

Avant!

**Star-Hspice
User Guide**

**Release 2002.2
June 2002**

Star-Hspice and Star-Hspice XT/RF User Guide, Release 2002.2, June 2002

Previous version 2001.4, December, 2001

Copyright © 1993-2002 Avant! Corporation and Avant! subsidiary. All rights reserved.

Unpublished--rights reserved under the copyright laws of the United States.

Avant! software V2002.2 Copyright © 1993-2002 Avant! Corp. All rights reserved. Unpublished--rights reserved under the copyright laws of the United States.

Use of copyright notices is precautionary and does not imply publication or disclosure.

Disclaimer

AVANT! RESERVES THE RIGHT TO MAKE CHANGES TO ANY PRODUCTS HEREIN, WITHOUT FURTHER NOTICE. AVANT! MAKES NO WARRANTY, REPRESENTATION, OR GUARANTEE REGARDING THE FITNESS OF ITS PRODUCTS FOR ANY PARTICULAR PURPOSE, AND SPECIFICALLY DISCLAIMS ANY WARRANTY OF MERCHANTABILITY AND ANY WARRANTY OF NON-INFRINGEMENT. AVANT! DOES NOT ASSUME ANY LIABILITY ARISING OUT OF THE APPLICATION OR USE OF ANY PRODUCT, AND SPECIFICALLY DISCLAIMS ANY AND ALL LIABILITY, INCLUDING WITHOUT LIMITATION, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES. AVANT!'S LIABILITY ARISING OUT OF THE MANUFACTURE, SALE OR SUPPLYING OF THE PRODUCTS OR THEIR USE OR DISPOSITION, WHETHER BASED UPON WARRANTY, CONTRACT, TORT, OR OTHERWISE, SHALL NOT EXCEED THE ACTUAL LICENSE FEE PAID BY CUSTOMER.

Proprietary Rights Notice

This document contains information of a proprietary nature. No part of this manual may be copied or distributed without the prior written consent of Avant! corporation. This document, and the software described herein, is provided only under a written license agreement, or a type of written non-disclosure agreement, with Avant! corporation or its subsidiaries. ALL INFORMATION CONTAINED HEREIN SHALL BE KEPT IN CONFIDENCE, AND USED STRICTLY IN ACCORDANCE WITH THE TERMS OF THE WRITTEN NON-DISCLOSURE AGREEMENT, OR WRITTEN LICENSE AGREEMENT, WITH AVANT! CORPORATION OR ITS SUBSIDIARIES.

Trademark/Service-Mark Notice

ApolloII, ApolloII-GA, Aurora, ASIC Synthesizer, AvantTestchip, AvantWaves, ChipPlanner, Columbia, Columbia-CE, Cosmos-Scope, Cyclelink, Davinci, DFM Workbench, Driveline, Dynamic Model Switcher, Electrically Aware, Enterprise, EnterpriseACE, Evaccess, Hercules, Hercules-Explorer, HotPlace, HSPICE, HSPICE-LINK, LTL, Libra-Passport, Lynx, Lynx-LB, Lynx-VHDL, Mars, Mars-Rail, Mars-Xtalk, MASTER Toolbox, Medici, Michelangelo, Milkyway, Optimum Silicon, Passport, Pathfinder, Planet, Planet-PL, Planet-RTL, Polaris, Polaris-CBS, Polaris-MT, Progen, Prospector, Raphael, Raphael-NES, Saturn, Sirius, Silicon Blueprint, Smart Extraction, Solar, Solar II, Star, Star-Sim, Star-Sim XT, Star-Hspice, Star-Hspice XT/RF, Star-HspiceLink, Star-DC, Star-RC, Star-RC XT, Star-Power, Star-Time, Star-MTB, Star-XP, Taurus, Taurus-Device, Taurus-Layout, Taurus-Lithography, Taurus-OPC, Taurus-Process, Taurus-Topography, Taurus-Visual, Taurus-Workbench, Technology File Manager, TimeSlice, True-Hspice, and TSUPREM-4 are trademarks of Avant! Corporation. Avant!, the Avant! logo, AvanLabs, and avanticorp are trademarks and service-marks of Avant! Corporation. All other trademarks and service-marks are the property of their respective owners.

Contacting Avant! Corporation

Telephone: (510) 413-8000
Facsimile: (510) 413-8080
Toll-Free Telephone: (800) 369-0080
URL: <http://www.avanticorp.com>

Avant! Corporation
46871 Bayside Parkway
Fremont, CA 94538



Using This Guide

This User Guide provides the following information about the Star-Hspice™ circuit and device simulation software:

- Setup for Simulation
- Simulation Input and Controls
- Elements
- Using Sources and Stimuli
- Multi-Terminal Networks
- Parameters and Functions
- Simulation Output
- Simulation Options
- Initializing DC/Operating Point Analysis
- Transient Analysis
- AC Sweep and Small Signal Analysis
- Statistical Analysis and Optimization
- Common Model Interface
- Characterizing Cells
- Signal Integrity
- Behavioral Modeling
- Pole/Zero Analysis
- FFT Spectrum Analysis
- Modeling Filters and Networks
- Timing Analysis Using Bisection
- Running Demonstration Files
- FAQ/Troubleshooting
- Interfaces For Design Environments
- Library Encryption
- Full Simulation Examples

Audience

This manual is intended for design engineers who use Star-Hspice to develop, test, analyze, and modify circuit designs.

How This Manual is Organized

The manual set is divided into two volumes, as follows:

- Volume I (Chapters 1 through 13) describes how to run simulations, using Star-Hspice, and how to evaluate the results.
- Volume II contains detailed applications and examples of how to use Star-Hspice for a wide variety of circuit simulations (Chapters 14 through 22). Volume II also contains reference material (Appendices).

Related Documents

The following documents pertain to this guide:

- Star-Hspice, Star-Time, and AvanWaves Installation Guide
- Star-Sim, Star-Sim XT, and Star-Time User Guides
- Star-Hspice and AvanWaves Release Notes

If you have questions or suggestions about this documentation, send them to:

`techpubs@avanticorp.com`

Conventions

Avant! documents use the following conventions, unless otherwise specified:

Convention	Description
Commands and options	Appear in all capital letters and spelled out, such as: BLOCK and SIM_ACS You can abbreviate all commands and options to three letters, with the exception of the .LIBRARY option, which should not be abbreviated, because a .LIB option also exists.
Menus and menu choices	Appear in <i>Times italic</i> with a >, surrounded by spaces, between levels, such as: <i>File > Save</i> When a tool name and a colon appear before the menu name, the command is available only through a tool. For example: <i>Data Prep: Pin Solution > Via</i> refers to the <i>Via</i> command on the <i>Pin Solution</i> menu, which you access by selecting <i>Data Prep</i> from the <i>Tools</i> menu in ApolloTM.
Window names	Appear in <i>boldface/italics</i>, such as: <i>Composer</i> window
Field names	Appear in square brackets ([]) and regular text, such as: [File name] [Mapping File]
Buttons	Appear in boldface, such as: Add Ok

Convention	Description
User input at system prompt	Appears in all small letters in Courier typeface, unless an element is a parameter, such as: <pre>% grdgenxo <ITF process file></pre> Items in Courier must be typed exactly as shown. Courier italic surrounded by quotation marks (“ ”) indicates a string.
Text from files and system messages	Examples appear in Courier exactly as they appear on the screen.
File names	Within the body of a paragraph, appear in lowercase <i>Times italic</i>, such as: <pre>mos_tech</pre> You can enter file names in uppercase and/or lowercase unless specifically indicated otherwise.
Variables	Appear in <i>Times italic</i>, surrounded by < >, such as: <pre><process_name>.nxtgrd</pre>
Models	Appear in uppercase for the acronyms, such as MOSFET, BJT, etc., but in Initial Caps for actual model names, such as Schlieman-Hodges Level 34, with the exception of IDS, another acronym.
Parameters	Appear in uppercase Courier typeface, such as: <pre>ACM=2</pre>
[] in syntax	Denotes optional arguments, such as: <pre>pin1 [pin2, ...pinN]</pre> In this example, you must enter at least one pin name; the other arguments are optional.

Convention	Description
{item} ...)	Indicates that you can repeat the construction enclosed in braces.
. . .	Indicates that text was omitted.
'(item1 item2)	An apostrophe, followed by parentheses, indicates that text within the parentheses encloses a list. If the list contains multiple items, spaces separate items. Type this information exactly as it appears in the syntax.
	Separates items in a list of choices. For example: On Off.
\	Indicates the continuation of a command line.

Obtaining Customer Support

If you have a maintenance contract with Avant!, you can obtain customer support by:

- Contacting your local Technical Support Engineer (TSE).
- Calling the Avant! Corporate office from 8:00 AM through 5:00 PM Pacific Standard Time (PST) at:

1-800-346-5953

Ask the receptionist for customer support.

- Faxing a description of the problem to the Avant! Corporate office at:

1-510-413-8080

Ensure that you write “Attn.: Customer Support Service Center” somewhere on the cover letter, so the FAX can be properly routed.

- Emailing a description of the problem to the Star-Hspice Support Center at:

`hspice_nw@avanticorp.com`

Other Sources of Information

The Avant! external web site provides information for various products. You can access our web site at:

`http://www.avanticorp.com`

From our web site, you can register to become a member of the Avant! Users Research Organization for Real Applications (AURORA™) user’s group. By participating, you can share and exchange information pertaining to Integrated Circuit Design Automation (ICDA).



Table of Contents

Audience	iv
How This Manual is Organized	iv
Related Documents	iv
Conventions	v
Obtaining Customer Support	viii
Other Sources of Information	viii
Chapter 1 - Overview	1-1
Applications	1-2
Features	1-3
Supported Platforms	1-6
Simulation Structure	1-7
Data Flow	1-8
Simulation Process Overview	1-10
Chapter 2 - Setup for Simulation	2-1
Setting Environment Variables	2-2
LM_LICENSE_FILE	2-2
SIM_CONFIG	2-2
Creating a Configuration File	2-4
Example	2-6

Using Wildcards	2-7
Syntax	2-7
Example	2-8
Netlist Overview	2-9
Basic Structure	2-9
First Character	2-10
Adding Elements	2-11
Comments and Line Continuation	2-12
Software Conventions	2-12
Chapter 3 - Simulation Input and Controls	3-1
Using Netlist Input Files	3-2
Input Netlist File Guidelines	3-2
Input Netlist File Sections and Chapter References	3-6
Input Netlist File Composition	3-8
Title of Simulation and .TITLE Statement	3-8
Comments	3-8
Element and Source Statements	3-9
.SUBCKT or .MACRO Statement	3-11
.ENDS or .EOM Statement	3-13
Subcircuit Call Statement	3-13
Element and Node Naming Conventions	3-14
.GLOBAL Statement	3-18
.TEMP Statement	3-19
.DATA Statement	3-20
.INCLUDE Statement	3-28
.MODEL Statement	3-28
.LIB Call and Definition Statements	3-30
.OPTION SEARCH Statement	3-34
.PARAM Statement	3-36
.PROTECT Statement	3-38

.UNPROTECT Statement	3-38
.ALTER Statement	3-39
.ALIAS Statement	3-41
.MALIAS Statement	3-43
.CONNECT Statement	3-44
.DEL LIB Statement	3-45
.END Statement	3-48
Condition-Controlled Netlists (if-else)	3-49
Using Subcircuits	3-51
Hierarchical Parameters	3-52
Undefined Subcircuit Search	3-54
Discrete Device Libraries	3-55
DDL Library Access	3-55
Vendor Libraries	3-56
Subcircuit Library Structure	3-57
Using Standard Input Files	3-58
Design and File Naming Conventions	3-58
Configuration File (<i>meta.cfg</i>)	3-59
Initialization File (<i>hspice.ini</i>)	3-59
DC Operating Point Initial Conditions File (<design>.ic#)	3-59
Starting Star-Hspice	3-60
Executing a Simulation	3-62
Interactive Simulation	3-64
Sample Star-Hspice Commands	3-65
Improving Simulation Performance Using Multithreading	3-67
Running Star-Hspice-MT	3-67
Performance Improvement Estimations	3-68
Using PKG and EBD Simulation	3-70
Options Statements	3-70
System-Level PKG and EBD Simulation	3-73

Stand-alone PKG Simulation	3-74
Stand-alone EBD Simulation	3-75
Limitation	3-76
Star-Hspice Output Files	3-78
Chapter 4 - Elements	4-1
Passive Elements	4-2
Resistors	4-2
Linear Resistors	4-4
Behavioral Resistors	4-5
Capacitors	4-6
Linear Capacitors	4-9
Behavioral Capacitors	4-10
Charge-Conserving Capacitors	4-11
Inductors	4-11
Mutual Inductors	4-14
Linear Inductors	4-16
Active Elements	4-18
Diode Element	4-18
Bipolar Junction Transistor (BJT) Element	4-20
JFETs and MESFETs	4-23
MOSFETs	4-25
Transmission Lines	4-29
Input Syntax for the W Element	4-29
W Element Statement	4-30
T Element Statement	4-33
U Element Statement	4-35
Frequency-Dependent Multi-Terminal (S) Element	4-36
Buffers	4-40
Syntax	4-40
Example	4-40

Chapter 5 - Using Sources and Stimuli.....	5-1
Independent Source Elements	5-2
Source Element Conventions	5-2
Independent Source Element	5-2
DC Sources	5-5
AC Sources	5-5
Transient Sources	5-6
Mixed Sources	5-6
Independent Source Functions	5-7
Pulse Source Function	5-7
Sinusoidal Source Function	5-11
Exponential Source Function	5-14
Piecewise Linear (PWL) Source Function	5-17
Data-Driven Piecewise Linear Source	5-19
Single-Frequency FM Source Function	5-21
Amplitude Modulation Source Function	5-23
Voltage and Current Controlled Elements	5-26
Polynomial Functions	5-27
Piecewise Linear Function	5-31
Voltage-Dependent Voltage Sources — E Elements	5-33
Voltage-Controlled Voltage Source (VCVS)	5-33
Behavioral Voltage Source	5-33
Ideal Op-Amp	5-34
Ideal Transformer	5-34
E Element Parameters	5-35
E Element Examples	5-37
Current-Dependent Current Sources — F Elements	5-40
Current-Controlled Current Source (CCCS)	5-40
F Element Parameters	5-41
F Element Examples	5-43

Voltage-Dependent Current Sources — G Elements	5-44
Voltage-Controlled Current Source (VCCS)	5-44
Behavioral Current Source	5-45
Voltage-Controlled Resistor (VCR)	5-45
Voltage-Controlled Capacitor (VCCAP)	5-46
G Element Parameters	5-47
G Element Examples	5-50
Current-Dependent Voltage Sources — H Elements	5-52
Current-Controlled Voltage Source (CCVS)	5-52
H Element Parameters	5-53
H Element Examples	5-55
Digital and Mixed Mode Stimuli	5-56
U Element Digital Input Elements and Models	5-56
U Element Digital Outputs	5-60
Replacing Sources With Digital Inputs	5-63
Specifying a Digital Vector File	5-68
Defining Vector Patterns	5-68
Defining Tabular Data	5-73
Using Tabular Data	5-77
Defining Waveform Characteristics	5-77
Modifying Waveform Characteristics	5-78
Comment Lines	5-85
Continuing a Line	5-85
Digital Vector File Example	5-85
Chapter 6 - Multi-Terminal Networks	6-1
Using Scattering Parameter Element	6-2
Syntax	6-3
Frequency Table Model	6-5
Syntax	6-5
Example	6-8

Chapter 7 - Parameters and Functions.....	7-1
Using Parameters in Simulation (.PARAM)	7-2
Defining Parameters	7-2
Assigning Parameters	7-4
User-Defined Function Parameters	7-5
Subcircuit Default Parameter Definitions	7-6
Predefined Analysis Function	7-7
Measurement Parameters	7-7
.PRINT .PROBE .PLOT .GRAPH Parameters	7-7
Multiply Parameter	7-7
Using Algebraic Expressions	7-9
Built-In Functions	7-10
Example	7-14
Parameter Scoping and Passing	7-15
Library Integrity	7-16
Reusing Cells	7-16
Creating Parameters in a Library	7-16
Parameter Defaults and Inheritance	7-19
Parameter Passing Solutions	7-22
Chapter 8 - Simulation Output.....	8-1
Overview of Output Statements	8-2
Output Commands	8-2
Output Variables	8-3
Displaying Simulation Results	8-4
.PRINT Statement	8-4
.PLOT Statement	8-8
.PROBE Statement	8-10
.GRAPH Statement	8-12

Using Wildcards in .PRINT, .PROBE, .PLOT, and .GRAPH	
Statements	8-16
Print Control Options	8-17
Printing the Subcircuit Output	8-22
Selecting Simulation Output Parameters	8-25
DC and Transient Output Variables	8-25
AC Analysis Output Variables	8-33
Element Template Output	8-39
Specifying User-Defined Analysis (.MEASURE)	8-40
.MEASURE Performance	8-41
.MEASURE Parameter Types	8-42
.MEASURE Statement: Rise, Fall, and Delay	8-43
Average, RMS, and Peak Measurements	8-48
FIND and WHEN Functions	8-49
Equation Evaluation	8-52
Average, RMS, MIN, MAX, INTEG, and PP	8-53
INTEGRAL Function	8-55
DERIVATIVE Function	8-55
ERROR Function	8-58
Arithmetic Expression Measurements	8-61
.DOUT Statement: Expected State of Digital Output Signal	8-62
Syntax	8-62
Example	8-64
.STIM Statement: Reuse Simulation Output as Input Stimuli	8-65
Syntax	8-65
Output Files	8-69
Element Template Listings	8-71

Chapter 9 - Simulation Options.....	9-1
Setting Control Options	9-2
.OPTION Statement	9-2
General Control Options	9-5
Input and Output Options	9-5
CPU Options	9-12
Interface Options	9-12
Analysis Options	9-15
Error Options	9-17
Version Options	9-17
Model Analysis Options	9-18
General Options	9-18
MOSFET Control Options	9-20
Inductor Options	9-21
BJT and Diode Options	9-21
DC Operating Point, DC Sweep, and Pole/Zero Options	9-22
Accuracy Options	9-22
Matrix Options	9-25
Pole/Zero Input and Output Options	9-28
Convergence Options	9-29
Pole/Zero Control Options	9-35
Transient and AC Small Signal Analysis Options	9-37
Accuracy Options	9-37
Speed Options	9-41
Timestep Options	9-44
Algorithm Options	9-47
Input and Output Options	9-51

Chapter 10 - Initializing DC/Operating Point Analysis	10-1
Simulation Flow	10-2
Initialization and Analysis	10-3
DC Initialization and Operating Point Statements	10-6
.OP Statement — Operating Point	10-6
Element Statement IC Parameter	10-8
.IC and .DCVOLT Initial Condition Statements	10-9
.NODESET Statement	10-10
.SAVE and .LOAD Statements	10-10
.DC Statement—DC Sweeps	10-14
Syntax	10-14
Keywords and Parameters	10-15
Examples	10-17
Schmitt Trigger Example	10-18
Other DC Analysis Statements	10-19
.SENS Statement — DC Sensitivity Analysis	10-19
.TF Statement — DC Small-Signal Transfer Function Analysis ..	10-20
.PZ Statement— Pole/Zero Analysis	10-21
DC Initialization Control Options	10-22
Pole/Zero Analysis Options	10-30
Accuracy and Convergence	10-32
Accuracy Tolerances	10-32
Accuracy Control Options	10-34
Convergence Control Options	10-34
Autoconverge Process	10-39
Reducing DC Errors	10-43
Shorted Element Nodes	10-44
Inserting Conductance, Using DCSTEP	10-45
Floating-Point Overflow	10-46

Diagnosing Convergence Problems	10-47
Non-Convergence Diagnostic Table	10-47
Traceback of Non-Convergence Source	10-49
Solutions for Non-Convergent Circuits	10-49
Chapter 11 - Transient Analysis.....	11-1
Simulation Flow	11-2
Overview of Transient Analysis	11-3
Using the .TRAN Statement	11-4
Syntax	11-4
.TRAN Keywords and Parameters	11-6
.TRAN Examples	11-7
.TRAN Options	11-8
.TRAN Output Syntax	11-9
.TRAN Output Format/Description	11-9
Transient Analysis of an RC Network	11-10
Transient Analysis of an Inverter	11-12
Using the .BIASCHK Statement	11-14
Syntax	11-14
Example	11-16
Options for the .biaschk Command	11-16
Transient Control Options	11-17
Method Options	11-18
Tolerance Options	11-23
Limit Options	11-28
Matrix Manipulation Options	11-30
Controlling Simulation Speed and Accuracy	11-31
Simulation Speed	11-31
Simulation Accuracy	11-31

Numerical Integration Algorithm Controls	11-35
Syntax	11-35
Gear and Trapezoidal Algorithms	11-35
Selecting Timestep Control Algorithms	11-38
Iteration Count Dynamic Timestep Algorithm	11-39
Local Truncation Error (LTE) Dynamic Timestep Algorithm	11-40
DVDT Dynamic Timestep Algorithm	11-40
Timestep Controls	11-42
Fourier Analysis	11-43
.FOUR Statement	11-44
.FFT Statement	11-47
Chapter 12 - AC Sweep and Small Signal Analysis.....	12-1
AC Small Signal Analysis	12-2
.AC Statement	12-4
Syntax	12-4
Examples	12-7
AC Control Options	12-9
AC Analysis of an RC Network	12-10
Other AC Analysis Statements	12-13
.DISTO Statement — AC Small-Signal Distortion Analysis	12-13
.NOISE Statement — AC Noise Analysis	12-15
.SAMPLE Statement — Noise Folding Analysis	12-17
.NET Statement - AC Network Analysis	12-18
References	12-27
Chapter 13 - Statistical Analysis and Optimization	13-1
Analytical Model Types	13-2
Simulating Circuit and Model Temperatures	13-4
Temperature Analysis	13-5
.TEMP Statement	13-6

Worst Case Analysis	13-8
Model Skew Parameters	13-8
Monte Carlo Analysis	13-13
Functions	13-13
Monte Carlo Setup	13-13
Monte Carlo Output	13-14
.PARAM Distribution Function Syntax	13-15
Monte Carlo Parameter Distribution	13-17
Monte Carlo Examples	13-17
Worst Case and Monte Carlo Sweep Example	13-25
Star-Hspice Input File	13-25
Transient Sigma Sweep Results	13-27
Monte Carlo Results	13-29
Optimization	13-34
Optimization Control	13-35
Simulation Accuracy	13-35
Curve Fit Optimization	13-36
Goal Optimization	13-36
Timing Analysis	13-37
Optimization Syntax	13-37
Optimization Examples	13-43
MOS Level 3 Model DC Optimization	13-43
MOS Level 13 Model DC Optimization	13-47
RC Network Optimization	13-50
Optimizing CMOS Tristate Buffer	13-54
BJT S Parameters Optimization	13-59
BJT Model DC Optimization	13-62
Optimizing GaAsFET Model DC	13-66
Optimizing MOS Op-amp	13-68

Chapter 14 - Common Model Interface.....	14-1
Overview of CMI	14-2
Directory Structure	14-3
Running Simulations with CMI Models	14-5
Adding Proprietary MOS Models	14-6
MOS Models on Unix Platforms	14-6
MOS Models on PC Platforms	14-10
Testing CMI Models	14-12
Model Interface Routines	14-13
Interface Variables	14-17
pModel, pInstance	14-18
CMI_ResetModel	14-19
CMI_ResetInstance	14-20
CMI_AssignModelParm	14-20
CMI_AssignInstanceParm	14-21
CMI_SetupModel	14-22
CMI_SetupInstance	14-23
CMI_Evaluate	14-23
CMI_DiodeEval	14-25
CMI_Noise	14-26
CMI_PrintModel	14-27
CMI_FreeModel	14-28
CMI_FreeInstance	14-28
CMI_WriteError	14-29
CMI_Start	14-30
CMI_Conclude	14-31
CMI Function Calling Protocol	14-32
Internal Routines	14-33
Extended Topology	14-35

Conventions	14-37
Bias Polarity, for N- and P-channel Devices	14-37
Source-Drain Reversal Conventions	14-38
Thread-Safe Model Code	14-38
Chapter 15 - Characterizing Cells.....	15-1
Typical Data Sheet Parameters	15-2
Rise, Fall, and Delay Calculations	15-2
Ripple Calculation	15-3
Sigma Sweep versus Delay	15-3
Delay versus Fanout	15-5
Pin Capacitance Measurement	15-6
Op-amp Characterization of ALM124	15-7
Characterizing Cells in Data-Driven Analysis	15-9
Cell Examples	15-9
Input File Examples	15-12
Chapter 16 - Signal Integrity	16-1
Preparing for Simulation	16-2
Signal Integrity Problems	16-3
Analog Side of Digital Logic	16-4
Optimizing TDR Packaging	16-9
Using TDR in Simulation	16-9
TDR Optimization Procedure	16-11
Simulating Circuits with Signetics Drivers	16-16
Example	16-17
Package Inductance	16-18
Simulating Circuits with Xilinx FPGAs	16-19
Syntax for IOB (xil_iob) and IOB4 (xil_iob4)	16-19
Ground Bounce Simulation	16-21
Coupled Line Noise	16-23

Chapter 17 - Behavioral Modeling.....	17-1
Behavioral Design Process	17-2
Using Behavioral Elements	17-3
Controlled Sources	17-4
Digital Stimulus Files	17-4
Behavioral Examples	17-5
Op-Amp Subcircuit Generators	17-5
Libraries	17-5
Voltage and Current Controlled Elements	17-6
Polynomial Functions	17-7
Piecewise Linear (PWL) Function	17-10
Voltage-Dependent Voltage Sources — E Elements	17-12
Voltage-Controlled Voltage Source (VCVS)	17-12
Behavioral Voltage Source	17-12
Ideal Op-Amp	17-13
Ideal Transformer	17-13
E Element Parameters	17-13
Examples	17-15
Current-Dependent Current Sources — F Elements	17-18
Current-Controlled Current Source (CCCS)	17-18
F Element Parameters	17-19
Examples	17-21
Voltage-Dependent Current Sources — G Elements	17-22
Voltage-Controlled Current Source (VCCS)	17-22
Behavioral Current Source	17-23
Voltage-Controlled Resistor (VCR)	17-23
Voltage-Controlled Capacitor (VCCAP)	17-24
G Element Parameters	17-24
Examples	17-27

Current-Dependent Voltage Sources – H Elements	17-30
Current-Controlled Voltage Source (CCVS)	17-30
H Element Parameters	17-31
Examples	17-33
Modeling with Digital Behavioral Components	17-34
Behavioral AND and NAND Gates	17-34
Behavioral D-Latch	17-36
Behavioral Double-Edge Triggered Flip-Flop	17-39
Calibrating Digital Behavioral Components	17-42
Building Behavioral Lookup Tables	17-42
Optimizing Behavioral CMOS Inverters	17-48
Optimizing Behavioral Ring Oscillators	17-52
Analog Behavioral Elements	17-54
Behavioral Integrator	17-54
Behavioral Differentiator	17-56
Ideal Transformer	17-58
Behavioral Tunnel Diode	17-59
Behavioral Silicon-Controlled Rectifier (SCR)	17-60
Behavioral Triode Vacuum Tube Subcircuit	17-61
Behavioral Amplitude Modulator	17-63
Behavioral Data Sampler	17-64
Op-Amps, Comparators, and Oscillators	17-65
Op-Amp Model Generator	17-65
Op-Amp Element Statement Format	17-66
Op-Amp .MODEL Statement Format	17-67
Op-Amp Subcircuit Example	17-75
741 Op-Amp from Controlled Sources	17-77
Inverting Comparator with Hysteresis	17-81
Voltage-Controlled Oscillator (VCO)	17-82
LC Oscillator	17-84

Phase Locked Loops (PLL)	17-87
Phase Detector, with Multi-Input NAND Gates	17-87
PLL BJT Behavioral Modeling	17-90
References	17-98
Chapter 18 - Pole/Zero Analysis.....	18-1
Overview of Pole/Zero Analysis	18-2
Using Pole/Zero Analysis	18-3
Muller Method	18-3
.PZ (Pole/Zero) Statement	18-4
Pole/Zero Analysis Examples	18-7
Example 1 – Low-Pass Filter	18-7
Example 2 – Kerwin’s Circuit	18-10
Example 3 – High-Pass Butterworth Filter	18-11
References	18-22
Chapter 19 - FFT Spectrum Analysis	19-1
Using Windows in FFT Analysis	19-2
Using the .FFT Statement	19-6
Syntax	19-6
Examples	19-8
Examining the FFT Output	19-9
AM Modulation	19-11
Input Listing	19-11
Output Listing	19-12
Graphical Output	19-12
Balanced Modulator and Demodulator	19-14
Input Listing	19-14
Output Listing	19-15

Signal Detection Test Circuit	19-22
Input Listing	19-22
Output	19-22
References	19-28
Chapter 20 - Modeling Filters and Networks.....	20-1
Transient Modeling	20-2
Using G and E Elements	20-4
Laplace Transform Function Call	20-4
Element Statement Parameters	20-10
G and E Element Notes	20-12
Laplace Band-Reject Filter	20-12
Laplace Low-Pass Filter	20-14
Circular Convolution Example	20-17
Laplace and Pole-Zero Modeling	20-19
Laplace Transform (LAPLACE) Function	20-19
Laplace Transform POLE (Pole/Zero) Function	20-27
AWE Transfer Function Modeling	20-33
Y Parameter Line Modeling	20-36
Comparison of Circuit and Pole/Zero Models	20-41
Modeling Switched Capacitor Filters	20-44
Switched Capacitor Network	20-44
Switched Capacitor Network Example	20-45
Switched Capacitor Filter Example	20-46
Input File for Switched Capacitor Filter	20-47
References	20-51

Chapter 21 - Timing Analysis Using Bisection.....	21-1
Overview of Bisection	21-2
Bisection Methodology	21-4
Measurement	21-4
Optimization	21-4
Using Bisection	21-5
Command Syntax	21-6
Examples	21-8
Setup Time Analysis	21-9
Input Listing	21-9
Results	21-12
Minimum Pulse Width Analysis	21-14
Input Listing	21-14
Results	21-16
Chapter 22 - Running Demonstration Files	22-1
Using the Demo Directory Tree	22-2
Two-Bit Adder Demo	22-3
One-Bit Subcircuit	22-3
MOS Two-Bit Adder Input File	22-4
MOS I-V and C-V Plotting Demo	22-6
Plotting Variables	22-6
MOS I-V and C-V Plot Example Input File	22-9
CMOS Output Driver Demo	22-10
Strategy	22-10
CMOS Output Driver Example Input File	22-13
Temperature Coefficients Demo	22-15
Example	22-15
Input File, for Optimized Temperature Coefficients	22-16
Optimization Section	22-16

Simulating Electrical Measurements	22-17
T2N2222 Optimization Example Input File	22-18
Transient Measurements	22-18
Modeling Wide-Channel MOS Transistors	22-20
Demonstration Input Files	22-23
Appendix A - FAQ/Troubleshooting.....	A-1
Analysis	A-2
Documentation	A-4
Environment Variables	A-6
Error Messages	A-7
Input	A-12
Installation Issues	A-13
Licensing/Access Issues	A-15
Limitations	A-17
Miscellaneous	A-18
Models	A-20
MS Windows/PC Issues	A-23
Netlist/Options	A-27
Output	A-33
W Element/Field Solver	A-35
Waveform Viewing	A-36
Appendix B - Interfaces For Design Environments.....	B-1
AvanLink to Cadence Composer and Analog Artist	B-2
Features	B-2
Environment	B-3

AvanLink Design Flow	B-5
Schematic Entry and Library Operations	B-6
Generating a Netlist	B-7
Simulation	B-7
Waveform Display	B-7
AvanLink for Design Architect	B-8
Features	B-8
Environment	B-9
AvanLink-DA Design Flow	B-10
Schematic Entry and Library Operations	B-10
Generating a Netlist	B-11
Simulation	B-12
Waveform Display	B-12
Viewlogic Links	B-13

Appendix C - Library Encryption..... C-1

Library Encryption	C-2
Controlling the Encryption Process	C-2
Library Structure	C-2
Encryption Guidelines	C-5
Installing and Running the Encryptor	C-7
Installing the Encryptor	C-7
Running the Encryptor	C-7
Metaencrypt Features	C-9
8-Byte Key Encryption	C-9
Encryption Structure	C-9
.sp File Encryption	C-10
.lib File Encryption	C-11
.inc File Encryption	C-12
.load Encryption	C-12

Encrypting 80+ Columns	C-12
Statements Not Supported	C-12
Additional Recommendations for Encryption	C-12
Encryption Structure Example	C-13
Appendix D - Full Simulation Examples	D-1
Simulation Example, with AvanWaves	D-2
Input Netlist and Circuit	D-2
Execution and Output Files	D-3
Simulation Graphical Output in AvanWaves	D-10
Simulation Example, with Cosmos-Scope	D-13
Input Netlist and Circuit	D-13
Execution and Output Files	D-15
View Star-Hspice Results in Cosmos-Scope	D-15



Chapter 1

Overview

Star-Hspice is an optimizing analog circuit simulator. It is Avant!'s industrial-grade circuit analysis product. You can use it to simulate electrical circuits in steady-state, transient, and frequency domains. Like traditional SPICE simulators, Star-Hspice is Fortran-based, but it is faster and has more capabilities than typical SPICE simulators. Star-Hspice accurately simulates, analyzes, and optimizes circuits, from DC, to microwave frequencies that are greater than 100 GHz.

Star-Hspice is ideal for cell design and process modeling. It is also the tool of choice for signal-integrity and transmission-line analysis.

This chapter explains the following topics:

- [Applications](#)
- [Features](#)
- [Supported Platforms](#)
- [Simulation Structure](#)

Applications

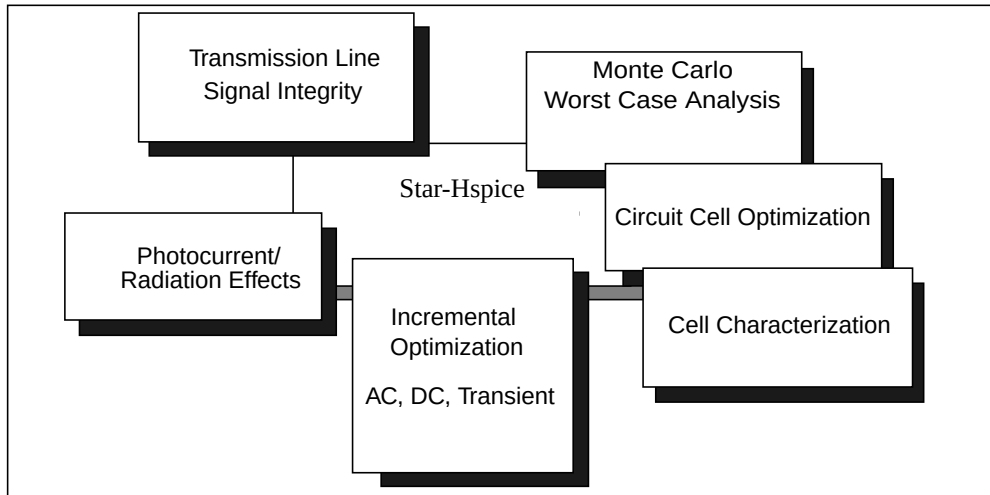
Star-Hspice is unequalled for fast, accurate circuit and behavioral simulation. They facilitate circuit-level analysis of performance and yield, using Monte Carlo, worst case, parametric sweep, and data-table sweep analysis, and employ the most reliable automatic-convergence capability.

Star-Hspice forms the cornerstone of a suite of Avant! tools and services that allow accurate calibration of logic and circuit model libraries, to actual silicon performance.

The size of the circuits that Star-Hspice can simulate, is limited only by the virtual memory of the computer that you are using. Star-Hspice software is optimized for each computer platform, with interfaces available to a variety of design frameworks.

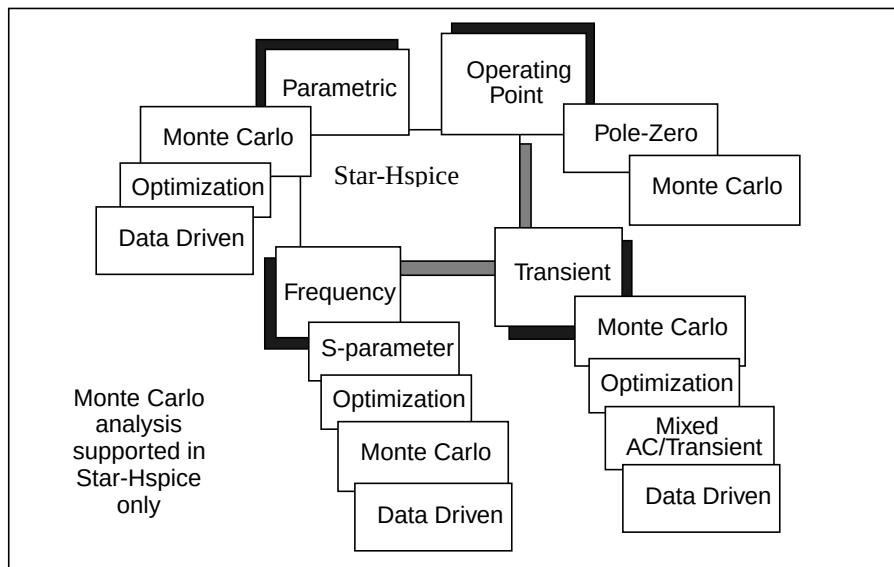
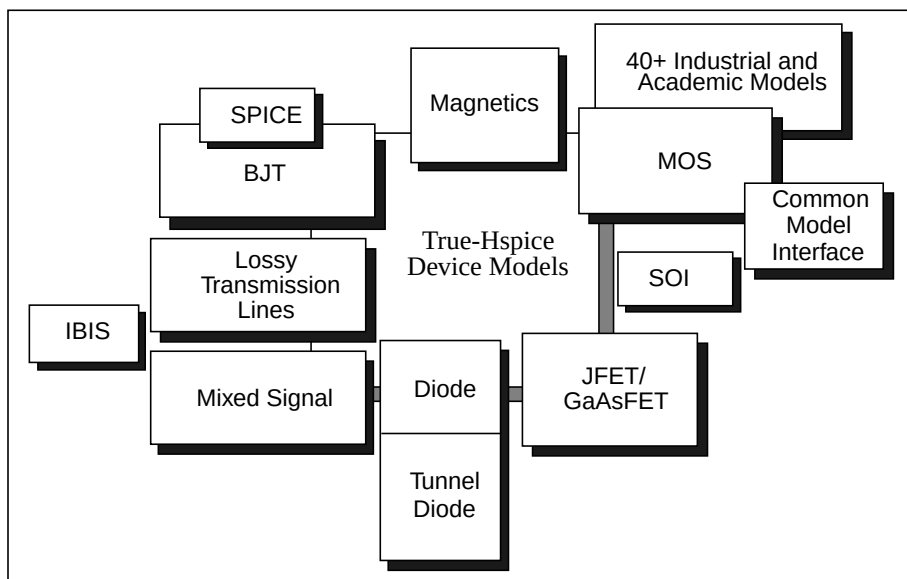
Features

Figure 1-1: Star-Hspice Design Features



Star-Hspice is compatible with most SPICE variations, and has the following additional features:

- Superior convergence.
- Accurate modeling, including many foundry models.
- Hierarchical node naming and reference.
- Circuit optimization for models and cells, with incremental or simultaneous multiparameter optimizations in AC, DC, and transient simulations.
- Interpreted Monte Carlo and worst-case design support.
- Input, output, and behavioral algebraics for cells with parameters.
- Cell characterization tools, to calibrate models for high-level logic simulators.
- Geometric lossy-coupled transmission lines for PCB, multi-chip, package, and IC technologies.
- Discrete component, pin, package, and vendor IC libraries.
- AvanWaves interactively graphs and analyzes multiple simulation waveforms.
- If you suspend a simulation job, LSF license manager sends a signal to that job, and Star-Hspice releases the occupied license. Another simulation job can use that license, or the stopped job can reclaim the license and continue from where you stopped it. You can also submit simulation jobs with priority into the LSG queue; LSF automatically suspends low-priority simulation jobs, to run high-priority simulation jobs. When a high-priority job completes, LSF automatically releases the license back to the lower-priority job, which resumes from the point where LSF suspended it.

Figure 1-2: Star-Hspice Circuit Analysis Types**Figure 1-3: True-Hspice Modeling Technologies**

Simulation at the integrated circuit level, and at the system level, requires careful planning of the organization and interaction between transistor models and subcircuits. Methods that worked for small circuits might have too many limitations when applied to higher-level simulations.

Use the following Star-Hspice features to organize how simulation circuits and models run:

- Explicit include files – `.INC` statement.
- Implicit include files – `.OPTION SEARCH = 'lib_directory'`.
- Algebraics and parameters for devices and models – `.PARAM` statement.
- Parameter library files – `.LIB` statement.
- Automatic model selector – `LMIN`, `LMAX`, `WMIN`, and `WMAX` model parameters.
- Parameter sweep – `SWEEP` analysis statement.
- Statistical analysis – `SWEEP MONTE` analysis statement.
- Multiple alternative – `.ALTER` statement.
- Automatic measurements – `.MEASURE` statement.
- Condition-controlled netlists (`if-elseif-else-endif` statements).

Supported Platforms

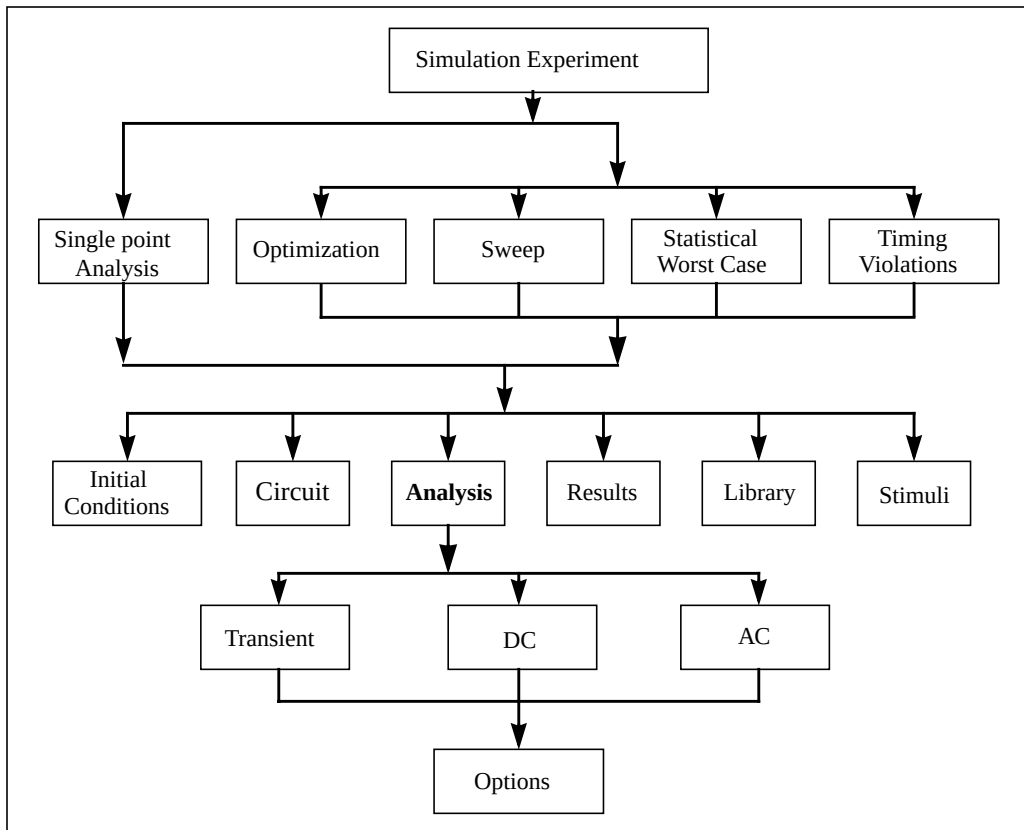
Star-Hspice is available for the following platforms and operating systems:

Platform	Operating System
Sun Ultra	Solaris 2.5, 2.7 and 2.8
Sun Sparc	Solaris 5.5
Sun Blade	Solaris 5.8
HP PA	UX 10.20, UX 11.00
IBM RS6000	AIX 4.3
DEC Alpha	OSF 4.0
SGI	IRIX 6.5
PC	Windows 95, 98, ME, 2000, NT 4.0, and XP.
Linux	RedHat 6.2, 7.0/7.1 (Does not support MOSFET level 29 and level 45).
Note: Star-Hspice supports a single AMD CPU for WinNT4.0, and RedHat 7.0/7.1	

Simulation Structure

Figure 1-4 shows the program structure for simulation experiments.

Figure 1-4: Simulation Program Structure



Analysis and verification of complex designs are typically organized around a series of experiments. These experiments can be simple sweeps, or more complex Monte Carlo and optimization analyses, and setup and hold violation analyses of DC, AC, and transient conditions.

For each simulation experiment, you must specify tolerances and limits to achieve the desired goals, such as optimizing or centering a design. Common factors for each experiment are:

- process
- voltage
- temperature
- parasitics

Star-Hspice supports two experimental methods:

- Single point – a simple procedure that produces a single result, or a single set of output data.
- Multipoint – performs an analysis (single point) sweep for each value in an outer loop (multipoint) sweep.

The following are examples of multipoint experiments:

- Process variation – Monte Carlo or worst-case model parameter variation.
- Element variation – Monte Carlo or element parameter sweeps.
- Voltage variation – VCC, VDD, or substrate supply variation.
- Temperature variation – design temperature sensitivity.
- Timing analysis – basic timing, jitter, and signal integrity analysis.
- Parameter optimization – balancing complex constraints, such as speed versus power, or frequency versus slew rate versus offset (analog circuits).

Data Flow

Star-Hspice accepts input and simulation control information from several different sources. They can output results in a number of convenient forms for review and analysis. [Figure 1-5 on page 1-10](#) shows the overall data flow.

1. To begin design entry and simulation, create an input netlist file.
Most schematic editors and netlisters support the SPICE or Star-Hspice hierarchical format.
2. Star-Hspice executes the analyses specified in the input file.
3. Star-Hspice stores the simulation results requested in either an output listing file or (if you specified `.OPTION POST`) a graph data file.

If you specified `POST`, Star-Hspice stores the circuit solution (in either steady state, time, or frequency domain).

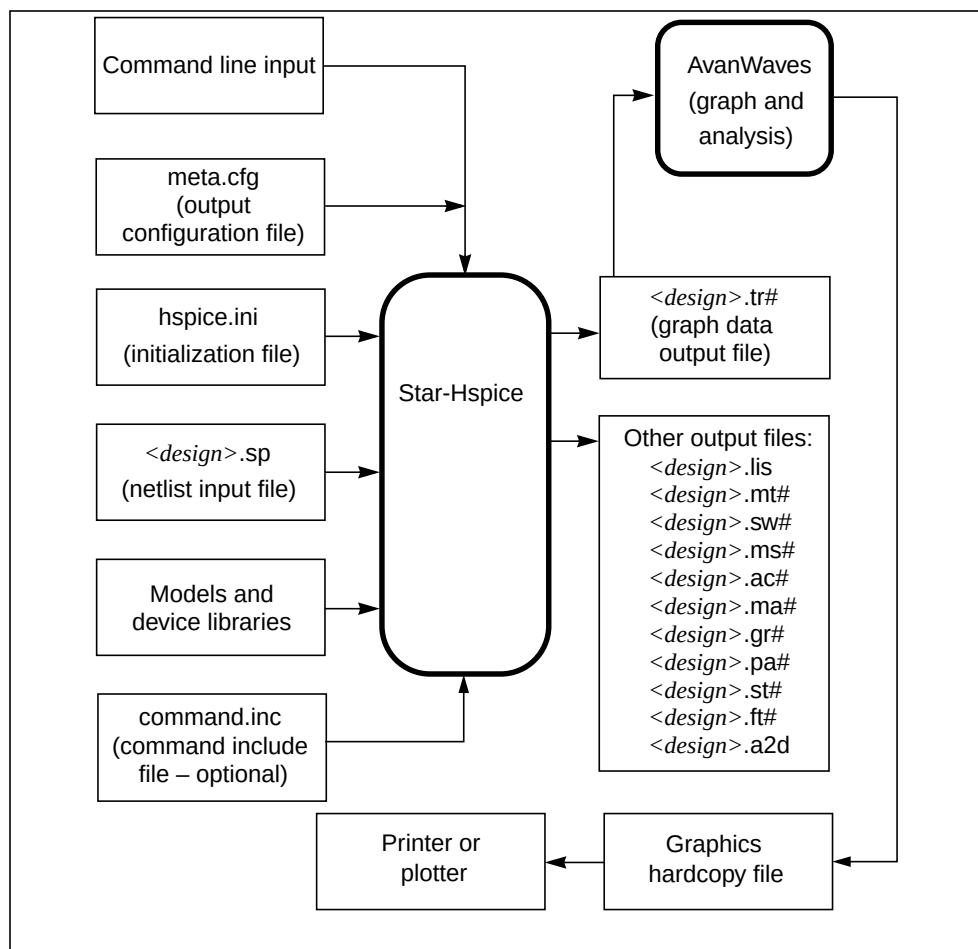
4. To view or plot the results for any nodal voltage or branch current, use a high-resolution graphic output terminal or laser printer.

Star-Hspice provides a complete set of print and plot variables for viewing analysis results.

The Star-Hspice programs include a textual command line interface. For example, to execute the program, enter the *hspice* command, the input file name, and the desired options. You can use the command line at the prompt in a Unix shell, or a DOS command line, or click on an icon in a Windows environment.

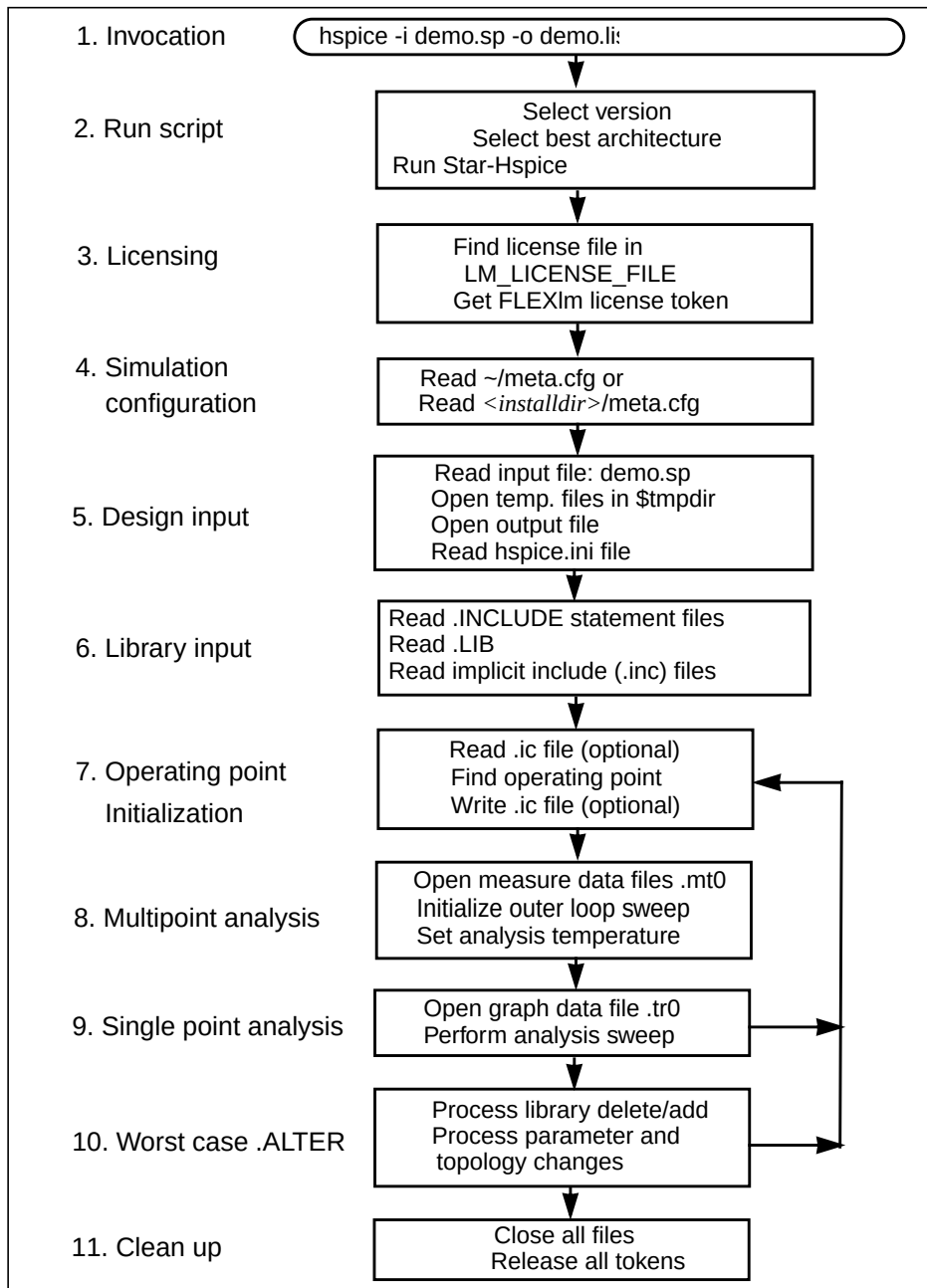
You can specify whether the Star-Hspice program simulation output appears in an output listing file, or in a graph data file. Star-Hspice creates standard output files to describe initial conditions (*.ic* extension) and output status (*.st0* extension). In addition, Star-Hspice creates various output files, in response to user-defined input options—for example, Star-Hspice creates a *<design>.tr0* file, in response to a *.TRAN* transient analysis statement.

The AvanWaves output display and analysis program includes a graphical user interface. Use the mouse to select options, and to execute commands, in various AvanWaves windows. Refer to the *AvanWaves User Guide* for instructions about how to use AvanWaves.

Figure 1-5: Overview of Data Flow

Simulation Process Overview

Figure 1-6 on page 1-11 is a diagram of the Star-Hspice simulation process. The following section summarizes the steps in a typical simulation.

Figure 1-6: Simulation Process



Chapter 2

Setup for Simulation

This chapter describes the required setup steps, and background information that you should understand, before you run Star-Hspice to perform IC circuit analyses.

This chapter includes the following examples:

- [Setting Environment Variables](#)
- [Creating a Configuration File](#)
- [Using Wildcards](#)
- [Netlist Overview](#)

Setting Environment Variables

Star-Hspice requires some environment variable settings, to generate the MOSFET technology file, called *mos_tech*. The *mos_tech* file contains device and process information (I-V and C-V data) obtained from either:

- Your host simulator, such as Star-Hspice, and its device models and model libraries, or
- The built-in common transistor models.

LM_LICENSE_FILE

The LM_LICENSE_FILE variable specifies the full path to the *license.dat* license file. Set the LM_LICENSE_FILE environment variable to point to the Star-Hspice license file.

Example

If your Star-Hspice license file is in */usr/cad/hspice/license.dat* path, then enter:

```
setenv LM_LICENSE_FILE /usr/cad/hspice/license.dat
```

SIM_CONFIG

The name of Star-Hspice configuration file defaults to *.admrc*. However, you can use the SIM_CONFIG environment variable to override this default name. SIM_CONFIG lets you use a different configuration file, with different options, when you run the simulation. This is particularly useful when you are simulating several different circuits, each with specific configuration file requirements. Using SIM_CONFIG, you can switch configuration files referenced in the simulation, instead of editing and re-editing the default file for each run. Also, if another tool is using *.admrc*, you can set SIM_CONFIG, to prevent Star-Hspice from reading an invalid file.

Example

```
setenv SIM_CONFIG ".starhspice_rc"
```

Note: The filename that you specify in SIM_CONFIG follows the same rules as the default .admrc configuration file. For information about creating this file, see [Chapter 3, “Simulation Input and Controls”](#).

Creating a Configuration File

You can create a configuration file, called *.admrc*, to customize your Star-Hspice simulation. Another Avant! IC circuit simulation product, Star-Sim XT, also uses the *.admrc* configuration file. Star-Hspice first searches for *.admrc* in your current working directory, then in your home directory, as defined in the *\$HOME* variable. You can use the configuration options listed in [Table 2-1](#).

Table 2-1: Configuration File Options

Keyword	Description	Example
hier_delimiter	Changes the delimiter for subcircuit hierarchies from “,” to the specified symbol.	hier_delimiter ^
flush_waveform	Flushes a waveform. If you do not specify a percentage, then the default value is 20%.	flush_waveform percent%
max_waveform_size	Automatically limits the waveform file size. • If the number is less than 5000, Star-Hspice resets it to 2G. • If you do not specify the number, Star-Hspice sets it to the default, 2G. • If you do not include this line, the file size has no limitation.	max_waveform_size 2000000000

Table 2-1: Configuration File Options (Continued)

Keyword	Description	Example
skip_nrd_nrs	Star-Hspice does not support this option, which directs Star-Sim XT to consider transistors with matching geometries (except for NRD and NRS) as identical for pre-characterization.	skip_nrd_nrs
unit_atto	Activates detection of the “atto” unit. Otherwise, Star-Hspice assumes that <i>a</i> is <i>amperes</i> .	unit_atto
v_supply	Star-Hspice does not support this option, which Star-Sim XT uses for characterization. This option changes the default voltage supply range.	v_supply 3
wildcard_match_all	Matches any group of characters.	wildcard_match_all *
wildcard_match_one	Matches any single character.	wildcard_match_one ?
wildcard_left_range	Begins a range expression.	wildcard_left_range [
wildcard_right_range	Ends a range expression.	wildcard_right_range]

Note: For more information about wildcards, see [Using Wildcards on page 2-7](#).

To insert comments into your configuration file, include a # character as the first character in a line.

Example

This configuration file shows how to use comments in the *.admrc* file:

```
# sample configuration file
# see Table 2-1
# the next line of code changes the delimiter
# for subcircuit hierarchies from "," to "^"
hier_delimiter ^
# the next line of code matches any groups of
# "*" characters
wildcard_match_all *
# the next line of code matches one "?" character
wildcard_match_one ?
# the next line of code begins the range expression with
# the "[" character
wildcard_left_range [
# the next line of code ends the range expression with
# the "]" character
wildcard_right_range ]
```

Using Wildcards

You can use wildcards to match node names. For more information about using wildcards in a configuration file, see [Creating a Configuration File on page 2-4](#).

The following statements support wildcards:

- .PRINT
- .PROBE

Syntax

`.PROBE wildcard_expression`

where the characters that formulate the *wildcard_expression* are:

*	Matches any string of characters.
?	Matches any single character.
[]	Matches any character that appears within the brackets. For example, [123] matches 1, 2, or 3. A hyphen, inside the brackets, indicates a character range. For example, [0-9] is the same as [0123456789], and matches any digit character.
any other character	Matches itself.

Wildcards must begin with a letter or a number. For example:

<code>.probe v(*)</code>	<--- correct format
<code>.probe *</code>	<--- incorrect format
<code>.probe x*</code>	<--- correct format

Example

The following examples use wildcards with the `.PRINT` and `.PROBE` statements. You must create an `.admrc` file, to use any of the wildcards.

- Probe all top-level nodes.
`.PROBE v( *)`
- Probe all top-level nodes whose names start with a. For example: a1, a2, a3, a00, ayz.
`.PROBE v(a*)`
- Print all first-level nodes, where zero-level are top-level nodes. For example: X1.A, X4.554, Xab.abc123.
`.PRINT v(*.*)`
- Probe all first-level nodes, where zero-level are top-level nodes. For example: x1.A, x4.554, xab.abc123. This creates only waveform data, without an ASCII table of results.
`.PROBE v(x*.*)`
- Print all second-level nodes. For example: x1.x2.a, xab.xdff.in.
`.PRINT v(x*.*.*)`
- Match all first-level nodes with names that are exactly two characters long. For example: x1.in, x4.12.
`.PROBE v(x*.??)`
- Probe nodes that combine variables, with a specific range of possible digits. This example outputs the `x*.00` through `x*.99` nodes. The node name must be two characters long, and must be integers from 00 to 99, inclusive.
`.PROBE v(x*.[0-9][0-9])`
- Print all first-level nodes, where zero-level are top-level nodes, that are only two characters long. However, the first character must be either 1, 2, or 3, and the second character must be either a, b, or c. For example: xdd.1b, xdd.2c, xy.3a.
`.PRINT v(x*.[123][abc])`
- Print all second-level nodes that start with a, b, c, d, or e.
`.PRINT v(*.*.[a-e]*)`

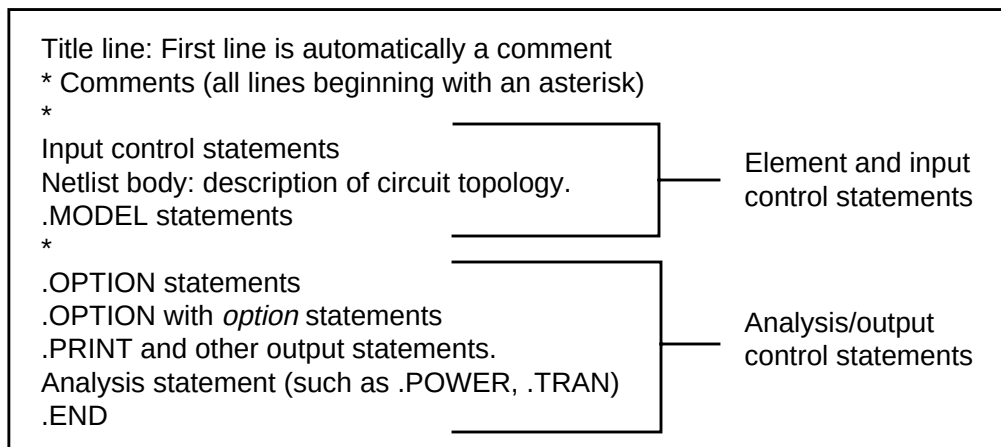
Netlist Overview

The circuit description syntax, for Star-Hspice, is compatible with the SPICE input netlist format.

Basic Structure

Figure 2-1 shows the basic structure of an input netlist.

Figure 2-1: Basic Netlist Structure



Example

This example of a simple netlist file, called *inv_ckt.in*, shows a small inverter test case, which measures the timing behavior of the inverter. To create the circuit:

1. Define the MOSFET models for the PMOS and NMOS transistors of the inverter.
2. Insert the power supplies for both VDD and GND power rails.
3. Insert the pulse source to the inverter input.

This circuit uses transient analysis, and produces output graphical waveform data, for the input and output ports of the inverter circuit.

```

* Sample inverter circuit
* **** MOS models ****
.MODEL n1 NMOS LEVEL=3 THETA=0.4 ...
.MODEL p1 PMOS LEVEL=3 ...
* **** Define power supplies and sources ****
VDD VDD 0 5
VPULSE VIN 0 PULSE 0 5 2N 2N 2N 98N 200N
VGND GND 0 0
* **** Actual circuit topology ****
M1 VOUT VIN VDD VDD p1
M2 VOUT VIN GND GND n1
* **** Analysis statement ****
.TRAN 1n 300n
* **** Output control statements ****
.OPTION POST PROBE
.PROBE V(VIN) V(VOUT)
.END

```

First Character

The first character in every line specifies how Star-Hspice interprets the remaining line. [Table 2-2](#) lists and describes the valid characters.

Table 2-2: First Character Descriptions

Line	If the First Character is...	Indicates
First line of a netlist	Any character	Comment line
Subsequent lines of netlist, and all lines of included files	.	Netlist keyword
	c, C, d, D, e, E, f, F, g, G, h, H, i, I, k, K, l, L, m, M, q, Q, r, R, v, V	Element instantiation
	*	Comment line
	+	Continues previous line

The first line of a netlist is a comment, no matter what letter starts the line. The first line of an included file is a normal line, not a comment.

Example

The . character at the start of the line below, indicates that .TRAN is a keyword:

```
.TRAN 0.5ns 20ns
```

Adding Elements

Lines that add an instance of an element begin with a specific letter, as shown in [Table 2-3](#).

Table 2-3: Element Identifiers

Letter (First Character)	Element	Example Line
C	Capacitor	Cbypass 1 0 10pf
D	Diode	D7 3 9 D1
E	Voltage-controlled voltage source	Ea 1 2 3 4 K
F	Current-controlled current source	Fsub n1 n2 vin 2.0
G	Voltage-controlled current source	G12 4 0 3 0 10
H	Current-controlled voltage source	H3 4 5 Vout 2.0
I	Current source	I A 2 6 1e-6
L	Linear inductor	LX a b 1e-9
M	MOS transistor	M834 1 2 3 4 N1
Q	Bipolar transistor	Q5 3 6 7 8 pnp1
R	Resistor	R10 21 10 1000
V	Voltage source	V1 8 0 5

Netlist input processing is case insensitive, except for file names and their paths. Star-Hspice does not limit the identifier length, line length, or file size.

Comments and Line Continuation

The first line of a netlist is always a comment, regardless of what character appears first; comment lines that are not the first line of the netlist require either an asterisk (*) as the first character of the line, or a dollar sign (\$) directly in front of the comment, anywhere on the line.

- You can insert all comment text, after and including the \$, anywhere in a line of code, not just at the beginning of a line (as required when you use *).
- \$ is the preferred way to indicate comments, because of the flexibility of its placement within the code.
- Line continuations require + as the first character in the line that follows.

Example

```
.ABC Title Line (Star-Hspice ignores the netlist
* keyword on this line, because the first line is
* always a comment)
* This line is a comment
.MODEL n1 NMOS $this is an example of an inline comment
* The following line is a continuation
+ LEVEL = 3
```

Software Conventions

Subcircuit Node Names

To assign subcircuit node names, Star-Hspice traces the node, from the top level of the circuit, to the final subcircuit level. It then concatenates the different level names, using . as a delimiter.

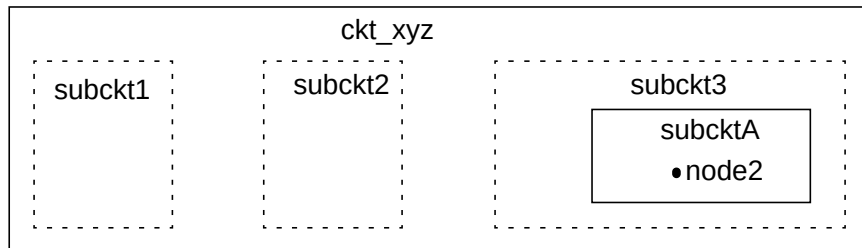
To change the delimiter, use the *hier_delimiter* entry in the configuration file (see [Creating a Configuration File on page 2-4](#)). The last name must be a node name or an element.

Example

In [Figure 2-2](#), the top-level (main) circuit is named *ckt_xyz*. This circuit contains three subcircuits: *subckt1*, *subckt2*, and *subckt3*. The *subckt3* subcircuit contains another subcircuit, called *subcktA*. To reference the *node2* node, specify the path as follows:

```
subckt3.subcktA.node2
```

Figure 2-2: Example Top-Level Circuit



Reserved Operator Keywords

The following symbols are reserved operator keywords:

, () = “ ’

Do not use these symbols as part of any parameter or node name that you define. Using any of these reserved operator keywords as names causes a syntax error, and Star-Hspice stops immediately.

Scale Factors for Numbers

Numbers can use any of the following formats:

- Integers.
- Floating-point values.
- A floating-point number, followed by an integer exponent (scientific notation).

- An integer, or a floating-point number, followed by one of the scale factors listed in [Table 2-4](#).

Table 2-4: Scale Factors

Scale Factor	Prefix	Multiplying Factor
T	tera	1e+12
G	giga	1e+9
MEG or X	mega	1e+6
K	kilo	1e+3
M	milli	1e-3
U	micro	1e-6
N	nano	1e-9
P	pico	1e-12
F	femto	1e-15
A	atto	1e-18

Note: A is not a scale factor in a character string that contains *amps*. For example, Star-Hspice interprets the 20ampb string as 20e-18mpb (20^{-18} mpb), but it correctly interprets 20amps as 20 amperes of current, not as 20e-18mps (20^{-18} mps).



Chapter 3

Simulation Input and Controls

This chapter describes the input requirements, methods of entering data, and Star-Hspice statements used to enter input. This chapter explains the following topics:

- [Using Netlist Input Files](#)
- [Input Netlist File Composition](#)
- [Using Subcircuits](#)
- [Discrete Device Libraries](#)
- [Using Standard Input Files](#)
- [Starting Star-Hspice](#)
- [Improving Simulation Performance Using Multithreading](#)
- [Using PKG and EBD Simulation](#)
- [Star-Hspice Output Files](#)

Using Netlist Input Files

This section describes how to use standard netlist input files.

Input Netlist File Guidelines

Star-Hspice operates on an input netlist file, and stores results in either an output listing file or a graph data file. An input file, with the name *<design>.sp* (use this form for clarity), contains the following:

- Design netlist (with subcircuits and macros, power supplies, and so on).
- Statement naming the library to use (optional).
- Specifies the type of analysis to run (optional).
- Specifies the type of output desired (optional).

To generate input netlist and library input files, Star-Hspice uses either a schematic netlister or a text editor.

Statements in the input netlist file can be in any order, except that the first line is a title line, and the last `.ALTER` submodule must appear at the end of the file, before the `.END` statement.

Note: If you do not place an `.END` statement and a [Return] at the end of the input netlist file, Star-Hspice issues an error message.

Input Line Format

- The input netlist file cannot be in a packed or compressed format.
- The input reader can accept an input token, such as:
 - a statement name.
 - a node name.
 - a parameter name or value.

Any valid string of characters, between two token delimiters, is a token.
See [Delimiters on page 3-3](#).

- An input filename, statement, or equation can be up to 1024 characters long.
- Star-Hspice ignores differences between upper and lower case in input lines, except in quoted filenames.

- To continue a statement on the next line, in Star-Hspice, enter a plus (+) sign as the first non-numeric, non-blank character in the next line.
- To continue all Star-Hspice statements, including quoted strings (such as paths and algebras), use a backslash (\) or a double backslash (\\) at the end of the line that you want to continue.
 - A single backslash preserves white space.
 - A double backslash squeezes out any white space between the continued lines. The double backslash guarantees that path names are joined without interruption.

Input lines can be 1024 characters long, so folding and continuing a line is generally necessary only to improve readability.

- You can add comments anywhere in a Star-Hspice file. Lines that begin with an asterisk (*) are comments. To place a comment on the same line as input text, enter a dollar sign (\$), preceded by one or more blanks, after the input text.
- If your input netlist file includes any special control characters, Star-Hspice reports an error. Most systems cannot print control characters, so the error message is ambiguous, because the error message cannot show the erroneous character. Use the .OPTION BADCHAR statement to locate such errors. The default for BADCHAR is off.

Names

- Names must begin with an alphabetic character, but thereafter can contain numbers and the following characters:
 ! # \$ % * + - / < > [] _
- Names are input tokens. Token delimiters must precede and follow these names. See [Delimiters on page 3-3](#).
- Names can be 1024 characters long.
- Names are not case sensitive.

Delimiters

- An input token is any item in the input file that Star-Hspice recognizes. Input token delimiters are: tab, blank, comma, equal sign (=), and parentheses ().
- Single or double quotes delimit expressions and filenames.
- Colons delimit element attributes (for example, M1:VGS).
- Periods indicate hierarchy. For example, X1.X2.n1 is the n1 node on the X2 subcircuit of the X1 circuit.

Nodes

- Node identifiers can be up to 1024 characters long, including periods and extensions.
- Numerical node names are valid in the range of 0 through 9999999999999999 (1-1E16).
- Star-Hspice ignores leading zeros in node numbers.
- Star-Hspice ignores trailing characters in node numbers. For example, node 1A is the same as node 1. Exception: Star-Hspice recognizes the following special alphabetic trailing characters (a, d, e, f, g, i, k, m, n, o, p, t, u, x).
- A node name can begin with any of the following characters: # _ ! %.
- To make nodes global across all subcircuits, use a .GLOBAL statement.
- The 0, GND, GND!, and GROUND node names all refer to the global Star-Hspice ground. Simulation treats nodes with any of these names as a *ground* node, and produces v(0) into the output files.

Instance Names

- The names of element instances begin with the element key letter (for example, M for a MOSFET element, D for a diode, R for a resistor, and so on), except in subcircuits.
- Subcircuit instance names begin with X. (Subcircuits are sometimes called macros or modules.)
- Instance names are limited to 1024 characters.
- .OPTION LENNAM defines the length of names in printouts (default = 8).

Hierarchy Paths

- A period indicates path hierarchy.
- Paths can be up to 1024 characters long.
- Path numbers compress the hierarchy, for post-processing and listing files.
- You can find path number cross references in the listing and in the <design>.pa0 file.
- .OPTION PATHNUM controls whether the list files show full path names or path numbers.

Numbers

- You can enter numbers as integer or real.
- Numbers can use exponential format or engineering key letter format, but not both (1e-12 or 1p, but not 1e-6u).

- To designate exponents, use D or E.
- `.OPTION EXPMAX` limits the exponent size.
- Star-Hspice interprets trailing alphabetic characters as units comments.
- Star-Hspice does not check units comments.
- `.OPTION INGOLD` controls the format of numbers in printouts.
- `.OPTION NUMDGT = x` controls the listing printout accuracy.
- `.OPTION MEASDGT = x` controls the measure file printout accuracy.
- `.OPTION VFLOOR = x` specifies the smallest voltage for which Star-Hspice prints the value. Smaller voltages print as 0.

Parameters and Expressions

- Parameter names in Star-Hspice use Hspice name syntax rules, except that names must begin with an alphabetic character. The other characters must be either a number, or one of these characters:
! # \$ % [] _
- To define parameter hierarchy overrides and defaults, use the `.OPTION PARHIER = global | local` statement.
- If you create multiple definitions for the same parameter or option, Star-Hspice uses the last parameter definition or `.OPTION` statement, even if that definition occurs *later* in the input than a reference to the parameter or option. Star-Hspice does not warn you when you redefine a parameter.
- You must define a parameter, *before* you use that parameter to define another parameter.
- When you select design parameter names, be careful to avoid conflicts with parameterized libraries.
- To delimit expressions, use single or double quotes.
- Expressions cannot exceed 256 characters.
- For improved readability, use a double slash (`\\`) at end of a line, to continue the line.
- You can nest functions up to three levels.
- Any function that you define can contain up to two arguments.
- Use the `PAR(expression or parameter)` function to evaluate expressions in output statements.

Input Netlist File Structure

An input netlist file should consist of one main program, and one or more optional submodules. Use a submodule (preceded by an `.ALTER` statement) to automatically change an input netlist file; then rerun the simulation with different options, netlist, analysis statements, and test vectors.

You can use several high-level call statements (`.INCLUDE`, `.LIB` and `.DEL LIB`) to restructure the input netlist file modules. These statements can call netlists, model parameters, test vectors, analysis, and option macros into a file, from library files or other files. The input netlist file also can call an external data file, which contains parameterized data for element sources and models.

Schematic Netlists

Star-Hspice typically uses netlists to generate circuits from schematics, and accept either hierarchical or flat netlists. The normal SPICE netlists flatten all subcircuits, and rename all nodes to numbers. Avoid flat netlists if possible.

The process of creating a schematic involves:

- Symbol creation with a symbol editor.
- Circuit encapsulation.
- Property creation.
- Symbol placement.
- Symbol property definition.
- Wire routing and definition.

Input Netlist File Sections and Chapter References

Sections	Examples	Chapter	Definition
Title	<code>.TITLE</code>	3	The first line in the netlist is the title of the input netlist file.

Sections	Examples	Chapter	Definition
Set-up	.OPTION	9	Sets conditions for simulation.
	.IC or .NODESET	10	Initial values in circuit and subcircuit.
	.PARAM	7	Set parameter values in the netlist.
	.GLOBAL	7	Set node name globally in netlist.
Sources	Sources and digital inputs	5	Sets input stimuli (I or V).
Netlist	Circuit elements	3-4	Circuit for simulation.
	.SUBKCT, .ENDS	3	Subcircuit definitions.
Analysis	.DC, .TRAN, .AC, etc.	10-12	Statements to perform analyses.
	.SAVE and .LOAD	10	Save and load operating point info.
	.DATA	3	Create table for data-driven analysis.
	.TEMP	3	Set analysis temperature.
Output	.PRINT, .PLOT, .GRAPH, .PROBE	8	Statements to output variables.
	.MEASURE	8	Statement to evaluate and report user-defined functions of a circuit.
Library, Model and File Inclusion	.INCLUDE	3	General include files.
	.MALIAS	3	Assigns an alias to a diode, BJT, JFET, or MOSFET.
	.MODEL	3, 8	Element model descriptions.
	.LIB	3	Library.
	.<UN>PROTECT	3	Control printback to output listing.
Alter blocks	.ALIAS	3	Renames a previous model.
	.ALTER	3	Sequence for in-line case analysis.
	.DELETE LIB	3	Removes previous library selection.
End of netlist	.END	3	Required statement, to end the netlist.

Input Netlist File Composition

Title of Simulation and .TITLE Statement

You set the simulation title in the first line of the input file. Star-Hspice always reads this line, and uses it as the title of the simulation, regardless of the contents of this line. The simulation prints the title verbatim, in each section heading of the output listing file.

As shown in the first syntax below, to set the title, you can place a .TITLE statement on the first line of the netlist. However, Star-Hspice does not *require* the .TITLE syntax. In the second form shown below, the string is the first line of the input file. The first line of the input file is always the implicit title. If any statement appears as the first line in a file, simulation interprets it as a title, and does not execute it.

An .ALTER statement does not support use the .TITLE statement. To change a title for a .ALTER statement, place the title content in the .ALTER statement itself.

Syntax

```
.TITLE <string_of_up_to_72_characters>
```

or

```
<string_of_up_to_72_characters>
```

Comments

An asterisk (*) as the first non-blank character, or an inline dollar sign (\$) that is not the first character on the line, indicates a comment statement.

Syntax

```
* <comment_on_a_line_by_itself>
```

or

```
<HSPICE_statement> $ <comment_following_HSPICE_input>
```

Example

```
*RF = 1K    GAIN SHOULD BE 100
$ MAY THE FORCE BE WITH MY CIRCUIT
VIN 1 0 PL 0 0 5V 5NS $ 10v 50ns
R12 1 0 1MEG $ FEED BACK
```

You can place comment statements anywhere in the circuit description. The * must be in the first space on the line.

The \$ must be used for comments that do *not* begin at the first space on a line (for example, for comments that follow simulator input on the same line). The \$ must be preceded by a space or comma, if it is not the first nonblank character. You can place the \$ within node or element names.

Element and Source Statements

Element statements describe the netlists of devices and sources. Use nodes to connect elements to one another. Nodes can be either numbers or names.

Element statements specify:

- Type of device.
- Nodes to which the device is connected.
- Operating electrical characteristics of the device.

Element statements can also reference model statements that define the electrical parameters of the element.

For descriptions of element statements for the various types of supported elements, see the chapters about individual types of elements, in this user guide.

Syntax

```
elname <node1 node2 ... nodeN> <mname>
+ <pname1 = val1> <pname2 = val2> <M = val>
```

or

```
elname <node1 node2 ... nodeN> <mname>
+ <pname = 'expression'> <M = val>
```

or

```
elname <node1 node2 ... nodeN> <mname>
+ <val1 val2 ... valn>
```

where:

<i>elname</i>	Element name that cannot exceed 1023 characters, and must begin with a specific letter for each element type:
B	IBIS buffer
C	Capacitor
D	Diode
E,F,G,H	Dependent current and voltage sources
I	Current (inductance) source
J	JFET or MESFET
K	Mutual inductor
L	Inductor
M	MOSFET
Q	BJT
R	Resistor
T,U,W	Transmission line
V	Voltage source
X	Subcircuit call
<i>node1 ...</i>	Node names identify the nodes that connect to the element. Node names must begin with a letter, followed by up to 1023 additional alphanumeric characters. You cannot use the following characters in node names: = (), ' <space>
<i>mname</i>	Star-Hspice requires a model reference name for all elements, except passive devices.
<i>pname1 ...</i>	An element parameter name identifies the parameter value that follows this name.
<i>expression</i>	Any mathematical expression containing values or parameters, such as <i>param1</i> * <i>val2</i>
<i>val1 ...</i>	Value of the <i>pname1</i> parameter, or to the corresponding model node. The value can be a number or an algebraic expression.
<i>M = val</i>	Element multiplier. Replicates the <i>val</i> element times, in parallel.

Example

```
Q1234567 4000 5000 6000 SUBSTRATE BJTMODEL AREA = 1.0
```

The above example specifies a bipolar junction transistor, with its collector connected to node 4000, its base connected to node 5000, its emitter connected to node 6000, and its substrate connected to the SUBSTRATE node. The BJTMODEL name references the model statement, which describes the transistor parameters.

```
M1 ADDR SIG1 GND SBS N1 10U 100U
```

The above example specifies a MOSFET named M1, where:

- drain node=ADDR.
- gate node=SIG1.
- source node=GND.
- substrate nodes= SBS.

The above element statement calls an associated model statement, N1. The MOSFET dimensions are width = 100 microns and length = 10 microns.

```
M1 ADDR SIG1 GND SBS N1 w1+w l1+l
```

The above example specifies a MOSFET named M1, where:

- drain node=ADDR.
- gate node=SIG1.
- source node=GND.
- substrate nodes= SBS.

The above element statement calls an associated model statement, N1. MOSFET dimensions are algebraic expressions (width = w1+w, and length = l1+l).

.SUBCKT or .MACRO Statement

You can use .subckt and .macro statements in Star-Hspice.

Syntax

```
.SUBCKT subnam n1 < n2 n3 ...> < parnam = val ...>
.ENDS
```

or

```
.MACRO subnam n1 < n2 n3 ... > < parnam = val ...>
.EOM
```

where:

<i>subnam</i>	Specifies a reference name for the subcircuit model call.
<i>n1 ...</i>	Node numbers for external reference; cannot be the ground node (zero). Any element nodes that are in the subcircuit, but are not in this list, are strictly local, with three exceptions: 1. Ground node (zero). 2. Nodes assigned using <code>BULK = node</code> in the MOSFET or BJT models. 3. Nodes assigned using the <code>.GLOBAL</code> statement.
<i>parnam</i>	A parameter name set to a value. For use only in the subcircuit. To override this value, use an assignment in the subcircuit call, or set a value in a <code>.PARAM</code> statement.

Example

```
*FILE SUB2.SP TEST OF SUBCIRCUITS
.OPTION LIST ACCT
    V1 1 0 1
.PARAM P5 = 5 P2 = 10
.SUBCKT SUB1 1 2 P4 = 4
    R1 1 0 P4
    R2 2 0 P5
    X1 1 2 SUB2 P6 = 7
    X2 1 2 SUB2
.ENDS
*
.MACRO SUB2 1 2 P6 = 11
    R1 1 2 P6
    R2 2 0 P2
.EOM
    X1 1 2 SUB1 P4 = 6
    X2 3 4 SUB1 P6 = 15
    X3 3 4 SUB2
*
.MODEL DA D CJA = CAJA CJP = CAJP VRB = -20 IS = 7.62E-18
+ PHI = .5 EXA = .5 EXP = .33
.PARAM CAJA = 2.535E-16 CAJP = 2.53E-16
.END
```

The preceding example defines two subcircuits: SUB1 and SUB2. These are resistor divider networks, whose resistance values are parameters (variables). The X1, X2, and X3 statements call these subcircuits. Because the resistor values are different in each call, these three calls produce different subcircuits.

.ENDS or .EOM Statement

Syntax

```
.ENDS <SUBNAM>
```

or

```
.EOM <SUBNAM>
```

Example

```
.ENDS OPAMP
.EOM MAC3
```

This statement must be the last for any subcircuit definition. The subcircuit name, if included, indicates which definition to terminate. You can nest subcircuit references (calls) within subcircuits, in Star-Hspice.

Subcircuit Call Statement

Syntax

```
Xyyy n1 <n2 n3 ...> subnam <parnam = val ...> <M = val>
```

where:

<i>Xyyy</i>	Subcircuit element name. Must begin with an <i>X</i> , followed by up to 15 alphanumeric characters.
<i>n1 ...</i>	Node names, for external reference.
<i>subnam</i>	Subcircuit model reference name.
<i>parnam</i>	A parameter name set to a value (<i>val</i>), for use only in the subcircuit. It overrides a parameter value in the subcircuit definition, but is overridden by a value set in a .PARAM statement.

M Multiplier. Makes the subcircuit appear as M subcircuits in parallel. You can use this multiplier to characterize circuit loading. Star-Hspice does not need additional calculation time, to evaluate multiple subcircuits.

Example 1

```
X1 2 4 17 31 MULTI WN = 100 LN = 5
```

The above example calls a subcircuit model named MULTI. It assigns the $WN = 100$ and $LN = 5$ parameters in the .SUBCKT statement (not shown). The subcircuit name is X1. All subcircuit names must begin with X.

Example 2

```
.SUBCKT YYY NODE1 NODE2 VCC = 5V
.IC NODEX = VCC
  R1 NODE1 NODEX 1
  R2 NODEX NODE2 1
.EOM
XXXX 5 6 YYY VCC = 3V
```

The preceding example defines a subcircuit named YYY. The subcircuit consists of two 1-ohm resistors in series. The .IC statement uses the VCC passed parameter to initialize the NODEX subcircuit node.

Note: If you initialize a non-existent subcircuit node, Star-Hspice generates a warning message. This can occur if you use an existing .ic file (initial conditions) to initialize a circuit that you modified since you created the .ic file.

Element and Node Naming Conventions

Node Names

Nodes are the points of connection between elements in the input netlist file. You can use either names or numbers to designate nodes. Node numbers can be from 1 to 999999999999999; node number 0 is always ground. Star-Hspice ignores letters that follow numbers in node names. Node names must begin with a letter, followed by up to 1023 characters.

In addition to letters and digits, node names can include the following characters:

+	plus sign
-	minus sign or hyphen
*	asterisk
/	slash
\$	dollar sign
#	pound sign
[]	left and right square brackets
!	exclamation mark
<>	left and right angle brackets
_	underscore
%	percent sign

You can use braces { } in node names, but Star-Hspice changes them to brackets []. Also, you cannot use the following characters in node names:

()	left and right parentheses
,	comma
=	equal sign
'	apostrophe
	blank space

Also, period (.) is reserved for use as a separator between the subcircuit name and the node name:

<subcircuitName>.<nodeName>

The sorting order for operating point nodes is:

a-z, !, #, \$, %, *, +, -, /

Instance and Element Names

Element names in Star-Hspice begin with a letter designating the element type, followed by up to 1023 alphanumeric characters. Element type letters are R for resistor, C for capacitor, M for a MOSFET device, and so on (see [Element and Source Statements on page 3-9](#)).

Subcircuit Node Names

Star-Hspice assigns two subcircuit node names.

- To assign the first name, Star-Hspice uses the (.) extension to concatenate the circuit path name with the node name—for example, X1.XBIAS.M5.

Note: Node designations that start with the same number, followed by any letter, are the same node. For example, 1c and 1d are the same node.

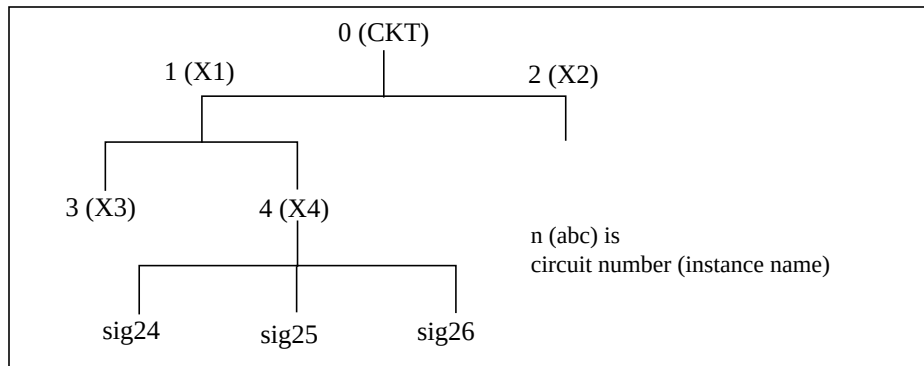
- The second subcircuit node name is a unique number that Star-Hspice automatically assigns to an input netlist file subcircuit. The (:) extension concatenates this number with the internal node name, to form the entire subcircuit's node name (for example, 10:M5). The output listing file cross-references the node name.

To indicate the ground node, use either the number 0, the name GND, or !GND. Every node should have at least two connections, except for transmission line nodes (unterminated transmission lines are permitted) and MOSFET substrate nodes (which have two internal connections). Floating power supply nodes are terminated with a 1-megohm resistor and a warning message.

Path Names of Subcircuit Nodes

A path name consists of a sequence of subcircuit names, starting at the highest-level subcircuit call, and ending at an element or bottom-level node. Periods separate the subcircuit names in the path name. The maximum length of the path name, including the node name, is 1024 characters.

You can use path names in the .PRINT, .PLOT, .NODESET, and .IC statements, as an alternative method to reference internal nodes (nodes not appearing on the parameter list). You can use the path name to reference any node, including any internal node. Subcircuit node and element names follow the rules shown in [Figure 3-1 on page 3-17](#).

Figure 3-1: Subcircuit Calling Tree, with Circuit Numbers and Instance Names

In [Figure 3-1](#), the path name of the *sig25* node, in the X4 subcircuit, is *X1.X4.sig25*. You can use this path in Star-Hspice statements, such as:

```
.PRINT v(X1.X4.sig25)
```

Abbreviated Subcircuit Node Names

In Star-Hspice, you can use circuit numbers as an alternative to path names, to reference nodes or elements in `.PRINT`, `.PLOT`, `.NODESET`, or `.IC` statements. Compiling the circuit assigns a circuit number to all subcircuits, creating an abbreviated path name:

```
<subckt-num> : <name>
```

The subcircuit name and a colon precede every occurrence of a node or element in the output listing file. For example, `4:INTNODE1` denotes a node named `INTNODE1`, in a subcircuit assigned the number 4.

Any node not in a subcircuit has a prefix of 0: (0 references the main circuit). To identify nodes and subcircuits in the output listing file, Star-Hspice uses a circuit number that references the subcircuit where the node or element appears. Abbreviated path names let you use DC operating point node voltage output, as input in a `.NODESET` statement, for a later run.

You can copy the part of the output listing titled *Operating Point Information* or you can type it directly into the input file, preceded by a `.NODESET` statement. This eliminates recomputing the DC operating point in the second simulation.

Automatic Node Name Generation

Star-Hspice can automatically assign internal node names. To check both nodal voltages and branch currents, you can use the assigned node name when you print or plot. Star-Hspice supports several special cases for node assignment — for example, simulation automatically assigns node 0 as a ground node.

For CSOS (CMOS Silicon on Sapphire), if you assign a value of -1 to the bulk node, the name of the bulk node is B#. Use this name to print the voltage at the bulk node. When printing or plotting current—for example `.PLOT I(R1)`—Star-Hspice inserts a zero-valued voltage source. This source inserts an extra node in the circuit named *vnn*, where *nn* is a number that Star-Hspice automatically generates; this number appears in the output listing file.

.GLOBAL Statement

The `.GLOBAL` statement globally assigns a node name, in Star-Hspice. This means that all references to a global node name, used at any level of the hierarchy in the circuit, will be connected to the same node.

The most common use of a `.GLOBAL` statement is if your netlist file includes subcircuits. This statement assigns a common node name to subcircuit nodes. Another common use of `.GLOBAL` statements is to assign power supply connections of all subcircuits. For example, `.GLOBAL VCC` connects all subcircuits with the internal node name VCC. Ordinarily, in a subcircuit, the node name consists of the circuit number, concatenated to the node name. When you use a `.GLOBAL` statement, Star-Hspice does not concatenate the node name with the circuit number, and assigns only the global name. You can then exclude the power node name in the subcircuit or macro call.

Syntax

```
.GLOBAL node1 node2 node3 ...
```

where:

<i>node1 ...</i>	Specifies global nodes, such as supply and clock names; overrides local subcircuit definitions.
------------------	---

Example

This example shows global definitions for the VDD and input_sig nodes.

```
.GLOBAL VDD input_sig
```

.TEMP Statement

To specify the temperature of a circuit for a Star-Hspice simulation, use the .TEMP statement, or the TEMP parameter in the .DC, .AC, and .TRAN statements. Star-Hspice compares the circuit simulation temperature that you set, against the reference temperature in the TNOM control option. Star-Hspice uses the difference between the circuit simulation temperature and the TNOM reference temperature to define derating factors for component values. For information about temperature analysis, see [Temperature Analysis on page 13-5](#).

Star-Hspice permits only one temperature for the entire circuit.

Syntax

```
.TEMP t1 <t2 <t3 ...>>
```

where:

<i>t1 t2 ...</i>	Specifies temperatures, in °C, at which Star-Hspice simulates the circuit.
------------------	--

Example 1

```
.TEMP -55.0 25.0 125.0
```

The .TEMP statement sets the circuit temperatures for the entire circuit simulation. Star-Hspice uses the temperature set in the .TEMP statement, along with the TNOM option setting (or the TREF model parameter) and the DTEMP element temperature, and simulates the circuit with individual elements or model temperatures.

Example 2

```
.TEMP 100
  D1 N1 N2 DMOD DTEMP = 30
  D2 NA NC DMOD
  R1 NP NN 100 TC1 = 1 DTEMP = -30
.MODEL DMOD D IS = 1E-15 VJ = 0.6 CJA = 1.2E-13
+ CJP = 1.3E-14 TREF = 60.0
```

The circuit simulation temperature from the `.TEMP` statement is 100°C. Because this example does not specify `TNOM`, it defaults to 25°C. The `DTEMP` parameter sets the temperature of the diode at 30°C above the circuit temperature; that is, $D1temp = 100^{\circ}C + 30^{\circ}C = 130^{\circ}C$. Star-Hspice simulates the `D2` diode at 100°C, and the `R1` resistor at 70°C. Because the diode model statement sets `TREF` at 60°C, the diode model parameters are derated by 70°C ($130^{\circ}C - 60^{\circ}C$) for the `D1` diode, and by 40°C ($100^{\circ}C - 60^{\circ}C$) for the `D2` diode. The value of `R1` is derated by 45°C ($70^{\circ}C - TNOM$).

.DATA Statement

In data-driven analysis, you can modify any number of parameters, then use the new parameter values to perform an operating point, DC, AC, or transient analysis. An array of parameter values can be either inline (in the simulation input file) or stored as an external ASCII file. The `.DATA` statement associates a list of parameter names with corresponding values in the array.

Star-Hspice supports the full `.DATA` functionality.

- Data-driven analysis.
- Inline or external data files.

Data-driven analysis syntax requires a `.DATA` statement, and an analysis statement that contains a `DATA = dataname` keyword.

You can use the `.DATA` statement to concatenate or column-laminate data sets, to optimize measured I-V, C-V, transient, or s-parameter data.

You can also use the `.DATA` statement for a first or second sweep variable, when you characterize cells, and test worst-case corners. Simulation reads data measured in a lab, such as transistor I-V data, one transistor at a time, in an outer analysis loop. Within the outer loop, the analysis reads data for each transistor (IDS curve, GDS curve, and so on), one curve at a time, in an inner analysis loop.

The `.DATA` statement specifies the parameters for which you want to change values, and specifies the sets of values to assign during each simulation. The required simulations run as an internal loop. This bypasses reading-in the netlist and setting-up the simulation, which saves computing time. In internal loop simulation, you can also plot simulation results against each other, and print them in a single output.

You can enter any number of parameters in a .DATA block. The .AC, .DC, and .TRAN statements can use external and inline data provided in .DATA statements. The number of data values per line does not need to correspond to the number of parameters. For example, you do not need to enter 20 values on each line in the .DATA block, if each simulation pass requires 20 parameters: the program reads 20 values on each pass, no matter how you format the values.

Each .DATA statement can contain up to 50 parameters. If you need more than 50 parameters in a single .DATA statement, place 50 or fewer parameters in the .DATA statement, and use .ALTER statements for the remaining parameters.

Star-Hspice refers to .DATA statements by their datanames, so each dataname must be unique. Star-Hspice support three .DATA statement formats:

- Inline data
- Data concatenated from external files
- Data column laminated from external files

These formats are described below.

- The MER and LAM keywords tell Star-Hspice to expect external file data, rather than inline data.
- The FILE keyword denotes the external filename.
- You can use simple file names, such as *out.dat*, without the single or double quotes (' ' or " "), but use the quotes when file names start with numbers, such as *1234.dat*.
- File names are case sensitive on Unix systems.

For more details about using the .DATA statement in different types of analysis, see [Chapter 9, “Simulation Options”](#), [Chapter 10, “Initializing DC/Operating Point Analysis”](#), and [Chapter 11, “Transient Analysis”](#).

Syntax

Operating point:

```
.DC DATA = dataname
```

DC sweep:

```
.DC vin 1 5 .25 SWEEP DATA = dataname
```

AC sweep:

```
.AC dec 10 100 10meg SWEEP DATA = dataname
```

TRAN sweep:

```
.TRAN 1n 10n SWEEP DATA = dataname
```

For any data-driven analysis, specify the start time (time 0) in the analysis statement, to ensure that the analysis correctly calculates the stop time.

Inline .DATA Statement

Inline data is parameter data, listed in a .DATA statement block. The *datanm* parameter, in a .DC, .AC, or .TRAN analysis statement, calls this statement.

Syntax

```
.DATA datanm pnam1 <pnam2 pnam3 ... pnamxxx >
+   pval1<pval2 pval3 ... pvalxxx>
+   pval1' <pval2' pval3' ... pvalxxx'>
.ENDDATA
```

where:

- datanm* Specifies the data name, referenced in the .TRAN, .DC, or .AC statement.
- pnam_i* Specifies the parameter names used for source value, element value, device size, model parameter value, and so on. You must declare these names in a .PARAM statement.
- pval_i* Specifies the parameter value.

The number of parameters that Star-Hspice reads, determines the number of columns of data. The physical number of data numbers per line does not need to correspond to the number of parameters. For example, if the simulation needs 20 parameters, you do not need 20 numbers per line.

Example

```
.TRAN          1n      100n          SWEEP DATA = devinf
.AC DEC        10      1hz  10khz    SWEEP DATA = devinf
.DC TEMP       -55     125  10       SWEEP DATA = devinf

.DATA devinf          width      length      thresh      cap
+                    50u          30u          1.2v          1.2pf
+                    25u          15u          1.0v          0.8pf
+                    5u           2u           0.7v          0.6pf
+ .                  ...          ...          ...          ...
.ENDDATA
```


Star-Hspice performs the above analyses for each set of parameter values defined in the .DATA statement. For example, the program first uses the width = 50u, length = 30u, thresh = 1.2v, and cap = 1.2pf parameters to perform .TRAN, .AC, and .DC analyses.

Star-Hspice then repeats the analyses for width = 25u, length = 15u, thresh = 1.0v, and cap = 0.8pf, and again for the values on each subsequent line in the .DATA block.

This is an example of .DATA as the inner sweep:

```
M1 1 2 3 0 N W = 50u L = LN
  VGS 2 0 0.0v
  VBS 3 0 VBS
  VDS 1 0 VDS
  .PARAM VDS = 0 VBS = 0 L = 1.0u
  .DC DATA = vdot
  .DATA vdot
      VBS      VDS      L
      0        0.1      1.5u
      0        0.1      1.0u
      0        0.1      0.8u
      -1       0.1      1.0u
      -2       0.1      1.0u
      -3       0.1      1.0u
      0        1.0      1.0u
      0        5.0      1.0u
  .ENDDATA
```

The preceding example performs a DC sweep analysis for each set of VBS, VDS, and L parameters in the .DATA vdot block. That is, Star-Hspice runs eight DC analyses, one for each line of parameter values in the .DATA block.

This is an example of .DATA as an outer sweep:

```
.PARAM W1 = 50u W2 = 50u L = 1u CAP = 0
.TRAN 1n 100n SWEEP DATA = d1
.DATA d1
  W1      W2      L      CAP
  50u     40u     1.0u   1.2pf
  25u     20u     0.8u   0.9pf
.ENDDATA
```

In the previous example:

- The default start time for the .TRAN analysis is 0.
- The time increment is 1 ns.
- The stop time is 100 ns.

This results in transient analyses at every time value from 0 to 100 ns, in steps of 1 ns, using the first set of parameter values in the *.DATA d1* block. Then Star-Hspice reads the next set of parameter values, and performs another 100 transient analyses. It sweeps time from 0 to 100 ns, in 1 ns steps. The outer sweep is time, and the inner sweep varies the parameter values. Star-Hspice performs two hundred analyses: 100 time increments, times 2 sets of parameter values.

External File .DATA Statement

Syntax

This is the syntax for concatenated data files:

```
.DATA datanm MER
  FILE = 'filename1' pname1 = colnum
+ <pname2 = colnum ...>
  <FILE = 'filename2' pname1 = colnum
+ <pname2 = colnum ...>>
...
<OUT = 'fileout'>
.ENDDATA
```

where:

<i>datanm</i>	Data name, referred to in the <i>.TRAN</i> , <i>.DC</i> or <i>.AC</i> statement.
<i>MER</i>	Specifies concatenated (series merging) data files to use.
<i>filenamei</i>	Name of the data file to read. Star-Hspice concatenates files in the order they appear in the <i>.DATA</i> statement. You can specify up to 10 files.
<i>pnamei</i>	Parameter names, used for source value, element value, device size, model parameter value, and so on. You must declare these names in a <i>.PARAM</i> statement.
<i>colnum</i>	Specifies the column number in the data file, for the parameter value. The column does not need to be the same between files.
<i>fileouti</i>	Data file name, where simulation writes concatenated data. This file contains the full syntax for an inline <i>.DATA</i> statement, and can replace the <i>.DATA</i> statement that created it in the netlist. You can output the file, and use it to generate one data file from many.

Concatenated data files are files with the same number of columns, placed one after another. For example, if you concatenate the three files (A, B, and C):

File A	File B	File C
a a a	b b b	c c c
a a a	b b b	c c c
a a a		

the data appears as follows:

```
a a a
a a a
a a a
b b b
b b b
c c c
c c c
```

Note: The number of lines (rows) of data in each file does not need to be the same. The simulator assumes that the associated parameter of each column of the A file is the same as each column of the other files.

Example

```
.DATA inputdata MER
  FILE = 'file1' p1 = 1 p2 = 3 p3 = 4
  FILE = 'file2' p1 = 1
  FILE = 'file3'
.ENDDATA
```

The above listing concatenates *file1*, *file2*, and *file3*, to form the *inputdata* dataset. The data in *file1* is at the top of the file, followed by the data in *file2*, and *file3*. The *inputdata* in the .DATA statement references the dataname specified in either the .DC, .AC, or .TRAN analysis statements. The parameter fields specify the column that contains the parameters (you must already have defined the parameter names in .PARAM statements). For example, the values for the p1 parameter are in column 1 of *file1* and *file2*. The values for the p2 parameter are in column 3 of *file1*.

For data files with fewer columns than others, Star-Hspice assigns values of zero to the missing parameters.

Column Laminated .DATA Statement

Syntax

This is the syntax for column-laminated data files:

```
.DATA datanm LAM
  FILE = 'filename1' pname1 = colnum
+ <pname2 = colnum ...>
  <FILE = 'filename2' pname1 = colnum
+ <pname2 = colnum ...>>
...
  <OUT = 'fileout'>
.ENDDATA
```

where:

<i>datanm</i>	Data name, referred to in the .TRAN, .DC or .AC statement.
<i>LAM</i>	Column-laminated (parallel merging) data files to use.
<i>filenamei</i>	Name of a data file to read. Star-Hspice concatenates files in the order listed in the .DATA statement. Specify up to 10 files.
<i>pnamei</i>	Parameter names used for source value, element value, device size, model parameter value, and so on. You must declare these names in a .PARAM statement.
<i>colnum</i>	Column number in the data file, that contains the parameter value. The column does not need to be the same between files.
<i>fileouti</i>	Name of the data file, where Star-Hspice writes concatenated data. This file contains the complete syntax for an inline .DATA statement, and can replace the .DATA statement that created it. You can output the file, and generate one data file from many.

Column lamination means that the columns of files with the same number of rows, are arranged side-by-side.

For example, three files (*D*, *E*, and *F*) contain the following columns of data:

File D	File E	File F
d1 d2 d3	e4 e5	f6
d1 d2 d3	e4 e5	f6
d1 d2 d3	e4 e5	f6

The laminated data appears as follows:

d1 d2 d3	e4 e5	f6
d1 d2 d3	e4 e5	f6
d1 d2 d3	e4 e5	f6

The number of columns of data does not need to be the same in the three files.

Note: The number of lines (rows) of data in each file does not need to be the same. Star-Hspice interprets missing data points as zero.

Example

```
.DATA dataname LAM
  FILE = 'file1' p1 = 1 p2 = 2 p3 = 3
  FILE = 'file2' p4 = 1 p5 = 2
  OUT = 'fileout'
.ENDDATA
```

This listing laminates columns from *file1*, and *file2*, into the *fileout* output file. Columns one, two, and three of *file1*, and columns one and two of *file2*, are designated as the columns to place in the output file. You can specify up to 10 files per .DATA statement.

Note: If you run Star-Hspice on a different machine than the one on which the input data files reside (such as when you work over a network), use full path names instead of aliases. Aliases might have different definitions on different machines.

.INCLUDE Statement

You can use the .INCLUDE statement in Star-Hspice.

Syntax

```
.INCLUDE '<filepath> filename'
```

where:

<i>filepath</i>	Path name of a file, for computer operating systems that support tree-structured directories.
<i>filename</i>	Name of a file to include in the data file. The file path, plus the file name, can be up to 1024 characters long. You can use any valid file name for the computer's operating system. You <i>must</i> enclose the file path and name in single or double quotation marks.

.MODEL Statement

You can use .MODEL statements in Star-Hspice.

Syntax

```
.MODEL mname type <VERSION = version_number>  
+ <pname1 = val1 pname2 = val2 ...>
```

where:

<i>mname</i>	Model name reference. Elements must use this name to refer to the model.
--------------	--

Note: If model names contain periods (.), the automatic model selector might fail.

type Selects a model type. Must be one of the following.

For Star-Hspice:

AMP	operational amplifier model
C	capacitor model
CORE	magnetic core model
D	diode model
L	magnetic core mutual inductor model
NJF	n-channel JFET model
NMOS	n-channel MOSFET model
NPN	npn BJT model
OPT	optimization model
PJF	p-channel JFET model
PLOT	plot model for the .GRAPH statement
PMOS	p-channel MOSFET model
PNP	pnp BJT model
R	resistor model
U	lossy transmission line model (lumped)
W	lossy transmission line model
SP	S parameter

pname1 ... Parameter name. Assign a model parameter name (*pname1*) from the parameter names for the appropriate model type. Each model section provides default values. For legibility, enclose the parameter assignment list in parentheses, and use either blanks or commas to separate each assignment. Use a plus sign (+) to start a continuation line.

VERSION Star-Hspice version number. Allows portability of the BSIM (LEVEL=13) and BSIM2 (LEVEL = 39) models, between Star-Hspice releases. Star-Hspice release numbers, and the corresponding version numbers, are:

<i>Star-Hspice release</i>	<i>Version number</i>
9007B	9007.02
9007D	9007.04
92A	92.01
92B	92.02
93A	93.01
93A.02	93.02
95.3	95.3
96.1	96.1

The **VERSION** parameter is valid only for LEVEL 13 and LEVEL 39 models. Use it with Star-Hspice Release H93A.02 and higher. If you use the parameter with any other model, or with a release before H93A.02, Star-Hspice issues a warning, but the simulation continues. You can also use **VERSION** to denote the BSIM3v3 version number only, in model LEVELs 49 and 53. For LEVELs 49 and 53, the **HSPVER** parameter denotes the Star-Hspice release number.

Example

```
.MODEL MOD1 NPN BF=50 IS=1E-13 VBF=50 AREA=2 PJ=3,
+ N=1.05
```

.LIB Call and Definition Statements

To create and read from libraries of commonly-used commands, device models, subcircuit analysis, and statements in library files, use the **.LIB** call statement. As Star-Hspice encounters each **.LIB** call name in the main data file, it reads the corresponding entry from the designated library file, until it finds an **.ENDL** statement.

You can also place a **.LIB** call statement in an **.ALTER** block.

.LIB Library Call Statement

Syntax

```
.LIB '<filepath> filename' entryname
```

where:

<i>filepath</i>	Path to a file. Used where a computer supports tree-structured directories. When the LIB file (or alias) is in the same directory where you run Star-Hspice, you do not need to specify a directory path; the netlist runs on any machine. Use the “../” syntax in the filepath, to designate the parent directory of the current directory.
<i>filename</i>	Name of a file to include in the data file. The combination of filepath plus filename can be up to 256 characters long, structured as any filename that is valid for the computer’s operating system. Enclose the file path and file name in single or double quotation marks. Use the “../” syntax in the filename, to designate the parent directory of the current directory.
<i>entryname</i>	Entry name, for the section of the library file to include. The first character of an entryname cannot be an integer.

Example

```
.LIB 'MODELS' cmos1
```

.LIB Library File Definition Statement

To build libraries, use the .LIB statement in a library file. For each macro in a library, use a library definition statement (.LIB entryname) and an .ENDL statement. The .LIB statement begins the library macro, and the .ENDL statement ends the library macro.

Syntax

```
.LIB entryname1
.
. $ ANY VALID SET OF Star-Hspice STATEMENTS
.
.ENDL entryname1
.LIB entryname2
.
. $ ANY VALID SET OF Star-Hspice STATEMENTS
.
.ENDL entryname2
.LIB entryname3
.
. $ ANY VALID SET OF Star-Hspice STATEMENTS
.
.ENDL entryname3
```

The text after a library file entry name must consist of Star-Hspice statements.

.LIB Nested Library Calls

Library calls can call other libraries, if they are different files.

Example

Shown below are an illegal example and a legal example, for the *file3* library.

Illegal:

```
.LIB MOS7
...
.LIB 'file3' MOS7 $ This call is illegal in MOS7 library
...
...
.ENDL
```

Legal:

```
.LIB MOS7
...
.LIB 'file1' MOS8
.LIB 'file2' MOS9
.LIB CTT $ file2 is already open for the CTT entry point
.ENDL
```

You can nest library calls to any depth. Use this capability, with the .ALTER statement, to construct a sequence of model runs. Each run can consist of similar components, using different model parameters, without duplicating the entire input file.

Library Building Rules

1. A library cannot contain .ALTER statements.
2. A library can contain nested .LIB calls to itself, or to other libraries. If you use a relative path in a nested .LIB call, the path starts from the directory of the parent library, not from the work directory. The depth of nested calls is limited only by the constraints of your system configuration.
3. A library *cannot* contain a call to a library of its own entry name, within the same library file.
4. A Star-Hspice library cannot contain the .END statement.
5. .ALTER processing cannot change .LIB statements, within a file that an .INCLUDE statement calls.

The simulator uses the .LIB statement and the .INCLUDE statement, to access the models and skew parameters. The library contains parameters that modify .MODEL statements. The example below is a .LIB of model skew parameters, and features both worst-case and statistical distribution data. The statistical distribution median value is the default, for all non-Monte Carlo analysis.

Example

```
.LIB TT
$TYPICAL P-CHANNEL AND N-CHANNEL CMOS LIBRARY
$ PROCESS: 1.0U CMOS, FAB7
$ following distributions are 3 sigma ABSOLUTE GAUSSIAN
.PARAM TOX = AGAUSS(200,20,3)           $ 200 angstrom +/- 20a
+ XL = AGAUSS(0.1u,0.13u,3)             $ polysilicon CD
+ DELVTON = AGAUSS(0.0,.2V,3)           $ n-ch threshold change
+ DELVTOP = AGAUSS(0.0,.15V,3)          $ p-ch threshold change
.INC '/usr/meta/lib/cmos1_mod.dat'       $ model include file
.ENDL TT

.LIB FF
$HIGH GAIN P-CH AND N-CH CMOS LIBRARY 3SIGMA VALUES
.PARAM TOX = 220 XL = -0.03 DELVTON = -.2V
+ DELVTOP = -0.15V
.INC '/usr/meta/lib/cmos1_mod.dat'       $ model include file
.ENDL FF
```

The model is contained in the */usr/meta/lib/cmos1_mod.dat* include file.

```
.MODEL NCH NMOS LEVEL = 2 XL = XL TOX = TOX  
+ DELVTO = DELVTON .....  
.MODEL PCH PMOS LEVEL = 2 XL = XL TOX = TOX  
+ DELVTO = DELVTOP .....
```

Note: The model keyword (left side) equates to the skew parameter (right side). A model keyword can be the same as a skew parameter.

.OPTION SEARCH Statement

Use this statement to automatically access a library in Star-Hspice.

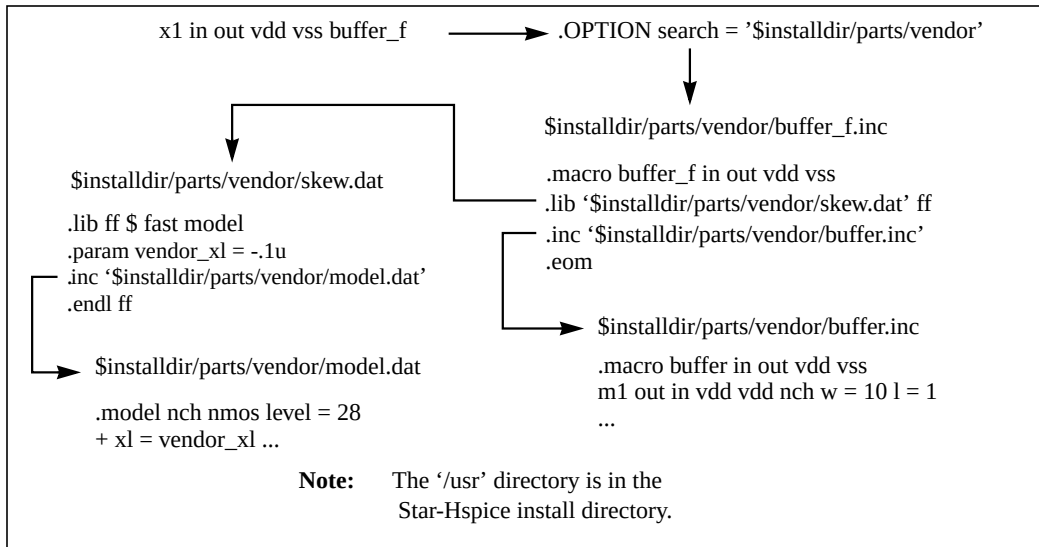
Syntax

```
.OPTION SEARCH = 'directory_path'
```

Example

```
.OPTION SEARCH = '$installdir/parts/vendor'
```

The above example searches for models in the *vendor* subdirectory, under the *\$installdir/parts* installation directory (see [Figure 3-1](#)). The *parts/* directory contains the DDL subdirectories.

Figure 3-1: Vendor Library Usage

Automatic Library Selection

Automatic library selection searches a sequence of up to 40 directories. In Star-Hspice, the *hspice.ini* file sets the default search paths.

Use this file for directories that you want Star-Hspice to always search. Star-Hspice searches for libraries in the order in which `.OPTION SEARCH` statements specify libraries. When Star Hspice encounters a subcircuit call, the search order is as follows:

1. Read the input file, for a `.SUBCKT` or `.MACRO` with the specified call name.
2. Read any `.INC` files or `.LIB` files, for a `.SUBCKT` or `.MACRO` with the specified call name.
3. Search the directory containing the input file, for the `call_name.inc` file.
4. Search the directories in the `.OPTION SEARCH` list.

You can use the Star-Hspice library search and selection features to simulate process corner cases, using `.OPTION SEARCH = '<libdir>'` to target different process directories. For example, if you store an input/output buffer subcircuit in a file named *iobuf.inc*, you can create three copies of the file, to simulate *fast*, *slow* and *typical* corner cases. Each file contains different Star-Hspice transistor models, representing the different process corners. Store these files (all named *iobuf.inc*) in separate directories.

.PARAM Statement

Use the `.PARAM` statement to define parameters. Parameters in Star-Hspice are names that have associated numeric values. You can use any of the following methods to define parameters:

- Simple Parameter Assignments
- Algebraic Parameter (Equation)
- User-Defined Function
- Subcircuit Default Definition
- Predefined Analysis
- Measurement Parameters

Simple Parameter Assignments

You can use simple parameter assignments in Star-Hspice.

A simple parameter assignment is a constant real number. The parameter keeps this value, unless a later definition changes its value, or an algebraic expression assigns a new value during simulation. Star-Hspice does not warn you if it reassigns a parameter.

Syntax

```
.PARAM <ParamName>=<RealNumber>
```

Algebraic Parameter (Equation)

You can use algebraic (equation) parameters in Star-Hspice.

To assign algebraic parameters, use an algebraic expression of real values, a predefined or user-defined function, or circuit or model values. Enclose a complex expression in single quotes to invoke the algebraic processor, *unless* the expression begins with an alphabetic character and contains no spaces. A simple expression consists of a single parameter name.

Syntax

```
.PARAM <ParamName>='<Expression>'
```

or

```
.PARAM <ParamName1>=<ParamName2>
```

To use an algebraic expression as an output variable in a .PRINT, .PLOT, or .PROBE statement, use the PAR keyword in Star-Hspice. For example:

```
.PRINT DC v(3) gain=PAR('v(3)/v(2)')
+ PAR('V(4)/V(2)')
```

Example

```
.para x=cos(2)+sin(2)
```

User-Defined Function

A user-defined function assignment is similar to the definition of an algebraic parameter. Star-Hspice extends the algebraic parameter definition to include function parameters, used in the algebraic that defines the function. You can nest user-defined functions up to three deep.

Syntax

```
.PARAM <ParamName>(<pv1>[<pv2>])='<Expression>'
```

Subcircuit Default Definition

When you use hierarchical subcircuits, you can pick default values for circuit elements. You can use this feature in cell definitions, to simulate the circuit with typical values.

Syntax

```
.SUBCKT <SubName><PinList>[<SubDefaultsList>]
```

where *SubDefaultsList* is:

```
<SubParam1>=<Expression>[<SubParam1>=<Expression>...]
```

Predefined Analysis

Star-Hspice provides several specialized analysis types, that require a way to control the analysis. For the syntax of these uses of .PARAM, see [.PARAM Distribution Function Syntax on page 13-15](#).

Star-Hspice supports the following predefined analysis parameters:

- Temperature functions (*fn*)
- Optimization guess/range
- frequency
- time
- Monte Carlo functions

Measurement Parameters

.MEASURE statements produce a *measurement* parameter. In general, the rules for measurement parameters are the same as those for standard parameters.

However, measurement parameters are not defined in a .PARAM statement, but directly in the .MEASURE statement. For more information, see [.MEASURE Parameter Types on page 8-42](#).

.PROTECT Statement

Use the .PROTECT statement to keep models and cell libraries private, in Star-Hspice.

- The .PROTECT statement suppresses printing text from the list file, such as when you use the BRIEF option.
- Use the .UNPROTECT command to restore normal output functions.
- Any elements and models located between a .PROTECT and an .UNPROTECT statement, inhibit the element and model listing from the LIST option.
- The .OPTION NODE nodal cross reference, and the .OP operating point printout, do not list any nodes that are contained within the .PROTECT and .UNPROTECT statements.

Syntax

```
.PROTECT
```

.UNPROTECT Statement

In Star-Hspice, the .UNPROTECT statement restores normal output functions that a .PROTECT statement restricted.

- Any elements and models located between .PROTECT and .UNPROTECT statements, inhibit the element and model listing from the LIST option.
- Neither the .OPTION NODE cross reference, nor the .OP operating point printout, list any nodes within the .PROTECT and .UNPROTECT statements.

Syntax

.UNPROTECT

.ALTER Statement

You can use the .ALTER statement to rerun a Star-Hspice simulation, using different parameters and data.

Use parameter (variable) values for print and plot statements, before you alter them. The .ALTER block *cannot* include .PRINT, .PLOT, .GRAPH or any other input/output statements. You can include analysis statements (.DC, .AC, .TRAN, .FOUR, .DISTO, .PZ, and so on) in a .ALTER block in an input netlist file.

However, if you are changing *only* the type of analysis, and you do not change the circuit itself, then simulation runs faster if you specify all analysis types in one block, instead of using separate .ALTER blocks for each analysis type.

The .ALTER sequence or block can contain:

- Element statements (except source elements)
- .DATA statements
- .DEL LIB statements
- .INCLUDE statements
- .IC (initial condition) and .NODESET statements
- .LIB statements
- .MODEL statements
- .OP statements
- .OPTION statements
- .PARAM statements
- .TEMP statements
- .TF statements
- .TRAN, .DC, and .AC statements
- .ALIAS statements

Altering Design Variables and Subcircuits

The following rules apply when you alter design variables and subcircuits in Star-Hspice.

1. If the name of a new element, `.MODEL` statement, or subcircuit definition is identical to the name of an original statement of the same type, then the new statement replaces the old. Add new statements in the input netlist file.
2. You can alter element and `.MODEL` statements within a subcircuit definition. You can also add a new element or `.MODEL` statement to a subcircuit definition. To modify the topology in subcircuit definitions, put the element into libraries. To add a library, use `.LIB`; to delete, use `.DEL LIB`.
3. If a parameter name in a new `.PARAM` statement in the `.ALTER` module is identical to a previous parameter name, then the new assigned value replaces the old value.
4. If you used parameter (variable) values for elements (or for model parameter values) when you used `.ALTER`, then use the `.PARAM` statement to change these parameter values. Do not use numerical values to redescribe the elements or model parameters.
5. If you turned on an option, using an `.OPTION` statement in either an original input file or a `.ALTER` block, you can turn that option off.
6. Each `.ALTER` simulation run prints only the actual altered input. A special `.ALTER` title identifies the run.
7. `.ALTER` processing cannot revise `.LIB` statements within a file that an `.INCLUDE` statement calls. However, `.ALTER` processing can accept `.INCLUDE` statements, within a file that a `.LIB` statement calls.

Using Multiple .ALTER Statements

1. For the first simulation run, Star-Hspice reads the input file, up to the first `.ALTER` statement, and performs the analyses up to that `.ALTER` statement.
2. After it completes the first simulation, Star-Hspice reads the input between the first `.ALTER` statement, and either the next `.ALTER` statement or the `.END` statement.

3. Star-Hspice then uses these statements to modify the input netlist file.
4. Star-Hspice then resimulates the circuit.
5. For each additional .ALTER statement, Star-Hspice performs the simulation that precedes the first .ALTER statement.
6. Star-Hspice then performs another simulation, using the input between the current .ALTER statement, and either the next .ALTER statement or the .END statement.

If you do not want to rerun the simulation that precedes the first .ALTER statement, every time you run a .ALTER simulation, then do the following:

1. Put the statements that precede the first .ALTER statement, into a library.
2. Use the .LIB statement in the main input file.
3. Put a .DEL LIB statement in the .ALTER section, to delete that library for the .ALTER simulation run.

Syntax

```
.ALTER <title_string>
```

The *title_string* is any string up to 72 characters. Star-Hspice prints the appropriate title string for each .ALTER run, in each section heading of the output listing, and in the graph data (.tr#) files.

.ALIAS Statement

As listed in the previous section, you can use .alter statements to rename a model, to rename a library containing a model, or to delete an entire library of models in Star-Hspice. If your netlist references the old model name, then after you use one of these types of .alter statements, Star-Hspice no longer finds this model.

For example, if you use .DEL LIB in the .ALTER block to delete a library, the .ALTER command deletes all models in this library. If your netlist references one or more models in the deleted library, then Star-Hspice no longer finds the models.

To resolve this issue, Star-Hspice provides a .ALIAS command, to let you alias the old model name to another model name that Star-Hspice *can* find in the existing model libraries.

For example, you might delete a library named *poweramp*, that contains a model named *pa1*. Another library might contain an equivalent model named *par1*. You can then alias the *pa1* model name to the *par1* model name:

```
.alias pa1 par1
```

During simulation, when Star-Hspice encounters a model named *pa1* in your netlist, it initially cannot find this model, because you used a *.ALTER* statement to delete the library that contained this model. However, the *.ALIAS* statement indicates to use the *par1* model, in place of the old *pa1* model. Star-Hspice *does* find this new model in another library, so simulation continues.

You must specify both an old model name, and a new model name to use in its place. You cannot use the *.ALIAS* command without *any* model names:

```
.ALIAS
```

or with only *one* model name:

```
.ALIAS pa1
```

You also cannot alias a model name to *more than one* model name, because then the simulator would not know which of these new models to use in place of the deleted or renamed model:

```
.ALIAS pa1 par1 par2
```

For the same reason, you cannot alias a model name to a second model name, and then alias the second model name to a third model name:

```
.ALIAS pa1 par1  
.ALIAS par1 par2
```

If your netlist does not contain a *.ALTER* command, and if the *.ALIAS* does not report a usage error, then the *.ALIAS* does not affect the simulation results. For example, your netlist might contain the statement

```
.ALIAS myfet nfet
```

Without a *.ALTER* statement, Star-Hspice does not use *nfet* to replace *myfet* during simulation.

If your netlist contains one or more `.ALTER` commands, the first simulation uses the original *myfet* model. After the first simulation, when the netlist references *myfet* from a deleted library, `.ALIAS` substitutes *nfet* in place of the missing model.

- If Star-Hspice finds model definitions for both *myfet* and *nfet*, it reports an error and aborts.
- If Star-Hspice finds a model definition for *myfet*, but not for *nfet*, it reports a warning, and simulation continues, using the original *myfet* model.
- If Star-Hspice finds a model definition for *nfet*, but not for *myfet*, it reports a *replacement successful* message.

.MALIAS Statement

You can use the `.MALIAS` statement to assign an alias (another name) to a diode, BJT, JFET, or MOSFET model that you defined in a `.MODEL` statement.

The syntax of the `.MALIAS` statement is:

```
.MALIAS model_name=alias_name1 <alias_name2 ...>
```

where:

- *model_name* is the model name defined in the `.model` card.
- *alias_name1...* is the alias that an instance (element) of the model references.

For example:

```
*file: test malias statement
.options acct tnom=50 list gmin=1e-14 post
.temp 0.0 25
.tran .1 2
vdd 2 0 pwl 0 -1 1 1
d1 2 1 zend dtemp=25
d2 1 0 zen dtemp=25

* malias statements
.malias zendef = zen zend

* model definition
.model zendef d (vj=.8 is=1e-16 ibv=1e-9 bv=6.0 rs=10
+ tt=0.11n n=1.0 eg=1.11 m=.5 cjo=1pf tref=50)
.end
```

where `zendef` is a diode model, and `zen` and `zend` are its aliases. The `zendef` model points to both the `zen` and `zend` aliases.

`.malias` differs from `.alias` in two ways:

- The alias name in an `.alias` statement is defined in a `.model` card, but the model card does not define the alias used in a `.malias` statement.
- The `.alias` command works only if you include `.alter` in the netlist. You can use `.malias` without `.alter`.

.CONNECT Statement

This statement connects two nodes in your Star-Hspice netlist, so that simulation evaluates the two nodes as only one node. Both nodes must be at the same level in the circuit design that you are simulating: you cannot connect nodes that belong to different subcircuits.

Syntax

```
.connect node1 node2
```

where:

node1 Name of the first of two nodes to connect to each other.

node2 Name of the second of two nodes to connect to each other. The first node replaces this node in the simulation.

If you connect *node2* to *node1*, Star-Hspice does not recognize *node2* at all. To apply any Star-Hspice statement to *node2*, apply it to *node1* instead. Then, to change the netlist construction to recognize *node2*, use a `.alter` statement. For example:

```
*example for .connect
vcc 0 cc 5v
r1  0 1 5k
r2  1 cc 5k
.tran 1n 10n
.print i(vcc) v(1)
.alter
.connect cc 1
.end
```

The first `.tran` simulation includes two resistors. Later simulations have only one resistor, because *r2* is shorted by connecting *cc* with *1*. `v(1)` does not print out, but `v(cc)` prints out instead.

You can use multiple `.connect` statements to connect multiple nodes together. For example:

```
.connect node1 node2
.connect node2 node3
```

connects both node2 and node3 to node1. All connected nodes must be in the same subcircuit, or all in the main circuit. The first Star-Hspice simulation evaluates only node1; node2 and node3 are the same node as node1. You can use `.alter` statements to simulate node2 and node3.

If you set `.option node`, Star-Hspice prints out a node connection table.

.DEL LIB Statement

You can use the `.DEL LIB` statement in Star-Hspice.

Use the `.DEL LIB` statement to remove library data from memory. The next time you run a simulation, the `.DEL LIB` statement removes the `.LIB` call statement, with the same library number and entry name, from memory. You can then use a `.LIB` statement to replace the deleted library.

In Star-Hspice, you can use the `.DEL LIB` statement with the `.ALTER` statement.

Syntax

```
.DEL LIB '<filepath>filename' entryname
.DEL LIB libnumber entryname
```

where:

<i>entryname</i>	Entry name, used in the library call statement to delete.
<i>filename</i>	Name of a file to delete from the data file. The file path, plus the file name, can be up to 64 characters long. You can use any file name that is valid for the operating system that you use. Enclose the file path and file name in single or double quote marks.
<i>filepath</i>	Path name of a file, if the operating system supports tree-structured directories.
<i>libnumber</i>	Library number, used in the library call statement to delete.

Example 1

This example uses an .ALTER block.

```

FILE1: ALTER1 TEST  CMOS INVERTER
.OPTION ACCT LIST
.TEMP 125
.PARAM WVAL = 15U VDD = 5
*
.OP
.DC VIN 0 5 0.1
.PLOT DC V(3) V(2)
*
VDD 1 0 VDD
VIN 2 0
*
M1 3 2 1 1 P 6U 15U
M2 3 2 0 0 N 6U W = WVAL
*
.LIB 'MOS.LIB' NORMAL
.ALTER
.DEL LIB 'MOS.LIB' NORMAL      $removes LIB from memory
$PROTECTION
.PROT                          $protect statements below .PROT
.LIB 'MOS.LIB' FAST           $get fast model library
.UNPROT
.ALTER
.OPTION NOMOD OPTS            $suppress printing model parameters
                                and print the option summary
*
.TEMP -50 0 50                 $run with different temperatures
.PARAM WVAL = 100U VDD = 5.5   $change the parameters
VDD 1 0 5.5                    $using VDD 1 0 5.5 to change the
                                $power supply VDD value doesn't
                                $work
VIN 2 0 PWL 0NS 0 2NS 5 4NS 0 5NS 5
                                $change the input source
.OP VOL                        $node voltage table of operating
                                $points
.TRAN 1NS 5NS                  $run with transient also
M2 3 2 0 0 N 6U WVAL           $change channel width
.MEAS SW2 TRIG V(3) VAL = 2.5 RISE = 1 TARG V(3)
+ VAL = VDD CROSS = 2          $measure output
*
.END

```


Example 1 calculates a DC transfer function for a CMOS inverter.

1. First, Star-Hspice simulates the device, using the NORMAL inverter model from the MOS.LIB library.
2. Using the .ALTER block and the .LIB command, Star-Hspice substitutes a faster CMOS inverter, FAST, for NORMAL.
3. Star-Hspice then resimulates the circuit.
4. Using the second .ALTER block, Star-Hspice executes DC transfer analysis simulations at three different temperatures, and with an n-channel width of 100 μm , instead of 15 μm .
5. Star-Hspice also runs a transient analysis, in the second .ALTER block. Use the .MEASURE statement to measure the rise time of the inverter.

Example 2

This example uses an .ALTER block.

```

FILE2: ALTER2.SP CMOS INVERTER USING SUBCIRCUIT
.OPTION LIST ACCT
.MACRO INV 1 2 3
M1 3 2 1 1 P 6U 15U
M2 3 2 0 0 N 6U 8U
.LIB 'MOS.LIB' NORMAL
.EOM INV
XINV 1 2 3 INV
VDD 1 0 5
VIN 2 0
.DC VIN 0 5 0. 1
.PLOT V(3) V(2)
.ALTER
.DEL LIB 'MOS.LIB' NORMAL
.TF V(3) VIN          $DC small-signal transfer function
*
.MACRO INV 1 2 3      $change data within subcircuit def
M1 4 2 1 1 P 100U 100U $change channel length,width,also
                        $stoplogy
M2 4 2 0 0 N 6U 8U    $change topology
R4 4 3 100            $add the new element
C3 3 0 10P            $add the new element
.LIB 'MOS.LIB' SLOW   $set slow model library
$.INC 'MOS2.DAT'      $not allowed to be used inside
                        $subcircuit allowed outside
                        $subcircuit

.EOM INV
.END

```

In this example, the `.ALTER` block adds a resistor and capacitor network to the circuit. The network connects to the output of the inverter, and Star-Hspice simulates a DC small-signal transfer function.

.END Statement

An `.END` statement must be the last statement in the input netlist file. The period preceding `END` is a required part of the statement.

Any text that follows the `.END` statement is a comment, and has no effect on that simulation. An input file that contains more than one simulation run, must include an `.END` statement for each simulation run. You can concatenate any number of simulations into a single file.

Syntax

```
.END <comment>
```

Example

```
MOS OUTPUT
.OPTION NODE NOPAGE
VDS 3 0
VGS 2 0
M1 1 2 0 0 MOD1 L = 4U W = 6U AD = 10P AS = 10P
.MODEL MOD1 NMOS VTO = -2 NSUB = 1.0E15 TOX = 1000 U0 = 550
VIDS 3 1
.DC VDS 0 10 0.5 VGS 0 5 1
.PRINT DC I(M1) V(2)
.END MOS OUTPUT

MOS CAPS
.OPTION SCALE = 1U SCALM = 1U WL ACCT
.OP
.TRAN .1 6
V1 1 0 PWL 0 -1.5V 6 4.5V
V2 2 0 1.5VOLTS
MODN1 2 1 0 0 M 10 3
.MODEL M NMOS VTO = 1 NSUB = 1E15 TOX = 1000 U0 = 800 LEVEL = 1
+ CAPOP = 2
.PLOT TRAN V(1) (0,5) LX18(M1) LX19(M1) LX20(M1) (0,6E-13)
.END MOS CAPS
```

Condition-Controlled Netlists (if-else)

```
.if (condition1)
    <statement_block1>
# The following statement block in {braces} is
# optional, and you can repeat it multiple times:
{ .elseif (condition2)
    <statement_block2>
}
# The following statement block in [brackets]
# is optional, and you cannot repeat it:
[ .else (condition3)
    <statement_block3>
]
.endif
```

You can use this if-else structure to change the circuit topology, expand the circuit, set parameter values for each device instance, or select different model cards in each if-else block.

- In an if, elseif, or else *condition* statement, complex Boolean expressions must not be ambiguous. For example, change (a==b && c>=d) to ((a==b) && (c>=d)).
- In an if, elseif, or else *statement_block*, you can include most valid Star-Hspice analysis and output statements. The exceptions are:

.end, .alter, .subckt, .ends, .macro, .eom, .global, .del,
.mailias, .alias, .list, .nolist, and .connect statements.

search, d_ibis, d_imic, d_lv56, biasfi, modsrh, cmiflag, nxx,
and brief options.

- You can include if-elseif-else statements in subcircuits, but you cannot include subcircuits within if-elseif-else statements. However, you can use if-elseif-else blocks to select different submodules, to structure the netlist (using .inc, .lib, and .vec statements).
- If two or more models in an if-else block have the same model name and model type, they must also be the same revision level.
- Parameters in an if-else block do not affect the parameter value within the condition expression. Star-Hspice updates the parameter value only *after* it selects the if-else block.
- You can nest if-else blocks.
- You can include an unlimited number of elseif statements within an if-else block.

- You cannot include sweep parameters or simulation results within an if-else block.
- You cannot use an if-else block within another statement. In the following example, Star-Hspice does not recognize the if-else block as part of the resistor definition:

```
r 1 0
  .if (r_val>10k)
    + 10k
  .else
    + r_val
  .endif
```

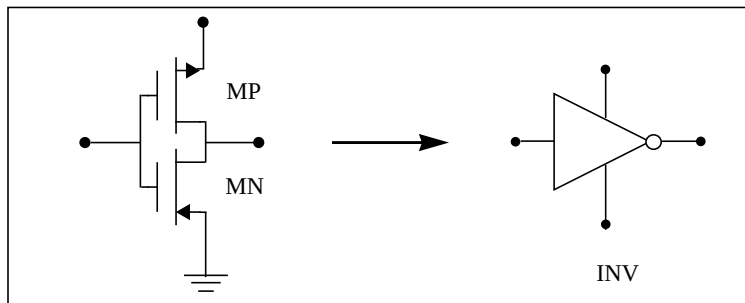
Using Subcircuits

Reusable cells are the key to saving labor in any CAD system. This also applies to circuit simulation, in Star-Hspice.

- To create and simulate a reusable circuit, you must construct it as a subcircuit.
- Use parameters to expand the utility of a subcircuit.

Traditional SPICE includes the basic subcircuit, but does not provide a way to consistently name nodes. However, Star-Hspice provides a simple method for naming subcircuit nodes and elements: use the subcircuit call name as a prefix to the node or element name.

Figure 3-2: Subcircuit Representation



The following input creates an instance named X1 of the INV cell macro, which consists of two MOSFETs, named MN and MP:

```
X1 IN OUT VD_LOCAL VS_LOCAL inv W = 20
  .MACRO INV IN OUT VDD VSS W = 10 L = 1 DJUNC = 0
    MP OUT IN VDD VDD PCH W = W L = L DTEMP = DJUNC
    MN OUT IN VSS VSS NCH W = 'W/2' L = L DTEMP = DJUNC
  .EOM
```

Note: To access the name of the MOSFET, inside of the INV subcircuit that X1 calls, the names are X1.MP and X1.MN. So to print the current that flows through the MOSFETs, use `.PRINT I (X1.MP)`

Hierarchical Parameters

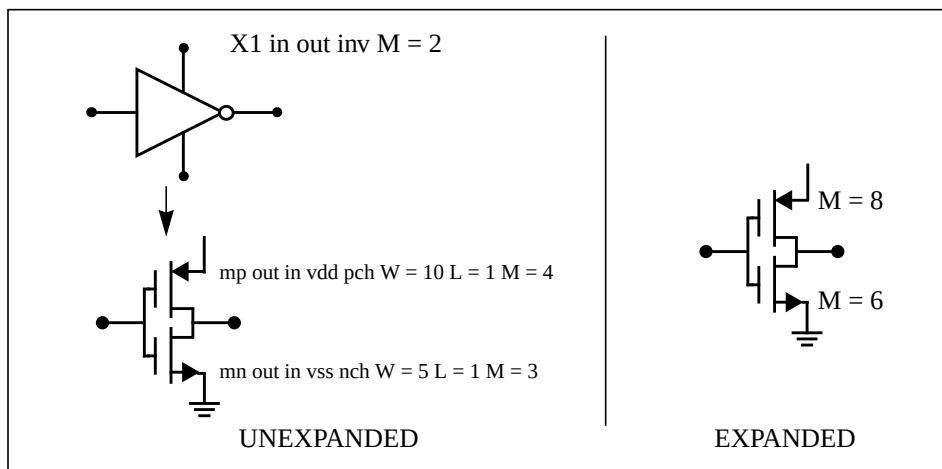
M (Multiply) Parameter

The most basic subcircuit parameter, in Star-Hspice, is the M (multiply) parameter. This keyword is common to all elements, including subcircuits, *except* for voltage sources. The multiply parameter multiplies the internal component values, which in effect creates parallel copies of the element or subcircuit. To simulate 32 output buffers switching simultaneously, you need to place only one subcircuit:

```
X1 in out buffer M = 32
```

Multiply works hierarchically. For a subcircuit within a subcircuit, Star-Hspice multiplies the product of both levels.

Figure 3-3: Hierarchical Parameters Simplify Flip-flop Initialization



Example

```
X1 D Q Qbar CL CLBAR dlatch flip = 0
macro dlatch
+ D Q Qbar CL CLBAR flip = vcc
.nodeset v(din) = flip
xinv1 din qbar inv
xinv2 Qbar Q inv
m1 q CLBAR din nch w = 5 l = 1
m2 D CL din nch w = 5 l = 1
.eom
```

S (Scale) Parameter

To scale a sub-circuit, use the S (local scale) parameter. This parameter behaves in much the same way as the M parameter in the preceding section.

Syntax

```
.OPTION hier_scale=value
.OPTION scale=value
X1 node1 node2 subname S = valueM parameter
```

The option `hier_scale` statement defines how Star-Hspice interprets the `S` parameter, where `value` is either:

- 0 (the default), indicating a user-defined parameter, or
- 1, indicating a scale parameter.

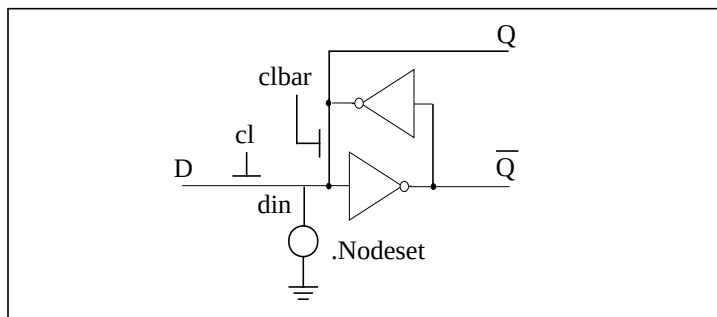
The `.OPTION SCALE` statement defines the original (default) scale of the sub-circuit. The specified `S` scale is relative to this default scale of the sub-circuit.

The scale in the `subname` sub-circuit is `value*scale`. Subcircuits can originate from multiple sources, so scaling is multiplicative (cumulative) throughout your design hierarchy. For example:

```
x1 a y inv S=1u
subckt inv in out
x2 a b kk S=1m
.ends
```

In this example:

1. Star-Hspice scales the `X1` sub-circuit by the first `S` scaling value, `1u*(SCALE)`.
2. Because scaling is cumulative, `X2` (a sub-circuit of `X1`) is then scaled, in effect, by the `S` scaling values of **both** `X1` and `X2`:
 $1m*1u*(SCALE)$

Figure 3-4: D Latch with Nodeset

Star-Hspice does not limit the size or complexity of subcircuits; they can contain subcircuit references, and any model or element statement. To specify subcircuit nodes in .PRINT or .PLOT statements, specify the full subcircuit path and node name.

Undefined Subcircuit Search

If a subcircuit call is in a data file that does not describe the subcircuit, Star-Hspice automatically searches the:

1. Present directory for the file.
2. Directories specified in any .OPTION SEARCH = "*directory_path_name*" statement.
3. Directory where the Discrete Device Library is located.

Star-Hspice searches for the model reference name file that has an .inc suffix. For example, if the data file includes an undefined subcircuit, such as X 1 1 2 INV, Star-Hspice searches the system directories for the *inv.inc* file and, when found, places that file in the calling data file.

Discrete Device Libraries

The Avant! Discrete Device Library (DDL) is a collection of True-Hspice device models of discrete components, which you can use with Star-Hspice. The `$installdir/parts` directory contains the various subdirectories that form the DDL. Avant! used its own ATEM discrete device characterization system to derive the BJT, MESFET, JFET, MOSFET, and diode models from laboratory measurements. The behavior of op-amp, timer, comparator, SCR, and converter models closely resembles that described in manufacturers' data sheets. Star-Hspice has a built-in op-amp model generator.

Note: The value of the `$installdir` environment variable is the path name to the directory where you installed Star-Hspice. This installation directory contains subdirectories, such as `/parts` and `/bin`. It also contains certain files, such as a prototype `meta.cfg` file, and the Star-Hspice license files.

DDL Library Access

To include a DDL library component in a data file, use the *X* subcircuit call statement with the DDL element call. The DDL element statement includes the model name, which the actual DDL library file uses. For example, the following element statement creates an instance of the 1N4004 diode model:

```
X1 2 1 D1N4004
```

where D1N4004 is the model name. See [Element and Source Statements on page 3-9](#) and the *True-Hspice Device Models Reference Manual* for descriptions of element statements.

Optional parameter fields in the element statement can override the internal specification of the model. For example, for op-amp devices, you can override the offset voltage, and the gain and offset current. Because the DDL library devices are based on True-Hspice circuit-level models, simulation automatically compensates for the effects of supply voltage, loading, and temperature.

On most computers, Star-Hspice accesses DDL models in several ways:

1. The installation script creates an *hspice.ini* initialization file.
2. Star-Hspice writes the search path for the DDL and vendor libraries into a `.OPTION SEARCH = '<lib_path>'` statement.

This provides immediate access to all libraries, for all users. It also automatically includes the models in the input netlist. When the input netlist references a model or subcircuit, Star-Hspice searches the directory to which the `= DDLPATH` environment variable points, for a file with the same name as the reference name. This file is an include file, so its filename suffix is *.inc*. Star-Hspice installation sets the DDLPATH variable in the *meta.cfg* configuration file.

3. Set `.OPTION SEARCH = '<library_path>'` in the input netlist.

Use this method to list the personal libraries to search. Star-Hspice first searches the default libraries referenced in the *hspice.ini* file, then searches libraries in the order in which the input file lists them.

4. Directly include a specific model, using the `.INCLUDE` statement. For example, to use a model named T2N2211, store the model in a file named *T2N2211.inc*, and put the following statement in the input file:

```
.INCLUDE <path>/T2N2211.inc
```

Because this method requires you to store each model in its own *.inc* file, it is not generally useful. However, you can use it to debug new models, when you test only a small number of models.

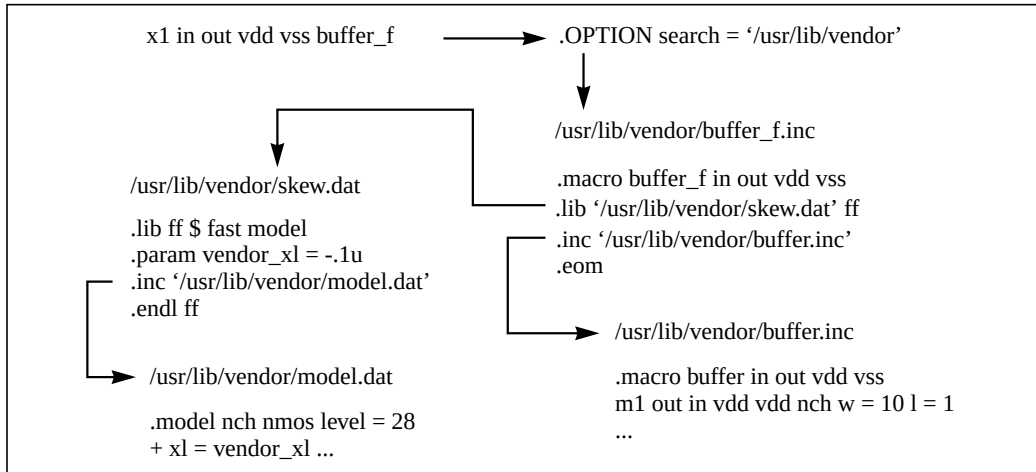
Vendor Libraries

The vendor library is the interface between commercial parts, and circuit or system simulation, in Star-Hspice.

- ASIC vendors provide comprehensive cells, corresponding to inverters, gates, latches, and output buffers.
- Memory and microprocessor vendors supply input and output buffers.
- Interface vendors supply complete cells, for simple functions and output buffers, to use in generic family output.
- Analog vendors supply behavioral models.

To avoid name and parameter conflicts, models in vendor cell libraries should be within the subcircuit definitions.

Figure 3-5: Vendor Library Usage



Subcircuit Library Structure

Your library structure must adhere to the `.INCLUDE` statement specification in the implicit subcircuit. You can use this function in Star-Hspice to specify the directory that contains the `<subname>.inc` subcircuit file, and then reference the name `<subname>` in each subcircuit call.

The Star-Hspice component naming conventions for each subcircuit is:

`<subname>.inc`

Store the subcircuit in a directory that is accessible through the `.OPTION SEARCH = '<libdir>'` statement.

Create subcircuit libraries in a hierarchical structure. Typically, the top-level subcircuit fully describes the input/output buffer, and any hierarchy is buried inside. The buried hierarchy can include lower-level components, model statements, and parameter assignments. Your library cannot use the `.LIB` or `.INCLUDE` statements anywhere in the hierarchy.

Using Standard Input Files

This section describes how to use standard input files in Star-Hspice.

Design and File Naming Conventions

The design name identifies the circuit and any related files, including:

- Schematic and netlist files.
- Simulator input and output files.
- Design configuration files.
- Hardcopy files.

Star-Hspice and AvanWaves extract the design name from their input files, and perform actions based on that name. For example, AvanWaves reads the *<design>.cfg* configuration file, to restore node setups used in previous AvanWaves runs.

Star-Hspice and AvanWaves read and write files related to the current circuit design. Files related to a design usually reside in one directory. The output file is standard output on Unix platforms, and you can redirect it.

[Table 3-1](#) lists input file types, and their standard names. The following sections describe these files.

Table 3-1: Input Files

Input File Type	File Name
Output configuration file	<i>meta.cfg</i>
Initialization file	<i>hspice.ini</i>
DC operating point initial conditions file	<i><design>.ic#</i>
Input netlist file	<i><design>.sp</i>
Library input file	<i><library_name></i>
Analog transition data file	<i><design>.d2a</i>

Configuration File (*meta.cfg*)

This file sets up the printer, plotter, and terminal. It includes a line, *default_include = file name*, which sets up a path to the default *.ini* file (for example, *hspice.ini*).

The *default_include* file name is case-sensitive (except for the PC and Windows versions of Star-Hspice).

Initialization File (*hspice.ini*)

Specify user defaults in an *hspice.ini* initialization file. If the run directory contains an *hspice.ini* file, Star-Hspice includes its contents at the top of the input file.

Other ways to include initialization files are to define `DEFAULT_INCLUDE = <filename>` in the system, or in a *meta.cfg* file.

You can use an initialization file to set options (in an `.OPTION` statement) and to access libraries, as the Avant! installation procedure does.

DC Operating Point Initial Conditions File (<design>.ic#)

The *<design>.ic#* file is an optional input file, which contains initial DC conditions for particular nodes. You can use this file to initialize DC conditions, with either a `.NODESET` or an `.IC` statement.

The `.SAVE` statement creates a *<design>.ic#* file. A subsequent `.LOAD` statement initializes the circuit to the DC operating point values, specified in the *<design>.ic#* file.

Starting Star-Hspice

Use the following syntax to start Star-Hspice:

```
hspice <-i> <path/>input_file <-o path/output_file>  
+ <-n number> <-html<path/html_file>> <-b>
```

where:

input_file Specifies the input netlist file name, for which an extension *<.ext>* is optional. If you do not specify an input filename extension in the command, Star-Hspice searches for the *<input_file>.sp* file. Precede the input file with *-i*. Star-Hspice uses the input filename as the root for the output files. Star-Hspice also checks for an initial conditions file (*.ic*) that has the input file root name. The following is an example of an input file name:

```
/usr/sim/work/rb_design.sp
```

where:

- */usr/sim/work/* is the directory path to the design.
- *rb_design* is the design root name.
- *.sp* is the filename suffix.

-n Specifies the starting number for numbering output data file revisions (*output_file.tr#*, *output_file.ac#*, *output_file.sw#*, where *#* is the revision number).

Table 3-2 lists available Star-Hspice command arguments..

Table 3-2: Star-Hspice Command Options

Option	Description
-b	Batch processing switch, for PC platforms only.
-html <path/>html_file	Specifies an HTML output file. If you do not specify a path, Star-Hspice saves the HTML output file in the same directory that you specified in the -o option. If you do not specify the -o option, Star-Hspice saves the HTML output in the running directory.
-i <input_file>	Name of the input netlist file. If you do not enter an extension, Star-Hspice assumes .sp.
-n <number>	Revision number at which to start numbering .gr#, .tr#, and other output files. By default, file numbers start at zero: .gr0, .tr0, and so on. Use this option to specify the number (-n 7 for .gr7, .tr7, for example).
-o <output_file>	Name of the output file. If you do not specify an extension, Star-Hspice assigns .lis.

You do not need to include a filename extension in the output file specification. Star-Hspice names it *output_file.lis*. In output file names, everything up to the final period is the root filename, and everything after the last period is the filename extension.

If you either do not use the -o option, or you use the -o option without pointing to a filename, then Star-Hspice uses the output root file name specified in the -html option. If you do not specify an output file name in either the -o or -html option, then Star-Hspice uses the input root filename as the output file root filename. If you include the .lis extension in the filename that you enter with -o, Star-Hspice does not append another .lis extension to the output file root filename.

If you do not specify an output file, Star-Hspice directs output to the terminal. Use the following syntax to redirect the output to a file, instead of to the terminal:

```
hspice input_file <-n number> > output_file
```

For example, for the invocation command

```
hspice demo.sp -n 7 > demo.out
```

where:

<i>demo.sp</i>	Is the input netlist file; the <i>.sp</i> extension to the input filename is optional.
-n 7	Starts the output data file revision numbers at 7: <i>demo.tr7</i> , <i>demo.ac7</i> , and <i>demo.sw7</i>
>	Redirects the program output listing to <i>demo.out</i>

Executing a Simulation

Perform these steps to execute a Star-Hspice simulation.

1. Invocation.

To invoke Star-Hspice, use a Unix command such as:

```
hspice demo.sp > demo.out &
```

This command invokes the Star-Hspice shell, and uses an input netlist file named *demo.sp*, and an output listing file named *demo.out*. The & at the end of the command invokes Star-Hspice in the background, so that you can still use the window and keyboard while Star-Hspice runs.

2. Script execution.

The Star-Hspice shell starts the *hspice* executable, from the appropriate architecture (machine type) directory. The Unix *run* script launches a Star-Hspice simulation. This procedure establishes the environment for the Star-Hspice executable. The script prompts for information, such as the platform that you are using, and the version of Star-Hspice to run. (Available versions are determined when you install Star-Hspice.)

3. Licensing.

Star-Hspice supports the FLEXlm *licensing management system*. When you use FLEXlm licensing, Star-Hspice reads the LM_LICENSE_FILE environment variable to find the location of the *license.dat* file.

If Star-Hspice cannot authorize access, the job terminates at this point, and prints an error message in the output listing file.

4. Simulation configuration.

Star-Hspice reads the appropriate *meta.cfg* file. The search order for the configuration file is the user login directory, and then the product installation directory.

5. Design input.

Star-Hspice opens the input netlist file. If the input netlist file does not exist, a *no input data* error appears in the output listing file.

Star-Hspice opens three scratch files in the /tmp directory. To change this directory, reset the TMPDIR environment variable in the Star-Hspice command script.

Star-Hspice opens the output listing file. If you do not own the current directory, Star-Hspice terminates with a *file open* error.

An example of a simple Star-Hspice input netlist is:

Inverter Circuit

```
.OPTION LIST NODE POST
.TRAN 200P 20N SWEEP TEMP -55 75 10
.PRINT TRAN V(IN) V(OUT)
M1 VCC IN OUT VCC PCH L = 1U W = 20U
M2 OUT IN 0 0 NCH L = 1U W = 20U
VCC VCC 0 5
VIN IN 0 0 PULSE .2 4.8 2N 1N 1N 5N 20N CLOAD OUT 0 .75P
.MODEL PCH PMOS
.MODEL NCH NMOS
.ALTER
CLOAD OUT 0 1.5P
.END
```

6. Library input.

Star-Hspice reads any files specified in .INCLUDE and .LIB statements.

7. Operating point initialization.

Star-Hspice reads any initial conditions that you specified in `.IC` and `.NODESET` statements, finds an operating point (that you can save with a `.SAVE` statement), and writes any operating point information that you requested.

8. Multipoint analysis.

Star-Hspice performs the experiments specified in analysis statements. In the above example, the `.TRAN` statement causes Star-Hspice to perform a multipoint transient analysis for 20 ns, for temperatures ranging from -55°C to 75°C, in steps of 10°C.

9. Single-point analysis.

Star-Hspice performs a single or double sweep of the designated quantity, and produces one set of output files.

10. Worst-case `.ALTER` (Star-Hspice only).

You can vary simulation conditions, and repeat the specified single or multipoint analysis. The above example changes `CLOAD` from 0.75 pF to 1.5 pF, and repeats the multipoint transient analysis.

11. Normal termination.

After you complete the simulation, Star-Hspice closes all files that it opened, and releases all license tokens.

Interactive Simulation

To invoke Star-Hspice in *interactive* mode, enter:

```
hspice -I
```

You can then use other Star-Hspice commands to help you simulate circuits interactively. You can also use the `help` command for detailed information about a command.

Star-Hspice interactive mode also supports saving commands into a script file. To save the commands that you use, and replay them later, enter:

```
hspice -I -L scriptfile.cmd
```

Sample Star-Hspice Commands

The following are some additional examples of Star-Hspice commands.

■ **hspice -i demo.sp**

demo is the root filename. Output files are named *demo.lis*, *demo.tr0*, *demo.st0*, and *demo.ic*.

■ **hspice -i demo.sp -o demo**

demo is the output file root name (designated with the -o option). Output files are named *demo.lis*, *demo.tr0*, *demo.st0*, and *demo.ic*.

■ **hspice -i rmdir/demo.sp**

demo is the root name. Star-Hspice writes the *demo.lis*, *demo.tr0*, and *demo.st0* output files into the directory where you executed the Star-Hspice command. It also writes the *demo.ic* output file into the same directory as the input source—that is, *rmdir*.

■ **hspice -i a.b.sp**

a.b is the root name. The output files are *./a.b.lis*, *./a.b.tr0*, *./a.b.st0*, and *./a.b.ic*.

■ **hspice -i a.b -o d.e**

a.b is the root name for the input file.

d.e is the root for output file names, except for the *.ic* file, to which Star-Hspice assigns the *a.b* input file root name. The output files are *d.e.lis*, *d.e.tr0*, *d.e.st0*, and *a.b.ic*.

■ **hspice -i a.b.sp -o outdir/d.e**

a.b is the root for the *.ic* file. Star-Hspice writes the *.ic* file into a file named *a.b.ic*.

d.e is the root for other output files. Output files are *outdir/d.e.lis*, *outdir/d.e.tr0*, and *outdir/d.e.st0*.

■ **hspice -i indir/a.b.sp -o outdir/d.e.lis**

a.b is the root for the *.ic* file. Star-Hspice writes the *.ic* file into a file named *indir/a.b.ic*.

d.e is the root for the output files.

■ hspice test.sp -o test.lis -html test.html

This command creates output file in both .lis and .html format, after simulating the test.sp input netlist.

■ hspice test.sp -html test.html

This command creates only a .html output file, after simulating the test.sp input netlist.

■ hspice test.sp -o test.lis

This command creates only a .lis output file, after simulating the test.sp input netlist.

■ hspice -i test.sp -o -html outdir/a.html

This command creates output files in both .lis and .html format. Both files are in the outdir directory, and their root filename is a.

■ hspice -i test.sp -o out1/a.lis -html out2/b.html

This command creates output files in both .lis and .html format. The .lis file is in the out1 directory, and its root filename is a. The .html file is in the out2 directory, and its root filename is b.

Improving Simulation Performance Using Multithreading

Star-Hspice simulations involve both device model evaluations and matrix solutions. You can run model evaluations concurrently on multiple CPUs, using multithreading, to significantly improve simulation performance. The model evaluation dominates most of the time. To determine how much time Star-Hspice spends in evaluating models and solving matrices, specify `.OPTION acct = 2` in the netlist. Using multithreading results in faster simulations, with no loss of accuracy.

Multithreaded (MT) Star-Hspice is supported on Sun Solaris 2.5.1 (SunOS 5.5.1), Sun Solaris 2.7 (SunOS 5.7), Sun Solaris 2.8 (SunOS 5.8), HP-UX 11.0, PC/RedHat Linux 7.0, PC/RedHat Linux 7.1, Windows NT, Windows 2000, and Windows XP.

Multithreading improves simulation speed, especially for circuit designs that contain many MOSFET, JFET, diode, or BJT models in the netlist.

Running Star-Hspice-MT

To run Star-Hspice-MT, use the syntax described below.

Unix/Linux Platform

On the command line, enter:

```
hspice -mt #num -i input_filename -o output_filename
```

Windows NT Platform

Under the Windows NT DOS prompt, type:

```
hsp_mt -mt #num -i input_filename -o output_filename
```

- If you omit the `#num`, or if the `#num` that you specify is larger than the number of online CPUs, then Star-Hspice sets the number of threads to the number of online CPUs.
- If you omit the `-o output_file` option, Star-Hspice prints the result to the standard output.

Under Windows NT Explorer:

1. Double click the **hsp_mt** application icon.
2. Select the **File/Simulate** button, to select the input netlist file.

In Windows, the program automatically detects the number of online CPUs.

Under the Avant! HSPUI (Star-Hspice User Interface):

1. Select the correct version of *hsp_mt.exe* in the Version Combo Box.
2. Select the correct number of processors in the MT Option Box.
3. Click the **Open** button, to select the input netlist file.
4. Click the **Simulate** button, to start the simulation.

Performance Improvement Estimations

For multithreaded Star-Hspice, the CPU time is:

$$T_{mt} = T_{serial} + T_{parallel}/N_{cpu} + T_{overhead}$$

where:

<i>T_{serial}</i>	Represents Star-Hspice calculations that are not threaded.
<i>T_{parallel}</i>	Represents threaded Star-Hspice calculations.
<i>N_{cpu}</i>	The number of CPUs used. <i>T_{overhead}</i> is the overhead from multithreading. Typically, this represents a small fraction of the total run time.

For example, for a 151-stage *nand* ring oscillator using LEVEL 49, *Tparallel* is about 80% of *T1cpu* (the CPU time associated with a single CPU), if you run with two threads on a multi-CPU machine. Ideally, assuming *Toverhead* = 0, you can achieve a speedup of:

$$T1cpu / (0.2T1cpu + 0.8T1cpu/2cpus) = 1.67$$

The typical value of *Tparallel* is 0.6 to 0.7, for moderate to large circuits.

Using PKG and EBD Simulation

In Star-Hspice, PKG & EBD simulation support package data from [Package], [Pins] and [Define Package Model] sections in *.ibs, *.pkg, and *.ebd files.

You can use Star-Hspice to simulate the packaging, and the board-level trace effects, in the whole system. You can also simulate the packaging effect, and the pin-connected trace effect, stand-alone, with the additional stimulus and loads on the corresponding pins.

Note: The subcircuit interface port names must be same as the pin names listed in the IBIS file.

Options Statements

To support the PKG and EBD feature in system simulation, the Star-Hspice netlist must include the following lines:

```
.OPTION PKGMAP="pkg.map"  
.OPTION EBDMAP="ebd.map"  
.OPTION PKGTYP=RLC / T_LINE  
.OPTION EBDTYP=RLC / T_LINE
```


Parameter	Description
PKGMAP	Specifies the map file name. This file lists the relationship between the Star-Hspice subcircuit and the IBIS component. You can assign this option (with the map file) up to 40 times.
EBDMAP	Specifies the name of a map file, which lists the relationship between Star-Hspice sub-circuit names and: ■ The IBIS board-level module. ■ The <i>X</i> element name in the sub-circuit. ■ The on-board component. You can assign this option (with the map file) up to 40 times.
PKGTYP	Specifies the types of elements to use, to represent the package effect. ■ If the value is RLC (the default), Star-Hspice uses RLC elements as the parasitic packaging elements. ■ If the value is T_LINE, Star-Hspice uses the transmission line. ■ If you specify the package data in matrix form, Star-Hspice uses the <i>w</i> element. ■ If the package data is in the [Package] or [Pin] section, Star-Hspice uses the RLC element. Use this option, only if you use the section form to specify the package data in [Define Package Model], and the section length is not 0.
EBDTYP	Specifies the type of elements to use, to represent the board-level pin connected traces. ■ If the value is RLC, Star-Hspice selects RLC element netlists as the traces. ■ If the value is T_LINE, Star-Hspice uses the transmission line.

PKG Map File

The PKG map file format is:

```
HSP_SUBCIRCUIT_NAME_1 IBIS_FILE_NAME1 COMPONENT_NAME1
HSP_SUBCIRCUIT_NAME_2 IBIS_FILE_NAME2 COMPONENT_NAME2
...           ...           ...
```

Parameter	Description
HSP_SUBCIRCUIT_NAME_1	Specifies the name of the sub-circuit to consider, with the package effect.
IBIS_FILE_NAME1	Specifies the name of the IBIS file that includes the package information, for the HSP_SUBCIRCUIT_NAME_1 sub-circuit.
COMPONENT_NAME1	Specifies the name of the component that corresponds to the sub-circuit.

EBD Map File

The EBD map file format is:

```
[FILE NAME] filename
[EBD MAP DATA]
[BOARD LEVEL SUBCIRCUIT] subcircuit_name
[EBD FILE Name] EBD_file_name
x_element_name1 component_on_board_name1
x_element_name2 component_on_board_name2
...
[END EBD MAP DATA]
```

Parameter	Description
[FILE NAME]	Keyword. The <i>filename</i> argument specifies the file name, to verify file consistency.
[EBD MAP DATA] [END EBD MAP DATA]	Between these two keywords, is information about the relationship between the Star-Hspice sub-circuit and a board-level module. You can specify multiple [EBD MAP DATA] & [END EBD MAP DATA] blocks, but you can include only one board-level module in each block.
[BOARD LEVEL SUBCIRCUIT]	A keyword. The <i>subcircuit_name</i> argument specifies the Star-Hspice sub-circuit, to consider with the trace effect.
[EBD FILE Name]	A keyword. The <i>EBD_file_name</i> argument specifies the name of the file that includes pin-related traces for the subcircuit.
X_ELEMENT_NAME1	Specifies the X element in the Star-Hspice subcircuit, which corresponds to the on-board component specified in COMPONENT_ON_BOARD_NAME1.
COMPONENT_ON_BOARD_NAME1	Specifies the on-board component, referenced in the EBD file, which corresponds to the X_ELEMENT_NAME1.

The PKG & EBD effect are shown from the first simulation.

System-Level PKG and EBD Simulation

To simulate an entire system (including PKG & EBD information), associate the Star-Hspice netlist with the PKG & EBD information. To do this, use the related options, the PKG map files, and the EBD map files. To obtain detailed information, contact your local Avant! Technical Support teams.

Stand-alone PKG Simulation

Use the Stand-alone PKG Simulation feature, to focus only on studying the packaging effect. To do this, follow these steps:

1. Use the hspice command, to produce a subcircuit that corresponds to the package component.
2. Prepare a Star-Hspice netlist that calls the generated subcircuit. Star-Hspice adds the subcircuit, with the suitable stimulus and loads.
3. Simulate the Star-Hspice netlist.

The command syntax for pkg2ckt is:

```
hspice -t RLC/T_LINE -p ibis_file -c component_name
```

Parameter	Description
hspice	Program name.
-t	Specifies elements that handle the package effect, either: ■ RLC (the default) for RLC elements, or ■ T_LINE (transmission line) for the W element. This argument is not available for the package data in matrix form.
-p	Specifies that the next argument is <i>ibis file</i> .
ibis_file	Specifies the name of the IBIS file that includes the package data. The file extension must be either .ibs or .pkg.
-c	Specifies that the next argument is <i>component name</i> .
component_name	Specifies the name of the component, for which simulation focuses only on the package (PKG).

The generated sub-circuit file name is the *component_name*, with a *.inc* extension.

The number of interface nodes is twice the number of pins listed in the *ibis* file, for internal nodes and external pins.

- Half of these nodes use the pin names from the *ibis* file. These node names, without new prefixes, are pins that connect outside of the circuit.
- The remaining interface nodes use the same name as the pin listed in the *ibis* file, but with the *P0_* prefix. These internal nodes connect to the original sub-circuit interface node, based on their names.

Stand-alone EBD Simulation

To study *only* the pin-connected trace effect, use the Stand-alone, Pin-connected Trace Simulation feature, as described in the following steps:

1. Use the *hspice* command, to create a subcircuit that corresponds to the pin-connected trace component.
2. Prepare a Star-Hspice netlist, that calls the generated subcircuit. Star-Hspice adds the subcircuit, with suitable stimulus and loads.
3. Simulate the Star-Hspice netlist.

The command syntax for *pkg2ckt* is:

```
hspice -t RLC/T_LINE -e ibis_file -o output_subckt_name
```

Parameter	Description
hspice	Program name.
-t	Specifies the elements that handle the package effect, either: ■ RLC (the default) for RLC elements, or ■ T_LINE (transmission line) for the W element. This argument is not available for the package data in matrix form.
-e	Specifies that the next argument is <i>ibis file</i> .
ibis_file	Specifies the name of the IBIS file, that includes the pin-connected trace data. The file extension must be .ebd.
-o	Specifies that the next argument is <i>subckt_name</i> .
subckt_name	Specifies the name of the sub-circuit, for which simulation focuses only on the pin-connected trace.

The generated file name is the *subckt_name*, with a .inc extension.

The number of interface nodes consists of the *pins* listed in the .ebd file, and the *nodes* listed in the .ebd file.

- The interface node from the pin, uses the same name as the pin.
- The interface node, from the node in the .ebd file, uses this name format:
{ *ref_name* + "_" + *pin_name* [+ *digit*] }

Limitation

In any specified package, the number of pins must not exceed 512.

If you use an EBD file, the EBD simulation feature does not support series components (such as resistors), because the current IBIS specification does not let you specify a resistance value. Currently, IBIS describes *only* the board pin-connect traces; IBIS ignores the other on-board traces.

You can use the PKG & EBD features for:

- Simulation, for both PKG and EBD.
- System simulation of a PKG.
- System simulation, using EBD.
- Simulation of a PKG circuit, to a PKG subcircuit.
- Simulation of an EBD circuit, to an EBD subcircuit.

You can select the parasitic element type, either RLC, or *w* transmission lines.

- The command:

```
hspice test_ebd1.sp
```

inserts the parasitic elements, due to the package and board-pin connected traces, into the original netlist. It then performs the simulation as usual.

- The command:

```
hspice -t RLC -p test_9_1_1.ibs -c test_9_1_1
```

outputs a sub-circuit into the `test_9_1_1.inc` file, which describes the package parasitic effects for the R/L/C elements.

- The command:

```
hspice -t T_LINE -e test_ebd1.ebd -o test_ebd
```

outputs the sub-circuit information into the `test_ebd.inc` file, which describes the board parasitic effect for the *w* transmission line.

Star-Hspice Output Files

Star-Hspice produces various types of output files, as listed in [Table 3-3](#).

Table 3-3: Star-Hspice Output Files and Suffixes

Output File Type	Extension
Output listing	.lis, or user-specified
Transient analysis results	.tr# †
Transient analysis measurement results	.mt#
DC analysis results	.sw# †
DC analysis measurement results	.ms#
AC analysis results	.ac# †
AC analysis measurement results	.ma#
Hardcopy graph data (from <i>meta.cfg</i> PRTDEFAULT)	.gr# ††
Digital output	.a2d
FFT analysis graph data	.ft#†††
Subcircuit cross-listing	.pa#
Output status	.st#
Operating point node voltages (initial conditions)	.ic#

is either a sweep number, or a hardcopy file number.

† Created only if you use .POST to generate graphical data.

†† Requires a .GRAPH statement, or a pointer to a file, in the *meta.cfg* file. The PC version of Star-Hspice does not generate this file.

††† Created only if you use a .FFT statement.

The files are listed in [Table 3-3](#) and described below.

Output listing can appear as *output_file* (no file extension), *output_file.lis*, or with a file extension that you specify, depending on which format you use to start the simulation. *Output_file* is the output file specification, not including any extension. This file includes the following information:

- Name of the simulator used.
- Version of the Star-Hspice simulator used.
- Avant! message block.
- Input file name.
- User name.
- License details.
- Copy of the input netlist file.
- Node count.
- Operating point parameters.
- Details of the volt drop, current, and power for each source and subcircuit.
- Low-resolution plots, originating from the .PLOT statement.
- Results of .PRINT statement.
- Results of .OPTION statements.

Star-Hspice places **transient analysis results** in *output_file.tr#*, where # is 0-9 or a-z, and follows the -n argument. This file lists the numerical results of transient analysis. A .TRAN statement in the input file, together with an .OPTION POST statement, creates this post-analysis file. The output file is in proprietary binary format if POST = 0 or 1, or in ASCII format if POST = 2. You can also use the explicit expressions POST = BINARY, or POST=ASCII.

Star-Hspice writes **transient analysis measurement results** to *output_file.mt#*. The .MEASURE TRAN statement creates this output file.

DC analysis results appear in *output_file.sw#*, which a .DC statement produces. This file contains the results of the applied stepped or swept DC parameters, defined in that statement. The results can include noise, distortion, or network analysis.

If the input file includes a .MEASURE DC statement, the *output_file.ms#* file specifies the **DC analysis measurement results**.

Star-Hspice places **AC analysis results** in *output_file.ac#*. These results list the output variables as a function of frequency, according to your specifications following the .AC statement.

If the input file contains a .MEASURE AC statement, then *output_file.ma#* contains **AC analysis measurement results**.

Star-Hspice places **hardcopy graph data** in *output_file.gr#*, which a .GRAPH statement produces. It is in the form of a printer file, typically in Adobe PostScript or HP PCL format. This facility is not available in the PC version of Star-Hspice.

Digital output contains data that the A2D conversion option of the U element converted to digital form.

FFT analysis graph data contains the graphical data needed to display the FFT analysis waveforms.

If the input netlist includes subcircuits, Star-Hspice automatically generates the **subcircuit cross-listing**, and writes it into *output_file.pa#*. This file relates the subcircuit node names, in the subcircuit call, to the node names used in the corresponding subcircuit definitions.

Use the output file specification, with a *.st#* extension, to name the output status. The output status contains the following runtime reports:

- Start and end times for each CPU phase.
- Options settings, with warnings for obsolete options.
- Status of pre-processing checks for licensing, input syntax, models, and circuit topology.
- Convergence strategies that Star-Hspice uses on difficult circuits.

You can use the information in this file to diagnose problems, particularly when communicating with Avant! Customer Support.

Operating point node voltages are DC operating point initial conditions, which the *.SAVE* statement stores.



Chapter 4

Elements

Elements are local, and sometimes customized, instances of a device model, specified in your design netlist.

For descriptions of the standard device models on which elements (instances) are based, see the *True-Hspice Device Models Reference Guide*. That manual describes the models that you can use as elements not only in Star-Hspice, but also in Star-Hspice XT/RF, and Star-Sim XT.

This chapter describes the syntax for the basic elements of a circuit netlist in Star-Hspice. Refer to the *True-Hspice Device Models Reference Manual* for detailed syntax descriptions and model descriptions.

This chapter explains the following topics:

- [Passive Elements](#)
- [Active Elements](#)
- [Transmission Lines](#)
- [Buffers](#)

Passive Elements

Resistors

The general syntax for a resistor element in a Star-Hspice netlist is:

Syntax

```
Rxxx n1 n2 <mname> <R = >resistance <<TC1 = >val>
+ <<TC2 = >val> <SCALE = val> <M = val> <AC = val>
+ <DTEMP = val> <L = val> <W = val> <C = val>
```

Resistance can be a value (in units of ohms) or an equation. Required fields are the two nodes, and the resistance or the model name. If you use the parameter labels, the node and model name must precede the labels. Other arguments can follow in any order. If you specify a resistor model (see Chapter 2 in the *True-Hspice Device Models Reference Manual*), the resistance value is optional.

The arguments are:

<i>Rxxx</i>	Resistor element name. Must begin with <i>R</i> , followed by up to 1023 alphanumeric characters.
<i>n1</i>	Positive terminal node name.
<i>n2</i>	Negative terminal node name.
<i>mname</i>	Resistor model name. Use this name in elements, to reference a resistor model.
<i>R = resistance</i>	Resistance value at room temperature. This can be: ■ a numeric value in ohms ■ a parameter in ohms ■ a function of any node voltages ■ a function of branch currents ■ any independent variables, such as: □ time □ frequency (HERTZ) □ temperature

<i>TC1</i>	First-order temperature coefficient for the resistor. Refer to Chapter 2, “Using Passive Device Models”, in the <i>True-Hspice Device Models Reference Manual</i> , for temperature-dependent relations.
<i>TC2</i>	Second-order temperature coefficient for the resistor.
<i>SCALE</i>	Element scale parameter; scales resistance by its value. Default = 1.0.
<i>M</i>	Multiplier, used to simulate parallel resistors. For example, to represent two parallel instances of a resistor, set $M = 2$, to multiply the number of resistors by 2. Default = 1.0.
<i>AC</i>	AC resistance, used in AC analysis. Default = R_{eff} .
<i>DTEMP</i>	Temperature difference between the element and the circuit, in degrees Celsius. Default = 0.0.
<i>L</i>	Resistor length in meters. Default = 0.0, if you did not specify L in a resistor model.
<i>W</i>	Resistor width. Default = 0.0, if you did not specify W in the model.
<i>C</i>	Capacitance connected from node $n2$ to bulk. Default = 0.0, if you did not specify C in a resistor model.

Star-Hspice Examples

In the following example, the $R1$ resistor connects from the $Rnode1$ node to the $Rnode2$ node, with a resistance of 100 ohms.

```
R1 Rnode1 Rnode2 100
```

The $RC1$ resistor connects from node 12 to node 17, with a resistance of 1 kilohm, and temperature coefficients of 0.001 and 0.

```
RC1 12 17 R = 1k TC1 = 0.001 TC2 = 0
```

The $Rterm$ resistor connects from the input node to ground, with a resistance determined by the square root of the analysis frequency (non-zero for AC analysis only).

```
Rterm input gnd R = 'sqrt(HERTZ)'
```

The Rxxx resistor, from node 989999999 to node 87654321, with a resistance of 1 ohm for DC and time-domain analyses, and 10 gigohms for AC analyses.

```
Rxxx 989999999 87654321 1 AC = 1e10
```

Linear Resistors

Syntax

The input syntax of a resistor is:

```
Rxxx node1 node2 < modelname > < R = > value
< TC1 = val >
+ < TC2 = val > < W = val > < L = val > < M = val >
+ < C = val > < DTEMP = val > < SCALE = val >
```

where:

Rxxx	Is the name of a resistor.
node1 and node2	Are the names or numbers of the connecting nodes.
modelname	Is the name of the resistor model.
value	Is the nominal resistance value, in ohms.
R	Specifies the resistance, in ohms, at room temperature.
TC1, TC2	Specifies the temperature coefficient.
W	Specifies the resistor width.
L	Specifies the resistor length.
M	Specifies the parallel multiplier.
C	Specifies the parasitic capacitance between node2 and the substrate.
DTEMP	Specifies the temperature difference between the element and the circuit.
SCALE	Specifies the scaling factor.

Example

The first resistor, R1, is a simple 10-ohm linear resistor. The second resistor, Rload, calls a resistor model named RVAL, defined later in the netlist.

Note: If a resistor calls a model, then you do not need to specify a constant resistance, as you do with R1.

- R3 takes its value from the *RX* parameter, and uses the *TC1* and *TC2* temperature coefficients, which become 0.001 and 0, respectively.
- RP spans across different circuit hierarchies, and is 0.5 ohms.

```
R1 1 2 10.0
Rload 1 GND RVAL
.param rx=100
R3 2 3 RX TC1 = 0.001 TC2 = 0
RP X1.A X2.X5.B .5
.MODEL RVAL R
```

Behavioral Resistors

Star-Hspice supports resistors with the following equation type:

```
Rxxx n1 n2 . . . <R=> 'equation' . . .
```

Note: The equation can be a function of any node voltage, and any branch current, but not a function of time, frequency, or temperature.

Example

```
R1 A B R = 'V(A) + I(VDD)'
```

Capacitors

The general syntax for a capacitor element is:

General Form

```
Cxxx n1 n2 <mname> <C = >capacitance <<TC1 = >val>
+ <<TC2 = >val> <SCALE = val> <IC = val> <M = val>
+ <W = val> <L = val> <DTEMP = val>
```

or

```
Cxxx n1 n2 <C = >'equation' <CTYPE = val>
+ <above_options...>
```

Polynomial Form

```
Cxxx n1 n2 POLY c0 c1... <above_options...>
```

where you can specify the capacitance as a numeric value, in units of farads, as an equation, or as a polynomial of the voltage. The only required fields are the two nodes, and the capacitance or model name.

- If you use the parameter labels, the nodes and model name must precede the labels. Other arguments can follow in any order.
- If you specify a capacitor model (see Chapter 2, in the *True-Hspice Device Models Reference Manual*), the capacitance value is optional.

If you use an equation to specify capacitance, the CTYPE parameter determines how Star-Hspice calculates the capacitance charge. The calculation is different, depending on whether the equation uses a self-referential voltage (that is, the voltage across the capacitor, whose capacitance is determined by the equation).

To avoid syntax conflicts, if a capacitor model has the same name as a parameter that specifies the capacitance, Star-Hspice uses the model name. In the following example, C1 assumes its capacitance value from the model, not the parameter.

```
.PARAMETER CAPXX = 1
C1 1 2 CAPXX
.MODEL CAPXX C CAP = 1
```

The arguments are:

<i>Cxxx</i>	Capacitor element name. Must begin with C, followed by up to 1023 alphanumeric characters.
<i>n1</i>	Positive terminal node name.
<i>n2</i>	Negative terminal node name.
<i>mname</i>	Capacitance model name. Elements use this name to reference a capacitor model.
<i>C = capacitance</i>	Capacitance at room temperature, as a numeric value, or a parameter, in farads.
<i>TC1</i>	First-order temperature coefficient for the capacitor. Refer to Chapter 2, “Using Passive Device Models”, in the <i>True-Hspice Device Models Reference Manual</i> , for temperature-dependant relations.
<i>TC2</i>	Second-order temperature coefficient, for the capacitor.
<i>SCALE</i>	Element scale parameter, scales capacitance by its value. Default = 1.0.
<i>IC</i>	Initial voltage across the capacitor, in volts. If you specify UIC in the .TRAN statement, Star-Hspice uses this value as the DC operating point voltage. The .IC statement overrides it.
<i>M</i>	Multiplier, used to simulate multiple parallel capacitors. Default = 1.0
<i>W</i>	Capacitor width in meters. Default = 0.0, if you did not specify w in a capacitor model.
<i>L</i>	Capacitor length in meters. Default = 0.0, if you did not specify L in a capacitor model.
<i>DTEMP</i>	Element temperature difference from the circuit temperature, in degrees Celsius. Default = 0.0.

$C = \text{'equation'}$	Capacitance at room temperature, specified as a function of: ■ any node voltages ■ any branch currents ■ any independent variables, such as: □ time □ frequency (HERTZ) □ temperature
<i>CTYPE</i>	Determines capacitance charge calculation, for elements with capacitance equations. If the capacitance equation is a function of $v(n1, n2)$, set <i>CTYPE</i> = 1. Use this setting correctly, to ensure proper capacitance calculations, and correct simulation results. Default = 0.
<i>POLY</i>	Keyword, to specify capacitance as a polynomial.
<i>c0 c1...</i>	Coefficients of a polynomial in voltage, describing the capacitor value. <i>c0</i> represents the magnitude of the 0th order term, <i>c1</i> represents the magnitude of the 1st order term, and so on. You cannot use parameters as coefficient values.

Example

In the following example, the C1 capacitors connect from node 1 to node 2, with a capacitance of 20 picofarads:

```
C1 1 2 20p
```

Cshunt refers to three capacitors in parallel, connected from the node output to ground, each with a capacitance of 100 femtofarads.

```
Cshunt output gnd C = 100f M = 3
```

The Cload capacitor connects from the driver node to the output node. The capacitance is determined by the voltage on the capcontrol node, times 1E-6. The initial voltage across the capacitor is 0 volts.

```
Cload driver output C = '1u*v(capcontrol)' CTYPE = 1
+ IC = 0v
```

The C99 capacitor connects from the in node to the out node. The capacitance is determined by the polynomial $C = c_0 + c_1*v + c_2*v*v$, where v is the voltage across the capacitor.

```
C99 in out POLY 2.0 0.5 0.01
```

Linear Capacitors

Syntax

The input syntax of a capacitor is:

```
Cxxx node1 node2 < modelname > < C = > value
< TC1 = val >
+ < TC2 = val > < W = val > < L = val > < DTEMP = val >
+ < M = val > < SCALE = val > < IC = val >
```

where:

Cxxx	Is the name of a capacitor.
node1 and node2	Are the names or numbers of the connecting nodes.
value	Is the nominal capacitance value, in Farads.
modelname	Is the name of the capacitor model.
C	Specifies the capacitance, in Farads, at room temperature.
TC1, TC2	Specifies the temperature coefficient.
W	Specifies the capacitor width.
L	Specifies the capacitor length.
M	Specifies the multiplier of parallel capacitors.
DTEMP	Specifies the temperature difference between the element and the circuit.
SCALE	Specifies the scaling factor.
IC	Specifies the initial capacitor voltage.

Example

```
Cbypass 1 0 10PF
C1 2 3 CBX
.MODEL CBX C
CB B 0 10P IC = 4V
CP X1.XA.1 0 0.1P
```

In this example:

- Cbypass is a straightforward, 10-picofarad (PF) capacitor.
- C1, which calls the CBX model, does not have a constant capacitance.
- CB is a 10 PF capacitor, with an initial voltage of 4V across it.
- CP is a 0.1 PF capacitor.

Behavioral Capacitors

Star-Hspice supports capacitors with the following equation type:

```
Cxxx n1 n2 . . . C='equation' CTYPE=0, 1 or 2
```

Note: You can describe the capacitor value as a function of any node voltage, and any branch current, but not as a function of time, frequency, or temperature.

CTYPE Parameter

CTYPE determines the calculation mode for elements that use capacitance equations. Set this parameter carefully, to ensure correct simulation results.

- C=0, if C depends only on its own terminal voltages—that is, a function of $V(n1, n2)$.
- C=1, if C depends only on outside voltages or currents.

Example

```
V1 1 0 pwl(0n 0v 100n 10v)
V2 2 0 pwl(0n 0v 100n 10v)
C1 1 0 C = '(V(1) + V(2))*1e-12'
```

Charge-Conserving Capacitors

Cxxx n1 n2 . . . Q = 'equation'

$C = \frac{dQ}{dV}$, $V = V(n1, n2)$

Cxxx a b Q = 'f(V(a, b))'

is equivalent to:

Cxxx a b Q = 'f(V(a, b))'

where: $d(x) = \frac{df(x)}{dx}$

Example

C1 a b Q = 'sin(V(a, b)) + V(c, d)*V(a, b)'

is equivalent to:

C1 a b C = 'cos (V(a, b)) + V(c, d)'

Note: Charge-conserving capacitors deliver a more-accurate solution.

Inductors

The general syntax for an inductor element is:

General Form

Lxxx n1 n2 <L = >inductance <<TC1 = >val> <<TC2 = >val>
+ <SCALE = val> <IC = val> <M = val> <DTEMP = val>
+ <R = val>

or

Lxxx n1 n2 L = 'equation' <LTYPE = val> <above_options...>

Polynomial Form

Lxxx n1 n2 POLY c0 c1... <above_options...>

Magnetic Winding Form

`Lxxx n1 n2 NT = turns <above_options...>`

where the inductance can be either a value (in units of henries), an equation, a polynomial of the current, or a magnetic winding. Required fields are the two nodes, and the inductance or model name.

- If you use parameter labels, the nodes and model name must be first. Other arguments can be in any order.
- If you specify an inductor model (see Chapter 2 in the *True-Hspice Device Models Reference Manual*), the inductance value is optional.

The arguments are:

<i>Lxxx</i>	Inductor element name. Must begin with L, followed by up to 1023 alphanumeric characters.
<i>n1</i>	Positive terminal node name.
<i>n2</i>	Negative terminal node name.
<i>TC1</i>	First-order temperature coefficient for the inductor. Refer to Chapter 2, “Using Passive Device Models”, in the <i>True-Hspice Device Models Reference Manual</i> , for temperature-dependent relations.
<i>TC2</i>	Second-order temperature coefficient for the inductor.
<i>SCALE</i>	Element scale parameter; scales inductance by its value. Default = 1.0.
<i>IC</i>	Initial current through the inductor, in amperes. Star-Hspice uses this value as the DC operating point voltage, when you specify UIC in the .TRAN statement. The .IC statement overrides it.

<i>L = inductance</i>	Inductance value. This can be: ■ a numeric value, in henries ■ a parameter in henries ■ a function of any node voltages ■ a function of branch currents ■ any independent variables, such as: □ time □ frequency (HERTZ) □ temperature
<i>M</i>	Multiplier, used to simulate parallel inductors. Default = 1.0.
<i>DTEMP</i>	Temperature difference between the element and the circuit, in degrees Celsius. Default = 0.0.
<i>R</i>	Resistance of inductor, in ohms. Default = 0.0.
<i>L = 'equation'</i>	Inductance at room temperature, specified as: ■ a function of any node voltages ■ a function of branch currents ■ any independent variables, such as: □ time □ frequency (HERTZ) □ temperature
<i>LTYPE</i>	Determines inductance flux calculation for elements, using inductance equations. If the inductance equation is a function of <i>i(Lxxx)</i> , then set <i>LTYPE</i> = 1. Use this setting correctly, to ensure proper inductance calculations, and correct simulation results. Default = 0.
<i>POLY</i>	Keyword that specifies the inductance, calculated by a polynomial.
<i>c0 c1...</i>	Coefficients of a polynomial in the current, describing the inductor value. <i>c0</i> is the magnitude of the 0th order term, <i>c1</i> is the magnitude of the 1st order term, and so on.
<i>NT = turns</i>	Number of turns of an inductive magnetic winding.

Example

In the following example, the L1 inductor connects from the coilin node to the coilout node, with an inductance of 100 nanohenries.

```
L1 coilin coilout 100n
```

The Lloop inductor connects from node 12 to node 17. Its inductance is 1 microhenry, and its temperature coefficients are 0.001 and 0.

```
Lloop 12 17 L = 1u TC1 = 0.001 TC2 = 0
```

The Lcoil inductor connects from the input node to ground. Its inductance is determined by the product of the current through the inductor, and 1E-6.

```
Lcoil input gnd L = '1u*i(input)' LTYPE = 0
```

The L99 inductor connects from the in node to the out node. Its inductance is determined by the polynomial $L = c0 + c1*i + c2*i*i$, where i is the current through the inductor. The inductor also has a specified DC resistance of 10 ohms.

```
L99 in out POLY 4.0 0.35 0.01 R = 10
```

The L inductor connects from node 1 to node, as a magnetic winding element, with 10 turns of wire.

```
L 1 2 NT = 10
```

Mutual Inductors

The general syntax for a mutual inductor element is:

General Form

```
Kxxx Lyyy Lzzz <K = >coupling
```

Mutual Core Form

```
Kaaa Lbbb <Lccc ... <Lddd>> mname <MAG = magnetization>
```

where *coupling* is a unitless value, from zero to one, representing the coupling strength. If you use parameter labels, the nodes and model name must be first. Other arguments can be in any order. If you specify an inductor model (see Chapter 2, “Using Passive Device Models”, in the *True-Hspice Device Models Reference Manual*), the inductance value is optional.

The arguments are:

<i>Kxxx</i>	Mutual inductor element name. Must begin with κ , followed by up to 1023 alphanumeric characters.
<i>Lyxx</i>	Name of the first of two coupled inductors.
<i>Lzzx</i>	Name of the second of two coupled inductors.
<i>K = coupling</i>	Coefficient of mutual coupling. κ is a unitless number, with magnitude > 0 and ≤ 1 . If κ is negative, the direction of coupling reverses. This is equivalent to reversing the polarity of either of the coupled inductors. Use the <i>K = coupling</i> syntax when using a parameter value or an equation.
<i>Kaaa</i>	Saturable core element name. Must begin with κ , followed by up to 1023 alphanumeric characters.
<i>Lbbb</i> , <i>Lccc</i> , <i>Lddd</i>	The names of the windings about the <i>Kaaa</i> core. One winding element is required, and each winding element must use the magnetic winding syntax.
<i>mname</i>	Saturable core model name. See Chapter 2, “Using Passive Device Models”, in the <i>True-Hspice Device Models Reference Manual</i> for model information.
<i>MAG = magnetization</i>	Initial magnetization of the saturable core. You can set this to +1, 0 and -1, where +/- 1 refer to positive and negative values of the BS model parameter (see Chapter 2, “Using Passive Device Models”, in the <i>True-Hspice Device Models Reference Manual</i>).

You can determine the coupling coefficient, based on geometric and spatial information. To determine the final coupling inductance, Star-Hspice divides the coupling coefficient by the square-root of the product of the self-inductances.

When using the mutual inductor element to calculate the coupling between more than two inductors, Star-Hspice can automatically calculate an approximate second-order coupling. See the third example below, for a specific situation.

Note: The automatic inductance calculation is an estimation, and is accurate for a subset of geometries. The second-order coupling coefficient is the product of the two first-order coefficients, which is not correct for many geometries.

Example

The L_{in} and L_{out} inductors are coupled, with a coefficient of 0.9.

```
K1 Lin Lout 0.9
```

The L_{high} and L_{low} inductors are coupled, with a coefficient equal to the value of the COUPLE parameter.

```
Kxfmr Lhigh Llow K = COUPLE
```

- The K1 mutual inductor couples L₁ and L₂.
- The K2 mutual inductor couples L₂ and L₃.

The coupling coefficients are 0.98 and 0.87. Star-Hspice automatically calculates the mutual inductance between L₁ and L₃, with a coefficient of $0.98 \times 0.87 = 0.853$.

```
K1 L1 L2 0.98
K2 L2 L3 0.87
```

Linear Inductors

Syntax

```
Lxxx node1 node2 <L => value <TC1 = val> <TC2 = val>
+ <M = val> <DTEMP = val> <IC = val>
```

where:

Lxxx	Name of an inductor.
node1 and node2	Names or numbers of the connecting nodes.
value	Nominal inductance value, in Henries.
L	Inductance, in Henries, at room temperature.

TC1, TC2	Temperature coefficient.
M	Multiplier of parallel inductors.
DTEMP	Temperature difference between the element and the circuit.
IC	Initial inductor current.

Example

```
LX A B 1E-9  
LR 1 0 1u IC = 10mA
```

In this example:

- LX is a 1 nH inductor.
- LR is a 1 uH inductor, with an initial current of 10 mA.

To properly simulate power line inductors, you must either set them to ANALOG mode, or invoke the SIM_RAIL option as follows:

```
.OPTION SIM_ANALOG = "L1"
```

or

```
.OPTION SIM_RAIL = ON
```

Active Elements

Diode Element

The general syntax for a diode element is:

Geometric (LEVEL=1) or Non-Geometric (LEVEL=3) Form

```
Dxxx nplus nminus mname <<AREA = >area> <<PJ = >val>
+ <WP = val> <LP = val> <WM = val> <LM = val> <OFF>
+ <IC = vd> <M = val> <DTEMP = val>
```

or

```
Dxxx nplus nminus mname <W = width> <L = length> <WP = val>
+ <LP = val> <WM = val> <LM = val> <OFF> <IC = vd> <M = val>
+ <DTEMP = val>
```

Fowler-Nordheim (LEVEL = 2) Form

```
Dxxx nplus nminus mname <W = val <L = val>> <WP = val>
+ <OFF> <IC = vd> <M = val>
```

The only required fields are the two nodes, and the model name. If you use the parameter labels, the nodes and model name must be first, and the other optional arguments can be in any order. The arguments are:

<i>Dxxx</i>	Diode element name. Must begin with D, followed by up to 1023 alphanumeric characters.
<i>nplus</i>	Positive terminal (anode) node name. The series resistor of the equivalent circuit is attached to this terminal.
<i>nminus</i>	Negative terminal (cathode) node name.
<i>mname</i>	Diode model name reference.

<i>AREA</i>	Area of the diode (unitless for LEVEL = 1 diode, and square meters for LEVEL = 3 diode). This affects saturation currents, capacitances, and resistances (diode model parameters are IK, IKR, JS, CJO, and RS). The SCALE option does not affect the area factor for the LEVEL = 1 diode. Default = 1.0. Overrides AREA from the diode model. If you do not specify the AREA, Star-Hspice calculates it from the width and length.
<i>PJ</i>	Periphery of junction (unitless for LEVEL = 1 diode, and meters for LEVEL = 3 diode). Overrides PJ from the diode model. If you do not specify PJ, Star-Hspice calculates it from the width and length specifications.
<i>WP</i>	Width of polysilicon capacitor, in meters (for LEVEL = 3 diode only). Overrides WP in the diode model. Default = 0.0.
<i>LP</i>	Length of polysilicon capacitor, in meters (for LEVEL = 3 diode only). Overrides LP in the diode model. Default = 0.0.
<i>WM</i>	Width of metal capacitor, in meters (for LEVEL = 3 diode only). Overrides WM in the diode model. Default = 0.0.
<i>LM</i>	Length of metal capacitor, in meters (for LEVEL = 3 diode only). Overrides LM in the diode model. Default = 0.0.
<i>OFF</i>	Sets initial condition for this element to OFF, in DC analysis. Default = ON.
<i>IC = vd</i>	Initial voltage across the diode element. Star-Hspice uses this value, when the .TRAN statement contains the UIC option. The .IC statement overrides it.
<i>M</i>	Multiplier, to simulate multiple diodes in parallel. The M setting affects all currents, capacitances, and resistances. Default = 1.
<i>DTEMP</i>	The difference between the element temperature and the circuit temperature, in degrees Celsius. Default = 0.0.
<i>W</i>	Width of the diode, in meters (LEVEL=3 diode model only)
<i>L</i>	Length of the diode, in meters (LEVEL = 3 diode model only)

Examples

The D1 diode, with anode and cathode, connects to nodes 1 and 2. Diode1 specifies the diode model.

```
D1 1 2 diode1
```

The Dprot diode, with anode and cathode, connects to the output node. Ground references the firstd diode model, and specifies an area of 10 (unitless for LEVEL = 1 model). The initial condition has the diode OFF.

```
Dprot output gnd firstd 10 OFF
```

The Ddrive diode, with anode and cathode, connects to the driver and output nodes. The width and length are 500 microns. This diode references the model_d diode model.

```
Ddrive driver output model_d W = 5e-4 L = 5e-4 IC = 0.2
```

Bipolar Junction Transistor (BJT) Element

The general syntax for a BJT element is:

Syntax

```
Qxxx nc nb ne <ns> mname <area> <OFF>
+ <IC = vbeval,vceval> <M = val> <DTEMP = val>
```

or

```
Qxxx nc nb ne <ns> mname <AREA = area> <AREAB = val>
+ <AREAC = val> <OFF> <VBE = vbeval> <VCE = vceval>
+ <M = val> <DTEMP = val>
```

The only required fields are the collector, base, and emitter nodes, and the model name. The nodes and model name must precede other fields in the netlist.

The arguments are:

<i>Qxxx</i>	BJT element name. Must begin with Q, followed by up to 1023 alphanumeric characters.
<i>nc</i>	Collector terminal node name.
<i>nb</i>	Base terminal node name.
<i>ne</i>	Emitter terminal node name.
<i>ns</i>	Substrate terminal node name, which is optional. You can also use the BULK parameter to set this name in the BJT model.
<i>mname</i>	BJT model name reference.
<i>area</i> , <i>AREA = area</i>	Emitter area multiplying factor, which affects currents, resistances, and capacitances. Default = 1.0.
<i>OFF</i>	Sets initial condition for this element to OFF, in DC analysis. Default = ON.
<i>IC = vbeval</i> , <i>vceval</i> , <i>VBE</i> , <i>VCE</i>	Initial internal base-emitter voltage (<i>vbeval</i>) and collector-emitter voltage (<i>vceval</i>). Star-Hspice uses this value when the .TRAN statement includes UIC. The .IC statement overrides it.
<i>M</i>	Multiplier, to simulate multiple BJTs in parallel. The M setting affects all currents, capacitances, and resistances. Default = 1.
<i>DTEMP</i>	The difference between the element temperature and the circuit temperature, in degrees Celsius. Default = 0.0.
<i>AREAB</i>	Base area multiplying factor, which affects currents, resistances, and capacitances. Default = AREA.
<i>AREAC</i>	Collector area multiplying factor, which affects currents, resistances, and capacitances. Default = AREA.

Example

In the Q1 BJT element below:

```
Q1 1 2 3 model_1
```

- The collector connects to node 1.
- The base connects to node 2.
- The emitter connects to node 3.
- model_1 references the BJT model.

In the Qopamp1 BJT element below:

```
Qopamp1 c1 b3 e2 s 1stagepnp AREA = 1.5 AREAB = 2.5  
AREAC = 3.0
```

- The collector connects to the c1 node.
- The base connects to the b3 node.
- The emitter connects to the e2 node.
- The substrate connects to the s node.
- 1stagepnp references the BJT model.
- The AREA area factor is 1.5.
- The AREAB area factor is 2.5.
- The AREAC area factor is 3.0.

In the Qdrive BJT element below:

```
Qdrive driver in output model_npn 0.1
```

- The collector connects to the driver node.
- The base connects to the in node.
- The emitter connects to the output node.
- model_npn references the BJT model.
- The area factor is 0.1.

JFETs and MESFETs

The general syntax for a JFET or MESFET element is:

Syntax

```
Jxxx nd ng ns <nb> mname <<<AREA> = area | <W = val>
+ <L = val>> <OFF> <IC = vdsval, vgsval> <M = val>
+ <DTEMP = val>
```

or

```
Jxxx nd ng ns <nb> mname <<<AREA> = area> | <W = val>
+ <L = val>> <OFF> <VDS = vdsval> <VGS = vgsval>
+ <M = val> <DTEMP = val>
```

The only required fields are the drain, gate, and source nodes, and the model name. The nodes and model name must precede other fields within the netlist.

The arguments are:

<i>Jxxx</i>	JFET or MESFET element name. Must begin with J, followed by up to 1023 alphanumeric characters.
<i>nd</i>	Drain terminal node name
<i>ng</i>	Gate terminal node name
<i>ns</i>	Source terminal node name
<i>nb</i>	Bulk terminal node name, which is optional.
<i>mname</i>	JFET or MESFET model name reference
<i>area</i> , <i>AREA = area</i>	Area multiplying factor that affects the BETA, RD, RS, IS, CGS, and CGD model parameters. Default = 1.0, in units of square meters.
<i>W</i>	FET gate width in meters
<i>L</i>	FET gate length in meters
<i>OFF</i>	Sets initial condition to OFF for this element, in DC analysis. Default = ON.
<i>IC = vdsval</i> , <i>vgsval</i> , <i>VDS</i> , <i>VGS</i>	Initial internal drain-source voltage (<i>vdsval</i>) and gate-source voltage (<i>vgsval</i>). Use this argument when the .TRAN statement contains UIC. The .IC statement overrides it.

<i>M</i>	Multiplier to simulate multiple JFETs or MESFETs in parallel. The <i>M</i> setting affects all currents, capacitances, and resistances. Default = 1.
<i>DTEMP</i>	The difference between the element temperature and the circuit temperature, in degrees Celsius. Default = 0.0.

Example

In the J1 JFET element below:

```
J1 1 2 3 model_1
```

- The drain connects to node 1.
- The source connects to node 2.
- The gate connects to node 3.
- model_1 references the JFET model.

In the Jopamp1 JFET element below:

```
Jopamp1 d1 g3 s2 b 1stage AREA = 100u
```

- The drain connects to the d1 node.
- The source connects to the g3 node.
- The gate connects to the s2 node.
- 1stage references the JFET model.
- The area is 100 microns.

In the Jdrive JFET element below:

```
Jdrive driver in output model_jfet W = 10u L = 10u
```

- The drain connects to the driver node.
- The source connects to the in node.
- The gate connects to the output node.
- model_jfet references the JFET model.
- The width is 10 microns.
- The length is 10 microns.

MOSFETs

The general syntax for a MOSFET element is:

Syntax

```
Mxxx nd ng ns <nb> mname <<L = >length> <<W = >width>
+ <AD = val> AS = val> <PD = val> <PS = val>
+ <NRD = val> <NRS = val> <RDC = val> <RSC = val> <OFF>
+ <IC = vds, vgs, vbs> <M = val> <DTEMP = val>
+ <GEO = val> <DELVT0 = val>
```

or

```
.OPTION WL
Mxxx nd ng ns <nb> mname <width> <length>
+ <other_options...>
```

The only required fields are the drain, gate and source nodes, and the model name. The nodes and model name must precede other fields in the netlist. If you did not specify a label, use the second syntax with the `.OPTION WL` statement, to exchange the width and length options.

The arguments are:

<i>Mxxx</i>	MOSFET element name. Must begin with M, followed by up to 1023 alphanumeric characters.
<i>nd</i>	Drain terminal node name.
<i>ng</i>	Gate terminal node name.
<i>ns</i>	Source terminal node name.
<i>nb</i>	Bulk terminal node name, which is optional. To set this argument in the MOSFET model, use the BULK parameter.
<i>mname</i>	MOSFET model name reference
<i>L</i>	MOSFET channel length, in meters. This parameter overrides DEFL in an <code>.OPTION</code> statement. Default = DEFL, with a maximum value of 0.1m.
<i>W</i>	MOSFET channel width, in meters. This parameter overrides DEFW in an <code>.OPTION</code> statement. Default = DEFW.

<i>AD</i>	Drain diffusion area. Overrides DEFAD in the .OPTION statement. Default = DEFAD, only when you set the MOSFET model parameter ACM = 0.
<i>AS</i>	Source diffusion area. Overrides DEFAS in the .OPTION statement. Default = DEFAS, only when you set the MOSFET model parameter ACM = 0.
<i>PD</i>	Perimeter of the drain junction, including the channel edge. Overrides DEFPD in the .OPTION statement. Default = DEFAD, when you set the MOSFET model parameter ACM = 0 or 1. Default = 0.0, when you set ACM = 2 or 3.
<i>PS</i>	Perimeter of the source junction, including the channel edge. Overrides DEFPS in the .OPTION statement. Default = DEFAS, when you set the MOSFET model parameter ACM = 0 or 1. Default = 0.0, when you set ACM = 2 or 3.
<i>NRD</i>	Number of squares of drain diffusion for resistance calculations. Overrides DEFNRD in the .OPTION statement. Default = , when you set the MOSFET model parameter ACM = 0 or 1. Default = 0.0, when you set ACM = 2 or 3.
<i>NRS</i>	Number of squares of source diffusion, for resistance calculations. Overrides DEFNRS in the .OPTION statement. Default = DEFNRS when you set the MOSFET model parameter ACM = 0 or 1. Default = 0.0, when you set ACM = 2 or 3.
<i>RDC</i>	Additional drain resistance due to contact resistance, in units of ohms. This value overrides the RDC setting in the MOSFET model specification. Default = 0.0.
<i>RSC</i>	Additional source resistance due to contact resistance, in units of ohms. This value overrides the RSC setting in the MOSFET model specification. Default = 0.0.
<i>OFF</i>	Sets initial condition for this element to OFF, in DC analysis. Default = ON. Note: this command does not work for depletion devices.

<i>IC = vds, vgs, vbs</i>	Initial voltage across the external drain and source (<i>vds</i>), gate and source (<i>vgs</i>), and bulk and source terminals (<i>vbs</i>). Use these arguments with UIC in the .TRAN statement. The .IC statement overrides these values.
<i>M</i>	Multiplier, to simulate multiple MOSFETs in parallel. The <i>M</i> setting affects all channel widths, diode leakages, capacitances, and resistances. Default = 1.
<i>DTEMP</i>	The difference between the element temperature and the circuit temperature, in degrees Celsius. Default = 0.0.
<i>GEO</i>	Source/drain sharing selector, for a MOSFET model parameter value of ACM = 3. Default = 0.0.
<i>DELVTO</i>	Zero-bias threshold voltage shift. Default = 0.0.

Example

In the M1 MOSFET element below:

```
M1 1 2 3 model_1
```

- The drain connects to node 1.
- The gate connects to node 2.
- The source connects to node 3.
- *model_1* references the MOSFET model.

In the Mopamp1 MOSFET element below:

```
Mopamp1 d1 g3 s2 b 1stage L = 2u W = 10u
```

- The drain connects to the d1 node.
- The gate connects to the g3 node.
- The source connects to the s2 node.
- 1stage references the MOSFET model.
- The length of the gate is 2 microns.
- The width of the gate is 10 microns.

In the Mdrive MOSFET element below:

```
Mdrive driver in output bsim3v3 W = 3u L = 0.25u  
+ DTEMP = 4.0
```

- The drain connects to the driver node.
- The gate connects to the in node.
- The source connects to the output node.
- bsim3v3 references the MOSFET model.
- The length of the gate is 3 microns.
- The width of the gate is 0.25 microns.
- The device temperature is 4 degrees Celsius higher than the circuit temperature.

Transmission Lines

A transmission line is a passive element that connects any two conductors, at any distance apart. One conductor sends the input signal through the transmission line, and the other conductor receives the output signal from the transmission line. The signal that is transmitted from one end of the pair to the other end, is voltage between the conductors.

Examples of transmission lines include:

- Power transmission lines.
- Telephone lines.
- Waveguides.
- Traces on printed circuit boards and multi-chip modules (MCMs).
- Bonding wires in semiconductor IC packages.
- On-chip interconnections.

Input Syntax for the W Element

The W element supports four different formats, to specify the transmission line properties:

- Model 1: RLGC-Model specification.
 - Internally specified in a `.model` statement.
 - Externally specified in a different file.
- Model 2: U-Model specification.
 - RLGC input for up to five coupled conductors.
 - Geometric input (planar, coax, twin-lead).
 - Measured-parameter input.
 - Skin effect.
- Model 3: Built-in field solver model.
- Model 4: Frequency-dependent tabular model.

The input syntax for the W element card is:

```
Wxxx i1 i2 ... iN iR o1 o2 ... oN oR N=val L=val
+ TABLEMODEL=name
```

where	specifies the
N	Number of signal conductors (excluding the reference conductor).
i1...iN	Node names for the near-end signal-conductor terminal.
iR	Node name for the near-end reference-conductor terminal.
o1...oN	Node names for the far-end signal-conductor terminal.
oR	Node name for the far-end reference-conductor terminal.
L	Length of the transmission line.
TABLEMODEL	Name of the frequency-dependent tabular model.

W Element Statement

The general syntax for a lossy (W Element) transmission line element is:

RLGC File Form

```
Wxxx in1 <in2 <...inx>> refin out1 <out2 <...outx>>
+ refout <RLGCfile = fname> N = val L = val
```

U-Model Form

```
Wxxx in1 <in2 <...inx>> refin out1 <out2 <...outx>>
+ refout <Umodel = mname> N = val L = val
```

Field Solver Form

```
Wxxx in1 <in2 <...inx>> refin out1 <out2 <...outx>>
+ refout <FSmodel = mname> N = val L = val
```

where the number of ports on a single transmission line are not limited. You must provide one input and output port, the ground references, a model or file reference, a number of conductors, and a length.

The arguments are:

<i>Wxxx</i>	Lossy (W Element) transmission line element name. Must start with <i>w</i> , followed by up to 1023 alphanumeric characters.
<i>inx</i>	Signal input node for <i>x</i> th transmission line (<i>in1</i> is required).
<i>refin</i>	Ground reference for input signal
<i>outx</i>	Signal output node for the <i>x</i> th transmission line (each input port must have a corresponding output port).
<i>refout</i>	Ground reference for output signal.
<i>RLGCfile = fname</i>	File name reference, for the file containing the RLGC information for the transmission lines (for syntax, see Chapter 6, “Using Transmission Lines”, in the <i>True-Hspice Device Models Reference Manual</i>).
<i>N</i>	Number of conductors (excluding the reference conductor).
<i>L</i>	Physical length of the transmission line, in units of meters.
<i>Umodel = mname</i>	U-model lossy transmission-line model reference name. A lossy transmission line model, used to represent the characteristics of the W-element transmission line.
<i>FSmodel = mname</i>	Internal field solver model name. References the PETL internal field solver, as the source of the transmission-line characteristics (for syntax, see Chapter 6, “Using Transmission Lines”, in the <i>True-Hspice Device Models Reference Manual</i>).

Example

The W1 lossy transmission line below connects the *in* node to the *out* node:

```
W1 in gnd out gnd RLGCfile = cable.rlgc N = 1 L = 5
```

- Both signal references are grounded.
- The RLGC file is named *cable.rlgc*.
- The transmission line is 5 meters long.

The `wcable` element below is a two-conductor lossy transmission line:

```
wcable in1 in2 gnd out1 out2 gnd Umodel = umod_1 N = 2
+ L = 10
```

- The `in1` and `in2` input nodes connect to the `out1` and `out2` output node.
- Both signal references are grounded.
- `umod_1` references the U-model.
- The transmission line is 10 meters long.

The `wnet1` element below is a five-conductor lossy transmission line:

```
wnet1 i1 i2 i3 i4 i5 gnd o1 gnd o3 gnd o5 gnd
+ FSmodel = board1 N = 5 L = 1m
```

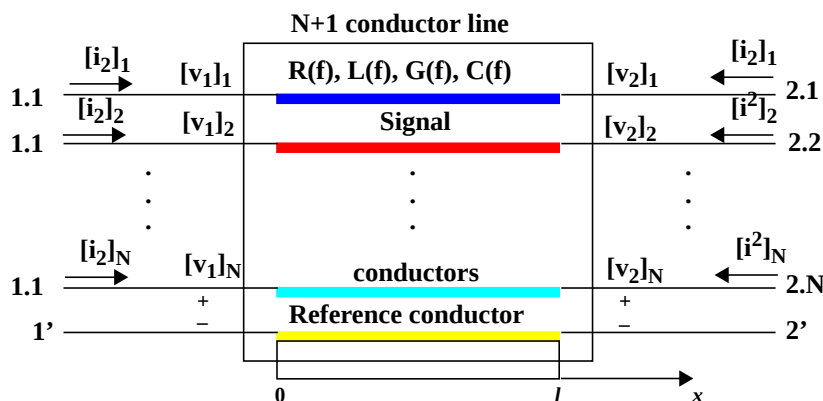
- The `i1`, `i2`, `i3`, `i4` and `i5` input nodes connect to the `o1`, `o3`, and `o5` output nodes.
- The `i5` input, and the three outputs (`o1`, `o3`, and `o5`) are all grounded.
- `board1` references the Field Solver model.
- The transmission line is 1 millimeter long.

The order of parameters in the W-element card does not matter. You can specify the number of signal conductors, N , after the list of nodes. Moreover, you can freely mix the nodes and parameters in the W-element card.

You can specify only one of the `RLGCFfile`, `FSmodel`, or `Umodel` models, in a single W-element card.

Figure 4-1 shows the node numbering convention for the element syntax.

Figure 4-1: Terminal Node Numbering for W Element



T Element Statement

The general syntax for a lossless (T Element) transmission line element is:

General Form

```
Txxx in refin out refout Z0 = val TD = val <L = val>
+ <IC = v1,i1,v2,i2>
```

or

```
Txxx in refin out refout Z0 = val F = val <NL = val>
+ <IC = v1,i1,v2,i2>
```

U-Model Form

```
Txxx in refin out refout mname L = val
```

where only one input and output port is allowed.

The arguments are defined:

<i>Txxx</i>	Lossless transmission line element name. Must begin with T, followed by up to 1023 alphanumeric characters.
<i>in</i>	Signal input node.
<i>refin</i>	Ground reference for the input signal.
<i>out</i>	Signal output node.
<i>refout</i>	Ground reference for the output signal.
<i>Z0</i>	Characteristic impedance of the transmission line.
<i>TD</i>	Signal delay from the transmission line, in units of seconds per meter.
<i>L</i>	Physical length of the transmission line, in units of meters. Default = 1.
<i>IC = v1,i1,v2,i2</i>	Initial conditions of the transmission line. Specify the voltage on the input port (v1), current into the input port (i1), voltage on the output port (v2), and the current into the output port (i2).
<i>F</i>	Frequency at which the transmission line has the electrical length specified in NL.

<i>NL</i>	Normalized electrical length of the transmission line (at the frequency specified in the F parameter), in units of wavelengths per line length. Default = 0.25, which is a quarter-wavelength.
<i>mname</i>	U-model reference name. A lossy transmission line model, representing the characteristics of the lossless transmission line.

Example

The T1 transmission line below connects the in node to the out node:

```
T1 in gnd out gnd Z0 = 50 TD = 5n L = 5
```

- Both signal references are grounded.
- Impedance is 50 ohms.
- The transmission delay is 5 nanoseconds per meter.
- The transmission line is 5 meters long.

The Tcable transmission line below connects the in1 node to the out1 node:

```
Tcable in1 gnd out1 gnd Z0 = 100 F = 100k NL = 1
```

- Both signal references are grounded.
- Impedance is 100 ohms.
- The normalized electrical length is 1 wavelength at 100 kHz.

The Tnet1 transmission line below connects the driver node to the output node:

```
Tnet1 driver gnd output gnd Umodel1 L = 1m
```

- Both signal references are grounded.
- Umodel1 references the U-model.
- The transmission line is 1 millimeter long.

U Element Statement

The general syntax for a lossy (U Element) transmission line element is:

Syntax

```
Uxxx in1 <in2 <...in5>> refin out1 <out2 <...out5>>
+ refout mname L = val <LUMPS = val>
```

where the number of ports on a single transmission line is limited to five in and five out. One input and output port, the ground references, a model reference, and a length are all required.

The arguments are:

Uxxx	Lossy (U Element) transmission line element name. Must begin with U, followed by up to 1023 alphanumeric characters.
inx	Signal input node for the x th transmission line (in1 is required).
refin	Ground reference for the input signal.
outx	Signal output node for the x th transmission line (each input port must have a corresponding output port).
refout	Ground reference for the output signal.
mname	Model reference name for the U-model lossy transmission-line.
L	Physical length of the transmission line, in units of meters.
LUMPS	Number of lumped-parameter sections used to simulate the element.

Example

The U1 transmission line below connects the in node to the out node:

```
U1 in gnd out gnd umodel_RG58 L = 5
```

- Both signal references are grounded.
- umodel_RG58 references the U-model.
- The transmission line is 5 meters long.

The Ucable transmission line below connects the in1 and in2 input nodes to the out1 and out2 output nodes:

```
Ucable in1 in2 gnd out1 out2 gnd twistpr L = 10
```

- Both signal references are grounded.
- twistpr references the U-model.
- The transmission line is 10 meters long.

The Unet1 element below is a five-conductor lossy transmission line:

```
Unet1 i1 i2 i3 i4 i5 gnd o1 gnd o3 gnd o5 gnd Umodel1  
+ L = 1m
```

- The i1, i2, i3, i4, and i5 input nodes connect to the o1, o3, and o5 output nodes.
- The i5 input, and the three outputs (o1, o3, and o5) are all grounded.
- Umodel1 references the U-model.
- The transmission line is 1 millimeter long.

Frequency-Dependent Multi-Terminal (S) Element

When used with the generic frequency-domain model (.MODEL SP), an S (scattering) model is a convenient way to describe a multi-terminal network.

This element uses the following parameters to define a frequency-dependent, multi-terminal network:

- **S** (scattering)
- **Y** (admittance)
- **Z** (impedance)

You can use an S Element in the following types of analyses:

- DC
- AC
- Transient
- Small Signal

For a description of the S Parameter and .sp analysis, see Chapter 6 of the *True-Hspice Device Models Reference Manual*.

Syntax

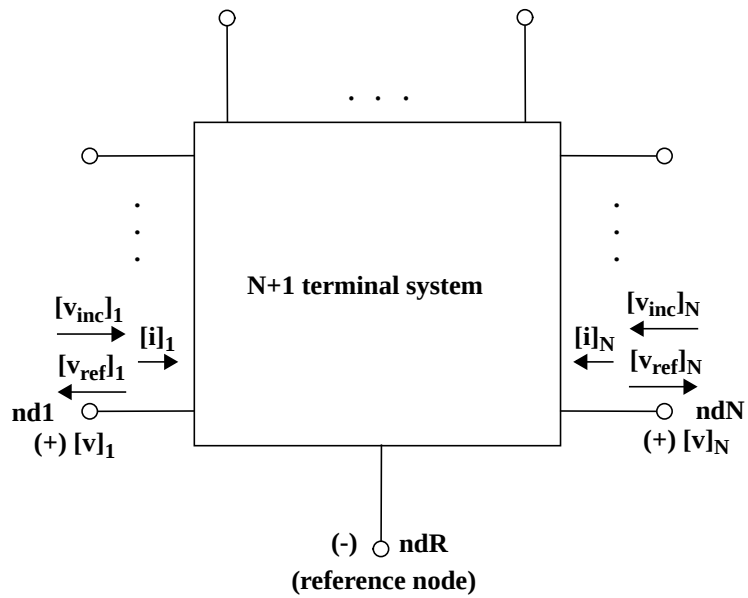
The syntax of the S Element is:

```
Sxxx nd1 nd2 ... ndN ndR FQMODEL=name [TYPE=val
      Zo=val Zof=name FBASE=value FMAX=value
      <IDC=vectorvalue|VDC=vectorvalue>]
```

Parameters

Parameter	Specifies
nd1 nd2 ... ndN	<i>N</i> signal nodes (see Figure 1).
ndR	Reference node.
FQMODEL	.MODEL statement of sp type, which defines the frequency behavior of the S , Y , or Z parameters.
Zo	Impedance value of the reference line (frequency-independent). For multiple terminals ($N > 1$), Star-Hspice assumes that the characteristic impedance matrix of the reference lines are diagonal, and that you set diagonal values to Zo. To specify general types of reference lines, use Zof. The default value is 50Ω .
Zof	Name of the frequency-varying model, which defines the frequency behavior of the reference system. If you defined both Zo and Zof, then Zof has precedence.
TYPE	Parameter type: ■ S (scattering), the default ■ Y (admittance) ■ Z (impedance)

Parameter	Specifies
FBASE	Base frequency to use for transient analysis. This value becomes the base frequency point for Inverse Fourier Transformation. ■ If you do not set this value, the base frequency is a reciprocal value of the transient period. ■ If you set a frequency that is smaller than the reciprocal value of the transient, then the transient analysis performs circular convolution, and uses the reciprocal value of FBASE as its base period.
FMAX	Maximum frequency to use for transient analysis. This value becomes the maximum frequency point for Inverse Fourier Transformation.
IDC	Terminal bias current at DC. Note: If you set this value, then the element acts as a current source at DC, instead of using the network parameter matrix.
VDC	Terminal bias voltage at DC. Note: If you set this value, then the element acts as a voltage source at DC, instead of using the network parameter matrix.

Figure 4-2: Terminal Node Notation

Buffers

The general syntax of an element card for input/output buffers is:

Syntax

```
bname node_1 node_2 ... node_N keyword_1 = value_1 ...
+ [keyword_M = value_M]
```

where:

bname	Is the buffer name, and starts with the letter B.
node_1 node_2 ... node_N	Is a list of input/output buffer external nodes. The number of nodes and their meaning are specific to different buffer types.
keyword_i = value_i	Assigns a value of <i>value_i</i> to the <i>keyword_i</i> keyword. Specify optional keywords in [brackets].

For information about the keywords, see Chapter 7, “Using IBIS Models”, in the *True-Hspice Device Models Reference Manual*.

Example

```
B1 nd_pc nd_gc nd_in nd_out_of_in
+ buffer = 1
+ file = 'test.ibs'
+ model = 'IBIS_IN'
```

- This example represents an input buffer named B1.
- The four terminals are named *nd_pc*, *nd_gc*, *nd_in* and *nd_out_of_in*.
- The IBIS model named IBIS_IN is located in the IBIS file named *test.ibs*.

Note: Star-Hspice connects the *nd_pc* and *nd_gc* nodes to the voltage sources. Do not manually connect these nodes to voltage sources.

For more examples, see Chapter 7, “Using IBIS Models”, in the *True-Hspice Device Models Reference Manual*.



Chapter 5

Using Sources and Stimuli

This chapter describes element and model statements for independent sources, dependent sources, analog-to-digital elements, and digital-to-analog elements. It also explains each type of element and model statement. Explicit formulas and examples show how various combinations of parameters affect the simulation.

The chapter explains the following topics:

- Independent Source Elements
- Independent Source Functions
- Voltage and Current Controlled Elements
- Voltage-Dependent Voltage Sources — E Elements
- Current-Dependent Current Sources — F Elements
- Voltage-Dependent Current Sources — G Elements
- Current-Dependent Voltage Sources — H Elements
- Digital and Mixed Mode Stimuli
- Replacing Sources With Digital Inputs
- Specifying a Digital Vector File

Independent Source Elements

Use independent source element statements to specify DC, AC, transient, and mixed independent voltage and current sources. Some types of analysis use the associated analysis sources. For example, in a DC analysis, if you specify both DC and AC sources in one independent source element statement, Star-Hspice removes the AC source from the circuit, for the DC analysis. If you specify an independent source for an AC, transient, and DC analysis, Star-Hspice removes transient sources, calculates the operating point, and removes DC sources, for the AC analysis. Initial transient values always override the DC value.

Source Element Conventions

You do not need to ground voltage sources. Star-Hspice assumes that positive current flows from the positive node, through the source, to the negative node. A positive current source forces current to flow out of the N+ node, through the source, and into the N- node.

You can use parameters as values in independent sources. Do not use any of the following reserved keywords to identify these parameters:

AC	ACI	AM	DC	EXP	PE	PL
PU	PULSE	PWL	R	RD	SFFM	SIN

Independent Source Element

The general syntax for an independent source is:

Syntax

```
Vxxx n+ n- <<DC=> dcval> <tranfun> <AC=acmag>
+ <acphase>>
```

or

```
Iyyy n+ n- <<DC=> dcval> <tranfun> <AC=acmag>
+ <acphase>> <M=val>
```

The arguments are:

<i>Vxxx</i>	Independent voltage source element name. Must begin with <i>V</i> , followed by up to 1023 alphanumeric characters.
<i>Iyyy</i>	Independent current source element name. Must begin with <i>I</i> , followed by up to 1023 alphanumeric characters.
<i>n+</i>	Positive node.
<i>n-</i>	Negative node.
<i>DC=dcval</i>	DC source keyword and value, in volts. The <i>tranfun</i> value at time zero overrides the DC value. Default=0.0.
<i>tranfun</i>	Transient source function (one or more of: AM, DC, EXP, PE, PL, PU, PULSE, PWL, SFFM, SIN). The functions specify the characteristics of a time-varying source. See the individual functions, for syntax.
<i>AC</i>	AC source keyword, for use in AC small-signal analysis.
<i>acmag</i>	Magnitude (RMS) of the AC source, in volts.
<i>acphase</i>	Phase of the AC source, in degrees. Default=0.0.
<i>M</i>	Multiplier, to simulate multiple parallel current sources. Star-Hspice multiplies the source current value by <i>M</i> . Default=1.0.

Example 1

- The *VX* voltage source has a 5 volt DC bias.
- The positive terminal connects to node 1.
- The negative terminal is grounded.

```
VX 1 0 5V
```

Example 2

- The *VCC* parameter specifies the DC bias for the *VB* voltage source.
- The positive terminal connects to node 2.
- The negative terminal is grounded.

```
VB 2 0 DC=VCC
```

Example 3

- The *VH* voltage source has a 2-volt DC bias, and a 1-volt RMS AC bias, with 90 degree phase offset.
- The positive terminal connects to node 3.
- The negative terminal connects to node 6.

```
VH 3 6 DC=2 AC=1, 90
```

Example 4

- The piecewise-linear relationship defines the time-varying response for the *IG* current source, which is 1 milliamp at time=0, and 5 milliamps at 25 milliseconds.
- The positive terminal connects to node 8.
- The negative terminal connects to node 7.

```
IG 8 7 PL(1MA 0S 5MA 25MS)
```

Example 5

- The *VCC* parameter specifies the DC bias for the *VCC* voltage source.
- The piecewise-linear relationship defines the time-varying response for the *VCC* voltage source, which is 0 volts at time=0, *VCC* from 10 to 15 nanoseconds, and back to 0 volts at 20 nanoseconds.
- The positive terminal connects to the *in* node.
- The negative terminal connects to the *out* node.
- Star-Hspice determines the operating point for this source, without the DC value (the result is 0 volts).

```
VCC in out VCC PWL 0 0 10NS VCC 15NS VCC 20NS 0
```

Example 6

- The *VIN* voltage source has a 0.001-volt DC bias, and a 1-volt RMS AC bias.
- The sinusoidal time-varying response ranges from 0 to 1 volts, with a frequency of 1 megahertz.
- The positive terminal connects to node 13.
- The negative terminal connects to node 2.

```
VIN 13 2 0.001 AC 1 SIN (0 1 1MEG)
```

- The *ISRC* current source has a 1/3-amp RMS AC response, with a 45-degree phase offset.

- The frequency-modulated, time-varying response ranges from 0 to 1 volts, with a carrier frequency of 10 kHz, a signal frequency of 1 kHz, and a modulation index of 5.
- The positive terminal connects to node 23.
- The negative terminal connects to node 21.

```
ISRC 23 21 AC 0.333 45.0 SFFM (0 1 10K 5 1K)
```

Example 7

- The VMEAS voltage source has a 0-volt DC bias.
- The positive terminal connects to node 12.
- The negative terminal connects to node 9.

```
VMEAS 12 9
```

DC Sources

For a DC source, you can specify the DC current or voltage in different ways:

```
V1 1 0 DC=5V
V1 1 0 5V
I1 1 0 DC=5mA
I1 1 0 5mA
```

- The first two examples specify a DC voltage source of 5 V, connected between node 1 and ground.
- The third and fourth examples specify a 5 mA DC current source, between node 1 and ground.

The direction of current in both sources is from node 1 to ground.

AC Sources

AC current and voltage sources are impulse functions, used for an AC analysis. To specify the magnitude and phase of the impulse, use the AC keyword.

```
V1 1 0 AC=10V,90
VIN 1 0 AC 10V 90
```

The above two examples specify an AC voltage source, with a magnitude of 10 V and a phase of 90 degrees. To specify the frequency sweep range of the AC analysis, use the .AC analysis statement. The AC or frequency domain analysis provides the impulse response of the circuit.

Transient Sources

For transient analysis, you can specify the source as a function of time. The following functions are available:

- pulse
- exponential
- damped sinusoidal
- single-frequency FM
- piecewise linear

Mixed Sources

Mixed sources specify source values for more than one type of analysis. For example, you can specify a DC source, an AC source, and a transient source, all of which connect to the same nodes. In this case, when you run specific analyses, Star-Hspice selects the appropriate DC, AC, or transient source. The exception is the zero-time value of a transient source, which over-rides the DC value; it is selected for operating-point calculation for all analyses.

```
VIN 13 2 0.5 AC 1 SIN (0 1 1MEG)
```

The above example specifies:

- a DC source of 0.5 V
- an AC source of 1 V
- a transient damped sinusoidal source

Each source connects between nodes 13 and 2.

For DC analysis, the program uses zero source value, because the sinusoidal source is zero at time zero.

Independent Source Functions

Star-Hspice provides the following types of independent source functions:

- Pulse (PULSE function)
- Sinusoidal (SIN function)
- Exponential (EXP function)
- Piecewise linear (PWL function)
- Single-frequency FM (SFFM function)
- Single-frequency AM (AM function)

Star-Hspice also provides a data-driven version of PWL. When you use the data-driven PWL, you can reuse the results of an experiment or of a previous simulation, as one or more input sources for a transient simulation.

When you use the independent sources supplied with Star-Hspice, you can specify several useful analog and digital test vectors, for steady state, time domain, or frequency domain analysis. For example, in the time domain, you can specify both current and voltage transient waveforms, as exponential, sinusoidal, piecewise linear, AM, or single-sided FM functions.

Pulse Source Function

Star-Hspice provides a trapezoidal pulse source function, which starts with an initial delay from the beginning of the transient simulation interval, to an onset ramp. During the onset ramp, the voltage or current changes linearly, from its initial value, to the pulse plateau value. After the pulse plateau, the voltage or current moves linearly, along a recovery ramp, back to its initial value. The entire pulse repeats, with a period named *per*, from onset to onset.

The syntax for a pulse source, in an independent voltage or current source, is:

Syntax

```
Vxxx n+ n- PU<LSE> <(>v1 v2 <td <tr <tf <pw  
+ <per>>>>> <)>
```

or

```
Ixxx n+ n- PU<LSE> <(>v1 v2 <td <tr <tf <pw  
+ <per>>>>> <)>
```

The arguments are:

<i>Vxxx</i> , <i>Ixxx</i>	Independent voltage source, which exhibits the pulse response.
<i>PULSE</i>	Keyword for a pulsed time-varying source. The short form is <i>PU</i> .
<i>v1</i>	Initial value of the voltage or current, before the pulse onset (units of volts or amps).
<i>v2</i>	Pulse plateau value (units of volts or amps).
<i>td</i>	Delay time in seconds, from the beginning of the transient interval, to the first onset ramp. Default=0.0; Star-Hspice sets negative values to zero.
<i>tr</i>	Duration of the onset ramp (in seconds), from the initial value, to the pulse plateau value (reverse transit time). Default=TSTEP.
<i>tf</i>	Duration of the recovery ramp (in seconds), from the pulse plateau, back to the initial value (forward transit time). Default=TSTEP.
<i>pw</i>	Pulse width (the width of the plateau portion of the pulse), in seconds. Default=TSTOP.
<i>per</i>	Pulse repetition period, in seconds. Default=TSTOP.

The following table shows the time-value relationship for a PULSE source:

Time	Value
0	v1
td	v1
td + tr	v2
td + tr + pw	v2
td + tr + pw + tf	v1
tstop	v1

Linear interpolation determines the intermediate points.

Note: TSTEP is the printing increment, and TSTOP is the final time.

Example 1

The following example shows the pulse source, connected between node 3 and node 0. In the pulse:

- The output high voltage is 1 V.
- The output low voltage is -1 V.
- The delay is 2 ns.
- The rise and fall time are each 2 ns.
- The high pulse width is 50 ns.
- The period is 100 ns.

```
VIN 3 0 PULSE (-1 1 2NS 2NS 2NS 50NS 100NS)
```

Example 2

The following example is a pulse source, which connects between node 99 and node 0. The syntax shows parameter values for all specifications.

```
V1 99 0 PU lv hv tdlay tris tfall tpw tper
```

Example 3

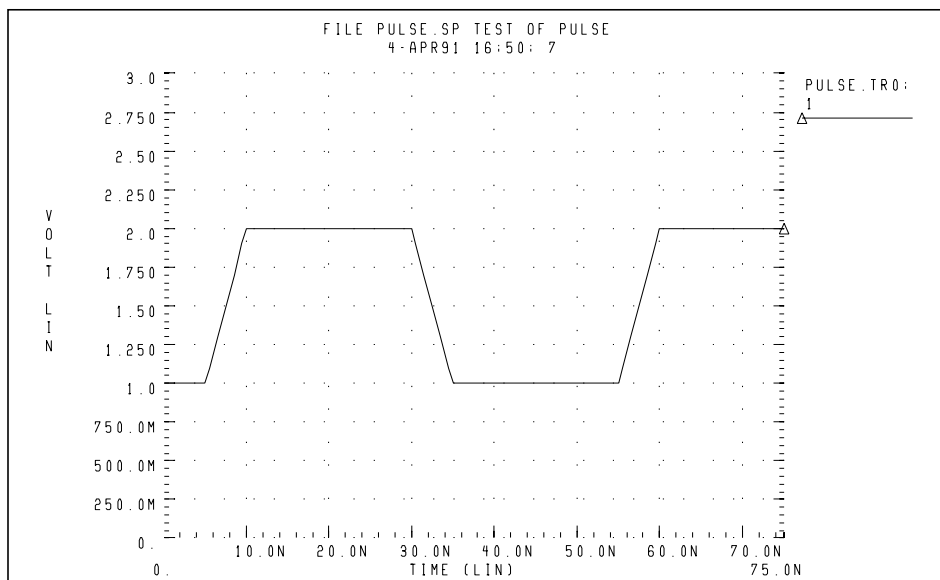
The following example shows an entire netlist, which contains a PULSE voltage source. In the source:

- The initial voltage is 1 volt.
- The pulse voltage is 2 volts.
- The delay time, rise time, and fall time are each 5 nanoseconds.
- The pulse width is 20 nanoseconds.
- The pulse period is 50 nanoseconds.

```
File pulse.sp test of pulse
.option post
.tran .5ns 75ns
vpulse 1 0 pulse( v1 v2 td tr tf pw per )
r1 1 0 1
.param v1=1v v2=2v td=5ns tr=5ns tf=5ns pw=20ns
+ per=50ns
.end
```

Figure 5-1 shows the result of simulating this netlist, in Star-Hspice.

Figure 5-1: Pulse Source Function



Sinusoidal Source Function

Star-Hspice provides a damped sinusoidal source, which is the product of a dying exponential with a sine wave. To apply this waveform, you must specify:

- the sine wave frequency
- the exponential decay constant
- the beginning phase
- the beginning time of the waveform

The syntax for a sinusoidal source in an independent voltage or current source is:

Syntax

```
Vxxx n+ n- SIN <( > vo va <freq <td <q <j >>>> <)>
```

or

```
Ixxx n+ n- SIN <( > vo va <freq <td <q <j >>>> <)>
```

The arguments are:

<i>Vxxx, Ixxx</i>	Independent voltage source that exhibits the sinusoidal response.
<i>SIN</i>	Keyword for a sinusoidal time-varying source.
<i>vo</i>	Voltage or current offset, in volts or amps.
<i>va</i>	Voltage or current RMS amplitude, in volts or amps.
<i>freq</i>	Source frequency in Hz. Default=1/TSTOP.
<i>td</i>	Time delay before beginning the sinusoidal variation, in seconds. Default=0.0. Response is 0 volts or amps, until Star-Hspice reaches the delay value, even with a non-zero DC voltage.
<i>q</i>	Damping factor, in units of 1/seconds. Default=0.0.
<i>j</i>	Phase delay, in units of degrees. Default=0.0.

The following table of expressions defines the waveform shape:

Time	Value
0 to td	$v_o + v_a \cdot \text{SIN}\left(\frac{2 \cdot \Pi \cdot \phi}{360}\right)$
td to tstop	$v_o + v_a \cdot \text{Exp}[-(\text{Time} - \text{td}) \cdot \theta] \cdot \text{SIN}\left\{2 \cdot \Pi \cdot \left[\text{freq} \cdot (\text{time} - \text{td}) + \frac{\phi}{360}\right]\right\}$

where TSTOP is the final time; see [Using the .TRAN Statement on page 11-4](#) for a detailed explanation.

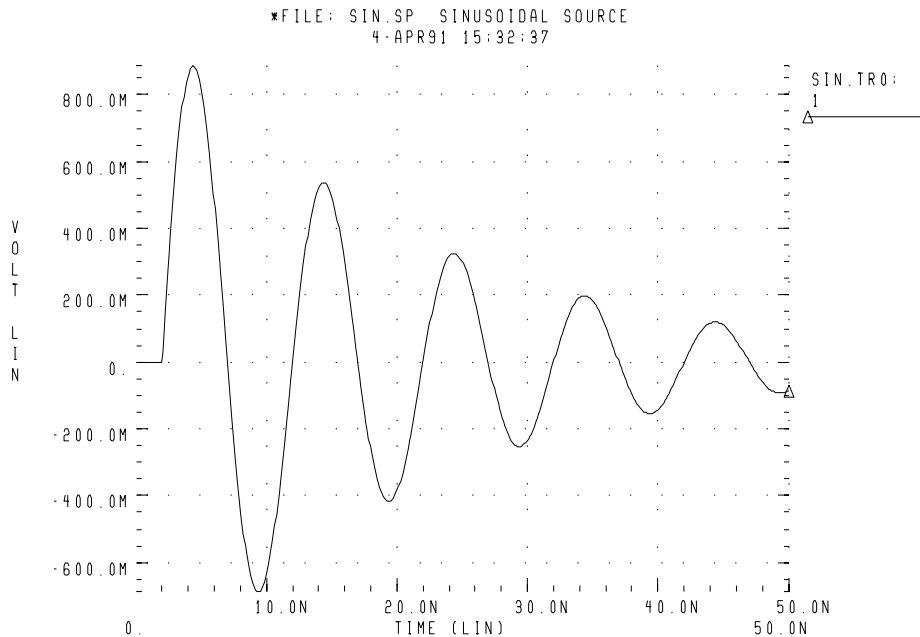
Example

```
VIN 3 0 SIN (0 1 100MEG 1NS 1e10)
```

This damped sinusoidal source connects between nodes 3 and 0. In this waveform:

- The peak value is 1 V.
- The offset is 0 V.
- The frequency is 100 MHz.
- The time delay is 1 ns.
- The damping factor is 1e10.
- The phase delay is zero degree.

See [Figure 5-2 on page 5-13](#) for a plot of the source output.

Figure 5-2: Sinusoidal Source Function

```
*File: SIN.SP THE SINUSOIDAL WAVEFORM
*<decay envelope>
.OPTION POST
.PARAM V0=0 VA=1 FREQ=100MEG DELAY=2N THETA=5E7
+ PHASE=0
V 1 0 SIN (V0 VA FREQ DELAY THETA PHASE)
R 1 0 1
.TRAN .05N 50N
.END
```

This example shows an entire netlist, which contains a SIN voltage source. In the source:

- The initial voltage is 0 volts.
- The pulse voltage is 1 volt.
- The delay time is 2 nanoseconds.
- The frequency is 100 MHz.
- The damping factor is 50 MHz.

Exponential Source Function

The general syntax for an exponential source, in an independent voltage or current source, is:

Syntax

```
Vxxx n+ n- EXP <( > v1 v2 <td1 <t 1 <td2 <t 2>>>> <)>
```

or

```
Ixxx n+ n- EXP <( > v1 v2 <td1 <t 1 <td2 <t 2>>>> <)>
```

The arguments are:

<i>Vxxx, Ixxx</i>	Independent voltage source, exhibiting an exponential response.
<i>EXP</i>	Keyword for an exponential time-varying source.
<i>v1</i>	Initial value of voltage or current, in volts or amps.
<i>v2</i>	Pulsed value of voltage or current, in volts or amps.
<i>td1</i>	Rise delay time, in seconds. Default=0.0.
<i>td2</i>	Fall delay time, in seconds. Default=td1+TSTEP.
<i>t 1</i>	Rise time constant, in seconds. Default=TSTEP.
<i>t 2</i>	Fall time constant, in seconds. Default=TSTEP.

TSTEP is the printing increment, and TSTOP is the final time.

The following table of expressions defines the waveform shape:

Time	Value
0 to td1	v1
td1 to td2	$v1 + (v2 - v1) \cdot \left[1 - \text{Exp}\left(-\frac{\text{Time} - \text{td1}}{\tau_1}\right) \right]$
td2 to tstop	$v1 + (v2 - v1) \cdot \left[1 - \text{Exp}\left(-\frac{\text{td2} - \text{td1}}{\tau_1}\right) \right] \cdot \text{Exp}\left[\frac{-(\text{Time} - \text{td2})}{\tau_2}\right]$

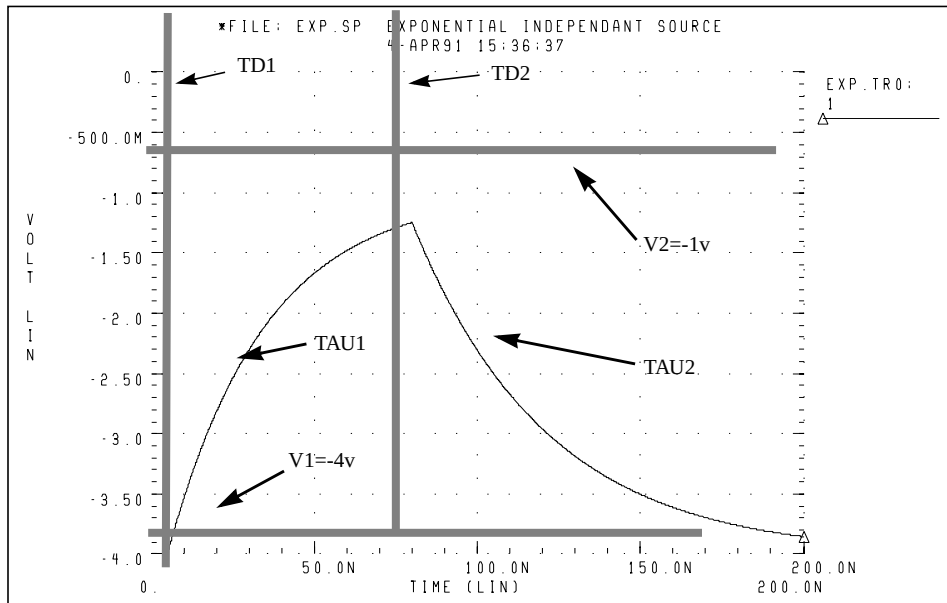
Example

```
VIN 3 0 EXP (-4 -1 2NS 30NS 60NS 40NS)
```

The above example describes an exponential transient source, which connects between nodes 3 and 0. In this source:

- The initial t=0 voltage is -4 V.
- The final voltage is -1 V.
- The waveform rises exponentially, from -4 V to -1 V, with a time constant of 30 ns.
- At 60 ns, the waveform starts dropping to -4 V again, with a time constant of 40 ns.

Figure 5-3: Exponential Source Function



```

*FILE: EXP.SP THE EXPONENTIAL WAVEFORM
.OPTION POST
.PARAM V1=-4 V2=-1 TD1=5N TAU1=30N TAU2=40N TD2=80N
V 1 0 EXP (V1 V2 TD1 TAU1 TD2 TAU2)
R 1 0 1
.TRAN .05N 200N
.END

```

This example shows an entire netlist, which contains an EXP voltage source. In this source:

- The initial $t=0$ voltage is -4 V.
- The final voltage is -1 V.
- The waveform rises exponentially, from -4 V to -1 V, with a time constant of 30 ns.
- At 80 ns, the waveform starts dropping to -4 V again, with a time constant of 40 ns.

Piecewise Linear (PWL) Source Function

The general syntax for a piecewise linear source, in an independent voltage or current source, is:

General Form

```
Vxxx n+ n- PWL <( > t1 v1 <t2 v2 t3 v3...> <R <=repeat>>
+ <TD=delay> <)>
```

or

```
Ixxx n+ n- PWL <( > t1 v1 <t2 v2 t3 v3...> <R <=repeat>>
+ <TD=delay> <)>
```

MSINC and ASPEC Form

```
Vxxx n+ n- PL <( > v1 t1 <v2 t2 v3 t3...> <R <=repeat>>
+ <TD=delay> <)>
```

or

```
Ixxx n+ n- PL <( > v1 t1 <v2 t2 v3 t3...> <R <=repeat>>
+ <TD=delay> <)>
```

The arguments are:

<i>Vxxx, Ixxx</i>	Independent voltage source; uses a piecewise linear response.
<i>PWL</i>	Keyword for a piecewise linear time-varying source.
<i>v1 v2 ... vn</i>	Current or voltage values at the corresponding timepoint.
<i>t1 t2 ... tn</i>	Timepoint values, where the corresponding current or voltage value is valid.
<i>R=repeat</i>	Keyword and time value to specify a repeating function. With no argument, the source repeats from the beginning of the function. <i>repeat</i> is the time, in units of seconds, which specifies the start point of the waveform to repeat. This time needs to be less than the greatest time point, <i>tn</i> .
<i>TD=delay</i>	Time, in units of seconds, which specifies the length of time to delay the piecewise linear function.

- Each pair of values ($t1$, $v1$) specifies that the value of the source is $v1$ (in volts or amps), at time $t1$.
- Linear interpolation between the time points determines the value of the source, at intermediate values of time.
- The PL form of the function accommodates ASPEC style formats, and reverses the order of the time-voltage pairs to voltage-time pairs.
- If you do not specify a time-zero point, Star-Hspice uses the DC value of the source, as the time-zero source value.
- Star-Hspice does not force the source to terminate at the TSTOP value, specified in the .TRAN statement.

If the slope of the piecewise linear function changes below a specified tolerance, the timestep algorithm might not choose the specified time points as simulation time points. To obtain a value for the source voltage or current, Star-Hspice extrapolates neighboring values. As a result, the simulated voltage might deviate slightly from the voltage specified in the PWL list. To force Star-Hspice to use the specified values, use the SLOPETOL option, which reduces the slope change tolerance (for more information about this option, see [Simulation Output on page 8-1](#)).

Specify R to cause the function to repeat. You can specify a value after this R , to indicate the beginning of the function to repeat. The repeat time must equal a breakpoint in the function. For example, if $t1 = 1$, $t2 = 2$, $t3 = 3$, and $t4 = 4$, then the *repeat* value can be 1, 2, or 3.

Specify $TD=val$, to cause a delay at the beginning of the function. You can use TD with or without the repeat function.

Example

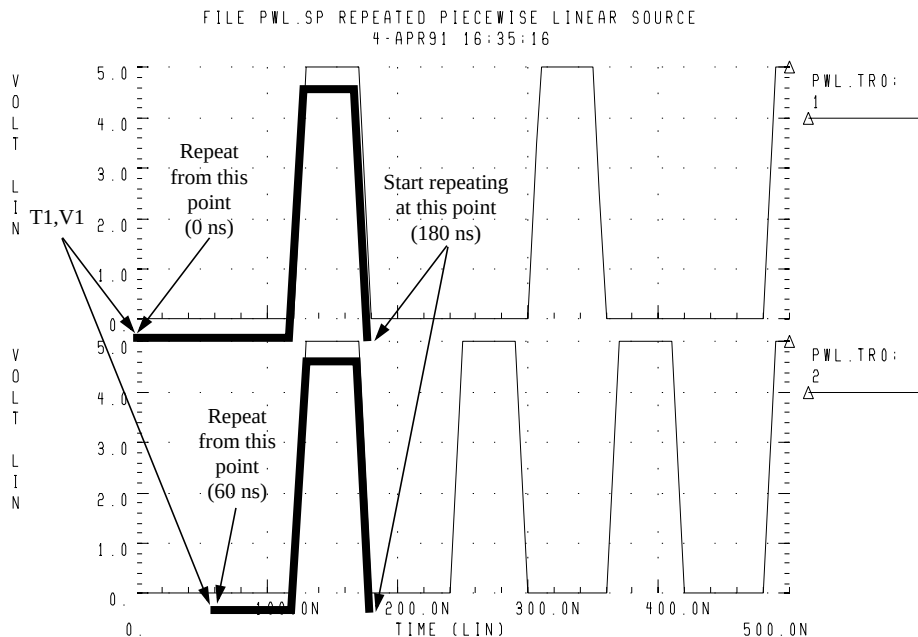
```
*FILE: PWL.SP THE REPEATED PIECEWISE LINEAR SOURCE
*ILLUSTRATION OF THE USE OF THE REPEAT FUNCTION "R"
*file pwl.sp REPEATED PIECEWISE LINEAR SOURCE
.OPTION POST
.TRAN 5N 500N
V1 1 0 PWL 60N 0V, 120N 0V, 130N 5V, 170N 5V, 180N 0V,
+ R 0N
R1 1 0 1
V2 2 0 PL 0V 60N, 0V 120N, 5V 130N, 5V 170N, 0V 180N,
+ R 60N
R2 2 0 1
.END
```

This example shows an entire netlist, which contains two piecewise linear voltage sources. The two sources have the same function:

- The first is in normal format. The repeat starts at the beginning of the function.
- The second is in ASPEC format. The repeat starts at the first timepoint.

See [Figure 5-4](#) for the difference in responses.

Figure 5-4: Results of Using the Repeat Function



Data-Driven Piecewise Linear Source

The general syntax for a data-driven piecewise linear source, in an independent voltage or current source, is:

Syntax

Vxxx n+ n- PWL (TIME, PV)

or

Ixxx n+ n- PWL (TIME, PV)

along with:

```
.DATA dataname
TIME PV
t1 v1
t2 v2
t3 v3
t4 v4
. . .
.ENDDATA
.TRAN DATA=datanam
```

The arguments are:

TIME Parameter name for time value, provided in a .DATA statement.

PV Parameter name for amplitude value, provided in a .DATA statement.

You must use this source with a .DATA statement that contains time-value pairs. For each *tn-vn* (time-value) pair that you specify in the .DATA block, the data-driven PWL function outputs a current or voltage of the specified *tn* duration and with the specified *vn* amplitude.

When you use this source, you can reuse the results of one simulation, as an input source in another simulation. The transient analysis must be data-driven.

Example

```
*DATA DRIVEN PIECEWISE LINEAR SOURCE
V1 1 0 PWL(TIME, pv1)
R1 1 0 1
V2 2 0 PWL(TIME, pv2)
R2 2 0 1
.DATA dsrc
TIME pv1 pv2
0n 5v 0v
5n 0v 5v
10n 0v 5v
.ENDDATA
.TRAN DATA=dsrc
.END
```


This example is an entire netlist, containing two data-driven, piecewise linear voltage sources. The `.DATA` statement contains the two sets of values referenced in the `pv1` and `pv2` sources. The `.TRAN` statement references the data name.

Single-Frequency FM Source Function

The general syntax for including a single-frequency, frequency-modulated source in an independent voltage or current source is:

Syntax

```
Vxxx n+ n- SFFM <( > vo va <fc <mdi <fs>>> <)>
```

or

```
Ixxx n+ n- SFFM <( > vo va <fc <mdi <fs>>> <)>
```

The arguments are:

<i>Vxxx</i> , <i>Ixxx</i>	Independent voltage source, which exhibits the frequency-modulated response.
<i>SFFM</i>	Keyword for a single-frequency, frequency-modulated, time-varying source.
<i>vo</i>	Output voltage or current offset, in volts or amps.
<i>va</i>	Output voltage or current amplitude, in volts or amps.
<i>fc</i>	Carrier frequency, in Hz. Default=1/TSTOP.
<i>mdi</i>	Modulation index, which determines the magnitude of deviation from the carrier frequency. Values normally lie between 1 and 10. Default=0.0.
<i>fs</i>	Signal frequency, in Hz. Default=1/TSTOP.

The following expression defines the waveform shape:

$$\text{sourcevalue} = \text{vo} + \text{va} \cdot \text{SIN}[2 \cdot \pi \cdot \text{fc} \cdot \text{Time} + \text{mdi} \cdot \text{SIN}(2 \cdot \pi \cdot \text{fc} \cdot \text{Time})]$$

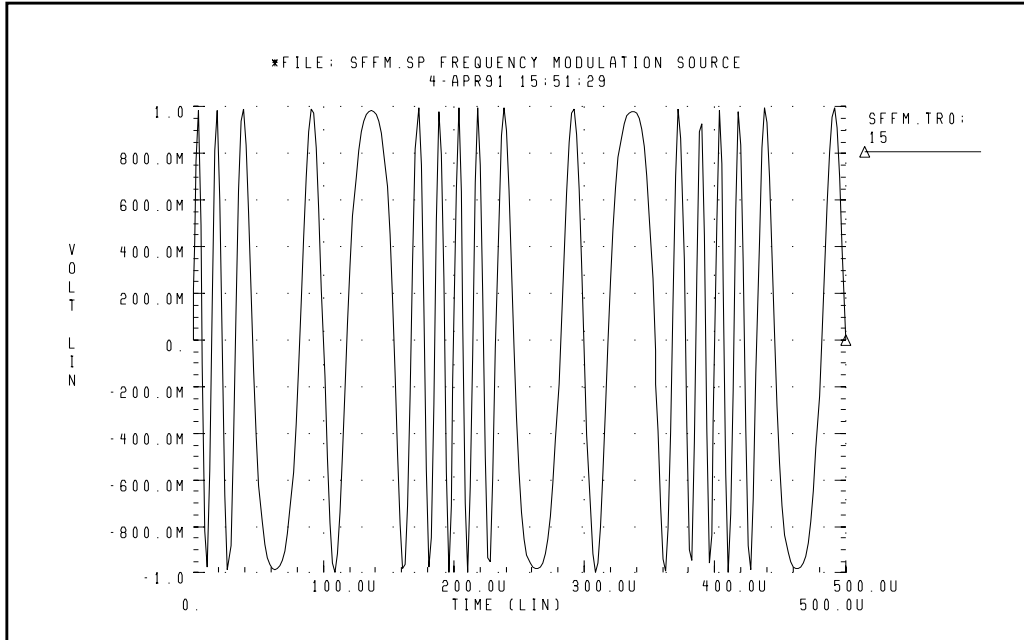
Note: For a description of TSTOP, see [Using the .TRAN Statement on page 11-4](#).

Example

```
*FILE: SFFM.SP THE SINGLE FREQUENCY FM SOURCE
.OPTION POST
V 1 0 SFFM (0, 1M, 20K. 10, 5K)
R 1 0 1
.TRAN .0005M .5MS
.END
```

This example shows an entire netlist, which contains a single-frequency, frequency-modulated voltage source. In this source.

- The offset voltage is 0 volts.
- The maximum voltage is 1 millivolt.
- The carrier frequency is 20 kHz.
- The signal is 5 kHz, with a modulation index of 10 (the maximum wavelength is roughly 10 times as long as the minimum).

Figure 5-5: Single Frequency FM Source

Amplitude Modulation Source Function

The general syntax for including a single-frequency, frequency-modulated source in an independent voltage or current source is:

Syntax

`Vxxx n+ n- AM <(> sa oc fm fc <td> <)>`

or

`Ixxx n+ n- AM <(> sa oc fm fc <td> <)>`

The arguments are:

Vxxx, Ixxx Independent voltage source, which exhibits the amplitude-modulated response.

AM Keyword for an amplitude-modulated, time-varying source.

<i>sa</i>	Signal amplitude, in volts or amps. Default=0.0.
<i>fc</i>	Carrier frequency, in hertz. Default=0.0.
<i>fm</i>	Modulation frequency, in hertz. Default=1/TSTOP.
<i>oc</i>	Offset constant, a unitless constant that determines the absolute magnitude of the modulation. Default=0.0.
<i>td</i>	Delay time before the start of the signal, in seconds. Default=0.0.

The following expression defines the waveform shape:

$$\text{sourcevalue} = \text{sa} \cdot \{ \text{oc} + \text{SIN}[2 \cdot \pi \cdot \text{fm} \cdot (\text{Time} - \text{td})] \} \cdot \text{SIN}[2 \cdot \pi \cdot \text{fc} \cdot (\text{Time} - \text{td})]$$

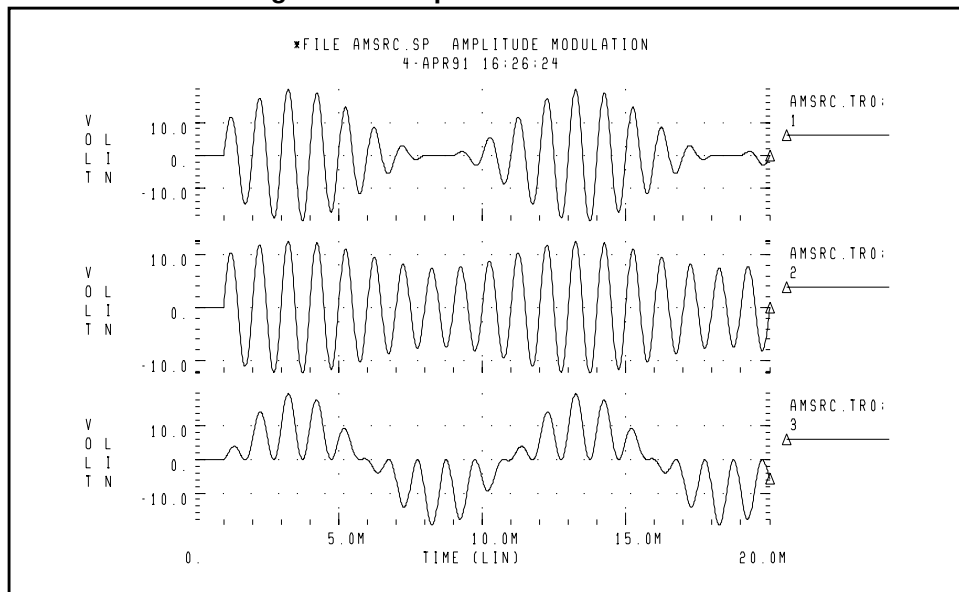
Example

```
.OPTION POST
.TRAN .01M 20M
V1 1 0 AM(10 1 100 1K 1M)
R1 1 0 1
V2 2 0 AM(2.5 4 100 1K 1M)
R2 2 0 1
V3 3 0 AM(10 1 1K 100 1M)
R3 3 0 1
.END
```

This example shows an entire netlist, which contains three amplitude-modulated voltage sources.

- In the first source:
 - The amplitude is 10.
 - The offset constant is 1.
 - The carrier frequency is 1 kHz.
 - The modulation frequency of 100 Hz.
 - The delay is 1 millisecond.

- In the second source, only the amplitude and offset constant differ from the first source:
 - The amplitude is 2.5.
 - The offset constant is 4.
 - The carrier frequency is 1 kHz.
 - The modulation frequency of 100 Hz.
 - The delay is 1 millisecond.
- The third source exchanges the carrier and modulation frequencies, compared to the first source:
 - The amplitude is 10.
 - The offset constant is 1.
 - The carrier frequency is 100 Hz.
 - The modulation frequency of 1 kHz.
 - The delay is 1 millisecond.

Figure 5-6: Amplitude Modulation Plot

Voltage and Current Controlled Elements

Star-Hspice provides two voltage-controlled and two current-controlled elements, known as E, G, H, and F Elements. You can use these controlled elements to model:

- MOS transistors
- bipolar transistors
- tunnel diodes
- SCRs
- analog functions, such as:
 - operational amplifiers
 - summers
 - comparators
 - voltage-controlled oscillators
 - modulators
 - switched capacitor circuits.

The controlled elements can be either linear or non-linear functions, of either controlling node voltages or branch currents, depending on whether you use the polynomial or piecewise linear functions.

Each controlled element has different functions:

- The E Element is a voltage-controlled and/or current-controlled voltage source. It can be:
 - an ideal op-amp.
 - an ideal transformer.
 - an ideal delay element.
 - a piecewise linear, voltage-controlled, multi-input AND, NAND, OR, or NOR gate.

- The G Element is a voltage-controlled and/or current-controlled current source. It can be:
 - a voltage controlled resistor.
 - a piecewise linear, voltage-controlled capacitor.
 - an ideal delay element.
 - or a piecewise linear, multi-input AND, NAND, OR, or NOR gate.
- The H Element is a current-controlled voltage source. It can be:
 - an ideal delay element.
 - a piecewise linear, current-controlled, multi-input AND, NAND, OR, or NOR gate.
- The F Element is a current-controlled current source. It can be:
 - an ideal delay element.
 - a piecewise linear, current-controlled, multi-input AND, NAND, OR, or NOR gate.

The following sections describe the polynomial and piecewise linear functions, and the element statements for linear or non-linear functions.

Polynomial Functions

You can use the controlled element statement to define the controlled output variable (current, resistance, or voltage), as a polynomial function of one or more voltages or branch currents. You can use the POLY(NDIM) parameter to select three polynomial equations.

POLY(1)	One-dimensional equation
POLY(2)	Two-dimensional equation
POLY(3)	Three-dimensional equation

The POLY(1) polynomial equation specifies a polynomial equation as a function of *one* controlling variable, POLY(2) as a function of *two* controlling variables, and POLY(3) as a function of *three* controlling variables.

Each polynomial equation includes polynomial coefficient parameters (P0, P1 ... Pn), which you can set to explicitly define the equation.

One-Dimensional Function

If the function is one-dimensional (a function of one branch current or node voltage), the following expression determines the FV function value:

$$FV = P0 + (P1 \cdot FA) + (P2 \cdot FA^2) + (P3 \cdot FA^3) + (P4 \cdot FA^4) + (P5 \cdot FA^5) + \dots$$

<i>FV</i>	Controlled voltage or current, from the controlled source.
<i>P0 . . . PN</i>	Coefficients of a polynomial equation.
<i>FA</i>	Controlling branch current, or nodal voltage.

Note: If the polynomial is one-dimensional, and you specify exactly one coefficient, Star-Hspice assumes that it is P1 (P0 = 0.0), to facilitate the input of linear-controlled sources.

Example

The following controlled source statement is an example of a one-dimensional function:

E1 5 0 POLY(1) 3 2 1 2.5

The above voltage-controlled voltage source connects to nodes 5 and 0.

1. The single-dimension polynomial function parameter, *POLY(1)*, informs Star-Hspice that E1 is a function of the difference of one nodal voltage pair.

In this example, the voltage difference is between nodes 3 and 2, so $FA = V(3,2)$.

2. The dependent source statement then specifies that P0=1 and P1=2.5. From the one-dimensional polynomial equation above, the defining equation for V(5,0) is:

$$V(5, 0) = 1 + 2.5 \cdot V(3,2)$$

Two-Dimensional Function

Where the function is two-dimensional (a function of two node voltages or two branch currents), the following expression determines FV:

$$\begin{aligned} FV = & P0 + (P1 \cdot FA) + (P2 \cdot FB) + (P3 \cdot FA^2) + (P4 \cdot FA \cdot FB) + (P5 \cdot FB^2) \\ & + (P6 \cdot FA^3) + (P7 \cdot FA^2 \cdot FB) + (P8 \cdot FA \cdot FB^2) + (P9 \cdot FB^3) + \dots \end{aligned}$$

For a two-dimensional polynomial, the controlled source is a function of two nodal voltages or currents. To specify a two-dimensional polynomial, set POLY(2) in the controlled source statement.

Example

For example, generate a voltage-controlled source that specifies the controlled voltage, $V(1, 0)$, as:

$$V(1, 0) = 3 \cdot V(3, 2) + 4 \cdot V(7, 6)^2$$

To implement this function, use the following controlled source element statement:

```
E1 1 0 POLY(2) 3 2 7 6 0 3 0 0 0 4
```

This example specifies a controlled voltage source, which connects between nodes 1 and 0, and is controlled by two differential voltages:

- The voltage difference between nodes 3 and 2.
- The voltage difference between nodes 7 and 6.

That is, $FA=V(3,2)$, and $FB=V(7,6)$. The polynomial coefficients are:

- $P0=0$
- $P1=3$
- $P2=0$
- $P3=0$
- $P4=0$
- $P5=4$

Three-Dimensional Function

For a three-dimensional polynomial function, with FA, FB, and FC as its arguments, the following expression determines the FV function value:

$$\begin{aligned}
 FV = & P0 + (P1 \cdot FA) + (P2 \cdot FB) + (P3 \cdot FC) + (P4 \cdot FA^2) \\
 & + (P5 \cdot FA \cdot FB) + (P6 \cdot FA \cdot FC) + (P7 \cdot FB^2) + (P8 \cdot FB \cdot FC) \\
 & + (P9 \cdot FC^2) + (P10 \cdot FA^3) + (P11 \cdot FA^2 \cdot FB) + (P12 \cdot FA^2 \cdot FC) \\
 & + (P13 \cdot FA \cdot FB^2) + (P14 \cdot FA \cdot FB \cdot FC) + (P15 \cdot FA \cdot FC^2) \\
 & + (P16 \cdot FB^3) + (P17 \cdot FB^2 \cdot FC) + (P18 \cdot FB \cdot FC^2) \\
 & + (P19 \cdot FC^3) + (P20 \cdot FA^4) + \dots
 \end{aligned}$$

Example

For example, generate a voltage-controlled source that specifies the voltage as:

$$V(1, 0) = 3 \cdot V(3,2) + 4 \cdot V(7,6)^2 + 5 \cdot V(9,8)^3$$

from the above equation, and the three-dimensional polynomial equation:

$$FA = V(3,2)$$

$$FB = V(7,6)$$

$$FC = V(9,8)$$

$$P1 = 3$$

$$P7 = 4$$

$$P19 = 5$$

Substituting these values into the voltage-controlled voltage source statement, yields the following:

```
V(1, 0) POLY(3) 3 2 7 6 9 8 0 3 0 0 0 0 0 4 0 0 0 0 0 0 0 0 0 0 5
```

The preceding example specifies a controlled voltage source, which connects between nodes 1 and 0, and is controlled by three differential voltages:

- The voltage difference between nodes 3 and 2.
- The voltage difference between nodes 7 and 6.
- The voltage difference between nodes 9 and 8.

That is:

- $FA = V(3,2)$
- $FB = V(7,6)$
- $FC = V(9,8)$.

The statement defines the polynomial coefficients as:

- $P1=3$
- $P7=4$
- $P19=5$
- The rest are zero.

Piecewise Linear Function

You can use the one-dimensional piecewise linear function to model some special element characteristics, such as those of:

- tunnel diodes
- silicon-controlled rectifiers
- diode breakdown regions

To describe the piecewise linear function, specify, measured data points. Although some data points describe the device characteristics, Star-Hspice automatically smooths the corners. This rounding ensures derivative continuity and, as a result, better convergence.

A DELTA parameter controls the curvature of the characteristic at the corners. The smaller the DELTA, the sharper the corners are. The maximum DELTA is limited to half of the smallest breakpoint distance. If the breakpoints are sufficiently separated, specify the DELTA to a proper value.

- You *can* specify up to 100 point pairs.
- You *must* specify at least two point pairs (four coefficients).

To model bidirectional switch or transfer gates, G Elements use the NPWL and PPWL functions, which behave the same way as NMOS and PMOS transistors.

The piecewise linear function can also model multi-input AND, NAND, OR, and NOR gates. In this usage, only one input determines the state of the output.

- In AND / NAND gates, the piecewise linear function uses the input with the smallest value, to determine the corresponding output of the gates.
- In OR / NOR gates, the input with the largest value determines the corresponding output of the gates.

Voltage-Dependent Voltage Sources — E Elements

This section explains E Element syntax statements, and defines their parameters.

Voltage-Controlled Voltage Source (VCVS)

Linear Syntax

```
Exxx n+ n- <VCVS> in+ in- gain <MAX=val> <MIN=val>
+ <SCALE=val> <TC1=val> <TC2=val><ABS=1> <IC=val>
```

Polynomial Syntax

```
Exxx n+ n- <VCVS> POLY(NDIM) in1+ in1- ... inndim
+ inndim-<TC1=val> <TC2=val> <SCALE=val> <MAX=val>
+ <MIN=val> <ABS=1> P0 <P1...> <IC=vals>
```

Piecewise Linear Syntax

```
Exxx n+ n- <VCVS> PWL(1) in+ in- <DELTA=val>
+ <SCALE=val> <TC1=val> <TC2=val> x1,y1 x2,y2 ...
+ x100,y100 <IC=val>
```

Multi-Input Gate Syntax

```
Exxx n+ n- <VCVS> gatetype(k) in1+ in1- ... ink+ ink-
+ <DELTA=val> <TC1=val> <TC2=val> <SCALE=val>
+ x1,y1 ... x100,y100 <IC=val>
```

Delay Element Syntax

```
Exxx n+ n- <VCVS> DELAY in+ in- TD=val <SCALE=val>
+ <TC1=val> <TC2=val> <NPDELAY=val>
```

Behavioral Voltage Source

The syntax is:

```
Exxx n+ n- VOL='equation' <MAX>=val> <MIN=val>
```

Ideal Op-Amp

The syntax is:

```
EXXX n+ n- OPAMP in+ in-
```

Ideal Transformer

The syntax is:

```
EXXX n+ n- TRANSFORMER in+ in- k
```

E Element Parameters

<i>ABS</i>	Output is an absolute value, if ABS=1.
<i>DELAY</i>	Keyword for the delay element. The delay element is the same as the voltage-controlled voltage source, except it has an associated propagation delay, τ_D. This element adjusts propagation delay in macro-modelling. Note: DELAY is a reserved word; do not use it as a node name.
<i>DELTA</i>	Controls the curvature of the piecewise linear corners. This parameter defaults to one-fourth of the smallest breakpoint distances. The maximum is limited to one-half of the smallest breakpoint distances.
<i>Exxx</i>	Voltage-controlled element name. The parameter starts with E, followed by up to 1023 alphanumeric characters.
<i>gain</i>	Voltage gain.
<i>gatetype(k)</i>	Can be one of AND, NAND, OR, or NOR. ■ (k) represents the number of inputs of the gate. ■ The x's and y's represent the piecewise linear variation of output, as a function of input. In multi-input gates, only one input determines the state of the output.
<i>IC</i>	Initial condition: the initial estimate of the value(s) of the controlling voltage(s). If you do not specify IC, the default=0.0.
<i>in +/-</i>	Positive or negative controlling nodes. Specify one pair for each dimension.

<i>k</i>	Ideal transformer turn ratio: $V(\text{in+}, \text{in-}) = k \cdot V(\text{n+}, \text{n-})$ or, number of gates input.
<i>MAX</i>	Maximum output voltage value. The default is undefined, and sets no maximum value.
<i>MIN</i>	Minimum output voltage value. The default is undefined, and sets no minimum value.
<i>n+/-</i>	Positive or negative node of a controlled element.
<i>NDIM</i>	Polynomial dimensions. If you do not specify <code>POLY(NDIM)</code> , Star-Hspice assumes a one-dimensional polynomial. NDIM must be a positive number.
<i>NPDELAY</i>	Sets the number of data points to use in delay simulations. The default value is the larger of either 10, or the smaller of <code>TD/tstep</code> and <code>tstop/tstep</code> . That is, $\text{NPDELAY}_{\text{default}} = \max\left[\frac{\min\langle \text{TD}, \text{tstop} \rangle}{\text{tstep}}, 10\right]$ The <code>.TRAN</code> statement specifies <code>tstep</code> and <code>tstop</code> values.
<i>OPAMP</i>	The keyword for an ideal op-amp element. <code>OPAMP</code> is a reserved word; do <i>not</i> use it as a node name.
<i>P0, P1 ...</i>	The polynomial coefficients. When you specify one coefficient, Star-Hspice assumes that it is <code>P1</code> (<code>P0=0.0</code>), and that the element is linear. When you specify more than one polynomial coefficient, the element is nonlinear, and <code>P0, P1, P2 ...</code> represent them (see Polynomial Functions on page 5-27).
<i>POLY</i>	Polynomial keyword function.
<i>PWL</i>	Piecewise linear keyword function.

<i>SCALE</i>	Element value multiplier.
<i>TC1,TC2</i>	First-order and second-order temperature coefficients. Star-Hspice updates the <i>SCALE</i> by temperature: $\text{SCALE}_{\text{eff}} = \text{SCALE} \cdot (1 + \text{TC1} \cdot \Delta t + \text{TC2} \cdot \Delta t^2)$
<i>TD</i>	Time delay keyword.
<i>TRANSFORMER</i>	Keyword for an ideal transformer. <i>TRANS</i> is a reserved word; do <i>not</i> use it as a node name.
<i>VCVS</i>	Keyword for voltage-controlled voltage source. <i>VCVS</i> is a reserved word; do <i>not</i> use it as a node name.
<i>x1,...</i>	Controlling voltage across the in+ and in- nodes. The <i>x</i> values must be in increasing order.
<i>y1,...</i>	Corresponding element values of <i>x</i> .

E Element Examples

Ideal OpAmp

You can use the voltage-controlled voltage source to build a voltage amplifier, with supply limits.

- The output voltage across nodes 2,3 is $v(14,1) \cdot 2$.
- The value of the voltage gain parameter is 2.
- The *MAX* parameter specifies a maximum *E1* voltage of 5 V.
- The *MIN* parameters specifies a minimum *E1* voltage output of -5 V.

If, for example, $V(14,1) = -4\text{V}$, Star-Hspice sets *E1* to -5 V, not to -8 V, as the equation produces.

```
Eopamp 2 3 14 1 MAX=+5 MIN=-5 2.0
```

You can define and use a parameter in the following format, to specify a value for polynomial coefficient parameters:

```
.PARAM CU = 2.0
E1 2 3 14 1 MAX=+5 MIN=-5 CU
```

Voltage Summer

An ideal voltage summer specifies the source voltage as a function of three controlling voltage(s):

- V(13,0)
- V(15,0)
- V(17,0)

To describe a voltage source, the voltage summer uses the following value:

$$V(13,0) + V(15,0) + V(17,0)$$

This example represents an ideal voltage summer. This example initializes the three controlling voltages, for a DC operating point analysis, to 1.5, 2.0, and 17.25 V, respectively.

```
EX 17 0 POLY(3) 13 0 15 0 17 0 0 1 1 1 IC=1.5,2.0,17.25
```

Polynomial Function

A voltage-controlled source can also output a non-linear function, using a one-dimensional polynomial. This example does not specify the POLY parameter, so Star-Hspice assumes a one-dimensional polynomial—that is, a function of one controlling voltage. The equation corresponds to the element syntax. Behavioral equations replace this older method.

$$V(3,4) = 10.5 + 2.1 * V(21,17) + 1.75 * V(21,17)^2$$

```
E2 3 4 POLY 21 17 10.5 2.1 1.75
```

Zero-Delay Inverter Gate

You can use a piecewise linear transfer function to build a simple inverter, with no delay.

```
Einv out 0 PWL(1) in 0 .7v,5v 1v,0v
```

Ideal Transformer

If the turn ratio is 10 to 1, the voltage relationship is $V(\text{out})=V(\text{in})/10$.

```
Etrans out 0 TRANSFORMER in 0 10
```

Voltage-Controlled Oscillator (VCO)

Use the VOL keyword to define a single-ended input, which controls the output of a VCO.

In the following example, the voltage at the *control* node controls the frequency of the sinusoidal output voltage at the *out* node. The *v0* parameter is the DC offset voltage, and *gain* is the amplitude. The output is a sinusoidal voltage, whose frequency is specified in *freq · control*.

```
Evco out 0 VOL='v0+gain*SIN(6.28 freq*v(control)*TIME)'
```

Note: This equation is valid only for a steady-state VCO (fixed voltage). This equation does not apply if you sweep the control voltage.

Current-Dependent Current Sources — F Elements

This section explains F Element syntax statements, and defines their parameters.

Current-Controlled Current Source (CCCS)

Linear Syntax

```
Fxxx n+ n- <CCCS> vn1 gain <MAX=val> <MIN=val>
<SCALE=val> <TC1=val> <TC2=val> <M=val> <ABS=1>
<IC=val>
```

Polynomial Syntax

```
Fxxx n+ n- <CCCS> POLY(NDIM) vn1 <... vnndim> <MAX=val>
<MIN=val> <TC1=val> <TC2=val> <SCALE=vals> <M=val>
<ABS=1> P0 <P1...> <IC=vals>
```

Piecewise Linear Syntax

```
Fxxx n+ n- <CCCS> PWL(1) vn1 <DELTA=val>
<SCALE=val><TC1=val> <TC2=val> <M=val> x1,y1 ...
x100,y100 <IC=val>
```

Multi-Input Gate Syntax

```
Fxxx n+ n- <CCCS> gatetype(k) vn1, ... vnk <DELTA=val>
<SCALE=val> <TC1=val> <TC2=val> <M=val> <ABS=1> x1,y1
... x100,y100 <IC=val>
```

Delay Element Syntax

```
Fxxx n+ n- <CCCS> DELAY vn1 TD=val <SCALE=val>
<TC1=val><TC2=val> NPDELAY=val
```

F Element Parameters

<i>ABS</i>	Output is an absolute value, if ABS=1.
<i>CCCS</i>	Keyword for current-controlled current source. CCCS is a reserved word; do not use it as a node name.
<i>DELAY</i>	Keyword for the delay element. The delay element is the same as a current-controlled current source, except it has an associated propagation delay, TD. Use this element to adjust the propagation delay in the macromodel process. DELAY is a reserved word; do not use it as a node name.
<i>DELTA</i>	Controls the curvature of piecewise linear corners. The default is 1/4 of the smallest breakpoint distances. The maximum is 1/2 of the smallest breakpoint distances.
<i>Fxxx</i>	Name of the current-controlled current source element. The parameter must begin with F, followed by up to 1023 alphanumeric characters.
<i>gain</i>	Current gain.
<i>gatetype(k)</i>	Can be one of AND, NAND, OR, or NOR. (<i>k</i>) is the number of inputs of the gate. <i>x</i> 's and <i>y</i> 's are the piecewise linear variation of output, as a function of input. In multi-input gates, one input determines the output state. Do not use any of the above keyword names as a node name.
<i>IC</i>	Initial condition (estimate) of the controlling current(s), in amps. If you do not specify IC, the default=0.0.
<i>M</i>	Number of elements in parallel.
<i>MAX</i>	Maximum output current value. The default is undefined, and sets no maximum value.
<i>MIN</i>	Minimum output current value. The default is undefined, and sets no minimum value.

<i>n+/-</i>	Positive or negative controlled-source connecting nodes.
<i>NDIM</i>	Polynomial dimensions. If you do not specify <code>POLY(NDIM)</code> , Star-Hspice assumes a one-dimensional polynomial. <i>NDIM</i> must be a positive number.
<i>NPDELAY</i>	Sets the number of data points to use in delay simulations. The default value is the larger of either 10, or the smaller of $TD/tstep$ and $tstop/tstep$. That is, $NPDELAY_{\text{default}} = \max \left[\frac{\min \langle TD, tstop \rangle}{tstep}, 10 \right]$ The <code>.TRAN</code> statement specifies the <i>tstep</i> and <i>tstop</i> values.
<i>P0, P1 ...</i>	■ If you specify one polynomial coefficient, Star-Hspice assumes that it is <i>P1</i> (<i>P0</i>=0.0), and the source is linear. ■ If you specify more than one polynomial coefficient, then the source is non-linear, and Star-Hspice assumes that the polynomials are <i>P0</i>, <i>P1</i>, <i>P2</i> ...
<i>POLY</i>	Polynomial keyword function.
<i>PWL</i>	Piecewise linear keyword function.
<i>SCALE</i>	Multiplier for the element value.
<i>TC1,TC2</i>	First-order and second-order temperature coefficients. Temperature updates the <i>SCALE</i>: $SCALE_{\text{eff}} = SCALE \cdot (1 + TC1 \cdot \Delta t + TC2 \cdot \Delta t^2)$
<i>TD</i>	Time-delay keyword.
<i>vn1 ...</i>	Names of voltage sources, through which the controlling current flows. Specify one name for each dimension.
<i>x1,...</i>	Controlling current, through the <i>vn1</i> source. Specify the <i>x</i> values in increasing order.
<i>y1,...</i>	Corresponding output current values of <i>x</i> .

F Element Examples

```
F1 13 5 VSENS MAX=+3 MIN=-3 5
```

The above example describes a current-controlled current source, connected between nodes 13 and 5. The current, which controls the value of the controlled source, flows through the voltage source named VSENS.

Note: To use a current-controlled current source, you can place a dummy independent voltage source into the path of the controlling current.

The defining equation is:

$$I(F1) = 5 \cdot I(VSENS)$$

- The current gain is 5.
- The maximum current flow through F1 is 3 A.
- The minimum current flow is -3 A.

If $I(VSENS) = 2$ A, this examples sets $I(F1)$ to 3 amps, not 10 amps as the equation suggests. You can define a parameter for the polynomial coefficient(s), as shown below:

```
.PARAM VU = 5
F1 13 5 VSENS MAX=+3 MIN=-3 VU
```

The next example describes a current-controlled current source, with the value:

$$I(F2) = 1e-3 + 1.3e-3 \cdot I(VCC)$$

```
F2 12 10 POLY VCC 1MA 1.3M
```

Current flows from the positive node, through the source, to the negative node. The direction of positive controlling current flow is from the positive node, through the source, to the negative node of vnam (linear), or to the negative node of each voltage source (nonlinear).

```
Fd 1 0 DELAY vin TD=7ns SCALE=5
```

The above example is a delayed, current-controlled current source.

```
Filim 0 out PWL(1) vsrc -1a, -1a 1a, 1a
```

The final example is a piecewise-linear, current-controlled current source.

Voltage-Dependent Current Sources — G Elements

This section explains G Element syntax statements, and their parameters.

Voltage-Controlled Current Source (VCCS)

Linear Syntax

```
Gxxx n+ n- <VCCS> in+ in- transconductance <MAX=val>
+ <MIN=val> <SCALE=val> <M=val> <TC1=val> <TC2=val>
+ <ABS=1> <IC=val>
```

Polynomial Syntax

```
Gxxx n+ n- <VCCS> POLY(NDIM) in1+ in1- ...
+ <inndim+ inndim-> MAX=val> <MIN=val> <SCALE=val>
+ <M=val> <TC1=val> <TC2=val> <ABS=1> P0<P1...> <IC=vals>
```

Piecewise Linear Syntax

```
Gxxx n+ n- <VCCS> PWL(1) in+ in- <DELTA=val>
+ <SCALE=val> <M=val> <TC1=val> <TC2=val>
+ x1,y1 x2,y2 ... x100,y100 <IC=val> <SMOOTH=val>

Gxxx n+ n- <VCCS> NPWL(1) in+ in- <DELTA=val>
+ <SCALE=val> <M=val> <TC1=val><TC2=val>
+ x1,y1 x2,y2 ... x100,y100 <IC=val> <SMOOTH=val>

Gxxx n+ n- <VCCS> PPWL(1) in+ in- <DELTA=val>
+ <SCALE=val> <M=val> <TC1=val> <TC2=val>
+ x1,y1 x2,y2 ... x100,y100 <IC=val> <SMOOTH=val>
```

Multi-Input Gate Syntax

```
Gxxx n+ n- <VCCS> gatetype(k) in1+ in1- ...
+ ink+ ink- <DELTA=val> <TC1=val> <TC2=val> <SCALE=val>
+ <M=val> x1,y1 ... x100,y100<IC=val>
```


Delay Element Syntax

```
Gxxx n+ n- <VCCS> DELAY in+ in- TD=val <SCALE=val>
+ <TC1=val> <TC2=val> NPDELAY=val
```

Behavioral Current Source

Syntax

```
Gxxx n+ n- CUR='equation' <MAX>=val> <MIN=val> <M=val>
+ <SCALE=val>
```

Voltage-Controlled Resistor (VCR)

Linear Syntax

```
Gxxx n+ n- VCR in+ in- transfactor <MAX=val> <MIN=val>
+ <SCALE=val> <M=val> <TC1=val> <TC2=val> <IC=val>
```

Polynomial Syntax

```
Gxxx n+ n- VCR POLY(NDIM) in1+ in1- ...
+ <inndim+ inndim-> <MAX=val> <MIN=val> <SCALE=val>
+ <M=val> <TC1=val> <TC2=val> P0 <P1...> <IC=vals>
```

Piecewise Linear Syntax

```
Gxxx n+ n- VCR PWL(1) in+ in- <DELTA=val> <SCALE=val>
+ <M=val> <TC1=val> <TC2=val> x1,y1 x2,y2 ... x100,y100
+ <IC=val> <SMOOTH=val>
```

```
Gxxx n+ n- VCR NPWL(1) in+ in- <DELTA=val> <SCALE=val>
+ <M=val> <TC1=val> <TC2=val> x1,y1 x2,y2 ... x100,y100
+ <IC=val> <SMOOTH=val>
```

```
Gxxx n+ n- VCR PPWL(1) in+ in- <DELTA=val> <SCALE=val>
+ <M=val> <TC1=val> <TC2=val> x1,y1 x2,y2 ... x100,y100
+ <IC=val> <SMOOTH=val>
```

Multi-Input Gate Syntax

```
Gxxx n+ n- VCR gatetype(k) in1+ in1- ... ink+ ink-
+ <DELTA=val> <TC1=val> <TC2=val> <SCALE=val> <M=val>
+ x1,y1 ... x100,y100 <IC=val>
```

Voltage-Controlled Capacitor (VCCAP)

Syntax (Piecewise Linear)

```
Gxxx n+ n- VCCAP PWL(1) in+ in- <DELTA=val>
+ <SCALE=val> <M=val> <TC1=val><TC2=val> x1,y1
+ x2,y2 ... x100,y100 <IC=val> <SMOOTH=val>
```

Use the NPWL and PPWL functions to interchange the $n+$ and $n-$ nodes, but use the same transfer function. The following summarizes this action:

NPWL Function

For node “in-” connected to “n+”:

- If $v(n+,n-) < 0$, then the controlling voltage is $v(in+,in-)$.
- Otherwise, the controlling voltage is $v(in+,n-)$.

For node “in-” connected to “n-”:

- If $v(n+,n-) > 0$, then the controlling voltage is $v(in+,in-)$.
- Otherwise, the controlling voltage is $v(in+,n+)$.

PPWL Function

For node “in-” connected to “n+”:

- If $v(n+,n-) > 0$, then the controlling voltage is $v(in+,in-)$.
- Otherwise, the controlling voltage is $v(in+,n-)$.

For node “in-” connected to “n-”:

- If $v(n+,n-) < 0$, then the controlling voltage is $v(in+,in-)$.
- Otherwise, the controlling voltage is $v(in+,n+)$.

If the $in-$ node does not connect to either $n+$ or $n-$, then Star-Hspice changes NPWL and PPWL to PWL.

G Element Parameters

<i>ABS</i>	Output is an absolute value, if ABS=1.
<i>CUR, VALUE</i>	Current output that flows from n+ to n-. The equation that you define can be a function of: ■ node voltages ■ branch currents ■ TIME ■ temperature (TEMPER) ■ frequency (HERTZ)
<i>DELAY</i>	Keyword for the delay element. The delay element is the same as in the voltage-controlled current source, except that it has an associated propagation delay, TD. Use this element to adjust the propagation delay in the macromodel process. DELAY is a keyword; do not use it as a node name.
<i>DELTA</i>	Controls the curvature of piecewise linear corners. Defaults to 1/4 of the smallest breakpoint distances. The maximum is 1/2 of the smallest breakpoint distances.
<i>Gxxx</i>	Name of the voltage-controlled element. Must begin with G, followed by up to 1023 alphanumeric characters.
<i>gatetype(k)</i>	Can be one of AND, NAND, OR, or NOR. The (k) parameter represents the number of inputs of the gate. The x's and y's represent the piecewise linear variation of the output, as a function of the input. In multi-input gates, only one input determines the state of the output.
<i>IC</i>	Initial condition. Initial estimate of the value(s) of controlling voltage(s). If you do not specify IC, the default=0.0.
<i>in +/-</i>	Positive or negative controlling nodes. Specify one pair for each dimension.
<i>M</i>	Number of elements in parallel.

<i>MAX</i>	Maximum current or resistance value. The default is undefined, and sets no maximum value.
<i>MIN</i>	Minimum current or resistance value. The default is undefined, and sets no minimum value.
<i>n+/-</i>	Positive or negative node of the controlled element.
<i>NDIM</i>	Polynomial dimensions. If you do not specify POLY(NDIM), Star-Hspice assumes a one-dimensional polynomial. NDIM must be a positive number.
<i>NPDELAY</i>	Sets the number of data points to use in delay simulations. The default value is the larger of either 10, or the smaller of TD/tstep and tstop/tstep. That is, $NPDELAY_{\text{default}} = \max \left[\frac{\min \langle TD, tstop \rangle}{tstep}, 10 \right]$ The .TRAN statement specifies the tstep and tstop values.
<i>NPWL</i>	Models the symmetrical bidirectional switch or transfer gate, NMOS.
<i>P0, P1 ...</i>	The polynomial coefficients. ■ If you specify one coefficient, Star-Hspice assumes that it is P1 (P0=0.0), and the element is linear. ■ If you specify more than one polynomial coefficient, the element is non-linear, and the coefficients are P0, P1, P2 ... (see Polynomial Functions on page 5-27).
<i>POLY</i>	Polynomial keyword function.
<i>PWL</i>	Piecewise linear keyword function.
<i>PPWL</i>	Models the symmetrical bidirectional switch or transfer gate, PMOS.
<i>SCALE</i>	Multiplier for the element value.

<i>SMOOTH</i>	For piecewise-linear, dependent-source elements, SMOOTH selects the curve-smoothing method. A curve-smoothing method simulates exact data points that you provide. For example, you can use this method to make Star-Hspice simulate specific data points that correspond to either measured data or data sheets. Choices for SMOOTH are 1 or 2: 1. Selects the smoothing method used in Hspice versions before release H93A. Use this method to maintain compatibility with simulations that you ran, using releases older than H93A. 2. Selects the smoothing method, which uses data points that you provide. This is the default for Hspice versions starting with release H93A.
<i>TC1,TC2</i>	First-order and second-order temperature coefficients. Temperature updates the SCALE: $CALE_{eff} = SCALE \cdot (1 + TC1 \cdot \Delta t + TC2 \cdot \Delta t^2)$
<i>TD</i>	Time delay keyword.
<i>transconductance</i>	Voltage-to-current conversion factor.
<i>transfactor</i>	Voltage-to-resistance conversion factor.
<i>VCCAP</i>	Keyword for the voltage-controlled capacitance element. VCCAP is a reserved word; do not use it as a node name.
<i>VCCS</i>	Keyword for voltage-controlled current source. VCCS is a reserved word; do not use it as a node name.
<i>VCR</i>	Keyword for voltage controlled resistor element. VCR is a reserved word; do not use it as a node name.
<i>x1,...</i>	Controlling voltage, across the <i>in+</i> and <i>in-</i> nodes. Specify the <i>x</i> values in increasing order.
<i>y1,...</i>	Corresponding element values of <i>x</i> .

G Element Examples

Switch

A voltage-controlled resistor represents a basic switch characteristic. The resistance between nodes 2 and 0 varies linearly, from 10 meg to 1 m ohms, when voltage across nodes 1 and 0 varies between 0 and 1 volt. The resistance remains at 10 meg when below the lower voltage limit, and at 1 m ohms when above the upper voltage limit.

```
Gswitch 2 0 VCR PWL(1) 1 0 0v,10meg 1v,1m
```

Switch-Level MOSFET

To model a switch level n-channel MOSFET, use the N-piecewise linear resistance switch. The resistance value does not change when you switch the *d* and *s* node positions.

```
Gnmos d s VCR NPWL(1) g s LEVEL=1 0.4v,150g  
+ 1v,10meg 2v,50k 3v,4k 5v,2k
```

Voltage-Controlled Capacitor

The capacitance value across the (*out*,0) nodes varies linearly, from 1 p to 5 p, when voltage across the (*ctrl*,0) nodes varies between 2 v and 2.5 v. The capacitance value remains constant at 1 picofarad when below the lower voltage limit, and at 5 picofarads when above the upper voltage limit.

```
Gcap out 0 VCCAP PWL(1) ctrl 0 2v,1p 2.5v,5p
```

Zero-Delay Gate

To implement a two-input AND gate, use an expression and a piecewise linear table. The inputs are voltages at the *a* and *b* nodes, and the output is the current flow from the *out* node to node 0. Star-Hspice multiplies the current by the *SCALE* value, which in this example is the inverse of the load resistance, connected across the (*out*,0) nodes.

```
Gand out 0 AND(2) a 0 b 0 SCALE='1/rload' 0v,0a 1v,.5a  
+ 4v,4.5a 5v,5a
```

Delay Element

A delay is a low-pass filter type delay, similar to that of an opamp. In contrast, a transmission line has an infinite frequency response. A glitch input to a G delay attenuates in a way that is similar to a buffer circuit. In this example, the output of the delay element is the current flow from the *out* node to node 1; the value is the voltage across the (*in*, 0) nodes, multiplied by the SCALE value, and delayed by the TD value.

```
Gdel out 0 DELAY in 0 TD=5ns SCALE=2 NPDELAY=25
```

Diode Equation

To model forward-bias diode characteristics, from node 5 to ground, use a runtime expression. The saturation current is 1e-14 amp, and the thermal voltage is 0.025 v.

```
Gdio 5 0 CUR='1e-14*(EXP(V(5)/0.025)-1.0)'
```

Diode Breakdown

The following example models a diode-breakdown region, to a forward region. When voltage across the diode is above or below the piecewise linear limit values (-2.2v, 2v), the diode current remains at the corresponding limit values (-1a, 1.2a).

```
Gdiode 1 0 PWL(1) 1 0 -2.2v, -1a -2v, -1pa .3v, .15pa  
+ 6v, 10ua 1v, 1a 2v, 1.2a
```

Triode

Both of the following voltage-controlled current sources implement a basic triode.

- The first example uses the `poly(2)` operator to multiply the anode and grid voltages together, and to scale by .02.
- The second example uses the explicit behavioral algebraic description.

```
gt i_anode cathode poly(2) anode, cathode  
+ grid, cathode 0 0 0 0 .02  
  
gt i_anode cathode  
+ cur='20m*v(anode, cathode)*v(grid, cathode)'
```

Current-Dependent Voltage Sources — H Elements

This section explains H Element syntax statements, and defines their parameters.

Current-Controlled Voltage Source (CCVS)

Linear Syntax

```
Hxxx n+ n- <CCVS> vn1 transresistance <MAX=val>
+ <MIN=val> <SCALE=val> <TC1=val><TC2=val> <ABS=1>
+ <IC=val>
```

Polynomial Syntax

```
Hxxx n+ n- <CCVS> POLY(NDIM) vn1 <... vnndim>
+ <MAX=val>MIN=val> <TC1=val> <TC2=val> <SCALE=val>
+ <ABS=1> P0 <P1...> <IC=vals>
```

Piecewise Linear Syntax

```
Hxxx n+ n- <CCVS> PWL(1) vn1 <DELTA=val> <SCALE=val>
+ <TC1=val> <TC2=val> x1,y1 ... x100,y100 <IC=val>
```

Multi-Input Gate Syntax

```
Hxxx n+ n- gatetype(k) vn1, ... vnk <DELTA=val>
+ <SCALE=val> <TC1=val> <TC2=val> x1,y1 ... x100,y100
+ <IC=val>
```

Delay Element Syntax

```
Hxxx n+ n- <CCVS> DELAY vn1 TD=val <SCALE=val> <TC1=val>
+ <TC2=val> <NPDELAY=val>
```


H Element Parameters

<i>ABS</i>	Output is an absolute value, if ABS=1.
<i>CCVS</i>	Keyword for the current-controlled voltage source. CCVS is a reserved word; do not use it as a node name.
<i>DELAY</i>	Keyword for the delay element. The delay element is the same as a current-controlled voltage source, except it has an associated propagation delay, TD. Use this element to adjust the propagation delay in the macromodel process. DELAY is a reserved word; do not use it as a node name.
<i>DELTA</i>	Controls the curvature of piecewise linear corners. The default is 1/4 of the smallest breakpoint distances. The maximum is 1/2 of the smallest breakpoint distances.
<i>gatetype(k)</i>	Can be AND, NAND, OR, or NOR. (k) is the number of inputs of the gate. x's and y's are the piecewise linear variation of the output, as a function of the input. In multi-input gates, one input determines the output state.
<i>Hxxx</i>	Name of a current-controlled voltage source. Must begin with H, followed by up to 1023 alphanumeric characters.
<i>IC</i>	Initial condition (estimate) of the controlling current(s), in amps. If you do not specify IC, the default=0.0.
<i>MAX</i>	Maximum voltage value. The default is undefined, and sets no maximum value.
<i>MIN</i>	Minimum voltage value. The default is undefined, and sets no minimum value.
<i>n+/-</i>	Positive/negative connecting node, for controlled source.
<i>NDIM</i>	Polynomial dimensions. If you do not specify POLY(NDIM), Star-Hspice assumes a one-dimensional polynomial. NDIM must be a positive number.

<i>NPDELAY</i>	Sets the number of data points in delay simulations. The default value is the larger of either 10, or the smaller of $TD/tstep$ and $tstop/tstep$. That is: $NPDELAY_{\text{default}} = \max \left[\frac{\min \langle TD, tstop \rangle}{tstep}, 10 \right]$ The .TRAN statement specifies the $tstep$ and $tstop$ values.
<i>P0, P1 ...</i>	■ If you specify one polynomial coefficient, the source is linear, and Star-Hspice assumes that the polynomial is $P1$ ($P0=0.0$). ■ If you specify more than one polynomial coefficient, the source is non-linear, and Star-Hspice assumes that the polynomials are $P0, P1, P2 ...$
<i>POLY</i>	Polynomial keyword function.
<i>PWL</i>	Piecewise linear keyword function.
<i>SCALE</i>	Multiplier for the element value.
<i>TC1,TC2</i>	First-order and second-order temperature coefficients. Temperature updates the SCALE: $SCALE_{\text{eff}} = SCALE \cdot (1 + TC1 \cdot \Delta t + TC2 \cdot \Delta t^2)$
<i>TD</i>	Time delay keyword.
<i>transresistance</i>	Current-to-voltage conversion factor.
<i>vn1 ...</i>	Names of voltage sources, through which the controlling current flows. Specify one name for each dimension.
<i>x1,...</i>	Controlling current, through the $vn1$ source. Specify the x values in increasing order.
<i>y1,...</i>	Corresponding output voltage values of x .

H Element Examples

```
HX 20 10 VCUR MAX=+10 MIN=-10 1000
```

The example above selects a linear current-controlled voltage source. The controlling current flows through the dependent voltage source (named VCUR). The defining equation of the CCVS is: $HX = 1000 \cdot VCUR$

The defining equation specifies that the voltage output of HX is 1000 times the value of the current flowing through CUR.

- If the equation produces a value of $HX > +10$ V, then the MAX= parameter sets HX to 10 V.
- If the equation produces a value of $HX < -10$ V, then the MIN= parameter sets HX to -10 V.

CUR is the independent voltage source, through which the controlling current flows. If the controlling current does not flow through an independent voltage source, insert a dummy independent voltage source.

```
.PARAM CT=1000
HX 20 10 VCUR MAX=+10 MIN=-10 CT
HXY 13 20 POLY(2) VIN1 VIN2 0 0 0 0 1 IC=0.5, 1.3
```

The example above describes a dependent voltage source, with the value:

$$V = I(VIN1) \cdot I(VIN2)$$

This two-dimensional polynomial equation specifies:

- FA1=VIN1
- FA2=VIN2
- P0=0
- P1=0
- P2=0
- P3=0
- P4=1

The initial controlling current is .5 mA through VIN1, and 1.3 mA for VIN2.

Positive controlling current flows from the positive node, through the source, to the negative node of vnam (linear). The (non-linear) polynomial specifies the source voltage, as a function of the controlling current(s).

Digital and Mixed Mode Stimuli

Star-Hspice input netlists support two types of digital stimuli:

- U Element digital input files.
- Vector input files.

This section describes both types.

U Element Digital Input Elements and Models

In Star-Hspice, the U Element can reference digital input and digital output models, for mixed-mode simulation. Viewlogic's Viewsim mixed mode simulator uses Star-Hspice, with digital input from Viewsim. If you run Star-Hspice in standalone mode, the state information originates from a digital file. Digital outputs are handled in a similar fashion. In digital input file mode, the input file is named *<design>.d2a*, and the output file is named *<design>.a2d*.

A2D and D2A functions accept the terminal “\” backslash character as a line-continuation character, to allow more than 255 characters in a line. Use line continuation if the first line of a digital file, which contains the signal name list, is longer than the maximum line length that your text editor accepts.

Do not put a blank first line in a digital D2A file. If the first line of a digital file is blank, Star-Hspice issues an error message.

The following example demonstrates how to use the “\” line continuation character, to format an input file for text editing. The example file contains a signal list for a 64-bit bus.

```

...
a00 a01 a02 a03 a04 a05 a06 a07 \
a08 a09 a10 a11 a12 a13 a14 a15 \
... * Continuation of signal names
a56 a57 a58 a59 a60 a61 a62 a63 End of signal names
... Remainder of file

```

The general syntax for a U Element digital source, in a Star-Hspice netlist, is:

General Form

```
Uxxx interface nlo nhi mname SIGNAME = sname IS = val
```

The arguments are:

<i>Uxxx</i>	Digital input element name. Must begin with U, followed by up to 1023 alphanumeric characters.
<i>interface</i>	Interface node in the circuit, to which the digital input attaches.
<i>nlo</i>	Node connected to the low-level reference.
<i>nhi</i>	Node connected to the high-level reference.
<i>mname</i>	Digital input model reference (U model).
<i>SIGNAME=</i> <i>sname</i>	Signal name, as referenced in the digital output file header. Can be a string of up to eight alphanumeric characters.
<i>IS=val</i>	Initial state of the input element. Must be a state that the model defines.

Model Syntax

```
.MODEL mname U LEVEL=5 <parameters...>
```

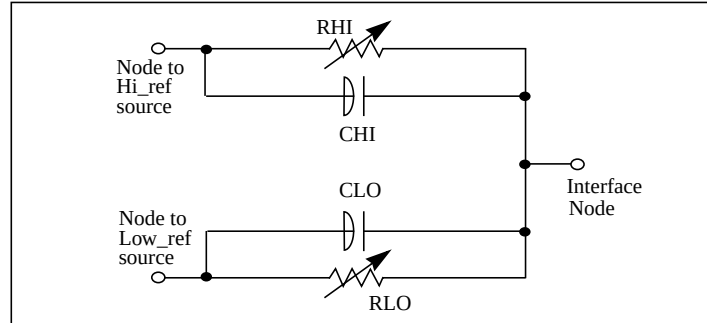
Digital input.

Digital-to-Analog Input Model Parameters

Names (Alias)	Units	Default	Description
CLO	farad	0	Capacitance, to low-level node.
CHI	farad	0	Capacitance, to high-level node.
S0NAME			State 0 character abbreviation. A string of up to four alphanumerical characters.
S0TSW	sec		State 0 switching time.
S0RLO	ohm		State 0 resistance, to low-level node.
S0RHI	ohm		State 0 resistance, to high-level node.
S1NAME			State 1 character abbreviation. A string of up to four alphanumerical characters.
S1TSW	sec		State 1 switching time.
S1RLO	ohm		State 1 resistance, to low-level node.
S1RHI	ohm		State 1 resistance, to high-level node.
S19NAME			State 19 character abbreviation. A string of up to four alphanumerical characters.
S19TSW	sec		State 19 switching time.
S19RLO	ohm		State 19 resistance, to low-level node.
S19RHI	ohm		State 19 resistance, to high-level node.
TIMESTEP	sec		Step size, for digital input files only.

To define up to 20 different states in the model definition, use the S_n NAME, S_n TSW, S_n RLO and S_n RHI parameters, where n ranges from 0 to 19.

[Figure 5-7 on page 5-59](#) shows the circuit representation of the element.

Figure 5-7: Digital-to-Analog Converter Element

Example

The following example shows how to use the U Element and model, as a digital input for a Star-Hspice netlist.

```
* EXAMPLE OF U-ELEMENT DIGITAL INPUT
UC carry-in VLD2A VHD2A D2A SIGNAME=1 IS=0
VLO VLD2A GND DC 0
VHI VHD2A GND DC 1
.MODEL D2A U LEVEL=5 TIMESTEP=1NS,
+ S0NAME=0 S0TSW=1NS S0RLO = 15, S0RHI = 10K,
+ S2NAME=x S2TSW=3NS S2RLO = 1K, S2RHI = 1K
+ S3NAME=z S3TSW=5NS S3RLO = 1MEG, S3RHI = 1MEG
+ S4NAME=1 S4TSW=1NS S4RLO = 10K, S4RHI = 60
.PRINT V(carry-in)
.TRAN 1N 100N
.END
```

where the associated digital input file is:

```
1
00 1:1
09 z:1
10 0:1
11 z:1
20 1:1
30 0:1
39 x:1
40 1:1
41 x:1
50 0:1
60 1:1
70 0:1
80 1:1
```

U Element Digital Outputs

The general syntax for a digital output in a Star-Hspice output is:

General Form

```
U<name> interface reference mname SIGNAME = sname
```

<i>Uxxx</i>	Digital output element name. Must begin with U, followed by up to 1023 alphanumeric characters.
<i>interface</i>	Interface node in the circuit, at which Star-Hspice measures the digital output.
<i>reference</i>	Node to use as a reference for the output.
<i>mname</i>	Digital output model reference (U model).
<i>SIGNAME=</i> <i>sname</i>	Signal name, as referenced in the digital output file header. A string of up to eight alphanumeric characters.

Model Syntax

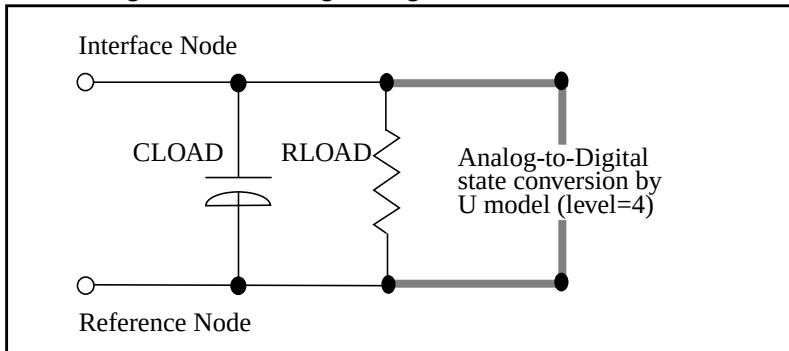
```
.MODEL mname U LEVEL=4 <parameters...>
```

Digital output.

Analog-to-Digital Output Model Parameters

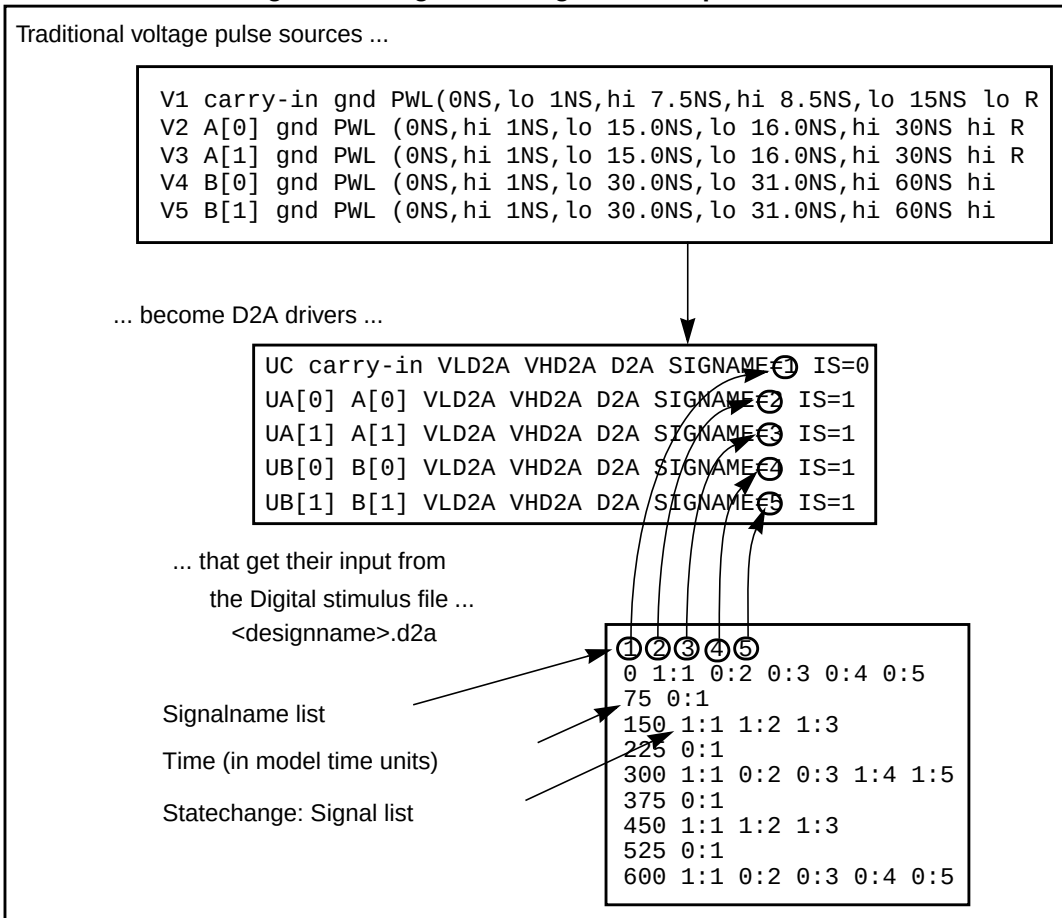
Name (Alias)	Units	Default	Description
RLOAD	ohm	1/gmin	Output resistance.
CLOAD	farad	0	Output capacitance.
S0NAME			State 0 character abbreviation. A string of up to four alphanumerical characters.
S0VLO	volt		State 0 low-level voltage.
S0VHI	volt		State 0 high-level voltage.
S1NAME			State 1 character abbreviation. A string of up to four alphanumerical characters.
S1VLO	volt		State 1 low-level voltage.
S1VHI	volt		State 1 high-level voltage.
S19NAME			State 19 character abbreviation. A string of up to four alphanumerical characters.
S19VLO	volt		State 19 low-level voltage.
S19VHI	volt		State 19 high-level voltage.
TIMESTEP	sec	1E-9	Step size for digital input file.
TIMESCALE			Scale factor, for time.

To define up to 20 different states in the model definition, use the S_n NAME, S_n VLO and S_n VHI parameters, where n ranges from 0 to 19. [Figure 5-8 on page 5-62](#) shows the circuit representation of the element.

Figure 5-8: Analog-to-Digital Converter Element

Replacing Sources With Digital Inputs

Figure 5-9: Digital File Signal Correspondence



The following is an example of replacing sources with digital inputs.

```
* EXAMPLE OF U-ELEMENT DIGITAL OUTPUT
VOUT carry_out GND PWL 0N 0V 10N 0V 11N 5V 19N 5V 20N 0V
+ 30N 0V 31N 5V 39N 5V 40N 0V

VREF REF GND DC 0.0V
UCO carry-out REF A2D SIGNAME=12
* DEFAULT DIGITAL OUTPUT MODEL (no "X" value)

.MODEL A2D U LEVEL=4 TIMESTEP=0.1NS TIMESCALE=1
+ S0NAME=0 S0VLO=-1 S0VHI= 2.7
+ S4NAME=1 S4VLO= 1.4 S4VHI=9.0
+ CLOAD=0.05pf

.TRAN 1N 50N
.END
```

and the digital output file should look like:

```
12
0      0:1
105    1:1
197    0:1
305    1:1
397    0:1
```

where:

- 12 represents the signal name
- The first column is the time, in units of 0.1 nanoseconds.
- The second column has the signal *value:name* pairs.
- This file uses more columns to represent subsequent outputs.

The following two-bit MOS adder uses the digital input file. In the plot below, the 'A[0], A[1], B[0], B[1], and CARRY-IN' nodes all originate from a digital file input (see [Figure 5-9 on page 5-63](#)). Star-Hspice outputs a digital file.

```
FILE: MOS2BIT.SP - ADDER - 2 BIT ALL-NAND-GATE
+ BINARY ADDER
*
.OPTION ACCT NOMOD FAST scale=1u gmindc=100n post
.param lmin=1.25 hi=2.8v lo=.4v vdd=4.5
.global vdd
*
.TRAN .5NS 60NS
.MEAS PROP-DELAY TRIG V(carry-in) TD=10NS VAL='vdd*.5'
+ RISE=1 TARG V(c[1]) TD=10NS VAL='vdd*.5' RISE=3
*
```

```

.MEAS PULSE-WIDTH TRIG V(carry-out_1) VAL='vdd*.5'
+ RISE=1 TARG V(carry-out_1) VAL='vdd*.5' FALL=1
*
.MEAS FALL-TIME TRIG V(c[1]) TD=32NS VAL='vdd*.9'
+ FALL=1 TARG V(c[1]) TD=32NS VAL='vdd*.1' FALL=1
*
VDD vdd gnd DC vdd
X1 A[0] B[0] carry-in C[0] carry-out_1 ONEBIT
X2 A[1] B[1] carry-out_1 C[1] carry-out_2 ONEBIT
*

* Subcircuit Definitions
.subckt NAND in1 in2 out wp=10 wn=5
    M1 out in1 vdd vdd P W=wp L=lmin ad=0
    M2 out in2 vdd vdd P W=wp L=lmin ad=0
    M3 out in1 mid gnd N W=wn L=lmin as=0
    M4 mid in2 gnd gnd N W=wn L=lmin ad=0
    CLOAD out gnd 'wp*5.7f'
.ends
*

.subckt ONEBIT in1 in2 carry-in out carry-out
    X1 in1 in2 #1_nand NAND
    X2 in1 #1_nand 8 NAND
    X3 in2 #1_nand 9 NAND
    X4 8 9 10 NAND
    X5 carry-in 10 half1 NAND
    X6 carry-in half1 half2 NAND
    X7 10 half1 13 NAND
    X8 half2 13 out NAND
    X9 half1 #1_nand carry-out NAND
.ENDS ONEBIT
*

* Stimulus
UC carry-in VLD2A VHD2A D2A SIGNAME=1 IS=0
UA[0] A[0] VLD2A VHD2A D2A SIGNAME=2 IS=1
UA[1] A[1] VLD2A VHD2A D2A SIGNAME=3 IS=1
UB[0] B[0] VLD2A VHD2A D2A SIGNAME=4 IS=1
UB[1] B[1] VLD2A VHD2A D2A SIGNAME=5 IS=1
*

uc0 c[0] vrefa2d a2d signame=10
uc1 c[1] vrefa2d a2d signame=11
uco carry-out_2 vrefa2d a2d signame=12
uci carry-in vrefa2d a2d signame=13
*

```

```

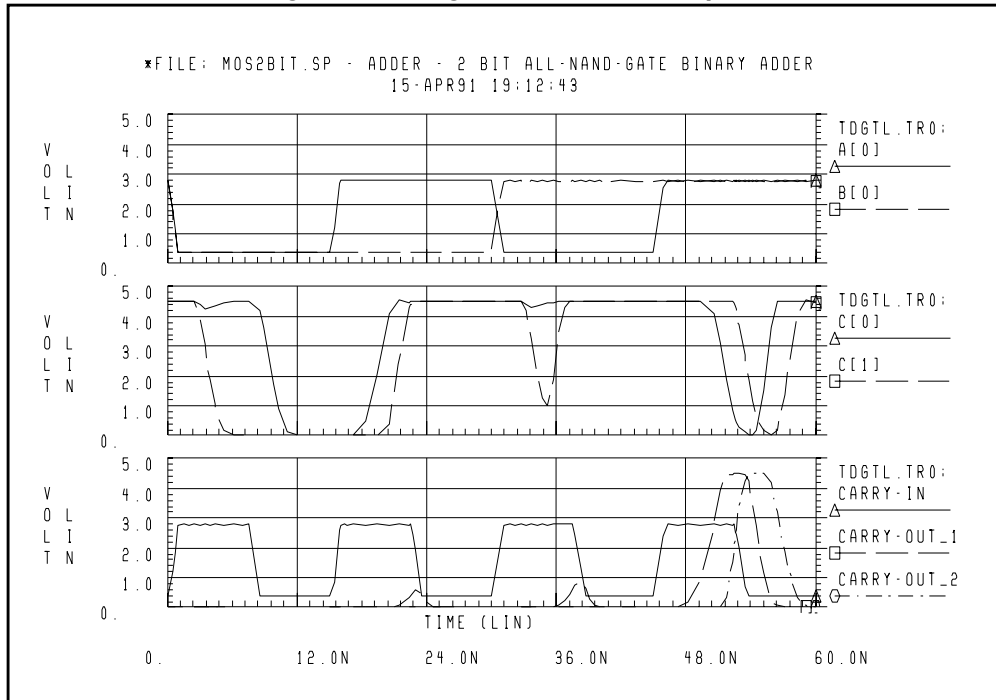
* Models
.MODEL N NMOS LEVEL=3 VT0=0.7 U0=500 KAPPA=.25 KP=30U
+ ETA=.01 THETA=.04 VMAX=2E5 NSUB=9E16 TOX=400
+ GAMMA=1.5 PB=0.6 JS=.1M XJ=0.5U LD=0.1U NFS=1E11
+ NSS=2E10 RSH=80 CJ=.3M MJ=0.5 CJSW=.1N MJSW=0.3
+ acm=2 capop=4
*

.MODEL P PMOS LEVEL=3 VT0=-0.8 U0=150 KAPPA=.25 KP=15U
+ ETA=.015 THETA=.04 VMAX=5E4 NSUB=1.8E16 TOX=400
+ GAMMA=.672 PB=0.6 JS=.1M XJ=0.5U LD=0.15U NFS=1E11
+ NSS=2E10 RSH=80 CJ=.3M MJ=0.5 CJSW=.1N MJSW=0.3
+ acm=2 capop=4
*

* Default Digital Input Interface Model
.MODEL D2A U LEVEL=5 TIMESTEP=0.1NS,
+ S0NAME=0 S0TSW=1NS S0RLO = 15, S0RHI = 10K,
+ S2NAME=x S2TSW=5NS S2RLO = 1K, S2RHI = 1K
+ S3NAME=z S3TSW=5NS S3RLO = 1MEG, S3RHI = 1MEG
+ S4NAME=1 S4TSW=1NS S4RLO = 10K, S4RHI = 60
VLD2A VLD2A 0 DC lo
VHD2A VHD2A 0 DC hi
*

* Default Digital Output Model (no "X" value)
.MODEL A2D U LEVEL=4 TIMESTEP=0.1NS TIMESCALE=1
+ S0NAME=0 S0VLO=-1 S0VHI= 2.7
+ S4NAME=1 S4VLO= 1.4 S4VHI=6.0
+ CLOAD=0.05pf
VREFA2D VREFA2D 0 DC 0.0V
.END

```

Figure 5-10: Digital Stimulus File Input

Specifying a Digital Vector File

The digital vector file consists of three parts:

- *Vector Pattern Definition* section
- *Waveform Characteristics* section
- *Tabular Data* section.

You can use a digital vector file in Star-Hspice.

To incorporate this information into your simulation, include this line in your netlist:

```
.VEC 'digital_vector_file'
```

Defining Vector Patterns

The *Vector Pattern Definition* section defines the vectors: their names, sizes, signal direction, sequence or order for each vector stimulus, and so on. It must occur first in the digital vector file. The statements within this section (except the *radix* statement) can appear in any order, and all keywords are case-insensitive.

A sample Vector Pattern Definition section follows:

```
radix 1111 1111
vname a b c d e f g h
io iiii iiii
tunit ns
```

For an explanation of keywords, such as *radix* and *vname*, see [Defining Tabular Data on page 5-73](#).

Radix Statement

The *radix* statement specifies the number of bits associated with each vector. Valid values for the number of bits range from 1 to 4.

# bits	Radix	Number System	Valid Digits
1	2	Binary	0, 1
2	4	–	0 – 3
3	8	Octal	0 – 7
4	16	Hexadecimal	0 – F

Only one *radix* statement must appear in the file, and it must be the first non-comment line.

Example

This example illustrates two 1-bit signals, followed by a 4-bit signal, followed by one each 1-bit, 2-bit, 3-bit, and 4-bit signals, and finally eight 1-bit signals.

```
; start of vector pattern definition section
radix 1 1 4 1234 1111 1111
```

Vname Statement

The *vname* statement defines the name of each vector. If you do not specify *vname*, Star-Hspice assigns a default name to each signal: V1, V2, V3, and so on. If you define more than one *vname* statement, the last statement overrides the previous statement.

```
radix 1 1 1 1 1 1 1 1 1 1 1 1 1
vname V1 V2 V3 V4 V5 V6 V7 V8 V9 V10 V11 V12
```

To provide the range of the bit indices, use square brackets [] and a colon:

```
[starting_index : ending_index]
```

The *vname* name is required for each bit. You can associate a single name with multiple bits (*such as bus notation*).

The bit order is MSB:LSB. You can also nest this bus notation syntax inside other grouping symbols, such as <>, (), [], and so on. The name of each bit is *vname*, followed by the index suffix (appended).

Example 1

If you specify:

```
radix 2 4
vname VA[0:1] VB[4:1]
```

Star-Hspice generates voltage sources with the following names:

```
VA0 VA1 VB4 VB3 VB2 VB1
```

where:

- VA0 and VB4 are the MSBs.
- VA1 and VB1 are the LSBs.

Example 2

If you specify:

```
vname VA[[0:1]] VB<[4:1]>
```

Star-Hspice generates voltage sources with the following names:

```
VA[0] VA[1] VB<4> VB<3> VB<2> VB<1>
```

Example 3

This example shows how to specify a single bit of a bus:

```
vname VA[[2:2]]
```

Example 4

This example generates signals named A0, A1, A2, ... A23:

```
radix 444444
vname A[0:23]
```

IO Statement

The *io* statement defines the type, for each vector. The line starts with the *io* keyword, followed by a string of *i*, *b*, *o*, or *u* definitions. These definitions indicate whether each corresponding vector is an input (*i*), bidirectional (*b*), output (*o*), or unused (*u*) vector.

- i* Input, which Star-Hspice uses to stimulate the circuit.
- o* Expected output, which Star-Hspice compares with the simulated outputs.
- b* Star-Hspice ignores.

Example

- If you do not specify the *io* statement, Star-Hspice assumes that all signals are input signals.
- If you define more than one *io* statement, the last statement overrules previous statements.

```
io i i i i bbbb iiiiioouu
```

Tunit Statement

The *tunit* statement defines the time unit in the digital vector file for *period*, *tdelay*, *slope*, *trise*, *tfall*, and *absolute time*. It must be one of the following:

fs	femto-second
ps	pico-second
ns	nano-second
us	micro-second
ms	milli-second

- If you do not specify the *tunit* statement, the default time unit value is **ns**.
- If you define more than one *tunit* statement, the last statement overrules the previous statement.

Example

The *tunit* statement in this example specifies that the absolute times in the *tabular data* section are 11.0ns, 20.0ns, and 33.0ns.

```
tunit ns
11.0 1000 1000
20.0 1100 1100
33.0 1010 1001
```

Period and Tskip Statements

The *period* statement defines the time interval for the *tabular data* section. You do not need to specify the absolute time at every time point. If you use a *period* statement, without the *tskip* statement, then the *tabular data* section contains only signal values, not absolute times. The *tunit* statement defines the time unit of the *period*.

Example

In this example:

- The first row of the tabular data (1000 1000) is at time 0ns.
- The second row (1100 1100) is at 10ns.
- The third row (1010 1001) is at 20ns.

```
radix 1111 1111
period 10
1000 1000
1100 1100
1010 1001
```

The *tskip* statement specifies to ignore the absolute time field in the tabular data. You can then keep, but ignore, the absolute time field of each row in the tabular data, when you use the *period* statement.

Example

If your netlist contains:

```
radix 1111 1111
period 10
tskip
11.0 1000 1000
20.0 1100 1100
33.0 1010 1001
```

then Star-Hspice ignores the absolute times 11.0, 20.0 and 33.0.

Enable Statement

The *enable* statement specifies the controlling signal(s) for bidirectional signals. All bidirectional signals require an *enable* statement. If you specify more than one *enable* statement, the last statement overrides the previous statement, and Star-Hspice issues a warning message.

The syntax is the *enable* keyword, followed by the controlling signal name, and the mask that defines the (bidirectional) signals to which *enable* applies.

The controlling signal, for bidirectional signals, must be an input signal, with a radix of 1. The bidirectional signals become output when the controlling signal is at state 1 (or high). To reverse this default control logic, start the control signal name with a tilde (~).

Example

In this example, the *x* and *y* signals are bidirectional, as defined by the *b* in the *io* line.

- The first enable statement indicates that *x* (as defined by the position of *F*) becomes output, when the *a* signal is 1.
- The second enable specifies that the *y* bidirectional bus becomes output, when the *a* signal is 0.

```
radix 144
io ibb
vname a x[3:0] y[3:0]
enable a 0 F 0
enable ~a 0 0 F
```

Defining Tabular Data

Although this section generally appears last in a digital vector file (after the *Vector Pattern* and *Waveform Characteristics* definitions), this chapter describes it first, to introduce the definitions of a *vector*.

The Tabular Data section defines (in *tabular* format) the values of the signals, at specified times. Its general format is:

```
time1 signal1_value1 signal2_value1 signal3_value1...
time2 signal1_value2 signal2_value2 signal3_value2...
time3 signal1_value3 signal2_value3 signal3_value3...
.
.
```

The set of values for a particular signal, over all times, is a *vector*, which appears as a vertical column in the tabular data and vector table. The set of all *signal1_valuex* constitute one vector. Signal values can have any of the legal states, described in the next section.

Rows in the tabular data section must appear in chronological order, because row placement carries sequential timing information.

Example

```
10.0 1000 0000
15.0 1100 1100
20.0 1010 1001
30.0 1001 1111
```

This example feature eight signals, and therefore eight vectors. The first signal (starting from the left) has a vector [1 1 1 1]; the second has a vector [0 1 0 0]; and so on.

Input Stimuli

Star-Hspice converts each input signal into a PWL (piecewise linear) voltage source, and a series resistance. The legal states for an input signal are:

0	Drive to ZERO (gnd).
1	Drive to ONE (vdd).
Z, z	Floating to HIGH IMPEDANCE.
X, x	Drive to ZERO (gnd).
L	Resistive drive to ZERO (gnd).
H	Resistive drive to ONE (vdd).
U, u	Drive to ZERO (gnd).

- For the 0, 1, X, x, U, and u states, Star-Hspice sets resistance to zero.
- For the L and H states, the *out* (or *outz*) statement defines the resistance value.
- For the Z and z states, the *triz* statement defines the resistance value.

Expected Output

Star-Hspice converts each output signal into a *.DOUT* statement, in the netlist. During simulation, Star-Hspice compares the actual results, with the expected output vector(s). If the states are different, an error message appears. The legal states for expected outputs include:

0	Expect ZERO.
1	Expect ONE.
X, x	Don't care.
U, u	Don't care.
Z, z	Expect HIGH IMPEDANCE (don't care).

Note: Simulation evaluates Z, z as *don't care*, because Star-Hspice cannot detect a high impedance state.

Example

An example of usage follows:

```

...
; start of tabular section data
11.0 1 0 0 1
20.0 1 1 0 0
30.0 1 0 0 0
35.0 x x 0 0

```

Verilog Value Format

Star-Hspice also accepts Verilog *sized* format, for specifying numbers:

```
<size> ' <base format> <number>
```

where:

- <size> specifies (in decimal) the number of bits
- <base format> indicates:
 - binary ('b or 'B)
 - octal ('o or 'O)
 - hexadecimal ('h or 'H).
- Valid <number> fields are combinations of the characters 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F. Depending on the <base format> chosen, only a subset of these characters might be legal.

You can also use unknown values (X) and high impedance (Z) in the <number> field. An X or Z sets four bits in the hexadecimal base, three bits in the octal base, or one bit in the binary base.

If the most significant bit of a number is 0, X, or Z, Star-Hspice automatically extends the number (if necessary), to fill the remaining bits with 0, X, or Z. If the most significant bit is 1, Star-Hspice uses 0 to extend it.

Example

```
4'b1111
12'hABx
32'bZ
8'h1
```

This example specifies values for:

- a 4-bit signal in binary
- a 12-bit signal in hexadecimal
- a 32-bit signal in binary
- an 8-bit signal in hexadecimal

Equivalents of these lines, in non-Verilog format, are:

```
1111
AB xxxx
ZZZZ ZZZZ ZZZZ ZZZZ ZZZZ ZZZZ ZZZZ ZZZZ
1000 0000
```


Periodic Tabular Data

Tabular data is often periodic, so you do not need to specify the absolute time at every time point. When you specify the *period* statement, the *tabular data* section omits the absolute times (see [Using Tabular Data on page 5-77](#)).

Example

```
radix 1111 1111
vname a b c d e f g h
io iiii iiii
tunit ns
period 10
; start of vector data section
1000 1000
1100 1100
1010 1001
```

Using Tabular Data

The *Tabular Data* section defines the values of the input signals, at specified times. The first column lists the time, followed by signal values in the subsequent columns, in the order specified in the *vname* statement.

An example of tabular data follows:

```
11.0 1000 1000
20.0 1100 1100
33.0 1010 1001
```

Defining Waveform Characteristics

The *Waveform Characteristics* section defines various attributes for signals, such as the rise or fall time, thresholds for logic ‘high’ or ‘low’, and so on. A sample Waveform Characteristics section follows:

```
trise 0.3 137F 0000
tfall 0.5 137F 0000
vih 5.0 137F 0000
vil 0.0 137F 0000
```

Modifying Waveform Characteristics

This section describes how to modify waveform characteristics of your circuit.

Tdelay, Idelay, and Odelay Statements

The *tdelay*, *idelay* and *odelay* statements define the delay time of the signal, relative to the absolute time of each row in the *tabular data* section.

- *idelay* applies to the input signals.
- *odelay* applies to the output signals.
- *tdelay* applies to both input and output signals.

The statement starts with a keyword (*tdelay*, *idelay*, or *odelay*), followed by a delay value, and then a *mask*. The mask defines the signals to which the delay applies. If you do not provide a mask, the delay value applies to all signals.

The *tunit* statement defines the time unit of *tdelay*, *idelay* and *odelay*. Normally, you need to use only the *tdelay* statement; use the *idelay* and *odelay* statements only to specify different input and output delay times, for bidirectional signals. Star-Hspice ignores *idelay* settings on output signals (or *odelay* settings on input signals), and issues a warning message.

You can specify more than one *tdelay*, *idelay*, or *odelay* statement.

- If you apply more than one *tdelay* (*idelay*, *odelay*) statement to a signal, the last statement overrides the previous statements, and Star-Hspice issues a warning.
- If you do not specify the signal delays in a *tdelay*, *idelay*, or *odelay* statement, Star-Hspice defaults to zero.

Example

The first *tdelay* statement indicates that all signals have the same delay time, 1.0. Subsequent *tdelay* statements override the delay time of some signals.

- The delay time for the V2 and Vx signals is -1.2.
- The delay time for the V4, V5[0:1], and V6[0:2] signals is 1.5.

- The `V7[0:3]` signals have an input delay time of 2.0, and an output delay time of 3.0.

```
radix 1 1 4 1234 11111111
io i i o iiib iiiiii
vname V1 V2 VX[3:0] V4 V5[1:0] V6[0:2] V7[0:3]
+V8 V9 V10 V11 V12 V13 V14 V15
tdelay 1.0
tdelay -1.2 0 1 1 0000 00000000
tdelay 1.5 0 0 0 1370 00000000
idelay 2.0 0 0 0 000F 00000000
odelay 3.0 0 0 0 000F 00000000
```

Slope Statement

- The *slope* statement specifies the fall times for the input signal.
- The *tunit* statement defines the time unit.

To specify the signals to which the *slope* applies, use a mask.

- If you do not specify the *slope* statement, the default slope value is 0.1 ns.
- If you specify more than one *slope* statement, the last statement overrules the previous statements, and Star-Hspice issues a warning message.

The *slope* statement has no effect on the expected output signals. You can specify the optional *trise* and *tfall* statements, to overrule the rise time and fall time of a signal.

Example

- In the first example, the rising and falling times of all signals are 1.2 ns.
- The second example specifies a rising/falling time of 1.1 ns for the first, second, sixth, and seventh signals.

```
slope 1.2
slope 1.1 1100 0110
```

Trise Statement

The *trise* statement specifies the rise time of each input signal (for which the mask applies). The *tunit* statement defines the time unit for *trise*.

Example

- If you do not use any *trise* statement to specify the rising time of the signals, Star-Hspice uses the value defined in the *slope* statement.
- If you apply more than one *trise* statement to a signal, the last statement overrides the previous statements, and Star-Hspice issues a warning message.

```
trise 0.3
trise 0.5 0 1 1 137F 00000000
trise 0.8 0 0 0 0000 11110000
```

The *trise* statements have no effect on the expected output signals.

Tfall Statement

The *tfall* statement specifies the falling time for each input signal (for which the mask applies). The *tunit* statement defines the time unit of *tfall*.

Example

- If you do not specify the falling time of the signals in a *tfall* statement, Star-Hspice uses the value defined in the *slope* statement.
- If you specify more than one *tfall* statement for a signal, the last statement overrides the previous statements. Star-Hspice issues a warning message.

```
tfall 0.5
tfall 0.3 0 1 1 137F 00000000
tfall 0.9 0 0 0 0000 11110000
```

The *tfall* statements have no effect on the expected output signals.

Out /Outz Statements

The *out* and *outz* keywords are equivalent, and specify output resistance for each signal (for which the mask applies); *out* (or *outz*) applies only to input signals.

Example

- If you do not specify the output resistance of a signal, in an *out* (or *outz*) statement, Star-Hspice uses the default (zero).
- If you specify more than one *out* (or *outz*) statement for a signal, the last statement overrules the previous statements, and Star-Hspice issues a warning message.

```
out 15.1
out 150 1 1 1 0000 00000000
outz 50.5 0 0 0 137F 00000000
```

The *out* (or *outz*) statements have no effect on the expected output signals.

Triz Statement

The *triz* statement specifies the output impedance, when the signal (for which the mask applies) is in *tristate*; *triz* applies only to the input signals.

Example

- If you do not specify the *tristate* impedance of a signal, in a *triz* statement, Star-Hspice assumes 1000M.
- If you apply more than one *triz* statement to a signal, the last statement overrules the previous statements, and Star-Hspice issues a warning.

```
triz 15.1M
triz 150M 1 1 1 0000 00000000
triz 50.5M 0 0 0 137F 00000000
```

The *triz* statements have no effect on the expected output signals.

Vih Statement

The *vih* statement specifies the logic high voltage, for each input signal to which the mask applies.

Example

- If you do not specify the logic high voltage of the signals, in a *vih* statement, Star-Hspice assumes 3.3.
- If you apply more than one *vih* statement to a signal, the last statement overrides the previous statements, and Star-Hspice issues a warning.

```

vih 5.0
vih 5.0 1 1 1 137F 00000000
vih 3.5 0 0 0 0000 11111111

```

The *vih* statements have no effect on the expected output signals.

Vil Statement

The *vil* statement specifies the logic-low voltage, for each input signal to which the mask applies.

Example

- If you do not specify the logic-low voltage of the signals, in a *vil* statement, Star-Hspice assumes 0.0.
- If you apply more than one *vil* statement to a signal, the last statement overrides the previous statements, and Star-Hspice issues a warning.

```

vil 0.0
vil 0.0 1 1 1 137F 11111111

```

The *vil* statements have no effect on the expected output signals.

Vref Statement

Similar to the *tdelay* statement, the *vref* statement specifies the name of the reference voltage, for each input vector to which the mask applies. *vref* applies only to input signals.

Example

If your netlist contains:

```

vname v1 v2 v3 v4 v5[1:0] v6[2:0] v7[0:3] v8 v9 v10
vref 0
vref 0 111 137F 000
vref vss 0 0 0 0000 111

```

when Star-Hspice implements it into the netlist, the voltage source realizes *v1*:

```
v1 V1 0 pwl( . . . . . )
```

as do *v2*, *v3*, *v4*, *v5*, *v6*, and *v7*.

However, *v8* is realized by

```
V8 V8 vss pwl( . . . . . )
```

as is *v9* and *v10*.

- If you do not specify the reference voltage name of the signals, in a *vref* statement, Star-Hspice assumes 0.
- If you apply more than one *vref* statement, the last statement overrules the previous statements, and Star-Hspice issues a warning.

The *vref* statements have no effect on the output signals.

Vth Statement

Similar to the *tdelay* statement, the *vth* statement specifies the logic threshold voltage, for each signal to which the mask applies. *vth* applies only to output signals. The threshold voltage determines the logic state of output signals, for comparison with the expected output signals.

Example

- If you do not specify the threshold voltage of the signals, in a *vth* statement, Star-Hspice assumes 1.65.
- If you apply more than one *vth* statement to a signal, the last statement overrules the previous statements, and Star-Hspice issues a warning.

```
vth 1.75
vth 2.5 1 1 1 137F 00000000
vth 1.75 0 0 0 0000 11111111
```

The *vth* statements have no effect on the input signals.

Voh Statement

The *voh* statement specifies the logic-high voltage of each output signal to which the mask applies.

Example

- If you do not specify the logic-high voltage, in a *voh* statement, Star-Hspice assumes 3.3.
- If you apply more than one *voh* statement to a signal, the last statement overrides the previous statements, and Star-Hspice issues a warning.

```
voh 4.75
voh 4.5 1 1 1 137F 00000000
voh 3.5 0 0 0 0000 11111111
```

The *voh* statements have no effect on input signals.

Note: If you do not define either *voh* or *vol*, Star-Hspice uses *vth* (default or defined).

Vol Statement

The *vol* statement specifies the logic-low voltage, for each output signal to which the mask applies.

Example

- If you do not specify the logic-low voltage, in a *vol* statement, Star-Hspice assumes 0.0.
- If you apply more than one *vol* statements to a signal, the last statement overrides the previous statements, and Star-Hspice issues a warning.

```
vol 0.5
vol 0.5 1 1 1 137F 11111111
```

The *vol* statements have no effect on input signals.

Note: If you do not define either *voh* or *vol*, Star-Hspice uses *vth* (default or defined).

Comment Lines

Any line that starts with a semi-colon (;) is a comment line. Comments can also start at any point on a line. Star-Hspice ignores characters after a semi-colon.

An example of usage follows:

```
; This is a comment line
radix 1 1 4 1234 ; This is a radix line
```

Continuing a Line

As in netlists, any line that starts with a plus sign (+) is a continuation from the previous line.

Digital Vector File Example

An example of a vector pattern definition follows:

```
; specifies # of bits associated with each vector
radix 1 2 444
; *****
; defines name for each vector. For multi-bit
; vectors, innermost [] provide the bit index range,
; MSB:LSB
vname v1 va[[1:0]] vb[12:1]
; actual signal names: v1, va[0], va[1], vb1 ... vb12
; *****
; defines vector as input, output, or bi-direc
io i o bbb
; defines time unit
tunit ns
; *****
; vb12-vb5 are output when 'v1' is 'high'
enable v1 0 0 FF0
; vb4-vb1 are output when 'v1' is 'low'
enable ~v1 0 0 00F
; *****
; all signals have delay of 1 ns
; Note: do not put unit (e.g., ns) again here because
; this value will be multiplied by the unit specified
; in the 'tunit' line.
tdelay 1.0
; signals va1 and va0 have delays of 1.5ns
tdelay 1.5 0 3 000
```

```

*****
; specify input rise and fall times (if you want
; different rise and fall times, use trise/
; tfallstmt.)
; Note: do not put unit (e.g., ns) again here because
; this value will be multiplied by the unit specified
; in the 'tunit' line.
slope 1.2
*****
; specify the logic 'high' voltage for input signals
vih 3.3 1 0 000
vih 5.0 0 0 FFF
; likewise, may specify logic 'low' with 'vil'
*****
; va & vb switch from 'lo' to 'hi' at 1.75 volts
vth 1.75 0 1 FFF
*****
; tabular data section
10.0 1 3 FFF
20.0 0 2 AFF
30.0 1 0 888
.
.
.

```



Chapter 6

Multi-Terminal Networks

You can use the S Element to describe a multi-terminal network, in AC and DC analyses of a circuit, in either Star-Hspice. This chapter explains various topics related to the S Element:

- [Using Scattering Parameter Element](#)
- [Frequency Table Model](#)

Using Scattering Parameter Element

The S Element, in conjunction with the generic frequency-domain model (.MODEL SP), provides a convenient way to describe a multi-terminal network. Currently, this element supports S (scattering) and Y parameters. You can use the S Element in AC and DC analyses.

In particular, the S parameter in the S Element represents the generalized scattering parameter (**S**) for a multi-terminal network, which is defined as:

$$\mathbf{v}_{\text{ref}} = \mathbf{S} \cdot \mathbf{v}_{\text{inc}} .$$

where:

- Boldface lower-case symbols denote vectors.
- Boldface upper-case symbols denote matrices.
- $\mathbf{v}_{\text{inc}}$ is the incident voltage wave vector.
- $\mathbf{v}_{\text{ref}}$ is the reflected voltage wave vector (see [Figure 6-1 on page 6-3](#)).

To convert the S parameter to the Y parameter, use the following formula:

$$\mathbf{Y} = \mathbf{Y}_{\text{rs}}(\mathbf{I} - \mathbf{S})(\mathbf{I} + \mathbf{S})^{-1}\mathbf{Y}_{\text{rs}} .$$

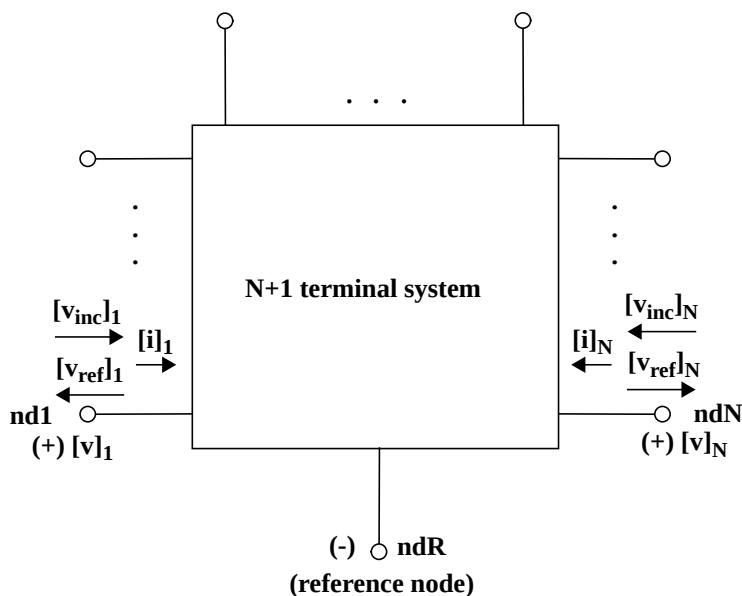
where $\mathbf{Y}_{\text{r}}$ is the characteristic admittance matrix of the reference system. The following formula relates $\mathbf{Y}_{\text{r}}$ to the $\mathbf{Z}_{\text{r}}$ characteristic impedance matrix:

$$\mathbf{Y}_{\text{r}} = \mathbf{Z}_{\text{r}}^{-1}, \mathbf{Y}_{\text{rs}}\mathbf{Y}_{\text{rs}} = \mathbf{Y}_{\text{r}}, \mathbf{Z}_{\text{rs}}\mathbf{Z}_{\text{rs}} = \mathbf{Z}_{\text{r}} .$$

Similarly, you can convert the Y parameter to the S parameter, as follows:

$$\mathbf{S} = (\mathbf{I} + \mathbf{Z}_{\text{rs}}\mathbf{Y}\mathbf{Z}_{\text{rs}})(\mathbf{I} - \mathbf{Z}_{\text{rs}}\mathbf{Y}\mathbf{Z}_{\text{rs}})^{-1} .$$

Figure 6-1: Terminal Node Notation



Syntax

The syntax of the S Element is:

```
Sxxx nd1 nd2 ... ndN ndR FQMODEL=name [TYPE=val Zo=val
Zof=name]
```

nd1 nd2 ... ndN

N signal nodes (see [Figure 6-1](#)).

ndR

Reference node.

FQMODEL

.MODEL statement of sp type, which defines the frequency behavior of the S or Y parameter.

TYPE

Parameter type:

- S: scattering parameter (default).
- Y: Y parameter.

Zo	Characteristic impedance value, for the reference line (frequency-independent). For multi-terminal cases ($N > 1$), Star-Hspice that the characteristic impedance matrix for the reference lines is diagonal, and that its values are set to Zo. To specify more general types of a reference line system, use Zof. Default=50 Ω .
Zof	Name of the frequency-varying model, which defines the frequency behavior of the reference system. If you define both Zo and Zof, then Zof has precedence.

Frequency Table Model

The Frequency Table Model is a generic model, which describes frequency-varying behavior. Currently, the S Element and .NOISENPT use this model.

Syntax

The syntax of the .MODEL model card for the S Element is:

```
.MODEL name sp [N=val FSTART=val FSTOP=val NI=val SPACING=val
+ MATRIX=val VALTYPE=val INFINITY=matrixval INTERPOLATION=val
+ EXTRAPOLATION=val] [DATA=(npts ...)] [DATAFILE=filename]
```

Name	Model name
N	Matrix dimension (number of signal terminals). Specify values other than 1, before you set INFINITY and DATA. Default=1.
FSTART	Starting frequency point for data. Default=0.
FSTOP	Final frequency point for data. Use this only for the LINEAR and LOG spacing formats.
NI	Number of frequency points per interval. Use this only for the DEC and OCT spacing formats. Default=10.
npts	Number of data points.
SPACING	Data sample spacing format: ■ LIN (LINEAR): uniform spacing, with the frequency step of (FSTOP-FSTART)/(npts-1). Default. ■ OCT: octave variation, with FSTART as the starting frequency, and NI points per octave. npts determines the final frequency. ■ DEC: decade variation, with FSTART as the starting frequency, and NI points per decade. npts determines the final frequency.

- LOG: logarithmic spacing, with FSTART and FSTOP as the starting and final frequencies.
- POI (NONUNIFORM): non-uniform spacing. Pairs data points with frequency points.

MATRIX

Matrix (data point) format:

- SYMMETRIC: symmetric matrix. Specifies only the lower-half triangle portion of a matrix. Default.
- HERMITIAN: similar to SYMMETRIC, but off-diagonal terms are complex-conjugates of each other.
- NONSYMMETRIC: non-symmetric matrix. Specifies a full matrix.

VALTYPE

Data type for matrix elements:

- REAL: real entry.
- CARTESIAN: complex number, in real/imaginary format. Default.
- POLAR: complex number, in polar format. Specifies angles in radians.

INFINITY

Data point, at infinity. Typically real-valued. The data format must be consistent with the MATRIX and VALTYPE specifications. npt s does not count this point.

DC

Data point, at DC. Typically real-valued. The data format must be consistent with the MATRIX and VALTYPE specifications. npt s does not count this point. You must specify a DC point or a data point, at frequency=0.

INTERPOLATION

Interpolation scheme:

- STEP: piecewise step. Default.
- LINEAR: piecewise linear.
- SPLINE: b-spline curve fit.

EXTRAPOLATION

Extrapolation scheme during simulation:

- NONE: does not allow extrapolation. Star-Hspice terminates, if a required data point is outside of the specified range.
- STEP: uses the last boundary point. Default.
- LINEAR: linear extrapolation, based on the last two boundary points.

If you specify the data point at infinity, then Star-Hspice does not extrapolate, and uses the infinity value.

DATA

Specifies data points.

- Syntax for LIN spacing:

```
.MODEL name sp SPACING=LIN
+ [N=dim] FSTART=f0 DF=f1
+ DATA=npts d1 d2 ...
```

- Syntax for OCT or DEC spacing:

```
.MODEL name sp SPACING=DEC or OCT
+ [N=dim] FSTART=f0 NI=n_per_intval
+ DATA=npts d1 d2 ...
```

Syntax for POI spacing:

```
.MODEL name sp
+ SPACING=NONUNIFORM [N=dim]
+ DATA=npts f1 d1 f2 d2 ...
```

DATAFILE

Use this option to specify data points in an external file. The content of this file must be only raw numbers, without any suffixes, comments, or continuation characters. The order of data must follow the DATA statement. This data file does not limit the line length, so you can enter a large number of data points.

Note: Star-Hspice interpolates and extrapolates *after* it internally converts S parameter data to the Y parameter.

Example

In this example, the two outputs from the resistor and S parameter modeling must match exactly. See [Table 6-1](#) for the input file listing. See [Figure 6-2](#) for an illustration of a transmission line, using a resistive termination.

Figure 6-2: Transmission Line, with Resistive Termination

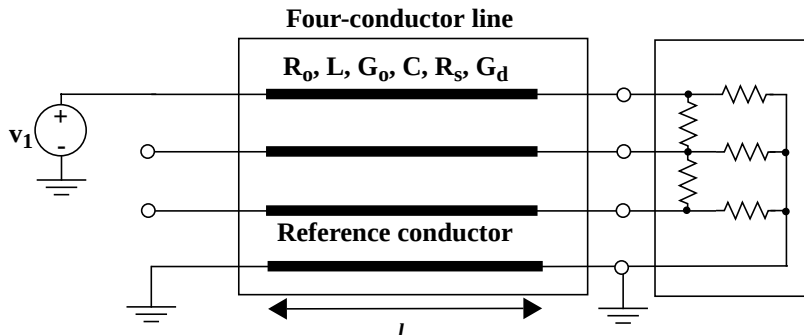


Table 6-1: Input File Listing

Header, options, and sources	<pre>*S parameter ex1: x-line, with a resistive + termination .OPTION POST V1 i1 0 ac=1v</pre>
Analysis	<pre>.AC lin 500 0Hz 30MegHz .DC v1 0v 5v 1v</pre>
Transmission line (W element)	<pre>W1 i1 i2 i3 0 o1 o2 o3 0 RLGCMODEL=wrlgc N=3 + L=0.97 .MODEL wrlgc W MODELTYPE=RLGC N=3 + Lo = 2.78310e-07 + 8.75304e-08 3.29391e-07 + 3.65709e-08 1.15459e-07 3.38629e-07 + Co = 1.41113e-10 + -2.13558e-11 9.26469e-11 + -8.92852e-13 -1.77245e-11 8.72553e-11</pre>
Termination	<pre>x1 o1 o2 o3 0 terminator</pre>

Frequency model definition	<pre>.MODEL fmod sp N=3 FSTOP=30MegHz + DATA= 1 + -0.270166 0.0 + 0.322825 0.0 -0.41488 0.0 + 0.17811 0.0 0.322825 0.0 -0.270166 0.0</pre>
Resistor elements	<pre>.SUBCKT terminator n1 n2 n3 ref R1 n1 ref 75 R2 n2 ref 75 R3 n3 ref 75 R12 n1 n2 25 R23 n2 n3 25 .ends terminator</pre>
Equivalent S parameter element	<pre>.ALTER S parameter case .SUBCKT terminator n1 n2 n3 ref S1 n1 n2 n3 ref FQMODEL=fmod .ENDS terminator .END</pre>

The following is an example of a transmission line, with a capacitive network termination.

Frequency model definition	<pre>.MODEL fmod sp N=3 FSTOP=30MegHz + DATA= 2 + 1.0 0.0 + 0.0 0.0 1.0 0.0 + 0.0 0.0 0.0 0.0 1.0 0.0 + 0.97409 -0.223096 + 0.00895303 0.0360171 0.964485 -0.25887 + -0.000651487 0.000242442 0.00895303 + 0.0360171 0.97409 -0.223096</pre>
Using capacitive elements	<pre>.SUBCKT terminator n1 n2 n3 ref C1 n1 ref 10pF C2 n2 ref 10pF C3 n3 ref 10pF C12 n1 n2 2pF C23 n2 n3 2pF .ENDS terminator</pre>

The two outputs from the resistor and S parameter modeling differ slightly, due to the linear frequency dependency, relative to the capacitor. To remove this difference, use the linear interpolation scheme in `.MODEL`.

Figure 6-3 and Table 6-2 show an example of a transmission line, using the S parameter.

Figure 6-3: 3-Conductor Transmission Line

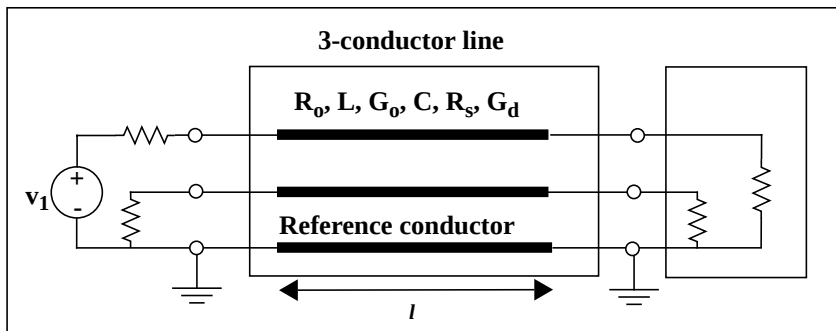


Table 6-2: Input File Listing

Header, options, and sources	<pre>*S parameter ex3: modeling x-line, using + S parameter .OPTION POST vin in0 0 ac=1</pre>
Analysis	<pre>.AC lin 100 0 1000meg .DC vin 0 1v 0.2v</pre>
Transmission line	<pre>W1 in1 in2 0 out1 out2 0 N=2 RLGCMODEL=m2</pre>
Termination	<pre>R1 in0 in1 28 R2 in2 0 28 R3 out1 0 28 R4 out2 0 28</pre>
W Element RLGC model definition	<pre>.MODEL m2 w ModelType=RLGC, N=2 + Lo= 0.178e-6 0.0946e-7 0.178e-6 + Co= 0.23e-9 -0.277e-11 0.23e-9 + Ro= 0.97 0 0.97 + Go= 0 0 0 + Rs= 0.138e-3 0 0.138e-3 + Gd= 0.29e-10 0 0.29e-10</pre>

Frequency model definition	<pre> .MODEL SM2 sp N=4 FSTART=0 FSTOP=1e+09 + SPACING=LINEAR + DATA= 60 + 0.00386491 0 + 0 0 0.00386491 0 + 0.996135 0 0 0 0.00386491 0 + 0 0 0.996135 0 0 0 0.00386491 0 + -0.0492864 -0.15301 + 0.00188102 0.0063569 -0.0492864 + -0.15301 0.926223 -0.307306 0.000630484 + -0.00154619 0.0492864 -0.15301 + 0.000630484 -0.00154619 0.926223 + -0.307306 0.00188102 0.0063569 + -0.0492864 -0.15301 -0.175236 -0.241602 + 0.00597 0.0103297 -0.175236 -0.241602 + 0.761485 -0.546979 0.00093508 + -0.00508414 -0.175236 -0.241602 + 0.00093508 -0.00508414 0.761485 + -0.546979 0.00597 0.0103297 -0.175236 + -0.241602 + ... </pre>
Equivalent S parameter element	<pre> .SUBCKT terminator n1 n2 n3 ref S1 n1 n2 n3 ref FQMODEL=SM2 .ENDS terminator .END </pre>



Chapter 7

Parameters and Functions

Parameters are similar to variables, which most programming languages use. They hold a value that you either assign when you create your circuit design, or the simulation calculates, based on circuit solution values. Parameters can store static values for a variety of quantities (resistance, source voltage, rise time, and so on). You can also use them in sweep or statistical analysis.

This chapter describes how to use parameters within a Star-Hspice netlist:

- [Using Parameters in Simulation \(.PARAM\)](#)
- [Using Algebraic Expressions](#)
- [Built-In Functions](#)
- [Parameter Scoping and Passing](#)

Using Parameters in Simulation (.PARAM)

Defining Parameters

Parameters in Star-Hspice are names that you associate with numeric values. You can use any of these methods to define parameters:

Simple assignment	<code>.PARAM <SimpleParam> = 1e-12</code>
Algebraic definition	<code>.PARAM <AlgebraicParam> = 'SimpleParam*8.2'</code> <i>SimpleParam</i> excludes the output variable. You can also use algebraic parameters in <code>.PRINT</code> and <code>.PROBE</code> statements, and in <code>.PLOT</code>, and <code>.GRAPH</code> statements. For example: <pre>.PRINT AlgebraicParam=par('algebraic expression')</pre> You can use the same syntax for <code>.PROBE</code>, <code>.PLOT</code>, and <code>.GRAPH</code> statements. See Using Algebraic Expressions on page 7-9.
User-defined function	<code>.PARAM <MyFunc(x, y)> = 'Sqrt((x*x)+(y*y))'</code>
Subcircuit default	<pre>.SUBCKT <SubName> <ParamDefName> = + <Value> .MACRO <SubName> <ParamDefName> = + <Value></pre>
Predefined analysis function	<pre>.PARAM <mcVar> = Agauss(1.0,0.1)</pre> (see Statistical Analysis and Optimization on page 13-1).

.MEASURE statement	.MEASURE <DC AC TRAN> result TRIG + ... + TARG ... <GOAL = val> <MINVAL = val> + <WEIGHT = val> <MeasType> + <MeasParam> (see Specifying User-Defined Analysis (.MEASURE) on page 8-40).
.PRINT .PROBE .PLOT .GRAPH	.PRINT .PROBE .PLOT .GRAPH <DC AC TRAN> <i>outParam = Par_Expression</i>

A parameter definition in Star-Hspice always uses the last value found in the input netlist (subject to local versus global parameter rules). The definitions below assign a value of 3 to the *DupParam* parameter.

```
.PARAM DupParam = 1
...
.PARAM DupParam = 3
```

Star-Hspice assigns 3 as the value for all instances of *DupParam*, including instances that are earlier in the input than the `.PARAM DupParam = 3` statement.

All parameter values in Star-Hspice are IEEE double floating point numbers. Parameter resolution order is:

1. Resolve all literal assignments.
2. Resolve all expressions.
3. Resolve all function calls.

[Table 7-1](#) shows the parameter passing order.

Table 7-1: Parameter Passing Order

.OPTION PARHIER = GLOBAL	.OPTION PARHIER = LOCAL
Analysis sweep parameters	Analysis sweep parameters
.PARAM statement (library)	.SUBCKT call (instance)
.SUBCKT call (instance)	.SUBCKT definition (symbol)
.SUBCKT definition (symbol)	.PARAM statement (library)

Assigning Parameters

You can assign the following types of values to parameters:

- A constant real number.
- An algebraic expression of real values.
- A predefined function.
- A function that you define.
- A circuit value.
- A model value.

To invoke the algebraic processor in Star-Hspice, you must enclose a complex expression in single quotes. A simple expression consists of a single parameter name.

The parameter keeps the assigned value, unless:

- a later definition changes its value, or
- an algebraic expression assigns a new value during simulation.

Star-Hspice does not warn you, if they reassign a parameter.

Syntax

```
.PARAM <ParamName> = <RealNumber>
.PARAM <ParamName> = '<Expression>' $ Quotes are mandatory
.PARAM <ParamName1> = <ParamName2> $ Cannot be recursive!
```

Numerical Example

```
.PARAM TermValue = 1g
  rTerm Bit0 0 TermValue
  rTerm Bit1 0 TermValue
...
```

Expression Example

```
.PARAM Pi          = '355/113'
.PARAM Pi2         = '2*Pi'

.PARAM npRatio     = 2.1
.PARAM nWidth      = 3u
.PARAM pwidth      = 'nWidth * npRatio'
Mp1               ... <pModelName> W = pwidth
Mn1               ... <nModelName> W = nWidth
...
```

Inline Parameter Assignments

To define circuit values, using a direct algebraic evaluation:

```
r1 n1 0 R = '1k/sqrt(HERTZ)' $ Resistance related to
+ $ frequency
```

Parameters in Output

To use an algebraic expression as an output variable in a .PRINT, .PLOT, .PROBE .GRAPH, or .MEASURE statement, use the PAR keyword (see [Simulation Output on page 8-1](#) for more information about simulation output). For example:

```
.PRINT DC v(3) gain = PAR('v(3)/v(2)') PAR('v(4)/v(2)')
```

User-Defined Function Parameters

You can define a function that is similar to the parameter assignment, but you cannot nest the functions more than two deep.

The format of a function is:

```
funcname1(arg1[,arg2]) = expression1
+ [funcname2(arg1[,arg2]) = expression2] off
```

- An expression can contain parameters that you have not yet defined.
- A function must have at least one argument, and not more than two.
- You can redefine functions.

where:

<i>funcname</i>	Specifies the function name. This parameter must be distinct from array names and built-in functions. In subsequently defined functions, all embedded functions must be previously defined.
<i>arg1, arg2</i>	Specifies variables used in the expression.
<i>off</i>	voids all user-defined functions.

For example:

```
f(a,b) = POW(a,2)+a*b   g(d) = SQRT(d)
+ h(e) = e*f(1,2)-g(3)
```

Syntax

The syntax for user-defined function parameters is:

```
.PARAM <ParamName>(<pv1>[, <pv2>]) = '<Expression>'
```

Example

```
.PARAM CentToFar (c)          = '(((c*9)/5)+32)'  
.PARAM F(p1,p2)              = 'Log(Cos(p1)*Sin(p2))'  
.PARAM SqrDProd (a,b)        = '(a*a)*(b*b)'
```

Subcircuit Default Parameter Definitions

When you use hierarchical sub-circuits, you can pick default values for circuit elements. You typically use defaults in cell definitions, to simulate the circuit using typical values (see [Using Subcircuits on page 3-51](#)).

Syntax

```
.SUBCKT <SubName> <PinList> [<SubDefaultsList>]
```

where <SubDefaultsList> is

```
<SubParam1> = <Expression>  
[<SubParam2> = <Expression> ...]
```

Subcircuit Parameter Example

This example implements an inverter that uses a *Strength* parameter. By default, the inverter can drive three devices. Enter a new value for the *Strength* parameter in the element line, to select larger or smaller inverters for the application.

```
.SUBCKT Inv a y Strength = 3  
    Mp1 <MosPinList> pMosMod L = 1.2u W = 'Strength * 2u'  
    Mn1 <MosPinList> nMosMod L = 1.2u W = 'Strength * 1u'  
.ENDS  
...  
xInv0 a y0 Inv                      $ Default devices: p device = 6u,  
                                     $ n device = 3u  
xInv1 a y1 Inv Strength = 5          $ p device = 10u, n device = 5u  
xInv2 a y2 Inv Strength = 1          $ p device = 2u, n device = 1u  
...
```

Predefined Analysis Function

Star-Hspice includes specialized analysis types, such as Optimization and Monte Carlo, that require a way to control the analysis. For definitions of the parameters for these analysis types, see [Statistical Analysis and Optimization on page 13-1](#).

Measurement Parameters

.MEASURE statements produce a *measurement* parameter. The rules for measurement parameters are the same as for standard parameters, except that measurement parameters are defined in a .MEASURE statement, not in a .PARAM statement. For a description of the .MEASURE statement, see [Specifying User-Defined Analysis \(.MEASURE\) on page 8-40](#).

.PRINT|.PROBE|.PLOT|.GRAPH Parameters

.PRINT | .PROBE | .PLOT | .GRAPH statements in Star-Hspice produce a *print* parameter. The rules for print parameters are the same as the rules for standard parameters, except that you define the parameter directly in a .PRINT | .PROBE | .PLOT | .GRAPH statement, not in a .PARAM statement.

For example:

```
.print p1 = 3  
.print p2 = par("p1*5")
```

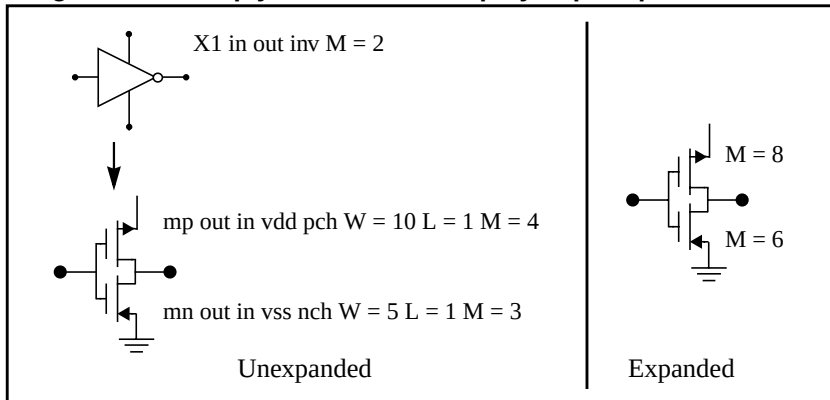
You can use p1 and p2 as parameters in netlist. The p1 value is 3; the p2 value is 15.

Multiply Parameter

The *M* multiply parameter is a special keyword, common to all elements (except for voltage sources) and sub-circuits. It multiplies the internal component values, which has the same effect as making parallel copies of the element or subcircuit. To simulate the effect of 32 output buffers that switch simultaneously, you need to place only one subcircuit call, such as:

```
X1 in out buffer M = 32
```

Multiply works hierarchically. Star-Hspice multiplies a subcircuit within a subcircuit, by the product of the multiply parameters, at both levels.

Figure 7-1: Multiply Parameters Simplify Flip-Flop Initialization

Using Algebraic Expressions¹

In Star-Hspice, an algebraic expression, with quoted strings, can replace any parameter in the netlist.

In Star-Hspice, you can then use these expressions as output variables, in .PLOT, .PRINT, and .GRAPH statements. Algebraic expressions can expand your options in the input netlist file. Some uses of algebraic expressions are:

■ Parameters:

```
.PARAM x = 'y+3'
```

■ Functions:

```
.PARAM rho(leff,weff) = '2+*leff*weff-2u'
```

■ Algebra in elements:

```
R1 1 0 r = 'ABS(v(1)/i(m1))+10'
```

■ Algebra in .MEASURE statements:

```
.MEAS vmax MAX V(1)
.MEAS imax MAX I(q2)
.MEAS ivmax PARAM = 'vmax*imax'
```

■ Algebra in output statements:

```
.PRINT conductance = PAR('i(m1)/v(22)')
```

The basic syntax for using algebraic expressions for output is:

```
PAR('algebraic expression')
```

In addition to using quotations, you must define the expression inside the PAR() statement, for output. The continuation character for quoted parameter strings, in Star-Hspice, is a double backslash (\). (Outside of quoted strings, the single backslash, \, is the continuation character.)

¹Star-Hspice uses double-precision numbers (15 digits) for expressions, user-defined parameters, and sweep variables. For better precision, use parameters (instead of constants) in algebraic expressions, because constants are only single-precision numbers (7 digits).

Built-In Functions

In addition to simple arithmetic operations (+, -, *, /), Star-Hspice provides several built-in functions, listed in [Table 7-2](#), that you can use in expressions:

Table 7-2: Star-Hspice Built-in Functions (Sheet 1 of 4)

HSPICE Form	Function	Class	Description
sin(x)	sine	trig	Returns the sine of x (radians)
cos(x)	cosine	trig	Returns the cosine of x (radians)
tan(x)	tangent	trig	Returns the tangent of x (radians)
asin(x)	arc sine	trig	Returns the inverse sine of x (radians)
acos(x)	arc cosine	trig	Returns the inverse cosine of x (radians)
atan(x)	arc tangent	trig	Returns the inverse tangent of x (radians)
sinh(x)	hyperbolic sine	trig	Returns the hyperbolic sine of x (radians)
cosh(x)	hyperbolic cosine	trig	Returns the hyperbolic cosine of x (radians)
tanh(x)	hyperbolic tangent	trig	Returns the hyperbolic tangent of x (radians)
abs(x)	absolute value	math	Returns the absolute value of x: x
sqrt(x)	square root	math	Returns the square root of the absolute value of x: $\text{sqrt}(-x) = -\text{sqrt}(x)$
pow(x,y)	absolute power	math	Returns the value of x raised to the integer part of y: $x^{(\text{integer part of } y)}$
pwr(x,y)	signed power	math	Returns the absolute value of x, raised to the y power, with the sign of x: $(\text{sign of } x) x ^y$

Table 7-2: Star-Hspice Built-in Functions (Sheet 2 of 4)

HSPICE Form	Function	Class	Description
log(x)	natural logarithm	math	Returns the natural logarithm of the absolute value of x, with the sign of x: (sign of x)log(x)
log10(x)	base 10 logarithm	math	Returns the base 10 logarithm of the absolute value of x, with the sign of x: (sign of x)log ₁₀ (x)
exp(x)	exponential	math	Returns e, raised to the power x: e^x
db(x)	decibels	math	Returns the base 10 logarithm of the absolute value of x, multiplied by 20, with the sign of x: (sign of x)20log ₁₀ (x)
int(x)	integer	math	Returns the integer portion of x. The fractional portion of the number is lost.
nint(x)	integer	math	Rounds x up or down, to the nearest integer.
sgn(x)	return sign	math	■ Returns -1 if x is less than 0. ■ Returns 0 if x is equal to 0. ■ Returns 1 if x is greater than 0
sign(x,y)	transfer sign	math	Returns the absolute value of x, with the sign of y: (sign of y) x
min(x,y)	smaller of two args	control	Returns the numeric minimum of x and y
max(x,y)	larger of two args	control	Returns the numeric maximum of x and y
val(element)	get value	various	Returns a parameter value for a specified element. For example, val(r1) returns the resistance value of the r1 resistor.

Table 7-2: Star-Hspice Built-in Functions (Sheet 3 of 4)

HSPICE Form	Function	Class	Description
<code>val(<i>element.parameter</i>)</code>	get value	various	Returns a value for a specified parameter of a specified element. For example, <code>val(rload.temp)</code> returns the value of the <i>temp</i> (temperature) parameter for the <i>rload</i> element.
<code>val(model_type:model_name.model_param)</code>	get value	various	Returns a value for a specified parameter of a specified model of a specific type. For example, <code>val(nmos:mos1.rs)</code> returns the value of the <i>rs</i> parameter for the <i>mos1</i> model, which is an <i>nmos</i> model type.
<code>lv(<Element>)</code> or <code>lx(<Element>)</code>	element templates	various	Returns various element values during simulation. See Element Template Output on page 8-39 for more information.
<code>v(<Node>),</code> <code>i(<Element>)</code> ...	circuit output variables	various	Returns various circuit values during simulation. See DC and Transient Output Variables on page 8-25 for more information.
<code>[cond] ?x : y</code>	ternary operator		Returns <i>x</i> if <i>cond</i> is not zero. Otherwise, returns <i>y</i> . Syntax: <code>.para x=[condition] ?y:z</code>
<code><</code>	relational operator (less than)		Returns 1 if the left operand is less than the right operand. Otherwise, returns 0. Syntax: <code>.para x=y<z</code> (<i>y</i> less than <i>z</i>)

Table 7-2: Star-Hspice Built-in Functions (Sheet 4 of 4)

HSPICE Form	Function	Class	Description
<=	relational operator (less than or equal)		Returns 1 if the left operand is less than or equal to the right operand. Otherwise, returns 0. Syntax: .para x=y<=z (y less than or equal to z)
>	relational operator (greater than)		Returns 1 if the left operand is greater than the right operand. Otherwise, returns 0. Syntax: .para x=y>z (y greater than z)
>=	relational operator (greater than or equal)		Returns 1 if the left operand is greater than or equal to the right operand. Otherwise, returns 0. Syntax: .para x=y>=z (y greater than or equal to z)
==	equality		Returns 1 if the operands are equal. Otherwise, returns 0. Syntax: .para x=y==z (y equal to z)
!=	inequality		Returns 1 if the operands are not equal. Otherwise, returns 0. Syntax: .para x=y!=z (y not equal to z)
&&	Logical AND		Returns 1 if neither operand is zero. Otherwise, returns 0. Syntax: .para x=y&&z (y AND z)
	Logical OR		Returns 1 if either or both operands are not zero. Returns 0 only if both operands are zero. Syntax: .para x=y z (y OR z)

Example

```
.parameters p1=4 p2=5 p3=6
r1 1 0 value=(p1 ? p2+1 : p3)
```

Star-Hspice reserves the variable names listed in [Table 7-3](#), for use in elements such as E, G, R, C, and L. You cannot use them for any other purpose in your netlist (for example, in .PARAM statements).

Table 7-3: Star-Hspice Special Variables

HSPICE Form	Function	Class	Description
time	current simulation time	control	Uses parameters to define the current simulation time, during transient analysis.
temper	current circuit temperature	control	Uses parameters to define the current simulation temperature, during transient/temperature analysis.
hertz	current simulation frequency	control	Uses parameters to define the frequency, during AC analysis.

Parameter Scoping and Passing

If you use parameters to define values in sub-circuits, you need to create fewer similar cells, to provide enough functionality in your library. You can pass circuit parameters into hierarchical designs, and assign different values to the same parameter within individual cells, when you run simulation.

For example, if you use parameters to set the initial state of a latch in its subcircuit definition, then you can override this initial default in the instance call. You need to create only one cell, to handle both initial state versions of the latch.

You can also use parameters to define the layout of a cell. For example, you can use parameters in a MOS inverter, to simulate a range of inverter sizes, with only one cell definition. Local instances of the cell can assign different values to the size parameter for the inverter.

In Star-Hspice, you can also perform Monte Carlo analysis or optimization on a cell that uses parameters.

How you handle hierarchical parameters depends on how you construct and analyze your cells. You can construct a design in which information flows from the top of the design, down into the lowest hierarchical levels.

- To centralize the control at the top of the design hierarchy, set *global* parameters.
- To construct a library of small cells that are individually controlled from within, set *local* parameters, and build upwards to the block level.

This section describes the scope of parameter names, and how Star-Hspice resolves naming conflicts between levels of hierarchy.

Library Integrity

Integrity is a fundamental requirement for any symbol library. Library integrity can be as simple as a consistent, intuitive name scheme, or as complex as libraries with built-in range checking.

You risk poor library integrity if you use libraries from different vendors in a single circuit design. Because names of circuit parameters are not standardized between vendors, two components can include the same parameter name for different functions. For example, one vendor might build a library that uses the name *Tau* as a parameter to control one or more subcircuits in their library. Another vendor might use *Tau* to control a different aspect of their library. If you set a global parameter named *Tau* to control one library, you also modify the behavior of the second library, which might not be the intent.

If the *scope* of a higher-level parameter is *global* to all sub-circuits at lower levels of the design hierarchy, higher-level definitions override lower-level parameter values that have the same names. The scope of a lower-level parameter is *local* to the subcircuit where you define the parameter (but global to all subcircuits that are even lower in the design hierarchy). Local scoping rules in Star-Hspice prevent higher-level parameters from overriding lower-level parameters of the same name, when that is not desired.

Reusing Cells

Problems with parameter names also occur if different groups collaborate on a design. Because global parameters prevail over local parameters, all circuit designers must know the names of all parameters, even those used in sections of the design for which they are not responsible. This can lead to a large investment in standardized libraries. To avoid this situation, use local parameter scoping, to encapsulate all information about a section of a design, within that section.

Creating Parameters in a Library

To ensure that the input netlist includes critical, user-supplied parameters when you run simulation, you can use “illegal defaults”—that is, defaults that cause the simulator to abort if you do not supply overrides for the defaults.

If a library cell includes illegal defaults, you must provide a value for each instance of those cells. If you do not, the simulation aborts.

For example, you might define a default MOSFET width of 0.0. Star-Hspice aborts, because MOSFET models require this parameter.

Example 1

```
* Subcircuit default definition
.SUBCKT Inv A Y Wid = 0      $ Inherit illegal values by default
    mp1 <NodeList> <Model> L = 1u W = 'Wid*2'
    mn1 <NodeList> <Model> L = 1u W = Wid
.ENDS

* Invoke symbols in a design
x1 A Y1 Inv                  $ Bad! No widths specified
x2 A Y2 Inv Wid = 1u         $ Overrides illegal value for Width
```

This simulation aborts on the *x1* subcircuit instance, because you never set the required *Wid* parameter on the subcircuit instance line. The *x2* subcircuit simulates correctly. Additionally, the instances of the *Inv* cell are subject to accidental interference, because the *Wid* global parameter is exposed outside the domain of the library. Anyone can specify an alternative value for the parameter, in another section of the library or the circuit design. This might prevent the simulation from catching the condition on *x1*.

Example 2

In this example, the name of a global parameter conflicts with the internal library parameter named *Wid*. Another user might specify such a global parameter, in a different library. In this example, the user of the library has specified a different meaning for the *Wid* parameter, to define an independent source.

```
.Param Wid = 5u              $ Default Pulse Width for source
v1 Pulsed 0 Pulse ( 0v 5v 0u 0.1u 0.1u Wid 10u )
...

* Subcircuit default definition
.SUBCKT Inv A Y Wid = 0      $ Inherit illegals by default
    mp1 <NodeList> <Model> L = 1u W = 'Wid*2'
    mn1 <NodeList> <Model> L = 1u W = Wid
.ENDS

* Invoke symbols in a design
x1 A Y1 Inv                  $ Incorrect width!
x2 A Y2 Inv Wid = 1u         $ Incorrect! Both x1 and x2
                             $ simulate with mp1 = 10u and
                             $ mn1 = 5u instead of 2u and 1u.
```

Under global parameter scoping rules, simulation succeeds, but incorrectly. Star-Hspice does not warn you that the x1 inverter has no assigned width, because the global parameter definition for *Wid* overrides the subcircuit default.

Note: Similarly, sweeping with different values of *Wid* dynamically changes both the *Wid* library internal parameter value, and the pulse width value to the *Wid* value of the current sweep.

In global scoping, the highest-level name prevails, when resolving name conflicts. Local scoping uses the lowest-level name.

When you use the parameter inheritance method, you can specify to use local scoping rules. This feature can cause different results than you obtained using Star-Hspice versions before release 95.1, on existing circuits.

When you use local scoping rules, the Example 2 netlist correctly aborts in x1, for $W = 0$ (default *Wid* = 0, in the .SUBCKT definition, has higher precedence, than the .PARAM statement). This results in the correct device sizes for x2. This change can affect your simulation results, if you intentionally or accidentally create a circuit such as the second one shown above.

As an alternative to width testing in the Example 2 netlist, you can use .OPTION DEFW to achieve a limited version of library integrity. This option specifies the default width for all MOS devices, during a simulation. Because part of the definition is still in the top-level circuit, this method can still make unwanted changes to library values, without notification from the Star-Hspice simulator.

[Table 7-4](#) compares the three primary methods for configuring libraries, to achieve required parameter checking for default MOS transistor widths.

Table 7-4: Methods for Configuring Libraries

Method	Parameter Location	Pros	Cons
Local	On a .SUBCKT definition line	Protects the library from global circuit parameter definitions, unless you override it. Single location for default values.	You cannot use it with versions of Star-Hspice before Release 95.1.
Global	At the global level and on .SUBCKT definition lines	Works with older Star-Hspice versions.	An indiscreet user, another vendor assignment, or the intervening hierarchy can change the library. Cannot override a global value at a lower level.
Special	.OPTION DEFW statement	Simple to do.	Third-party libraries, or other sections of the design, might depend on the DEFW option.

Parameter Defaults and Inheritance

Use the .OPTION PARHIER parameter to specify scoping rules. The syntax is:

```
.OPTION PARHIER = < GLOBAL | LOCAL >
```

The default setting is GLOBAL, which uses the same scoping rules that Star-Hspice used before Release 95.1.

Parameter Scoping Example

The following example explicitly shows the difference between local and global scoping, for using parameters in sub-circuits.

The input netlist includes the following:

```
.OPTION parhier=<global | local>
.PARAM DefPwid = 1u
.SUBCKT Inv a y DefPwid = 2u DefNwid = 1u
    Mp1 <MosPinList> pMosMod L = 1.2u W = DefPwid
    Mn1 <MosPinList> nMosMod L = 1.2u W = DefNwid
.ENDS
```

Set the `.OPTION PARHIER =` parameter scoping option to `GLOBAL`. The netlist also includes the following input statements:

```
xInv0 a y0 Inv                $override DefPwid default,
                                $ xInv0.Mp1 width = 1u
xInv1 a y1 Inv DefPwid = 5u    $override DefPwid=5u,
                                $ xInv1.Mp1 width = 1u
.measure tran Wid0 param = 'lv2(xInv0.Mp1)' $ lv2 is the
                                           $ template for the
.measure tran Wid1 param = 'lv2(xInv1.Mp1)' $ channel width
                                           $ 'lv2(xInv1.Mp1)'
.ENDS
```

Simulating this netlist produces the following results in the listing file:

```
wid0 = 1.0000E-06
wid1 = 1.0000E-06
```

If you change the `.OPTION PARHIER =` parameter scoping option to `LOCAL`:

```
xInv0 a y0 Inv                $not override .param DefPwid=2u,
                                $ xInv0.Mp1 width = 2u
xInv1 a y1 Inv DefPwid = 5u    $override .param DefPwid=2u,
                                $ xInv1.Mp1 width = 5u:
.measure tran Wid0 param = 'lv2(xInv0.Mp1)' $ override the
.measure tran Wid1 param = 'lv2(xInv1.Mp1)' $ global .PARAM
...
```

then simulation produces the following results in the listing file:

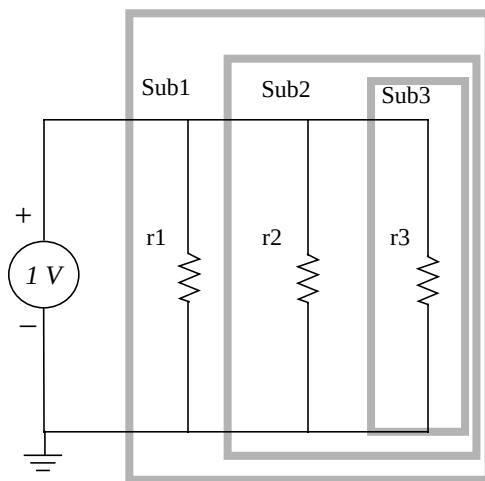
```
wid0 = 2.0000E-06
wid1 = 5.0000E-06
```

Parameter Passing

Figure 7-2 shows a flat representation of a hierarchical circuit, which contains three resistors.

Each of the three resistors obtains its simulation time resistance from the *Val* parameter. The netlist defines the *Val* parameter in four places, with three different values.

Figure 7-2: Hierarchical Parameter Passing Problem



```

TEST OF PARHIER
.OPTION list node post = 2
+ ingold = 2
+ parhier = <Local|Global>
.PARAM Val = 1
x1 n0 0 Sub1
.SubCkt Sub1 n1 n2 Val = 1
    r1 n1 n2 Val
    x2 n1 n2 Sub2
.Ends Sub1
.SubCkt Sub2 n1 n2 Val = 2
    r2 n1 n2 Val
    x3 n1 n2 Sub3
.Ends Sub2
.SubCkt Sub3 n1 n2 Val = 3
    r3 n1 n2 Val
.Ends Sub3
.OP
.END

```

The total resistance of the chain has two possible solutions: 0.3333Ω and 0.5455Ω .

You can use the PARHIER option to specify which parameter value prevails, when you define parameters with the same name at different levels of the design hierarchy.

Under global scoping rules, in the case of name conflicts, the top-level assignment `.PARAM Val = 1` overrides the subcircuit defaults, and the total is 0.3333Ω . Under local scoping rules, the lower level assignments prevail, and the total is 0.5455Ω (one, two and three ohms in parallel).

The example in [Figure 7-2 on page 7-21](#) produces the results in [Table 7-5](#), based on how you set the local/global PARHIER option:

Table 7-5: PARHIER = LOCAL vs. PARHIER = GLOBAL Results

Element	PARHIER = Local	PARHIER = Global
r1	1.0	1.0
r2	2.0	1.0
r3	3.0	1.0

Parameter Passing Solutions

Changes in scoping rules can cause different simulation results, for circuit designs created before Star-Hspice Release 95.1, than for designs created after that release. Use the following checklist to determine whether you will see simulation differences when you use the new default scoping rules. These checks are especially important if your netlists contain devices from multiple vendor libraries.

- Check your sub-circuits for parameter defaults, on the .SUBCKT or .MACRO line.
- Check your sub-circuits for a .PARAM statement, within a .SUBCKT definition.
- To check your circuits for global parameter definitions, use the .PARAM statement.
- If any of the names from the first three checks are identical, set up two Star-Hspice simulation jobs: one with .OPTION PARHIER = GLOBAL, and one with .OPTION PARHIER = LOCAL. Then look for differences in the output.



Chapter 8

Simulation Output

Use output format statements and variables to display steady state, frequency, and time domain simulation results. You can also use these variables in behavioral circuit analysis, modeling, and simulation techniques. To display electrical specifications (such as rise time, slew rate, amplifier gain, and current density), use the output format features.

This chapter explains the following topics:

- [Overview of Output Statements](#)
- [Displaying Simulation Results](#)
- [Selecting Simulation Output Parameters](#)
- [Specifying User-Defined Analysis \(.MEASURE\)](#)
- [.DOUT Statement: Expected State of Digital Output Signal](#)
- [.STIM Statement: Reuse Simulation Output as Input Stimuli](#)
- [Element Template Listings](#)

Overview of Output Statements

Output Commands

The input netlist file contains output statements, including `.PRINT`, `.PLOT`, `.GRAPH`, `.PROBE`, `.MEASURE`, and `.DOUT`. Each statement specifies the output variables, and the type of simulation result, to display—such as `.DC`, `.AC`, or `.TRAN`. When you specify `.OPTION POST`, Star-Hspice puts all output variables, referenced in `.PRINT`, `.PLOT`, `.GRAPH`, `.PROBE`, `.MEASURE`, `.DOUT`, and `.STIM` statements, into AvanWaves interface files.

AvanWaves provides high-resolution, post-simulation, and interactive display of waveforms.

Output Statement	Description
<code>.PRINT</code>	Prints numeric analysis results in the output listing file (and post-processor data, if you specify <code>.OPTION POST</code>).
<code>.PLOT</code> (<i>Star-Hspice only</i>)	Generates low-resolution (ASCII) plots in the output listing file (and post-processor data, if you specify <code>.OPTION POST</code>), in Star-Hspice.
<code>.GRAPH</code> (<i>Star-Hspice only</i>)	Generates high-resolution plots, for specific printing devices (such as HP LaserJet), or in PostScript format (intended for hard-copy outputs, without a using a post-processor).
<code>.PROBE</code>	Outputs data to post-processor output files, but not to the output listing (used with <code>.OPTION PROBE</code> , to limit output).
<code>.MEASURE</code>	Prints the results of specific user-defined analyses (and post-processor data, if you specify <code>.OPTION POST</code>), to the output listing file. You can use the <code>.MEASURE</code> statement in Star-Hspice.
<code>.DOUT</code>	Specifies the expected final state of an output signal.
<code>.STIM</code>	Specifies simulation results to transform to PWL, Data Card, or Digital Vector File format.

Output Variables

The output format statements require special output variables, to print or plot analysis results, for nodal voltages and branch currents. Star-Hspice uses the following groups of output variables:

- DC and transient analysis
- AC analysis
- element template
- .MEASURE statement
- parametric analysis

For Star-Hspice, **DC and transient analysis** displays:

- individual nodal voltages:

$V(n1 [,n2])$

- branch currents:

$I(Vxx)$

- element power dissipation:

$In(element)$

AC analysis displays imaginary and real components of a nodal voltage or branch current, and the magnitude and phase of a nodal voltage or branch current. AC analysis results also print impedance parameters, and input and output noise.

Element template analysis displays element-specific nodal voltages, branch currents, element parameters, and the derivatives of the element's node voltage, current, or charge.

The .MEASURE statement variables define the electrical characteristics to measure in a .MEASURE statement analysis, in Star-Hspice.

Parametric analysis variables are mathematically-defined expressions, which operate on nodal voltages, branch currents, element template variables, or other parameters that you specify. You can use these variables when you run behavioral analysis of simulation results. See [Using Algebraic Expressions on page 7-9](#) for information about parameters in Star-Hspice.

Displaying Simulation Results

The following section describes the statements that you can use to display simulation results for your specific requirements.

.PRINT Statement

The .PRINT statement specifies output variables, for which Star-Hspice prints values.

- The maximum number of variables in a single .PRINT statement, was 32 before Release 2002.2, but has been extended. For example, you can enter:

```
.PRINT v(1) v(2) ... v(32) v(33) v(34)
```

which previously required two .PRINT statements:

```
.PRINT v(1) v(2) ... v(32)  
.PRINT v(33) v(34)
```

- To simplify parsing of the output listings, Star-Hspice prints a single *x* in the first column, to indicate the beginning of the .PRINT output data. A single *y* in the first column indicates the end of the .PRINT output data.

Syntax

```
.PRINT antype ov1 <ov2 ... >
```

- | | |
|----------------|--|
| <i>antype</i> | Specifies the type of analysis for outputs. <i>Antype</i> is one of the following types: DC, AC, TRAN, NOISE, or DISTO. |
| <i>ov1 ...</i> | Specifies the output variables to print. These are voltage, current, or element template variables, from a DC, AC, TRAN, NOISE, or DISTO analysis. |

You can include wildcards in .PRINT statements. For example:

```
.PRINT TRAN V(9?t*u)
```

This example prints out the results of a transient analysis, for the voltage at the matched node name.

- The ? wildcard matches any single character. For example, 9? matches 92, 9a, 9A, and 9%.
- The * wildcard matches any string of zero or more characters. For example:
 - If your netlist includes a resistor named *r1* and a voltage source named *vin*, then `.print i(*)` prints the current for both of these elements: *i(r1)* and *i(vin)*.
 - `.print v(o*)` prints the voltages for all nodes whose names start with *o*; if your netlist contains nodes named *in* and *out*, this example prints only the *v(out)* voltage.
 - If your netlist contains nodes named 0, 1, 2, and 3, then `.print v(0,*)` or `.print v(0 *)` prints the voltage between node 0 and each of the other nodes: *v(0,1)*, *v(0,2)*, and *v(0,3)*.

You can also use the `iall` keyword in a .PRINT statement, to print all branch currents of all diode, BJT, JFET, or MOSFET elements in your circuit design. For example, if your circuit contains four MOSFET elements (named *m1*, *m2*, *m3*, and *m4*), then `.print iall (m*)` is equivalent to `.print i(m1) i(m2) i(m3) i(m4)`, and prints the output currents of all four MOSFET elements.

Statement Order

Star-Hspice creates different `.sw0` and `.tr0` files, based on the order of the `.print` and `.dc` statements. If you do not specify an analysis type for a `.print` command, the analysis type matches the last analysis command in the netlist, before the `.print` statement.

For example:

```
CASE 1
.print v(din) i(mxn18)
.dc vdin 0 5.0 0.05
.tran 1ns 60ns
```

```
CASE 2
.dc vdin 0 5.0 0.05
.tran 1ns 60ns
.print v(din) i(mxn18)
```

```
CASE 3
.dc vdin 0 5.0 0.05
.print v(din) i(mxn18)
.tran 1ns 60ns
```

- If you replace the `.print` statement with:
`.print TRAN v(din) i(mnx)`
then all three cases have identical `.sw0` and `.tr0` files.
- If you replace the `.print` statement with:
`.print DC v(din) i(mnx)`
then the `.sw0` and `.tr0` files are different.

Example 1

```
.PRINT TRAN V(4) I(VIN) PAR('V(OUT)/V(IN)')
```

This example prints the results of a transient analysis, for the nodal voltage named 4. It also prints the current, through the voltage source named VIN. It also prints the ratio of the nodal voltage at the OUT and IN nodes.

Example 2

```
.PRINT AC VM(4,2) VR(7) VP(8,3) II(R1)
```

- Depending on the value of the ACOUT option, `VM(4,2)` prints the AC magnitude of the voltage difference, or the difference of the voltage magnitudes, between nodes 4 and 2.
- `VR(7)` prints the real part of the AC voltage, between node 7 and ground.
- Depending on the value of the ACOUT option, `VP(8,3)` prints the phase of the voltage difference between nodes 8 and 3, or the difference of the phase of voltage at node 8 and voltage at node 3.
- `II(R1)` prints the imaginary part of the current, through R1.

Example 3

```
.PRINT AC ZIN YOUT(P) S11(DB) S12(M) Z11(R)
```

This example prints:

- The magnitude of the input impedance.
- The phase of the output admittance.
- Several S and Z parameters.

This statement accompanies a network analysis, using the .AC and .NET analysis statements.

Example 4

```
.PRINT DC V(2) I(VSRC) V(23,17) I1(R1) I1(M1)
```

This example prints the DC analysis results for several different nodal voltages and currents, through:

- The resistor named R1.
- The voltage source named VSRC.
- The drain-to-source current of the MOSFET named M1.

Example 5

```
.PRINT NOISE INOISE
```

This example prints the equivalent input noise.

Example 6

```
.PRINT DISTO HD3 SIM2(DB)
```

This example prints the magnitude of the third-order harmonic distortion, and the decibel value of the intermodulation distortion sum, through the load resistor that you specify in the .DISTO statement.

Example 7

```
.PRINT AC INOISE ONOISE VM(OUT) HD3
```

This statement includes NOISE, DISTO, and AC output variables in the same .PRINT statement in Star-Hspice.

Example 8

```
.PRINT pj1 = par('p(rd) +p(rs)')
```

This statement prints the value of `pj1`, with the specified function.

Note: Star-Hspice ignores `.PRINT` statement references to nonexistent netlist part names, and prints those names in a warning message.

.PLOT Statement

The `.PLOT` statement plots the output values of one or more variables, in a selected Star-Hspice analysis. Each `.PLOT` statement defines the contents of one plot, which can contain more than one output variable.

If you do not specify plot limits, Star-Hspice automatically determines the minimum and maximum values of each output variable that it plots, and scales each plot to fit common limits. To force Star-Hspice to set limits for certain variables, set the plot limits to (0,0) for those variables.

To make Star-Hspice find plot limits for each plot individually, use `.OPTION PLIM` to create a different axis for each plot variable. The `PLIM` option is similar to the plot limit algorithm in SPICE2G.6, where each plot can have limits different from any other plot. A number from 2 through 9 indicates the overlap of two or more traces on a plot.

If more than one output variable appears on the same plot, Star-Hspice prints *and* plots the first variable specified. To print out more than one variable, include another `.PLOT` statement.

You can specify an unlimited number of `.PLOT` statements for each type of analysis. To set the plot width, use the `C0` (columns out) option. If you set `C0` to 80, the plot has 50 columns. If `C0` is 132, the plot has 100 columns.

You can include wildcards in `.PLOT` statements. For example:

```
.PLOT TRAN V(9?t*u)
```

This example plots the results of a transient analysis, for the voltage at the matched node name.

- The `?` wildcard matches any single character. For example, `9?` matches `92`, `9a`, `9A`, and `9%`.
- The `*` wildcard matches any string of zero or more characters. For example:
 - If your netlist includes a resistor named `r1` and a voltage source named `vin`, then `.plot i(*)` plots the current for both of these elements: `i(r1)` and `i(vin)`.
 - `.plot v(o*)` plots the voltages for all nodes whose names start with `o`; if your netlist contains nodes named `in` and `out`, this example plots only the `v(out)` voltage.
 - If your netlist contains nodes named `0`, `1`, `2`, and `3`, then `.plot v(0,*)` or `.plot v(0 *)` plots the voltage between node `0` and each of the other nodes: `v(0,1)`, `v(0,2)`, and `v(0,3)`.

Syntax

```
.PLOT antype ov1 <(plo1,phi1)> <ov2> <(plo2,phi2)> ...>
```

where:

<i>antype</i>	Type of analysis for the specified plots. Analysis types are: DC, AC, TRAN, NOISE, or DISTO.
<i>ov1</i> ...	Output variables to plot. These are voltage, current, or element template variables, from a DC, AC, TRAN, NOISE, or DISTO analysis. See the following sections for syntax.
<i>plo1,phi1</i> ...	Lower and upper plot limits. The plot for each output variable uses the first set of plot limits, after the output variable name. Set a new plot limit for each output variable, after the first plot limit. For example, to plot all output variables that use the same scale, specify one set of plot limits at the end of the <code>.PLOT</code> statement. If you set the plot limits to (0,0) Star-Hspice automatically sets the plot limits.

Example

In the following example, PAR plots the ratio of the collector current and the base current, for the Q1 transistor.

```
.PLOT DC V(4) V(5) V(1) PAR('I1(Q1)/I2(Q1)')
.PLOT TRAN V(17,5) (2,5) I(VIN) V(17) (1,9)
.PLOT AC VM(5) VM(31,24) VDB(5) VP(5) INOISE
```

The second of the two preceding examples uses the VDB output variable to plot the AC analysis results (in decibels), for the node named 5. The AC plot can also include NOISE results and other variables that you specify.

```
.PLOT AC ZIN YOUT(P) S11(DB) S12(M) Z11(R)
.PLOT DISTO HD2 HD3(R) SIM2
.PLOT TRAN V(5,3) V(4) (0,5) V(7) (0,10)
.PLOT DC V(1) V(2) (0,0) V(3) V(4) (0,5)
```

In the last example above, Star-Hspice sets the plot limits for V(1) and V(2), but you specify 0 and 5 volts as the plot limits for V(3) and V(4).

.PROBE Statement

The .PROBE statement saves output variables into interface and graph data files. Star-Hspice usually saves all voltages, supply currents, and output variables. Set .OPTION PROBE, to save output variables only. Use the .PROBE statement to specify the quantities to print in the output listing.

If you are interested only in the output data file, and you do not want tabular or plot data in your listing file, set .OPTION PROBE, and use the .PROBE statement to specify the values to save in the output listing.

You can include wildcards in .PROBE statements. For example:

```
.PROBE TRAN V(9?t*u)
```

This example probes the results of a transient analysis, for the voltage at the matched node name.

- The `?` wildcard matches any single character. For example, `9?` matches `92`, `9a`, `9A`, and `9%`.
- The `*` wildcard matches any string of zero or more characters. For example:
 - If your netlist includes a resistor named `r1` and a voltage source named `vin`, then `.probe i(*)` probes for the current for both of these elements: `i(r1)` and `i(vin)`.
 - `.probe v(o*)` probes for the voltages for all nodes whose names start with `o`; if your netlist contains nodes named `in` and `out`, this example probes for only the `v(out)` voltage.
 - If your netlist contains nodes named `0`, `1`, `2`, and `3`, then `.probe v(0,*)` or `.probe v(0 *)` probes for the voltage between node `0` and each of the other nodes: `v(0,1)`, `v(0,2)`, and `v(0,3)`.

Syntax

```
.PROBE antype ov1 <ov2 ...>
```

where:

<i>antype</i>	Type of analysis for the specified plots. Analysis types are: DC, AC, TRAN, NOISE, or DISTO.
<i>ov1</i> ...	Output variables to plot. These are voltage, current, or element template variables from a DC, AC, TRAN, NOISE, or DISTO analysis. A <code>.PROBE</code> statement can include more than one output variable.

Example

```
.PROBE DC V(4) V(5) V(1) beta = PAR(`I1(Q1)/I2(Q1)')
```

.GRAPH Statement

Use the .GRAPH statement when you need high-resolution plots of Star-Hspice simulation results.

This statement is similar to the .PLOT statement, with the addition of an optional model. When you specify a model, you can add or change graphing properties for the graph. The .GRAPH statement generates a *.gr#* graph data file and sends this file directly to the default high resolution graphical device (to specify this device, set PRTDEFAULT in the *meta.cfg* configuration file).

Each .GRAPH statement creates a new *.gr#* file, where # ranges first from 0 to 9, and then from a to z. You can create up to 36 graph files. If you specify more than 36 .GRAPH statements, Star-Hspice overwrites the graph files, starting with the *.gr0* file. To overcome this limitation, use the ALT999 or ALT9999 option, to extend the number of digits allowed in the file name extension, to either *.gr####* (ALT999) or *.gr#####* (ALT9999), where # ranges from 0 to 9.

You can include wildcards in .GRAPH statements. For example:

```
.GRAPH TRAN V(9?t*u)
```

This example graphs the results of a transient analysis, for the voltage at the matched node name.

- The ? wildcard matches any single character. For example, 9? matches 92, 9a, 9A, and 9%.
- The * wildcard matches any string of zero or more characters. For example:
 - If your netlist includes a resistor named *r1* and a voltage source named *vin*, then *.graph i(*)* graphs the current for both of these elements: *i(r1)* and *i(vin)*.
 - *.graph v(o*)* graphs the voltages for all nodes whose names start with *o*; if your netlist contains nodes named *in* and *out*, this example graphs only the *v(out)* voltage.
 - If your netlist contains nodes named 0, 1, 2, and 3, then *.graph v(0,*)* or *.graph v(0 *)* graphs the voltage between node 0 and each of the other nodes: *v(0,1)*, *v(0,2)*, and *v(0,3)*.

Note: You cannot use .GRAPH statements in the PC version of Star-Hspice.

Syntax

```
.GRAPH antype <MODEL = mname> <unam1 = > ov1,  
+ <unam2 = >ov2 ...> (plo,phi)
```

where:

<i>antype</i>	Type of analysis for the specified plots. Analysis types are: DC, AC, TRAN, NOISE, or DISTO.
<i>mname</i>	Plot model name, referenced in the .GRAPH statement. Use .GRAPH and its plot name to create high-resolution plots directly from Star-Hspice.
<i>unam1</i> ...	You can define output names, which correspond to the <i>ov1 ov2 ...</i> output variables (<i>unam1 unam2 ...</i>), and use them as labels, instead of output variables, for a high resolution graphic output.
<i>ov1</i> ...	Output variables to print. These can be voltage, current, or element template variables, from a different type of analysis. You can also use algebraic expressions as output variables, but you must define them inside the PAR() statement.
<i>plo</i> , <i>phi</i>	Lower and upper plot limits. Set the plot limits only at the end of the .GRAPH statement.

.MODEL Statement for .GRAPH

This section describes the model statement for .GRAPH in Star-Hspice.

Syntax

```
.MODEL mname PLOT (pnam1 = val1 pnam2 = val2...)
```

<i>mname</i>	Plot model name, referenced in .GRAPH statements
<i>PLOT</i>	Keyword for a .GRAPH statement model
<i>pnam1</i> = <i>val1</i> ...	Each .GRAPH statement model includes several model parameters. If you do not specify model parameters, Star-Hspice uses the default values of the model parameters, described in the following table. Pnam <i>n</i> is one of the model parameters of a .GRAPH statement, and val <i>n</i> is the value of pnam <i>n</i> . Val <i>n</i> can be more than one parameter.

Example

```
.GRAPH DC cgb = lx18(m1) cgd = lx19(m1) cgs = lx20(m1)
.GRAPH DC MODEL = plotbjt
+ model_ib = i2(q1)      meas_ib = par(ib)
+ model_ic = i1(q1)      meas_ic = par(ic)
+ model_beta = par('i1(q1)/i2(q1)')
+ meas_beta = par('par(ic)/par(ib)')(1e-10,1e-1)
.MODEL plotbjt PLOT MONO = 1 YSCAL = 2 XSCAL = 2
+ XMIN = 1e-8 XMAX = 1e-1
```

Model Parameters

Name (Alias)	Default	Description
<i>FREQ</i>	0.0	Plots symbol frequency. ■ A value 0 does not generate plot symbols. ■ A value of n generates a plot symbol every n points.
<i>MONO</i>	0.0	Monotonic option. <i>MONO</i> = 1 automatically resets the x-axis, if any change occurs in the x direction.
<i>TIC</i>	0.0	Shows tick marks.
<i>XGRID</i> , <i>YGRID</i>	0.0	Set these values to 1.0, to turn on the axis grid lines.
<i>XMIN</i> , <i>XMAX</i>	0.0	■ If <i>XMIN</i> is not equal to <i>XMAX</i>, then <i>XMIN</i> and <i>XMAX</i> determine the x-axis plot limits. ■ If <i>XMIN</i> equals <i>XMAX</i>, or if you do not set <i>XMIN</i> and <i>XMAX</i>, then Star-Hspice automatically sets the plot limits. These limits apply to the actual x-axis variable value, regardless of the <i>XSCAL</i> type.
<i>XSCAL</i>	1.0	Scale for the x-axis. Two common axis scales are: Linear(LIN) (<i>XSCAL</i> = 1) Logarithm(LOG) (<i>XSCAL</i> = 2)
<i>YMIN</i> , <i>YMAX</i>	0.0	■ If <i>YMIN</i> is not equal to <i>YMAX</i>, then <i>YMIN</i> and <i>YMAX</i> determine the y-axis plot limits. ■ The y-axis limits in the .GRAPH statement, override <i>YMIN</i> and <i>YMAX</i> in the model. ■ If you do not specify plot limits then Star-Hspice automatically sets the plot limits. These limits apply to the actual y-axis variable value, regardless of the <i>YSCAL</i> type.
<i>YSCAL</i>	1.0	Scale for the y-axis. Two common axis scales are: Linear(LIN) (<i>YSCAL</i> = 1) Logarithm(LOG) (<i>YSCAL</i> = 2)

Using Wildcards in .PRINT, .PROBE, .PLOT, and .GRAPH Statements

You can include wildcards in .PRINT and .PROBE statements, and in .PLOT and .GRAPH statements. For example:

```
* test wildcard
.option post=2
v1 1 0 10
r1 1 n20 10
r20 n20 n21 10
r21 n21 0 10
.dc v1 1 10 1

***Wildcard equivalent for:
*.print i(r1) i(r20) i(r21) i(v1)
.print i(*)

***Wildcard equivalent for:
*.probe v(0) v(1)
.probe v(?)

***Wildcard equivalent for:
*.plot v(n20) v(n21)
.plot v(n2?)

***Wildcard equivalent for:
*.graph v(n20, 1) v(n21, 1)
.graph v(n2*, 1)

.end
```

Supported wildcard characters are:

- ? Matches any single character that Star-Hspice supports.
- * Matches zero or more characters that Star-Hspice supports.

Templates

The following are supported wildcard templates:

```
v vm vr vi vp vdb vt
i im ir ii ip idb it
p pm pr pi pp pdb pt
lxn<n> lvn<n> (n is a number 0~9)
i1 im1 ir1 ii1 ip1 idb1 it1
i2 im2 ir2 ii2 ip2 idb2 it2
i3 im3 ir3 ii3 ip3 idb3 it3
i4 im4 ir4 ii4 ip4 idb4 it4
iall
```

For detailed information about the templates, see [Selecting Simulation Output Parameters on page 8-25](#).

Examples

Using wildcards in statements such as `v(n2?)` and `v(n2*,1)` in the preceding test case (named `test wildcard`), you can also use the following in statements (they are *not* equivalent), if you use an `.ac` statement instead of a `.dc` statement:

```
vm(n2?) vr(n2?) vi(n2?) vp(n2?) vdb(n2?) vt(n2?)
vm(n2*,1) vr(n2*,1) vi(n2*,1) vp(n2*,1) vdb(n2*,1)
vt(n2*,1)
```

Using wildcards in statements such as `i(*)` in the preceding test case (named `test wildcard`), you can also use the following in statements (they are *not* equivalent), if you use an `.ac` statement instead of a `.dc` statement:

```
im(*) ir(*) ip(*) idb(*) it(*)
```

`iall` is an output template, for all branch currents of diode, BJT, JFET, or MOSFET output. For example, `iall(m*)` is equivalent to:

```
i1(m*) i2(m*) i3(m*) i4(m*).
```

Print Control Options

.OPTION CO for Printout Width

The number of output variables that print on a single line of output, is a function of the number of columns, which you use the `CO` option to set in Star-Hspice.

Typical values are `CO = 80` (the default) for narrow printouts, and `CO = 132` for wide printouts. You can set up to five output variables per 80-column output, and up to eight output variables per 132-column output, with twelve characters per column. Star-Hspice automatically creates additional print statements and tables, for all output variables beyond the number that the `CO` option specifies.

.WIDTH Statement

You can use the `.WIDTH` statement to define the print-out width in Star-Hspice.

Syntax

```
.WIDTH OUT = {80 |132}
```

where *OUT* is the output print width

Example

```
.WIDTH OUT = 132 $ SPICE compatible style
.OPTION CO = 132 $ preferred style
```

Permissible values for *OUT* are 80 and 132. You can also use the *CO* option to set the *OUT* value.

.OPTION ALT999 or ALT9999, to Extend Output File Name

The output files for a postprocessor (from `.OPTION POST` in Star-Hspice) or `.GRAPH` statements have unique extensions `.xx#`, where:

- *xx* is a two-character text string, to denote the output type (see [Simulation Output on page 8-1](#) for more information).
- *#* is an alphanumeric character, that denotes the `.ALTER` number of the current simulation. This limits the total number of `.ALTER` statements in a netlist to 36, before the outputs begin overwriting the current files.

The `ALT999` and `ALT9999` options extend the output file name suffix to `.xx####` and `.xx#####`, respectively, where *#* represents a numerical character (0 to 9) only. Use this syntax to include up to 1000 or 10,000 `.ALTER` statements in the input netlist, which creates a unique file name for each output file.

.OPTION INGOLD for Printout Numerical Format

By default, Star-Hspice prints variable values in engineering notation:

<code>F = 1e-15</code>	<code>M = 1e-3</code>
<code>P = 1e-12</code>	<code>K = 1e3</code>
<code>N = 1e-9</code>	<code>X = 1e6</code>
<code>U = 1e-6</code>	<code>G = 1e9</code>

In contrast to exponential form, engineering notation provides two to three extra significant digits, and aligns columns to facilitate comparison. To obtain output in exponential form, specify `INGOLD = 1` or `2`, with an `.OPTION` statement.

<code>INGOLD = 0</code> (default)	Engineering Format	1.234K 123M
<code>INGOLD = 1</code>	G Format (fixed and exponential)	1.234e+03 .123
<code>INGOLD = 2</code>	E Format (exponential SPICE)	1.234e+03 .123e-1

.OPTION POST for High Resolution Graphics

Use an `.OPTION POST` statement to display high-resolution AvanWaves plots of simulation results, on either a graphics terminal or a high-resolution laser printer. Use `.OPTION POST` to provide output, without specifying other parameters. `POST` has defaults, which supply usable data to most parameters.

<code>POST = 0,1,BINARY</code>	Output format is binary.
<code>POST = 2,ASCII</code>	Output format is ASCII.

.OPTION ACCT Summary of Job Statistics

The `ACCT` option in Star-Hspice generates a detailed accounting report, where:

<code>.OPTION ACCT</code>	Enables reporting.
<code>.OPTION ACCT = 1</code> (default)	Is the same as <code>ACCT</code> , without arguments.
<code>.OPTION ACCT = 2</code>	Enables reporting, and matrix statistic reporting.

The following output example appears at the end of an output listing.

```
**** job statistics summary tnom = 25.000 temp = 25.000
# nodes = 15 # elements = 29 # real*8
mem avail/used = 333333/13454
# diodes = 0 # bjts = 0 # jfets = 0 # mosfets = 24
  analysis   time   # points  tot. iter  conv.iter
  op point   0.24    1         11
  transient  5.45   161        265      103 rev = 1
  pass1      0.08
  readin     0.12
  errchk     0.05
  setup      0.04
  output     0.00
```

The analysis time includes the following time statistics:

```
  load       5.22
  solver      0.16
# external nodes = 15 # internal nodes = 0
# branch currents = 5 total matrix size = 20
pivot based and non pivoting solution times
non pivoting: ---- decompose 0.08 solve 0.08
matrix size(109) = initial size(105) + fill(4)
words copied = 111124
total cpu time 6.02 seconds
job started at 11:54:11 21-sep92
job ended   at 11:54:36 21-sep92
```

The definitions for the items in the previous listing follow:

# BJTS	Number of bipolar transistors in the circuit.
# ELEMENTS	Total number of elements.
# JFETS	Number of JFETs in the circuit.
# MOSFETS	Number of MOSFETs in the circuit.
# NODES	Total number of nodes.
# POINTS	Number of transient points that you specify in the .TRAN statement. JTRFLG is usually at least 50, unless you set the DELMAX option.

<i>CONV.ITER</i>	Number of points that the simulator needs, to preserve the accuracy that the tolerances specify.
<i>DC</i>	DC operating point analysis time, and number of iterations required. The <i>ITL1</i> option sets the maximum number of iterations.
<i>ERRCHK</i>	Part of the input processing.
<i>MEM +</i>	Amount of workspace available, and amount used in the simulation.
<i>AVAILUSED</i>	Measured in 64-bit (8-byte) words.
<i>OUTPUT</i>	Time required, to process all prints and plots.
<i>LOAD</i>	Constructs the matrix equation.
<i>SOLVER</i>	Solves equations.
<i>PASS1</i>	Part of the input processing.
<i>READIN</i>	Specifies the input reader reads the user data file and any additional library files, and generates an internal representation of the information.
<i>REV</i>	Number of times that the simulator had to cut time (reversals). This measures how difficult the design is to simulate.
<i>SETUP</i>	Constructs a sparse matrix pointer system.
<i>TOTAL JOB TIME</i>	Total amount of CPU time required, to process the simulation. This is not the amount of actual (clock) time used to simulate, and can differ slightly from run to run, even if the runs are identical.

The ratio of *TOT . ITER* to *CONV . ITER* is the best measure of simulator efficiency. The theoretical ratio is 2:1. In this example the ratio was 2.57:1. SPICE generally has a ratio of anywhere from 3:1 to 7:1.

In transient analysis, the ratio of CONV.ITER to # POINTS is the measure of the number of points evaluated, to the number of points printed. If this ratio is greater than about 4:1, the convergence and time step control tolerances might be too tight for the simulation.

Changing the File Descriptor Limit

A simulation that uses a large number of .ALTER statements might fail, because of the limit on the number of file descriptors. For example, for a Sun workstation, the default number of file descriptors is 64, so a design with more than 50 .ALTER statements probably fails, with the following error message:

```
error could not open output spool file /tmp/tmp.nnn
a critical system resource is inaccessible or exhausted
```

To prevent this error on a Sun workstation, enter the following operating system command, before you start the simulation:

```
limit descriptors 128
```

For platforms other than Sun workstations, ask your system administrator to help you increase the number of files that you can open concurrently.

Printing the Subcircuit Output

The following examples demonstrate how to print or plot voltages of nodes that are in subcircuit definitions, using .PRINT, .PLOT, .PROBE, or .GRAPH.

Note: In the following example, you can substitute .PROBE, .PLOT, or .GRAPH for .PRINT.

Example 1

```
.GLOBAL vdd vss
X1 1 2 3 nor2
X2 3 4 5 nor2
.SUBCKT nor2 A B Y
.PRINT v(B) v(N1) $ Print statement 1
M1 N1 A vdd vdd pch w = 6u l = 0.8u
M2 Y B N1 vdd pch w = 6u l = 0.8u
M3 Y A vss vss vss nch w = 3u l = 0.8u
M4 Y B vss vss nch w = 3u l = 0.8u
.ENDS
```

Print statement 1 invokes a printout of the voltage on the B input node, and on the N1 internal node, for every instance of the nor2 subcircuit.

```
.PRINT v(1) v(X1.A) $ Print statement 2
```

The .PRINT statement above specifies two ways to print the voltage on the A input of the X1 instance.

```
.PRINT v(3) v(X1.Y) v(X2.A) $ Print statement 3
```

This print statement specifies three different ways to print the voltage at the Y output of the X1 instance (or the A input of the X2 instance).

```
.PRINT v(X2.N1) $ Print statement 4
```

The print statement above prints out the voltage on the N1 internal node of the X2 instance.

```
.PRINT i(X1.M1) $ Print statement 5
```

The print statement above prints out the drain-to-source current, through the M1 MOSFET in the X1 instance.

Example 2

```
X1 5 6 YYY
.SUBCKT YYY 15 16
X2 16 36 ZZZ
R1 15 25 1
R2 25 16 1
.ENDS
.SUBCKT ZZZ 16 36
C1 16 0 10P
R3 36 56 10K
C2 56 0 1P
.ENDS
.PRINT V(X1.25) V(X1.X2.56) V(6)
```

The .PRINT statement voltages are:

V(X1.25)	Local node to the YYY subcircuit definition, which the X1 subcircuit calls.
V(X1.X2.56)	Local node to the ZZZ subcircuit definition, which the X2 subcircuit calls. In turn, X1 calls X2.
V(6)	Represents the voltage of node 16, in the X1 instance of the YYY subcircuit.

This example prints analysis results for the voltage at node 56, within the X2 and X1 subcircuits. The full path name, X1.X2.56, specifies that node 56 is within the X2 subcircuit, which in turn is within the X1 subcircuit.

Selecting Simulation Output Parameters

This section explains how to define parameters, that provide the appropriate simulation output. To define simulation parameters, use the `.OPTION` and `.MEASURE` statements, and define specific variable elements.

DC and Transient Output Variables

Some types of output variables for DC and transient analyses are:

- Voltage differences between specified nodes (or between one specified node and ground).
- Current output, for an independent voltage source.
- Current output, for any element.
- Element templates. For each device type, these templates contain:
 - values of variables that you set
 - state variables
 - element charges
 - capacitance currents
 - capacitances
 - derivatives

[Print Control Options on page 8-17](#) summarizes the codes that you can use, to specify the element templates for output in Star-Hspice.

Nodal Voltage Syntax

$V (n1<, n2>)$

$n1, n2$ Star-Hspice prints or plots the voltage difference ($n1-n2$) between the specified nodes. If you omit $n2$, Star-Hspice prints or plots the voltage difference between $n1$ and ground (node 0).

Current: Voltage Sources

Syntax

I (Vxxx)

where:

Vxxx Voltage source element name. If an independent power supply is within a subcircuit, then to access its current output, append a dot and the subcircuit name to the element name. For example, I(X1.Vxxx).

Example

```
.PLOT TRAN I(VIN)
.PRINT DC I(X1.VSRC)
.PLOT DC I(XSUB.XSUBSUB.VY)
```

Current: Element Branches

Syntax

In (Wwww)
Iall (Wwww)

where:

n Node position number, in the element statement. For example, if the element contains four nodes, I3 is the branch current output for the third node. If you do not specify *n*, Star-Hspice assumes the first node.

Wwww Element name. If the element is within a subcircuit, then to access its current output, append a dot and the subcircuit name to the element name. For example, I3(X1.Wwww).

Iall An alias just for diode, BJT, JFET, and MOSFET devices.

(Wwww) ■ If Wwww is a diode, it is equivalent to:

I1(Wwww) I2(Wwww) .

■ If Wwww is one of the other device types, it is equivalent to:

I1(Wwww) I2(Wwww) I3(Wwww) I4(Wwww)

Example

`I1(R1)`

This example specifies the current, through the first node of the R1 resistor.

`I4(X1.M1)`

The above example specifies the current, through the fourth node (the substrate node) of the M1 MOSFET, which is defined in the X1 subcircuit.

`I2(Q1)`

The last example specifies the current, through the second node (the base node) of the Q1 bipolar transistor.

To define each branch circuit, use a single element statement. When Star-Hspice evaluates branch currents, it inserts a zero-volt power supply, in series with branch elements.

If Star-Hspice cannot interpret a `.PRINT` or `.PLOT` statement that contains a branch current, it generates a warning.

Branch current direction for the elements in [Figure 8-1](#) through [Figure 8-6](#) is defined in terms of arrow notation (current direction), and node position number (terminal type).

Figure 8-1: Resistor (node1, node2)

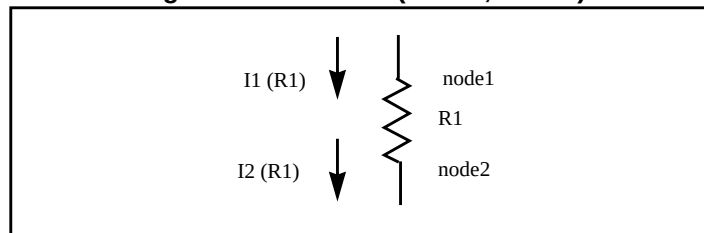


Figure 8-2: Capacitor (node1, node2); Inductor (node 1, node2)

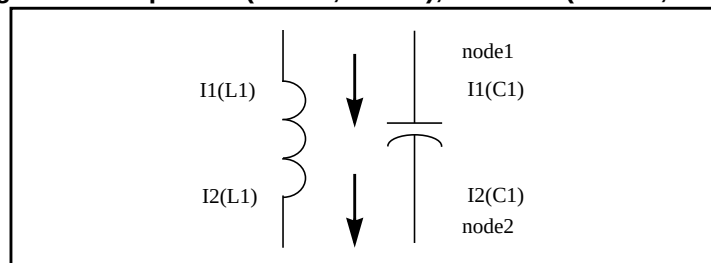
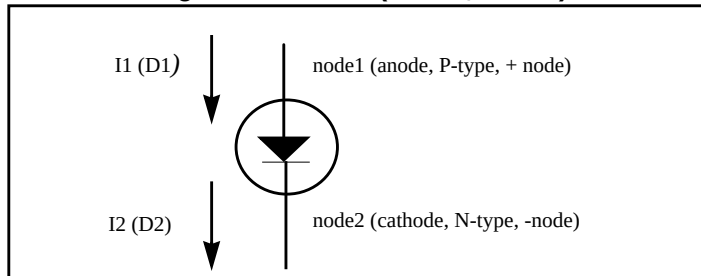
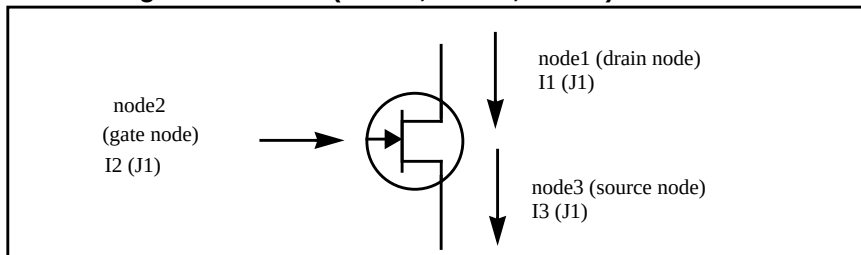
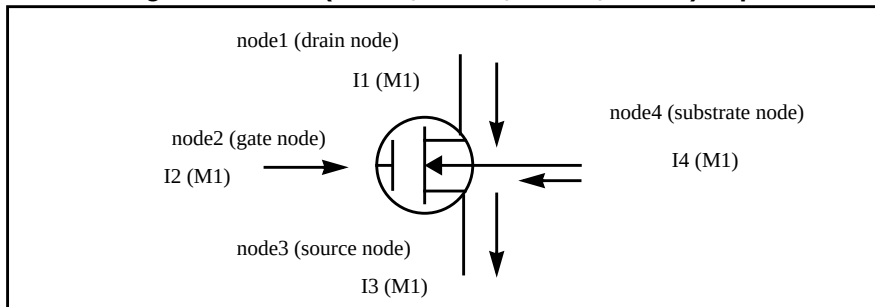
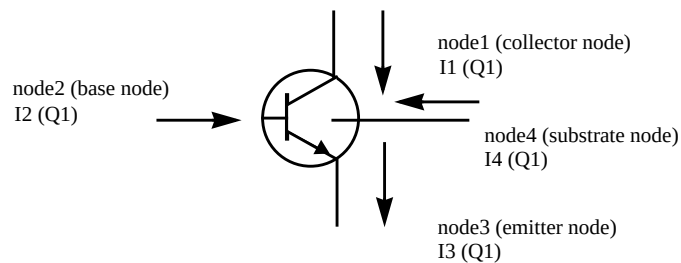


Figure 8-3: Diode (node1, node2)**Figure 8-4: JFET (node1, node2, node3) - n-channel****Figure 8-5: BJT (node1, node2, node3, node4) - npn****Figure 8-6: MOSFET (node1, node2, node3, node4) - n-channel**

Power Output

For power calculations, Star-Hspice computes dissipated or stored power in each passive element (R, L, C), and source (V, I, G, E, F, and H). To compute this power, Star-Hspice multiplies the voltage across an element, and its corresponding branch current.

However, for semiconductor devices, Star-Hspice calculates only the dissipated power. It excludes the power stored in the device junction or parasitic capacitances, from the device power computation. The following sections show equations for calculating the power that different types of devices dissipate.

Star-Hspice also computes the total power dissipated in the circuit, which is the sum of the power dissipated in:

- devices
- resistors
- independent current sources
- all dependent sources

For hierarchical designs, Star-Hspice also computes the power dissipation for each subcircuit.

Note: For the total power (dissipated power + stored power), Star-Hspice does not add the power of each independent source (voltage and current sources).

Print or Plot Power

To output the instantaneous element power, and the total power dissipation, use a `.PRINT` or `.PLOT` statement in Star-Hspice.

Syntax

```
.PRINT <DC | TRAN> P(element_or_subcircuit_name)POWER
```

Star-Hspice calculates power only for transient and DC sweep analyses. Use the `.MEASURE` statement to compute the average, rms, minimum, maximum, and peak-to-peak value of the power. The `POWER` keyword invokes the total power dissipation output.

Example

```
.PRINT TRAN      P(M1)      P(VIN)      P(CLOAD)      POWER
.PRINT TRAN      P(Q1)      P(DI0)      P(J10)      POWER
.PRINT TRAN      POWER      $ Total transient analysis
* power dissipation

.PLOT DC POWER   P(IIN)     P(RLOAD)  P(R1)
.PLOT DC POWER   P(V1)     P(RLOAD)  P(VS)

.PRINT TRAN P(Xf1) P(Xf1.Xh1)
```

Diode Power Dissipation

$$P_d = V_{pp'} \cdot (I_{do} + I_{cap}) + V_{p'n} \cdot I_{do}$$

P_d	Power dissipated in the diode.
I_{do}	DC component of the diode current.
I_{cap}	Capacitive component of the diode current.
$V_{p'n}$	Voltage across the junction.
$V_{pp'}$	Voltage across the series resistance, RS.

BJT Power Dissipation

■ Vertical

$$P_d = V_{c'e'} \cdot I_{co} + V_{b'e'} \cdot I_{bo} + V_{cc'} \cdot I_{ctot} + V_{ee'} \cdot I_{etot} + V_{sc'} \cdot I_{so} - V_{cc'} \cdot I_{stot}$$

■ Lateral

$$P_d = V_{c'e'} \cdot I_{co} + V_{b'e'} \cdot I_{bo} + V_{cc'} \cdot I_{ctot} + V_{bb'} \cdot I_{btot} + V_{ee'} \cdot I_{etot} + V_{sb'} \cdot I_{so} - V_{bb'} \cdot I_{stot}$$

I_{bo}	DC component of the base current.
I_{co}	DC component of the collector current.
I_{so}	DC component of the substrate current.

Pd	Power dissipated in a BJT.
Ibtot	Total base current (excluding the substrate current).
Ictot	Total collector current (excluding the substrate current).
Ietot	Total emitter current.
Istot	Total substrate current.
Vb'e'	Voltage across the base-emitter junction.
Vbb'	Voltage across the series base resistance, RB.
Vc'e'	Voltage across the collector-emitter terminals.
Vcc'	Voltage across the series collector resistance, RC.
Vee'	Voltage across the series emitter resistance, RE.
Vsb'	Voltage across the substrate-base junction.
Vsc'	Voltage across the substrate-collector junction.

JFET Power Dissipation

$$Pd = Vd's' \cdot Ido + Vgd' \cdot Igdo + Vgs' \cdot Igso + \\ Vs's \cdot (Ido + Igso + Icgs) + Vdd' \cdot (Ido - Igdo - Icgd)$$

Icgd	Capacitive component of the gate-drain junction current.
Icgs	Capacitive component of the gate-source junction current.
Ido	DC component of the drain current.
Igdo	DC component of the gate-drain junction current.
Igso	DC component of the gate-source junction current.
Pd	Power dissipated in a JFET.
Vd's'	Voltage across the internal drain-source terminals.

Vdd'	Voltage across the series drain resistance, RD.
Vgd'	Voltage across the gate-drain junction.
Vgs'	Voltage across the gate-source junction.
Vs's	Voltage across the series source resistance, RS.

MOSFET Power Dissipation

$$P_d = V_{d's'} \cdot I_{do} + V_{bd'} \cdot I_{bdo} + V_{bs'} \cdot I_{bso} + V_{s's} \cdot (I_{do} + I_{bso} + I_{cbs} + I_{cgs}) + V_{dd'} \cdot (I_{do} - I_{bdo} - I_{cbd} - I_{cgd})$$

Ibdo	DC component of the bulk-drain junction current.
Ibso	DC component of the bulk-source junction current.
Icbd	Capacitive component of the bulk-drain junction current.
Icbs	Capacitive component of the bulk-source junction current.
Icgd	Capacitive component of the gate-drain current.
Icgs	Capacitive component of the gate-source current.
Ido	DC component of the drain current.
Pd	Power dissipated in the MOSFET.
Vbd'	Voltage across the bulk-drain junction.
Vbs'	Voltage across the bulk-source junction.
Vd's'	Voltage across the internal drain-source terminals.
Vdd'	Voltage across the series drain resistance, RD.
Vs's	Voltage across the series source resistance, RS.

AC Analysis Output Variables

Output variables for AC analysis include:

- Voltage differences between specified nodes (or between one specified node and ground).
- Current output, for an independent voltage source.
- Element branch current.
- Impedance (Z), admittance (Y), hybrid (H), and scattering (S) parameters.
- Input and output impedance, and admittance.

Table 8-1 lists AC output variable types. In this table, the type symbol is appended to the variable symbol, to form the output variable name. For example, VI is the imaginary part of the voltage, or IM is the magnitude of the current.

Table 8-1: AC Output Variable Types

Type Symbol	Variable Type
DB	decibel
I	imaginary part
M	magnitude
P	phase
R	real part
T	group delay

Specify real or imaginary parts, magnitude, phase, decibels, and group delay, for voltages and currents.

Nodal Voltage

Syntax

Vx (n1, <, n2>)

where:

x	Specifies the voltage output type (see Table 8-1 on page 8-33)
n1, n2	Specifies node names. If you omit n2, Star-Hspice assumes ground (node 0).

Example 1

```
.PLOT AC VM(5) VDB(5) VP(5)
```

The above example plots the magnitude of the AC voltage of node 5, using the VM output variable. Star-Hspice uses the VDB output variable to plot the *voltage* at node 5, and uses the VP output variable to plot the *phase* of the nodal voltage at node 5.

To produce complex results, an AC analysis uses either the SPICE or Star-Hspice method, and the ACOUT control option, to calculate the values of real or imaginary parts, for complex voltages of AC analysis, and their magnitude, phase, decibel, and group delay values. The default for Star-Hspice is ACOUT = 1. To use the SPICE method, set ACOUT = 0.

A typical use of the SPICE method is to calculate the nodal vector difference, when comparing adjacent nodes in a circuit. You can use this method to find the phase or magnitude across a capacitor, inductor, or semiconductor device.

Use the Star-Hspice method to calculate an inter-stage gain in a circuit (such as an amplifier circuit), and to compare its gain, phase, and magnitude.

The following example defines the AC analysis output variables for the Star-Hspice method, and then for the SPICE method.

Example 2: Star-Hspice Method (ACOUT = 1, Default)

■ Real and imaginary:

```
VR(N1,N2) = REAL [V(N1,0)] - REAL [V(N2,0)]
VI(N1,N2) = IMAG [V(N1,0)] - IMAG [V(N2,0)]
```

■ Magnitude:

$$\begin{aligned} VM(N1, 0) &= [VR(N1, 0)^2 + VI(N1, 0)^2]^{0.5} \\ VM(N2, 0) &= [VR(N2, 0)^2 + VI(N2, 0)^2]^{0.5} \\ VM(N1, N2) &= VM(N1, 0) - VM(N2, 0) \end{aligned}$$

■ Phase:

$$\begin{aligned} VP(N1, 0) &= ARCTAN[VI(N1, 0)/VR(N1, 0)] \\ VP(N2, 0) &= ARCTAN[VI(N2, 0)/VR(N2, 0)] \\ VP(N1, N2) &= VP(N1, 0) - VP(N2, 0) \end{aligned}$$

■ Decibel:

$$VDB(N1, N2) = 20 \cdot \text{LOG}_{10}(VM(N1, 0)/VM(N2, 0))$$

Example 3: SPICE Method (ACOUT = 0)

■ Real and imaginary:

$$\begin{aligned} VR(N1, N2) &= \text{REAL} [V(N1, 0) - V(N2, 0)] \\ VI(N1, N2) &= \text{IMAG} [V(N1, 0) - V(N2, 0)] \end{aligned}$$

■ Magnitude:

$$VM(N1, N2) = [VR(N1, N2)^2 + VI(N1, N2)^2]^{0.5}$$

■ Phase:

$$VP(N1, N2) = ARCTAN[VI(N1, N2)/VR(N1, N2)]$$

■ Decibel:

$$VDB(N1, N2) = 20 \cdot \text{LOG}_{10}[VM(N1, N2)]$$

Current: Independent Voltage Sources

Syntax

Iz (Vxxx)

where:

- z Current output type (see [Table 8-1 on page 8-33](#)).
- Vxxx Voltage source element name. If an independent power supply is within a subcircuit, then to access its current output, append a dot and the subcircuit name to the element name. For example, IM(X1.Vxxx).

Example

```
.PLOT AC IR(V1) IM(VN2B) IP(X1.X2.VSRC)
```

Current: Element Branches**Syntax**

```
IZn (Wwww)
```

where:

- z* Current output type (see [Table 8-1 on page 8-33](#)).
- n* Node position number, in the element statement. For example, if the element contains four nodes, IM3 denotes the magnitude of the branch current output, for the third node.
- Wwww* Element name. If the element is within a subcircuit, then to access its current output, append a dot and the subcircuit name to the element name. For example, IM3(X1.Wwww).

Example

```
.PRINT AC IP1(Q5) IM1(Q5) IDB4(X1.M1)
```

If you use the form *IN*(XXXX) for AC analysis output, then Star-Hspice prints the magnitude value, *IMn*(XXXX).

Group Time Delay

The group time delay, *TD*, is associated with AC analysis. *TD* is the negative derivative of phase, in radians, with respect to radian frequency. Star-Hspice uses the difference method to compute *TD*, as follows:

$$TD = -\frac{1}{360} \cdot \frac{(phase2 - phase1)}{(f2 - f1)}$$

where *phase1* and *phase2* are the phases (in degrees) of the specified signal, at the *f1* and *f2* frequencies (in Hertz).

Syntax

```
.PRINT AC VT(10) VT(2, 25) IT(RL)
.PLOT AC IT1(Q1) IT3(M15) IT(D1)
```

Note: Because the phase has a discontinuity every 360°, TD shows the same discontinuity, even though TD is continuous.

Example

```
INTEG.SP ACTIVE INTEGRATOR
***** INPUT LISTING
*****
V1      1      0      .5      AC      1
R1      1      2      2K
C1      2      3      5NF
E3      3      0      2 0 -1000.0

.AC DEC      15      1K      100K
.PLOT AC      VT(3)      (0, 4U)      VP(3)
.END
```

Network**Syntax**

$$X_{ij}(z), ZIN(z), ZOUT(z), YIN(z), YOUT(z)$$

where:

- X Specifies Z for impedance, Y for admittance, H for hybrid, or S for scattering parameters.
- ij i and j can be 1 or 2. They identify the matrix parameter to print.
- z Output type (see [Table 8-1 on page 8-33](#)). If you omit z, Star-Hspice prints the magnitude of the output variable.
- ZIN Input impedance. For a one-port network, ZIN, Z11, and H11 are the same.
- $ZOUT$ Output impedance.

YIN Input admittance. For a one-port network, *YIN* and *Y11* are the same.

YOUT Output admittance.

Example

```
.PRINT AC Z11(R) Z12(R) Y21(I) Y22 S11 S11(DB)
.PRINT AC ZIN(R) ZIN(I) YOUT(M) YOUT(P) H11(M)
.PLOT AC S22(M) S22(P) S21(R) H21(P) H12(R)
```

Noise and Distortion

This section describes the variables used for noise and distortion analysis.

Syntax

ovar <(z)>

where:

ovar Noise and distortion analysis parameter. It can be *ONoise* (output noise), *INoise* (equivalent input noise), or any of the distortion analysis parameters (*HD2*, *HD3*, *SIM2*, *DIM2*, *DIM3*).

z Output type (only for distortion). If you omit *z*, Star-Hspice outputs the magnitude of the output variable.

Example

```
.PRINT DISTO HD2(M) HD2(DB)
```

Prints the magnitude and decibel values of the second harmonic distortion component, through the load resistor that you specified in the *.DISTO* statement (not shown).

```
.PLOT NOISE INoise ONoise
```

Note: You can specify the noise and distortion output variable, and other AC output variables, in the *.PRINT AC* or *.PLOT AC* statements.

Element Template Output

.PRINT and .PROBE, or .PLOT and .GRAPH statements use element templates to output user-input parameters, state variables, stored charges, capacitor currents, capacitances, and derivatives of variables. The Star-Hspice element templates are listed at the end of this chapter.

Syntax

Elname:Property

<i>Elname</i>	Name of the element.
<i>Property</i>	Property name of an element, such as a user-input parameter, state variable, stored charge, capacitance current, capacitance, or derivative of a variable.

The alias is:

LVnn(Elname)

or

LXnn(Elname)

<i>LV</i>	Form to obtain output of user-input parameters, and state variables.
<i>LX</i>	Form to obtain output of stored charges, capacitor currents, capacitances, and derivatives of variables.
<i>nn</i>	Code number for the desired parameter, as listed in the tables in this section.
<i>Elname</i>	Name of the element.

Example

```
.PLOT TRAN V(1,12) I(X2.VSIN) I2(Q3) DI01:GD
.PRINT TRAN X2.M1:CGGB0 M1:CGDB0 X2.M1:CGSBO
```

Specifying User-Defined Analysis (.MEASURE)

Use the .MEASURE statement to modify information, and to define the results of successive Star-Hspice simulations. The .MEASURE statement prints user-defined electrical specifications of a circuit. Optimization uses .MEASURE statements extensively. The specifications include:

- propagation
- delay
- rise time
- fall time
- peak-to-peak voltage
- minimum and maximum voltage over a specified period
- other user-defined variables

You can also use .MEASURE with either the error function or GOAL parameter, to optimize circuit component values, and to curve-fit measured data to model parameters.

Computing the measurement results is based on postprocessing output. If you use the INTERP option to reduce the size of the postprocessing output, then the measurement results can contain interpolation errors. See [Input and Output Options on page 9-51](#) for more information about the INTERP option.

The .MEASURE statement can use several different formats, depending on the application. You can use it for either DC sweep, AC, or transient analysis.

Fundamental measurement modes in Star-Hspice are:

- Rise, fall, and delay
- Find-when
- Equation evaluation
- Average, RMS, min, max, and peak-to-peak
- Integral evaluation
- Derivative evaluation
- Relative error

If a .MEASURE statement does not execute, then Star-Hspice writes 0.0e0 in the .mt# file as the .MEASURE result, and writes FAILED in the output listing file. Use the MEASFAIL option to write results to the .mt#, .ms#, or .ma# files. See [Input and Output Options on page 9-51](#) for information about the MEASFAIL option.

To control the output variables, listed in .MEASURE statements, use the .PUTMEAS option. See [Input and Output Options on page 9-50](#) for more information.

.MEASURE Performance

If you specify a large number of .measure statements, Star-Hspice might not complete for several minutes, or several hours. Overall simulation run time depends on the number of .measure statements to process for each iteration, and the number of iterations required to achieve convergence.

To reduce simulation run time, place similar variables together, when you list them in the .measure statement.

Examples

Example 1 - Original Case (Slower, due to repeated switching between the v1 and v2 variables):

```
.meas tran val1 AVG v(1) FROM=0ms TO=50ms
.meas tran val2 AVG v(2) FROM=0ms TO=50ms
.meas tran val3 AVG v(1) FROM=50ms TO=100ms
.meas tran val4 AVG v(2) FROM=50ms TO=100ms
```

Example 2 - Improved Case (Faster):

```
.meas tran val1 AVG v(1) FROM=0ms TO=50ms
.meas tran val3 AVG v(1) FROM=50ms TO=100ms
.meas tran val2 AVG v(2) FROM=0ms TO=50ms
.meas tran val4 AVG v(2) FROM=50ms TO=100ms
```

The second example lists all v(1) variables consecutively, followed by all v(2) variables. In this second case, Star-Hspice applies all measurements to a single variable (v1) at the same time. This reduces overall simulation run time, compared to switching back to the same variable repeatedly, when you do **not** sort the .measure list by variable name.

To automatically sort large numbers of `.measure` statements in this way, use the `.option meassort` statement.

Syntax

`.option meassort=0` (the default; does not sort `.measure` statements)
`.option meassort=1` (internally sorts `.measure` statements)

Set this option to 1 only if you use a large number of `.measure` statements, where you need to list similar variables together, to reduce simulation run time. For a small number of `.measure` statements, turning on internal sorting might slow-down the simulation while sorting, compared to **not** sorting first.

.MEASURE Parameter Types

Measurement parameter results produced by `.PARAM` statements in `.SUBCKT` blocks produce measurement results, but you cannot use those results outside of the subcircuit. That is, you cannot pass any measurement parameters defined in `.SUBCKT` statements, as bottom-up parameters in hierarchical designs.

Measurement parameter names must not conflict with standard parameter names. Star-Hspice issues an error message, if it encounters a measurement parameter with the same name as a standard parameter definition.

To prevent parameter values in `.MEASURE` statements from overwriting parameter assignments in other statements, Star-Hspice keeps track of parameter types. If you use the same parameter name in both a `.MEASURE` statement and a `.PARAM` statement at the same hierarchical level, simulation terminates and reports an error. No error occurs if the parameter assignments are at different hierarchical levels. `PRINT` statements that occur at different levels, do not print hierarchical information for parameter name headings.

The following example illustrates how Star-Hspice handles .MEASURE statement parameters.

```

...
.MEASURE tran length TRIG v(clk) VAL = 1.4
+ TD = 11ns RISE = 1 TARGv(neq) VAL = 1.4 TD = 11ns
+ RISE = 1
.SUBCKT path out in width = 0.9u length = 600u
+ rm1 in m1 m2mg w = 'width' l = 'length/6'
...
.ENDS

```

In the above listing, the *length* in the resistor statement:

```
rm1 in m1 m2mg w = 'width' l = 'length/6'
```

does not inherit its value from the *length* in the .MEASURE statement:

```
.MEASURE tran length ...
```

because they are of different types.

The correct value of *l* in *rm1* should be:

```
l = length/6 = 100u
```

instead of a value derived from the measured value in transient analysis.

.MEASURE Statement: Rise, Fall, and Delay

Use this format to measure independent-variable (time, frequency, or any parameter or temperature) differential measurements such as rise time, fall time, slew rate, and any measurement that requires the determination of independent variable values. The format specifies substatements TRIG and TARG. These two statements specify the beginning and ending of a voltage or current amplitude measurement.

The rise, fall, and delay measurement mode computes the time, voltage, or frequency between a trigger value and a target value. Examples for transient analysis include rise/fall time, propagation delay, and slew rate measurement. Applications for AC analysis are the measurement of the bandwidth of an amplifier or the frequency at which a certain gain is achieved.

Syntax

```
.MEASURE <DC|AC|TRAN> result TRIG ... TARG ...  
+ <GOAL = val> <MINVAL = val> <WEIGHT = val>
```

where:

<i>MEASURE</i>	Specifies measurements. You can abbreviate to MEAS.
<i>result</i>	Name associated with the measured value, in the Star-Hspice output. This example measures the independent variable, beginning at the trigger, and ending at the target: ■ Transient analysis measures time. ■ AC analysis measures frequency. ■ DC analysis measures the DC sweep variable. If simulation reaches the target before the trigger activates, the resulting value is negative. Do not use DC, TRAN, or AC as the <i>result</i> name.
<i>TRIG...</i>	Identifies the beginning of trigger specifications.
<i>TARG ...</i>	Identifies the beginning of target specifications.
<DC AC TRAN>	Specifies the analysis type of the measurement. If you omit this parameter, Star-Hspice uses the last analysis mode that you requested.
<i>GOAL</i>	Specifies the desired measure value in ERR calculation for optimization. To calculate the error, the simulation uses the equation: $\text{ERRfun} = (\text{GOAL} - \text{result}) / \text{GOAL} .$
<i>MINVAL</i>	If the absolute value of GOAL is less than MINVAL, the MINVAL replaces the GOAL value, in the denominator of the ERRfun expression. Used only in ERR calculation for optimization. Default = 1.0e-12.

WEIGHT Multiplies the calculated error by the weight value. Used only in ERR calculation for optimization. Default = 1.0.

You can use the LAST keyword in *TARG_SPEC* to indicate the last event. *TRIG_SPEC* and *TARG_SPEC* can also use the syntax:

```
TRIG AT = time
```

Trigger

```
TRIG trig_var VAL = trig_val <TD = time_delay>
+ <CROSS = c> <RISE = r> <FALL = f>
```

or

```
TRIG AT = val
```

Target

```
TARG targ_var VAL = targ_val <TD = time_delay>
+ <CROSS = c | LAST> <RISE = r | LAST> <FALL = f | LAST>
```

where:

<i>TRIG</i>	Indicates the beginning of the trigger specification.
<i>trig_val</i>	Value of <i>trig_var</i> , which increments the counter for crossings, rises, or falls, by one.
<i>trig_var</i>	Specifies the name of the output variable, which determines the logical beginning of the measurement. If Star-Hspice reaches the target before the trigger activates, .MEASURE reports a negative value.
<i>TARG</i>	Indicates the beginning of the target signal specification.
<i>targ_val</i>	Specifies the value of the <i>targ_var</i> , which increments the counter for crossings, rises, or falls, by one.
<i>targ_var</i>	Name of the output variable, at which Star-Hspice determines the propagation delay with respect to the <i>trig_var</i> .

time_delay Amount of simulation time that must elapse, before Star-Hspice enables the measurement. The simulation counts the number of crossings, rises, or falls, only after the *time_delay* value. The default trigger delay is zero.

CROSS = c The numbers indicate which occurrence of a CROSS,
RISE = r FALL, or RISE event causes Star-Hspice to measure. For
FALL = f example:

```
.meas tran tdlay trig v(1) val=1.5 td=10n  
+ rise=2 targ v(2) val=1.5 fall=2
```

In the above example, *rise=2* specifies to measure the *v(1)* voltage, only on the first two rising edges of the waveform. The value of these first two rising edges is 1. However, *trig v(1) val=1.5* indicates to trigger when the voltage on the rising edge voltage is 1.5, which never occurs on these first two rising edges. So the *v(1)* voltage measurement never finds a trigger.

- *RISE = r*, the WHEN condition is met, and measurement occurs after the designated signal has risen *r* rise times.
- *FALL = f*, measurement occurs when the designated signal has fallen *f* fall times.
- A crossing is either a rise or a fall, so for *CROSS = c*, measurement occurs when the designated signal has achieved a total of *c* crossing times, as a result of either rising or falling.
- For TARG, the LAST keyword specifies the last event.

LAST

Star-Hspice measures when the last CROSS, FALL, or RISE event occurs.

- CROSS = LAST, measurement occurs the last time the WHEN condition is true, for a rising *or* falling signal.
- FALL = LAST, measurement occurs the last time the WHEN condition is true, for a *falling* signal.
- RISE = LAST, measurement occurs the last time the WHEN condition is true, for a *rising* signal.
- LAST is a reserved word; you cannot use it as a parameter name in the above .MEASURE statements.

AT = val

Special case for trigger specification. *val* is:

- the time for TRAN analysis
 - the frequency for AC analysis
 - the parameter for DC analysis
- at which measurement starts.

Star-Hspice Example

```
.MEASURE TRAN tdlay TRIG V(1) VAL = 2.5 TD = 10n
+ RISE = 2 TARG V(2) VAL = 2.5 FALL = 2
```

This example measures propagation delay between nodes 1 and 2, for a transient analysis. Star-Hspice measures the delay from the second rising edge of the voltage at node 1, to the second falling edge of node 2. The measurement begins when the second rising voltage at node 1 is 2.5 V, and ends when the second falling voltage at node 2 is 2.5 V. The TD = 10n parameter counts the crossings, after 10 ns has elapsed. Star-Hspice prints the results as tdlay = <value>.

```
.MEASURE TRAN riset TRIG I(Q1) VAL = 0.5m RISE = 3
+ TARG I(Q1) VAL = 4.5m RISE = 3

.MEASURE pwidth TRIG AT = 10n TARG V(IN) VAL = 2.5
+ CROSS = 3
```

In the last example, TRIG. AT = 10n starts measuring time at $t = 10$ ns, in the transient analysis. The TARG parameters end time measurement, when $V(IN) = 2.5$ V, on the third crossing. *pwidth* is the printed output variable.

Note: If you use the `.TRAN` statement with a `.MEASURE` statement, do not use a non-zero `START` time in `.TRAN` statement, or the `.MEASURE` results might be incorrect.

Average, RMS, and Peak Measurements

This `.MEASURE` statement reports the average, RMS, or peak value of the specified output variable.

Syntax

```
.MEASURE < TRAN > varname func var FROM = start TO = end
```

where:

varname Is the user-defined variable name for the measurement.

func Is one of the following keywords:

AVG: Average: calculates the area under *var*, divided by the period of interest.

MAX: Maximum: reports the maximum value of *var* over the specified interval.

MIN: Minimum: reports the minimum value of *var* over the specified interval.

PP: Peak-to-peak: reports the maximum value, minus the minimum of *var* over the specified interval.

RMS: Root mean squared: calculates the square root of the area under the var^2 curve, divided by the period of interest.

INTEG: Integral: reports the integral of *var* over the specified period.

var Is the name of the output variable, which can be either the node voltage or the branch current of the circuit. You can also use an expression, consisting of the node voltages or the branch current.

start Is the starting time of the measurement period.

end Is the ending time of the measurement period.

Example

1. In the example below, the `.MEASURE` statement calculates the RMS voltage of the OUT node, from 0ns to 10ns. It then labels the result RMSVAL:
`.MEAS TRAN RMSVAL RMS V(OUT) FROM = 0NS TO = 10NS`
2. In the example below, the `.MEASURE` statement finds the maximum current of the VDD voltage supply, between 10ns and 200ns in the simulation. The result is called MAXCUR.
`.MEAS MAXCUR MAX I(VDD) FROM = 10NS TO = 200NS`
3. In the example below, the `.MEASURE` statement uses the ratio of `V(OUT)` and `V(IN)` to find the peak-to-peak value, during the interval of 0ns to 200ns.
`.MEAS P2P PP PAR('V(OUT)/V(IN)') FROM = 0NS TO = 200NS`

FIND and WHEN Functions

The FIND and WHEN functions specify to measure:

- Any independent variables (time, frequency, parameter).
- Any dependent variables (voltage or current, for example).
- The derivative of any dependent variable, when some specific event occurs.

You can use these measure statements in unity gain frequency or phase measurements. You can also use these statements to measure the time, frequency, or any parameter value:

- when two signals cross each other.
- when a signal crosses a constant value.

The measurement starts after a specified time delay, TD. To find a specific event, set RISE, FALL, or CROSS to a value (or parameter), or specify LAST for the last event. LAST is a reserved word; you cannot use it as a parameter name in the above measure statements. For definitions of parameters of the measure statement, see [Displaying Simulation Results on page 8-4](#).

Syntax

```
.MEASURE <DC|TRAN|AC> result WHEN out_var = val <TD = val>
+ < RISE = r | LAST > < FALL = f | LAST > < CROSS = c | LAST >
+ <GOAL = val> <MINVAL = val> <WEIGHT = val>
```

or

```
.MEASURE <DC|TRAN|AC> result WHEN out_var1 = out_var2
+ < TD = val > < RISE = r | LAST > < FALL = f | LAST >
+ < CROSS = c | LAST > <GOAL = val> <MINVAL = val>
+ <WEIGHT = val>
```

or

```
.MEASURE <DC|TRAN|AC> result FIND out_var1
+ WHEN out_var2 = val < TD = val > < RISE = r | LAST >
+ < FALL = f | LAST > < CROSS = c | LAST > <GOAL = val>
+ <MINVAL = val> <WEIGHT = val>
```

or

```
.MEASURE <DC|TRAN|AC> result FIND out_var1
+ WHEN out_var2 = out_var3 <TD = val > < RISE = r | LAST >
+ < FALL = f | LAST > <CROSS = c | LAST> <GOAL = val>
+ <MINVAL = val> <WEIGHT = val>
```

or

```
.MEASURE <DC|TRAN|AC> result FIND out_var1 AT = val
+ <GOAL = val> <MINVAL = val> <WEIGHT = val>
```

Parameter Definitions

<i>CROSS</i> = <i>c</i> <i>RISE</i> = <i>r</i> <i>FALL</i> = <i>f</i>	The numbers indicate which occurrence of a CROSS, FALL, or RISE event starts measuring. ■ For <i>RISE</i> = <i>r</i>, after the designated signal rises <i>r</i> rise times, the WHEN condition is met, and measurement begins. ■ For <i>FALL</i> = <i>f</i>, measurement starts when the designated signal has fallen <i>f</i> fall times. ■ A crossing is a rise or a fall. For <i>CROSS</i> = <i>c</i>, measurement starts when the designated signal has achieved a total of <i>c</i> crossing times, as a result of either rising or falling.
<DC AC TRAN>	Specifies the analysis type for the measurement. If you omit this parameter, Star-Hspice assumes the last analysis type that you requested.
<i>FIND</i>	Selects the FIND function.
<i>GOAL</i>	Specifies the desired .MEASURE value. Optimization uses this value in ERR calculation. The following equation calculates the error: $\text{ERRfun} = (\text{GOAL} - \text{result}) / \text{GOAL}$
<i>LAST</i>	Starts measurement at the last CROSS, FALL, or RISE event. ■ For <i>CROSS</i> = <i>LAST</i>, measurement starts the last time the WHEN condition is true, for either a rising or falling signal. ■ For <i>FALL</i> = <i>LAST</i>, measurement starts the last time the WHEN condition is true, for a falling signal. ■ For <i>RISE</i> = <i>LAST</i>, measurement starts the last time the WHEN condition is true for a rising signal. LAST is a reserved word. Do not use it as a parameter name in these .MEASURE statements.
<i>MINVAL</i>	If the absolute value of GOAL is less than MINVAL, then MINVAL replaces the GOAL value in the denominator of the ERRfun expression. Used only in ERR calculation for optimization. Default = 1.0e-12.

<i>out_var(1,2,3)</i>	These variables establish conditions that start a measurement.
<i>result</i>	Name associated with the measured value, in the Star-Hspice output.
<i>TD</i>	Identifies the time at which measurement starts.
<i>WEIGHT</i>	Multiplies the calculated error by the weight value. Used only in ERR calculation for optimization. Default = 1.0.
<i>WHEN</i>	Selects the WHEN function.

Example

In the example below, the first measurement, TRT, calculates the difference between $V(3)$ and $V(4)$, when $V(1)$ is half the voltage of $V(2)$ at the last rise event.

The second measurement, STIME, finds the time when $V(4)$ is 2.5V at the third rise-fall event. A CROSS event is either a rising or a falling edge.

```
.MEAS TRAN TRT FIND PAR('V(3)-V(4)')
WHEN V(1)=PAR('V(2)/2') + RISE = LAST
.MEAS STIME WHEN V(4) = 2.5 CROSS = 3
```

Equation Evaluation

Use this statement to evaluate an equation, that is a function of the results of previous .MEASURE statements. The equation must not be a function of node voltages or branch currents. The syntax is:

```
.MEASURE <DC|TRAN|AC> result PARAM = 'equation'
+ <GOAL = val> <MINVAL = val>
```


Average, RMS, MIN, MAX, INTEG, and PP

Average (AVG), RMS, MIN, MAX, and peak-to-peak (PP) measurement modes report statistical functions of the output variable, rather than analysis values.

- AVG calculates the area under an output variable, divided by the periods of interest.
- RMS divides the square root of the area under the output variable square, by the period of interest.
- MIN reports the minimum value of the output function, over the specified interval.
- MAX reports the maximum value of the output function, over the specified interval.
- PP (peak-to-peak) reports the maximum value, minus the minimum value, over the specified interval.

Note: AVG, RMS, and INTEG have no meaning in a DC data sweep, so if you use them, Star-Hspice issues a warning message.

Syntax

```
.MEASURE <DC|AC|TRAN> result func out_var <FROM = val>
+ <TO = val> <GOAL = val> <MINVAL = val> <WEIGHT = val>
```

where:

<DC|AC|TRAN> Specifies the analysis type for the measurement. If you omit this parameter, Star-Hspice assumes the last analysis mode that you requested.

FROM Specifies the initial value for the *func* calculation. For transient analysis, this value is in units of time.

TO Specifies the end of the *func* calculation.

GOAL Specifies the .MEASURE value. Optimization uses this value for ERR calculation. This equation calculates the error:

$$\text{ERRfun} = (\text{GOAL} - \text{result}) / \text{GOAL}$$

<i>MINVAL</i>	If the absolute value of GOAL is less than MINVAL, MINVAL replaces the GOAL value in the denominator of the ERRfun expression. Used only in ERR calculation for optimization. Default = 1.0e-12.
<i>func</i>	Indicates one of the measure statement types: ■ AVG (average): Calculates the area under the <i>out_var</i>, divided by the periods of interest. ■ MAX (maximum): Reports the maximum value of the <i>out_var</i>, over the specified interval. ■ MIN (minimum): Reports the minimum value of the <i>out_var</i>, over the specified interval. ■ PP (peak-to-peak): Reports the maximum value, minus the minimum value, of the <i>out_var</i>, over the specified interval. ■ RMS (root mean squared): Calculates the square root of the area under the <i>out_var</i>² curve, divided by the period of interest.
<i>result</i>	Name of the measured value, in the output. The value is a function of the variable (<i>out_var</i>) and <i>func</i> .
<i>out_var</i>	Name of any output variable whose function (<i>func</i>) the simulation measures.
<i>WEIGHT</i>	Multiplies the calculated error, by the weight value. Used only in ERR calculation for optimization. Default = 1.0.

Example

```
.MEAS TRAN avgval AVG V(10) FROM = 10ns TO = 55ns
```

The example above calculates the average nodal voltage value for node 10, during the transient sweep, from the time 10 ns to 55 ns. It prints out the result as *avgval*.

```
.MEAS TRAN MAXVAL MAX V(1,2)
FROM = 15ns TO = 100ns
```

The preceding example finds the maximum voltage difference between nodes 1 and 2, for the time period from 15 ns to 100 ns.

```
.MEAS TRAN MINVAL MIN V(1,2) FROM = 15ns TO = 100ns
.MEAS TRAN P2PVAL PP I(M1) FROM = 10ns TO = 100ns
```

INTEGRAL Function

The INTEGRAL function reports the integral of an output variable, over a specified period.

Syntax

```
.MEASURE <DC|AC|TRAN> result INTEGRAL out_var
+ <FROM = val> <TO = val> <GOAL = val> <MINVAL = val>
+ <WEIGHT = val>
```

The INTEGRAL function (with *func*), uses the same syntax as the average (AVG), RMS, MIN, MAX, and peak-to-peak (PP) measurement mode, to defined the INTEGRAL (INTEG).

Example

The following example calculates the integral of $I(cload)$, from 10 ns to 100 ns.

```
.MEAS TRAN charge INTEG I(cload) FROM = 10ns TO = 100ns
```

DERIVATIVE Function

The DERIVATIVE function provides the derivative of:

- An output variable, at a specified time or frequency.
- Any sweep variable, depending on the type of analysis.
- A specified output variable, when some specific event occurs.

Syntax

```
.MEASURE <DC|AC|TRAN> result DERIVATIVE out_var
+ AT = val <GOAL = val> <MINVAL = val> <WEIGHT = val>
```

or

```
.MEASURE <DC|AC|TRAN> result DERIVATIVE out_var
+ WHEN var2 = val <RISE = r | LAST> <FALL = f | LAST>
+ <CROSS = c | LAST> <TD = tdval> <GOAL = goalval>
+ <MINVAL = minval> <WEIGHT = weightval>
```

or

```
.MEASURE <DC|AC|TRAN> result DERIVATIVE out_var
+ WHEN var2 = var3 <RISE = r | LAST> <FALL = f | LAST>
+ <CROSS = c | LAST> <TD = tdval> <GOAL = goalval>
+ <MINVAL = minval> <WEIGHT = weightval>
```

where:

<i>AT = val</i>	Value of <i>out_var</i> , at which the derivative is found.
<i>CROSS = c</i>	The numbers indicate which occurrence of a CROSS, FALL, or RISE event starts a measurement. ■ For RISE = <i>r</i>, when the designated signal has risen <i>r</i> rise times, the WHEN condition is met, and measurement starts. ■ For FALL = <i>f</i>, measurement starts when the designated signal has fallen <i>f</i> fall times. ■ A crossing is either a rise or a fall, so for CROSS = <i>c</i>, measurement starts when the designated signal has achieved a total of <i>c</i> crossing times, as a result of either rising or falling.
<i>RISE = r</i>	
<i>FALL = f</i>	
<i><DC AC TRAN></i>	Specifies the analysis type to measure. If you omit this parameter, Star-Hspice assumes the last analysis mode that you requested.
<i>DERIVATIVE</i>	Selects the derivative function. You can abbreviate to DERIV.
<i>GOAL</i>	Specifies the desired .MEASURE value. Optimization uses this value for ERR calculation. This equation calculates the error: $\text{ERRfun} = (\text{GOAL} - \text{result}) / \text{GOAL}$

<i>LAST</i>	Measures when the last CROSS, FALL, or RISE event occurs. ■ CROSS = LAST, measures the last time the WHEN condition is true, for a rising or falling signal. ■ FALL = LAST, measures the last time the WHEN condition is true, for a falling signal. ■ RISE = LAST, measures the last time the WHEN condition is true, for a rising signal. ■ LAST is a reserved word; do not use it as a parameter name in the above .MEASURE statements.
<i>MINVAL</i>	If the absolute value of GOAL is less than MINVAL, MINVAL replaces the GOAL value in the denominator of the ERRfun expression. Used only in ERR calculation for optimization. Default = 1.0e-12.
<i>out_var</i>	Variable for which Star-Hspice finds the derivative.
<i>result</i>	Name of the measured value, in the output.
<i>TD</i>	Identifies the time when measurement starts.
<i>var(2,3)</i>	These variables establish conditions that start a measurement.
<i>WEIGHT</i>	Multiplies the calculated error, between result and GOAL, by the weight value. Used only in ERR calculation for optimization. Default = 1.0.
<i>WHEN</i>	Selects the WHEN function.

Example

The following example calculates the derivative of $v(out)$, at 25 ns:

```
.MEAS TRAN slew rate DERIV V(out) AT = 25ns
```

The following example calculates the derivative of $v(1)$, when $v(1)$ is equal to $0.9 \cdot vdd$:

```
.MEAS TRAN slew DERIV v(1) WHEN v(1) = '0.90*vdd'
```

The following example calculates the derivative of $VP(output)/360.0$, when the frequency is 10 kHz:

```
.MEAS AC delay DERIV 'VP(output)/360.0' AT = 10khz
```

ERROR Function

The relative error function reports the relative difference between two output variables. You can use this format in optimization and curve-fitting of measured data. The relative error format specifies the variable to measure and calculate, from the .PARAM variable. To calculate the relative error between the two, Star-Hspice uses the ERR, ERR1, ERR2, or ERR3 function. With this format, you can specify a group of parameters to vary, to match the calculated value and the measured data.

Syntax

```
.MEASURE <DC|AC|TRAN> result ERRfun meas_var calc_var
+ <MINVAL = val> <IGNORE | YMIN = val> <YMAX = val>
+ <WEIGHT = val> <FROM = val> <TO = val>
```

where:

<DC AC TRAN>	Specifies the analysis type, for the measurement. If you omit this parameter, Star-Hspice assumes the last analysis mode that you requested.
<i>result</i>	Name of the measured result, in the output.
<i>ERRfun</i>	ERRfun indicates which error function to use: ERR, ERR1, ERR2, or ERR3.
<i>meas_var</i>	Name of any output variable or parameter, in the data statement. <i>M</i> denotes the <i>meas_var</i> , in the error equation.
<i>calc_var</i>	Name of the simulated output variable or parameter, in the .MEASURE statement, to compare with <i>meas_var</i> . <i>C</i> denotes the <i>calc_var</i> , in the error equation.

IGNOR YMIN	If the absolute value of <i>meas_var</i> is less than the IGNOR value, then the ERRfun calculation does not consider this point. Default = 1.0e-15.
FROM	Specifies the beginning of the ERRfun calculation. For transient analysis, the <i>from</i> value is in units of time. Defaults to the first value of the sweep variable.
WEIGHT	Multiplies the calculated error, by the weight value. Used only in ERR calculation for optimization. Default = 1.0.
YMAX	If the absolute value of <i>meas_var</i> is greater than the YMAX value, then the ERRfun calculation does not consider this point. Default = 1.0e+15.
TO	Specifies the end of the ERRfun calculation. Defaults to the last value of the sweep variable.
MINVAL	If the absolute value of <i>meas_var</i> is less than MINVAL, MINVAL replaces the <i>meas_var</i> value in the denominator of the ERRfun expression. Used only in ERR calculation for optimization. Default = 1.0e-12.

Error Equations

ERR

1. ERR sums the squares of $(M-C)/\max(M, MINVAL)$ for each point.
2. It then divides by the number of points.
3. Finally, it calculates the square root of the result.
 - M (*meas_var*) is the measured value of the device or circuit response.
 - C (*calc_var*) is the calculated value of the device or circuit response.
 - NPTS is the number of data points.

$$ERR = \left[\frac{1}{NPTS} \cdot \sum_{i=1}^{NPTS} \left(\frac{M_i - C_i}{\max(MINVAL, M_i)} \right)^2 \right]^{1/2}$$

ERR1

ERR1 computes the relative error at each point. For NPTS points, Star-Hspice calculates NPTS ERR1 error functions. For device characterization, the ERR1 approach is more efficient than the other error functions (ERR, ERR2, ERR3).

$$ERR1_i = \frac{M_i - C_i}{\max(MINVAL, M_i)} \quad i = 1, NPTS$$

Star-Hspice does not print out each calculated ERR1 value. When you set the ERR1 option, Star-Hspice calculates an ERR value, as follows:

$$ERR = \left[\frac{1}{NPTS} \cdot \sum_{i=1}^{NPTS} ERR1_i^2 \right]^{1/2}$$

ERR2

This option computes the absolute relative error, at each point. For NPTS points, Star-Hspice calls NPTS error functions.

$$ERR2_i = \left| \frac{M_i - C_i}{\max(MINVAL, M_i)} \right|, \quad i = 1, NPTS$$

The returned value printed for ERR2 is:

$$ERR = \frac{1}{NPTS} \cdot \sum_{i=1}^{NPTS} ERR2_i$$

ERR3

$$ERR3_i = \frac{\pm \log \left| \frac{M_i}{C_i} \right|}{\left| \log [\max(MINVAL, |M_i|)] \right|} = 1, NPTS$$

The + and - signs correspond to a positive and negative M/C ratio.

Note: If the M measured value is less than MINVAL, Star-Hspice uses MINVAL instead. Also, if the absolute value of M is less than the IGNOR | YMIN value, or greater than the YMAX value, then the error calculation does not consider this point.

Arithmetic Expression Measurements

The *expression* option is an arithmetic expression, that uses results from other prior .MEASURE statements.

Syntax

```
.MEASURE < TRAN > varname PARAM = "expression"
```

Example

In the example below, the first two measurements, **V3MAX** and **V2MIN**, set up the variables for the third measurement statement:

```
.MEAS TRAN V3MAX MAX V(3) FROM 0NS TO 100NS  
.MEAS TRAN V2MIN MIN V(2) FROM 0NS TO 100NS  
.MEAS VARG PARAM = "(V2MIN + V3MAX)/2"
```

- **V3MAX** is the maximum voltage of $V(3)$ between 0ns and 100ns of the simulation.
- **V2MIN** is the minimum voltage of $V(2)$ during that same interval.
- **VARG**, is the mathematical average of the **V3MAX** and **V2MIN** measurements.

Note: Expressions used in arithmetic expression must not be a function of node voltages or branch currents. Expressions used in all other .MEASURE statements can contain node voltages or branch currents, but must not use results from other .MEASURE statements.

.DOUT Statement: Expected State of Digital Output Signal

The digital output (*.DOUT*) statement specifies the expected final state of an output signal, in Star-Hspice.

During simulation, Star-Hspice compares the simulated results, with the expected output vector. If the states are different, Star-Hspice reports an error.

Syntax

The *.DOUT* statement can use either of two syntaxes. In both syntaxes, the *time* and *state* parameters describe the expected output of the *nd* node.

- The first syntax specifies a single threshold voltage, *VTH*. Any voltage level above *VTH* is high; any level below *VTH* is low.

```
.DOUT nd VTH ( time state < time state > )
```

where:

- *nd* is the node name.
- *VTH* is the single voltage threshold.
- *time* is an absolute time-point.
- *state* is one of the following expected conditions of the *nd* node, at the specified *time*:
 - 0 expect ZERO,LOW.
 - 1 expect ONE,HIGH.
 - else Don't care.
- The second syntax defines a threshold for both a logic high (*VHI*) and low (*VLO*).

```
.DOUT nd VLO VHI ( time state < time state > )
```

where:

- *nd* is the node name.
- *VLO* is the voltage of the logic low state.
- *VHI* is the voltage of the logic high state.
- *time* is an absolute time-point.
- *state* is one of the following expected conditions of the *nd* node, at the specified *time*:
 - 0 expect ZERO,LOW.
 - 1 expect ONE,HIGH.
 - else Don't care.

Note: If you specify both syntaxes (*VTH*, plus *VHI* and *VLO*), then Star-Hspice processes only *VTH*, and ignores *VHI* and *VLO*.

For both cases, the *time*, *state* pair describes the expected output. During simulation, Star-Hspice compares the simulated results against the expected output vector. If the states are different, Star-Hspice reports an error message. The legal values for *state* are:

0	Expect ZERO .
1	Expect ONE .
X, x	Do not care.
U, u	Do not care.
Z, z	Expect HIGH IMPEDANCE (do not care).

Example

The `.PARAM` statement in the example below sets the value of the `VTH` variable to 3. The `.DOUT` statement, operating on the `node1` node, uses `VTH` as its threshold voltage.

```
.PARAM VTH = 3.0
.DOUT node1 VTH(0.0n 0 1.0n 1
+ 2.0n X 3.0n U 4.0n Z 5.0n 0)
```

When `node1` is above 3V, it is considered a logic **1**; otherwise, it is a logic **0**.

- At 0ns, the expected state of `node1` is logic-low.
- At 1ns, the expected state is logic-high.
- At 2ns, 3ns, and 4ns, the expected state is “do not care”.
- At 5ns, the expected state is again logic-low.

.STIM Statement: Reuse Simulation Output as Input Stimuli

You can use the `.STIM` statement to reuse the results (output) of one simulation, as input stimuli in a new simulation.

Note: `.STIM` is an abbreviation of `.STIMULI`. You can use either form to specify this statement in Star-Hspice.

The `.STIM` statement specifies:

- Expected stimulus (PWL Source, DATA CARD, or VEC FILE).
- Signals to transform.
- Independent variables.

One `.STIM` statement produces one corresponding output file.

Syntax

Brackets `[ ]` enclose comments, which are optional.

```
.stim <tran|ac|dc> PWL|DATA|VEC <filename=output_filename> ...
```

PWL Source

You can use this syntax only in transient analysis.

```
.stim [tran] PWL [filename=output_filename]
+      [name1=] ovar1 [node1=n+] [node2=n-]
+      [[name2=] ovar2 [node1=n+] [node2=n-] ...]
+      [from=val] [to=val] [npoints=val]
```

or

```
.stim [tran] PWL [filename=output_filename]
+      [name1=] ovar1 [node1=n+] [node2=n-]
+      [[name2=] ovar2 [node1=n+] [node2=n-] ...]
+      indepvar=[(] t1 [t2 ...)] ] ]
```

where:

<i>tran</i>	Transient simulation.
<i>filename</i>	Output file name. If you do not specify a filename, Star-Hspice uses the input filename.
<i>namei</i>	PWL Source Name that you specify. The name must start with V (for a voltage source) or I (for a current source).
<i>ovari</i>	Output variable that you specify. <i>ovar</i> can be: ■ Node voltage. ■ Element current. ■ Parameter string. If you use a parameter string, you must specify <i>name1</i>. For example: <pre style="margin-left: 40px;">v(1), i(r1), v(2,1), par('v(1)+v(2)')</pre>
<i>node1</i>	Positive terminal node name.
<i>node2</i>	Negative terminal node name.
<i>from</i>	Keyword; specifies the time to start output of simulation results. For transient analysis, uses the time units that you specified.
<i>to</i>	Keyword; specifies the time to end output of simulation results. For transient analysis, uses the time units that you specified. The <i>from</i> value can be greater than the <i>to</i> value.
<i>npoints</i>	Number of output time points.
<i>indepvar</i>	Keyword; specifies dispersed (independent-variable) time points. You must specify dispersed time points in <i>increasing</i> order.

Data Card

```
.stim [tran |ac |dc] DATA [filename=output_filename]
+   dataname [name1=] ovar1
+   [[name2=]ovar2 ...] [from= val] [to=val]
+   [npoints=val] [indepout=val]
```

or

```
.stim [tran |ac |dc] DATA [filename=output_filename]
+   dataname [name1=] ovar1
+   [[name2=]ovar2 ...] indepvar=[(]t1 [t2 ...[)]
+   [indepout=val]
```

where:

<i>tran ac dc</i>	Selects the simulation type: transient, AC, or DC.
<i>filename</i>	Output file name. If you do not specify a filename, Star-Hspice uses the input filename.
<i>dataname</i>	Name of the data card to generate.
<i>namei</i>	Name of a parameter of the data card to generate.
<i>ovari</i>	Output variable that you specify. <i>ovar</i> can be: ■ Node voltage. ■ Element current. ■ Element templates. ■ Parameter string. For example: <pre>v(1), i(r1), v(2,1), par('v(1)+v(2)'), LX1(m1), LX2(m1)</pre>
<i>from</i>	Keyword; specifies the time to start output of simulation results. For transient analysis, uses the time units that you specified.
<i>to</i>	Keyword; specifies the time to end output of simulation results. For transient analysis, uses the time units that you specified.
<i>npoints</i>	Number of output independent-variable points.
<i>indepvar</i>	Keyword; specifies dispersed independent-variable points.

- indepout* Indicates whether to generate the independent variable column.
- *indepout*, *indepout* = 1, or *on*, produces the independent variable column. You can specify the independent-variables in any order.
 - *indepout*= 0 or *off* (the default), does not produce the independent variable column.

Note: You can place the *indepout* field anywhere, after the *ovari* field.

Digital Vector File

You can use this syntax only in transient analysis.

```
.stim [tran] VEC [filename=output_filename]
+    vth=val vtl=val [name1=] ovar1
+    [[name2=]ovar2 ...] [from=val] [to=val]
+    [npoints=val]
```

or

```
.stim [tran] VEC [filename=output_filename]
+    vth=val vtl=val [name1=] ovar1
+    [[name2=]ovar2 ...] indepvar=[(] t1 [t2 ...[)]]
```

where:

- namei* Signal name that you specify.
- filename* Output file name. If you do not specify a filename, Star-Hspice uses the input filename.
- ovari* Output variable that you specify. *ovar* must be a node voltage.
- from* Keyword; specifies the time to start output of simulation results. For transient analysis, uses the time units that you specified.
- to* Keyword; specifies the time to end output of simulation results. For transient analysis, uses the time units that you specified. The *from* value can be greater than the *to* value.

<i>npoints</i>	Number of output time points.
<i>indepvar</i>	Keyword; specifies dispersed independent-variable points. You must specify dispersed time points in <i>increasing</i> order.
<i>vth</i>	High voltage threshold.
<i>vtl</i>	Low voltage threshold.

Output Files

The .STIM statement generates the following output files:

Output File Type Extension

PWL Source	.pwl\$_tr# The .STIM statement writes PWL source results to output_file.pwl\$_tr#. This output file is the result of a .stim <tran> pwl statement in the input file.
Data Card	.dat\$_tr#, .dat\$_ac#, or .dat\$_sw# The .STIM statement writes DATA Card results to output_file.dat\$_sw#, output_file.dat\$_ac#, or output_file.dat\$_tr#. This output file is the result of a .stim <tran ac dc> data statement in the input file.
Digital Vector File	.vec\$_tr# The .STIM statement writes Digital Vector File results to output_file.vec\$_tr#. This output file is the result of a .stim <tran> vec statement in the input file.

where:

\$ Serial number of the current .STIM statement, within statements of the same stimulus type (pwl, data, or vec). $\$=0 \sim n-1$ (n is the number of the .STIM statement of that type). The initial \$ value is 0.

For example, if you specify three .STIM pwl statements, Star-Hspice generates three PWL output files, with the suffix names pwl0_tr#, pwl1_tr#, and pwl2_tr#.

tr | ac | sw

- tr = transient analysis.
- ac = AC analysis.
- sw = DC sweep analysis.

Either a sweep number, or a hard-copy file number. For a single sweep run, the default number is 0.

Element Template Listings

Table 8-2: Resistor

Name	Alias	Description
G	LV1	Conductance, at analysis temperature.
R	LV2	Resistance, at reference temperature.
TC1	LV3	First temperature coefficient.
TC2	LV4	Second temperature coefficient.

Table 8-3: Capacitor

Name	Alias	Description
CEFF	LV1	Computed effective capacitance.
IC	LV2	Initial condition.
Q	LX0	Charge, stored in capacitor.
CURR	LX1	Current, flowing through capacitor.
VOLT	LX2	Voltage, across capacitor.
–	LX3	Capacitance (not used after Star-Hspice release 95.3).

Table 8-4: Inductor

Name	Alias	Description
LEFF	LV1	Computed effective inductance.
IC	LV2	Initial condition.
FLUX	LX0	Flux, in the inductor.
VOLT	LX1	Voltage, across inductor.
CURR	LX2	Current, flowing through inductor.
—	LX4	Inductance (not used after Star-Hspice release 95.3).

Table 8-5: Mutual Inductor

Name	Alias	Description
K	LV1	Mutual inductance.

Table 8-6: Voltage-Controlled Current Source

Name	Alias	Description
CURR	LX0	Current, through the source, if VCCS.
R	LX0	Resistance value, if VCR.
C	LX0	Capacitance value, if VCCAP.
CV	LX1	Controlling voltage.
CQ	LX1	Capacitance charge, if VCCAP.
DI	LX2	Derivative of the source current, relative to the control voltage.

Table 8-6: Voltage-Controlled Current Source (*Continued*)

Name	Alias	Description
ICAP	LX2	Capacitance current, if VCCAP.
VCAP	LX3	Voltage, across capacitance, if VCCAP.

Table 8-7: Voltage-Controlled Voltage Source

Name	Alias	Description
VOLT	LX0	Source voltage.
CURR	LX1	Current, through source.
CV	LX2	Controlling voltage.
DV	LX3	Derivative of the source voltage, relative to the control current.

Table 8-8: Current-Controlled Current Source

Name	Alias	Description
CURR	LX0	Current, through source.
CI	LX1	Controlling current.
DI	LX2	Derivative of the source current, relative to the control current.

Table 8-9: Current-Controlled Voltage Source

Name	Alias	Description
VOLT	LX0	Source voltage.
CURR	LX1	Source current.
CI	LX2	Controlling current.
DV	LX3	Derivative of the source voltage, relative to the control current.

Table 8-10: Independent Voltage Source

Name	Alias	Description
VOLT	LV1	DC/transient voltage.
VOLTM	LV2	AC voltage magnitude.
VOLTP	LV3	AC voltage phase.

Table 8-11: Independent Current Source

Name	Alias	Description
CURR	LV1	DC/transient current.
CURRM	LV2	AC current magnitude.
CURRP	LV3	AC current phase.

Table 8-12: Diode

Name	Alias	Description
AREA	LV1	Diode area factor.
AREAX	LV23	Area, after scaling.
IC	LV2	Initial voltage, across diode.
VD	LX0	Voltage, across diode (VD), excluding RS (series resistance).
IDC	LX1	DC current, through diode (ID), excluding RS. Total diode current is the sum of IDC and ICAP.
GD	LX2	Equivalent conductance (GD).
QD	LX3	Charge of diode capacitor (QD).
ICAP	LX4	Current, through the diode capacitor. Total diode current is the sum of IDC and ICAP.
C	LX5	Total diode capacitance.
PID	LX7	Photo current, in diode.

Table 8-13: BJT (*Sheet 1 of 3*)

Name	Alias	Description
AREA	LV1	Area factor.
ICVBE	LV2	Initial condition, for base-emitter voltage (VBE).
ICVCE	LV3	Initial condition, for collector-emitter voltage (VCE).
MULT	LV4	Number of multiple BJTs.
FT	LV5	FT (Unity gain bandwidth).

Table 8-13: BJT (Sheet 2 of 3)

Name	Alias	Description
ISUB	LV6	Substrate current.
GSUB	LV7	Substrate conductance.
LOGIC	LV8	LOG 10 (IC).
LOGIB	LV9	LOG 10 (IB).
BETA	LV10	BETA.
LOGBETAI	LV11	LOG 10 (BETA) current.
ICTOL	LV12	Collector current tolerance.
IBTOL	LV13	Base current tolerance.
RB	LV14	Base resistance.
GRE	LV15	Emitter conductance, 1/RE.
GRC	LV16	Collector conductance, 1/RC.
PIBC	LV18	Photo current, base-collector.
PIBE	LV19	Photo current, base-emitter.
VBE	LX0	VBE.
VBC	LX1	Base-collector voltage (VBC).
CCO	LX2	Collector current (CCO).
CBO	LX3	Base current (CBO).
GPI	LX4	$g_{\pi} = ib / vbe$, constant vbc .
GU	LX5	$g_{\mu} = ib / vbc$, constant vbe .
GM	LX6	$g_m = ic / vbe + ic / vbe$, constant vce .
G0	LX7	$g_0 = ic / vce$, constant vbe .

Table 8-13: BJT (Sheet 3 of 3)

Name	Alias	Description
QBE	LX8	Base-emitter charge (QBE).
CQBE	LX9	Base-emitter charge current (CQBE).
QBC	LX10	Base-collector charge (QBC).
CQBC	LX11	Base-collector charge current (CQBC).
QCS	LX12	Current-substrate charge (QCS).
CQCS	LX13	Current-substrate charge current (CQCS).
QBX	LX14	Base-internal base charge (QBX).
CQBX	LX15	Base-internal base charge current (CQBX).
GXO	LX16	1/Rbeff Internal conductance (GXO).
CEXBC	LX17	Base-collector equivalent current (CEXBC).
–	LX18	Base-collector conductance (GEQCB0), (not used in Star-Hspice releases after 95.3).
CAP_BE	LX19	cbe capacitance (C_{Π}).
CAP_IBC	LX20	cbc internal base-collector capacitance (C_{μ}).
CAP_SCB	LX21	■ csc substrate-collector capacitance, for vertical transistors. ■ csb substrate-base capacitance, for lateral transistors.
CAP_XBC	LX22	cbcx external base-collector capacitance.
CMCMO	LX23	$(TF \cdot IBE) / vbc$.
VSUB	LX24	Substrate voltage.

Table 8-14: JFET

Name	Alias	Description
AREA	LV1	JFET area factor.
VDS	LV2	Initial condition, for drain-source voltage.
VGS	LV3	Initial condition, for gate-source voltage.
PIGD	LV16	Photo current, gate-drain in JFET.
PIGS	LV17	Photo current, gate-source in JFET.
VGS	LX0	VGS.
VGD	LX1	Gate-drain voltage (VGD).
CGSO	LX2	Gate-to-source (CGSO).
CDO	LX3	Drain current (CDO).
CGDO	LX4	Gate-to-drain current (CGDO).
GMO	LX5	Transconductance (GMO).
GDSO	LX6	Drain-source transconductance (GDSO).
GGSO	LX7	Gate-source transconductance (GGSO).
GGDO	LX8	Gate-drain transconductance (GGDO).
QGS	LX9	Gate-source charge (QGS).
CQGS	LX10	Gate-source charge current (CQGS).
QGD	LX11	Gate-drain charge (QGD).
CQGD	LX12	Gate-drain charge current (CQGD).
CAP_GS	LX13	Gate-source capacitance.
CAP_GD	LX14	Gate-drain capacitance.

Table 8-14: JFET (Continued)

Name	Alias	Description
–	LX15	Body-source voltage (not used after Star-Hspice release 95.3).
QDS	LX16	Drain-source charge (QDS).
CQDS	LX17	Drain-source charge current (CQDS).
GMBS	LX18	Drain-body (backgate) transconductance (GMBS).

Table 8-15: MOSFET (Sheet 1 of 4)

Name	Alias	Description
L	LV1	Channel length (L).
W	LV2	Channel width (W).
AD	LV3	Area of the drain diode (AD).
AS	LV4	Area of the source diode (AS).
ICVDS	LV5	Initial condition, for drain-source voltage (VDS).
ICVGS	LV6	Initial condition, for gate-source voltage (VGS).
ICVBS	LV7	Initial condition, for bulk-source voltage (VBS).
–	LV8	Device polarity: ■ 1 = forward ■ -1 = reverse (not used after Star-Hspice release 95.3).
VTH	LV9	Threshold voltage (bias dependent).
VDSAT	LV10	Saturation voltage (VDSAT).
PD	LV11	Drain diode periphery (PD).
PS	LV12	Source diode periphery (PS).

Table 8-15: MOSFET (Sheet 2 of 4)

Name	Alias	Description
RDS	LV13	Drain resistance (squares), (RDS).
RSS	LV14	Source resistance (squares), (RSS).
XQC	LV15	Charge-sharing coefficient (XQC).
GDEFF	LV16	Effective drain conductance ($1/R_{Deff}$).
GSEFF	LV17	Effective source conductance ($1/R_{Seff}$).
CDSAT	LV18	Drain-bulk saturation current, at -1 volt bias.
CSSAT	LV19	Source-bulk saturation current, at -1 volt bias.
VDBEFF	LV20	Effective drain bulk voltage.
BETAEFF	LV21	BETA, effective.
GAMMAEFF	LV22	GAMMA, effective.
DELTA	LV23	ΔL (MOS6 amount of channel length modulation), (valid only for LEVELs 1, 2, 3 and 6).
UBEFF	LV24	UB effective (valid only for LEVELs 1, 2, 3 and 6).
VG	LV25	VG drive (valid only for LEVELs 1, 2, 3 and 6).
VFBEFF	LV26	VFB effective.
–	LV31	Drain current tolerance (not used in Star-Hspice releases after 95.3).
IDSTOL	LV32	Source-diode current tolerance.
IDDTOL	LV33	Drain-diode current tolerance.
COVLGS	LV36	Gate-source overlap capacitance.
COVLGD	LV37	Gate-drain overlap capacitance.

Table 8-15: MOSFET (Sheet 3 of 4)

Name	Alias	Description
COVLGB	LV38	Gate-bulk overlap capacitance.
VBS	LX1	Bulk-source voltage (VBS).
VGS	LX2	Gate-source voltage (VGS).
VDS	LX3	Drain-source voltage (VDS).
CDO	LX4	DC-drain current (CDO).
CBSO	LX5	DC source-bulk diode current (CBSO).
CBDO	LX6	DC drain-bulk diode current (CBDO).
GMO	LX7	DC-gate transconductance (GMO).
GDSO	LX8	DC drain-source conductance (GDSO).
GMBSO	LX9	DC-substrate transconductance (GMBSO).
GBDO	LX10	Conductance of the drain diode (GBDO).
GBSO	LX11	Conductance of the source diode (GBSO).
<i>Meyer and Charge Conservation Model Parameters</i>		
QB	LX12	Bulk charge (QB).
CQB	LX13	Bulk-charge current (CQB).
QG	LX14	Gate charge (QG).
CQG	LX15	Gate-charge current (CQG).
QD	LX16	Channel charge (QD).
CQD	LX17	Channel-charge current (CQD).
CGGBO	LX18	$CGGBO = \partial Qg / \partial V_{gb} = CGS + CGD + CGB$

Table 8-15: MOSFET (Sheet 4 of 4)

Name	Alias	Description
CGDBO	LX19	$\text{CGDBO} = \partial Q_g / \partial V_{db}$, (for Meyer CGD = -CGDBO)
CGSBO	LX20	$\text{CGSBO} = \partial Q_g / \partial V_{sb}$, (for Meyer CGS = -CGSBO)
CBGBO	LX21	$\text{CBGBO} = \partial Q_b / \partial V_{gb}$, (for Meyer CGB = -CBGBO)
CBDBO	LX22	$\text{CBDBO} = \partial Q_b / \partial V_{db}$
CBSBO	LX23	$\text{CBSBO} = \partial Q_b / \partial V_{sb}$
QBD	LX24	Drain-bulk charge (QBD).
–	LX25	Drain-bulk charge current (CQBD), (not used in Star-Hspice releases after 95.3).
QBS	LX26	Source-bulk charge (QBS).
–	LX27	Source-bulk charge current (CQBS), (not used after Star-Hspice release 95.3).
CAP_BS	LX28	Bulk-source capacitance.
CAP_BD	LX29	Bulk-drain capacitance.
CQS	LX31	Channel-charge current (CQS).
CDGBO	LX32	$\text{CDGBO} = \partial Q_d / \partial V_{gb}$
CDDBO	LX33	$\text{CDDBO} = \partial Q_d / \partial V_{db}$
CDSBO	LX34	$\text{CDSBO} = \partial Q_d / \partial V_{sb}$

Table 8-16: Saturable Core Element

Name	Alias	Description
MU	LX0	Dynamic permeability (μ), Weber/(amp-turn-meter).
H	LX1	Magnetizing force (H), Ampere-turns/meter.
B	LX2	Magnetic flux density (B), Webers/meter ² .

Table 8-17: Saturable Core Winding

Name	Alias	Description
LEFF	LV1	Effective winding inductance (Henry).
IC	LV2	Initial condition.
FLUX	LX0	Flux, through winding (Weber-turn).
VOLT	LX1	Voltage, across winding (Volt).



Chapter 9

Simulation Options

This chapter describes the options that you can use to modify various aspects of a Star-Hspice simulation, including:

- output types
- accuracy
- speed
- convergence

This chapter explains all options available in the `.OPTION` statement in Star-Hspice, including the following topics:

- [Setting Control Options](#)
- [General Control Options](#)
- [Model Analysis Options](#)
- [DC Operating Point, DC Sweep, and Pole/Zero Options](#)
- [Transient and AC Small Signal Analysis Options](#)

Setting Control Options

This section describes how to set control options.

.OPTION Statement

To set control options, use `.OPTION` statements. You can set any number of options in one `.OPTION` statement, and you can include any number of `.OPTION` statements in an input netlist file. [Table 9-1 on page 9-3](#) lists all control options. Descriptions of the options follow the table. For descriptions of options that are relevant to a specific simulation type, see the appropriate DC, transient, and AC analysis chapters.

Most options default to 0 (OFF) when you do not assign a value, using either `.OPTION <opt> = <val>` or the option with no assignment: `.OPTION <opt>`. Each option description in this section also shows the default value.

Syntax

```
.OPTION opt1 <opt2 opt3 ...>
```

opt1 ... Specifies any input control options. Many options are in the form `<opt> = x`, where `<opt>` is the option name and `x` is the value assigned to that option. This section describes all options.

Example

To reset options, set them to zero (`.OPTION <opt> = 0`). To redefine an option, enter a new `.OPTION` statement; Star-Hspice uses the last definition. For example, set the BRIEF option to 1, to suppress the printout. Then reset BRIEF to 0 later in the input file, to resume the printout.

```
.OPTION BRIEF $ Sets BRIEF to 1 (turns it on)
* Netlist, models,
...
.OPTION BRIEF = 0 $ Turns BRIEF off
Options Keyword Summary
```

Table 9-1 lists the keywords for the .OPTION statement, grouped by their typical application. The sections that follow the table, describe the options listed under each type of analysis.

Table 9-1: .OPTION Keyword Application Table (Sheet 1 of 2)

GENERAL CONTROL OPTIONS		MODEL ANALYSIS	DC OPERATING POINT, DC SWEEP, and POLE/ZERO		TRANSIENT and AC SMALL SIGNAL ANALYSIS	
<i>Input, Output</i>	<i>Interfaces</i>	<i>General</i>	<i>Accuracy</i>	<i>Convergence</i>	<i>Accuracy</i>	<i>Timestep</i>
ACCT	ARTIST	DCAP	ABSH	CONVERGE	ABSH	ABSVAR
ACOUT	CDS	SCALE	ABSI ABSTOL	CSHDC	ABSV, VNTOL	DELMAX
ALT999	CSDF	TNOM	ABSMOS	DCFOR	ACCURATE	DVDT
ALT9999	MEASOUT		ABSV VNTOL	DCHOLD	ACOUT	FS
ALTER	DLENCSD					
BEEP						
BINPRNT	MENTOR	MOSFETs	ABSVDC	DCON	CHGTOL	FT
BRIEF	POST	CVTOL	DI	DCSTEP	CSHUNT	IMIN, ITL3
CO	PROBE	DEFAD	KCLTEST	DCTAN	GSHUNT	IMAX, ITL4
INGOLD	PSF	DEFAS	MAXAMP	DV	DI	ITL5
LENNAM	SDA	DEFL	RELH	GMAX	GMIN	RELVAR
LIST	ZUKEN	DEFNRD	RELI	GMINDC	GSHUNT	RMAX
MEASDGT						
MEASFAIL						
MEASSORT		DEFNRS	RELMOS	GRAMP	CSHUNT	RMIN
NODE	Analysis	DEFPD	RELV RLTOL	GSHUNT	MAXAMP	SLOPETOL
NOELCK	ASPEC	DEFPS	RELVDC	ICSWEEP	RELH	TIMERES
NOMOD	LIMPTS	DEFW		ITLPTRAN	RELI	

Table 9-1: .OPTION Keyword Application Table (Sheet 2 of 2)

GENERAL CONTROL OPTIONS		MODEL ANALYSIS	DC OPERATING POINT, DC SWEEP, and POLE/ZERO		TRANSIENT and AC SMALL SIGNAL ANALYSIS	
NOPAGE	PARHIER	SCALM	Matrix	NEWTOL	RELQ	Algorithm
NOTOP	SPICE	WL	ITL1	OFF	RELTOL RELV	DVTR
NUMDGT	SEED		ITL2	RESMIN	RISETIME	IMAX
NXX		Inductors	NOPIV		TRTOL	IMIN
OPTLST	Error	GENK	PIVOT, SPARSE	Pole/Zero	VNTOL, ABSV	LVLTIM
OPTS	BADCHR	KLIM		CSCAL	Speed	MAXORD
PATHNUM	DIAGNOSTIC		PIVREF	FMAX	AUTOSTOP	METHOD PURETP
PLIM	NOWARN	BJTs	PIVREL	FSCAL	BKPSIZ	MU
POSTTOP				GSCAL		
POST_VERSION	WARNLIMIT	EXPLI	PIVTOL	LSCAL	BYPASS	
PUTMEAS						
SEARCH			SPARSE, PIVOT	PZABS	BYTOL	Input, Output
STATFL	Version	Diodes		PZTOL	FAST	INTERP
VERIFY	H9007	EXPLI		RITOL	ITLPZ	ITRPRT
CPU			Input, Output	XnR, XnI	MBYPASS	UNWRAP
CPTIME			CAPTAB	NEWTOL	TRCON	
EPSMIN			DCCAP			
EXPMAX			VFLOOR			
LIMTIM						

General Control Options

Descriptions of the general control options follow. The descriptions are alphabetical by keyword, under the sections presented in the table.

Input and Output Options

<i>ACCT</i>	Reports job accounting and runtime statistics, at the end of the output listing. The ratio of output points to total iterations, determines simulation efficiency. Reporting is automatic, unless you disable it. Choices for ACCT are: 0 disables reporting 1 enables reporting 2 enables reporting of matrix statistics
<i>ACOUT</i>	AC output calculation method, for the difference in values of magnitude, phase, and decibels. Use these values for prints and plots. The default value is 1. The default value, ACOUT = 1, selects the Star-Hspice method, which calculates the difference of the magnitudes of the values. The SPICE method, ACOUT = 0, calculates the magnitude of the differences in Star-Hspice.
<i>ALT999, ALT9999</i>	This option generates up to 1000 (ALT999) or 10,000 (ALT9999) unique output files, from .ALTER simulation runs. Star-Hspice appends a number from 0-999 (ALT999) or 0-9999 (ALT9999) to the output file extension. For example, if a .TRAN analysis has 50 .ALTER statements, the filenames are filename.tr0, filename.tr1, ..., filename.tr50. Without this option, Star-Hspice overwrites files after the 36th .ALTER statement.

altchk	By default, Star-Hspice automatically reports topology errors in the latest elements, in your top-level netlist. It also reports errors in elements that you redefine, using the .ALTER statement (altered netlist). To disable topology checking in redefined elements (that is, to check topology <i>only</i> in the top-level netlist, but <i>not</i> in the altered netlist), set: <pre>.option altchk=0</pre> By default, .OPTION ALTCHK is set to 1: <pre>.option altchk=1</pre> or <pre>.option altchk</pre> This enables topology checking, in elements that you redefine using the .ALTER statement.
BEEP	■ BEEP=1 sounds an audible tone when simulation returns a message, such as <i>info: hspice job completed</i>. ■ BEEP=0 turns off the audible tone.
BINPRINT	Outputs the binning parameters of the CMI MOSFET model. Currently available only for Level 57.
BRIEF, NXX	Stops printback of the data file, until Star-Hspice finds an .OPTION BRIEF = 0, or the .END statement. It also resets the LIST, NODE, and OPTS options, and sets NOMOD. BRIEF = 0 enables printback. NXX is the same as BRIEF.
CO = x	Sets the number of columns for printout: x can be either 80 (for narrow printout) or 132 (for wide carriage printouts). You also can use the .WIDTH statement to set the output width. The default value is 80.

INGOLD = *x*

Specifies the printout data format. Use *INGOLD* = 2 for SPICE compatibility in Star-Hspice. The default value is 0. You can print numeric output from Star-Hspice, in one of three ways:

INGOLD = 0

Specifies engineering format, which expresses exponents as a single character:

1G = 1e9 1X = 1e6 1K = 1e3 1M = 1e-3
 1U = 1e-6 1N = 1e-9 1P = 1e-12
 1F = 1e-15

INGOLD = 1

Combines fixed and exponential format (G Format). Uses fixed format for numbers 0.1 to 999. Uses exponential format for numbers greater than 999, or less than 0.1.

INGOLD = 2

Uses exponential format exclusively (SPICE2G style). Exponential format generates constant number sizes, suitable for post-analysis tools.

Use *.OPTION MEASDGT*, with *INGOLD*, to control the output data format for *.MEASURE* results.

LENNAM = *x*

Specifies the maximum length of names, in the printout of operating point analysis results. The default value is 8. The maximum value of *x* is 16.

LIST, *VERIFY*

Produces an element summary listing, of the input data to print. Calculates effective sizes of elements, and the key values. *BRIEF* suppresses the *LIST* option. *VERIFY* is an alias for *LIST*.

MEASDGT = *x* Formats the .MEASURE statement output, in both the listing file and the .MEASURE output files (.ma0, .mt0, .ms0, and so on).

The value of *x* is typically between 1 and 7, although you can set it as high as 10. The default value is 4.0.

For example, if MEASDGT = 5, then .MEASURE displays numbers as:

- Five decimal digits, for numbers in scientific notation.
- Five digits to the right of the decimal, for numbers between 0.1 and 999.

In the listing (.lis), file, all .MEASURE output values are in scientific notation, so .OPTION MEASDGT = 5 results in five decimal digits.

Use MEASDGT, with .OPTION INGOLD = *x*, to control the output data format.

NODE Prints a node cross reference table. The BRIEF option suppresses NODE. The table lists each node, and all elements connected to it. A code indicates the terminal of each element, and a colon (:) separates the code from the element name. The codes are:

+	Diode anode
-	Diode cathode
B	BJT base
B	MOSFET or JFET bulk
C	BJT collector
D	MOSFET or JFET drain
E	BJT emitter
G	MOSFET or JFET gate
S	BJT substrate
S	MOSFET or JFET source

For example, part of a cross reference might look like:

```
1 M1:B D2:+ Q4:B
```

This line indicates that the bulk of M1, the anode of D2, and the base of Q4, all connect to node 1.

<i>NOELCK</i>	No element check; bypasses element checking, to reduce pre-processing time for very large files.
<i>NOMOD</i>	Suppresses the printout of model parameters.
<i>NOPAGE</i>	Suppresses page ejects, for title headings.
<i>NOTOP</i>	Suppresses the topology check, which increases speed for pre-processing very large files.
<i>NUMDGT = x</i>	Sets the number of significant digits to print, for output variable values. The value of <i>x</i> is typically between 1 and 7, although you can set it as high as 10. The default value is 4.0. This option does not affect the accuracy of the simulation.
<i>NXX</i>	Stops printback of the data file, until Star-Hspice finds an <code>.OPTION BRIEF = 0</code> , or the <code>.END</code> statement. It also resets the <code>LIST</code> , <code>NODE</code> , and <code>OPTS</code> options, and sets <code>NOMOD</code> . <code>BRIEF = 0</code> enables printback. <code>NXX</code> is the same as <code>BRIEF</code> .
<i>OPTLST = x</i>	Outputs additional optimization information: 0 No information (default). 1 Prints parameter, Broyden update, and bisection results information. 2 Prints gradient, error, Hessian, and iteration information. 3 Prints all of the above, and Jacobian.
<i>OPTS</i>	Prints the current settings, for all control options. If you change any of the default values of the options, the <code>OPTS</code> option prints the values that the simulation actually uses. The <code>BRIEF</code> option suppresses <code>OPTS</code> .

PATHNUM

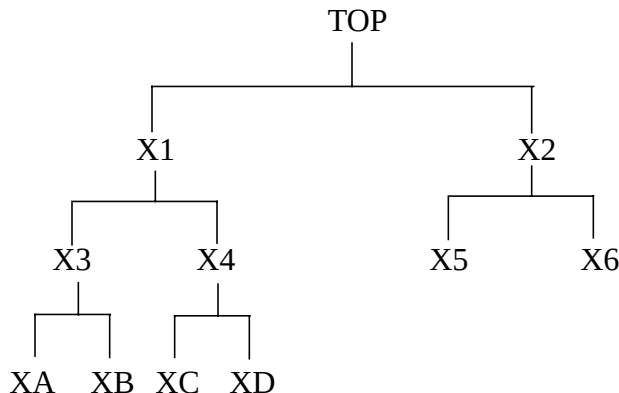
Prints subcircuit path numbers, instead of path names.

PLIM = x

Specifies plot size limits, for current and voltage plots:

- 0 Finds a common plot limit, and plots all variables on one graph, at the same scale
- 1 Enables SPICE-type plots in Star-Hspice, which create a separate scale and axis for each plot variable.

This option does not affect post- processing of graph data.

POSTTOP= n Outputs instances, up to n levels deep. For example, if your design hierarchy is:

Then:

- `.OPTION POST` saves all nodes, at all levels of hierarchy.
- `.OPTION POSTTOP` or `.OPTION POSTTOP=1` saves only the TOP node.
- `.OPTION POSTTOP=2` saves only nodes at the top two levels—that is, TOP, X1, and X2.

POST_VERSION**= x**

Sets the post-processing output version:

- $x = 9007$ truncates the node name in the post-processor output file, to a maximum of 16 characters.
- $x = 9601$ sets the node name length for the output file, consistent with the input restrictions (1024 characters).

POST_VERSION=2001 Sets the post-processing output version to 2001. This option shows you the new output file header, which includes the right number of output variables, rather than **** when the number exceeds 9999. If you set `.OPTION post_version=2001 post=2` in the netlist, then Star-Hspice returns more-accurate ASCII results.

The syntax is:

```
.option post_version=2001
```

To use binary values (with double precision) in the output file, include the following in the input file:

```
*****
.option post (or post=1) post_version=2001
*****
```

For more accurate simulation results, comment this format.

STATFL Controls whether Star-Hspice creates a `.st0` file.

- `statfl=0` (default) outputs a `.st0` file.
- `statfl=1` suppresses the `.st0` file.

SEARCH Sets the search path for libraries and included files. Star-Hspice searches the directory specified in `.OPTION SEARCH`, for libraries that the simulation references.

VERIFY Produces an element summary list, of input data to print. Calculates effective sizes and key values of elements.

- `BRIEF` suppresses `LIST`.
- `VERIFY` is an alias for `LIST`.

CPU Options

<i>CPTIME</i> = <i>x</i>	Sets the maximum CPU time, in seconds, allotted for this simulation job. When the time allowed for the job exceeds <i>CPTIME</i> , Star-Hspice prints or plots the results up to that point, and concludes the job. Use this option if you are uncertain how long the simulation will take, especially when you debug new data files. Also see <i>LIMTIM</i> . The default value is 1e7 (400 days).
<i>EPSMIN</i> = <i>x</i>	Specifies the smallest number that a computer can add or subtract, a constant value. The default value is 1e-28.
<i>EXPMAX</i> = <i>x</i>	Specifies the largest exponent that you can use for an exponential, before overflow occurs. Typical value for an IBM platform is 350.
<i>LIMTIM</i> = <i>x</i>	Sets the amount of CPU time reserved for generating prints and plots, in case a CPU time limit (<i>CPTIME</i> = <i>x</i>) terminates simulation. The default value is 2 (seconds), which is normally sufficient for short printouts and plots.

Interface Options

<i>ARTIST</i> = <i>x</i>	<i>ARTIST</i> = 2 enables the Cadence Analog Artist interface. This option requires a specific license. Supported on Sun Solaris 2.5/2.7/2.8, HP-UX 10.20 and 11.20, and IBM AIX 4.3 platforms only. Not available on Linux platforms.
<i>CDS</i> , <i>SDA</i>	<i>CDS</i> = 2 produces a Cadence WSF (ASCII format) post-analysis file, for Opus™. This option requires a specific license. <i>SDA</i> is the same as <i>CDS</i> .
<i>CSDF</i>	Selects Common Simulation Data Format (Viewlogic-compatible graph data file format).

- DLENCSDF** If you use the Common Simulation Data Format (Viewlogic graph data file format) as the output format, this *digit length* option specifies how many digits to include, in scientific notation (exponents), or to the right of the decimal point.
- Valid values are any integer from 1 to 10.
 - The default value is 5.
- If you assign a *floating* decimal point, or if you specify less than 1 or more than 10 digits, Star-Hspice uses the default. For example, it places 5 digits to the right of a decimal point.
- MEASOUT** Outputs .MEASURE statement values, and sweep parameters, into an ASCII file. Post-analysis processing (AvanWaves or other analysis tools) uses this *<design>.mt#* file, where # increments for each .TEMP or .ALTER block.
- For example, for a parameter sweep of an output load, which measures the delay, the .mt# file contains data for a delay-versus-fanout plot. The default value is 1. You can set this option to 0 (off) in the hspice.ini file.
- MENTOR = x** MENTOR = 2 enables the Mentor MSPICE-compatible (ASCII) interface. Requires a specific license.
- MONTECON** Continues a Monte Carlo analysis in Star-Hspice. Retrieves the next random value, even if non-convergence occurs. A random value can be too large, or too small, to cause convergence to fail. Other types of analysis can use this Monte Carlo random value.
- POST = x** Stores simulation results for analysis, using the AvanWaves graphical interface or other methods.
- POST = 1 saves the results in binary.
 - POST = 2 saves the results in ASCII format.
 - POST = 3 saves the results in New Wave binary format.

Set the POST option, and use the .PROBE statement to specify the data to save. The default value is 1. To use binary values (with double precision) in the output file, include the following in the input file:

```
*****
.option post (or post=1) post_version=2001
*****
```

For more accurate simulation results, comment this format.

PROBE

Limits the post-analysis output to just the variables designated in .PROBE, .PRINT, .PLOT, and .GRAPH statements. By default, Star-Hspice outputs all voltages and power supply currents, in addition to variables listed in .PROBE/.PRINT/.PLOT/.GRAPH statements. PROBE significantly decreases the size of simulation output files.

PSF = x

Specifies whether Star-Hspice outputs binary or ASCII data, when you run an Avant! IC circuit simulation from Cadence Analog Artist. Supported on Sun Solaris 2.5/2.7/2.8, HP-UX 10.20 and 11.20, and IBM AIX 4.3 platforms only. Not available on Linux platforms.

The value of *x* can be 1 or 2.

- If *x* is 2, Star-Hspice produces ASCII output.
- If .OPTION ARTIST PSF = 1, Star-Hspice produces binary output.

SDA

CDS = 2 produces a Cadence WSF (ASCII) format, post-analysis file, for Opus. This option requires a specific license. SDA is the same as CDS.

ZUKEN = x

- If *x* is 2, enables the Zuken interactive interface.
- If *x* is 1 (the default), disables this interface.

Analysis Options

<i>ASPEC</i>	Sets Star-Hspice to ASPEC-compatibility mode. When you set this option, the simulator reads ASPEC models and netlists, and the results are compatible. The default is 0 (Star-Hspice mode). If you set the ASPEC option, the following model parameters default to ASPEC values: ACM = 1: Changes the default values for CJ, IS, NSUB, TOX, U0, and UTRA. Diode Model: TLEV = 1 affects temperature compensation for PB. MOSFET Model: TLEV = 1 affects PB, PHB, VTO, and PHI. SCALM, SCALE: Sets the model scale factor to microns, for length dimensions. WL: Reverses the implicit order of the MOSFET element that has the specified width and length.
FFTOUT	Prints out 30 harmonic fundamentals, sorted by size, THD, SNR, and SFDR, but only if you specify a .OPTION fftout statement and a .fft freq=xxx statement.
LIMPTS = x	Sets the number of points to print or plot in AC analysis. You do not need to set LIMPTS for DC or transient analysis, because Star-Hspice spools the output file to disk. The default is 2001.

PARHIER Selects parameter-passing rules in Star-Hspice, for the evaluation order of subcircuit parameters. Applies only to parameters with the same name, at different levels of subcircuit hierarchy. The options are:

LOCAL A parameter name in a subcircuit, prevails over the same parameter name at a higher level of hierarchy.
 GLOBAL A parameter name at a higher level of hierarchy.
 Overrides the same parameter name at a lower level.

SPICE Star-Hspice is compatible with Berkeley SPICE. If you set this option, Star-Hspice uses these options and model parameters:

Example of general parameters, used with .OPTION SPICE:

```
TNOM = 27 DEFNRD = 1 DEFNRS = 1 INGOLD = 2
ACOUT = 0 DC
PIVOT PIVTOL = 1E-13 PIVREL = 1E-3 RELTOL = 1E-3
ITL1 = 100
ABSMOS = 1E-6 RELMOS = 1E-3 ABSTOL = 1E-12
VNTOL = 1E-6
ABSVDC = 1E-6 RELVDC = 1E-3 RELI = 1E-3
```

Example of transient parameters, used with .OPTION SPICE:

```
DCAP = 1 RELQ = 1E-3 CHGTOL=1E-14 ITL3 = 4 ITL4 = 10
ITL5 = 5000 FS = 0.125 FT = 0.125
```

Example of model parameters, used with .OPTION SPICE:

```
For BJT: MJS = 0
For MOSFET, CAPOP = 0
LD = 0 if not user-specified
UTRA = 0 not used by SPICE for LEVEL = 2
NSUB must be specified
NLEV = 0 for SPICE noise equation
```

SEED Sets a starting seed for a random-number generator, for Monte Carlo analysis in Star-Hspice. The minimum value is 1; the maximum value is 259200.

Error Options

You can use the following error options in Star-Hspice:

<i>BADCHR</i>	Generates a warning, when it finds a non-printable character in an input file.
<i>DIAGNOSTIC</i>	Logs the occurrence of negative model conductances.
<i>NOWARN</i>	Suppresses all warning messages, except those generated from statements in <code>.ALTER</code> blocks.
<i>WARNLIMIT</i> = <i>x</i>	Limits how many times certain warnings appear in the output listing. This reduces the output listing file size. <i>x</i> is the maximum number of warnings for each warning type. This limit applies to these warning messages: ■ MOSFET has negative conductance. ■ Node conductance is zero. ■ Saturation current is too small. ■ Inductance or capacitance is too large. The default value is 1.

Version Options

You can use the following version options in Star-Hspice:

<i>H9007</i>	Sets default values for general-control options, to correspond to the values for Star-Hspice Release H9007D. If you set this option, Star-Hspice does not use the <code>EXPLI</code> model parameter.
--------------	---

Model Analysis Options

General Options

DCAP The DCAP option selects equations, which Star-Hspice uses to calculate depletion capacitance for Level 1 and 3 diodes, and BJTs. The *True-Hspice Device Models Reference Manual* describes these equations.

MODSRH If MODSRH=1, Star-Hspice does not load or reference a model described in a .MODEL statement, if the netlist does not use that model. This option shortens simulation run time, when the netlist references many models, but no element in the netlist calls those models. The default value is MODSRH=0. If MODSRH=1, then the read-in time increases slightly.

```
example.sp:
* modsrh used incorrectly
.option post modsrh=1
xi1 net8 b c t6
xi0 a b net8 t6
v1 a 0 pulse 3.3 0.0 10E-6 1E-9 1E-9
+ 25E-6 50E-6
v2 b 0 2
v3 c 0 3
.model nch nmos level=49 version=3.2
.end
```

This input file automatically searches for t6.inc. If t6.inc includes the nch model, and you set MODSRH to 1, Star-Hspice does not load nch. Do not set MODSRH=1 in this type of file call. Use this option in front of the .MODEL card definition.

SCALE Element scaling factor, in Star-Hspice. This option scales parameters in element cards, by their value. The default value is 1.

- HIER_SCALE** If you set the HIER_SCALE option, you can use the S parameter to scale sub-circuits.
- 0 interprets S as a user-defined parameter.
 - 1 interprets S as a scale parameter.
- For more information about the S parameter, see [S \(Scale\) Parameter on page 3-53](#).
- TNOM** The reference temperature for Star-Hspice simulation. At this temperature, component derating is zero. The default is 25 degrees Celsius; if you enable .OPTION SPICE, the default is 27 degrees Celsius.
- MODMONTE**
- If MODMONTE=1, then within a single simulation run, each device that shares the same model card and is in the same Monte Carlo index, receives a different random value for its parameters that have a Monte Carlo definition.
 - If MODMONTE=0 (the default), then within a single simulation run, each device that shares the same model card and is in the same Monte Carlo index, receives the same random value for its parameters that have a Monte Carlo definition.

MOSFET Control Options

<i>CVTOL</i>	Changes the number of numerical integration steps, when calculating the gate capacitor charge for a MOSFET, using $CAPOP = 3$. See the discussion of $CAPOP = 3$ in the “Overview of MOSFETS” chapter of the <i>True-Hspice Device Models Reference Manual</i> , for explicit equations and discussion.
<i>DEFAD</i>	Default value, for MOSFET drain diode area, in Star-Hspice. The default value is 0.
<i>DEFAS</i>	Default value, for MOSFET source diode area, in Star-Hspice. The default value is 0.
<i>DEFL</i>	Default value, for MOSFET channel length, in Star-Hspice. The default value is $1e^{-4}m$.
<i>DEFNRD</i>	Default value, for the number of squares for the drain resistor, on a MOSFET. The default value is 0.
<i>DEFNRS</i>	Default value, for the number of squares for the source resistor, on a MOSFET. The default value is 0.
<i>DEFPD</i>	Default value, for the MOSFET drain diode perimeter, in Star-Hspice. The default value is 0.
<i>DEFPS</i>	Default value, for the MOSFET source diode perimeter, in Star-Hspice. Default value is 0.
<i>DEFW</i>	Default value, for the MOSFET channel width, in Star-Hspice. The default value is $1e^{-4}m$.
<i>SCALM</i>	Model scaling factor, in Star-Hspice. Scales model parameters by their value. Default is 1. See the <i>True-Hspice Device Models Reference Manual</i> , for parameters this option scales.
<i>WL</i>	Changes the specified order, in the VSIZE MOS element. Default order is length-width; this option changes the order to width-length. The default value is 0.

Inductor Options

You can use the following inductor options in Star-Hspice:

- GENK* Automatically computes second-order mutual inductance, for several coupled inductors. A value of 1 (the default) enables the calculation.
- KLIM* Minimum mutual inductance, below which automatic second-order mutual inductance calculation no longer proceeds. *KLIM* is unitless (analogous to coupling strength, specified in the K Element). Typical *KLIM* values are between .5 and 0.0. The default value is 0.01.

BJT and Diode Options

- EXPLI* Current-explosion model parameter. PN junction characteristics, above the explosion current, are linear. Star-Hspice determines the slope at the explosion point. This improves simulation speed and convergence. The default is 0.0 amp/AREAeff.

DC Operating Point, DC Sweep, and Pole/Zero Options

Accuracy Options

ABSH = *x* Sets the absolute current change, through voltage-defined branches (voltage sources and inductors). Use ABSH with DI and RELH, to check for current convergence. The default is 0.0.

ABSI = *x* Sets the absolute branch current error tolerance (in diodes, BJTs, and JFETs), during DC and transient analysis. Decrease ABSI, if accuracy is more important than convergence time.

- To analyze currents less than 1 nanoamp, change ABSI to a value at least two orders of magnitude smaller than the minimum expected current.
- The default value is 1e-9 for KCLTEST = 0, or 1e-6 for KCLTEST = 1.

ABSMOS = *x* Current error tolerance (for a MOSFET device), in DC or transient analysis. The ABSMOS setting determines whether the drain-to-source current solution has converged. The drain-to-source current converged if:

- The difference between the drain-to-source current in the last iteration, versus the present iteration, is less than ABSMOS, or
- This difference is greater than ABSMOS, but the percent change is less than RELMOS.

If other accuracy tolerances also indicate convergence, Star-Hspice solves the circuit at that timepoint, and calculates the next timepoint solution. For low-power circuits, optimization, and single transistor simulations, set ABSMOS = 1e-12. The default value is 1e-6 (amperes).

ABSTOL = *x* Sets the absolute error tolerance for branch currents. Decrease ABSTOL, if accuracy is more important than convergence time. ABSTOL is the same as ABSI.

ABSVDC = *x* Sets the minimum voltage, for DC and transient analysis. If accuracy is more critical than convergence, decrease ABSVDC. For voltages less than 50 microvolts, reduce ABSVDC, to two orders of magnitude less than the smallest desired voltage; this ensures at least two digits of significance. Typically, you do not need to change ABSVDC, unless you simulate a high-voltage circuit. For 1000-volt circuits, a reasonable value is 5 to 50 millivolts. The default value is VNTOL (VNTOL default = 50 μ V).

DI = *x* Sets the maximum iteration-to-iteration current change, through voltage-defined branches (voltage sources and inductors). Use this option only if the value of the ABSH control option is greater than 0. The default value is 0.0.

KCLTEST Activates the KCL (Kirchhoff's Current Law) test. This test increases simulation time, especially for large circuits, but it very accurately checks the solution. The default value is 0.

If you set this value to 1, Star-Hspice sets these options:

- Sets RELMOS and ABSMOS options to 0 (off).
- Sets ABSI to 1e-6 A.
- Sets RELI to 1e-6.

To satisfy the KCL test, each node must satisfy this condition:

$$|\sum i_b| < RELI \cdot \sum |i_b| + ABSI$$

where the i_b s are the node currents.

MAXAMP = *x* Sets the maximum current, through voltage-defined branches (voltage sources and inductors). If the current exceeds the MAXAMP value, Star-Hspice issues an error. The default is 0.0.

<i>RELH</i> = <i>x</i>	Sets the relative current tolerance, through voltage-defined branches (voltage sources and inductors), and checks current convergence. Use this option only if the ABSH control value is greater than zero. The default is 0.05.
<i>RELI</i> = <i>x</i>	Sets the relative error/tolerance change, from iteration to iteration. This parameter determines convergence for all currents, in diode, BJT, and JFET devices. (RELMOS sets the tolerance for MOSFETs). This is the change in current, from the value calculated at the previous timepoint. The default value is 0.01 for KCLTEST = 0, or 1e-6 for KCLTEST = 1.
<i>RELMOS</i> = <i>x</i>	Sets the error-tolerance percent, for the relative drain-to-source current, from iteration to iteration. This parameter determines convergence, for currents in MOSFET devices. (RELI sets the tolerance for other active devices.) This is the change in current, since the previous timepoint. Star-Hspice uses RELMOS only when the current is greater than the ABSMOS floor value. The default value is 0.05.
<i>RELV</i> = <i>x</i>	Sets the relative error tolerance, for voltages. When voltages or currents exceed their absolute tolerances, the RELV test determines convergence. Increasing RELV increases the relative error. You should generally keep RELV at its default value. RELV controls simulator charge conservation. For voltages, RELV is the same as RELTOL. The default value is 1e-3.
<i>RELVDC</i> = <i>x</i>	Sets the relative error tolerance, for voltages. When voltages or currents exceed their absolute tolerances, the RELVDC test determines convergence. Increasing RELVDC increases the relative error. You should generally keep RELVDC at its default value. RELVDC controls simulator charge conservation. The default value is RELTOL (RELTOL default = 1e-3).

Matrix Options

You can use the following matrix-related options in Star-Hspice:

- ITL1* = *x* Sets the maximum DC iteration limit. Increasing this value rarely improves convergence for small circuits. Values as high as 400 have resulted in convergence for some large circuits with feedback, such as operational amplifiers and sense amplifiers. However, most models do not require more than 100 iterations for convergence. Set `.OPTION ACCT`, to list how many iterations are required for an operating point. The default value is 200.
- ITL2* = *x* Sets the iteration limit for the DC transfer curve. Increasing the iteration limit improves convergence, *only* if the circuit is very large. The default value is 50.
- NOPIV* Prevents Star-Hspice from automatically switching to pivoting matrix factors, when a node conductance is less than `PIVTOL`. `NOPIV` inhibits pivoting (see `PIVOT`).
- PIVOT* = *x* Provides different pivot algorithm selections. These algorithms reduce simulation time, and achieve convergence in circuits that produce hard-to-solve matrix equations. To select the pivot algorithm, set `PIVOT` to one of the following values:
- 0: Original non-pivoting algorithm.
 - 1: Original pivoting algorithm.
 - 2: Picks the algorithm for the largest pivot in the row.
 - 3: Picks the best-in-row algorithm.
 - 10: Fast, non-pivoting algorithm; requires more memory.
 - 11: Fast, pivoting algorithm; requires more memory than `PIVOT` values less than 11.
 - 12: Picks the algorithm for the largest pivot in the row; requires more memory than `PIVOT` values less than 12.
 - 13: Fast, best pivot: faster; requires more memory than `PIVOT` values less than 13.

The default value is 10.

The fastest algorithm is `PIVOT = 13`, which can improve simulation time by up to ten times, on very large circuits. However, the `PIVOT = 13` option requires substantially more memory for the simulation. Some circuits with large conductance ratios, such as switching regulator circuits, might need pivoting. If `PIVTOL = 0`, Star-Hspice automatically changes from non-pivoting, to a row pivot strategy, if it detects any diagonal matrix entry that is less than `PIVTOL`. This strategy provides the time and memory advantages of non-pivoting inversion, but it also avoids unstable simulations and incorrect results. Use `.OPTION NOPIV` to prevent Star-Hspice from using pivots.

For very large circuits, `PIVOT = 10, 11, 12, or 13` can require excessive memory.

If Star-Hspice switches to pivoting during a simulation, it prints the message:

```
        pivot change on the fly
```

followed by the node numbers that cause the problem. Use `.OPTION NODE` to obtain a node-to-element cross reference.

`SPARSE` is the same as `PIVOT`.

PIVREF Pivot reference. Use `PIVREF` in `PIVOT = 11, 12, or 13`, to limit the size of the matrix. The default value is `1e+8`.

PIVREL = x Sets the maximum and minimum row/matrix ratio. Use only for `PIVOT = 1`. Large values for `PIVREL` can result in very long matrix pivot times. If the value is too small, however, no pivoting occurs. Start with small values of `PIVREL`, using an adequate (but not excessive) value, for convergence and accuracy. The default value is `1E-20` (`max = 1e-20, min = 1`).

PIVTOL = *x* Sets the absolute minimum value for which Star-Hspice accepts a matrix entry as a pivot. PIVTOL is the minimum conductance in the matrix, when PIVOT = 0. The default value is 1.0e-15.

Note: Set PIVTOL to a value less than GMIN or GMINDC. Values that approach 1, yield increased pivot.

SPARSE = *x* Selects different pivoting algorithms. These algorithms reduce simulation time, and achieve convergence in circuits that produce hard-to-solve matrix equations. To select the pivot algorithm, set PIVOT to one of the following values:

- 0: Original non-pivoting algorithm.
- 1: Original pivoting algorithm.
- 2: Picks the algorithm for the largest pivot in the row.
- 3: Picks the best-in-row algorithm.
- 10: Fast, non-pivoting algorithm; requires more memory.
- 11: Fast, pivoting algorithm; requires more memory than PIVOT values less than 11.
- 12: Picks the algorithm for the largest pivot in the row; requires more memory than PIVOT values less than 12.
- 13: Fast, best pivot: faster; requires more memory than PIVOT values less than 13.

The default value is 10.

The fastest algorithm is PIVOT = 13, which can improve simulation time by up to ten times, on very large circuits. However, the PIVOT = 13 option requires substantially more memory for the simulation. Some circuits with large conductance ratios, such as switching regulator circuits, might need pivoting.

If `PIVTOL = 0`, Star-Hspice automatically changes from non-pivoting, to a row pivot strategy, if it detects any diagonal matrix entry that is less than `PIVTOL`. This strategy provides the time and memory advantages of non-pivoting inversion, but it also avoids unstable simulations and incorrect results. Use `.OPTION NOPIV` to prevent Star-Hspice from using pivots.

For very large circuits, `PIVOT = 10, 11, 12, or 13` can require excessive memory.

If Star-Hspice switches to pivoting during a simulation, it prints the message:

pivot change on the fly

followed by the node numbers that cause the problem. Use `.OPTION NODE` to obtain a node-to-element cross reference.

`SPARSE` is the same as `PIVOT`.

Pole/Zero Input and Output Options

You can use the following pole/zero input and output options in Star-Hspice:

<i>CAPTAB</i>	Prints a table of single-plate node capacitances, for diodes, BJTs, MOSFETs, JFETs, and passive capacitors, at each operating point.
<i>DCCAP</i>	Generates C-V plots, and prints the capacitance values of a circuit (both model and element), during a DC analysis. You can use a DC sweep of the capacitor, to generate C-V plots. The default value is 0 (off).
<i>VFLOOR = x</i>	Sets a lower limit, for voltages that print in the output listing. Star-Hspice prints all voltages that are lower than <code>VFLOOR</code> , as 0. This affects only the output listing: <code>VNTOL (ABSV)</code> sets the minimum voltage used in a simulation.

Convergence Options

CONVERGE	Invokes different methods to solve non-convergence problems. CONVERGE = -1 Together with DCON = -1, disables autoconvergence. CONVERGE = 0 Autoconvergence (default). CONVERGE = 1 Uses the Damped Pseudo Transient algorithm. If simulation fails to converge within the amount of CPU time (set in the CPTIME control option), simulation halts. CONVERGE = 2 Uses a combination of DCSTEP and GMINDC ramping. Not used in the autoconvergence flow. CONVERGE = 3 Invokes the source-stepping method. Not used in the autoconvergence flow. CONVERGE = 4 Uses the <i>gmath</i> ramping method. Even you did not set it in an .OPTION statement, the CONVERGE option activates if a matrix floating-point overflows, or if Star-Hspice reports a <i>timestep too small</i> error. Default = 0. If a matrix floating-point overflows, then CONVERGE = 1.
CSHDC	The same option as CSHUNT; use only with the CONVERGE option.
DCFOR = x	Used in conjunction with the DCHOLD option, and the .NODESET statement, to enhance the DC convergence properties of a simulation. DCFOR sets the number of iterations to calculate, after a circuit converges in the steady state. The number of iterations after convergence is usually zero, so DCFOR adds iterations (and computation time) to the DC circuit solution. DCFOR ensures that a circuit actually, not falsely, converges. The default is 0.

DCHOLD = *x* Use DCFOR and DCHOLD together, to initialize a DC analysis. DCFOR and DCHOLD enhance the convergence properties of a DC simulation. DCFOR and DCHOLD work with the .NODESET statement.

DCHOLD specifies the number of iterations, during which Star-Hspice maintains a node at the voltage values specified in a .NODESET statement. The effects of DCHOLD on convergence differ, according to the DCHOLD value, and the number of iterations needed to obtain DC convergence.

- If a circuit converges in the steady state, in fewer than DCHOLD iterations, then the DC solution includes the values set in the .NODESET statement.
- If a circuit requires more than DCHOLD iterations to converge, Star-Hspice ignores the values set in the .NODESET statement. Star-Hspice then calculates the DC solution, with the .NODESET fixed-source voltages open circuited.

The default value is 1.

DCON = *X* If a circuit cannot converge, Star-Hspice automatically sets *DCON* = 1, and calculates the following:

$$DV = \max\left(0.1, \frac{V_{max}}{50}\right), \text{ if } DV = 1000$$

$$GRAMP = \max\left(6, \log_{10}\left(\frac{I_{max}}{GMINDC}\right)\right)$$

$$ITL1 = ITL1 + 20 \cdot GRAMP$$

where V_{max} is the maximum voltage, and I_{max} is the maximum current.

If convergence problems persist, Star-Hspice sets $DCON = 2$ (the same as above, except $DV = 1e6$). Star-Hspice uses the above calculations if $DCON = 1$ or 2. Star-Hspice automatically invokes $DCON = 1$, if the circuit fails to converge, and then invokes $DCON = 2$ if $DCON = 1$ fails.

If the circuit contains uninitialized flip-flops or discontinuous models, the simulation might not converge. Set $DCON = -1$ and $CONVERGE = -1$, to disable autoconvergence. Star-Hspice then lists all non-convergent nodes and devices.

DCSTEP = x Converts DC model and element capacitors to conductance, to enhance DC convergence properties. Star-Hspice divides the value of the element capacitors by *DCSTEP*, to obtain a DC conductance model. The default value is 0 (seconds).

DCTRAN Invokes different methods, to solve non-convergence problems in Star-Hspice:

CONVERGE = -1

Together with $DCON = -1$, disables auto-convergence.

CONVERGE = 0

Autoconvergence (default).

CONVERGE = 1

Damped Pseudo Transient algorithm. If simulation fails to converge, within the amount of CPU time set in the CPTIME control option, then simulation halts.

CONVERGE = 2

Uses a combination of *DCSTEP* and *GMINDC* ramping. Not used in the autoconvergence flow.

CONVERGE = 3

Invokes the source-stepping method. Not used in the autoconvergence flow.

CONVERGE = 4

Uses the *gmath* ramping method.

Even if you do not set it in an `.OPTION` statement, the `CONVERGE` option activates, if a matrix floating point overflows, or if a timestep is too small. The default is 0.

If a matrix floating-point overflows, Star-Hspice sets `CONVERGE = 1`. `DCTRAN` is an alias for `CONVERGE`.

- DV = x* Specifies the maximum iteration-to-iteration voltage change, for all circuit nodes (in both DC and transient analysis). Requires high-gain bipolar amplifiers values of 0.5 to 5.0, to achieve a stable DC operating point. CMOS circuits frequently require a value of about 1 volt, for large digital circuits. The default value is 1000 (or 1e6 if `DCON = 2`).
- GMAX = x* Specifies conductance, in parallel with the current source, used for `.IC` and `.NODESET` initialization circuitry. Some large bipolar circuits require you to set `GMAX` to 1, for convergence. The default value is 100 (mho).
- GMINDC = x* DC analysis uses conductance, in parallel to all pn junctions and MOSFET nodes. `GMINDC` helps overcome DC convergence problems, caused by low values of off-conductance, for pn junctions and MOSFETs. You can use `GRAMP` to reduce `GMINDC`, by one order of magnitude, for each step. Set `GMINDC` between 1e-4 and the `PIVTOL` value. The default is 1e-12.
- Large values of `GMINDC` can cause unreasonable circuit response. If convergence requires large values, suspect a bad model or circuit. If a matrix floating-point overflows, and if `GMINDC` is 1.0e-12 or less, Star-Hspice sets it to 1.0e-11. Star-Hspice manipulates `GMINDC` in auto-converge mode.

<i>GRAMP</i> = <i>x</i>	Star-Hspice sets the value during autoconvergence. Use GRAMP, with the GMINDC convergence control option, to find the smallest value of GMINDC that results in DC convergence. GRAMP specifies the conductance range, over which a DC operating-point analysis sweeps GMINDC. Star-Hspice substitutes values of GMINDC over this range, and simulates at each value. It then picks the lowest value of GMINDC, at which the circuit converges in a steady state. If you sweep GMINDC between 1e-12 mhos (the default) and 1e-6 mhos, Star-Hspice sets GRAMP to 6 (the value of the exponent difference, between the default and the maximum conductance limit). In this case, Star-Hspice sets GMINDC to 1e-6 mhos, and simulates the circuit. If convergence occurs, Star-Hspice sets GMINDC to 1e-7 mhos, and simulates the circuit again. The sweep continues, until Star-Hspice has simulated all GRAMP values. If the combined conductance of GMINDC and GRAMP is greater than 1e-3 mho, a false convergence can occur. The default value is 0.
<i>GSHUNT</i>	Conductance, added from each node to ground. The default value is zero. Add a small GSHUNT to each node, to help solve <i>Timestep too small</i> problems, caused by either high-frequency oscillations or numerical noise.
<i>ICSWEEP</i>	Saves the current analysis result of a parameter or temperature sweep, as the starting point in the next analysis in the sweep. ■ If ICSWEEP = 1 (the default), Star-Hspice uses the current results in the next analysis. ■ If ICSWEEP = 0, Star-Hspice does not use the results of the current analysis in the next analysis.

<i>ITLPT</i> <i>TRAN</i>	Controls the iteration limit used in the final try of the pseudo-transient method, in OP or DC analysis. If your simulation fails in the final try of the pseudo-transient method, you can enlarge this option. The default value is 30.
<i>NEWTOL</i>	Calculates one or more iterations past convergence, for every calculated DC solution, and timepoint circuit solution. If you do not set <i>NEWTOL</i> , after Star-Hspice determines convergence, the convergence routine ends, and the next program step begins. The default value is 0.
<i>OFF</i>	For all active devices, initializes terminal voltages to zero, if you did not initialize them to other values. For example, if you did not initialize both the drain and source nodes of a transistor (using <i>.NODESET</i> or <i>.IC</i> statements, or connecting them to sources), then <i>OFF</i> initializes all nodes of the transistor to zero. Star-Hspice checks the <i>OFF</i> option, before element <i>IC</i> parameters. If you assigned an element <i>IC</i> parameter to a node, Star-Hspice initializes the node to the element <i>IC</i> parameter value, even if the <i>OFF</i> option previously set it to zero. You can use the <i>OFF</i> element parameter to initialize terminal voltages to zero, for particular active devices. Use the <i>OFF</i> option to help find exact DC operating-point solutions, for large circuits.
<i>RESMIN</i> = <i>x</i>	Specifies the minimum resistance value, for all resistors, including parasitic and inductive resistances. The default value is 1e-5 (ohm). Range: 1e-15 to 10 ohm.

Pole/Zero Control Options

You can use the following pole/zero options in Star-Hspice:

<i>CSCAL</i>	Sets the capacitance scale. Star-Hspice multiplies capacitances by <i>CSCAL</i> . The default value is 1e+12 (capacitances in pF).
<i>FMAX</i>	Sets the maximum value for pole and zero angular frequency. The default value is 1.0e+12 rad/sec.
<i>FSCAL</i>	Sets the frequency scale. Star-Hspice multiplies the frequency by <i>FSCAL</i> . The default value is 1e-9 (that is, by default, you enter all frequencies in units of GHz).
<i>GSCAL</i>	Sets the conductance scale. Star-Hspice multiplies conductances by <i>GSCAL</i> , and divides resistances by <i>GSCAL</i> . The default value is 1e+3 (that is, by default, you enter all resistances in units of k Ω).
<i>LSCAL</i>	Sets the inductance scale. Star-Hspice multiplies inductances by <i>LSCAL</i> . The default value is 1e+6 (that is, by default, you enter all inductances in units of μ H).

The scale factors must satisfy the following relations:

$$GSCA = CSCAL \cdot FSCAL$$

$$GSCAL = \frac{1}{LSCAL} \cdot FSCAL$$

If you change scale factors, you might need to modify the initial Muller points (X0R, X0I), (X1R, X1I), and (X2R, X2I), even though Star-Hspice multiplies initial values by (1e-9/GSCAL).

<i>PZABS</i>	Sets absolute tolerances, for poles and zeros. This option affects low-frequency poles or zeros. Use it as follows:
--------------	---

$$\text{If } (|X_{real}| + |X_{imag}| < PZABS),$$

$$\text{then } X_{real} = 0 \text{ and } X_{imag} = 0.$$

You can also use it for convergence tests. The default is 1e-2.

PZTOL Sets the relative error tolerance, for poles or zeros. The default value is 1.0×10^{-6} .

RITOL Sets a minimum ratio for (real/imaginary), or (imaginary/real) parts of the poles or zeros. Use *RITOL* as follows:

If $|X_{imag}| \leq RITOL \cdot |X_{real}|$, then $X_{imag} = 0$.

If $|X_{real}| \leq RITOL \cdot |X_{imag}|$, then $X_{real} = 0$.

The default value is 1.0×10^{-6} .

(X0R,X0I), The three complex starting points, in the Muller pole/zero analysis algorithm, are:

(X1R,X1I), $X0R = -1.23456 \times 10^6$ $X0I = 0.0$
 $X1R = -1.23456 \times 10^5$ $X1I = 0.0$
(X2R,X2I) $X2R = +1.23456 \times 10^6$ $X2I = 0.0$

Star-Hspice multiplies these initial points by *FSCAL*.

Transient and AC Small Signal Analysis Options

Accuracy Options

ABSH = *x* Sets the absolute current change, through voltage-defined branches (voltage sources and inductors). Use ABSH with DI and RELH, to check for current convergence. The default is 0.0.

ABSV = *x* Sets the minimum voltage for DC and transient analysis. ABSV is the same as VNTOL. If accuracy is more critical than convergence, decrease VNTOL. For voltages less than 50 microvolts, reduce VNTOL to two orders of magnitude less than the smallest desired voltage. This ensures at least two significant digits. Typically, you do not need to change VNTOL, except to simulate a high-voltage circuit. For 1000-volt circuits, a reasonable value is 5 to 50 millivolts. The default value is 50 (microvolts).

ACCURATE Selects a time algorithm that uses LVLTIM = 3 and DVDT = 2, for circuits such as high-gain comparators. Use this option with circuits that combine high gain and large dynamic range, to guarantee accurate solutions in Star-Hspice. When set to 1, ACCURATE sets these control options:

- LVLTIM = 3
- DVDT = 2
- RELVAR = 0.2
- ABSVAR = 0.2
- FT = 0.2
- RELMOS = 0.01

The default value is 0.

<i>ACOUT</i>	AC output calculation method, for the difference in values of magnitude, phase, and decibels. Use this option for prints and plots. The default value is 1. The default value, <i>ACOUT</i> = 1, selects the Star-Hspice method, which calculates the difference of the magnitudes of the values. The SPICE method, <i>ACOUT</i> = 0, calculates the magnitude of the differences in Star-Hspice.
<i>CHGTOL</i> = <i>x</i>	Sets the charge error tolerance, when <i>LVLTIM</i> = 2 is set. <i>CHGTOL</i>, along with <i>RELQ</i>, sets the absolute and relative charge tolerance, for all Star-Hspice capacitances. The default value is 1e-15 (coulomb).
<i>CSHUNT</i>	Capacitance, added from each node to ground, in Star-Hspice. Add a small <i>CSHUNT</i> to each node, to solve some <i>internal timestep too small</i> problems, caused by high-frequency oscillations or numerical noise. The default is 0.
<i>DI</i> = <i>x</i>	Sets the maximum iteration-to-iteration current change, through voltage-defined branches (voltage sources and inductors). Use this option only when the value of the <i>DI</i> control option is greater than 0. The default value is 0.0.
<i>GMIN</i> = <i>x</i>	Sets the minimum conductance, in a transient analysis time sweep. The default value is 1e-12.
<i>GSHUNT</i>	Conductance, added from each node to ground. The default is zero. Adding a small <i>GSHUNT</i> to each node can solve some <i>internal timestep too small</i> problems, caused by high-frequency oscillations or numerical noise.
<i>MAXAMP</i> = <i>x</i>	Sets the maximum current, through voltage-defined branches (voltage sources and inductors). If the current exceeds the <i>MAXAMP</i> value, Star-Hspice issues an error. The default is 0.0.

<i>RELH</i> = <i>x</i>	Sets relative current tolerance, through voltage-defined branches (voltage sources and inductors). Use RELH to check current convergence, but <i>only</i> if the value of the ABSH control option is greater than zero. The default value is 0.05.
<i>RELI</i> = <i>x</i>	Sets the relative error/tolerance change, from iteration to iteration. Determines convergence for all currents in diode, BJT, and JFET devices. (RELMOS sets tolerance for MOSFETs). This is the change in current, from the value calculated at the previous timepoint. The default is 0.01 for KCLTEST = 0, or 1e-6 for KCLTEST = 1.
<i>RELQ</i> = <i>x</i>	Used in the timestep algorithm, for local truncation error (LVLTIM = 2). RELQ changes the size of the timestep. If the capacitor charge calculation for the present iteration, exceeds that of the past iteration, by a percentage greater than the value of RELQ, then Star-Hspice reduces the internal timestep (Delta). The default value is 0.01.
<i>RELTOL</i> , <i>RELV</i>	Sets the relative error tolerance for voltages. Use RELV, with the ABSV control option, to determine voltage convergence. Increasing RELV increases relative error. RELV is the same as RELTOL. RELI and RELVDC options default to the RELTOL value. The default value is 1e-3.
<i>RISETIME</i>	Specifies the smallest risetime of a signal, .OPTION RISETIME = <i>x</i> . Use it only in transmission line models, in Star-Hspice. In the U Element, the following equation determines the number of lumps:

$$\text{MIN}\left[20, 1 + \left(\frac{TDeff}{RISETIME}\right) \cdot 20\right]$$

where *TDeff* is the end-to-end delay in a transmission line. The W Element uses RISETIME, *only* if *R_s* or *G_d* is non-zero. In such cases, RISETIME determines the maximum signal frequency.

<i>TRTOL</i> = <i>x</i>	Used in the timestep algorithm for local truncation error (<i>LVLTIM</i> = 2). Star-Hspice multiplies <i>TRTOL</i> by the internal timestep, which the timestep algorithm for the local truncation error generates. <i>TRTOL</i> reduces simulation time, and maintains accuracy. It estimates the amount of error introduced, when the algorithm truncates the Taylor series expansion. This error reflects the minimum time-step, to reduce simulation time and maintain accuracy. The range of <i>TRTOL</i> is 0.01 to 100; typical values are 1 to 10. If you set <i>TRTOL</i> to 1 (the minimum value), Star-Hspice uses a very small timestep. As you increase the <i>TRTOL</i> setting, the timestep size increases. The default is 7.0.
<i>VNTOL</i> = <i>x</i> , <i>ABSV</i>	Sets the minimum voltage, for DC and transient analysis. <i>ABSV</i> is the same as <i>VNTOL</i> . Decrease <i>VNTOL</i> , if accuracy is more critical than convergence. If you need voltages less than 50 microvolts, reduce <i>VNTOL</i> to two orders of magnitude less than the smallest desired voltage. This ensures at least two significant digits. Typically, you change <i>VNTOL</i> only if you simulate a high-voltage circuit. For 1000-volt circuits, a reasonable value is 5 to 50 millivolts. The default value is 50 (microvolts).

Speed Options

AUTOSTOP Stops a transient analysis in Star-Hspice, after calculating all TRIG-TARG and FIND-WHEN measure functions. This option can substantially reduce CPU time. By default, if the data file contains measure functions (such as AVG, RMS, MIN, MAX, PP, ERR, ERR1, 2, 3, or PARAM), then AUTOSTOP is disabled (that is, .OPTION autostop or .OPTION autostop from_to=0 is set). To use AUTOSTOP with these measure functions, set .OPTION autostop from_to or .OPTION autostop from_to=1.

For trig-targ and find-when measure functions, if you set autostop, do not use the preceding measure result as the measured parameter. Otherwise, the measured result is probably inaccurate.

BKPSIZ = x Sets the size of the breakpoint table. The default value is 5000. This is an old option, provided only for backward-compatibility.

BYPASS To speed-up simulation in Star-Hspice, this option does not update the status of latent devices. Set .OPTION BYPASS = 1, to enable bypassing. BYPASS applies to MOSFETs, MESFETs, JFETs, BJTs, and diodes. The default value is 1.

Note: Use the BYPASS algorithm cautiously. Some circuit types might not converge, and might lose accuracy in both transient analysis and operating-point calculations.

BYTOL = x Specifies the tolerance for the voltage, at which a MOSFET, MESFET, JFET, BJT, or diode becomes latent. Star-Hspice does not update the status of latent devices. The default value is MBYPASSxVNTOL.

FAST

To speed-up simulation, does not update the status of latent devices. Use this option for MOSFETs, MESFETs, JFETs, BJTs, and diodes. The default is 0.

A device is latent, if its node voltage variation (from one iteration to the next) is less than the value of either the BYTOL control option, or the BYPASSTOL element parameter. (When FAST is on, Star-Hspice sets BYTOL to different values, for different types of device models.)

Besides the FAST option, you can also use the NOTOP and NOELCK options to reduce input pre-processing time. Increasing the value of the MBYPASS or BYTOL option, also helps simulations to run faster, but can reduce accuracy.

ITLPZ

Sets the iteration limit, for pole/zero analysis. The default value is 100.

MBYPASS = x

Computes the default value of the BYTOL control option:

$$\text{BYTOL} = \text{MBYPASS} \times \text{VNTOL}$$

Also multiplies the RELV voltage tolerance. Set MBYPASS to about 0.1, for precision analog circuits.

- The default value is 1, for DVDT = 0, 1, 2, or 3.
- The default value is 2, for DVDT = 4.

TRCON

Controls the speed of some special circuits. For some large non-linear circuits with large TSTOP/TSTEP values, analysis might run for an excessively long time. In this case, Star-Hspice might automatically set a new and bigger RMAX value, to speed up the analysis for primary reference. In most cases, however, Star-Hspice does not activate this type of autospeedup process.

For autospeedup to occur, all three of the following conditions must occur:

- N1 (Number of Nodes) > 1,000
- N2 (TSTOP/TSTEP) >= 10,000
- N3 (Total Number of Diode, BJTs, JFETs and MOSFETs) > 300

Autospeedup is most likely to occur if the circuit also meets either of the following conditions:

- N2 >= 1e+8, and N3 > 500, or
- N2 >= 2e+5, and N3 > 1e+4

If Star-Hspice *does* activate autospeedup, you might need to disable it. To do this, set TRCON=-1, and increase TSTEP or RMAX (or both), to balance accuracy and speed.

- TRCON = 0 or TRCON=1 enables autospeedup for circuits that meet necessary conditions.
- TRCON = -1 disables autospeedup.

The default value of TRCON is 1.

TRCON also controls the automatic convergence process. See [Algorithm Options on page 9-47](#).

Timestep Options

- ABSVAR* = *x* Sets the limit for the maximum voltage change, from one time point to the next. Use this option with the DVDT algorithm. If the simulator produces a convergent solution that is greater than ABSVAR, then Star-Hspice discards the solution, sets the timestep to a smaller value, and recalculates the solution. This is called a timestep reversal. The default value is 0.5 (volts).
- DELMAX* = *x* Sets the maximum value for the Delta of the internal timestep. Star-Hspice automatically sets the DELMAX value, based on the factors listed in [Timestep Control for Accuracy on page 11-32](#). The initial DELMAX value, shown in the Star-Hspice output listing, is generally not the value used for simulation.
- DVDT* Adjusts the timestep, based on rates of change for node voltage. Choices are:
- 0 - original algorithm
 - 1 - fast
 - 2 - accurate
 - 3,4 - balance speed and accuracy
- The default value is 4.
- FS* = *x* Sets the fraction of a timestep (TSTEP), by which the Delta (internal timestep) decreases, for the first time point of a transient. Decreasing the FS value helps circuits that have timestep convergence difficulties. You can also use it in the DVDT = 3 method, to control the timestep.

$$\Delta = FS \times [\text{MIN}(TSTEP, DELMAX, BKPT)]$$

where you specify DELMAX, and BKPT relates to the breakpoint of the source. Set TSTEP in the .TRAN statement. The default value is 0.25.

$FT = x$	Sets the fraction of a timestep (TSTEP), by which the Delta (internal timestep) decreases, for an iteration set that does not converge. You can also use this value in DVDT = 2 and DVDT = 4, to control the timestep. The default is 0.25.
$IMIN = x,$ $ITL3 = x$	Determines the timestep, in transient analysis simulations. IMIN sets the minimum number of iterations required, to obtain convergence. If the number of iterations is less than IMIN, then the internal timestep (Delta) doubles. Use this option to decrease simulation times, in circuits where the nodes are stable most of the time (such as digital circuits). If the number of iterations is greater than IMIN, the timestep stays the same, unless the timestep exceeds the IMAX option. ITL3 is the same as IMIN. The default is 3.0.
$IMAX = x,$ $ITL4 = x$	Determines the maximum timestep, in transient analysis simulations. IMAX sets the maximum number of iterations, to obtain a convergent solution at a timepoint. If the number of iterations needed is greater than IMAX, the internal timestep (Delta) decreases by a factor equal to the FT transient control option, and uses the new timestep to calculate a new solution. IMAX also works with the IMIN transient control option. ITL4 is the same as IMAX. The default value is 8.0.
$ITL3 = x$	Determines the minimum timestep, in transient analysis simulations. IMIN sets the minimum number of iterations required, to obtain convergence. If the number of iterations is less than IMIN, the internal timestep (Delta) doubles. Use this option to decrease simulation times, in circuits where the nodes are stable most of the time (such as digital circuits). If the number of iterations is greater than IMIN, the timestep remains the same, unless the timestep exceeds the IMAX option. ITL3 is the same as IMIN. The default value is 3.0.

<i>ITL4</i> = <i>x</i>	Determines the maximum timestep, in transient analysis simulations. IMAX sets the maximum number of iterations, to obtain a convergent solution at a timepoint. If the number of iterations needed is greater than IMAX, then the internal timestep Delta decreases, by a factor equal to the FT transient control option. Star-Hspice uses the new timestep to calculate a new solution. IMAX also works with the IMIN transient control option. ITL4 is the same as IMAX. The default is 8.0.
<i>ITL5</i> = <i>x</i>	Sets the iteration limit, for transient analysis. If a circuit uses more than ITL5 iterations, the program prints all results, up to that point. The default value, 0.0, allows an infinite number of iterations.
<i>RELVAR</i> = <i>x</i>	Used with ABSVAR, and the DVDT timestep option. RELVAR sets the relative voltage change, for LVLTIM = 1 or 3. If the node voltage at the <i>current</i> time point, exceeds the node voltage at the <i>previous</i> time point by RELVAR, then Star-Hspice reduces the timestep, and calculates a new solution at a new time point. The default value is 0.30 (30%).
<i>RMAX</i> = <i>x</i>	Sets the TSTEP multiplier, which controls the maximum value (DELMAX) to use for the Delta of the internal timestep: $\text{DELMAX} = \text{TSTEP} \times \text{RMAX}$ The default value is 5, when dvdt = 4, and lvltim = 1. Otherwise, the default = 2. The maximum value is 1e+9, the minimum value is 1e-9. The recommended maximum value is 1e+5.
<i>RMIN</i> = <i>x</i>	Sets the minimum value of Delta (internal timestep). An internal timestep, that is smaller than RMIN×TSTEP, terminates the transient analysis, with the error message <i>internal timestep too small</i> . Delta decreases by the amount set in the FT option, if the circuit has not converged within IMAX iterations. The default is 1.0e-9.

- SLOPETOL = x** Sets a minimum value, for breakpoint table entries in a piecewise linear (PWL) analysis, performed in Star-Hspice. If the difference in the slopes of two consecutive PWL segments is less than the SLOPETOL value, Star-Hspice ignores the breakpoint, for the point between the segments. The default is 0.5.
- TIMERES = x** Sets a minimum separation between breakpoint values, for the breakpoint table. If two breakpoints are closer together (in time) than the TIMERES value, Star-Hspice enters only one of them in the breakpoint table. The default value is 1 ps.

Algorithm Options

You can use the following algorithm options in Star-Hspice:

- DVTR** Limits the voltage, in transient analysis. The default is 1000.
- IMAX = x,**
ITL4 = x Determines the maximum timestep, in transient analysis simulations. IMAX sets the maximum number of iterations, to obtain a convergent solution at a timepoint. If the number of iterations needed is greater than IMAX, then the internal timestep Delta decreases by a factor equal to the FT transient control option. Star-Hspice uses the new timestep to calculate a new solution. IMAX also works with the IMIN transient control option. ITL4 is the same as IMAX. The default is 8.0.
- IMIN = x,**
ITL3 = x Determines the timestep, in transient analysis simulations. IMIN sets the minimum number of iterations, to obtain convergence. If the number of iterations is less than IMIN, the Delta for the internal timestep doubles. Use this option to decrease simulation times, in circuits where nodes are stable most of the time (such as digital circuits). If the number of iterations is greater than IMIN, the timestep remains the same, unless the iterations exceed the IMAX option (see IMAX). ITL3 is the same as IMIN. The default is 3.0.

- LVLTIM = x** Selects the timestep algorithm, used for transient analysis.
- If **LVLTIM = 1** (the default), Star-Hspice uses the DVDT timestep algorithm.
 - If **LVLTIM = 2**, Star-Hspice uses the timestep algorithm for the local truncation error.
 - If **LVLTIM = 3**, Star-Hspice uses the DVDT timestep algorithm, with timestep reversal.
- To use the GEAR method of numerical integration and linearization, select **LVLTIM = 2**. To use the TRAP linearization algorithm, select **LVLTIM = 1** or **3**. Using **LVLTIM = 1** (the DVDT option) helps avoid *internal timestep too small* non-convergence.
- The local truncation algorithm (**LVLTIM = 2**) offers a higher degree of accuracy, and prevents errors propagating from time point to time point, which can cause an unstable solution.
- MAXORD = x** Sets the maximum order of integration, for the GEAR method in Star-Hspice (see **METHOD**). The *x* value can be either 1 or 2.
- If **MAXORD = 1**, Star-Hspice uses the backward Euler method of integration.
 - **MAXORD = 2**, however, is more stable, accurate, and practical. The default is 2.0.
- METHOD = name** Sets the numerical integration method, for a transient analysis, to either GEAR or TRAP. To use GEAR, set **METHOD = GEAR**, which sets **LVLTIM = 2**.
- To change **LVLTIM** from 2 to 1 or 3, set **LVLTIM = 1** or **3**. This overrides **METHOD = GEAR**, which sets **LVLTIM = 2**.

TRAP (trapezoidal) integration usually reduces program execution time, with more accurate results. However, this method can introduce an apparent oscillation on printed or plotted nodes, which might not result from circuit behavior. To test this, run a transient analysis, using a small timestep. If oscillation disappears, the cause was the trapezoidal method.

The GEAR method acts as a filter, removing oscillations that occur in the trapezoidal method. Highly non-linear circuits, such as operational amplifiers, can require long execution times, when you use the GEAR method. Circuits that do not converge in trapezoidal integration, often converge in GEAR.

The default value is TRAP (trapezoidal).

PURETP

Sets the integration method to use, for the reversal time point. The default value is 0. If you set puretp=1, when Star-Hspice encounters non-convergence, it uses TRAP (instead of B.E) for the reversed time point.

Use this option to help an oscillating circuit to oscillate, if the default simulation process cannot satisfy the result.

Use this option with the method=TRAP statement.

MU = x

Coefficient, for trapezoidal integration. The range for MU is 0.0 to 0.5. The default is 0.5.

TRCON

Controls the automatic convergence (*autoconvergence*) process. If the circuit fails to converge using the trapezoidal (TRAP) numerical integration method (for example, because of trapezoidal oscillation), Star-Hspice uses the GEAR method and LTE timestep algorithm, to run the transient analysis again from time=0. This process is called *autoconvergence*.

Note: Star-Hspice also uses autoconvergence in DC analysis, if the Newton-Raphson (N-R) method fails to converge.

Autoconvergence sets the following options to their default values before the second try:

```
METHOD=GEAR, LVLTIM=2, MBYPASS=1.0,  
BYPASS=0.0, SLOPETOL=0.5  
BYTOL= min{mbypas*vntol and reltol}
```

RMAX=2.0, if it was 5.0 in the first simulation run. Otherwise, RMAX does not change.

- TRCON=1 (the default) enables the autoconvergence process, if the previous simulation run fails.
- To disable autoconvergence, set TRCON=0 or TRCON=-1.

Input and Output Options

You can use the following input and output options in Star-Hspice:

INTERP	Limits output to post-analysis tools, such as Cadence or Zuken, to only the .TRAN timestep intervals. By default, Star-Hspice outputs all convergent iterations. INTERP typically produces a much smaller <i>design.tr#</i> file. Use INTERP = 1 with caution, when the netlist includes .MEASURE statements. To compute measure statements, Star-Hspice uses the postprocessing output. Reducing postprocessing output can cause interpolation errors in measure results. When you run a data-driven transient analysis (.TRAN DATA statement) within optimization routines, Star-Hspice forces INTERP to 1. The results of all measurements are at the time points of the data, in the data-driven sweep. If the measurement needs to use converged internal timesteps (such as AVG or RMS calculations), set INTERP = 1.
ITRPRT	Prints output variables, at their internal timepoint values. This option might generate a long output list.
MEASFAIL	■ Produces 0 into .mt#, .ms# or .ma# and prints <i>failed</i> to the listing file, when measfail=0. ■ Prints <i>failed</i> into the .mt#, .ms#, or .ma# file, and into the listing file, when measfail=1.

Note: The default value is 1.

Use the following syntax:

```
.option measfail=1 | 0
```

MEASSORT

To automatically sort large numbers of .MEASURE statements, use the .OPTION MEASSORT statement.

- .OPTION MEASSORT=0 (the default; does not sort .MEASURE statements).
- .OPTION MEASSORT=1 (internally sorts .MEASURE statements).

Set this option to 1, *only* if you use a large number of .MEASURE statements, where it is important to list similar variables together, to reduce simulation run time. For a small number of .MEASURE statements, turning on internal sorting might slow-down the simulation while sorting, compared to **not** sorting first.

PUTMEAS

Controls the output variables, listed in the .MEASURE statement. The syntax is:

.option putmeas=0 or (1)

The default value is 1.

- 0 Does not save variable values, which are listed in the .MEASURE statement, into the corresponding output file (such as .tr#, .ac# or .sw#). This option decreases the size of the output file.
- 1 Saves variable values, which are listed in the .MEASURE statement, into the corresponding output file (such as .tr#, .ac# or .sw#). This option is similar to the output of Hspice 2000.4.

UNWRAP

Displays phase results from AC analysis, in unwrapped form (with a continuous phase plot). This allows accurate calculation of group delay. Star-Hspice always computes the group delay, based on unwrapped phase results, even if you do not set the UNWRAP option.



Chapter 10

Initializing DC/Operating Point Analysis

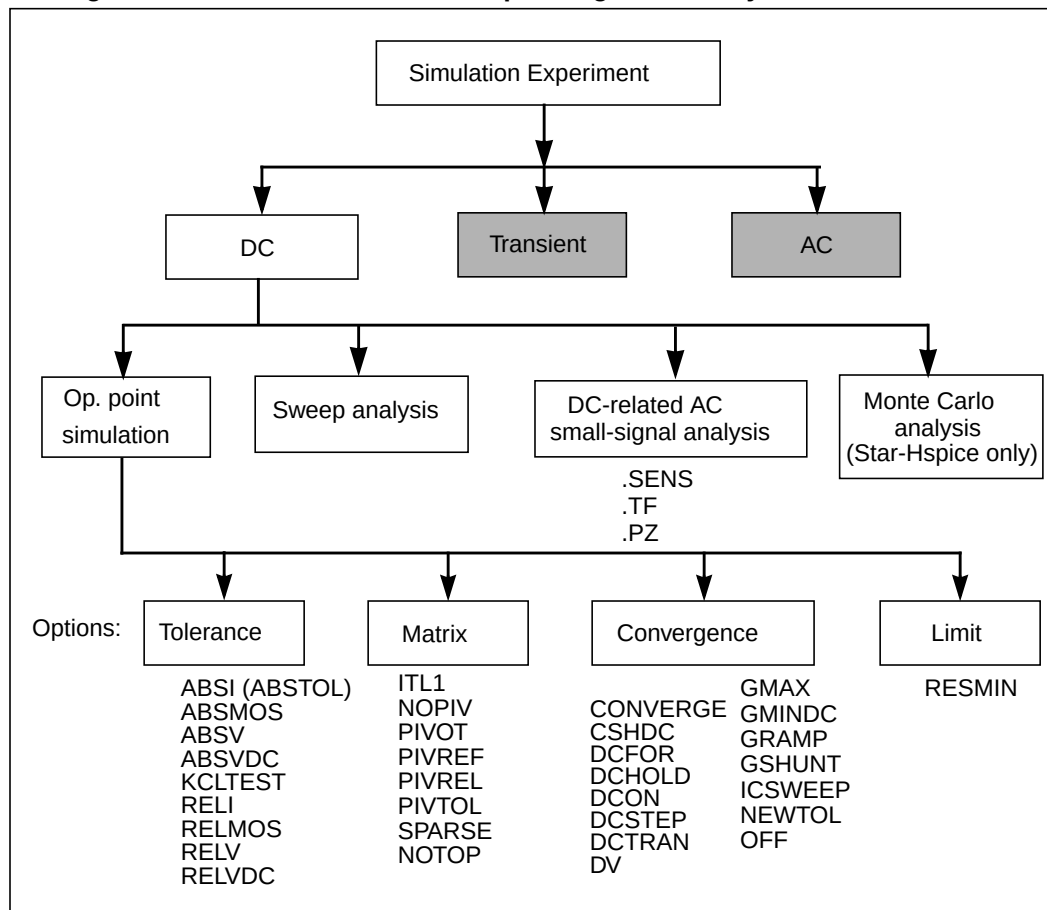
This chapter describes DC initialization and operating point analysis. It explains the following topics:

- [Simulation Flow](#)
- [Initialization and Analysis](#)
- [DC Initialization and Operating Point Statements](#)
- [.DC Statement—DC Sweeps](#)
- [Other DC Analysis Statements](#)
- [DC Initialization Control Options](#)
- [Pole/Zero Analysis Options](#)
- [Accuracy and Convergence](#)
- [Reducing DC Errors](#)
- [Diagnosing Convergence Problems](#)

Simulation Flow

Figure 10-1 shows the simulation flow, for Star-Hspice.

Figure 10-1: DC Initialization and Operating Point Analysis Simulation Flow



Initialization and Analysis

Before it performs .OP, .DC sweep, .AC, or .TRAN analyses, Star-Hspice first sets the DC operating point values, for all nodes and sources. To do this, Star-Hspice does one of the following:

- calculates all values
- applies values specified in .NODESET and .IC statements
- applies values stored in an initial conditions file.

The .OPTION OFF statement, and the OFF and IC = val element parameters, also control initialization.

Initialization is fundamental to simulation. Star-Hspice starts any analysis with known nodal voltages (or initial estimates for unknown voltages), and some branch currents. It then iteratively finds the exact solution. Initial estimates that are close to the exact solution, increase the likelihood of a convergent solution and a lower simulation time.

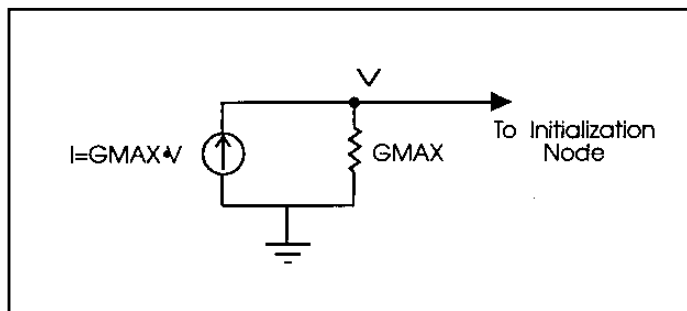
A transient analysis first calculates a DC operating point, using the DC equivalent model of the circuit (unless you specify the UIC parameter in the .TRAN statement). Star-Hspice then uses the resulting DC operating point as an initial estimate, to solve the next timepoint in the transient analysis.

1. If you do not provide an initial guess, or if you provide only partial information, Star-Hspice provides a default estimate, for each node in the circuit.
2. Star-Hspice then uses this estimate to iteratively find the exact solution.
The .NODESET and .IC statements supply an initial guess, for the exact DC solution of nodes within a circuit.
3. To set any circuit node to any value, use the .NODESET statement.
4. Star-Hspice then connects a voltage source equivalent, to each initialized node (a current source, with a GMAX parallel conductance, set with a .OPTION statement).
5. Star-Hspice next calculates a DC operating point, with the .NODESET voltage source equivalent connected.

6. Star-Hspice disconnects the equivalent voltage sources, which you set in the `.NODESET` statement, and recalculates the DC operating point.

This is the DC operating point solution.

Figure 10-2: Equivalent Voltage Source: `.NODESET` and `.IC`



Use the `.IC` statement, to provide both an initial guess and a solution, for selected nodes within the circuit. Nodes that you initialize with the `.IC` statement, become part of the solution of the DC operating point.

You can also use the `OFF` option to initialize active devices. The `OFF` option works with `.IC` and `.NODESET` voltages, as follows:

1. If the netlist includes any `.IC` or `.NODESET` statements, Star-Hspice sets node voltages, according to those statements.
2. If you set the `OFF` option, then Star-Hspice sets values to zero, for the terminal voltages of all active devices (BJTs, diodes, MOSFETs, JFETs, MESFETs) that are not set in `.IC` or `.NODESET` statements, or by sources.
3. If element statements specify any `IC` parameters, Star-Hspice sets those initial conditions.
4. Star-Hspice uses the resulting voltage settings, as the initial guess at the operating point.

Use `OFF` to find an exact solution, during an operating point analysis, in a large circuit. The majority of device terminals are at zero volts, for the operating point solution. To initialize the terminal voltages to zero, for selected active devices, set the `OFF` parameter, in the element statements for those devices.

After Star-Hspice finds a DC operating point, use `.SAVE` to store operating-point node voltages in a `<design>.ic` file. Then use the `.LOAD` statement to restore operating-point values, from the `ic` file for later analyses.

When you set initial conditions for Transient Analysis:

- If you include UIC in a .TRAN statement, Star-Hspice starts a transient analysis, using node voltages specified in an .IC statement.
- Use the .OP statement, to store an estimate of the DC operating point, during a transient analysis.
- An *internal timestep too small* error message indicates that the circuit failed to converge. The cause of the failure can be that Star-Hspice cannot use stated initial conditions to calculate the actual DC operating point.

DC Initialization and Operating Point Statements

.OP Statement — Operating Point

When you include an .OP statement in an input file, Star-Hspice calculates the DC operating point of the circuit. You can also use the .OP statement to produce an operating point, during a transient analysis. You can include only one .OP statement in a simulation.

If an analysis requires calculating an operating point, then you do not need to specify the .OP statement; Star-Hspice calculates an operating point. If you specify a .OP statement, and if you include the UIC keyword in a .TRAN analysis statement, then the simulation omits the time = 0 operating point analysis, and issues a warning in the output listing.

Syntax

`.OP <format> <time> <format> <time>`

format	Any of the following keywords. (Only the first letter is required. Default = ALL.)
	■ ALL Full operating point, including voltage, currents, conductances, and capacitances. This parameter outputs voltage/current for the specified time. ■ BRIEF Produces a one-line summary of each element's voltage, current, and power. Current is stated in milliamperes, and power is in milliwatts. ■ CURRENT Voltage table, with a brief summary of element currents and power.

- **DEBUG**
Usually invoked only if a simulation does not converge. Debug prints back the non-convergent nodes, with the new voltage, old voltage, and the tolerance (degree of non-convergence). It also prints back the non-convergent elements, with their tolerance values.
- **NONE**
Inhibits node and element print-outs, but performs additional analysis that you specify.
- **VOLTAGE**
Voltage table only.

The preceding keywords are mutually- exclusive; use only one at a time.

time

Place this parameter directly after ALL, VOLTAGE, CURRENT, or DEBUG. It specifies the time at which Star-Hspice prints the report.

Example

The following example calculates:

- Operating point voltages and currents, for the DC solution.
- Currents at 10 ns, for the transient analysis.
- Voltages at 17.5 ns, 20 ns and 25 ns, for the transient analysis.

```
.OP .5NS CUR 10NS VOL 17.5NS 20NS 25NS
```

The following example calculates the complete DC operating point solution. The next section shows a printout of the solution.

```
.OP
```

Output

```
***** OPERATING POINT INFORMATION      TNOM = 25.000 TEMP = 25.000
***** OPERATING POINT STATUS IS ALL     SIMULATION TIME IS 0.

NODE      VOLTAGE      NODE      VOLTAGE      NODE      VOLTAGE
+ 0:2  =      0      0:3  = 437.3258M      0:4  = 455.1343M
```

```

+ 0:5 = 478.6763M 0:6 = 496.4858M 0:7 = 537.8452M
+ 0:8 = 555.6659M 0:10 = 5.0000 0:11 = 234.3306M

```

**** VOLTAGE SOURCES

SUBCKT

ELEMENT	0:VNCE	0:VN7	0:VPCE	0:VP7
VOLTS	0	5.00000	0	-5.00000
AMPS	-2.07407U	-405.41294P	2.07407U	405.41294P
POWER	0.	2.02706N	0.	2.02706N

TOTAL VOLTAGE SOURCE POWER DISSIPATION = 4.0541 N WATTS

**** BIPOLAR JUNCTION TRANSISTORS

SUBCKT

ELEMENT	0:QN1	0:QN2	0:QN3	0:QN4
MODEL	0:N1	0:N1	0:N1	0:N1
IB	999.99912N	2.00000U	5.00000U	10.00000U
IC	-987.65345N	-1.97530U	-4.93827U	-9.87654U
VBE	437.32588M	455.13437M	478.67632M	496.48580M
VCE	437.32588M	17.80849M	23.54195M	17.80948M
VBC	437.32588M	455.13437M	478.67632M	496.48580M
VS	0.	0.	0.	0.
POWER	5.39908N	875.09107N	2.27712U	4.78896U
BETAD	-987.65432M	-987.65432M	-987.65432M	-987.65432M
GM	0.	0.	0.	0.
RPI	2.0810E+06	1.0405E+06	416.20796K	208.10396K
RX	250.00000M	250.00000M	250.00000M	250.00000M
RO	2.0810E+06	1.0405E+06	416.20796K	208.10396K
CPI	1.43092N	1.44033N	1.45279N	1.46225N
CMU	954.16927P	960.66843P	969.64689P	977.06866P
CCS	800.00000P	800.00000P	800.00000P	800.00000P
BETAAC	0.	0.	0.	0.
FT	0.	0.	0.	0.

Element Statement IC Parameter

Use the element statement parameter, IC = <val>, to set DC terminal voltages, for selected active devices.

- Star-Hspice uses the value, set in IC = <val>, as the DC operating point value, in the DC solution.

The following example describes an H element dependent-voltage source:

```
HXCC 13 20 VIN1 VIN2 IC = 0.5, 1.3
```

The current, through VIN1, initializes to 0.5 mA. The current, through VIN2, initializes to 1.3 mA.

.IC and .DCVOLT Initial Condition Statements

Use the `.IC` statement, or the `.DCVOLT` statement, to set transient initial conditions in Star-Hspice. How it initializes depends on whether the `.TRAN` analysis statement includes the `UIC` parameter.

If you specify the `UIC` parameter in the `.TRAN` statement, Star-Hspice does not calculate the initial DC operating point, but directly enters transient analysis. Transient analysis uses the `.IC` initialization values as part of the solution, for timepoint zero (calculating the zero timepoint applies a fixed equivalent voltage source). The `.IC` statement is equivalent to specifying the `IC` parameter on each element statement, but is more convenient. You can still specify the `IC` parameter, but it does not have precedence over values set in the `.IC` statement.

If you do *not* specify the `UIC` parameter in the `.TRAN` statement, Star-Hspice computes the DC operating point solution, before the transient analysis. The node voltages that you specify in the `.IC` statement are fixed, to determine the DC operating point. Transient analysis releases the initialized nodes, to calculate the second and later time points.

Syntax

```
.IC V(node1) = val1 V(node2) = val2 ...
```

or

```
.DCVOLT V(node1) = val1 V(node2) = val2 ...
```

or

```
.DCVOLT V node1 val1 <node2 val2 ...>
```

where:

val1 ... Specifies voltages. The significance of these voltages depends on whether you specify the `UIC` parameter in the `.TRAN` statement.

node1 ... Node numbers or names can include full paths, or circuit numbers.

Example

```
.IC V(11) = 5 V(4) = -5 V(2) = 2.2
.DCVOLT 11 5 4 -5 2 2.2
```

.NODESET Statement

.NODESET initializes all specified nodal voltages, for DC operating point analysis. Use the .NODESET statement, to correct convergence problems in DC analysis. If you set the node values in the circuit, close to the actual DC operating point solution, you enhance convergence of the simulation. The Star-Hspice simulator uses the NODESET voltages, *only* in the first iteration.

Syntax

```
.NODESET V(node1) = val1 <V(node2) = val2 ...>
```

or

```
.NODESET node1 val1 <node2 val2>
```

node1 ... Node numbers or names can include full paths or circuit numbers.

val1 Specifies voltages.

Example

```
.NODESET V(5:SETX) = 3.5V V(X1.X2.VINT) = 1V
.NODESET V(12) = 4.5 V(4) = 2.23
.NODESET 12 4.5 4 2.23 1 1
```

.SAVE and .LOAD Statements

Star-Hspice saves the operating point, unless you use the .SAVE LEVEL = NONE statement. Star-Hspice restores the saved operating-point file, only if the input file contains a .LOAD statement.

If any node initialization commands, such as .NODESET and .IC, appear in the netlist *after* the .LOAD command, then they overwrite the .LOAD initialization. If you use this feature to set particular states for multistate circuits (such as flip-flops), you can still use the .SAVE command to speed up the DC convergence.

.SAVE and .LOAD continue to work, even on changed circuit topologies. Adding or deleting nodes results in a new circuit topology. Star-Hspice initializes the new nodes, as if you did not save an operating point. Star-Hspice ignores references to deleted nodes, but initializes coincidental nodes to the values that you saved from the previous run.

When you initialize nodes to voltages, Star-Hspice inserts Norton-equivalent circuits at each initialized node. The conductance value of a Norton-equivalent circuit is $G_{MAX} = 100$, which might be too large for some circuits.

If using .SAVE and .LOAD does not speed up the simulation, or causes simulation problems, you can use `.OPTION GMAX = 1e-12`, to minimize the effect of the Norton-equivalent circuits on matrix conductances. Star-Hspice still uses the initialized node voltages to initialize devices.

.SAVE Statement

The .SAVE statement in Star-Hspice stores the operating point of a circuit, in a file that you specify. For quick DC convergence in subsequent simulations, use the .LOAD statement to input the contents of this file. Star-Hspice saves the operating point by default, even if the Star-Hspice input file does not contain a .SAVE statement. To not save the operating point, specify `.SAVE LEVEL = NONE`.

You can save the operating point data as either an .IC or a .NODESET statement.

The syntax is:

```
.SAVE <TYPE = type_keyword> <FILE = save_file>  
+ <LEVEL = level_keyword> <TIME = save_time>
```

where:

type_keyword	Storage method, for saving the operating point. The type can be one of the following. The default is NODESET. ■ .NODESET Stores the operating point as a .NODESET statement. Later simulations initialize all node voltages to these values, if you use the .LOAD statement. If circuit conditions change incrementally, DC converges within a few iterations. ■ .IC Stores the operating point as a .IC statement. Subsequent simulations initialize node voltages to these values. if the netlist file includes the .LOAD statements.
save_file	Name of the file that stores DC operating point data. The file name format is <design>.ic#. The default is <design>.ic0.
level_keyword	Circuit level, at which you save the operating point. The level can be one of the following. ■ ALL (default) Saves all nodes, from the top to the lowest circuit level. This option offers the greatest improvement in simulation time. ■ TOP Saves only nodes in the top-level design. Does not save any subcircuit nodes. ■ NONE Does not save the operating point.
save_time	Time during transient analysis, when Star-Hspice saves the operating point. Star-Hspice requires a valid transient analysis statement, to save a DC operating point. Default = 0.

A parameter or temperature sweep saves only the first operating point. For example, if the input netlist file contains the statement:

```
.TEMP -25 0 25
```

then Star-Hspice saves the operating point that corresponds to .TEMP -25.

.LOAD Statement

Use the .LOAD statement to input the contents of a file, that you stored using the .SAVE statement in Star-Hspice.

Files stored with the .SAVE statement contain operating point information, for the point in the analysis at which you executed .SAVE.

Do not use the .LOAD command for concatenated netlist files.

The syntax is:

```
.LOAD <FILE = load_file>
```

<i>load_file</i>	Name of the file, in which .SAVE saved an operating point, for the circuit under simulation. The format of the file name is <design>.ic#. The default is <design>.ic0, where <i>design</i> is the root name of the design.
------------------	--

.DC Statement—DC Sweeps

You can use the `.DC` statement in DC analysis, to:

- Sweep any parameter value.
- Sweep any source value.
- Sweep temperature range.
- Perform a DC Monte Carlo (random sweep) analysis.
- Perform a data-driven sweep.
- Perform a DC circuit optimization, for a data-driven sweep.
- Perform a DC circuit optimization, using start and stop.
- Perform a DC model characterization.

The format for the `.DC` statement depends on the application that uses it, as shown in the following examples.

Syntax

Sweep or Parameterized Sweep:

```
.DC var1 START = start1 STOP = stop1 STEP = incr1
```

or

```
.DC var1 START = <param_expr1> STOP = <param_expr2>  
+ STEP = <param_expr3>
```

or

```
.DC var1 start1 stop1 incr1 <SWEEP var2 type np start2 stop2>
```

or

```
.DC var1 start1 stop1 incr1 <var2 start2 stop2 incr2>
```

Data-Driven Sweep:

```
.DC var1 type np start1 stop1 <SWEEP DATA = datanm>
```

or

```
.DC DATA = datanm<SWEEP var2 start2 stop2 incr2>
```

or

```
.DC DATA = datanm
```

Monte Carlo:

```
.DC var1 type np start1 stop1 <SWEEP MONTE = val>
```

or

```
.DC MONTE = val
```

Optimization:

```
.DC DATA = datanm OPTIMIZE = opt_par_fun  
+ RESULTS = measnames MODEL = optmod
```

or

```
.DC var1 start1 stop1 SWEEP OPTIMIZE = OPTxxx  
+ RESULTS = measname MODEL = optmod
```

Keywords and Parameters

The .DC statement keywords and parameters are:

DATA = <i>datanm</i>	<i>Datanm</i> is the reference name of a .DATA statement.
incr1 ...	Voltage, current, element, or model parameters; or temperature increment values.
MODEL	Specifies the optimization reference name. The .MODEL OPT statement uses this name in an optimization analysis
MONTE = <i>val</i>	<i>val</i> is the number of randomly-generated values, which you can use to select parameters from a distribution. The distribution can be <i>Gaussian</i> , <i>Uniform</i> , or <i>Random Limit</i> .

np	Number of points per decade or per octave, or just number of points, based on which keyword precedes it.
OPTIMIZE	Specifies the parameter reference name, used for optimization in the .PARAM statement
RESULTS	Specifies the measure name, used for optimization in the .MEASURE statement
<i>start1 ...</i>	Starting voltage, current, element, or model parameters; or temperature values. If you use the POI (list of points) variation type, specify a list of parameter values, instead of <i>start stop</i> .
<i>stop1 ...</i>	Final voltage, current, any element, model parameter, or temperature values.
SWEEP	Keyword, to indicate that a second sweep has a different type of variation (DEC, OCT, LIN, POI, or DATA statement; or MONTE = <i>val</i>)
TEMP	Keyword, to indicate a temperature sweep.
type	Can be any of the following keywords: DEC — decade variation OCT — octave variation LIN — linear variation POI — list of points
<i>var1 ...</i>	■ Name of an independent voltage or current source, or ■ Name of any element or model parameter, or ■ TEMP keyword (indicating a temperature sweep). Star-Hspice supports a source value sweep, which refers to the source name (SPICE style). However, if you select a parameter sweep, a .DATA statement, and a temperature sweep, then you must select a parameter name for the source value. A later .DC statement must refer to this name. The parameter name must not start with V, I, or TEMP.

Examples

The following example sweeps the value of the VIN voltage source, from 0.25 volts to 5.0 volts, in increments of 0.25 volts.

```
.DC VIN 0.25 5.0 0.25
```

The following example sweeps the drain-to-source voltage, from 0 to 10 V, in 0.5 V increments, at VGS values of 0, 1, 2, 3, 4, and 5 V.

```
.DC VDS 0 10 0.5 VGS 0 5 1
```

The following example starts a DC analysis of the circuit, from -55°C to 125°C, in 10°C increments.

```
.DC TEMP -55 125 10
```

The following script runs a DC analysis, at five temperatures: 0, 30, 50, 100, and 125°C.

```
.DC TEMP POI 5 0 30 50 100 125
```

The following example runs a DC analysis on the circuit, at each temperature value. The temperatures result from a linear temperature sweep, from 25°C to 125°C (five points), which sweeps a resistor value named *xval*, from 1 k to 10 k, in 0.5 k increments.

```
.DC xval 1k 10k .5k SWEEP TEMP LIN 5 25 125
```

The example below specifies a sweep of the *par1* value, from 1 k to 100 k, in increments of 10 points per decade.

```
.DC DATA = datanm SWEEP par1 DEC 10 1k 100k
```

The next example also requests a DC analysis, at specified parameters in the `.DATA datanm` statement. It also sweeps the *par1* parameter, from 1k to 100k, in increments of 10 points per decade.

```
.DC par1 DEC 10 1k 100k SWEEP DATA = datanm
```

The final example invokes a DC sweep of the parameter *par1* from 1k to 100k by 10 points per decade, using 30 randomly generated (Monte Carlo) values.

```
.DC par1 DEC 10 1k 100k SWEEP MONTE = 30
```

Schmitt Trigger Example

```
*file: bjtschmt.sp          bipolar schmitt trigger
.OPTION post = 2
vcc 6 0 dc 12
vin 1 0 dc 0 pwl(0,0 2.5u,12 5u,0)
cb1 2 4 .1pf
rc1 6 2 1k
rc2 6 5 1k
rb1 2 4 5.6k
rb2 4 0 4.7k
re 3 0 .47k
*
diode 0 1 dmod
q1 2 1 3 bmod 1 ic = 0,8
q2 5 4 3 bmod 1 ic = .5,0.2
*
.dc vin 0,12,.1
*
.model dmod d is = 1e-15 rs = 10
.model bmod npn is = 1e-15 bf = 80 tf = 1n
+ cjc = 2pf cje = 1pf rc = 50 rb = 100 vaf = 200
.plot v(1) v(5)
.graph dc model = schmittplot input = v(1)
+ output = v(5) 4.0 5.0
.model schmittplot plot xscal = 1 yscal = 1 xmin = .5u
+ xmax = 1.2u
.end
```

Other DC Analysis Statements

Star-Hspice also provides the following DC analysis statements. Each statement uses the DC-equivalent model of the circuit, in its analysis. For `.PZ`, the equivalent circuit includes capacitors and inductors.

- | | |
|--------------------|---|
| <code>.PZ</code> | Performs pole/zero analysis (you do not need to specify <code>.OP</code>) |
| <code>.SENS</code> | Obtains DC small-signal sensitivities of output variables, for circuit parameters (you do not need to specify <code>.OP</code>) |
| <code>.TF</code> | Calculates DC small-signal values for transfer functions (ratio of output variable, to input source). You do not need to specify <code>.OP</code> . |

Star-Hspice provides DC control options, and DC initialization statements, which model resistive parasitics and initialize nodes. These statements enhance convergence properties, and accuracy, of simulation. This section describes how to perform DC-related, small-signal analysis.

.SENS Statement — DC Sensitivity Analysis

If the input file includes a `.SENS` statement, Star-Hspice determines DC small-signal sensitivities for each specified output variable, relative to every circuit parameter. The sensitivity measurement is the partial derivative of each output variable, for a specified circuit element, measured at the operating point, and normalized to the total change in output magnitude. Therefore, the sum of the sensitivities of all elements is 100%. Star-Hspice calculates sensitivities for

- resistors
- voltage sources
- current sources
- diodes
- BJTs (including Level 4, the VBIC95 model)
- MOSFETs (Level49 and Level53, Version=3.22).

You can perform only one `.SENS` analysis per simulation. If you specify more than one `.SENS` statement, Star-Hspice runs only the *last* `.SENS` statement.

Syntax

```
.SENS ov1 <ov2 ...>
```

ov1 ov2 ... Branch currents, or nodal voltage, for DC component-sensitivity analysis.

Example

```
.SENS V(9) V(4,3) V(17) I(VCC)
```

Note: The .SENS statement can generate very large amounts of output for large circuits.

.TF Statement — DC Small-Signal Transfer Function Analysis

The transfer function statement (.TF) defines small-signal output and input, for DC small-signal analysis. When you use the .TF statement, Star-Hspice computes:

- DC small-signal value of the transfer function (output/input),.
- Input resistance.
- Output resistance.

Syntax

```
.TF ov srcnam
```

where:

<i>ov</i>	Small-signal output variable.
<i>srcnam</i>	Small-signal input source.

Example

```
.TF V(5,3) VIN
.TF I(VLOAD) VIN
```

For the first example, Star-Hspice computes the ratio of $V(5,3)$ to VIN . This is the ratio of small-signal input resistance at VIN , to the small-signal output resistance (measured across nodes 5 and 3). If you specify more than one `.TF` statement in a single simulation, Star-Hspice runs only the *last* `.TF` statement.

.PZ Statement— Pole/Zero Analysis

Syntax

```
.PZ ov srcnam
```

where:

ov Output variable: a node voltage $V(n)$ or branch current $I(element)$.

srcnam Input source: the name of an independent voltage or current source.

Example

```
.PZ V(10) VIN
.PZ I(RL) ISORC
```

See [Pole/Zero Analysis on page 18-1](#), for complete information about pole/zero analysis.

DC Initialization Control Options

Use control options in a DC operating-point analysis, to control DC convergence properties and simulation algorithms. Many of these options also affect transient analysis, because DC convergence is an integral part of transient convergence. Include the following options for *both* DC and transient convergence:

- Absolute and relative voltages.
- Current tolerances.
- Matrix options.

Use .OPTION statements to specify the following options, which control DC analysis (see [Simulation Options on page 9-1](#)):

ABSTOL	GSHUNT	
CAPTAB	ICSWEEP	OFF
CSHDC	ITLTRAN	PIVOT
DCCAP	ITL1	PIVREF
DCFOR	ITL2	PIVREL
DCHOLD	KCLTEST	PIVTOL
DCSTEP	MAXAMP	RESMIN
DV	NEWTOL	SPARSE
GRAMP	NOPIV	

Some of these options also are used in DC and AC analysis. Many of these options also affect the transient analysis, because DC convergence is an integral part of transient convergence. For a description of transient analysis, see [Transient Analysis on page 11-1](#).

Table 10-1: DC Initialization Control Options (*Sheet 1 of 8*)

ABSTOL = x	Sets the absolute node voltage error tolerance, for DC and transient analysis. Decrease ABSTOL, if accuracy is more important than convergence time. ABSTOL is the same as ABSI.
CAPTAB	Prints single-plate node capacitances, for diodes, BJTs, JFETs, MOSFETs, and passive capacitors, at each operating point.
CSHDC	Same option as CSHUNT, but used <i>only</i> with the CONVERGE option.

Table 10-1: DC Initialization Control Options (Sheet 2 of 8)

DCCAP	Generates C-V plots. Prints capacitance values of a circuit (both model and element) during a DC analysis. C-V plots are often generated during a DC sweep of the capacitor. Default = 0 (off).
DCFOR = x	Use with DCHOLD, and the .NODESET statement, to enhance DC convergence. DCFOR sets how many iterations to calculate, after a circuit converges in a steady state. The number of iterations after convergence is usually zero. DCFOR adds iterations (and computing time) when calculating a DC circuit solution, to ensure that a circuit did not falsely converge. Default = 0.
DCHOLD = x	Use DCFOR and DCHOLD together, to initialize DC analysis. These statements enhance convergence properties in DC simulation. DCFOR and DCHOLD work with .NODESET. DCHOLD specifies how many iterations to hold a node, at the .NODESET voltage values. The effects of DCHOLD on convergence differ, according to the DCHOLD value, and the number of iterations before DC converges. ■ If a circuit converges in a steady state, in fewer than DCHOLD iterations, the DC solution includes the values set in .NODESET. ■ If the circuit requires more than DCHOLD iterations to converge, Star-Hspice ignores the values in the .NODESET statement, and calculates the DC solution, using the .NODESET fixed-source voltages, open-circuited. Default = 1.
DCSTEP = x	Converts DC model and element capacitors, to a conductance, enhancing DC convergence. Divides element capacitor values by DCSTEP, to model DC conductance. Default = 0 (seconds).
DV = x	Maximum iteration-to-iteration voltage change, for all circuit nodes, in both DC and transient analysis. High-gain bipolar amplifiers can require values of 0.5 to 5.0, to achieve a stable DC operating point. Large CMOS digital circuits frequently require about 1 volt. Default = 1000 (or 1e6 if DCON = 2).

Table 10-1: DC Initialization Control Options (Sheet 3 of 8)

GRAMP = x	Star-Hspice sets the value during auto-convergence (default=0). Use GRAMP, with the GMINDC convergence-control option, to find the smallest GMINDC value that converges. For a description of GMINDC, see Convergence Control Options on page 10-34. GRAMP specifies the conductance range, over which DC operating point analysis sweeps GMINDC. Star-Hspice replaces GMINDC values over this range, simulates each value, and uses the lowest GMINDC value where the circuit converged. If you sweep GMINDC between 1e-12 mhos (default) and 1e-6 mhos, GRAMP is 6 (value of the exponent difference, between the default and the maximum conductance limit). In this example: 1. Star-Hspice first sets GMINDC to 1e-6 mhos, and simulates the circuit. 2. If circuit simulation converges, Star-Hspice sets GMINDC to 1e-7 mhos, and simulates the circuit. 3. The sweep continues, until Star-Hspice has simulated all values on the GRAMP ramp. If the combined GMINDC and GRAMP conductance is greater than 1e-3 mho, false convergence can occur.
GSHUNT	Conductance added from each node, to ground. The default is zero. Add a small GSHUNT value to each node, to help solve <i>Timestep too small</i> problems, caused by high-frequency oscillations or numerical noise.
ICSWEEP	For a parameter or temperature sweep, saves the results of the current analysis. You can use this result as the starting point in the next analysis in the sweep. Default = 1. ■ If ICSWEEP = 1, the next analysis uses the current results. ■ If ICSWEEP = 0, the next analysis does not use the results of the current analysis.

Table 10-1: DC Initialization Control Options (Sheet 4 of 8)

ITLPTRAN	Controls the iteration limit used in the final try of the pseudo-transient method, in OP or DC analysis. If your simulation fails in the final try of the pseudo-transient method, you can enlarge this option. The default value is 30.
ITL1 = x	Sets the maximum DC iterations (default=200). Increasing this value rarely improves convergence in <i>small</i> circuits. Values up to 400 can converge <i>large</i> circuits, with feedback (such as operational or sense amplifiers). Most models do not require more than 100 iterations to converge. Set .OPTION ACCT to list the number of iterations for an operating point.
ITL2 = val	Sets the DC transfer curve iteration limit. Increasing this limit improves convergence, <i>only</i> on very large circuits. Default = 50.
KCLTEST	Activates a KCL (Kirchhoff's Current Law) test. This test adds to simulation time for large circuits, but it accurately checks the solution. Default = 0. If set to 1, Star-Hspice sets the following options: ■ RELMOS and ABSMOS options, to 0 (off). ■ ABSI, to 1e-6 A. ■ RELI, to 1e-6. To satisfy the KCL test, each node must satisfy the following condition, where i_bs are the node currents: $ \Sigma i_b < RELI \cdot \Sigma i_b + ABSI$
MAXAMP = x	Sets the maximum current, through voltage-defined branches (voltage sources and inductors). If the current exceeds the MAXAMP value, Star-Hspice reports an error. Default = 0.0.
NEWTOL	Calculates one iteration past convergence, for each DC solution and timepoint circuit solution (default is 0). If you do not set NEWTOL after converging, then the routine ends, and the next program step begins.

Table 10-1: DC Initialization Control Options (Sheet 5 of 8)

NOPIV	Prevents (inhibits) Star-Hspice from automatically switching to pivoting-matrix factoring, if a nodal conductance is less than PIVTOL. See also PIVOT.
OFF	Initializes terminal voltages of all active devices to zero, if they are not initialized to other values. For example, if the drain and source nodes of a transistor are not initialized (using .NODESET or .IC statements, or by connecting them to sources), then the OFF option initializes all nodes of the transistor to zero. Star-Hspice checks the OFF option before element IC parameters. If a node includes an element IC parameter assignment, simulation initializes the node to the element IC parameter value, even if the OFF option previously set it to zero. (You can use the OFF element parameter to initialize the terminal voltages to zero, for specific active devices). Use the OFF option to find exact DC operating point solutions, for large circuits.
PIVOT = x (same as SPARSE = x)	Selects different pivoting algorithms. Use these to reduce simulation time, and to achieve convergence in circuits that produce hard-to-solve matrix equations. To select the pivot algorithm, set PIVOT to one of these values: ■ 0 Original non-pivoting algorithm. ■ 1 Original pivoting algorithm. ■ 2 Algorithm to pick the largest pivot in a row. ■ 3 Algorithm to pick the best pivot in a row. ■ 10 Fast nonpivoting algorithm; requires more memory. ■ 11 Fast pivoting algorithm; requires more memory than for PIVOT values less than 11. ■ 12 Algorithm to pick the largest pivot in a row; requires more memory than for PIVOT values less than 12. ■ 13 Fast best pivot: faster; requires more memory than for PIVOT values less than 13. Default = 10.

Table 10-1: DC Initialization Control Options (Sheet 6 of 8)

The fastest algorithm is $PIVOT = 13$, which can improve simulation time up to ten times, on very large circuits. However, $PIVOT = 13$ requires substantially more memory for simulation.

Some circuits with large conductance ratios, such as switching regulator circuits, might require pivoting.

If $PIVOT = 0$, Star-Hspice automatically changes from non-pivoting, to a row-pivot strategy, when it detects any diagonal-matrix entry less than $PIVTOL$. This strategy provides the time and memory advantages of non-pivoting inversion, and avoids unstable simulations and incorrect results. Use `.OPTION NOPIV`, to prevent Star-Hspice from pivoting. For very large circuits, $PIVOT = 10, 11, 12$, or 13 , can require excessive memory.

If Star-Hspice switches to pivoting during a simulation, it prints the message *pivot change on the fly*, followed by the node numbers that caused the problem. Use `.OPTION NODE` to cross-reference a node to an element.

PIVREF	Pivot reference. Used in $PIVOT = 11, 12$, or 13 , to limit the size of the matrix. Default = $1e+8$.
PIVREL = x	Sets the maximum/minimum ratio of a row or matrix. Use only if $PIVOT = 1$. Large values for PIVREL can result in very long matrix-pivot times. If the value is too small, however, pivoting does not occur. Start with small values of PIVREL, using an adequate (but not excessive) value, for convergence and accuracy. Default = $1E-20$ (max = $1e-20$, min = 1).
PIVTOL = x	The minimum value for which Star-Hspice accepts a matrix entry as a pivot. If $PIVOT=0$, PIVTOL is the minimum conductance in the matrix. Default = $1.0e-15$.

Note: PIVTOL must be less than GMIN or GMINDC. Values approaching 1 increase the pivot.

Table 10-1: DC Initialization Control Options (Sheet 7 of 8)

RESMIN = x	Specifies the minimum resistance for all resistors, including parasitic and inductive resistances. Default = 1e-5 (ohm). Range: 1e-15 to 10 ohm.
SPARSE = x (same as PIVOT = x)	Selects different pivoting algorithms. Use these to reduce simulation time, and to achieve convergence in circuits that produce hard-to-solve matrix equations. To select the pivot algorithm, set SPARSE to one of these values: ■ 0 Original non-pivoting algorithm. ■ 1 Original pivoting algorithm. ■ 2 Algorithm to pick the largest pivot in a row. ■ 3 Algorithm to pick the best pivot in a row. ■ 10 Fast nonpivoting algorithm; requires more memory. ■ 11 Fast pivoting algorithm; requires more memory than for PIVOT values less than 11. ■ 12 Algorithm to pick the largest pivot in a row; requires more memory than for PIVOT values less than 12. ■ 13 Fast best pivot: faster; requires more memory than for PIVOT values less than 13. Default = 10. The fastest algorithm is PIVOT = 13, which can improve simulation time by up to ten times, on very large circuits. However, the PIVOT = 13 option requires substantially more memory for the simulation. Some circuits with large conductance ratios, such as switching regulator circuits, might require pivoting.

Table 10-1: DC Initialization Control Options (Sheet 8 of 8)

SPARSE = 0 automatically changes from a non-pivoting strategy, to a row-pivot strategy, when it detects any diagonal-matrix entry less than PIVTOL. This strategy provides the time and memory advantages of non-pivoting inversion, and avoids unstable simulations and incorrect results. Use .OPTION NOPIV, to prevent Star-Hspice from pivoting. For very large circuits, PIVOT = 10, 11, 12, or 13, can require excessive memory.

If Star-Hspice switches to pivoting during a simulation, it prints the message *pivot change on the fly*, followed by the node numbers that caused the problem. Use .OPTION NODE to cross-reference a node to an element.

Pole/Zero Analysis Options

To set control options, use the `.OPTION` statement. Pole/zero analysis uses the following control options.

<i>CSCAL</i>	Sets the capacitance scale. Star-Hspice multiplies capacitances by <i>CSCAL</i> . Default = $1\text{e}+12$.
<i>FMAX</i>	Sets the limit for the maximum pole and zero frequency value. Default = $1.0\text{e}+12 \cdot \text{FSCAL}$.
<i>FSCAL</i>	Sets the frequency scale. Star-Hspice multiplies the frequency by <i>FSCAL</i> . Default = $1\text{e}-9$.
<i>GSCAL</i>	Sets the conductance scale. Star-Hspice multiplies conductances by, and divides resistances by <i>GSCAL</i> . Default = $1\text{e}+3$.
<i>ITLPZ</i>	Sets maximum iterations for pole/zero analysis. Default = 100.
<i>LSCAL</i>	Sets the inductance scale. Star-Hspice multiplies inductances by <i>LSCAL</i> . Default = $1\text{e}+6$. Scale factors must satisfy the following:

$$GSCA = CSCAL \cdot FSCAL$$

$$GSCAL = \frac{1}{LSCAL} \cdot FSCAL$$

If you change scale factors, you might need to modify the initial Muller points (*X0R*, *X0I*), (*X1R*, *X1I*) and (*X2R*, *X2I*), even though Star-Hspice multiplies initial values by $(1\text{e}-9/\text{GSCAL})$.

<i>PZABS</i>	Sets absolute tolerances, for poles and zeros. Affects only low-frequency poles or zeros. Use it as follows:
--------------	--

$$\text{If } (|X_{real}| + |X_{imag}| < PZABS),$$

$$\text{then } X_{real} = 0 \text{ and } X_{imag} = 0 .$$

You can also use it in convergence tests. Default = $1\text{e}-2$.

PZTOL Sets a relative error tolerance for poles or zeros. Default = 1.0e-6.

RITOL Sets the minimum ratio value, for the (real/imaginary) or (imaginary/real) parts of the poles or zeros. Use *RITOL* as follows:

$$\text{If } |X_{imag}| \leq RITOL \cdot |X_{real}|, \text{ then } X_{imag} = 0$$

$$\text{If } |X_{real}| \leq RITOL \cdot |X_{imag}|, \text{ then } X_{real} = 0$$

Default = 1.0e-6.

(*X0R,X0I*), The three complex starting points, in the Muller pole/zero analysis algorithm, are:

(*X1R,X1I*), *X0R* = -1.23456e6 *X0I* = 0.0
 X1R = -1.23456e5 *X1I* = 0.0
 (*X2R,X2I*) *X2R* = +.23456e6 *X2I* = 0.0

Star-Hspice multiplies these initial points by *FSCAL*.

Accuracy and Convergence

Convergence is the ability to solve a set of circuit equations, within specified tolerances, and within a specified number of iterations. In numerical circuit simulation, a designer specifies a relative and absolute accuracy for the circuit solution. The simulator iteration algorithm then attempts to converge to a solution that is within these set tolerances. That is, if consecutive simulations achieve results within the specified accuracy tolerances, circuit simulation has converged. How quickly the simulator converges, is often a primary concern to a designer—especially for preliminary design trials. So designers willingly sacrifice some accuracy, for simulations that converge quickly.

Accuracy Tolerances

Star-Hspice uses accuracy tolerances that you specify, to help assure convergence. These tolerances determine when, and whether, to exit the convergence loop. For each iteration of the convergence loop, Star-Hspice subtracts the value of the previously-calculated solution, from the present solution, then compares this result with the accuracy tolerances. If the difference between solutions for two consecutive iterations, is within the specified accuracy tolerances, then the circuit simulation has converged.

$$| V_n^k - V_n^{k-1} | < = \text{accuracy tolerance}$$

where:

- V_n^k is the solution at the n timepoint, for iteration k .
- V_n^{k-1} is the solution at the n timepoint, for iteration $k - 1$.

As shown in [Table 10-2 on page 10-33](#), Star-Hspice defaults to specific absolute and relative values. You can change these tolerance levels, so that simulation time is not excessive, but accuracy is not compromised. [Accuracy Control Options on page 10-34](#) describes the options in [Table 10-2](#).

Table 10-2: Absolute and Relative Accuracy Tolerances

Type	Option	Default
Nodal Voltage Tolerances	ABSVDC	50 μ v
	RELVDC	.001
Current Element Tolerances	ABSI	1 nA
	RELI	.01
	ABSMOS	1 μ A
	RELMOS	.05

Star-Hspice compares nodal voltages and element currents, to the values from the previous iteration.

- If the absolute value of the difference is less than ABSVDC or ABSI, then the node or element has converged.

ABSV and ABSI set the floor value, below which Star-Hspice ignores values. Values above the floor use RELVDC and RELI as relative tolerances. If the iteration-to-iteration absolute difference is less than these tolerances, then it is convergent.

Note: ABSMOS and RELMOS are the tolerances for MOSFET drain currents.

Accuracy settings directly affect the number of iterations before convergence.

- If accuracy tolerances are tight, the circuit requires more time to converge.
- If the accuracy setting is too loose, the resulting solution can be inaccurate and unstable.

[Table 10-3](#) shows an example of the relationship between the RELVDC value, and the number of iterations.

Table 10-3: RELV vs. Accuracy and Simulation Time for 2 Bit Adder

RELVDC	Iteration	Delay (ns)	Period (ns)	Fall time (ns)
.001	540	31.746	14.336	1.2797
.005	434	31.202	14.366	1.2743
.01	426	31.202	14.366	1.2724
.02	413	31.202	14.365	1.3433
.05	386	31.203	14.365	1.3315
.1	365	31.203	14.363	1.3805
.2	354	31.203	14.363	1.3908
.3	354	31.203	14.363	1.3909
.4	341	31.202	14.363	1.3916
.4	344	31.202	14.362	1.3904

Accuracy Control Options

The default control option settings are designed to maximize accuracy, without significantly degrading performance. For a description of these options and their settings, see [Controlling Simulation Speed and Accuracy on page 11-31](#).

Convergence Control Options

This section describes the following options:

ABSH	DCON	RELH
ABSI	DCTRAN	RELI
ABSMOS	DI	RELMOS
ABSVDC	GMAX	RELV
CONVERGE	GMINDC	RELVDC

- $ABSH = x$ Sets the absolute current change, through voltage-defined branches (voltage sources and inductors). Use ABSH with DI and RELH, to check for current convergence. Default = 0.0.
- $ABSI = x$ Error tolerance for branch currents, in diodes, BJTs, and JFETs, during DC and transient analysis. Decrease ABSI, if accuracy is more important than convergence time.
- To analyze currents less than 1 nanoamp, change ABSI to a value, at least two orders of magnitude smaller than the minimum expected current.
- Default: $1e-9$ for KCLTEST = 0, $1e-16$ for KCLTEST = 1.
- $ABSMOS = x$ Current error tolerance for MOSFET devices, in DC and transient analysis. This value determines whether the drain-to-source current solution has converged. If the difference in drain-to-source current, between the last and the present iteration, is less than ABSMOS (or if it is greater than ABSMOS, but the percent change is less than RELMOS), drain-to-source current converges.
- Star-Hspice then checks the other accuracy tolerances. If all indicate convergence, the circuit solution at that timepoint is solved, and Star-Hspice calculates the next timepoint solution. For low-power circuits, optimization, and single transistor simulations, set $ABSMOS = 1e-12$. Default = $1e-6$ (amperes).
- $ABSVDC = x$ Sets the minimum voltage for DC analysis. If accuracy is more critical than convergence, decrease ABSVDC. If you need voltages less than 50 micro-volts, reduce ABSVDC, to two orders of magnitude less than the smallest voltage. This ensures at least two digits of significance. Typically, you do not need to change ABSVDC, unless you are simulating a high-voltage circuit. For 1000-volt circuits, a reasonable value can be 5 to 50 millivolts. Default = VNTOL (VNTOL default = 50 μ V).

CONVERGE Invokes different methods to solve non-convergence problems

CONVERGE = -1

Use with DCON = -1, to disable auto-convergence.

CONVERGE = 0

Autoconvergence (default).

CONVERGE = 1

Uses the Damped Pseudo Transient algorithm. If simulation does not converge within the specified CPTIME, then simulation halts.

CONVERGE = 2

Uses a combination of DCSTEP and GMINDC ramping. Not used in the autoconvergence flow.

CONVERGE = 3

Invokes the source-stepping method. Not used in the autoconvergence flow.

CONVERGE = 4

Uses the *gmath* ramping method.

Even you did not set it in an .OPTION statement, the CONVERGE option activates if a matrix floating-point overflows, or if Star-Hspice reports a *timestep too small* error. Default = 0.

If a matrix floating-point overflows, then CONVERGE = 1.

DCON = *x* If a circuit cannot converge, Star-Hspice automatically sets *DCON* = 1, and performs these calculations:

$$DV = \max\left(0.1, \frac{V_{max}}{50}\right), \text{ if } DV = 1000$$

$$GRAMP = \max\left(6, \log_{10}\left(\frac{I_{max}}{GMINDC}\right)\right)$$

$$ITL1 = ITL1 + 20 \cdot GRAMP$$

where V_{max} is maximum voltage, and I_{max} is maximum current.

If the circuit still cannot converge, Star-Hspice sets *DCON* = 2, which sets *DV* = 1e6.

If a circuit contains discontinuous models or uninitialized flip-flops, simulation might not converge. Setting *DCON* = -1 and *CONVERGE* = -1 disables auto-convergence, and lists non-convergent nodes and devices.

DCTRAN *DCTRAN* is an alias for *CONVERGE*. See *CONVERGE*.

DI = *x* Sets the maximum iteration-to-iteration change in current, through voltage-defined branches (voltage sources and inductors). Use this option *only* if the value of the *ABSH* control option is greater than 0. Default = 0.0.

GMAX = *x* The conductance, in parallel with the current source, used for circuitry that initializes the *.IC* and *.NODESET* conditions. Some large bipolar circuits can require *GMAX* set to 1, for convergence. Default = 100 (mho).

GMINDC = x Place this conductance in parallel with all pn junctions, and all MOSFET nodes except *gate* (see [Figure 6-1 on page 6-3](#)), for DC analysis. GMINDC helps overcome DC convergence problems, caused by low values of off-conductance, for pn junctions and MOSFET devices. You can use GRAMP to reduce GMINDC by one order of magnitude, for each step. You can set GMINDC between $1\text{e-}4$ and the PIVTOL value. Default = $1\text{e-}12$.

Large values of GMINDC can cause unreasonable circuit response. If your circuit requires large values to converge, suspect a bad model or circuit. If a matrix floating-point overflows, and if GMINDC is $1.0\text{e-}12$ or less, Star-Hspice sets it to $1.0\text{e-}11$.

Star-Hspice manipulates GMINDC in auto-converge mode, as described in [Autoconverge Process on page 10-39](#).

RELH = x Sets relative tolerance for currents, through voltage-defined branches (voltage sources and inductors). Use it to check current convergence, but *only* if the value of the ABSH control option is greater than zero. Default = 0.05 .

RELI = x Relative error tolerance for currents, from iteration to iteration. Determines all current convergence in diodes, BJTs, and JFETs. (RELMOS sets the tolerance for MOSFETs). Sets the change in current, from the value calculated at the previous timepoint. Default = 0.01 for KCLTEST = 0 , $1\text{e-}6$ for KCLTEST = 1 .

RELMOS = x Relative error tolerance for drain-to-source current, from iteration-to-iteration. Determines MOSFET current convergence. (RELI sets the tolerance for other active devices.) Sets the change in current, from the value calculated at the previous timepoint. Star-Hspice uses the RELMOS value, *only* if the current is greater than the ABSMOS floor value. Default = 0.05 .

$RELV = x$	Relative error tolerance for voltages. If voltage or current exceeds the absolute tolerance, a RELV test determines convergence. Increasing RELV increases the relative error. You should generally maintain RELV at its default value. RELV conserves simulator charge. For voltages, RELV is the same as RELTOL. Default = $1e-3$.
$RELVDC = x$	Relative error tolerance for voltages. If voltage or current exceeds the absolute tolerance, a RELVDC test determines convergence. Increasing RELVDC increases the relative error. You should generally maintain RELVDC at its default value. RELVDC conserves simulator charge. Default = RELTOL (RELTOL default = $1e-3$).

Autoconverge Process

If a circuit does not converge in the number of iterations that *ITL1* specifies, Star-Hspice initiates an auto-convergence process. This process manipulates DCON, GRAMP, and GMINDC, and even CONVERGE in some cases. [Figure 10-3 on page 10-41](#) shows the autoconverge process.

Note: Star-Hspice uses autonvergence in transient analysis, but it also uses autonvergence in DC analysis, if the Newton-Raphson (N-R) method fails to converge.

Notes:

1. Setting `.OPTION DCON = -1` disables steps 2 and 3.
2. Setting `.OPTION CONVERGE = -1` disables steps 4 and 5.
3. Setting `.OPTION DCON = -1 CONVERGE = -1` disables steps 2, 3, 4, and 5.
4. If you set the DV option to a value other than the default, step 2 uses the value you set for DV, but step 3 changes DV to $1e6$.
5. Setting GRAMP in an `.OPTION` statement has no effect on the auto-converge process. Auto-converge sets GRAMP independently.
6. If you specify a value for GMINDC in an `.OPTION` statement, GMINDC ramps to the value you set, instead of to $1e-12$, in steps 2 and 3.

DCON and GMINDC

GMINDC helps stabilize the circuit, during DC operating-point analysis. For MOSFETs, GMINDC helps stabilize the device, in the vicinity of the threshold region. Star-Hspice inserts GMINDC between:

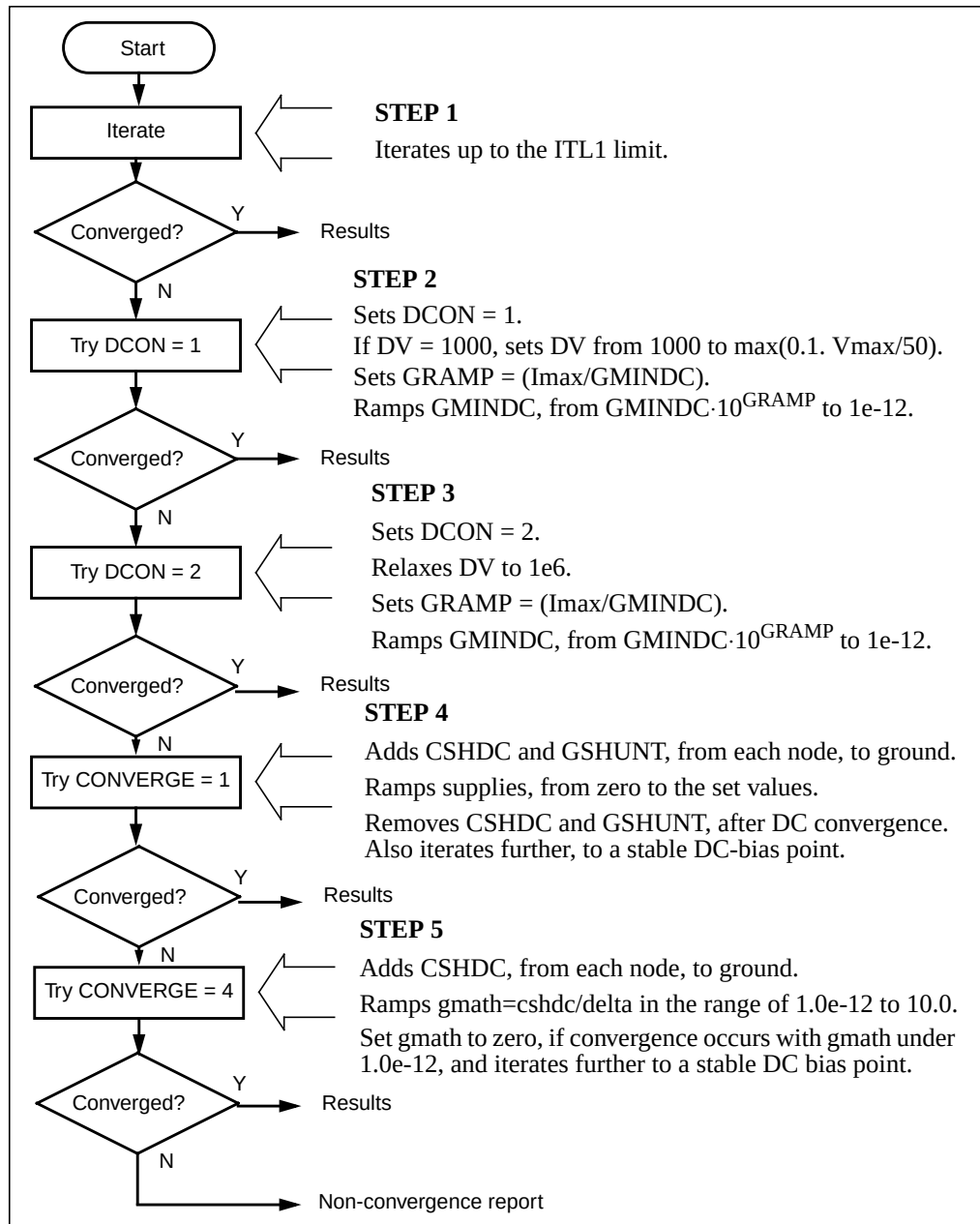
- Drain and bulk.
- Source and bulk.
- Drain and source.

The drain-to-source GMINDC helps to:

- Linearize the transition, from cutoff to weakly-on.
- Smooth-out model discontinuities.
- Compensate for the effects of negative conductances.

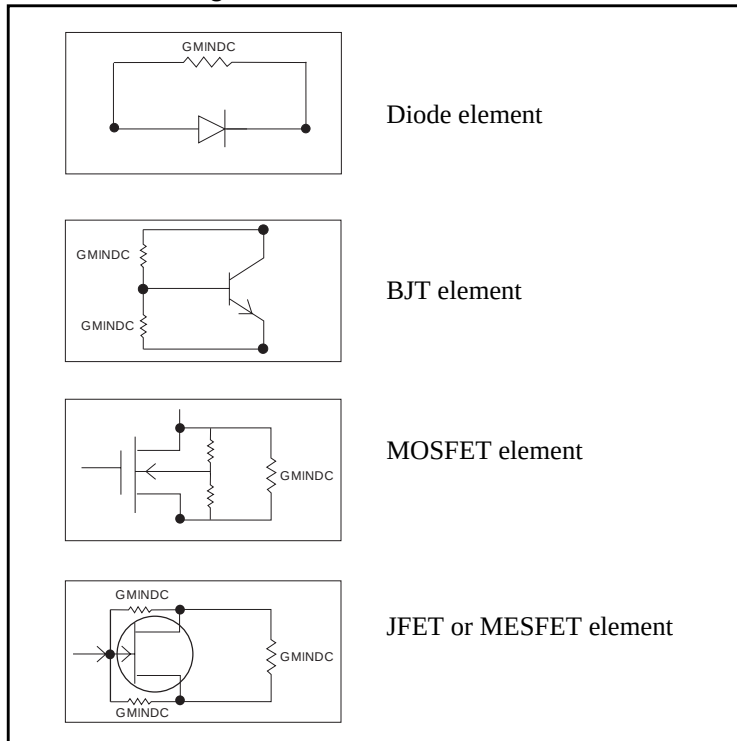
The pn junction insertion of GMINDC, in junction diodes, linearizes the low conductance region. As a result, the device behaves like a resistor, in the low-conductance region. This prevents the occurrence of zero conductance, and improves the convergence of the circuit.

Figure 10-3: Autoconvergence Process Flow Diagram



If a circuit does not converge, Star-Hspice automatically sets the DCON option. This option invokes GMINDC ramping, in steps 2 and 3 of [Figure 10-3 on page 10-41](#). [Figure 10-4](#) shows GMINDC, for various elements.

Figure 10-4: GMINDC Insertion



Reducing DC Errors

To reduce DC errors, perform the following steps:

1. To check topology, set `.OPTION NODE`, to list nodal cross-references.
 - ❑ Do all MOS p-channel substrates connect to either VCC or positive supplies?
 - ❑ Do all MOS n-channel substrates connect to either GND or negative supplies?
 - ❑ Do all vertical NPN substrates connect to either GND or negative supplies?
 - ❑ Do all lateral PNP substrates connect to negative supplies?
 - ❑ Do all latches have either an OFF transistor, a `.NODESET`, or an `.IC`, on one side?
 - ❑ Do all series capacitors have a parallel resistance, or is `.OPTION DCSTEP` set?
2. Check your `.MODEL` statements.
 - ❑ Check all model parameter units. Use model printouts to verify actual values and units, because Star-Hspice multiplies some model parameters by scaling options.
 - ❑ Are sub-threshold parameters of MOS models, set with reasonable value (such as `NFS = 1e11` for SPICE 1, 2, and 3 models, or `N0 = 1.0` for True-Hspice BSIM1, BSIM2, and Level 28 device models)?
 - ❑ Avoid setting UTRA in MOS Level 2 models.
 - ❑ Are JS and JSW set in the MOS model, for the DC portion of a diode model? A typical JS value is $1e-4A/M^2$.
 - ❑ Are CJ and CJSW set, in MOS diode models?
 - ❑ Do JFET and MESFET models have weak-inversion NG and ND set?
 - ❑ If you use the MOS Level 6 LGAMMA equation, is `UPDATE = 1`?
 - ❑ Make sure that DIODE models have non-zero values, for saturation current, junction capacitance, and series resistance?
 - ❑ Use MOS `ACM = 1`, `ACM = 2`, or `ACM = 3` source and drain diode calculations, to automatically generate parasitics.

3. General remarks:

- ❑ Ideal current sources require large values of `.OPTION GRAMP`, especially for BJT and MESFET circuits. Such circuits do not ramp up with the supply voltages, and can force reverse-bias conditions, leading to excessive nodal voltages.
- ❑ Schmitt triggers are unpredictable for DC sweep, and sometimes for operating points, for the same reasons that oscillators and flip-flops are unpredictable. Use slow transient.
- ❑ Large circuits tend to have more convergence problems, because they have a higher probability of uncovering a modeling problem.
- ❑ Circuits that converge individually, but fail when combined, are almost guaranteed to have a modeling problem.
- ❑ Open-loop op-amps have high gain, which can lead to difficulties in converging. Start op-amps in unity-gain configuration, and open them up in transient analysis, using a voltage-variable resistor, or a resistor with a large AC value (for AC analysis).

4. Check your options:

- ❑ Remove all convergence-related options, and try first with no special options settings.
- ❑ Check non-convergence diagnostic tables, for non-convergent nodes. Look up non-convergent nodes in the circuit schematic. They are generally latches, Schmitt triggers, or oscillating nodes.
- ❑ For stubborn convergence failures, bypass DC all together, and use `.TRAN` with `UIC` set. Continue transient analysis until transients settle out, then specify the `.OP` time, to obtain an operating point during the transient analysis. To specify an AC analysis during the transient analysis, add an `.AC` statement to the `.OP` time statement.
- ❑ `SCALE` and `SCALM` scaling options have a significant effect on parameter values in both elements and models. Be careful with units.

Shorted Element Nodes

Star-Hspice disregards any capacitor, resistor, inductor, diode, BJT, or MOSFET, if all of its leads connect together. Simulation does not count the component in its component tally, and issues a warning:

```
** warning ** all nodes of element x:<name> are
connected together
```


Inserting Conductance, Using DCSTEP

In a DC operating-point analysis, failure to include conductances in a capacitor model results in broken circuit loops (because a DC analysis opens all capacitors). This might not be solvable. If you include a small conductance in the capacitor model, the circuit loops are complete, and Star-Hspice can solve them.

Modeling capacitors as complete opens, can result in the following error:

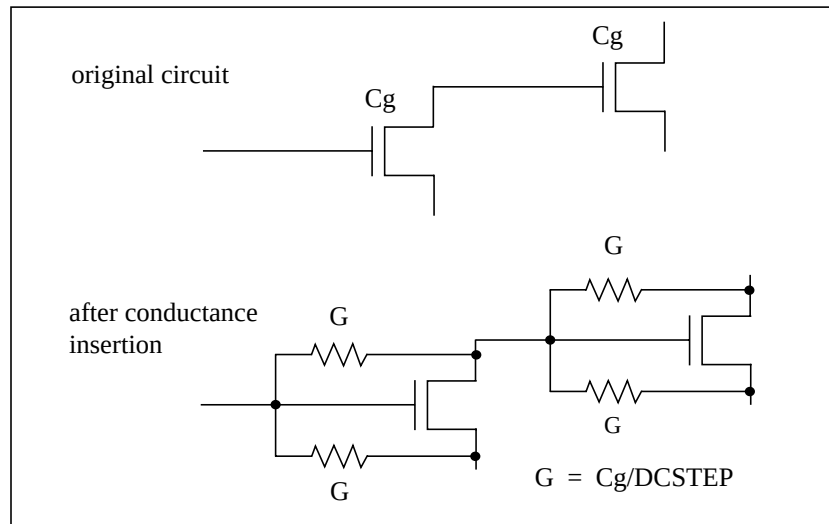
“No DC Path to Ground”

For a DC analysis, use `.OPTION DCSTEP`, to assign a conductance value to all capacitors in the circuit. DCSTEP calculates the value as follows:

$$\text{conductance} = \text{capacitance} / \text{DCSTEP}$$

Figure 10-5 shows how Star-Hspice inserts conductance (G), in parallel with capacitance (Cg). This provides current paths around capacitances, in DC analysis.

Figure 10-5: Conductance Insertion



Floating-Point Overflow

If MOS conductance is negative or zero, Star-Hspice might have difficulty converging. An indication of this type of problem is a floating-point overflow, during matrix solutions. Star-Hspice detects floating-point overflow, and invokes the Damped Pseudo Transient algorithm (`CONVERGE = 1`), to try to achieve DC convergence without requiring you to intervene. If `GMINDC` is `1.0e-12` or less when a floating-point overflows, Star-Hspice sets it to `1.0e-11`.

Diagnosing Convergence Problems

Before simulation, Star-Hspice diagnoses potential convergence problems in the input circuit, and provides an early warning, to help you in debugging your circuit. If Star-Hspice detects a circuit condition that might cause convergence problems, it prints the following message into the output file:

```
"Warning: Zero diagonal value detected at node ( ) in
equation solver, which might cause convergence problems. If
your simulation fails, try adding a large resistor between
node ( ) and ground."
```

Non-Convergence Diagnostic Table

If a circuit cannot converge, Star-Hspice automatically generates two printouts, called the *diagnostic tables*:

- Nodal voltage printout, which prints the names of all no-convergent node voltages, and the associated voltage error tolerances (*tol*).
 - Element printout, which lists all non-convergent elements, and their associated element currents, element voltages, model parameters, and current error tolerances (*tol*).
1. To locate the branch current, or the nodal voltage, that causes non-convergence, analyze the diagnostic tables. Look for unusually large values of branch currents, nodal voltages or tolerances.
 2. After you locate the cause, use the .NODESET or .IC statements, to initialize the node or branch.

If circuit simulation does not converge, Star-Hspice automatically generates a non-convergence diagnostic table, indicating:

- The quantity of recorded voltage failures.
- The quantity of recorded branch element failures.

Any node in the circuit can generate voltage failures, including *hidden* nodes (such as extra nodes that parasitic resistors can create).

3. Check the element printout for the sub-circuit, model, and element name, for all parts of the circuit where node voltages or currents do not converge.

For example, [Table 10-4](#) identifies the *xinv21*, *xinv22*, *xinv23*, and *xinv24* inverters, as problem sub-circuits in a ring oscillator. It also indicates that the p-channel transistors, in the *xinv21*, *xinv22*, *xinv24* sub-circuits, are nonconvergent elements. The n-channel transistor of *xinv23* is also a nonconvergent element.

The table lists voltages and currents for the transistors, so you can check whether they have reasonable values. The *tolds*, *tolbd*, and *tolbs* error tolerances indicate how close the element currents (drain to source, bulk to drain, and bulk to source) are, to a convergent solution. For *tol* variables, a value close to or below 1.0 is a convergent solution. In [Table 10-4](#), the *tol* values that are around 100, indicate that the currents were far from convergence. The element current and voltage values are also shown (*id*, *ibs*, *ibd*, *vgs*, *vds*, and *vbs*). Examine whether these values are realistic, and determine the transistor regions of operation.

Table 10-4: Voltages, Currents, and Tolerances for Subcircuits

subckt element model	xinv21 21:mphc1 0:p1	xinv22 22:mphc1 0:p1	xinv23 23:mphc1 0:p1	xinv23 23:mnch1 0:n1	xinv24 24:mphc1 0:p1
id	27.5809f	140.5646u	1.8123p	1.7017m	5.5132u
ibs	205.9804f	3.1881f	31.2989f	0.	200.0000f
ibd	0.	0.	0.	-168.7011f	0.
vgs	4.9994	-4.9992	69.9223	4.9998	-67.8955
vds	4.9994	206.6633u	69.9225	-64.9225	2.0269
vbs	4.9994	206.6633u	69.9225	0.	2.0269
vth	-653.8030m	-745.5860m	-732.8632m	549.4114m	-656.5097m
tolds	114.8609	82.5624	155.9508	104.5004	5.3653
tolbd	0.	0.	0.	0.	0.
tolbs	3.534e-19	107.1528m	0.	0.	0.

Traceback of Non-Convergence Source

To locate a non-convergence source, trace the circuit path, for error tolerance. For example, in an inverter chain, the last inverter can have a very high error tolerance. If this is the case, examine the error tolerance of the elements that drive the inverter. If the driving tolerance is high, the driving element could be the source of non-convergence. However, if the tolerance is low, check the driven element as the source of non-convergence.

When you examine the voltages and current levels of a non-convergent MOSFET, you can discover the operating region of the MOSFET. This information can flow to the location of the discontinuity in the model—for example, subthreshold-to-linear, or linear-to-saturation.

When considering error tolerances, check the current and nodal voltage values. If these values are extremely low, then a relatively large number is being divided by a very small number. This produces a large calculation result, which probably causes the non-convergence errors. To solve this, increase the value of the absolute-accuracy options.

Use the diagnostic table, with the DC iteration limit (ITL1 statement), to find the sources of non-convergence. When you increase or decrease ITL1, Star-Hspice prints output for the problem nodes and elements, for a new iteration—that is, the last iteration of the analysis that you set in ITL1.

Solutions for Non-Convergent Circuits

Non-convergent circuits generally result from:

- [Poor Initial Conditions](#)
- [Inappropriate Model Parameters](#)
- [PN Junctions \(Diodes, MOSFETs, BJTs\)](#)

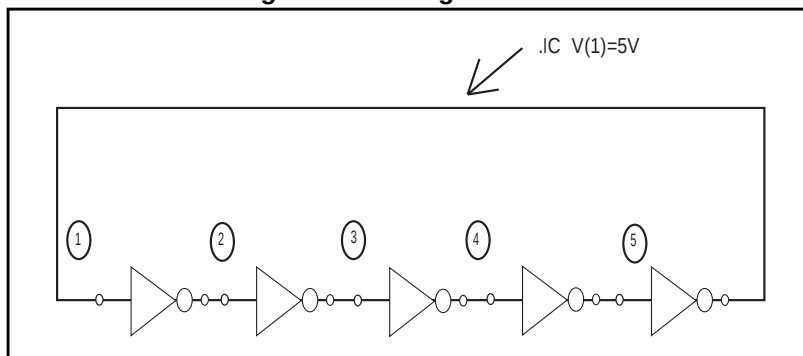
The following sections explain these conditions.

Poor Initial Conditions

Multi-stable circuits need state information, to guide the DC solution. You must initialize ring oscillators and flip-flops. These multi-stable circuits can either produce an intermediate forbidden state, or cause a DC convergence problem.

To initialize a circuit, use the `.IC` statement, which forces a node to the requested voltage. Ring oscillators usually require you to set only one stage.

Figure 10-6: Ring Oscillator



The best way to set up the flip-flop is to use an `.IC` statement in the subcircuit definition. The following example sets the local `Qset` parameter to 0, and uses this value for the `.IC` statement, to initialize the `Q` latch output node. As a result, all latches have a default state of `Q` low. Setting `Qset` to `vdd` calls a latch, which overrides this state.

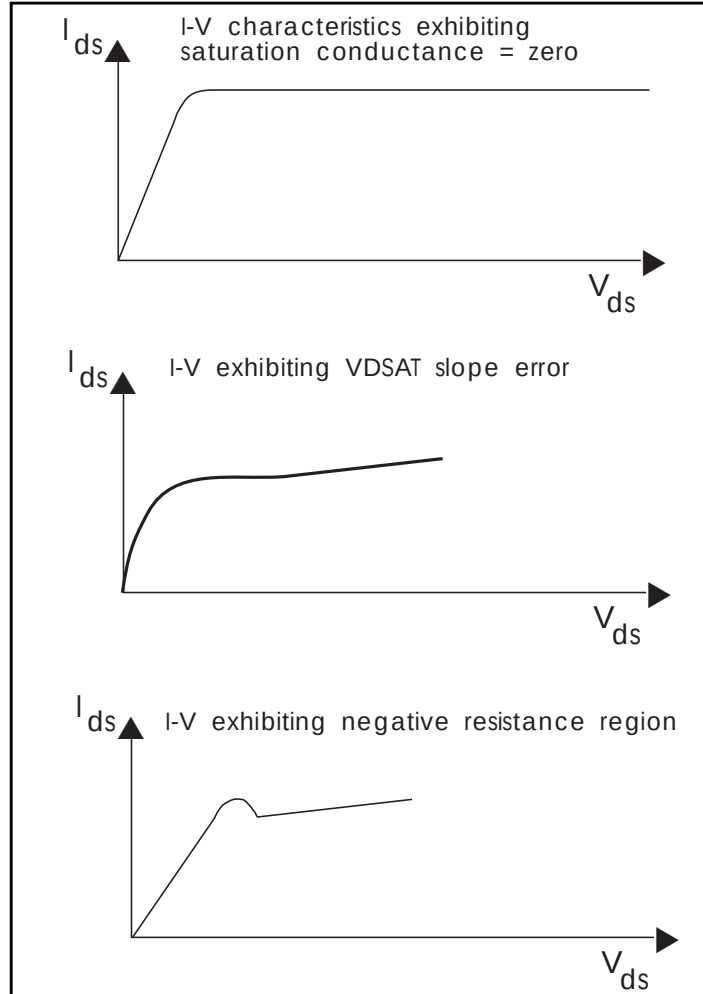
For example:

```
.subckt latch in Q Q/ d Qset = 0
.ic Q = Qset
...
.ends
.Xff data_in[1] out[1] out[1]/ strobe LATCH Qset = vdd
```

Inappropriate Model Parameters

If you impose non-physical model parameters, you might create a discontinuous `IDS` or capacitance model. This can cause an *internal timestep too small* error, during the transient simulation. The `mosivcv.sp` demonstration file shows `IDS`, `VGS`, `GM`, `GDS`, `GMB`, and `CV` plots, for MOS devices. A sweep near threshold, from $V_{th}-0.5$ V to $V_{th}+0.5$ V (using a delta of 0.01 V), sometimes discloses a possible discontinuity in the curves.

Figure 10-7: Discontinuous I-V Characteristics



If the simulation no longer converges, when you add a component or change a component value, then the model parameters are inappropriate, or do not correspond to the physical values that they represent.

1. Check the input netlist file, for non-convergent elements.
Devices with a *TOL* value greater than 1, are non-convergent.
2. Find the devices, at the beginning of the combined-logic string of gates, that seem to start the non-convergent string.

3. Check the operating point of these devices very closely, to see what region they operate in.

Model parameters associated with this region are probably inappropriate.

Circuit simulation is based on using single-transistor characterization, to simulate a large collection of devices. If a circuit fails to converge, the cause can be a single transistor, anywhere in the circuit.

PN Junctions (Diodes, MOSFETs, BJTs)

PN junctions found in diode, BJT, and MOSFET models, might exhibit non-convergent behavior, in both DC and transient analysis.

For example, PN junctions often have a high *off* resistance, resulting in an ill-conditioned matrix. To overcome this, the `GMINDC` and `GMIN` options automatically parallel every PN junction in a design, with a conductance.

Non-convergence can occur if you overdrive the PN junction. This happens if you omit a current-limiting resistor, or if the resistor has a very small value. In transient analysis, protection diodes are often temporarily forward-biased (due to the inductive switching effect). This overdrives the diode, and can result in non-convergence, if you omit a current-limiting resistor.



Chapter 11

Transient Analysis

Transient analysis computes the circuit solution, as a function of time, over a time range specified in the .TRAN statement.

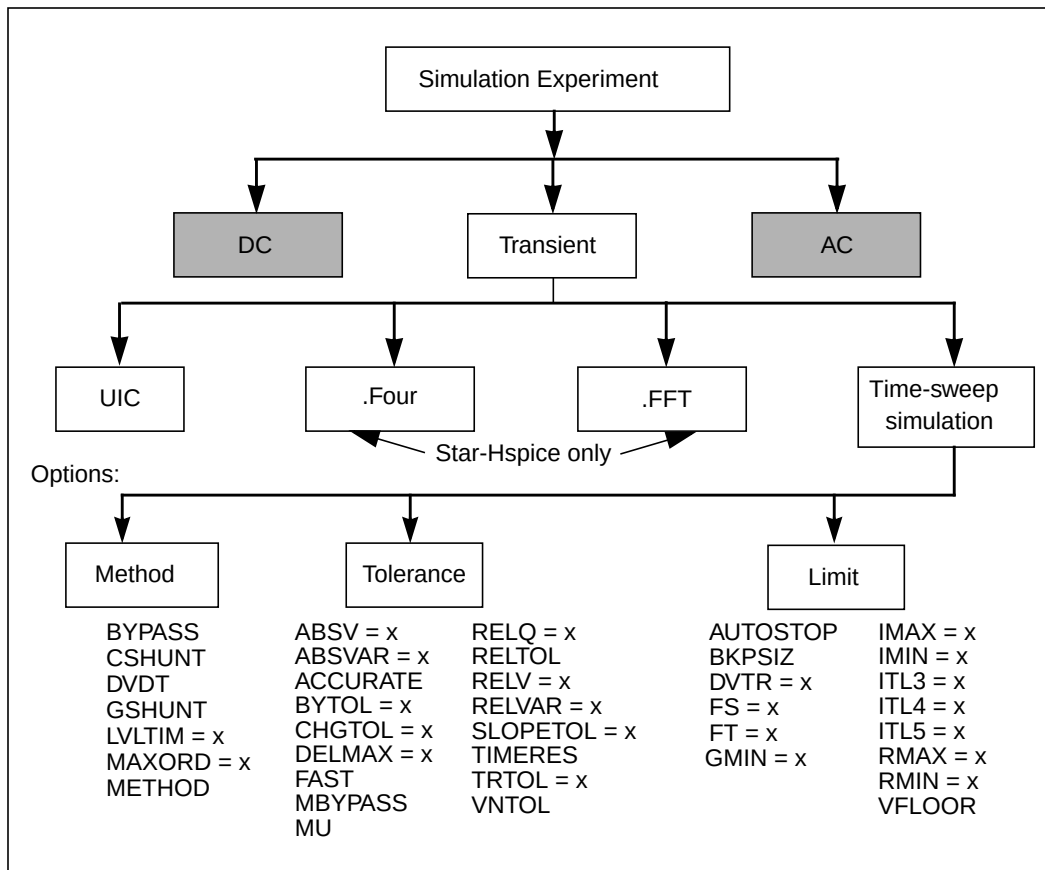
This chapter explains the following topics:

- [Simulation Flow](#)
- [Overview of Transient Analysis](#)
- [Using the .TRAN Statement](#)
- [Transient Analysis of an RC Network](#)
- [Transient Analysis of an Inverter](#)
- [Using the .BIASCHK Statement](#)
- [Transient Control Options](#)
- [Controlling Simulation Speed and Accuracy](#)
- [Numerical Integration Algorithm Controls](#)
- [Selecting Timestep Control Algorithms](#)
- [Fourier Analysis](#)

Simulation Flow

Figure 11-1 illustrates the simulation flow, for transient analysis in Star-Hspice.

Figure 11-1: Transient Analysis Simulation Flow



Overview of Transient Analysis

Transient analysis simulates a circuit at a specific time. *Some* of its algorithms, control options, convergence-related issues, and initialization parameters are different than those used in DC analysis. However, a transient analysis first performs a DC operating point analysis, unless you specify the UIC option in the .TRAN statement. Therefore, *most* DC analysis algorithms, control options, initialization issues, and convergence issues, also apply to transient analysis.

Unless you set the initial circuit operating conditions, some circuits (such as oscillators, or circuits with feedback) do not have stable operating point solutions. For these circuits, either:

- Break the feedback loop, to calculate a stable DC operating point, or
- Specify the initial conditions, in the simulation input.

If you include the UIC parameter in the .TRAN statement, Star-Hspice bypasses the DC operating point analysis. Instead, it uses node voltages, specified in an .IC statement, to start a transient analysis. For example, if a .IC statement sets a node to 5 V in, the value at that node for the first time point (time 0) is 5 V.

You can use the .OP statement to store an estimate of the DC operating point, during a transient analysis.

In the following example, the UIC parameter (in the .TRAN statement) bypasses the initial DC operating point analysis. The .OP statement calculates the transient operating point (at $t = 20$ ns), during the transient analysis.

```
.TRAN 1ns 100ns UIC  
.OP 20ns
```

Although a transient analysis might provide a convergent DC solution, the transient analysis itself can still fail to converge. In a transient analysis, the *internal timestep too small* error message indicates that the circuit failed to converge. The cause of this convergence failure might be that stated initial conditions are not close enough to the actual DC operating point values. Use the commands in this chapter to help achieve convergence in a transient analysis.

Using the .TRAN Statement

Syntax

Single-Point Analysis

```
.TRAN tincr1 tstop1 <tincr2 tstop2 ...tincrN tstopN>
+ <START = val> <UIC>
```

Double-Point Analysis

```
.TRAN tincr1 tstop1 <tincr2 tstop2 ...tincrN tstopN>
+ <START = val> <UIC>
+ <SWEEP var type np pstart pstop>
```

or

```
.TRAN tincr1 tstop1 <tincr2 tstop2 ...tincrN tstopN>
+ <START = val> <UIC> <SWEEP var START="param_expr1"
+ STOP="param_expr2"
+ STEP="param_expr3">
```

or

```
.TRAN tincr1 tstop1 <tincr2 tstop2 ... tincrN tstopN>
+ <START=val> <UIC> <SWEEP var start_expr stop_expr
+ step_expr>
```

Data-Driven Sweep

Star-Hspice supports the following types of data-driven sweep syntax:

```
.TRAN DATA = datanm
```

or

```
TRAN tincr1 tstop1 <tincr2 tstop2 ...tincrN tstopN>
+ <START = val> <UIC> <SWEEP DATA = datanm>
```

or

```
.TRAN DATA = datanm<SWEEP var type np pstart pstop>
```

or

```
.TRAN DATA=datanm <SWEEP var START="param_expr1"  
+STOP="param_expr2" STEP="param_expr3">
```

or

```
.TRAN DATA=datanm <SWEEP var start_expr stop_expr step_expr>
```

Monte Carlo Analysis

Star-Hspice supports the following Monte Carlo syntax for transient analysis.

```
.TRAN tincr1 tstop1 <tincr2 tstop2 ...tincrN tstopN>  
+ <START = val> <UIC><SWEEP MONTE = val>
```

Optimization

Star-Hspice supports the following Optimization syntax for transient analysis.

```
.TRAN DATA = datanm OPTIMIZE = opt_par_fun  
+ RESULTS = measnames MODEL = optmod
```

.TRAN Keywords and Parameters

Transient sweep specifications can include these keywords and parameters:

Table 11-1: Keywords and Parameters in a Transient Sweep (Sheet 1 of 2)

<i>DATA = datanm</i>	Data name, referenced in the .TRAN statement.
<i>MONTE = val</i>	Produces a specified number (<i>val</i>) of randomly-generated values, which Star-Hspice uses to select parameters from a distribution. The distribution can be <i>Gaussian</i> , <i>Uniform</i> , or <i>Random Limit</i> .
<i>np</i>	Number of points, or number of points per decade or octave, depending on what keyword precedes it.
<i>param_expr...</i>	Expressions you specify: <i>param_expr1...param_exprN</i> .
<i>pincr</i>	Voltage, current, element, or model parameter; or temperature increment value. If you set the <i>type</i> variation, use <i>np</i> (number of points), instead of <i>pincr</i> .
<i>pstart</i>	Starting voltage, current, or temperature; or any element or model parameter value. If you set the <i>type</i> variation to POI (list of points), use a list of parameter values, instead of <i>pstart pstop</i> .
<i>pstop</i>	Final voltage, current, or temperature; or any element or model parameter value.
<i>START</i>	Time when printing or plotting begins. The <i>START</i> keyword is optional: you can specify a start time without the keyword. If you use .TRAN with a .MEASURE statement, a non-zero <i>START</i> time can cause incorrect .MEASURE results. Do not use non-zero <i>START</i> times in .TRAN statements, when you also use .MEASURE.
<i>SWEEP</i>	This keyword indicates that the .TRAN statement specifies a second sweep.

Table 11-1: Keywords and Parameters in a Transient Sweep (Sheet 2 of 2)

<i>tincr1...</i>	Specifies the printing or plotting increment for printer output, and the suggested computing increment for post-processing.
<i>tstop1...</i>	Time when a transient analysis stops incrementing by the first specified time increment (<i>tincr1</i>). If another <i>tincr</i> - <i>tstop</i> pair follows, analysis continues with a new increment.
<i>type</i>	Specifies any of the following keywords: ■ DEC – decade variation. ■ OCT – octave variation (the value of the designated variable is eight times its previous value). ■ LIN – linear variation. ■ POI – list of points.
<i>UIC</i>	Uses the nodal voltages specified in the .IC statement (or in the <i>IC</i> = parameters of the various element statements) to calculate initial transient conditions, rather than solving for the quiescent operating point.
<i>var</i>	Name of an independent voltage or current source, any element or model parameter, or the TEMP keyword (indicating a temperature sweep). You can use a source value sweep, referring to the source name (SPICE style). However, if you specify a parameter sweep, a .DATA statement, and a temperature sweep, you must choose a parameter name for the source value, and subsequently refer to it in the .TRAN statement. The parameter name must not start with V or I.

.TRAN Examples

- The following example performs and prints the transient analysis, every 1 ns, for 100 ns.

```
.TRAN 1NS 100NS
```

2. The following example performs the calculation every 0.1 ns, for the first 25 ns; and then every 1 ns, until 40 ns. Printing and plotting begin at 10 ns.

```
.TRAN .1NS 25NS 1NS 40NS START = 10NS
```

3. The following example performs the calculation every 10 ns, for 1 μ s. This example bypasses the initial DC operating point calculation. It uses the nodal voltages, specified in the .IC statement (or by IC parameters in element statements), to calculate the initial conditions.

```
.TRAN 10NS 1US UIC
```

4. The following example increases the temperature by 10 °C, through the range -55 °C to 75 °C. It also performs transient analysis, for each temperature.

```
.TRAN 10NS 1US UIC SWEEP TEMP -55 75 10
```

5. The following example analyzes each load parameter value, at 1 pF, 5 pF, and 10 pF.

```
.TRAN 10NS 1US SWEEP load POI 3 1pf 5pf 10pf
```

6. The following example is a data-driven time sweep. It uses a data file as the sweep input. If the parameters in the data statement are controlling sources, then a piecewise linear specification must reference them.

```
.TRAN data = dataname
```

.TRAN Options

BYPASS	Bypasses model evaluations for MOSFETs, if the terminal voltages do not change. Can be 0 (off) or 1 (on). Default is 1.
CSHUNT	Shunt capacitance value.
AUTOSTOP	If on, .TRAN simulation stops when it finds all .MEASURE results. Can be 0 (off) or 1 (on). Default is 0.
GMIN	Minimum conductance added to all PN junctions.

.TRAN Output Syntax

```
.print tran ov1 [ov2 ... ovN]
.probe tran ov1 [ov2 ... ovN]
.measure tran measspec
.plot tran ov1 [ov2 ... ovN]
.graph tran ov1 [ov2 ... ovN]
```

The *ov1*, ... *ovN* output variables can include the following:

- $V(n)$: voltage at node *n*.
- $V(n1,n2)$: voltage between the *n1* and *n2* nodes.
- $Vn(d1)$: voltage at *n*th terminal of the *d1* device.
- $In(d1)$: current into *n*th terminal of the *d1* device.
- '*expression*': expression, involving the plot variables above

You can use wildcards (* or as specified in `.admr c`) to specify multiple output variables in a single command. Output is affected by `.OPTION post` and `.OPTION probe`.

.TRAN Output Format/Description

- *.print Writes the output from the `.PRINT` statement to a *.print file. Star-Hspice does not generate a *.print# file.
- The header line contains column labels.
 - The first column is time.
 - The remaining columns represent the output variables specified with `.PRINT`.
 - The rows that follow the header, contain the data values for the simulated time points.
- *.tr# Writes output from the `.PROBE`, `.PRINT`, `.PLOT`, `.GRAPH`, or `.MEASURE` statement to a *.tr# file.

Transient Analysis of an RC Network

You can run a transient analysis, using an RC network, with a pulse source, a DC source, and an AC source.

1. Type the following netlist into a file named *quickTRAN.sp*.

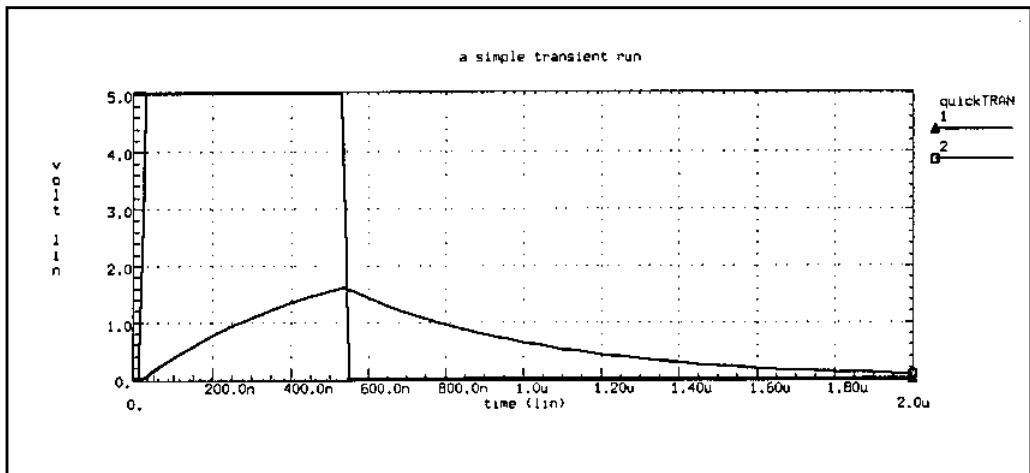
```
A SIMPLE TRANSIENT RUN
.OPTION LIST NODE POST
.OP
.TRAN 10N 2U
.PRINT TRAN V(1) V(2) I(R2) I(C1)
V1 1 0 10 AC 1 PULSE 0 5 10N 20N 20N 500N 2U
R1 1 2 1K
R2 2 0 1K
C1 2 0 .001U
.END
```

Note: The V1 source specification includes a pulse source. For the syntax of pulse sources and other types of sources, see [Using Sources and Stimuli on page 5-1](#).

2. To run Star-Hspice, type the following:
`hspice quickTRAN.sp > quickTRAN.lis`
3. To examine the simulation results and status, use an editor and view the `.lis` and `.st0` files.
4. Run AvanWaves and open the `.sp` file.
5. To view the waveform, select the *quickTRAN.tr0* file from the **Results Browser** window.
6. Display the voltage at nodes 1 and 2 on the x-axis.

Figure 11-2 shows the waveforms.

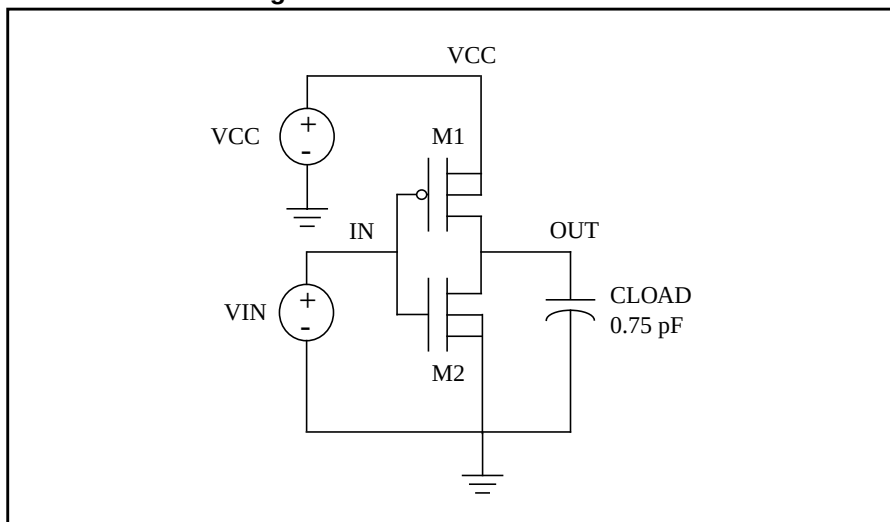
Figure 11-2: Voltages at RC Network Circuit Node 1 and Node 2



Transient Analysis of an Inverter

As a final example, analyze the behavior of the simple MOS inverter shown in [Figure 11-3](#).

Figure 11-3: MOS Inverter Circuit



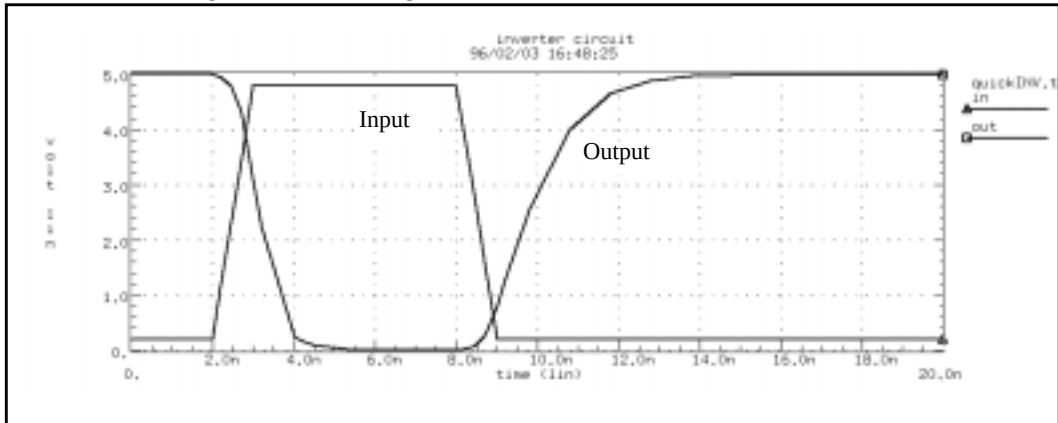
1. Type the following netlist data into a file named *quickINV.sp*.

```
Inverter Circuit
.OPTION LIST NODE POST
.TRAN 200P 20N
.PRINT TRAN V(IN) V(OUT)
M1 OUT IN VCC VCC PCH L = 1U W = 20U
M2 OUT IN 0 0 NCH L = 1U W = 20U
VCC VCC 0 5
VIN IN 0 0 PULSE .2 4.8 2N 1N 1N 5N 20N
CLOAD OUT 0 .75P
.MODEL PCH PMOS LEVEL = 1
.MODEL NCH NMOS LEVEL = 1
.END
```

2. To run Star-Hspice, type the following
hspice quickINV.sp > quickINV.lis
3. Use AvanWaves to examine the voltage waveforms, at the inverter IN and OUT nodes.

Figure 11-4 shows the waveforms.

Figure 11-4: Voltage at MOS Inverter Node 1 and Node 2



Using the .BIASCHK Statement

Breakdown can occur if the voltage bias, between some terminals of an element, is too large. The .BIASCHK statement monitors the voltage bias, using the limits and noise that you define. Bias monitoring checks the specified bias, during transient analysis, and reports the following:

- element name
- time
- terminals
- bias that exceeds the limit
- number of times the bias exceeds the limit, for an appointed element

Star-Hspice saves the information as both a warning and a BIASCHK summary, in the *.lis file.

You can use this command *only* for MOS and capacitors. For example, a .BIASCHK statement might check for voltages that exceed a specified limit, for MOS dielectric breakdown. BIASCHK can check voltages from the gate, to the source, drain, or bulk.

BIASCHK cannot detect the bias that exceeds the limit, if the bias is always the same value during transient analysis.

If a model name, referenced in an active element statement, contains a period (.), then .BIASCHK reports an error. This occurs because it is unclear whether a reference such as x.123 is a model name or a sub-circuit name (123 model in the x sub-circuit).

Instance (element) and model names *can* contain wildcards, either ? (stands for one character) or * (stands for 0 or more characters).

Syntax

```
.biaschk type terminal1=t1 terminal2=t2 limit=lim
+ <noise=ns><name=devname1><name=devname2>...
+ <mname=modelname1><mname=modelname2> ...
```

where:

<i>type</i>	Element type that you want to check MOS (R, C . . .) The <i>type</i> can be NMOS, PMOS, or C.
<i>terminal 1, 2</i>	Terminals, between which Star-Hspice checks (that is, checks between <i>terminal1</i> and <i>terminal2</i>): ■ For MOS level 57: <i>nd, ng, ns, ne, np, n6</i> ■ For MOS level 58: <i>nd, ngf, ns, ngb</i> ■ For MOS level 59: <i>nd, ng, ns, ne, np</i> ■ For other MOS level: <i>nd, ng, ns, nb</i> ■ For Capacitor: <i>n1, n2</i>
<i>limit</i>	Biaschk limit that you define. Reports an error, if the bias voltage (between appointed terminals, of appointed elements and models), is larger than the limit.
<i>noise</i>	Biaschk noise that you define. The default is 0.1v. Noise-filter some of the results (the local maximum bias voltage, that is larger than the limit). The next local max replaces the local max, if all of the following conditions are satisfied: 1. <code>local_max-local_min<noise></code>. 2. <code>next local_max-local_min<noise></code>. 3. This local max is smaller than the next local max.
<i>name</i>	Element name, that you want to check.
<i>mname</i>	Model name. Star-Hspice checks elements of this model, for bias.

If you do not set name and mname, Star-Hspice checks all elements of this type (*type* is a required keyword in the `.biaschk` card), for bias voltage.

You can use a wild card, to describe name and mname, in the `biaschk` card.

- ? stands for one character.
- * stands for 0 or more characters.

Example

```
.biaschk NMOS terminal1=ng terminal2=nb limit=2v  
+ noise=0.01v name=x1.x3.m1 mname=nch.1 name=m3
```

Options for the `.biaschk` Command

`biasfile` Option

- If you use this option, Star-Hspice outputs the results of all `.biaschk` commands in this netlist, to a file that you specify.
- If you do not set this option, Star-Hspice outputs the results to the `*.lis` file.

For example:

```
.option biasfile='biaschk/mos.bias'
```

`biawarn` Option

- If you set this option to 1, Star-Hspice immediately outputs a warning message, when any local max bias voltage exceeds the limit *during* transient analysis. *After* this transient analysis, Star-Hspice outputs the results summary, as filtered by noise.
- If you set this option to 0 (the default), Star-Hspice does not output a warning message *during* transient analysis. Star-Hspice outputs the results, *after* this transient analysis.

For example:

```
.option biawarn=1
```


Transient Control Options

The options in this section modify the behavior of the transient analysis integration routines. Delta refers to the internal timestep. TSTEP and TSTOP refer to the step and stop values entered with the .TRAN statement. The options are grouped into three categories: method, tolerance, and limit:

Table 11-2: Transient Control Options, Arranged by Category

Method	Tolerance		Limit	
BYPASS	ABSH	RELQ	AUTOSTOP	ITL3
CSHUNT	ABSV	RELTOL	BKPSIZ	ITL4
DVDT	ABSVAR	RELV	DELMAX	ITL5
GSHUNT	ACCURATE	RELVAR	DVTR	RMAX
INTERP	BYTOL	SLOPETOL	FS	RMIN
ITRPRT	CHGTOL	TIMERES	FT	VFLOOR
LVLTIM	DI	TRTOL	GMIN	
MAXORD	FAST	VNTOL		
METHOD	MBYPASS			
PURETP	MAXAMP			
TRCON	MU			
	RELH			
	RELI			

Method Options

Table 11-3: Method Options (Sheet 1 of 5)

<i>BYPASS</i>	Bypasses model evaluations, if the terminal voltages do not change. Can be 0 (off) or 1 (on). To speed-up simulation, this option does not update the status of latent devices. To enable bypassing, set <code>.OPTION BYPASS = 1</code>, for MOSFETs, MESFETs, JFETs, BJTs, or diodes. Default = 1. Note: For some circuit types, BYPASS can result in non-convergence, and loss of accuracy in both transient analysis, and operating-point calculations.
<i>CSHUNT</i>	Added capacitance, from each node, to ground. Add a small CSHUNT to each node, to solve some <i>internal timestep too small</i> errors, caused by high-frequency oscillations or numerical noise. Default = 0.
<i>DVDT</i>	Adjusts the timestep, based on rates of change for node voltages.: ■ 0 - original algorithm. ■ 1 - fast. ■ 2 - accurate. ■ 3,4 - balance speed and accuracy. Default = 4.
<i>GSHUNT</i>	Conductance added, from each node, to ground. The default is zero. Adding a small GSHUNT to each node solves some <i>internal timestep too small</i> errors, caused by high-frequency oscillations or numerical noise.

Table 11-3: Method Options (Sheet 2 of 5)

<i>INTERP</i>	Limits output to post-analysis tools, such as Cadence or Zuken, to only the .TRAN timestep intervals. By default, Star-Hspice outputs all convergent iterations into the design.tr# file. INTERP typically produces a much smaller design.tr# file. Use INTERP = 1 with caution, when you also use a .MEASURE statement. To compute measure statements, Star-Hspice uses the post-processing output. Reducing post-processing output can lead to interpolation errors, in measure results. When you run data-driven transient analysis (.TRAN DATA) in an optimization routine, Star-Hspice forces INTERP to 1. All measurement results are at the time points specified in the data-driven sweep. To measure <i>only</i> at converged internal timesteps (for example, to calculate the AVG or RMS), set ITRPRT = 1.
<i>ITRPRT</i>	Prints output variables. at their internal time points. This option might generate a long output list.
<i>LVLTIM = x</i>	Selects the timestep algorithm, for transient analysis. ■ LVLTIM = 1 uses the DVDT timestep algorithm. ■ LVLTIM = 2 uses the timestep algorithm for local truncation error. ■ LVLTIM = 3 uses the DVDT timestep algorithm, with timestep reversal. To use the TRAP linearization algorithm, select LVLTIM 1 or 3. LVLTIM = 1 (the DVDT option) is the default, and helps avoid the <i>internal timestep too small</i> non-convergence error. To use the GEAR method of numerical integration and linearization, select LVLTIM = 2. The local truncation algorithm (LVLTIM = 2) provides a higher degree of accuracy than the TRAP method. If you use this option, errors do not propagate from time point to time point, which can result in an unstable solution.

Table 11-3: Method Options (Sheet 3 of 5)

<i>MAXORD = x</i>	Sets the maximum order of integration, for the GEAR method (see METHOD). The value of <i>x</i> can be 1 or 2. ■ If you specify MAXORD = 1, Star-Hspice uses the backward Euler method of integration. ■ MAXORD = 2 (the default) is more stable, accurate, and practical.
<i>METHOD = name</i>	Sets the numerical integration method, for a transient analysis, to either GEAR or TRAP. ■ To use GEAR, set METHOD = GEAR. This automatically sets LVLTIM = 2. ■ To change LVLTIM from 2, to either 1 or 3, set LVLTIM = 1 or 3, after the METHOD = GEAR option. This overrides the LVLTIM = 2 setting, which is the default in METHOD = GEAR.
	TRAP (trapezoidal) integration generally reduces program execution time, and returns more accurate results. However, trapezoidal integration can introduce an apparent oscillation on printed or plotted nodes, which might not be caused by circuit behavior. To test if this is the case, run a transient analysis with a small timestep. If the oscillation disappears, it was due to the trapezoidal method. The GEAR method is a filter, removing the oscillations found in the trapezoidal method. Highly non-linear circuits (such as operational amplifiers) can require very long execution times, if you use the GEAR method. Circuits that do not converge if you use trapezoidal integration, often converge if you use GEAR. Default = TRAP (trapezoidal).

Table 11-3: Method Options (Sheet 4 of 5)

PURETP	Sets the integration method to use, for the reversal time point. The default value is 0. If you set puretp=1, then if Star-Hspice encounters non-convergence, it uses TRAP (instead of B.E) for the reversed time point. You can use this option to help some oscillating circuits to oscillate, if the default simulation process cannot satisfy the result. Use this option with the method=TRAP statement.
TRCON	Controls the automatic convergence (<i>autoconvergence</i>) and automatic speedup (<i>autospeedup</i>) processes in Star-Hspice. ■ TRCON=1 (the default) enables both <i>autoconvergence</i> and <i>autospeedup</i>. ■ TRCON= 0 enables <i>autospeedup</i> only. ■ TRCON =-1 disables both <i>autoconvergence</i> and <i>autospeedup</i>. Aautoconvergence If the circuit fails to converge using the trapezoidal (TRAP) numerical integration method (for example, because of trapezoidal oscillation), Star-Hspice uses the GEAR method and LTE timestep algorithm, to run the transient analysis again from time=0. This process is called <i>autoconvergence</i>. Autoconvergence sets the following options to their default values before the second try: <pre style="margin-left: 40px;">METHOD=GEAR, LVLTIM=2, MBYPASS=1.0, BYPASS=0.0, SLOPETOL=0.5 BYTOL= min{mbypas*vntol and reltol}</pre> RMAX=2.0, if it was 5.0 in the first simulation run. Otherwise, RMAX does not change.

Table 11-3: Method Options (Sheet 5 of 5)**Autospeedup**

For some large non-linear circuits with large TSTOP/TSTEP values, analysis might run for an excessively long time. In this case, Star-Hspice might automatically set a new and bigger RMAX value, to speed up the analysis for primary reference. In most cases, however, Star-Hspice does not activate this type of autospeedup process.

For autospeedup to occur, all three of the following conditions must occur:

- N1 (Number of Nodes) > 1,000
- N2 (TSTOP/TSTEP) >= 10,000
- N3 (Total Number of Diode, BJTs, JFETs and MOSFETs) > 300

Autospeedup is most likely to occur if the circuit also meets either of the following conditions:

- N2 >= 1e+8, and N3 > 500, or
- N2 >= 2e+5, and N3 > 1e+4

If Star-Hspice *does* activate autospeedup, you might need to disable it. To do this, set TRCON=-1, and increase TSTEP or RMAX (or both), to balance accuracy and speed.

Tolerance Options

Table 11-4: Tolerance Options (*Sheet 1 of 5*)

<i>ABSH</i> = <i>x</i>	Sets the absolute current change, through voltage- defined branches (voltage sources and inductors). Use ABSH with DI and RELH, to check for current convergence. Default = 0.0.
<i>ABSV</i> = <i>x</i>	Sets the absolute minimum voltage, for DC and transient analysis. ABSV is the same as VNTOL. Decrease VNTOL, if accuracy is more important than convergence. If you need voltages less than 50 microvolts, reduce VNTOL to two orders of magnitude less than the smallest desired voltage. This ensures at least two digits of significance. Typically, you do not need to change VNTOL, unless you are simulating a high-voltage circuit. For 1000-volt circuits, a reasonable value can be 5 to 50 millivolts. Default = 50 (microvolts).
<i>ABSVAR</i> = <i>x</i>	Maximum voltage change, from one time point to the next. Use this option with the DVDT algorithm. If the simulator produces a convergent solution that is greater than ABSVAR, it: 1. Discards the solution. 2. Sets the timestep to a smaller value. 3. Recalculates the solution. This is a <i>timestep reversal</i>. Default is 0.5 (volts).
<i>ACCURATE</i>	Selects a time algorithm, which uses LVLTIM = 3 and DVDT = 2, for circuits such as high-gain comparators. Use this option in circuits that combine high gain and large dynamic range, to guarantee solution accuracy.

Table 11-4: Tolerance Options (Sheet 2 of 5)

	If you set ACCURATE to 1, Star-Hspice uses the following control options: ■ LVLTIM = 3 ■ DVDT = 2 ■ RELVAR = 0.2 ■ ABSVAR = 0.2 ■ FT = 0.2 ■ RELMOS = 0.01 Default = 0.
<i>BYTOL</i> = <i>x</i>	Specifies a voltage tolerance, at which a MOSFET, MESFET, JFET, BJT, or diode becomes latent. Star-Hspice does not update the status of latent devices. Default = MBYPASS <i>x</i> VNTOL.
<i>CHGTOL</i> = <i>x</i>	Sets a charge error tolerance, if you set LVLTIM = 2. Use CHGTOL with RELQ, to set the absolute and relative charge tolerance, for all Star-Hspice capacitances. Default = 1e-15 (coulomb).
<i>DI</i> = <i>x</i>	Sets the maximum iteration-to-iteration current change, through voltage-defined branches (voltage sources and inductors). Use this option <i>only</i> if the value of the DI control option is greater than 0. Default = 0.0.
<i>FAST</i>	To speed-up simulation, this option does not update the status of latent devices. Use this option for MOSFETs, MESFETs, JFETs, BJTs, and diodes. Default = 0. A device is latent if its node voltage variation (from one iteration to the next) is less than the value of the BYTOL control option, or the BYPASSTOL element parameter. (If FAST is on, Star-Hspice sets BYTOL to different values, for different types of device models.)

Table 11-4: Tolerance Options (Sheet 3 of 5)

	In addition to the FAST option, you can use the NOTOP and NOELCK options, to reduce input pre-processing time. Increasing the value of the MBYPASS or BYTOL option also helps simulations to run faster, but can reduce accuracy.
$MAXAMP = x$	Sets the maximum current, through voltage-defined branches (voltage sources and inductors). If the current exceeds the MAXAMP value, Star-Hspice reports an error. Default = 0.0.
$MBYPASS = x$	Computes a default value for the BYTOL option: $BYTOL = MBYPASS \times VNTOL$ Also multiplies the RELV voltage tolerance. Set MBYPASS to about 0.1, for precision analog circuits. ■ Default = 1, for DVDT = 0, 1, 2, or 3. ■ Default = 2 for DVDT = 4.
$MU = x$	Coefficient, for trapezoidal integration. MU can range from 0.0 to 0.5. Default=0.5.
$RELH = x$	Sets relative current tolerance, through voltage-defined branches (voltage sources and inductors). RELH checks current convergence. Use this option <i>only</i> if the value of the ABSH control option is greater than zero. Default = 0.05.
$RELI = x$	Sets the relative error/tolerance change, from iteration to iteration. This value determines convergence for all currents, in diode, BJT, and JFET devices. (RELMOS sets the tolerance for MOSFETs). This is the percent change in current, from the value calculated at the previous timepoint. ■ Default = 0.01 for KCLTEST = 0. ■ Default = 1e-6 for KCLTEST = 1.

Table 11-4: Tolerance Options (Sheet 4 of 5)

<i>RELQ = x</i>	Used in the timestep algorithm for local truncation error (LVLTIM = 2). RELQ changes the size of the timestep. If the capacitor charge calculation (in the present iteration) exceeds that of the past iteration, by a percentage greater than the value of RELQ, then Star-Hspice reduces the internal timestep (Delta). Default = 0.01.
<i>RELTOL, RELV</i>	Sets the relative error tolerance, for voltages. Use RELV, with the ABSV control option, to determine voltage convergence. Increasing RELV increases the relative error. RELV is the same as RELTOL. The RELI and RELVDC options default to the RELTOL value. Default = 1e-3.
<i>RELVAR = x</i>	Use this option with ABSVAR and the DVDT timestep algorithm, to set the relative voltage change for LVLTIM = 1 or 3. If the nodal voltage (at the current time point) is RELVAR volts higher than the nodal voltage at the previous time point, then Star-Hspice reduces the timestep, and calculates a new solution at a new time point. Default = 0.30 (30%).
<i>SLOPETOL = x</i>	Sets a lower limit for breakpoint table entries, in a piecewise linear (PWL) analysis. If the difference in the slopes of two consecutive PWL segment is less than the SLOPETOL value, then Star-Hspice ignores the breakpoint table entry, for the point between the segments. Default = 0.5.
<i>TIMERES = x</i>	Sets a minimum separation between breakpoint values, for the breakpoint table. If two breakpoints are closer together in time than the TIMERES value, then Star-Hspice enters only one of them in the breakpoint table. Default = 1 ps.

Table 11-4: Tolerance Options (Sheet 5 of 5)

$TRTOL = x$	Used in the timestep algorithm for local truncation error (LVLTIM = 2). After this algorithm generates TRTOL, Star-Hspice multiplies the internal timestep by TRTOL. Although TRTOL reduces simulation time, it also maintains accuracy. This factor estimates the amount of error introduced, when you truncate the Taylor series expansion, which the algorithm uses. This error reflects the minimum timestep required, to reduce simulation time and maintain accuracy. The range of TRTOL is 0.01 to 100; typical values range from 1 to 10. If you set TRTOL to 1 (the minimum value), Star-Hspice uses a very small timestep. As you increase the TRTOL setting, the timestep size increases. Default = 7.0.
$VNTOL = x,$	Sets the minimum voltage, for DC and transient analysis. If accuracy is more important than convergence, decrease VNTOL. If you need voltages less than 50 microvolts, reduce VNTOL to two orders of magnitude less than the smallest desired voltage. This ensures at least two significant digits. Typically, you do not need to change VNTOL, unless you are simulating a high-voltage circuit. For 1000-volt circuits, a reasonable value can be 5 to 50 millivolts. ABSV is the same as VNTOL. Default = 50 (microvolts).

Limit Options

Table 11-5: Limit Options (Sheet 1 of 3)

<i>AUTOSTOP</i>	Stops the transient analysis, after calculating all TRIG-TARG and FIND-WHEN measure functions. This option can substantially reduce CPU time. If the data file contains measure functions (such as AVG, RMS, MIN, MAX, PP, ERR, ERR1, 2, 3, and PARAM), then Star-Hspice disables AUTOSTOP. If on, .TRAN simulation stops when it finds all .MEASURE results. Can be 0 (off, the default) or 1 (on).
<i>BKPSIZ = x</i>	Sets the size of the breakpoint table. Default = 5000.
<i>DELMAX = x</i>	Sets the maximum value for the internal timestep (Delta). Star-Hspice automatically sets the DELMAX value, based on that factors listed in Timestep Control for Accuracy on page 11-32. The initial DELMAX value, shown in the Star-Hspice output listing, is generally not the value used for simulation.
<i>DVTR</i>	Limits voltage in a transient analysis. Default = 1000.
<i>FS = x</i>	Decreases Delta (internal timestep) by the specified fraction of a timestep (TSTEP), for the first time point of a transient. Decrease the FS value to help circuits that have timestep convergence difficulties. DVDT = 3 uses FS to control the timestep. $Delta = FS \times [MIN(TSTEP, DELMAX, BKPT)]$ ■ You specify DELMAX. ■ BKPT is related to the breakpoint of the source. The .TRAN statement sets TSTEP. Default = 0.25.
<i>FT = x</i>	Decreases Delta (the internal timestep), by a specified fraction of a timestep (TSTEP), for an iteration set that does not converge. If DVDT = 2 or DVDT = 4, FT controls the timestep. Default = 0.25.
<i>GMIN = x</i>	Sets the minimum conductance added to all PN junctions, for a time sweep in transient analysis. Default = 1e-12.

Table 11-5: Limit Options (Sheet 2 of 3)

$IMIN = x$, $ITL3 = x$	Minimum timestep, in timestep algorithms for transient analysis. IMIN is the minimum number of iterations, to converge. If the number of iterations is less than IMIN, the internal timestep (Delta) doubles. This decreases simulation times, in circuits where nodes are stable most of the time (such as digital circuits). If the number of iterations is greater than IMIN, then the timestep stays the same, unless the number of iterations exceeds IMAX (see IMAX). ITL3 is the same as IMIN. Default = 3.0.
$IMAX = x$, $ITL4 = x$	Maximum timestep, in timestep algorithms for transient analysis. IMAX is the maximum iterations, to converge at a timepoint. If the number of required iterations is greater than IMAX, the internal timestep (Delta) decreases, by a factor equal to the FT transient control option. Star-Hspice uses the new timestep, to calculate a new solution. IMAX also works with the IMIN transient control option. ITL4 is the same as IMAX. Default = 8.0.
$ITL5 = x$	Sets an iteration limit for transient analysis. If a circuit uses more than ITL5 iterations, the program prints all results, up to that point. The default (0.0) allows an infinite number of iterations.
$RMAX = x$	Sets the TSTEP multiplier, which determines the maximum value (DELMAX) for the internal timestep (Delta): $DELMAX = TSTEP \times RMAX$ ■ Default = 5, if dvdt = 4 and lvltim = 1. ■ Otherwise, the default = 2.
$RMIN = x$	Sets the minimum value of Delta (internal timestep). An internal timestep smaller than $RMIN \times TSTEP$, terminates the transient analysis, and reports an <i>internal timestep too small</i> error. If the circuit does not converge in IMAX iterations, Delta decreases by the amount you set in the FT option. Default = 1.0e-9.

Table 11-5: Limit Options (Sheet 3 of 3)

$V_{FLOOR} = x$	Sets a minimum voltage to print in the output listing. All voltages lower than V_{FLOOR} , print as 0. This affects only the output listing: V_{NTOL} (ABS V) sets the minimum voltage to use in a simulation.
-----------------	---

Matrix Manipulation Options

After Star-Hspice generates individual linear elements (in an input netlist file), it constructs the linear equations for the matrix. You can set variables that affect how Star-Hspice constructs and solves the matrix equation, including the `PIVOT` and `GMIN` options. `GMIN` places a variable into the matrix, to prevent the matrix becoming *ill-conditioned*.

Use the `PIVOT` option to select a pivoting method, which reduces simulation time, and assists in both DC and transient convergence. Pivoting reduces the error, resulting from elements in the matrix that are widely different in magnitude. `PIVOT` searches the matrix, to find the largest element value, and then uses this value as the pivot.

Controlling Simulation Speed and Accuracy

Convergence is the ability to solve a set of circuit equations, within specified tolerances, and within a specified number of iterations. In numerical circuit simulation, a designer specifies a relative and absolute accuracy for the circuit solution. The simulator iteration algorithm then attempts to converge to a solution that is within these set tolerances. That is, if consecutive simulations achieve results within the specified accuracy tolerances, circuit simulation has converged. How quickly the simulator converges, is often a primary concern to a designer—especially for preliminary design trials. So designers willingly sacrifice some accuracy, for simulations that converge quickly.

Simulation Speed

Star-Hspice can substantially reduce the computer time needed to solve complex problems. You can use the following options to alter the internal algorithms, to increase simulation efficiency.

- `.OPTION FAST` – sets additional options, which increase simulation speed, with minimal loss of accuracy
- `.OPTION AUTOSTOP` – terminates the simulation, after completing all `.MEASURE` statements. This is of special interest, when testing corners.

For descriptions of the `FAST` and `AUTOSTOP` options, see [Transient Control Options on page 11-17](#).

Simulation Accuracy

In Star-Hspice, the default values of the control option aim for superior accuracy, within an acceptable amount of simulation time. The control options, and their default settings (to maximize accuracy), are:

```
DVDT = 4    LVLTIM = 1    RMAX = 5    SLOPETOL = 0.75
FT = FS = 0.25    BYPASS = 1
BYTOL = MBYPASSxVNTOL = 0.100m
```

Note: BYPASS is on (set to 1), only when DVDT = 4. For other DVDT settings, BYPASS is off (0). The SLOPETOL value is 0.75, only if DVDT = 4 and LVLTIM = 1. For all other values of DVDT or LVLTIM, SLOPETOL defaults to 0.5.

Timestep Control for Accuracy

The DVDT control option selects the timestep control algorithm. For a description of the relationships between DVDT and other control options, see [Selecting Timestep Control Algorithms on page 11-38](#).

The DELMAX control option also affects simulation accuracy. DELMAX specifies the maximum allowed timestep size. If you do not set DELMAX in an .OPTION statement, Star-Hspice computes a DELMAX value. Factors that determine the computed DELMAX value are:

- .OPTION RMAX and FS.
- Breakpoint locations, for a PWL source.
- Breakpoint locations, for a PULSE source.
- Smallest period, for a SIN source.
- Smallest delay, for a transmission line component.
- Smallest ideal delay, for a transmission line component.
- TSTEP value, in a .TRAN analysis.
- Number of points, in an FFT analysis.

Use the FS and RMAX control options, to control the DELMAX value.

- The FS option, which defaults to 0.25, scales the breakpoint interval in the DELMAX calculation.
- The RMAX option, which defaults to 5 (if DVDT = 4 and LVLTIM = 1), scales the TSTEP (timestep) size in the DELMAX calculation.

For circuits that contain oscillators or ideal delay elements, use an .OPTION statement, to set DELMAX to one-hundredth of the period or less.

The ACCURATE control option tightens the simulation options, to output the most accurate set of simulation algorithms and tolerances. If you set ACCURATE to 1, Star-Hspice uses the following control options:

DVDT = 2	RELVAR = 0.2	BYPASS = 0	FT = FS = 0.2	RELMOS = 0.01
BYTOL = 0	LVLTIM = 3	ABSVAR = 0.2	RMAX = 2	SLOPETOL = 0.5

Models and Accuracy

Simulation accuracy depends on the sophistication and accuracy of the models you use. Advanced MOS, BJT, and GaAs models provide superior results for critical applications. The following model types increase simulation accuracy:

- Algebraic models, which describe parasitic interconnect capacitances as a function of the width of the transistor. The wire model extension of the resistor can model the metal, diffusion, or poly interconnects, to preserve the relationship between the physical layout and the electrical property.
- The ACM parameter in MOS models, which calculates defaults for source and drain junction parasitics. Star-Hspice uses ACM equations to calculate:
 - size of the bottom wall
 - length of the sidewall diodes
 - length of a lightly doped structure.

SPICE defaults do not calculate the junction diode. Specify AD, AS, PD, PS, NRD, NRS, to override the default calculations.

- The CAPOP = 4 parameter in MOS models; models the most advanced charge conservation, non-reciprocal gate capacitances. Star-Hspice calculates the gate capacitors and overlaps, from the IDS model for LEVEL 49 or 53. Simulation ignores the CAPOP parameter; instead, use the CAPMOD model parameter, with a reasonable value.

Guidelines for Choosing Accuracy Options

Use the ACCURATE option for:

- Analog or mixed signal circuits.
- Circuits with long time constants, such as RC networks.
- Circuits with ground bounce.

Use the default options (DVDT = 4) for:

- Digital CMOS.
- CMOS cell characterization.
- Circuits with fast moving edges (short rise and fall times).

For ideal delay elements, use one of the following:

- ACCURATE.
- DVDT = 3.
- DVDT = 4. If the minimum pulse width of any signal is less than the minimum ideal delay, set DELMAX to a value smaller than the minimum pulse width.

Numerical Integration Algorithm Controls

If you use Star-Hspice for transient analysis, you can select one of three options:

- Gear
- Backward-Euler
- Trapezoidal

to convert differential terms into algebraic terms.

Syntax

Gear algorithm:

```
.OPTION METHOD = GEAR
```

Backward-Euler:

```
.OPTION METHOD = GEAR MU = 0
```

Trapezoidal algorithm (default):

```
.OPTION METHOD = TRAP
```

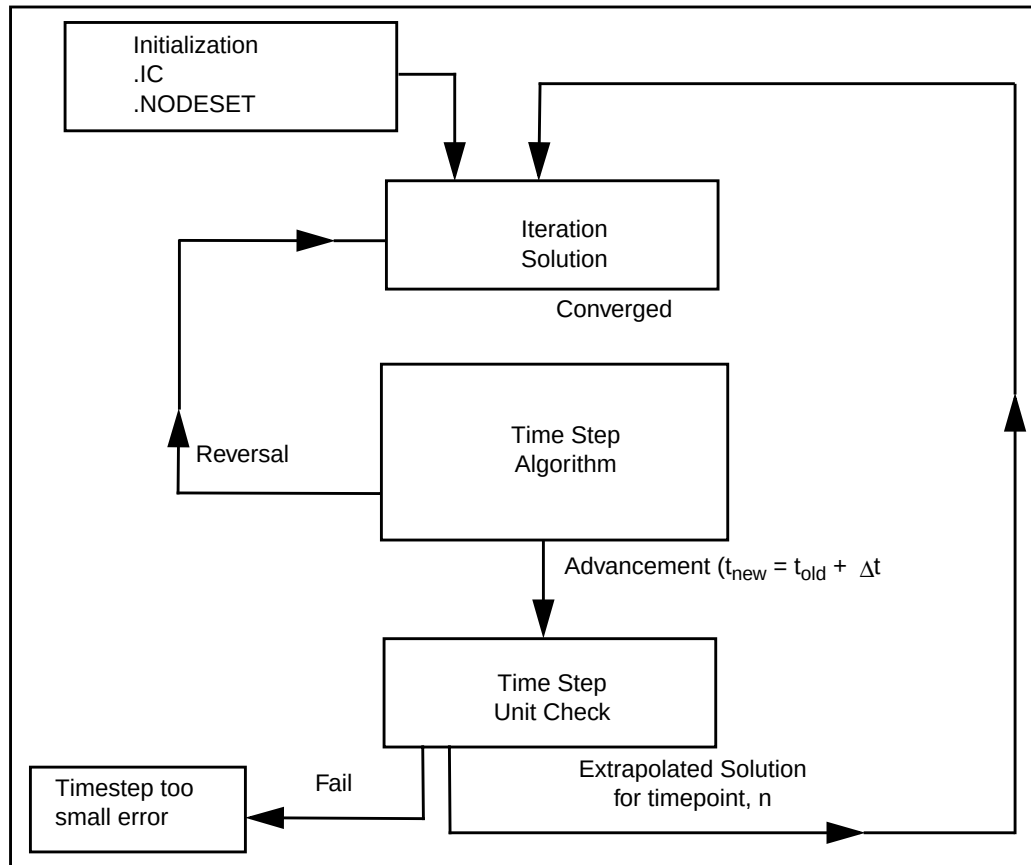
Each algorithm has advantages and disadvantages. Ideally, the trapezoidal is the preferred algorithm overall, because of its highest accuracy level and lowest simulation time.

However, selecting the appropriate algorithm for convergence is not always that easy or ideal. Which algorithm you select, largely depends on the type of circuit, and its associated behavior when you use different input stimuli.

Gear and Trapezoidal Algorithms

The algorithm that you select, automatically sets the timestep control algorithm. In Star-Hspice, if you select the GEAR algorithm (including Backward-Euler), the timestep control algorithm defaults to the truncation timestep algorithm. However, if you select the trapezoidal algorithm, the DVDT algorithm is the default. To change these Star-Hspice default s, use the timestep control options.

Figure 11-5: Time Domain Algorithm



The trapezoidal algorithm can cause computational oscillation—that is, oscillation that the algorithm itself causes, not oscillation from the circuit design. This also produces an unusually long simulation time. If this occurs in inductive circuits (such as switching regulators), use the GEAR algorithm.

If transient analysis fails to converge using METHOD=TRAP and DVDT timesteps (for example, due to trapezoidal oscillation), and Star-Hspice reports an *internal timestep too small* error, by default Star-Hspice then starts the *autonvergence* process. This process sets METHOD=GEAR and LVLTIM=2, and uses the Local Truncation Error (LTE) timestep algorithm. Star-Hspice then runs another transient analysis, to automatically obtain convergent results.

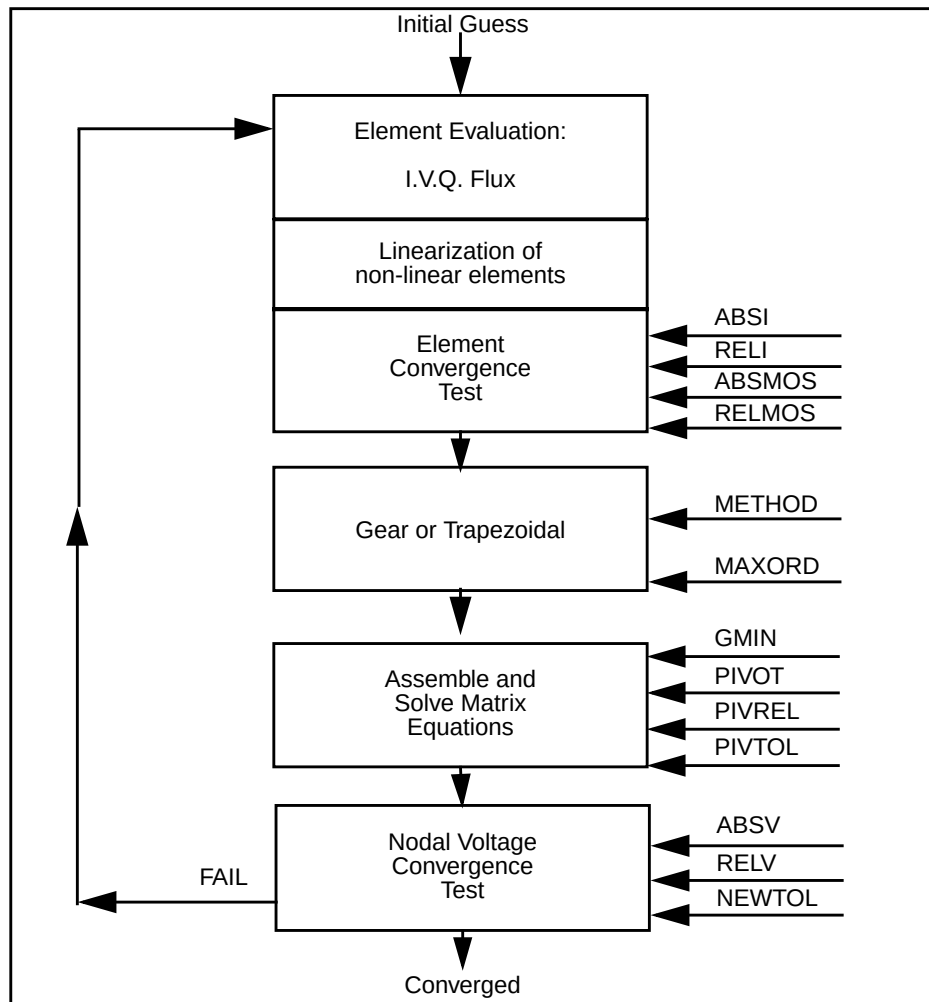
To manually improve on autoconvergence results, or if autoconvergence fails to converge, you can do either of the following:

- Set METHOD=GEAR in the netlist, and try to obtain convergent results directly.

To improve accuracy or speed, you can adjust t step in a .TRAN statement, or in transient control options (such as RMAX, RELQ, CHGTOL, or TRTOL).

- Set METHOD=TRAP in the netlist, then manually adjust t step and the relevant control options (such as CSHUNT or GSHUNT).

Figure 11-6: Iteration Algorithm



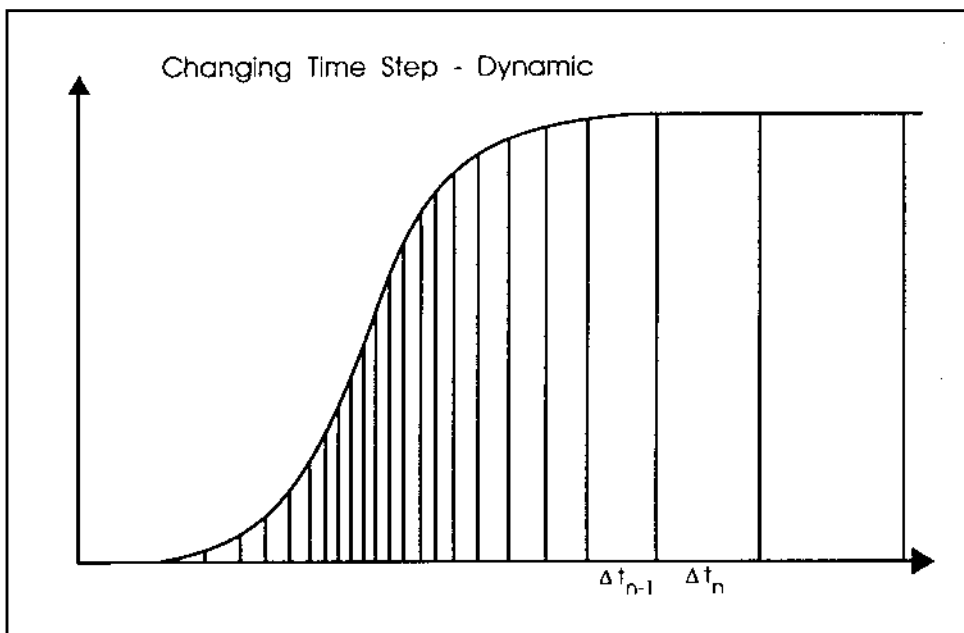
Selecting Timestep Control Algorithms

In Star-Hspice, you can select one of three dynamic timestep-control algorithms:

- [Iteration Count Dynamic Timestep Algorithm.](#)
- [Local Truncation Error \(LTE\) Dynamic Timestep Algorithm.](#)
- [DVDT Dynamic Timestep Algorithm.](#)

Each algorithm uses a dynamically-changing timestep. A dynamically-changing timestep increases the accuracy of simulation, and reduces the simulation time. To do this, simulation varies the value of the timestep, over the transient analysis sweep, depending on the stability of the output. Dynamic timestep algorithms increase the timestep value when internal nodal voltages are stable, and decrease the timestep value when nodal voltages change quickly.

Figure 11-7: Internal Variable Timestep



The LVLTIM option selects the timestep algorithm:

- LVLTIM = 0 selects the iteration count algorithm.
- LVLTIM = 1 selects the DVDT timestep algorithm, and the iteration count algorithm. To control operation of the timestep control algorithm, set the DVDT control option. For LVLTIM = 1 and DVDT = 0, 1, 2, or 3, the algorithm does not use timestep reversal. For DVDT = 4, the algorithm uses timestep reversal.

For more information about the DVDT algorithm, see [DVDT Dynamic Timestep Algorithm on page 11-40](#).

- LVLTIM = 2 selects the truncation timestep algorithm, and the iteration count algorithm (with reversal).
- LVLTIM = 3 selects the DVDT timestep algorithm (with timestep reversal), and the iteration count algorithm. For LVLTIM = 3 and DVDT = 0, 1, 2, 3, or 4, the algorithm uses timestep reversal.

If Star-Hspice starts the autoconvergence process, it sets LVLTIM = 2.

Iteration Count Dynamic Timestep Algorithm

The simplest dynamic timestep algorithm is the iteration count algorithm. The following options control this algorithm:

<i>IMAX</i>	Controls the internal timestep size, based on the number of iterations required for a timepoint solution. If the number of iterations per timepoint exceeds the IMAX value, the internal timestep decreases. Default = 8.
<i>IMIN</i>	Controls the internal timestep size, based on the number of iterations required for the previous timepoint solution. If the last timepoint solution completes in fewer than IMIN iterations, the internal timestep increases. Default = 3.

Local Truncation Error (LTE) Dynamic Timestep Algorithm

The local truncation error timestep method uses a Taylor-series approximation, to calculate the next timestep for a transient analysis. This method uses the allowed local truncation error, to calculate an internal timestep. If the calculated timestep is smaller than the current timestep, then Star-Hspice sets back the timepoint (timestep reversal), and uses the calculated timestep to increment the time. If the calculated timestep is larger than the current timestep, then Star-Hspice does not reverse the timestep. The next timepoint uses a new timestep.

To select the timestep algorithm for local truncation error, set `LVLTIM = 2` or `METHOD=GEAR`. The control options, available with the algorithm for local truncation error, are:

```
TRTOL (default = 7)
CHGTOL (default = 1e-15)
RELQ (default = 0.01)
```

For some circuits (such as magnetic core circuits), GEAR and LTE provide more accurate result than TRAP. You can use this method with circuits containing inductors, diodes, BJTs (even Level 4 and above), MOSFETs, or JFETs.

DVDT Dynamic Timestep Algorithm

To select this algorithm, set the `LVLTIM` option to 1 or 3.

- If you set `LVLTIM = 1`, the DVDT algorithm does not use timestep reversal. Star-Hspice saves the results for the current timepoint, and uses a new timestep for the next timepoint.
- If you set `LVLTIM = 3`, the algorithm uses timestep reversal. If the results do not converge at a specified iteration, Star-Hspice ignores the results of the current timepoint, sets back the time by the old timestep, and then uses a new timestep. Therefore, `LVLTIM = 3` is more accurate, and more time-consuming, than `LVLTIM = 1`.

This algorithm uses different tests, to decide whether to reverse the timestep, depending on how you set the DVDT control option.

- For DVDT = 0, 1, 2, or 3, the decision is based on the SLOPETOL control option.
- For DVDT = 4, the decision is based on how you set the SLOPETOL, RELVAR, and ABSVAR control options.

The DVDT algorithm calculates the internal timestep, based on the rate of nodal voltage changes.

- For circuits with rapidly-changing nodal voltages, the DVDT algorithm uses a small timestep.
- For circuits with slowly-changing nodal voltages, the DVDT algorithm uses larger timesteps.

The DVDT = 4 setting selects a timestep control algorithm for non-linear node voltages. If you set the LVLTIM option to either 1 or 3, then DVDT = 4 also uses timestep reversals. To measure non-linear node voltages, Star-Hspice measures changes in slopes of the voltages. If the change in slope is larger than the SLOPETOL control setting, simulation reduces the timestep, by the factor specified in the FT control option. The FT option defaults to 0.25.

Star-Hspice sets the SLOPETOL value to 0.75 for LVLTIM = 1, and to 0.50 for LVLTIM = 3. Reducing the value of SLOPETOL increases simulation accuracy, but also increases simulation time.

- For LVLTIM = 1, SLOPETOL and FT control the simulation accuracy.
- For LVLTIM = 3, the RELVAR and ABSVAR control options also affect the timestep, and therefore affect the simulation accuracy.

You can use the RELVAR and ABSVAR options, with the DVDT option, to improve simulation time or accuracy. For faster simulation time, increase RELVAR and ABSVAR (although this might decrease accuracy).

Note: If you need backward compatibility with Star-Hspice Release 95.3, use these options. Setting .OPTION DVDT = 3 automatically sets all of these values.

LVLTIM = 1	RMAX = 2	SLOPETOL = 0.5
FT = FS = 0.25	BYPASS = 0	BYTOL = 0.050

Timestep Controls

The RMIN, RMAX, FS, FT, and DELMAX control options define the minimum and maximum internal timestep, for the DVDT algorithm. If the timestep is below the minimum timestep, program execution stops. For example, if the timestep becomes less than the minimum internal timestep (defined as TSTEP×RMIN), Star-Hspice reports an *internal timestep too small* error.

Note: RMIN is the minimum timestep coefficient. Its default value is 1e-9. TSTEP is the time increment, as set in the .TRAN statement.

If you set DELMAX in an .OPTION statement, Star-Hspice uses DVDT = 0. If you do not specify DELMAX in an .OPTION statement, then Star-Hspice computes a DELMAX value. For DVDT = 0, 1, or 2, the maximum internal timestep is:

$$\min[(TSTOP/50), \text{DELMAX}, (TSTEP \times RMAX)]$$

The TSTOP time is the transient sweep range, as set in the .TRAN statement.

One exception is in the *autospeedup* process. When dealing with large non-linear circuit with very big TSTOP/TSTEP values (for example, .TRAN 1n 1), Star-Hspice might activate autospeedup. This process automatically sets RMAX to a bigger value, and sets the maximum internal timestep to:

$$\min[(TSTOP/20), (TSTEP \times RMAX)]$$

Set TRCON=-1 to disable autospeedup. You can then adjust TSTEP and RMAX, to balance accuracy and speed.

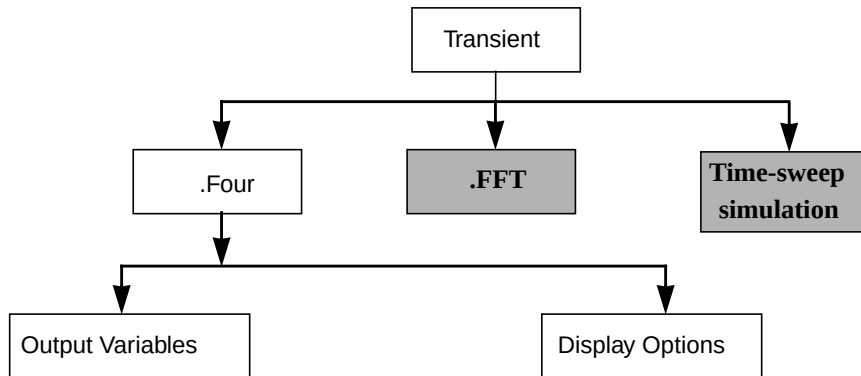
In circuits with piecewise linear (PWL) transient sources, the SLOPETOL option also affects the internal timestep. A PWL source, with a large number of voltage or current segments, contributes a correspondingly-large number of entries to the internal breakpoint table. The number of breakpoint table entries contributes to the internal timestep control.

If the difference in the slope, for consecutive segments of a PWL source, is less than the SLOPETOL value, then Star-Hspice ignores the breakpoint table entry, for the point between the segments. For a PWL source, with a signal that changes value slowly, ignoring its breakpoint table entries can help reduce the simulation time. Data in the breakpoint table is a factor in the internal timestep control, so setting a high SLOPETOL reduces the number of usable breakpoint table entries, which reduces the simulation time.

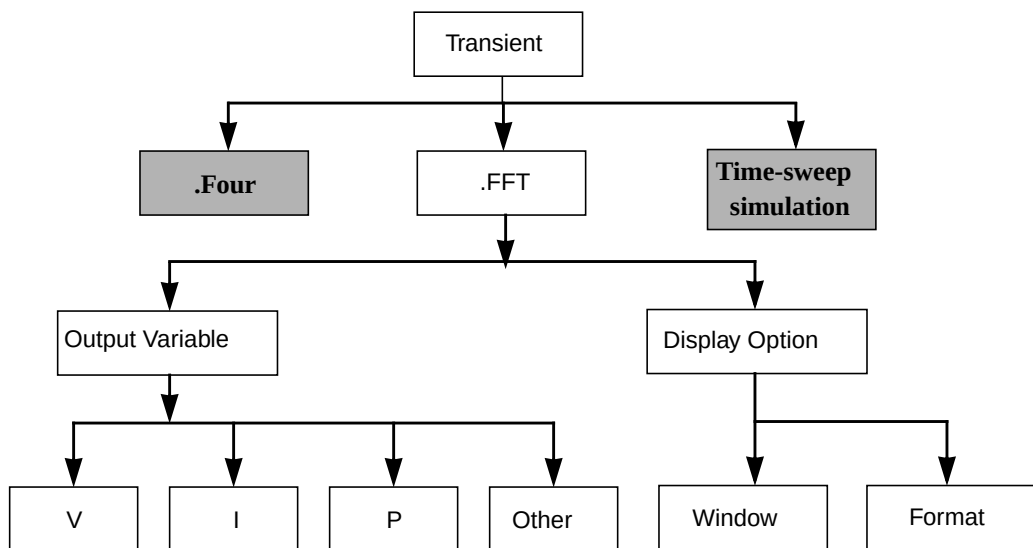
Fourier Analysis

This section describes the flow for Fourier and FFT Analysis in Star-Hspice.

Figure 11-8: Fourier and FFT Analysis



.FOUR Statement



.FFT Statement

Star-Hspice provides two different Fourier analyses:

- .FOUR is the same as is available in SPICE 2G6: a standard, fixed-window analysis tool.
- .FFT is a much more flexible Fourier analysis tool. Use it for analysis tasks that require more detail and precision.

.FOUR Statement

This statement performs a Fourier analysis, as part of the transient analysis in Star-Hspice. Star-Hspice performs the Fourier analysis over the interval (tstop-fperiod, tstop), where:

- tstop is the final time, specified for the transient analysis (see [Using the .TRAN Statement on page 11-4](#)).
- fperiod is one period of the fundamental frequency (*freq* parameter).

Star-Hspice performs Fourier analysis on 501 points of transient analysis data on the last $1/f$ time period, where f is the fundamental Fourier frequency. Star-Hspice interpolates transient data, to fit on 501 points, running from (tstop- $1/f$) to tstop. To calculate the phase, the normalized component, and the Fourier component, Star-Hspice uses 10 frequency bins. The Fourier analysis determines the DC component, and the first nine AC components.

For improved accuracy, the .FOUR statement can use non-linear, instead of linear, interpolation.

Syntax

```
.FOUR freq ov1 <ov2 ov3 ...>
```

freq Fundamental frequency.

ov1 ... Output variables to analyze.

Example

```
.FOUR 100K V(5)
```

Accuracy and DELMAX

For better accuracy, set small values for the RMAX or DELMAX options. For maximum accuracy, set .OPTION DELMAX to (period/500). For circuits with very high resonance factors (high-Q circuits, such as crystal oscillators, tank circuits, and active filters), set DELMAX to less than (period/500).

Fourier Equation

The total harmonic distortion is the square root of the sum of the squares, of the second through ninth normalized harmonic, times 100, expressed as a percent:

$$THD = \frac{1}{R1} \cdot \left(\sum_{m=2}^9 R_m^2 \right)^{1/2} \cdot 100\%$$

This interpolation can result in various inaccuracies. For example, if the transient analysis runs at intervals longer than $1/(501 \cdot f)$, then the frequency response of the interpolation dominates the power spectrum. Furthermore, this interpolation does not derive an error range for the output.

The following equation calculates the Fourier coefficients:

$$g(t) = \sum_{m=0}^9 C_m \cdot \cos(mt) + \sum_{m=0}^9 D_m \cdot \sin(mt)$$

where

$$C_m = \frac{1}{\pi} \cdot \int_{-\pi}^{\pi} g(t) \cdot \cos(m \cdot t) \cdot dt$$

$$D_m = \frac{1}{\pi} \cdot \int_{-\pi}^{\pi} g(t) \cdot \sin(m \cdot t) \cdot dt$$

$$g(t) = \sum_{m=0}^9 C_m \cdot \cos(m \cdot t) + \sum_{m=0}^9 D_m \cdot \sin(m \cdot t)$$

The following equations approximate the C and D values:

$$C_m = \sum_{n=0}^{500} g(n \cdot \Delta t) \cdot \cos\left(\frac{2 \cdot \pi \cdot m \cdot n}{501}\right)$$

$$D_m = \sum_{n=0}^{500} g(n \cdot \Delta t) \cdot \sin\left(\frac{2 \cdot \pi \cdot m \cdot n}{501}\right)$$

The following equations calculate the magnitude and phase:

$$R_m = (C_m^2 + D_m^2)^{1/2}$$

$$\Phi_m = \arctan\left(\frac{C_m}{D_m}\right)$$

Input Example

The following is Star-Hspice input for an .OP, .TRAN, or .FOUR analysis.

```
CMOS INVERTER
*
M1 2 1 0 0 NMOS W = 20U L = 5U
M2 2 1 3 3 PMOS W = 40U L = 5U
VDD 3 0 5
VIN 1 0 SIN 2.5 2.5 20MEG

.MODEL NMOS NMOS LEVEL = 3 CGDO = .2N CGSO = .2N
+ CGBO = 2N
.MODEL PMOS PMOS LEVEL = 3 CGDO = .2N CGSO = .2N
+CGBO = 2N
.OP
.TRAN 1N 100N
.FOUR 20MEG V(2)
.PRINT TRAN V(2) V(1)
.END
```

Output Example

```

*****
cmos inverter
**** fourier analysis      tnom = 25.000 temp = 25.000 ****
fourier components of transient response v(2)
dc component = 2.430D+00

```

harmonic no	frequency (hz)	fourier component	normalized component	phase (deg)	normalized phase (deg)
1	20.0000x	3.0462	1.0000	176.5386	0.
2	40.0000x	115.7006m	37.9817m	-106.2672	-282.8057
3	60.0000x	753.0446m	247.2061m	170.7288	-5.8098
4	80.0000x	77.8910m	25.5697m	-125.9511	-302.4897
5	100.0000x	296.5549m	97.3517m	164.5430	-11.9956
6	120.0000x	50.0994m	16.4464m	-148.1115	-324.6501
7	140.0000x	125.2127m	41.1043m	157.7399	-18.7987
8	160.0000x	25.6916m	8.4339m	172.9579	-3.5807
9	180.0000x	47.7347m	15.6701m	154.1858	-22.3528

total harmonic distortion = 27.3791 percent

For information about Fourier analysis, see [Fourier Analysis on page 11-43](#).

.FFT Statement

The following is the syntax of the .FFT statement in Star-Hspice. [Table 11-6 on page 11-48](#) describes the parameters.

Syntax

```

.FFT <output_var> <START = value> <STOP = value>
+ <NP = value> <FORMAT = keyword> <WINDOW = keyword>
+ <ALFA = value> <FREQ = value> <FMIN = value>
+ <FMAX = value>

```

Table 11-6: .FFT Statement Parameters

Parameter	Default	Description
output_var	—	Any valid output variable, such as voltage, current, or power.
START	see Description	Start of the output variable waveform to analyze. Defaults to the START value in the .TRAN statement, which defaults to 0.
FROM	see START	In .FFT statements, FROM is an alias for START.
STOP	see Description	End of the output variable waveform to analyze. Defaults to the TSTOP value in the .TRAN statement.
TO	see STOP	In .FFT statements, TO is an alias for STOP.
NP	1024	Number of points in the FFT analysis. NP must be a power of 2. If NP is not a power of 2, Star-Hspice automatically adjusts it, to the closest higher number that is a power of 2.
FORMAT	NORM	Output format: ■ NORM = normalized magnitude. ■ UNORM = unnormalized magnitude.
WINDOW	RECT	Window type to use: ■ RECT = rectangular truncation window. ■ BART = Bartlett (triangular) window. ■ HANN = Hanning window. ■ HAMM = Hamming window. ■ BLACK = Blackman window. ■ HARRIS = Blackman-Harris window. ■ GAUSS = Gaussian window. ■ KAISER = Kaiser-Bessel window.

Table 11-6: .FFT Statement Parameters (Continued)

Parameter	Default	Description
ALFA	3.0	Parameter used in GAUSS and KAISER windows, to control the highest side-lobe level, bandwidth, and so on. $1.0 \leq \text{ALFA} \leq 20.0$
FREQ	0.0 (Hz)	Frequency of interest. If FREQ is non-zero, the output lists only harmonics of this frequency (based on FMIN and FMAX), and the THD for these harmonics.
FMIN	1.0/T (Hz)	Minimum frequency to list in the FFT output file, or to use in THD calculations. $T = (\text{STOP} - \text{START})$
FMAX	0.5*NP*FMIN (Hz)	Maximum frequency to list in the output file, or to use in THD calculations.

Example

Below are four examples of valid .FFT statements.

```
.fft v(1)
.fft v(1,2) np = 1024 start = 0.3m stop = 0.5m
+ freq = 5.0k window = kaiser alpha = 2.5
.fft I(rload) start = 0m to = 2.0m fmin = 100k
+ fmax = 120k format = unorm
.fft par('v(1) + v(2)') from = 0.2u stop = 1.2u
+ window = harris
```

You can include only one output variable in an .FFT command. The following is an *incorrect* use of the command.

```
.fft v(1) v(2) np = 1024
```

The example below shows the correct use of the command. In this case, Star-Hspice generates an `.ft0` file for the FFT of `v(1)`, and a `.ft1` file for the FFT of `v(2)`.

```
.fft v(1) np = 1024  
.fft v(2) np = 1024
```

FFT Analysis Output

Star-Hspice prints the results of the FFT analysis in a tabular format, in the `.lis` file, based on the parameters in the `.FFT` statement. Star-Hspice also prints either the normalized magnitude values, or (if you specify `FORMAT = UNORM`) the unnormalized magnitude values. The number of printed frequencies is half the number of points (NP) specified in the `.FFT` statement. If you use `FMIN` or `FMAX` to specify a minimum or a maximum frequency, Star-Hspice prints only the specified frequency range. Moreover, if you use `FREQ` to specify a frequency, then the output lists only the harmonics of this frequency, and the percent of total harmonic distortion.

Star-Hspice generates a `.ft#` file, and the listing file, for each FFT output variable that contains data to display in FFT analysis waveforms. You can view the magnitude in dB, and the phase in degrees.

In the sample FFT analysis `.lis` file output below, the header defines all parameters used in the FFT analysis.

```
***** Sample FFT output extracted from the .lis file  
  
fft test ... sine  
*****  fft analysis tnom =   25.000 temp =   25.000  
*****  
fft components of transient response v(1)  
  
Window: Rectangular  
First Harmonic:    1.0000k  
Start Freq:       1.0000k  
Stop  Freq:      10.0000k  
  
dc component: mag(db) =  -1.132D+02 mag =  2.191D-06 phase =  
1.800D+02
```

frequency index	frequency (hz)	fft_mag (db)	fft_mag	fft_phase (deg)
2	1.0000k	0.	1.0000	-3.8093m
4	2.0000k	-125.5914	525.3264n	-5.2406
6	3.0000k	-106.3740	4.8007u	-98.5448
8	4.0000k	-113.5753	2.0952u	-5.5966
10	5.0000k	-112.6689	2.3257u	-103.4041
12	6.0000k	-118.3365	1.2111u	167.2651
14	7.0000k	-109.8888	3.2030u	-100.7151
16	8.0000k	-117.4413	1.3426u	161.1255
18	9.0000k	-97.5293	13.2903u	70.0515
20	10.0000k	-114.3693	1.9122u	-12.5492
total harmonic distortion =			1.5065m	percent

The preceding example specifies a frequency of 1 kHz, and the THD up to 10 kHz, which corresponds to the first ten harmonics.

Note: The highest frequency in the Star-Hspice FFT output might not be identical to the specified FMAX, because of Star-Hspice adjustments.

Table 11-7 describes the output of the Star-Hspice FFT analysis.

Table 11-7: .FFT Statement Output Description

Column Heading	Description
frequency index	Runs from 1 to NP/2, or the corresponding index for FMIN and FMAX. The DC component, corresponding to index 0, displays independently.
frequency	Actual frequency, associated with the index.
fft_mag (db), fft_mag	The output includes two FFT magnitude columns: the first in dB, and the second in units of the output variable. Star-Hspice normalizes magnitude, unless you specify UNORM format.
fft_phase	Associated phase, in degrees.

FFT Notes

1. Use the following formula as a guideline, when you specify a frequency range for FFT output:

$$\text{frequency increment} = 1.0 / (\text{STOP} - \text{START})$$

Each frequency index corresponds to a multiple of this increment. To obtain a finer frequency resolution, maximize the duration of the time window.

2. FMIN and FMAX have no effect on the `.ft0`, `.ft1`, ..., `.ftn` files.

For more information about the `.FFT` statement in Star-Hspice, see [Using the .FFT Statement on page 19-6](#).



Chapter 12

AC Sweep and Small Signal Analysis

This chapter describes how to perform an AC sweep, and a small signal analysis. It explains the following topics:

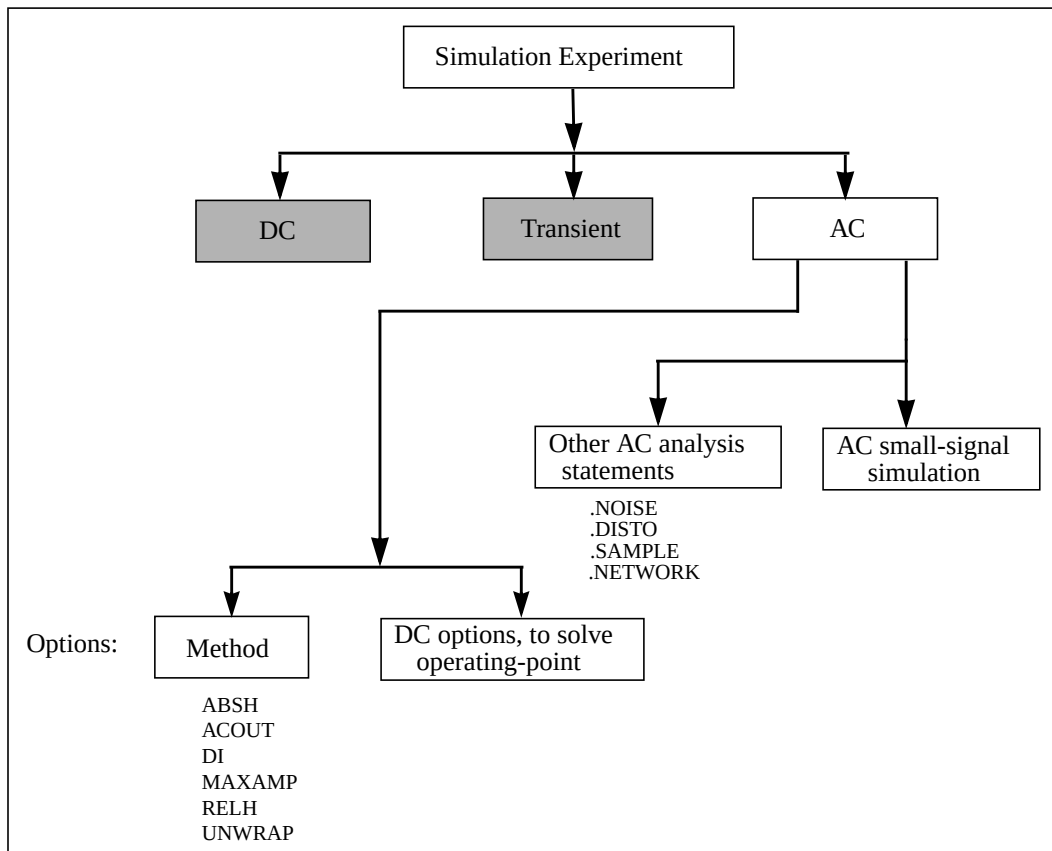
- AC Small Signal Analysis
- .AC Statement
- AC Control Options
- AC Analysis of an RC Network
- Other AC Analysis Statements

AC Small Signal Analysis

The AC small signal analysis portion of Star-Hspice computes AC output variables as a function of frequency (see [Figure 12-1](#)). Star-Hspice first solves for the DC operating point conditions. It then uses these conditions to develop linear, small-signal models, for all non-linear devices in the circuit.

In Star-Hspice, the output of AC Analysis includes voltages and currents.

Figure 12-1: AC Small Signal Analysis Flow



Star-Hspice converts capacitor and inductor values to their corresponding admittances:

$$Y_C = j\omega C \quad \text{for capacitors}$$

$$Y_L = 1/j\omega L \quad \text{for inductors}$$

Resistors can have different DC and AC values. If you specify $AC = \langle value \rangle$ in a resistor statement, Star-Hspice uses the DC value of resistance to calculate the operating point, but uses the AC resistance value in the AC analysis. When you analyze operational amplifiers, Star-Hspice uses a low value for the feedback resistance, to compute the operating point for the unity gain configuration. You can then use a very large value for the AC resistance, in AC analysis of the open loop configuration.

AC analysis of bipolar transistors is based on the small-signal equivalent circuit, as described in Chapter 4, “Using BJT Models”, in the *True-Hspice Device Models Reference Manual*. MOSFET AC-equivalent circuit models are described in Chapter 8, “Introducing MOSFETs”, in the *True-Hspice Device Models Reference Manual*.

The AC analysis statement can sweep values for:

- Frequency.
- Element.
- Temperature.
- Model parameter.
- Randomized (Monte Carlo) distribution.
- Optimization and AC design analysis.

Additionally, as part of the small-signal analysis tools, Star-Hspice provides:

- Noise analysis.
- Distortion analysis.
- Network analysis.
- Sampling noise.

.AC Statement

You can use the `.AC` statement in several different formats, depending on the application, as shown in the examples below.

You can also use the `.AC` statement to perform data-driven analysis in Star-Hspice.

Syntax

Single/Double Sweep

```
.AC type np fstart fstop
```

or

```
.AC type np fstart fstop <SWEEP var <START=>start  
+ <STOP=>stop <STEP=>incr>
```

or

```
.AC type np fstart fstop <SWEEP var type np start stop>
```

or

```
.AC type np fstart fstop <SWEEP var START="param_expr1"  
+ STOP="param_expr2" STEP="param_expr3">
```

or

```
.AC type np fstart fstop <SWEEP var start_expr  
+ stop_expr step_expr>
```

Sweep Using Parameters

You can use the following syntax in Star-Hspice:

```
.AC type np fstart fstop <SWEEP DATA = datanm>
```

or

```
.AC DATA = datanm
```


or

```
.AC DATA = datanm <SWEEP var <START=>start <STOP=>stop  
+ <STEP=>incr>
```

or

```
.AC DATA = datanm <SWEEP var START="param_expr1"  
+ STOP="param_expr2" STEP="param_expr3">
```

or

```
.AC DATA = datanm <SWEEP var start_expr stop_expr  
+ step_expr>
```

Optimization

You can use the following syntax in Star-Hspice:

```
.AC DATA = datanm OPTIMIZE = opt_par_fun
+ RESULTS = measnames MODEL = optmod
```

Random/Monte Carlo

You can use the following syntax in Star-Hspice:

```
.AC type np fstart fstop <SWEEP MONTE = val>
```

The .AC statement keywords and parameters are:

<i>DATA = datanm</i>	Data name, referenced in the .AC statement
<i>incr</i>	Increment value of the voltage, current, element, or model parameter. If you use <i>type</i> variation, specify the <i>np</i> (number of points) instead of <i>incr</i> .
<i>fstart</i>	Starting frequency. If you use POI (list of points) type variation, use a list of frequency values, not <i>fstart fstop</i> .
<i>fstop</i>	Final frequency.
<i>MONTE = val</i>	Produces a number (<i>val</i>) of randomly-generated values. Star-Hspice uses these values to select parameters from a distribution, either <i>Gaussian</i> , <i>Uniform</i> , or <i>Random Limit</i> (see Monte Carlo Analysis on page 13-13).
<i>np</i>	Number of points, or points per decade or octave, depending on which keyword precedes it.
<i>start</i>	Starting voltage or current, or any parameter value for an element or a model.
<i>stop</i>	Final voltage or current, or any parameter value for an element or a model.
<i>SWEEP</i>	This keyword indicates that the .AC statement specifies a second sweep.

<i>TEMP</i>	This keyword indicates a temperature sweep
<i>type</i>	Can be any of the following keywords: ■ DEC – decade variation. ■ OCT – octave variation. ■ LIN – linear variation. ■ POI – list of points.
<i>var</i>	An independent voltage or current source, element or model parameter, or the TEMP (temperature sweep) keyword. Star-Hspice supports source value sweep, referring to the source name (<i>SPICE</i> style). If you select a parameter sweep, a .DATA statement, and a temperature sweep, then you must choose a parameter name for the source value. You must also later refer to it in the .AC statement. The parameter name cannot start with V or I.

Examples

The following example performs a frequency sweep, by 10 points per decade, from 1 kHz to 100 MHz.

```
.AC DEC 10 1K 100MEG
```

The next line calls for a 100-point frequency sweep, from 1 Hz to 100 Hz.

```
.AC LIN 100 1 100HZ
```

The following example performs an AC analysis, for each value of cload. This results from a linear sweep of cload between 1 pF and 10 pF (20 points), sweeping the frequency by 10 points per decade, from 1 Hz to 10 kHz.

```
.AC DEC 10 1 10K SWEEP cload LIN 20 1pf 10pf
```

The following example performs an AC analysis, for each value of rx, 5 k and 15 k, sweeping the frequency by 10 points per decade, from 1 Hz to 10 kHz.

```
.AC DEC 10 1 10K SWEEP rx POI 2 5k 15k
```

The next example uses the `.DATA` statement to perform a series of AC analyses, modifying more than one parameter. The `datanm` file contains the parameters.

```
.AC DEC 10 1 10K SWEEP DATA = datanm
```

The following example illustrates a frequency sweep, and a Monte Carlo analysis, with 30 trials.

```
.AC DEC 10 1 10K SWEEP MONTE = 30
```

If the input file includes an `.AC` statement, Star-Hspice runs AC analysis for the circuit, over a selected frequency range, for each parameter in the second sweep.

For an AC analysis, the data file must include at least one independent AC source element statement (for example, `VI INPUT GND AC 1V`). Star-Hspice checks for this condition, and reports a fatal error if you did not specify such AC sources (see [Using Sources and Stimuli on page 5-1](#)).

AC Control Options

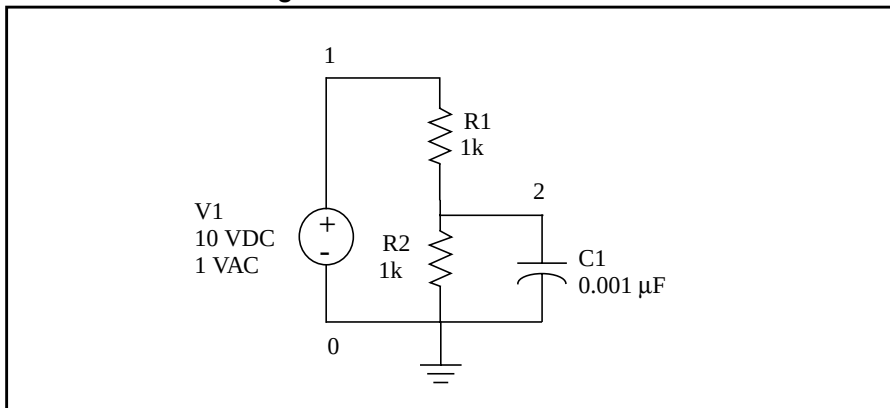
$ABSH = x$	Sets the absolute current change, through voltage-defined branches (voltage sources and inductors). Use ABSH with DI and RELH, to check for current convergence. Default = 0.0.
$ACOUT$	AC output calculation method, for the difference in values of magnitude, phase, and decibels. Use this option for prints and plots. Default = 1. ■ The default value, $ACOUT = 1$, selects the Star-Hspice method, which calculates the difference of the magnitudes of the values. ■ The SPICE method, $ACOUT = 0$, calculates the magnitude of the differences.
$DI = x$	Sets the maximum iteration-to-iteration current change, through voltage-defined branches (voltage sources and inductors). Use this option only if the value of the DI control option is greater than 0. Default = 0.0.
$MAXAMP = x$	Sets the maximum current, through voltage-defined branches (voltage sources and inductors). If the current exceeds the MAXAMP value, Star-Hspice issues an error message. Default = 0.0.
$RELH = x$	Sets the relative current tolerance, through voltage-defined branches (voltage sources and inductors). Use this option to check current convergence, but only if the value of the ABSH control option is greater than zero. Default = 0.05.
$UNWRAP$	Displays phase results for AC analysis, in <i>unwrapped</i> form (with a continuous phase plot). Star-Hspice uses these results to accurately calculate group delay. It also uses unwrapped phase results to compute group delay, even if you do not set the UNWRAP option.

AC Analysis of an RC Network

Figure 12-2 shows a simple RC network, with a DC and AC source applied. The circuit consists of:

- Two resistors, R1 and R2.
- Capacitor C1.
- Voltage source V1.
- Node 1 is the connection between the source positive terminal and R1.
- Node 2 is where R1, R2, and C1 are connected.
- Star-Hspice ground is always node 0.

Figure 12-2: RC Network Circuit



The input netlist for the RC network circuit is:

```
A SIMPLE AC RUN
.OPTION LIST NODE POST
.OP
.AC DEC 10 1K 1MEG
.PRINT AC V(1) V(2) I(R2) I(C1)
V1 1 0 10 AC 1
R1 1 2 1K
R2 2 0 1K
C1 2 0 .001U
.END
```

Follow the procedure below to perform an AC analysis for the RC network circuit.

1. Type the above netlist into a file named `quickAC.sp`.
2. To run a Star-Hspice analysis, type:
`hspice quickAC.sp > quickAC.lis`

When the run finishes, Star-Hspice displays:

```
>info:      ***** hspice job concluded
```

followed by a line that shows the amount of real time, user time, and system time needed for the analysis.

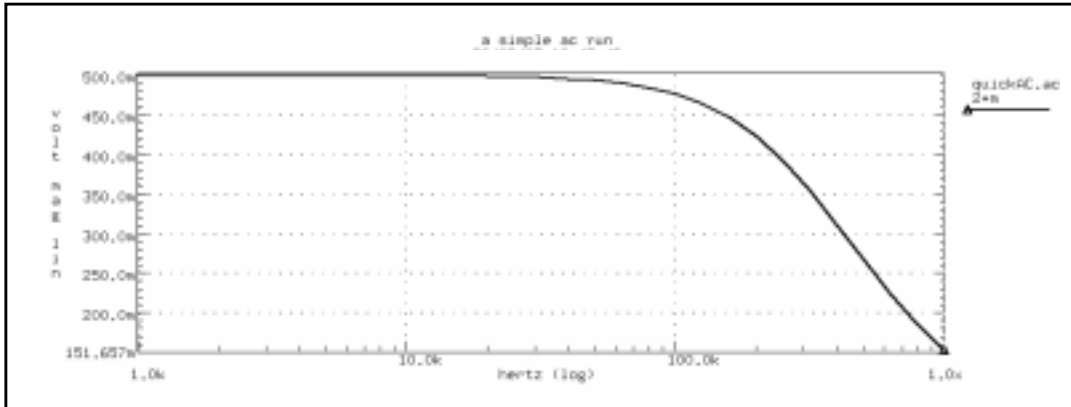
The following new files are present in your run directory:

- ☐ `quickAC.ac0`
- ☐ `quickAC.ic0`
- ☐ `quickAC.lis`
- ☐ `quickAC.st0`

3. Use an editor to view the `.lis` and `.st0` files, to examine the simulation results and status.
4. Run AvanWaves and open the `.sp` file.
5. To view the waveform, select the `quickAC.ac0` file from the **Results Browser** window.
6. Display the voltage at node 2, using a log scale on the x-axis.

Figure 12-3 shows the waveform that Star-Hspice produces if you sweep the response of node 2, as you vary the frequency of the input from 1 kHz to 1 MHz.

Figure 12-3: RC Network Node 2 Frequency Response



As you sweep the input from 1 kHz to 1 MHz, the `quickAC.lis` file displays:

- the input netlist
- details about the elements and topology
- operating point information
- the table of requested data

The `quickAC.ic0` file contains information about DC operating point conditions. The `quickAC.st0` file contains information about the simulation run status.

To use the operating point conditions for subsequent simulation runs, execute the `.LOAD` statement.

Other AC Analysis Statements

This section describes how to use other AC analysis statements.

.DISTO Statement — AC Small-Signal Distortion Analysis

The `.DISTO` statement computes the distortion characteristics of the circuit in an AC small-signal, sinusoidal, steady-state analysis.

The program computes and reports five distortion measures at the specified load resistor. The analysis assumes that the input uses one or two signal frequencies.

- Star-Hspice uses the first frequency (F1, the nominal analysis frequency) to calculate harmonic distortion. The `.AC` statement frequency-sweep sets it.
- Star-Hspice uses the optional second input frequency (F2) to calculate intermodulation distortion. To set it implicitly, specify the `skw2` parameter, which is the $F2/F1$ ratio.

<i>DIM2</i>	Intermodulation distortion, first difference. Relative magnitude and phase of the frequency component $(F1 - F2)$.
<i>DIM3</i>	Intermodulation distortion, second difference. The relative magnitude and phase of the frequency component $(2 \cdot F1 - F2)$.
<i>HD2</i>	Second-order harmonic distortion. Relative magnitude and phase of the frequency component $2 \cdot F1$ (ignores F2).
<i>HD3</i>	Third-order harmonic distortion. Relative magnitude and phase of the frequency component $3 \cdot F1$ (ignores F2).
<i>SIM2</i>	Intermodulation distortion, sum. Relative magnitude and phase of the frequency component $(F1 + F2)$.

The `.DISTO` summary report includes:

- A set of distortion measures, for each component in each element.
- A summary of distortion measures for each element.
- A summary of distortion measures for the entire circuit.

Syntax

`.DISTO Rload <inter <skw2 <refpwr <spwf>>>>`

where:

<i>Rload</i>	The resistor element name of the output load resistor, into which the output power feeds.
<i>inter</i>	Interval at which Star-Hspice prints a distortion-measure summary. Specifies a number of frequency points in the AC sweep (see the <i>np</i> parameter, in .AC Statement on page 12-4). If you omit <i>inter</i>, or set it to zero, Star-Hspice does not print a summary. To print or plot the distortion measures, use the .PRINT or .PLOT statement. If you set <i>inter</i> to 1 or higher, Star-Hspice prints a summary of the first frequency, and of each subsequent inter-frequency increment. To obtain a summary printout for only the first and last frequencies, set <i>inter</i> equal to the total number of increments needed, to reach <i>fstop</i> in the .AC statement. For a summary printout of only the first frequency, set <i>inter</i> to greater than the total number of increments required, to reach <i>fstop</i>.
<i>skw2</i>	Ratio of the second frequency (F2) to the nominal analysis frequency (F1), in the range $1e-3 < skw2 < 0.999$. If you omit <i>skw2</i> , the default value is 0.9.
<i>refpwr</i>	Reference power level, used to compute the distortion products. If you omit <i>refpwr</i> , the default value is 1mW, measured in decibels magnitude (dbM). The value must be $\geq 1e-10$.
<i>spwf</i>	Amplitude of the second frequency (F2). The value must be $\geq 1e-3$. Default = 1.0.

Example

```
.DISTO RL 2 0.95 1.0E-3 0.75
```

Star-Hspice performs only one distortion analysis per simulation. If your design contains more than one .DISTO statement, Star-Hspice runs only the last statement. The .DISTO statement calculates distortions for diodes, BJTs (levels 1, 2, 3, and 4), and MOSFETs (Level49 and Level53, Version 3.22).

Note: Star-Hspice prints an extensive summary from the distortion analysis, for each frequency listed. Use the *inter* parameter, in the .DISTO statement, to limit the amount of output generated.

.NOISE Statement — AC Noise Analysis

Syntax

```
.NOISE ovv srcnam inter
```

where:

<i>ovv</i>	Nodal voltage output variable. Defines the node at which Star-Hspice sums the noise.
<i>srcnam</i>	Name of the independent voltage or current source, to use as the noise input reference
<i>inter</i>	Interval at which Star-Hspice prints a noise analysis summary. <i>inter</i> specifies how many frequency points to summarize in the AC sweep. If you omit <i>inter</i> , or set it to zero, Star-Hspice does not print a summary. If <i>inter</i> is equal to or greater than one, Star-Hspice prints summary for the first frequency, and once for each subsequent increment of the <i>inter</i> frequency. The noise report is sorted according to the contribution of each node to the overall noise level.

Example

```
.NOISE V(5) VIN 10
```

Use the .NOISE and .AC statements, to control the noise analysis of the circuit.

Noise Calculations

Noise calculations in Star-Hspice are based on complex AC nodal voltages, which in turn are based on the DC operating point. For descriptions of noise models for each device type, see the *True-Hspice Device Models Reference Manual*. Each noise source does not statistically correlate to other noise sources in the circuit; the Star-Hspice simulator calculates each noise source independently. The total output noise voltage is the RMS sum of the individual noise contributions:

$$onoise = \sum_{n=1}^n |Z_n \cdot I_n|^2$$

<i>onoise</i>	Total output noise.
<i>I</i>	Equivalent current due to thermal, shot, or flicker noise.
<i>Z</i>	Equivalent transimpedance, between noise source and output.
<i>n</i>	Number of noise sources, associated with all resistors, MOSFETs, diodes, JFETs, and BJTs.

The input noise (*inoise*) voltage is the total output noise, divided by the gain or transfer function of the circuit. Star-Hspice prints the contribution of each noise generator in the circuit, for each *inter* frequency point. The simulator also normalizes the output and input noise levels, relative to the square root of the noise bandwidth. The units are volts/Hz^{1/2} or amps/Hz^{1/2}.

To simulate flicker noise sources in the noise analysis, include values for the KF and AF parameters, on the appropriate device model statements. Use the .PRINT or .PLOT statement, to print or plot output noise, and the equivalent input noise.

If you specify more than one .NOISE statement in a single simulation, Star-Hspice runs only the last statement.

.SAMPLE Statement — Noise Folding Analysis

To acquire data from analog signals, use the .SAMPLE statement, with the .NOISE and .AC statements, to analyze data sampling noise in Star-Hspice. The SAMPLE analysis performs a noise-folding analysis, at the output node.

Syntax

```
.SAMPLE FS = freq <TOL = val> <NUMF = val>
+ <MAXFLD = val> <BETA = val>
```

Parameters and Keywords

- FS* = *freq* Sample frequency, in Hertz.
- TOL* Sampling-error tolerance: the ratio of the noise power (in the highest folding interval) to the noise power (in baseband). Default = $1.0\text{e-}3$.
- NUMF* Maximum number of frequencies that you can specify. The algorithm requires about ten times this number of internally-generated frequencies, so keep this value small. Default = 100.
- MAXFLD* Maximum number of folding intervals (default = 10.0). The highest frequency (in Hertz) that you can specify is:
- $$F_{\text{MAX}} = \text{MAXFLD} \cdot FS$$
- BETA* Optional noise integrator (duty cycle), at the sampling node:
- BETA = 0 no integrator
 - BETA = 1 simple integrator (default)
- If you clock the integrator (integrates during a fraction of the $1/FS$ sampling interval), then set BETA to the duty cycle of the integrator.

.NET Statement - AC Network Analysis

You can use the .NET statement in Star-Hspice to compute parameters for:

- Z impedance matrix.
- Y admittance matrix.
- H hybrid matrix
- S scattering matrix.

Star-Hspice also computes:

- Input impedance.
- Output impedance.
- Admittance.

This analysis is a part of the AC small-signal analysis. Therefore, to run network analysis, you must specify the frequency sweep for the AC statement.

Syntax

One-Port Network

```
.NET input <RIN = val>
.NET input <val>
```

Two-Port Network

```
.NET output input <ROUT = val> <RIN = val>
```

<i>input</i>	Name of the voltage or current source for AC input.
<i>output</i>	Output port. It can be: ■ An output voltage, $V(n1, n2)$. ■ An output current, $I(source)$, or $I(element)$.
<i>RIN</i>	Keyword, for input or source resistance. The RIN value calculates output impedance, output admittance, and scattering parameters. The default RIN value is 1 ohm.
<i>ROUT</i>	Keyword, for output or load resistance. The ROUT value calculates input impedance, admittance, and scattering parameters. The default ROUT value is 1 ohm.

Example

One-Port Network

```
.NET          VINAC      RIN = 50
.NET          IIN        RIN = 50
```

Two-Port Network

```
.NET V(10,30)      VINAC      ROUT = 75      RIN = 50
.NET I(RX)          VINAC      ROUT = 75      RIN = 50
```

AC Network Analysis - Output Specification

Syntax

$X_{ij}(z)$, $ZIN(z)$, $ZOUT(z)$, $YIN(z)$, $YOUT(z)$

- X In Star-Hspice, can be Z (impedance), Y (admittance), H (hybrid), or S (scattering).
- ij i and j identify the matrix parameter to print, in Star-Hspice. Value can be 1 or 2. Use with the X value above (for example, S_{ij} , Z_{ij} , Y_{ij} , or H_{ij}).
- z Output type (Star-Hspice):
- R: real part.
 - I: imaginary part.
 - M: magnitude.
 - P: phase.
 - DB: decibel.
 - T: group time delay.
- ZIN Input impedance. For the one-port network, ZIN , $Z11$, and $H11$ are the same.
- $ZOUT$ Output impedance.
- YIN Input admittance. For a one-port network, YIN is the same as $Y11$.
- $YOUT$ Output admittance.

If you omit z, output includes the magnitude of the output variable. The output of AC Analysis includes voltages and currents.

Example

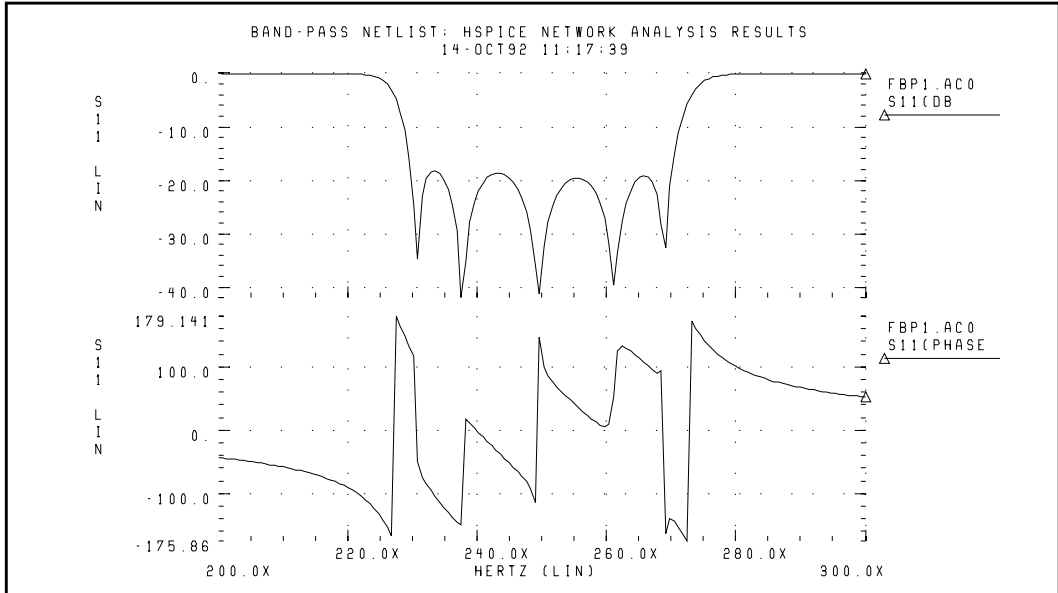
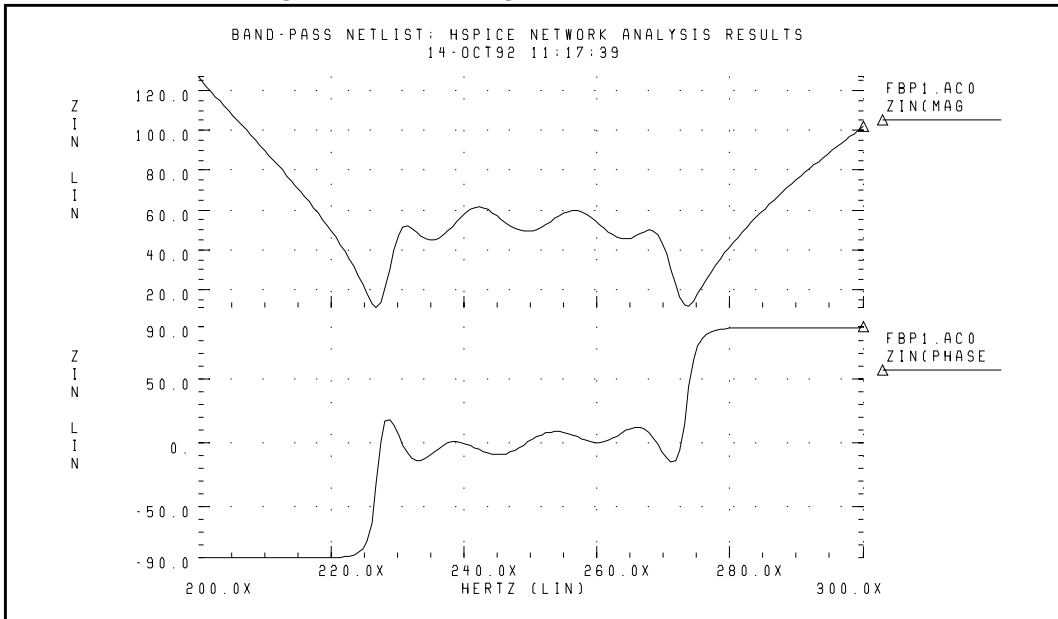
```
.PRINT AC Z11(R) Z12(R) Y21(I) Y22 S11 S11(DB) Z11(T)
.PRINT AC ZIN(R) ZIN(I) YOUT(M) YOUT(P) H11(M) H11(T)
.PLOT AC S22(M) S22(P) S21(R) H21(P) H12(R) S22(T)
```

Bandpass Netlist:¹ Network Analysis Results

```
*FILE: FBP_1.SP
.OPTION DCSTEP = 1 POST
*BAND PASS FILTER

C1      IN      2      3.166PF
L1      2      3      203NH
C2      3      0      3.76PF
C3      3      4      1.75PF
C4      4      0      9.1PF
L2      4      0      36.81NH
C5      4      5      1.07PF
C6      5      0      3.13PF
L3      5      6      233.17NH
C7      6      7      5.92PF
C8      7      0      4.51PF
C9      7      8      1.568PF
C10     8      0      8.866PF
L4      8      0      35.71NH
C11     8      9      2.06PF
C12     9      0      4.3PF
L5      9      10     200.97NH
C13     10     OUT     2.97PF
RX      OUT     0      1E14
VIN     IN      0      AC 1

.AC LIN 41 200MEG 300MEG
.NET V(OUT) VIN ROUT = 50 RIN = 50
.PLOT AC S11(DB) (-50,10) S11(P) (-180,180)
.PLOT AC ZIN(M) (5,130) ZIN(P) (-90,90)
.END
```


Figure 12-4: S11 Magnitude and Phase Plots**Figure 12-5: ZIN Magnitude and Phase Plots**

NETWORK Variable Specification

Star-Hspice uses the results of AC analysis, to perform network analysis. The .NET statement defines the Z, Y, H, and S parameters to calculate.

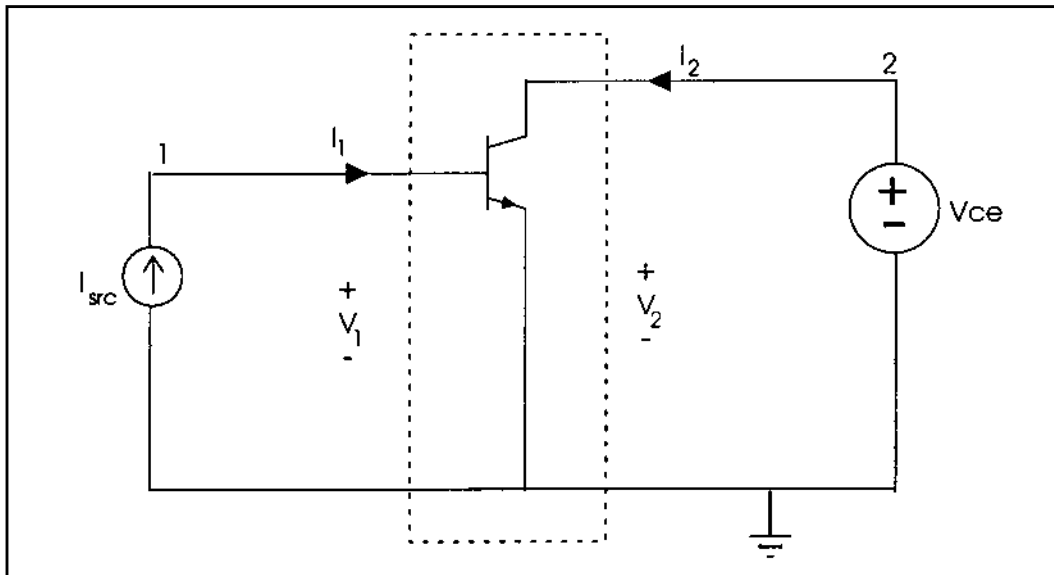
The following list shows various combinations of the .NET statement, for network matrices that Star-Hspice calculates:

1. .NET Vout Isrc V = [Z] [I]
2. .NET Iout Vsrc I = [Y] [V]
3. .NET Iout Isrc [V1 I2]^T = [H] [I1 V2]^T
4. .NET Vout Vsrc [I1 V2]^T = [S] [V1 I2]^T

([M]^T represents the *transpose* of the M matrix)

Note: The preceding list does not mean that you must use combination (1) to calculate the Z parameters. However, if you specify .NET Vout Isrc, Star-Hspice initially evaluates the Z matrix parameters. It then uses standard conversion equations, to determine the S parameters, or any other requested parameters.

The example in [Figure 12-6](#) shows the importance of the variables in the .NET statement. Here, *Isrc* and *Vce* are the DC biases, applied to the BJT.

Figure 12-6: Parameters with .NET V(2) Isrc

This .NET statement provides an incorrect result for the Z parameter calculation:

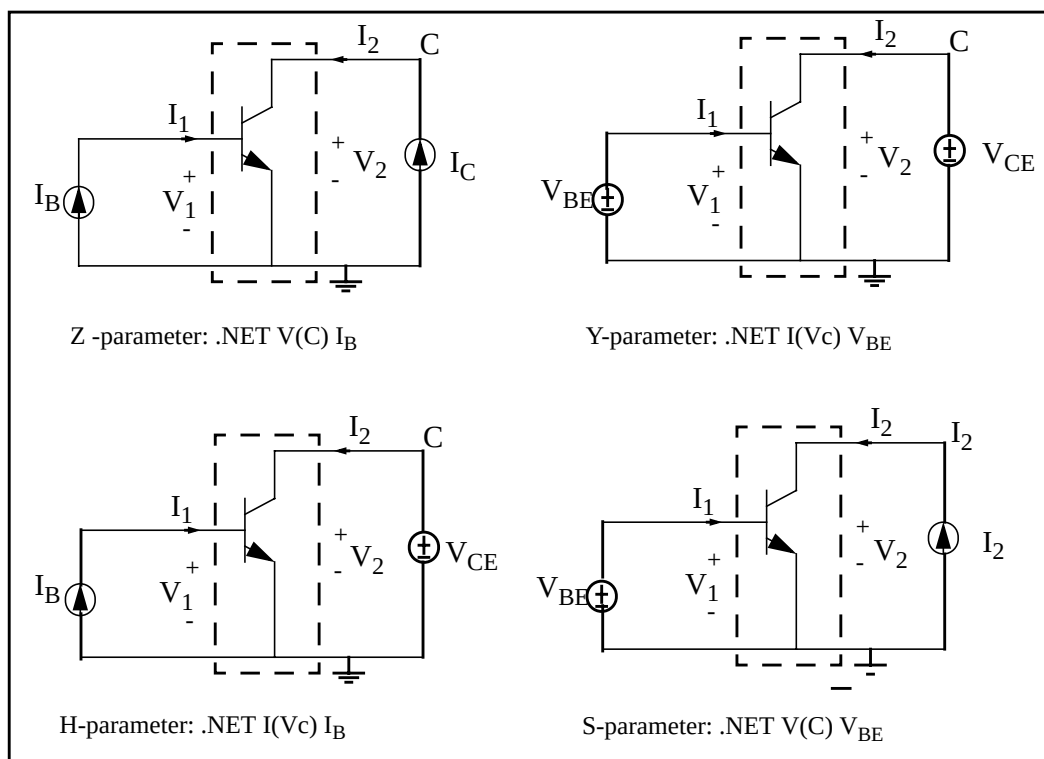
```
.NET V(2) Isrc
```

When Star-Hspice runs AC analysis, it shorts all DC voltage sources; all DC current sources are open-circuited. As a result, $v(2)$ shorts to ground, and its value is zero in AC analysis. This affects the results of the network analysis.

In this example, Star-Hspice attempts to calculate the Z parameters (Z_{11} and Z_{21}), defined as $Z_{11} = V_1/I_1$ and $Z_{21} = V_2/I_1$ with $I_2=0$. The above example does not satisfy the requirement that I_2 must be zero. Instead, V_2 is zero, which results in incorrect values for Z_{11} and Z_{21} .

Figure 12-7 shows the correct biasing configurations, for performing network analysis for the Z, Y, H, and S parameters.

Figure 12-7: Network Parameter Configurations



For example, to calculate the H parameters, Star-Hspice uses the .NET statement.

`.NET I(Vc) I_B`

V_C denotes the voltage at the C node, which is the collector of the BJT. With this statement, Star-Hspice uses the following equations to calculate H parameters, immediately after AC analysis:

$$V_1 = H_{11} \cdot I_1 + H_{12} \cdot V_2$$

$$I_2 = H_{21} \cdot I_1 + H_{22} \cdot V_2$$

To calculate Hybrid parameters (H_{11} and H_{21}), the DC voltage source (V_{CE}) sets V_2 to zero, and the DC current source (I_B) sets I_1 to zero. Setting I_1 and V_2 to zero, precisely meets the conditions of the circuit under examination: the input current source is open-circuited, and the output voltage source shorts to ground.

A data file, containing measured results, can drive external DC biases, applied to a BJT. In some cases, not all DC currents and voltages (at input and output ports) are available. When you analyze a network analysis, examine the circuit, and select suitable input and output variables. This helps you to obtain correctly-calculated results. The following examples demonstrate network analysis of a BJT, using Star-Hspice.

Network Analysis Example: Bipolar Transistor

```
BJT network analysis
.option nopage list
+ newtol reli = 1e-5 absi = 1e-10 relv = 1e-5
+ relvdc = 1e-7 nomod post gmindc = 1e-12
.op
.param vbe = 0 ib = 0 ic = 0 vce = 0
$ H-parameter
.NET i(vc) ibb rin = 50 rout = 50
ve      e      0      0
ibb     0      b      dc = 'ib' ac = 0.1
vc      c      0      'vce'
q1      c      b      e 0      bjt
```

```
.model bjt npn subs = 1
+ bf = 1.292755e+02 br = 8.379600e+00
+ is = 8.753000e-18 nf = 9.710631e-01
+ nr = 9.643484e-01 ise = 3.428000e-16
+ isc = 1.855000e-17 iss = 0.000000e+00
+ ne = 2.000000e+00 nc = 9.460594e-01
+ ns = 1.000000e+00 vaf = 4.942130e+01
+ var = 4.589800e+00 ikf = 5.763400e-03
+ ikr = 5.000000e-03 irb = 8.002451e-07
+ rc = 1.216835e+02 rb = 1.786930e+04
+ rbm = 8.123460e+01 re = 2.136400e+00
+ cje = 9.894950e-14 mje = 4.567345e-01
+ vje = 1.090217e+00 cjc = 5.248670e-14
+ mjc = 1.318637e-01 vjc = 5.184017e-01
+ xcjc = 6.720303e-01 cjs = 9.671580e-14
+ mjs = 2.395731e-01 vjs = 5.000000e-01
+ tf = 3.319200e-11 itf = 1.457110e-02
+ xtf = 2.778660e+01 vtf = 1.157900e+00
+ ptf = 6.000000e-05 xti = 4.460500e+00
+ xtb = 1.456600e+00 eg = 1.153300e+00
+ tikf1 = -5.397800e-03 tirb1 = -1.071400e-03
+ tre1 = -1.121900e-02 trb1 = 3.039900e-03
+ trc1 = -4.020700e-03 trm1 = 0.000000e+00

.print ac par('ib') par('ic')
+ h11(m) h12(m) h21(m) h22(m)
+ z11(m) z12(m) z21(m) z22(m)
+ s11(m) s21(m) s12(m) s22(m)
+ y11(m) y21(m) y12(m) y22(m)

.ac Dec 10 1e6 5g sweep data = bias
.data bias
```

vbe	vce	ib	ic
771.5648m	292.5047m	1.2330u	126.9400u
797.2571m	323.9037m	2.6525u	265.0100u
821.3907m	848.7848m	5.0275u	486.9900u
843.5569m	1.6596	8.4783u	789.9700u
864.2217m	2.4031	13.0750u	1.1616m
884.3707m	2.0850	19.0950u	1.5675m

```
.enddata
.end
```

Other possible biasing configurations, for the network analysis, are:

\$S-parameter

```
.NET v(c) vbb rin = 50 rout = 50
ve      e      0      0
vbb     b      0      dc = 'vbe' ac = 0.1
icc     0      c      'ic'
q1      c      b      e 0      bjt
```

\$Z-parameter

```
.NET v(c) ibb rin = 50 rout = 50
ve      e      0      0
ibb     0      b      dc = 'ib' ac = 0.1
icc     0      c      'ic'
q1      c      b      e 0      bjt
```

\$Y-parameter

```
.NET i(vc) vbb rin = 50 rout = 50
ve      e      0      0
vbb     b      0      'vbe' ac = 0.1
vc      c      0      'vce'
q1      c      b      e 0      bjt
```

References

1. Goyal, Ravender. "S-Parameter Output From SPICE Program", *MSN & CT*, February 1988, pp. 63 and 66.



Chapter 13

Statistical Analysis and Optimization

When you design an electrical circuit, it must meet tolerances for the specific manufacturing process. The *electrical yield* is the number of parts that meet the electrical test specifications. Overall process efficiency requires maximum yield. To analyze and optimize the yield, Star-Hspice uses statistical techniques, and observes the effects of variations in element and model parameters.

- [Analytical Model Types](#)
- [Simulating Circuit and Model Temperatures](#)
- [Worst Case Analysis](#)
- [Monte Carlo Analysis](#)
- [Worst Case and Monte Carlo Sweep Example](#)
- [Optimization](#)
- [Optimization Examples](#)

Analytical Model Types

To model parametric and statistical variation in circuit behavior, use:

- The .PARAM statement, which investigates the performance of a circuit, as you change circuit parameters. See [Simulation Input and Controls on page 3-1](#), for details about the .PARAM statement.
- Temperature Variation Analysis, which varies the circuit and component temperatures, and compares the circuit responses. You can study the temperature-dependent effects of the circuit, in detail.
- Monte Carlo Analysis. If you know the statistical standard deviations of component values, use this analysis to center a design. This provides maximum process yield, and determines component tolerances.
- Worst Case Corners Analysis. If you know the component value limit, use this analysis to automate quality assurance, for:
 - Basic circuit function.
 - Process extremes.
 - Quick estimation of speed and power trade-offs.
 - Best case and worst case model selection.
 - Parameter corners.
 - Library files.
- Data-Driven Analysis. Use for cell characterization, response surface, or Taguchi analysis. See [Characterizing Cells on page 15-1](#). Automates characterization of cells, and calculates the coefficient of polynomial delay, for timing simulation. You can simultaneously vary any number of parameters, and perform an unlimited number of analyses. This analysis uses ASCII file format, so Star-Hspice can automatically generate parameter values. This analysis can replace hundreds or thousands of Star-Hspice simulation runs.

Use yield analyses to modify:

- DC operating points.
- DC sweeps.
- AC sweeps.
- Transient analysis.

These analyses can generate scatter plots, for operating point analysis. They can also generate a family of curve plots for DC, AC, and transient analysis.

Use the `.MEASURE` statement, with yield analyses, to view distributions of delay times, power, or any other characteristic described in a `.MEASURE` statement. Often, this is more useful than viewing a family of curves, that a Monte Carlo analysis generates.

When you use the `.MEASURE` statement, Star-Hspice generates a table of results, in an `.mt#` file. You can read this file in ASCII format, and you can use *AvanWaves* to display it. Also, if you use `.MEASURE` statements in a Monte Carlo or data-driven analysis, then the Star-Hspice output file includes calculations for standard statistical descriptors:

$$\text{Mean} = \frac{x_1 + x_2 + \dots + x_n}{N}$$

$$\text{Variance} = \frac{(x_1 - \text{Mean})^2 + \dots + (x_n - \text{Mean})^2}{N - 1}$$

$$\text{Sigma} = \sqrt{\text{Variance}}$$

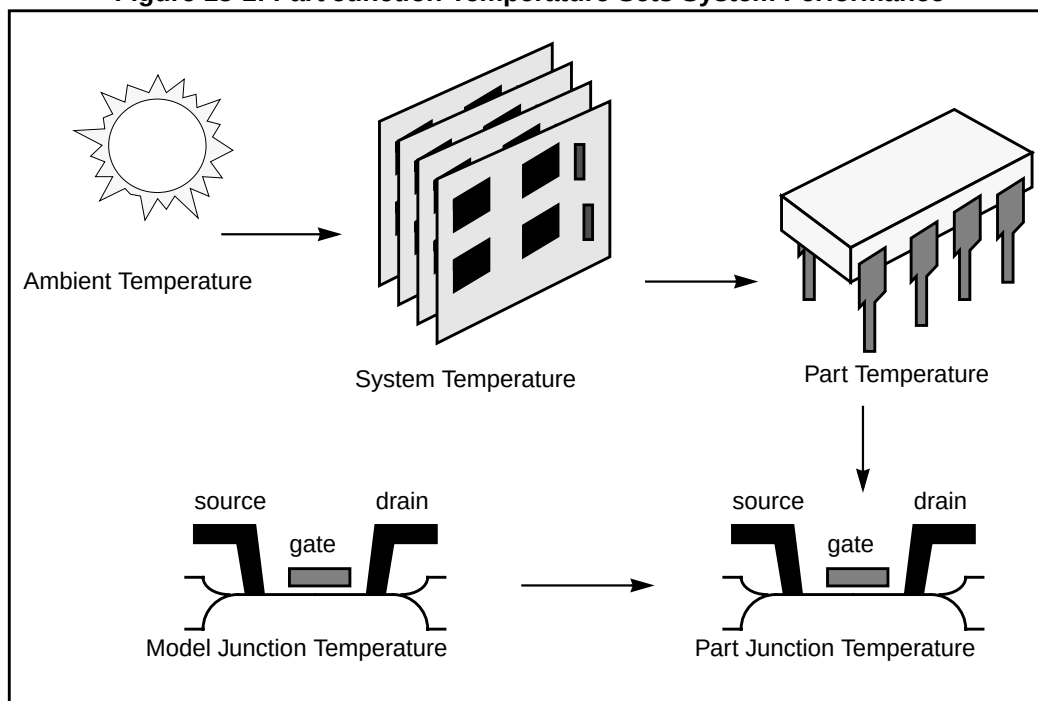
$$\text{Average Deviation} = \frac{|x_1 - \text{Mean}| + \dots + |x_n - \text{Mean}|}{N - 1}$$

Simulating Circuit and Model Temperatures

Temperature affects *all* electrical circuits. [Figure 13-1](#) shows the key temperature parameters, associated with circuit simulation:

- Model reference temperature – you can model different models at different temperatures. Each model has a TREF (temperature reference) parameter.
- Element junction temperature – each resistor, transistor, or other element generates heat, so an element is hotter than the ambient temperature.
- Part temperature – at the system level, each part has its own temperature.
- System temperature – a collection of parts form a system, which has a local temperature.
- Ambient temperature – the ambient temperature is the air temperature of the system.

Figure 13-1: Part Junction Temperature Sets System Performance



Star-Hspice calculates temperatures as differences from the ambient temperature:

$$T_{ambient} + \Delta_{system} + \Delta_{part} + \Delta_{junction} = T_{junction}$$

$$I_{ds} = f(T_{junction}, T_{model})$$

Every element includes a DTEMP keyword, which defines the difference between junction and ambient temperature. The following example uses DTEMP in a MOSFET element statement:

```
M1 drain gate source bulk Model_name W=10u
+ L=1u DTEMP=+20
```

Temperature Analysis

You can specify three temperatures:

- Model reference temperature, specified in a .MODEL statement. The temperature parameter is usually TREF, but can be TEMP or TNOM in some models. This parameter specifies the temperature, in °C, at which Star-Hspice measures and extracts the model parameters. Set the value of TNOM in a .OPTION statement. Its default value is 25 °C.
- Circuit temperature, which you specify using a .TEMP statement or the TEMP parameter. This is the temperature, in °C, at which Star-Hspice simulates all elements. To modify the temperature for a particular element, use the DTEMP parameter. The default circuit temperature is the value of TNOM.
- Individual element temperature, which is the circuit temperature, plus an optional amount that you specify in the DTEMP parameter.

To specify the temperature of a circuit in a simulation run, use either the .TEMP statement, or the TEMP parameter in the .DC, .AC, or .TRAN statements. Star-Hspice compares the circuit simulation temperature that one of these statements sets, against the reference temperature that the TNOM option sets. TNOM defaults to 25 °C, unless you use the SPICE option, which defaults to 27 °C. To calculate the derating of component values and model parameters, Star-Hspice uses the difference between the circuit simulation temperature, and the TNOM reference temperature.

Elements and models within a circuit can operate at different temperatures. For example, a high-speed input/output buffer, that switches at 50 MHz, is much hotter than a low-drive NAND gate, that switches at 1 MHz). To simulate this temperature difference, specify both an element temperature parameter (DTEMP), and a model reference parameter (TREF). If you specify DTEMP in an element statement, the element temperature for the simulation is:

$$\text{element temperature} = \text{circuit temperature} + \text{DTEMP}$$

Specify the DTEMP value in the element statement (resistor, capacitor, inductor, diode, BJT, JFET, or MOSFET statement). Assign a parameter to DTEMP, then use the .DC statement to sweep the parameter. The DTEMP value defaults to zero.

If you specify TREF in the model statement, the model reference temperature changes (TREF overrides TNOM). Derating the model parameters is based on the difference between circuit simulator temperature, and TREF (instead of TNOM).

.TEMP Statement

Syntax

```
.TEMP t1 <t2 <t3 ...>>
```

t1 t2 ... The temperatures, in °C, at which Star-Hspice simulates the circuit.

Example

```
.TEMP -55.0 25.0 125.0
```

The .TEMP statement sets the circuit temperatures for the entire circuit simulation. To simulate the circuit, using individual elements or model temperatures, Star-Hspice uses:

- Temperature, as set in the .TEMP statement.
- TNOM option setting (or the TREF model parameter).
- DTEMP element temperature.

```
.TEMP 100
D1 N1 N2 DMOD DTEMP=30
D2 NA NC DMOD
R1 NP NN 100 TC1=1 DTEMP=-30

.MODEL DMOD D IS=1E-15 VJ=0.6 CJA=1.2E-13 CJP=1.3E-14
+ TREF=60.0
```

In this example:

- The .TEMP statement sets the circuit simulation temperature to 100°C.
- You do not specify TNOM, so it defaults to 25°C.
- The temperature of the diode is 30°C *above* the circuit temperature, as set in the DTEMP parameter.

That is:

- $D1temp = 100^{\circ}C + 30^{\circ}C = 130^{\circ}C$.
- Star-Hspice simulates the D2 diode at 100°C.
- R1 simulates at 70°C.

Because the diode model statement specifies TREF at 60°C, Star-Hspice derates the specified model parameters by:

- 70°C (130°C - 60°C) for the D1 diode.
- 40°C (100°C - 60°C) for the D2 diode.
- 45°C (70°C - TNOM) for the R1 resistor.

Worst Case Analysis

You can use *Worst Case* analysis (.wcase statement) when you design and analyze MOS and BJT IC circuits in Star-Hspice. To simulate the worst case, Star-Hspice sets all variables to their 2-sigma or 3-sigma worst case values. Because several independent variables rarely attain their worst-case values simultaneously, this technique tends to be overly pessimistic, and can lead to over-designing the circuit. However, this analysis is useful as a fast check.

Model Skew Parameters

The Avant! True-Hspice Device Models include physically-measurable model parameters. The circuit simulator uses parameter variations, to predict how an actual circuit responds to extremes in the manufacturing process. Physically-measurable model parameters are called *skew* parameters, because they skew from a statistical mean, to obtain the predicted performance variations.

Examples of skew parameters are the difference between the drawn and physical dimension of metal, polysilicon, or active layers, on an integrated circuit.

Generally, you specify skew parameters independently of each other, so you can use combinations of skew parameters to represent worst cases. Typical skew parameters for CMOS technology include:

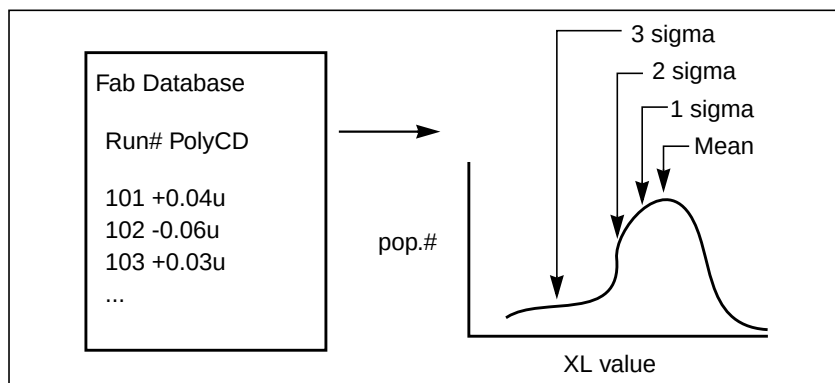
- XL – polysilicon CD (critical dimension of the poly layer, representing the difference between drawn and actual size).
- XW_n , XW_p – active CD (critical dimension of the active layer, representing the difference between drawn and actual size).
- TOX – thickness of the gate oxide.
- RSH_n , RSH_p – resistivity of the active layer.
- $DELVTOn$, $DELVTOp$ – variation in threshold voltage.

You can use these parameters in any level of MOS model, within the True-Hspice device models. The $DELVT0$ parameter shifts the threshold value. Star-Hspice adds this value to VTO for the Level 3 model, and adds or subtracts it from $VFB0$ for the BSIM model. [Table 13-1 on page 13-9](#) shows whether Star-Hspice adds or subtracts deviations from the average.

Table 13-1: Sigma Deviations

Type	Param	Slow	Fast
NMOS	XL	+	-
	RSH	+	-
	DELVTO	+	-
	TOX	+	-
	XW	-	+
PMOS	XL	+	-
	RSH	+	-
	DELVTO	-	+
	TOX	+	-
	XW	-	+

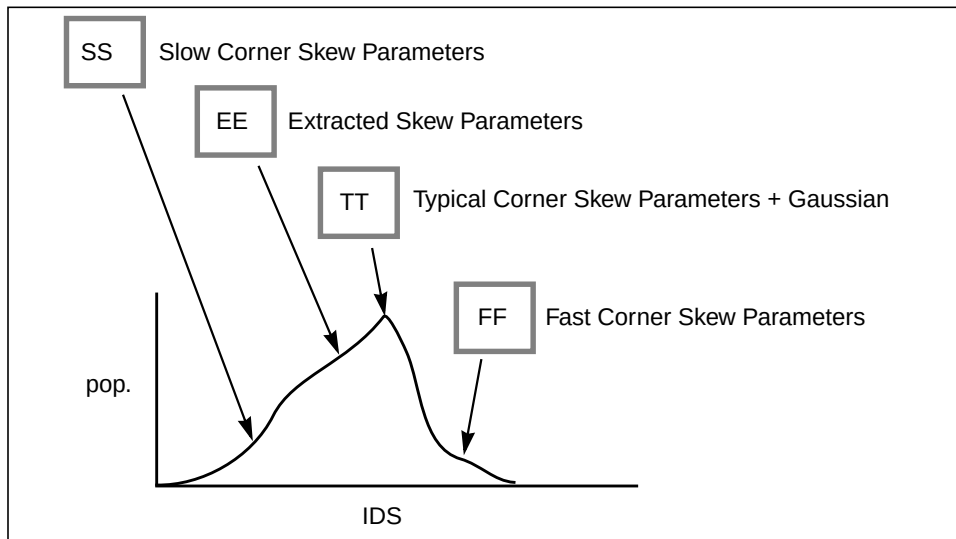
Star-Hspice selects skew parameters, based on the available historical data that it collects, either during fabrication or electrical test. For example, Star-Hspice collects the *XL skew* parameter, for poly CD, during fabrication. This parameter is usually the most important skew parameter for a MOS process. [Figure 13-2](#) is an example of data that historical records produce.

Figure 13-2: Historical Records for Skew Parameters in a MOS Process

Using Skew Parameters in Star-Hspice

Figure 13-3 shows how to create a worst-case, corners library file, for a CMOS process model in Star-Hspice. Specify the physically-measured parameter variations, so that their proper minimum and maximum values are consistent with measured current (IDS) variations. For example, Star-Hspice can generate a 3-sigma variation in IDS, from a 2-sigma variation in physically-measured parameters.

Figure 13-3: Worst Case Corners Library File for a CMOS Process Model



The `.LIB` (library) statement, and the `.INCLUDE` (include file) statement, access the models and skew. The library contains parameters that modify `.MODEL` statements. The following example of `.LIB`, using model skew parameters, features both worst-case and statistical-distribution data. In statistical distribution, the median value is the default for all non-Monte Carlo analysis.

Example

```
.LIB TT
$TYPICAL P-CHANNEL AND N-CHANNEL CMOS LIBRARY DATE:3/4/91
$ PROCESS: 1.0U CMOS, FAB22, STATISTICS COLLECTED 3/90-2/91
$ following distributions are 3 sigma ABSOLUTE GAUSSIAN

.PARAM
$ polysilicon Critical Dimensions
+ polycd=agauss(0,0.06u,1) xl='polycd-sigma*0.06u'
$ Active layer Critical Dimensions
+ nactcd=agauss(0,0.3u,1) xwn='nactcd+sigma*0.3u'
+ pactcd=agauss(0,0.3u,1) xwp='pactcd+sigma*0.3u'
$ Gate Oxide Critical Dimensions (200 angstrom +/- 10a at 1
$ sigma)
+ toxcd=agauss(200,10,1) tox='toxcd-sigma*10'

$ Threshold voltage variation
+ vtoncd=agauss(0,0.05v,1) delvton='vtoncd-sigma*0.05'
+ vtopcd=agauss(0,0.05v,1) delvtop='vtopcd+sigma*0.05'

.INC '/usr/meta/lib/cmos1_mod.dat'      $ model include file
.ENDL TT
.LIB FF
$HIGH GAIN P-CH AND N-CH CMOS LIBRARY 3SIGMA VALUES

.PARAM TOX=230 XL=-0.18u DELVTON=-.15V DELVTOP= 0.15V
.INC '/usr/meta/lib/cmos1_mod.dat'      $ model include file
.ENDL FF
```

The /usr/meta/lib/cmos1_mod.dat include file contains the model.

```
.MODEL NCH NMOS LEVEL=2 XL=XL TOX=TOX DELVTO=DELVTON . .
.MODEL PCH PMOS LEVEL=2 XL=XL TOX=TOX DELVTO=DELVTOP . .
```

Note: The model keyname (left side) equates to the skew parameter (right side). Model keynames and skew parameters can use the same names.

Skew File Interface to Device Models

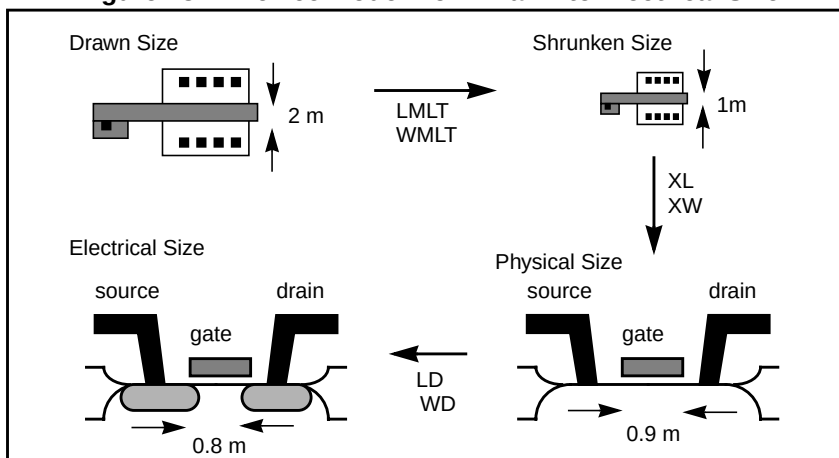
Skew parameters are model parameters, for transistor models or passive components. A typical device model set includes:

- MOSFET models, for all device sizes, using an automatic model selector.
- RC wire models, for polysilicon, metal1, and metal2 layers, in the drawn dimension. Models include temperature coefficients and fringe capacitance.
- Single-diode, and distributed-diode models, for N+, P+, and well (includes temperature, leakage, and capacitance, based on the drawn dimension).
- BJT models, for parasitic bipolar transistors. You can also use these for any special BJTs, such as a BiCMOS for ECL BJT process (includes current and capacitance as a function of temperature).
- Metal1 and metal2 transmission line models, for long metal lines.
- Models must accept elements. Sizes are based on a drawn dimension. If you draw a cell at 2 μ dimension, and shrink it to 1 μ , the physical size is 0.9 μ . The effective electrical size is 0.8 μ . Account for the four dimension levels:

drawn size
shrunk size
physical size
electrical size

Most simulator models scale directly from *drawn* to *electrical* size. True-Hspice MOS models support all four size levels, as explained in [Figure 13-4](#).

Figure 13-4: Device Model from Drawn to Electrical Size



Monte Carlo Analysis

Monte Carlo analysis uses a random number generator, to create the following types of functions.

Functions

Gaussian Parameter Distribution

- Relative variation—variation is a ratio of the average.
- Absolute variation—adds variation to the average.
- Bimodal—multiplies distribution, to statistically reduce nominal parameters.

Uniform Parameter Distribution

- Relative variation—variation is a ratio of the average.
- Absolute variation—adds variation to the average.
- Bimodal—multiplies distribution, to statistically reduce nominal parameters.

Random Limit Parameter Distribution

- Absolute variation—adds variation to the average.
- Monte Carlo analysis randomly selects the *min* or *max* variation.

The value of the MONTE analysis keyword determines how many times to perform operating point, DC sweep, AC sweep, or transient analysis.

Monte Carlo Setup

To set up a Monte Carlo analysis, use the following Star-Hspice statements:

- .PARAM statement—sets a model or element parameter, to a Gaussian, Uniform, or Limit function distribution.
- .DC, .AC, or .TRAN analysis—enables MONTE.
- .MEASURE statement—calculates the output mean, variance, sigma, and standard deviation.

Analysis Syntax

Select the type of analysis to run, such as operating point, DC sweep, AC sweep, or TRAN sweep.

Operating Point

```
.DC MONTE=val
```

DC Sweep

```
.DC vin 1 5 .25 SWEEP MONTE=val
```

AC Sweep

```
.AC dec 10 100 10meg SWEEP MONTE=val
```

TRAN Sweep

```
.TRAN 1n 10n SWEEP MONTE=val
```

The *val* value specifies the number of Monte Carlo iterations to perform. A reasonable number is 30. The statistical significance of 30 iterations is quite high. If the circuit operates correctly for all 30 iterations, there is a 99% probability that over 80% of all possible component values operate correctly. The relative error of a quantity, determined through Monte Carlo analysis, is proportional to $val^{-1/2}$.

Monte Carlo Output

- .MEASURE statements are the most convenient way to summarize the results.
- .PRINT statements generate tabular results, and print the values of all Monte Carlo parameters.

If one iteration is out of specification, you can obtain the component values from the tabular listing. A detailed resimulation of that iteration might help identify the problem.

- .GRAPH generates a high-resolution plot for each iteration.

In contrast, AvanWaves superimposes all iterations as a single plot, so you can analyze each iteration individually.

.PARAM Distribution Function Syntax

You can assign a .PARAM parameter to the keywords of elements and models, and assign a distribution function to each .PARAM parameter. Star-Hspice recalculates the distribution function each time that an element or model keyword uses a parameter. When you use this feature, Monte Carlo analysis can use a parameterized schematic netlist, without additional modifications.

The syntax is:

```
.PARAM xx=UNIF(nominal_val, rel_variation
+ <, multiplier>)
```

or

```
.PARAM xx=AUNIF(nominal_val, abs_variation <,
+ multiplier>)
```

or

```
.PARAM xx=GAUSS(nominal_val, rel_variation, sigma <,
+ multiplier>)
```

or

```
.PARAM xx=AGAUSS(nominal_val, abs_variation, sigma <,
+ multiplier>)
```

or

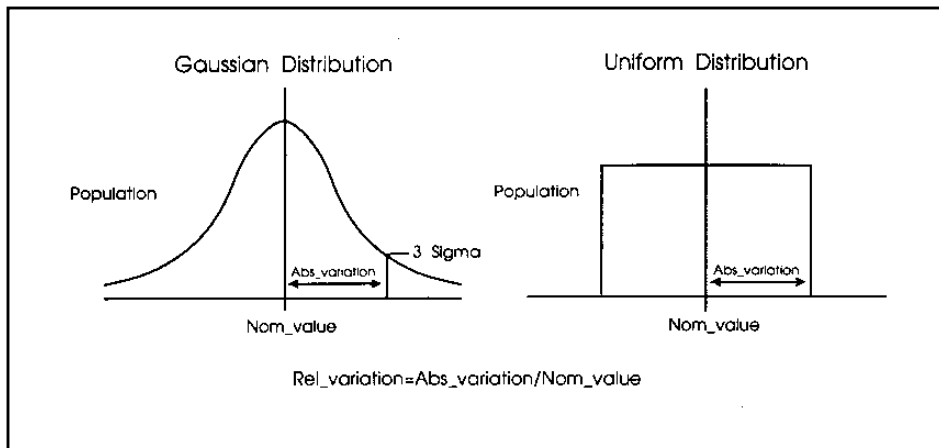
```
.PARAM xx=LIMIT(nominal_val, abs_variation)
```

where:

<i>xx</i>	Distribution function calculates the value of this parameter.
<i>UNIF</i>	Uniform distribution function, using relative variation.
<i>AUNIF</i>	Uniform distribution function, using absolute variation.
<i>GAUSS</i>	Gaussian distribution function, using relative variation.
<i>AGAUSS</i>	Gaussian distribution function, using absolute variation

<i>LIMIT</i>	Random-limit distribution function, using absolute variation. Adds +/- <i>abs_variation</i> to <i>nominal_val</i> , based on whether the random outcome of a -1 to 1 distribution is greater than or less than 0.
<i>nominal_val</i>	Nominal value for Monte Carlo analysis, and default value for all other analyses.
<i>abs_variation</i>	AUNIF and AGAUSS vary the <i>nominal_val</i> , by +/- <i>abs_variation</i> .
<i>rel_variation</i>	UNIF and GAUSS vary the <i>nominal_val</i> , by +/- (<i>nominal_val</i> · <i>rel_variation</i>).
<i>sigma</i>	Specifies <i>abs_variation</i> or <i>rel_variation</i> at the <i>sigma</i> level. For example, if <i>sigma</i> =3, then the standard deviation is <i>abs_variation</i> divided by 3.
<i>multiplier</i>	If you do not specify a multiplier, the default is 1. Star-Hspice repeats the calculation many times, and saves the largest deviation. The resulting parameter value might be greater than or less than <i>nominal_val</i> . The resulting distribution is bimodal.

Figure 13-5: Monte Carlo Distribution



Monte Carlo Parameter Distribution

Each time you use a parameter, Monte Carlo calculates a new random variable.

- If you do not specify a Monte Carlo distribution, then Star-Hspice assumes the nominal value.
- If you specify a Monte Carlo distribution for only one analysis, Star-Hspice uses the nominal value for all other analyses.

You can assign a Monte Carlo distribution to all elements that share a common model. The actual element value varies, according to the element distribution. If you assign a Monte Carlo distribution to a model keyword, then all elements that share the model, use the same keyword value. You can use this feature to create double element and model distributions.

For example, the MOSFET channel length varies from transistor to transistor, by a small amount that corresponds to the die distribution. The die distribution is responsible for offset voltages in operational amplifiers, and for the tendency of flip-flops to settle into random states. However, all transistors on a die site vary, according to the wafer or fabrication run distribution. This value is much larger than the die distribution, but affects all transistors the same way. You can specify the wafer distribution in the MOSFET model, to set the speed and power dissipation characteristics.

Monte Carlo Examples

Gaussian, Uniform, and Limit Functions

Test of monte carlo gaussian, uniform, and limit functions

```
.OPTION post
.dc monte=60
* setup plots
.model histo plot ymin=80 ymax=120 freq=1
.graph model=HISTO aunif_1=v(au1)
.graph model=HISTO aunif_10=v(au10)
.graph model=HISTO agauss_1=v(ag1)
.graph model=HISTO agauss_10=v(ag10)
.graph model=HISTO limit=v(L1)
```

```
* uniform distribution relative variation +/- .2
.param ru_1=unif(100,.2)
Iu1 u1 0 -1
ru1 u1 0 ru_1

* absolute uniform distribution absolute variation +/- 20
* single throw and 10 throw maximum
.param rau_1=aunif(100,20)
.param rau_10=aunif(100,20,10)
Iau1 au1 0 -1
rau1 au1 0 rau_1
Iau10 au10 0 -1
rau10 au10 0 rau_10

* gaussian distribution relative variation +/- .2
* at 3 sigma
.param rg_1=gauss(100,.2,3)
Ig1 g1 0 -1
rg1 g1 0 rg_1

* absolute gaussian distribution absolute variation +/- .2
* at 3 sigma
* single throw and 10 throw maximum
.param rag_1=agauss(100,20,3)
.param rag_10=agauss(100,20,3,10)
Iag1 ag1 0 -1
rag1 ag1 0 rag_1
Iag10 ag10 0 -1
rag10 ag10 0 rag_10

* random limit distribution absolute variation +/- 20
.param RL=limit(100,20)
IL1 L1 0 -1
rL1 L1 0 RL
.end
```

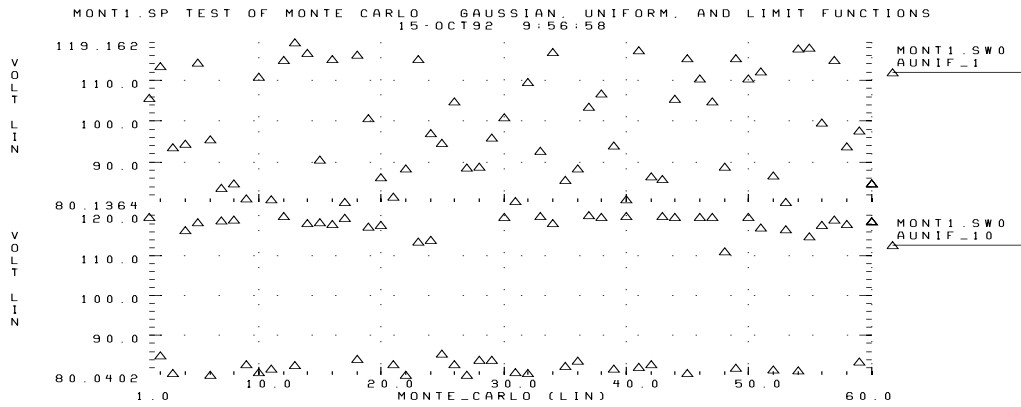
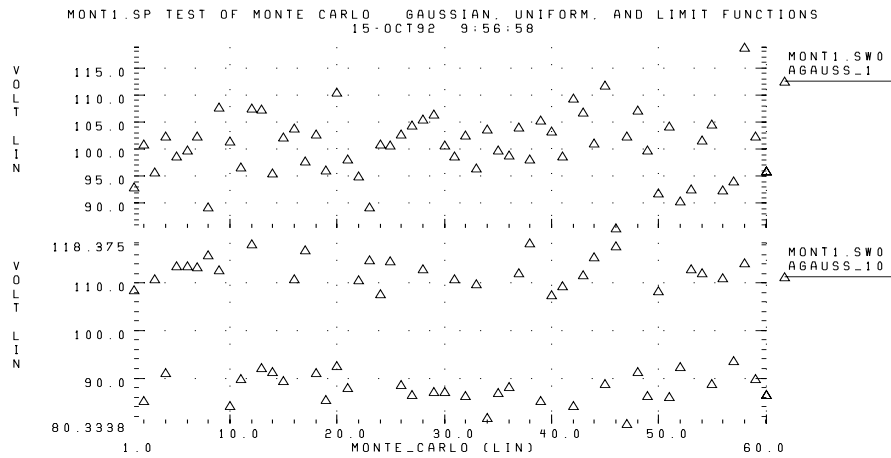
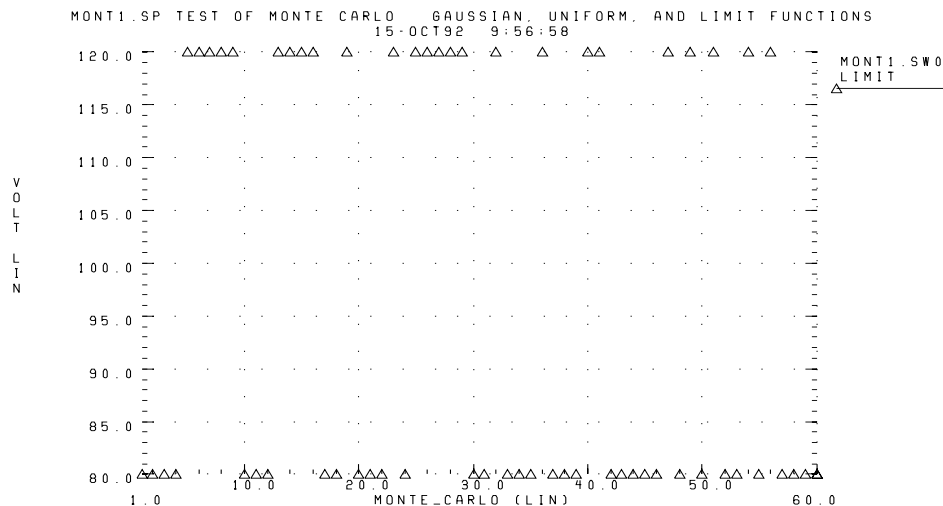
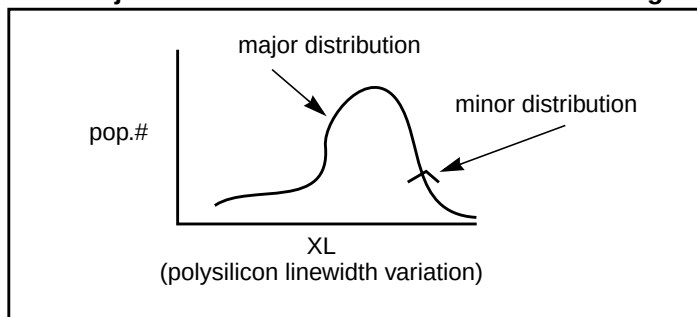
Figure 13-6: Gaussian Functions**Figure 13-7: Uniform Functions**

Figure 13-8: Limit Functions

Major and Minor Distribution

In MOS IC processes, manufacturing tolerance parameters have both a major and a minor statistical distribution.

- The major distribution is the wafer-to-wafer and run-to-run variation. It determines electrical yield.
- The minor distribution is the transistor-to-transistor process variation. It is responsible for critical second-order effects, such as amplifier offset voltage and flip-flop preference.

Figure 13-9: Major and Minor Distribution of Manufacturing Variations

The example below is a Monte Carlo analysis of a DC sweep, in Star-Hspice. Monte Carlo sweeps the VDD supply voltage, from 4.5 volts to 5.5 volts.

```
File: MONDC_A.SP
.DC VDD 4.5 5.5 .1 SWEEP MONTE=30
.PARAM LENGTH=1U LPHOTO=.1U
.PARAM LEFF=GAUSS (LENGTH, .05, 3)
+ XPHOTO=GAUSS (LPHOTO, .3, 3)
.PARAM PHOTO=XPHOTO

M1 1 2 GND GND NCH W=10U L=LEFF
M2 1 2 VDD' VDD PCH W=20U L=LEFF
M3 2 3 GND GND NCH W=10U L=LEFF
M4 2 3 VDD VDD PCH W=20U L=LEFF

.MODEL NCH NMOS LEVEL=2 U0=500 TOX=100 GAMMA=.7 VTO=.8
+ XL=PHOTO

.MODEL PCH PMOS LEVEL=2 U0=250 TOX=100 GAMMA=.5 VTO=-.8
+ XL=PHOTO
.INC Model.dat
.END
```

- The M1 through M4 transistors form two inverters.
- The nominal value of the LENGTH parameter sets the channel lengths for the MOSFETs, which are set to 1u in this example.
- All transistors are on the same integrated circuit die. The LEFF parameter specifies the distribution, which in this example is a $\pm 5\%$ distribution in the variation of the channel lengths, at the ± 3 -sigma level.
- Each MOSFET has an independent random Gaussian value.

The PHOTO parameter controls the difference between the physical gate length, and the drawn gate length. Because both n-channel and p-channel transistors use the same layer for the gates, Monte Carlo analysis sets the XPHOTO distribution to the PHOTO local parameter.

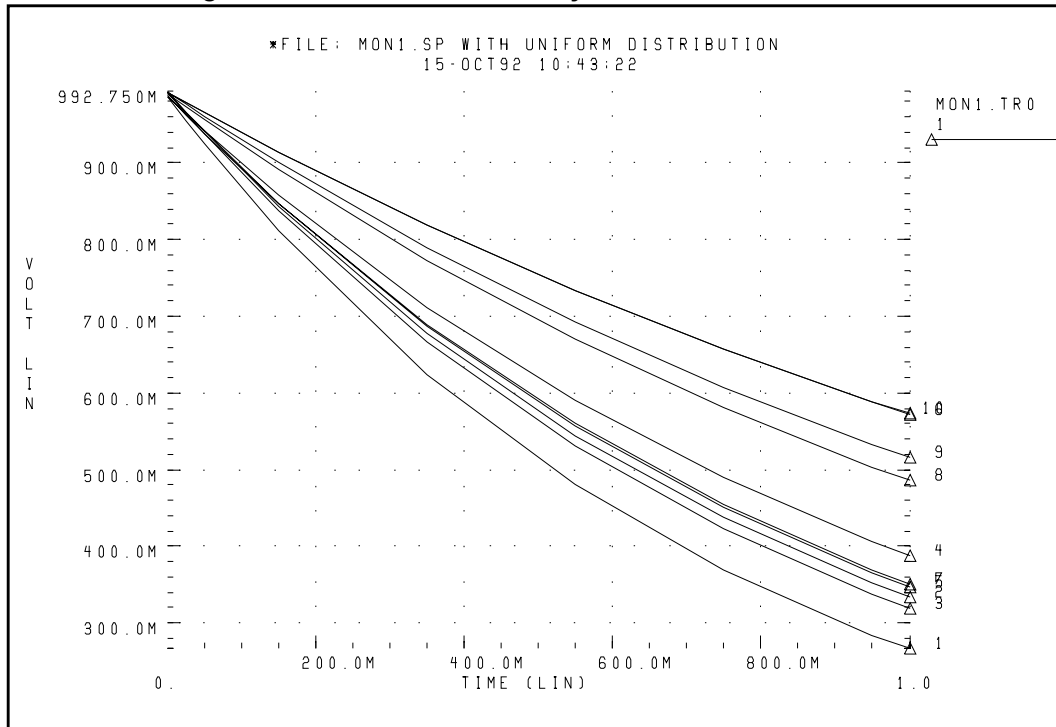
XPHOTO controls PHOTO lithography, for both NMOS and PMOS devices, which is consistent with the physics of manufacturing.

RC Time Constant

This simple example shows uniform distribution, for resistance and capacitance. It also shows the resulting transient waveforms, for 10 different random values.

```
*FILE: MON1.SP WITH UNIFORM DISTRIBUTION
.OPTION LIST POST=2
.PARAM RX=UNIF(1, .5) CX=UNIF(1, .5)
.TRAN .1 1 SWEEP MONTE=10
.IC 1 1
R1 1 0 RX
C1 1 0 CX
.END
```

Figure 13-10: Monte Carlo Analysis of RC Time Constant

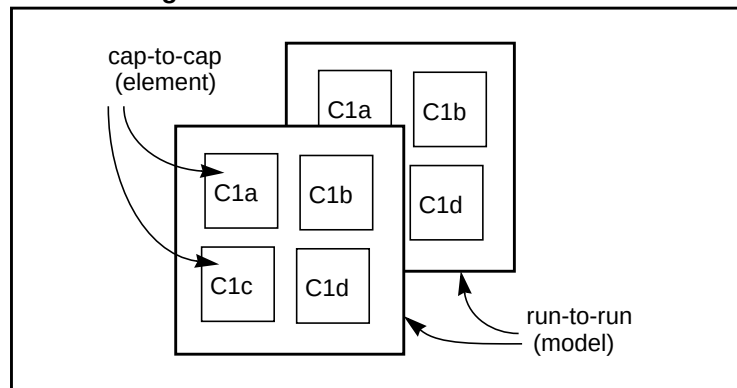


Switched Capacitor Filter Design

Capacitors, used in switched-capacitor filters, consist of parallel connections of a basic cell. Use Monte Carlo techniques in Star-Hspice to estimate the variation in total capacitance. The capacitance calculation uses two distributions:

- Minor (element) distribution of cell capacitance, from cell-to-cell, on a single die.
- Major (model) distribution of the capacitance, from wafer-to-wafer, or from manufacturing run-to-run.

Figure 13-11: Monte Carlo Distribution



You can approach this problem from either physical or electrical levels.

- The physical level relies on physical distributions, such as oxide thickness and polysilicon line width control.
- The electrical level relies on actual capacitor measurements.

Physical Approach

1. Oxide thickness control is excellent for small areas on a single wafer. Therefore, you can use a local variation in polysilicon to control the variation in capacitance, for adjacent cells.
2. Next, define a local poly line-width variation, and a global (model-level) poly line-width variation. In this example:
 - The local polysilicon linewidth control for a line 10 μ wide, manufactured with process A, is $\pm 0.02 \mu$, for a 1-sigma distribution.
 - The global (model level) polysilicon line-width control is much wider; use 0.1 μ for this example.

3. The global oxide thickness is 200 angstroms, with a ± 5 angstrom variation at 1 sigma.
4. The cap element is square, with local poly variation in both directions.
5. The *cap* model has two distributions:
 - poly line-width distribution
 - oxide thickness distribution.

The effective length is:

$$L_{eff} = L_{drawn} - 2 \cdot DEL$$

so the model poly distribution is half the physical per-side values:

```
C1a 1 0 CMOD W=ELPOLY L=ELPOLY
C1b 1 0 CMOD W=ELPOLY L=ELPOLY
C1C 1 0 CMOD W=ELPOLY L=ELPOLY
C1D 1 0 CMOD W=ELPOLY L=ELPOLY
$ 10U POLYWIDTH,0.05U=1SIGMA
$ CAP MODEL USES 2*MODPOLY .05u= 1 sigma
$ 5angstrom oxide thickness AT 1SIGMA
.PARAM ELPOLY=AGAUSS(10U,0.02U,1)
+ MODPOLY=AGAUSS(0,.05U,1)
+ POLYCAP=AGAUSS(200e-10,5e-10,1)
.MODEL CMOD C THICK=POLYCAP DEL=MODPOLY
```

Electrical Approach

The electrical approach assumes no physical interpretation, but requires a local (element) distribution, and a global (model) distribution. In this example:

- You can match the capacitors to $\pm 1\%$, for the 2-sigma population.
- The process can maintain a $\pm 10\%$ variation, from run to run, for a 2-sigma distribution.

```
C1a 1 0 CMOD SCALE=ELCAP
C1b 1 0 CMOD SCALE=ELCAP
C1C 1 0 CMOD SCALE=ELCAP
C1D 1 0 CMOD SCALE=ELCAP
.PARAM ELCAP=Gauss(1,.01,2) $ 1% at 2 sigma
+ MODCAP=Gauss(.25p,.1,2) $10% at 2 sigma
.MODEL CMOD C CAP=MODCAP
```

Worst Case and Monte Carlo Sweep Example

The following example measures the delay of a pair of inverters.

- An inverter buffers the input.
- Another inverter loads the output.

The model is prepared according to the scheme described in the previous sections:

- The first .TRAN analysis statement sweeps from the worst-case 3-sigma slow, to 3-sigma fast.
- The second .TRAN performs 100 Monte Carlo sweeps.

Star-Hspice Input File

The Star-Hspice input file can contain the following sections.

Analysis Setup Section

To accelerate the simulation, the AUTOSTOP option automatically stops the simulation, when the .MEASURE statements achieve their target values.

```
$ inv.sp sweep mosfet -3 sigma to +3 sigma,  
$ then Monte Carlo  
.option nopage nomod acct  
+ autostop post=2  
.tran 20p 1.0n sweep sigma -3 3 .5  
.tran 20p 1.0n sweep monte=20  
.option post co=132  
.param vref=2.5  
.meas m_delay trig v(2) val=vref fall=1  
+ targ v(out) val=vref fall=1  
.meas m_power rms power to=m_delay  
.param sigma=0
```

Circuit Netlist Section

```
.global 1
vcc 1 0 5.0
vin in 0 pwl 0,0 0.2n,5
x1 in 2 inv
x2 2 3 inv
x3 3 out inv
x4 out 5 inv
.macro inv in out
    mn out in 0 0 nch W=10u L=1u
    mp out in 1 1 pch W=10u L=1u
.eom
```

Skew Parameter Overlay for Model Section

* overlay of gaussian and algebraic for best case worst case
 + and + monte carlo
 * +/- 3 sigma is the maximum value for parameter sweep

```
.param
+ mult1=1
+ polycd=agauss(0,0.06u,1) xl='polycd-sigma*0.06u'
+ nactcd=agauss(0,0.3u,1) xwn='nactcd+sigma*0.3u'
+ pactcd=agauss(0,0.3u,1) xwp='pactcd+sigma*0.3u'
+ toxcd=agauss(200,10,1) tox='toxcd-sigma*10'
+ vtoncd=agauss(0,0.05v,1) delvton='vtoncd-sigma*0.05'
+ vtopcd=agauss(0,0.05v,1) delvtop='vtopcd+sigma*0.05'
+ rshncd=agauss(50,8,1) rshn='rshncd-sigma*8'
+ rshpcd=agauss(150,20,1) rshp='rshpcd-sigma*20'
```

MOS Model for N-Channel and P-Channel Transistors Section

```
* LEVEL=28 example model for high accuracy model
.model nch nmos
+ LEVEL=28
+ lmlt=mult1 wmlt=mult1 wref=22u lref=4.4u
+ xl=xl xw=xwn tox=tox delvto=delvton rsh=rshn
+ ld=0.06u wd=0.2u
+ acm=2 ldif=0 hdif=2.5u
+ rs=0 rd=0 rdc=0 rsc=0
+ js=3e-04 jsw=9e-10
+ cj=3e-04 mj=.5 pb=.8 cjsw=3e-10 mjsw=.3 php=.8 fc=.5
+ capop=4 xqc=.4 meto=0.08u
```

```

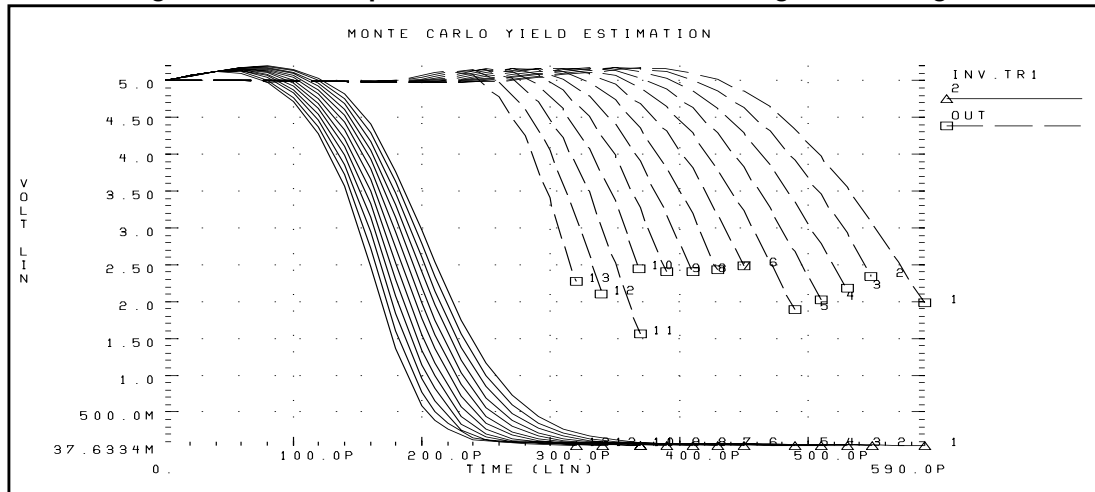
+ tlev=1 cta=0 ctp=0 tlevc=0 nlev=0
+ trs=1.6e-03 bex=-1.5 tcv=1.4e-03
* dc model
+ x2e=0 x3e=0 x2u1=0 x2ms=0 x2u0=0 x2m=0
+ vfb0=-.5 phi0=0.65 k1=.9 k2=.1 eta0=0
+ muz=500 u00=.075
+ x3ms=15 u1=.02 x3u1=0
+ b1=.28 b2=.22 x33m=0.000000e+00
+ alpha=1.5 vcr=20
+ n0=1.6 wfac=15 wfacu=0.25
+ lvfb=0 lk1=.025 lk2=.05
+ lalpha=5
.model pch pmos
+ LEVEL=28
+ lmlt=mult1 wmlt=mult1 wref=22u lref=4.4u
+ xl=xl xw=xwp tox=tox delvto=delvtop rsh=rshp
+ ld=0.08u wd=0.2u
+ acm=2 ldif=0 hdif=2.5u
+ rs=0 rd=0 rdc=0 rsc=0 rsh=rshp
+ js=3e-04 jsw=9e-10
+ cj=3e-04 mj=.5 pb=.8 cjsw=3e-10 mjsw=.3 php=.8 fc=.5
+ capop=4 xqc=.4 meto=0.08u
+ tlev=1 cta=0 ctp=0 tlevc=0 nlev=0
+ trs=1.6e-03 bex=-1.5 tcv=-1.7e-03
* dc model
+ x2e=0 x3e=0 x2u1=0 x2ms=0 x2u0=0 x2m=5
+ vfb0=-.1 phi0=0.65 k1=.35 k2=0 eta0=0
+ muz=200 u00=.175
+ x3ms=8 u1=0 x3u1=0.0
+ b1=.25 b2=.25 x33m=0.0
+ alpha=0 vcr=20
+ n0=1.3 wfac=12.5 wfacu=.2
+ lvfb=0 lk1=-.05
.end

```

Transient Sigma Sweep Results

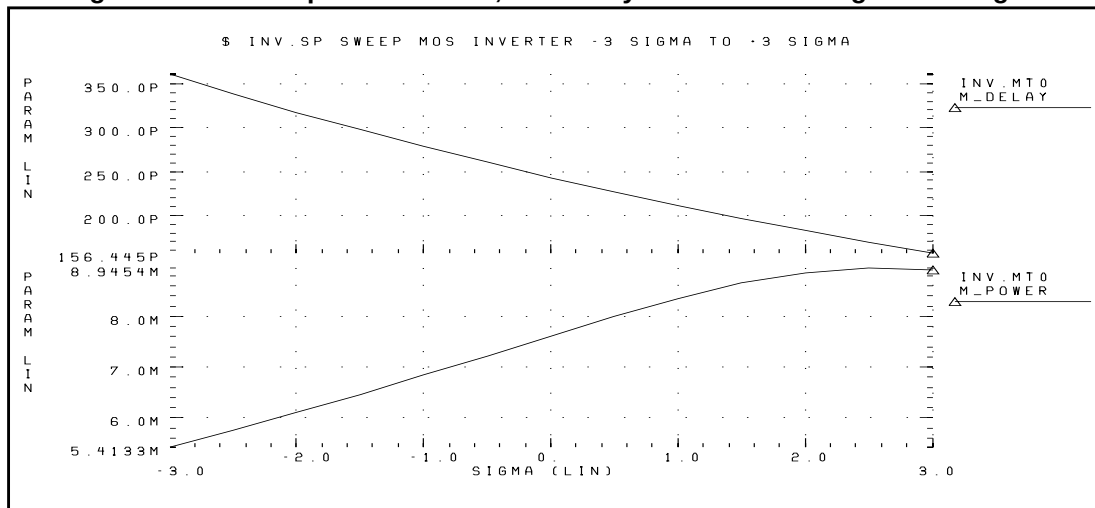
The plot in [Figure 13-12 on page 13-28](#) shows the family of transient analysis curves, for the transient sweep of the sigma parameter, from -3 to +3. Star-Hspice then algebraically couples sigma into the skew parameters. The resulting parameters modify the actual NMOS and PMOS models.

Figure 13-12: Sweep of Skew Parameters from -3 Sigma to +3 Sigma



To view the transient family of curves, plot the .MEASURE output file. The plot in Figure 13-13 shows the measured pair delay, and the total dissipative power, against the SIGMA parameter.

Figure 13-13: Sweep MOS Inverter, Pair Delay and Power: -3 Sigma to 3 Sigma



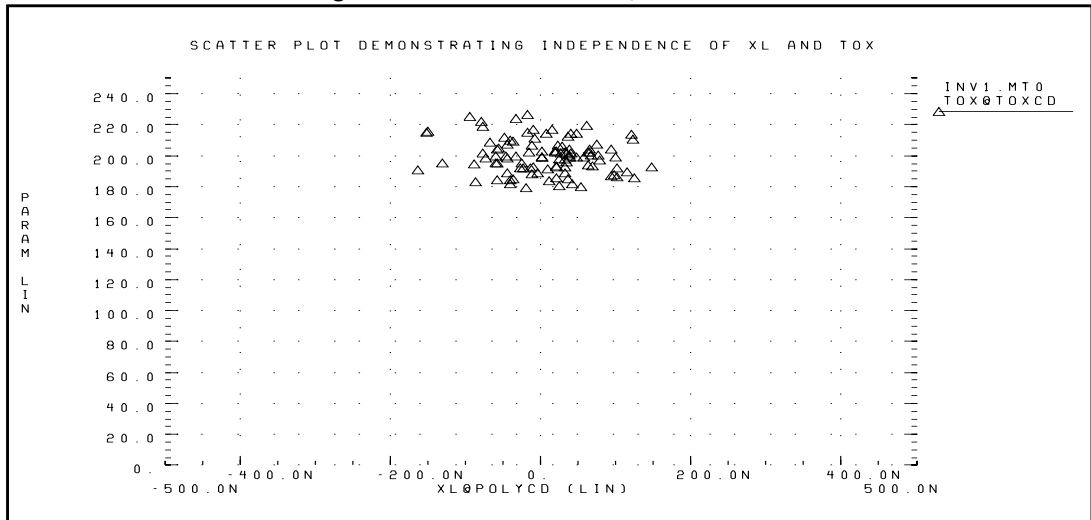
Monte Carlo Results

This section evaluates the output of the Monte Carlo analysis in Star-Hspice. The plot in [Figure 13-14](#) is a quality-control step, which plots *TOX* against *XL* (polysilicon critical dimension). Avant!'s graphing software returned the cloud of points, based on:

- Setting *XL* as the X-axis independent variable.
- Plotting *TOX*, with a symbol frequency of 1.

These settings plot points, without connecting lines. The resulting graph demonstrates that the *TOX* model parameter is randomly independent of *XL*.

Figure 13-14: Scatter Plot, XL and TOX



The next graph (see [Figure 13-15 on page 13-30](#)) is a standard scatter plot, showing the measured delay for the inverter pair, against the Monte Carlo index number. If a particular result looks interesting—for example, if the simulation 68 (*monte carlo index* = 68) produces the smallest delay—then you can read the output listing file, and obtain the Monte Carlo parameters for that simulation.

```

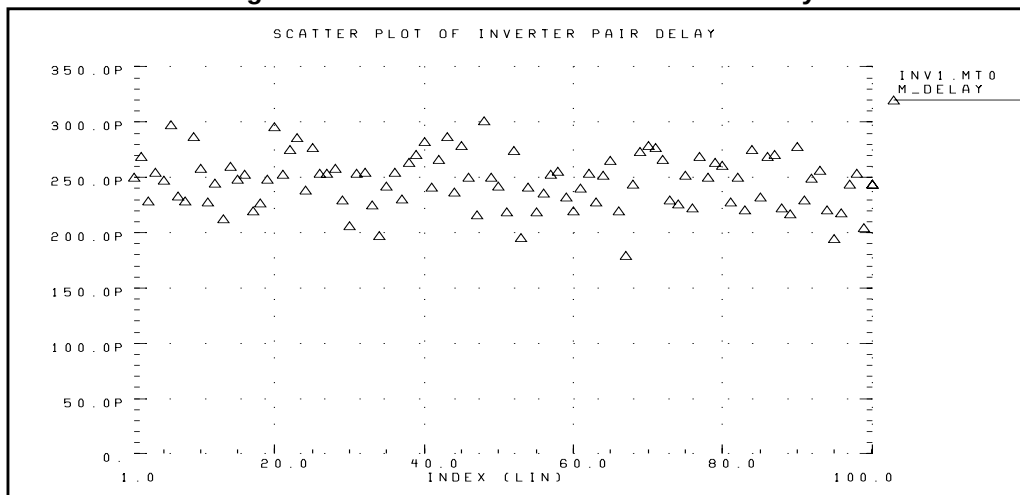
*** monte carlo index = 68 ***
MONTE CARLO PARAMETER DEFINITIONS
polycd: xl = -1.6245E-07
nactcd: xwn = 3.4997E-08
pactcd: xwp = 3.6255E-08
toxcd: tox = 191.0
vtoncd: delvton = -2.2821E-02
vtopcd: delvtop = 4.1776E-02
rshncd: rshn = 45.16
rshpcd: rshp = 166.2

m_delay = 1.7946E-10 targ= 3.4746E-10 trig= 1.6799E-10
m_power = 7.7781E-03 from= 0.0000E+00 to= 1.7946E-10

```

In the preceding listing, the m_delay value of $1.79\text{e-}10$ seconds is the fastest pair delay. You can also examine the Monte Carlo parameters.

Figure 13-15: Scatter Plot of Inverter Pair Delay

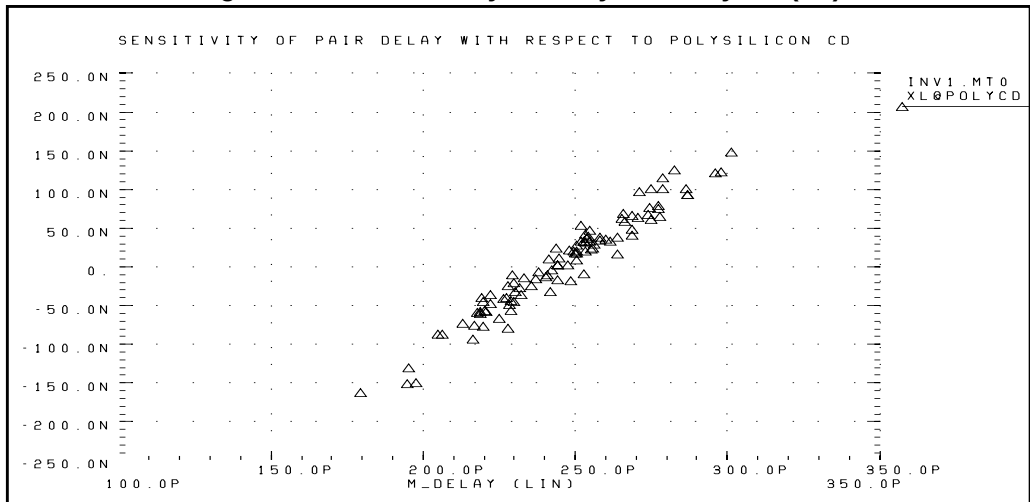


Plotting against the Monte Carlo index number does not help to center the design. To center the design:

1. Graph the various process parameters against the pair delay.
This graph determines the most sensitive process variables.
2. Select the pair delay, as the X-axis independent variable.
3. Set the symbol frequency to 1, to obtain the scatter plot.

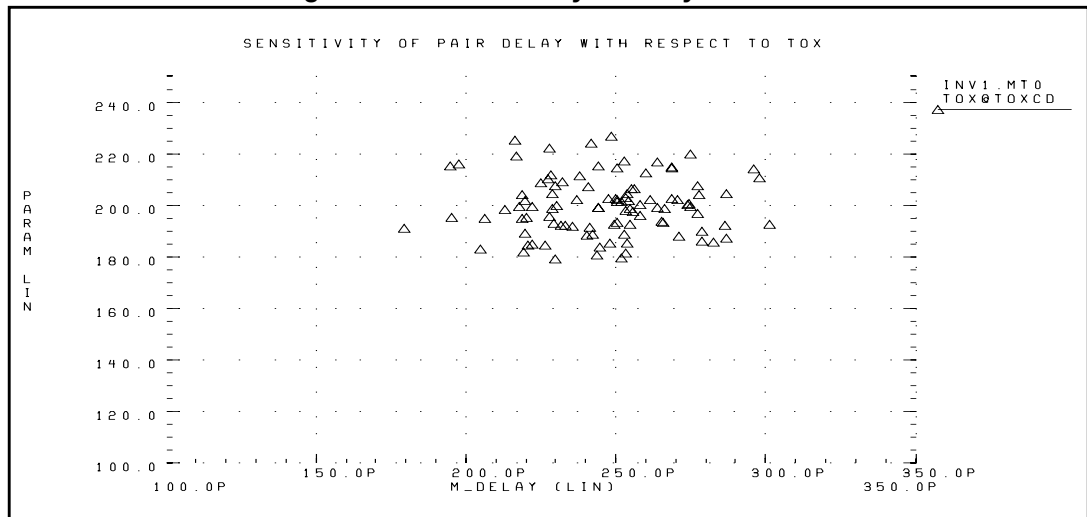
Figure 13-16 plots the expected sensitivity of the output pair delay, to channel length variation (polysilicon variation).

Figure 13-16: Sensitivity of Delay with Poly CD (XL)



The next plot shows the *TOX* parameter, against the pair delay (Figure 13-17). The scatter plot does not have a clear tilt, because *TOX* is a secondary process parameter, compared to *XL*. To explore this in more detail, set the *XL* skew parameter to a constant, and run a simulation.

Figure 13-17: Sensitivity of Delay with TOX

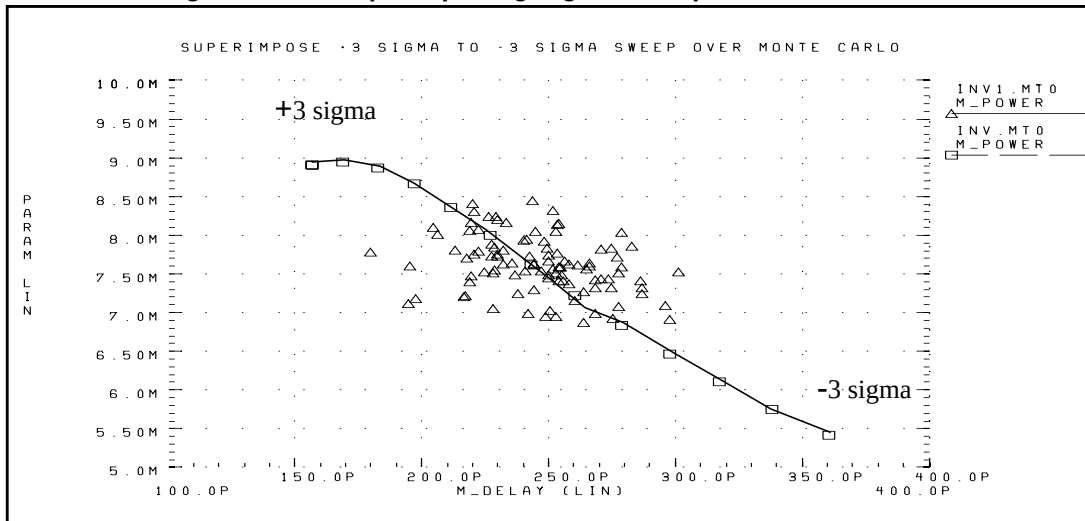


The plot in [Figure 13-18](#) overlays a 3-sigma, worst-case corners response, on a 100-point Monte Carlo analysis. The actual (Monte Carlo) distribution for power/delay is very different from the +3 sigma to -3 sigma plot.

- This example simulated the worst case in 0.5 sigma steps.
- The actual response is closer to ± 1.5 sigma, instead of ± 3 sigma.
- This produces a predicted delay variation of 100 ps, instead of 200 ps.

Therefore, the advantage of using Monte Carlo over traditional 3-sigma, worst-case corners, is a 100% improvement in accuracy of simulated-to-actual distribution. This shows how the worst-case procedure is overly pessimistic.

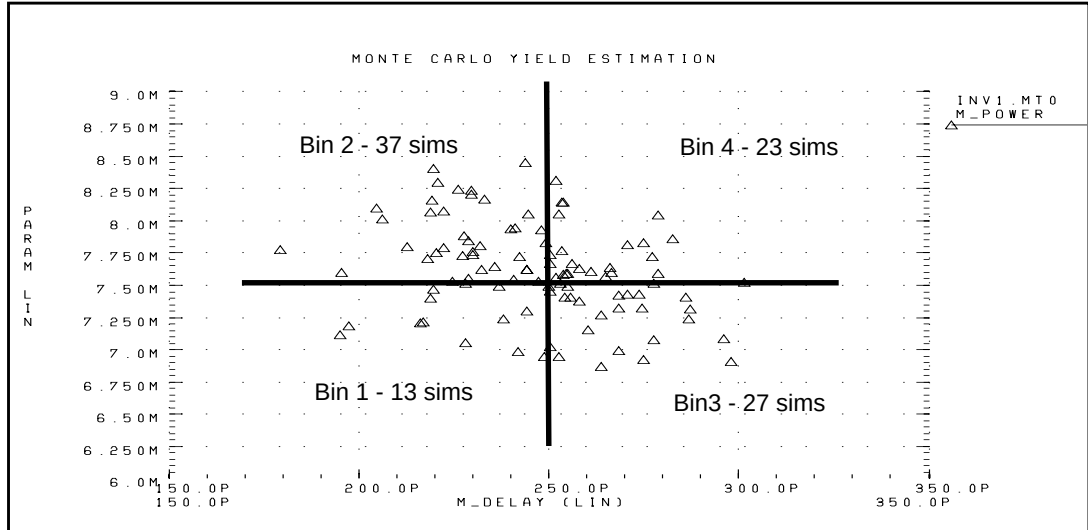
Figure 13-18: Superimposing Sigma Sweep Over Monte Carlo



[Figure 13-19 on page 13-33](#) superimposes the assumed part grades from marketing studies, onto the Monte Carlo plot. This example uses a 250 ps delay, and 7.5 mW power dissipation, to determine the four binning grades. A manual count shows:

- Bin1 - 13%
- Bin2 - 37%
- Bin3 - 27%
- Bin4 - 23%

If this circuit is representative of the entire chip, then the present yield should be 13% for the premium *Bin 1* parts, assuming variations in design and process.

Figure 13-19: Speed/Power Yield Estimation

Optimization

Optimization automatically generates model parameters and component values, from a set of electrical specifications or measured data. With you define an optimization program and a circuit topology, Star-Hspice automatically selects the design components and model parameters, to meet your DC, AC, and transient electrical specifications.

Star-Hspice optimization is the result of more than ten years of research, in both optimizing algorithms and user interface.

- The optimizing function is integrated into the core of Star-Hspice, for optimum efficiency.
- The circuit-result targets are part of the .MEASURE command structure.
- Star-Hspice optimize its own internally-defined parameter functions.

Use a .MODEL statement to set up the optimization.

Note: Star-Hspice uses post-processing output to compute the .MEASURE statements. If you set INTERP=1 to reduce the post-processing output, the measurement results might contain interpolation errors. See [Input and Output Options on page 9-51](#) for more information about these options.

The most powerful feature of Star-Hspice optimization is its incremental optimization technique. You can use this technique to solve the DC parameters first, then the AC parameters, and finally the transient parameters. A set of optimizer measurement functions not only makes transistor optimization easy, but significantly improves cell and circuit optimization.

To perform optimization, create an input netlist file that specifies:

- Minimum and maximum parameter and component limits.
- Variable parameters and components.
- An initial estimate of the selected parameter and component values.
- Circuit performance goals, or a model-versus-data error function.

If you provide the input netlist file, optimization specifications, component limits, and initial guess, then the optimizer reiterates the circuit simulation, until it either meets the target electrical specification, or finds an optimized solution.

For improved optimization, reduced simulation time, and increased likelihood of a convergent solution, the initial estimate of the component values should produce a circuit whose specifications are near those of the original target. This reduces the number of times the optimizer reselects component values, and resimulates the circuit.

Optimization Control

How much time an optimization requires, before it completes, depends on:

- Number of iterations allowed.
- Relative input tolerance.
- Output tolerance.
- Gradient tolerance.

The default values are satisfactory for most applications. Generally, 10 to 30 iterations are sufficient, to obtain accurate optimizations.

Simulation Accuracy

For optimization, set the simulator with tighter convergence options than normal. The following are suggested options:

For DC MOS model optimizations:

```
absmos=1e-8  
relmos=1e-5  
relv=1e-4
```

For DC JFET, BJT, and diode model optimizations:

```
absi=1e-10  
reli=1e-5  
relv=1e-4
```

For transient optimizations:

```
relv=1e-4  
relvar=1e-2
```

Curve Fit Optimization

Use optimization to curve-fit the DC, AC, or transient data that you define.

1. Use the `.DATA` statement to store the numeric data for curves, in the data file, as in-line data.
2. Use the `.PARAM xxx=OPTxxx` statement to specify the variable circuit components, and the parameter values, for the netlist.

The optimization analysis statements use the `DATA=` keyword to call the in-line data.

3. Use the `.MEASURE` statement to compare the simulation result to the values in the data file

In this statement, use the `ERR1` keyword to control the comparison.

If the calculated value is not within the error tolerances specified in the optimization model, Star-Hspice selects a new set of component values. Star-Hspice then simulates the circuit again, and repeats this process, until it obtains the closest fit to the curve, or until the set of error tolerances is satisfied.

Goal Optimization

Goal optimization differs from curve-fit optimization, because it usually optimizes *only* a particular electrical specification, such as rise time or power dissipation.

1. To specify goal optimizations, use the `GOAL` keyword.
2. In the `.MEASURE` statement, select a relational operator, where `GOAL` is the target electrical specification to measure.

For example, you can choose a relational operator in multiple-constraint optimizations, when the absolute accuracy of some criteria is less important than for others.

Timing Analysis

To analyze circuit timing violation, Star-Hspice uses a binary search algorithm. This algorithm generate a set of operational parameters, which produce a failure in the required behavior of the circuit. When a circuit timing failure occurs, you can identify a timing constraint, which can lead to a design guideline. Typical types of timing constraint violations include:

- Data setup time, before a clock.
- Data hold time, after a clock.
- Minimum pulse width required, to allow a signal to propagate to the output.
- Maximum toggle frequency of the component(s).

Bisection Optimization finds the value of an input variable (target value), associated with a goal value for an output variable. You can use various types of input and output variables, and a transfer function to relate them. For example:

- voltage
- current
- delay time
- gain

You can use the bisection feature, in either a pass-fail mode or a bisection mode. In each case, the process is largely the same.

Optimization Syntax

Optimization requires several Star-Hspice statements:

- `.MODEL modname OPT . . .`
- `.PARAM parameter=OPTxxx (init, min, max)`

Use the `.PARAM` statement to specify initial, lower, and upper bounds.

- A `.DC`, `.AC`, or `.TRAN` analysis statement, with:
 - `MODEL=modname`
 - `OPTIMIZE=OPTxxx`
 - `RESULTS=measurename`

Use the `.PRINT`, `.PLOT`, and `.GRAPH` output statements, with the `.DC`, `.AC`, or `.TRAN` analysis statements.

Use an analysis statement, with the OPTIMIZE keyword, *only* for optimization. To generate output for the optimized circuit, specify another analysis statement (.DC, .AC, or .TRAN), and the output statements.

- .MEASURE *measurename* ... <GOAL = | < | > *val*>

Include a space on either side of the relational operator:

=
<
>

For a description of the types of .MEASURE statements that you can use in optimization, see [Simulation Output on page 8-1](#).

The proper specification order is:

1. Analysis statement, with OPTIMIZE.
2. .MEASURE statements, specifying optimization goals or error functions.
3. Ordinary analysis statement.
4. Output statements.

Analysis Statement (.DC, .TRAN, .AC)

The syntax is:

```
.DC <DATA=filename> SWEEP OPTIMIZE=OPTxxx
+ RESULTS=ierr1 ...
+ ierrn MODEL=optmod
```

or

```
.AC <DATA=filename> SWEEP OPTIMIZE=OPTxxx
+ RESULTS=ierr1 ... ierrn MODEL=optmod
```

or

```
.TRAN <DATA=filename> SWEEP OPTIMIZE=OPTxxx
+ RESULTS=ierr1 ... ierrn MODEL=optmod
```

where:

<i>DATA</i>	Specifies the in-line file of parameter data, to use in the optimization.
<i>OPTIMIZE</i>	Indicates that the analysis is for optimization. Specifies the parameter reference name, used in the .PARAM optimization statement. In a .PARAM optimization statements, if OPTIMIZE selects the parameter reference name, then the associated parameters vary during an optimization analysis.
<i>MODEL</i>	The optimization reference name, which you also specify in the .MODEL optimization statement.
<i>RESULTS</i>	The measurement reference name. You also specify this name in the .MEASURE optimization statement. RESULTS passes the analysis data to the .MEASURE optimization statement.

.PARAM Statement

The syntax is:

```
.PARAM parameter=OPTxxx (initial_guess, low_limit,  
+ upper_limit)
```

or

```
.PARAM parameter=OPTxxx (initial_guess, low_limit,  
+ upper_limit, delta)
```

where:

- OPTxxx* Specifies the optimization parameter reference name. The associated optimization analysis references this name. This must agree with the *OPTxxx* name, as specified in the analysis command associated with the OPTIMIZE keyname.
- parameter* Specifies:
- Parameter to vary.
 - Initial value estimate
 - Lower limit.
 - Upper limit.
- If the optimizer does not find the best solution within these constraints, it attempts to find the best solution without constraints.
- delta* The final parameter value is the initial guess $\pm (n \cdot \text{delta})$. If you do not specify *delta*, the final parameter value is between *low_limit* and *upper_limit*. For example, you can use this parameter to optimize transistor drawn widths and lengths, which must be quantized.

In the following example, *uox* and *vtx* are the variable model parameters, which optimize a model for a selected set of electrical specifications.

```
.PARAM vtx=OPT1(.7,.3,1.0) uox=OPT1(650,400,900)
```

The estimated initial value for the *vtx* parameter is 0.7 volts. You can vary this value within the limits of 0.3 and 1.0 volts, for the optimization procedure. The optimization parameter reference name (OPT1) references the associated optimization analysis statement (not shown).

.MODEL Statement

For each optimization within a data file, specify a .MODEL statement. Star-Hspice can then execute more than one optimization per simulation run. The .MODEL optimization statement defines:

- Convergence criteria.
- Number of iterations.
- Derivative methods.

The syntax is:

```
.MODEL mname OPT <parameter=val ...>
```

You can specify the following OPT parameters in the .MODEL statement:

- | | |
|--------------|--|
| <i>mname</i> | Model name. Elements use this name to refer to the model. |
| CENDIF | The point when optimizing needs more-accurate derivatives. If the gradient of the RESULTS functions are less than CENDIF, Star-Hspice uses more time-consuming derivative methods. You can use values of 0.1 to 0.01 in most applications. If you use too large a value, the optimizer requires more CPU time. If you use too small a value, the optimizer might not find as accurate an answer. Default=1.0e-9. |
| CLOSE | <p>Initial estimate of how close parameter initial value estimates are, to the solution. CLOSE multiplies changes in new parameter estimates. If you use a large CLOSE value, the optimizer takes large steps toward the solution. For a small value, the optimizer takes smaller steps toward the solution. CLOSE ranges from 0.01 (very close parameter estimates) to 10 (rough initial guesses). Default=1.0.</p> <p>If CLOSE is greater than 100, the steepest descent in the Levenburg-Marquardt algorithm dominates. If CLOSE is less than 1, the Gauss-Newton method dominates. For more details, see L. Spruiell, "Optimization Error Surfaces," <i>Meta-Software Journal</i>, Vol. 1, No. 4, December 1994.</p> |
| CUT | Modifies CLOSE, depending on how successful iterations are, toward the solution. If the last iteration succeeds, descent toward the CLOSE solution decreases by the CUT value. If the last iteration was not a successful descent to the solution, CLOSE increases by CUT squared. CUT drives CLOSE up or down, depending on the relative success in finding the solution. The CUT value must be > 1. Default = 2.0. |
| DIFSIZ | Increment change in a parameter value, for gradient calculations ($\Delta x = DIFSIZ \cdot \max(x, 0.1)$). If you specify delta in a .PARAM statement, then $\Delta x = \text{delta}$. Default = 1e-3. |

GRAD	Represents possible convergence, if the gradient of the RESULTS function is less than GRAD. Most applications use values of 1e-6 to 1e-5. Too large a value can stop the optimizer before finding the best solution. Too small a value requires more iterations. Default=1.0e-6.
ITROPT	Maximum number of iterations. Typically, you need no more than 20-40 iterations, to find a solution. Too many iterations can imply that the RELIN, GRAD, or RELOUT values are too small. Default=20.
LEVEL	Selects an optimizing algorithm. Currently, the only option is LEVEL=1, a modified Levenburg-Marquardt algorithm.
MAX	Sets the upper limit on CLOSE. Use values > 100. Default=6000.
PARMIN	Allows better control of incremental parameter changes, during error calculations. Default=0.1. This produces more control over the trade-off between simulation time and optimization result accuracy. To calculate parameter increments, Star-Hspice uses the relationship: $\Delta par_val = DIFSIZ \cdot \text{MAX}(par_val, \text{PARMIN})$
RELIN	Variation in the relative input parameter, for convergence. If all optimizing input parameters vary by no more than RELIN between iterations, the solution converges. RELIN is a relative variance test, so a value of 0.001 implies that optimizing parameters vary by less than 0.1%, from one iteration to the next. Default=0.001.
RELOUT	Represents the variance in the relative output RESULTS function, for convergence. For RELOUT=0.001, the difference in the RMS error of the RESULTS functions, vary less than 0.001. Default=0.001.

Optimization Examples

This section provides examples of the following types of Star-Hspice optimizations:

- [MOS Level 3 Model DC Optimization](#)
- [MOS Level 13 Model DC Optimization](#)
- [RC Network Optimization](#)
- [Optimizing CMOS Tristate Buffer](#)
- [BJT S Parameters Optimization](#)
- [BJT Model DC Optimization](#)
- [Optimizing GaAsFET Model DC](#)
- [Optimizing MOS Op-amp](#)

MOS Level 3 Model DC Optimization

This example shows an optimization of I-V data, to a Level 3 MOS model. The data consists of gate curves (*ids* versus *vgs*) and drain curves (*ids* versus *vds*).

This example optimizes the Level 3 parameters:

- VTO
- GAMMA
- UO
- VMAX
- THETA
- KAPPA

After optimization, Star-Hspice compares the model to the data for the gate, and then to the drain curves. The POST option generates AvanWaves files, for comparing the model to the data.

Input Netlist File, for Level 3 Model DC Optimization

```
$LEVEL 3 mosfet optimization
$.tighten the simulator convergence properties
.OPTION nomod post=2 newtol relmos=1e-5 absmos=1e-8
.MODEL optmod OPT itropt=30
```

Circuit Input

```

vds 30 0 vds
vgs 20 0 vgs
vbs 40 0 vbs
m1 30 20 0 40 nch w=50u l=4u

$..
$.process skew parameters for this data
.PARAM xwn=-0.3u xln=-0.1u toxn=196.6 rshn=67

$.the model and initial guess
.MODEL nch NMOS LEVEL=3
+ acm=2 ldif=0 hdif=4u tlev=1 n=2
+ capop=4 meto=0.08u xqc=0.4

$...note capop=4 is ok for H8907 and later, otherwise
$...use Capop=2
$...fixed parameters
+ wd=0.15u ld=0.07u
+ js=1.5e-04 jsw=1.8e-09
+ cj=1.7e-04 cjsw=3.8e-10
+ nfs=2e11 xj=0.1u delta=0 eta=0

$...process skew parameters
+ tox=toxn rsh=rshn
+ xw=xwn xl=xln

```

Optimized Parameters

```

+ vto=vto gamma=gamma
+ uo=uo vmax=vmax theta=theta kappa=kappa

.PARAM
+ vto      = opt1(1,0.5,2)
+ gamma    = opt1(0.8,0.1,2)
+ uo       = opt1(480,400,1000)
+ vmax     = opt1(2e5,5e4,5e7)
+ theta    = opt1(0.05,1e-3,1)
+ kappa    = opt1(2,1e-2,5)

```

Optimization Sweeps

```

.DC DATA=all optimize=opt1 results=comp1 model=optmod
.MEAS DC comp1 ERR1 par(ids) i(m1) minval=1e-04 ignor=1e-05

```

DC Sweeps

```
.DC DATA=gate
.DC DATA=drain
```

Print Sweeps

```
.PRINT DC vds=par(vds) vgs=par(vgs) im=i(m1) id=par(ids)
.PRINT DC vds=par(vds) vgs=par(vgs) im=i(m1) id=par(ids)
```

DC Sweep Data

```
$.data
.PARAM vds=0 vgs=0 vbs=0 ids=0
.DATA all vds vgs vbs ids
1.000000e-01 1.000000e+00 0.000000e+00 1.655500e-05
5.000000e+00 5.000000e+00 0.000000e+00 4.861000e-03
.ENDATA

.DATA gate vds vgs vbs ids
1.000000e-01 1.000000e+00 0.000000e+00 1.655500e-05
1.000000e-01 5.000000e+00 -2.000000e+00 3.149500e-04
.ENDDATA

.DATA drain vds vgs vbs ids
2.500000e-01 2.000000e+00 0.000000e+00 2.302000e-04
5.000000e+00 5.000000e+00 0.000000e+00 4.861000e-03
.ENDDATA

.END
```

The Star-Hspice input netlist shows:

- Using .OPTION to tighten tolerances, which increases the accuracy of Star-Hspice simulation. Use this method for I-V optimization.
- .MODEL optmod OPT itropt=30 limits the number of iterations to 30.
- The circuit is one transistor. The VDS, VGS, and VBS parameter names, match names used in the data statements.
- .PARAM statements specify XL, XW, TOX, and RSH process variation parameters, as constants. The device characterizes these measured parameters.
- The model references parameters. In GAMMA= GAMMA, the left side is a *Level 3* model parameter name; the right side is a .PARAM parameter name.
- The long .PARAM statement specifies initial, min and max values, for the optimized parameters. Optimization initializes UO at 480, and maintains it within the range 400 to 1000.

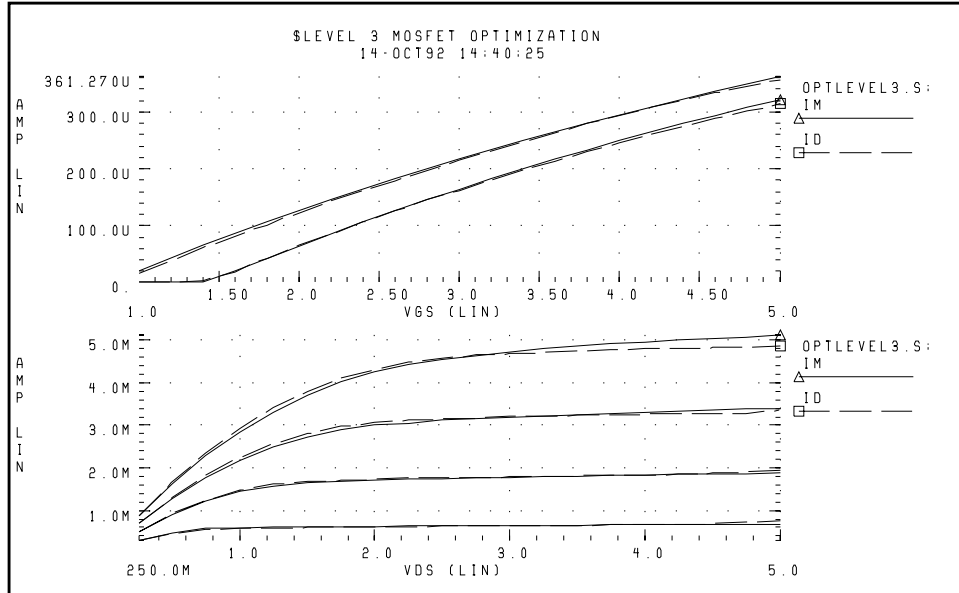
- The first .DC statement indicates that:
 - Data is in the in-line .DATA *all* block, which contains merged gate and drain curve data.
 - Parameters that you declared as OPT1 (in this example, all optimized parameters) are optimized.
 - The COMP1 error function matches the name of a .MEASURE statement.
 - The OPTMOD model sets the iteration limit.
- The .MEASURE statement specifies least-squares relative error. Star-Hspice divides the difference between data *par(ids)* and model *i(m1)*, by either:
 - the absolute value of *par(ids)*, or
 - *minval*=10e-6

whichever is larger. If you use *minval*, low current data does not dominate the error.
- Use the remaining .DC and .PRINT statements for print-back, after optimization. You can place them anywhere in the netlist input file, because parsing the file correctly assigns them.
- The .PARAM VDS=0 VGS=0 VBS=0 IDS=0 statements declare these data column names, as parameters.

The .DATA statements contain data for IDS versus VDS, VGS, and VBS. Select data that matches the model parameters to optimize. For example, to optimize GAMMA, use data with back bias (VBS= -2 in this case). To optimize KAPPA, the saturation region must contain data. In this example, the *all* data set contains:

- Gate curves: vds=0.1 vbs=0,-2 vgs=1 to 5, in steps of 0.25.
- Drain curves: vbs=0 vgs=2,3,4,5 vds=0.25 to 5, in steps of 0.25.

Figure 13-20 on page 13-47 shows the results.

Figure 13-20: Level 3 MOSFET Optimization

MOS Level 13 Model DC Optimization

This example shows optimization of I-V data, to a Level 13 MOS model. The data consists of gate curves (*ids* versus *vgs*) and drain curves (*ids* versus *vds*). This example demonstrates two-stage optimization.

1. Star-Hspice optimizes the *vfb0*, *k1*, *muz*, *x2m*, and *u00* Level 13 parameters, to the gate data.
2. Then Star-Hspice optimizes the *MUS*, *x3MS*, and *u1* Level 13 parameters, and the ALPHA impact ionization parameter, to the drain data.

After optimization, Star-Hspice compares the model to the data. The POST option generates AvanWaves files, to compare the model to the data.

[Figure 13-21 on page 13-50](#) shows the results.

DC Optimization Input Netlist File, for Level 13 Model

```
$LEVEL 13 mosfet optimization
$.tighten the simulator convergence properties
.OPTION nomod post=2 newtol relmos=1e-5 absmos=1e-8
.MODEL optmod OPT itropt=30
```

Circuit Input

```
vds 30 0 vds
vgs 20 0 vgs
vbs 40 0 vbs
m1 30 20 0 40 nch w=50u l=4u
$.
$.process skew parameters for this data
.PARAM xwn=-0.3u xln=-0.1u toxn=196.6 rshn=67
$.the model and initial guess
.MODEL nch NMOS LEVEL=13
+ acm=2 ldif=0 hdif=4u tlev=1 n=2 capop=4 meto=0.08u
+ xqc=0.4
$.parameters obtained from measurements
+ wd=0.15u ld=0.07u js=1.5e-04 jsw=1.8e-09
+ cj=1.7e-04 cjsw=3.8e-10
$.parameters not used for this data
+ k2=0 eta0=0 x2e=0 x3e=0 x2u1=0 x2ms=0 x2u0=0 x3u1=0
$.process skew parameters
+ toxm=toxn rsh=rshn
+ xw=xwn xl=xln
$.optimized parameters
+ vfb0=vfb0 k1=k1 x2m=x2m muz=muz u00=u00
+ mus=mus x3ms=x3ms u1=u1
$.impact ionization parameters
+ alpha=alpha vcr=15
.PARAM
+ vfb0      = opt1(-0.5, -2, 1)
+ k1        = opt1(0.6,0.3,1)
+ muz       = opt1(600,300,1500)
+ x2m       = opt1(0, -10,10)
+ u00       = opt1(0.1,0,0.5)
+ mus       = opt2(700,300,1500)
+ x3ms      = opt2(5,0,50)
+ u1        = opt2(0.1,0,1)
+ alpha     = opt2(1,1e-3,10)
```


Optimization Sweeps

```
.DC DATA=gate optimize=opt1 results=comp1 model=optmod  
.MEAS DC comp1 ERR1 par(ids) i(m1) minval=1e-04 ignor=1e-05  
.DC DATA=drain optimize=opt2 results=comp2 model=optmod  
.MEAS DC comp2 ERR1 par(ids) i(m1) minval=1e-04 ignor=1e-05
```

DC Data Sweeps

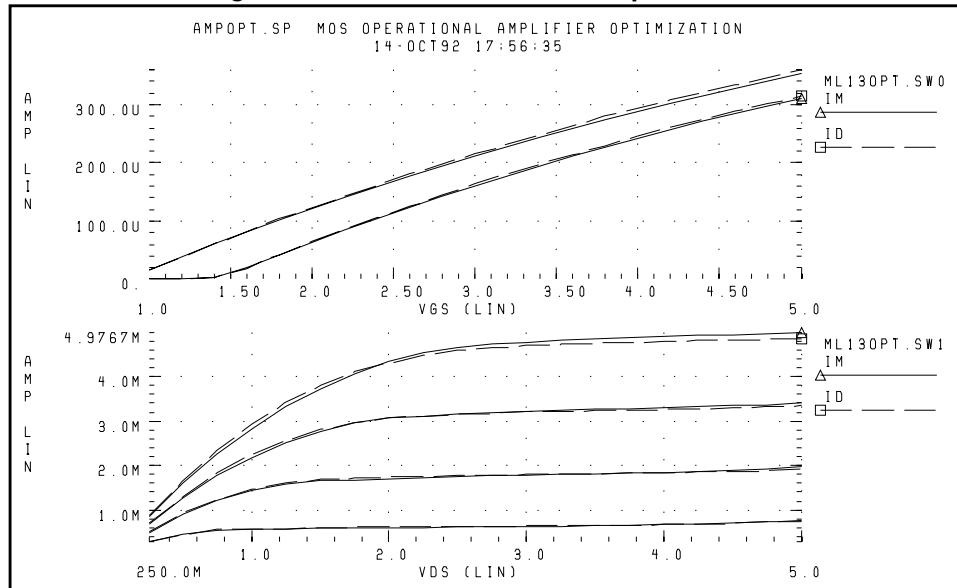
```
.DC DATA=gate  
.DC DATA=drain
```

Print Sweeps

```
.PRINT DC vds=par(vds) vgs=par(vgs) im=i(m1) id=par(ids)  
.PRINT DC vds=par(vds) vgs=par(vgs) im=i(m1) id=par(ids)
```

DC Sweep Data

```
$.data  
.PARAM vds=0 vgs=0 vbs=0 ids=0  
.DATA gate vds vgs vbs ids  
1.000000e-01 1.000000e+00 0.000000e+00 1.655500e-05  
1.000000e-01 5.000000e+00 -2.000000e+00 3.149500e-04  
.ENDDATA  
.DATA drain vds vgs vbs ids  
2.500000e-01 2.000000e+00 0.000000e+00 2.809000e-04  
5.000000e+00 5.000000e+00 0.000000e+00 4.861000e-03  
.ENDDATA  
.END
```

Figure 13-21: Level 13 MOSFET Optimization

RC Network Optimization

The following example optimizes the power dissipation and time constant, for an RC network. The circuit is a parallel resistor and capacitor. Design targets are:

- 1 s time constant.
- 50 mW rms power dissipation, through the resistor.

The Star-Hspice strategy is:

- The RC1 .MEASURE statement calculates the RC time constant, where the GOAL of .3679 V corresponds to 1 s time constant e^{-RC} .
- The RC2 .MEASURE statement calculates the rms power, where the GOAL is 50 mW.
- OPTrc identifies RX and CX as optimization parameters, and sets their starting, minimum, and maximum values.

Network optimization uses these Star-Hspice features:

- Measure voltages, and report times that are subject to a goal.
- Measure device power dissipation, subject to a goal.
- Measure statements replace the tabular or plot output.
- Parameters used as element values.
- Parameter optimizing function.
- Transient analysis, with SWEEP optimizing.

RC Network Optimization Input Netlist File

```
.title RCOPT.sp
.option post

.PARAM RX=OPTRC(.5, 1E-2, 1E+2)
.PARAM CX=OPTRC(.5, 1E-2, 1E+2)

.MEASURE TRAN RC1 TRIG AT=0 TARG V(1) VAL=.3679 FALL=1
+ GOAL=1sec
.MEASURE TRAN RC2 RMS P(R1) GOAL=50mwatts

.MODEL OPT1 OPT

.tran .1 2          $ initial values
.tran .1 2 SWEEP OPTIMIZE=OPTrc RESULTS=RC1,RC2 MODEL=OPT1
.tran .1 2          $ analysis using final optimized values

.ic 1 1
R1 1 0 RX
c1 1 0 CX
```

Optimization Results

```
RESIDUAL SUM OF SQUARES      = 1.323651E-06
NORM OF THE GRADIENT          = 6.343728E-03
MARQUARDT SCALING PARAMETER  = 2.799235E-06
NO. OF FUNCTION EVALUATIONS  =          24
NO. OF ITERATIONS             =          12
```

Residual Sum of Squares

The residual sum of squares is a measure of the total error. The smaller this value is, the more accurate the optimization results are.

$$\text{residual sum of squares} = \sum_{i=1}^{ne} E_i^2$$

where E is the error function, and ne is the number of error functions.

Norm of the Gradient

The norm of the gradient is another measure of the total error. The smaller this value is, the more accurate the optimization results are. The following equations calculates the G gradient:

$$G_j = \sum_{i=1}^{ne} E_i \cdot (\Delta E_i / \Delta P_j)$$

$$\text{norm of the gradient} = 2 \cdot \sqrt{\sum_{j=1}^{np} G_j^2}$$

where P is the parameter, and np is the number of parameters to optimize.

Marquardt Scaling Parameter

The Levenburg-Marquardt algorithm uses this parameter to find the actual solution for the optimizing parameters. The search direction is a combination of the Steepest Descent method, and the Gauss-Newton method.

The optimizer initially uses the Steepest Descent method, as the fastest approach to the solution. It then uses the Gauss-Newton method, to find the solution. During this process, the Marquardt Scaling Parameter becomes very small, but starts to increase again, if the solution starts to deviate. If this happens, the optimizer chooses between the two methods, to work toward the solution again.

If the optimizer does not attain the optimal solution, it prints both an error message, and a large Marquardt Scaling Parameter value.

Number of Function Evaluations

This is the number of analyses (for example, finite difference or central difference) needed, to find a minimum of the function.

Number of Iterations

This is the number of iterations needed, to find the optimized or actual solution.

Optimized Parameters OPTRC

*			%NORM-SEN	%CHANGE
.PARAM RX	=	6.7937	\$ 54.5260	50.2976M
.PARAM CX	=	147.3697M	\$ 45.4740	33.7653M

Figure 13-22: Power Dissipation and Time Constant (VOLT) RCOPT.TR0 = Before Optimization, RCOPT.TR1 = Optimized Result

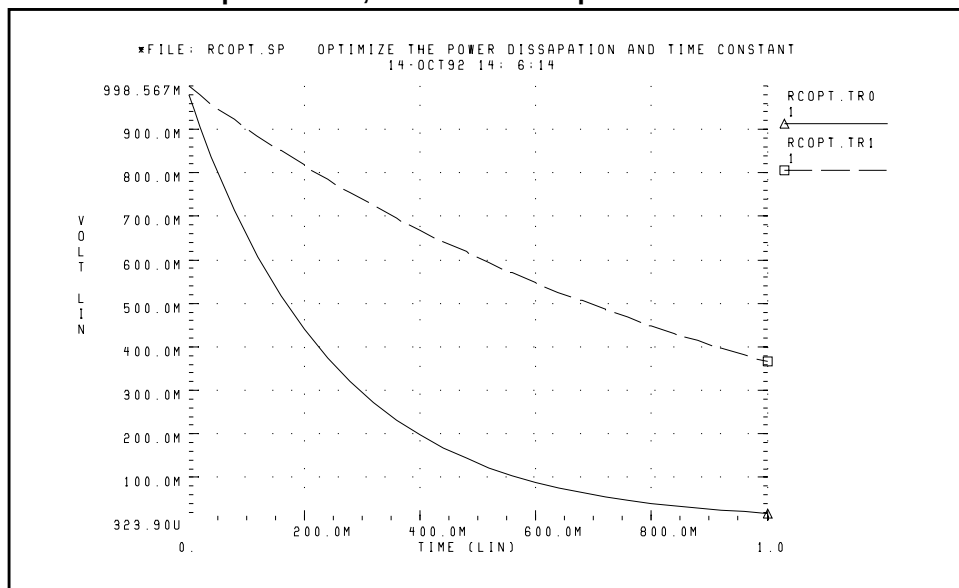
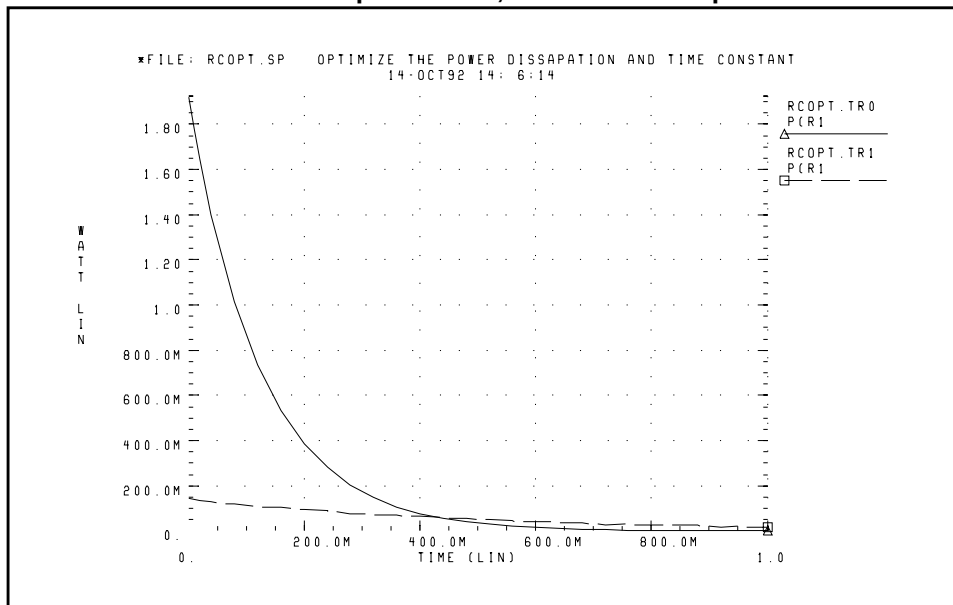


Figure 13-23: Power Dissipation and Time Constant (WATT)
RCOPT.TR0 = Before Optimization, RCOPT.TR1 = Optimized Result



Optimizing CMOS Tristate Buffer

The example circuit is an inverting CMOS tristate buffer. The design targets are:

1. Rising edge delay of 5 ns (input 50% voltage, to output 50% voltage).
2. Falling edge delay of 5 ns (input 50% voltage, to output 50% voltage).
3. RMS power dissipation should be as low as possible.
4. Output load consists of:
 - pad capacitance
 - leadframe inductance
 - 50 pF capacitive load

The Star-Hspice strategy is:

- Simultaneously optimize both the rising and falling delay buffer.
- Set up the internal power supplies, and the tristate enable, as global nodes.
- Optimize all device widths, except:
 - Initial inverter (assumed to be standard size).
 - Tristate inverter, and part of the tristate control (optimizing is not sensitive to this path).
- Perform an initial transient analysis, for plotting purposes. Then optimize and perform a final transient analysis, for plotting purposes.
- To use a weighted RMS power measure, specify an unrealistically-low power goal. Then use MINVAL to attenuate the error.

Input Netlist File, to Optimize a CMOS Tristate Buffer

```
*Tri-State input/output Optimization
.OPTION defl=1.2u nomod post=2
+ relv=1e-3 absvar=.5 relvar=.01
```

Circuit Input

```
.global lgnd lvcc enb
.macro buff data out
mp1 DATAN DATA LVCC LVCC p w=35u
mn1 DATAN DATA LGND LGND n w=17u

mp2 BUS DATAN LVCC LVCC p w=wp2
mn2 BUS DATAN LGND LGND n w=wn2
mp3 PEN PENN LVCC LVCC p w=wp3
mn3 PEN PENN LGND LGND n w=wn3
mp4 NEN NENN LVCC LVCC p w=wp4
mn4 NEN NENN LGND LGND n w=wn4

mp5 OUT PEN LVCC LVCC p w=wp5 l=1.8u
mn5 OUT NEN LGND LGND n w= wn5 l=1.8u

mp10 NENN BUS LVCC LVCC p w=wp10
mn12 PENN ENB NENN LGND n w=wn10
mn10 PENN BUS LGND LGND n w=wn10
mp11 NENN ENB LVCC LVCC p w=wp11
mp12 NENN ENBN PENN LVCC p w=wp11
mn11 PENN ENBN LGND LGND n w=80u
mp13 ENBN ENB LVCC LVCC p w=35u
mn13 ENBN ENB LGND LGND n w=17u

cbus BUS LGND 1.5pf
cpad OUT LGND 5.0pf

.ends
```

```

* * input signals *
vcc VCC GND 5V
lvcc vcc lvcc 6nh
lgnd lgnd gnd 6nh
vin DATA LGND pl (0v 0n, 5v 0.7n)
vinb DATABar LGND pl (5v 0n, 0v 0.7n)
ven ENB GND 5V
** circuit **
x1 data out buff
cext1 out GND 50pf
x2 databar outbar buff
cext2 outbar GND 50pf

```

Optimization Parameters

```

.param
+ wp2=opt1(70u,30u,330u)
+ wn2=opt1(22u,15u,400u)
+ wp3=opt1(400u,100u,500u)
+ wn3=opt1(190u,80u,580u)
+ wp4=opt1(670u,150u,800u)
+ wn4=opt1(370u,50u,500u)
+ wp5=opt1(1200u,1000u,5000u)
+ wn5=opt1(600u,400u,2500u)
+ wp10=opt1(240u,150u,450u)
+ wn10=opt1(140u,30u,280u)
+ wp11=opt1(240u,150u,450u)

```

Control Section

```

.tran 1ns 15ns
.tran .5ns 15ns sweep optimize=opt1
+ results=tfopt,tropt,rmspowo model=optmod
** put soft limit for power with minval setting (i.e. values
** less than 1000mw are less important)
.measure rmspowo rms power goal=100mw minval=1000mw
.meas tran tfopt trig v(data) val=2.5 rise=1 targ v(out)
+ val=2.5 fall=1 goal 5.0n
.meas tran tropt trig v(databar) val=2.5 fall=1 targ
+ v(outbar) val=2.5 rise=1 goal 5.0n
.model optmod opt itropt=30 max=1e+5
.tran 1ns 15ns
* output section *
*.plot tran v(data) v(out)
*.plot tran v(databar) v(outbar)

```


Model Section

```
.MODEL N NMOS LEVEL=3 VTO=0.7 U0=500 KAPPA=.25 KP=30U
+ ETA=.03 THETA=.04 VMAX=2E5 NSUB=9E16 TOX=500E-10
+ GAMMA=1.5 PB=0.6 JS=.1M XJ=0.5U LD=0.0 NFS=1E11 NSS=2E10
+ CGSO=200P CGDO=200P CGB0=300P

.MODEL P PMOS LEVEL=3 VTO=-0.8 U0=150 KAPPA=.25 KP=15U
+ ETA=.03 THETA=.04 VMAX=5E4 NSUB=1.8E16 TOX=500E-10
+ NFS=1E11 GAMMA=.672 PB=0.6 JS=.1M XJ=0.5U LD=0.0
+ NSS=2E10 CGSO=200P CGDO=200P CGB0=300P

.end
```

Optimization Results

```
residual sum of squares      = 2.388803E-02
norm of the gradient         = 0.769765
marquardt scaling parameter = 12624.2
no. of function evaluations  = 175
no. of iterations            = 23
```

Optimization Completed

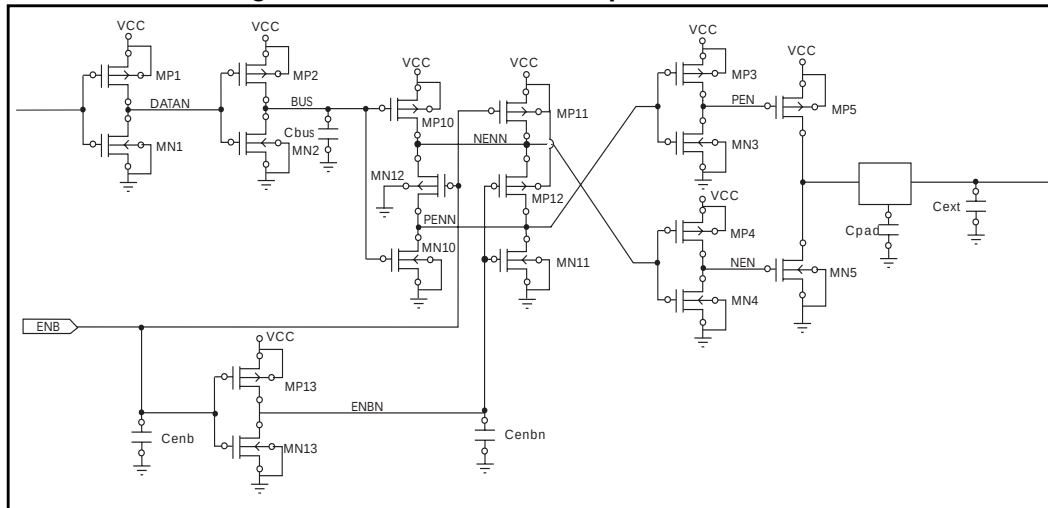
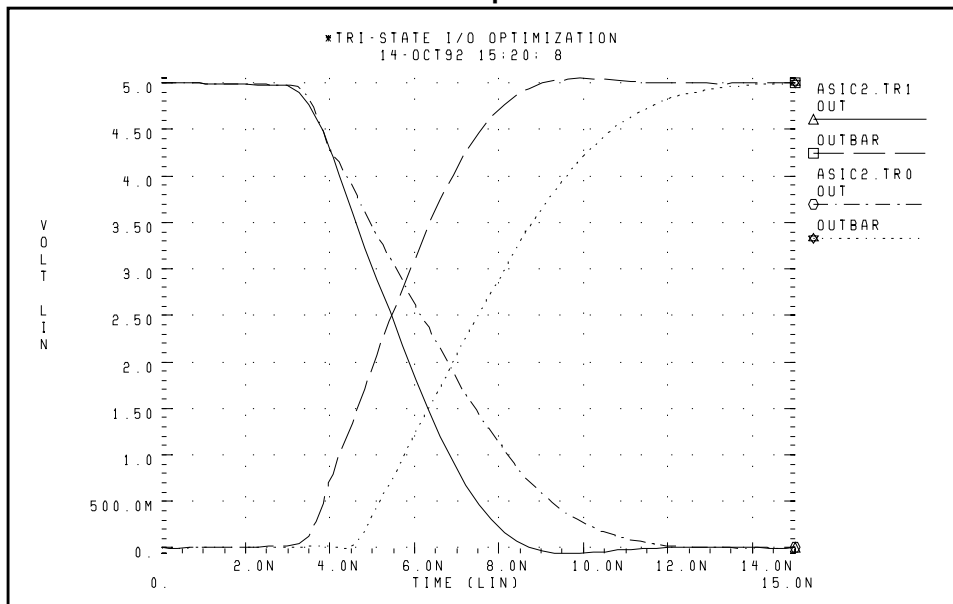
Parameters relin= 1.0000E-03 on last iterations

Optimized Parameters OPT1

```
*                                     %norm-sen  %change

.param wp2          = 84.4981u  $ 22.5877  -989.3733u
.param wn2          = 34.1401u  $ 7.6568   -659.2874u
.param wp3          = 161.7354u $ 730.7865m -351.7833u
.param wn3          = 248.6829u $ 8.1362   -2.2416m
.param wp4          = 238.9825u $ 1.2798   -1.5774m
.param wn4          = 61.3509u  $ 315.4656m 43.5213m
.param wp5          = 1.7753m   $ 4.1713   2.1652m
.param wn5          = 1.0238m   $ 5.8506   413.9667u
.param wp10         = 268.3125u $ 8.1917   -2.0266m
.param wn10         = 115.6907u $ 40.597   -422.8411u
.param wp11         = 153.1344u $ 482.0655m -30.6813m

*** optimize results measure names and values
* tfopt             = 5.2056n
* tropt             = 5.5513n
* rmspowo           = 200.1808m
```

Figure 13-24: Tristate Buffer Optimization Circuit**Figure 13-25: Tristate Input/Output Optimization ACIC2B.TR0 = Before Optimization, ACIC2B.TR1 = Optimized Result**

BJT S Parameters Optimization

The following example optimizes the S parameters, to match those specified for a set of measurements. The .DATA MEASURED in-line data statement contains these measured S parameters, as a function of frequency. The model parameters of the microwave transistor (LBB, LCC, LEE, TF, CBE, CBC, RB, RE, RC, and IS) vary. As a result, the measured S parameters (in the .DATA statement) match the calculated S parameters, from the simulation results.

This optimization uses a 2n6604 microwave transistor, and an equivalent circuit that consists of a BJT, with parasitic resistances and inductances. The BJT is biased at a 10 mA collector current (0.1 mA base current at DC bias and bf=100).

Key Star-Hspice Features Used

- NET command, to simulate network analyzer action.
- AC optimization.
- Optimized element and model parameters.
- Optimizing, compares measured S parameters to calculated parameters.
- S parameters, used in magnitude and phase (real and imaginary available).
- Weighting of data-driven frequency, versus S parameter table. Used for the phase domain.

Input Netlist File, for Optimizing BJT S Parameters

```
* BJTOPT.SP BJT S PARAMETER OPTIMIZATION
.OPTION ACCT NOMOD POST=2
```

BJT Equivalent Circuit Input

```
* NET COMMAND AUTOMATICALLY REVERSES THE SIGN OF THE POWER
* SUPPLY CURRENT, FOR NETWORK CALCULATIONS
.NET I(VCE) IBASE ROUT=50 RIN=50
VCE VCE 0 10V
IBASE 0 IIN AC=1 DC=.1MA
LBB IIN BASE LBB
LCC VCE COLLECT LCC
LEE EMIT 0 LEE
Q1 COLLECT BASE EMIT T2N6604
.MODEL T2N6604 NPN RB=RB BF=100 TF=TF CJE=CBE CJC=CBC
+ RE=RE RC=RC IS=IS
.PARAM
+ LBB= OPT1(100P, 1P, 10N)
```

```

+ LCC= OPT1(100P, 1P, 10N)
+ LEE= OPT1(100P, 1P, 10N)
+ TF = OPT1(1N, 5P, 5N)
+ CBE= OPT1(.5P, .1P, 5P)
+ CBC= OPT1(.4P, .1P, 5P)
+ RB= OPT1(10, 1, 300)
+ RE= OPT1(.4, .01, 5)
+ RC= OPT1(10, .1, 100)
+ IS= OPT1(1E-15, 1E-16, 1E-10)
.AC DATA=MEASURED OPTIMIZE=OPT1
+ RESULTS=COMP1,COMP3,COMP5,COMP6,COMP7
+ MODEL=CONVERGE

.MODEL CONVERGE OPT RELIN=1E-4 RELOUT=1E-4 CLOSE=100
+ ITROPT=25
.MEASURE AC COMP1 ERR1 PAR(S11M) S11(M)
.MEASURE AC COMP2 ERR1 PAR(S11P) S11(P) MINVAL=10
.MEASURE AC COMP3 ERR1 PAR(S12M) S12(M)
.MEASURE AC COMP4 ERR1 PAR(S12P) S12(P) MINVAL=10
.MEASURE AC COMP5 ERR1 PAR(S21M) S21(M)
.MEASURE AC COMP6 ERR1 PAR(S21P) S21(P) MINVAL=10
.MEASURE AC COMP7 ERR1 PAR(S22M) S22(M)

.AC DATA=MEASURED
.PRINT PAR(S11M) S11(M) PAR(S11P) S11(P)
.PRINT PAR(S12M) S12(M) PAR(S12P) S12(P)
.PRINT PAR(S21M) S21(M) PAR(S21P) S21(P)
.PRINT PAR(S22M) S22(M) PAR(S22P) S22(P)

.DATA MEASURED
FREQ  S11M  S11P  S21M  S21P  S12M  S12P  S22M  S22P
100ME .6    -52   19.75  148   .02   65   .87   - 21
200ME .56   -95   15.30  127   .032  49   .69   - 33
500ME .56   -149  7.69   97    .044  41   .45   - 41
1000ME .58   -174  4.07   77    .061  42   .39   - 47
2000ME .61   159   2.03   50    .095  40   .39   - 70
.ENDDATA

.PARAM FREQ=100ME S11M=0, S11P=0, S21M=0, S21P=0, S12M=0,
+ S12P=0, S22M=0, S22P=0
.END

```

Optimization Results

```

RESIDUAL SUM OF SQUARES      = 5.142639e-02
NORM OF THE GRADIENT         = 6.068882e-02
MARQUARDT SCALING PARAMETER  = 0.340303
CO. OF FUNCTION EVALUATIONS  = 170
NO. OF ITERATIONS            = 35

```

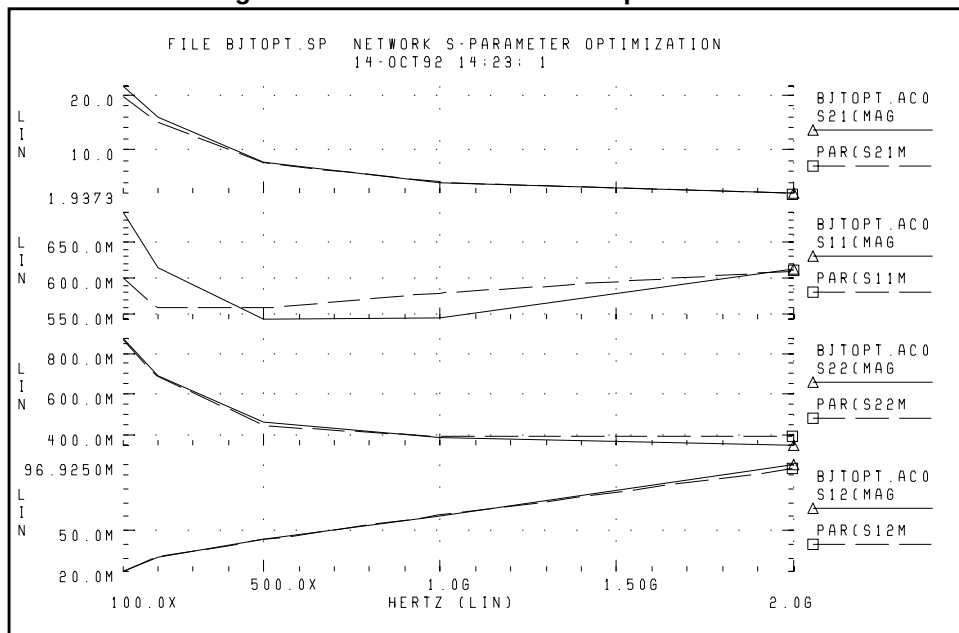
The maximum number of iterations (25) was exceeded. However, the results probably are accurate. Increase ITROPT accordingly.

Optimized Parameters OPT1- Final Values

***OPTIMIZED PARAMETERS OPT1 SENS %NORM-SEN

.PARAM	LBB	=	1.5834N	\$	27.3566X	2.4368
.PARAM	LCC	=	2.1334N	\$	12.5835X	1.5138
.PARAM	LEE	=	723.0995P	\$	254.2312X	12.3262
.PARAM	TF	=	12.7611P	\$	7.4344G	10.0532
.PARAM	CBE	=	620.5195F	\$	23.0855G	1.5300
.PARAM	CBC	=	1.0263P	\$	346.0167G	44.5016
.PARAM	RB	=	2.0582	\$	12.8257M	2.3084
.PARAM	RE	=	869.8714M	\$	66.8123M	4.5597
.PARAM	RC	=	54.2262	\$	3.1427M	20.7359
.PARAM	IS	=	99.9900P	\$	3.6533X	34.4463M

Figure 13-26: BJT-S Parameter Optimization



BJT Model DC Optimization

The goal is to match the forward and reverse Gummel plots, obtained from a HP4145 semiconductor analyzer, using the True-Hspice LEVEL=1 Gummel-Poon BJT model. Because the Gummel plots are at low base currents, Star-Hspice does not optimize the base resistance. Star-Hspice also does not optimize the forward and reverse Early voltages (VAF and VAR), because simulation did not measure VCE data.

The key feature, used in this optimization, is incremental optimization.

1. Star-Hspice first optimizes the forward-Gummel data points.
2. Star-Hspice updates the forward-optimized parameters, into the model.

After updating, you cannot change these parameters.

3. Star-Hspice next optimizes the reverse-Gummel data points.

BJT Model DC Optimization Input Netlist File

```
* FILE OPT_BJT.SP BJT OPTIMIZATION T2N3947
* OPTIMIZE THE DC FORWARD AND REVERSE CHARACTERISTICS
+ FROM A GUMMEL PLOT
* ALL DC GUMMEL-POON DC PARAMETERS EXCEPT BASE
+ RESISTANCE AND EARLY VOLTAGES OPTIMIZED
*

$. .TIGHTEN THE SIMULATOR CONVERGENCE PROPERTIES
.OPTION NOMOD INGOLD=2 NOPAGE VNTOL=1E-10 POST
+ NUMDGT=5 RELI=1E-4 RELV=1E-4

$. .OPTIMIZATION CONVERGENCE CONTROLS
.MODEL OPTMOD OPT RELIN=1E-4 ITROPT=30 GRAD=1E-5 CLOSE=10
+ CUT=2 CENDIF=1E-6 RELOUT=1E-4 MAX=1E6
```

Room Temp Device

```
VBER BASE 0 VBE
VBCR BASE COL VBC
Q1 COL BASE 0 BJTMOD
```

Model and Initial Estimates

```

.MODEL BJTMOD NPN
+ ISS = 0. XTF = 1. NS = 1.
+ CJS = 0. VJS = 0.50000 PTF = 0.
+ MJS = 0. EG = 1.10000 AF = 1.
+ ITF = 0.50000 VTF = 1.00000
+ FC = 0.95000 XCJC = 0.94836
+ SUBS = 1
+ TF=0.0 TR=0.0 CJE=0.0 CJC=0.0 MJE=0.5 MJC=0.5 VJE=0.6
+ VJC=0.6 RB=0.3 RC=10 VAF=550 VAR=300
$. .THESE ARE THE OPTIMIZED PARAMETERS
+ BF=BF IS=IS IKF=IKF ISE=ISE RE=RE
+ NF=NF NE=NE
$. .THESE ARE FOR REVERSE BASE OPT
+ BR=BR IKR=IKR ISC=ISC
+ NR=NR NC=NC

.PARAM VBE=0 IB=0 IC=0 VCE_EMIT=0 VBC=0 IB_EMIT=0 IC_EMIT=0
+ BF= OPT1(100, 50, 350)
+ IS= OPT1(5E-15, 5E-16, 1E-13)
+ NF= OPT1(1.0, 0.9, 1.1)
+ IKF=OPT1(50E-3, 1E-3, 1)
+ RE= OPT1(10, 0.1, 50)
+ ISE=OPT1(1E-16, 1E-18, 1E-11)
+ NE= OPT1(1.5, 1.2, 2.0)
+ BR= OPT2(2, 1, 10)
+ NR= OPT2(1.0, 0.9, 1.1)
+ IKR=OPT2(50E-3, 1E-3, 1)
+ ISC=OPT2(1E-12, 1E-15, 1E-10)
+ NC= OPT2(1.5, 1.2, 2.0)

.DC DATA=BASEF SWEEP OPTIMIZE=OPT1 RESULTS=IBVBE,ICVBE
+ MODEL=OPTMOD

.MEAS DC IBVBE ERR1 PAR(IB) I2(Q1) MINVAL=1E-14
+ IGNORE=1E-16

.MEAS DC ICVBE ERR1 PAR(IC) I1(Q1) MINVAL=1E-14
+ IGNORE=1E-16

.DC DATA=BASER SWEEP OPTIMIZE=OPT2 RESULTS=IBVBER,ICVBER
+ MODEL=OPTMOD

.MEAS DC IBVBER ERR1 PAR(IB) I2(Q1) MINVAL=1E-14
+ IGNORE=1E-16

.MEAS DC ICVBER ERR1 PAR(IC) I1(Q1) MINVAL=1E-14
+ IGNORE=1E-16

.DC DATA=BASEF
.PRINT DC PAR(IC) I1(Q1) PAR(IB) I2(Q1)

.DC DATA=BASER
.PRINT DC PAR(IC) I1(Q1) PAR(IB) I2(Q1)

```

Optimization Results OPT1

RESIDUAL SUM OF SQUARES = 2.196240E-02

Optimized Parameters OPT1

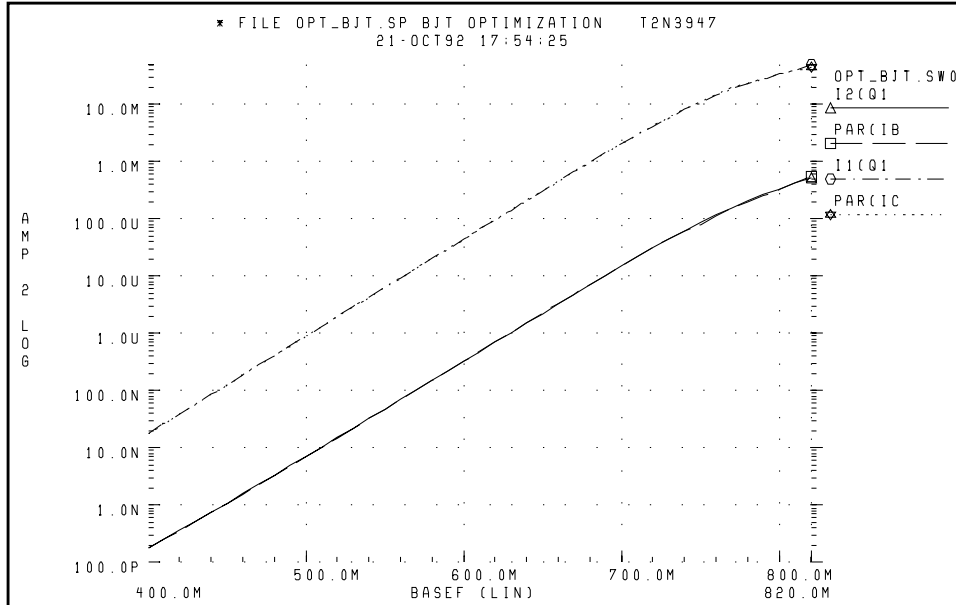
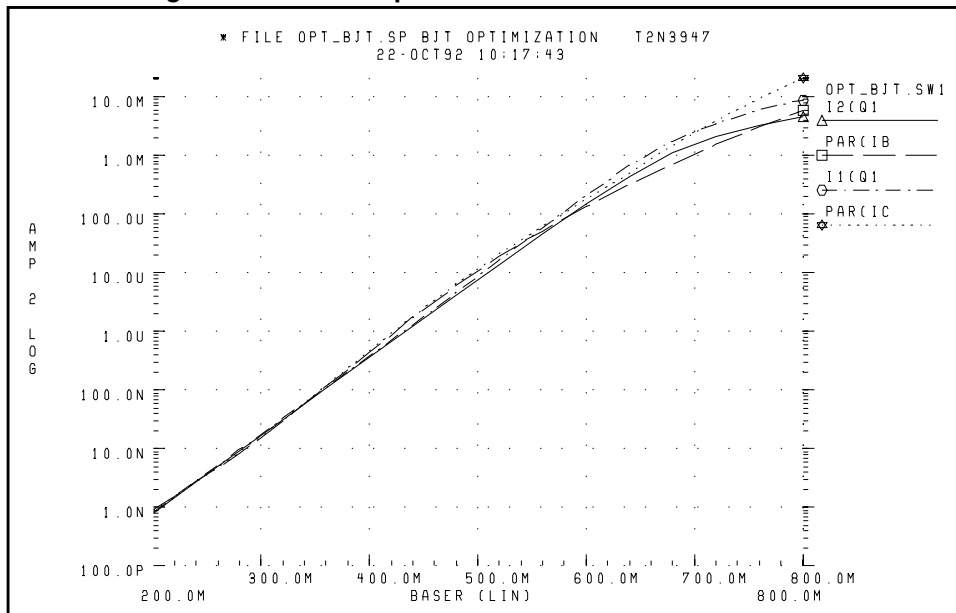
+ %NORM-SEN %CHANGE
.PARAM BF = 1.4603E+02 \$ 2.7540E+00 -7.3185E-07
.PARAM IS = 2.8814E-15 \$ 3.7307E+00 -5.0101E-07
.PARAM NF = 9.9490E-01 \$ 9.1532E+01 -1.0130E-08
.PARAM IKF = 8.4949E-02 \$ 1.3782E-02 -8.8082E-08
.PARAM RE = 6.2358E-01 \$ 8.6337E-02 -3.7665E-08
.PARAM ISE = 5.0569E-16 \$ 1.0221E-01 -3.1041E-05
.PARAM NE = 1.3489E+00 \$ 1.7806E+00 2.1942E-07

Optimization Results OPT2

RESIDUAL SUM OF SQUARES = 1.82776

Optimized Parameters OPT2

* %NORM-SEN %CHANGE
.PARAM BR = 1.0000E+01 \$ 1.1939E-01 1.7678E+00
.PARAM NR = 9.8185E-01 \$ 1.4880E+01 -1.1685E-03
.PARAM IKR = 7.3896E-01 \$ 1.2111E-03 -3.5325E+01
.PARAM ISC = 1.8639E-12 \$ 6.6144E+00 -5.2159E-03
.PARAM NC = 1.2800E+00 \$ 7.8385E+01 1.6202E-03

Figure 13-27: BJT Optimization Forward Gummel Plots**Figure 13-28: BJT Optimization Reverse Gummel Plots**

Optimizing GaAsFET Model DC

This example circuit is a high-performance, GaAsFET transistor. The design target is to match HP4145 DC measured data, to the True-Hspice LEVEL=3 JFET model.

The Star-Hspice strategy is:

- MEASURE IDSERR is an ERR1 type function. It provides linear attenuation of the error results, starting at 20 mA. This function ignores all currents below 1 mA. The high-current fit is the most important for this model.
- The OPT1 function simultaneously optimizes all DC parameters.
- The .DATA statement merges raw data files TD1.dat and TD2.dat together.
- The graph plot model sets the MON0=1 parameter, to remove the retrace lines from the family of curves.

GaAsFET Model DC Optimization Input Netlist File

```
*FILE JOPT.SP JFET OPTIMIZATION
.OPTION ACCT NOMOD POST
+ RELI=2E-4 RELV=2E-4
VG GATE 0 XVGS
VD DRAIN 0 XVDS
J1 DRAIN GATE 0 JFETN1

.MODEL JFETN1 NJF LEVEL=3 CAPOP=1 SAT=3
+ NG=1
+ CGS=1P CGD=1P RG=1
+ VTO=VTO BETA=BETA LAMBDA=LAMBDA
+ RS=RDS RD=RDS IS=1E-15 ALPHA=ALPHA
+ UCRIT=UCRIT SATEXP=SATEXP
+ GAMDS=GAMDS VGEXP=VGEXP

.PARAM
+ VTO=OPT1(-.8,-4,0)
+ VGEXP=OPT1(2,1,3.5)
+ GAMDS=OPT1(0,-.5,0)
+ BETA= OPT1(6E-3, 1E-5,9E-2)
+ LAMBDA=OPT1(30M,1E-7,5E-1)
+ RDS=OPT1(1,.001,40)
+ ALPHA=OPT1(2,1,3)
+ UCRIT=OPT1(.1,.001,1)
+ SATEXP=OPT1(1,.5,3)

.DC DATA=DESIRED OPTIMIZE=OPT1 RESULTS=IDSERR MODEL=CONV
.MODEL CONV OPT RELIN=1E-4 RELOUT=1E-4 CLOSE=100 ITROPT=25
.MEASURE DC IDSERR ERR1 PAR(XIDS) I(J1) MINVAL=20M
+ IGNORE=1M
.DC DATA=DESIRED
```

```
.GRAPH PAR(XIDS) I(J1)
.MODEL GRAPH PLOT MONO=1
.PRINT PAR(XVGS) PAR(XIDS) I(J1)

.DATA DESIRED MERGE
+ FILE=JDC.DAT XVDS=1 XVGS=2 XIDS=3
.ENDDATA
.END
```

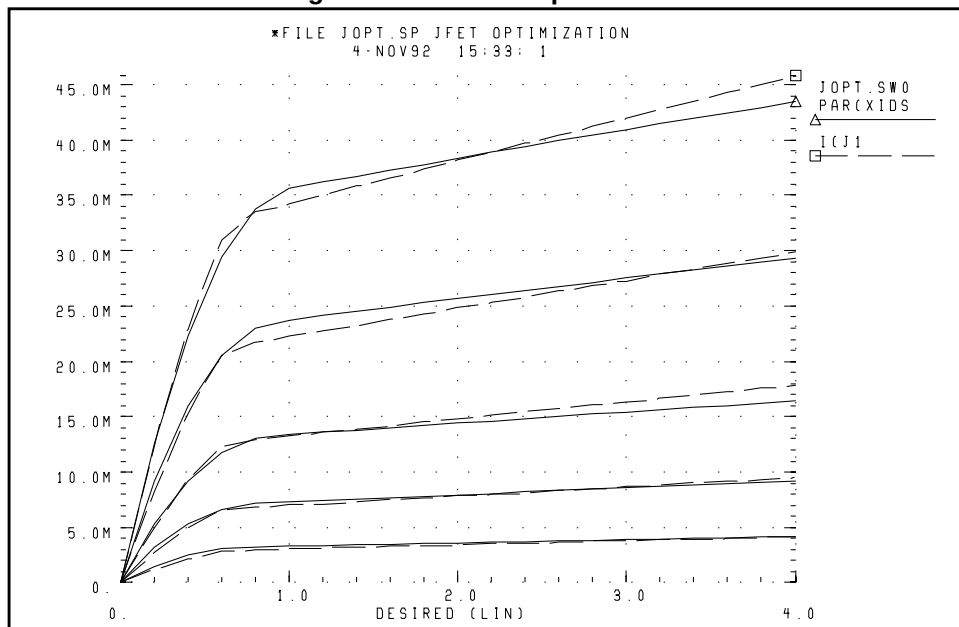
Optimization Results

RESIDUAL SUM OF SQUARES = 7.582202E-02

Optimized Parameters Opt1

* %NORM-SEN	%CHANGE			
.PARAM VTO	= -1.1067	\$	64.6110	43.9224M
.PARAM VGEXP	= 2.9475	\$	13.2024	219.4709M
.PARAM GAMDS	= 0.	\$	0.	0.
.PARAM BETA	= 11.8701M	\$	17.2347	136.8216M
.PARAM LAMBDA	= 138.9821M	\$	2.2766	-1.5754
.PARAM RDS	= 928.3216M	\$	704.3204M	464.0863M
.PARAM ALPHA	= 2.2914	\$	728.7492M	168.4004M
.PARAM UCRIT	= 1.0000M	\$	18.2438M	-125.0856
.PARAM SATEXP	= 1.4211	\$	1.2241	2.2218

Figure 13-29: JFET Optimization



Optimizing MOS Op-amp

The design goals for the MOS operational amplifier are:

- Minimize the gate area (and therefore the total cell area).
- Minimize the power dissipation.
- Open-loop transient step response of 100 ns, for rising and falling edges.

The Star-Hspice strategy is:

- Simultaneously optimize two amplifier cells, for rising and falling edges.
- Total power is power for two cells.
- The optimization transient analysis must be longer, to allow for a range of values in intermediate results.
- All transistor widths and lengths are optimized.
- Calculate the transistor area algebraically, use a voltage value, and minimize the resulting voltage.
- The transistor area measure statement uses MINVAL, which assigns less weight to the area minimization.
- Optimizes the bias voltage.

MOS Op-amp Optimization Input Netlist File

```
AMPOPT.SP MOS OPERATIONAL AMPLIFIER OPTIMIZATION
.OPTION RELV=1E-3 RELVAR=.01 NOMOD ACCT POST
.PARAM VDD=5 VREF='VDD/2'
VDD VSUPPLY 0 VDD
VIN+ VIN+ 0 PWL(0 , 'VREF-10M' 10NS 'VREF+10M' )
VINBAR+ VINBAR+ 0 PWL(0 , 'VREF+10M' 10NS 'VREF-10M' )
VIN- VIN- 0 VREF
VBIAS VBIAS 0 BIAS
.GLOBAL VSUPPLY VBIAS
XRISE VIN+ VIN- VOUTR AMP
CLOADR VOUTR 0 .4P
XFALL VINBAR+ VIN- VOUTF AMP
CLOADF VOUTF 0 .4P

.MACRO AMP VIN+ VIN- VOUT
M1 2 VIN- 3 3 MOSN W=WM1 L=LM
M2 4 VIN+ 3 3 MOSN W=WM1 L=LM
M3 2 2 VSUPPLY VSUPPLY MOSP W=WM1 L=LM
M4 4 2 VSUPPLY VSUPPLY MOSP W=WM1 L=LM
M5 VOUT VBIAS 0 0 MOSN W=WM5 L=LM
M6 VOUT 4 VSUPPLY VSUPPLY MOSP W=WM6 L=LM
M7 3 VBIAS 0 0 MOSN W=WM7 L=LM
.ENDS
```

```

.PARAM AREA='4*WM1*LM + WM5*LM + WM6*LM + WM7*LM'
VX 1000 0 AREA
RX 1000 0 1K

.MODEL MOSP PMOS (VT0=-1 KP=2.4E-5 LAMBDA=.004
+ GAMMA =.37 TOX=3E-8 LEVEL=3)

.MODEL MOSN NMOS (VT0=1.2 KP=6.0E-5 LAMBDA=.0004
+ GAMMA =.37 TOX=3E-8 LEVEL=3)

.PARAM WM1=OPT1(60U,20U,100U)
+ WM5=OPT1(40U,20U,100U)
+ WM6=OPT1(300U,20U,500U)
+ WM7=OPT1(70U,40U,200U)
+ LM=OPT1(10U,2U,100U)
+ BIAS=OPT1(2.2,1.2,3.0)

.TRAN 2.5N 300N SWEEP OPTIMIZE=OPT1
+ RESULTS=DELAYR,DELAYF,TOT_POWER,AREA MODEL=OPT
.MODEL OPT OPT CLOSE=100

.TRAN 2N 150N
.MEASURE DELAYR TRIG AT=0 TARG V(VOUTR) VAL=2.5 RISE=1
+ GOAL=100NS

.MEASURE DELAYF TRIG AT=0 TARG V(VOUTF) VAL=2.5 FALL=1
+ GOAL=100NS

.MEASURE TOT_POWER AVG POWER GOAL=10MW
.MEASURE AREA MIN PAR(AREA) GOAL=1E-9 MINVAL=100N
.PRINT V(VIN+) V(VOUTR) V(VOUTF)

.END

```

Optimization Results

```

RESIDUAL SUM OF SQUARES = 4.654377E-04
NORM OF THE GRADIENT    = 6.782920E-02

```

Optimized Parameters Opt1

```

*          %NORM-SEN%CHANGE

.PARAM WM1          = 47.9629U $ 1.6524 -762.3661M
.PARAM WM5          = 66.8831U $ 10.1048 23.4480M
.PARAM WM6          = 127.1928U $ 12.7991 22.7612M
.PARAM WM7          = 115.8941U $ 9.6104 -246.4540M
.PARAM LM           = 6.2588U $ 20.3279 -101.4044M
.PARAM BIAS         = 2.7180 $ 45.5053 5.6001M

*** OPTIMIZE RESULTS MEASURE NAMES AND VALUES
* DELAYR            = 100.4231N
* DELAYF            = 99.5059N
* TOT_POWER         = 10.0131M
* AREA              = 3.1408N

```

Figure 13-30: CMOS Op-amp

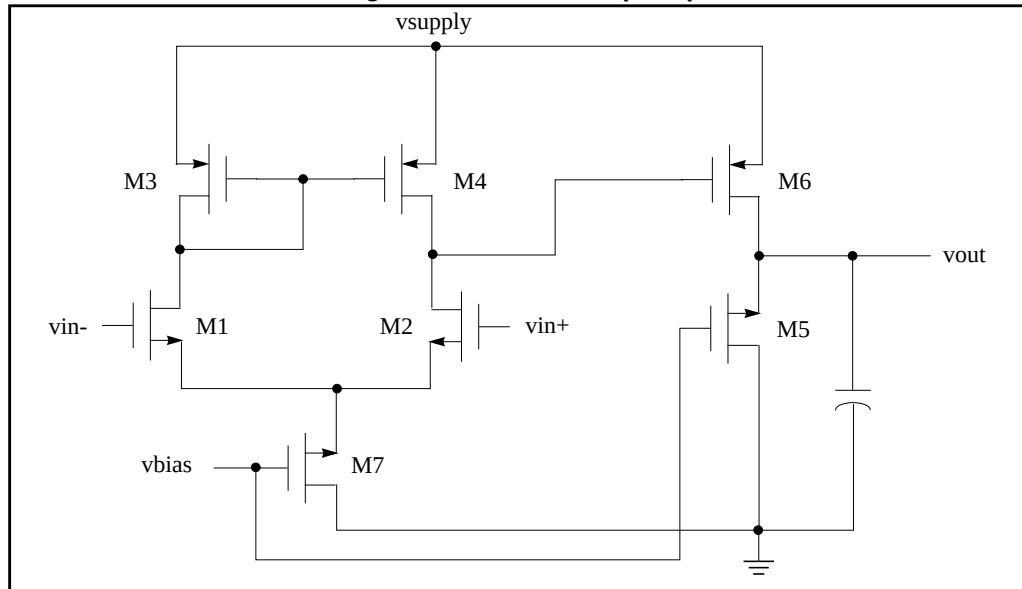
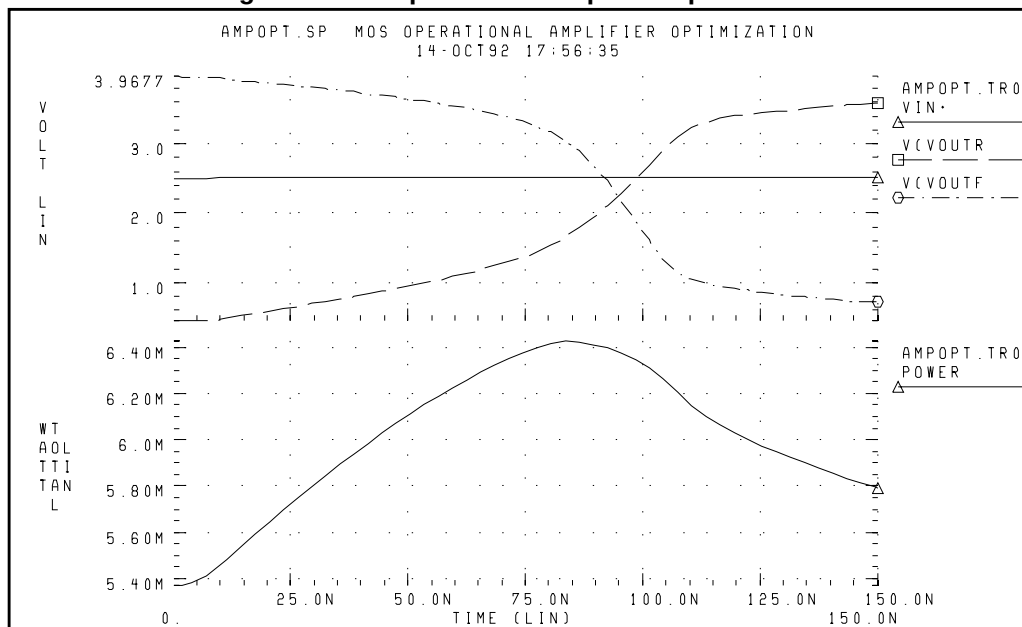


Figure 13-31: Operational Amplifier Optimization





Chapter 14

Common Model Interface

The Common Model Interface (CMI) is an Avant! program interface that you can use to add proprietary models into the Star-Hspice simulator.

This chapter explains the following topics:

- [Overview of CMI](#)
- [Directory Structure](#)
- [Running Simulations with CMI Models](#)
- [Adding Proprietary MOS Models](#)
- [Testing CMI Models](#)
- [Model Interface Routines](#)
- [Interface Variables](#)
- [Internal Routines](#)
- [Extended Topology](#)
- [Conventions](#)

Overview of CMI

Star-Hspice uses a dynamically-linked shared library, to integrate models with CMI. Add the `cmiflag` global option, to load the dynamically-linked CMI library (`libCMImodel`). Simulation searches for the `libCMImodel` shared library, in the `$hspice_lib_models` path. If the simulator does not find the library, it searches in `$installdir/$ARCH/lib/models`.

Dynamic loading shares resources, more efficiently than static binding does. If you run several simulations concurrently, Star-Hspice needs only one copy of the dynamically-linked CMI model in memory, which speeds-up the process. Theoretically, the static-linking version is always slightly faster, if you run only one simulation at a time. However, the performance difference between dynamic loading and static binding is usually less than 5%.

CMI includes several source code examples, for integration of MOS, JFET, and MESFET models in simulation. They are standard Berkeley SPICE MOSFET models (LEVEL 1, 2, 3, BSIM1, 2, 3) and JFET/MESFET models. To minimize the effort required for adding models, Avant! provides installation scripts, which automate the shared-library generation process.

If you derive your proprietary models from SPICE models, the integration process is similar to the examples, with minimal modifications.

Note: Star-Hspice includes equations and programs for bias calculation, numerical integration, convergence checking, and matrix loading. You do not need to use these programs to complete a new model integration, so the source code examples do not include them.

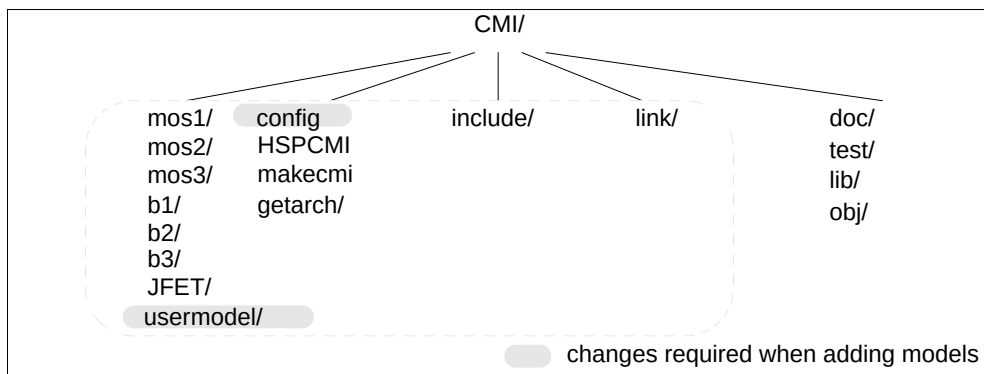
CMI supports the following platforms:

- Sun Solaris 2.5, 2.7, 2.8
- HP-UX 10.20, 11.0
- RedHat Linux6.2, Linux7.0, Linux7.1
- Windows95, Windows98, Windows2000, Windows NT, and Windows XP.

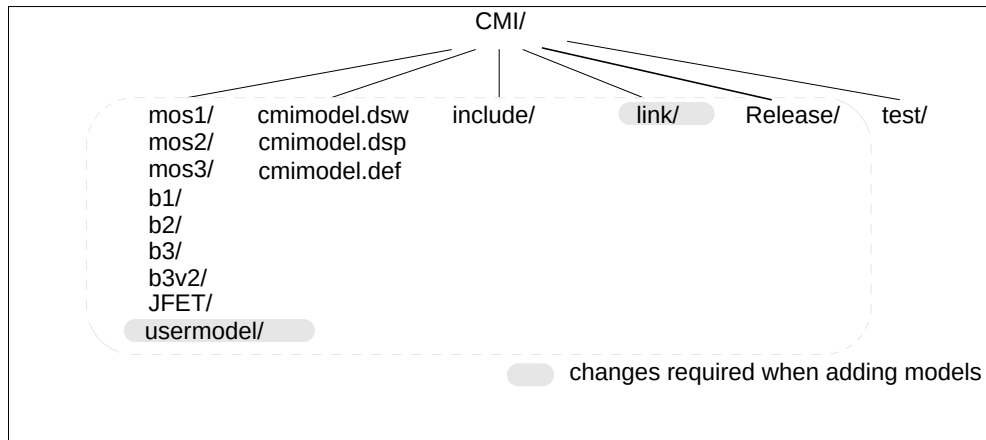
Directory Structure

Figure 14-1 shows the structure of the CMI distribution for Unix (Sun and HP) and Linux platforms. Figure 14-2 shows the structure of the CMI distribution for PC (Windows 95, Win98, Win2000, Windows NT, and Windows XP) platforms. You must modify the shaded files, or add new ones, for new models

Figure 14-1: CMI Directory Structure, Unix/Linux Platforms.



HSPCMI	Subdirectory, containing the utility that processes configuration files and makefiles.
get_arch	C shell script, for identifying platforms.
config	Configuration file.
doc/	CMI documentation.
link/	Main CMI routines.
include/	CMI header files.
makecmi	Master makefile.
test/	Model testing example.
lib	Shared library directory.
obj	Object code.
mos1/, mos2/, mos3, b1,b2/,b3, JFET	Model directories.

Figure 14-2: CMI Directory Structure, PC Platforms.

cmimodel.dsw

Project workspace file.

cmimodel.dsp

Project file.

cmimodel.def

Definition file.

include/

CMI header files.

link/

Main CMI routines.

test/

Model testing example.

Release/

Shared library directory.

mos1/, mos2/, mos3, b1,b2/,b3, b3v2/, JFET

Model directories.

Running Simulations with CMI Models

To specify CMI models, use the *level* model parameter. Levels used in the example models are (same as those in Berkeley Spice-3):

- | | |
|--------------------------|---------------------|
| ■ <i>LEVEL 1 (mos1/)</i> | LEVEL 1 MOS model |
| ■ <i>LEVEL 2 (mos2/)</i> | LEVEL 2 MOS model |
| ■ <i>LEVEL 3 (mos3/)</i> | LEVEL 3 MOS model |
| ■ <i>LEVEL 4 (b1/)</i> | BSIM model |
| ■ <i>LEVEL 5 (b2/)</i> | BSIM2 model |
| ■ <i>LEVEL 8 (b3/)</i> | BSIM3v3 model |
| ■ <i>LEVEL 9 (JFET)</i> | JFET & MESFET model |

To perform a simulation run on a CMI model, add the following line in the input netlist:

```
.OPTION cmiflag
```

The LEVEL 8 example code (located in the *b3* directory) and the True-Hspice LEVEL 49 model, are both based on BSIM3, version 3. However, the speed of LEVEL 8 is sometimes 20 percent slower than LEVEL 49, in True-Hspice models. This occurs because LEVEL 49 is carefully implemented, to ensure high accuracy and performance. In contrast, LEVEL 8 in the example code is only an example of CMI interface implementation. Therefore, the slower performance of the example code, compared to True-Hspice LEVEL 49, is expected.

The Level 9 example code is located in the JFET directory, and is based on True-Hspice JFET&MESFET Level 3 (Statz model) and Spice3. This is example code, only for CMI interface implementation.

Adding Proprietary MOS Models

You can use the CMI interface to enter proprietary models into Star-Hspice. This section describes how to use CMI, to add a new MOS model and simplify the integration process.

MOS Models on Unix Platforms

In the following examples, the percent sign (%) is the UNIX shell prompt, and `$(installdir)` points to the directory where you installed Star-Hspice. `$ARCH` is the OS type for the computer. CMI supports Sun4, Solaris, and HP platforms.

To create a CMI shared library and add a new model:

1. Create a directory environment.
2. Modify the configuration file.
3. Prepare and modify model routines.
4. Compile the shared library.
5. Set up the runtime shared-library path.

Creating the Directory Environment

To create the CMI directory environment:

1. Copy the CMI directory from the Star-Hspice release directory, to a new location, as shown in the following example:

```
% cp -r $(installdir)/cmi /home/user1/userx/model
```

The new CMI directory (`/home/user1/userx/model/cmi`) is your working directory. You must have read and write access to this directory.

2. Copy an existing model subdirectory to a new model directory, and create a subdirectory for the new model under the working CMI directory.

For example, if your MOSFET model is LEVEL 222, you can copy the subdirectory from the existing MOS model LEVEL 3, as follows:

```
%cp -r mos3 mos222
```

3. Add the following line in the `config` configuration file:

```
mos222      222      "my own MOSFET model"
```

where:

- *mos222* is the model name.
- 222 is the model LEVEL.
- "my own MOSFET model" is the descriptive comment for the model.

The model name and level must be unique, in the configuration file. For more information, see the in-line comment in the configuration file.

Preparing Model Routine Files

In the new *mos222* model subdirectory, rename *mos3* in all filenames to *mos222*. For example:

```
% mv CMImos3defs.h CMImos222defs.h
```

After you rename all files, the new model subdirectory should contain the following group of files:

- CMImos222defs.h
- CMImos222.c
- CMImos222GetIpar.c
- CMImos222SetIpar.c
- CMImos222GetMpar.c
- CMImos222SetMpar.c
- CMImos222eval.c
- CMImos222set.c
- CMImos222temp.c

For a detailed description of each routine, see [Model Interface Routines on page 14-13](#). You must modify the functions, as necessary. To add a new model, most of the work required is modifying these files.

Compiling the Shared Library

1. Follow the steps in the preceding section, to modify the model routine files and the configuration file.
2. Manually set the HSPICE_CMI environment variable to the working CMI directory (see [Creating the Directory Environment on page 14-6](#)):

```
% setenv HSPICE_CMI /home/user1/userx/model/cmi
```
3. Use a single *make* operation, to compile the model routines, and the shared library.

4. To check the syntax of your C functions, before you launch the compilation process, enter the following command:

```
% make -f makecmi lint
```

This command lists any syntax errors in your model routines.

5. To invoke the compilation process, enter the following:

```
% make -f makecmi
```

The simulator creates the new `libCMImodel` shared library, in the `lib/` subdirectory. It also generates all object files in the `obj/` subdirectory.

Note: During compilation, CMI creates files (`makefile.SUN` on SUN, `makefile.HP` on HP), and subdirectories (`obj/`, `lib/`), in the CMI working directory. Do **not** manually modify these generated files.

Choosing a Compiler

To use any functional compiler, set the `CC` environment variable to the location of the compiler (the auto-generated `makefile`, `makefile.SUN` or `makefile.HP`, uses `CC`). Set the appropriate compiler and link flags properly, so the final CMI library build is in position-independent code (PIC). Dynamic linking uses PIC.

For SUNOS 5.4 platforms or later, `-KPIC` (the automatically-generated compiler flag), and the `-G -z` link flag, are for the Sun workshop compiler (`cc` or `acc`). These compilers are typically installed in a directory, such as `/usr1/opt/SUNWsp/SC4.2/bin`.

For HP9000/700 platforms, `cc` is installed in `/opt/ansic/bin`. For more information, type `man cc` or `man acc`, to display the on-line manual page for these commands.

You can use any optimization flags for the CMI library, are not restricted. However, for best results, use the `-fast` flag for the `cc` or `acc` Sun compiler, and use the `-O` flag for the `cc` HP compiler.

Using the gcc Compiler

If you use the `gcc` compiler, modify the `makefile` (`makefile.SUN` or `makefile.HP`), to set the compiler flags correctly.

For *gcc*, set the *CC* environment variable to *gcc*, and modify the makefile to use the *-fPIC* flag for compiling, and the *-r* flag for linking. For example:

```
gcc -c -I../include -fPIC CMImain.c
...
gcc -r -o ../lib/libCMImodel obj/*.o
```

Using the */usr/ucb/cc* Compiler

If you use the */usr/ucb/cc* compiler, modify the makefile (*makefile.SUN* or *makefile.HP*), to set the compiler flags correctly.

The */usr/ucb/cc* compiler cannot compile C source files, until you install the Language Optional Source Package. Verify that this source package is installed, then set the *CC* environment variable to */usr/ucb/cc*.

Note: The Language Optional Source Package is not installed in Solaris by default, but it is installed in SUNOS 4.1.x by default. You must either install the optional language software, or use a workable compiler (such as *cc* or *acc* in the Sun workshop). You can also use the *gcc* compiler, with some minor modifications to the makefile.

Runtime Shared Library Path

The shared library is now ready to use. You must update the shared model search path (defined in the *hspice_lib_models* environment variable), so that the system dynamic loader can find the new CMI shared library. Enter the following:

```
setenv hspice_lib_models $HSPICE_CMI/lib
```

Troubleshooting

Sometimes, even if you successfully build the CMI dynamic library, when you run simulation, Star-Hspice returns an error message:

```
**error**: Unable to load
/home/ant/lib/models/libCMImodel and /home/ant/lib/
models/libCMImodel.so
```

The cause of this problem is usually symbols that are undefined when you run the simulation.

Note: Different compilers usually generate *nm* output, in different formats.

The following is an example output of undefined symbols, using Sun workshop cc as the compiler, on a SUNOS 5.5 machine.

```
nm libCMImodel | grep UNDEF
[35]          | 0 | 0 | NOTY | LOCL | 0 | UNDEF |
[34]          | 0 | 0 | NOTY | LOCL | 0 | UNDEF |
[1779]        | 0 | 0 | NOTY | GLOB | 0 | UNDEF | .mul
[1853]        | 0 | 0 | NOTY | GLOB | 0 | UNDEF | __dtou
[1810]        | 0 | 0 | NOTY | GLOB | 0 | UNDEF | __iob
[1822]        | 0 | 0 | NOTY | WEAK | 0 | UNDEF | _ex_deregister
[1749]        | 0 | 0 | NOTY | WEAK | 0 | UNDEF | _ex_register
[1857]        | 0 | 0 | NOTY | GLOB | 0 | UNDEF | atan
[1878]        | 0 | 0 | NOTY | GLOB | 0 | UNDEF | cos
[1820]        | 0 | 0 | NOTY | GLOB | 0 | UNDEF | exp
[1777]        | 0 | 0 | NOTY | GLOB | 0 | UNDEF | fabs
[1872]        | 0 | 0 | NOTY | GLOB | 0 | UNDEF | fprintf
[1764]        | 0 | 0 | NOTY | GLOB | 0 | UNDEF | log
[1767]        | 0 | 0 | NOTY | GLOB | 0 | UNDEF | malloc
[1842]        | 0 | 0 | NOTY | GLOB | 0 | UNDEF | memset
[1772]        | 0 | 0 | NOTY | GLOB | 0 | UNDEF | pow
[1869]        | 0 | 0 | NOTY | GLOB | 0 | UNDEF | sin
[1790]        | 0 | 0 | NOTY | GLOB | 0 | UNDEF | sqrt
[1758]        | 0 | 0 | NOTY | GLOB | 0 | UNDEF | strcasecmp
[1848]        | 0 | 0 | NOTY | GLOB | 0 | UNDEF | strcpy
[1858]        | 0 | 0 | NOTY | GLOB | 0 | UNDEF | strlen
[1800]        | 0 | 0 | NOTY | GLOB | 0 | UNDEF | strncpy
```

To satisfy the above undefined symbols when you run simulation, use libraries such as *libc* or *libm*. However, any unsatisfied symbols, other than those shown in the example, can cause the *Unable to load* problem.

MOS Models on PC Platforms

To add proprietary MOS models on a PC platform, follow the steps below to create a Dynamic Link Library.

1. Copy the CMI directory from the Star-Hspice release directory, to a new location. For example:

```
D:/xcopy d:\avanti\cmimodel project\cmimodel /e
```
2. Use Microsoft Visual C++ to open the cmimodel/cmimodel.dsw file.

3. In the `cmimodel/project` directory, create a new subdirectory named `mos222`.
4. To copy all files from the MOS LEVEL 3 model to the new `mos222` subdirectory:

```
D: project\cmimodel> xcopy mos3 mos222 /e
```
5. In the source files, globally replace `mos3` with `mos222`.
6. In all file names, replace `mos3` with `mos222`. For example:

```
D: project\cmimodel rename CMImos3defs.h CMImos222defs.h
```

After you rename all files, the new model subdirectory contains the following files:

```
CMImos222defs.h
CMImos222.c
CMImos222GetIpar.c
CMImos222SetIpar.c
CMImos222GetMpar.c
CMImos222SetMpar.c
CMImos222eval.c
CMImos222set.c
CMImos222temp.c
```
7. Load all of the above files from the `mos222` subdirectory, into the project directory.
8. Add the following declaration to the `cmimodel/link/CMImdlDec.h` file:

```
extern CMI_MOSDEF* pCMI_mos222def;
```
9. Add the following branch to the switch clause in the `cmimodel/link/CMImdlLevel.h` file:

```
case222:
    pCMIDevice = (char *)pCMI_mos222def;
    break;
```
10. Rebuild the `libCMImodel.dll` file.
11. Put the dynamic link library into the same directory as the `hspice.exe` or `hspice_mt.exe` executable.

Testing CMI Models

To test a new model in the shared library, run a simulation on the `mos3.sp` input file, in the `test` subdirectory. This file contains a simple CMOS inverter, using MOS LEVEL-3 models. Modify transistor sizes and model cards as necessary.

```
%hspice mos3.sp >mos3.lis
```

You can then use *Avanwaves* to inspect the I-V and C-V characteristics, at different biasing conditions.

Use *AvanWaves* to carefully check the following aspects:

- Sign and value of channel current (*ids*).
- Monotone characteristics of channel current, versus *vgs* and *vds*.
- Sign and value of capacitance (*cgs*, *cgb*, *csb*, *cdb*).

Refer to the *AvanWaves User Guide* for more information.

To verify the CMI integration of your new model, run a DC sweep analysis, and a transient analysis, on the test netlist.

Note: LEVELs from 100 to 200 are reserved for CMI customer models. Choose levels from this range, so your custom models do not conflict with existing True-Hspice model levels. Also, add a special prefix or suffix for some of the auxiliary functions used in CMI, especially those from the public domain (such as the *modchk* function, or *dc3p1* from Berkeley Spice3. This ensures that the function names are different from those used in the simulator's core code.

After testing, if you are satisfied with your CMI library, put it in the default CMI library directory, `$install_dir/$ARCH/lib/models`. In this syntax, *\$ARCH* can be *sun4*, *sol4*, or *pa*, depending on the platform you use to compile your CMI library.

Model interface routines accept input parameters from CMI. For each set of input conditions, model routines must return transistor characteristics to CMI.

Model Interface Routines

Model interface routines accept these input parameters from CMI:

- Circuit and nominal model temperatures (*CKTtemp*, *CKTnomtemp*).
- Input biases (*vds*, *vgs*, *vbs*).
- Model parameters (*level*, *vto*, *tox*, *uo* ...).
- Instance parameters (*w*, *l*, *as*, *ad* ...).
- Mode of the transistor (*mode*, 1 for normal, -1 for reverse).
- AC frequency (*freq*, passes from the simulator to the model code).
- Integration order (*intorder*) for transient simulation. Star-Hspice returns the following codes:
 - 0 - Trapezoidal
 - 1 - 1st order Gear
 - 2 - 2nd order Gear
- Transient time step (*timestep*).
- Transient time point (*timepoint*).

For each set of input conditions, the model routines must return the following transistor characteristics to CMI:

- Flag for computing charge and capacitance (1 for computation, 0 for no computation, *qflag*).
- Selector for charge, capacitance model (0 for Meyer capacitance model*, 13 for charge-based model, *capop*).
- Channel current (*ids*).
- Channel conductance (*gds*).
- Trans-conductance (*gm*).
- Substrate trans-conductance (*gmbs*).
- Turn-on voltage (*von*).
- Saturation voltage (*vsat*).
- Gate-overlap capacitances (*cgso*, *cgdo*, *cgbo*).
- Intrinsic MOSFET charges (*qg*, *qd*, *qs*).
- Intrinsic MOSFET capacitances, referenced to bulk (*cggb*, *cgdb*, *cgsb*, *cbgb*, *cbdb*, *cbsb*, *cdgb*, *cddb*, *cdsb*).
- Parasitic source and drain conductances (*gs*, *gd*).

- Substrate diode current (*ibd*, *ibs*).
- Substrate diode conductance (*gbd*, *gbs*).
- Substrate diode charge (*qbd*, *qbs*).
- Substrate diode junction capacitance (*capbd*, *capbs*).
- Substrate impact ionization current (*isub*).
- Substrate impact ionization trans-conductances ($gbgs=dI_{sub}/dV_{gs}$, $gbds=dI_{sub}/dV_{ds}$, $gbbs=dI_{sub}/dV_{bs}$).
- Current, for source resistance noise, squared (*nois_irs*).
- Current, for drain resistance noise, squared (*nois_ird*).
- Current, for noise from the Thermal or Shot channel, squared (*nois_idsth*).
- Current, for source resistance noise, squared (*nois_idsfl*).

You cannot use Meyer capacitance models in Star-Hspice.

The *CMI_VAR* variable type, in the *include/CMIdef.h* file, transfers transistor biases and output characteristics, between CMI and model interface routines.

The *vds*, *vgs* and *vbs* entries provide bias conditions. The other entries carry the results, from evaluating the model equations.

```
/* must be consistent with its counterpart in HSPICE */
typedef struct CMI_var {
    /* device input formation */
    int    mode;          /* device mode */
    int    qflag;         /* flag for charge/cap computing */
    double vds;          /* vds bias */
    double vgs;          /* vgs bias */
    double vbs;          /* vbs bias */

    /* device DC information */
    double gd;           /* drain conductance */
    double gs;           /* source conductance */
    double cgso; /* gate-source overlap capacitance */
    double cgdo; /* gate-drain overlap capacitance */
    double cgbo; /* gate-bulk overlap capacitance */
    double von;          /* turn-on voltage */
    double vdsat;        /* saturation voltage */
    double ids;          /* drain dc current */
    double gds; /* output conductance (dIds/dVds) */
    double gm;           /* trans-conductance (dIds/dVgs) */
    double gmbs; /* substrate trans-conductance
                  (dIds/dVbs)*/
}
```

```

' /* MOSFET capacitance model selection */
/* capop can have following values
* 13      charge model
* 0 or else Meyer's model

NOTE: Star-Hspice does not support Meyer's model.

int    capop;      /* capacitor selector */

/* Meyer's capacitances: intrinsic capacitance + overlap
capacitance. Star-Hspice ignores these 3 capacitances. A charge-
based model formulation is required. */

double capgs; /* Meyer's gate capacitance
              (dQg/dVgs + cgso) */
double capgd; /* Meyer's gate capacitance
              (dQg/dVds + cgdo) */
double capgb; /* Meyer's gate capacitance
              (dQg/dVbs + cgbo) */

/* substrate-junction information */
double ibs; /* substrate source-junction leakage current */
double ibd; /* substrate drain-junction leakage current */
double gbs; /* substrate source junction-conductance */
double gbd; /* substrate drain junction-conductance */
double capbs; /* substrate source-junction capacitance */
double capbd; /* substrate drain-junction capacitance */
double qbs; /* substrate source-junction charge */
double qbd; /* substrate drain-junction charge */

/* substrate impact ionization current */
double isub; /* substrate current */
double gbgs; /* substrate trans-conductance (dIsub/dVgs) */
double gbds; /* substrate trans-conductance (dIsub/dVds) */
double gbbs; /* substrate trans-conductance (dIsub/dVbs) */

/* charge-based model intrinsic terminal charges */
/* NOTE: these are intrinsic charges ONLY */
double qg; /* gate charge */
double qd; /* drain charge */
double qs; /* source charge */

/* charge-based model intrinsic trans-capacitances */
/* NOTE: these are intrinsic capacitances ONLY */
double cggb;
double cgdb;
double cgbs;
double cbgb;
double cbdb;
double cbsb;
double cdgb;
double cddb;
double cdsb;

```

```

/* noise parameters */
double nois_irs; /* Source noise current^2 */
double nois_ird; /* Drain noise current^2 */
double nois_idsth; /* channel thermal or shot
                  noise current^2 */
double nois_idsfl; /* 1/f channel noise current^2 */
double freq;      /* ac frequency */

/* extended model topology */
char *topovar;    /* topology variables */
double leff;      /* effective channel length */
double weff;      /* effective channel width */
} CMI_VAR;

```

The *CMIenv* global variable defines the nominal temperature and device temperature. You can use the *pCMIenv* (pointer to the global *CMIenv* struct) global variable to access the *CMIenv* structure. The *CMI_ENV* type, in the *include/CMIdef.h* file, defines the structure for *CMIenv*:

```

/* environment variables */
typedef struct CMI_env {
    double CKTtemp;      /* simulation temperature */
    double CKTnomTemp;   /* nominal temperature */
    double CKTgmin;      /* GMIN for the circuit */
    int    CKTtempGiven; /* temp setting flag */
    /* following are hspice-specific options */
    double aspec;
    double spice;
    double scalm;
} CMI_ENV;

/* model parameters for JFET&MESFET */
/* JFET&MESFET model parameter for CMI_VAR in CMIdef.h */
double gg; /* gate conductance */
double cigs; /* gate-to-source current */
double gigs; /* gate-to-source conductance */
double cigd; /* gate-to-drain current */
double gigd; /* gate-to-drain conductance */
double csat; /* diode saturation current */
double capds; /* drain-to-source capacitance */
double nois_irg; /* Gate noise current^2 */
double qgso; /* gate-to-source old charge */
double qgdo; /* gate-to-drain old charge */
double qgs; /* gate-to-source charge */
double qgd; /* gate-to-drain charge */
double vgsold; /* gate-to-source old voltage */
double vgdold; /* gate-to-drain old voltage */

```

Interface Variables

To assign model/instance parameter values, and to evaluate I-V, C-V response, you need fifteen interface routines. For each new model, an *interface variable* (in the *CMI_MOSDEF* type) defines pointers to these routines, and the model/instance variables. This variable is in the `include/CMIDef.h` file.

```
typedef struct CMI_MosDef {
char   ModelName[100];
char   InstanceName[100];
char   *pModel;
char   *pInstance;
int    modelSize;
int    instSize;
int    (*CMI_ResetModel)(char*, int, int);
int    (*CMI_ResetInstance)(char*);
int    (*CMI_AssignModelParm)(char*, char*, double);
int    (*CMI_AssignInstanceParm)(char*, char*, double);
int    (*CMI_SetupModel)(char*);
int    (*CMI_SetupInstance)(char*, char*);
int    (*CMI_Evaluate)(CMI_VAR*, char*, char*);
int    (*CMI_DiodeEval)(CMI_VAR*, char*, char*);
int    (*CMI_Noise)(CMI_VAR*, char*, char*);
int    (*CMI_PrintModel)(char*);
int    (*CMI_FreeModel)(char*);
int    (*CMI_FreeInstance)(char*, char*);
int    (*CMI_WriteError)(int, char*);
int    (*CMI_Start)(void);
int    (*CMI_Conclude)(void);
/* extended model topology, 0 is normal mos, 1 is
   berkeley SOI, etc. */
int    topoid;
} CMI_MOSDEF;
```

All routines return 0 if they succeed, or a non-zero integer (an error code that you define) for a warning or error. The following sections describe each entry.

Star-Hspice extracts examples for the first seven functions, from the MOS3 implementation. Examples for the remaining eight functions are not part of the actual MOS3 code, but the code includes them for demonstration. The example MOS3 implementation contains one header file, and eight C files. All routines are based on SPICE-3 code.

pModel, pInstance

Star-Hspice initializes structures for an interface variable, when you compile.
For example:

```
/* function declaration */
int CMImos3ResetModel(char*, int, int);
int CMImos3ResetInstance(char*);
int CMImos3AssignMP(char*, char*, double);
int CMImos3AssignIP(char*, char*, double);
int CMImos3SetupModel(char*);
int CMImos3SetupInstance(char*, char*);
int CMImos3Evaluate(CMI_VAR*, char*, char*);
int CMImos3DiodeEval(CMI_VAR*, char*, char*);
int CMImos3Noise(CMI_VAR*, char*, char*);
int CMImos3PrintModel(char*);
int CMImos3FreeModel(char*);
int CMImos3FreeInstance(char*, char*);
int CMImos3WriteError(int, char*);
int CMImos3Start(void);
int CMImos3Conclude(void);
/* extended model topology, 0=normal mos, 1=berkeley SOI */
/* local */
static MOS3model _Mos3Model;
static MOS3instance _Mos3Instance;
static CMI_MOSDEF CMI_mos3def = {
    (char*)&_Mos3Model,
    (char*)&_Mos3Instance,
    CMImos3ResetModel,
    CMImos3ResetInstance,
    CMImos3AssignMP,
    CMImos3AssignIP,
    CMImos3SetupModel,
    CMImos3SetupInstance,
    CMImos3Evaluate,
    CMImos3DiodeEval,
    CMImos3Noise,
    CMImos3PrintModel,
    CMImos3FreeModel,
    CMImos3FreeInstance,
    CMImos3,
    CMImos3Start,
    CMImos3Conclude
};
/* export */
CMI_MOSDEF *pCMI_mos3def = &CMI_mos3def;

*Note: the last 8 functions are optional. If a function is not
defined, replace it with NULL.
```


CMI_ResetModel

This routine initializes all parameters of a model. All model parameters become undefined, after initialization. Undefined means that the parameter is not defined in a netlist model card. The pmos flag sets the transistor type, after initialization.

Syntax

```
int CMI_ResetModel(char* pmodel, int pmos, int level)
```

pmodel	Pointer to the model instance.
pmos	1 if PMOS, or 0 if NMOS.
level	Model level value, passed from the parser.

Example

```
int
#ifdef __STDC__
    CMImos3ResetModel(
        char *pmodel,
        int    pmos)
#else
    CMImos3ResetModel(pmodel, pmos)
    char *pmodel;
    int    pmos;
#endif
{
    /* reset all flags to undefined */
    (void)memset(pmodel, 0, sizeof(MOS3model));
    /* Note: contains model value passed from parser */
    if(pmos)
    {
        ((MOS3model*)pmodel)->MOS3type = PMOS;
        ((MOS3model*)pmodel)->MOS3typeGiven = 1;
    }
    return 0;
} /* int CMImos3ResetModel() */
```

CMI_ResetInstance

This routine initializes all parameter settings of an instance. All instance parameters become undefined, after initialization. Undefined means the parameter does not have a definition in a netlist MOS instance.

Syntax

```
int CMI_ResetInstance(char* pinst)
```

pinst Pointer to the instance.

Example

```
int
#ifdef __STDC__
    CMImos3ResetInstance(
        char *ptran)
#else
    CMImos3ResetInstance(ptran)
    char *ptran;
#endif
{
    (void)memset(ptran, 0, sizeof(MOS3instance));
    ((MOS3instance*)ptran)->MOS3w = 1.0e-4;
    ((MOS3instance*)ptran)->MOS3l = 1.0e-4;
    return 0;
} /* int CMImos3ResetInstance() */
```

CMI_AssignModelParm

This routine sets the value of a model parameter.

Syntax

```
int CMI_AssignModelParm(char* pmodel, char* pname, double value)
```

pmodel Pointer to the model instance.

pname String of the parameter name.

value Parameter value.

Example

```
int
#ifdef __STDC__
    CMIImos3AssignMP(
        char    *pmodel,
        char    *pname,
        double value)
#else
    CMIImos3AssignMP(pmodel, pname, value)
    char    *pmodel;
    char    *pname;
    double value;
#endif
{
    int param;
    CMIImos3GetMpar(pname, &param);
    CMIImos3SetMpar(param, value, (MOS3model*)pmodel);
    return 0;
} /* int CMIImos3AssignMP() */
```

CMI_AssignInstanceParm

This routine sets the value of an instance parameter.

Syntax

```
int CMI_AssignInstanceParm(char *pinst, char* pname, double
value)
```

pinst Pointer to the instance.

pname String of the parameter name.

value Parameter value.

Example

```
int
#ifdef __STDC__
    CMIImos3AssignIP(
        char    *ptran,
        char    *pname,
        double value)
```

```

    #else
        CMImos3AssignIP(ptran, pname, value)
        char    *ptran;
        char    *pname;
        double value;
    #endif
    {
        int param;
        CMImos3GetIpar(pname, &param);
        CMImos3SetIpar(param, value, (MOS3instance*)ptran);
        return 0;
    } /* int CMImos3AssignIP() */

```

CMI_SetupModel

This routine sets up a model, after you specify all model parameters.

Syntax

```
int CMI_SetupModel(char* pmodel)
```

pmodel Pointer to the model.

Example

```

int
#ifdef __STDC__
    CMImos3SetupModel(
        char *pmodel)
#else
    CMImos3SetupModel(pmodel)
    char *pmodel;
#endif
{
    CMImos3setupModel((MOS3model*)pmodel);
    return 0;
} /* int CMImos3SetupModel() */

```

CMI_SetupInstance

This routine sets up an instance, after you specify all instance parameters. Star-Hspice typically processes temperature and geometry.

Syntax

```
int CMI_SetupInstance(char* pinst)
```

pinst Pointer to the instance.

Example

```
int
#ifdef __STDC__
    CMImos3SetupInstance(
        char    *pmodel,
        char    *ptran)
#else
    CMImos3SetupInstance(pmodel, ptran)
        char    *pmodel;
        char    *ptran;
#endif
{
    /* temperature modified parameters */
    CMImos3temp((MOS3model*)pmodel, (MOS3instance*)ptran);
    return 0;
} /* int CMImos3SetupInstance() */
```

CMI_Evaluate

Based on the bias conditions and model/instance parameter values, this routine evaluates model equations. It then passes all transistor characteristics, via the *CMI_VAR* variable.

Syntax

```
int CMI_Evaluate(CMI_VAR *pvar, char *pmodel, char *pinst)
```

pvar Pointer to *CMI_VAR* variable.

pmodel Pointer to the model.

pinst Pointer to the instance.

Example

```

int
#ifdef __STDC__
    CMImos3Evaluate(
        CMI_VAR *pslot,
        char *pmodel,
        char *ptr)
#else
    CMImos3Evaluate(pslot, pmodel, ptr)
    CMI_VAR *pslot;
    char *pmodel;
    char *ptr;
#endif
{
    CMI_ENV *penv;
    MOS3instance *ptran;
    penv = pCMIenv; /* pCMIenv is a global */
    ptran = (MOS3instance*)ptr;
    /* call model evaluation */

    (void)CMImos3evaluate(penv, (MOS3model*)pmodel, ptran,
        pslot->vgs, pslot->vds, pslot->vbs);
    pslot->gd = ptran->MOS3drainConductance;
    pslot->gs = ptran->MOS3sourceConductance;
    pslot->von = ptran->MOS3von;
    pslot->ids = ptran->MOS3cd;
    pslot->gds = ptran->MOS3gds;
    pslot->gm = ptran->MOS3gm;
    pslot->gmbs = ptran->MOS3gmbs;
    pslot->gbd = ptran->MOS3gbd;
    pslot->gbs = ptran->MOS3gbs;
    pslot->cgs = ptran->MOS3capgs;
    pslot->cgd = ptran->MOS3capgd;
    pslot->cgb = ptran->MOS3capgb;
    pslot->capdb = ptran->MOS3capbd;
    pslot->capsb = ptran->MOS3capbs;
    pslot->cbso = ptran->MOS3cbs;
    pslot->cbdo = ptran->MOS3cbd;

    ...Assign additional CMI_VAR elements here, for substrate
    model and overlap capacitances.

    return 0;
} /* int CMImos3Evaluate() */

```

CMI_DiodeEval

Based on the bias conditions and model/instance parameter values, this routine evaluates the MOS junction diode model equations. It then passes all transistor characteristics, via the *CMI_VAR* variable.

Syntax

```
int CMI_DiodeEval(CMI_VAR *pvar, char *pmodel, char *pinst)
```

pvar	Pointer to <i>CMI_VAR</i> variable.
pmodel	Pointer to the model.
pinst	Pointer to the instance.

Example

```
int
#ifdef __STDC__
    CMImos3DiodeEval(
        CMI_VAR *pslot,
        char *pmodel,
        char *ptr)
#else
    CMImos3Diode(pslot, pmodel, ptr)
    CMI_VAR *pslot;
    char *pmodel;
    char *ptr;
#endif
{
    CMI_ENV *penv;
    MOS3instance *ptran;
    penv = pCMIenv; /* pCMIenv is global */
    ptran = (MOS3instance*)ptr;
    /* call model evaluation */
    (void)CMImos3diode(penv, (MOS3model*)pmodel, ptran,
        pslot->vgs, pslot->vds, pslot->vbs);
    pslot->ibs = ptran->MOS3ibs;
    pslot->ibd = ptran->MOS3ibd;
    pslot->gbs = ptran->MOS3gbs;
    pslot->gbd = ptran->MOS3gbd;
    pslot->capbs = ptran->MOS3capbs;
    pslot->capbd = ptran->MOS3capbd;
    pslot->qbs = ptran->MOS3qbs;
    pslot->qdb = ptran->MOS3qbd;
    return 0;
} /* int CMImos3DiodeEval() */
```

CMI_Noise

Based on the bias conditions, temperature, and model/instance parameter values, this routine evaluates the noise model equations. It then returns noise characteristics, via the *CMI_VAR* variable.

Star-Hspice passes values to:

- *pslot->nois_irs*. Thermal noise, associated with parasitic source resistance, expressed as a mean square noise current (in parallel with *Rs*).
- *pslot->nois_ird*. Thermal noise, associated with parasitic drain resistance, expressed as a mean square noise current (in parallel with *Rd*).
- *pslot->nois_idsth*. Thermal noise, associated with a MOSFET, expressed as a mean square noise current, referenced across the MOSFET channel.
- *pslot->nois_idsfl*. Flicker noise, associated with a MOSFET, expressed as a mean square noise current, referenced across the MOSFET channel.

Star-Hspice also passes the frequency into *CMI_Noise*, via *pslot->freq*.

Syntax

```
int CMI_Noise(CMI_VAR *pvar, char *pmodel, char *pinst)
```

pvar Pointer to the *CMI_VAR* variable.

pmodel Pointer to the model.

pinst Pointer to the instance.

Example

```
int
#ifdef __STDC__
    CMImos3Noise(
        CMI_VAR *pslot,
        char *pmodel,
        char *ptr)
#else
    CMImos3Noise(pslot, pmodel, ptr)
    CMI_VAR *pslot;
    char *pmodel;
    char *ptr;
#endif
    {double freq, fourkt;
      CMI_ENV *penv
      MOS3instance *ptran;
```



```

    penv = pCMIenv; /* pCMIenv is a global */
    ptran = (MOS3instance*)ptr;
    fourkt = 4.0 * BOLTZMAN * ptran->temp; /* 4kT */
    freq = pslot->freq;

    /* Drain resistor thermal noise as current^2 source*/
    pslot->nois_ird = fourkt * ptran->gdpr;
    /* Source resistor thermal noise as current^2 source */
    pslot->nois_irs = fourkt * ptran->gspr;
    /* thermal noise assumed to be current^2 source referenced
       to channel. The source code for thermalnoise() is not
       shown here*/
    pslot->nois_idsth = thermalnoise(model, here, fourkt);
    /* flicker (1/f) noise assumed to be current^2 source
       referenced to channel. The source code for flickernoise()
       is not shown here */
    pslot->nois_idsfl = flickernoise(model, here, freq);
    return 0;
} /* int CMImos3Noise() */

```

CMI_PrintModel

This routine prints all model parameter names, values, and units, to standard output. Star-Hspice calls this routine for each model after the CMI_SetupModel.

Syntax

```
int CMI_PrintModel(char *pmodel)
```

pmodel Pointer to the model.

Example

```

int
#ifdef __STDC__
    CMImos3PrintModel(
        char *pmodel)
#else
    CMImos3PrintModel(pmodel)
        char *pmodel;
#endif
{
    CMI_ENV *penv

    /* Note: source for CMImos3printmodel() not shown*/
    (void)CMImos3printmodel((MOS3model*)pmodel);
    return 0;
} /* int CMImos3PrintModel() */

```

CMI_FreeModel

This routine frees memory that Star-Hspice previously allocated for model-related data. After simulation, Star-Hspice calls this routine, during a loop over all models.

Syntax

```
int CMI_FreeModel(char *pmodel)
```

pmodel Pointer to the model.

Example

```
int
#ifdef __STDC__
    CMImos3FreeModel(
        char    *pmodel)
#else
    CMImos3FreeModel(pmodel)
        char    *pmodel;
#endif
{ /* free memory allocated for model data. Note
   CMImos3freemodel() source code not shown. */
(void)CMImos3freemodel((MOS3model*)pmodel);
return 0;
} /* int CMImos3FreeModel() */
```

CMI_FreeInstance

This routine frees memory that Star-Hspice previously allocated for storing instance-related data. After simulation, Star-Hspice calls this routine, during an outer loop over all models, and an inner loop over all instances associated with each model.

Syntax

```
int CMI_FreeInstance(char *pmodel. char *pinst)
```

pmodel Pointer to the model.

pinst Pointer to the instance.

Example

```

int
#ifdef __STDC__
    CMImos3FreeInstance(
        char    *pmodel,
        char    *ptr)
#else
    CMImos3FreeInstance(pmodel, ptr)
        char    *pmodel;
        char    *ptr;
#endif
{
    CMI_ENV      *penv
    MOS3instance *ptran;
    ptran = (MOS3instance*)ptr;
    /* free memory allocated for model data. Note
    CMImos3freeinstance()source code not shown.
    */
    (void)CMImos3freeinstance((MOS3model*)pmodel, ptran);
    return 0;
} /* int CMImos3FreeInstance() */

```

CMI_WriteError

If model evaluation detects an error, this routine writes error messages that you define, to standard output. All CMI functions return an error code, and pass it to CMI_WriteError().

- In CMI_writeError(), you define an error statement, and copy it to err_str (based on the error code value).
- CMI_WriteError() returns the error status:
 - If *err_status*>0, Star-Hspice writes the error message, and aborts.
 - If *err_status*=0, Star-Hspice writes the warning message, and continues.

Star-Hspice calls CMI_WriteError() after every CMI function call.

Syntax

```
int CMI_WriteError(int err_code, char *err_str)
```

<i>err_code</i>	Error code.
<i>err_str</i>	Pointer to error message.

Example

```

int
#ifdef __STDC__
    CMImos3WriteError(
        void err_code,
        char *err_str)
#else
    CMImos3WriteError(err_code, err_str)
    int err_code;
    char *err_str;
#endif
{
    /* */
    int err_status=0;
    switch err_code
    {
    case 1:
        strcpyn(err_str, "User Err: Eval()", CMI_ERR_STR_LEN);
        err_status=1;
    case 2:
        strcpyn(err_str, "User Warn: Eval()", CMI_ERR_STR_LEN);
        err_status=1;
    default:
        strcpyn(err_str, "User Err:Generic", CMI_ERR_STR_LEN);
        err_status=1;
    }
    return err_status;
} /* int CMImos3WriteError() */

```

CMI_Start

Before simulation, this routine runs startup functions that you define.

Syntax

```
int CMI_Start(void)
```

Example

```

int
#ifdef __STDC__
    CMImos3Start(void)
#else
    CMImos3Start(void)
#endif
{
    (void)CMImos3start();
    return 0;
} /* int CMImos3Start() */

```

CMI_Conclude

After simulation, this routine runs conclude functions that you define.

Syntax

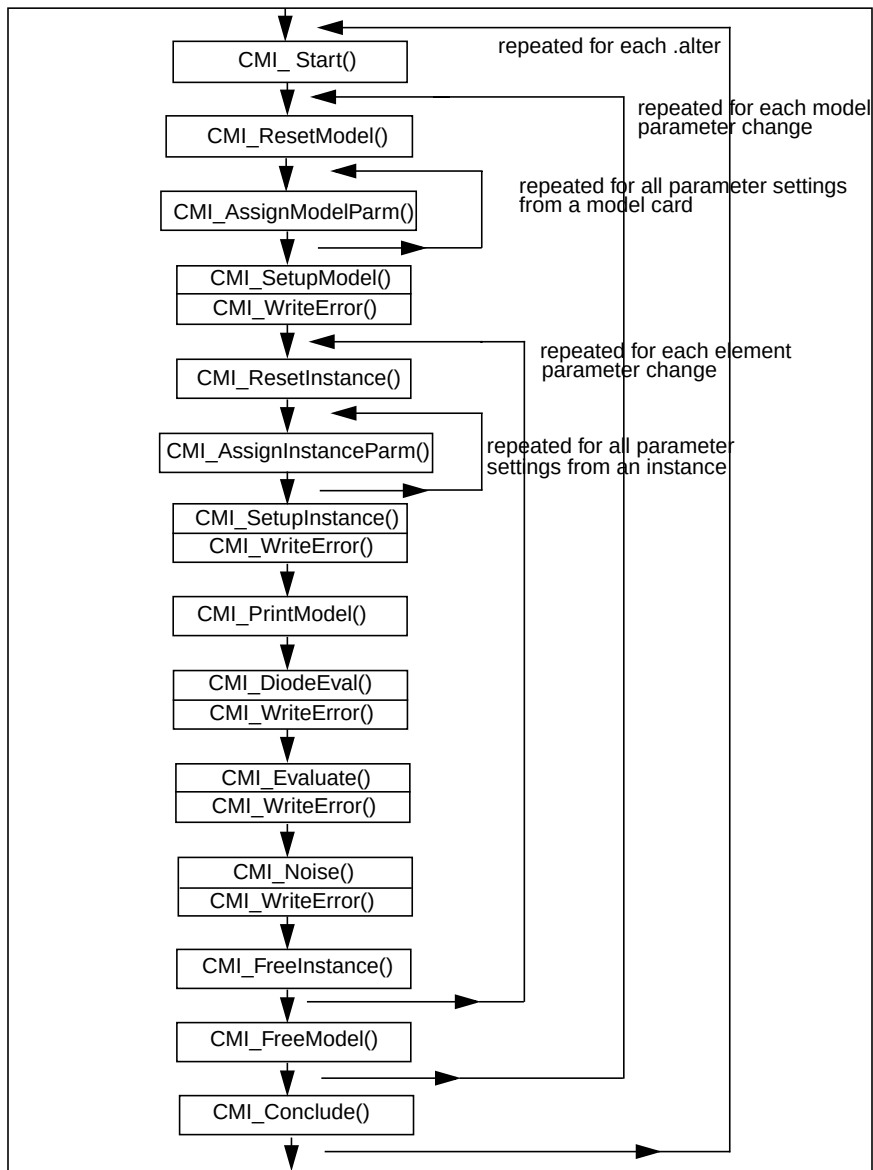
```
int CMI_Conclude(void)
```

Example

```
int
#ifdef __STDC__
    CMIImos3Conclude(void)
#else
    CMIImos3Conclude(void)
#endif
{
    (void)CMIImos3conclude();
    return 0;
} /* int CMIImos3Conclude() */Internal Routines */
```

CMI Function Calling Protocol

Figure 14-3: Interface Routines Calling Sequence



Internal Routines

In the example MOS3 implementation, the interface routines in *CMImos3.c* also call the following internal routines:

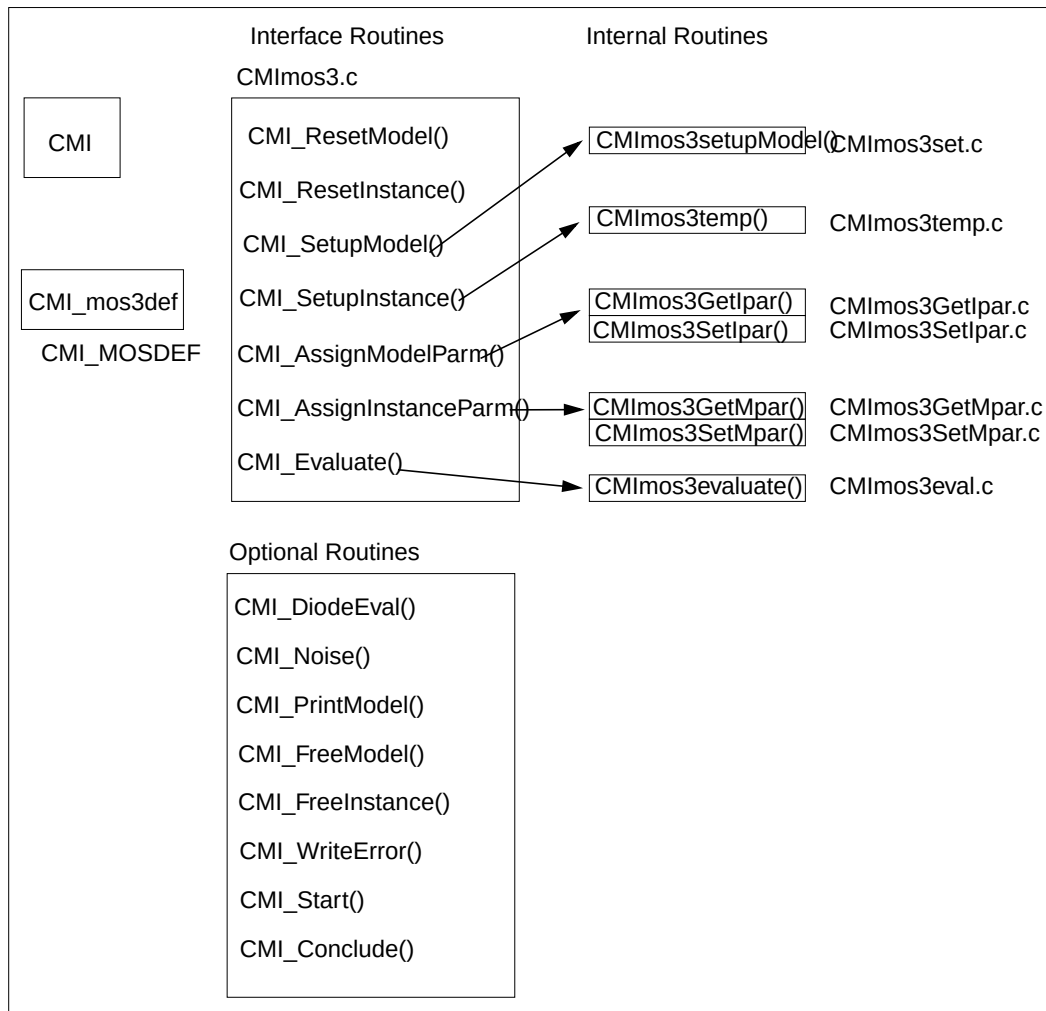
<code>CMImos3GetIpar.c</code>	get instance parameter index
<code>CMImos3SetIpar.c</code>	set instance parameter
<code>CMImos3GetMpar.c</code>	get model parameter index
<code>CMImos3SetMpar.c</code>	set model parameter
<code>CMImos3eval.c</code>	evaluate model equations
<code>CMImos3set.c</code>	setup a model
<code>CMImos3temp.c</code>	setup an instance, including temperature-effect

[Figure 14-4 on page 14-34](#) shows the hierarchical relationship between the interface routines, and the internal routines.

For the automatic script to work, the name of the interface variable (and all routine files) must follow the naming convention, as follows:

<code>pCMI_xxxdef</code>	
<code>CMIxxx.c</code>	<code>CMIxxxGetIpar.c</code>
<code>CMIxxxSetIpar.c</code>	<code>CMIxxxGetMpar.c</code>
<code>CMIxxxSetMpar.c</code>	<code>CMIxxxeval.c</code>

where *xxx* is the model name.

Figure 14-4: Hierarchy of Interface and Internal Routines

Extended Topology

In addition to conventional four terminal (`topoid = 0`) MOSFET topology, Star-Hspice can support other topologies. You must assign a unique `topoid` for different topologies.

CMI implements BSIM SOI topology, with an assigned `topoid` of 1.

- If you create your own model named `topovar`, and it is the same as the BSIM SOI model, you can specify a `topoid` of 1, and use the Star-Hspice topology structure for stamping information.
- If your model topology is different from the conventional four-terminal model, or the BSIM SOI, then you must specify the `topovar` structure. Star-Hspice assigns a unique `topoid` for your topology.

The naming convention for the structure fields is the same as in the BSIM SOI model. For detailed information about fields in the structure, refer to the *BSIM3PD2.0 MOSFET MODEL User's Manual*, which you can find at:

<http://www-device.eecs.berkeley.edu/~bsim3soi>

For example, the following is the *topovar* structure for the BSIM SOI model, used in Star-Hspice CMI.

```
struct TOP01 {
    double vps;
    double ves;
    double delTemp;      /* T node */
    double selfheat;
    double qsub;
    double qth;
    double cbodcon;
    double gbps;
    double gbpr;
    double gcde;
    double gcse;
    double gjdg;
    double gjdd;
    double gjdb;
    double gjdT;
    double gjsg;
    double gjsd;
    double gjsb;
    double gjdT;
    double cdeb;
```

```
double cbeb;  
double ceeb;  
double cgeo;  
/* add 4 for T */  
double cgT;  
double cdT;  
double cbT;  
double ceT;  
double rth;  
double cth;  
double gmT;  
double gbT;  
double gbpT;  
double gTtg;  
double gTtd;  
double gTtb;  
double gTtt;  
};
```

Conventions

Bias Polarity, for N- and P-channel Devices

The *vds*, *vgs*, and *vbs* input biases, in CMI_VAR, are:

```
vds = vd - vs
vgs = vg - vs
vbs = vb - vs
```

You must negate these biases for the P-channel device, if your model code does not distinguish between n-channel and p-channel bias. The example routines multiply the biases by the *type* model parameter, which is 1 for N-device, or -1 for P-device. For example, see the MOS3 model code:

```
if (model->MOS3type < 0) { /* P-channel */
    vgs = -VgsExt;
    vds = -VdsExt;
    vbs = -VbsExt;
}
else { /* N-channel */
    vgs = VgsExt;
    vds = VdsExt;
    vbs = VbsExt;
}
```

Use this code in both the *CMI_Evaluate()* and *CMI_DiodeEval()* functions.

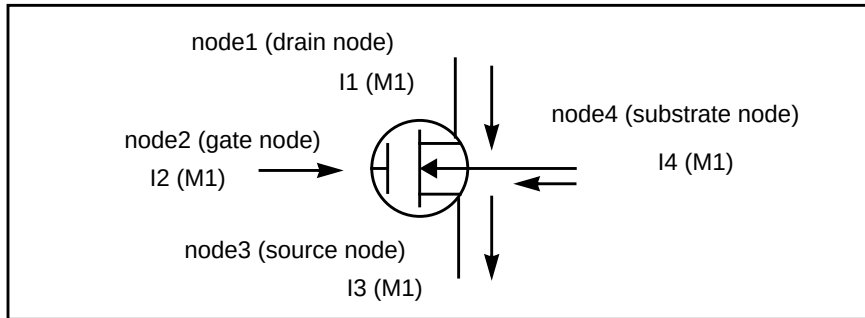
[Figure 14-5 on page 14-38](#) shows the convention to output current components.

- For channel current, drain-to-source is the positive direction.
- For substrate diodes, bulk-to-source/drain are the positive directions.

These conventions are the same for both N-channel and P-channel devices.

The conventions for *von* are:

- N-channel, device is on, if *vgs* > *von*.
- P-channel, device is on, if *vgs* < *von*.

Figure 14-5: MOSFET (node1, node2, node3, node4) - N-channel

Base the derivatives (conductances and capacitances) on the polarity conventions of the bias and current. The following code demonstrates the required polarity reversal for currents, and V_{on} , V_{dsat} for PMOS devices.

```
if (model->type < 0)
{
    pslot->ids = -pslot->ids;
    pslot->ibs = -pslot->ibs;
    pslot->ibd = -pslot->ibd;
    pslot->von = -pslot->von;
    pslot->vdsat = -pslot->vdsat;
}
```

Source-Drain Reversal Conventions

Star-Hspice performs the appropriate computations, when you operate the MOSFET in the reverse mode (when $V_{ds} < 0$ for N-channel, or $V_{ds} > 0$ for P-channel). This includes a variable transformation ($V_{ds} \rightarrow -V_{ds}$, $V_{gs} \rightarrow V_{gd}$, $V_{bs} \rightarrow V_{bd}$), and interchange of the source and drain terminals. You do not see this transformation, but it simplifies the model coding task.

Thread-Safe Model Code

Star-Hspice uses shared-memory, multithreading algorithms during model evaluation. To ensure thread-safe model code, adhere to the following rules:

- Do not use static variables in *CMI_Evaluate()*, *CMIDiodeEval()*, *CMIWriteError()*, and *CMI_Noise()*, or in functions that these routines call.
- Never write to a global variable, when you execute *CMI_Evaluate()*, *CMIDiodeEval()*, *CMIWriteError()*, and *CMI_Noise()*.



Chapter 15

Characterizing Cells

Most ASIC vendors use the basic capabilities of the `.MEASURE` statement in Star-Hspice to characterize standard cell libraries, and to prepare data sheets.

Star-Hspice stores input sweep parameters, and measure output parameters, in measure output data files (*design.mt0*, *design.sw0*, and *design.ac0*). This file stores multiple sweep data.

- You can use AvanWaves to plot this data—for example, to generate fanout plots of delay versus load.
- You can use the slope and intercept of the loading curves, to calibrate VHDL, Verilog, Lsim, TimeMill, and Synopsys models.

This chapter explains the following topics:

- **Typical Data Sheet Parameters**

A series of typical data sheet examples, show the flexibility of the `.MEASURE` statement.

- **Characterizing Cells in Data-Driven Analysis**

Automates cell characterization, including calculating the delay coefficient for the timing-simulator polynomial. You can simultaneously vary an unlimited number of parameters, or the number of analyses to perform. Cell characterization uses a convenient ASCII file format, for automated parameter input to Star-Hspice.

Typical Data Sheet Parameters

This section describes how to determine typical data sheet parameters.

Rise, Fall, and Delay Calculations

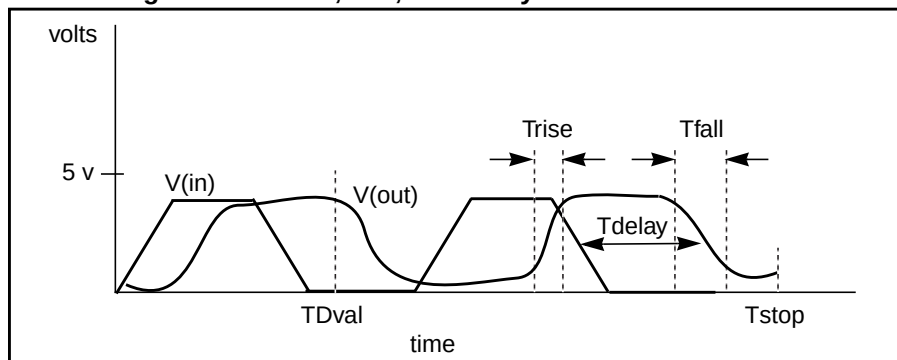
The following example does the following:

1. Uses the MAX function to calculate v_{max} , over the time region of interest.
2. Uses the MIN function to calculate v_{min} .
3. Uses the measured parameters in subsequent calculations, for accurate 10% and 90% points, when determining the rise and fall time.
RISE=1 is relative to the time window that the $TDval$ delay forms.
4. Uses a fixed value for the measure threshold, to calculate the T_{delay} delay.

The following is an example:

```
.MEAS TRAN vmax MAX V(out) FROM=TDval TO=Tstop
.MEAS TRAN vmin MIN V(out) FROM=TDval TO=Tstop
.MEAS TRAN Trise TRIG V(out) val='vmin+0.1*vmax'
+ TD=TDval RISE=1 TARG V(out) val='0.9*vmax' RISE=1
.MEAS TRAN Tfall TRIG V(out) val='0.9*vmax' TD=TDval
+ FALL=2 TARG V(out) val='vmin+0.1*vmax' FALL=2
.MEAS TRAN Tdelay TRIG V(in) val=2.5 TD=TDval FALL=1
+ TARG V(out) val=2.5 FALL=2
```

Figure 15-1: Rise, Fall, and Delay Time Demonstration



Ripple Calculation

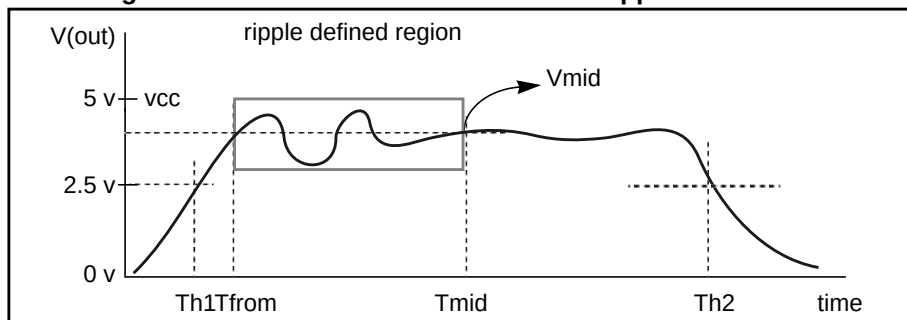
Ripple calculation performs the following:

- Delimits the wave at the 50% of VCC points
- Finds the *Tmid* midpoint
- Defines a bounded region by finding the pedestal voltage (*Vmid*) and then finding the first time that the signal crossed this value, *Tfrom*
- Measures the ripple in the defined region using the peak-to-peak (PP) measure function from *Tfrom* to *Tmid*

The following is an example:

```
.MEAS TRAN Th1 WHEN V(out)='0.5*vcc' CROSS=1
.MEAS TRAN Th2 WHEN V(out)='0.5*vcc' CROSS=2
.MEAS TRAN Tmid PARAM='(Th1+Th2)/2'
.MEAS TRAN Vmid FIND V(out) AT='Tmid'
.MEAS TRAN Tfrom WHEN V(out)='Vmid' RISE=1
.MEAS TRAN Ripple PP V(out) FROM='Tfrom' TO='Tmid'
```

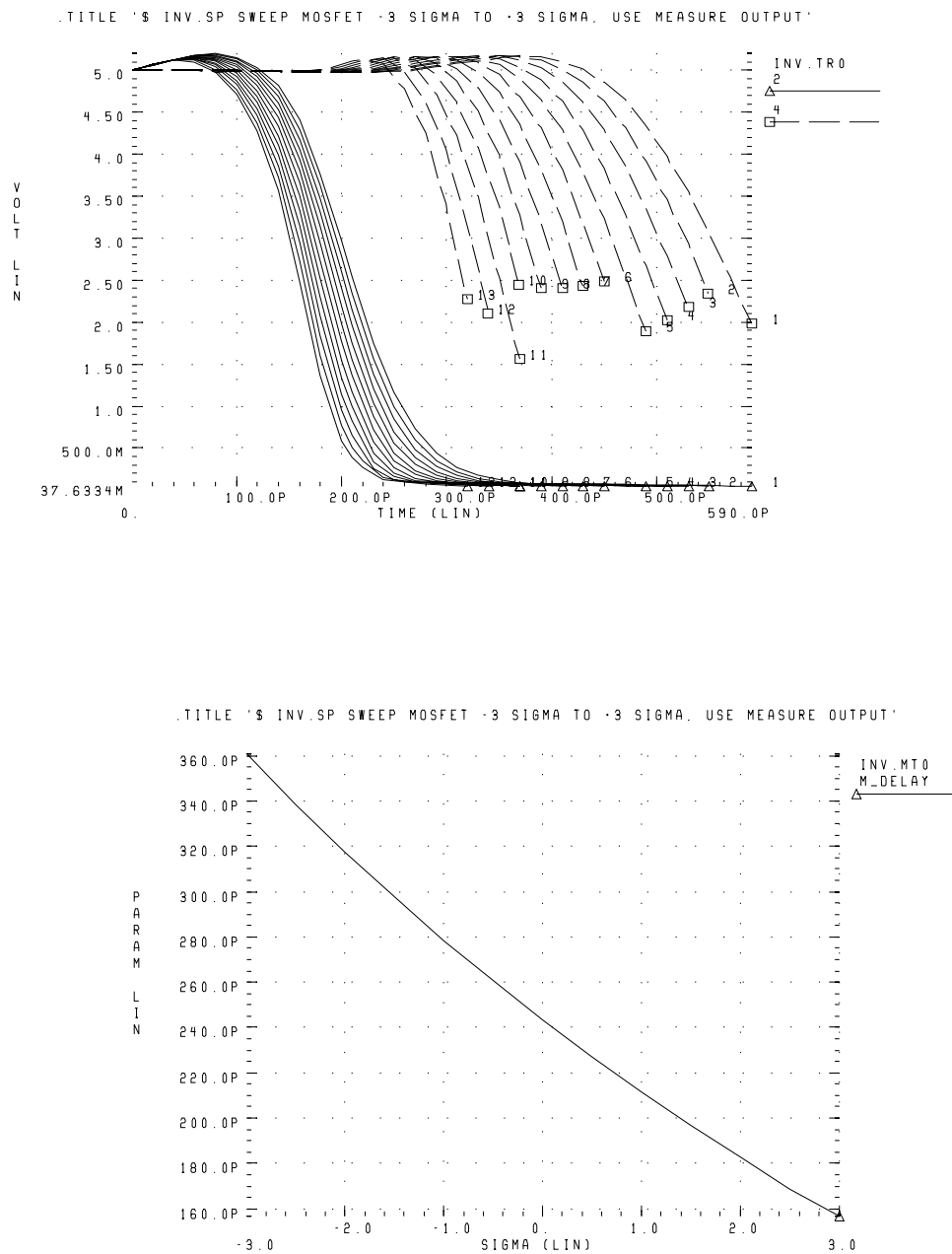
Figure 15-2: Waveform to Demonstrate Ripple Calculation



Sigma Sweep versus Delay

This file sweeps the sigma of the model parameter distribution, while it examines the delay. It shows you the delay derating curve, for the worst cases in the model. This example is based on the demonstration file, in `$installdir/demo/hspice/cchar/sigma.sp`. For a description of this technique for building a worst-case sigma library, see [Worst Case Analysis on page 13-8](#).

```
.tran 20p 1.0n sweep sigma -3 3 .5
.meas m_delay trig v(2) val=vref fall=1 targ v(4)
+ val=vref fall=1
.param xlnew ='polycd-sigma*0.06u'
+ toxnew='tox-sigma*10'
.model nch nmos LEVEL=28 xl = xlnew tox=toxnew
```

Figure 15-3: Inverter Pair Transfer Curves and Sigma Sweep vs. Delay

Delay versus Fanout

This example sweeps the sub-circuit multiplier, to quickly generate five load curves. To obtain more accurate results, buffer the input source with one stage.

For each second-sweep variable (*m_delay* and *rms_power*), the following example calculates:

- mean
- variance
- sigma
- average deviance

This example is based on the demonstration file in `$installdir/demo/hspice/cchar/load1.sp`.

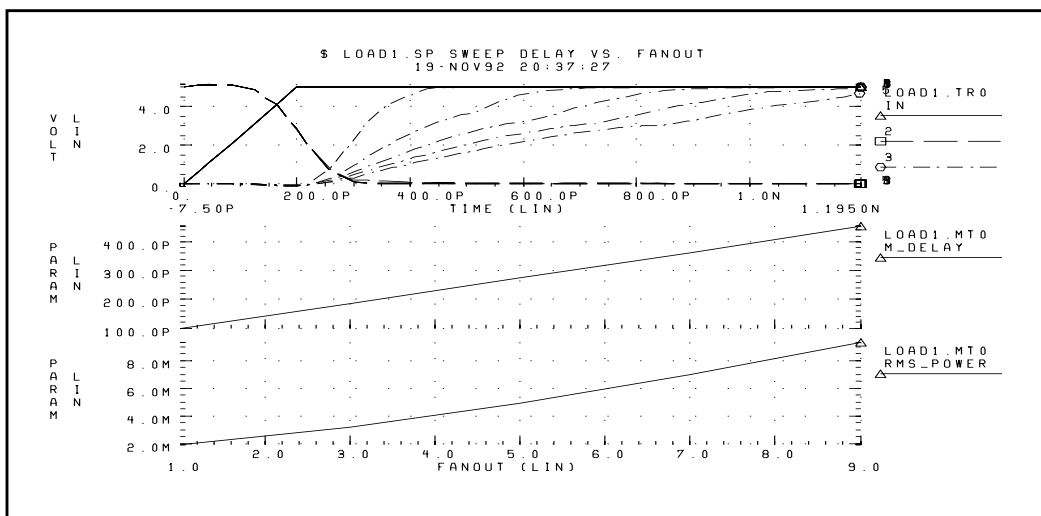
Input File Example

```
tran 100p 2.0n sweep fanout 1 10 2
.param vref=2.5
.meas m_delay trig v(2) val=vref fall=1
+ targ v(3) val=vref rise=1
.meas rms_power rms power
x1 in 2 inv
x2 2 3 inv
x3 3 4 inv m=fanout
```

Output Statistical Results

```
meas_variable = m_delay
mean = 273.8560p varian = 1.968e-20
sigma = 140.2711p avgdev = 106.5685p
meas_variable = rms_power
mean = 5.2544m varian = 8.7044u
sigma = 2.9503m avgdev = 2.2945m
```

Figure 15-4: Inverter Delay and Power, versus Fanout



Pin Capacitance Measurement

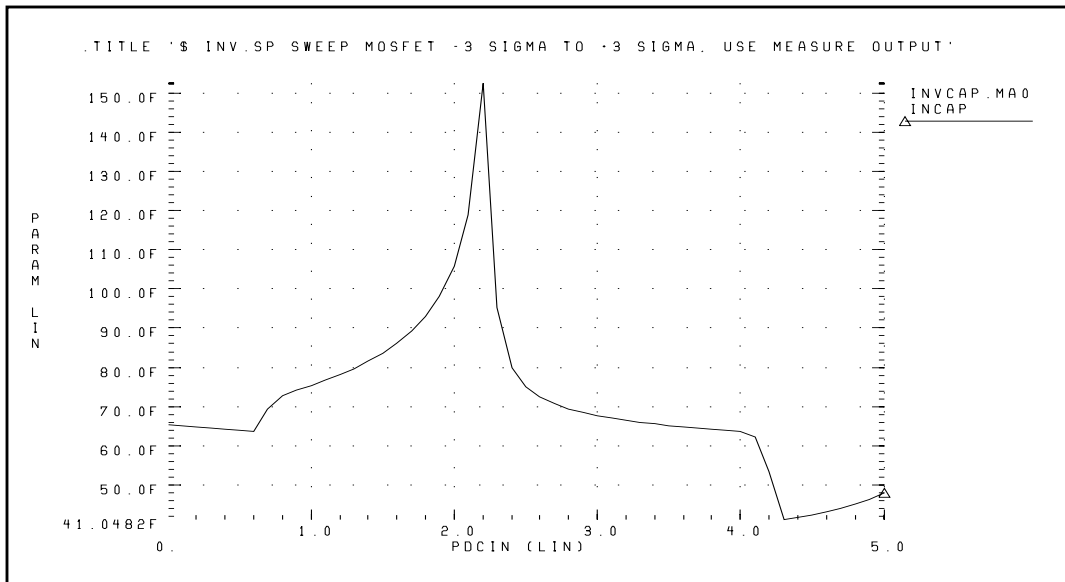
This example does the following:

1. Shows the effect of dynamic capacitance, at the switch point.
2. Sweeps the DC input voltage (*pdcin*) to the inverter.
3. Performs an AC analysis, at each 0.1 volt increment.
4. Calculates the *incap* measure parameter, from the imaginary current through the voltage source, at 10 kilohertz in the AC curve (not shown).

The peak capacitance (at the switch point) occurs when the voltage at the output side changes, in the direction opposite the input side of the Miller capacitor. This adds the Miller capacitance, times the inverter gain, to the effective capacitance.

The following is an example

```
mp out in 1 1 mp w=10u l=3u
mn out in 0 0 mn w=5u l=3u
vin in 0 DC= pdcin AC 1 0
.ac lin 2 10k 100k sweep pdcin 0 5 .1
.measure ac incap find par( '-1 * ii(vin)/
+ (hertz*twopi)' ) AT=10000hertz
```

Figure 15-5: Graph of Pin Capacitance versus Inverter Input Voltage

Op-amp Characterization of ALM124

This example analyzes op-amps. This example uses:

1. .MEASURE statements, to present a very complete data sheet.
2. Four .MEASURE statements, to reference the *out0* output node of an op-amp circuit. These statements use output variable operators for:
 - decibels *vdb(out0)*
 - voltage magnitude *vm(out0)*
 - phase *vp(out0)*

The example is based on the demonstration file, in *demo/apps/alm124.sp*.

Input File Example

```
.measure ac 'unitfreq' trig at=1 targ vdb(out0)
+ val=0 fall=1
.measure ac 'phasemargin' find vp(out0)
+ when vdb(out0)=0
.measure ac 'gain(db)' max vdb(out0)
.measure ac 'gain(mag)' max vm(out0)
```

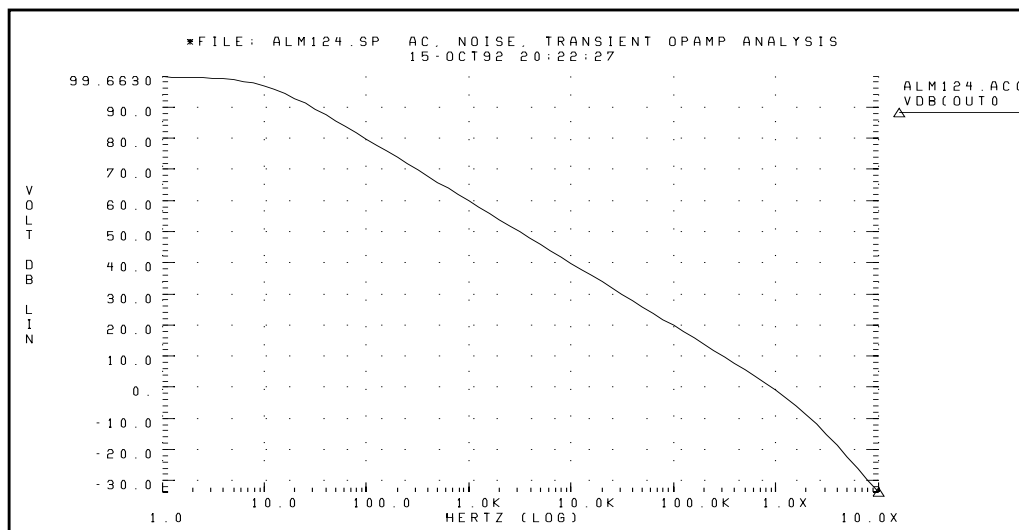
Measure Results

unitfreq = 9.0786E+05 targ= 9.0786E+05 trig= 1.0000E+00
 phasemargin = 6.6403E+01

gain(db) = 9.9663E+01 at= 1.0000E+00 from= 1.0000E+00
 + to= 1.0000E+07

gain(mag)= 9.6192E+04 at= 1.0000E+00 from= 1.0000E+00
 + to= 1.0000E+07

Figure 15-6: Magnitude Plot of Op-Amp Gain



Characterizing Cells in Data-Driven Analysis

This section provides example input files, which characterize cells for an inverter, based on 3-micron MOSFET technology. The program finds the propagation delay, and the rise and fall times, for the inverter, using best, worst, and typical cases for different fanouts. You can use this library data for digital-based simulators, such as those used to simulate gate arrays and standard cells.

The example is based on the demonstration file in `$installdir/demo/hspice/apps/cellchar.sp`. It demonstrates how to use the following, to characterize a CMOS inverter:

- `.MEASURE` statement.
- `.DATA` statement.
- `AUTOSTOP` option.

Cell Examples

[Figure 15-7](#) and [Figure 15-8](#) are identical, except that their input signals are complementary.

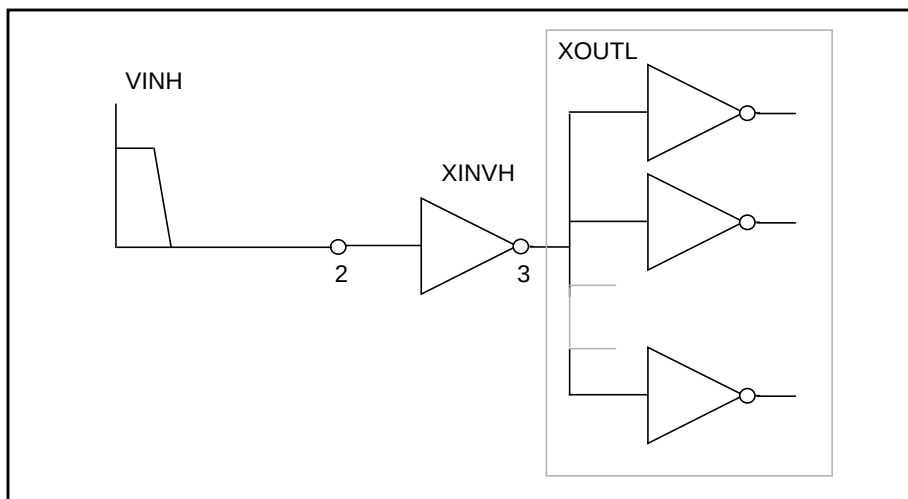
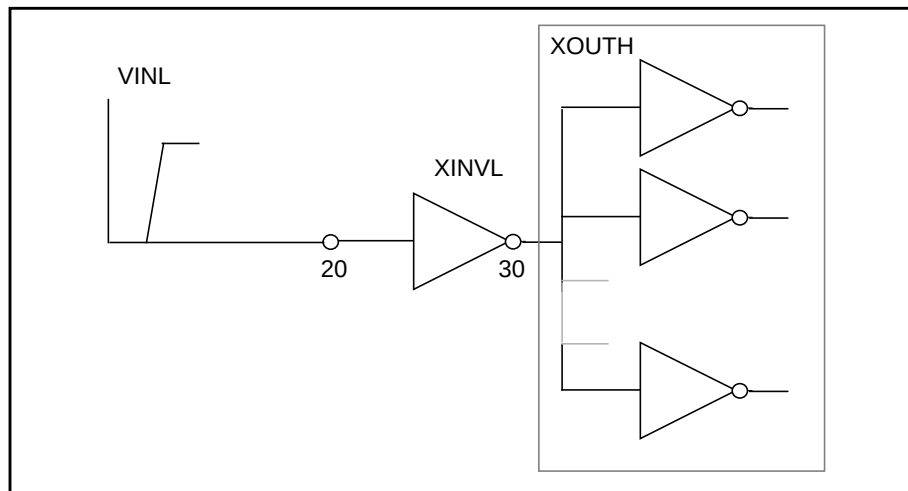
- The circuit in [Figure 15-7](#) calculates the rise time, and the low-to-high propagation delay time.
- The circuit in [Figure 15-8](#) calculates the fall time, and the high-to-low propagation delay time.

If you use only one circuit, CPU time increases, because analysis time increases when Star-Hspice calculates both rise and fall times.

The `XOUTL` or `XOUTH` sub-circuit represents the fanout of the cell (inverter). To modify fanout, specify different multipliers (m) in the sub-circuit calls.

You can also specify local and global temperatures. This example characterizes the cell at a global temperature of 27, but the temperature of the M1 and M2 devices is (27+DTEMP). The `.DATA` statement specifies the DTEMP value.

The example uses a transient parameterized sweep, with `.DATA` and `.MEASURE` statements, to determine the inverter timing, for best, typical, and worst cases.

Figure 15-7: Cell Characterization Circuit 1**Figure 15-8: Cell Characterization Circuit 2**

This example varies the following parameters:

- power supply
 - input rise and fall time
 - fanout
 - MOSFET temperature
 - *n*-channel and *p*-channel threshold
 - drawn width and length of the MOSFET
1. Use the `.MEASURE` statement to specify a parameter to measure.
 2. Use the `AUTOSTOP` option, to speed simulation time.
 3. The `AUTOSTOP` option terminates the transient sweep, although it has not completely swept the specified transient sweep range.

The `.MEASURE` statement uses quoted string parameter variables, to measure the rise time, fall time, and propagation delays.

- Rise time starts when the voltage at node 3 (the output of the inverter) is equal to $0.1 \cdot VDD$ (that is, $V(3) = 0.1VDD$).
- Rise time ends when the voltage at node 3 is equal to $0.9 \cdot VDD$ (that is, $V(3) = 0.9VDD$).

For more accurate results, start the `.MEASURE` calculation after either:

- A time delay, or
- A simulation cycle, specifying delay time in the `.MEASURE` statement, or
- An input pulse statement.

The following example features:

- `AUTOSTOP` and `.MEASURE` statements.
- Mean, variance, sigma, and avgdev calculations.
- Circuit and element temperature.
- Algebraic equation handling.
- `PAR( )` as an output variable, in the `.MEASURE` statement.
- Sub-circuit parameter passing, and sub-circuit multiplier.
- `.DATA` statement.

Input File Examples

```

FILE: CELLCHAR.SP
.OPTION SPICE NOMOD AUTOSTOP
.PARAM TD=10N PW=50N TRR=5N TRF=5N VDD=5 LDEL=0 WDEL=0
+ NVT=0.8 PVT=-0.8 DTEMP=0 FANOUT=1
.GLOBAL VDD
* - global supply name
.TEMP 27

```

SUBCKT Definition

```

.SUBCKT INV IN OUT
M1 OUT IN VDD VDD P L=3U W=15U DTEMP=DTEMP
M2 OUT IN 0 0 N L=3U W=8U DTEMP=DTEMP
CL OUT 0 200E-15 .001
CI IN 0 50E-15 .001
.ENDS

```

SUBCKT Calls

```

XINVH 2 3 INV $-- INPUT START HIGH
XOUTL 3 4 INV M=FANOUT
XINVL 2030 INV $-- INPUT START LOW
XOUTH 30 40INV M=FANOUT
* - INPUT VOLTAGE SOURCES
VDD VDD 0 VDD
VINH 2 0 PULSE(VDD,0,TD,TRR,TRF,PW,200NS)
VINL 20 0 PULSE(0,VDD,TD,TRR,TRF,PW,200NS)
* - MEASURE STATEMENTS FOR RISE, FALL, AND PROPAGATION
+ DELAYS
.MEAS RISETIME TRIG PAR('V(3) -0.1*VDD') VAL=0 RISE=1
+ TARG PAR('V(3) -0.9*VDD') VAL=0 RISE=1
.MEAS FALLTIME TRIG PAR('V(30)-0.9*VDD') VAL=0 FALL=1
+ TARG PAR('V(30)-0.1*VDD') VAL=0 FALL=1
.MEAS TPLH TRIG PAR('V(2) -0.5*VDD') VAL=0 FALL=1
+ TARG PAR('V(3) -0.5*VDD') VAL=0 RISE=1
.MEAS TPHL TRIG PAR('V(20)-0.5*VDD') VAL=0 RISE=1
+ TARG PAR('V(30)-0.5*VDD') VAL=0 FALL=1
* - ANALYSIS SPECIFICATION
.TRAN 1N 500N SWEEP DATA=DATNM
* - DATA STATEMENT SPECIFICATION
.DATA DATNM
VDD TRR TRF FANOUT DTEMP NVT PVT LDEL WDEL
5.0 2N 2N 2 0 0.8 -0.8 0 0 $ TYPICAL
5.5 1N 1N 1 -80 0.6 -0.6 -0.2U 0.2U $ BEST
4.5 3N 3N 10 100 1.0 -1.0 +0.2U -0.2U $ WORST
5.0 2N 2N 2 0 1.0 -0.6 0 0 $ STRONG P
$ WEAK N
5.0 2N 2N 2 0 0.6 -1.0 0 0 $ WEAK P
$ STRONG N
5.0 2N 2N 4 0 0.8 -0.8 0 0 $ FANOUT=4
5.0 2N 2N 8 0 0.8 -0.8 0 0 $ FANOUT=8
.ENDDATA

```


Models

```
.MODEL N NMOS LEVEL=2 LDEL=LDEL WDEL=WDEL
+ VTO=NVT      TOX =300      NSUB=1.34E16  U0=600
+ LD=0.4U      WD =0.6U      UCRIT=4.876E4  UEXP=.15
+ VMAX=10E4    NEFF=15      PHI=.71      PB=.7
+ RS=10        RD =10        GAMMA=0.897    LAMBDA=0.004
+ DELTA=2.31   NFS =6.1E11   CAPOP=4
+ CJ=3.77E-4   CJSW=1.9E-10  MJ=.42    MJSW=.128

.MODEL P PMOS LEVEL=2 LDEL=LDEL WDEL=WDEL
+ VTO=PVT      TOX=300      NSUB=0.965E15  U0=250
+ LD=0.5U      WD=0.65U     UCRIT=4.65E4   UEXP=.25
+ VMAX=1E5     NEFF=10      PHI=.574      PB=.7
+ RS=15        RD=15        GAMMA=0.2      LAMBDA=.01
+ DELTA=2.486  NFS=5.2E11   CAPOP=4
+ CJ=1.75E-4   CJSW=2.3E-10  MJ=.42      MJSW=.128
.END
```

A sample of measure statements is printed:

```
*** MEASURE STATEMENT RESULTS FROM THE FIRST ITERATION
*** ($ TYPICAL)
RISETIME = 3.3551E-09 TARG= 1.5027E-08 TRIG= 1.1672E-08
FALLTIME = 2.8802E-09 TARG= 1.4583E-08 TRIG= 1.1702E-08
TPLH     = 1.8537E-09 TARG= 1.2854E-08 TRIG= 1.1000E-08
TPHL     = 1.8137E-09 TARG= 1.2814E-08 TRIG= 1.1000E-08
*** MEASURE STATEMENT RESULTS FROM THE LAST ITERATION
*** ($ FANOUT=8)
RISETIME = 8.7909E-09 TARG= 2.0947E-08 TRIG= 1.2156E-08
FALLTIME = 7.6526E-09 TARG= 1.9810E-08 TRIG= 1.2157E-08
TPLH     = 3.9922E-09 TARG= 1.4992E-08 TRIG= 1.1000E-08
TPHL     = 3.7995E-09 TARG= 1.4800E-08 TRIG= 1.1000E-08
MEAS_VARIABLE = RISETIME
MEAN  = 6.5425E-09  VARIAN = 4.3017E-17
SIGMA = 6.5588E-09  AVGDEV = 4.6096E-09
MEAS_VARIABLE = FALLTIME
MEAN  = 5.7100E-09  VARIAN = 3.4152E-17
SIGMA = 5.8440E-09  AVGDEV = 4.0983E-09
MEAS_VARIABLE = TPLH
MEAN  = 3.1559E-09  VARIAN = 8.2933E-18
SIGMA = 2.8798E-09  AVGDEV = 1.9913E-09
MEAS_VARIABLE = TPHL
MEAN  = 3.0382E-09  VARIAN = 7.3110E-18
SIGMA = 2.7039E-09  AVGDEV = 1.8651E-0
```

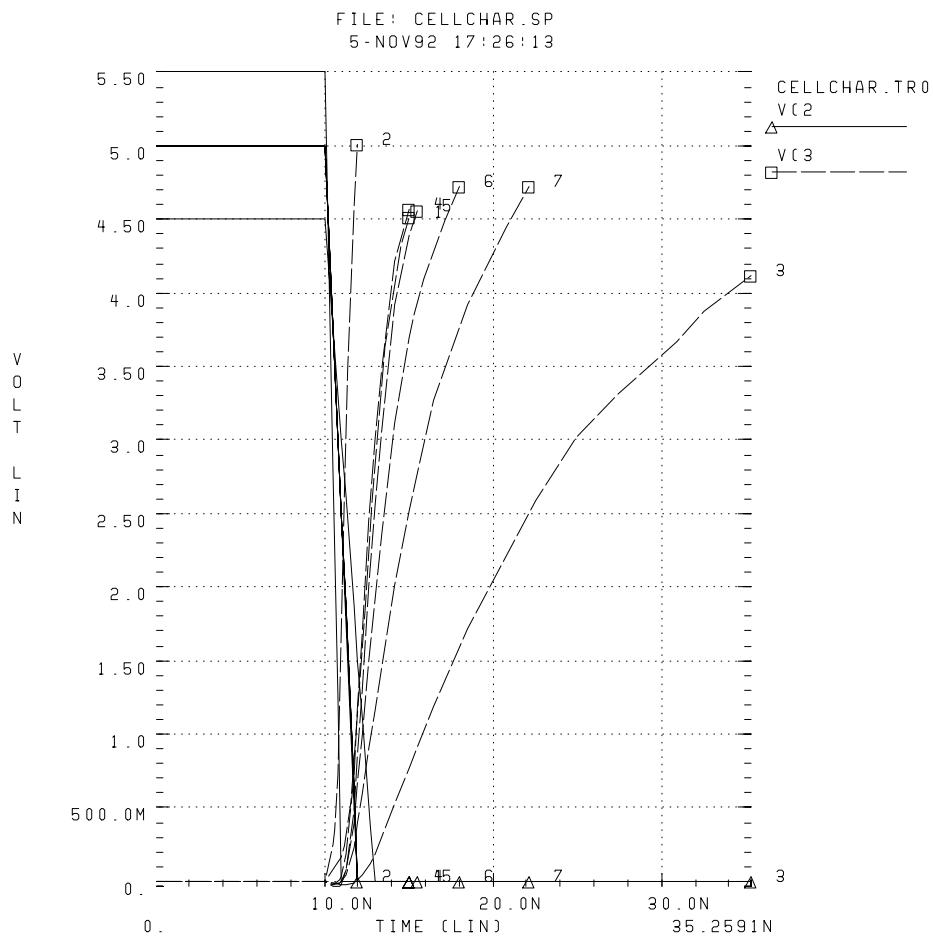
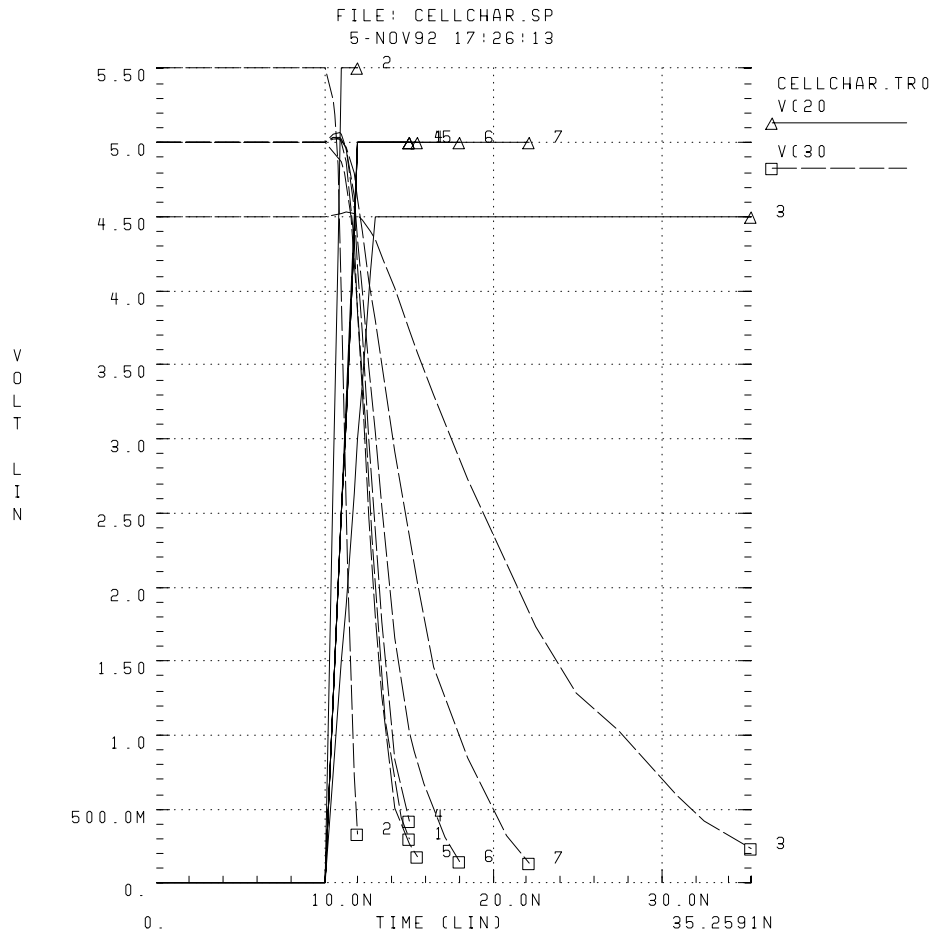
Figure 15-9: Plotting the Simulation Outputs

Figure 15-10: Verifying the Measure Statement Results by the Plots



Chapter 16

Signal Integrity

The performance of an IC design is no longer limited to how many million transistors a vendor fits on a single chip. With tighter packaging space, and increasing clock frequencies, packaging issues and system-level performance issues (such as crosstalk and transmission lines) are becoming increasingly significant.

At the same time, with the popularity of multi-chip packages, and increased I/O counts, package design itself is becoming more like chip design.

This chapter describes how to maintain signal integrity for your design, and explains the following topics:

- [Preparing for Simulation](#)
- [Optimizing TDR Packaging](#)
- [Simulating Circuits with Signetics Drivers](#)
- [Simulating Circuits with Xilinx FPGAs](#)

Preparing for Simulation

To simulate a PC board or backplane, you must use models for:

- Driver cell, including parasitic pin capacitances and package lead inductances.
- Transmission lines.
- A receiver cell, with parasitic pin capacitances and package lead inductances.
- Terminations, or other electrical elements, on the line.

Model the transmission line as closely as possible—that is, to maintain the integrity of the simulation, include all electrical elements, exactly as they are laid out on the backplane or printed circuit board.

You can use readily-available I/O drivers from ASIC vendors, and the True-Hspice advanced lossy transmission lines, to simulate the electrical behavior of the board interconnect, bus, or backplane. You can analyze the transmission line behavior, under various conditions.

You can simulate, because the critical models and simulation technology exist.

- Many manufacturers of high-speed components already use Star-Hspice.
- You can hide the complexity, from the system level.
- Star-Hspice preserves the necessary electrical characteristics, with full transistor-level library circuits.

Star-Hspice can simulate systems, using:

- System-level behavior, such as local component temperature and independent models, to accurately predict electrical behavior.
- Automatic inclusion of library components, using the SEARCH option.
- Lossy transmission line models that:
 - Support common-mode simulation.
 - Include ground-plane reactance.
 - Include resistive loss, of conductor and ground plane.
 - Allow multiple signal conductors.
 - Require minimum CPU computation time.

Signal Integrity Problems

Table 16-1 lists some signal integrity problems, which can cause failures in high-speed designs.

Table 16-1: High-Speed Design Problems and Solutions

Signal Integrity Problem	Causes	Solution
Noise: delta I (current)	Multiple simultaneously-switching drivers; high-speed devices create larger delta I.	Adjust or evaluate location, size, and value of decoupling capacitors.
Noise: coupled (crosstalk)	Closely-spaced parallel traces.	Establish design rules for lengths of parallel lines.
Noise: reflective	Impedance mismatch.	Reduce the number of connectors, and select proper impedance connectors.
Delay: path length	Poor placement and routing; too many or too few layers; chip pitch.	Choose MCM or other high-density packaging technology.
Propagation speed	Dielectric medium.	Choose the dielectric with the lowest dielectric constant.
Delay: rise time degradation	Resistive loss and impedance mismatch.	Adjust width, thickness, and length of line.

Analog Side of Digital Logic

Circuit simulation of a digital system becomes necessary, *only* when the analog characteristics of the digital signals become electrically important. Is the digital circuit a new design, or simply a fast version of the old design? Many new digital products are actually faster versions of existing designs. For example, the transition from a 100 MHz to a 150 MHz Pentium PC might not require extensive logic simulations. However, the integrity of the digital quality of the signals might require careful circuit analysis.

The source of a signal integrity problem is the digital output driver. A high-speed digital output driver can drive only a few inches, before the noise and delay (because of the wiring) become a problem. To speed-up circuit simulation and modeling, you can create analog behavioral models, which mimic the full analog characteristics, at a fraction of the traditional evaluation time.

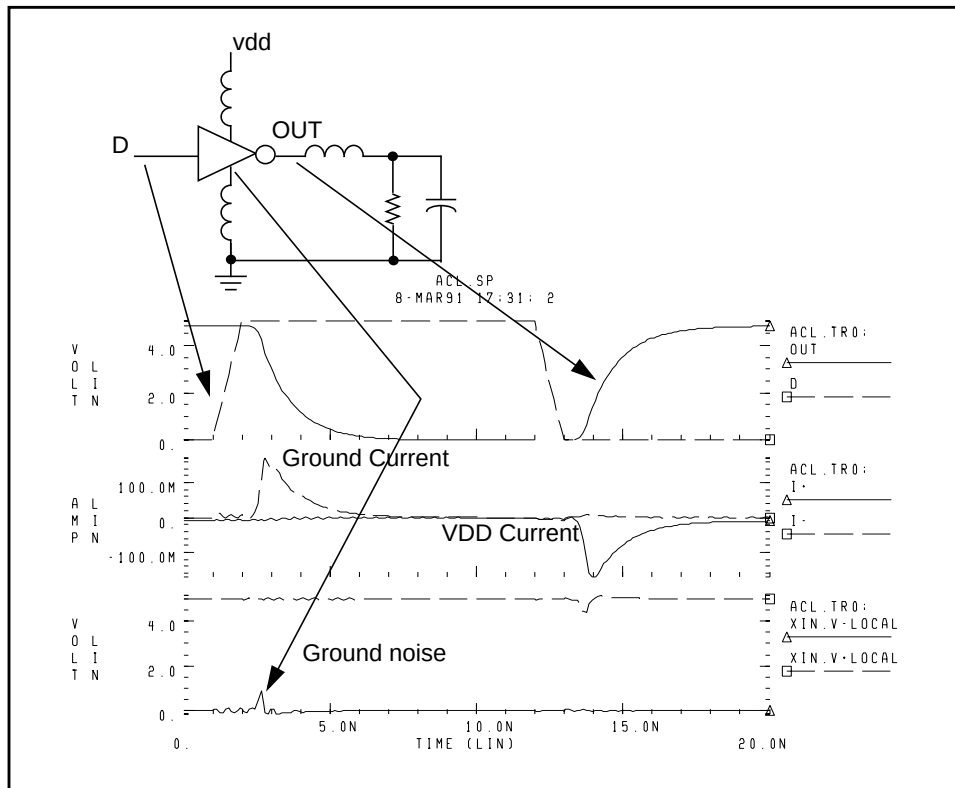
The roadblocks to successful high-speed digital designs are noise and signal delays. Digital noise can originate from several sources. The fundamental digital noise sources are:

- *Line termination noise*: additional voltage, reflected from the load back to the driver, because of impedance mismatch. Digital output buffers are not designed to accurately control the output impedance. Most buffers have different rising and falling edge impedances.
- *Ground bounce noise*: generated where leadframes, or other circuit wires, cannot form into transmission lines. The resulting inductance creates an induced voltage in:
 - the ground circuit
 - the supply circuit
 - the output driver circuit

Ground bounce noise lowers the noise margins for the rest of the system.

- *Coupled line noise*: noise induced from lines that are physically adjacent. This noise is generally most severe for data lines that are next to clock lines.

Simulating the output buffer in [Figure 16-1 on page 16-5](#) demonstrates the analog behavior of a digital gate circuit, as simulated in Star-Hspice.

Figure 16-1: Simulating Output Buffer with 2 ns Delay and 1.8 ns Rise/Fall Times

Circuit delays become critical, as timing requirements become tighter. The key circuit delays are:

- Gate delays.
- Line turnaround delays, for tristate buffers.
- Line length delays (clock skew).

Logic analysis addresses only gate delays. You can compute the variation in the gate delay from a circuit simulation, only if you understand the best case and worst case manufacturing conditions.

The line turnaround delays add to the gate delays, because you must add extra margin, so that multiple tristate buffer drivers do not simultaneously turn on. The line-length delay affects the clock skew most directly, in most systems.

As system cycle times approach the speed of electromagnetic signal propagation for the printed circuit board, consideration of the line length becomes critical. The system noises and line delays interact with the electrical characteristics of the gates, and might require circuit level simulation.

Analog details find digital systems problems. Exceeding the noise quota might not cause a system to fail. Maximum noise becomes a problem, only when Star-Hspice is accepting a digital input. If a digital systems engineer can decouple the system, Star-Hspice can accept much higher noise.

Common decoupling methods are:

- Multiple ground and power planes, on the PCB, MCM, and PGA.
- Separating signal traces, with ground traces.
- Decoupling capacitors.
- Series resistors, on output buffer drivers.
- Twisted-pair line driving.

In present systems designs, you must select the best packaging methods at three levels:

- printed circuit board
- multi-chip module
- pin grid array

Extra ground and power planes are often necessary, to lower the supply inductance, and to provide decoupling.

- Decoupling capacitors must have very low internal inductance, to be effective for high-speed designs.
- Newer designs frequently use series resistance in the output drivers, to lower circuit ringing.
- Critical high-speed driver applications use twisted differential-pair transmission lines.

A systems engineer must determine how to partition the logic. The propagation speed of signals on a printed circuit board is about 6 in/ns. As digital designs become faster, wiring interconnects become a factor in how you partition logic.

The critical wiring systems are:

- IC-level wiring.
- Package wiring, for SIPs, DIPs, PGAs, and MCMs.
- Printed circuit-board wiring.
- Backplane and connector wiring.
- Long lines – power, coax, or twisted pair.

If you use ASIC or custom integrated circuits (as part of your system logic partitioning strategy), you must make decisions about integrated circuit level wiring. The more-familiar decisions involve selecting packages, and arranging packages on a printed circuit board. Large systems generally have a central backplane, which becomes the primary challenge at the system partition level.

Use the following equation to estimate wire length, when transmission line effects become noticeable:

$$\text{critical length} = (\text{rise time}) * \text{velocity} / 8$$

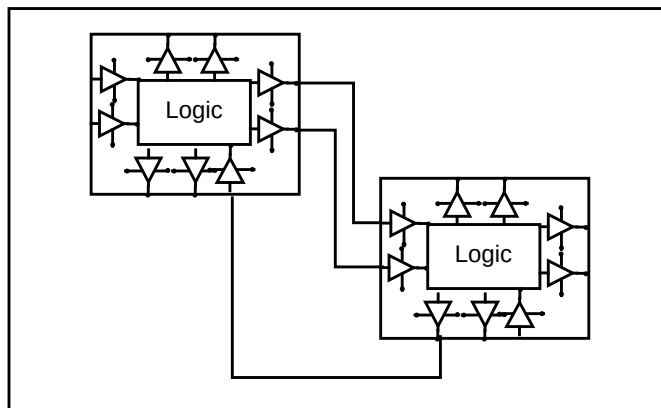
For example, if rise time is 1 ns, and board velocity is 6 in/ns, then distortion becomes noticeable when wire length is 3/4 in. The Star-Hspice circuit simulator automatically generates models for each type of wire, to define effects of full loss transmission lines.

To partition a system, ECL logic design engineers typically calculated the noise quota for each line. Now, you must design most high-speed digital logic with respect to the noise quota, so that the engineer knows how much noise and delay are acceptable, before timing and logic levels fail.

To solve the noise quota problem, you must calculate the noise associated with the wiring. You can separate large integrated circuits into two parts:

- Internal logic.
- External input and output amplifiers.

When you use mixed digital and analog tools, such as Avant! Star-Hspice and Viewlogic Viewsim A/D, you can merge a complete system together, with full analog-quality timing constraints, and full digital representation. You can simultaneously evaluate noise-quota calculations, subject to system timing.

Figure 16-2: Analog Drivers and Wires

Optimizing TDR Packaging

Packaging plays an important role in determining the overall speed, cost, and reliability of a system. With today's small feature sizes, and high levels of integration, a significant portion of the total delay is the time required for a signal to travel between chips.

Multi-layer ceramic technology has proven to be well suited for high-speed GaAs IC packages.

A multi-chip module (MCM) minimizes the chip-to-chip spacing. It also reduces the inductive and capacitive discontinuity, between the chips mounted on the substrate. An MCM uses a more direct path (die-bump-interconnect-bump-die), which eliminates wire bonding. In addition, narrower and shorter wires on the ceramic substrate have much less capacitance and inductance, than PC board interconnections have.

Time domain reflectometry (TDR) is the closest measurement to actual digital component functions. It provides a transient display of the impedance versus time, for pulse behavior.

Using TDR in Simulation

When you use a digitized TDR file, you can use the Star-Hspice optimizer to automatically select design components. To extract critical points from digitized TDR files, use the `.MEASURE` statement, and use the results as electrical specifications for optimization. This process eliminates recurring design cycles, to find component values that curve-fit the TDR files.

Figure 16-3: Optimization Process

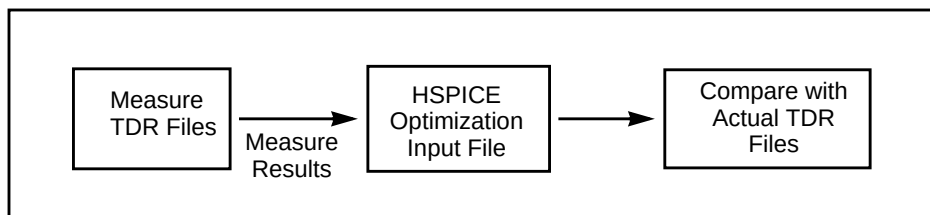
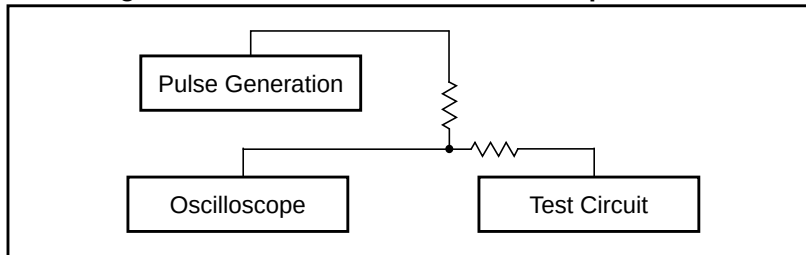


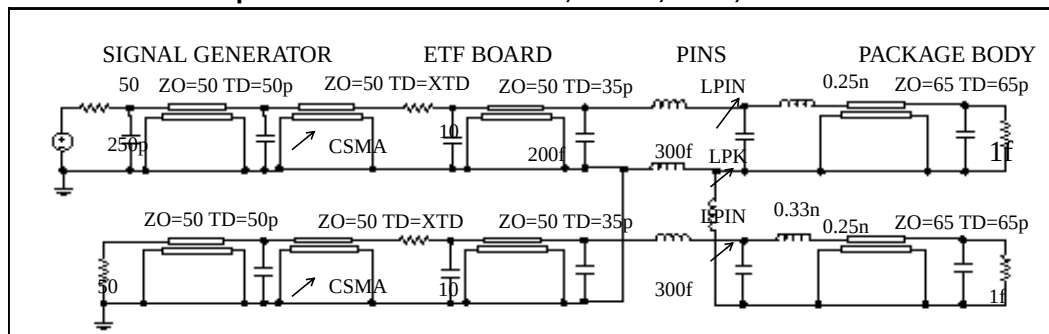
Figure 16-4: General Method for TDR Optimization

Use the following method for realistic high-speed testing of packaging.

- Test fixtures closely emulate a high-speed system environment.
- A True-Hspice device model uses ideal transmission lines and discrete components, for measurements.

The tested circuit contains the following components:

- Signal generator.
- Coax, connecting the signal generator to ETF (engineering test fixture) board.
- ETF board.
- Package pins.
- Package body.

**Figure 16-5: SPICE Model for Package-Plus-Test Fixture
Optimized Parameters: XTD, CSMA, LPIN, and LPK**

The package tests use a digital sampling oscilloscope to perform traditional time-domain measurements. Use these tests to observe the reflected and transmitted signals. These signals are derived from the built-in high-speed pulse generator, and translated output signals, into digitized time-domain reflectometer files (voltage versus time).

Use a fully-developed SPICE model to simulate the package-plus-test fixture, then compare the simulated and measured reflected/transmitted signals.

The next section shows the input netlist file for this experiment. [Figure 16-6](#) through [Figure 16-9](#) show the output plots. To investigate this experiment, use the advanced lossy transmission lines, to include attenuation and dispersion.

TDR Optimization Procedure

Measure Critical Points in the TDR Files

```
Vin 1 0 PWL(TIME,VOLT)
.DATA D_TDR
      TIME      VOLT
      0          0.5003mV
      0.1n       0.6900mV
      ...
      2.0n       6.4758mV
.ENDDATA
.TRAN DATA=D_TDR
.MEAS .....
.END
```

Set Up an Input Optimization File

```
$ SPICE MODEL FOR PACKAGE-PLUS-TEST FIXTURE
$ AUTHOR: DAVID H. SMITH & RAJ M. SAVARA
.OPTION POST RELV=1E-4 RELVAR=1E-2
$ DEFINE PARAMETERS

.PARAM LV=-0.05 HV=0.01 TD=1P TR=25P TF=50P TPW=10N
+ TPER=15N

$ PARAMETERS TO BE OPTIMIZED
.PARAM CSMA=OPT1(500f,90f,900f,5f)
+ XTD=OPT1(150p,100p,200p)
+ LPIN=OPT1(0.65n,0.10n,0.90n,0.2n)
+ LPK=OPT1(1.5n,0.75n,3.0n,0.2n)
+ LPKCL=0.33n
+ LPKV=0.25n
```

Signal Generator

```
VIN    S1 GND PULSE LV HV TD TR TF TPW TPER
RIN    S1 S2 50
CIN1   S2 GND 250f
TCOAX  S2 GND SIG_OUT GND Z0=50 TD=50p
```

ETF Board

```
CSNAL  SIG_OUT GND CSMA
TEFT2  SIG_OUT GND E3 GND Z0=50 TD=XTD
RLOSS1 E3 E4 10
CRPAD1 E4 GND 200f
TLIN2  E4 GND ETF_OUT GND Z0=50 TD=35p
CPAD2  ETF_OUT GND 300f
TLIN1  E5 GND E6 GND Z0=50 TD=35p
CPAD1  E5 GND 300f
CRPAD2 E6 GND 200f
RLOS1  E6 E7 10
TEFT1  E7 GND E8 GND Z0=50 TD=XTD
CSMA2  E8 GND CSMA
TCOAX2 E8 GND VREF1 GND Z0=50 TD=50p
RIN1   VREF1 GND 50
```

Package Body

```
LPIN1  ETF_OUT P1 LPIN
LPK1   GND P5 LPK
LPKGCL P5 NVOUT2 LPKCL
CPKG1  P1 P5 250f
LPKV1  P1 P2 LPKV
TPKGL  P2 NVOUT2 VOUT NVOUT2 Z0=65 TD=65P
CBPL   VOUT NVOUT 1f
ROUT1  VOUT NVOUT 50meg
LPIN2  E5 P3 LPIN
CPKG2  P3 NVOUT2 250f
LPKV2  P3 P4 LPKV
TPKG2  P4 NVOUT2 VOUT2 NVOUT2 Z0=65 TD=65p
CBPD1  VOUT2 NVOUT2 1f
ROUT2  VOUT2 NVOUT2 50meg

$ BEFORE OPTIMIZATION
.TRAN .004NS 2NS

$ OPTIMIZATION SETUP
.MODEL OPTMOD OPT ITROPT=30
.TRAN .05NS 2NS SWEEP OPTIMIZE=OPT1
+ RESULTS=MAXV, MINV, MAX_2, COMP1, PT1, PT2, PT3
+ MODEL=OPTMOD
```



```

$ MEASURE CRITICAL POINTS IN THE REFLECTED SIGNAL
$ GOALS ARE SELECTED FROM MEASURED TDR FILES
.MEAS TRAN COMP1 MIN V(S2) FROM=100p TO=500p
+ GOAL=-27.753
.MEAS TRAN PT1 FIND V(S2) AT=750p GOAL=-3.9345E-3
+ WEIGHT=5
.MEAS TRAN PT2 FIND V(S2) AT=775p GOAL=2.1743E-3
+ WEIGHT=5
.MEAS TRAN PT3 FIND V(S2) AT=800p GOAL=5.0630E-3
+ WEIGHT=5

$ MEASURE CRITICAL POINTS IN THE TRANSMITTED SIGNAL
$ GOALS ARE SELECTED FROM MEASURED TDR FILES
.MEAS TRAN MAXV FIND V(VREF1) AT=5.88E-10 GOAL=6.3171E-
+ WEIGHT=7
.MEAS TRAN MINV FIND V(VREF1) AT=7.60E-10 GOAL=-
+ 9.9181E-3
.MEAS TRAN MAX_2 FIND V(VREF1) AT=9.68E-10
+ GOAL=4.9994E-3

$ COMPARE SIMULATED RESULTS WITH MEASURED TDR VALUES
.TRAN .004NS 2NS
.PRINT C_REF=V(S2) C_TRAN=V(VREF1)
.END

```

Figure 16-6: Reflected Signals Before Optimization

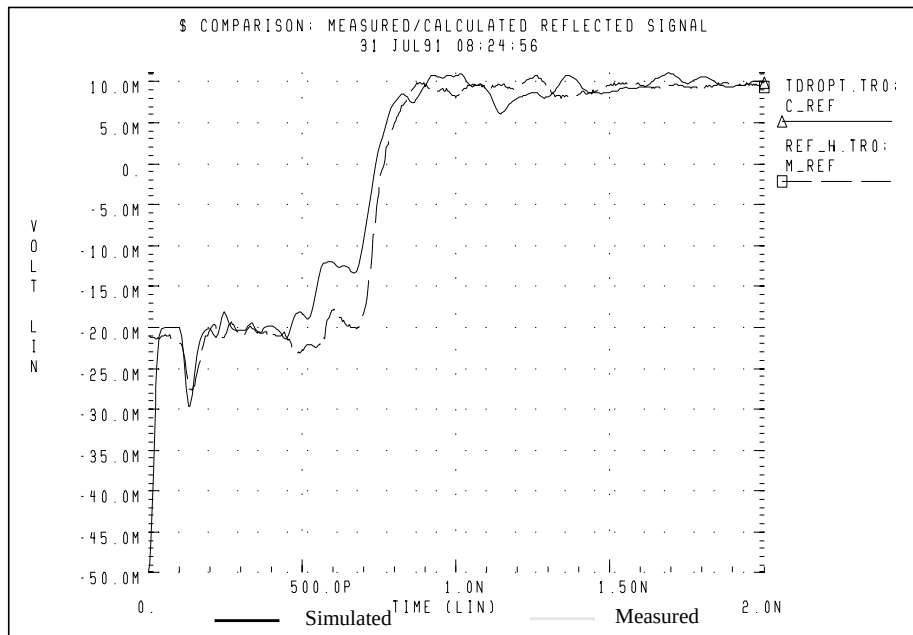


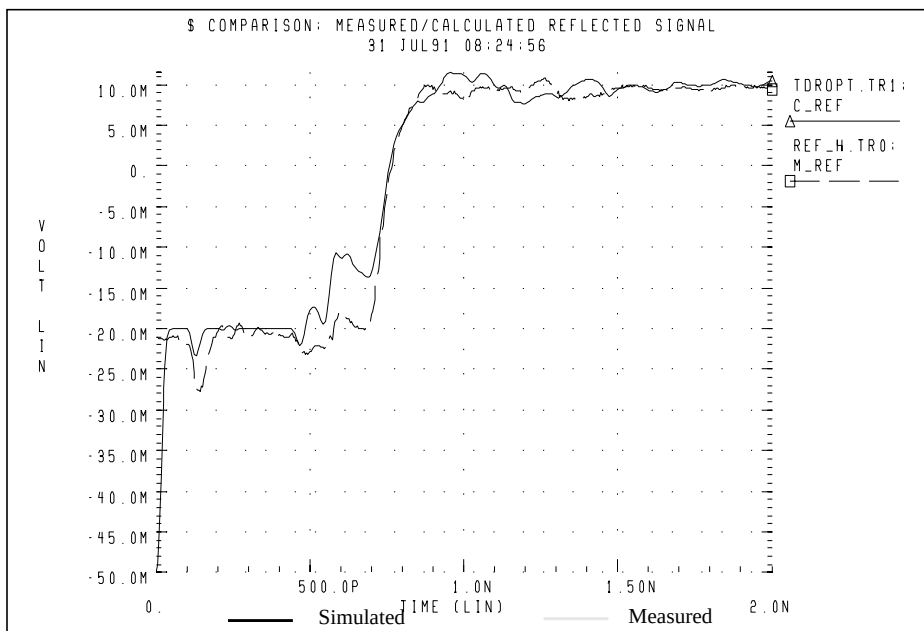
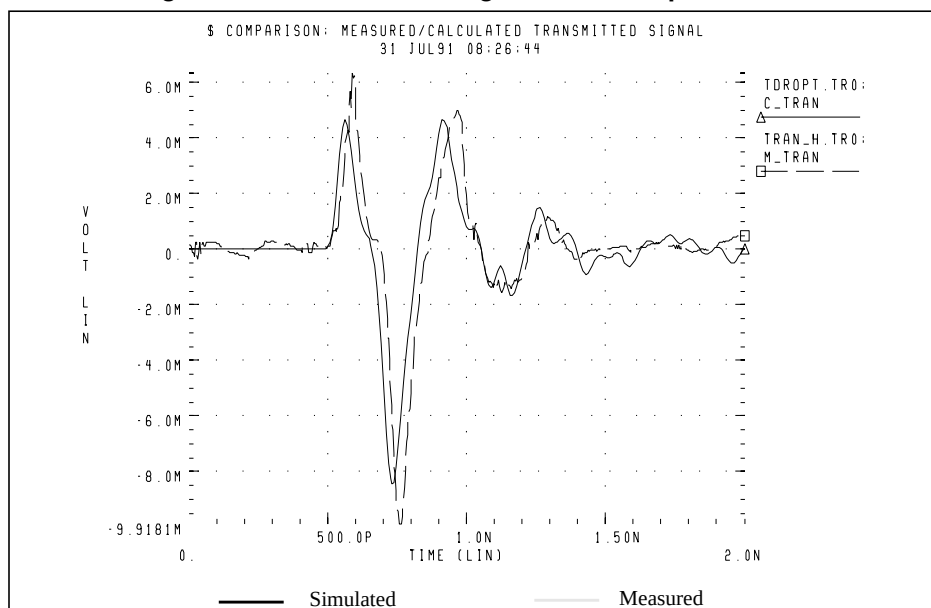
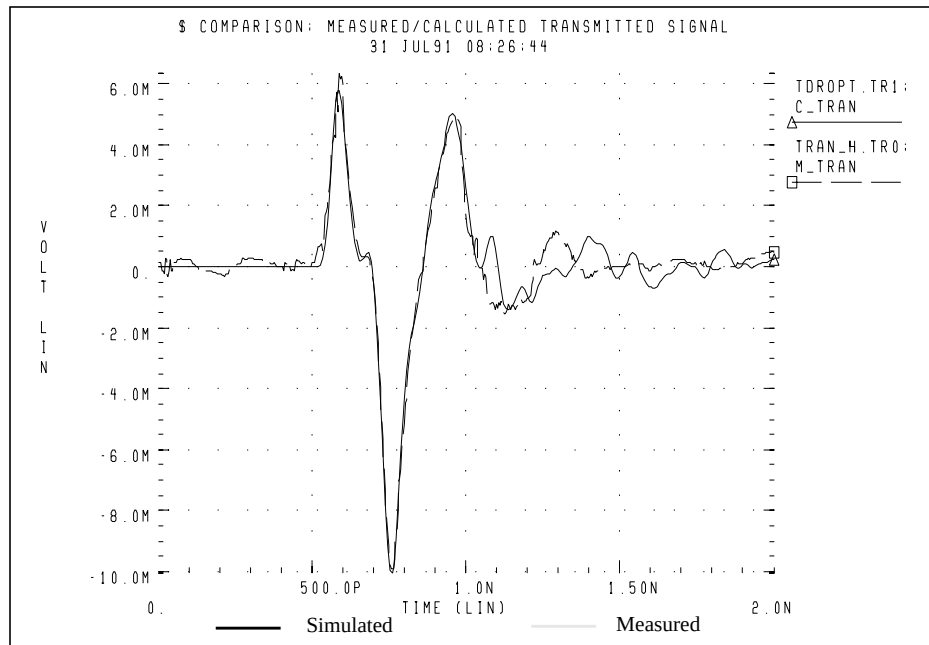
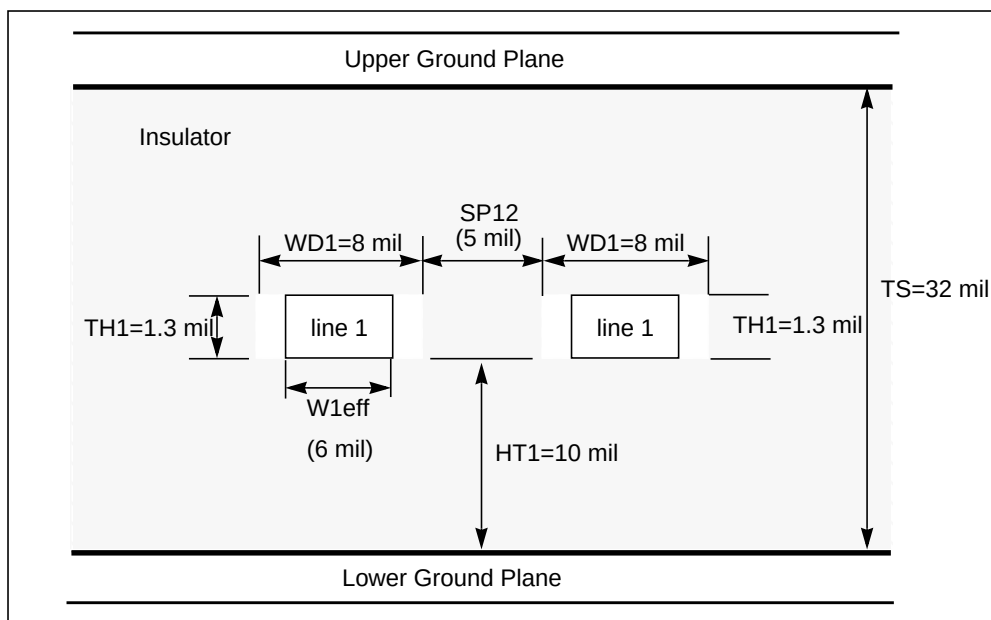
Figure 16-7: Reflected Signals After Optimization**Figure 16-8: Transmitted Signals Before Optimization**

Figure 16-9: Transmitted Signals after Optimization

Simulating Circuits with Signetics Drivers

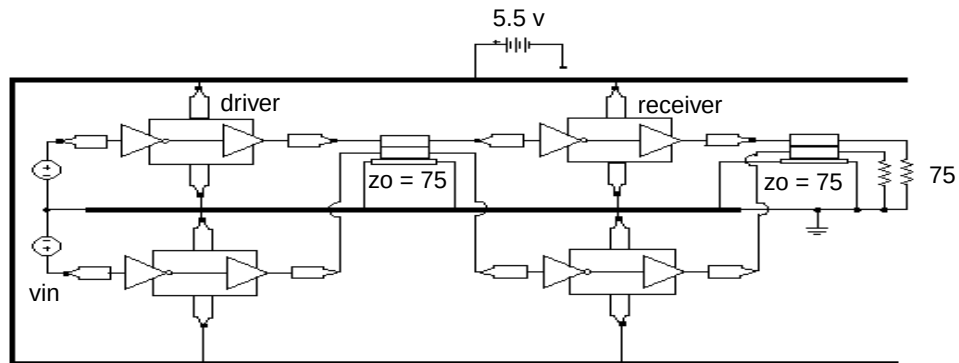
Star-Hspice includes a Signetics I/O buffer library, in the `$installdir/parts/signet` directory. You can use these high-performance parts in backplane design. Transmission line models describe two conductors.

Figure 16-10: Planar Transmission Line DLEV=2: Microstrip Sea of Dielectric



In the following application, a pair of drivers are driving about 2.5 inches of adjacent lines, to a pair of receivers that drive about four inches of line.

Figure 16-11: I/O Drivers/Receivers with Package Lead Inductance, Parallel 4" Lossy Microstrip Connectors



Example

This example connects I/O chips with transmission lines.

```
.OPTION SEARCH='$installdir/parts/signet'
.OPTION POST=2 TNOM=27 NOMOD LIST METHOD=GEAR
.TEMP 27
$ DEFINE PARAMETER VALUES
.PARAM LV=0 HV=3 TD1=10n TR1=3n TF1=3n TPW=20n
+ TPER=100n TD2=20n TR2=2n TF2=2n LNGTH=101.6m
$ POWER SUPPLY
VCC VCC 0 DC 5.5
$ INPUT SOURCES
VIN1 STIM1 0 PULSE LV HV TD1 TR1 TF1 TPW TPER
VIN2 STIM2 0 PULSE LV HV TD2 TR2 TF2 TPW TPER

$ FIRST STAGE: DRIVER WITH TLINE
X1ST_TOP STIM1 OUTPIN1 VCC GND IO_CHIP PIN_IN=2.6n PIN_OUT=4.6n
X1ST_DN STIM2 OUTPIN2 VCC GND IO_CHIP PIN_IN=2.9n PIN_OUT=5.6n
U_1ST OUTPIN1 OUTPIN2 GND TLOUT1 TLOUT2 GND USTRIP L=LNGTH

$ SECOND STAGE: RECEIVER WITH TLINE
X2ST_TOP TLOUT1 OUTPIN3 VCC GND IO_CHIP PIN_IN=4.0n PIN_OUT=2.5n
X2ST_DN TLOUT2 OUTPIN4 VCC GND IO_CHIP PIN_IN=3.6n PIN_OUT=5.1n
U_2ST OUTPIN3 OUTPIN4 GND TLOUT3 TLOUT4 GND USTRIP L=LNGTH

$ TERMINATING RESISTORS
R1 TLOUT3 GND 75
R2 TLOUT4 GND 75

$ IO CHIP MODEL - SIGNETICS
.SUBCKT IO_CHIP IN OUT VCC XGND PIN_VCC=7n PIN_GND=1.8n
X1 IN1 INVOUT VCC1 XGND1 ACTINPUT
X2 INVOUT OUT1 VCC1 XGND1 AC109EQ
```

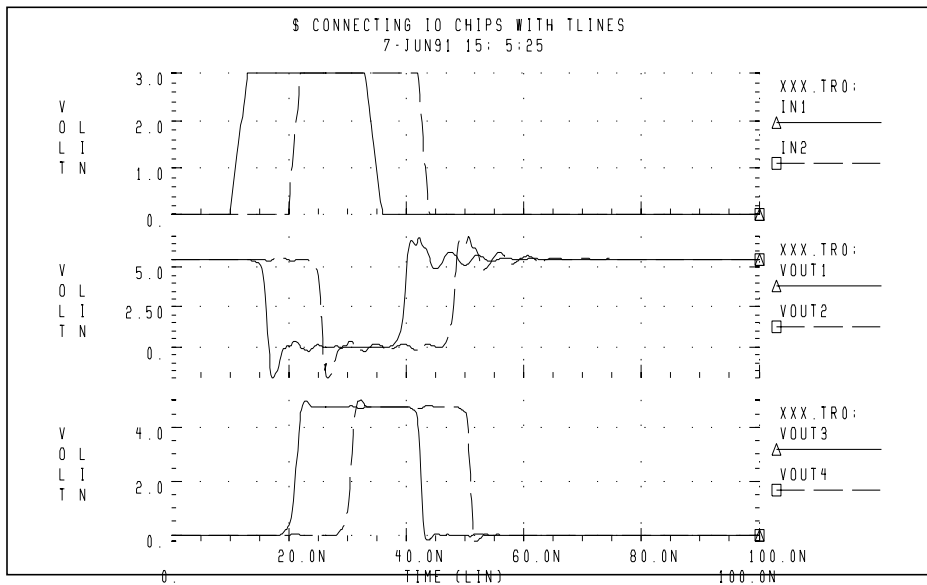
Package Inductance

```

LIN_PIN IN IN1 PIN_IN
LOUT_PIN OUT1 OUT PIN_OUT
LVCC VCC VCC1 PIN_VCC
LGND XGND1 XGND PIN_GND
.ENDS
$ TLINE MODEL - 2 SIGNAL CONDUCTORS WITH GND
$ PLANE
.MODEL USTRIP U LEVEL=3 ELEV=1 PLEV=1
+ TH1=1.3mil HT1=10mil TS=32mil KD1=4.5 DLEV=0 WD1=8mil
+ XW=-2mil KD2=4.5 NL=2 SP12=5mil
$ ANALYSIS / PRINTS
.TRAN .1NS 100NS
.GRAPH IN1=V(STIM1) IN2=V(STIM2) VOUT1=V(TLOUT1)
+ VOUT2=V(TLOUT2)
.GRAPH VOUT3=V(TLOUT3) VOUT4=V(TLOUT4)
.END

```

Figure 16-12: Connecting I/O Chips with Transmission Lines



Simulating Circuits with Xilinx FPGAs

Avant! and Xilinx maintain a library of True-Hspice, transistor-level sub-circuits, for the Xilinx 3000 and 4000 series Field Programmable Gate Arrays (FPGAs). These sub-circuits model the input and output buffer.

The following simulations use the Xilinx input/output buffer (`xil_iob.inc`) to simulate ground-bounce effects, for the 1.08 μ m process, at room temperature and at nominal model conditions. In the IOB and IOB4 sub-circuits, you can set parameters to specify:

- Local temperature.
- Fast, slow, or typical speed.
- 1.2 μ or 1.08 μ technology.

You can use these choices to perform a variety of simulations, to measure:

- Ground bounce, as a function of package, temperature, part speed, and technology.
- Coupled noise, both on-chip and chip-to-chip.
- Full transmission line effects at, the package level and the printed circuit board level.
- Peak current, and instantaneous power consumption, for power supply bus considerations, and chip capacitor placement.

Syntax for IOB (`xil_iob`) and IOB4 (`xil_iob4`)

```
* EXAMPLE OF CALL FOR 1.2U PART:
* X1 I 0 PAD TS FAST PPUB TTL VDD GND XIL_IOB
*+ XIL_SIG=0 XIL_DTEMP=0 XIL_SHRINK=0

* EXAMPLE OF CALL FOR 1.08U PART:
* X1 I 0 PAD TS FAST PPUB TTL VDD GND XIL_IOB
*+ XIL_SIG=0 XIL_DTEMP=0 XIL_SHRINK=1
```

Nodes	Description
I (IOB only)	output of the TTL/CMOS receiver
O (IOB only)	input pad driver stage

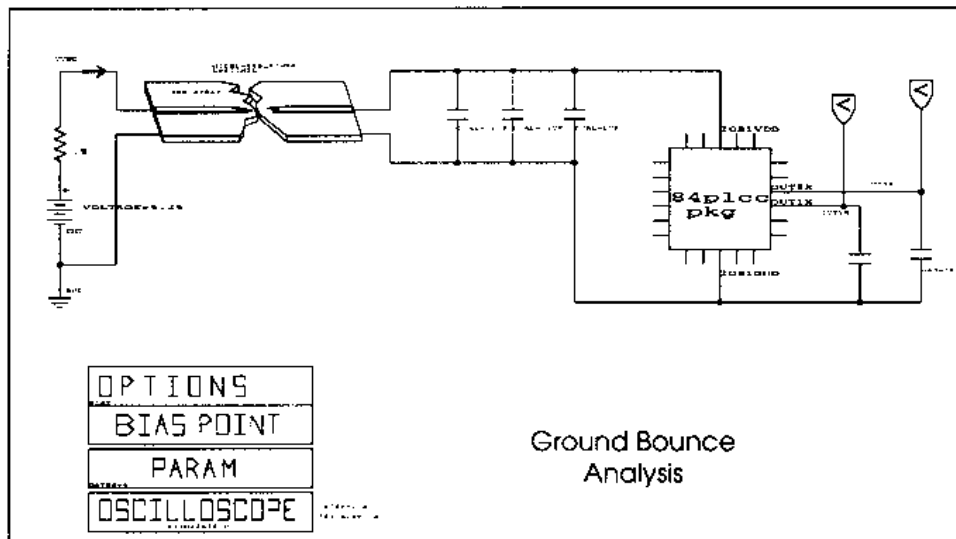
Nodes	Description
I1 (IOB4 only)	input data 1
I2 (IOB4 only)	input data 2
DRIV_IN (IOB4 only)	
PAD	bonding pad connection
TS	three-state control input (5 V disables)
FAST	slew rate control (5 V fast)
PPUB (IOB only)	pad pull-up enable (0 V enables)
PUP (IOB4 only)	pad pull-up enable (0 V enables)
PDOWN (IOB4 only)	pad pull-up enable (5 V enables)
TTL (IOB only)	CMOS/TTL input threshold (5 V selects TTL)
VDD	5-volt supply
GND	ground
XIL_SIG	model distribution: (default 0) -3==> slow 0==> typical +3==> fast
XIL_DTEMP	Buffer temperature difference, from ambient. The default = 0 degrees, if ambient is 25 degrees, and if the buffer is 10 degrees hotter than XIL_DTEMP=10.
XIL_SHRINK	Old or new part; (default is new): ■ 0==>old ■ 1==>new

All grounds and supplies are common to the external nodes, for ground and VDD. You can redefine grounds, to add package models.

Ground Bounce Simulation

Ground-bounce simulation duplicates the Xilinx internal measurements methods. It simultaneously toggles 8 to 32 outputs. The simulation loads each output with a 56-pF capacitance. Simulation also uses an 84-pin package mode, and an output buffer held at chip ground, to measure the internal ground bounce.

Figure 16-13: Ground Bounce Simulation



Star-Hspice adjusts the simulation model for the oscilloscope recordings, so you can use it for the two-bond wire ground.

Input File for Ground Bounce

```
qabounce.sp test of xilinx i/o buffers
* The following is the netlist for the above schematic
* (Figure 16-13)

.op
.option post list
.tran 1ns 50ns sweep gates 8 32 4
.measure bounce max v(out1x)
*.tran .1ns 7ns
.param gates=8
.print v(out1x) v(out8x) i(vdd) power
```

```

$.param xil_dtemp=-65 $ -40 degrees c
$ (65 degrees from +25 degrees)
vdd vdd gnd 5.25
vgnd return gnd 0

upower1 vdd return iob1vdd iob1gnd pcb_power
+ L=600mil

* local power supply capacitors
xc1a iob1vdd iob1gnd cap_mod cval=.1u
xc1b iob1vdd iob1gnd cap_mod cval=.1u
xc1c iob1vdd iob1gnd cap_mod cval=1u
xgnd_b iob1vdd iob1gnd out8x out1x xil_gnd_test
xcout8x out8x iob1gnd cap_mod m=gates
xcout1x out1x iob1gnd cap_mod m=1

.model pcb_power u LEVEL=3 elev=1 plev=1 nl=1 llev=1
+ th=1.3mil ht=10mil kd=4.5 dlev=1 wd=500mil xw=-2mil

.macro cap_mod node1 node2 cval=56p
Lr1 node1 node1x L=2nh R=0.05
cap node1x node2x c=cval
Lr2 node2x node2 L=2nh R=0.05
.eom

.macro xil_gnd_test vdd gnd outx outref
+ gates=8
* example of 8 iobuffers simultaneously switching
* through approx. 4nh lead inductance
* 1 iob is active low for ground bounce measurements

vout drive chipgnd pwl 0ns 5v, 10ns 5v, 10.5ns 0v,
$+ 20ns 0v, 20.5ns 5v, 40ns 5v R

x8 I8 drive PAD8x TS FAST PPUB TTL chipvdd chipgnd
+ xil_iob xil_sig=0 xil_dtemp=0 xil_shrink=1 M=gates
x1 I1 gnd PAD1x TS FAST PPUB TTL chipvdd chipgnd
+ xil_iob xil_sig=0 xil_dtemp=0 xil_shrink=1 m=1

```

Control Settings

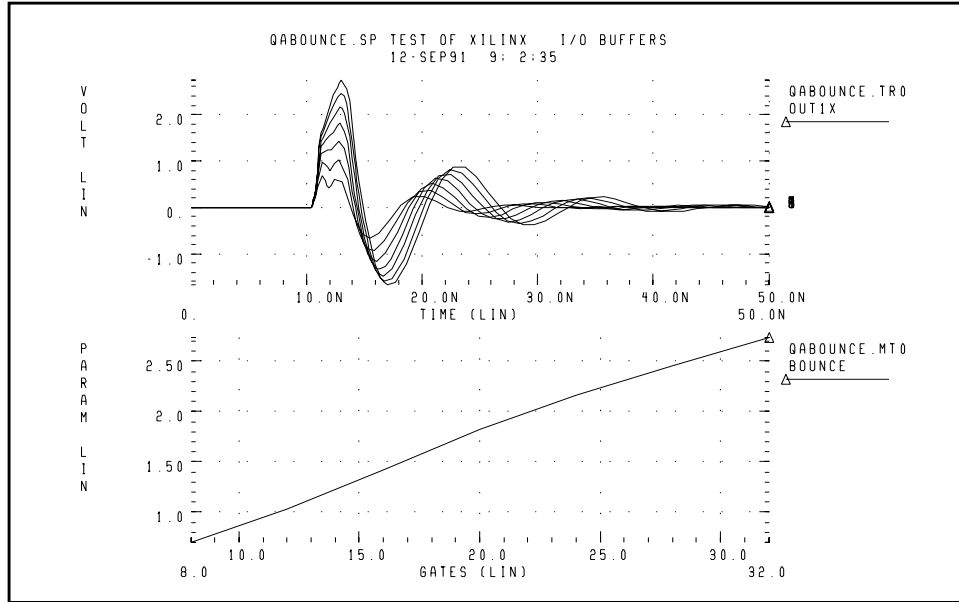
```

rts    ts    chipgnd 1
rfast  fast  chipvdd 1
rppub  ppub  chipgnd 1
rttl   ttl   chipvdd 1

* pad model plcc84 rough estimates
lvdd   vdd   chipvdd L=3.0nh r=.02
lgnd   gnd   chipgnd L=3.0nh r=.02
lout8x outx pad8x L='5n/gates' r='0.05/gates'
lout1x outref pad1x L=5nh r=0.05
c_vdd_gnd chipvdd chipgnd 100n

.eom
.end

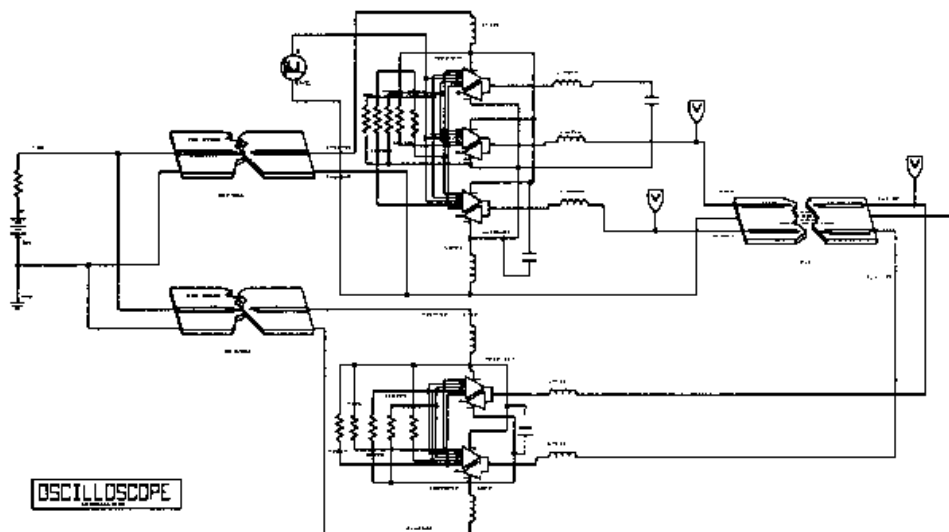
```

Figure 16-14: Results of Ground Bounce Simulation

Coupled Line Noise

This example uses coupled noise, to separate IOB parts. The output of one part drives the input of the other part, through 0.6 inch of PCB. The example also monitors an adjacent quiet line.

Figure 16-15: Coupled Noise Simulation



Coupled Noise

Input File, for qa8.sp test of xilinx 0.8u i/o buffers

* This netlist is for the schematic in [Figure 16-15](#).

```
.op
.option nomod post=2
*.tran .1ns 5ns sweep xil_sig -3 3 3
.tran .1ns 15ns
.print v(out1x) v(out3x) i(vdd) v(irec)
vdd vdd gnd 5
vgnd return gnd 0
upower1 vdd return iob1vdd iob1gnd pcb_power L=600mil
upower2 vdd return iob2vdd iob2gnd pcb_power L=600mil
x4io iob1vdd iob1gnd out3x out1x outrec irec xil_iob4
cout3x out3x iob1gnd 9pf
u1x out1x outrec iob1gnd i_o_in i_o_out iob2gnd pcb_top
L=2000mil
xrec iob2vdd iob2gnd i_o_in i_o_out xil_rec
.ic i_o_out 0v
.model pcb_top u LEVEL=3 elev=1 plev=1 nl=2 llev=1
+ th=1.3mil ht=10mil sp=5mil kd=4.5 dlev=1 wd=8mil xw=-2mil
.model pcb_power u LEVEL=3 elev=1 plev=1 nl=1 llev=1
+ th=1.3mil ht=10mil kd=4.5 dlev=1 wd=500mil xw=-2mil
.macro xil_rec vdd gnd tri1 tri2
* example of 2 iobuffers in tristate
```

```

xtri1 Irec 0 pad_tri1 TSrec FAST PPUB TTL
+ chipvdd chipgnd xil_iob xil_sig=0 xil_dtemp=0 xil_shrink=1 m=1
xtri2 Irec 0 pad_tri2 TSrec FAST PPUB TTL
+ chipvdd chipgnd xil_iob xil_sig=0 xil_dtemp=0
+ xil_shrink=1 m=1

```

Control Setting

```

rin_output      0      chipgnd 1
rtsrec tsrec chipvdd 1
rfast fast chipvdd 1
rppub ppub chipgnd 1
rttl ttl chipvdd 1

* pad model plcc84 rough estimates
lvdd vdd chipvdd L=1nh r=.01
lgnd gnd chipgnd L=1nh r=.01
ltri1 tri1 pad_tri1 L=3nh r=0.01
ltri2 tri2 pad_tri2 L=3nh r=.01
c_vdd_gnd chipvdd chipgnd 100n
.eom

.macro xil_iob4 vdd gnd out3x out1x outrec Irec
* example of 4 iobuffers simultaneously switching
* through approx. 3nh lead inductance
* 1 iob is a receiver (tristate)

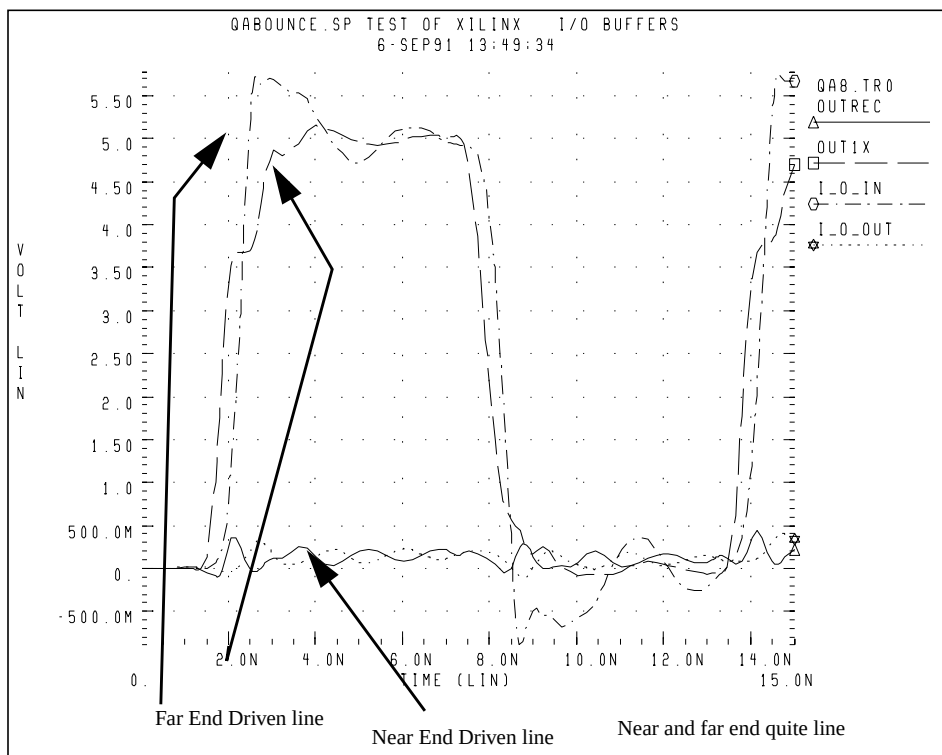
vout 0 chipgnd pwl 0ns 0v, 1ns 0v, 1.25ns 4v, 7ns 4v,
+ 7.25ns 0v, 12ns 0v R

x3 I3 0 PAD3x TS FAST PPUB TTL chipvdd chipgnd xil_iob
+ xil_sig=0 xil_dtemp=0 xil_shrink=1 m=3
x1 I1 0 PAD1x TS FAST PPUB TTL chipvdd chipgnd xil_iob
+ xil_sig=0 xil_dtemp=0 xil_shrink=1 m=1
xrec Irec 0 PADrec TSrec FAST PPUB TTL chipvdd chipgnd xil_iob
+ xil_sig=0 xil_dtemp=0 xil_shrink=1 m=1

* control settings
rts ts chipgnd 1
rtsrec tsrec chipvdd 1
rfast fast chipvdd 1
rppub ppub chipgnd 1
rttl ttl chipvdd 1

* pad model plcc84 rough estimates
lvdd vdd chipvdd L=1nh r=.01
lgnd gnd chipgnd L=1nh r=.01
lout3x out3x pad3x L=1nh r=.0033
lout1x out1x pad1x L=4nh r=0.01
loutrec outrec padrec L=4nh r=.01
c_vdd_gnd chipvdd chipgnd 100n
.eom
.end

```

Figure 16-16: Results of Coupled Noise Simulation

IOB Buffer Module

* INPUT/OUTPUT BLOCK MODEL

* PINS:

* I OUTPUT OF THE TTL/CMOS INPUT RECEIVER.
 * O INPUT TO THE PAD DRIVER STAGE.
 * PAD BONDING PAD CONNECTION.
 * TS THREE-STATE CONTROL INPUT. HIGH LEVEL
 * DISABLES PAD DRIVER.
 * FAST SLEW RATE CONTROL. HIGH LEVEL SELECTS
 * FAST SLEW RATE.
 * PPUB PAD PULLL-UP ENABLE. ACTIVE LOW.
 * TTL CMOS/TTL INPUT THRESHOLD SELECT. HIGH
 * SELECTS TTL.
 * VDD POSITIVE SUPPLY CONNECTION FOR INTERNAL
 * CIRCUITRY.

```

*   ALL SIGNALS ABOVE ARE REFERENCED TO NODE 0.
*   THIS MODEL CAUSES SOME DC CURRENT TO FLOW
*   INTO NODE 0, WHICH IS AN ARTIFACT OF THE MODEL.
*
*   GND      CIRCUIT GROUND

```

Buffer Module Description

```

* THIS SUBCIRCUIT MODELS THE INTERFACE BETWEEN XILINX
* 3000 SERIES PARTS AND THE BONDING PAD. IT IS NOT
* USEFUL FOR PREDICTING DELAY TIMES FROM THE OUTSIDE
* WORLD TO INTERNAL LOGIC IN THE XILINX CHIP. RATHER,
* IT CAN BE USED TO PREDICT THE SHAPE OF WAVEFORMS
* GENERATED AT THE BONDING PAD AS WELL AS THE RESPONSE
* OF THE INPUT RECEIVERS TO APPLIED WAVEFORMS.

* THIS MODEL IS INTENDED FOR USE BY SYSTEM DESIGNERS
* WHO ARE CONCERNED ABOUT TRANSMISSION EFFECTS IN
* CIRCUIT BOARDS CONTAINING XILINX 3000 SERIES PARTS.

* THE PIN CAPACITANCE AND BONDING WIRE INDUCTANCE,
* RESISTANCE ARE NOT CONTAINED IN THIS MODEL. THESE
* ARE A FUNCTION OF THE CHOSEN PACKAGE AND MUST BE
* INCLUDED EXPLICITLY IN A CIRCUIT BUILT WITH THIS
* SUBCIRCUIT.

* NON-IDEALITIES SUCH AS GROUND BOUNCE ARE ALSO A
* FUNCTION OF THE SPECIFIC CONFIGURATION OF THE
* XILINX PART, SUCH AS THE NUMBER OF DRIVERS WHICH
* SHARE POWER PINS SWITCHING SIMULTANEOUSLY. ANY
* SIMULATION TO EXAMINE THESE EFFECTS MUST ADDRESS
* THE CONFIGURATION-SPECIFIC ASPECTS OF THE DESIGN.
*

.SUBCKT XIL_IOB I 0 PAD_IO TS FAST PPUB TTL VDD GND
+ XIL_SIG=0 XIL_DTEMP=0 XIL_SHRINK=1

.prot FREELIB
;]= $.[;qW.261DW3Eu0
VO\;;n[ $.[;qW.2'4%S+%X;;0[(3'1:67*8-:1:\[
kp39H2J9#Yo%XpVY#0!rDI$UqhmE%:\7%(3e%:\7\50)1-5i# ;

.ENDS XIL_IOB

```




Chapter 17

Behavioral Modeling

Behavioral modeling substitutes more abstract, less computationally intensive circuit models, for lower-level descriptions of analog functions. These simpler models emulate the transfer characteristics of the circuit elements that they replace, but with increased efficiency. Behavioral modeling substantially reduces the actual simulation time per circuit. At the level of an entire design and simulation cycle, design efficiency greatly increases, and you can complete a design (from concept to marketable product) in substantially less time.

This chapter describes how to create behavioral models, and explains the following topics:

- Behavioral Design Process
- Using Behavioral Elements
- Voltage and Current Controlled Elements
- Voltage-Dependent Voltage Sources — E Elements
- Current-Dependent Current Sources — F Elements
- Voltage-Dependent Current Sources — G Elements
- Current-Dependent Voltage Sources — H Elements
- Modeling with Digital Behavioral Components
- Calibrating Digital Behavioral Components
- Analog Behavioral Elements
- Op-Amps, Comparators, and Oscillators
- Phase Locked Loops (PLL)
- References

Behavioral Design Process

Star-Hspice provides specific modeling elements that promote the use of behavioral and mixed signal techniques. These models include controllable sources that you can configure, to emulate op-amps, single-input or multi-input logic gates, or any system with a continuous algebraic transfer function.

- You can create these functions in algebraic form, or in the form of coordinate pairs.
- You can use digital stimulus files, to enter logic waveforms into the simulation deck, rather than using piecewise linear sources to enter digital waveforms.
- You can define clock rise times, fall times, periods, and voltage levels.

The typical design cycle for a circuit or system, using Star-Hspice behavioral models, is:

1. Fully simulate a subcircuit, with pertinent inputs, characterizing its transfer functions.
2. Determine which Star-Hspice elements, singularly or in combination, accurately describe the transfer function.
3. Reconfigure the subcircuit appropriately.
4. After you verify the behavioral model, substitute the model into the larger system, in place of the lower-level subcircuit.

Using Behavioral Elements

Behavioral elements offer a higher level of abstraction, and faster processing, compared to a lower-level description of an analog function.

- System-level designers can use function libraries of sub-circuits, containing these elements, to describe parts such as:
 - op-amps
 - vendor specific output buffer drivers
 - TTL drivers
 - logic-to-analog converters
 - analog-to-logic simulator converters
- Integrated Circuit designers can use these elements to reduce design time, especially when designing filters and signal processors.

Behavioral elements use an arbitrary algebraic equation, as a transfer function to either a voltage (E) or current (G) source. This function can include:

- nodal voltages
- element currents
- time
- other parameters, which you define

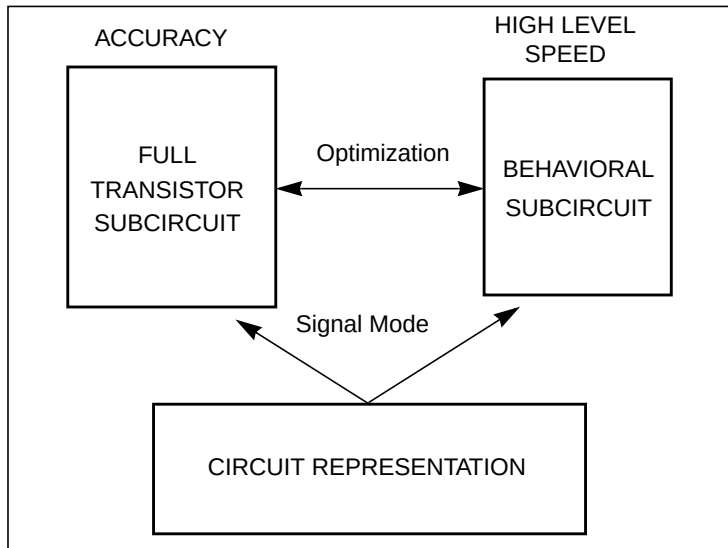
A good example of this is a VCO, where *control* is the input voltage node, and *osc* is the oscillator output:

```
Evco osc 0
VOL='voff+gain*SIN(6.28*freq*(1+V(control))*TIME)'
```

You can use sub-circuits to encapsulate a function.

- If you split the function definition from the use, you create a hierarchy.
- If you pass parameters into the subcircuit, you create a *parameterized* cell.
- If you create a full transistor cell library, and a behavioral representation library, you can include mixed-signal functions within Star-Hspice.

You can use the built-in OPTIMIZE function to calibrate the behavioral elements, from a full transistor circuit.

Figure 17-1: Netlisting by Signal Mode

Controlled Sources

Controlled sources model both analog and digital circuits, at the behavioral level. This reduces simulation times for mixed signals, and models system-level operations. Controlled sources also model gate-switching action, for behavioral modeling of digital circuits. For analog behavioral modeling, you can program the controlled sources as mathematical functions. These functions can be either linear or non-linear, depending on other nodal voltages and branch currents.

Digital Stimulus Files

Complex transition files are difficult to process, if you use piecewise linear sources. You can use the A2D and D2A conversion functions, in the U Element, to simplify processing of transition files.

- The A2D function converts analog output to digital data.
- The D2A function converts digital input data to analog.

You can also export the output to either logic or VHDL simulators.

Behavioral Examples

The examples of analog and digital elements, in this chapter, show how the behavioral elements operate.

Op-Amp Subcircuit Generators

The subcircuit generator automatically designs operational amplifiers, to meet electrical specifications (such as PSRR, CMRR, and Vos). The generator produces component values, for each element in the design. When Star-Hspice combines these values, the resulting subcircuits simulate faster than conventional circuit-level implementations.

Libraries

The Discrete Device Library contains standard industry IC components. You can use this library to model board-level designs that contain any of the following:

- transistors
- diodes
- opamps
- comparators
- converters
- IC pins
- printed circuit board traces
- coaxial cables

You can also model drivers and receivers, to analyze transmission line effects, power line noise, and signal line noise.

Voltage and Current Controlled Elements

Star-Hspice uses four voltage-controlled and current-controlled elements, known as E, F, G, and H Elements. Use these controlled elements to model the following:

- MOS and bipolar transistors
- Tunnel diodes
- SCRs

and analog functions such as:

- Operational amplifiers
- Summers
- Comparators
- Voltage-controlled oscillators
- Modulators
- Switched-capacitor circuits

Depending on whether you used the polynomial or piecewise linear functions, the controlled elements can be:

- Linear functions of controlling-node voltages.
- Non-linear functions of controlling-node voltages.
- Linear functions of branch currents.
- Non-linear functions of branch currents.

The functions of the E, F, G, and H controlled elements are different.

- The E Elements can be:
 - A voltage-controlled voltage source
 - A current-controlled voltage source
 - An ideal op-amp.
 - An ideal transformer.
 - An ideal delay element.
 - A piecewise linear, voltage-controlled, multi-input AND, NAND, OR, or NOR gate.

- The F Element can be:
 - A current-controlled current source.
 - An ideal delay element.
 - A piecewise linear, current-controlled, multi-input AND, NAND, OR, or NOR gate.
- The G Element can be:
 - A voltage-controlled current source.
 - A current-controlled current source.
 - A voltage-controlled resistor.
 - A piecewise linear, voltage-controlled capacitor.
 - An ideal delay element.
 - A piecewise linear, multi-input AND, NAND, OR, or NOR gate.
- The H Element can be:
 - A current-controlled voltage source.
 - An ideal delay element.
 - A piecewise linear, current-controlled, multi-input AND, NAND, OR, or NOR gate.

The next section describes polynomial and piecewise linear functions. Later sections describe element statements for linear or non-linear functions.

Polynomial Functions

Use the controlled element statement, to define the controlled output variable (current, resistance, or voltage), as a polynomial function of one or more voltages or branch currents. You can select three polynomial equations, using the POLY(ndim) parameter in the E, F, G, or H Element statement.

- POLY(1) one-dimensional equation (function of one controlling variable).
- POLY(2) two-dimensional equation (function of two controlling variables).
- POLY(3) three-dimensional equation (function of three controlling variables).

Each polynomial equation include polynomial coefficient parameters (P0, P1 ... Pn), which you can use to explicitly define the equation.

One-Dimensional Function

If the function is one-dimensional (a function of one branch current or one node voltage), then the following expression determines the FV function value:

$$FV = P0 + (P1 \cdot FA) + (P2 \cdot FA^2) + (P3 \cdot FA^3) + (P4 \cdot FA^4) + (P5 \cdot FA^5) + \dots$$

FV Controlled voltage or current, from the controlled source.

P0. . . Pn Coefficients of the polynomial equation.

FA Controlling branch current or nodal voltage.

Note: If you specify one coefficient in a one-dimensional polynomial, Star-Hspice assumes that the coefficient is P1 ($P0 = 0.0$). This facilitates the input of linear controlled sources.

The following controlled source statement is a one-dimensional function:

```
E1 5 0 POLY(1) 3 2 1 2.5
```

The above voltage-controlled voltage source connects to nodes 5 and 0. POLY(1) is a single-dimension polynomial function parameter, which informs Star-Hspice that *E1* is a function of the difference in one nodal voltage pair—in this example, the voltage difference between nodes 3 and 2, so $FA = V(3, 2)$. The dependent source statement then specifies that $P0=1$, and $P1=2.5$. From the one-dimensional polynomial equation above, the defining equation for *E1* is:

$$E1 = 1 + 2.5 \cdot V(3,2)$$

Two-Dimensional Function

If the function is two-dimensional (that is, a function of two node voltages or two branch currents), the following expression determines FV:

$$FV = P0 + (P1 \cdot FA) + (P2 \cdot FB) + (P3 \cdot FA^2) + (P4 \cdot FA \cdot FB) + (P5 \cdot FB^2) \\ + (P6 \cdot FA^3) + (P7 \cdot FA^2 \cdot FB) + (P8 \cdot FA \cdot FB^2) + (P9 \cdot FB^3) + \dots$$

For a two-dimensional polynomial, the controlled source is a function of two nodal voltages or currents. To specify a two-dimensional polynomial, set POLY(2) in the controlled-source statement.

For example, generate a voltage-controlled source that specifies the controlled voltage (*E1*) as:

$$E1 = 3 \cdot V(3,2) + 4 \cdot V(7,6)^2$$

To implement this function, use this controlled-source element statement:

```
E1 1 0 POLY(2) 3 2 7 6 0 3 0 0 0 4
```

This statement specifies a controlled voltage source, connected between nodes 1 and 0. Two differential voltages control this voltage source:

- The voltage difference between nodes 3 and 2.
- The voltage difference between nodes 7 and 6.

That is, $FA = V(3, 2)$ and $FB = V(7, 6)$. The polynomial coefficients are $P0=0$, $P1=3$, $P2=0$, $P3=0$, $P4=0$, and $P5=4$.

Three-Dimensional Function

For a three-dimensional polynomial function, with *FA*, *FB*, and *FC* as its arguments, the following expression determines the *FV* function value:

$$\begin{aligned} FV = & P0 + (P1 \cdot FA) + (P2 \cdot FB) + (P3 \cdot FC) + (P4 \cdot FA^2) \\ & + (P5 \cdot FA \cdot FB) + (P6 \cdot FA \cdot FC) + (P7 \cdot FB^2) + (P8 \cdot FB \cdot FC) \\ & + (P9 \cdot FC^2) + (P10 \cdot FA^3) + (P11 \cdot FA^2 \cdot FB) + (P12 \cdot FA^2 \cdot FC) \\ & + (P13 \cdot FA \cdot FB^2) + (P14 \cdot FA \cdot FB \cdot FC) + (P15 \cdot FA \cdot FC^2) \\ & + (P16 \cdot FB^3) + (P17 \cdot FB^2 \cdot FC) + (P18 \cdot FB \cdot FC^2) \\ & + (P19 \cdot FC^3) + (P20 \cdot FA^4) + \dots \end{aligned}$$

For example, generate a voltage-controlled source that specifies the voltage as:

$$E1 = 3 \cdot V(3,2) + 4 \cdot V(7,6)^2 + 5 \cdot V(9,8)^3$$

The resulting three-dimensional polynomial equation:

$$FA = V(3,2)$$

$$FB = V(7,6)$$

$$FC = V(9,8)$$

$$P1 = 3$$

$$P7 = 4$$

$$P19 = 5$$

Substitute these values into the voltage controlled voltage source statement:

```
E1 1 0 POLY(3) 3 2 7 6 9 8 0 3 0 0 0 0 0 4 0 0 0 0 0 0
+ 0 0 0 0 0 5
```

The above statement specifies a voltage source, connected between nodes 1 and 0. Three differential voltages control this voltage source: the voltage differences between nodes 3 and 2, between nodes 7 and 6, and between nodes 9 and 8. That is, $FA=V(3, 2)$, $FB=V(7, 6)$, and $FC=V(9, 8)$. The statement specifies the polynomial coefficients as $P1=3$, $P7=4$, $P19=5$. The other coefficients are zero.

Piecewise Linear (PWL) Function

You can use the one-dimensional piecewise linear (PWL) function to model special element characteristics, such as those of tunnel diodes, silicon-controlled rectifiers, and diode breakdown regions. To describe the piecewise linear function, specify measured data points. Although data points describe the device characteristic, Star-Hspice automatically smooths the corners, to ensure derivative continuity. This, in turn, results in better convergence.

The DELTA parameter controls the curvature of the characteristic, at the corners. The smaller the DELTA, the sharper the corners are. The maximum value allowed for DELTA is half of the smallest distance between breakpoints. Specify a DELTA that provides satisfactory sharpness of the function corners. You can specify up to 100 breakpoint pairs. You must specify at least two point pairs (each point consists of an x and a y coefficient).

You can use the NPWL and PPWL functions, and G Elements, to model bidirectional switch or transfer gates. The NPWL and PPWL functions behave like NMOS and PMOS transistors.

You can also use the piecewise linear function, to model multi-input AND, NAND, OR, and NOR gates. In this case, only one input determines the state of the output.

- In AND and NAND gates, the piecewise linear function uses the input with the smallest value, to determine the corresponding output of the gates.
- In OR and NOR gates, the input with the largest value determines the corresponding output of the gates.

Voltage-Dependent Voltage Sources — E Elements

Voltage-Controlled Voltage Source (VCVS)

The syntax is:

Linear

```
Exxx n+ n- <VCVS> in+ in- gain <MAX=val> <MIN=val>
+ <SCALE=val> <TC1=val> <TC2=val> <ABS=1> <IC=val>
```

Polynomial

```
Exxx n+ n- <VCVS> POLY(ndim) in1+ in1- ...
+ inndim+ inndim- <TC1=val> <TC2=val> <SCALE=val>
+ <MAX=val> <MIN=val> <ABS=1> p0 <p1...> <IC=val>
```

Piecewise Linear

```
Exxx n+ n- <VCVS> PWL(1) in+ in- <DELTA=val>
+ <SCALE=val> <TC1=val> <TC2=val> x1,y1 x2,y2 ...
+ x100,y100 <IC=val>
```

Multi-Input Gates

```
Exxx n+ n- <VCVS> gatetype(k) in1+ in1- ... inj+ inj-
+ <DELTA=val> <TC1=val> <TC2=val> <SCALE=val>
+ x1,y1 ... x100,y100 <IC=val>
```

Delay Element

```
Exxx n+ n- <VCVS> DELAY in+ in- TD=val <SCALE=val>
+ <TC1=val> <TC2=val> <NPDELAY=val>
```

Behavioral Voltage Source

The syntax is:

```
Exxx n+ n- VOL='equation' <MAX=val> <MIN=val>
```

Ideal Op-Amp

The syntax is:

```
EXXX n+ n- OPAMP in+ in-
```

Ideal Transformer

The syntax is:

```
EXXX n+ n- TRANSFORMER in+ in- k
```

E Element Parameters

<i>ABS</i>	Output is an absolute value, if ABS=1.
<i>DELAY</i>	Keyword for the delay element. The delay element is the same as for the voltage-controlled voltage source, except that it has a propagation delay (TD). Subcircuit modeling uses this element to adjust the propagation delay. DELAY is a Star-Hspice keyword; do not use it as a node name.
<i>DELTA</i>	Controls the curvature of the piecewise linear corners. Defaults to 1/4 of the smallest distance between breakpoints. The maximum is 1/2 of the smallest distance between breakpoints.
<i>Exxx</i>	Voltage-controlled element name. Must begin with E, followed by up to 15 alphanumeric characters.
<i>gain</i>	Voltage gain.
<i>gatetype(k)</i>	Can be AND, NAND, OR, or NOR. The value of <i>k</i> is the number of inputs of the gate. The <i>x</i> and <i>y</i> terms represent the piecewise linear variation of the output, as a function of the input. In the multi-input gates, only one input determines the state of the output.
<i>IC</i>	Initial condition: the initial estimate of the value(s) for the controlling voltage(s). Default=0.0.

<i>in +/-</i>	Positive or negative controlling nodes. Specify one pair for each dimension.
<i>j</i>	Ideal transformer turn ratio: $V(in+,in-) = j \cdot V(n+,n-)$
<i>MAX</i>	Maximum output voltage value. The default is undefined, and sets no maximum value.
<i>MIN</i>	Minimum output voltage value. The default is undefined, and sets no minimum value.
<i>n+/-</i>	Positive or negative node, of a controlled element.
<i>NPDELAY</i>	Sets the number of data points to use, in delay simulations. The default value is the larger of either 10, or the smaller of $TD/tstep$ and $tstop/tstep$. That is, $NPDELAY_{\text{default}} = \max \left[\frac{\min \langle TD, tstop \rangle}{tstep}, 10 \right]$ The .TRAN statement specifies the values of $tstep$ and $tstop$.
<i>OPAMP</i>	Keyword for ideal op-amp element. OPAMP is a Star-Hspice keyword. Do not use it as a node name.
<i>p0, p1 ...</i>	Polynomial coefficients. If you specify one coefficient, Star-Hspice assumes that it is $p1$, with $p0=0.0$, and the element is linear. If you specify more than one polynomial coefficient ($p0, p1, p2, \dots$), then the element is nonlinear. See Polynomial Functions on page 17-7 .
<i>POLY</i>	Polynomial dimension. If you do not specify $POLY(ndim)$, Star-Hspice assumes that you are using a one-dimensional polynomial. $Ndim$ must be a positive number.
<i>PWL</i>	Keyword for the piecewise linear function.
<i>SCALE</i>	Multiplier for the element value.

<i>TC1, TC2</i>	First-order and second-order temperature coefficients. Temperature updates the SCALE: $\text{SCALE}_{\text{eff}} = \text{SCALE} \cdot (1 + \text{TC1} \cdot \Delta t + \text{TC2} \cdot \Delta t^2)$
<i>TD</i>	Keyword for the time delay.
<i>TRANSFORMER</i>	Keyword for the ideal transformer. TRANSFORMER is a Star-Hspice keyword. Do not use it as a node name.
<i>VCVS</i>	Keyword for the voltage-controlled voltage source. VCVS is a Star-Hspice keyword. Do not use it as a node name.
<i>x1,...</i>	Controlling voltage, across the <i>in+</i> and <i>in-</i> nodes. The <i>x</i> values must be in increasing order.
<i>y1,...</i>	Corresponding element values of <i>x</i> .

Examples

Ideal Op-Amp

You can use the voltage-controlled voltage source to build a voltage amplifier, with supply limits.

- The output voltage across nodes 2,3 = $v(14,1) \cdot 2$.
- The voltage gain is 2.
- The MAX and MIN parameters specify a maximum *E1* voltage of 5V, and a minimum *E1* voltage output of -5V.

For example, if $v(14,1) = -4\text{V}$, then *E1* is -5V, and not -8V as the equation suggests.

```
Eopamp 2 3 14 1 MAX=+5 MIN=-5 2.0
```

To specify a value for polynomial coefficient parameters, you can use the following format to define a parameter:

```
.PARAM CU = 2.0
E1 2 3 14 1 MAX=+5 MIN=-5 CU
```

Voltage Summer

An ideal voltage summer specifies the source voltage, as a function of three controlling voltage(s):

- V(13,0)
- V(15,0)
- V(17,0).

It describes a voltage source, with the value:

$$V(13,0) + V(15,0) + V(17,0)$$

This example represents an ideal voltage summer. It initializes the three controlling voltages, for a DC operating point analysis, to 1.5, 2.0, and 17.25 V.

```
EX 17 0 POLY(3) 13 0 15 0 17 0 0 1 1 1 IC=1.5,2.0,17.25
```

Polynomial Function

The voltage-controlled source can also output a non-linear function, using the one-dimensional polynomial. Because this example does not specify the POLY parameter, Star-Hspice assumes that this is a one-dimensional polynomial (that is, a function of one controlling voltage). The equation corresponds to the element syntax. Behavioral equations replace this older method.

```
E2 3 4 VOLT = "10.5 + 2.1 *V(21,17) + 1.75 *V(21,17)^2"
E2 3 4 POLY 21 17 10.5 2.1 1.75
```

Zero-Delay Inverter Gate

You can use a piecewise linear transfer function to build a simple inverter, with no delay.

```
Einv out 0 PWL(1) in 0 .7v,5v 1v,0v
```

Ideal Transformer

If the turn ratio is 10 to 1, the voltage relationship is $V(\text{out})=V(\text{in})/10$.

```
Etrans out 0 TRANSFORMER in 0 10
```


Voltage-Controlled Oscillator (VCO)

The `VOL` keyword defines a single-ended input, which controls the output of a VCO.

In the following example, the voltage at the *control* node controls the frequency of the sinusoidal output voltage at the *out* node. The *v0* parameter is the DC offset voltage, and *gain* is the amplitude. The output is a sinusoidal voltage, with a frequency of *freq*control*.

```
Evco out 0  
VOL='v0+gain*SIN(6.28*freq*v(control)*TIME)'
```

Note: This equation is valid only for a steady-state VCO (fixed voltage). If you sweep the control voltage, this equation does not apply.

Current-Dependent Current Sources — F Elements

Current-Controlled Current Source (CCCS)

The syntax is:

Linear

```
Fxxx n+ n- <CCCS> vn1 gain <MAX=val> <MIN=val>
+ <SCALE=val> <TC1=val> <TC2=val> <M=val> <ABS=1>
+ <IC=val>
```

Polynomial

```
Fxxx n+ n- <CCCS> POLY(ndim) vn1 <... vnndim> <MAX=val>
+ <MIN=val> <TC1=val> <TC2=val> <SCALE=val> <M=val>
+ <ABS=1> p0 <p1...> <IC=val>
```

Piecewise Linear

```
Fxxx n+ n- <CCCS> PWL(1) vn1 <DELTA=val>
+ <SCALE=val> <TC1=val> <TC2=val> <M=val>
+ x1,y1 ... x100,y100 <IC=val>
```

Multi-Input Gates

```
Fxxx n+ n- <CCCS> gatetype(k) vn1, ... vnk <DELTA=val>
+ <SCALE=val> <TC1=val> <TC2=val> <M=val> <ABS=1>
+ x1,y1 ... x100,y100 <IC=val>
```

Delay Element

```
Fxxx n+ n- <CCCS> DELAY vn1 TD=val <SCALE=val>
+ <TC1=val> <TC2=val> NPDELAY=val
```

Note: G Elements with algebraics make CCCS elements obsolete. You can still use CCCS elements for backward-compatibility with existing designs.

F Element Parameters

<i>ABS</i>	Output is an absolute value, if ABS=1.
<i>CCCS</i>	Keyword for current-controlled current source. CCCS is a Star-Hspice keyword. Do not use it as a node name
<i>DELAY</i>	Keyword for the delay element. The delay element is the same as for a current-controlled current source, except that it has a propagation delay (TD). In subcircuit modeling, this element adjusts the propagation delay. DELAY is a Star-Hspice keyword. Do not use it as a node name.
<i>DELTA</i>	Controls the curvature of piecewise linear corners. Defaults to 1/4 of the smallest distance between breakpoints. The maximum is 1/2 the smallest distance between breakpoints.
<i>Fxxx</i>	Element name of the current-controlled current source. Must begin with F, followed by up to 15 alphanumeric characters.
<i>gain</i>	Current gain.
<i>gatetype(k)</i>	Can be AND, NAND, OR, or NOR. <i>k</i> is the number of inputs for the gate. The <i>x</i> and <i>y</i> terms represent the piecewise linear variation of the output, as a function of the input. In multi-input gates, only one input determines the state of the output. Do not use the above keywords as node names.
<i>IC</i>	Initial condition: the initial estimate of the value(s) for the controlling current(s), in amps. Default=0.0.
<i>M</i>	Number of replications of the element, in parallel.
<i>MAX</i>	Maximum value of the output current. The default is undefined, and sets no maximum value.
<i>MIN</i>	Minimum value of the output current. The default is undefined, and sets no minimum value.
<i>n+/-</i>	Connecting nodes for a positive or negative controlled source.

NPDELAY Sets the number of data points to use, in delay simulations. The default value is the larger of either 10, or the smaller of TD/tstep and tstop/tstep. That is,

$$\text{NPDELAY}_{\text{default}} = \max\left[\frac{\min\langle \text{TD}, \text{tstop} \rangle}{\text{tstep}}, 10\right]$$

The .TRAN statement specifies the values of tstep and tstop.

p0, p1 ... The polynomial coefficients.

If you specify one coefficient, Star-Hspice assumes that it is p1, with p0=0.0, and the element is linear.

If you specify more than one polynomial coefficient (p0, p1, p2, ...), then the element is non-linear. See [Polynomial Functions on page 17-7](#).

POLY Polynomial dimension. If you do not specify POLY(*ndim*), Star-Hspice assumes that this is a one-dimensional polynomial. *Ndim* must be a positive number.

PWL Keyword for the piecewise linear function.

SCALE Multiplier for the element value.

TC1, TC2 First-order and second-order temperature coefficients. The temperature updates the SCALE:

$$\text{SCALE}_{\text{eff}} = \text{SCALE} \cdot (1 + \text{TC1} \cdot \Delta t + \text{TC2} \cdot \Delta t^2)$$

TD Keyword for the time delay.

vn1... Names of voltage sources, through which the controlling current flows. Specify one name for each dimension.

x1,... Controlling current, through the *vn1* source. The x values must be in increasing order.

y1,... Corresponding output-current values of x.

Examples

```
$ Current-controlled current sources - F Elements,
F1 13 5 VSENS MAX=+3 MIN=-3 5
F2 12 10 POLY VCC 1MA 1.3M
Fd 1 0 DELAY vin TD=7ns SCALE=5
Filim 0 out PWL(1) vsrc -1a,-1a 1a,1a
```

The first example describes a current-controlled current source, connected between nodes 13 and 5. The current, which controls the value of the controlled source, flows through the voltage source named VSENS.

Note: To use a current-controlled current source, you can place a dummy independent voltage source, into the path of the controlling current.

The defining equation is:

$$I(F1) = 5 \cdot I(VSENS)$$

- The current gain is 5.
- The maximum current flow, through F1, is 3 A.
- The minimum current flow is -3 A.

If $I(VSENS) = 2$ A, then $I(F1)$ is 3 amps, not 10 amps (as the equation suggests). You can define a parameter for the polynomial coefficient(s):

```
.PARAM VU = 5
F1 13 5 VSENS MAX=+3 MIN=-3 VU
```

The second example is a current-controlled current source, with the value:

$$I(F2) = 1e-3 + 1.3e-3 \cdot I(VCC)$$

Current flows from the positive node, through the source, to the negative node. The direction of positive controlling-current flow is from the positive node, through the source, to the negative node of *vnam* (linear), or to the negative node of each voltage source (non-linear).

The third example is a delayed current-controlled current source.

The fourth example is a piecewise linear, current-controlled current source.

Voltage-Dependent Current Sources — G Elements

Voltage-Controlled Current Source (VCCS)

The syntax is:

Linear

```
Gxxx n+ n- <VCCS> in+ in- trans-conductance <MAX=val>
+ <MIN=val> <SCALE=val> <M=val> <TC1=val> <TC2=val>
+ <ABS=1> <IC=val>
```

Polynomial

```
Gxxx n+ n- <VCCS> POLY(ndim) in1+ in1- ...
+ <inndim+ inndim-> <MAX=val> <MIN=val> <SCALE=val>
+ <M=val> <TC1=val> <TC2=val> <ABS=1>
+ p0 <p1...> <IC=val>
```

Piecewise Linear

```
Gxxx n+ n- <VCCS> PWL(1) in+ in- <DELTA=val>
+ <SCALE=val> <M=val> <TC1=val> <TC2=val>
+ x1,y1 x2,y2 ... x100,y100 <IC=val> <SMOOTH=val>
```

```
Gxxx n+ n- <VCCS> NPWL(1) in+ in- <DELTA=val>
+ <SCALE=val> <M=val> <TC1=val> <TC2=val>
+ x1,y1 x2,y2 ... x100,y100 <IC=val> <SMOOTH=val>
```

```
Gxxx n+ n- <VCCS> PPWL(1) in+ in- <DELTA=val>
+ <SCALE=val> <M=val> <TC1=val> <TC2=val>
+ x1,y1 x2,y2 ... x100,y100 <IC=val> <SMOOTH=val>
```

Multi-Input Gates

```
Gxxx n+ n- <VCCS> gatetype(k) in1+ in1- ...
+ ink+ ink- <DELTA=val> <TC1=val> <TC2=val> <SCALE=val>
+ <M=val> x1,y1 ... x100,y100 <IC=val>
```

Delay Element

```
Gxxx n+ n- <VCCS> DELAY in+ in- TD=val <SCALE=val>
+ <TC1=val> <TC2=val> NPDELAY=val
```

Behavioral Current Source

The syntax is:

```
Gxxx n+ n- CUR='equation' <MAX=val> <MIN=val>
```

Voltage-Controlled Resistor (VCR)

The syntax is:

Linear

```
Gxxx n+ n- VCR in+ in- transfactor <MAX=val> <MIN=val>
+ <SCALE=val> <M=val> <TC1=val> <TC2=val> <IC=val>
```

Polynomial

```
Gxxx n+ n- VCR POLY(ndim) in1+ in1- ...
+ <inndim+ inndim-> <MAX=val> <MIN=val> <SCALE=val>
+ <M=val> <TC1=val> <TC2=val> p0 <p1...> <IC=val>
```

Piecewise Linear

```
Gxxx n+ n- VCR PWL(1) in+ in- <DELTA=val> <SCALE=val>
+ <M=val> <TC1=val> <TC2=val> x1,y1 x2,y2 ... x100,y100
+ <IC=val> <SMOOTH=val>
```

```
Gxxx n+ n- VCR NPWL(1) in+ in- <DELTA=val> <SCALE=val>
+ <M=val> <TC1=val> <TC2=val> x1,y1 x2,y2 ... x100,y100
+ <IC=val> <SMOOTH=val>
```

```
Gxxx n+ n- VCR PPWL(1) in+ in- <DELTA=val> <SCALE=val>
+ <M=val> <TC1=val> <TC2=val> x1,y1 x2,y2 ... x100,y100
+ <IC=val> <SMOOTH=val>
```

Multi-Input Gates

```
Gxxx n+ n- VCR gatetype(k) in1+ in1- ... ink+ ink-
+ <DELTA=val> <TC1=val> <TC2=val> <SCALE=val> <M=val>
+ x1,y1 ... x100,y100 <IC=val>
```

Voltage-Controlled Capacitor (VCCAP)

The syntax is:

```
Gxxx n+ n- VCCAP PWL(1) in+ in- <DELTA=val>
+ <SCALE=val> <M=val> <TC1=val> <TC2=val>
+ x1, y1 x2, y2 ... x100, y100 <IC=val> <SMOOTH=val>
```

You can use the NPWL and PPWL functions to interchange the $n+$ and $n-$ nodes, and keep the same transfer function. The following summarizes this action:

NPWL Function

For the $in-$ node, connected to $n-$:

- If $v(n+, n-) > 0$, then the controlling voltage is $v(in+, in-)$.
- Otherwise, the controlling voltage is $v(in+, n+)$.

For the $in-$ node, connected to $n+$:

- If $v(n+, n-) < 0$, then the controlling voltage is $v(in+, in-)$.
- Otherwise, the controlling voltage is $v(in+, n+)$.

PPWL Function

For the $in-$ node, connected to $n-$:

- If $v(n+, n-) < 0$, then the controlling voltage is $v(in+, in1-)$.
- Otherwise, the controlling voltage is $v(in+, n+)$.

For the $in-$ node, connected to $n+$:

- If $v(n+, n-) > 0$, then the controlling voltage is $v(in+, in-)$.
- Otherwise, the controlling voltage is $v(in+, n+)$.

G Element Parameters

ABS Output is an absolute value, if ABS=1.

CUR=equation Current output, which flows from $n+$ to $n-$. The *equation*, which you define, can be a function of:

- node voltages
- branch currents
- TIME
- temperature (TEMPER)
- frequency (HERTZ)

<i>DELAY</i>	Keyword for the delay element. The delay element is the same as for a voltage-controlled current source, except it has a propagation delay (TD). This element helps you to adjust the propagation delay in the subcircuit model process. <i>DELAY</i> is a Star-Hspice keyword. Do not use it as a node name.
<i>DELTA</i>	Controls the curvature of piecewise linear corners. Defaults to 1/4 of the smallest distance between breakpoints. The maximum is 1/2 the smallest distance between breakpoints.
<i>Gxxx</i>	Name of the voltage-controlled element. Must begin with G, followed by up to 15 alphanumeric characters.
<i>gatetype(k)</i>	Can be AND, NAND, OR, or NOR. The value of <i>k</i> is the number of inputs of the gate. The <i>x</i> and <i>y</i> terms represent the piecewise linear variation of the output, as a function of the input. In multi-input gates, only one input determines the state of the output.
<i>IC</i>	Initial condition. The initial estimate of the value(s) of the controlling voltage(s). If you do not specify <i>IC</i> , default=0.0.
<i>in +/-</i>	Positive or negative controlling nodes. Specify one pair for each dimension.
<i>M</i>	Number of replications of the element, in parallel
<i>MAX</i>	Maximum value of the current or resistance. The default is undefined, and sets no maximum value.
<i>MIN</i>	Minimum value of the current or resistance. The default is undefined, and sets no minimum value.
<i>n +/-</i>	Positive or negative node of the controlled element
<i>NPDELAY</i>	Sets the number of data points to use, in delay simulations. The default value is the larger of either 10, or the smaller of TD/tstep and tstop/tstep. That is,

$$NPDELAY_{\text{default}} = \max \left[\frac{\min \langle TD, tstop \rangle}{tstep}, 10 \right]$$

	The .TRAN statement specifies the values of <code>tstep</code> and <code>tstop</code> .
<i>NPWL</i>	Models the symmetrical bidirectional switch or transfer gate, NMOS
<i>p0, p1 ...</i>	Polynomial coefficients. If you specify one coefficient, Star-Hspice assumes that it is <code>p1</code> , with <code>p0=0.0</code> , and the element is linear. If you specify more than one polynomial coefficient (<code>p0</code> , <code>p1</code> , <code>p2</code> , ...), then the element is non-linear. See Polynomial Functions on page 17-7 .
<i>POLY</i>	Polynomial dimension. If you do not specify <code>POLY(ndim)</code> , Star-Hspice assumes that it is a one-dimensional polynomial. <i>Ndim</i> must be a positive number.
<i>PWL</i>	Keyword for the piecewise linear function.
<i>PPWL</i>	Models the symmetrical bidirectional switch or transfer gate, PMOS
<i>SCALE</i>	Multiplier for the element value.
<i>SMOOTH</i>	For piecewise linear dependent source elements, SMOOTH selects the curve-smoothing method. A curve-smoothing method simulates exact data points that you provide. You can use this method to make Star-Hspice simulate specific data points, which correspond to measured data or data sheets. Choices for SMOOTH are 1 or 2. n 1 selects the smoothing method prior to Release H93A. n 2 selects the smoothing method, which uses data points that you provide. This is the default method, starting with release H93A.

<i>TC1,TC2</i>	First-order and second-order temperature coefficients. The temperature updates the SCALE: $\text{SCALE}_{\text{eff}} = \text{SCALE} \cdot (1 + \text{TC1} \cdot \Delta t + \text{TC2} \cdot \Delta t^2)$
<i>TD</i>	Keyword for the time delay.
<i>transconductance</i>	Voltage-to-current conversion factor.
<i>transfactor</i>	Voltage-to-resistance conversion factor.
<i>VCCAP</i>	Keyword for the voltage-controlled capacitance element. VCCAP is a Star-Hspice keyword. Do not use it as a node name.
<i>VCCS</i>	Keyword for the voltage-controlled current source. VCCS is a Star-Hspice keyword. Do not use it as a node name.
<i>VCR</i>	Keyword for the voltage-controlled resistor element. VCR is a Star-Hspice keyword. Do not use it as a node name.
<i>x1,...</i>	Controlling voltage, across the <i>in+</i> and <i>in-</i> nodes. The <i>x</i> values must be in increasing order.
<i>y1,...</i>	Corresponding element values of <i>x</i> .

Examples

Switch

A voltage-controlled resistor represents a basic switch characteristic. The resistance between nodes 2 and 0 varies linearly, from 10meg to 1m ohms, when voltage across nodes 1 and 0 varies between 0 and 1 volt. Beyond the voltage limits, the resistance remains at 10meg and 1m ohms.

```
Gswitch 2 0 VCR PWL(1) 1 0 0v,10meg 1v,1m
```

Switch-Level MOSFET

You can use the N-piecewise linear resistance switch, to model a switch-level, n-channel MOSFET. The resistance value does not change, when you switch the *d* node and *s* positions.

```
Gnmos d s VCR NPWL(1) g s LEVEL=1 0.4v,150g 1v,10meg  
+ 2v,50k 3v,4k 5v,2k
```

Voltage-Controlled Capacitor

The capacitance value, across the *out,0* nodes, varies linearly (from 1p to 5p), when voltage across the *ctrl,0* nodes varies between 2v and 2.5v. Beyond the voltage limits, capacitance remains constant, at 1 picofarad and 5 picofarads.

```
Gcap out 0 VCCAP PWL(1) ctrl 0 2v,1p 2.5v,5p
```

Zero-Delay Gate

To implement a two-input AND gate, use an expression and a piecewise linear table.

- The inputs are voltages at the *a* and *b* nodes.
- The output is the current flow from the *out* to 0 node.
- The current is multiplied by the *SCALE* value—which in this example, is the inverse of the load resistance, connected across the *out,0* nodes.

```
Gand out 0 AND(2) a 0 b 0 SCALE='1/rload' 0v,0a 1v,.5a  
+ 4v,4.5a 5v,5a
```

Delay Element

A delay is a low-pass filter delay, similar to that of an opamp. In contrast, a transmission line has an infinite frequency response. A glitch input to a G delay attenuates similarly to a buffer circuit. In this example, the output of the delay element is the current flow, from the *out* node to the 1 node, with a value equal to the voltage across the *in,0* nodes, multiplied by the *SCALE* value, and delayed by the *TD* value.

```
Gdel out 0 DELAY in 0 TD=5ns SCALE=2 NPDELAY=25
```

Diode Equation

You can use a run-time expression to model a forward-bias diode characteristic, from node 5 to ground. The saturation current is 1e-14 amp, and the thermal voltage is 0.025v.

```
Gdio 5 0 CUR='1e-14*(EXP(V(5)/0.025)-1.0)'
```

Diode Breakdown

You can model the diode breakdown region to forward region. When voltage across a diode is above or below the piecewise linear limit values (-2.2v, 2v), the diode current remains at the corresponding limit values (-1a, 1.2a).

```
Gdiode 1 0 PWL(1) 1 0 -2.2v, -1a -2v, -1pa .3v, .15pa
+.6v, 10ua 1v, 1a 2v, 1.2a
```

Triodes

Both of the following voltage-controlled current sources implement a basic triode.

- The first example uses the `poly(2)` operator, to multiply the anode and grid voltages together, and to scale by `.02`.
- The next example uses the explicit behavioral algebraic description.

```
gt i_anode cathode poly(2) anode, cathode grid, cathode
+ 0 0 0 0 .02 gt i_anode cathode
+ cur='20m*v(anode, cathode) v(grid, cathode)'
```

Current-Dependent Voltage Sources – H Elements

Current-Controlled Voltage Source (CCVS)

The syntax is:

Linear

```
Hxxx n+ n- <CCVS> vn1 transresistance <MAX=val>
+ <MIN=val> <SCALE=val> <TC1=val> <TC2=val> <ABS=1>
+ <IC=val>
```

Polynomial

```
Hxxx n+ n- <CCVS> POLY(ndim) vn1 <... vnndim>
+ <MAX=val> <MIN=val> <TC1=val> <TC2=val> <SCALE=val>
+ <ABS=1> p0 <p1...> <IC=val>
```

Piecewise Linear

```
Hxxx n+ n- <CCVS> PWL(1) vn1 <DELTA=val> <SCALE=val>
+ <TC1=val> <TC2=val> x1,y1 ... x100,y100 <IC=val>
```

Multi-Input Gates

```
Hxxx n+ n- gatetype(k) vn1, ... vnk <DELTA=val>
+ <SCALE=val> <TC1=val> <TC2=val> x1,y1 ...
+ x100,y100 <IC=val>
```

Delay Element

```
Hxxx n+ n- <CCVS> DELAY vn1 TD=val <SCALE=val>
+ <TC1=val> <TC2=val> <NPDELAY=val>
```

Note: E Elements with algebraics make CCVS elements obsolete. You can still use CCVS elements for backward-compatibility with existing designs.

H Element Parameters

<i>ABS</i>	Output is an absolute value, if ABS=1.
<i>CCVS</i>	Keyword for a current-controlled voltage source. CCVS is a Star-Hspice keyword. Do not use it as a node name.
<i>DELAY</i>	Keyword for the delay element. The delay element is the same as for a current-controlled voltage source, except it has a propagation delay (TD). In subcircuit modeling, this element adjusts propagation delay. DELAY is a Star-Hspice keyword. Do not use it as a node name.
<i>DELTA</i>	Controls the curvature of piecewise linear corners. Defaults to 1/4 of the smallest distance between breakpoints. The maximum is 1/2 the smallest distance between breakpoints.
<i>gatetype(k)</i>	Can be AND, NAND, OR, or NOR. The <i>k</i> value is the number of inputs of the gate. The <i>x</i> and <i>y</i> terms are the piecewise linear variation of output, as a function of input. In multi-input gates, only one input determines the state of the output.
<i>Hxxx</i>	Element name of the current-controlled voltage source. Must begin with H, followed by up to 15 alphanumeric characters.
<i>IC</i>	Initial condition. This is the initial estimate of the value(s), for the controlling current(s) in amps. Default=0.0.
<i>MAX</i>	Maximum voltage value. The default is undefined, which sets no maximum value.
<i>MIN</i>	Minimum voltage value. The default is undefined, which sets no minimum value.
<i>n+/-</i>	Connecting nodes, for a positive or negative controlled source.

NPDELAY Sets the number of data points to use in delay simulations. The default value is the larger of either 10, or the smaller of TD/tstep and tstop/tstep. That is:

$$\text{NPDELAY}_{\text{default}} = \max\left[\frac{\min\langle \text{TD}, \text{tstop} \rangle}{\text{tstep}}, 10\right]$$

The .TRAN statement specifies the tstep and tstop values.

p0, p1 . . . Polynomial coefficients.

If you specify one coefficient, Star-Hspice assumes that it is p1, with p0=0.0, and the element is linear.

If you specify more than one polynomial coefficient (p0, p1, p2, ...), the element is nonlinear. See [Polynomial Functions on page 17-7](#).

POLY Polynomial dimension. If you do not specify POLY(*ndim*), Star-Hspice assumes a one-dimensional polynomial. *Ndim* must be a positive number.

PWL Keyword for a piecewise linear function.

SCALE Multiplier for the element value.

TC1, TC2 First-order and second-order temperature coefficients. The temperature updates the SCALE:

$$\text{SCALE}_{\text{eff}} = \text{SCALE} \cdot (1 + \text{TC1} \cdot \Delta t + \text{TC2} \cdot \Delta t^2)$$

TD Keyword for the time delay.

transresistance Current-to-voltage conversion factor.

vn1... Names of voltage sources, through which controlling current flows. You must specify one name for each dimension.

x1,... Controlling current, through *vn1* source. The *x* values must be in increasing order.

y1,... Corresponding output voltage values of *x*.

Examples

```
HX 20 10 VCUR MAX=+10 MIN=-10 1000
```

The example above selects a linear current-controlled voltage source. The controlling current flows through the dependent voltage source, called VCUR. The defining equation of the C CVS is:

$$HX = 1000 \cdot VCUR$$

The defining equation states that the voltage output of HX is 1000 times the value of current flowing through CUR. If the equation produces a value of HX greater than +10V, or less than -10V, because of the MAX= and MIN= parameters, Star-Hspice sets HX to either 10V or -10V. CUR is the name of the independent voltage source, through which the controlling current flows. If the controlling current does not flow through an independent voltage source, you must insert a dummy independent voltage source.

```
.PARAM CT=1000
HX 20 10 VCUR MAX=+10 MIN=-10 CT
HXY 13 20 POLY(2) VIN1 VIN2 0 0 0 0 1 IC=0.5, 1.3
```

The example above describes a dependent voltage source, with the value:

$$V = I(VIN1) \cdot I(VIN2)$$

This two-dimensional polynomial equation specifies FA1=VIN1, FA2=VIN2, P0=0, P1=0, P2=0, P3=0, and P4=1. Star-Hspice initializes the controlling current to flow through VIN1, at 5mA. For VIN2, the initial current is 1.3mA.

The direction of positive controlling current flow is from the positive node, through the source, to the negative node of vnam (linear). The polynomial (nonlinear) specifies the source voltage, as a function of controlling current(s).

Modeling with Digital Behavioral Components

This section shows how to model, using digital behavioral components.

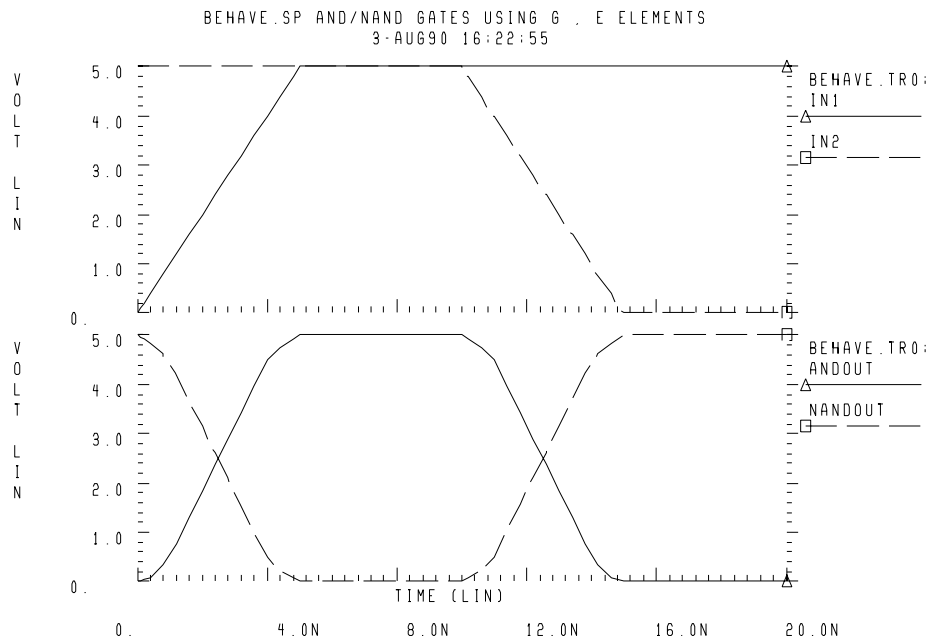
Behavioral AND and NAND Gates

The following input file example uses a G Element to model a two-input AND gate. An E Element models a two-input NAND gate. [Figure 17-2 on page 17-35](#) shows the resulting waveforms.

```

behave.sp and/nand gates using g, e Elements
.OPTION post=2
.op
.tran .5n 20n
.probe v(in1) v(in2) v(andout) v(in1) v(in2) v(nandout)
g 0 andout and(2) in1 0 in2 0
+ 0.0 0.0ma
+ 0.5 0.1ma
+ 1.0 0.5ma
+ 4.0 4.5ma
+ 4.5 4.8ma
+ 5.0 5.0ma
*
e nandout 0 nand(2) in1 0 in2 0
+ 0.0 5.0v
+ 0.5 4.8v
+ 1.0 4.5v
+ 4.0 0.5v
+ 4.5 0.2v
+ 5.0 0.0v
*
vin1 in1 0 0 pwl(0,0 5ns,5)
vin2 in2 0 5 pwl(0,5 10ns,5 15ns,0)
rin1 in1 0 1k
rin2 in2 0 1k
rand andout 0 1k
rnand nandout 0 1k
.end

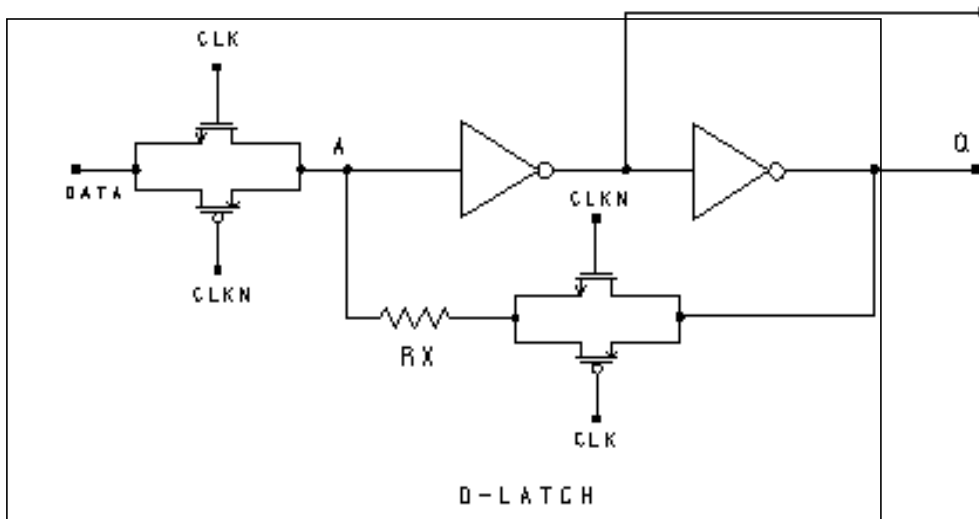
```

Figure 17-2: NAND/AND Gates

Behavioral D-Latch

This example uses one input NAND gates, and NPWL/PPWL functions, to model a D flip-flop.

Figure 17-3: D-Latch Circuit



Example

```

dlatch.sp--- cmos d-latch
.option post
.tran .2n 60ns
.probe tran clock=v(clck)data=v(d) q=v(q)
.ic v(q)=0

```

Waveforms

```

vdata d 0 pulse(0,5 2n,1n,1n 19n,40n)
vclk clck 0 pulse(0,5 7n,1n,1n 10n,20n)
vclkn clckn 0 pulse(5,0 7n,1n,1n 10n,20n)
xdlatch d clck clckn q qb dlatch cinv=.2p

```

Subcircuit Definitions for Behavioral N-Channel MOSFET

```

* DRAIN GATE SOURCE
.SUBCKT nmos 1 2 3 capm=.5p
cd 1 0 capm
cs 3 0 capm
gn 3 1 VCR NPWL(1) 2 3
+ 0. 495.8840G
+ 200.00000M 456.0938G
+ 400.00000M 141.6902G
+ 600.00000M 7.0624G
+ 800.00000M 258.9313X
+ 1.00000 6.4866X
+ 1.20000 842.9467K
+ 1.40000 321.6882K
+ 1.60000 170.8367K
+ 1.80000 106.4944K
+ 2.00000 72.7598K
+ 2.20000 52.4632K
+ 2.40000 38.5634K
+ 2.60000 8.8056K
+ 2.80000 5.2543K
+ 3.00000 4.3553K
+ 3.20000 3.8407K
+ 3.40000 3.4950K
+ 3.60000 3.2441K
+ 3.80000 3.0534K
+ 4.00000 2.9042K
+ 4.20000 2.7852K
+ 4.40000 2.6822K
+ 4.60000 2.5k
+ 5.0 2.3k
.ENDS nmos

```

Behavioral P-Channel MOSFET

```

* DRAIN GATE SOURCE
.SUBCKT pmos 1 2 3 capm=.5p
cd 1 0 capm
cs 3 0 capm
gp 1 3 VCR PPWL(1) 2 3
+ -5.0000 2.3845K
+ -4.8000 2.4733K
+ -4.6000 2.5719K
+ -4.4000 2.6813K
+ -4.2000 2.8035K
+ -4.0000 2.9415K
+ -3.8000 3.1116K
+ -3.6000 3.3221K

```

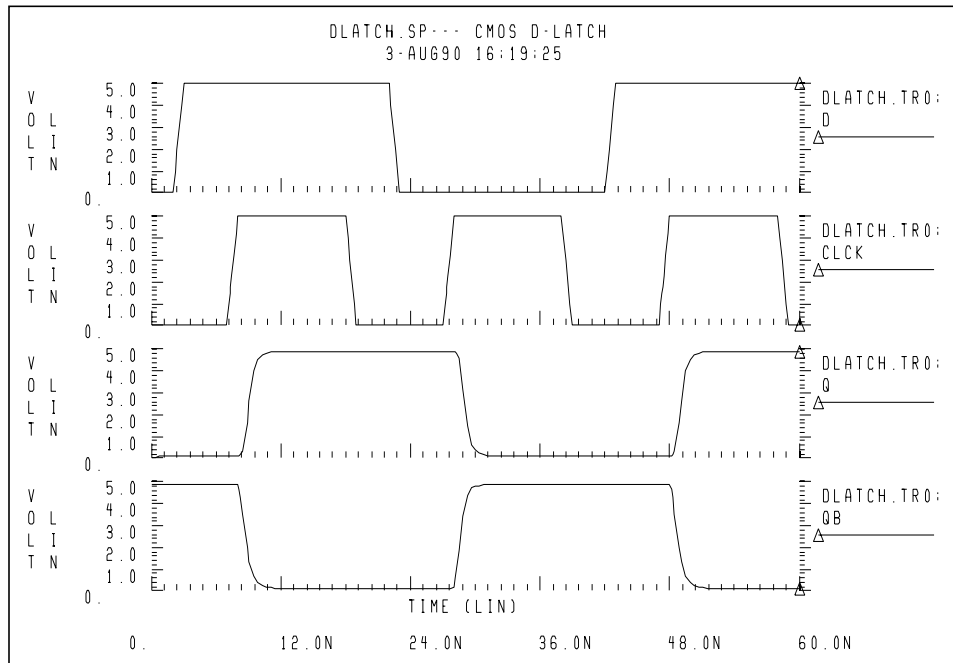
```

+ -3.4000 3.5895K
+ -3.2000 3.9410K
+ -3.0000 4.4288K
+ -2.8000 5.1745K
+ -2.6000 6.6041K
+ -2.4000 29.6203K
+ -2.2000 42.4517K
+ -2.0000 58.3239K
+ -1.8000 83.4296K
+ -1.6000 128.1517K
+ -1.4000 221.2640K
+ -1.2000 471.8433K
+ -1.0000 1.6359X
+ -800.00M 41.7023X
+ -600.00M 1.3394G
+ -400.00M 38.3449G
+ -200.00M 267.7325G
+ 0. 328.7122G
.ENDS pmos
*
.subckt tgate in out clk clkn ctg=.5p
xmn in clk out nmos capm=ctg
xmp in clkn out pmos capm=ctg
.ends tgate

.SUBCKT inv in out capout=1p
cout out 0 capout
rout out 0 1.0k
gn 0 out nand(1) in 0 scale=1
+ 0. 4.90ma
+ 0.25 4.88ma
+ 0.5 4.85ma
+ 1.0 4.75ma
+ 1.5 4.42ma
+ 3.5 1.00ma
+ 4.000 0.50ma
+ 4.5 0.2ma
+ 5.0 0.1ma
.ENDS inv

.subckt dlatch data clck clckn q qb cinv=1p
xtg1 data a clck clckn tgate ctg='cinv/2'
xtg2 q ax clckn clck tgate ctg='cinv/2'
rx ax a 5
xinv1 a qb inv capout=cinv
xinv2 qb q inv capout=cinv
.ends dlatch
.end

```

Figure 17-4: D-Latch Response

Behavioral Double-Edge Triggered Flip-Flop

This example uses the D_LATCH subcircuit from the previous example, and several NAND gates, to model a double-edged, triggered flip-flop.

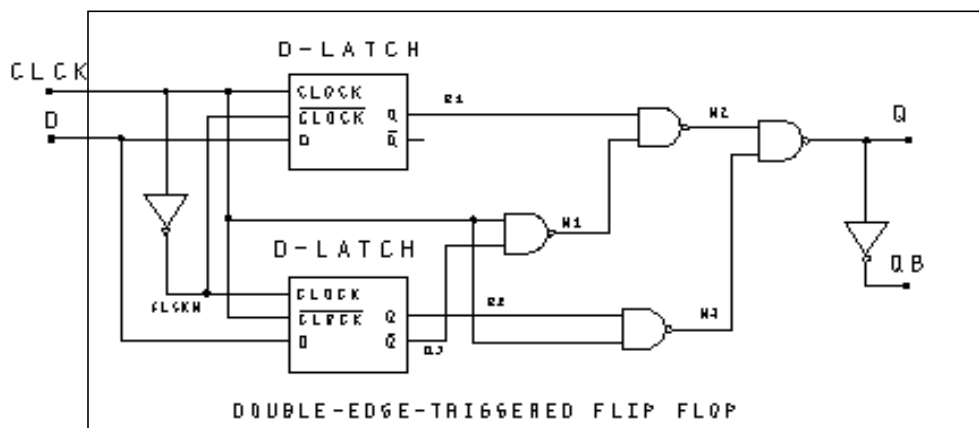
Example

```
det_dff.sp--- double edge triggered flip-flop
.option post=2
.tran .2n 100ns
.probe tran clock=v(clck) data=v(d) q=v(q)
```

Waveforms

```
vdata d 0 pulse(0,5 2n,1n,1n 28n,50n)
vclock clck 0 pulse(0,5 7n,1n,1n 10n,20n)
```

Figure 17-5: Double-Edge Triggered Flip-Flop Schematic



Main Circuit

```

xclk clck clckn inv cinv=.1p
xd1 d clck clckn q1 qb1 dlatch cinv=.2p
xd2 d clckn clck q2 qb2 dlatch cinv=.2p
xnand1 clck qb2 n1 nand2 capout=.5p
xnand2 q1 n1 n2 nand2 capout=.5p
xnand3 q2 clck n3 nand2 capout=.5p
xnand4 n2 n3 q nand2 capout=.5p
xin v q qb inv capout=.5p

```

Subcircuit Definitions

```

* Note: Subcircuit definitions for NMOS, PMOS, and INV
* are specified in the D-Latch examples; therefore they
* are not repeated here.
.SUBCKT nand2 in1 in2 out capout=2p
cout out 0 capout
rout out 0 1.0k
gn 0 out nand(2) in1 0 in2 0 scale=1
+ 0. 4.90ma
+ 0.25 4.88ma
+ 0.5 4.85ma
+ 1.0 4.75ma
+ 1.5 4.42ma
+ 3.5 1.00ma
+ 4.000 0.50ma
+ 4.5 0.2ma
+ 5.0 0.1ma
.ENDS nand2

```

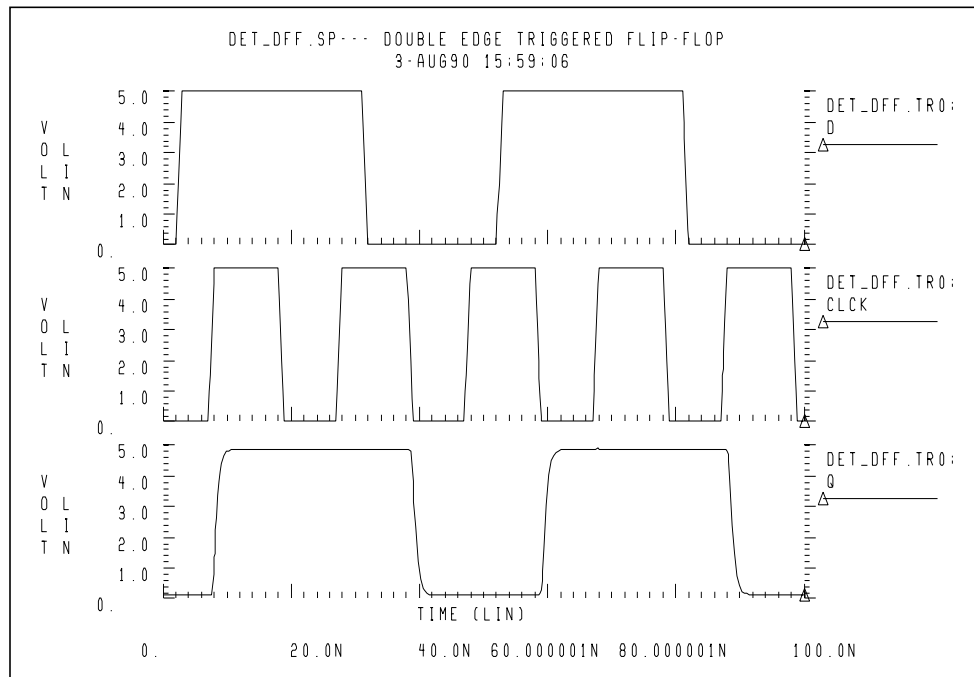


```

.subckt dlatch data clck clckn q qb cinv=1p
xtg1 data a clck clckn tgate ctg='cinv/2'
xtg2 q ax clckn clck tgate ctg='cinv/2'
rx ax a 10
xinv1 a qb inv capout=cinv
xinv2 qb q inv capout=cinv
.ends dlatch
.end

```

Figure 17-6: Double Edge Triggered Flip-Flop Response



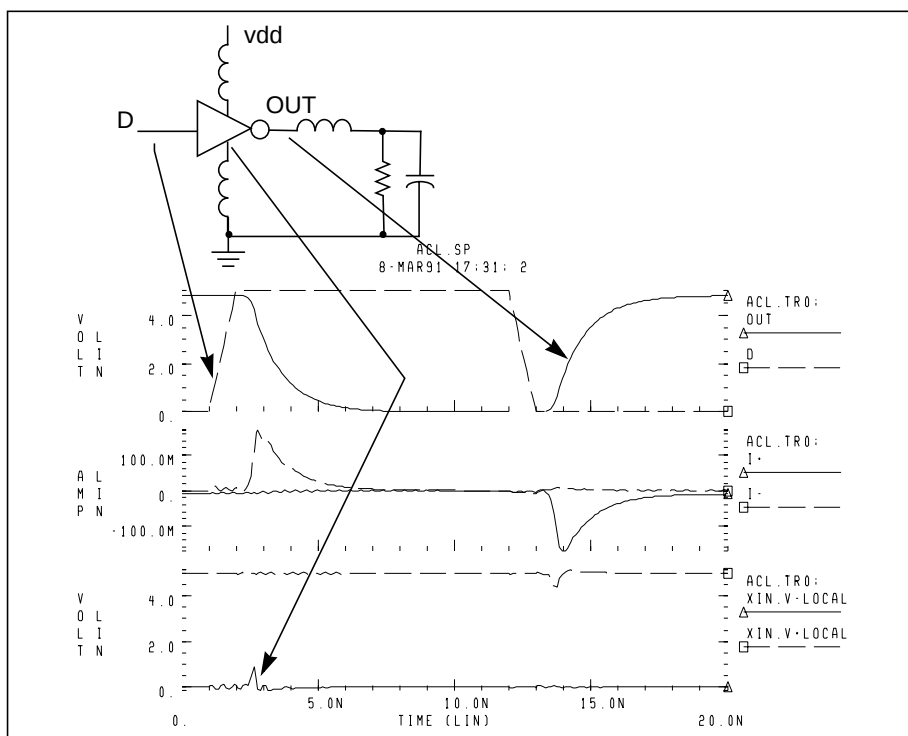
Calibrating Digital Behavioral Components

This section describes how to calibrate, using digital behavioral components.

Building Behavioral Lookup Tables

The following simulation demonstrates an ACL family output buffer, with 2 ns delay, and 1.8 ns rise and fall time. It also shows ground and VDD supply currents, and internal ground bounce due to the package.

Figure 17-7: ACL Family Output Buffer

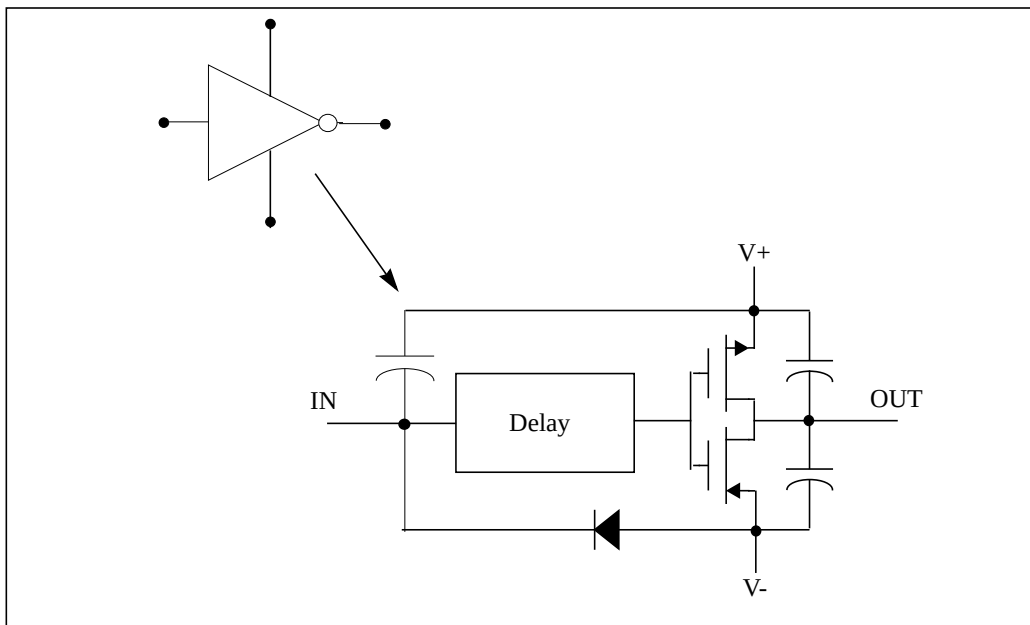


Star-Hspice uses the following commands to automatically measure the datasheet quantities, such as TPHL, risetime, maximum power dissipation, and ground bounce.

```
.MEAS tphl trig v(D) val='.5*vdd' rise=1
+ targ v(out) val='.5*vdd' fall=1
.MEAS risetime trig v(out) val='.1*vdd' rise=1
+ targ v(out) val='.9*vdd' rise=1
.MEAS max_power max power
.MEAS bounce max v(xin.v_local)
```

The inverter consists of capacitors, diodes, one-dimensional lookup table MOSFETs, and a special low-pass delay element. A property of the low-pass delay element, attenuates pulses that are narrower than the delay value.

Figure 17-8: Inverter



Subcircuit Definition

```
.subckt inv in out v+ v-
cout+ out_l v+ 2p
cout- out_l v- 2p
xmp out_l inx v+ pmos
xmn out_l inx v- nmos
e inx v- delay in v- td=1n
din v- in dx
.model dx d cjo=2pf
chi in v+ .5pf
.ends inv
```

One-dimensional lookup tables represent the behavioral MOSFETs. The equivalent n-channel lookup table is shown below.

Behavioral N-Channel MOSFET

The following example is a Drain Gate source.

```
.subckt nmos 1 2 3
gn 3 1 VCR npwl(1) 2 3 scale=0.008
* VOLTAGE RESISTANCE
+ 0. 495.8840g
+ 200.00000m 456.0938g
+ 400.00000m 141.6902g
+ 600.00000m 7.0624g
+ 800.00000m 258.9313meg
+ 1.00000 6.4866meg
+ 1.20000 842.9467k
+ 1.40000 21.6882k
+ 1.60000 170.8367k
+ 1.80000 106.4944k
+ 2.00000 72.7598k
+ 2.20000 52.4632k
+ 2.40000 38.5634k
+ 2.60000 8.8056k
+ 2.80000 5.2543k
+ 3.00000 4.3553k
+ 3.40000 3.4950k
+ 3.80000 2.0534k
+ 4.20000 2.7852k
+ 4.60000 2.5k
+ 5.0 2.3k
.ends nmos
```

The preceding example is a voltage-versus-resistance table. It shows, for example, that the resistance at 5 volts is 2.3k ohms.

Creating a Behavioral Inverter Lookup Table

You can create an inverter lookup table, in three simple steps.

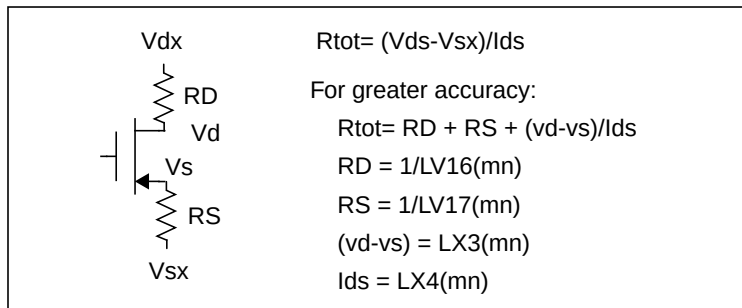
1. Simulate an actual transistor level inverter, using a DC sweep of the input.
2. Print the V/I output, for the output pullup and pulldown transistors.
3. Copy the printed output into the volt lookup table element, for the controlled resistor.

The following test file, *inv_vin_vout.sp*, calculates RN (the effective pulldown resistor transfer function) and RP (the pullup transfer function).

- RN is calculated as $V_{out}/I(mn)$, where *mn* is the pulldown transistor.
- RP is calculated as $(VCC - V_{out})/I(mp)$, where *mp* is the pullup transfer function.

The actual calculation uses a more-accurate method, to obtain the series resistance of the transistor, as in [Figure 17-9](#).

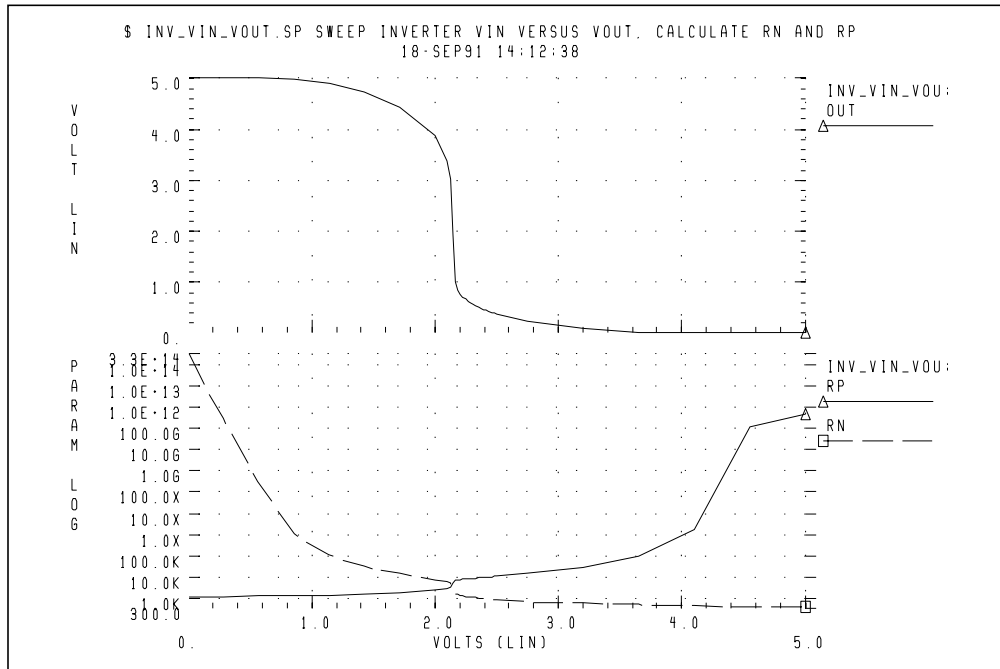
Figure 17-9: VIN versus VOUT



The first graph in [Figure 17-10](#) shows VIN versus VOUT.

The second graph shows the computed transfer resistances (RP and RN), as a function of VIN.

Figure 17-10: RP and RN as a Function of VIN



The Star-Hspice file used to calculate RP and RN is

```
$ inv_vin_vout.sp sweep inverter vin versus vout,
$ calculate rn and rp
```

The triple range DC sweep allows coarse grid before and after:

```
* use dc sweep with 3 ranges; 0-1.5v, 1.6-2.5, 2.6 5
.dc vin lin 8 0 2.0 lin 20 2.1 2.5 lin 6 2.75 5
$$ rn=par('v(out)/i(x1.mn)')
.print rn=
+ par('1/lv16(x1.mn)+1/lv17(x1.mn)+abs(lx3(x1.mn)/
+ lx4(x1.mn))')
.print rp=par('(-vcc+v(out))/i(x1.mp)')
.param sigma=0 vcc=5
.global vcc
vcc vcc 0 vcc
```

```

vin in 0 pwl 0,0 0.2n,5
x1 in out inv
.macro inv in out
mn out in 0 0 nch w=10u l=1u
mp out in vcc vcc pch w=10u l=1u
.eom

```

Star-Hspice produces the following tabular listing:

```

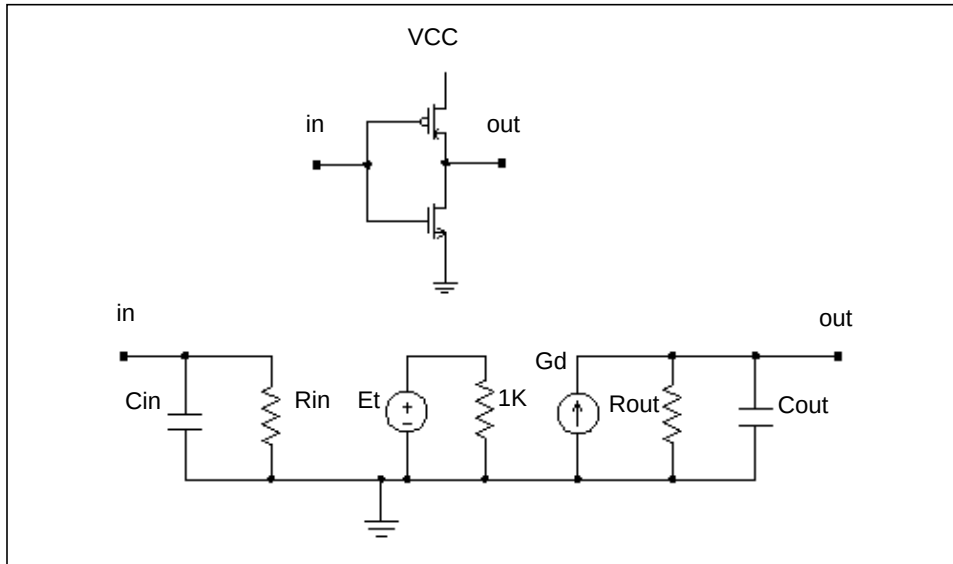
***** dc transfer curves tnom= 25.000 temp= 25.000
volt          rn
0.            3.312e+14
285.71429m    317.3503g
571.42857m    304.0682x
857.14286m    1.1222x
1.14286       107.6844k
1.42857       32.1373k
1.71429       14.6984k
2.00000       7.7108k
2.10000       5.8210k
2.12105       5.1059k
2.14211       3.2036k
2.16316       1.6906k
2.18421       1.4421k
2.20526       1.3255k
2.22632       1.2470k
2.24737       1.1860k
2.26842       1.1360k
2.28947       1.0935k
2.31053       1.0565k
2.33158       1.0238k
2.35263       994.3804
2.37368       967.7559
2.39474       943.4266
2.41579       921.0413
2.43684       900.3251
2.45789       881.0585
2.47895       863.0632
2.50000       846.1922
2.75000       701.5119
3.20000       560.6908
3.65000       479.8893
4.10000       426.4486
4.55000       387.7524
5.00000       357.4228

```

Optimizing Behavioral CMOS Inverters

To calibrate behavioral models, run Star-Hspice on the full transistor version of a cell. Then optimize the behavioral model to this data.

Figure 17-11: CMOS Inverter and its Equivalent Circuit



In this example, Star-Hspice uses the LEVEL 3 MOSFET model to simulate the CMOS inverter.

1. To obtain the input and output resistances, Star-Hspice performs a .TF transfer function analysis (.TF V(out) Vin).
2. To obtain the transfer function table of the inverter, Star-Hspice performs the .DC analysis, and sweeps the input voltage (.DC Vin 0 5 .1).
3. Star-Hspice uses this table, in the PWL element, to represent the transfer function of the inverter.
4. A voltage-controlled PWL capacitance adjusts the rise and fall time of the inverter, in the equivalent circuit, across the output resistance.
5. The delay element obtains the propagation delay, across the output rc circuit.
6. Star-Hspice uses the inverter in a ring oscillator, to adjust the input capacitance.

Star-Hspice uses optimization analysis for all adjustments in this example. The data file and the results are shown below.

Example

```

INVB_OP.SP---OPTIMIZATION OF CMOS MACROMODEL INVERTER
.OPTION POST PROBE NOMOD METHOD=GEAR
.GLOBAL VCC VCCM
.PARAM VCC=5 ROUT=2.5K CAPIN=.5P
+ TDELAY=OPTINV(1.0N, .5N, 3N)
+ CAPL=OPTINV(.2P, .1P, .6P)
+ CAPH=OPTINV(.2P, .1P, .6P)
.TRAN .25N 120NS
+ SWEEP OPTIMIZE=OPTINV RESULTS=RISEX, FALLX, PROPFX, PROPRX
+ MODEL=OPT1
.MODEL OPT1 OPT ITROPT=30 RELIN=1.0E-5 RELOUT=1E-4
.MEAS TRAN PROPFM TRIG V(INM) VAL='.5*VCC' RISE=2
+ TARG V(OUTM) VAL='.5*VCC' FALL=2
.MEAS TRAN PROPFX TRIG V(IN) VAL='.5*VCC' RISE=2
+ TARG V(OUT) VAL='.5*VCC' FALL=2
+ GOAL='PROPFM' WEIGHT=0.8
.MEAS TRAN PROPRM TRIG V(INM) VAL='.5*VCC' FALL=2
+ TARG V(OUTM) VAL='.5*VCC' RISE=2
.MEAS TRAN PROPRX TRIG V(IN) VAL='.5*VCC' FALL=2
+ TARG V(OUT) VAL='.5*VCC' RISE=2
+ GOAL='PROPRM' WEIGHT=0.8
.MEAS TRAN FALLM TRIG V(OUTM) VAL='.9*VCC' FALL=2
+ TARG V(OUTM) VAL='.1*VCC' FALL=2
.MEAS TRAN FALLX TRIG V(OUT) VAL='.9*VCC' FALL=2
+ TARG V(OUT) VAL='.1*VCC' FALL=2
+ GOAL='FALLM'
.MEAS TRAN RISEM TRIG V(OUTM) VAL='.1*VCC' RISE=2
+ TARG V(OUTM) VAL='.9*VCC' RISE=2
.MEAS TRAN RISEX TRIG V(OUT) VAL='.1*VCC' RISE=2
+ TARG V(OUT) VAL='.9*VCC' RISE=2
+ GOAL='RISEM'

.TRAN 0.5N 120N
.PROBE V(out) V(outm)
VC VCC 0 VCC
VCCM VCCM 0 VCC
X1 IN OUT INV
X1M INM OUTM INVM
VIN IN GND PULSE(0,5 1N,5N,5N 20N,50N)
VINM INM GND PULSE(0,5 1N,5N,5N 20N,50N)

```

Subcircuit Definition

```
.SUBCKT INV IN OUT
RIN IN 0 1E12
CIN IN 0 CAPIN
ET 1 0 PWL(1) IN 0
+ 1.00000 5.0
+ 1.50000 4.93
+ 2.00000 4.72
+ 2.40000 4.21
+ 2.50000 3.77
+ 2.60000 0.90
+ 2.70000 0.65
+ 3.00000 0.30
+ 3.50000 0.092
+ 4.00000 0.006
+ 4.60000 0.
RT 1 0 1K
GD 0 OUT DELAY 1 0 TD=TDELAY SCALE='1/ROUT'
GCOUT OUT 0 VCCAP PWL(1) IN 0 1V,CAPL 2V,CAPH
ROUT OUT 0 ROUT
.ENDS
```

Inverter Using Model

```
.SUBCKT INVM IN OUT
XP1 OUT IN VCCM VCCM MP
XN1 OUT IN GND GND MN
.ENDS
.MODEL N NMOS LEVEL=3 TOX=850E-10 LD=.85U NSUB=2E16
+ VTO=1 GAMMA=1.4 PHI=.9 U0=823 VMAX=2.7E5 XJ=0.9U
+ KAPPA=1.6 ETA=.1 THETA=.18 NFS=1.6E11 RSH=25
+ CJ=1.85E-4 MJ=.42 PB=.7 CJSW=6.2E-10 MJSW=.34
+ CGS0=5.3E-10 CGD0=5.3E-10 CGB0=1.75E-9
.MODEL P PMOS LEVEL=3 TOX=850E-10 LD=.6U
+ NSUB=1.4E16 VTO=-.86 GAMMA=.65 PHI=.76 U0=266
+ VMAX=.8E5 XJ=0.7U KAPPA=4 ETA=.25 THETA=.08
+ NFS=2.3E11 RSH=85 CJ=1.78E-4 MJ=.4 PB=.6 CJSW=5E-10
+ MJSW=.22 CGS0=5.3E-10 CGD0=5.3E-10 CGB0=.98E-9
SUBCKT MP 1 2 3 4
M1 1 2 3 4 P W=45U L=5U AD=615P AS=615P
+ PD=65U PS=65U NRD=.4 NRS=.4
.ENDS MP
.SUBCKT MN 1 2 3 4
M1 1 2 3 4 N W=17U L=5U AD=440P AS=440P
+ PD=80U PS=80U NRD=.85 NRS=.85
.ENDS MN
.END
```

Result

```

OPTIMIZATION RESULTS
  RESIDUAL SUM OF SQUARES      = 4.589123E-03
  NORM OF THE GRADIENT         = 1.155285E-04
  MARQUARDT SCALING PARAMETER  = 130.602
  NO. OF FUNCTION EVALUATIONS  = 51
  NO. OF ITERATIONS            = 15
OPTIMIZATION COMPLETED
MEASURED RESULTS < RELOUT= 1.0000E-04 ON LAST
+ ITERATIONS

```

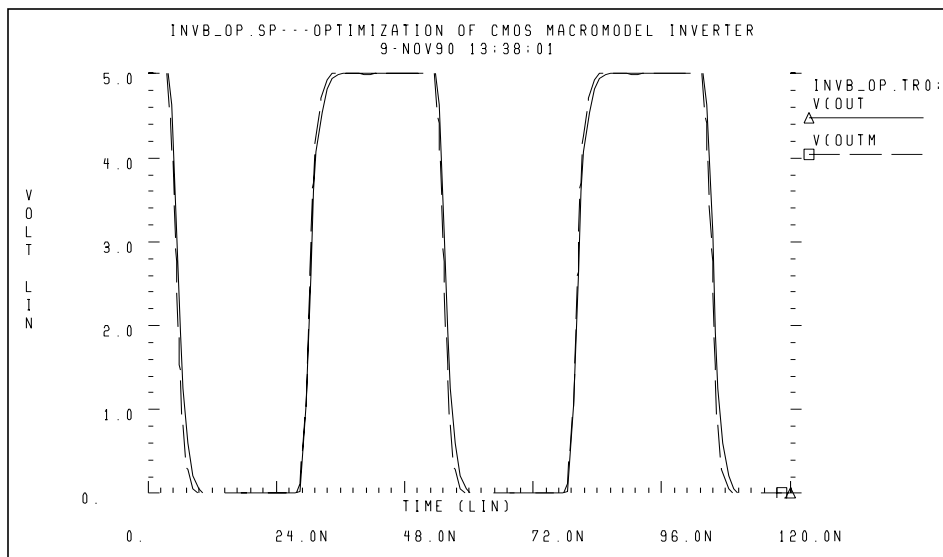
Optimized Parameters OPTINV

*			%NORM-SEN	%CHANGE
.PARAM	TDELAY	= 1.3251N	\$ 37.6164	-48.6429U
.PARAM	CAPL	= 390.2613F	\$ 37.2396	60.2596U
.PARAM	CAPH	= 364.2716F	\$ 25.1440	62.1922U

Optimize Results Measure Names and Values

* RISEX	=	2.7018N
* FALLX	=	2.5388N
* PROPFX	=	2.0738N
* PROPRX	=	2.1107N

Figure 17-12: CMOS Inverter Response



Optimizing Behavioral Ring Oscillators

To optimize behavioral ring oscillator performance, review the examples in this section.

Example Five-Stage Ring Oscillator

```

RING5BM.SP-5 STAGERING OSCILLATOR--MACROMODEL CMOS INVERTER
.IC V(IN)=5 V(OUT1)=0 V(OUT2)=5 V(OUT3)=0
.IC V(INM)=5 V(OUT1M)=0 V(OUT2M)=5 V(OUT3M)=0
.GLOBAL VCCM
.OPTION NOMOD POST=2 PROBE METHOD=GEAR DELMAX=0.5N
.PARAM VCC=5 $ CAPIN=0.92137P
.PARAM TDELAY=1.32N CAPL=390.26F CAPH=364.27F ROUT=2.5K
+ CAPIN=OPTOSC(0.8P,0.1P,1.0P)
.TRAN 1NS 150NS UIC
+ SWEEP OPTIMIZE=OPTOSC RESULTS=PERIODX MODEL=OPT1
.MODEL OPT1 OPT RELIN=1E-5 RELOUT=1E-4 DIFSIZ=.02 ITROPT=25
.MEAS TRAN PERIODM TRIG V(OUT3M) VAL='.8*VCC' RISE=2
+ TARG V(OUT3M) VAL='.8*VCC' RISE=3
.MEAS TRAN PERIODX TRIG V(OUT3) VAL='.8*VCC' RISE=2
+ TARG V(OUT3) VAL='.8*VCC' RISE=3
+ GOAL='PERIODM'

.TRAN 1NS 150NS UIC
.PROBE V(OUT3) V(OUT3M)
X1 IN OUT1 INV
X2 OUT1 OUT2 INV
X3 OUT2 OUT3 INV
X4 OUT3 OUT4 INV
X5 OUT4 IN INV
CL IN 0 1P
VCCM VCCM 0 VCC
X1M INM OUT1M INVM
X2M OUT1M OUT2M INVM
X3M OUT2M OUT3M INVM
X4M OUT3M OUT4M INVM
X5M OUT4M INM INVM
CLM INM 0 1P

*Subcircuit definitions given in the previous example
*are not repeated here.

.END

```

Result

Optimization Results

```

RESIDUAL SUM OF SQUARES      = 4.704516E-10
NORM OF THE GRADIENT          = 2.887249E-04
MARQUARDT SCALING PARAMETER   = 32.0000
NO. OF FUNCTION EVALUATIONS   = 52
NO. OF ITERATIONS             = 20

```

OTIMIZATION COMPLETED

```

MEASURED RESULTS < RELOUT= 1.0000E-04 ON LAST
+ ITERATIONS

```

**** OPTIMIZED PARAMETERS OPTOSC

```

*                                     %NORM-SEN  %CHANGE
.PARAM CAPIN                        = 921.4155F $ 100.0000  8.5740U

```

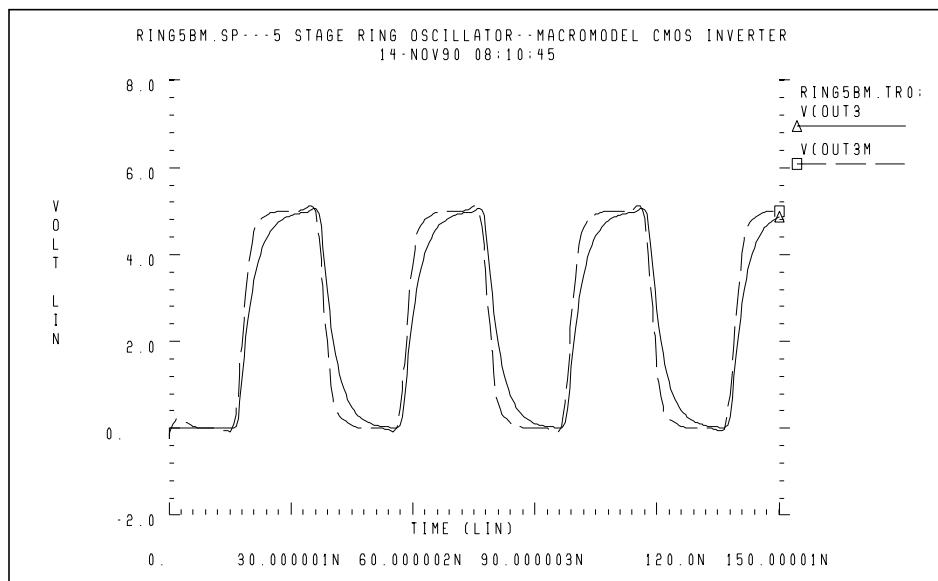
*** OPTIMIZE RESULTS MEASURE NAMES AND VALUES

```

* PERIODX                          = 40.3180N

```

Figure 17-13: Ring Oscillator Response



Analog Behavioral Elements

The following components are examples of analog behavioral building blocks. Each component demonstrates a basic Star-Hspice feature:

- | | |
|--------------------------------|------------------------------------|
| ■ integrator | ideal op-amp E Element source |
| ■ differentiator | ideal op-amp E Element source |
| ■ ideal transformer | ideal transformer E Element source |
| ■ tunnel diode | lookup table G Element source |
| ■ silicon-controlled rectifier | lookup table H Element source |
| ■ triode vacuum tube | algebraic G Element source |
| ■ AM modulator | algebraic G Element source |
| ■ data sampler | algebraic E Element source |

Behavioral Integrator

Star-Hspice uses an ideal op-amp to model the integrator circuit, and a VCVS to adjust the output voltage. The following equation calculates the output of the integrator:

$$V_{out} = -\frac{\text{gain}}{R_i \cdot C_i} \cdot \int_0^t V_{in} \cdot dt + V_{out}(0)$$

Figure 17-14: Integrator Circuit

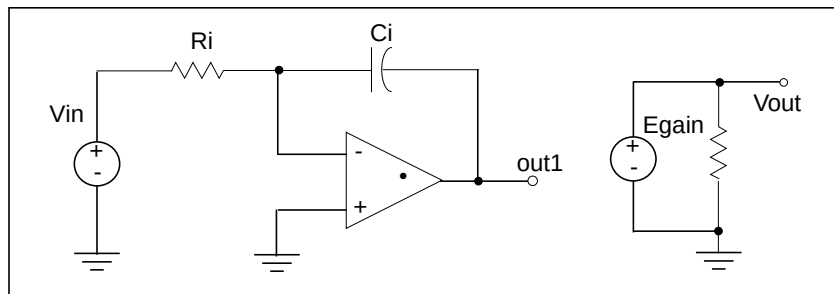
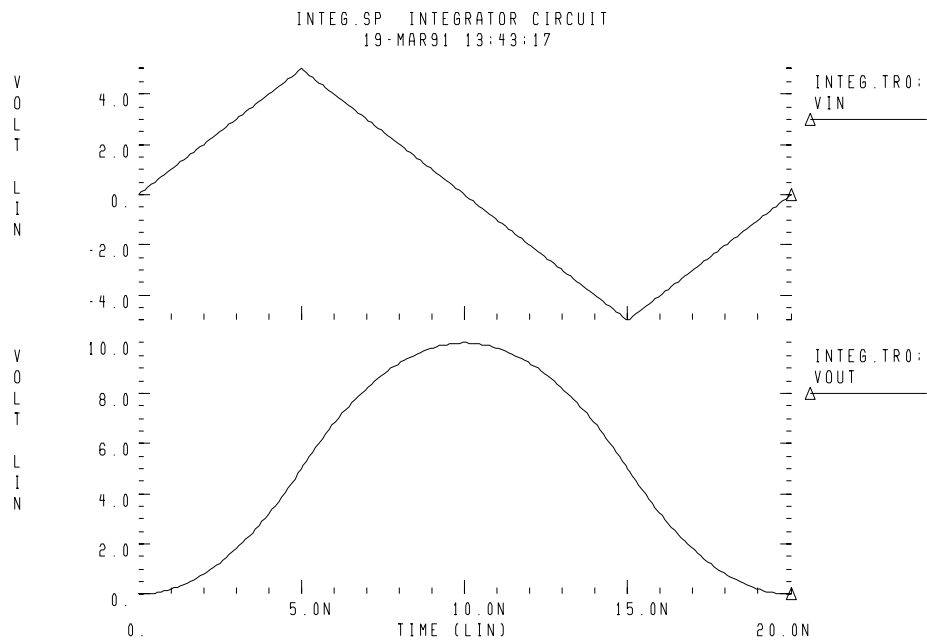


Figure 17-15: Response of Integrator to a Triangle Waveform



Example

```
Integ.sp integrator circuit
```

Control and Options

```
.TRAN 1n 20n
.OPTION POST PROBE DELMAX =.1n
.PROBE Vin=V(in) Vout=V(out)
```

Subcircuit Definition

```
.SUBCKT integ in out gain=-1 rval=1k cval=1p
EOP out1 0 OPAMP in- 0
Ri in in- rval
Ci in- out1 cval
Egain out 0 out1 0 gain
Rout out 0 1e12
.ENDS
```

Circuit

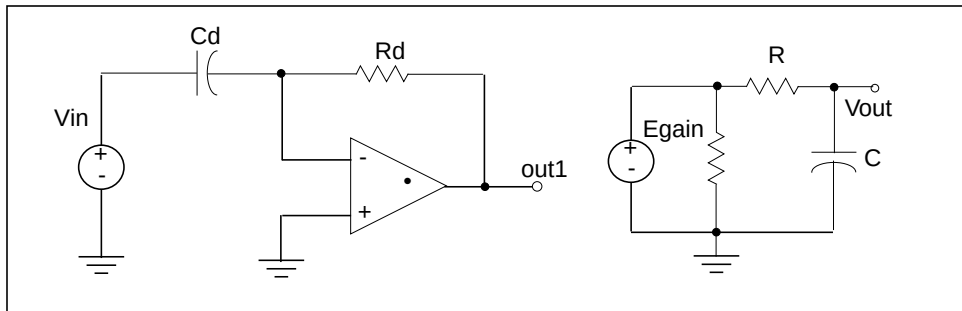
```
Xint in out integ gain=-0.4
Vin in 0 PWL(0,0 5n,5v 15n,-5v 20n,0)
.END
```

Behavioral Differentiator

Star-Hspice uses an ideal op-amp to model a differentiator, and a VCVS to adjust the magnitude and polarity of the output. The following equation calculates the differentiator response:

$$V_{out} = -\text{gain} \cdot R_d \cdot C_d \cdot \frac{d}{dt} V_{in}$$

For a high-frequency signal, the output of a differentiator can overshoot the edges. To smooth this out, you can use a simple RC filter.

Figure 17-16: Differentiator Circuit

Example

```
Diff.sp differentiator circuit
* V(out)=Rval * Cval * gain * (dV(in)/dt)
```

Control and Options

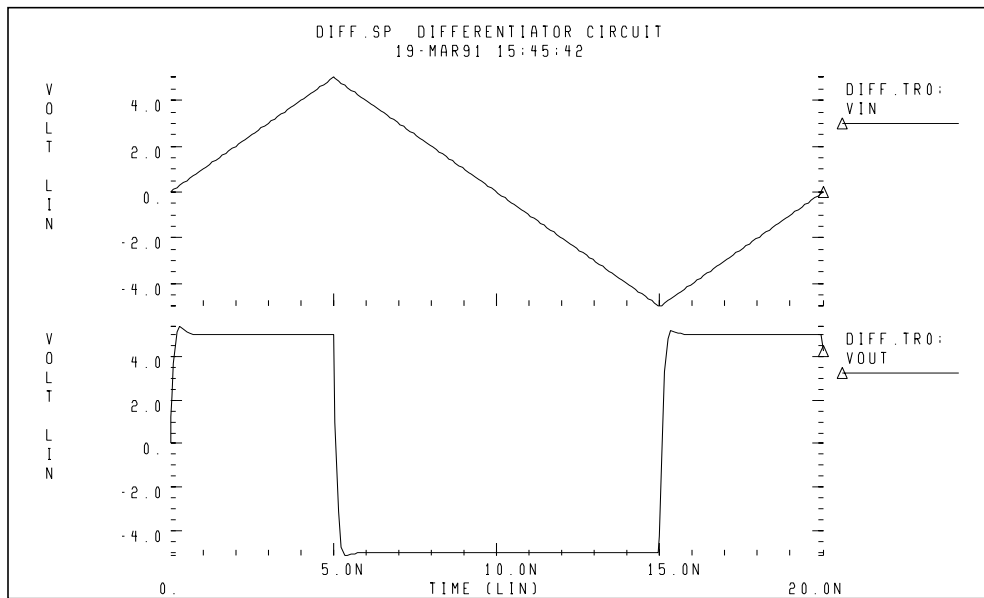
```
.TRAN 1n 20n
.PROBE Vin=V(in) Vout=V(out)
.OPTION PROBE POST
```

Subcircuit Definition

```
.SUBCKT diff in out gain=-1 rval=1k cval=1pf
EOP out1 0 OPAMP in- 0
Cd in in- cval
Rd in- out1 rval
Egain out2 0 out1 0 gain
Rout out2 0 1e12
*rc filter to smooth the output
R out2 out 75
C out 0 1pf
.ENDS
```

Circuit

```
Xdiff in out diff rval=5k
Vin in 0 PWL(0,0 5n,5v 15n,-5v 20n,0)
.END
```

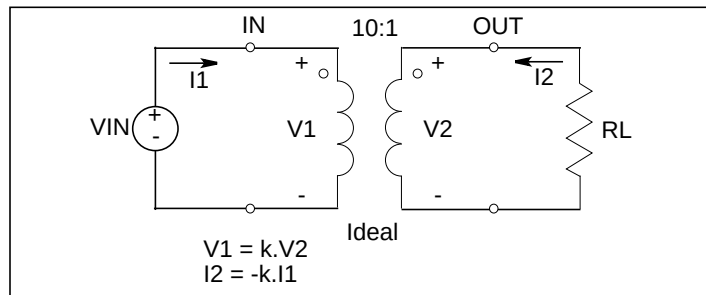
Figure 17-17: Response Of a Differentiator to a Triangle Waveform

Ideal Transformer

The following example uses the ideal transformer to convert 8-ohms impedance of a loudspeaker, to 800 ohms impedance. This is a proper load value for a power amplifier, $R_{in} = n^2 \cdot R_L$.

```
MATCHING IMPEDANCE BY USING IDEAL TRANSFORMER
E OUT 0 TRANSFORMER IN 0 10
RL OUT 0 8
VIN IN 0 1
.OP
.END
```

Figure 17-18: Ideal Transformer Example

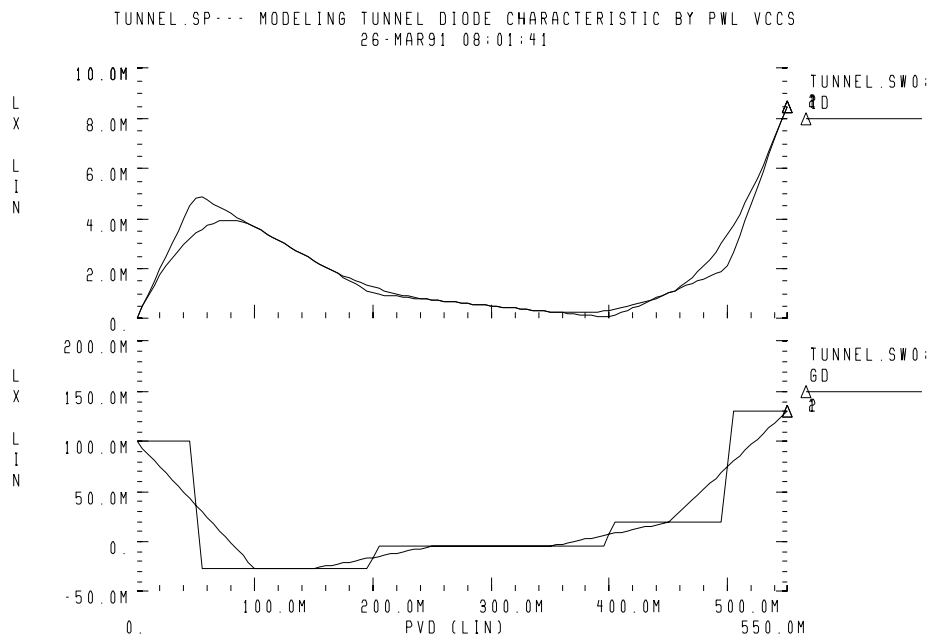


Behavioral Tunnel Diode

The following example uses a PWL VCCS to model a tunnel diode. Star-Hspice obtains the current characteristics for two DELTA values (50 μv and 10 μv). The IV characteristics, corresponding to DELTA=10 μv , have sharper corners. Star-Hspice also displays the derivative of the current, with respect to voltage (GD). The GD value, around the breakpoints, changes in a linear fashion.

The following is an example:

```
tunnel.sp-- modeling tunnel diode characteristic
+ by pwl vccs
* pwl function is tested for two different delta values.
+ The smaller delta will create the sharper corners.
.OPTION post=2
vin 1 0 pvd
.dc pvd 0 550m 5m sweep delta poi 2 50mv 5mv
.probe dc id=lx0(g) gd=lx2(g)
g 1 0 pwl(1) 1 0 delta=delta
+ -50mv, -5ma 50mv, 5ma 200mv, 1ma 400mv, .05ma
+ 500mv, 2ma 600mv, 15ma
.end
```

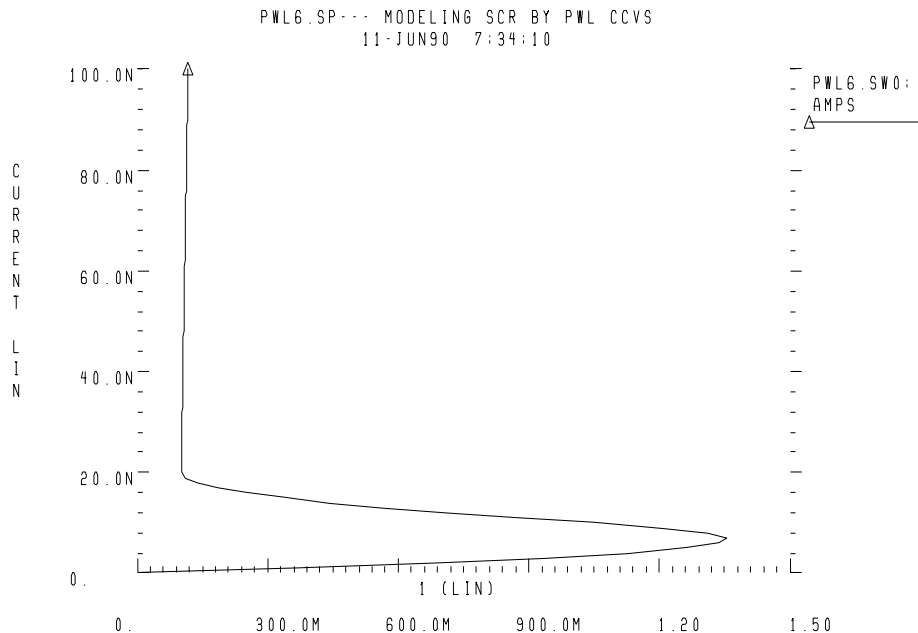
Figure 17-19: Tunnel Diode Characteristic

Behavioral Silicon-Controlled Rectifier (SCR)

To model the silicon controlled rectifier (SCR) characteristic, use a PWL CCVS, which provides a unique voltage value for any current value.

The following is an example:

```
pwl6.sp--- modeling SCR by pwl ccvs
*The silicon controlled rectifier (SCR) characteristic
*is modelled by a piecewise linear current controlled
*voltage source (PWL_CCVS), because for any current
*value there is a unique voltage value.
*use iscr as y-axis and v(1) as x-axis
.OPTION post=2
iscr 0 2 0
vdum 2 1 0
.dc iscr 0 1u 1n
.probe vscr=lx0(h) transr=lx3(h)
h 1 0 pwl(1) vdum -5na,-2v 5na,2v 15na,.1v 1ua,.3v
+ 10ua,.5 delta=5na
.end
```

Figure 17-20: Silicon Controlled Rectifier

Behavioral Triode Vacuum Tube Subcircuit

The following example shows how to include the behavioral elements in a subcircuit, for very good triode tube action. The *ea* voltage source modifies the basic power law equation (current source *gt*), for better response in saturation.

Example

```
triode.sp triode model family of curves using
+ behavioral elements
```

Control and Options

```
.OPTION post acct
.dc va 20v 60v 1v vg 1v 10v 1v
.probe ianode=i(xt.ra) v(anode) v(grid) eqn=lv6(xt.gt)
.print v(xt.int_anode) v(xt.i_anode) inode=i(xt.ra)
+ eqn=lv6(xt.gt)
```

Circuit

```

vg grid 0 1v
va anode 0 20v
vc cathode 0 0v
xt anode grid cathode triode

```

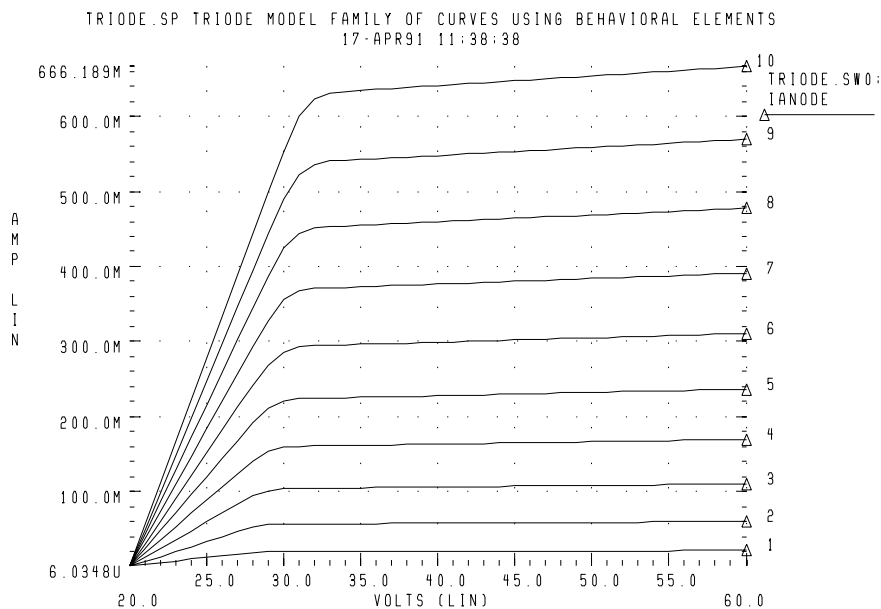
Subcircuit Definition

```

.subckt triode anode grid cathode
* 5 ohm anode resistance
* ea creates saturation region conductance
ra anode i_anode 5
ea int_anode cathode pwl(1) i_anode cathode delta=.01
+ 20,0 27,.85 28,.95 29,.99 30,1 130,1.2
gt i_anode cathode
+ cur='20m*v(int_anode,cathode)*pwr
+ (max(v(grid,cathode),0),1.5)'
cga grid i_anode 30p
cgc grid cathode 20p
cac i_anode cathode 5p
.ends
*
.end

```

Figure 17-21: Triode Family of Curves

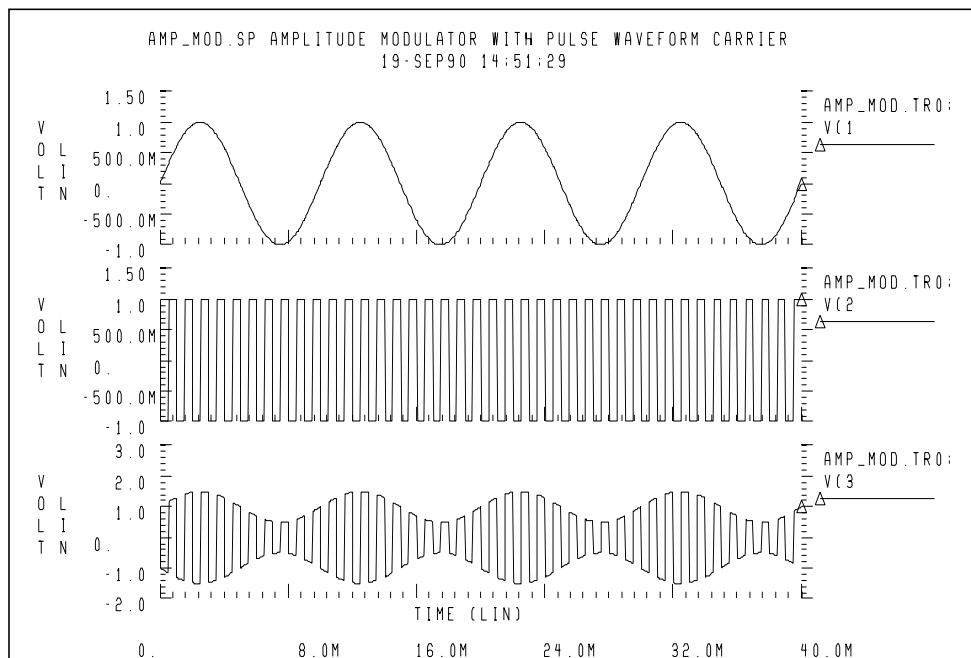


Behavioral Amplitude Modulator

This example uses a G Element as an amplitude modulator, with a pulse waveform carrier.

```
amp_mod.sp amplitude modulator with pulse waveform
+ carrier
.OPTION POST
.TRAN .05m 40m
.PROBE V(1) V(2) V(3)
Vs 1 0 SIN(0,1,100)
Vc 2 0 PULSE(1,-1,0,1n,1n,.5m,1m)
Rs 1 0 1+
Rc 2 0 1
G 0 3 CUR='(1+.5*V(1))*V(2) '
Re 3 0 1
.END
```

Figure 17-22: Amplitude Modulator Waveforms

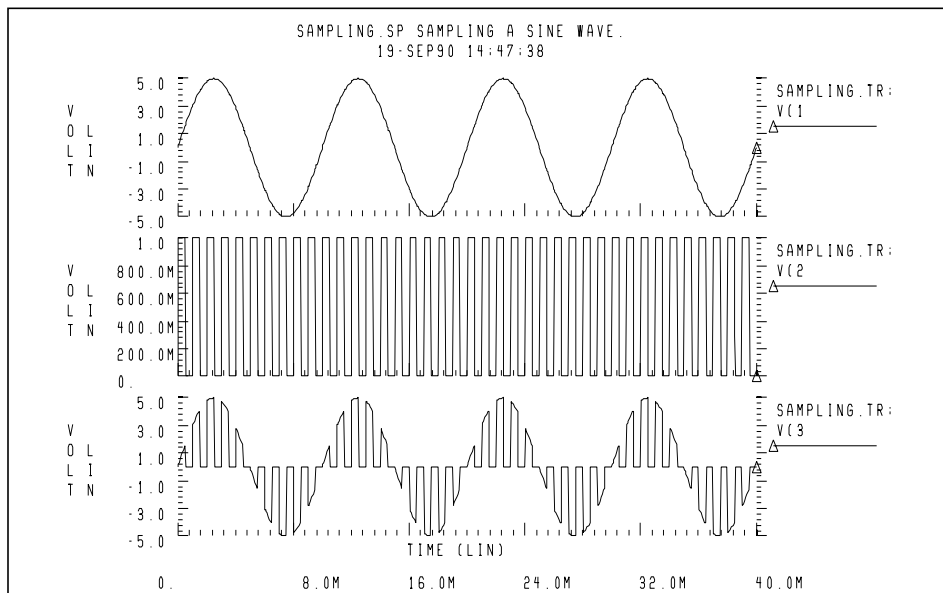


Behavioral Data Sampler

The following is an example of sampling behavioral data:

```
sampling.sp sampling a sine wave.
.OPTION POST
.TRAN .05m 40m
.PROBE V(1) V(2) V(3)
Vc 1 0 SIN(0,5,100)
Vs 2 0 PULSE(0,1,0,1n,1n,.5m,1m)
Rc 1 0 1
Rs 2 0 1
E 3 0 VOL='V(1)*V(2)'
Re 3 0 1
.END
```

Figure 17-23: Sampled Data



Op-Amps, Comparators, and Oscillators

This section describes the benefits of using op-amps, comparators, and oscillators, when you perform Star-Hspice simulation.

Op-Amp Model Generator

Star-Hspice uses the model generator to automatically design and simulate both board-level and IC op-amp designs.

1. Start from the existing electrical specifications for a standard industrial operational amplifier.
2. Enter the specifications in the op-amp model statement.

Star-Hspice automatically generates the internal components of the op-amp, to meet the specifications.

3. You can then call the design from a library, for a board-level simulation.

The Star-Hspice op-amp model is a subcircuit. It is about 20 times faster to simulate, than an actual transistor level op-amp. You can adjust the AC gain and phase to within 20 percent of the actual measured values, and set the transient slew rates accurately. This model does not contain high-order frequency response poles and zeros, so it can significantly differ from actual amplifiers, in predicting high-frequency instabilities.

You can use this model to represent normal amplifier characteristics, including:

- input offsets
- small signal gain
- transient effects

The op-amp subcircuit generator consists of a model, and one or more elements. Each element is in the form of a subcircuit call.

1. The model generates an output file of the op-amp equivalent circuit, which you can collect into libraries.

The file name is the name of the model (*mname*), with an *.inc* extension.

2. After you generate the output file, other Star-Hspice input files can reference this subcircuit, using a `.SUBCKT` call to the model name.
3. The `.SUBCKT` call automatically searches for the file in the present directory.
4. It then searches the directories specified in any `.OPTION SEARCH = 'directory_path_name' .`
5. Finally, it searches the directory where the DDL (Discrete Device Library) is located.

The amplifier element references the amplifier model.

If the model generator creates op-amp model that do not converge in DC analysis, use the `.IC` or `.NODESET` statement, to set the input nodes to the voltage that is halfway between the VCC and VEE. This balances the input nodes, and stabilizes the model.

Op-Amp Element Statement Format

COMP=0 (internal compensation)

The syntax is:

```
xa1 in- in+ out vcc vee modelname AV=val
```

COMP=1 (external compensation)

The syntax is:

```
xa1 in- in+ out comp1 comp2 vcc vee modelname AV=val
```

<i>in-</i>	inverting input
<i>in+</i>	noninverting input
<i>out</i>	output, single ended
<i>vcc</i>	positive voltage supply
<i>vee</i>	negative voltage supply
<i>modelname</i>	subcircuit reference name

Op-Amp .MODEL Statement Format

The syntax is:

```
.MODEL mname AMP parameter=value ...
```

mname Model name. Elements use this name to reference the model.
AMP Identifies an amplifier model.
parameter Any model parameter, as described below.
value Value assigned to a parameter.

Example

```
X0 IN- IN+ OUT0 VCC VEE ALM124
.MODEL ALM124 AMP
+ C2=      30.00P      SRPOS=    .5MEG      SRNEG=    .5MEG
+ IB=      45N         IBOS=     3N          VOS=     4M
+ FREQ=    1MEG        DELPHS=   25          CMRR=    85
+ ROUT=    50          AV=       100K        ISC=     40M
+ VOPOS=   14.5        VONEG=   -14.5       PWR=     142M
+ VCC=     16          VEE=     -16         TEMP=    25.00
+ PSRR=    100         DIS=      8.00E-16    JIS=     8.00E-16
```

Op-Amp Model Parameters

[Table 17-1](#) shows the model parameters for op-amps. The defaults for these parameters depend on the DEF parameter setting. [Table 17-2](#) shows defaults for each of the three DEF settings.

Table 17-1: Op-Amp Model Parameters (Sheet 1 of 7)

Names (Alias)	Units	Default	Description
AV (AVD)	volt/ volt		Amplifier gain, in volts out, per volt in. This is the DC ratio of the voltage in, to the voltage out. Typical gains are from 25k to 250k. If the frequency is too low, increase the negative and positive slew rates, or decrease DELPHS.

Table 17-1: Op-Amp Model Parameters (Sheet 2 of 7)

Names (Alias)	Units	Default	Description
AV1K	volt/ volt		Amplifier gain, at 1 kilohertz. This method estimates the unity-gain bandwidth. You can express gain as actual voltage gain, or in decibels (a standard unit conversion for Star-Hspice). If you set AV1K, Star-Hspice ignores FREQ. A typical value for AV1K is $AV1K = (unity\ gain\ freq) / 1000$.
C2	farad		Internal feedback compensation capacitance. ■ For an internally-compensated amplifier, if you do not specify a capacitance value, the default is 30 pF. ■ If the gain is high (above 500k), the internal compensation capacitor is probably different (typically 10 pF). ■ For an externally-compensated amplifier (COMP=1), set C2 to 0.5 pF, as the residual internal capacitance.
CMRR	volt/ volt		Common mode rejection ratio. This is usually between 80 and 110 dB. You can enter this value as 100 dB, or as 100000.
COMP			Compensation level selector. If you set this parameter to 1, it modifies the number of nodes in the equivalent, to include external compensation nodes. See C2 for external compensation settings. ■ COMP=0 internal compensation (default). ■ COMP=1 external compensation.

Table 17-1: Op-Amp Model Parameters (Sheet 3 of 7)

Names (Alias)	Units	Default	Description
DEF			Default model selector. Choose one of three default settings. ■ 0= generic (0.6 MHz bandwidth) (Default). ■ 1= ua741 (1.2 MHz bandwidth) ■ 2= mc4560 (3 MHz bandwidth).
DELPHS	deg		Excess phase, at the unity gain frequency. Also called the <i>phase margin</i>. Star-Hspice measures DELPHS in degrees. Typical excess phases range from 5° to 50°. 1. To determine DELPHS, subtract the phase at unity gain, from 90°. The result is the phase margin. 2. Use the same chart as used for the FREQ determination above. DELPHS interacts with FREQ (or AV1K). Values of DELPHS tend to lower the unity gain bandwidth, especially values greater than 20°. 3. Pick the DELPHS closest to measured value, that does not reduce unity gain bandwidth more than 20%. Otherwise, the model might not have enough poles, to always return correct phase and frequency responses.
DIS	amp	1e-16	Saturation current, for diodes and BJTs.

Table 17-1: Op-Amp Model Parameters (Sheet 4 of 7)

Names (Alias)	Units	Default	Description
<i>FREQ</i> (GBW,BW)	Hz		Unity gain frequency, measured in hertz. Typical frequencies are from 100 kHz to 3 MHz. If you do not specify this parameter, measure the open-loop frequency response at 0 dB voltage gain, and measure the actual compensation capacitance. Typical compensation is 30 pF, and single-pole compensation configuration. Note: If AV1K is > zero, Star-Hspice calculates unity gain frequency from AV1K, and ignores FREQ.
IB	amp		Input bias current. The amount of current required to bias the input differential transistors. This is usually a fundamental electrical characteristic. Typical values are between 20 and 400 nA.
IBOS	amp		Input bias offset current, also called <i>input offset current</i> . This is the amount of unbalanced current, between the input differential transistors, and is usually a fundamental electrical characteristic. Typical values are 10% to 20% of the IB.

Table 17-1: Op-Amp Model Parameters (Sheet 5 of 7)

Names (Alias)	Units	Default	Description
ISC	amp		Input short circuit current – not always specified. Typical values are from 5 to 25 mA. Star-Hspice can determine ISC from output characteristics (current sinking), as the maximum output sink current. ISC and ROUT interact with each other. If ROUT is too large for the value of ISC, Star-Hspice automatically reduces ROUT.
JIS	amp		JFET saturation current. Default=1e-16. You do not need to change this value.
LEVIN			Input level type selector. You can create only a BJT differential pair. LEVIN=1 BJT differential input stage.
LEVOUT			Output level type selector. You can create only a single-ended output stage. LEVOUT=1 single-ended output stage.
MANU			Manufacturer's name. Add this to the model parameter list, to identify the source of model parameters. Star-Hspice prints the name in the final equivalent circuit.
PWR (PD)	watt		Total power dissipation, for the amplifier. This includes the calculated value, for the op-amp input differential pair. If you specify a high slew rate, and very low power, then Star-Hspice issues a warning, and shows the power dissipation <i>only</i> for the input differential pair.

Table 17-1: Op-Amp Model Parameters (Sheet 6 of 7)

Names (Alias)	Units	Default	Description
RAC (r0ac, roac)	ohm		High-frequency output resistance. This typically is about 60% of ROUT. RAC usually ranges between 40 to 70 ohms, for op-amps with video drive capabilities.
ROUT	ohm		Low-frequency output resistance. To determine this value, use the closed-loop output impedance graph. The impedance at about 1kHz, using the maximum gain, is close to ROUT. Gains of 1,000 and above show the effective DC impedance, generally in the frequency region between 1k and 10 kHz. Typical values for ROUT are 50 to 100 ohms.
SRNEG (SRN)	volt		Negative output slew rate. Star-Hspice extracts this value from a graph that shows the response for the voltage follower pulse. This is usually a 4-volt or 5-volt output change, with 10 to 20 volt supplies. Measures the negative change in voltage, and the amount of time for the change.
SRPOS (SRP)	volt		Positive output slew rate. Star-Hspice extracts this value from a graph that shows the response for a voltage follower pulse. This is usually a 4-volt or 5-volt output change, with 10 to 20 volt supplies. Measures the positive change in voltage, and the amount of time for the change. Typical slew rates are from 70k to 700k.

Table 17-1: Op-Amp Model Parameters (Sheet 7 of 7)

Names (Alias)	Units	Default	Description
TEMP	°C		Temperature, in degrees Celsius. This is usually the temperature at which Star-Hspice measured the model parameters, which is typically 25 °C.
VCC	volt		Positive power-supply reference voltage, for VOPOS. Star-Hspice measures the VOPOS amplifier, with respect to VCC.
VEE	volt		Negative power-supply voltage. Star-Hspice measures the VONEG amplifier, with respect to VCC.
VONEG (VON)	volt		Maximum negative output voltage. This is less than VEE (the negative power-supply voltage), by the internal voltage drop.
VOPOS (VOP)	volt		Maximum positive output voltage. This is less than VCC (the positive power supply voltage), by the internal voltage drop.
VOS	volt		Input offset voltage. The required voltage between input differential transistors, to zero the output voltage. This is usually a fundamental electrical characteristic. Typical values for bipolar amplifiers range from 0.1 mV to 10 mV. Star-Hspice measures VOS in volts. In some amplifiers, VOS can cause a failure to converge. If this occurs, set VOS to 0, or use the initial conditions for convergence.

Op-Amp Model Parameter Defaults

Table 17-2: Op-Amp Model Parameter Defaults

Parameter	Description	Defaults		
		DEF=0	DEF=1	DEF=2
AV	Amplifier voltage gain	160k	417k	200k
AV1K	Amplifier voltage gain, at 1 kHz	-	1.2 k	3 k
C2	Feedback capacitance	30 p	30 p	10 p
CMRR	Common-mode rejection ratio	96 db 63.1k	106 db 199.5k	90 db 31.63k
COMP	Compensation level selector	0	0	0
DEF	Default level selector	0	1	2
DELPHS	Delta phase, at unity gain	25°	17°	52°
DIS	Diode saturation current	8e-16	8e-16	8e-16
FREQ	Frequency, for unity gain	600 k	-	-
IB	Current, for input bias	30 n	250 n	40 n
IBOS	Current, for input bias offset	1.5 n	0.7 n	5 n
ISC	Current, for output short circuit	25 mA	25 mA	25 mA
LEVIN	Circuit-level selector, for input	1	1	1
LEVOUT	Circuit-level selector, for output	1	1	1
MANU	Manufacturer's name	-	-	-
PWR	Power dissipation	72 mW	60 mW	50 mW
RAC	AC output resistance	0	75	70
ROUT	DC output resistance	200	550	100

Table 17-2: Op-Amp Model Parameter Defaults (Continued)

Parameter	Description	Defaults		
		DEF=0	DEF=1	DEF=2
SRPOS	Positive output slew rate	450 k	1 meg	1 meg
SRNEG	Negative output slew rate	450 k	800 k	800 k
TEMP	Temperature of model	25 deg	25 deg	25 deg
VCC	Positive supply voltage, for VOPOS	20	15	15
VEE	Negative supply voltage, for VONEG	-20	-15	-15
VONEG	Maximum negative output	-14	-14	-14
VOPOS	Maximum positive output	14	14	14
VOS	Input offset voltage	0	0.3 m	0.5 m

Op-Amp Subcircuit Example

AUTOSTOP Option

This example uses the `.OPTION AUTOSTOP` option, to shorten simulation time. After Star-Hspice completes the measurements specified in the `.MEASURE` statement, the associated transient analysis and AC analysis stops, even if the analysis has not yet completed the full sweep range.

AC Resistance

`AC=10000G` parameter, in the `Rfeed` element statement, installs a $10000\text{ G}\Omega$ feedback resistor. The AC analysis uses this resistor, in place of the $10\text{ k}\Omega$ feedback resistor (used in the DC operating point and transient analysis), which is open-circuited for the AC measurements.

Simulation Results

The simulation results include the DC operating point analysis, for an input voltage of 0 v, and power supply voltages of 15v.

- The DC offset voltage is 3.3021 mv, which is less than that specified for the original VOS specification, in the op-amp .MODEL statement.
- The unity-gain frequency is 907.885 kHz, which is within 10% of the 1 MHz that the FREQ parameter (in the .MODEL statement) specifies.
- The required time rate, for a 1-volt change in the output (from the .MEASURE statement), is 2.3 μ s (from the SRPOS simulation result listing). This provides a slew rate of 0.434 Mv/s, which is within about 12% of the 0.5 Mv/s, specified in the SRPOS parameter of the .MODEL statement.
- The negative slew rate is almost exactly 0.5 Mv/s, which is within 1% of the slew rate specified in the .MODEL statement.

Example

```

$$ FILE ALM124.SP
.OPTION NOMOD AUTOSTOP SEARCH=' '
.OP VOL
.AC DEC 10 1HZ 10MEGHZ
.MODEL PLOTDB PLOT XSCAL=2 YSCAL=3
.MODEL PLOTLOGX PLOT XSCAL=2
.GRAPH AC MODEL=PLOTDB VM(OUT0)
.GRAPH AC MODEL=PLOTLOGX VP(OUT0)
.TRAN 1U 40US 5US .15MS
.GRAPH V(IN) V(OUT0)
.MEASURE TRAN 'SRPOS' TRIG V(OUT0) VAL=2V RISE=1
+ TARG V(OUT0) VAL=3V RISE=1
.MEASURE TRAN 'SRNEG' TRIG V(OUT0) VAL=-2V FALL=1
+ TARG V(OUT0) VAL=-3V FALL=1
.MEASURE AC 'UNITFREQ' TRIG AT=1
+ TARG VDB(OUT0) VAL=0 FALL=1
.MEASURE AC 'PHASEMARGIN' FIND VP(OUT0)
+ WHEN VDB(OUT0)=0
.MEASURE AC 'GAIN(DB)' MAX VDB(OUT0)
.MEASURE AC 'GAIN(MAG)' MAX VM(OUT0)
VCC VCC GND +15V
VEE VEE GND -15V
VIN IN GND AC=1 PWL 0US 0V 1US 0V 1.1US +10V 15US +10V
+ 15.2US -10V 100US -10V

```

```

.MODEL ALM124 AMP
+ C2=      30.00P      SRPOS=      .5MEG      SRNEG=      .5MEG
+ IB=      45N        IBOS=      3N          VOS=      4M
+ FREQ=    1MEG       DELPHS=    25          CMRR=      85
+ ROUT=    50         AV=      100K         ISC=      40M
+ VOPOS=   14.5       VONEG=    -14.5       PWR=      142M
+ VCC=     16         VEE=     -16         TEMP=     25.00
+ PSRR=    100       DIS=      8.00E-16     JIS=      8.00E-16
*
```

Unity Gain Resistor Divider Mode

```

*
Rfeed      OUT0      IN-      10K          AC=10000G
RIN        IN        IN-      10K
RIN+       IN+       GND      10K
X0         IN-       IN+      OUT0
ROUT0      OUT0      GND      2K          VCC VEE ALM124
COUT0      OUT0      GND      100P
.END

***** OPERATING POINT STATUS IS VOLTAGE
***** SIMULATION TIME IS 0.

      NODE      =VOLTAGE      NODE      =VOLTAGE      NODE      =VOLTAGE
+ 0:IN        = 0.            0:IN+     =-433.4007U    0:IN-     = 3.3021M
+ 0:OUT0      = 7.0678M      0:VCC    = 15.0000      0:VEE     = -15.0000

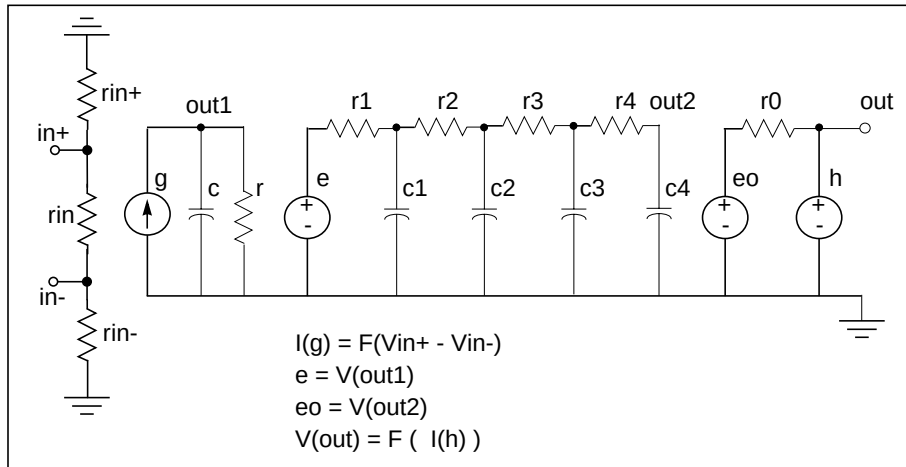
unitfreq      = 907.855K    TARG      = 907.856K    TRIG      = 1.000
PHASEMARGIN   = 66.403
gain(db)      = 99.663      AT        = 1.000
FROM          = 1.000      TO        = 10.000X
gain(mag)     = 96.192K    AT        = 1.000
FROM          = 1.000      TO        = 10.000X

srpos         = 2.030U      TARG      = 35.471U    TRIG      = 33.442U
srneg         = 1.990U      TARG      = 7.064U    TRIG      = 5.074U
```

741 Op-Amp from Controlled Sources

To model the μ A741 op-amp, use PWL controlled sources. A piecewise linear CCVS (source “h”) limits the output to ± 15 volts.

Figure 17-24: Op-Amp Circuit



Example

```

op_amp.sp --- operational amplifier
*
.OPTION post=2
.tran .001ms 2ms
.ac dec 10 .1hz 10me'
*.graph tran vout=v(output)
*.graph tran vin=v(input)
*.graph ac model=grap voutdb=vdb(output)
*.graph ac model=grap vphase=vp(output)
.probe tran vout=v(output) vin=v(input)
.probe ac voutdb=vdb(output) vphase=vp(output)
.model grap plot xscal=2

```

Main Circuit

```

xamp input 0 output opamp
vin input 0 sin(0,1m,1k) ac 1
* subcircuit definitions
* input subckt
.subckt      opin    in+    in-    out
             rin     in+    in-    2meg
             rin+    in+    0      500meg
             rin-    in-    0      500meg
g 0 out pwl(1) in+ in- -68mv, -68ma 68mv, 68ma delta=1mv
c          out    0      .136uf
r          out    0      835k
.ends

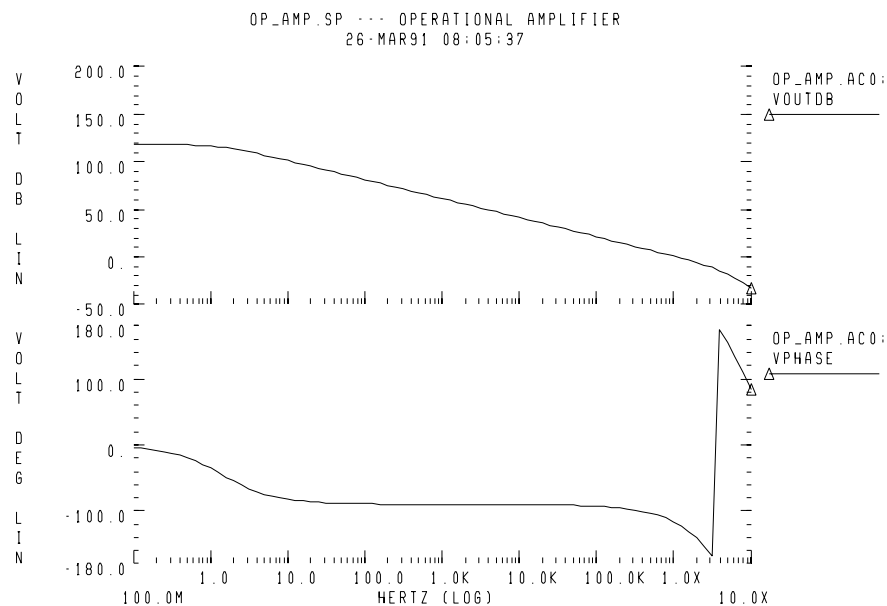
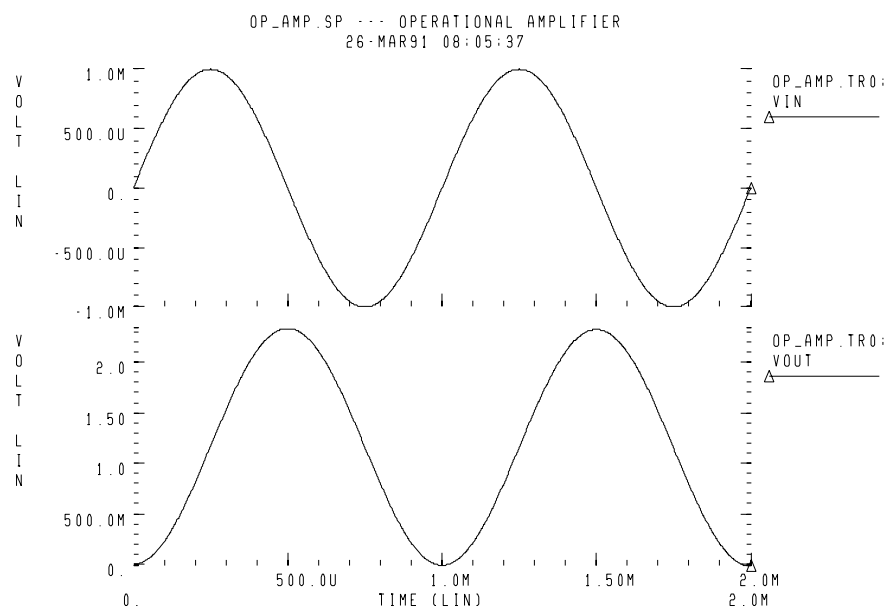
```

RC Circuit With Pole At 9 MHz

```
.subckt oprc in out
e out1 0 in 0 1
r1 out1 out2 168
r2 out2 out3 1.68k
r3 out3 out4 16.8k
r4 out4 out 168k
c1 out2 0 100p
c2 out3 0 10p
c3 out4 0 1p
c4 out 0 .1p
r out 0 1e12
.ends
```

Output Limiter to 15 v

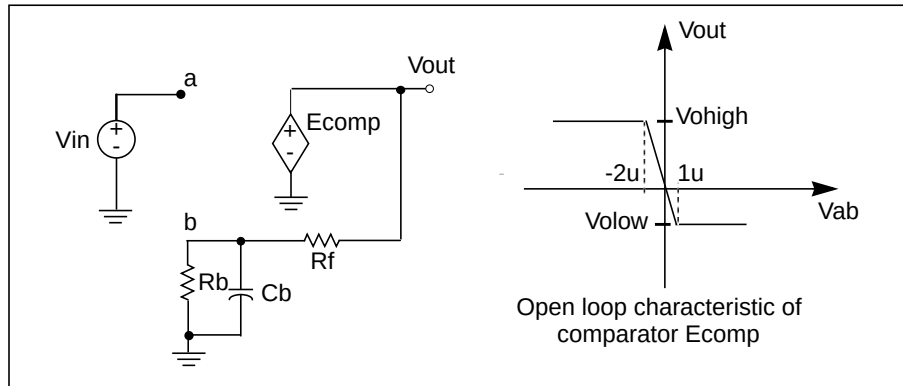
```
.subckt opout in out
eo out1 0 in 0 1
ro out1 out 75
vdum out dum 0
h dum 0 pwl(1) vdum delta=.01ma -.1ma, -15v .1ma, 15v
.ends
* op-amp subckt
.subckt opamp in+ in- out
xin in+ in- out1 opin
xrc out1 out2 oprc
xout out2 out opout
.ends
.end
```

Figure 17-25: AC Analysis Response**Figure 17-26: Transient Analysis Response**

Inverting Comparator with Hysteresis

A piecewise linear VCVS models an inverting comparator.

Figure 17-27: Inverting Comparator with Hysteresis



Two reference voltages correspond to the *volow* and *vohigh* voltages of Ecomp:

$$V_{reflow} = \frac{V_{olow} \cdot R_b}{R_b + R_f}$$

$$V_{refhigh} = \frac{V_{ohigh} \cdot R_b}{R_b + R_f}$$

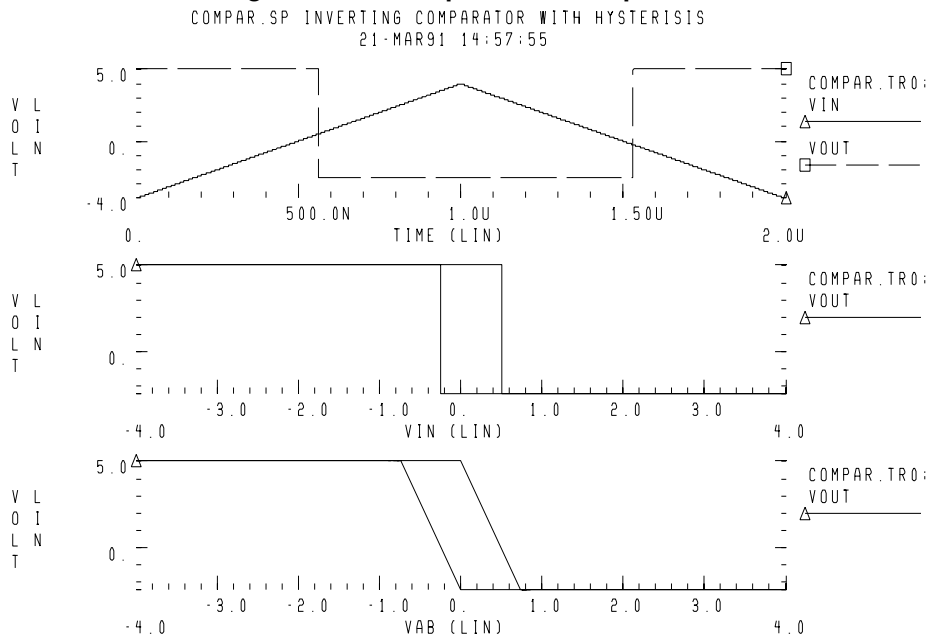
When V_{in} exceeds $V_{refhigh}$, the V_{out} output changes to V_{olow} . For V_{in} values less than V_{reflow} , the output changes to V_{ohigh} .

The following is an example:

```

Compar.sp Inverting comparator with hysteresis
.OPTION POST PROBE
.PARAM vohigh=5v volow=-2.5v rbval=1k rfval=9k
Ecomp out 0 PWL(1) a b -2u,vohigh 1u,volow
Rb b 0 rbval
Rf b out rfval
Cb b 0 1ff
Vin a 0 PWL(0, -4 1u, 4 2u, -4)
.TRAN .1n 2u
.PROBE Vin=V(a) Vab=V(a,b) Vout=V(out)
.END

```

Figure 17-28: Response of Comparator

Voltage-Controlled Oscillator (VCO)

In this example, a one-input NAND (functioning as an inverter) models a five-stage ring oscillator. PWL capacitance switches the load capacitance of this inverter, from 1pF to 3 pF. As the simulation results indicate, the oscillation frequency decreases, as the load capacitance increases.

Example

```
vcob.sp voltage controlled oscillator using
+ pwl functions
.OPTION POST
.GLOBAL ctrl
.TRAN 1n 100n
.IC V(in)=0 V(out1)=5
.PROBE TRAN V(in) V(out1) V(out2) V(out3) V(out4)
X1 in out1 inv
X2 out1 out2 inv
X3 out2 out3 inv
X4 out3 out4 inv
X5 out4 in inv
Vctrl ctrl 0 PWL(0,0 35n,0 40n,5)
```

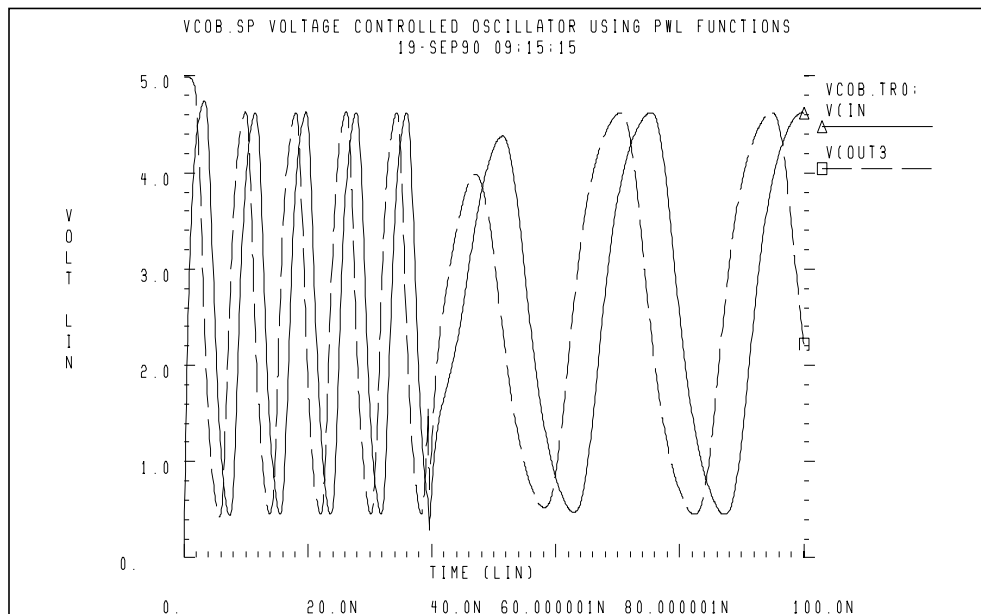
Subcircuit Definition

```
.SUBCKT inv in out rout=1k
* The following G Element is functioning as PWL
* capacitance.
Gcout out 0 VCCAP PWL(1) ctrl 0 DELTA=.01
+ 4.5 1p
+ 4.6 3p

Rout out 0 rout
Gn 0 out NAND(1) in 0 SCALE='1.0k/rout'
+ 0. 5.00ma
+ 0.25 4.95ma
+ 0.5 4.85ma
+ 1.0 4.75ma
+ 1.5 4.42ma
+ 3.5 1.00ma
+ 4.000 0.50ma
+ 4.5 0.20ma
+ 5.0 0.05ma
.ENDS inv
*
```

.END

Figure 17-29: Voltage Controlled Oscillator Response



LC Oscillator

The initial capacitor charge is 5 volts. The value of capacitance is the function of voltage, at node 10. The capacitance value becomes four times higher, at the t2 time. The following equation calculates the frequency of this LC circuit:

$$\text{freq} = \frac{1}{6.28 \cdot \sqrt{L \cdot C}}$$

At the t2 time, the frequency must be halved. The amplitude of oscillation depends on the condition of the circuit, when the capacitance value changes.

The stored energy is:

$$E = (0.5 \cdot C \cdot V^2) + (0.5 \cdot L \cdot I^2)$$

$$E = 0.5 \cdot C \cdot V_m^2, I = 0$$

$$E = 0.5 \cdot L \cdot I_m^2, V = 0$$

At the t2 time, when $V=0$, if C changes to $A \cdot C$, then:

$$0.5 \cdot L \cdot I_m^2 = 0.5 \cdot V_m^2 = 0.5 \cdot (A \cdot C) \cdot V_m'^2$$

and from the above equation:

$$V_m' = \frac{V_m}{\sqrt{A}}$$

$$Q_m' = \sqrt{A} \cdot V_m$$

The second condition that Star-Hspice considers is when $V=V_{in}$, if C changes to $A \cdot C$, then:

$$Q_m = Q_m'$$

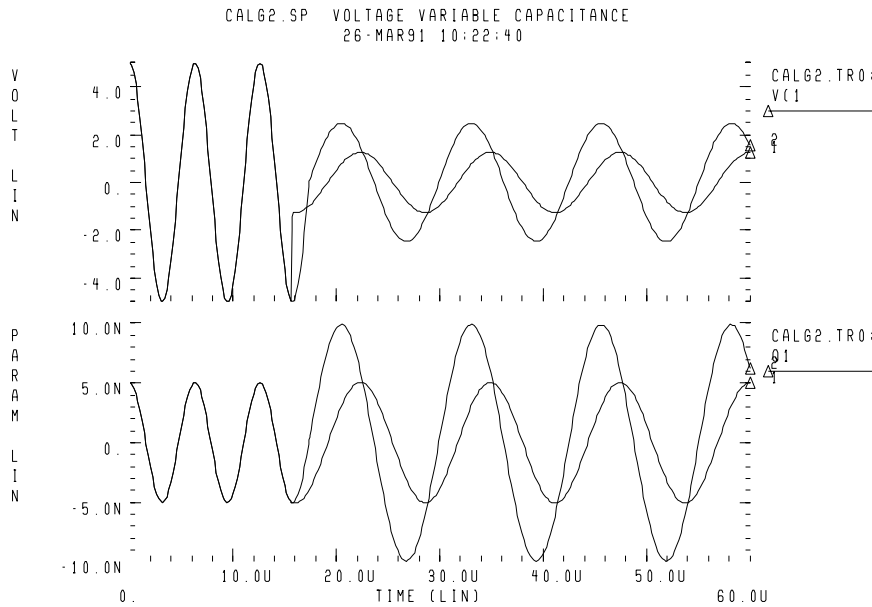
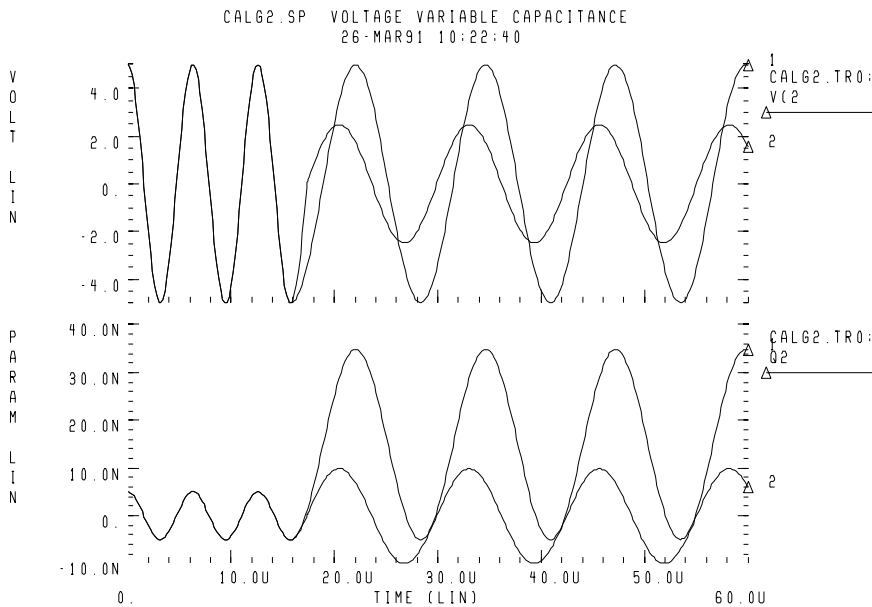
$$C \cdot V_m = A \cdot C \cdot V_m'$$

$$V_m' = \frac{V_m}{A}$$

Therefore, Star-Hspice modifies the voltage amplitude, between $V_m/\sqrt{A}$ and V_m/A , depending on the circuit condition when the circuit switches. This example tests the CTYPE 0 and 1 results. The result for CTYPE=1 must be correct, because capacitance is a function of voltage at node 10, not a function of the voltage across the capacitor itself.

The following is an example:

```
calg2.sp voltage variable capacitance
*
.OPTION POST
.IC v(1)=5 v(2)=5
C1 1 0 C='1e-9*v(10)' CTYPE=1
L1 1 0 1m
*
C2 2 0 C='1e-9*v(10)' CTYPE=0
L2 2 0 1m
*
V10 10 0 PWL(0sec,1v t1,1v t2,4v)
R10 10 0 1
*
.TRAN .1u 60u UIC SWEEP DATA=par
.MEAS TRAN period1 TRIG V(1) VAL=0 RISE=1
+ TARG V(1) VAL=0 RISE=2
.MEAS TRAN period2 TRIG V(1) VAL=0 RISE=5
+ TARG V(1) VAL=0 RISE=6
.PROBE TRAN V(1) q1=LX0(C1)
*
.PROBE TRAN V(2) q2=LX0(C2)
.DATA par t1 t2
15.65us 15.80us
17.30us 17.45us
.ENDDATA
.END
```

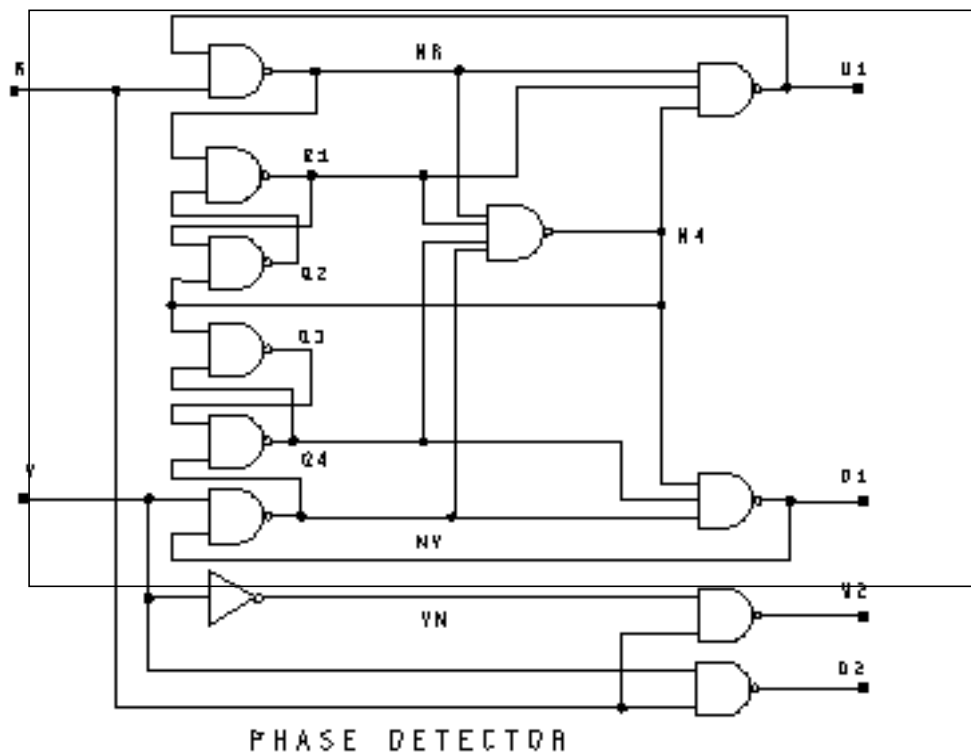
Figure 17-30: Correct Result Corresponding to CTYPE=1**Figure 17-31: Incorrect Result Corresponding to CTYPE=0**

Phase Locked Loops (PLL)

Phase Detector, with Multi-Input NAND Gates

This circuit uses behavioral elements, to implement the inverters, with 2, 3, and 4 input NAND gates.

Figure 17-32: Circuit Schematic of Phase Detector



Example

```

pdb.sp phase detector using behavioral nand gates.
.option post=2
.tran .25n 50ns
*.graph tran v(r) v(v) v(u1)
*.graph tran v(r) v(v) v(u2) $ v(d2)
.probe tran v(r) v(v) v(u1)
.probe tran v(r) v(v) v(u2) $ v(d2)
xnr r u1 nr nand2 capout=.1p
xq1 nr q2 q1 nand2 capout=.1p
xq2 q1 n4 q2 nand2
xq3 q4 n4 q3 nand2
xq4 q3 nv q4 nand2
xnv v d1 nv nand2
xu1 nr q1 n4 u1 nand3
xd1 nv q4 n4 d1 nand3
xvn v vn inv
xu2 vn r u2 nand2
xd2 r v d2 nand2
xn4 nr q1 q4 nv n4 nand4
*
* waveform vv lags waveform vr
vr r 0 pulse(0,5,0n,1n,1n,15n,30n)
vv v 0 pulse(0,5,5n,1n,1n,15n,30n)
*
* waveform vr lags waveform vv
*vr r 0 pulse(0,5,5n,1n,1n,15n,30n)
*vv v 0 pulse(0,5,0n,1n,1n,15n,30n)

```

Subcircuit Definitions

```

.SUBCKT inv in out capout=.1p
cout out 0 capout
rout out 0 1.0k
gn 0 out nand(1) in 0 scale=1
+ 0. 4.90ma
+ 0.25 4.88ma
+ 0.5 4.85ma
+ 1.0 4.75ma
+ 1.5 4.42ma
+ 3.5 1.00ma
+ 4.5 0.2ma
+ 5.0 0.1ma
.ENDS inv

```



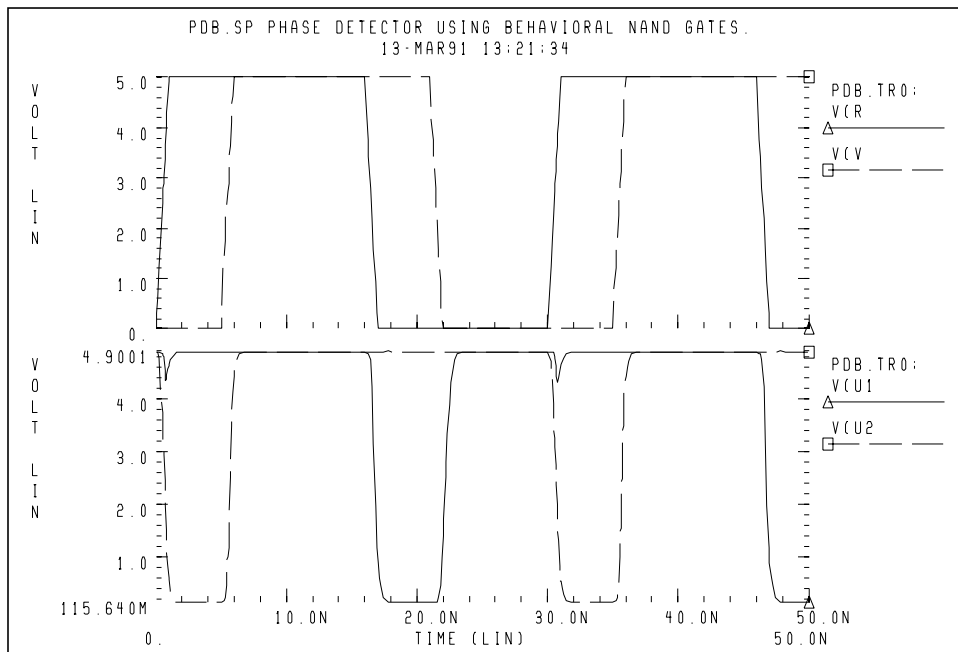
```

.SUBCKT nand2 in1 in2 out capout=.15p
cout out 0 capout
rout out 0 1.0k
gn 0 out nand(2) in1 0 in2 0 scale=1
+ 0. 4.90ma
+ 0.25 4.88ma
+ 0.5 4.85ma
+ 1.0 4.75ma
+ 1.5 4.42ma
+ 3.5 1.00ma
+ 4.000 0.50ma
+ 4.5 0.2ma
+ 5.0 0.1ma
.ENDS nand2

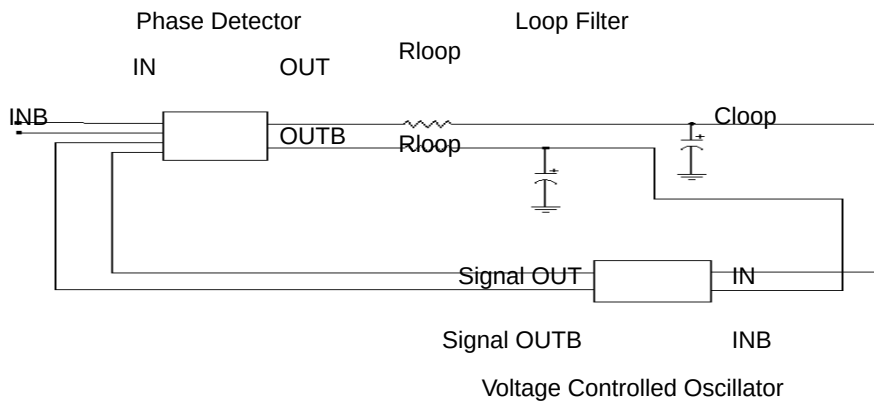
.SUBCKT nand3 in1 in2 in3 out capout=.2p
cout out 0 capout
rout out 0 1.0k
gn 0 out nand(3) in1 0 in2 0 in3 0 scale=1
+ 0. 4.90ma
+ 0.25 4.88ma
+ 0.5 4.85ma
+ 1.0 4.75ma
+ 1.5 4.42ma
+ 3.5 1.00ma
+ 4.000 0.50ma
+ 4.5 0.2ma
+ 5.0 0.1ma
.ENDS nand3

.SUBCKT nand4 in1 in2 in3 in4 out capout=.5p
cout out 0 capout
rout out 0 1.0k
gn 0 out nand(4) in1 0 in2 0 in3 0 in4 0 scale=1
+ 0. 4.90ma
+ 0.25 4.88ma
+ 0.5 4.85ma
+ 1.0 4.75ma
+ 1.5 4.42ma
+ 3.5 1.00ma
+ 4.000 0.50ma
+ 4.5 0.2ma
+ 5.0 0.1ma
.ENDS nand4
.end

```

Figure 17-33: Phase Detector Response

PLL BJT Behavioral Modeling

Figure 17-34: PLL Schematic

A Phase Locked Loop (PLL) circuit synchronizes to an input waveform, within a selected frequency range. This returns an output voltage that is proportional to variations in the input frequency. It has three basic components:

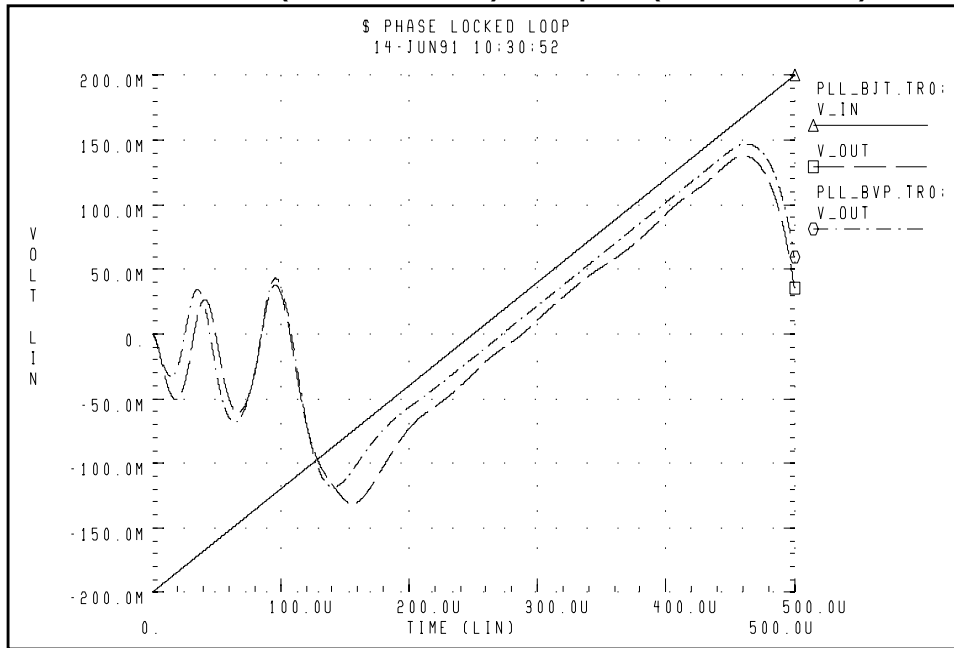
- A voltage-controlled oscillator (VCO), which returns an output waveform that is proportional to its input voltage.
- A phase detector, which compares the VCO output to the input waveform, and returns an output voltage, depending on their phase difference.
- A loop filter, which filters the phase detector voltage. It returns an output voltage, which forms the VCO input (and the external voltage output) of the PLL.

The following example shows a Star-Hspice simulation for a full bipolar implementation of a PLL.

1. Its transfer function shows a linear region of voltage vs. (periodic) time, which is, the *lock* range.
2. Star-Hspice behaviorally models the phase detector, which effectively implements a logical XNOR function.
3. Star-Hspice then substitutes this model into the full PLL circuit, and simulates again.
4. Star-Hspice then substitutes the behavioral model for the VCO, into the PLL circuit, and simulates this behavioral PLL.

The results of the transient simulations ([Figure 17-35](#)) show minimal difference between implementations. However, run time statistics show that the behavioral model reduces simulation time, to one-fifth that of the full circuit.

If you use this PLL in a larger system simulation (for example, an AM tracking system), include the behavioral model. This model substantially reduces simulation run time, and still accurately represents the subcircuit.

Figure 17-35: Behavioral (PLL_BVP Curve) vs. Bipolar (PLL_BJT Curve) Simulation

Example

This is an example of a phase locked loop:

```
$ phase locked loop
.option post probe acct
.option relv=1e-5
$
$ wideband FM example, Grebene gives:
$ f0=1meg kf=250kHz/V
$ kd=0.1 V/rad
$ R=10K C=1000p
$ f_lock = kf*kd*pi/2 = 39kHz, v_lock = kd*pi/2 = 0.157
$ f_capture/f_lock ~= 1/sqrt(2*pi*R*C*f_lock)
$ = 0.63, v_capture ~= 0.100
*.ic v(out)=0 v(fin)=0
.tran .2u 500u
.option delmax=0.01u interp
.probe v_in=v(inc,0) v_out=v(out,outb)
.probe v(in) v(osc) v(mout) v(out)
```

Input

```

vin inc 0 pwl 0u, -0.2 500u, 0.2
*vin inc 0 0

xin inc 0 in inb vco f0=1meg kf=125k phi=0 out_off=0
+ out_amp=0.3

$ vco
xvco e eb osc oscb vco f0=1meg kf=125k phi=0
+ out_off=-1 out_amp=0.3

$ phase detector
xpd in inb osc oscb mout moutb pd kd=0.1 out_off=-2.5

$ filter
rf mout e 10k
cf e 0 1000p
rfb moutb eb 10k
cfb eb 0 1000p

$ final output
rout out e 100k
cout out 0 100p
routb outb eb 100k
coub outb 0 100p

.macro vco in inb out outb f0=100k kf=50k phi=0.0
+ out_off=0.0 out_amp=1.0

gs 0 s poly(2) c 0 in inb 0 '6.2832e-9*f0'
+ 0 0 '6.2832e-9*kf'
gc c 0 poly(2) s 0 in inb 0 '6.2832e-9*f0'
+ 0 0 '6.2832e-9*kf'
cs s 0 1e-9
cc c 0 1e-12
e1 s_clip 0 pwl(1) s 0 -0.1, -0.1 0.1, 0.1
eout 0 s_clip 0 out_off vol='10*out_amp'
eboutb 0 s_clip 0 out_off vol='-10*out_amp'
.ic v(s)='sin(phi)' v(c)='cos(phi)'
.eom

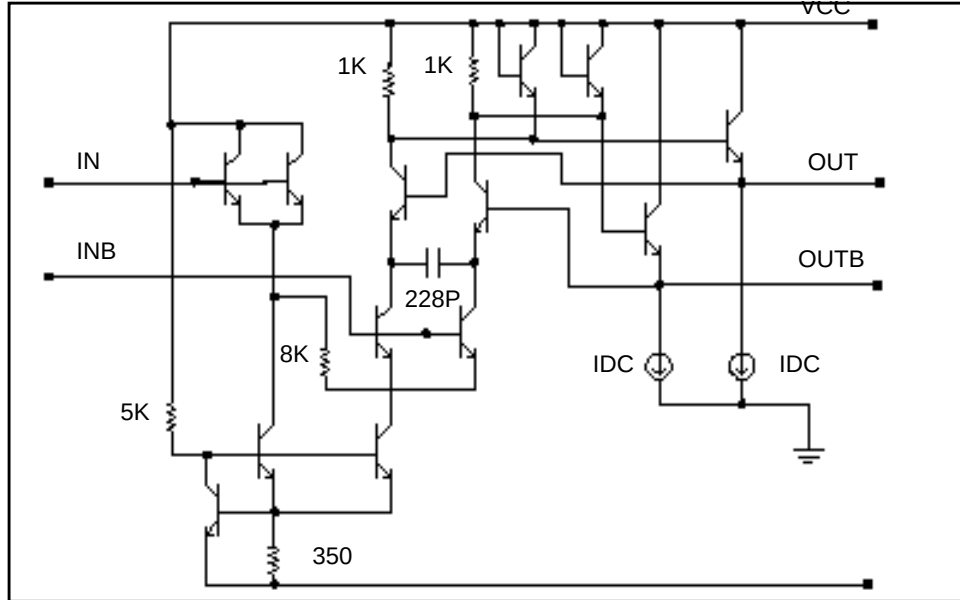
.macro pd in inb in2 in2b out outb kd=0.1 out_off=0
e1 clip1 0 pwl(1) in inb -0.1, -0.1 0.1, 0.1
e2 clip2 0 pwl(1) in2 in2b -0.1, -0.1 0.1, 0.1
e3 n1 0 poly(2) clip1 0 clip2 0 0 0 0 '78.6*kd'
e4 outb 0 n1 0 out_off 1
e5 out 0 n1 0 out_off -1
.eom
.end

```

VCO Example

This is an example of a BJT LEVEL, Voltage Controlled Oscillator (VCO):

```
$ phase locked loop
.option post probe acct
.option relv=1e-5
$
$ wideband FM example, Grebene gives:
$ f0=1meg kf=250kHz/V
$ kd=0.1 V/rad
$ R=10K C=1000p
$ f_lock = kf*kd*pi/2 = 39kHz, v_lock = kd*pi/2 = 0.157
$ f_capture/f_lock ~= 1/sqrt(2*pi*R*C*f_lock)
$ = 0.63, v_capture ~= 0.100
*.ic v(out)=0 v(fin)=0
.tran .2u 500u
.option delmax=0.01u interp
.probe v_in=v(inc,0) v_out=v(out,outb)
.probe v(in) v(osc) v(mout) v(out) v(e)
vcc vcc 0 6
vee vee 0 -6
$ input
vin inc 0 pwl 0u,-0.2 500u,0.2
xin inc 0 in inb vco f0=1meg kf=125k phi=0 out_off=0
+ out_amp=0.3
$ vco
xvco1 e eb osc oscb 0 vee vco1
.ic v(osc)=-1.4 v(oscb)=-0.7
```

Figure 17-36: Voltage Controlled Oscillator Circuit

BJT Level Phase Detector

Example

```
$ phase detector
xpd1 in inb osc oscb mout mouthb vcc vee pd1
```

Filter

```
rf mout e 10k
cf e 0 1000p
rfb mouthb eb 10k
cfb eb 0 1000p
```

Final Output

```
rout out e 100k
cout out 0 100p
routb outb eb 100k
coutb outb 0 100p
.macro vco in inb out outb f0=100k kf=50k phi=0.0
+ out_off=0.0 out_amp=1.0
gs 0 s poly(2) c 0 in inb 0 '6.2832e-9*f0'
+ 0 0 '6.2832e-9*kf'
gc c 0 poly(2) s 0 in inb 0 '6.2832e-9*f0'
+ 0 0 '6.2832e-9*kf'
cs s 0 1e-9
```

```

cc c 0 1e-9
e1 s_clip 0 pwl(1) s 0 -0.1,-0.1 0.1,0.1
e out 0 s_clip 0 out_off '10*out_amp'
eb outb 0 s_clip 0 out_off '-10*out_amp'
.ic v(s)='sin(phi)' v(c)='cos(phi)'
.eom
.macro pd in inb in2 in2b out outb kd=0.1 out_off=0
e1 clip1 0 pwl(1) in inb -0.1,-0.1 0.1,0.1
e2 clip2 0 pwl(1) in2 in2b -0.1,-0.1 0.1,0.1
e3 n1 0 poly(2) clip1 0 clip2 0 0 0 0 '78.6*kd'
e4 outb 0 n1 0 out_off 1
e5 out 0 n1 0 out_off -1
.eom
.macro vco1 in inb e7 e8 vcc vee vco_cap=228.5p
qout vcc vcc b7 npn1
qoutb vcc vcc b8 npn1
rb vcc c0 5k $ 1ma
q0 c0 b0 vee npn1
q7 vcc b7 e7 npn1
r4 vcc b7 1k
i7 e7 0 1m
q8 vcc b8 e8 npn1
r5 vcc b8 1k
i8 e8 0 1m
q9 b7 e8 e9 npn1
q10 b8 e7 e10 npn1
c0 e9 e10 vco_cap
q11 e9 in 2 npn1 $ ic=i0
q12 e10 in 2 npn1 $ ic=i0
q15 2 c0 b0 npn1 $ ic=2*i0
q16 3 c0 b0 npn1 $ ic=2*i0
rx 2 3 8k
q13 vcc inb 3 npn1
q14 vcc inb 3 npn1
rt b0 vee 350 $ i=4*i0=2m
.eom
.model npn1 npn
+ eg=1.1 af=1 xcjc=0.95 subs=1
+ cjs=0 tf=5p
+ tr=500p cje=0.2p cjc=0.2p fc=0.8
+ vje=0.8 vjc=0.8 mje=0.33 mjc=0.33
+ rb=0 rbm=0 irb=10u
+ is=5e-15 ise=1.5e-14 isc=0
+ vaf=150 bf=100 ikf=20m
+ var=30 br=5 ikr=15m
+ rc=0 re=0
+ nf=1 ne=1.5 nc=1.2
+ tbf1=8e-03

```

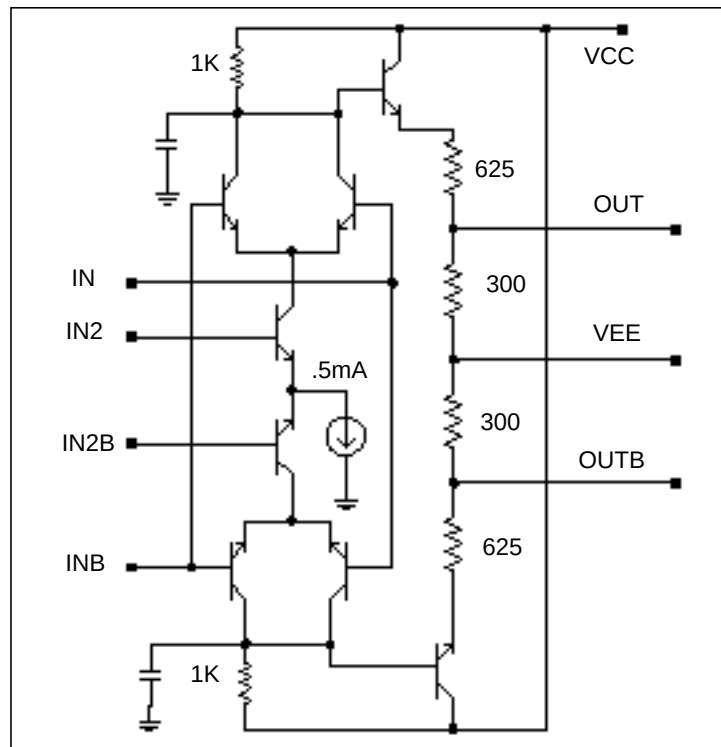


```

.macro pd1 in inb in2 in2b out outb vcc vee
r1 vcc n1 1k
rlb vcc n1b 1k
q3 n1 in c1 npn1
q4 n1b inb c1 npn1
q5 n1 inb c2 npn1
q6 n1b in c2 npn1
q1 c1 in2 e npn1
q2 c2 in2b e npn1
ie e 0 0.5m
c1 n1 0 1p
c1b n1b 0 1p
q7 vcc n1 e7 npn1
q8 vcc n1b e8 npn1
r1 e7 out 625
r2 out vee 300
r1b e8 outb 625
r2b outb vee 300
.eom
.end

```

Figure 17-37: Phase Detector Circuit



References

1. Chua & Lin. *Computer Aided Analysis of Electronic Circuits*. Englewood Cliffs: Prentice-Hall, 1975, page 117. See also “SPICE2 Application Notes for Dependent Sources,” by Bert Epler, *IEEE Circuits & Devices Magazine*, September 1987.



Chapter 18

Pole/Zero Analysis

You can use pole/zero analysis in Star-Hspice, to study the behavior of linear, time-invariant networks. You can apply the results to the design of analog circuits, such as amplifiers and filters. You can use pole/zero analysis to determine the stability of a design, or to calculate the poles and zeroes to specify in a POLE statement (see [Using Pole/Zero Analysis on page 18-3](#)).

Pole/zero analysis uses the .PZ statement, instead of pole/zero (POLE function) and Laplace (LAPLACE function) transfer function modeling, which are also described in [Using Pole/Zero Analysis on page 18-3](#).

This chapter explains the following topics:

- [Overview of Pole/Zero Analysis](#)
- [Using Pole/Zero Analysis](#)
- [Pole/Zero Analysis Examples](#)
- [References](#)

Overview of Pole/Zero Analysis

In pole/zero analysis, a network transfer function describes a network. For any linear time-invariant network, you can use this general form to write the function:

$$H(s) = \frac{N(s)}{D(s)} = \frac{a_0 s^m + a_1 \cdot s^{(m-1)} + \dots + a_m}{b_0 s^n + b_1 \cdot s^{(n-1)} + \dots + b_n}$$

In the factorized form, the general function is:

$$H(s) = \frac{a_0}{b_0} \cdot \frac{(s + z_1)(s + z_2) \dots (s + z_i) \dots (s + z_m)}{(s + p_1)(s + p_2) \dots (s + p_j) \dots (s + p_m)}$$

- The roots of the numerator $N(s)$ (that is, z_i) are the *zeros* of the network function.
- The roots of the denominator $D(s)$ (that is, p_j) are the *poles* of the network function.
- S is a complex frequency¹.

The dynamic behavior of the network depends on the location of the poles and zeros, on the network function curve (complex plane). The (real) poles are the natural frequencies of the network. You can graphically deduce the magnitude and phase curve of most network functions, from the location of its poles and zeros ([see 2 on page 18-22](#)).

[References on page 18-22](#), lists a variety of source material that address:

- Transfer functions of physical systems³.
- Design of systems and physical modeling ([see 4 on page 18-22](#)).
- Interconnect transfer function modeling ([see 5 on page 18-22](#)).

Using Pole/Zero Analysis

Star-Hspice uses only the Muller method ([see 7 on page 18-22](#)), to calculate the roots of the $N(s)$ and $D(s)$ polynomials.

Muller Method

You can apply the Muller method if the network contains frequency-dependent elements (such as S or W elements).

The Muller method approximates the polynomial, using a quadratic equation that fits through three points in the vicinity of a root. To obtain successive iterations toward a particular root, Star-Hspice finds the nearer root of a quadratic, whose curve passes through the last three points.

In Muller's method, selecting the three initial points affects both the convergence of the process, and the accuracy of the roots obtained.

1. If the poles or zeros occupy a wide frequency range, then choose ($X0R$, $X0I$) close to the origin, to find poles or zeros at the zero frequency first.
2. Find the remaining poles or zeros, in increasing order.

The ($X1R$, $X1I$) and ($X2R$, $X2I$) values can be orders of magnitude larger than ($X0R$, $X0I$). If any poles or zeros occur at high frequencies, adjust $X1I$ and $X2I$ accordingly.

Pole/zero analysis results are based on the circuit's DC operating point, so the operating point solution must be accurate. Use the `.NODESET` statement (not `.IC`) for initialization, to avoid DC convergence problems.

.PZ (Pole/Zero) Statement

The syntax is:

`.PZ output input`

PZ	Invokes the pole/zero analysis.
input	Input source, which can be the name of any independent voltage or current source.
output	Output variables, which can be: ■ Any node voltage, $V(n)$. ■ Any branch current, $I(branch_name)$.

Examples

```
.PZ    V(10)    VIN
.PZ    I(RL)    ISRC
.PZ    I1(M1)   VSRC
.pz    V(10)    VIN
```

...means voltage at node 10 as output and VIN source as input.

```
.pz    I(r20)   ISRC
```

...means branch current at r20 as output and ISRC source as input.

Pole/Zero Control Options

<i>CSCAL</i>	Sets the capacitance scale. Star-Hspice multiplies capacitances by CSCAL. Default=1.0e+12.
<i>FMAX</i>	Sets the maximum frequency value of angular velocity, for poles and zeros. Default=1.0e+12 rad/sec.
<i>FSCAL</i>	Sets the frequency scale. Star-Hspice multiplies the frequency by FSCAL. Default=1e-9.
<i>GSCAL</i>	Sets the conductance scale. Star-Hspice multiplies the conductance, and divides the resistance, by GSCAL. Default=1e+3.

<i>ITLPZ</i>	Sets the iteration limit for pole/zero analysis. Default=100.
<i>LSCAL</i>	Sets the inductance scale. Star-Hspice multiplies inductances by LSCAL. Default=1e+6.

Note: Scale factors must satisfy the following relations.

$$GSCAL = CSCAL \cdot FSCAL$$

$$GSCAL = \frac{1}{LSCAL \cdot FSCAL}$$

If you change scale factors, you might need to modify the initial Muller points, (X0R, X0I), (X1R, X1I) and (X2R, X2I), even though the program internally multiplies the initial values by (1.0e-9/GSCAL).

<i>PZABS</i>	Sets absolute tolerances for poles and zeros. This option affects low-frequency poles or zeros. Use this option as follows:
--------------	---

$$\text{If } (|X_{\text{real}}| + |X_{\text{imag}}| < PZABS),$$

$$\text{then } X_{\text{real}} = 0 \text{ and } X_{\text{imag}} = 0.$$

You can also use this option for convergence tests.
Default=1.0e-2.

<i>PZTOL</i>	Sets the relative error tolerance, for poles or zeros. Default=1.0e-6.
--------------	---

<i>RITOL</i>	Sets the minimum ratio value for the (real/imaginary) or (imaginary/real) parts of the poles or zeros. Default=1.0e-6. Use RITOL as follows:
--------------	--

$$\text{If } |X_{\text{imag}}| \leq RITOL \cdot |X_{\text{real}}|, \text{ then } X_{\text{imag}} = 0$$

$$\text{If } |X_{\text{real}}| \leq RITOL \cdot |X_{\text{imag}}|, \text{ then } X_{\text{real}} = 0$$

(X0R,X0I) The three complex starting-trial points, in the Muller
(x1R,X1I) algorithm, for pole/zero analysis. Defaults:
(X2R,X2I) X0R=-1.23456e6 X0I=0.0
 X1R=1.23456e5 X1I=0.0
 X2R=+1.23456e6 X2I=0.0
Star-Hspice multiplies these initial points, and FMAX, by
FSCAL.

.PZ Prerequisites

The .PZ statement requires the operating points. You can include the .OP statement in the netlist. If you do not, the simulator automatically invokes an operating point calculation.

Pole/Zero Analysis Examples

Example 1 – Low-Pass Filter

The following is a Star-Hspice input file, for a fifth-order low-pass prototype filter, used in pole/zero and AC analysis ([see 8 on page 18-22](#)). This file is in the \$installdir/demo/hspice/filters/flp5th.sp directory.

```
*FILE: FLP5TH.SP
5TH-ORDER LOW_PASS FILTER
****
* T = I(R2) / IIN
*   = 0.113*(S**2 + 1.6543)*(S**2 + 0.2632) /
*       (S**5 + 0.9206*S**4 + 1.26123*S**3 +
*       0.74556*S**2 + 0.2705*S + 0.09836)
*****
.OPTION POST
.PZ I(R2) IN
.AC DEC 100 .001HZ 10HZ
.PLOT AC IDB(R2) IP(R2)
IN 0 1 1.00 AC 1
R1  1 0 1.0
C3  1 0 1.52
C4  2 0 1.50
C5  3 0 0.83
C1  1 2 0.93
L1  1 2 0.65
C2  2 3 3.80
L2  2 3 1.00
R2  3 0 1.00
.END
```

Figure 18-1: Low-Pass Prototype Filter

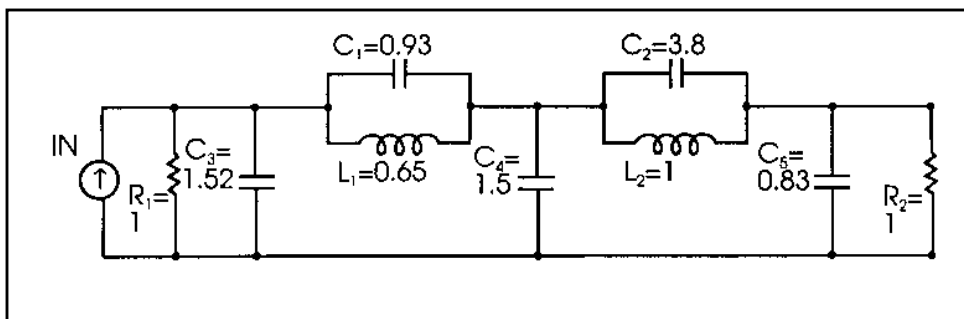


Table 18-1 shows the magnitude and phase variation of the current output, resulting from AC analysis. These results are consistent with pole/zero analysis. The pole/zero unit is radians per second, or hertz. The X-axis unit, in the plot, is in hertz.

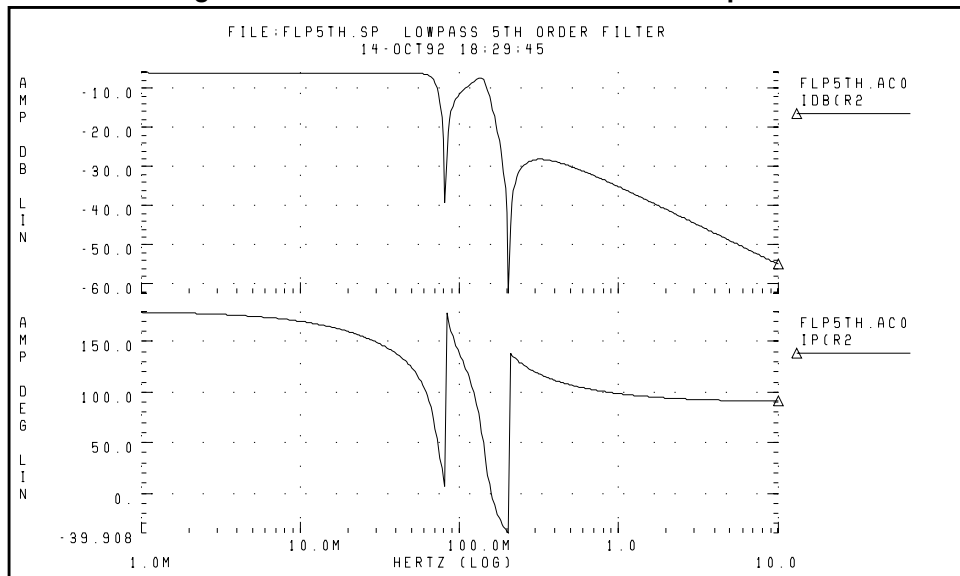
Table 18-1: Pole/Zero Analysis Results for Low-Pass Filter

Poles (rad/sec)		Poles (hertz)	
Real	Imag	Real	Imag
-6.948473e-02	-4.671778e-01	-1.105884e-02	-7.435365e-02
-6.948473e-02	4.671778e-01	-1.105884e-02	7.435365e-02
-1.182742e-01	-8.914907e-01	-1.882392e-02	-1.418852e-01
-1.182742e-01	8.914907e-01	-1.882392e-02	1.418852e-01
-5.450890e-01	0.000000e+00	-8.675361e-02	0.000000e+00

Zeros (rad/sec)		Zeros (hertz)	
Real	Imag	Real	Imag
0.000000e+00	-1.286180e+00	0.000000e+00	-2.047019e-01
0.000000e+00	-5.129892e-01	0.000000e+00	-8.164476e-02

Table 18-1: Pole/Zero Analysis Results for Low-Pass Filter (Continued)

0.000000e+00	5.129892e-01	0.000000e+00	8.164476e-02
0.000000e+00	1.286180e+00	0.000000e+00	2.047019e-01
Constant Factor = 1.129524e-01			

Figure 18-2: Fifth-Order Low-Pass Filter Response

Example 2 – Kerwin's Circuit

This example is a Star-Hspice input file, for pole/zero analysis of Kerwin's circuit (see 9 on page 18-22). Table 18-2 lists analysis results.

```
*FILE: $installdir/demo/hspice/filters/FKERWIN.SP
KERWIN'S CIRCUIT   HAVING JW-AXIS TRANSMISSION ZEROS.
**
* T = V(5) / VIN
*   = 1.2146 (S**2 + 2) / (S**2 + 0.1*S + 1)
* POLES = (-0.05004, +0.9987), (-0.05004, -0.9987)
* ZEROS = (0.0, +1.4142), (0.0, -1.4142)
*****
.PZ V(5) VIN
VIN 1 0 1
C1  1 2 0.7071
C2  2 4 0.7071
C3  3 0 1.4142
C4  4 0 0.3536
R1  1 3 1.0
R2  3 4 1.0
R3  2 5 0.5
E1  5 0 4 0 2.4293
.END
```

Figure 18-3: Design Example for Kerwin's Circuit

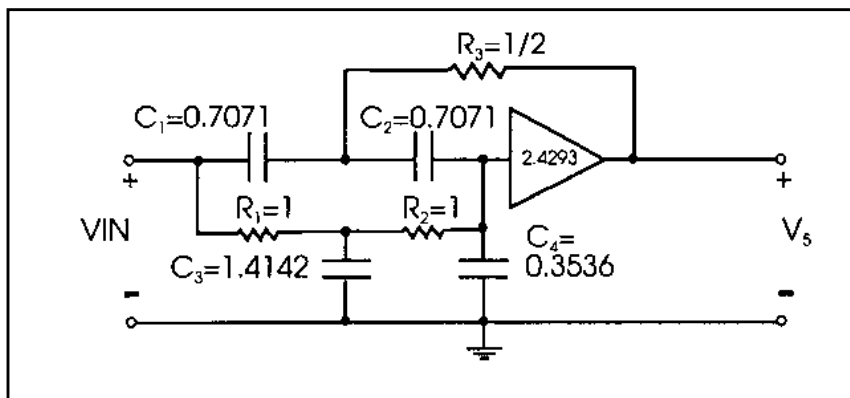


Table 18-2: Pole/Zero Analysis Results for Kerwin's Circuit

Poles (rad/sec)		Poles (hertz)	
Real	Imag	Real	Imag
-5.003939e-02	9.987214e-01	-7.964016e-03	1.589515e-01
-5.003939e-02	-9.987214e-01	-7.964016e-03	-1.589515e-01
-1.414227e+00	0.000000e+00	-2.250812e-01	0.000000e+00
Zeros (rad/sec)		Zeros (hertz)	
Real	Imag	Real	Imag
0.000000e+00	-1.414227e+00	0.000000e+00	-2.250812e-01
0.000000e+00	1.414227e+00	0.000000e+00	2.250812e-01
-1.414227e+00	0.000000e+00	-2.250812e-01	0.000000e+00
Constant Factor = 1.214564e+00			

Example 3 – High-Pass Butterworth Filter

This example is a Star-Hspice input file, for pole/zero analysis of a fourth-order high-pass Butterworth filter (see 10 on page 18-22). This file is in `$install_dir/demo/hspice/filters/fhp4th.sp`. Table 18-3 on page 18-12 shows the analysis results.

```
*FILE: FHP4TH.SP
*****
* T = V(10) / VIN
* = (S**4) / ((S**2 + 0.7653*S + 1) * (S**2
* + 1.8477*S + 1))
*
* POLES, (-0.38265, +0.923895), (-0.38265, -0.923895)
*          (-0.9239, +0.3827), (-0.9239, -0.3827)
* ZEROS, FOUR ZEROS AT (0.0, 0.0)
*****
```

```

.OPTION ITLPZ=200
.PZ    V(10)    VIN
VIN 1 0 1
C1    1 2 1
C2    2 3 1
R1    3 0 2.613
R2    2 4 0.3826
E1    4 0 3 0 1
C3    4 5 1
C4    5 6 1
R3    6 0 1.0825
R4    5 10 0.9238
E2    10 0 6 0 1
RL    10 0 1E20
.END

```

Figure 18-4: Fourth-Order High-Pass Butterworth Filter

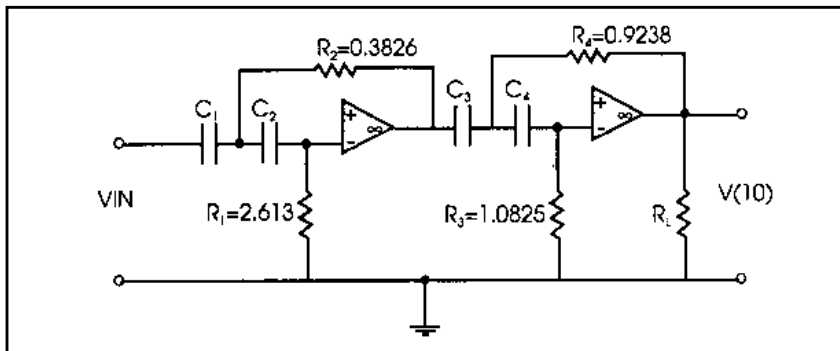


Table 18-3: Pole/Zero Analysis Results for High-Pass Butterworth Filter

Poles (rad/sec)		Poles (hertz)	
Real	Imag	Real	Imag
-3.827019e-01	-9.240160e-01	-6.090889e-02	1.470617e-01
-3.827019e-01	9.240160e-01	-6.090890e-02	-1.470617e-01
-9.237875e-01	3.828878e-01	-1.470254e-01	6.093849e-02
-9.237875e-01	-3.828878e-01	-1.470254e-01	-6.093849e-02

Table 18-3: Pole/Zero Analysis Results for High-Pass Butterworth Filter (Continued)

Poles (rad/sec)		Poles (hertz)	
Zeros (rad/sec)		Zeros (hertz)	
Real	Imag	Real	Imag
0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00
0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00
0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00
0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00
Constant Factor = 1.000000e+00			

Example 4 – CMOS Differential Amplifier

This example is a Star-Hspice input file, for pole/zero and AC analysis of a CMOS differential amplifier. The file is in `$installdir/demo/hspice/apps/mcdiff.sp`. [Table 18-4](#) shows the analysis results.

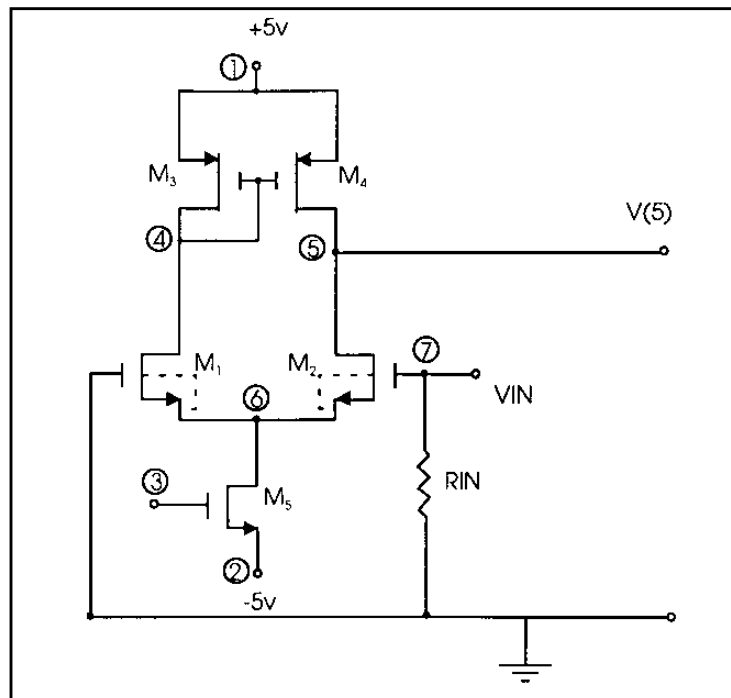
```

FILE: MCDIFF.SP
CMOS DIFFERENTIAL AMPLIFIER
.OPTION PIVOT SCALE=1E-6 SCALM=1E-6 WL
.PZ V(5) VIN
VIN 7 0 0 AC 1
.AC DEC 10 20K 500MEG
.PRINT AC VDB(5) VP(5)
M1 4 0 6 6 MN 100 10 2 2
M2 5 7 6 6 MN 100 10 2 2
M3 4 4 1 1 MP 60 10 1.5 1.5
M4 5 4 1 1 MP 60 10 1.5 1.5
M5 6 3 2 2 MN 50 10 1.0 1.0
VDD 1 0 5
VSS 2 0 -5
VGG 3 0 -3
RIN 7 0 1
.MODEL MN NMOS LEVEL=5 VT=1 UB=700 FRC=0.05 DNB=1.6E16
+ XJ=1.2 LATD=0.7 CJ=0.13 PHI=1.2 TCV=0.003 TOX=800
$
.MODEL MP PMOS LEVEL=5 VT=-1 UB=245 FRC=0.25 TOX=800
+ DNB=1.3E15 XJ=1.2 LATD=0.9 CJ=0.09 PHI=0.5 TCV=0.002
.END

```

Table 18-4: Pole/Zero Analysis Results for CMOS Differential Amplifier

Poles (rad/sec)		Poles (hertz)	
Real	Imag	Real	Imag
-1.798766e+06	0.000000e+00	-2.862825e+05	0.000000e+00
-1.126313e+08	-6.822910e+07	-1.792583e+07	-1.085900e+07
-1.126313e+08	6.822910e+07	-1.792583e+07	1.085900e+07
Zeros (rad/sec)		Zeros (hertz)	
Real	Imag	Real	Imag
-1.315386e+08	7.679633e+07	-2.093502e+07	1.222251e+07
-1.315386e+08	-7.679633e+07	-2.093502e+07	-1.222251e+07
7.999613e+08	0.000000e+00	1.273178e+08	0.000000e+00
Constant Factor = 3.103553e-01			

Figure 18-5: CMOS Differential Amplifier**Example 5 – Simple Amplifier**

This example is a Star-Hspice input file, for pole/zero analysis of an equivalent circuit, for a simple amplifier with:

- $R_S = R_{PI} = R_L = 1000$ ohms.
- $g_m = 0.04$ mho.
- $C_{MU} = 1.0 \times 10^{-11}$ farad.
- $C_{PI} = 1.0 \times 10^{-9}$ farad (see 11 on page 18-22).

The file is in `$installdir/demo/hspice/apps/ampg.sp`. Table 18-5 on page 18-16 shows the analysis results.

```

FILE: AMPG.SP
A SIMPLE AMPLIFIER.
* T = V(3) / VIN
* T = 1.0D6*(S - 4.0D9) / (S**2 + 1.43D8*S + 2.0D14)
* POLES = (-0.14D7, 0.0), (-14.16D7, 0.0)
* ZEROS = (+4.00D9, 0.0)
.PZ V(3) VIN
RS 1 2 1K
RPI 2 0 1K
RL 3 0 1K
GMU 3 0 2 0 0.04
CPI 2 0 1NF
CMU 2 3 10PF
VIN 1 0 1
.END

```

Figure 18-6: Simple Amplifier

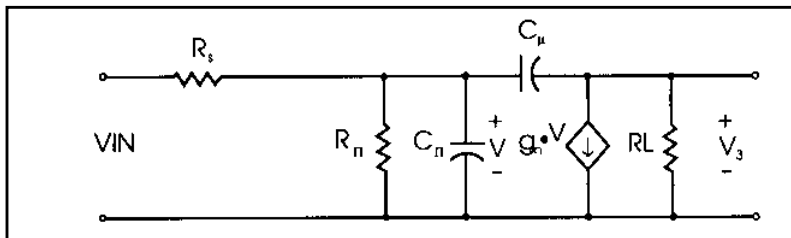


Table 18-5: Pole/Zero Analysis Results for Amplifier

Poles (rad/sec)		Poles (hertz)	
Real	Imag	Real	Imag
-1.412555+06	0.000000e+00	-2.248151e+05	0.000000e+00
-1.415874+08	0.000000e+00	-2.253434e+07	0.000000e+00
Zeros (rad/sec)		Zeros (hertz)	
Real	Imag	Real	Imag
4.000000e+09	0.000000e+00	6.366198e+08	0.000000e+00
Constant Factor = 1.000000e+06			

Example 6— Active Low-Pass Filter

This example is a Star-Hspice input file, for pole/zero analysis of an active ninth-order low-pass filter ([see 12 on page 18-22](#)), using the ideal op-amp element. This example performs an AC analysis. The file is in \$installdir/demo/hspice/filters/flp9th.sp. [Table 18-6 on page 18-19](#) is the analysis results.

```

FILE: FLP9TH.SP
VIN IN 0 AC 1
.PZ V(OUT) VIN
.AC DEC 50 .1K 100K
.OPTION POST DCSTEP=1E3 X0R=-1.23456E+3 X1R=-1.23456E+2
+ X2R=1.23456E+3 FSCAL=1E-6 GSCAL=1E3 CSCAL=1E9 LSCAL=1E3
.PLOT AC VDB(OUT)
.SUBCKT OPAMP IN+ IN- OUT GM1=2 RI=1K CI=26.6U GM2=1.33333
RL=75
RII IN+ IN- 2MEG
RI1 IN+ 0 500MEG
RI2 IN- 0 500MEG
G1 1 0 IN+ IN- GM1
C1 1 0 CI
R1 1 0 RI
G2 OUT 0 1 0 GM2
RLD OUT 0 RL
.ENDS

.SUBCKT FDNR 1 R1=2K C1=12N R4=4.5K
RLX=75
R1 1 2 R1
C1 2 3 C1
R2 3 4 3.3K
R3 4 5 3.3K
R4 5 6 R4
C2 6 0 10N
XOP1 2 4 5 OPAMP
XOP2 6 4 3 OPAMP
.ENDS

RS IN 1 5.4779K
R12 1 2 4.44K
R23 2 3 3.2201K
R34 3 4 3.63678K
R45 4 OUT 1.2201K
C5 OUT 0 10N
X1 1 FDNR R1=2.0076K C1=12N R4=4.5898K
X2 2 FDNR R1=5.9999K C1=6.8N R4=4.25725K
X3 3 FDNR R1=5.88327K C1=4.7N R4=5.62599K
X4 4 FDNR R1=1.0301K C1=6.8N R4=5.808498K
.END

```

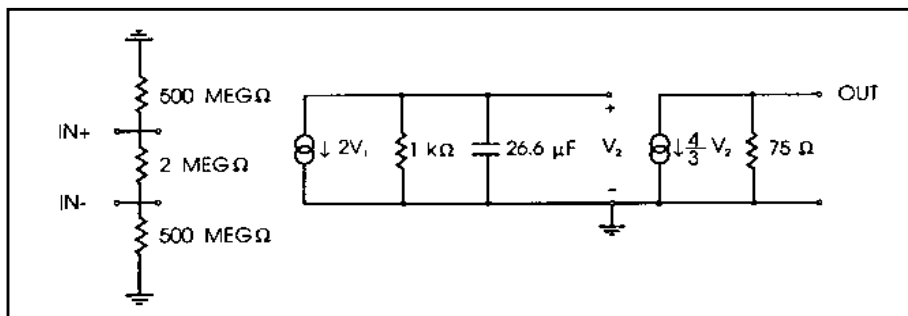
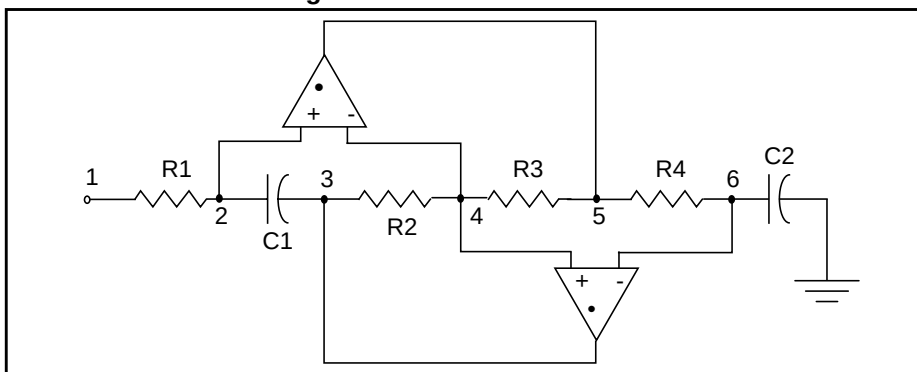
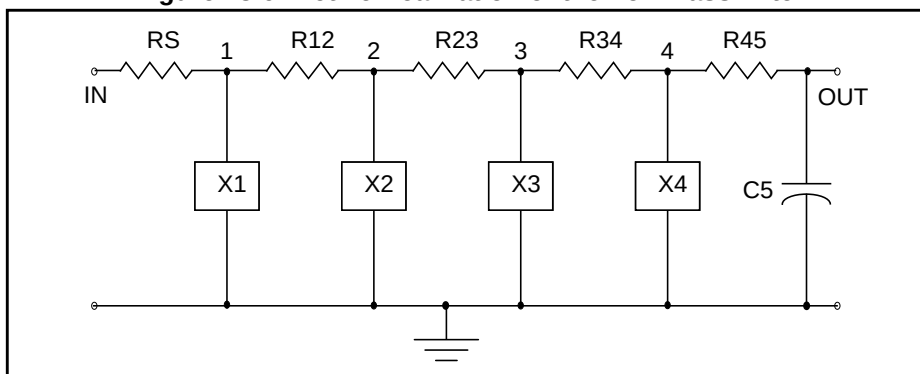
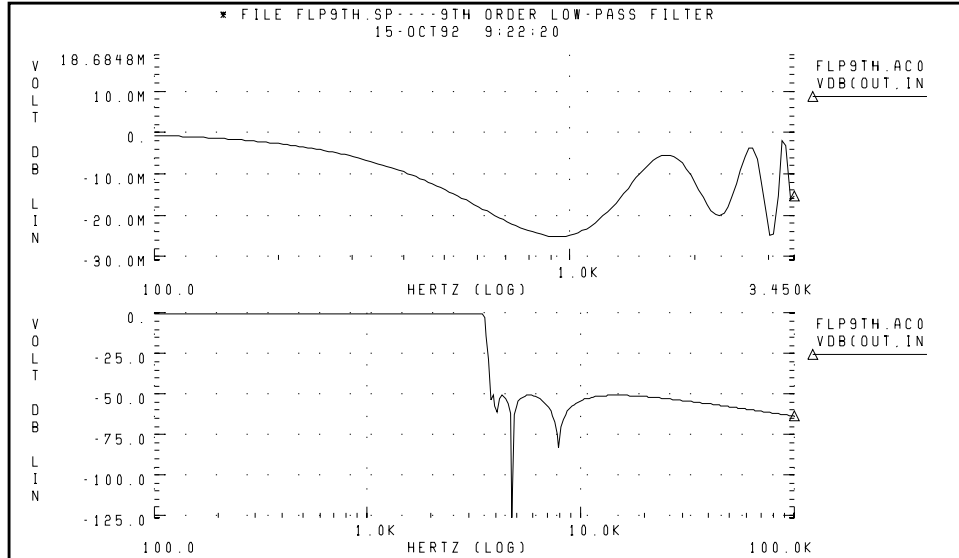
Figure 18-7: Linear Model of the 741C Op-Amp**Figure 18-8: FDNR Subcircuit****Figure 18-9: Active Realization of the Low-Pass Filter**

Table 18-6: Pole/Zero Analysis Results for the Active Low-Pass Filter (Sheet 1 of 2)

Poles (rad/sec)		Poles (hertz)	
Real	Imag	Real	Imag
-4.505616e+02	-2.210451e+04	-7.170911e+01	-3.518042e+03
-4.505616e+02	2.210451e+04	-7.170911e+01	3.518042e+03
-1.835284e+03	2.148369e+04	-2.920944e+02	3.419236e+03
-1.835284e+03	-2.148369e+04	-2.920944e+02	-3.419236e+03
-4.580172e+03	1.944579e+04	-7.289571e+02	3.094894e+03
-4.580172e+03	-1.944579e+04	-7.289571e+02	-3.094894e+03
-9.701962e+03	1.304893e+04	-1.544115e+03	2.076802e+03
-9.701962e+03	-1.304893e+04	-1.544115e+03	-2.076802e+03
-1.353908e+04	0.000000e+00	-2.154811e+03	0.000000e+00
-3.668995e+06	-3.669793e+06	-5.839386e+05	-5.840657e+05
-3.668995e+06	3.669793e+06	-5.839386e+05	5.840657e+05
-3.676439e+06	-3.676184e+06	-5.851234e+05	-5.850828e+05
-3.676439e+06	3.676184e+06	-5.851234e+05	5.850828e+05
-3.687870e+06	3.687391e+06	-5.869428e+05	5.868665e+05
-3.687870e+06	-3.687391e+06	-5.869428e+05	-5.868665e+05
-3.695817e+06	-3.695434e+06	-5.882075e+05	-5.881466e+05
-3.695817e+06	+3.695434e+06	-5.882075e+05	5.881466e+05

Table 18-6: Pole/Zero Analysis Results for the Active Low-Pass Filter (Sheet 2 of 2)

Zeroes (rad/sec)		Zeroes (hertz)	
Real	Imag	Real	Imag
-3.220467e-02	-2.516970e+04	-5.125532e-03	-4.005882e+03
-3.220467e-02	2.516970e+04	-5.125533e-03	4.005882e+03
2.524420e-01	-2.383956e+04	4.017739e-02	-3.794184e+03
2.524420e-01	2.383956e+04	4.017739e-02	3.794184e+03
1.637164e+00	2.981593e+04	2.605627e-01	4.745353e+03
1.637164e+00	-2.981593e+04	2.605627e-01	-4.745353e+03
4.888484e+00	4.852376e+04	7.780265e-01	7.722796e+03
4.888484e+00	-4.852376e+04	7.780265e-01	-7.722796e+03
-3.641366e+06	-3.642634e+06	-5.795413e+05	-5.797432e+05
-3.641366e+06	3.642634e+06	-5.795413e+05	5.797432e+05
-3.649508e+06	-3.649610e+06	-5.808372e+05	-5.808535e+05
-3.649508e+06	3.649610e+06	-5.808372e+05	5.808535e+05
-3.683700e+06	3.683412e+06	-5.862790e+05	5.862333e+05
-3.683700e+06	-3.683412e+06	-5.862790e+05	-5.862333e+05
-3.693882e+06	3.693739e+06	5.878995e+05	5.878768e+05
-3.693882e+06	-3.693739e+06	-5.878995e+05	-5.878768e+05
Constant Factor = 4.451586e+02			

Figure 18-10: 9th Order Low-Pass Filter Response

The top graph in [Figure 18-10](#) plots the bandpass response of the Pole/Zero 6 low-pass filter. The bottom graph shows overall response of the low-pass filter.

References

1. Desoer, Charles A. and Kuh, Ernest S. *Basic Circuit Theory*. New York: McGraw-Hill.1969. Chapter 15.
2. Van Valkenburg, M. E. *Network Analysis*. Englewood Cliffs, New Jersey: Prentice Hall, Inc., 1974, chapters 10 & 13.
3. R.H. Canon, Jr. *Dynamics of Physical Systems*. New York: McGraw-Hill, 1967. This text describes electrical, mechanical, pneumatic, hydraulic, and mixed systems.
4. B.C. Kuo. *Automatic Control Systems*. Englewood Cliffs, New Jersey: Prentice-Hall, 1975. This source discusses control system design, and provides background material about physical modeling.
5. L.T. Pillage, and R.A. Rohrer. "Asymptotic Waveform Evaluation for Timing Analysis", *IEEE Trans CAD*. Apr. 1990, pp. 352 - 366. This paper is a good references about interconnect transfer function modeling, and discusses extracting transfer functions for timing analysis.
6. S. Lin, and E.S. Kuh. "Transient Simulation of Lossy Interconnects Based on the Recursive Convolution Formulation", *IEEE Trans CAS*. Nov. 1992, pp. 879 - 892. This paper is another source for how to model interconnect transfer functions.
7. Muller, D. E., *A Method for Solving Algebraic Equations Using a Computer, Mathematical Tables, and Other Aids to Computation (MTAC)*. 1956, Vol. 10,. pp. 208-215.
8. Temes, Gabor C. and Mitra, Sanjit K. *Modern Filter Theory And Design*. J. Wiley, 1973, page 74.
9. Temes, Gabor C. and Lapatra, Jack W. *Circuit Synthesis And Design*, McGraw-Hill. 1977, page 301, example 7-6.
10. Temes, Gabor C. and Mitra, Sanjit K., *Modern Filter Theory And Design*. J. Wiley, 1973, page 348, example 8-3.
11. Desoer, Charles A. and Kuh, Ernest S. *Basic Circuit Theory*. McGraw-Hill, 1969, page 613, example 3.
12. Vlach, Jiri and Singhal, Kishore. *Computer Methods For Circuit Analysis and Design*. Van Nostrand Reinhold Co., 1983, pages 142, 494-496.



Chapter 19

FFT Spectrum Analysis

Spectrum analysis represents a time-domain signal, within the frequency domain. It most commonly uses the Fourier transform. A Discrete Fourier Transform (DFT) uses sequences of time values to determine the frequency content of analog signals, in circuit simulation. The Fast Fourier Transform (FFT) calculates the DFT, which Star-Hspice uses for spectrum analysis.

The `.FFT` statement in Star-Hspice uses the internal time point values.

This chapter explains the following topics:

- [Using Windows in FFT Analysis](#)
- [Using the `.FFT` Statement](#)
- [Examining the FFT Output](#)
- [AM Modulation](#)
- [Balanced Modulator and Demodulator](#)
- [Signal Detection Test Circuit](#)

By default, FFT uses a second-order interpolation to obtain waveform samples, based on the number of points that you specify.

`.OPTION FFT_ACCURATE`, or `.OPTION ACCURATE` (turns on `FFT_ACCURATE`), forces Star-Hspice to dynamically adjust the time step, so that each FFT point is a real simulation point. This eliminates interpolation error, and provides the highest FFT accuracy, with minimal overhead in simulation time.

You can use windowing functions to reduce the effects of waveform truncation on the spectral content. You can also use the `.FFT` command to specify:

- output format
- frequency
- number of harmonics
- total harmonic distortion (THD)

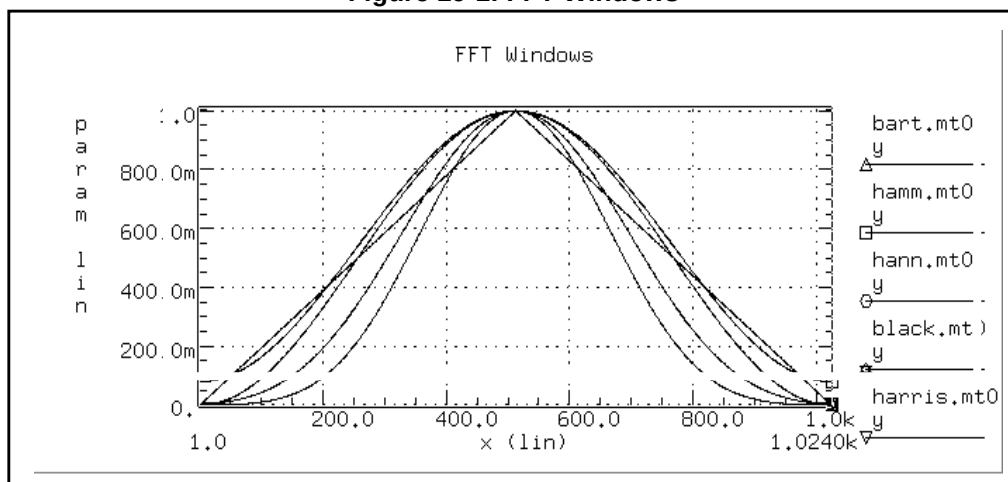
Using Windows in FFT Analysis

One problem with spectrum analysis in circuit simulators is that the duration of the signals is finite, although adjustable. Applying the FFT method to finite-duration sequences can produce inadequate results. This occurs because DFT assumes periodic extensions, causing *spectral leakage*.

The effect occurs when the finite duration of the signal does not result in a sequence that contains a whole number of periods. This is especially true when you use FFT to detect or estimate signals – that is, to detect weak signals in the presence of strong signals, or to resolve a cluster of equal-strength frequencies.

In FFT analysis, *windows* are frequency-weighting functions, applied to the time-domain data, to reduce the spectral leakage associated with finite-duration time signals. Windows are smoothing functions, which peak in the middle frequencies, and decrease to zero at the edges. Windows reduce the effects of discontinuities, as a result of finite duration. [Figure 19-1](#) shows the windows available in Star-Hspice. [Table 19-1 on page 19-3](#) lists the common performance parameters, for FFT windows available in Star-Hspice.

Figure 19-1: FFT Windows



The most important parameters in [Table 19-1 on page 19-3](#) are:

- Highest side-lobe level (to reduce bias, the lower the better).
- Worst-case processing loss (to increase detectability, the lower the better).

Table 19-1: Window Weighting Characteristics in FFT Analysis

Window	Equation	Highest Side-Lobe (dB)	Side-Lobe Roll-Off (dB/octave)	3.0-dB Bandwidth (1.0/T)	Worst Case Process Loss (dB)
Rectangular	$W(n)=1,$ $0 \leq n < NP^{\dagger}$	-13	-6	0.89	3.92
Bartlett	$W(n)=2n/(NP-1),$ $0 \leq n \leq (NP/2)-1$ $W(n)=2-2n/(NP-1),$ $NP/2 \leq n < NP$	-27	-12	1.28	3.07
Hanning	$W(n)=0.5-0.5[\cos(2\pi n/(NP-1))],$ $0 \leq n < NP$	-32	-18	1.44	3.18
Hamming	$W(n)=0.54-0.46[\cos(2\pi n/(NP-1))],$ $0 \leq n < NP$	-43	-6	1.30	3.10
Blackman	$W(n)=0.42323$ $-0.49755[\cos(2\pi n/(NP-1))]$ $+0.07922\cos[\cos(4\pi n/(NP-1))],$ $0 \leq n < NP$	-58	-18	1.68	3.47
Blackman-Harris	$W(n)=0.35875$ $-0.48829[\cos(2\pi n/(NP-1))]$ $+0.14128[\cos(4\pi n/(NP-1))]$ $-0.01168[\cos(6\pi n/(NP-1))],$ $0 \leq n < NP$	-92	-6	1.90	3.85
Gaussian a=2.5 a=3.0 a=3.5	$W(n)=\exp[-0.5a^2(NP/2-1-n)^2/(NP)^2],$ $0 \leq n \leq (NP/2)-1$ $W(n)=\exp[-0.5a^2(n-NP/2)^2/(NP)^2],$ $NP/2 \leq n < NP$	-42 -55 -69	-6 -6 -6	1.33 1.55 1.79	3.14 3.40 3.73

Table 19-1: Window Weighting Characteristics in FFT Analysis (*Continued*)

Window	Equation	Highest Side-Lobe (dB)	Side-Lobe Roll-Off (dB/octave)	3.0-dB Bandwidth (1.0/T)	Worst Case Process Loss (dB)
Kaiser-Bessel	$W(n)=I_0(x_2)/I_0(x_1)$				
$a=2.0$	$x_1=pa$	-46	-6	1.43	3.20
$a=2.5$	$x_2=x_1*\sqrt{1-(2(NP/2-1-n)/NP)^2}$,	-57	-6	1.57	3.38
$a=3.0$	$0 \leq n \leq (NP/2)-1$	-69	-6	1.71	3.56
$a=3.5$	$x_2=x_1*\sqrt{1-(2(n-NP/2)/NP)^2}$, $NP/2 \leq n < NP$ I_0 is the zero-order modified Bessel function	-82	-6	0.89	3.74

[†]NP is the number of points used for the FFT analysis.

Some compromise usually is necessary, to find a suitable window filtering for each application. As a rule, window performance improves with functions of higher complexity (those listed lower in the table).

- The Kaiser window has an ALFA parameter, which adjusts the compromise between different figures of merit for the window.
- The simple rectangular window produces a simple bandpass truncation, in the classical Gibbs phenomenon.
- The Bartlett or triangular window has good processing loss, and good side-lobe roll-off, but lacks sufficient bias reduction.
- The Hanning, Hamming, Blackman, and Blackman-Harris windows use progressively more complicated cosine functions. These functions provide smooth truncation, and a wide range of side-lobe level and processing loss.
- The last two windows in the table are parameterized windows. Use these windows to adjust the side-lobe level, the 3 dB bandwidth, and the processing loss.¹

Figure 19-2 and Figure 19-3 on page 19-5 show the characteristics of two typical windows.

Figure 19-2: Bartlett Window Characteristics

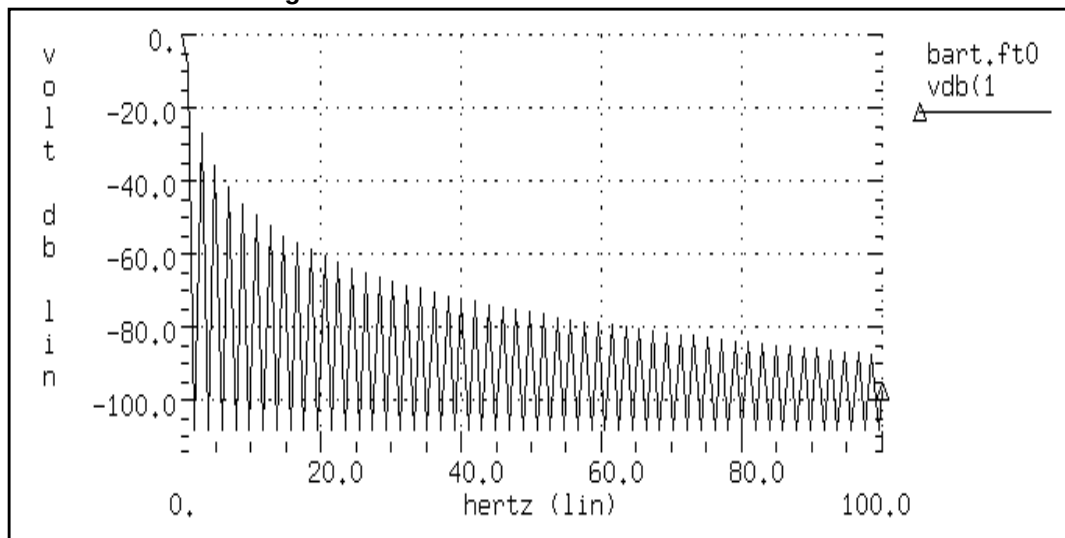
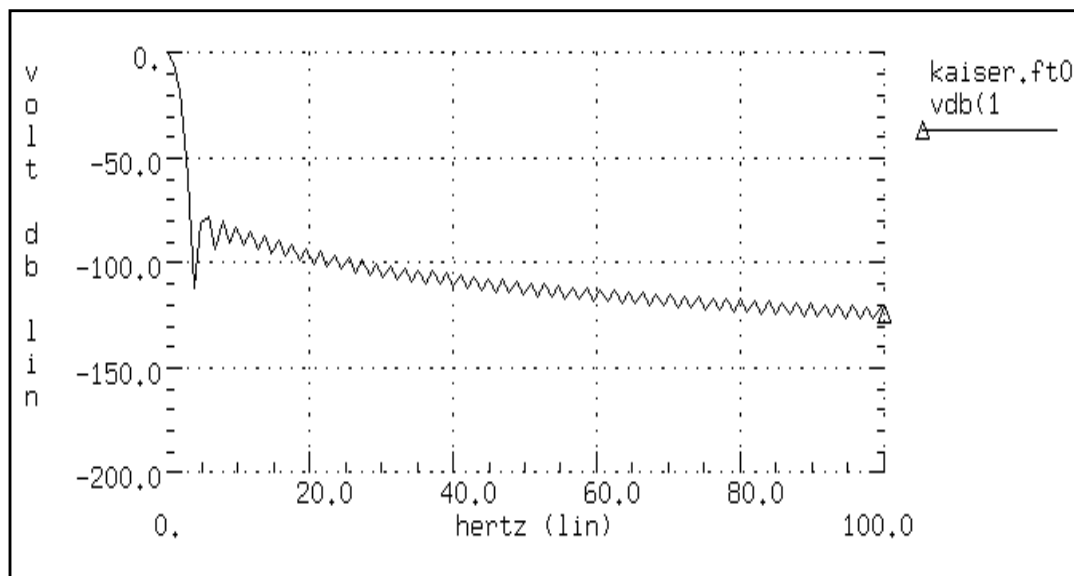


Figure 19-3: Kaiser-Bessel Window Characteristics, ALFA=3.0



Using the .FFT Statement

The general form of the .FFT statement is shown below. [Table 19-2](#) describes the parameters.

Syntax

```
.FFT <output_var> <START=value> <STOP=value> <NP=value>
+ <FORMAT=keyword> <WINDOW=keyword> <ALFA=value>
+ <FREQ=value> <FMIN=value> <FMAX=value>
```

Table 19-2: .FFT Statement Parameters

Parameter	Default	Description
output_var		Can be any valid output variable, such as voltage, current, or power.
START	see Description	Specifies the beginning of the output variable waveform to analyze. Default is the START value in the .TRAN statement, which defaults to 0 s.
FROM	see START	An alias for START in .FFT statements.
STOP	see Description	Specifies the end of the output variable waveform to analyze. Defaults to the TSTOP value in the .TRAN statement.
TO	see STOP	An alias for STOP, in .FFT statements
NP	1024	Specifies the number of points to use in the FFT analysis. NP must be a power of 2; if NP is not a power of 2, Star-Hspice automatically adjusts it to the closest higher number that is a power of 2.
FORMAT	NORM	Specifies the output format: ■ NORM= normalized magnitude ■ UNORM=unnormalized magnitude

Table 19-2: .FFT Statement Parameters (Continued)

Parameter	Default	Description
WINDOW	RECT	Specifies the window type to use: ■ RECT=simple rectangular truncation window. ■ BART=Bartlett (triangular) window. ■ HANN=Hanning window. ■ HAMM=Hamming window. ■ BLACK=Blackman window. ■ HARRIS=Blackman-Harris window. ■ GAUSS=Gaussian window. ■ KAISER=Kaiser-Bessel window.
ALFA	3.0	Specifies the parameter to use in GAUSS and KAISER windows, to control the highest side-lobe level, bandwidth, and so on. $1.0 \leq \text{ALFA} \leq 20.0$
FREQ	0.0 (Hz)	Specifies a frequency to analyze. If FREQ is non-zero, the output lists only the harmonics of this frequency, based on FMIN and FMAX. Star-Hspice also prints the THD for these harmonics.
FMIN	1.0/T (Hz)	Specifies the minimum frequency for which Star-Hspice prints FFT output into the listing file. THD calculations also use this frequency. $T = (\text{STOP} - \text{START})$
FMAX	0.5*NP*FM IN (Hz)	Specifies the maximum frequency for which Star-Hspice prints FFT output into the listing file. THD calculations also use this frequency.

Examples

The following are four examples of valid .FFT statements.

```
.fft v(1)
.fft v(1,2) np=1024 start=0.3m stop=0.5m freq=5.0k
+ window=kaiser alfa=2.5
.fft I(rload) start=0m to=2.0m fmin=100k fmax=120k
+ format=unorm
.fft par('v(1) + v(2)') from=0.2u stop=1.2u
+ window=harris
```

You can specify only one output variable in an .FFT command. The following is an incorrect use of the command.

```
.fft v(1) v(2) np=1024
```

The following example shows the correct use of the command. This example generates an .ft0 file for the FFT of *v(1)*, and an .ft1 file for the FFT of *v(2)*.

```
.fft v(1) np=1024
.fft v(2) np=1024
```


Examining the FFT Output

Star-Hspice prints the results of the FFT analysis in a tabular format, in the .lis file. These results are based on parameters in the .FFT statement. Star-Hspice prints normalized magnitude values, unless you specify `FORMAT= UNORM`, in which case it prints unnormalized magnitude values. The number of printed frequencies is half the number of points (NP) specified in the .FFT statement.

- If you use `FMIN` to specify a minimum frequency, or `FMAX` to specify a maximum frequency, Star-Hspice prints only the specified frequency range.
- If you use `FREQ` to specify a frequency, Star-Hspice outputs only the harmonics of this frequency, and the percent of total harmonic distortion.

In the sample output below, the header defines parameters in the FFT analysis.

```
***** Sample FFT output extracted from the .lis file
fft test ... sine
***** fft analysis      tnom= 25.000    temp= 25.000
***** fft components of transient response v(1)
Window:                  Rectangular
First Harmonic:          1.0000k
Start Freq:              1.0000k
Stop Freq:               10.0000k
dc component: mag(db)= -1.132D+02 mag= 2.191D-06 phase= 1.800D+02

frequency  frequency  fft_mag  fft_mag  fft_phase
index      (hz)        (db)
2          1.0000k      0.         1.0000    -3.8093m
4          2.0000k    -125.5914    525.3264n  -5.2406
6          3.0000k    -106.3740     4.8007u   -98.5448
8          4.0000k    -113.5753     2.0952u   -5.5966
10         5.0000k    -112.6689     2.3257u  -103.4041
12         6.0000k    -118.3365     1.2111u   167.2651
14         7.0000k    -109.8888     3.2030u  -100.7151
16         8.0000k    -117.4413     1.3426u   161.1255
18         9.0000k    -97.5293     13.2903u    70.0515
20        10.0000k   -114.3693     1.9122u   -12.5492

total harmonic distortion =      1.5065m percent
```

The preceding example specifies a frequency of 1 kHz, and a THD up to 10 kHz, which corresponds to the first ten harmonics.

Note: The highest frequency in the Star-Hspice FFT output might not match the specified FMAX, due to adjustments that Star-Hspice makes.

Table 19-3 describes the output of the Star-Hspice FFT analysis.

Table 19-3: .FFT Output Description

Column Heading	Description
Frequency Index	Runs from 1 to NP/2, or the corresponding index for FMIN and FMAX. The DC component, corresponding to the 0 index, displays independently.
Frequency	The actual frequency, associated with the index.
fft_mag (dB), fft_mag	■ The first FFT magnitude column is in dB. ■ The second FFT magnitude column is in units of the output variable. Star-Hspice normalizes the magnitude, unless you specify UNORM format.
fft_phase	The associated phase, in degrees.

Star-Hspice generates a .ft# file, and a listing file, for each FFT output variable. The .ft# file contains graphical data needed, to display the FFT analysis results in AvanWaves. You can display the magnitude in dB, and the phase in degrees.

Notes:

1. Use the following formula as a guideline, to specify a frequency range for FFT output:

$$\text{frequency increment} = 1.0 / (\text{STOP} - \text{START})$$

Each frequency index is a multiple of this increment. To obtain a finer frequency resolution, maximize the duration of the time window.
2. FMIN and FMAX have no effect on the .ft0, .ft1, ..., .ftn files.

AM Modulation

The example input listing, in the [Input Listing](#) section below, shows a 1 kHz carrier (FC), which a 100 Hz signal (FM) modulates. The following equation describes the voltage at node 1, which is an AM signal:

$$v(1) = sa \cdot (\text{offset} + \sin(\omega_m(\text{Time} - td))) \cdot \sin(\omega_c(\text{Time} - td))$$

You can expand the preceding equation, as follows.

$$v(1) = (sa \cdot \text{offset} \cdot \sin(\omega_c(\text{Time} - td)) + 0.5 \cdot sa \cdot \cos((\omega_c - \omega_m)(\text{Time} - td))) \\ - 0.5 \cdot sa \cdot \cos((\omega_c + \omega_m)(\text{Time} - td))$$

where

$$\omega_c = 2\pi f_c$$

$$\omega_f = 2\pi f_m$$

The preceding equations indicate that $v(1)$ is a summation of three signals, with the following frequency:

$$f_c, (f_c - f_m), \text{ and } (f_c + f_m)$$

This is the carrier frequency, and the two sidebands.

Input Listing

```
AM Modulation
.OPTION post
.PARAM sa=10 offset=1 fm=100 fc=1k td=1m
VX 1 0 AM(sa offset fm fc td)
RX 1 0 1
.TRAN 0.01m 52m
.FFT V(1) START=10m STOP=40m FMIN=833 FMAX=1.16K
.END
```

Output Listing

The relevant portion of the listing file is:

```
*****
am modulation
***** fft analysis  tnom= 25.000 temp= 25.000
***** fft components of transient response v(1)
Window:      Rectangular
Start Freq:   833.3333
Stop Freq:    1.1667k
dc component: mag(db)= -1.480D+02 mag=    3.964D-08
+ phase=     0.000D+00

frequency frequency  fft_mag      fft_mag      fft_phase
index      (hz)      (db)
25      833.3333      -129.4536      336.7584n      -113.0047
26      866.6667      -143.7912      64.6308n      45.6195
27      900.0000      -6.0206      500.0008m      35.9963
28      933.3333      -125.4909      531.4428n      112.6012
29      966.6667      -142.7650      72.7360n      -32.3152
30      1.0000k      0.      1.0000      -90.0050
31      1.0333k      -132.4062      239.7125n      -9.0718
32      1.0667k      -152.0156      25.0738n      3.4251
33      1.1000k      -6.0206      499.9989m      143.9933
34      1.1333k      -147.0134      44.5997n      -3.0046
35      1.1667k      -147.7864      40.8021n      -4.7543

***** job concluded
```

Graphical Output

Figure 19-4 and Figure 19-5 on page 19-13 display the results.

- Figure 19-4 shows the time-domain curve for node 1.
- Figure 19-5 shows the frequency-domain components, for the magnitude of node 1.

The carrier frequency is 1 kHz, with two sideband frequencies 100 Hz apart.

The third, fifth, and seventh harmonics are more than 100 dB below the fundamental, indicating excellent numerical accuracy. The time-domain data contains an integer multiple of the period, so you do not need to use windowing.

Figure 19-4: AM Modulation

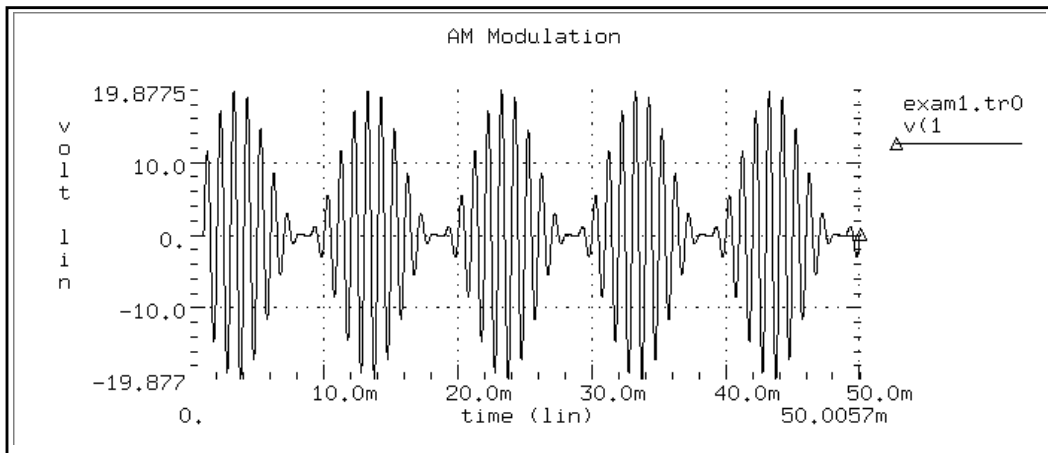
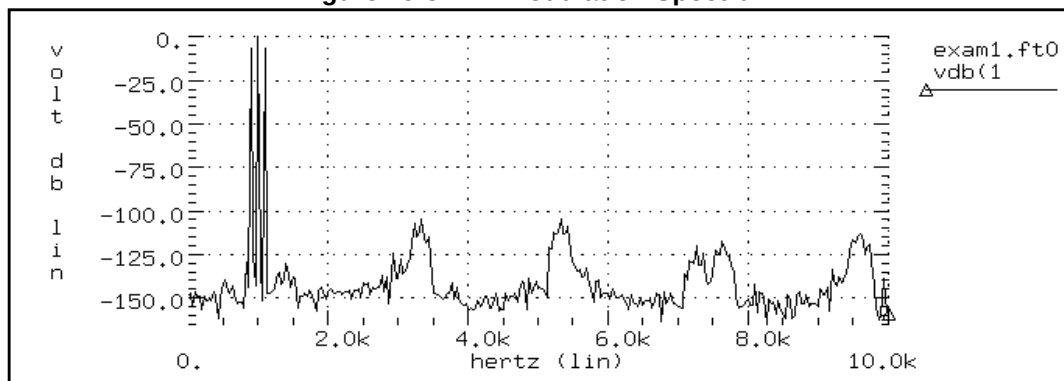


Figure 19-5: AM Modulation Spectrum



Balanced Modulator and Demodulator

Demodulation, or detection, recovers a modulating signal from the modulated output voltage. The netlist, in the [Input Listing](#) section below, shows this process. This example uses Star-Hspice behavioral models, and FFT analysis, to confirm the validity of the process, in the frequency domain.

The low-pass filter uses the Laplace element. This filter introduces some delay in the output signal, which causes *spectral leakage* if you do not use windowing in FFT. However, if you use window weighting to perform FFT, you eliminate most of the spectral leakage. The THD of the two outputs, shown in the [Output Listing on page 19-15](#), verifies this. Star-Hspice expects a 1 kHz output signal, so specify a 1 kHz frequency in the .FFT command. Also specify FMAX, to provide the first few harmonics in the output listing, for THD calculations.

Input Listing

```
Balanced Modulator & Demodulator Circuit
V1 mod1 GND sin(0 5 1K 0 0 0) $ modulating signal
r1 mod1 2 10k
r2 2 3 10k
r3 2 GND 10K

E1 3 GND OPAMP 2 GND
$ buffered output of modulating signal
V2 mod2 GND sin(0 5 10K 0 0 0) $ modulated signal
E2 modout GND vol='(v(3)*v(mod2))/10.0'
$ multiply to modulate

V3 8 GND sin(0 5 10K 0 0 0)
E3 demod GND vol='(v(modout)*v(8))/10.0'
$ multiply to demodulate
* use a laplace element for filtering
E_filter lpout 0 laplace demod 0 67.11e6 / 66.64e6
+ 6.258e3 1.0 $ filter out +the modulating signal
*

.tran 0.2u 4m
.fft v(mod1)
.fft v(mod2)
.fft v(modout)
.fft v(demod)
.fft v(lpout) freq=1.0k fmax=10k
$ ask to see the first few harmonics
```

```
.fft v(lpout) window=harris freq=1.0k fmax=10k
$ window should reduce spectral leakage
.probe tran v(mod1) v(mod2) v(modout) v(demod) v(lpout)
.option acct post probe
.end
```

Output Listing

The following portion of an output listing shows the effect of windowing, to reduce spectral leakage. This, in turn, reduces the THD.

```
balanced modulator & demodulator circuit
***** fft analysis tnom= 25.000 temp= 25.000
***** fft components of transient response v(lpout)
Window: Rectangular
First Harmonic: 1.0000k
Start Freq: 1.0000k
Stop Freq: 10.0000k

dc component: mag(db)= -3.738D+01 mag= 1.353D-02
+ phase= 1.800D+02
```

frequency index	frequency (hz)	fft_mag (db)	fft_mag	fft_phase (deg)
4	1.0000k	0.	1.0000	35.6762
8	2.0000k	-26.6737	46.3781m	122.8647
12	3.0000k	-31.4745	26.6856m	108.1100
16	4.0000k	-34.4833	18.8728m	103.6867
20	5.0000k	-36.6608	14.6880m	101.8227
24	6.0000k	-38.3737	12.0591m	100.9676
28	7.0000k	-39.7894	10.2455m	100.6167
32	8.0000k	-40.9976	8.9150m	100.5559
36	9.0000k	-42.0524	7.8955m	100.6783
40	10.0000k	-42.9888	7.0886m	100.9240

```
total harmonic distortion = 6.2269 percent
*****
balanced modulator & demodulator circuit
***** fft analysis tnom= 25.000 temp= 25.000
***** fft components of transient response v(lpout)
Window: Blackman-Harris
First Harmonic: 1.0000k
Start Freq: 1.0000k
Stop Freq: 10.0000k

dc component: mag(db)= -8.809D+01 mag= 3.938D-05
+ phase= 1.800D+02
```

frequency index	frequency (hz)	fft_mag (db)	fft_mag	fft_phase (deg)
4	1.0000k	0.	1.0000	34.3715
8	2.0000k	-66.5109	472.5569u	-78.8512
12	3.0000k	-97.5914	13.1956u	-55.7167
16	4.0000k	-107.8004	4.0736u	-41.6389
20	5.0000k	-117.9984	1.2592u	-23.9325
24	6.0000k	-125.0965	556.1309n	33.3195
28	7.0000k	-123.6795	654.6722n	74.0461
32	8.0000k	-122.4362	755.4258n	86.5049
36	9.0000k	-122.0336	791.2570n	91.6976
40	10.0000k	-122.0388	790.7840n	94.5380
total harmonic distortion =			47.2763m percent	

Figure 19-6 through Figure 19-14 show the signals, and their spectral content. The modulated signal contains only the sum, and the difference of the carrier frequency and the modulating signal (1 kHz and 10 kHz). At the receiver end, this example recovers the carrier frequency, in the demodulated signal. This example also shows a 10 kHz frequency shift, in the above signals (to 19 kHz and 21 kHz).

A low-pass filter uses a second-order Butterworth filter, to extract the carrier frequency. A Harris window significantly improves the noise floor in the filtered output spectrum, and reduces THD in the output listing (from 9.23% to 0.047%). However, this example needs a filter with a steeper transition region, and better delay characteristics, to suppress modulating frequencies below -60 dB.

Figure 19-9 on page 19-18 is a normalized “Filtered Output Signal” waveform.

Figure 19-6: Modulating and Modulated Signals

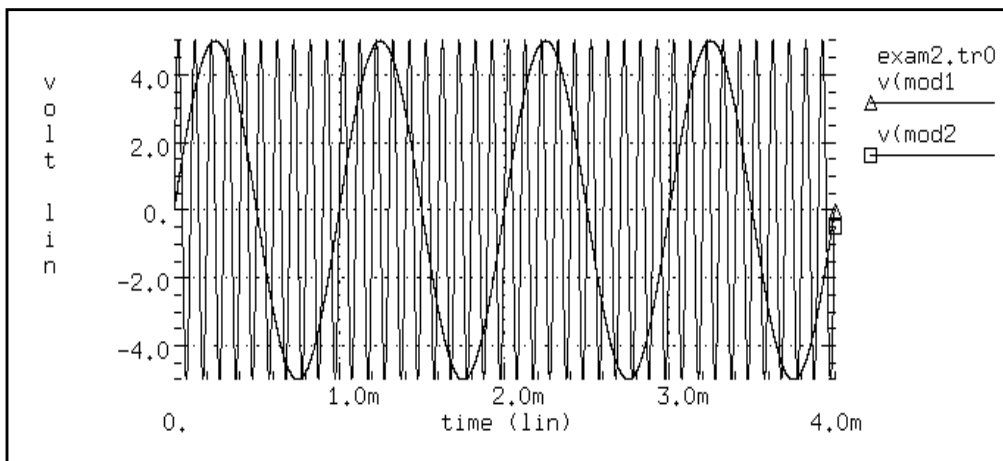


Figure 19-7: Modulated Signal

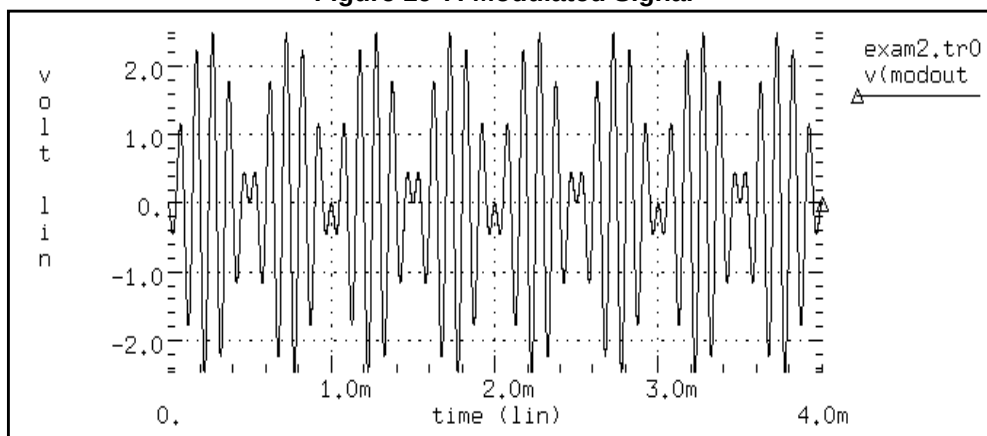


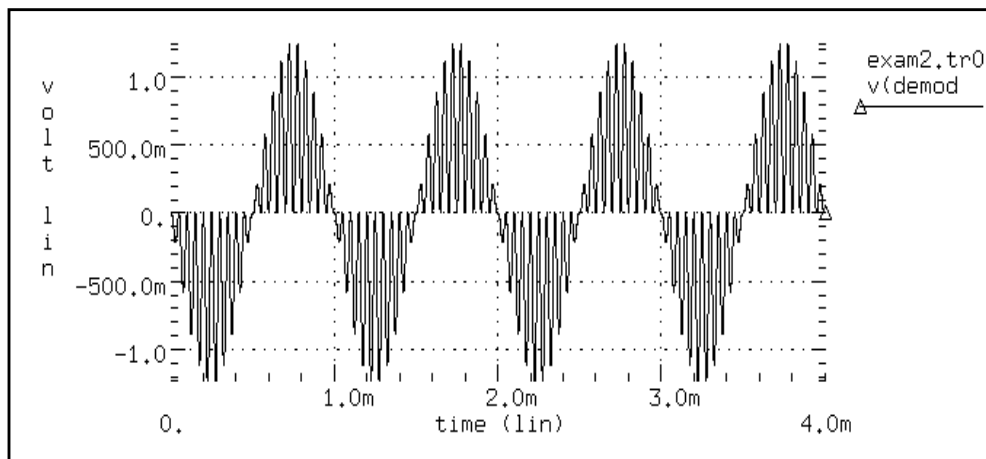
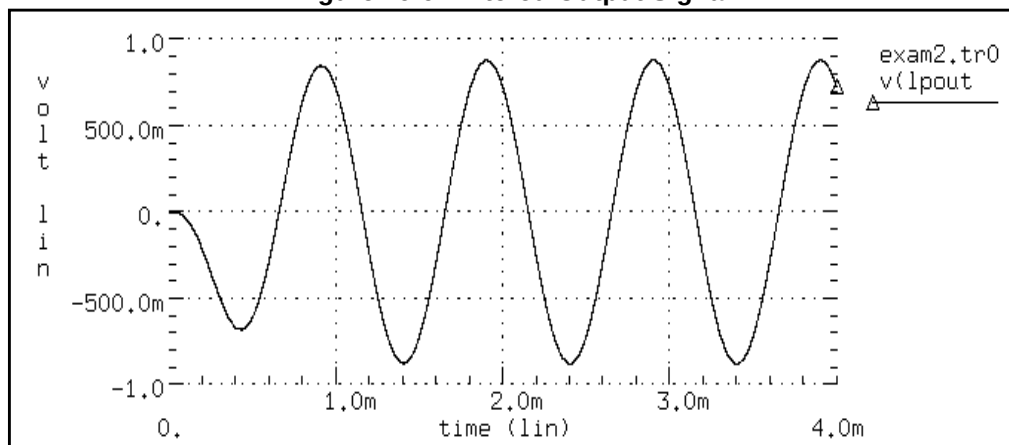
Figure 19-8: Demodulated Signal**Figure 19-9: Filtered Output Signal**

Figure 19-10: Modulating and Modulated Signal Spectrum

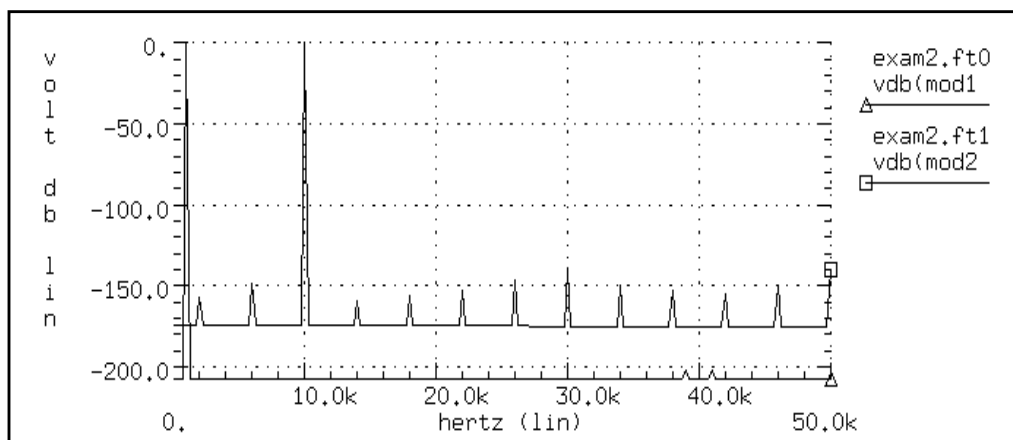


Figure 19-11: Modulated Signal Spectrum

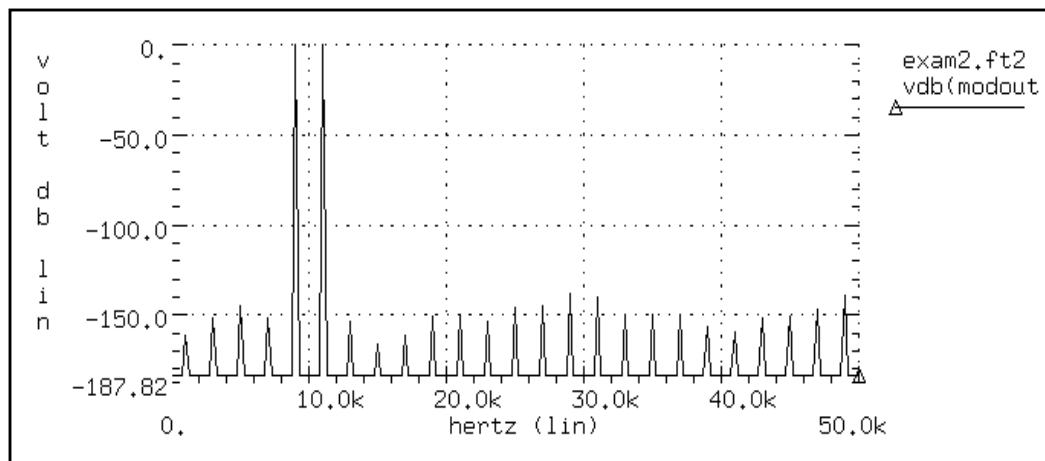


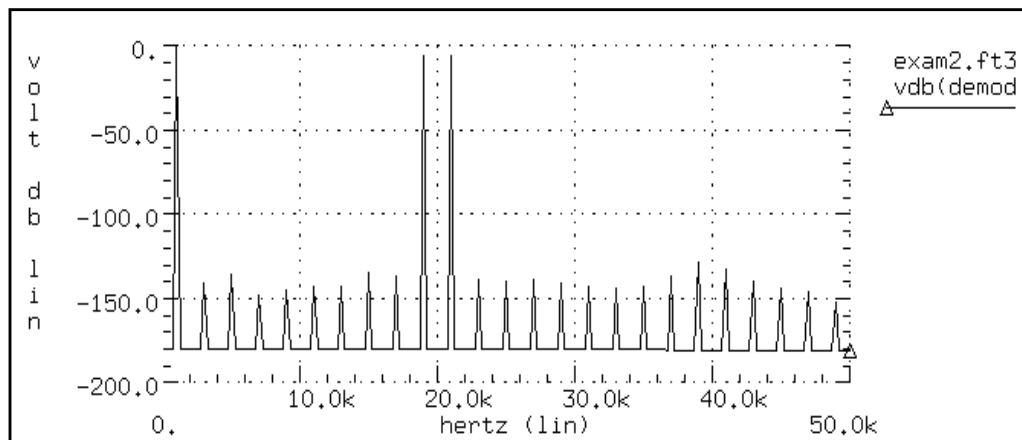
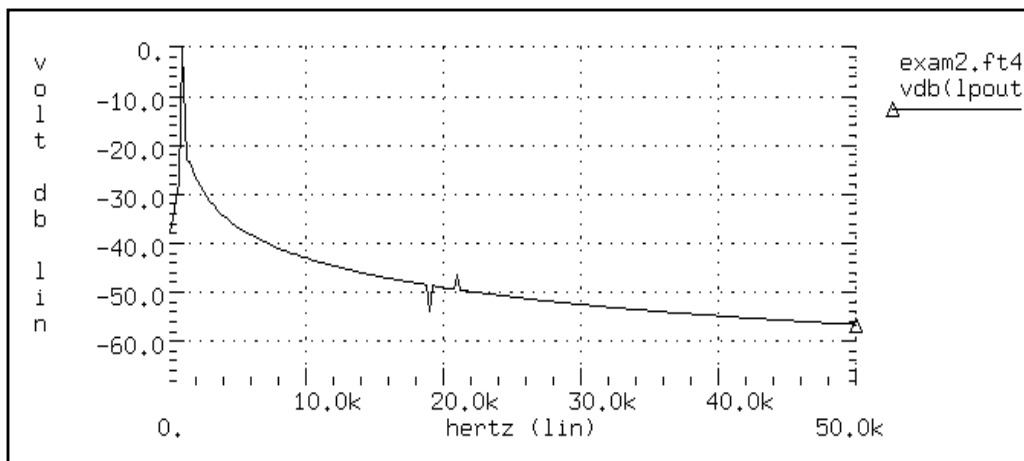
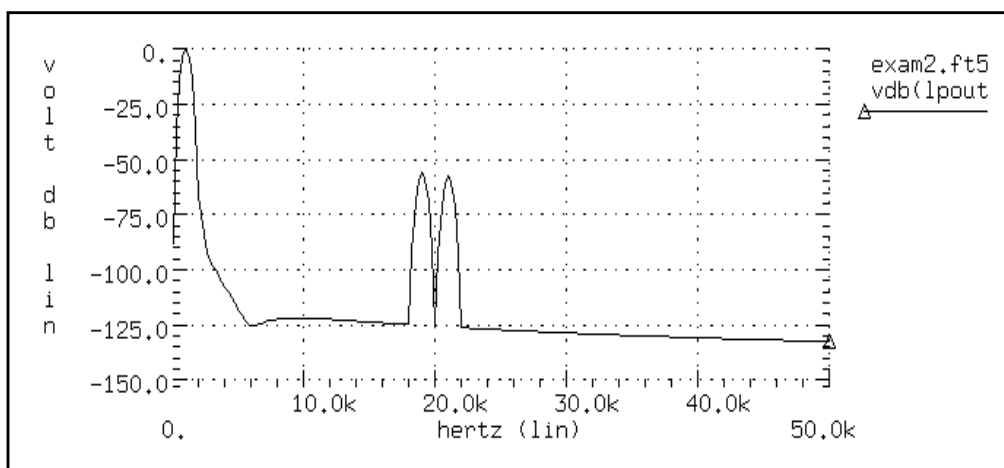
Figure 19-12: Demodulated Signal Spectrum**Figure 19-13: Filtered Output Signal (no window)**

Figure 19-14: Filtered Output Signal (Blackman-Harris window)

Signal Detection Test Circuit

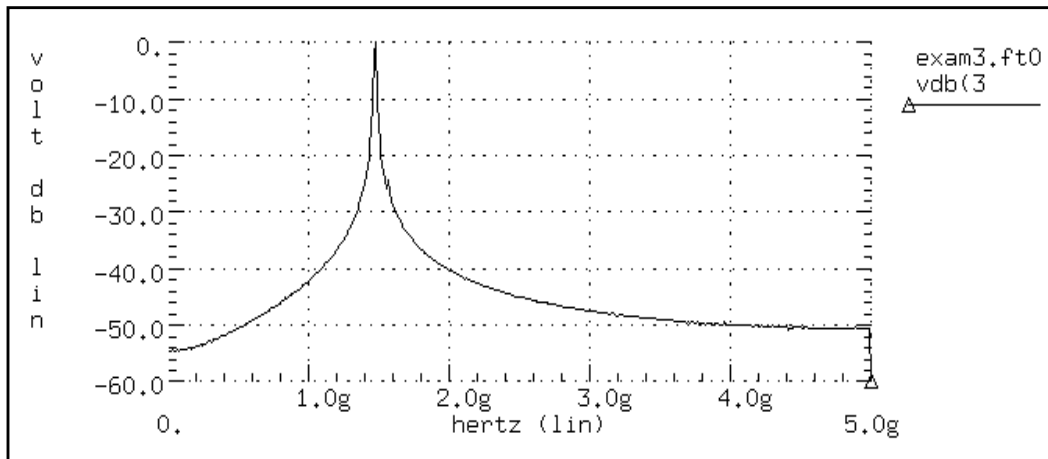
This example is a high-frequency mixer test circuit. It illustrates the effect of using a window to detect a weak signal, in the presence of a strong signal that is at a nearby frequency. This example adds two high-frequency signals, with a 40 dB separation (amplitudes are 1.0 and 0.01).

Input Listing

```
Signal Detection Test Circuit For FFT
v1 1 0 sin(0 1 1470.2Meg 0 0 90)
r1 1 0 1
v2 2 0 sin(0 0.01 1560.25Meg 0 0 90)
r2 2 0 1
E1 3 0 vol='v(1)+v(2)'
r3 3 0 1
.tran 0.1n 102.4n
.option post probe
.fft v(3)
.fft v(3) window=Bartlett fmin=1.2g fmax=2.2g
.fft v(3) window=hanning fmin=1.2g fmax=2.2g
.fft v(3) window=hamminn fmin=1.2g fmax=2.2g
.fft v(3) window=blackman fmin=1.2g fmax=2.2g
.fft v(3) window=harris fmin=1.2g fmax=2.2g
.fft v(3) window=gaussian fmin=1.2g fmax=2.2g
.fft v(3) window=kaiser fmin=1.2g fmax=2.2g
.end
```

Output

[Figure 19-15 on page 19-23](#) shows the rectangular window. Compare this with the spectra of the output for all FFT window types, as shown in [Figure 19-16](#) through [Figure 19-22](#). Without windowing, Star-Hspice does not detect the weak signal, because of spectral leakage.

Figure 19-15: Mixer Output Spectrum, Rectangular Window

- In the Bartlett window ([Figure 19-16 on page 19-24](#)), the noise floor increases dramatically, compared to the rectangular window (from -55, to more than -90 dB).
- The cosine windows (Hanning, Hamming, Blackman, and Blackman-Harris) all produce better results than the Bartlett window. However, the Blackman-Harris window provides the highest degree of separation for the two tones, and the lowest noise floor.
- The final two windows ([Figure 19-21 on page 19-26](#) and [Figure 19-22 on page 19-27](#)) use the ALFA=3.0 parameter, which is the default value in Star-Hspice. These two windows also produce acceptable results, especially the Kaiser-Bessel window, which sharply separates the two tones, and has a noise floor of almost -100-dB.

Processing such high frequencies, as demonstrated in this example, shows the numerical stability and accuracy of the FFT spectrum analysis algorithms, in Star-Hspice.

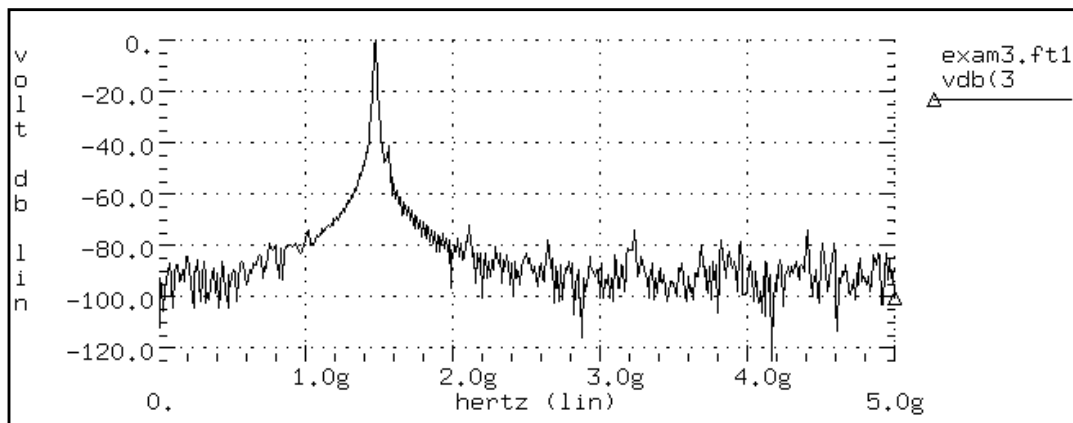
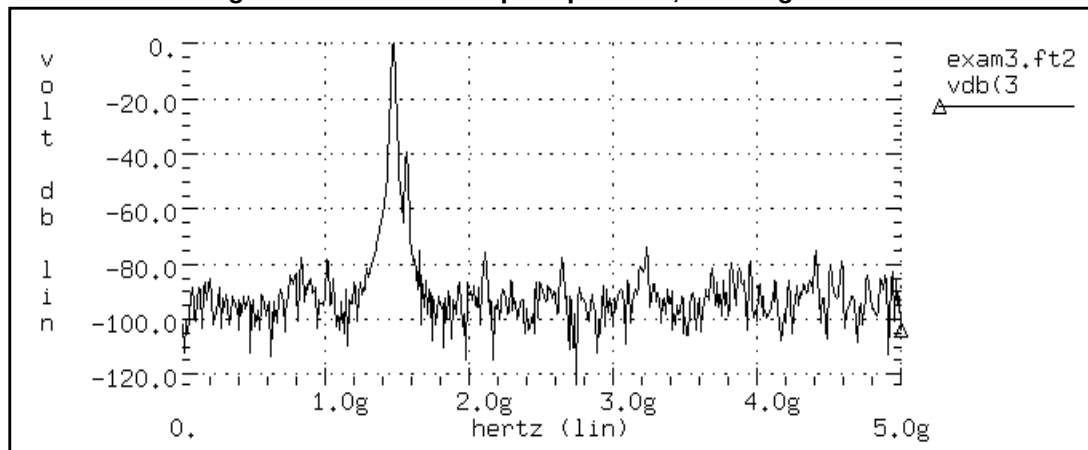
Figure 19-16: Mixer Output Spectrum, Bartlett Window**Figure 19-17: Mixer Output Spectrum, Hanning Window**

Figure 19-18: Mixer Output Spectrum, Hamming Window

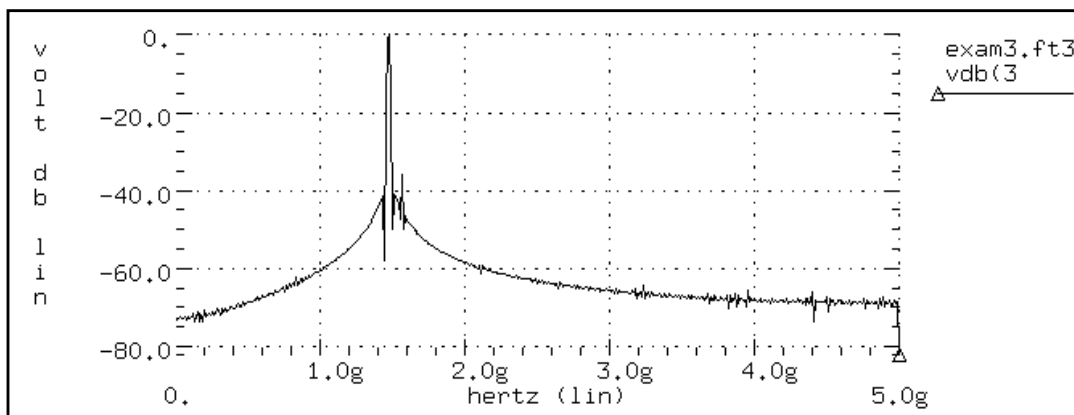


Figure 19-19: Mixer Output Spectrum, Blackman Window

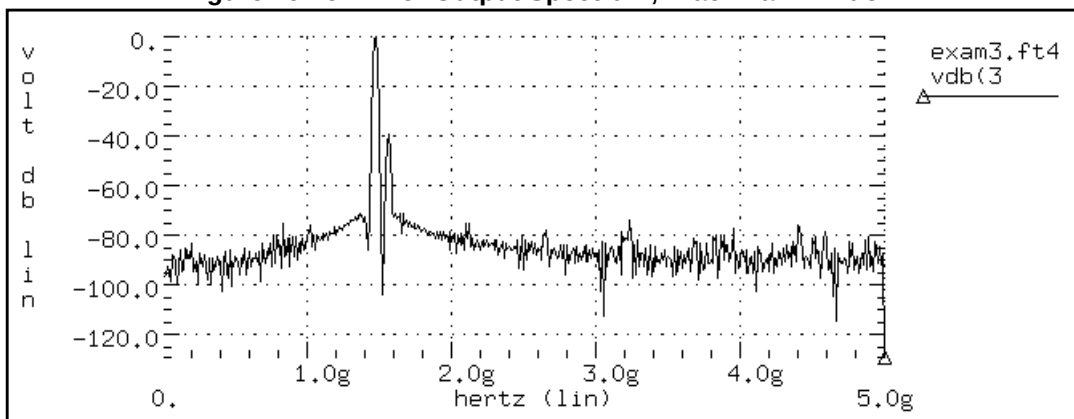


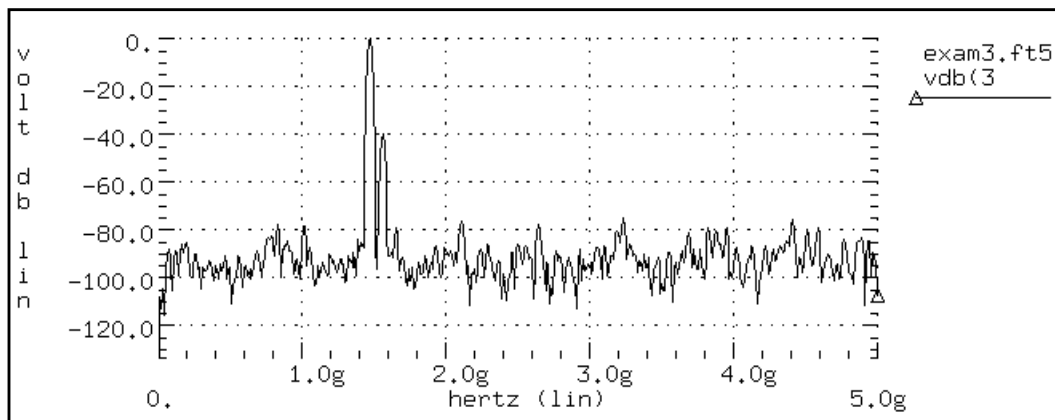
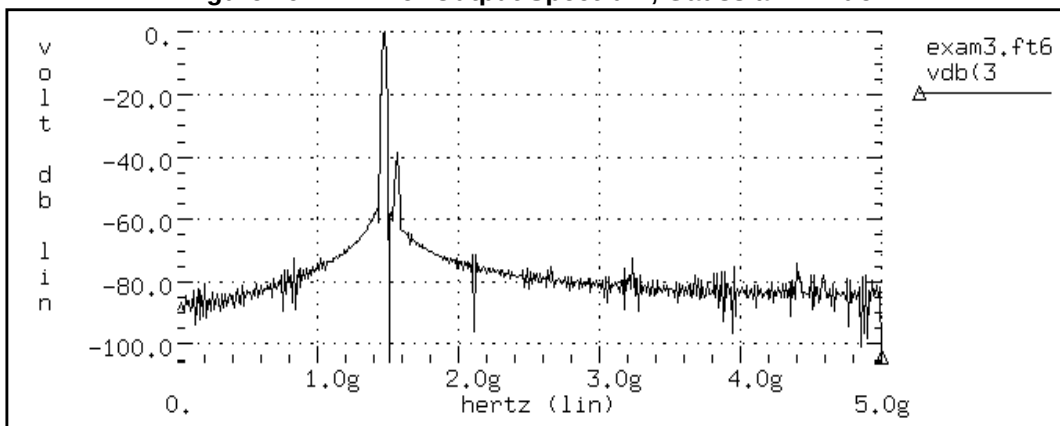
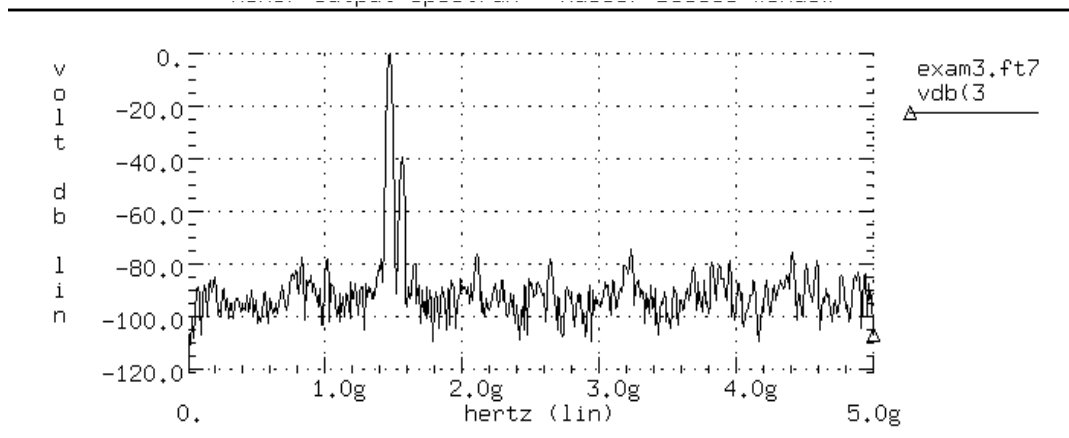
Figure 19-20: Mixer Output Spectrum, Blackman-Harris Window**Figure 19-21: Mixer Output Spectrum, Gaussian Window**

Figure 19-22: Mixer Output Spectrum, Kaiser-Bessel Window



References

1. For an excellent discussion of DFT windows, see Fredric J. Harris, “On the Use of Windows for Harmonic Analysis with Discrete Fourier Transform”, *Proceedings of the IEEE*, Vol. 66, No. 1, Jan. 1978.



Chapter 20

Modeling Filters and Networks

When you apply Kirchhoff's laws to circuits that contain energy storage elements, the result is simultaneous differential equations, in the time domain. A simulator must solve these equations, to analyze the circuit's behavior. Solving any equation that is higher than first order can be difficult, and classical methods cannot easily solve some driving functions.

In both cases, to simplify the solution, you can use Laplace transforms. These transforms convert time domain equations, containing integral and differential terms, into algebraic equations in the frequency domain.

This chapter explains the following topics:

- [Transient Modeling](#)
- [Using G and E Elements](#)
- [Laplace and Pole-Zero Modeling](#)
- [Modeling Switched Capacitor Filters](#)
- [References](#)

Transient Modeling

The Laplace transform method provides an easy way to relate a circuit's behavior, in time and frequency-domains. This facilitates simultaneous work in those domains.

The algorithm that Star-Hspice uses for Laplace and pole/zero transient modeling, offers better performance than the Fast Fourier Transform (FFT) algorithm. To invoke Laplace and pole/zero transient modeling, use a LAPLACE or POLE function call in a source element statement.

Laplace transfer functions are especially useful in top-down system design, when you use ideal transfer functions instead of detailed circuit designs. In Star-Hspice, you can also mix Laplace transfer functions, with transistors and passive components. Using this capability, you can model a system as the sum of the contributing ideal transfer functions. You can then progressively replace these functions with detailed circuit models, as they become available. Conventional uses of Laplace transfer functions include control systems, and behavioral models that contain non-linear elements.

Laplace transforms reduce the time needed to design and simulate large interconnect systems, such as clock distribution networks. You can use asymptotic waveform evaluation (AWE) and other methods, to create a Laplace transfer function model. The AWE model can use only a few poles to represent the large circuit. You can input these poles through a Laplace transform model, to closely approximate the delay and overshoot characteristics of many networks, in a fraction of the original simulation time.

You can use pole/zero analysis to help determine the stability of the design. You can use the POLE function in Star-Hspice when the poles and zeros of the circuit are specified, or you can use the .PZ statement (see [.PZ \(Pole/Zero\) Statement on page 18-4](#)) to derive the poles and zeros from the transfer function.

Frequency response is an important analog circuit property. It is normally the ratio of two complex polynomials (functions of complex frequencies), with positive real coefficients. The form of frequency response can be either the locations of poles and zeros, or a frequency table.

The usual way to design complex circuits is to interconnect smaller functional blocks of known frequency responses, either in pole/zero or frequency table form. For example, to design a band-reject filter, you can interconnect a low-pass filter, a high-pass filter, and an adder. Study the function of the complex circuit, in terms of its component blocks, before you design the actual circuit. After you test the functionality of the component blocks, you can use these blocks as a reference in optimization techniques, to determine the value of the complex element.

Using G and E Elements

This section describes how to use the G and E Elements.

Laplace Transform Function Call

Use the G and E Elements (controlled behavioral sources) as linear functional blocks, or as elements with specific frequency responses, in the following forms:

- [Laplace Transform](#)
- [Pole-Zero Function](#)
- [Frequency Response Table](#)

$H(s)$ denotes the frequency response (also called the impulse response), where s is a complex frequency variable ($s = j2\pi f$). To obtain the frequency response, perform an AC analysis, and set AC=1 in the input source (the Laplace transform of an impulse is 1). The following expression relates the input and output of the G and E Elements, with specified frequency response:

$$Y(j2\pi f) = H(j2\pi f) \cdot X(j2\pi f)$$

where X is the input, Y is the output, and H is the transfer function, at the f frequency.

AC analysis uses the above relation, at any frequency, to determine the frequency response. For operating point and DC sweep analysis, the relation is the same, but the frequency is zero.

The transient analysis is more complicated than the frequency response. The output is a convolution of the input waveform, with the impulse response $h(t)$:

$$y(t) = \int_{-\infty}^t x(\tau) \cdot h(t - \tau) \cdot d\tau$$

In discrete form, the output is:

$$y(k\Delta) = \Delta \sum_{m=0}^k x(m\Delta) \cdot h[(k-m) \cdot \Delta], \quad k = 0, 1, 2, \dots$$

where you can obtain $h(t)$ from $H(f)$, using the inverse Fourier integral:

$$h(t) = \int_{-\infty}^{\infty} H(f) \cdot e^{j2\pi ft} \cdot df$$

The following equation calculates the inverse discrete Fourier transform:

$$h(m\Delta) = \frac{1}{N \cdot \Delta} \sum_{n=0}^{N-1} H(f_n) \cdot e^{\frac{j2\pi nm}{N}}, \quad m = 0, 1, 2, \dots, N-1$$

where N is the number of equally-spaced time points, and Δ is the time interval or time resolution.

For the frequency response table form (FREQ) of the LAPLACE function, Star-Hspice uses a performance-enhanced algorithm, to convert $H(f)$ to $h(t)$. This algorithm requires N to be a power of 2. The following equation determines the f_n frequency point:

$$f_n = \frac{n}{N \cdot \Delta}, \quad n = 0, 1, 2, \dots, N-1$$

where $n > N/2$ represents the negative frequencies. The following equation determines the Nyquist critical frequency:

$$f_c = f_{N/2} = \frac{1}{2 \cdot \Delta}$$

Because the negative frequency responses are the image of the positive responses, you need to specify only $N/2$ frequency points, to evaluate N time points of $h(t)$. The larger the value of f_c is, the more accurate the transient analysis results are. However, for large f_c values, the Δ becomes smaller, and computation time increases.

The maximum frequency of interest depends on the functionality of the linear network. For example, in a low-pass filter, you can set f_c to the frequency at which the response drops by 60 dB (a factor of 1000).

$$|H(f_c)| = \frac{|H_{\max}|}{1000}$$

After you select or calculate f_c , the following equation can determine Δ :

$$\Delta = \frac{1}{2 \cdot f_c}$$

The following equation calculates the frequency resolution:

$$\Delta f = f_1 = \frac{1}{N \cdot \Delta}$$

which is inversely proportional to the maximum time ($N \cdot \Delta$), over which Star-Hspice evaluates $h(t)$. Therefore, the transient analysis accuracy also depends on the frequency resolution, or the number of points (N).

1. You can specify the frequency resolution (DELF), and the maximum frequency (MAXF), in the G or E Element statement.
2. To calculate N , Star-Hspice uses $2 \cdot \text{MAXF} / \text{DELF}$.
3. Then, Star-Hspice modifies N as a power of 2. The effective DELF is $2 \cdot \text{MAXF} / N$, to reflect the changes in N .

Laplace Transform

Syntax

Transconductance $H(s)$:

```
Gxxx n+ n- LAPLACE in+ in- k0, k1, ..., kn / d0,
+ d1, ..., dm
<SCALE=val> <TC1=val> <TC2=val> <M=val>
```

Voltage Gain $H(s)$:

```
Exxx n+ n- LAPLACE in+ in- k0, k1, ..., kn / d0,
+ d1, ..., dm
<SCALE=val> <TC1=val> <TC2=val>
```

$H(s)$ is a rational function, in the following form:

$$H(s) = \frac{k_0 + k_1s + \dots + k_ns^n}{d_0 + d_1s + \dots + d_ms^m}$$

You can use parameters to define the values of all coefficients ($k_0, k_1, \dots, d_0, d_1, \dots$).

Example

```

Glowpass 0 out LAPLACE in 0 1.0 / 1.0 2.0 2.0 1.0
Ehipass out 0 LAPLACE in 0 0.0,0.0,0.0,1.0 /
1.0,2.0,2.0,1.0

```

The *Glowpass* element statement describes a third-order low-pass filter, with the transfer function:

$$H(s) = \frac{1}{1 + 2s + 2s^2 + s^3}$$

The *Ehipass* element statement describes a third-order high-pass filter, with the transfer function:

$$H(s) = \frac{s^3}{1 + 2s + 2s^2 + s^3}$$

Pole-Zero Function

Syntax

Transconductance $H(s)$:

```

Gxxx n+ n- POLE in+ in- a  $\alpha_{z1}, f_{z1}, \dots, \alpha_{zn}, f_{zn}$  / b,
+  $\alpha_{p1}, f_{p1}, \dots, \alpha_{pm}, f_{pm}$  <SCALE=val> <TC1=val>
+ <TC2=val> <M=val>

```

Voltage Gain $H(s)$:

```

Exxx n+ n- POLE in+ in- a  $\alpha_{z1}, f_{z1}, \dots, \alpha_{zn}, f_{zn}$  / b,
+  $\alpha_{p1}, f_{p1}, \dots, \alpha_{pm}, f_{pm}$  <SCALE=val> <TC1=val>
+ <TC2=val>

```

The following equation defines $H(s)$ in terms of poles and zeros:

$$H(s) = \frac{a \cdot (s + \alpha_{z1} - j2\pi f_{z1}) \dots (s + \alpha_{zn} - j2\pi f_{zn})(s + \alpha_{zn} + j2\pi f_{zn})}{b \cdot (s + \alpha_{p1} - j2\pi f_{p1}) \dots (s + \alpha_{pm} - j2\pi f_{pm})(s + \alpha_{pm} + j2\pi f_{pm})}$$

The complex poles or zeros are in conjugate pairs. The element description specifies only one of them, and the program includes the conjugate. You can use parameters to specify the a, b, α , and f values.

Example

```
Ghigh_pass 0 out POLE in 0 1.0 0.0,0.0 / 1.0 0.001,0.0
Elow_pass out 0 POLE in 0 1.0 / 1.0, 1.0,0.0 0.5,0.1379
```

The *Ghigh_pass* statement describes a high-pass filter, with the transfer function:

$$H(s) = \frac{1.0 \cdot (s + 0.0 + j \cdot 0.0)}{1.0 \cdot (s + 0.001 + j \cdot 0.0)}$$

The *Elow_pass* statement describes a low-pass filter, with the transfer function:

$$H(s) = \frac{1.0}{1.0 \cdot (s + 1)(s + 0.5 + j2\pi \cdot 0.1379)(s + 0.5 - (j2\pi \cdot 0.1379))}$$

Frequency Response Table

Syntax

Transconductance $H(s)$:

```
Gxxx n+ n- FREQ in+ in- f1, a1,  $\phi_1$ , ..., fi, ai,  $\phi_1$ 
+ <DELF=val> <MAXF=val> <SCALE=val> <TC1=val>
+ <TC2=val> <M=val> <LEVEL=val>
+ <INTERPOLATION=val> <EXTRAPOLATION=val>
+ <ACCURACY=val>
```

Voltage Gain $H(s)$:

```
Exxx n+ n- FREQ in+ in- f1, a1,  $\phi_1$ , ..., fi, ai,  $\phi_1$ 
+ <DELF=val> <MAXF=val> <SCALE=val> <TC1=val>
+ <TC2=val>
```

- Each f_i is a frequency point, in hertz.
- a_i is the magnitude, in dB.
- ϕ_1 is the phase, in degrees.

At each frequency, Star-Hspice uses interpolation to calculate the network response, magnitude, and phase. Star-Hspice interpolates the magnitude (in dB) logarithmically, as a function of frequency. It also interpolates the phase (in degrees) linearly, as a function of frequency.

$$|H(j2\pi f)| = \left(\frac{a_i - a_k}{\log f_i - \log f_k} \right) (\log f - \log f_i) + a_i$$

$$\angle H(j2\pi f) = \left(\frac{\phi_i - \phi_k}{f_i - f_k} \right) (f - f_i) + \phi_i$$

Note: You can choose how to interpolate and extrapolate the frequency-table G element, as described in [Element Statement Parameters on page 20-10](#).

Example

```

Eftable output    0 FREQ input    0
+ 1.0k   -3.97m   293.7
+ 2.0k   -2.00m   211.0
+ 3.0k   17.80m   82.45
+
+ 10.0k -53.20   -1125.5

```

- The first column is frequency, in hertz.
- The second column is magnitude, in dB.
- The third column is phase, in degrees.

Set the LEVEL to 1 for a high-pass filter.

Set the last frequency point to the highest frequency response value that is a real number, with zero phase.

You can use parameters to set the frequency, magnitude, and phase, in the table.

Element Statement Parameters

These keywords are common to the three forms described above:

- Laplace
- pole-zero
- frequency response table

<i>ACCURACY</i>	Used only in G element, with the frequency response table. 0: default. To achieve the most accurate control voltage, at each time point, use linear interpolation of the two closest time points. 1: faster mode. Star-Hspice finds the control voltage at each time point, from the simulated time point just before the target. The smaller you set the time step, the closer the result is to the most accurate mode. 2: method used in release 2000.4 and before.
<i>DELTA, DELF</i>	Frequency resolution Δf. The inverse of DELF is the time window, over which Star-Hspice calculates $h(t)$ from $H(s)$. The smaller the DELF value is, the more accurate the transient analysis is, and the longer the CPU time. The number of points (N) used to convert $H(s)$ to $h(t)$, is $N=2 \cdot \text{MAXF}/\text{DELF}$. Because N must be a power of 2, Star-Hspice adjusts the DELF value. The default is $1/\text{TSTOP}$. In the G element, with FREQ and ACCURACY = 0 or 1, to perform circular convolution for periodic input, Star-Hspice limits the period. To do this, Star-Hspice sets $\text{DELF} = 1/T : T < \text{TSTOP}$.
<i>EXTRAPOLATION</i>	Extrapolation scheme, used only in the G element, with the frequency response table. 0: Linear interpolation (default); uses last 2 boundary points. 1: uses the last boundary point.

<i>FREQ</i>	Keyword, to indicate that a frequency response table describes the transfer function. Do not use FREQ as a node name in a G or E Element.
<i>INTERPOLATION</i>	Interpolation scheme, used only in the G element, with a frequency response table. 0: Piecewise linear (default) 1: Piecewise step. 2: B-spline curve fit.
<i>LAPLACE</i>	Keyword, to indicate that a Laplace transform function describes the transfer function. Do not use LAPLACE as a node name, on a G or E Element.
<i>LEVEL</i>	Used only in elements with a frequency-response table. Set this parameter to 1, if the element is a high-pass filter.
<i>M</i>	G Element multiplier. This parameter represents <i>M</i> G Elements in parallel. Default is 1.
<i>MAXF, MAX</i>	Maximum, or the Nyquist critical frequency. The larger the MAXF value, the more accurate the transient results are, and the longer the CPU time. The default is $1024 \cdot \text{DELF}$. These parameters apply only when you also use the FREQ parameter.
<i>POLE</i>	Keyword, to indicate that the pole and zero location describes the transfer function. Do not use POLE as a node name, on a G or E Element.
<i>SCALE</i>	Element value multiplier.
<i>TC1, TC2</i>	First-order and second-order temperature coefficients. The default is zero. The temperature updates the SCALE:

$$\text{SCALE}_{\text{eff}} = \text{SCALE} \cdot (1 + \text{TC1} \cdot \Delta t + \text{TC2} \cdot \Delta t^2)$$

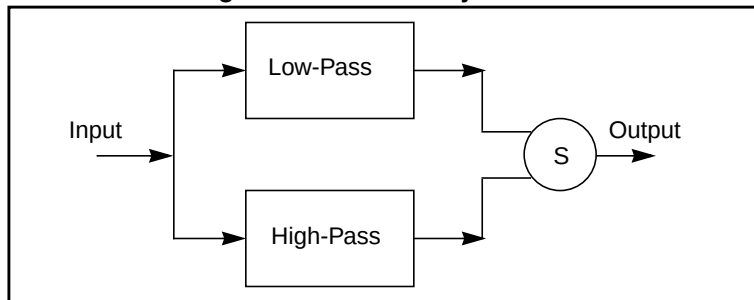
G and E Element Notes

- If elements in the data file specify frequency responses, do not use pole/zero analysis. If you specify `MAXF=<par>` in a G or E Element Statement, Star-Hspice warns that it is ignoring MAXF. This is normal.
- Use Interpolation, Extrapolation, and Accuracy options only for the G element, with the `FREQ` option.
- You can use the interpolation and extrapolation parameters, noted above, if `ACCURACY = 0` or `1`. If `ACCURACY = 2`, Star-Hspice performs interpolation/extrapolation, as mentioned in [Frequency Response Table on page 20-8](#).
- Star-Hspice performs circular convolution, when G element `ACCURACY = 0` or `1` (see [Figure 20-7](#)).

Laplace Band-Reject Filter

This example models an active band-reject filter ([see 1 on page 20-51](#)), with 3-dB points at 100 and 400 Hz, and <35 dB of attenuation, between 175 and 225 Hz. The band-reject filter contains low-pass and high-pass filters, and an adder. The low-pass and high-pass filters are fifth-order Chebyshev, with 0.5-dB ripple.

Figure 20-1: Band-Reject Filter



Example

```

BandstopL.sp band_reject filter
.OPTION PROBE POST=2
.AC DEC 50 10 5k
.PROBE AC VM(out_low) VM(out_high) VM(out)
.PROBE AC VP(out_low) VP(out_high) VP(out)
.TRAN .01m 12m
.PROBE V(out_low) V(out_high) V(out)
.GRAPH v(in) V(out)
Vin in 0 AC 1 SIN(0,1,250)
  
```


Band-Reject Filter Circuit

```

Elp3 out_low3 0 LAPLACE in 0
+ 1 / 1 6.729m 15.62988u 27.7976n
Elp out_low 0 LAPLACE out_low3 0
+ 1 / 1 0.364m 2.7482u
Ehp3 out_high3 0 LAPLACE in 0 0,0,0,9.261282467p /
+ 1,356.608u,98.33419352n,9.261282467p
Eph out_high 0 LAPLACE out_high3 0
+ 0 0 144.03675n / 1 83.58u 144.03675n
Eadd out 0 VOL='-V(out_low) - V(out_high)'
Rl out 0 1e6
.END

```

Figure 20-2: Frequency Response of the Band-Reject Filter

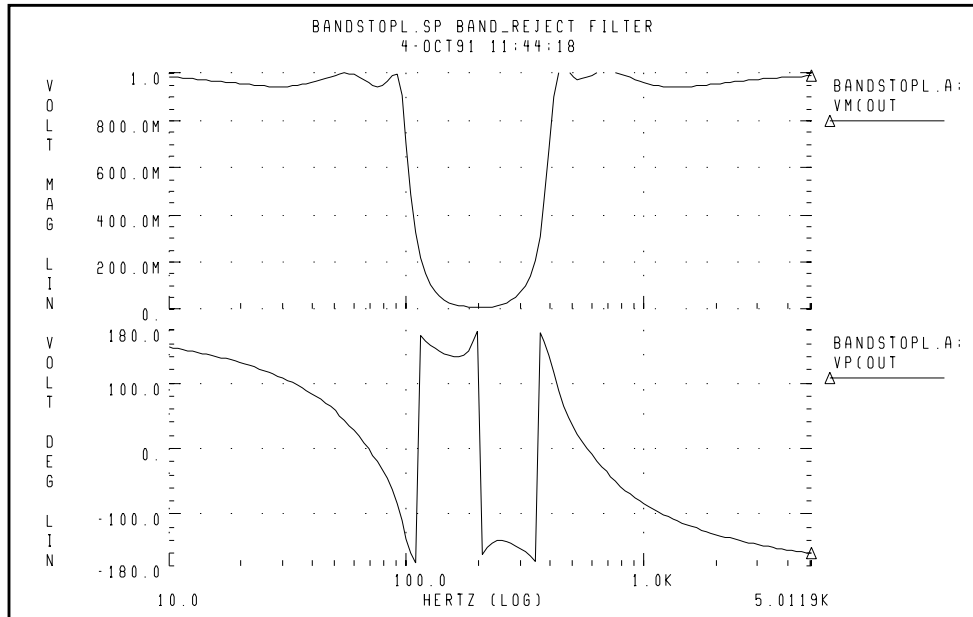
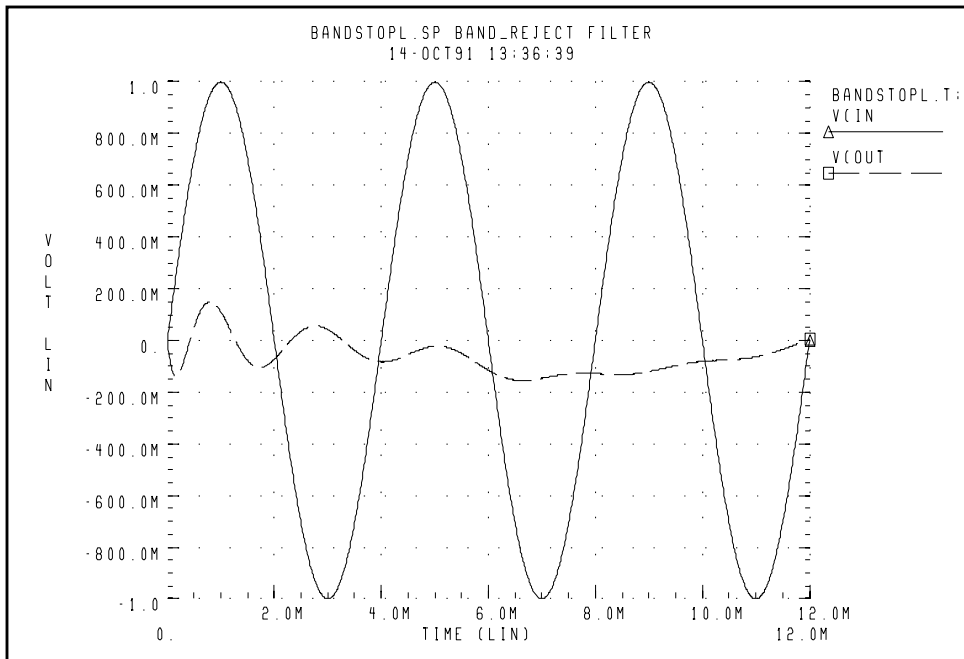
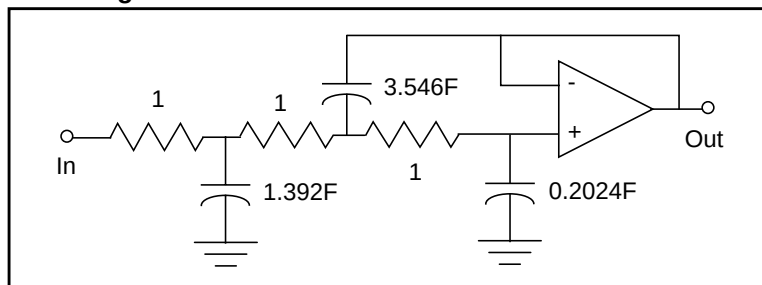


Figure 20-3: Transient Response of the Band-Reject Filter to a 250 Hz Sine Wave

Laplace Low-Pass Filter

This example simulates a third-order low-pass filter, with a Butterworth transfer function. It also compares the results of both the actual circuit and the functional G Element, with the third-order Butterworth transfer function, for AC and transient analysis.

Figure 20-4: Third-Order Active Low-Pass Filter

The third-order Butterworth transfer function that describes the above circuit is:

$$H(s) = \frac{1.0}{1.0 \cdot (s + 1)(s + 0.5 + j2\pi \cdot 0.1379)(s + 0.5 - (j2\pi \cdot 0.1379))}$$

The following example is the input listing for the above filter. Parameters set the pole locations for the G Element. Also, this listing specifies only one of the complex poles. The program derives the conjugate pole. The output of the circuit is the *out* node, and the output of the functional element is *outg*.

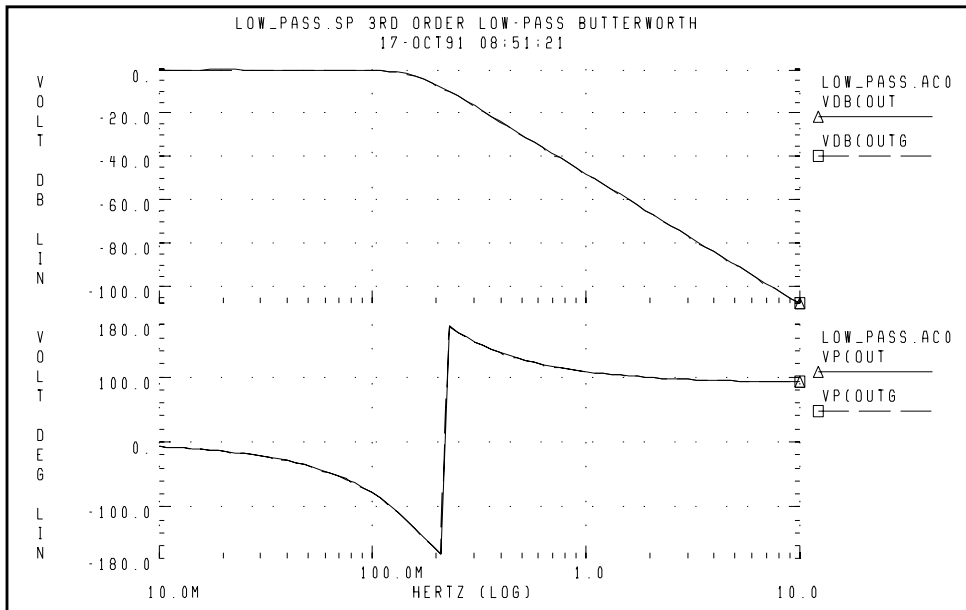
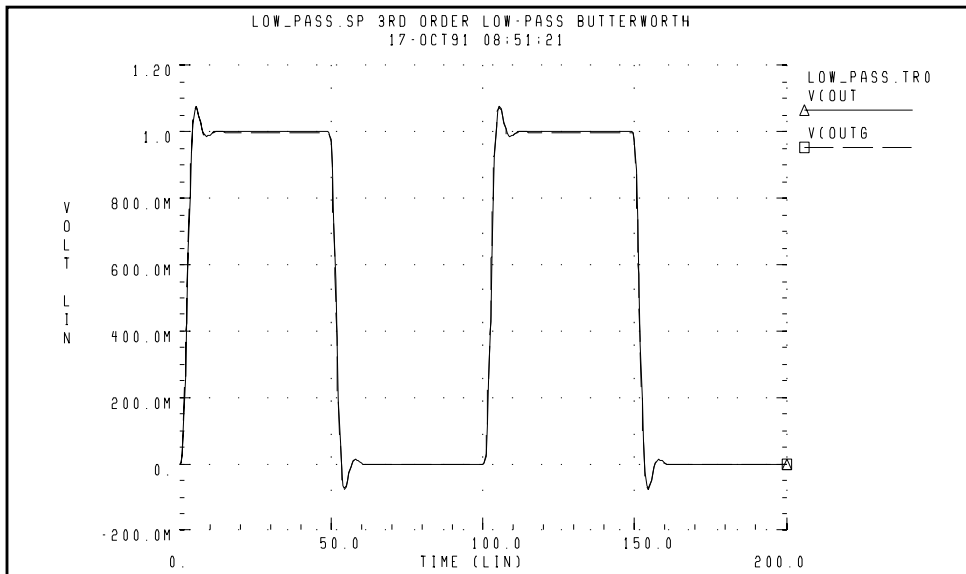
Example

The following is an example of a third-order low-pass Butterworth filter:

```
Low_Pass.sp 3rd order low-pass Butterworth
.OPTION POST=2 PROBE INTERP=1 DCSTEP=1e8
.PARAM a=1.0 b=1.0 ap1=1.0 fp1=0.0 ap2=0.5 fp2=0.1379
.AC DEC 25 0.01 10
.PROBE AC VDB(out) VDB(outg) VP(out) VP(outg)
.TRAN .5 200
.PROBE V(in) V(outg) V(out)
.GRAPH V(outg) V(out)
VIN in 0 AC 1 PULSE(0,1,0,1,1,48,100)
* 3rd order low-pass described by G Element
Glow_pass 0 outg POLE in 0 a / b ap1,fp1 ap2,fp2
Rg outg 0 1
```

Circuit Description

```
R1 in 2 1
R2 2 3 1
R3 3 4 1
C1 2 0 1.392
C2 4 0 0.2024
C3 3 out 3.546
Eopamp out 0 OPAMP 4 out
.END
```

Figure 20-5: Frequency Response of Circuit and Functional Element**Figure 20-6: Transient Response of Circuit and Functional Element to a Pulse**

Circular Convolution Example

This 30-degree phase-shift filter uses circular convolution. If `DELF = 10MHz`, Star-Hspice uses `IFFT` to obtain the period of time domain response for the `G` element. This value is based on the input frequency table, and is 100 nanoseconds. The `FREQ G` element performs the convolution integral, from $t - T$ to t , assuming that all control voltages at $t < 0$ are zero. t is the target time point, and T is the period of the time domain response, for the `G` element.

In this example, during time points from 0 to 100 nanoseconds, Star-Hspice uses harmonic components higher than 10MHz, due to the input transition at $t=0$. So the circuit does not behave as a phase shift filter. After one period ($t > 100$ nanoseconds), Star-Hspice performs circular convolution, based on a period of 100 nanoseconds. The transient result represents a 30-degree phase shift, for continuous periodic control voltage.

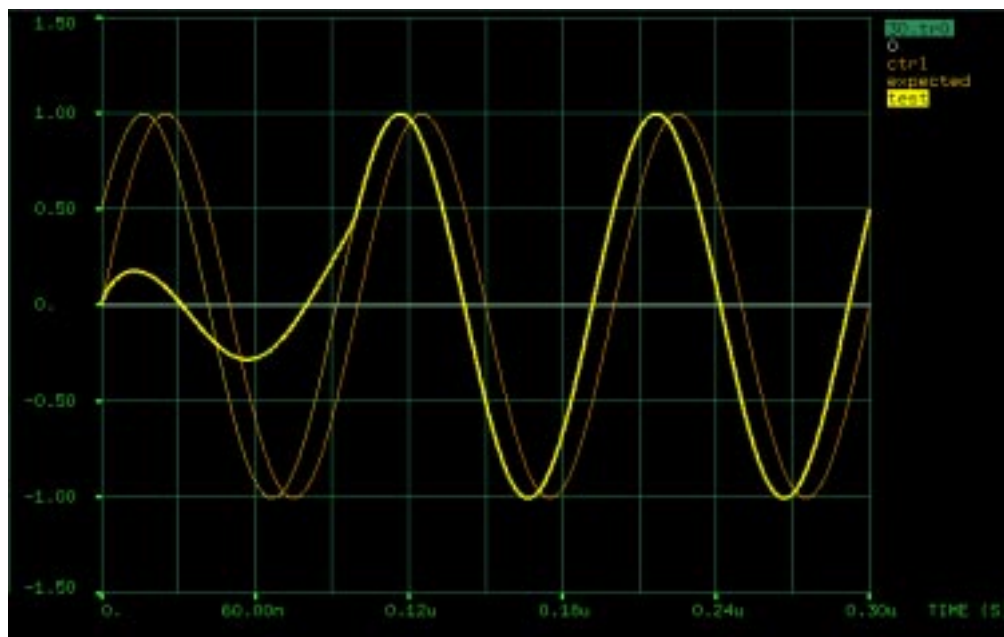
Notes

- `V(ctrl)`: control voltage input.
- `V(expected)`: node. Represents an ideal 30-degree shifted wave for the input.
- `V(test)`: output of the `G` element.

30-Degree Phase Shift Circuit File

This example illustrates a 30-degree, phase-shift filter.

```
.tran 0.1n 300n
.OPTION post=2 ingold=2 accurate
Vctrl ctrl gnd sin (0 1 10e6)
Gtest gnd test freq ctrl gnd
+ 1.0e00 0 30
+ 1.0e01 0 30
+ 1.0e02 0 30
+ 1.0e03 0 30
+ 1.0e04 0 30
+ 1.0e05 0 30
+ 1.0e06 0 30
+ 1.0e07 0 30
+ 1.0e08 0 30
+ 1.0e09 0 30
+ 1.0e10 0 30
+ MAXF=1.0e9 DELF=10e6
Rtest test gnd 1
Iexpected gnd 3 sin (0 1 10e6 0 0 30)
Vmes 2 3 expected 0v
Rexpected expected gnd 1
.end
```

Figure 20-7: Transient Response of the 30 Degree Phase Shift Filter

Laplace and Pole-Zero Modeling

Laplace Transform (LAPLACE) Function

Star-Hspice provides two types of LAPLACE function calls: one for transconductance, and one for voltage gain transfer functions. See [Using G and E Elements on page 20-4](#) for the general forms, and [Element Statement Parameters on page 20-10](#) for descriptions of the parameters.

General Form of the Transfer Function

To use LAPLACE modeling function, you must find the $k_0, \dots, k_n$ and $d_0, \dots, d_m$ coefficients of the transfer function. The transfer function is the s-domain (frequency domain) ratio, of the output for a single-source circuit, to the input, with initial conditions set to zero. The following equation represents the Laplace transfer function:

$$H(s) = \frac{Y(s)}{X(s)},$$

where:

- s is the complex frequency, $j2\pi f$.
- $Y(s)$ is the Laplace transform of the output signal.
- $X(s)$ is the Laplace transform of the input signal.

Note: To obtain the impulse response $H(s)$, Star-Hspice performs AC analysis, where AC=1 represents the input source. The Laplace transform of an impulse is 1. For an element with an infinite response at DC (such as a unit step function $H(s)=1/s$), Star-Hspice uses the value of the EPSMIN option (the smallest number possible on the platform) as the transfer function, in its calculations.

The general form of the transfer function $H(s)$, in the frequency domain, is:

$$H(s) = \frac{k_0 + k_1 s + \dots + k_n s^n}{d_0 + d_1 s + \dots + d_m s^m}$$

The order of the numerator for the transfer function cannot be greater than the order of the denominator. The exception is differentiators, for which the transfer function $H(s) = ks$. You can use parameter values for all k and d coefficients of the transfer function, in the circuit descriptions.

Finding the Transfer Function

The first step in determining the transfer function of a circuit is to convert the circuit to the s -domain. To do this, transform the value for each element, into its s -domain equivalent form.

[Table 20-1](#) and [Table 20-2](#) show transforms that convert some common functions to the s -domain ([see 2 on page 20-51](#)). The next section provides examples of using transforms, to determine transfer functions.

Table 20-1: Laplace Transforms for Common Source Functions

$f(t), t > 0$	Source Type	$L \{f(t)\} = F(s)$
$\delta(t)$	impulse	1
$u(t)$	step	$\frac{1}{s}$
t	ramp	$\frac{1}{s^2}$
e^{-at}	exponential	$\frac{1}{s + a}$
$\sin \omega t$	sine	$\frac{\omega}{s^2 + \omega^2}$

Table 20-1: Laplace Transforms for Common Source Functions (Continued)

f(t), t>0	Source Type	$L \{f(t)\} = F(s)$
$\cos \omega t$	cosine	$\frac{s}{s^2 + \omega^2}$
$\sin(\omega t + \theta)$	sine	$\frac{s \sin(\theta) + \omega \cos(\theta)}{s^2 + \omega^2}$
$\cos(\omega t + \theta)$	cosine	$\frac{s \cos(\theta) - \omega \sin(\theta)}{s^2 + \omega^2}$
$\sinh \omega t$	hyperbolic sine	$\frac{\omega}{s^2 - \omega^2}$
$\cosh \omega t$	hyperbolic cosine	$\frac{s}{s^2 - \omega^2}$
te^{-at}	damped ramp	$\frac{1}{(s + a)^2}$
$e^{-at} \sin \omega t$	damped sine	$\frac{\omega}{(s + a)^2 + \omega^2}$
$e^{-at} \cos \omega t$	damped cosine	$\frac{s + a}{(s + a)^2 + \omega^2}$

Table 20-2: Laplace Transforms for Common Operations

f(t)	$L \{f(t)\} = F(s)$
$Kf(t)$	$KF(s)$
$f_1(t) + f_2(t) - f_3(t) + \dots$	$F_1(s) + F_2(s) - F_3(s) + \dots$

Table 20-2: Laplace Transforms for Common Operations (Continued)

$f(t)$	$L \{f(t)\} = F(s)$
$\frac{d}{dt}f(t)$	$sF(s) - f(0^-)$
$\frac{d^2}{dt^2}f(t)$	$s^2F(s) - sf(0) - \frac{d}{dt}f(0^-)$
$\frac{d^n}{dt^n}f(t)$	$s^nF(s) - s^{n-1}f(0) - s^{n-2}\frac{d}{dt}f(0)$
$\int_{-\infty}^t f(t)dt$	$\frac{F(s)}{s} + \frac{f^{-1}(0)}{s}$
$f(t-a)u(t-a), a > 0$ (u is the step function)	$e^{-as}F(s)$
$e^{-at}f(t)$	$F(s+a)$
$f(at), a > 0$	$\frac{1}{a}F\left(\frac{s}{a}\right)$
$tf(t)$	$-\frac{d}{ds}(F(s))$
$t^n f(t)$	$-(1)^n \frac{d^n}{ds^n}F(s)$
$\frac{f(t)}{t}$	$\int_s^\infty F(u)du$ (u is the step function)
$f(t-t_1)$	$e^{-t_1s}F(s)$

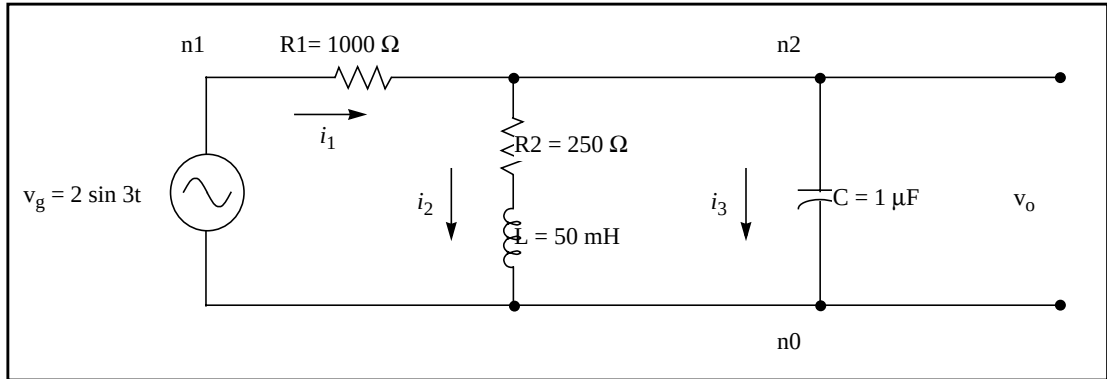
Determining the Laplace Coefficients

The following examples describe how to determine the appropriate coefficients, for the Laplace modeling function call.

LAPLACE Example 1 – Voltage Gain Transfer Function

To find the voltage-gain transfer function for the circuit in [Figure 20-8](#), convert the circuit to its equivalent s-domain circuit, and solve for v_o / v_g .

Figure 20-8: LAPLACE Example 1 Circuit



Use transforms from [Table 20-2](#), to convert the inductor, capacitor, and resistors. $L\{f(t)\}$ represents the Laplace transform of $f(t)$:

$$L\left\{L \frac{d}{dt} f(t)\right\} = L \cdot (sF(s) - f(0)) = 50 \times 10^{-3} \cdot (s - 0) = 0.05s$$

$$L\left\{\frac{1}{C} \int_0^t f(t) d\tau\right\} = \frac{1}{C} \cdot \left(\frac{F(s)}{s} + \frac{f^{-1}(0)}{s}\right) = \frac{1}{10^{-6}} \cdot \left(\frac{1}{s} + 0\right) = \frac{10^6}{s}$$

$$L\{R1 \cdot f(t)\} = R1 \cdot F(s) = R1 = 1000 \, \Omega$$

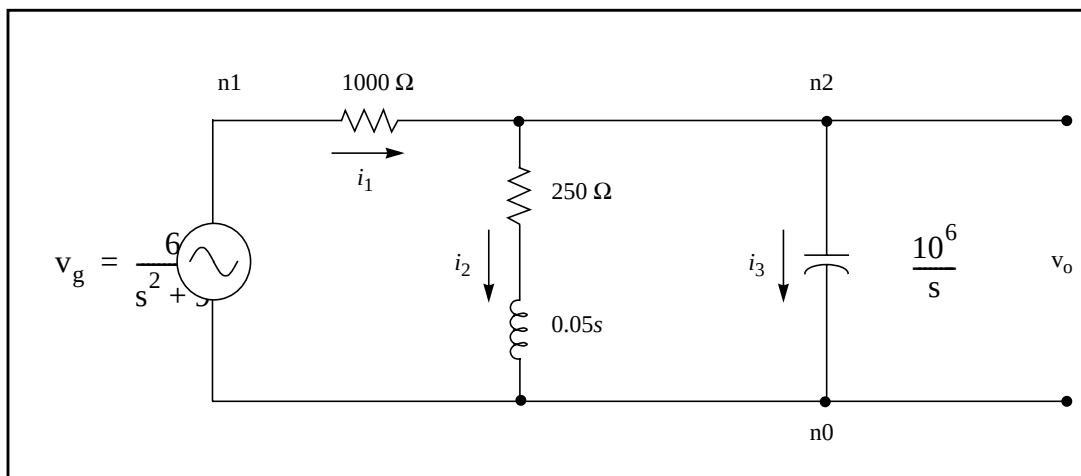
$$L\{R2 \cdot f(t)\} = R2 \cdot F(s) = R2 = 250 \, \Omega$$

To convert the voltage source to the s-domain, use the $\sin \omega t$ transform from Table 20-1:

$$L\{2 \sin 3t\} = 2 \cdot \frac{3}{s^2 + 3^2} = \frac{6}{s^2 + 9}$$

Figure 20-9 displays the s-domain equivalent circuit.

Figure 20-9: S-Domain Equivalent of the LAPLACE Example 1 Circuit



Summing the output currents from the n2 node:

$$\frac{v_o - v_g}{1000} + \frac{v_o}{250 + 0.05s} + \frac{v_o s}{10^6} = 0$$

Solve for v_o :

$$v_o = \frac{1000(s + 5000)v_g}{s^2 + 6000s + 25 \times 10^6}$$

The voltage-gain transfer function is:

$$H(s) = \frac{v_o}{v_g} = \frac{1000(s + 5000)}{s^2 + 6000s + 25 \times 10^6} = \frac{5 \times 10^6 + 1000s}{25 \times 10^6 + 6000s + s^2}$$

For the Laplace function call, use the k_n and d_m coefficients for the transfer function, in the form:

$$H(s) = \frac{k_0 + k_1s + \dots + k_ns^n}{d_0 + d_1s + \dots + d_ms^m}$$

The coefficients, from the above voltage-gain transfer function, are:

$$\begin{array}{lll} k_0 = 5 \times 10^6 & k_1 = 1000 & \\ d_0 = 25 \times 10^6 & d_1 = 6000 & d_2 = 1 \end{array}$$

Using these coefficients, the following is a Laplace modeling function call, for the voltage-gain transfer function of the circuit in [Figure 20-8 on page 20-23](#):

```
Example1 n1 n0 LAPLACE n2 n0 5E6 1000 / 25E6 6000 1
```

LAPLACE Example 2 – Differentiator

To model a differentiator, use either G or E Elements, as shown in the following example.

In the frequency domain:

$$\text{E Element: } V_{\text{out}} = ksV_{\text{in}}$$

$$\text{G Element: } I_{\text{out}} = ksV_{\text{in}}$$

In the time domain:

$$\text{E Element: } v_{\text{out}} = k \frac{dV_{\text{in}}}{dt}$$

$$\text{G Element: } i_{\text{out}} = k \frac{dV_{\text{in}}}{dt}$$

For a differentiator, the voltage gain transfer function is:

$$H(s) = \frac{V_{\text{out}}}{V_{\text{in}}} = ks$$

In the general form of the transfer function:

$$H(s) = \frac{k_0 + k_1s + \dots + k_ns^n}{d_0 + d_1s + \dots + d_ms^m},$$

If you set $k_1 = k$ and $d_0 = 1$, and the remaining coefficients are zero, then the equation becomes:

$$H(s) = \frac{ks}{1} = ks$$

Using the $k_1 = k$ and $d_0 = 1$ coefficients, in the Laplace modeling, the circuit descriptions for the differentiator are:

```
Edif out GND LAPLACE in GND 0 k / 1
Gdif out GND LAPLACE in GND 0 k / 1
```

LAPLACE Example 3 – Integrator

You can use G or E Elements to model an integrator, as follows:

In the frequency domain:

$$\text{E Element: } V_{\text{out}} = \frac{k}{s} V_{\text{in}}$$

$$\text{G Element: } I_{\text{out}} = \frac{k}{s} V_{\text{in}}$$

In the time domain:

$$\text{E Element: } v_{\text{out}} = k \int V_{\text{in}} dt$$

$$\text{G Element: } i_{\text{out}} = k \int V_{\text{in}} dt$$

For an integrator, the voltage gain transfer function is:

$$H(s) = \frac{V_{\text{out}}}{V_{\text{in}}} = \frac{k}{s}$$

In the general form of the transfer function:

$$H(s) = \frac{k_0 + k_1s + \dots + k_ns^n}{d_0 + d_1s + \dots + d_ms^m}$$

As in the previous example, if you set $k_0 = k$ and $d_1 = 1$, then the equation becomes:

$$H(s) = \frac{k + 0 + \dots + 0}{0 + s + \dots + 0} = \frac{k}{s}$$

Laplace Transform POLE (Pole/Zero) Function

This section describes the general form of the pole/zero transfer function. It also provides examples of converting specific transfer functions, into pole/zero circuit descriptions.

POLE Function Call

You can use the POLE function if the poles and zeros of the circuit are available. You can derive the poles and zeros from the transfer function, as described in this chapter, or you can use the .PZ statement to find them, as described in [.PZ \(Pole/Zero\) Statement on page 18-4](#).

Star-Hspice provides two forms of the LAPLACE function call: one for transconductance, and one for voltage gain transfer functions. See [Using G and E Elements](#) for the general forms, and for optional parameters.

To use the POLE pole/zero modeling function, find the a , b , f , and α coefficients of the transfer function. The transfer function is the s -domain (frequency domain) ratio of the output, for a single-source circuit to the input, with initial conditions set to zero.

General Form of the Transfer Function

The general expanded form of the pole/zero transfer function $H(s)$ is:

$$H(s) = \frac{a(s + \alpha_{z1} + j2\pi f_{z1})(s + \alpha_{z1} - j2\pi f_{z1}) \dots (s + \alpha_{zn} + j2\pi f_{zn})(s + \alpha_{zn} - j2\pi f_{zn})}{b(s + \alpha_{p1} + j2\pi f_{p1})(s + \alpha_{p1} - j2\pi f_{p1}) \dots (s + \alpha_{pm} + j2\pi f_{pm})(s + \alpha_{pm} - j2\pi f_{pm})} \quad (1)$$

You can use parameters to set the a , b , α , and f values.

The following is an example:

```
Ghigh_pass 0 out    POLE in 0 1.0    0.0,0.0 / 1.0    0.001,0.0
Elow_pass out 0     POLE in 0 1.0 / 1.0, 1.0,0.0    0.5,0.1379
```

The *Ghigh_pass* statement describes a high pass filter, with the transfer function:

$$H(s) = \frac{1.0 \cdot (s + 0.0 + j \cdot 0.0)}{1.0 \cdot (s + 0.001 + j \cdot 0.0)}$$

The *Elow_pass* statement describes a low-pass filter, with the transfer function:

$$H(s) = \frac{1.0}{1.0 \cdot (s + 1)(s + 0.5 + j2\pi \cdot 0.1379)(s + 0.5 - (j2\pi \cdot 0.1379))}$$

To write a pole/zero circuit description for an element, you need to know $H(s)$ transfer function of the element, in terms of the a , b , f , and α coefficients.

Before you use the values of these coefficients in POLE function calls (in the circuit description), you must simplify the transfer function, as described in the next section.

Reduced Form of the Transfer Function

Complex poles and zeros occur in conjugate pairs (a set of complex numbers differ only in the signs of their imaginary parts):

$$(s + \alpha_{pm} + j2\pi f_{pm})(s + \alpha_{pm} - j2\pi f_{pm}), \text{ for poles.}$$

$$(s + \alpha_{zn} + j2\pi f_{zn})(s + \alpha_{zn} - j2\pi f_{zn}), \text{ for zeros.}$$

To write the transfer function in pole/zero format, supply coefficients for one term of each conjugate pair. Star-Hspice provides the coefficients for the other term. If you omit the negative complex roots, the result is the reduced form of the transfer function, *Reduced*{ $H(s)$ }.

To find the reduced form, collect all general-form terms that have negative complex roots:

$$H(s) = \frac{a(s + \alpha_{z1} + j2\pi f_{z1}) \dots (s + \alpha_{zn} + j2\pi f_{zn})}{b(s + \alpha_{p1} + j2\pi f_{p1}) \dots (s + \alpha_{pm} + j2\pi f_{pm})} \cdot \frac{a(s + \alpha_{z1} - j2\pi f_{z1}) \dots (s + \alpha_{zn} - j2\pi f_{zn})}{b(s + \alpha_{p1} - j2\pi f_{p1}) \dots (s + \alpha_{pm} - j2\pi f_{pm})} \quad (1)$$

Then discard the right-hand term, which contains all terms with negative roots. What remains is the reduced form:

$$\text{Reduced}\{H(s)\} = \frac{a(s + \alpha_{z1} + j2\pi f_{z1}) \dots (s + \alpha_{zn} + j2\pi f_{zn})}{b(s + \alpha_{p1} + j2\pi f_{p1}) \dots (s + \alpha_{pm} + j2\pi f_{pm})} \quad (2)$$

For this function, find the a , b , f , and α coefficients to use in a POLE function, for a voltage-gain transfer function. The following examples show how to determine the coefficients, and write POLE function calls for a high-pass filter and a low-pass filter.

POLE Example 1 – Highpass Filter

For a high-pass filter with a transconductance transfer function, such as:

$$H(s) = \frac{s}{(s + 0.001)}$$

Find the a , b , α , and f coefficients needed to write the transfer function in the general form (1) shown previously. You can then see the conjugate pairs of complex roots. You need to supply only one of each conjugate pair of roots, in the Laplace function call. Star-Hspice automatically inserts the other root.

To transform the function into a form that is more similar to the general form of the transfer function, rewrite the transconductance transfer function as:

$$H(s) = \frac{1.0(s + 0.0)}{1.0(s + 0.001)}$$

Because this function has no negative imaginary parts, it is already in the Star-Hspice reduced form (2) shown previously.

You can now identify the a , b , f , and α coefficients, so that the $H(s)$ transfer function matches the reduced form. This matching process obtains the values:

$$\begin{aligned} n &= 1, m = 1, \\ a &= 1.0 \quad \alpha_{z1} = 0.0 \quad f_{z1} = 0.0 \\ b &= 1.0 \quad \alpha_{p1} = 0.001 \quad f_{p1} = 0.0 \end{aligned}$$

Using these coefficients in the reduced form, provides the transfer function:

$$\frac{s}{(s + 0.001)}.$$

So the general transconductance transfer function POLE function call:

$$\begin{aligned} &G_{xxx} \quad n+ \quad n- \quad \text{POLE} \quad in+ \quad in- \quad a \quad \alpha_{z1}, f_{z1} \dots \alpha_{zn}, f_{zn} / \\ &b \quad \alpha_{p1}, f_{p1} \dots \alpha_{pm}, f_{pm} \end{aligned}$$

for an element named *Ghigh_pass*, becomes:

$$\begin{aligned} &G_{high_pass} \quad gnd \quad out \quad \text{POLE} \quad in \quad gnd \quad 1.0 \quad 0.0, 0.0 / \\ &+ \quad 1.0 \quad 0.001, 0.0 \end{aligned}$$

POLE Example 2 – Low-Pass Filter

For a low-pass filter, with the following voltage-gain transfer function:

$$H(s) = \frac{1.0}{1.0(s + 1.0 + j2\pi \cdot 0.0)(s + 0.5 + j2\pi \cdot 0.15)(s + 0.5 - j2\pi \cdot 0.15)}$$

you need to find the a , b , α , and f coefficients, to write the transfer function in the general form, so that you can identify the complex roots with negative imaginary parts.

To separate the reduced form, $Reduced\{H(s)\}$, from the terms with negative imaginary parts, rewrite the voltage-gain transfer function as:

$$\begin{aligned} H(s) &= \frac{1.0}{1.0(s + 1.0 + j2\pi \cdot 0.0)(s + 0.5 + j2\pi \cdot 0.15)} \cdot \frac{1.0}{(s + 0.5 - j2\pi \cdot 0.15)} \\ &= Reduced\{H(s)\} \cdot \frac{1.0}{(s + 0.5 - j2\pi \cdot 0.15)} \end{aligned}$$

So:

$$\text{Reduced}\{H(s)\} = \frac{1.0}{1.0(s + 1.0)(s + 0.5 + j2\pi \cdot 0.15)}$$

or:

$$\frac{a(s + \alpha_{z1} + j2\pi f_{z1}) \dots (s + \alpha_{zn} + j2\pi f_{zn})}{b(s + \alpha_{p1} + j2\pi f_{p1}) \dots (s + \alpha_{pm} + j2\pi f_{pm})} = \frac{1.0}{1.0(s + 1.0 + j2\pi \cdot 0.0)(s + 0.5 + j2\pi \cdot 0.15)}$$

Now assign coefficients in the reduced form, to match the specified voltage transfer function. The following coefficient values produce the transfer function:

$$n = 0, m = 2,$$

$$a = 1.0 \quad b = 1.0 \quad \alpha_{p1} = 1.0 \quad f_{p1} = 0 \quad \alpha_{p2} = 0.5 \quad f_{p2} = 0.15$$

You can substitute these coefficients in the POLE function-call, for a voltage-gain transfer function:

$$\text{EXXX } n+ \quad n- \quad \text{POLE } in+ \quad in- \quad a \quad \alpha_{z1}, f_{z1} \dots \alpha_{zn}, f_{zn} \quad / \\ b \quad \alpha_{p1}, f_{p1} \dots \alpha_{pm}, f_{pm}$$

for an element named *Elow_pass*, to obtain the following statement:

$$\text{Elow_pass out GND POLE in } 1.0 \quad / \quad 1.0 \quad 1.0, 0.0 \quad 0.5, 0.15$$

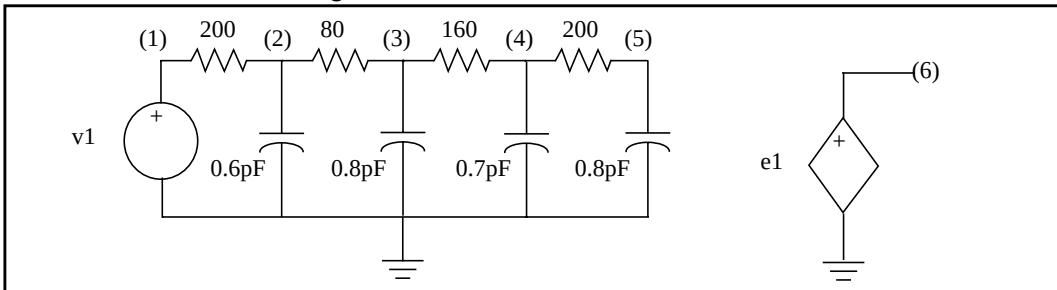
RC Line Modeling

Most RC lines can use very simple models, with only a single dominant pole. You can use AWE methods to find the dominant pole, computed based on the total series resistance and capacitance ([see 4 on page 20-51](#)), or determined using the *Elmore* delay ([see 5 on page 20-51](#)).

The *Elmore* delay uses the (d1-k1) value as the time constant, for a single-pole approximation to the complete $H(s)$, where $H(s)$ is the transfer function of the RC network for a specified output. The inverse Laplace transform of $h(t)$ is $H(s)$:

$$\tau_{DE} = \int_0^{\infty} t \cdot h(t) dt$$

Actually, the *Elmore* delay is the first moment of the impulse response, and so corresponds to a first-order AWE result.

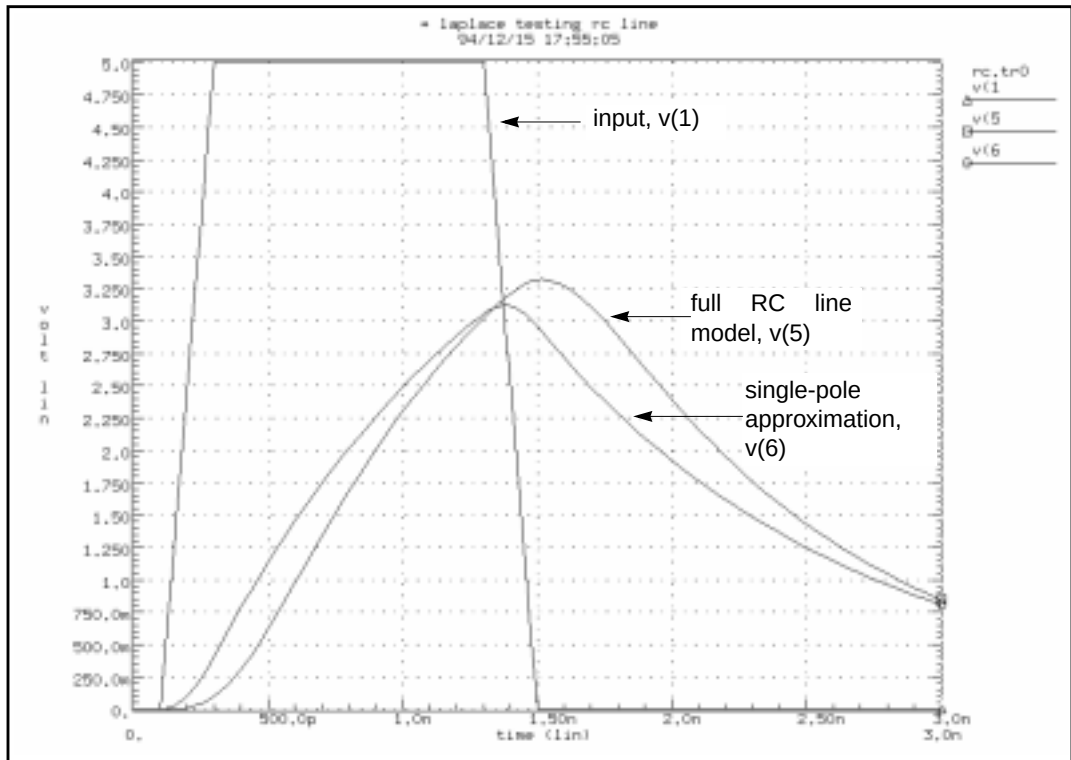
Figure 20-10: Circuits for an RC Line**RC Line Circuit File**

```

* Laplace testing RC line
.Tran 0.02ns 3ns
.OPTION Post Accurate List Probe
v1 1 0 PWL 0ns 0 0.1ns 0 0.3ns 5 1.3ns 5 1.5ns 0
r1 1 2 200
c1 2 0 0.6pF
r2 2 3 80
c2 3 0 0.8pF
r3 3 4 160
c3 4 0 0.7pF
r4 4 5 200
c4 5 0 0.8pF
e1 6 0 LAPLACE 1 0 1 / 1 1.16n
.Probe v(1) v(5) v(6)
.Print v(1) v(5) v(6)
.End

```

To closely approximate the output of the RC circuit (shown in [Figure 20-10](#)), you can use a single-pole response, as shown in [Figure 20-11](#) on page 20-33.

Figure 20-11: Transient Response of the RC Line and Single-Pole Approximation

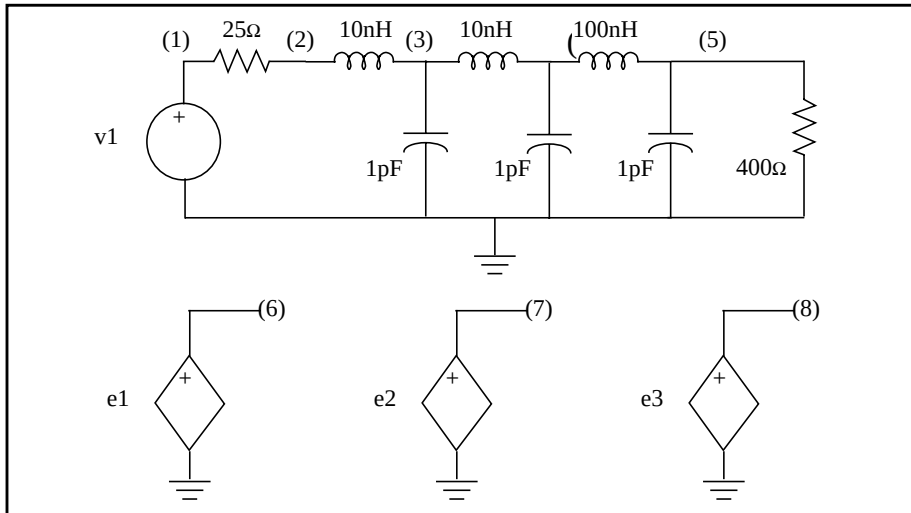
In [Figure 20-11](#), the single-pole approximation has less delay: 1 ns, compared to 1.1 ns for the full RC line model, at 2.5 volts. The single-pole approximation also has a lower peak value than the RC line model. All other things being equal, a circuit with a shorter time constant results in less filtering, and allows a higher maximum voltage value. The single-pole approximation produces a lower amplitude (and less delay) than the RC line, because the single pole neglects the other three poles, in the actual circuit. However, a single-pole approximation still provides very good results for many problems.

AWE Transfer Function Modeling

Approximations, using single-pole transfer functions, can cause larger errors for low-loss lines than for RC lines, because lower resistance allows ringing. Circuit ringing creates complex pole pairs in transfer function approximation. You need

at least one complex pole pair, to represent low-loss line response. Figure 20-12 is a typical low-loss line, and the transfer function sources used to test various approximations. To obtain the transfer functions, Star-Hspice uses asymptotic waveform evaluation (see 6 on page 20-51).

Figure 20-12: Circuits for a Low-Loss Line

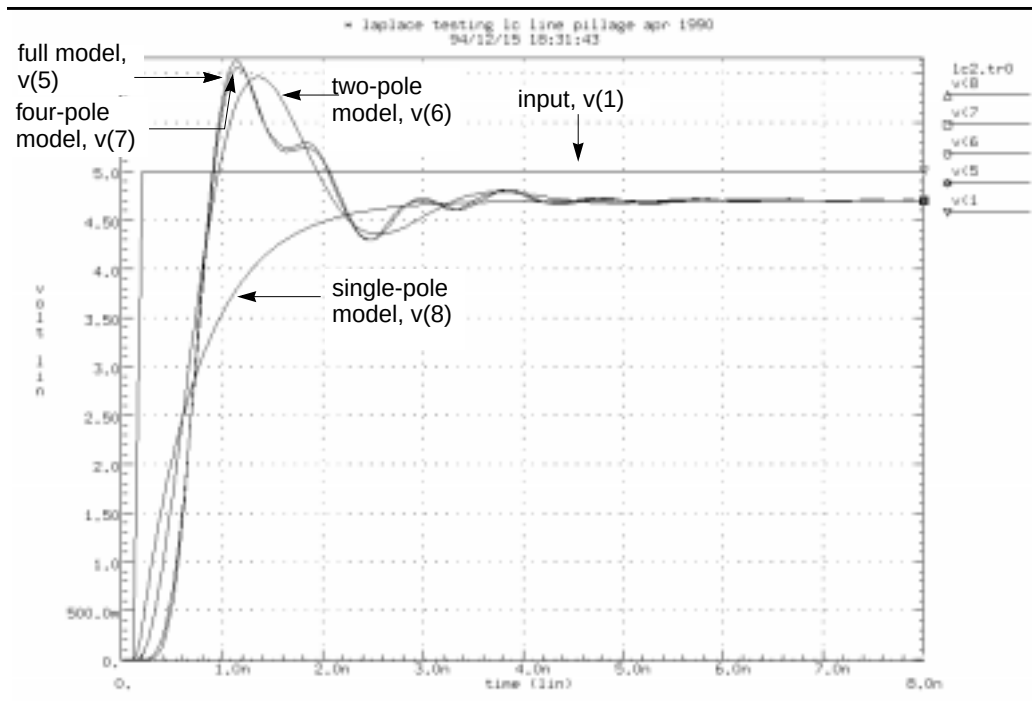


The following is a Low-Loss Line circuit file:

```
* Laplace testing LC line Pillage Apr 1990
.Tran 0.02ns 8ns
.OPTION Post Accurate List Probe
v1 1 0 PWL 0ns 0 0.1ns 0 0.2ns 5
r1 1 2 25
L1 2 3 10nH
c2 3 0 1pF
L2 3 4 10nH
c3 4 0 1pF
L3 4 5 100nH
c4 5 0 1pF
r4 5 0 400
e3 8 0 LAPLACE 1 0 0.94 / 1.0 0.6n
e2 7 0 LAPLACE 1 0 0.94e20 / 1.0e20 0.348e11 14.8
+ 1.06e-9 2.53e-19 SCALE=1.0e-20
e1 6 0 LAPLACE 1 0 0.94 / 1 0.2717e-9 0.12486e-18
.Probe v(1) v(5) v(6) v(7) v(8)
.Print v(1) v(5) v(6) v(7) v(8)
.End
```

Figure 20-13 on page 20-35 shows a transient response of a low-loss line. It also shows E Element Laplace models, using one, two, and four poles (see 6 on page 20-51). The single-pole model shows none of the ringing of the higher-order models. Also, all E models must adjust the gain of their response, for the finite load resistance, so the models are not independent of the load impedance. The 0.94-gain multiplier in the models take care of the 25-ohm source, and the 400-ohm load-voltage divider. These approximations are good delay estimations.

Figure 20-13: Transient Response of the Low-Loss Line



Although the two-pole approximation provides reasonable agreement with the transient overshoot, the four-pole model offers almost perfect agreement. The actual circuit has six poles. You can use scaling to bring some of the very small numbers in the Laplace model, above the 1e-28 limit of Star-Hspice. The SCALE parameter multiplies every parameter in the LAPLACE specification, by the same value (in this case $1.0\text{E}-20$).

A low-loss line allows reflections between the load and source, compared to the loss of an RC line, which usually isolates the source from the load. So you can either incorporate the load into the AWE transfer function approximation, or create a True-Hspice device model that allows source/load interaction. If you allow source/load interaction, you do not need to perform the AWE expansions each time that you change load impedances. This allows Star-Hspice to handle non-linear loads, and removes the need for a gain multiplier, as in the circuit file shown. You can use four voltage-controlled current sources, or G Elements, to create a Y parameter model for a transmission line. The Y parameter network provides the needed source/load interaction. The next example shows such a Y parameter model, for a low-loss line.

Y Parameter Line Modeling

A model that is independent of load impedance, is more complicated. You can still use AWE techniques, but you need a way for the load voltage and current to interact with the source impedance. For a transmission line of 100 ohms and 0.4 ns total delay (as shown in [Figure 20-14 on page 20-37](#)), to compare the response of the line, use a Y parameter model and a single-pole model.

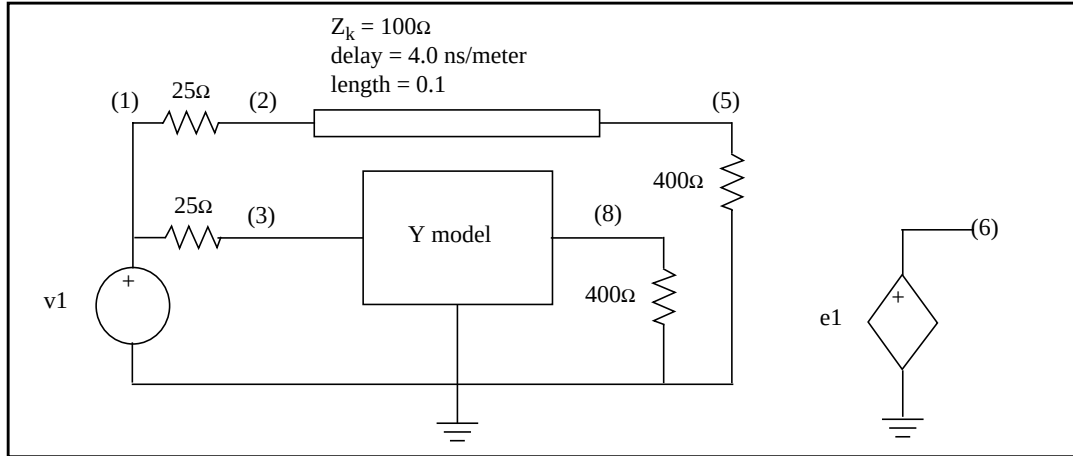
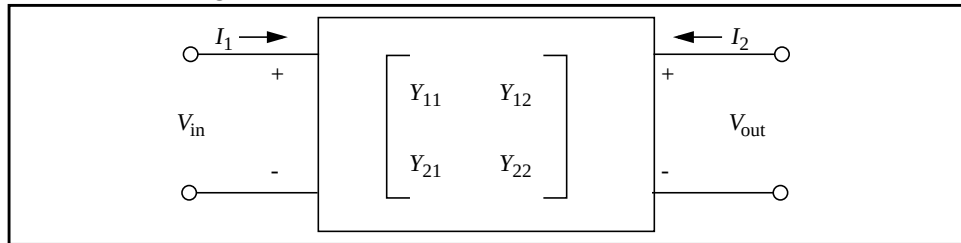
Figure 20-14: Line and Y Parameter Modeling

Figure 20-15 shows the voltage and current definitions for a Y parameter model.

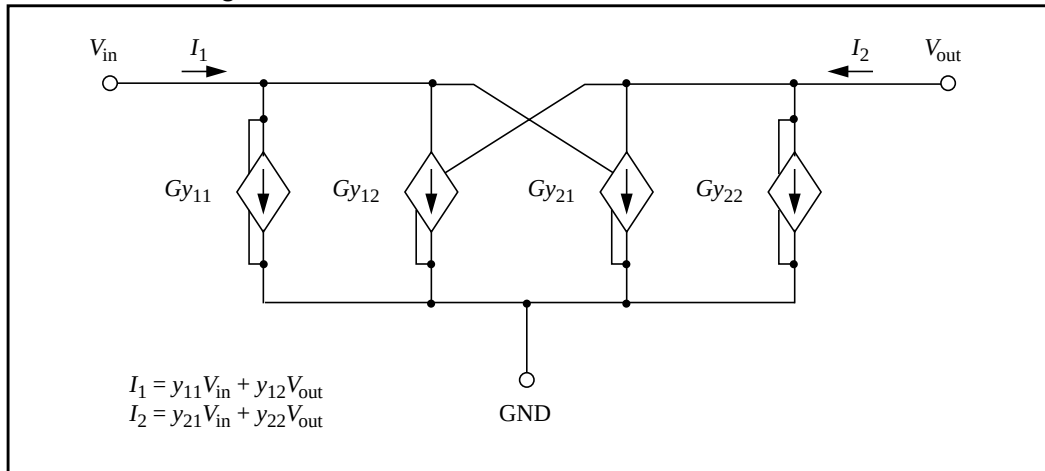
Figure 20-15: Y Matrix for the Two-Port Network

The following equations describe the general network in [Figure 20-15 on page 20-37](#), which you can translate into G Elements:

$$I_1 = Y_{11} V_{in} + Y_{12} V_{out}$$

$$I_2 = Y_{21} V_{in} + Y_{22} V_{out}$$

[Figure 20-16 on page 20-38](#) shows a schematic for a set of two-port Y parameters. The circuit consists mostly of G Elements.

Figure 20-16: Schematic for the Y Parameter Network

A Pade expansion of the Y parameters for a transmission line, determines the Laplace parameters for the Y parameter model, as shown in matrix form in the following equation:

$$Y = \frac{1}{Z_o} \cdot \begin{bmatrix} \coth(p) & -\operatorname{csch}(p) \\ -\operatorname{csch}(p) & \coth(p) \end{bmatrix},$$

where p is the product of the propagation constant, times the line length ([see 7 on page 20-51](#)).

A Pade approximation contains polynomials, in both the numerator and the denominator. A Pade approximation can model both poles and zeros, and $\coth$ and csch functions also contain both poles and zeros, so a Pade approximation provides a better low-order model, than a series approximation does.

The following equation calculates the Pade expansion of $\coth(p)$ and $\operatorname{csch}(p)$, with a second-order numerator and a third-order denominator:

$$\coth(p) \rightarrow \frac{\left(1 + \frac{2}{5} \cdot p^2\right)}{\left(p + \frac{1}{15} \cdot p^3\right)} \quad \operatorname{csch}(p) \rightarrow \frac{\left(1 - \frac{1}{20} \cdot p^2\right)}{\left(p + \frac{7}{60} \cdot p^3\right)}$$

When you substitute $(s \cdot \text{length} \cdot \sqrt{LC})$ for p , Star-Hspice generates polynomial expressions for each G Element. When you substitute 400 nH for L , 40 pF for C , 0.1 meter for length, and 100 for Z_0 ($Z_0 = \sqrt{L/C}$) in the matrix equation above, you can use the resulting values in a circuit file.

The circuit file shown below uses all of the above substitutions. The Pade approximations have different denominators for *csch* and *coth*, but the circuit file contains identical denominators. Although the actual denominators for *csch* and *coth* are only slightly different, using them can cause oscillations in the Star-Hspice response. To avoid this problem, use the same denominator in the *coth* and *csch* functions in the example. The simulation results might vary, depending on which denominator you use as the common denominator, because the coefficient of the third-order term changes (but by less than a factor of 2).

The following is an LC Line circuit file:

```
* Laplace testing LC line Pade
.Tran 0.02ns 5ns
.OPTION Post Accurate List Probe
v1 1 0 PWL 0ns 0 0.1ns 0 0.2ns 5
r1 1 2 25
r3 1 3 25
u1 2 0 5 0 wire1 L=0.1
r4 5 0 400
r8 8 0 400
e1 6 0 LAPLACE 1 0 1 / 1 0.4n

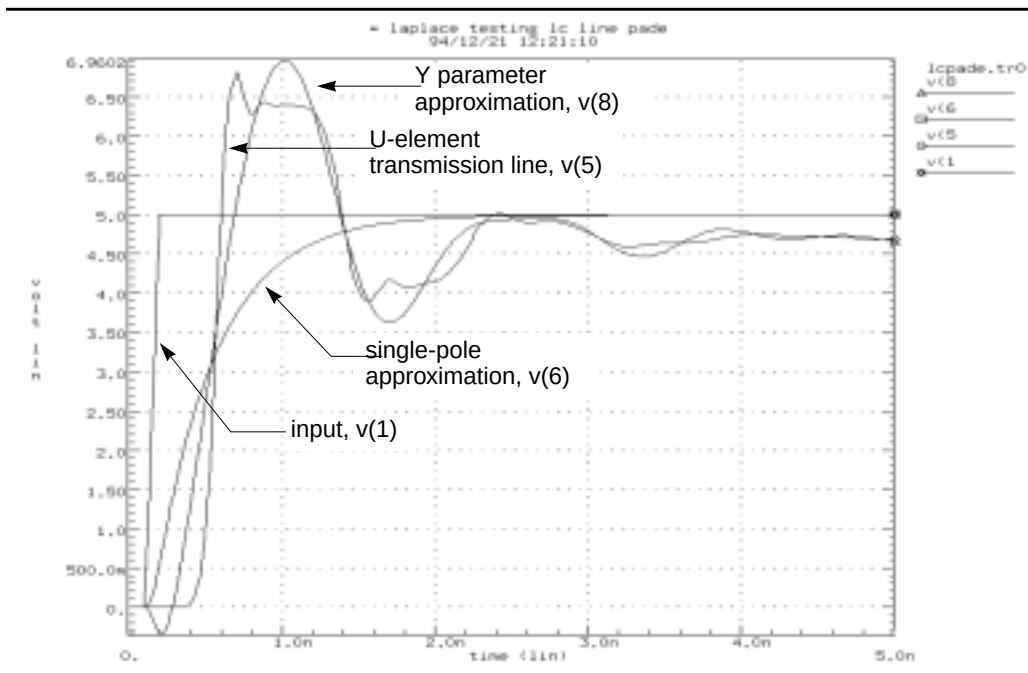
Gy11 3 0 LAPLACE 3 0 320016 0.0 2.048e-14 /
+ 0.0 0.0128 0.0 2.389e-22
Gy12 3 0 LAPLACE 8 0 -320016 0.0 2.56e-15 /
+ 0.0 0.0128 0.0 2.389e-22
Gy21 8 0 LAPLACE 3 0 -320016 0.0 2.56e-15 /
+ 0.0 0.0128 0.0 2.389e-22
Gy22 8 0 LAPLACE 8 0 320016 0.0 2.048e-14 /
+ 0.0 0.0128 0.0 2.389e-22

.model wire1 U Level=3 PLEV=1 ELEV=3 LLEV=0 MAXL=20
+ ZK=100 DELAY=4.0n

.Probe v(1) v(5) v(6) v(8)
.Print v(1) v(5) v(6) v(8)
.End
```

Figure 20-17 compares the output of the Y parameter model, with that of a full transmission line simulation, and with that obtained for a single-pole transfer function. In the latter case, the gain for the load impedance is incorrect, so the function produces an incorrect final voltage level. As expected, the Y parameter model provides the correct final voltage level. Although the Y parameter model provides a good approximation of the circuit delay, it contains too few poles to model all of the transient details. However, the Y parameter model provides excellent agreement with the overshoot and settling times.

Figure 20-17: Transient Response of the Y Parameter Line Model



Comparison of Circuit and Pole/Zero Models

This example simulates a ninth-order, low-pass filter circuit, and compares the results with its equivalent pole/zero description, using an E Element. The results are identical, but the pole/zero model runs about 40% faster. [Simulation Time Summary on page 20-42](#) shows the total CPU times for the two methods. For larger circuits, the computation time saving can be much higher.

The input listings for each model type are shown below. [Figure 20-18 on page 20-43](#) and [Figure 20-19 on page 20-43](#) display the transient and frequency response comparisons, resulting from the two modeling methods.

Circuit Model Input Listing

```
low_pass9a.sp 9th order low_pass filter.
* Reference: Jiri Vlach and Kishore Singhal, "Computer *
Methods for Circuit Analysis and Design", Van Nostrand
* Reinhold Co., 1983, pages 142, 494-496.
.PARAM freq=100 tstop='2.0/freq'
*.PZ v(out) vin
.AC dec 50 .1k 100k
.OPTION dcstep=1e3 post probe unwrap
.PROBE ac vdb(out) vp(out)
.TRAN STEP='tstop/200' STOP=tstop
.PROBE v(out)
vin in GND sin(0,1,freq) ac 1
.SUBCKT fdnr 1 r1=2k c1=12n r4=4.5k
r1 1 2 r1
c1 2 3 c1
r2 3 4 3.3k
r3 4 5 3.3k
r4 5 6 r4
c2 6 0 10n
eop1 5 0 opamp 2 4
eop2 3 0 opamp 6 4
.ENDS
*
rs in 1 5.4779k
r12 1 2 4.44k
r23 2 3 3.2201k
r34 3 4 3.63678k
r45 4 out 1.2201k
c5 out 0 10n
x1 1 fdnr r1=2.0076k c1=12n r4=4.5898k
x2 2 fdnr r1=5.9999k c1=6.8n r4=4.25725k
x3 3 fdnr r1=5.88327k c1=4.7n r4=5.62599k
x4 4 fdnr r1=1.0301k c1=6.8n r4=5.808498k
.END
```

Pole/Zero Model Input Listing

```
ninth.sp 9th order low_pass filter.
.PARAM twopi=6.2831853072
.PARAM freq=100 tstop='2.0/freq'
.AC dec 50 .1k 100k
.OPTION dcstep=1e3 post probe unwrap
.PROBE ac vdb(outp) vp(outp)
.TRAN STEP='tstop/200' STOP=tstop
.PROBE v(outp)
vin in GND sin(0,1,freq) ac 1
Epole outp GND POLE in GND 417.6153
+ 0. 3.8188k
+ 0. 4.0352k
+ 0. 4.7862k
+ 0. 7.8903k / 1.0
+ '73.0669*twopi' 3.5400k
+ '289.3438*twopi' 3.4362k
+ '755.0697*twopi' 3.0945k
+ '1.5793k*twopi' 2.1105k
+ '2.1418k*twopi' 0.
repole outp GND 1e12
.END
```

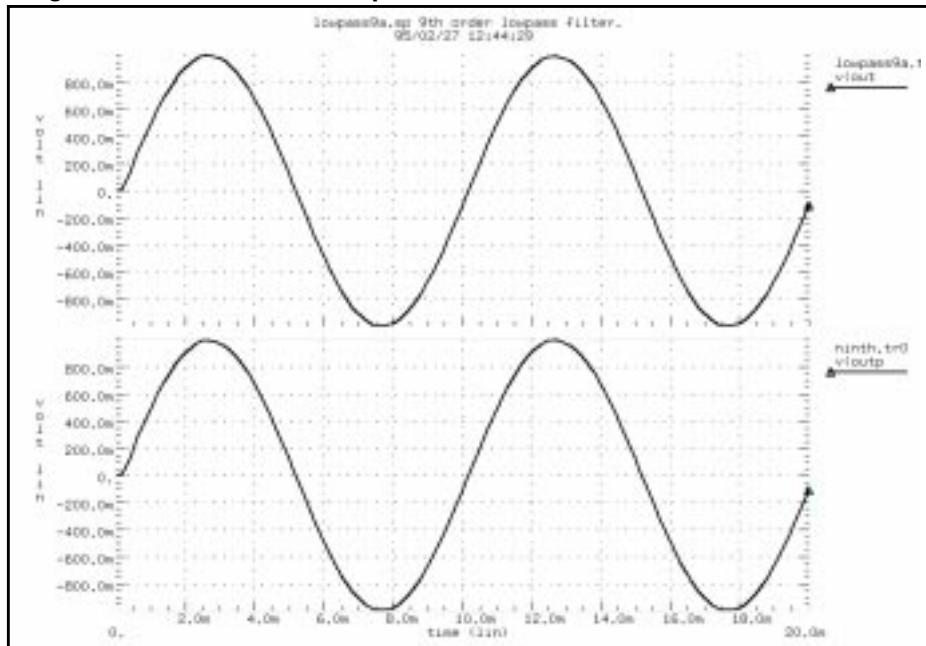
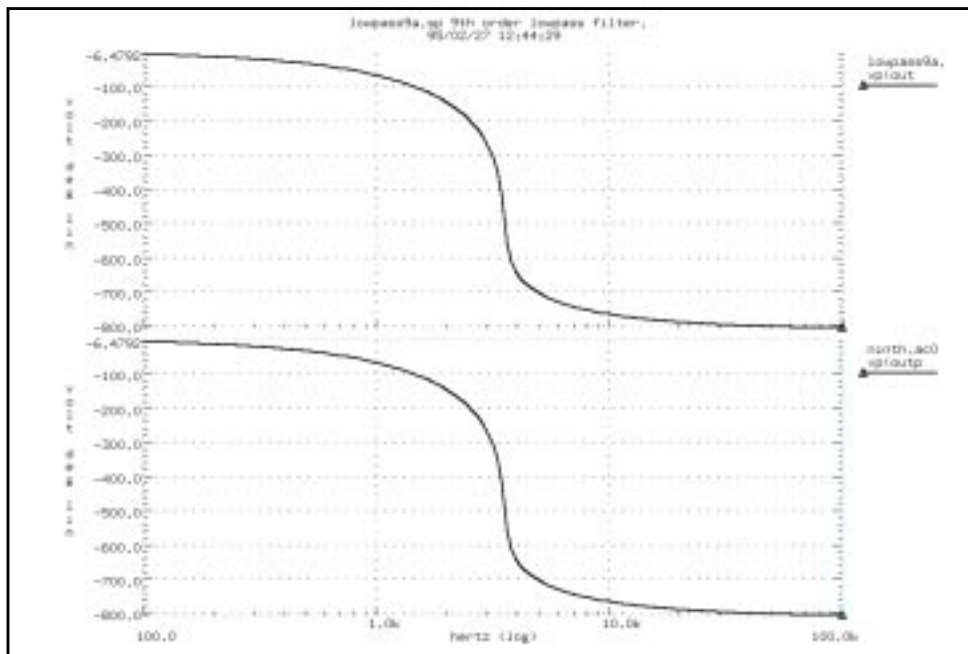
Simulation Time Summary

Circuit model simulation times:

analysis	time	# points	# iter	conv.iter
op point	0.23	1	3	
ac analysis	0.47	151	151	
transient	0.75	201	226	113 rev=0
readin	0.22			
errchk	0.13			
setup	0.10			
output	0.00			
total cpu time 1.98 seconds				

Pole/zero model simulation times:

analysis	time	# points	# iter	conv.iter
op point	0.12	1	3	
ac analysis	0.22	151	151	
transient	0.40	201	222	111 rev=0
readin	0.23			
errchk	0.13			
setup	0.02			
output	0.00			
total cpu time 1.23 seconds				

Figure 20-18: Transient Responses of the Circuit and Pole/Zero Models**Figure 20-19: AC Analysis Responses of the Circuit and Pole/Zero Models**

Modeling Switched Capacitor Filters

Switched Capacitor Network

You can model a resistor as a capacitor and switch combination. The value of the equivalent is proportional to the frequency of the switch, divided by the capacitance.

Construct a filter from MOSFETs and capacitors, where the filter characteristics are a function of the switching frequency of the MOSFETs.

To quickly determine the filter characteristics, use ideal switches (voltage controlled resistors), instead of MOSFETs. The resulting simulation speed-up can be as great as 7 to 10 times faster than a circuit using MOSFETs.

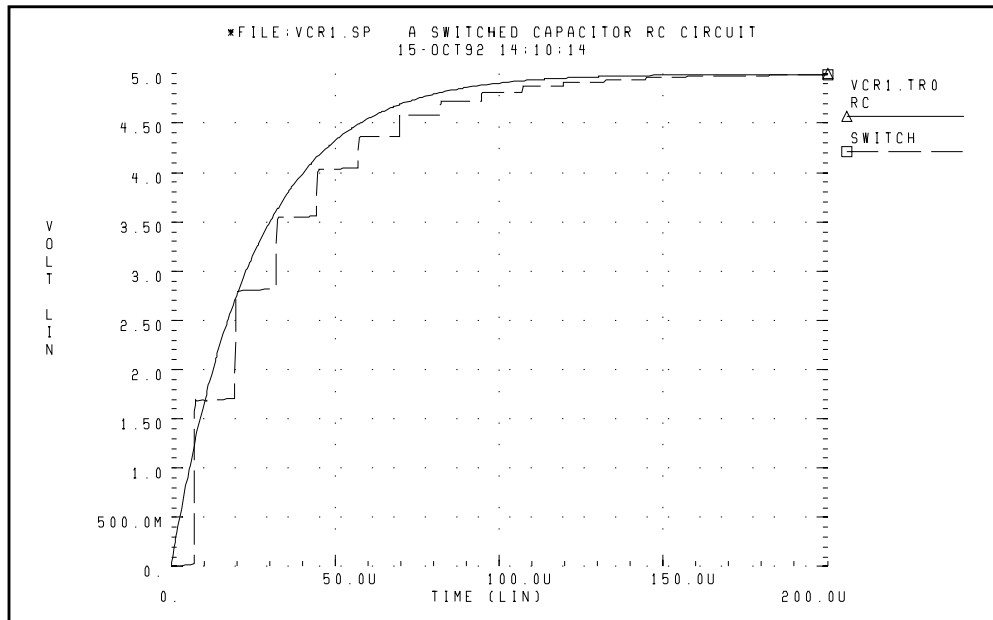
To construct an RC network, the model uses a resistor and a capacitor, along with a switched-capacitor equivalent network. The RCOUT node is the resistor/capacitor output, and VCROUT is the switched-capacitor output.

The GVCR1 and GVCR2 switches, and the C3 capacitance, model the resistor. The following equation calculates the resistor value:

$$R_{\text{res}} = \frac{T_{\text{switch}}}{C3}$$

where T_{switch} is the period of the PHI1 and PHI2 pulses.

Figure 20-20: VCR1.SP Switched Capacitor RC Circuit



Switched Capacitor Network Example

```

*FILE:VCR1.SP  A SWITCHED CAPACITOR RC CIRCUIT
.OPTION acct NOMOD POST
.IC V(SW1)=0 V(RCOUT)=0 V(VCROUT)=0
.TRAN 5U 200U
.GRAPH RC=V(RCOUT) SWITCH=V(VCROUT) (0,5)
VCC  VCC      GND      5V
C     RCOUT   GND      1NF
R     VCC     RCOUT    25K
C6    VCROUT  GND      1NF

* equivalent circuit for 25k resistor r=12.5us/.5nf
VA  PHI1 GND  PULSE 0 5 1US .5US .5US 3US 12.5US
VB  PHI2 GND  PULSE 0 5 7US .5US .5US 3US 12.5US
GVCR1 VCC SW1 PHI1 GND LEVEL=1 MIN=100 MAX=1MEG 1.MEG
+ -.5MEG
GVCR2 SW1 VCROUT PHI2 GND LEVEL=1 MIN=100 MAX=1MEG
+ 1.MEG .5MEG
C3    SW1     GND     .5NF
.END

```

Switched Capacitor Filter Example

This example is a fifth-order elliptic, switched-capacitor filter. The passband is 0-1 kHz, with loss less than 0.05 dB. This results from cascading models of the switches. The resistance is 1 ohm when the switch is closed, and 100 Megohm when it is open. The E Element models op-amps as an ideal op-amp. This example provides the transient response of the filter, for 1 kHz and 2 kHz sinusoidal input signals ([see 8 on page 20-51](#)).

Figure 20-21: Linear Section

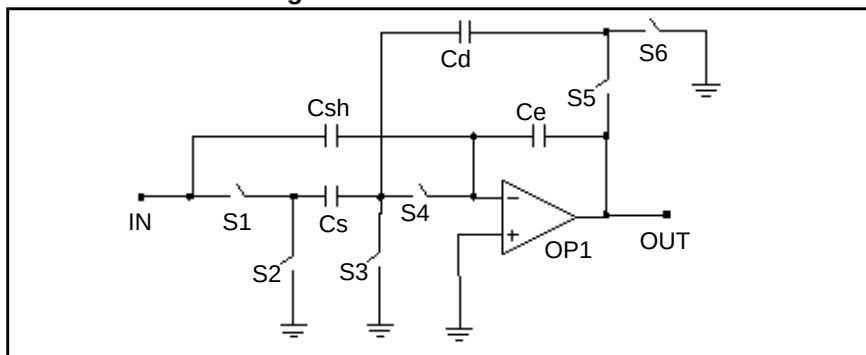


Figure 20-22: High_Q Biquad Section

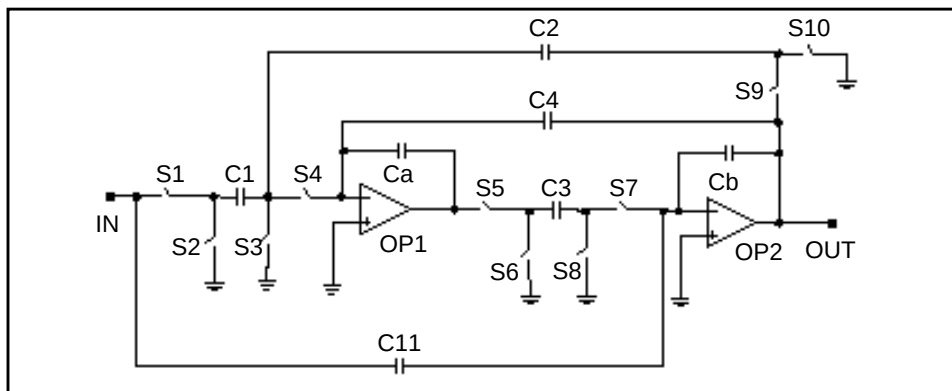
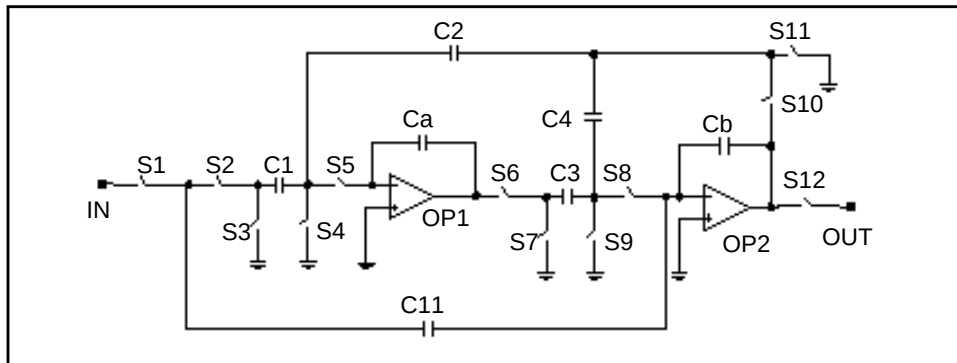


Figure 20-23: Low_Q Biquad Section



Input File for Switched Capacitor Filter

```

SWCAP5.SP Fifth Order Elliptic Switched Capacitor
+ Filter.
.OPTION POST PROBE
.GLOBAL phi1 phi2
.TRAN 2u 3.2m UIC
*.GRAPH v(phi1) v(phi2) V(in)
.PROBE V(out)
*.PLOT v(in) v(phi1) v(phi2) v(out)
*Iin 0 in SIN(0,1ma,1.0khz)
Iin 0 in SIN(0,1v,2khz)
Vphi1 phi1 0 PULSE(0,2 00u,.5u,.5u,7u,20u)
Vphi2 phi2 0 PULSE(0,2 10u,.5u,.5u,7u,20u)
Rsrc in 0 1k
Rload out 0 1k
Xsh in out1 sh
Xlin out1 out2 linear
Xhq out2 out3 hqbiq
Xlq out3 out4 lqbiq

```

Sample and Hold

```

.SUBCKT sh in out
Gs1 in 1 VCR PWL(1) phi1 0 0.5v,100meg 1.0v,1.0
Eop1 out 0 OPAMP 1 out
Ch 1 0 1.0pf
.ENDS

```

Linear Section

```
.SUBCKT linear in out
Gs1 in 1 VCR PWL(1) phi1 0 0.5v,100meg 1.0v,1.0
Gs2 1 0 VCR PWL(1) phi2 0 0.5v,100meg 1.0v,1.0
Cs 1 2 1.0pf
Gs3 2 0 VCR PWL(1) phi1 0 0.5v,100meg 1.0v,1.0
Gs4 2 3 VCR PWL(1) phi2 0 0.5v,100meg 1.0v,1.0
Eop1 out 0 OPAMP 0 3
Ce out 3 9.6725pf
Gs5 out 4 VCR PWL(1) phi2 0 0.5v,100meg 1.0v,1.0
Gs6 4 0 VCR PWL(1) phi1 0 0.5v,100meg 1.0v,1.0
Cd 4 2 1.0pf
Csh in 3 0.5pf
.ENDS
```

High_Q Biquad Section

```
.SUBCKT hqbiq in out
Gs1 in 1 VCR PWL(1) phi2 0 0.5v,100meg 1.0v,1.0
Gs2 1 0 VCR PWL(1) phi1 0 0.5v,100meg 1.0v,1.0
C1 1 2 0.5pf
Gs3 2 0 VCR PWL(1) phi1 0 0.5v,100meg 1.0v,1.0
Gs4 2 3 VCR PWL(1) phi2 0 0.5v,100meg 1.0v,1.0
Eop1 4 0 OPAMP 0 3
Ca 3 4 7.072pf
Gs5 4 5 VCR PWL(1) phi1 0 0.5v,100meg 1.0v,1.0
Gs6 5 0 VCR PWL(1) phi2 0 0.5v,100meg 1.0v,1.0
C3 5 6 0.59075pf
Gs7 6 7 VCR PWL(1) phi2 0 0.5v,100meg 1.0v,1.0
Gs8 6 0 VCR PWL(1) phi1 0 0.5v,100meg 1.0v,1.0
Eop2 out 0 OPAMP 0 7
Cb 7 out 4.3733pf
Gs9 out 8 VCR PWL(1) phi2 0 0.5v,100meg 1.0v,1.0
Gs10 8 0 VCR PWL(1) phi1 0 0.5v,100meg 1.0v,1.0
C4 out 3 1.6518pf
C2 8 2 0.9963pf
C11 7 in 0.5pf
.ENDS
```

Low_Q Biquad Section

```
.SUBCKT lqbiq in out
Gs1 in 1 VCR PWL(1) phi2 0 0.5v,100meg 1.0v,1.0
Gs2 1 2 VCR PWL(1) phi2 0 0.5v,100meg 1.0v,1.0
C1 2 3 0.9963pf
Gs3 2 0 VCR PWL(1) phi1 0 0.5v,100meg 1.0v,1.0
Gs4 3 0 VCR PWL(1) phi1 0 0.5v,100meg 1.0v,1.0
Gs5 3 4 VCR PWL(1) phi2 0 0.5v,100meg 1.0v,1.0
```

```

Ca  4 5 8.833pf
Eop1 5 0 OPAMP 0 4
Gs6 5 6 VCR PWL(1) phi1 0 0.5v,100meg 1.0v,1.0
Gs7 6 0 VCR PWL(1) phi2 0 0.5v,100meg 1.0v,1.0
C3  6 7 1.0558pf
Gs8 7 8 VCR PWL(1) phi2 0 0.5v,100meg 1.0v,1.0
Gs9 7 0 VCR PWL(1) phi1 0 0.5v,100meg 1.0v,1.0
Eop2 9 0 OPAMP 0 8
Cb  8 9 3.8643pf
Gs10 9 10 VCR PWL(1) phi2 0 0.5v,100meg 1.0v,1.0
Gs11 10 0 VCR PWL(1) phi1 0 0.5v,100meg 1.0v,1.0
C4  10 7 0.5pf
C2  10 3 0.5pf
C11 8 1 3.15425pf
Gs12 9 out VCR PWL(1) phi2 0 0.5v,100meg 1.0v,1.0
.ENDS
.END

```

Figure 20-24: Response to 1-kHz Sinusoidal Input

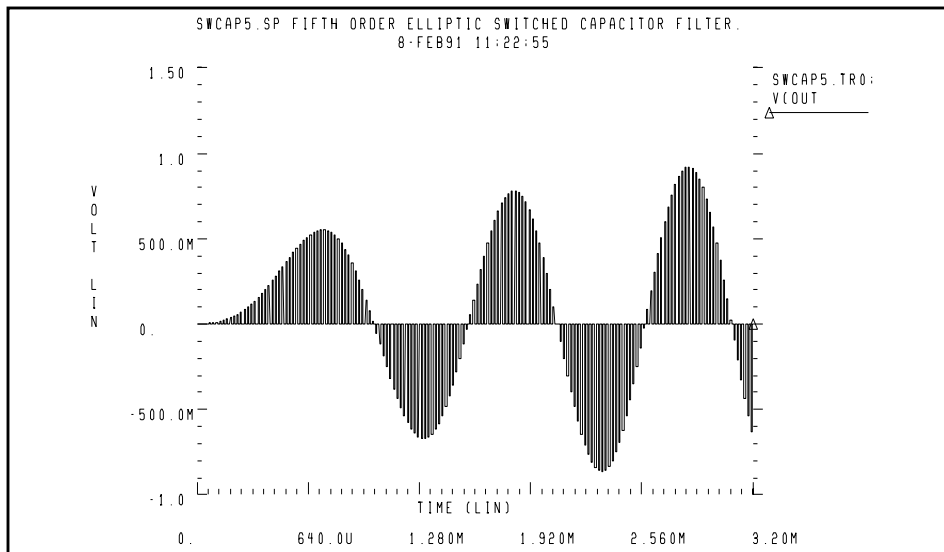
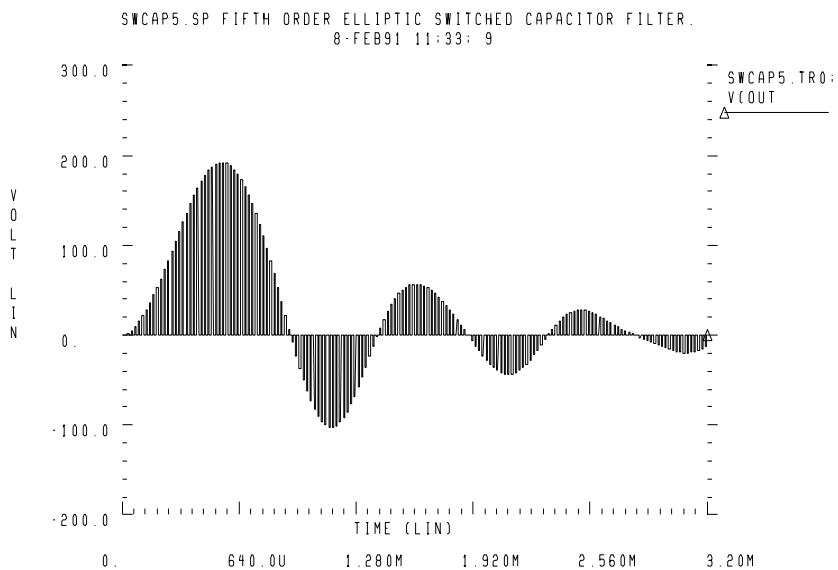


Figure 20-25: Response to 2-kHz Sinusoidal Input

References

1. Williams, Arthur B., and Taylor, Fred J. *Electronic Filter Design Handbook*. New York: McGraw-Hill, 1988, pp. 6-20 to 6-23.
2. Nillson, James W. *Electric Circuits*, 4th Edition. Reading, Massachusetts: Addison-Wesley, 1993.
3. Edminister, Joseph A. *Electric Circuits*. New York: McGraw-Hill, 1965.
4. Ghausi, Kelly, and M.S. "On the Effective Dominant Pole of the Distributed RC Networks", *Jour. Franklin Inst.*, June 1965, pp. 417- 429.
5. Elmore, W.C. and Sands, M. *Electronics, National Nuclear Energy Series*, New York: McGraw-Hill, 1949.
6. Pillage, L.T. and Rohrer, R.A. "Asymptotic Waveform Evaluation for Timing Analysis", *IEEE Trans. CAD*, Apr. 1990, pp. 352 - 366.
7. Kuo, F. F. *Network Analysis and Synthesis*. John Wiley and Sons, 1966.
8. Gregorian, Roubik & Temes, Gabor C. *Analog MOS Integrated Circuits*. J. Wiley, 1986, page 354.



Chapter 21

Timing Analysis Using Bisection

To analyze circuit timing violations, a typical methodology is to generate a set of operational parameters, which produce a failure in the required behavior of the circuit. When a circuit timing failure occurs, you can identify a timing constraint, which can lead to a design guideline. You must perform an iterative analysis, to define the violation specification.

Typical types of timing constraint violations include:

- Data setup time, before the clock.
- Data hold time, after the clock.
- Minimum pulse width required, for a signal to propagate to the output.
- Maximum toggle frequency of the component(s).

This chapter describes how to use the bisection function, in timing optimization. For more information about optimization, see [Statistical Analysis and Optimization on page 13-1](#).

This chapter explains the following topics:

- [Overview of Bisection](#)
- [Bisection Methodology](#)
- [Using Bisection](#)
- [Setup Time Analysis](#)
- [Minimum Pulse Width Analysis](#)

Overview of Bisection

Before bisection methods were developed, engineers built external drivers to submit multiple parameterized simulations to SPICE-type simulators. Each simulation explored a region of the operating envelope for the circuit. To provide part of the analysis, the driver also post-processed the simulation results, to deduce the limiting conditions.

If you characterize small circuits this way, analysis times are relatively small, compared with the overall job time. This method is inefficient, due to overhead of submitting the job, reading and checking the netlist, and setting up the matrix. The newer bisection methods increase efficiency when you analyze timing violations, to find the causes of timing failure. Bisection optimization is an efficient cell-characterization method, in Star-Hspice.

The bisection methodology saves time in three ways:

- Reduces multiple jobs to a single characterization job.
- Removes post-processing requirements.
- Uses accuracy-driven iterations.

Figure 21-1 on page 21-3 shows a typical analysis of setup-time constraints. Clock and data input waveforms drive a cell. Two input transitions (rise and fall) occur at times T_1 and T_2 . The result is an output transition, when $V(out)$ changes from low to high. The following relationship between the T_1 (data) and T_2 (clock) times must be true, for the $V(out)$ transition to occur:

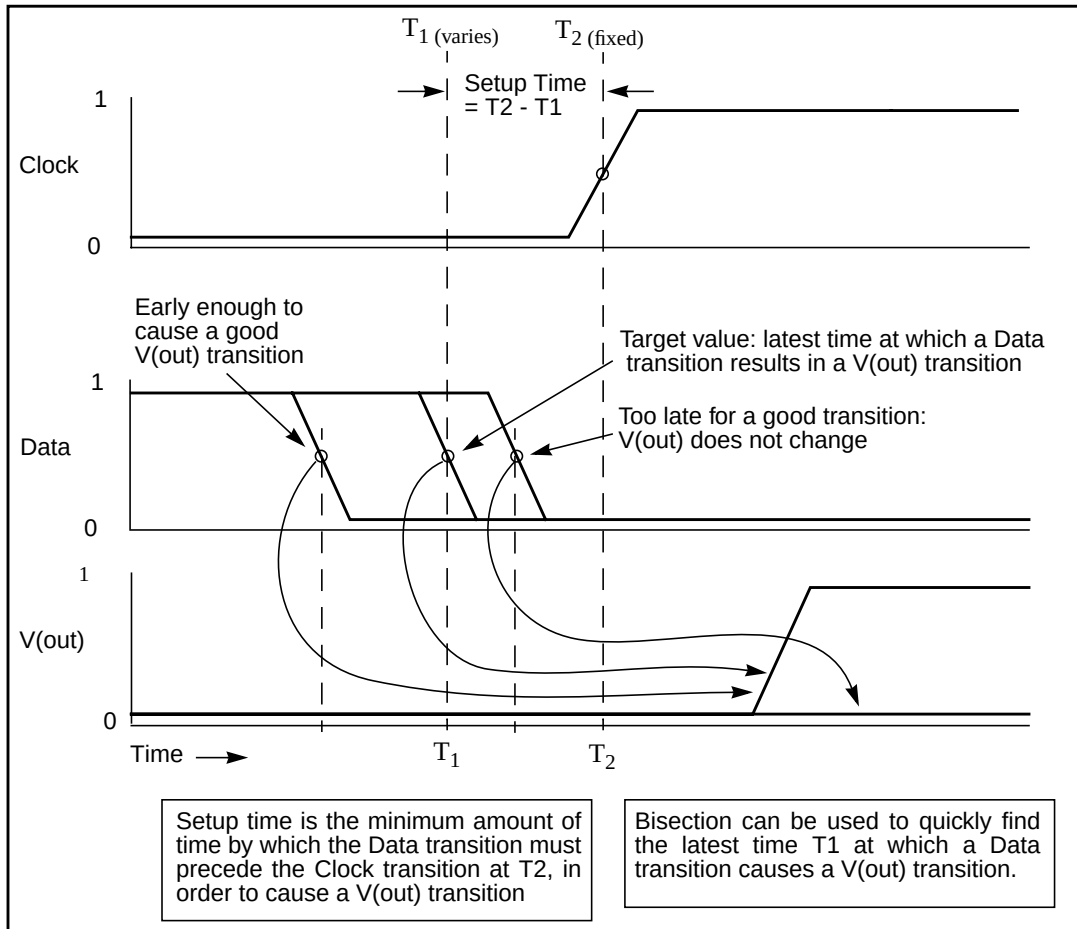
$$T_2 > (T_1 + \text{setup time})$$

Characterization, or violation analysis, determines the setup time. To do this, Star-Hspice keeps T_2 fixed, repeats the simulation with different T_1 values. It then observes which T_1 values produce an output transition, and which do not.

Before bisection, you had to run tight sweeps of the delay, between the data setup and clock edge, looking for the value at which no transition occurs. To do this, you swept a value that specifies how far the data edge precedes a fixed clock edge. This method is time consuming, and is accurate *only* if the sweep step is very small. Linear search methods cannot accurately determine the setup time value, unless you use extremely small steps from T_1 to T_2 to simulate the circuit at each point, and monitor the outcome.

For example, even if you know that the desired transition occurs during a particular five-nanosecond period, you might need to run 50 simulations to search for the setup time, to within 0.1 nanoseconds, over that five nanosecond period. Even after this, the error in the result can be as large as 0.05 nanoseconds.

Figure 21-1: Determining Setup Time with Bisection Violation Analysis



The bisection feature greatly reduces the amount of work and computational time required, to find an accurate solution for this type of problem. The following pages show examples of using this feature, to identify timing violations for the setup, hold, and minimum clock pulse width.

Bisection Methodology

Bisection is an optimization method that uses a binary search method, to find the value of an input variable (target value). This variable is associated with a *goal* value of an output variable.

The type of the input and output variables can be voltage, current, delay time, or gain, related by some transfer function. In general, use a binary search to locate the goal value of the output variable, within a search range of the input variable. Then iteratively halve that range, to rapidly converge on the target value. At each iteration, Star-Hspice compares the *measured value* of the output variable, with the goal value. Both the *pass/fail* method and the *bisection* method use bisection (see [Using Bisection on page 21-5](#)).

The bisection procedure involves two measurement and optimization steps, when solving the timing violation problem:

1. Detects whether the output transition occurred.
2. Automatically varies the input parameter (T_1 in [Figure 21-1](#)), to find the value for which the transition barely occurs.

Measurement

Use the MAX measurement function, to detect the success or failure of an output transition. For a low-to-high output transition, a MAX measurement produces zero on failure, or approximately the V_{dd} supply voltage on success. This measurement, using a goal of V_{dd} (minus a suitable small value to ensure a solution), is sufficient to drive the optimization.

Optimization

The bisection method is straightforward, if you specify a single measurement, with a goal, and known upper and lower boundary values, for the input parameter. The characterization engineer must specify acceptable upper and lower boundary values.

Using Bisection

Before you can use bisection, you must specify the following:

- A pair of values, for the upper and lower boundaries of the input variables. To find a solution, one of these values must result in an output variable $|\text{goal value}|$ and the other must result in $< |\text{goal value}|$.
- A goal value.
- Error tolerance value. The bisection process stops, when the difference between successive test values $\leq$ error tolerance. If the other criteria are also met, see below.
- Related variables. Use a monotonic transfer function to relate variables, where a steadily-progressing time (increase or decrease) results in a single occurrence of the *goal* value at the *target* input variable value.

Star-Hspice includes the error tolerance in a relation, used as a process-termination criterion.

[Figure 21-2 on page 21-11](#) shows an example of the binary search process, which the bisection algorithm uses. This example is the *pass/fail* type, and is appropriate for a setup-time analysis that tests for the presence of an output transition (see [Figure 21-1 on page 21-3](#)). In this example:

1. A long setup time $T_S (= T_2 - T_1)$ results in a V_{OUT} transition (a *pass*).
2. A too-short setup time (where the latch has not stabilized the input data, before the clock transition) results in a *fail*.
3. For example, you might define a *pass* time value as any setup time, T_S , that produces a V_{OUT} output *minimum high* logic output level of 2.7 V, which is the *goal* value.
4. The *target* value is that setup time that *just* produces the V_{OUT} value of 2.7 V. Finding the exact value is impractical, if not impossible, so you must specify an error tolerance, to calculate a solution that is arbitrarily close to the target value.
5. The bisection algorithm performs tests for each specified boundary value, to determine the direction in which to pursue the target value, after the first bisection. In this example, the upper boundary has a *pass* value, and the lower boundary has a *fail* value.

6. To start the binary search, specify the lower and upper boundaries.

The program tests the point midway between the lower and upper boundaries (see [Figure 21-2](#)).

- If the initial value passes the test, the target value must be less than the tested value (in this example). The bisection algorithm moves the upper search limit to the value that it just tested.
- If the test fails, the target value must be greater than the tested value. Bisection moves the lower limit to the value that it just tested.

7. The algorithm tests a value midway between the new limits.
8. The search continues in this manner, moving one limit or the other to the last midpoint, and testing the value midway between the new limits.
9. The process stops when the difference between the latest test values is less than or equal to the error tolerance that you specified. To normalize this value, multiply by the initial boundary range.

Command Syntax

```
.MODEL <OptModelName> OPT METHOD=BISECTION ...
.MODEL <OptModelName> OPT METHOD=PASSFAIL ...
```

OptModel- The model to use. Refer to [Statistical Analysis and Optimization on page 13-1](#) for *Name* information about specifying optimization models.

METHOD Keyword, to indicate the optimization method to use. For bisection, the method can be one of the following:

BISECTION

If the difference between the two latest test input values is within the error tolerance, and the latest measured value exceeds the goal, then bisection succeeds and stops. The process reports the optimized parameter, which corresponds to the test value that satisfies this error tolerance, and this goal (passes).

PASSFAIL

If the difference between the two latest test input values is within the error tolerance, and one of the values $\geq$ goal (passes) and the other fails, then bisection succeeds and stops. The process reports the input parameter value associated with the *pass* measurement.

OPT Keyword, to perform optimization.

A normal optimization specification passes the parameters:

```
.PARAM <ParamName>=<OptParFun> (<Initial>, <Lower>, <Upper>)
```

In the BISECTION method, the measure results for <Lower> and <Upper> limits of the <ParamName>, must be on opposite sides of the GOAL value in the .MEASURE statement. For the PASSFAIL method, the measure must pass for one limit, and fail for the other limit. Bisection ignores the <Initial> value.

The error tolerance is a parameter, in the model to optimize.

The bisectional search applies to only one parameter.

When you set the OPTLST option (.OPTION OPTLST=1), the process prints the following information for the BISECTION method:

```
bisec-opt iter = <num_iterations> xlo = <low_val>
+ xhi = <high_val>
x = <result_low_val> xnew = <result_high_val>
err = <error_tolerance>
```

where *x* is the old parameter value, and *xnew* is the new parameter value.

When .OPTION OPTLST=1, the process prints the following information for the PASSFAIL method:

```
bisec-opt iter = <num_iterations> xlo = <low_val>
+ xhi = <high_val>
x = <result_low_val> xnew = <result_high_val>
measfail = 1
```

(measfail = 0 for a test failure for the *x* value).

Examples

transient analysis .TRAN statement:

```
.TRAN <TranStep> <TranTime> SWEEP OPTIMIZE=<OptParFun>  
+ RESULTS=<MeasureName> MODEL=<OptModelName>
```

transient .MEASURE statement:

```
.MEASURE TRAN <MeasureName> <MeasureClause>  
+ GOAL=<GoalValue>
```


Setup Time Analysis

This example uses a bisectional search, to find the minimum setup time for a D flip-flop. The circuit for this example is `/bisect/dff_top.sp` in the Star-Hspice `$installdir/demo/hspice` demonstration file directory. The files in [Figure 21-2](#) and [Figure 21-3](#) show the results of this demo. Star-Hspice does not directly optimize the setup time, but extracts it from its relationship with the DelayTime parameter (the time before the data signal), which is the parameter to optimize.

Input Listing

```
File: $installdir/demo/hspice/bisect/dff_top.sp
* DFF_top Bisection Search for Setup Time
* PWL Stimulus
v28 data gnd PWL
+ 0s      5v
+ 1n      5v
+ 2n      0v
+ Td = "DelayTime"    $ Offsets Data from time 0
+ by DelayTime
v27 clock gnd PWL
+ 0s      0v
+ 3n      0v
+ 4n      5v
* Specify DelayTime as the search parameter and provide
* the lower and upper limits.
.PARAM DelayTime= Opt1 ( 0.0n, 0.0n, 5.0n )
* Transient simulation with Bisection Optimization
.TRAN 1n 8n Sweep      Optimize = Opt1
+                      Result    = MaxVout    $ Look at measure
+                      Model     = OptMod
* This measure finds the transition if it exists
.MEASURE Tran MaxVout Max v(D_Output) Goal = 'v(Vdd)'
* This measure calculates the setup time value
.MEASURE Tran SetupTime Trig v(Data)      Val = 'v(Vdd)/2'
+                                          Fall = 1
+                                          Val = 'v(Vdd)/2'
+                                          Rise = 1
+                                          Targ v(Clock)
* Optimization Model
.MODEL OptMod Opt
+ Method = Bisection
.OPTION Post Brief NoMod
```

```

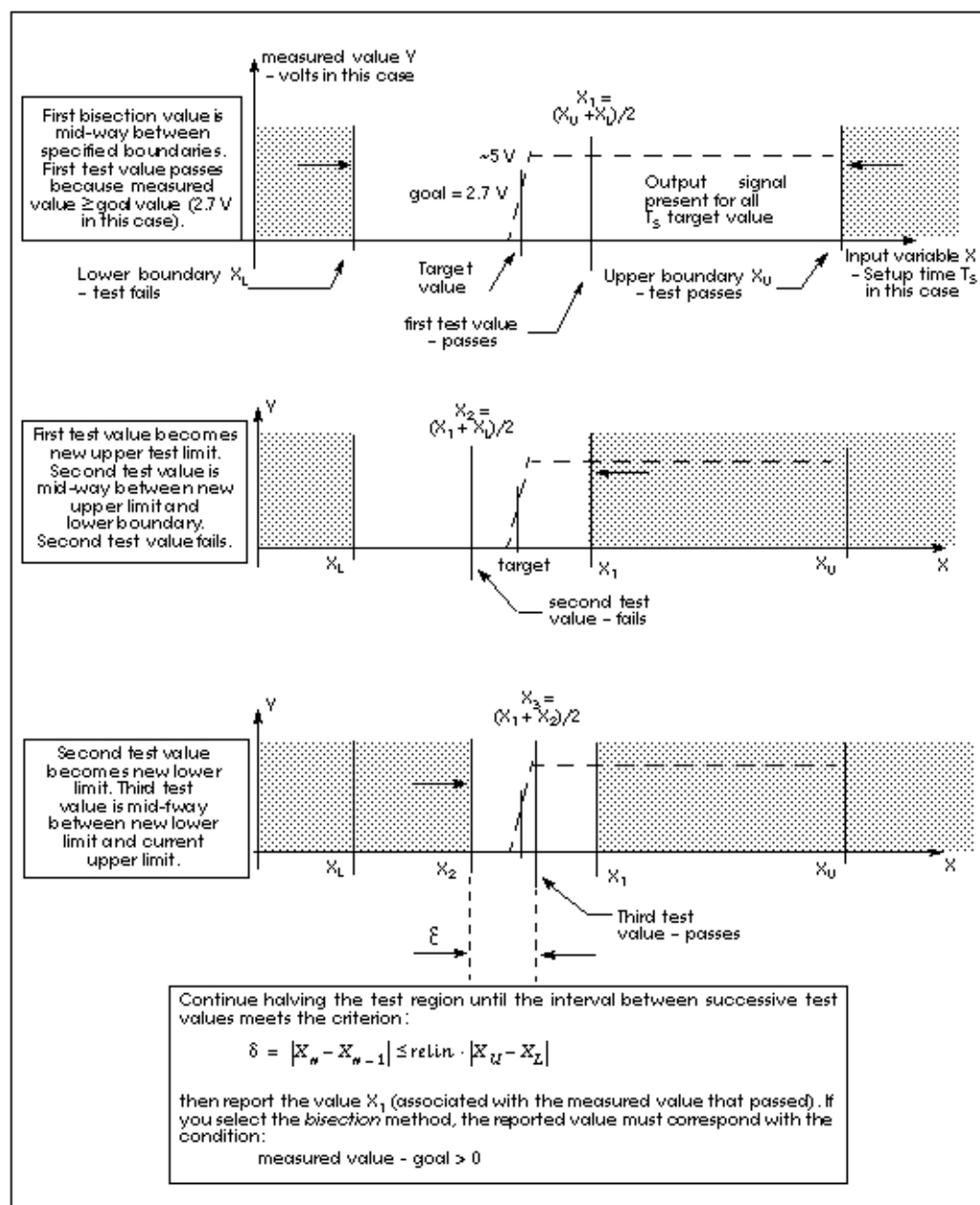
*****
* AvanLink to Cadence Composer by Avant!
* Hspice Netlist, May 31 15:24:09 1994
*****
.MODEL nmos nmos LEVEL=2
.MODEL pmos pmos LEVEL=2
.Global vdd gnd
.SUBCKT XGATE control in n_control out
m0 in n_control out vdd pmos l=1.2u w=3.4u
m1 in control out gnd nmos l=1.2u w=3.4u
.ends

.SUBCKT INV in out wp=9.6u wn=4u l=1.2u
mb2 out in gnd gnd nmos l=l w=wn
mb1 out in vdd vdd pmos l=l w=wp
.ends

.SUBCKT DFF c d nc nq
Xi64 nc net46 c net36 XGATE
Xi66 nc net38 c net39 XGATE
Xi65 c nq nc net36 XGATE
Xi62 c d nc net39 XGATE
Xi60 net722 nq INV
Xi61 net46 net38 INV
Xi59 net36 net722 INV
Xi58 net39 net46 INV
c20 net36 gnd c=17.09f
c15 net39 gnd c=15.51f
c12 net46 gnd c=25.78f
c4 nq gnd c=25.28f
c3 net722 gnd c=19.48f
c16 net38 gnd c=16.48f
.ENDS
*-----
* Main Circuit Netlist:
*-----
v14 vdd gnd dc=5
c10 vdd gnd c=35.96f
c15 d_output gnd c=21.52f
c12 dff_nq gnd c=11.73f
c11 net31 gnd c=42.01f
c14 net27 gnd c=34.49f
c13 net25 gnd c=41.73f
c8 clock gnd c=5.94f
c7 data gnd c=7.93f
Xi3 net25 net31 net27 dff_nq DFF l=1u wn=3.8u wp=10u
Xi6 data net31 INV
Xi5 net25 net27 INV
Xi4 clock net25 INV
Xi2 dff_nq d_output INV wp=26.4u wn=10.6u
.END

```

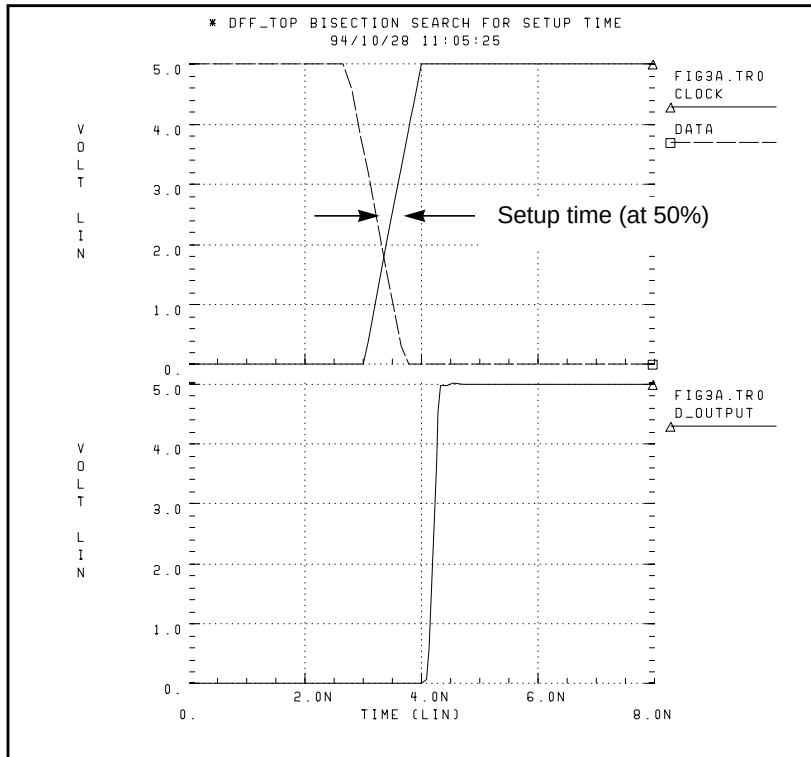
Figure 21-2: Bisection Example for Three Iterations



Results

The top plot in [Figure 21-3](#) shows the relationship between the clock and the data pulses that determine the setup time. The bottom plot is the output transition.

Figure 21-3: Transition at Minimum Setup Time



Find the actual value for the setup time in the “Optimization, Results” section of the input listing file:

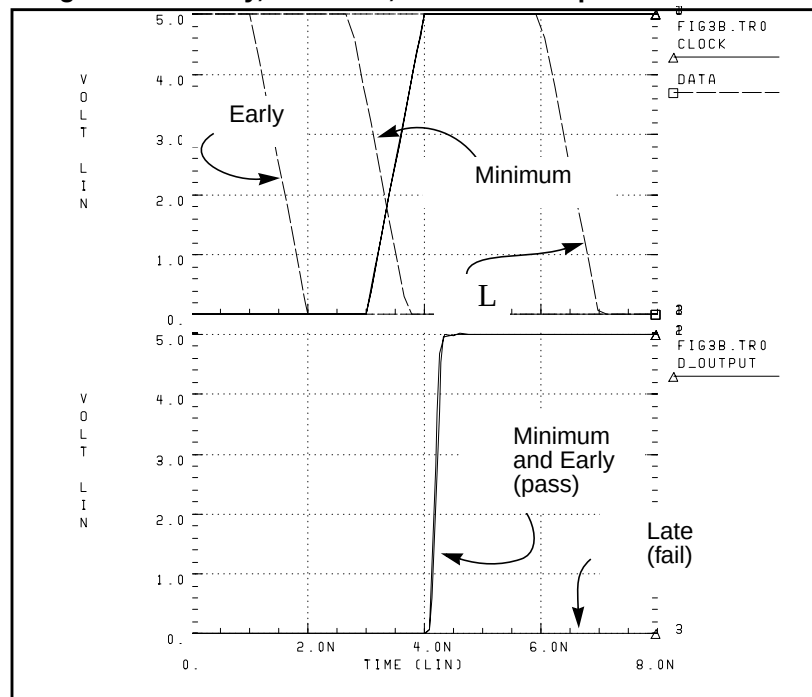
```
optimization completed, the condition
relin = 1.0000E-03 is satisfied
**** optimized parameters opt1
.PARAM DelayTime = 1.7188n
...
maxvout = 5.0049E+00    at= 4.5542E-09
from    = .0000E+00    to= 8.0000E-09
setupime= 2.8125E-10 targ= 3.5000E-09 trig= 3.2188E-09
```

This listing file excerpt shows that the optimal value for the setup time is 0.28125 nanoseconds.

The top plot in [Figure 21-4](#) shows examples of early and late data transitions, and the transition at the minimum setup time. The bottom plot shows how the timing of the data transition affects the output transition. The following analysis statement produces these results:

```
* Sweep 3 values for DelayTime      Early    Optim    Late
.TRAN 1n 8n Sweep DelayTime Poi 3   0.0n    1.7188n  5.0n
```

Figure 21-4: Early, Minimum, and Late Setup and Hold Times



This analysis produces the following results:

```
*** parameter DelayTime = .000E+00 *** $ Early
setuptime= 2.0000E-09 targ= 3.5000E-09 trig= 1.5000E-09
*** parameter DelayTime = 1.719E-09 *** $ Optimal
setuptime= 2.8120E-10 targ= 3.5000E-09 trig= 3.2188E-09
*** parameter DelayTime = 5.000E-09 *** $ Late
setuptime= -3.0000E-09 targ= 3.5000E-09 trig= 6.5000E-09
```

Minimum Pulse Width Analysis

This example uses a pass/fail bisectional search, to find a minimum pulse width required, so the input pulse can propagate to the output of an inverter. The circuit for this example is `/bisect/inv_a.sp`, in the `$installdir/demo/hspice` directory. [Figure 21-5 on page 21-16](#) shows the results of this demo.

Input Listing

```
File: $installdir/demo/bisect/inv_a.sp
$ Inv_a.sp testing bisectional search, cload=10p & 20p
*
* Parameters
.PARAM Cload      =10p                $ See end of deck for Alter
.PARAM Tpw        =opt1(0,0,15n)      $ Used in Pulsed Voltage
                                           $ Source, v1

* Transient simulation with PassFail Optimization
.TRAN .1n 20n Sweep      Optimize      = Opt1
+                               results    = Tprop
+                               Model       = Optmod
.MODEL OptMod Opt        Method        = PassFail
.MEASURE Tran Tprop      Trig v(in)     Val=2.5 Rise=1
+                               Targ v(out) val=2.5 Fall=1
.OPTION nomod acct=3 post autostop
.GLOBAL 1

* The Circuit
vcc 1 0 5
vin in 0 pulse(0,5 1n 1n 1n Tpw 20n)
rin in 0 1e13
rout out 0 10k
cout out 0 cload
x1 in out inv
.SUBCKT Inv in out
mn out in 0 0 nch W=10u L=1u
mp out in 1 1 pch W=10u L=1u
.ENDS

* Models
.PARAM
+ mult1=1 xl=0.06u xwn=0.3u xwp=0.3u
+ tox=200 delvton=0 delvtop=0 rshn=50
+ rshp=150
```

```

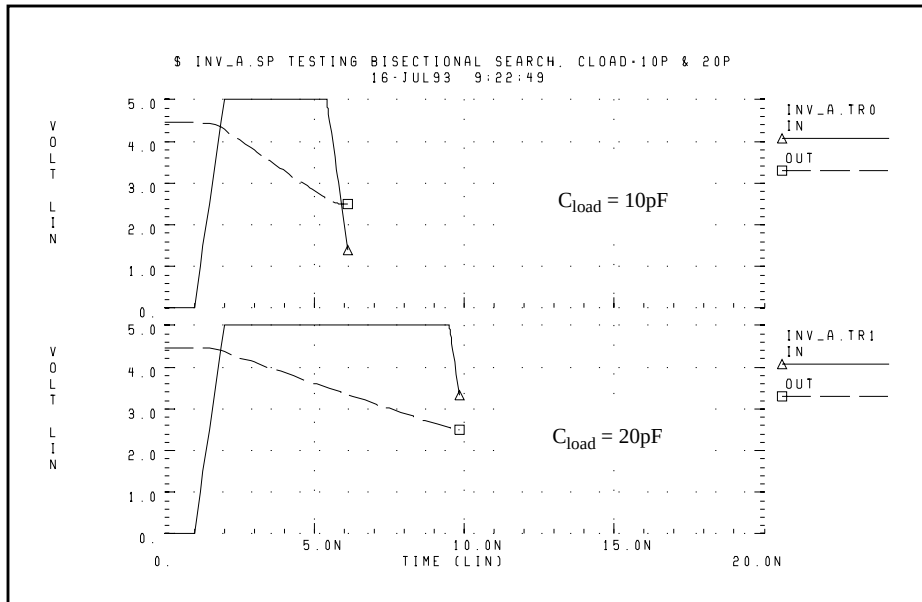
.MODEL nch nmos
+ LEVEL=28
+ lmlt=mult1 wmlt=mult1 wref=22u lref=4.4u
+ xl=xl xw=xwn tox=tox delvto=delvton rsh=rshn
+ ld=0.06u wd=0.2u acm=2 ldif=0 hdif=2.5u
+ rs=0 rd=0 rdc=0 rsc=0 js=3e-04 jsw=9e-10
+ cj=3e-04 mj=.5 pb=.8 cjsw=3e-10 mjsw=.3 php=.8
+ fc=.5 capop=4 xqc=.4 meto=0.08u
+ tlev=1 cta=0 ctp=0 tlevc=0 nlev=0
+ trs=1.6e-03 bex=-1.5 tcv=1.4e-03
* dc model
+ x2e=0 x3e=0 x2u1=0 x2ms=0 x2u0=0 x2m=0
+ vfb0=-.5 phi0=0.65 k1=.9 k2=.1 eta0=0
+ muz=500 u00=.075 x3ms=15 u1=.02 x3u1=0
+ b1=.28 b2=.22 x33m=0.000000e+00
+ alpha=1.5 vcr=20 n0=1.6 wfac=15 wfacu=0.25
+ lvfb=0 lk1=.025 lk2=.05 lalpha=5
.Model pch pmos
+ LEVEL=28
+ lmlt=mult1 wmlt=mult1 wref=22u lref=4.4u
+ xl=xl xw=xwp tox=tox delvto=delvtop rsh=rshp
+ ld=0.08u wd=0.2u acm=2 ldif=0 hdif=2.5u
+ rs=0 rd=0 rdc=0 rsc=0 rsh=rshp js=3e-04 jsw=9e-10
+ cj=3e-04 mj=.5 pb=.8 cjsw=3e-10 mjsw=.3 php=.8
+ fc=.5 capop=4 xqc=.4 meto=0.08u
+ tlev=1 cta=0 ctp=0 tlevc=0 nlev=0
+ trs=1.6e-03 bex=-1.5 tcv=-1.7e-03
* dc model
+ x2e=0 x3e=0 x2u1=0 x2ms=0 x2u0=0 x2m=5
+ vfb0=-.1 phi0=0.65 k1=.35 k2=0 eta0=0
+ muz=200 u00=.175 x3ms=8 u1=0 x3u1=0.0
+ b1=.25 b2=.25 x33m=0.0 alpha=0 vcr=20
+ n0=1.3 wfac=12.5 wfacu=.2 lvfb=0 lk1=-.05
*
* Alter for second load value
*
.ALTER $ repeat optimization for 20p load
.PARAM Cload=20p
.END

```

Results

Figure 21-5 shows the results of the pass/fail search, for two different capacitive loads.

Figure 21-5: Results of Bisectional Pass/Fail Search





Chapter 22

Running Demonstration Files

This chapter contains examples of basic file construction techniques, advanced features, and simulation tricks. It also lists and describes several Star-Hspice input files.

This chapter explains the following topics:

- [Using the Demo Directory Tree](#)
- [Two-Bit Adder Demo](#)
- [MOS I-V and C-V Plotting Demo](#)
- [CMOS Output Driver Demo](#)
- [Temperature Coefficients Demo](#)
- [Simulating Electrical Measurements](#)
- [Modeling Wide-Channel MOS Transistors](#)
- [Demonstration Input Files](#)

Using the Demo Directory Tree

[Demonstration Input Files on page 22-23](#) lists demonstration files, which are designed as good training examples. Most Star-Hspice distributions include these examples in the *demo* directory tree, where *\$installdir* is the installation directory environment variable:

<i>\$installdir</i> /demo/ hspice	/alge	algebraic output
	/apps	general applications
	/behave	analog behavioral components
	/bench	standard benchmarks
	/bjt	bipolar components
	/cchar	characteristics of cell prototypes
	/ciropt	circuit level optimization
	/ddl	Discrete Device Library
	/devopt	device level optimization
	/fft	Fourier analysis
	/filters	filters
	/mag	transformers, magnetic core components
	/mos	MOS components
	/rad	radiation effects (photocurrent)
	/sources	dependent and independent sources
	/tline	filters and transmission lines

Two-Bit Adder Demo

This two-bit adder shows how to improve efficiency, accuracy, and productivity in circuit simulation. The adder is in the `$install_dir/demo/hspice/apps/mos2bit.sp` demonstration file. It consists of two-input NAND gates, defined using the NAND sub-circuit. CMOS devices include length, width, and output loading parameters. Descriptive names enhance the readability of this circuit.

One-Bit Subcircuit

The ONEBIT subcircuit defines the two half adders, with carry in and carry out. To create the two-bit adder, Star-Hspice uses two calls to ONEBIT. Independent piecewise linear voltage sources provide the input stimuli. The *R* repeat function creates complex waveforms.

Figure 22-1: One-bit Adder sub-circuit

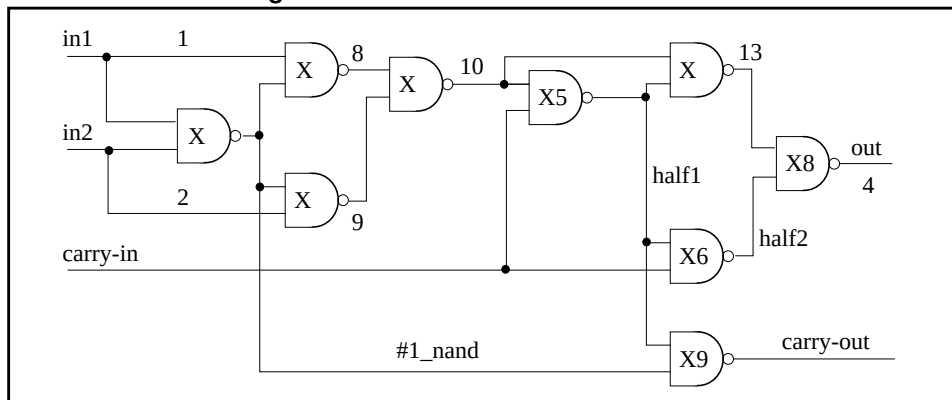


Figure 22-2: Two-bit Adder Circuit

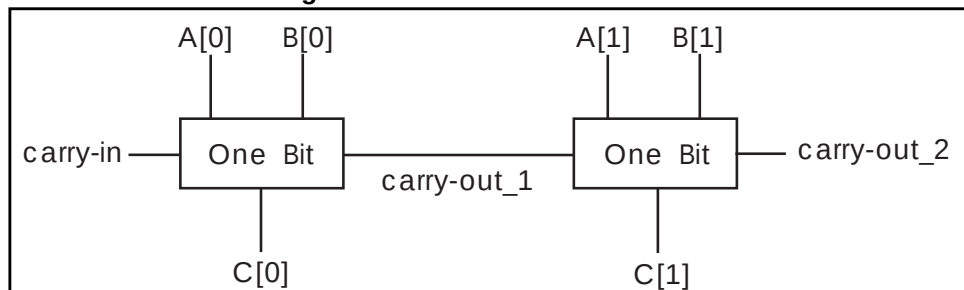
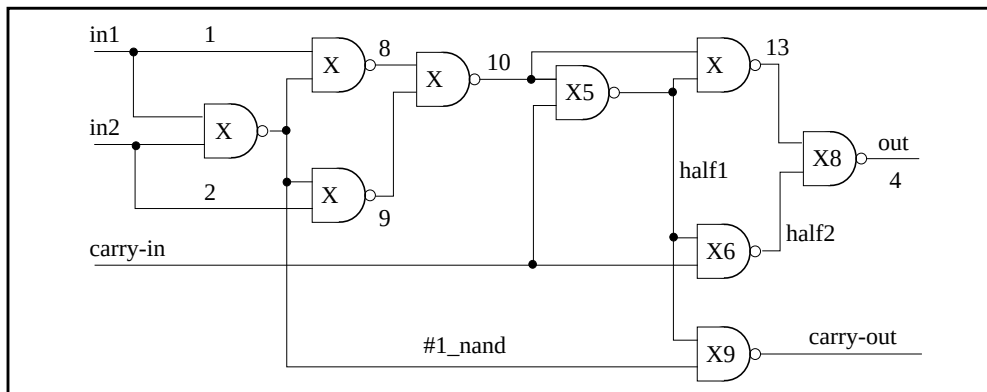


Figure 22-3: 1-bit NAND Gate Binary Adder



MOS Two-Bit Adder Input File

```

*FILE: MOS2BIT.SP - ADDER - 2 BIT ALL-NAND-GATE BINARY ADDER
.OPTION ACCT NOMOD FAST autostop scale=1u gmindc=100n
.param lmin=1.25 hi=2.8v lo=.4v vdd=4.5
.global vdd
.TRAN .5NS 60NS
.graph TRAN V(c[0]) V(carry-out_1) V(c[1]) V(carry-out_2)
+ par('V(carry-in)/6 + 1.5')
+ par('V(a[0])/6 + 2.0')
+ par('V(b[0])/6 + 2.5') (0,5)
.MEAS PROP-DELAY TRIG V(carry-in) TD=10NS VAL='vdd*.5' RISE=1
+ TARG V(c[1]) TD=10NS VAL='vdd*.5' RISE=3
.MEAS PULSE-WIDTH TRIG V(carry-out_1) VAL='vdd*.5' RISE=1
+ TARG V(carry-out_1) VAL='vdd*.5' FALL=1
.MEAS FALL-TIME TRIG V(c[1]) TD=32NS VAL='vdd*.9' FALL=1
+ TARG V(c[1]) TD=32NS VAL='vdd*.1' FALL=1
vdd vdd gnd DC vdd
X1 A[0] B[0] carry-in C[0] carry-out_1 ONEBIT
X2 A[1] B[1] carry-out_1 C[1] carry-out_2 ONEBIT
sub-circuit Definitions
.subckt NAND in1 in2 out wp=10 wn=5
M1 out in1 vdd vdd P W=wp L=lmin ad=0
M2 out in2 vdd vdd P W=wp L=lmin ad=0
M3 out in1 mid gnd N W=wn L=lmin as=0
M4 mid in2 gnd gnd N W=wn L=lmin ad=0
CLOAD out gnd 'wp*5.7f'
.ends
* switch model equivalent of the NAND. Gives a 10 times
* speedup over the MOS version.
.subckt NANDx in1 in2 out wp=10 wn=5

```

```

G1 out vdd vdd in1 LEVEL=1 MIN=1200 MAX=1MEG 1.MEG -.5MEG
G2 out vdd vdd in2 LEVEL=1 MIN=1200 MAX=1MEG 1.MEG -.5MEG
G3 out mid in1 gnd LEVEL=1 MIN=1200 MAX=1MEG 1.MEG -.5MEG
G4 mid gnd in2 gnd LEVEL=1 MIN=1200 MAX=1MEG 1.MEG -.5MEG
cout out gnd 300f
.ends
.subckt ONEBIT in1 in2 carry-in out carry-out
    X1 in1 in2 #1_nand NAND
    X2 in1 #1_nand 8 NAND
    X3 in2 #1_nand 9 NAND
    X4 8 9 10 NAND
    X5 carry-in 10 half1 NAND
    X6 carry-in half1 half2 NAND
    X7 10 half1 13 NAND
    X8 half2 13 out NAND
    X9 half1 #1_nand carry-out NAND
.ENDS ONEBIT

```

Stimuli

```

V1 carry-in gnd PWL(0NS,lo 1NS,hi 7.5NS,hi 8.5NS,lo 15NS lo R
V2 A[0] gnd PWL (0NS,hi 1NS,lo 15.0NS,lo 16.0NS,hi 30NS hi R
V3 A[1] gnd PWL (0NS,hi 1NS,lo 15.0NS,lo 16.0NS,hi 30NS hi R
V4 B[0] gnd PWL (0NS,hi 1NS,lo 30.0NS,lo 31.0NS,hi 60NS hi
V5 B[1] gnd PWL (0NS,hi 1NS,lo 30.0NS,lo 31.0NS,hi 60NS hi

```

Models

```

.MODEL N NMOS LEVEL=3 VTO=0.7 U0=500 KAPPA=.25 KP=30U
+ ETA=.01 THETA=.04 VMAX=2E5 NSUB=9E16 TOX=400 GAMMA=1.5
+ PB=0.6 JS=.1M XJ=0.5U LD=0.1U NFS=1E11 NSS=2E10
+ RSH=80 CJ=.3M MJ=0.5 CJSW=.1N MJSW=0.3 acm=2 capop=4
.MODEL P PMOS LEVEL=3 VTO=-0.8 U0=150 KAPPA=.25 KP=15U
+ ETA=.015 THETA=.04 VMAX=5E4 NSUB=1.8E16 TOX=400
+ GAMMA=.672 PB=0.6 JS=.1M XJ=0.5U LD=0.15U
+ NFS=1E11 NSS=2E10 RSH=80 CJ=.3M MJ=0.5 CJSW=.1N
+ MJSW=0.3 acm=2 capop=4
.END

```

MOS I-V and C-V Plotting Demo

To diagnose a simulation or modeling problem, you usually need to review the basic characteristics of the transistors. You can use this demonstration template file, `$installdir/demo/hspice/mos/mosivcv.sp`, with any MOS model. The example shows how to easily create input files, and how to display the complete graphical results. The following features aid model evaluations:

SCALE=1u	Sets the element units to microns, from meters, because circuit designs typically use microns.
DCCAP	Forces Star-Hspice to evaluate the voltage variable capacitors, during a DC sweep.
node names	Makes the circuit easy to understand. The symbolic name can contain up to 16 characters.
.GRAPH	.GRAPH statements create high-resolution plots. To set additional characteristics, add a graph model.

Plotting Variables

Use this template to plot internal variables, such as:

i(mn1)	i1, i2, i3, or i4 can specify the true branch currents, for each transistor node.
LV18(mn6)	Total gate capacitance (C-V plot).
LX7(mn1)	GM gate transconductance. (LX8 specifies GDS, and LX9 specifies GMB).

Figure 22-4: MOS IDS Plot

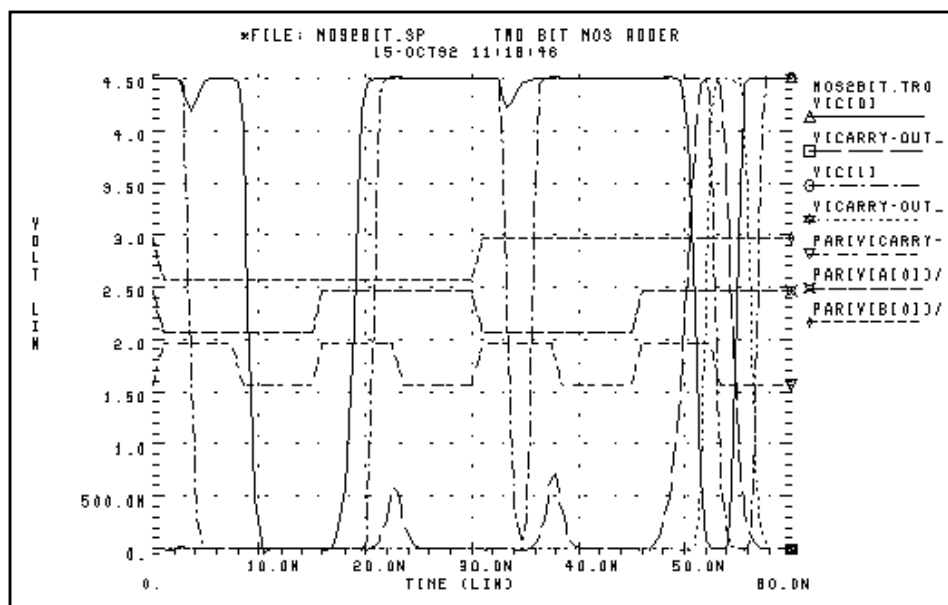


Figure 22-5: MOS VGS Plot

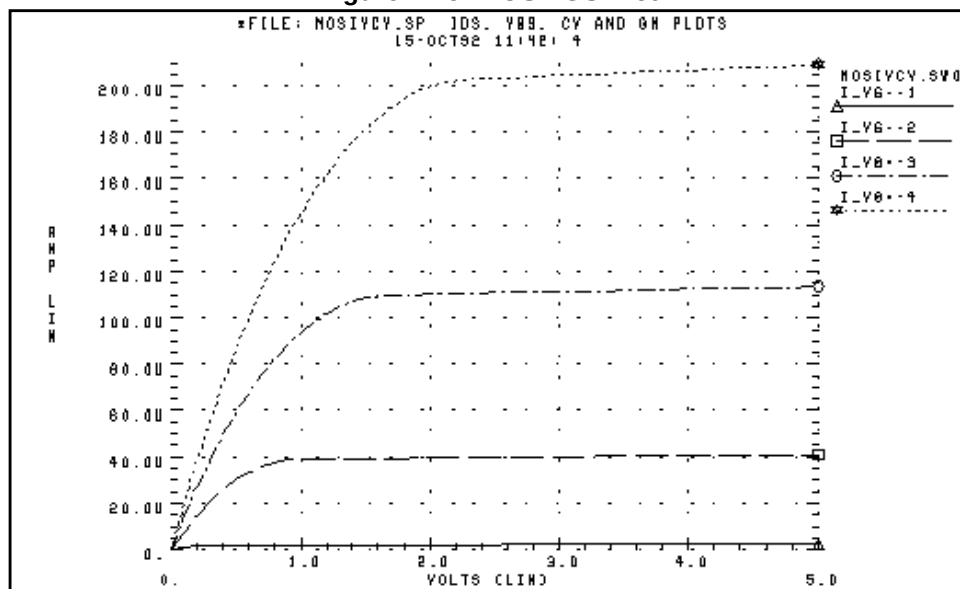


Figure 22-6: MOS GM Plot

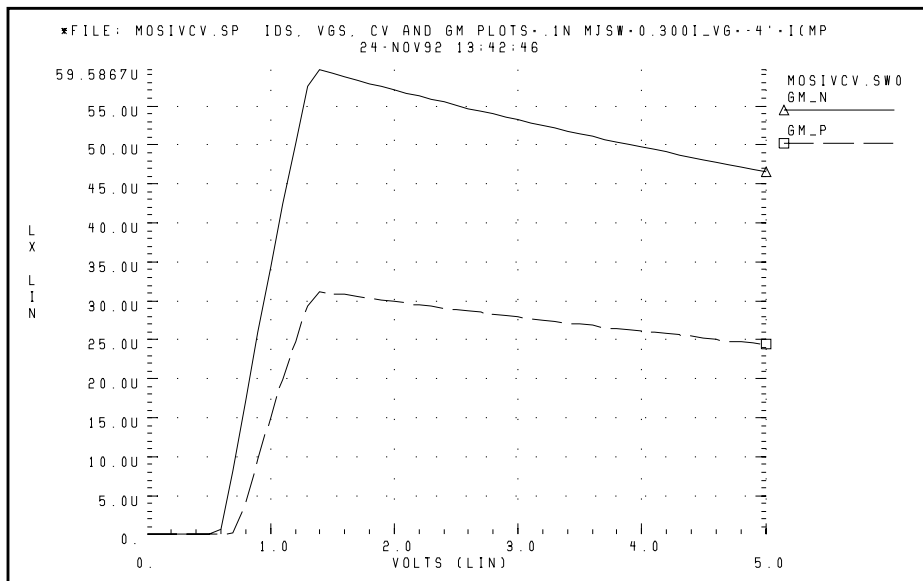
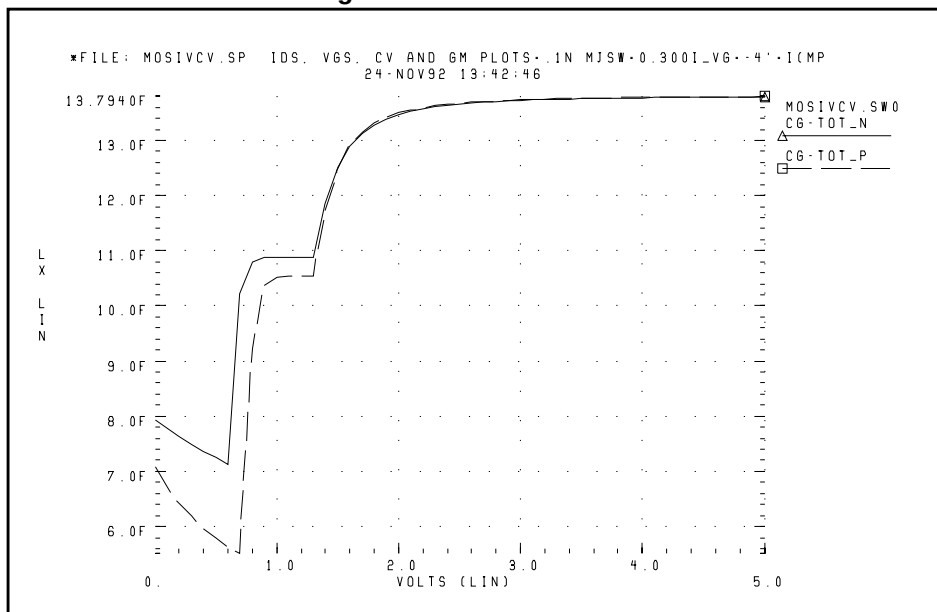


Figure 22-7: MOS C-V Plot



MOS I-V and C-V Plot Example Input File

```

*FILE: MOSIVCV.SP IDS, VGS, CV AND GM PLOTS
.OPTION SCALE=1U DCCAP
.DC VDDN 0 5.0 .1 $VBBN 0 -3 -3          sweep supplies
.PARAM ww=8 LL=2
$ ids-vds curves
.GRAPH 'I_VG=1' =I(MN1) 'I_VG=2' =I(MN2) 'I_VG=3' =I(MN3)
+ 'I_VG=4' =I(MN4)
.GRAPH 'I_VG=-1' =I(MP1) 'I_VG=-2' =I(MP2) 'I_VG=-3' =I(MP3)
+ 'I_VG=-4' =I(MP4)
$ ids-VGs curves
.GRAPH 'I_VD=.5' =I(MN6) 'I_VD=-.5' =I(MP6)
$ gate caps (cgs+cgd+cgb)
.GRAPH 'CG-TOT_N' =LX18(MN6) 'CG-TOT_P' = LX18(MP6)
$ gm
.GRAPH 'GM_N' =LX7(MN6) 'GM_P' =LX7(MP6)
VDDN vdd_n gnd 5.0
VBBN vbb_n gnd 0.0
EPD vdd_p gnd vdd_n gnd -1 $ reflect vdd for P devices
EPB vbb_p gnd vbb_n gnd -1 $ reflect vbb for P devices
V1 vg1 gnd 1
V2 vg2 gnd 2
V3 vg3 gnd 3
V4 vg4 gnd 4
V5 vddlow_n gnd .5
V-1 vg-1 gnd -1
V-2 vg-2 gnd -2
V-3 vg-3 gnd -3
V-4 vg-4 gnd -4
V-5 vddlow_p gnd -.5
MN1 vdd_n vg1 gnd vbb_n N W=ww L=LL
MN2 vdd_n vg2 gnd vbb_n N W=ww L=LL
MN3 vdd_n vg3 gnd vbb_n N W=ww L=LL
MN4 vdd_n vg4 gnd vbb_n N W=ww L=LL
MP1 gnd vg-1 vdd_p vbb_p P W=ww L=LL
MP2 gnd vg-2 vdd_p vbb_p P W=ww L=LL
MP3 gnd vg-3 vdd_p vbb_p P W=ww L=LL
MP4 gnd vg-4 vdd_p vbb_p P W=ww L=LL
MN6 vddlow_n vdd_n gnd vbb_n N W=ww L=LL
MP6 gnd vdd_p vddlow_p vbb_p P W=ww L=LL
.MODEL N NMOS LEVEL=3 VTO=0.7 UO=500 KAPPA=.25 ETA=.01
+ THETA=.04 VMAX=2E5 NSUB=9E16 TOX=400 KP=30U GAMMA=1.5
+ PB=0.6 JS=.1M XJ=0.5U LD=0.1U NFS=1E11 NSS=2E10
+ RSH=80 CJ=.3M MJ=0.5 CJSW=.1N MJSW=0.3 acm=2 capop=4
.MODEL P PMOS LEVEL=3 VTO=-0.8 UO=150 KAPPA=.25 KP=15U
+ ETA=.015 THETA=.04 VMAX=5E4 NSUB=1.8E16 TOX=400 GAMMA=.67
+ PB=0.6 JS=.1M XJ=0.5U LD=0.15U NFS=1E11 NSS=2E10
+ RSH=80 CJ=.3M MJ=0.5 CJSW=.1N MJSW=0.3 acm=2 capop=4
.END

```

CMOS Output Driver Demo

ASIC designers need to integrate high-performance IC parts onto a printed circuit board (PCB). The output driver circuit is critical to the overall performance of the system. The demonstration file, `$install_dir/demo/hspice/apps/asic1.sp`, shows the models for an output driver, the bond wire and leadframe, and a six-inch length of copper transmission line.

This simulation demonstrates how to:

- Define parameters, and measure test outputs.
- Use the LUMP5 macro to input geometric units, and convert them to electrical units.
- Use `.MEASURE` statements to calculate the peak local supply current, voltage drop, and power.
- Measure RMS power, delay, rise times, and fall times.
- Simulate and measure an output driver under load. The load consists of:
 - Bondwire and leadframe inductance.
 - Bondwire and leadframe resistance.
 - Leadframe capacitance.
 - Six inches of 6-mil copper, on an FR-4 printed circuit board.
 - Capacitive load, at the end of the copper wire.

Strategy

The Star-Hspice strategy is to:

- Create a five-lump transmission line model, for the copper wire.
- Create single lumped models, for leadframe loads.

Figure 22-8: Noise Bounce

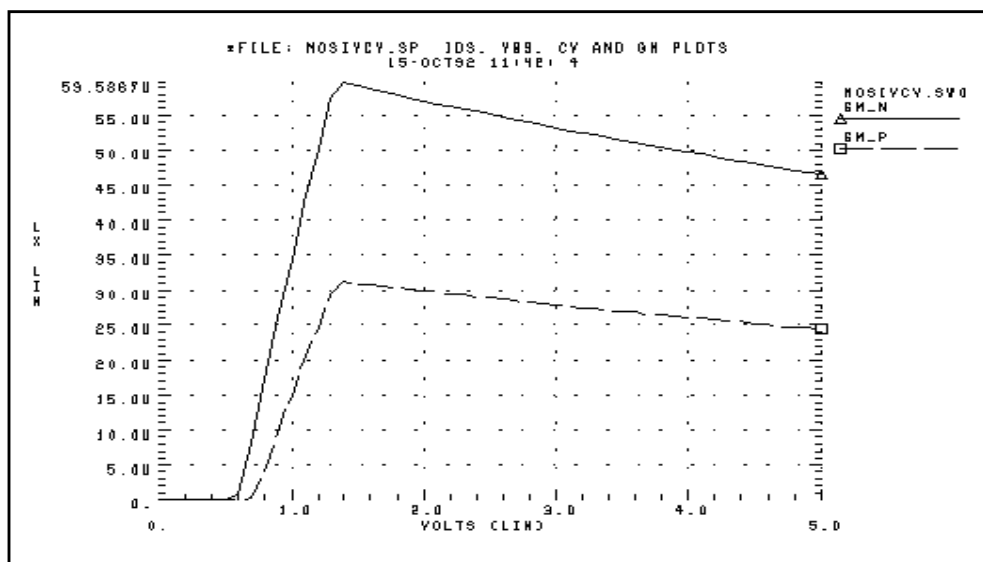


Figure 22-9: Asic1.sp Demo Local Supply Voltage

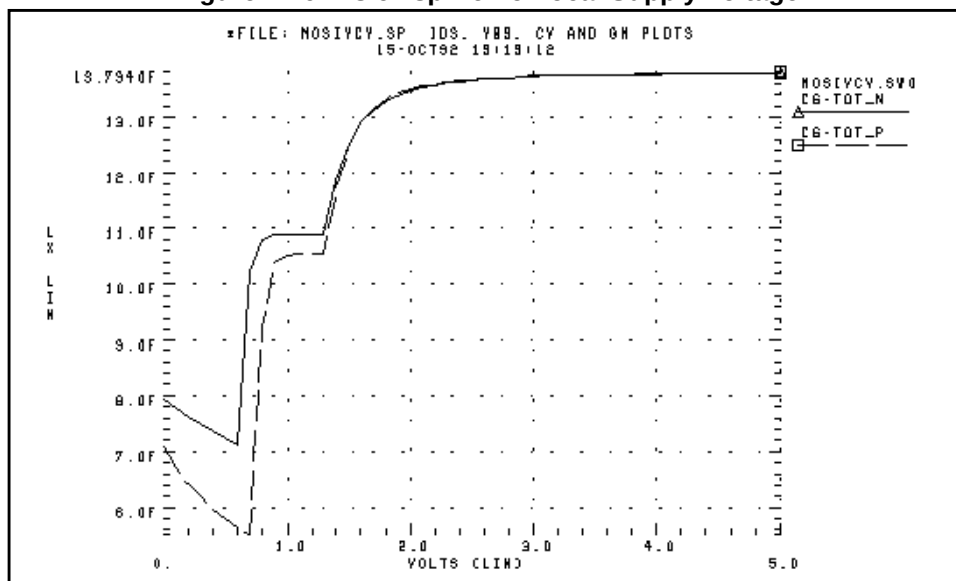


Figure 22-10: Asic1.sp Demo Local Supply Current

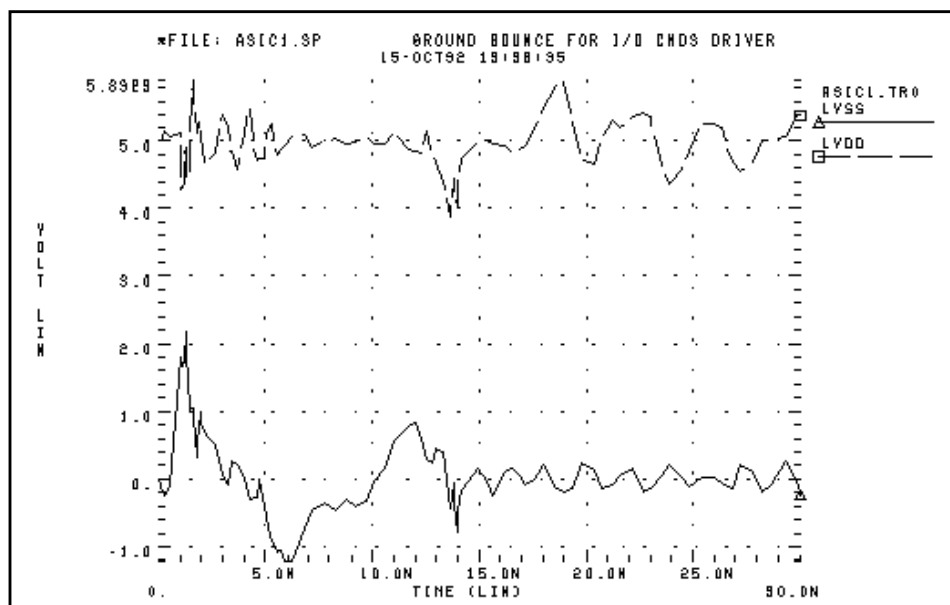
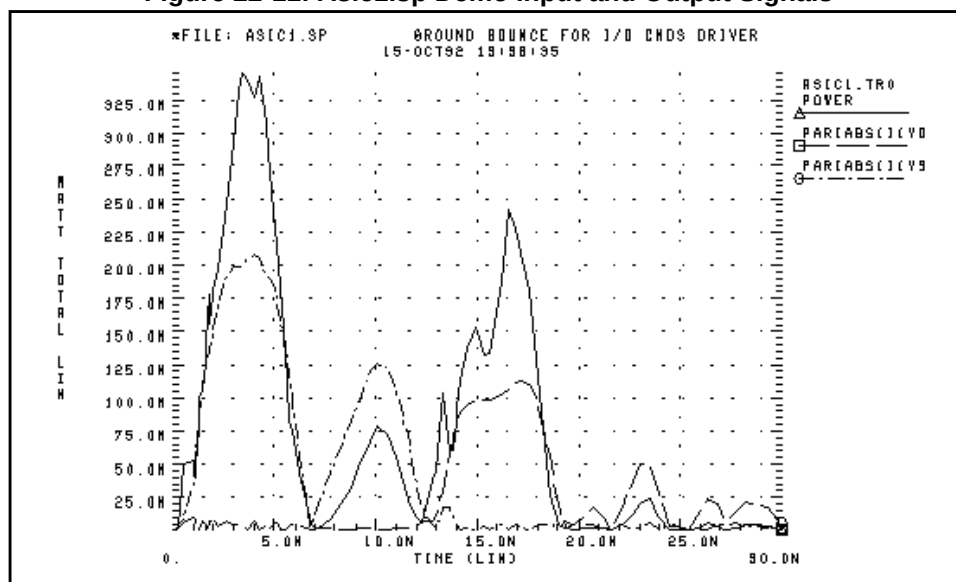


Figure 22-11: Asic1.sp Demo Input and Output Signals



CMOS Output Driver Example Input File

```
* FILE: ASIC1.SP
* SIMULATE AN OUTPUT DRIVER DRIVING 6 INCHES OF 6MIL PRINTED
*   CIRCUIT BOARD COPPER WITH 25PF OF LOAD CAPACITANCE
* MEASURE PEAK TO PEAK GROUND VOLTAGE
* MEASURE MAXIMUM GROUND CURRENT
* MEASURE MAXIMUM SUPPLY CURRENT
GROUND BOUNCE FOR I/O CMOS DRIVER 1200/1.2 & 800/1.2 MICRONS
.OPTION POST=2 RELVAR=.05
.TRAN .25N 30N
.MEASURE IVDD_MAX MAX PAR('ABS(I(VD))')
.MEASURE IVSS_MAX MAX PAR('ABS(I(VS))')
.MEASURE PEAK_GNDV PP V(LVSS)
.MEASURE PEAK_IVD PP PAR(' ABS(I(VD)*V(VDD,OUT)) ')
.MEASURE PEAK_IVS PP PAR(' ABS(I(VS)*V(VSS,OUT)) ')
.MEASURE RMS_POWER RMS POWER
.MEASURE FALL_TIME TRIG V(IN) RISE=1 VAL=2.5V
+ TARG V(OUT) FALL=1 VAL=2.5V
.MEASURE RISE_TIME TRIG V(IN) FALL=1 VAL=2.5V
+ TARG V(OUT) RISE=1 VAL=2.5V
.MEASURE TLINE_DLY TRIG V(OUT) RISE=1 VAL=2.5V
+ TARG V(OUT2) RISE=1 VAL=2.5V
```

Input Signals

```
VIN IN LGND PWL(0N 0V, 2N 5V, 12N 5, 14N 0)
* OUTPUT DRIVER
MP1 LOUT IN LVDD LVDD P W=1400U L=1.2U
MN1 LOUT IN LVSS LVSS N W=800U L=1.2U
xout LOUT OUT LEADFRAME
*POWER AND GROUND LINE PARASITICS
Vd VDD GND 5V
xdd vdd lvdd leadframe
Vs VSS gnd 0v
xss vss lvss leadframe
*OUTPUT LOADING – 3 INCH FR-4 PC BOARD + 5PF LOAD +
*3 INCH FR-4 + 5PF LOAD
XLOAD1 OUT OUT1 GND LUMP5 LEN=3 WID=.006
CLOAD1 OUT1 GND 5PF
XLOAD2 OUT1 OUT2 GND LUMP5 LEN=3 WID=.006
CLOAD2 OUT2 GND 5PF
.macro leadframe in out
rframe in mid .01
lframe mid out 10n
cframe mid gnd .5p
.ends
```

```

*Transmission Line Parameter Definitions
.param rho=.6mho/sq cap=.55nf/in**2 ind=60ph/sq
*The 5-lump macro defines a parameterized transmission line
.macro lump5 in out ref len_lump5=1 wid_lump5=.1
.prot
.param reseff='len_lump5*rho/wid_lump5*5'
+      capeff='len_lump5*wid_lump5*cap/5'
+      indeff='len_lump5*ind/wid_lump5*5'
r1 in 1 reseff
c1 1 ref capeff
l1 1 2 indeff
r2 2 3 reseff
c2 3 ref capeff
l2 3 4 indeff
r3 4 5 reseff
c3 5 ref capeff
l3 5 6 indeff
r4 6 7 reseff
c4 7 ref capeff
l4 7 8 indeff
r5 8 9 reseff
c5 9 ref capeff
l5 9 out indeff
.unprot
.ends

```

Model Section

```

.MODEL N NMOS LEVEL=3 VTO=0.7 U0=500 KAPPA=.25 ETA=.03
+ THETA=.04 VMAX=2E5 NSUB=9E16 TOX=200E-10 GAMMA=1.5 PB=0.6 +
JS=.1M XJ=0.5U LD=0.0 NFS=1E11 NSS=2E10 capop=4
.MODEL P PMOS LEVEL=3 VTO=-0.8 U0=150 KAPPA=.25 ETA=.03
+ THETA=.04 VMAX=5E4 NSUB=1.8E16 TOX=200E-10 GAMMA=.672
+ PB=0.6 JS=.1M XJ=0.5U LD=0.0 NFS=1E11 NSS=2E10 capop=4
.end
IVDD_MAX      = 0.1141      AT= 1.7226E-08
               FROM= 0.0000E+00  TO= 3.0000E-08
IVSS_MAX      = 0.2086      AT= 3.7743E-09
               FROM= 0.0000E+00  TO= 3.0000E-08
PEAK_GNDV = 3.221      FROM= 0.0000E+00 TO= 3.0000E-08
PEAK_IVD  = 0.2929      FROM= 0.0000E+00 TO= 3.0000E-08
PEAK_IVS  = 0.3968      FROM= 0.0000E+00 TO= 3.0000E-08
RMS_POWER = 0.1233      FROM= 0.0000E+00 TO= 3.0000E-08
FALL_TIME = 1.2366E-09 TARG= 1.9478E-09 TRIG= 7.1121E-10
RISE_TIME = 9.4211E-10 TARG= 1.4116E-08 TRIG= 1.3173E-08
TLINE_DLY = 1.6718E-09 TARG= 1.5787E-08 TRIG= 1.4116E-08

```

Temperature Coefficients Demo

SPICE-type simulators do not always automatically compensate for variations in temperature. The simulators make many assumptions that are not valid for all technologies. Many of the critical model parameters in Star-Hspice provide first-order and second-order temperature coefficients, to ensure accurate simulations. You can optimize these temperature coefficients in either of two ways.

- The first method uses the TEMP DC sweep variable.

All analysis sweeps allow two sweep variables. To optimize the temperature coefficients, one of these must be the optimize variable.

Sweeping TEMP limits the component to a linear element, such as a resistor, inductor, or capacitor.

- The second method uses multiple components at different temperatures.

Example

The following example, the `$installdir/demo/hspice/ciropt/opttemp.sp` demo file, simulates three circuits of a voltage source. It also simulates a resistor at -25, 0, and +25 °C from nominal, using the DTEMP parameter for element delta temperatures. The resistors share a common model.

You need three temperatures to solve a second-order equation. You can extend this simulation template to a transient simulation of non-linear components (such as bipolar transistors, diodes, and FETs).

This example uses some simulation shortcuts. In the internal output templates for resistors, LV1 (resistor) is the conductance (reciprocal resistance) at the desired temperature.

- You can run optimization in the resistance domain.
- To optimize more complex elements, use the current or voltage domain, with measured sweep data.

The error function expects a sweep on at least two points, so the data statement must include two duplicate points.

Input File, for Optimized Temperature Coefficients

```
*FILE OPTTEMP.SP    OPTIMIZE RESISTOR TC1 AND TC2
v-25 1 0 1v
v0   2 0 1v
v+25 3 0 1v
r-25 1 0 rmod dtemp=-25
r0   2 0 rmod dtemp=0
r+25 3 0 rmod dtemp=25
.model rmod R res=1k tc1r=tc1r_opt tc2r=tc2r_opt
```

Optimization Section

```
.model optmod opt
.dc data=RES_TEMP optimize=opt1
+      results=r@temp1,r@temp2,r@temp3
+      model=optmod
.param tc1r_opt=opt1(.001,-.1,.1)
.param tc2r_opt=opt1(1u,-1m,1m)
.meas r@temp1 err2 par(R_meas_t1) par('1.0 / lv1(r-25)')
.meas r@temp2 err2 par(R_meas_t2) par('1.0 / lv1(r0) ')
.meas r@temp3 err2 par(R_meas_t3) par('1.0 / lv1(r+25) ')
* * Output section *
.dc data=RES_TEMP
.print 'r1_diff'=par('1.0/lv1(r-25)')
+      'r2_diff'=par('1.0/lv1(r0) ')
+      'r3_diff'=par('1.0/lv1(r+25)')
.data RES_TEMP R_meas_t1 R_meas_t2 R_meas_t3
950 1000 1010
950 1000 1010
.enddata
.end
```

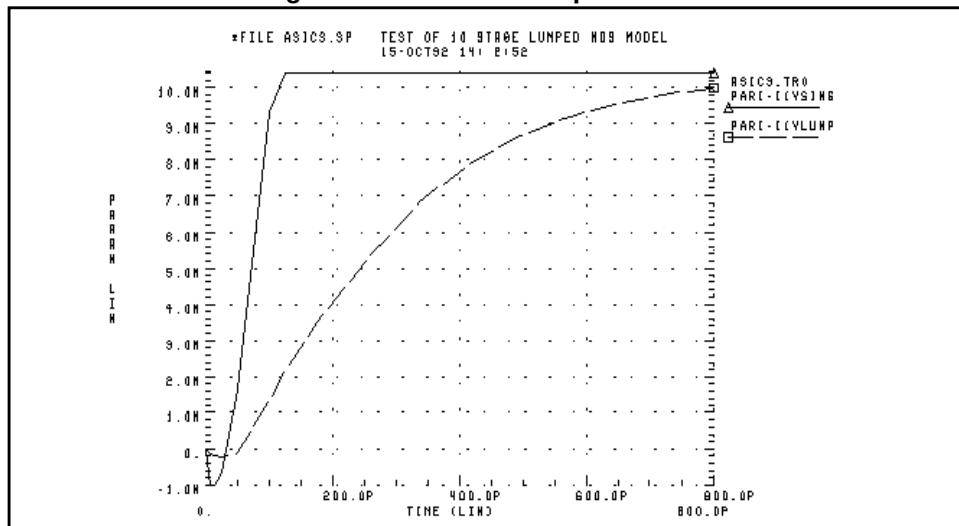

Simulating Electrical Measurements

In this example, Star-Hspice simulates electrical measurements, which return device characteristics, for data sheet information. The demonstration file for this example is `$installdir/demo/hspice/ddl/t2n2222.sp`. This example automatically includes DDL models by reference, using either the `DDLPATH` environment variable, or the `.OPTION SEARCH=path` statement. It also combines an AC circuit and measurement, with a transient circuit and measurement.

The AC circuit measures the maximum Hfe, which is the small-signal common emitter gain. Star-Hspice uses the `.MEASURE WHEN` statement to calculate the unity gain frequency, and the phase at the specified frequency. In the *Transient Measurements* section of the input file, a segmented transient statement speeds-up simulation, and compresses the output graph. Measurements include:

- TURN ON from 90% of input rising, to 90% of output falling.
- OUTPUT FALL from 90% to 10% of output falling.
- TURN OFF from 10% of input falling, to 10% of output rising.
- OUTPUT RISE from 10% to 90% of output rising.

Figure 22-12: T2N2222 Optimization



T2N2222 Optimization Example Input File

```
* FILE: T2N2222.SP
** assume beta=200 ft250meg at ic=20ma and vce=20v
** for 2n2222
.OPTION nopage autostop search=' '
*** ft measurement
* the net command automatically reverses the sign of
* the power supply current, for the network calculations
.NET I(vce) IBASE ROUT=50 RIN=50
VCE C 0 vce
IBASE 0 b AC=1 DC=ibase
xqft c b 0 t2n2222
.ac dec 10 1 1000meg
.graph s21(m) h21(m)
.measure 'phase @h21=0db' WHEN h21(db)=0
.measure 'h21_max' max h21(m)
.measure 'phase @h21=0deg' FIND h21(p) WHEN h21(db)=0
.param ibase=1e-4 vce=20 tauF=5.5e-10
```

Transient Measurements

```
** vccf power supply for forward reverse step recovery time
** vccr power supply for inverse reverse step recovery time
** VPLUSF positive voltage for forward pulse generator
** VPLUSr positive voltage for reverse pulse generator
** Vminusf positive voltage for forward pulse generator
** Vminusr positive voltage for reverse pulse generator
** rloadf load resistor for forward
** rloadr load resistor for reverse
.param vccf=30v
.param VPLUSF=9.9v
.param VMINUSF=-0.5v
.param rloadf=200
.TRAN 1N 75N 25N 200N 1N 300N 25N 1200N
.measure 'turn-on time' trig par('v(inf)-0.9*vplusf') val=0
+ rise=1 targ par('v(outf)-0.9*vccf') val=0 fall=1
.measure 'fall time' trig par('v(outf)-0.9*vccf') val=0
+ fall=1 targ par('v(outf)-0.1*vccf') val=0 fall=1
.measure 'turn-off time' trig par('v(inf)-0.1*vplusf')
+ val=0 fall=1 targ par('v(outf)-0.1*vccf') val=0 rise=1
.measure 'rise time' trig par('v(outf)-0.1*vccf') val=0
+ rise=1 targ par('v(outf)-0.9*vccf') val=0 rise=1
.graph V(INF) V(OUTF)
VCCF VCCF 0 vccf
RLOADF VCCF OUTF RLOADF
RINF INF VBASEF 1000
RPARF INF 0 58
XSCOPf OUTF 0 SCOPE
```

```
VINF      INF      0      PL      VMINUSF 0S      VMINUSF 5NS
+ VPLUSF 7NS VPLUSF 207NS VMINUSF 209NS
* CCX0F      VBASEF 0UTF      CCX0F
* CEX0F      VBASEF 0      CEX0F
XQF      0UTF VBASEF 0      t2n2222
.MACRO SCOPE VLOAD VREF
RIN      VLOAD VREF 100K
CIN      VLOAD VREF 12P
.EOM
.END
```

Modeling Wide-Channel MOS Transistors

If you select an appropriate model for I/O cell transistors, simulation accuracy improves. For wide-channel devices, model the transistor as a *group* of transistors, connected in parallel, with appropriate RC delay networks. If you model the device as only *one* transistor, the polysilicon gate introduces delay.

When you scale to higher-speed technologies, the area of the polysilicon gate decreases, reducing the gate capacitance. However, if you scale the gate oxide thickness, the capacitance per unit area increases, which also increases the RC product. The following example illustrates how scaling affects the delay. For example, for a device with:

- channel width = 100 microns
- channel length = 5 microns
- gate oxide thickness = 800 Angstroms

the resulting RC product for the polysilicon gate is:

$$R_{poly} = \frac{W}{L} \cdot 40 \quad poly = \frac{E_{sio} \cdot n_{si}}{t_{ox}} \cdot L \cdot W$$

$$R_{poly} = \frac{100}{5} \cdot 40 = 800, \quad C_o = \frac{3.9 \cdot 8.86}{800} \cdot 100 \cdot 5 = 215 \text{ fF} \quad RC = 138 \text{ ps}$$

For a transistor with:

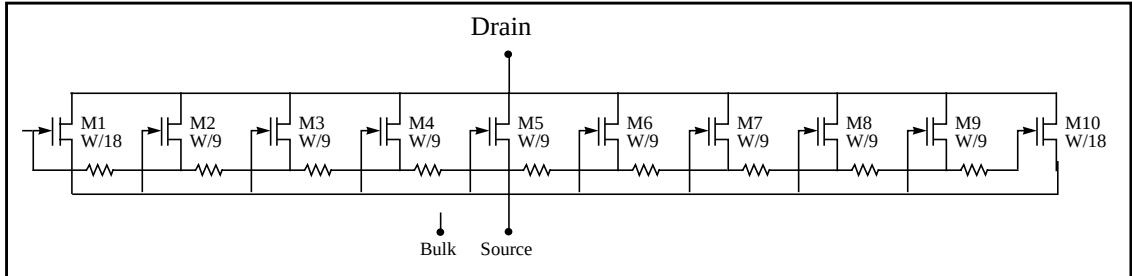
- channel width = 100 microns
- channel length = 1.2 microns
- gate oxide thickness = 250 Angstroms

the resulting RC product for the polysilicon gate is:

$$R_{poly} = \frac{\text{channel width}}{\text{channel length}} \cdot 40$$

$$C_o = \frac{3.9 \cdot 8.86}{T_{ox}} \cdot \text{channel width} \cdot \text{channel length} \quad RC = 546 \text{ ps}$$

You can use a nine-stage ladder model, to model the RC delay in CMOS devices.

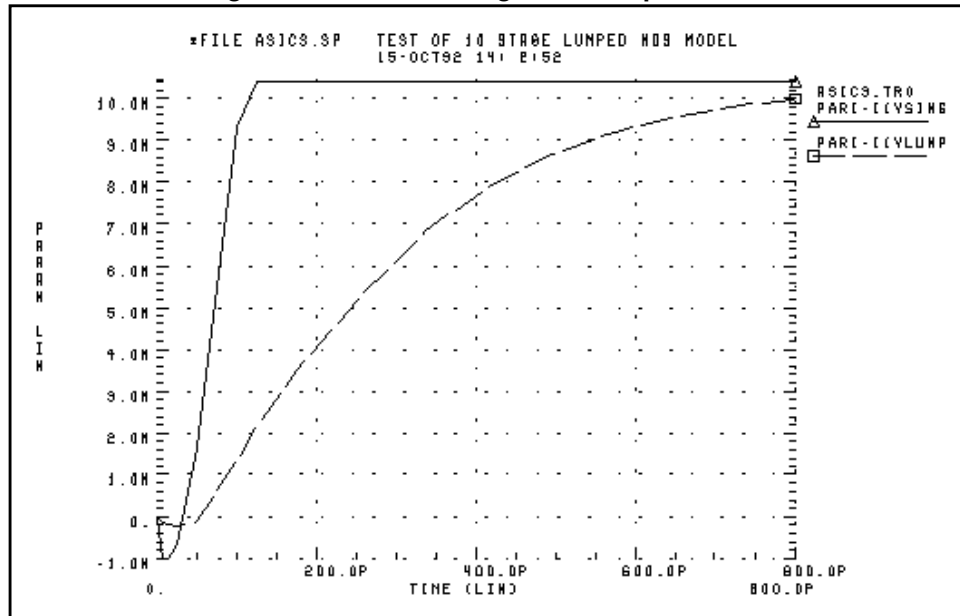
Figure 22-13: Nine-stage Ladder Model

In this example, the nine-stage ladder model is in a data file, `$installdir/demo/hspice/apps /asic3.sp`, which Star-Hspice optimizes (using measured data from a wide channel transistor, as the target data). Optimization produces a nine-stage ladder model, which matches the timing characteristics of the physical data. Star-Hspice compares the simulation results for the nine-stage ladder model, and the one-stage model, using the nine-stage ladder model as the reference. The one-stage model results are about 10% faster than actual physical data indicates.

The following is an example of a Nine-Stage Ladder model:

```
* FILE: ASIC3.SP Test of 9 Stage Ladder Model
.subckt lrgtp drain gate source bulk
m1 drain gate source bulk p w='wt/18' l=lt
m2 drain g1 source bulk p w='wt/9' l=lt
m3 drain g2 source bulk p w='wt/9' l=lt
m4 drain g3 source bulk p w='wt/9' l=lt
m5 drain g4 source bulk p w='wt/9' l=lt
m6 drain g5 source bulk p w='wt/9' l=lt
m7 drain g6 source bulk p w='wt/9' l=lt
m8 drain g7 source bulk p w='wt/9' l=lt
m9 drain g8 source bulk p w='wt/9' l=lt
m10 drain g9 source bulk p w='wt/18' l=lt
r1 gate g1 'wt/lt*rpoly/9'
r2 g1 g2 'wt/lt*rpoly/9'
r3 g2 g3 'wt/lt*rpoly/9'
r4 g3 g4 'wt/lt*rpoly/9'
r5 g4 g5 'wt/lt*rpoly/9'
r6 g5 g6 'wt/lt*rpoly/9'
r7 g6 g7 'wt/lt*rpoly/9'
r8 g7 g8 'wt/lt*rpoly/9'
r9 g8 g9 'wt/lt*rpoly/9'
.ends lrgtp
.end pro
.end
```

Figure 22-14: Asic3 Single vs. Lumped Model



Demonstration Input Files

Table 22-1: Demonstration Input Files (Sheet 1 of 18)

File Name	Description
<i>Algebraic Output Variable Examples \$installdir/demo/hspice/alge</i>	
alg.sp	demonstrates algebraic parameters
alg_fil.sp	magnitude response of the behavioral filter model
alg_vco.sp	voltage-controlled oscillator
alg_vf.sp	voltage-to-frequency converter, behavioral model
xalg1.sp	QA of parameters
xalg2.sp	QA of parameters
<i>Applications of General Interest \$installdir/demo/hspice/apps</i>	
alm124.sp	AC, noise, and transient op-amp analysis
alter2.sp	.ALTER examples
ampg.sp	pole/zero analysis of a G source amplifier
asic1.sp	ground bounce, for I/O CMOS driver
asic3.sp	ten-stage lumped MOS model
bjt2bit.sp	BJT two-bit adder
bjt4bit.sp	four-bit, all NAND gate, binary adder
bjtdiff.sp	BJT diff amp, with every analysis type
bjtschmt.sp	bipolar Schmidt trigger
bjtsense.sp	bipolar sense amplifier
cellchar.sp	characteristics of ASIC inverter cell

Table 22-1: Demonstration Input Files (Sheet 3 of 18)

File Name	Description
dlatch.sp	CMOS D-latch, using behaviorals
galg1.sp	sampling a sine wave
idealop.sp	ninth-order low-pass filter
integ.sp	integrator circuit
invb_op.sp	optimizes the CMOS macromodel inverter
ivx.sp	characteristics of the PMOS and NMOS, as a switch
op_amp.sp	op-amp, from Chua and Lin
pd.sp	phase detector, modeled as switches
pdb.sp	phase detector, using behavioral NAND gates
pwl10.sp	operational amplifier, used as a voltage follower
pwl2.sp	PPW-VCCS, with a gain of 1 amp/volt
pwl4.sp	eight-input NAND gate
pwl7.sp	modeling inverter, using a PWL VCVS
pwl8.sp	smoothing the triangle waveform, using the PWL CCCS
ring5bm.sp	five-stage ring oscillator – macromodel CMOS inverter
ringb.sp	ring oscillator, using behavioral model
sampling.sp	sampling a sine wave
scr.sp	silicon-controlled rectifier, modeled using the PWL CCVS
swcap5.sp	fifth-order elliptic switched capacitor filter
switch.sp	test for PWL switch element
swrc.sp	switched capacitor RC circuit

Table 22-1: Demonstration Input Files (Sheet 4 of 18)

File Name	Description
triode.sp	triode model family of curves, using behavioral elements
triorex.sp	triode model family of curves, using behavioral elements
tunnel.sp	modeling tunnel diode characteristic, using PWL VCCS
vcob.sp	voltage-controlled oscillator, using PWL functions
<i>Benchmarks</i> <i>\$installdir/demo/hspice/bench</i>	
bigmos1.sp	large MOS simulation
demo.sp	quick demo file, to test installation
m2bit.sp	72-transistor two-bit adder – typical cell simulation
m2bitf.sp	fast simulation example
m2bitsw.sp	fast simulation example – same as m2bitf.sp, but using behavioral elements
senseamp.sp	bipolar analog test case
<i>Timing Analysis</i> <i>\$installdir/demo/hspice/bisect</i>	
fig3a.sp	DFF bisection search, for setup time
fig3b.sp	DFF early, optimum, and late setup times
inv_a.sp	inverter bisection (pass-fail)
<i>BJT and Diode Devices</i> <i>\$installdir/demo/hspice/bjt</i>	
bjtbeta.sp	plot BJT beta
bjtft.sp	plot BJT FT, using s-parameters
bjtgmm.sp	plot BJT Gm, Gpi
dpntun.sp	junction tunnel diode

Table 22-1: Demonstration Input Files (Sheet 5 of 18)

File Name	Description
snaphsp.sp	convert SNAP to Star-Hspice
tun.sp	tunnel oxide diode
<i>Cell Characterization</i> <i>\$installdir/demo/hspice/cchar</i>	
dff.sp	DFF bisection search, for setup time
inv3.sp	characteristics of an inverter
inva.sp	characteristics of an inverter
invb.sp	characteristics of an inverter
load1.sp	inverter sweep, delay versus fanout
setupbsc.sp	setup characteristics
setupold.sp	setup characteristics
setuppas.sp	setup characteristics
sigma.sp	sweep MOSFET -3 sigma to +3 sigma, using measure output
tdgtl.a2d	Viewsim A2D Star-Hspice input file
tdgtl.d2a	Viewsim D2A Star-Hspice input file
tdgtl.sp	two-bit adder, using D2A Elements
<i>Circuit Optimization</i> <i>\$installdir/demo/hspice/ciropt</i>	
ampgain.sp	set unity gain frequency of a BJT diff pair
ampopt.sp	optimize area, power, speed of a MOS amp
asic2.sp	optimize speed, power of a CMOS output buffer
asic6.sp	find best width of a CMOS input buffer
delayopt.sp	optimize group delay of an LCR circuit

Table 22-1: Demonstration Input Files (Sheet 6 of 18)

File Name	Description
lpopt.sp	match lossy filter to ideal filter
opttemp.sp	find first and second temperature coefficients of a resistor
rcopt.sp	optimize speed or power, for an RC circuit
<i>DDL</i> <i>\$installdir/demo/hspice/ddl</i>	
ad8bit.sp	eight-bit A/D flash converter
alf155.sp	characteristics of National JFET op-amp
alf156.sp	characteristics of National JFET op-amp
alf157.sp	characteristics of National JFET op-amp
alf255.sp	characteristics of National JFET op-amp
alf347.sp	characteristics of National JFET op-amp
alf351.sp	characteristics of National wide-bandwidth, JFET input, op-amp
alf353.sp	characteristics of National wide-bandwidth, dual JFET input, op-amp
alf355.sp	characteristics of Motorola JFET, op-amp
alf356.sp	characteristics of Motorola JFET, op-amp
alf357.sp	characteristics of Motorola JFET, op-amp
alf3741.sp	
alm101a.sp	
alm107.sp	characteristics of National op-amp
alm108.sp	characteristics of National op-amp
alm108a.sp	characteristics of National op-amp

Table 22-1: Demonstration Input Files (Sheet 7 of 18)

File Name	Description
alm118.sp	characteristics of National op-amp
alm124.sp	characteristics of National low-power, quad op-amp
alm124a.sp	characteristics of National low-power, quad op-amp
alm158.sp	characteristics of National op-amp
alm158a.sp	characteristics of National op-amp
alm201.sp	characteristics of LM201 op-amp
alm201a.sp	characteristics of LM201 op-amp
alm207.sp	characteristics of National op-amp
alm208.sp	characteristics of National op-amp
alm208a.sp	characteristics of National op-amp
alm224.sp	characteristics of National op-amp
alm258.sp	characteristics of National op-amp
alm258a.sp	characteristics of National op-amp
alm301a.sp	characteristics of National op-amp
alm307.sp	characteristics of National op-amp
alm308.sp	characteristics of National op-amp
alm308a.sp	characteristics of National op-amp
alm318.sp	characteristics of National op-amp
alm324.sp	characteristics of National op-amp
alm358.sp	characteristics of National op-amp
alm358a.sp	characteristics of National op-amp

Table 22-1: Demonstration Input Files (Sheet 8 of 18)

File Name	Description
alm725.sp	characteristics of National op-amp
alm741.sp	characteristics of National op-amp
alm747.sp	characteristics of National op-amp
alm747c.sp	characteristics of National op-amp
alm1458.sp	characteristics of National dual op-amp
alm1558.sp	characteristics of National dual op-amp
alm2902.sp	characteristics of National op-amp
alm2904.sp	characteristics of National op-amp
amc1458.sp	characteristics of Motorola internally-compensated, high-performance op-amp
amc1536.sp	characteristics of Motorola internally-compensated, high-voltage op-amp
amc1741.sp	characteristics of Motorola internally-compensated, high-performance op-amp
amc1747.sp	characteristics of Motorola internally-compensated, high-performance op-amp
ane5534.sp	characteristics of TI low-noise, high-speed op-amp
anjm4558.sp	characteristics of TI dual op-amp
anjm4559.sp	characteristics of TI dual op-amp
anjm4560.sp	characteristics of TI dual op-amp
aop04.sp	characteristics of PMI op-amp
aop07.sp	characteristics of PMI ultra-low offset voltage, op-amp

Table 22-1: Demonstration Input Files (Sheet 9 of 18)

File Name	Description
aop14.sp	characteristics of PMI op-amp
aop15b.sp	characteristics of PMI precision JFET input, op-amp
aop16b.sp	characteristics of PMI precision JFET input, op-amp
at094cns.sp	characteristics of TI op-amp
atl071c.sp	characteristics of TI low-noise, op-amp
atl072c.sp	characteristics of TI low-noise, op-amp
atl074c.sp	characteristics of TI low-noise, op-amp
atl081c.sp	characteristics of TI JFET op-amp
atl082c.sp	characteristics of TI JFET op-amp
atl084c.sp	characteristics of TI JFET op-amp
atl092cp.sp	characteristics of TI op-amp
atl094cn.sp	characteristics of TI op-amp
aupc358.sp	characteristics of NEC general, dual op-amp
aupc1251.sp	characteristics of NEC general, dual op-amp
j2n3330.sp	characteristics of JFET 2n3330 I-V
mirf340.sp	characteristics of IRF340 I-V
t2n2222.sp	characteristics of BJT 2n2222

Table 22-1: Demonstration Input Files (Sheet 10 of 18)

File Name	Description
<i>Device Optimization</i> <i>\$installdir/demo/hspice/devopt</i>	
beta.sp	LEVEL=2 beta optimization
bjtopt.sp	s-parameter optimization of a 2n6604 BJT
bjtopt1.sp	2n2222 DC optimization
bjtopt2.sp	2n2222 Hfe optimization
d.sp	diode, multiple temperatures
dcopt1.sp	1n3019 diode, I-V and C-V optimization
gaas.sp	JFET optimization
jopt.sp	300u/1u GaAs FET, DC optimization
jopt2.sp	JFET optimization
joptac.sp	300u/1u GaAs FET, 40 MHz–20 GHz, s-parameter optimization
l3.sp	MOS LEVEL 3 optimization
l3a.sp	MOS LEVEL 3 optimization
l28.sp	LEVEL=28 optimization
ml2opt.sp	MOS LEVEL=2 I-V optimization
ml3opt.sp	MOS LEVEL=3 I-V optimization
ml6opt.sp	MOS LEVEL=6 I-V optimization
ml13opt.sp	MOS LEVEL=13 I-V optimization
opt_bjt.sp	2n3947 forward and reverse Gummel optimization

Table 22-1: Demonstration Input Files (Sheet 11 of 18)

File Name	Description
<i>Fourier Analysis</i> <i>\$installdir/demo/hspice/fft</i>	
am.sp	FFT analysis, AM source
bart.sp	FFT analysis, Bartlett window
black.sp	FFT analysis, Blackman window
dist.sp	FFT analysis, second harmonic distortion
exam1.sp	FFT analysis, AM source
exam3.sp	FFT analysis, high-frequency signal detection test
exam4.sp	FFT analysis, small-signal harmonic distortion test
exp.sp	FFT analysis, exponential source
fft.sp	FFT analysis, transient, sweeping a resistor
fft1.sp	FFT analysis, transient
fft2.sp	FFT analysis, on the product of three waveforms
fft3.sp	FFT analysis, transient, sweeping frequency
fft4.sp	FFT analysis, transient, Monte Carlo Gaussian distribution
fft5.sp	FFT analysis, data-driven transient analysis
fft6.sp	FFT analysis, sinusoidal source
gauss.sp	FFT analysis, Gaussian window
hamm.sp	FFT analysis, Hamming window
hann.sp	FFT analysis, Hanning window
harris.sp	FFT analysis, Blackman-Harris window
intermod.sp	FFT analysis, intermodulation distortion

Table 22-1: Demonstration Input Files (Sheet 12 of 18)

File Name	Description
kaiser.sp	FFT analysis, Kaiser window
mod.sp	FFT analysis, modulated pulse
pulse.sp	FFT analysis, pulse source
pwl.sp	FFT analysis, piecewise linear source
rect.sp	FFT analysis, rectangular window
rectan.sp	FFT analysis, rectangular window
sffm.sp	FFT analysis, single-frequency FM source
sine.sp	FFT analysis, sinusoidal source
swcap5.sp	FFT analysis, fifth-order elliptic, switched-capacitor filter
tri.sp	FFT analysis, rectangular window
win.sp	FFT analysis, window test
window.sp	FFT analysis, window test
winreal.sp	FFT analysis, window test
<i>Filters</i> <i>\$installdir/demo/hspice/filters</i>	
fbp_1.sp	bandpass LCR filter, measurement
fbp_2.sp	bandpass LCR filter, pole/zero
fbpnet.sp	bandpass LCR filter, s-parameters
fbprlc.sp	LCR AC analysis, for resonance
fhp4th.sp	high-pass LCR, fourth-order Butterworth filter
fkerwin.sp	pole/zero analysis of Kerwin's circuit
flp5th.sp	low-pass, fifth-order filter

Table 22-1: Demonstration Input Files (Sheet 13 of 18)

File Name	Description
flp9th.sp	low-pass, ninth-order FNDP, with ideal op-amps
micro1.sp	test of microstrip
micro2.sp	test of microstrip
tcoax.sp	test of RG58/AU coax
trans1m.sp	FR-4, printed-circuit, lumped transmission line
<i>Magnetics</i> <i>\$installdir/demo/hspice/mag</i>	
aircore.sp	air-core transformer circuit
bhloop.sp	b-h loop, non-linear, magnetic-core transformer
mag2.sp	three primary, two secondary, magnetic-core transformer
magcore.sp	magnetic-core transformer circuit
royerosc.sp	Royer magnetic-core oscillator
<i>MOSFET Devices</i> <i>\$installdir/demo/hspice/mos</i>	
bsim3.sp	LEVEL=47 BSIM3 model
cap13.sp	plot MOS capacitances, LEVEL=13 model
cap_b.sp	capacitances for LEVEL=13 model
cap_m.sp	capacitance for LEVEL=13 model
capop0.sp	plot MOS capacitances, LEVEL=2
capop1.sp	plot MOS capacitances, LEVEL=2
capop2.sp	plot MOS capacitances, LEVEL=2
capop4.sp	plot MOS capacitances, LEVEL=6
chrgpump.sp	charge-conservation test, LEVEL=3

Table 22-1: Demonstration Input Files (Sheet 14 of 18)

File Name	Description
iiplot.sp	plot of impact ionization current
ml6fex.sp	plot temperature effects, LEVEL=6
ml13fex.sp	plot temperature effects, LEVEL=13
ml13ft.sp	s-parameters, for LEVEL=13
ml13iv.sp	plot I-V, for LEVEL=13
ml27iv.sp	plot I-V, for LEVEL=27 SOSFET
mosiv.sp	plot I-V, for files that you include
mosivcv.sp	plot I-V and C-V, for LEVEL=3
qpulse.sp	charge-conservation test, LEVEL=6
qswitch.sp	charge-conservation test, LEVEL=6
selector.sp	automatic model selector for width and length
tgam2.sp	LEVEL=6, gamma model
tmos34.sp	MOS LEVEL=34 EPFL, test DC
<i>Radiation Effects</i> <i>\$installdir/demo/hspice/rad</i>	
brad1.sp	example of bipolar radiation effects
brad2.sp	example of bipolar radiation effects
brad3.sp	example of bipolar radiation effects
brad4.sp	example of bipolar radiation effects
brad5.sp	example of bipolar radiation effects
brad6.sp	example of bipolar radiation effects
drad1.sp	example of diode radiation effects

Table 22-1: Demonstration Input Files (Sheet 15 of 18)

File Name	Description
drad2.sp	example of diode radiation effects
drad4.sp	example of diode radiation effects
drad5.sp	example of diode radiation effects
drad6.sp	example of diode radiation effects
dradarb2.sp	example of diode radiation effects
jex1.sp	example of JFET radiation effects
jex2.sp	example of JFET radiation effects
jprad1.sp	example of JFET radiation effects
jprad2.sp	example of JFET radiation effects
jprad4.sp	example of JFET radiation effects
jrad1.sp	example of JFET radiation effects
jrad2.sp	example of JFET radiation effects
jrad3.sp	example of JFET radiation effects
jrad4.sp	example of JFET radiation effects
jrad5.sp	example of JFET radiation effects
jrad6.sp	example of JFET radiation effects
mrاد1.sp	example of MOSFET radiation effects
mrاد2.sp	example of MOSFET radiation effects
mrاد3.sp	example of MOSFET radiation effects
mrاد3p.sp	example of MOSFET radiation effects
mrاد3px.sp	example of MOSFET radiation effects

Table 22-1: Demonstration Input Files (Sheet 16 of 18)

File Name	Description
rad1.sp	example of total MOSFET dose
rad2.sp	diode photo-current test circuit
rad3.sp	diode photo-current test circuit, RLEV=3
rad4.sp	diode photo-current test circuit
rad5.sp	BJT photo-current test circuit, with an NPN transistor
rad6.sp	BJT secondary photo-current effect, which varies with R1
rad7.sp	BJT RLEV=6 example (semi-empirical model)
rad8.sp	JFET RLEV=1 example, with Wirth-Rogers square pulse
rad9.sp	JFET stepwise-increasing radiation source
rad10.sp	GaAs RLEV=5 example (semi-empirical model)
rad11.sp	NMOS E-model, LEVEL=8, with Wirth-Rogers square pulse
rad12.sp	NMOS 0.5x resistive voltage-divider
rad13.sp	three-input NMOS NAND gate, with non-EPI, EPI, and SOS examples
rad14.sp	GaAs differential-amplifier circuit
rad14dc.sp	n-channel JFET, DC I-V curves
<i>Sources</i> <i>\$installdir/demo/hspice/sources</i>	
amsrc.sp	amplitude modulation
exp.sp	exponential independent source
pulse.sp	test of pulse
pwl.sp	repeated piecewise-linear source

Table 22-1: Demonstration Input Files (Sheet 17 of 18)

File Name	Description
pwl10.sp	op-amp, voltage follower
rtest.sp	voltage-controlled resistor, inverter chain
sffm.sp	single-frequency, FM modulation source
sin.sp	sinusoidal source, waveform
vcr1.sp	switched-capacitor network, using G-switch
<i>Transmission Lines</i> <i>\$installdir/demo/hspice/tline</i>	
fr4.sp	microstrip test, FR-4 PC board material
fr4o.sp	optimizing model, for microstrip FR-4 PC board material
fr4x.sp	FR4 microstrip test
hd.sp	ground bounce, for I/O CMOS driver
rcsnubts.sp	ground bounce, for I/O CMOS driver, at snubber output
rcsnubtt.sp	ground bounce, for I/O CMOS driver
strip1.sp	two microstrips, in series (8 mil and 16 mil wide)
strip2.sp	two microstrips, coupled together
t14p.sp	1400 mil by 140 mil, 50-ohm tline, on FR-4, 50 MHz to 10.05 GHz
t14xx.sp	1400 mil by 140 mil, 50-ohm tline, on FR-4 optimization
t1400.sp	1400 mil by 140 mil, 50-ohm tline, on FR-4 optimization
tcoax.sp	RG58/AU coax, with 50-ohm termination
tfr4.sp	microstrip test
tfr4o.sp	microstrip test

Table 22-1: Demonstration Input Files (Sheet 18 of 18)

File Name	Description
tl.sp	series source, coupled and shunt-terminated transmission lines
transmis.sp	algebraics, and lumped transmission lines
twin2.sp	twin-lead model
xfr4.sp	microstrip test sub-circuit, expanded
xfr4a.sp	microstrip test sub-circuit, expanded, larger ground-resistance
xfr4b.sp	microstrip test
xulump.sp	test 5-, 20-, and 100-lump, U models



Appendix A

FAQ/Troubleshooting

This appendix contains frequently asked questions (FAQ), and their solutions, for the following topics:

- [Analysis](#)
- [Documentation](#)
- [Environment Variables](#)
- [Error Messages](#)
- [Input](#)
- [Installation Issues](#)
- [Licensing/Access Issues](#)
- [Limitations](#)
- [Miscellaneous](#)
- [Models](#)
- [MS Windows/PC Issues](#)
- [Netlist/Options](#)
- [Output](#)
- [W Element/Field Solver](#)
- [Waveform Viewing](#)

Solution numbers, at the end of each question and answer, refer to the solution number in the Avant! internal Defect/Enhancement (D/E) system.

Analysis

Q: How do I make .TRAN and .TRAN SWEEP results agree?

A: If transient simulations return different results than a parameterized transient sweep (and if you are using transmission line elements), you must explicitly set the RISETIME option. For example:

```
.OPTION risetime=<rise time of fastest component>
```

Solution: 28

Q: Why does Star-Hspice show *Internal timestep too small* when I use UIC in the .TRAN statement?

A: If you use UIC in the .TRAN statement, Star-Hspice bypasses the DC operating point analysis, and starts a transient analysis, using node voltages that you specify in an .IC statement.

During a simulation, DC operating point analysis is an important step. Most DC analysis algorithms, control options, initializations, and convergence, also apply to transient analysis. Although a transient analysis might provide a convergent DC solution, transient analysis can still fail to converge, if you use UIC. In transient analysis, the *internal timestep too small error* message indicates that the circuit failed to converge. The convergence failure might be due to stated initial conditions that are not close enough to the actual DC operating point values.

Therefore, the best solution is to comment out the UIC from the .TRAN statement, and rerun the simulation. At that point, Star-Hspice performs the DC operating point analysis, and uses the correct values in the transient analysis.

Solution: 383

Q: Can I change the seed that pseudo-randomly generates the initial conditions in the Monte Carlo simulation?

A: Yes, you can set `.OPTION seed=<value>`, where *value* is between 1 and 259200. This changes the starting seed for the random number generator.

Solution: 408

Documentation

Q: Whom should I contact to order Star-Hspice manuals, and other documents?

A: To purchase the latest *Star-Hspice* documents, call Ana Wagem at (510) 413-8383 or email:

ana_wageman@avanticorp.com

Solution: 41

Q: Are the demo files, used in the *Star-Hspice User Guide*, available on-line?

A: No, they are not on-line. However, they are in the directory where you install the Star-Hspice and AvanWaves executables:

1. From the directory where you install Star-Hspice (for example: /hspice), point to:
/hspice/99.4/demo/
2. Two subdirectories appear:
 - /hspice directory, which contains several subdirectories (such as alge, apps, bench, bisect, bjt, and so on). Each subdirectory includes several demo netlists, relative to the title of that subdirectory.
 - /awaves directory contains a subdirectory, named tutorial, which contains AC, DC, and TRAN subdirectories. Each subdirectory contains one demo file, relative to its title.

Solution: 376

Q: How do I obtain a copy of the *Star-Hspice* documentation?

A: First, check the Star-Hspice installation directory for a docs folder. If the manual is not there, check the Star-Hspice CD for the documents.

- ❑ For Unix, they are in a file named 200*.*_hspice_docs.tar.gz, or something similar.
- ❑ For Windows, rerun the installation. One of the options during the installation is to select the items to install (Star-Hspice, AvanWaves, and documentation).

If none of these options work, send an email to:

`hspice_nw@avanticorp.com`

You can set up an FTP account, to download the electronic documents.

For hardcopies, email:

`hspice_nw@avanticorp.com`

for the necessary contact information.

Solution: 460

Q: Are any demo files included with the PC version of the software?

A: Yes, during installation, you can install demo files.

Solution: 503

Environment Variables

Q: What does the META_QUEUE environment variable do?

A: If you enable META_QUEUE, you can queue licenses. For example, if you purchased five Star-Hspice floating licenses, if all five licenses are checked out, and if you enabled META_QUEUE, then the next simulation job that you submit waits in a queue, until a license is available. If you did not enable META_QUEUE, then Star-Hspice reports an error message, that no licenses are available.

Note: If your license file contains more than one Star-Hspice license token (INCREMENT line), and if the version dates are different, then the FLEXlm license manager queues only the first token in your license file. FLEXlm queues for the first INCREMENT line that satisfies the request. If you specify two INCREMENT lines with different versions, FLEXlm creates two license pools on the server. When you issue the queuing request, the server tries to satisfy the request. If it cannot, then the server queues for the first INCREMENT line that satisfies the request. After it queues for that particular INCREMENT line, the server waits for this line to become free. It does not continually look for any other line that might satisfy this request. This is normal operation for FLEXlm.

Solution: 367

Error Messages

Q: FLEXlm reports *inconsistent license key* error messages when I start up lmgrd. Why? What should I do?

A: This occurs if the encryption code that avantd calculates, is different from the encryption key used to generate the license key. FLEXlm uses the following pieces of information from the license key: the host id of the server, the software expiration date, the increment name, and the software version.

If you change any of these fields, FLEXlm reports an inconsistent license key error. This can also happen if you use different versions of avantd and lmgrd; however, it is very unlikely that this will cause a problem.

Avant! does not recommend using lmreread to start a new license. To shut down the license server, use:

```
lmdown -c /path/to/license/file
```

and then start the new license server in the normal manner.

Solution: 134

Q: Star-Hspice reports a TCL error: *Cannot find config.status*, when I run the \$installdir/bin/config script to configure Star-Hspice/AvanWaves. Why?

A: Your installation is probably fine. No matter what products you choose to install, the configuration script tries to display the config.status file. However, not all installation options generate a config.status file.

To verify that the configuration was successful, read the automatically-generated logfile. The configuration script places this file in /tmp/avanti_####/config.log, where #### are numbers specific to the date and time of the installation.

Solution: 137

Q: Star-Hspice reports the following error message:

```
** error**: iob_loads2:9612: Number of input voltage
sources in IOB = 0
```

What does this mean?

A: You must connect certain nodes in IBIS elements, to ideal sources. For example, the input node in an output buffer, must connect to an ideal source. For more information about which nodes must be connected, see Chapter 7, “Using IBIS Models”, in the *True-Hspice Device Models Reference Manual*.

Solution: 145

Q: Why is the simulate button grayed- out (disabled) in the Star-Hspice User Interface (HSPUI), even if Star-Hspice is not currently running a simulation?

A: This is usually caused by a computer crash during a simulation. To enable the simulate button, type in a command prompt:

```
“del %tmp%\job.run”
```

Alternatively, you can search for the job.run file, and delete it.

Solution: 293

Q: Why does Star-Hspice show an *inductor/voltage source loop* error?

A: This is normally caused by:

1. A voltage source, with an inductor connected to the same point.
2. A voltage source, with an inductor connected directly across its nodes.
3. Two or more inductors, connected in a loop, with unlimited current.

The best way to solve this type of error is to connect a small series resistance (1 nano-ohm or smaller usually works).

Solution: 464

Q: Star-Hspice reports *Time: command terminated abnormally*. Why?

A: This means that Star-Hspice stops unexpectedly, which can be due to a number of problems (such as convergence problems or a numerical overflow). Look at the *.lis and *.st0 files for clues to what happened. Also, try to find out if the simulation concluded using a different version of Star-Hspice, or on other platforms. If it did, it could be a bug. Otherwise, it is most likely a problem in the designs or model cards.

The Unix TIME utility, not Star-Hspice itself, outputs the *Time: command terminated abnormally* message. The TIME utility reports various system use information about the processes it monitors. If a monitored process stops unexpectedly, TIME issues this warning.

Solution: 520

Q: Why does my license.log file show the following error, when I try to start the avantd daemon:

ATTENTION ... "avantd" vendor daemon CAN NOT co-exist with monarc lockfile * error (-1) with "monarc", program aborted... Please correct problem and restart daemons

A: This is caused by an earlier Avanti vendor daemon. Look in the /usr/tmp directory, for a file named either lockmonarcd or lock.monarcd. Delete the lock file, and restart lmgrd. The avantd daemon can now start.

Solution: 528

Q: Why do Star-Hspice and AvanWaves report a *configure unsuccessful* error?

A: If you previously set \$installdir, when you install a new version of Star-Hspice, the path for the configuration is not correct (it will try to install over your previous installation). This causes a *configure unsuccessful* error. To fix this, set the correct path to your install directory, in the configure utility.

Solution: 561

Q: Why does Star-Hspice report: ***error**: system file descriptors may be used up?*

A: The operating system reports this error, if Star-Hspice tries to open more files than your operating system permits. If you use a large number of .ALTER statements, or if you read-in a lot of files (.LIB or .INCLUDE), increase the number of files that you can open at one time.

In sh/ksh/bash, use `ulimit` to display or change the limits for the current shell, and for all processes that you launch from this shell.

To display the current limit, use:

```
$ ulimit -n
```

For example, if the limit is 64, then any process that you launch from this shell cannot open more than 64 files at once. To raise this limit (in this example, to 256), type:

```
$ ulimit -n 256
```

Operating systems also have a “hard” limit that you cannot exceed (although the root user change the hard limit). To see what the hard limit is on your system, type:

```
$ ulimit -Hn
```

On csh/tcsh, use the built-in limit to do this (type `limit` to see a list of limits). The following is a limit example:

```
cputime      unlimited
filesize     unlimited
datasize     2097148 kbytes
stacksize    8192 kbytes
coredumpsize unlimited
descriptors  64
memorysize   unlimited
```

To increase the number of open files (file descriptors) to 256, type:

```
$ limit descriptors 256
```

To view the hard limits, type:

```
$ limit -h
```

Solution: 613

Q: Why does Star-Hspice report: ****error** reference 0:nch not found**, even when NCH appears in my library?

A: Star-Hspice has an *automatic model selection* feature, which your library probably uses. If a model name is followed by a period and a suffix, then the model uses this feature. For example:

```
.MODEL nch.1 nmos . . .
```

Your input netlist probably includes multiple `.model nch.*` statements, which define the model's parameters for different sizes of the transistor. These models use LMIN, LMAX, WMIN, and WMAX to determine which model applies to a specified transistor dimension.

For example, if `model nch.1` includes `LMIN=0.15u LMAX=0.25u WMIN=0.15u WMAX=1.0u`, then the model applies only to transistors with lengths between 0.15um and 0.25um, and widths between 0.15um and 1.0um.

If you see a *reference not found* error, then your transistor dimensions (L and W) do not fall within the specified ranges, in any of the models in your library.

Solution: 642

Input

Q: Why does Star-Hspice report *input file contains no data*?

A: This occurs if you did not include a `.END` statement, followed by `Return`, at the end of the netlist. Ensure that `.END` is the last statement in the netlist, and do not forget to insert a `Return`.

Solution: 44

Q: Why does an I/O Buffer (used as an input) have a much smaller input current, than an input buffer has?

A: You must include the power clamp and ground clamp nodes in the calling card, when you use an I/O buffer as an input buffer.

Solution: 455

Installation Issues

Q: Whom should I contact, to answer questions about Star-Hspice installation?

A: To contact the Star-Hspice technical support team, submit a question to:
hspice_nw@avantiacorp.com

To ensure that Avant! thoroughly understands your question, provide *all* pertinent details.

Solution: 41

Q: I am having trouble installing Star-Hspice on my Windows NT machine. Do you have any suggestions?

A: A common solution is to set Windows NT environment variables (such as \$installdir, LM_LICENSE_FILE, and PATH) in *ControlPanel->System*; these are not set by default.

Solution: 56

Q: I recently upgraded or installed a newer (post-97.4) Star-Hspice version; however, now I cannot simulate, using Analog Artist from Cadence. Star-Hspice issues an error message about not being able to find `permit.hsp`. Or perhaps Analog Artist is spawning the incorrect version of Star-Hspice. What do I do?

A: Most Cadence Composer/Analog Artist installations create a script that opens the Cadence framework. This script sets the \$installdir environment variable. Edit this script, to point the \$installdir variable to the new Star-Hspice installation directory (this directory contains the bin directory, which contains the Star-Hspice and AvanWaves scripts).

Solution: 136

Q: I am having problems installing my new node-locked Star-Hspice license. Star-Hspice reports an *invalid hostid* error, and shows a hostid of 0, but I have the dongle installed! What should I do?

A: Node-locked versions of Star-Hspice on the PC require a parallel-port dongle. Under WinNT, you must install a driver that can use the dongle. The installation program is `setupx86.exe` in the `%installdir%` directory. If you use Win98/95, you do not need to run the `setupx86.exe` sentinel driver.

Solution: 146

Licensing/Access Issues

Q: Whom should I contact to answer questions about Star-Hspice license files?

A: To request a new license file, send email to:
license@avanticorp.com

To ensure that Avant! thoroughly understands your question, provide *all* pertinent details.

Solution: 41

Q: My older FLEX license does not work with my new Star-Hspice. What should I do?

A: FLEXlm v5.0 needs a 4-digit year in the expiration date; you need a new license.

Solution: 53

Q: How can I run older Star-Hspice versions, without having to source the cshrc.meta file of the older version?

A: Edit the versions file in the latest version of Star-Hspice (for example, *.install*dir. ./99.2/bin/versions), and include the paths of the older versions in the versions file. Then at the command line, type:

hspice

After you enter the input and output file names, you are prompted for a version to run. Select the version number (1,2,3...) to run.

On PC platforms, modify the *...install*dir\99.2\versions file in the same way. To select the version to run, modify the last line in the Star-Hspice User Interface (HSPUI) window.

Solution: 57

Q: When I start-up lmgrd, FLEXlm reports an *inconsistent license key* error.

A: See this solution in [Error Messages on page A-7](#).

Solution: 134

Q: Star-Hspice reports a TCL error: *Cannot find config.status*, when I run the `$installdir/bin/config` script to configure Star-Hspice/AvanWaves.

A: See this solution in [Error Messages on page A-7](#).

Solution: 137

Q: How can I reserve licenses for a particular user, group, or host?

A: In FLEXlm, you can use a configuration file to reserve licenses. The configuration file is an ASCII text file, containing comments and statements. Lines that start with # are comments. To create groups of users, use the following statement:

```
GROUP <groupname> <user> [...]
```

To reserve a specific license to either a user, a group, a host or a display, using one of the following:

```
RESERVE <reserved_num> <feature> USER <username>
RESERVE <reserved_num> <feature> GROUP <groupname>
RESERVE <reserved_num> <feature> HOST <hostname>
RESERVE <reserved_num> <feature> DISPLAY <dispname>
```

Reserving a license for a specific HOST does not prevent users from setting their DISPLAY environment variable to their local DISPLAY and running the tool. In fact, reserving a license for a HOST is very similar to using a node-locked license.

Reserving a license for a DISPLAY, ensures that the DISPLAY environment variable is set to the value that you specified.

Solution: 654

Limitations

Q: What are the known limitations of Metaencryption?

A: The known limitations are:

- ❑ If you use the `.option` search decryption structure, the filenames of the encrypted files must have the same name as the subcircuit definition that they contain (with an `.inc` extension).
- ❑ If you want to use `.LIB` to reference the encrypted files, put `.prot` and `.unpr` between the `.lib` and `.endl` statements in the files to encrypt.
- ❑ Lines must no more than 254 characters long, and must not include semicolons (;).
- ❑ For Star-Hspice 1998.4 and earlier versions, insert dummy models, before you use the `LEVEL 49` model.
- ❑ For every `.PROTECT` statement, include an `.UNPROTECT` statement.
- ❑ Use `.PROTECT` and `.UNPROTECT` in the following order:

Recommended	Not Recommended
<code>.SUBCKT</code>	<code>.SUBCKT</code>
<code>.PROTECT</code>	<code>.PROTECT</code>
~~~	~~~
<code>.UNPROTECT</code>	<code>.ENDS</code>
<code>.ENDS</code>	<code>.UNPROTECT</code>

*Solution: 40*

*****

---

## Miscellaneous

Q: Should I include the gate edge of the source and drain, when I specify the PD (periphery of the drain) and PS (periphery of the source), for calculating parasitic diodes?

A: For ACM=0 or 2, include the gate edge of the source and drain, when you calculate PS and PD. For ACM=3, do *not* include the source or drain edges when you specify PS and PD. ACM=1 does not use PS and PD.

*Solution:* 133

*****

Q: Are parameters defined on the .SUBCKT line different from parameters defined in the subcircuit body?

A: Parameters defined on the .SUBCKT line are in the same namespace as parameters defined within the body of the subcircuit.

The following two subcircuits are equivalent:

```
.SUBCKT sub1 in out l=10u w=5u
.ENDS
```

Or:

```
.SUBCKT sub2 in out
.PARAM l=10u w=5u
.ENDS
```

---

**Note:** Both forms of the subcircuit accept the *l* and *w* parameters on element instance lines.

---

*Solution:* 139

*****

Q: How can I simplify repetitive complex measurements?

A: Encapsulate the functionality of your measurements/probes in subcircuits. Subcircuits do not need to contain elements. They can contain complex measurement routines, to make measurements less tedious. If you carefully choose the instance names, then parsing the output becomes relatively easy. The following example calculates AC transfer functions, for arbitrary nodes in your netlist:

```
.PARAM mag(a,b) = 'sqrt(a*a+b*b)'
.SUBCKT Xfer in out
.PROBE AC xfer

par('mag(vr(in)*vr(out)+vi(in)*vi(out),
+ vi(in)*vr(out)-vi(out)*vr(in))/
+ mag(vr(out),vi(out))')

*xfer = mag(in/out) (note the voltages at in and
+ out are complex)

.ENDS
```

*Solution: 142*

*****

# Models

Q: Why doesn't .OPTION SCALE work in my simulation?

A: Check to see if your MOSFET and model card are wrapped inside a .SUBCKT call. For example:

```
.SUBCKT pmos d g s b w=1 l=1 m=1 mp d g s b p w=w l=l
+ m=m
.MODEL p pmos LEVEL=.....
.ENDS
```

If so, check the model parameters for any dependencies on width or length. In this example, Star-Hspice does not scale these parameters. .OPTION SCALE affects width and length, only when you pass it into the model card, as physical length and width. The model parameter uses width and length before scaling.

*Solution:* 209

*****

Q: How do I obtain proprietary MOSFET models?

A: The following MOSFET model level numbers are vendor-proprietary, and require a special license before you can use them:

AMD	AMD	LEVEL 32
Dallas Semi	DALLAS	LEVEL 19
Hitachi	HIT	LEVEL 43
IBM	IBM	LEVEL 48
Lucent	ATT	LEVEL 45
Motorola	MOT	LEVEL 31
	MOTSSIM	LEVEL 29
National	NAT	LEVEL 33

SGS Thomson	SGSTHOM	LEVEL 46
Sharp	SHARP	LEVEL 36
TI	TI	LEVEL 37
		LEVEL 41
Vitesse	VITESSE	LEVEL 6

As of July 12, 2000, these are the contact people:

AMD	Barbara Vervenne	(512) 602-5783
Dallas Semi	Roy Hensley	(214) 450-3858
Hitachi	Len Mizrah	(408) 456-2081
IBM	Cal Bittner	(902) 769-6528
Lucent	Roberta Kulp	(610) 712-2614
Motorola	Ralph Sokel	(512) 933-5264
National	Chen Young Tao	(408) 721-5665
SGS Thomson	Claude Frank	33 476 92 63 47 (Tel) 33 476 08 96 52 (Fax)
Sharp	Sigenobu Kiyoi	81 7 4365 4273
TI	Jeff Brunson	(972) 480-2481
Vitesse	Aubrey Grey	(805) 388-7517

To receive a license from Avant! (the vendors do not cut the licenses), you must ask the vendor to submit an authorization letter to Avant! After Avant! receives the letter, you can obtain a license file.

*Solution: 263*

*****

Q: Why doesn't BJT LV5 - FT agree with net analysis results?

A: FT is a simple estimate, for a cut-off frequency of the internal transistor:

$$f_t = \text{abs}(g_m - g_o) / (2 * \pi * \max(1.0d-18, C_{be} + C_{bc} + C_{bx}))$$

Star-Hspice extracts the parameters for the equation, from the equivalent circuit for the BJT (see Chapter 4, "Using BJT Models", in the *True-Hspice Device Models Reference Manual*).

FT does not include parasitic elements (such as Rb, Rc, and Re), and does not include the external circuit. FT simply estimates the cut-off frequency of the internal transistor, which cannot replace actual simulation results.

For more information about this topic, see *Semiconductor Device Modeling with SPICE* by Massobrio and Antognetti, McGraw Hill, 1993, page 99.

*Solution:* 560

*****

Q: What is the importance of the following excerpt from the attached .lis file:

```
*****
**warning BSIM3v3 VERSION parameter does not appear in
**the model card!!!
**Failure to explicitly set VERSION may result in
**inaccurate results!
*****
```

A: Avant! implemented the BSIM3v3 models, released from UC Berkeley, in the forms of LEVEL 49 and LEVEL 53. Since implementation, UC Berkeley has released patches, and Avant! has implemented them as 3.1 (default for 98.2), 3.2 (default for 98.2 and 98.4) and 3.22 (default for 99.2, 99.4 and 2000.2).

If your models were developed for the VERSION = 3.1 release, and this parameter is not present in your models/libraries, then Star-Hspice defaults to 3.22 (in the Star-Hspice 2000.2 release), which might return unexpected results.

*Solution:* 562

*****

---

## MS Windows/PC Issues

Q: My Star-Hspice license does not work on my PC. Why?

A: Problems can occur on PC platforms, if license files contain truncated lines. Repair the license, or contact Star-Hspice technical support:

`hspice_nw@avantiacorp.com`

To ensure that Avant! thoroughly understands your question, provide *all* pertinent details.

*Solution:* 55

*****

Q: I am having trouble installing Star-Hspice on my Windows NT machine.

A: Use *ControlPanel->System* to set the Windows NT environment variables (`installdir`, `LM_LICENSE_FILE`, and `PATH`); these are not set by default.

*Solution:* 56

*****

Q: How can I run older Star-Hspice versions, without having to source the `csorc.meta` file of the older version?

A: See this solution in [Licensing/Access Issues on page A-15](#).

*Solution:* 57

*****

- Q: Why does Star-Hspice report *Bad date found in the running machine*, when I start Star-Hspice on Win NT?
- A: Before you try to check out a license, make sure that dates on the Windows NT machine are consistent with the current time (that is, no future creation or modification times). you must change any files with bad dates.

However, Windows NT does not have a utility to easily change file dates. Changing these dates requires a command such as the UNIX touch command, which Windows NT does not provide. A third-party software package *does* have this touch command. *UnixDos Toolkit 4.3d* includes several UNIX commands for the Window NT environment. You can download this toolkit from software sites. After you install this utility, you can *touch* the files, and change their dates to the current date. However, touch does not work with folders. To update the date of a folder, copy the contents of the folder to a new folder (example: *xxx* folder to *xxx1* folder), delete the old folder, and rename the folder to the original name (*xxx*).

Star-Hspice works when you change files and directories in the WINNT folder, to the current date. Other files that have bad (future) dates, but are *not* in the WINNT folder, do not cause any problems. For example, MS Office files and folders can have bad dates, but Star-Hspice still runs.

*Solution:* 341

*****

- Q: The Windows path to license.dat is correct, but Star-Hspice cannot find it.
- A: By default, Windows Explorer does not show the extension of a file. So if, by mistake, your license file has an additional extension besides .dat, then you will see it in the Explorer view, but Star-Hspice cannot find it.

To remedy the problem, open a DOS prompt, switch to the directory that contains your license file, and type `dir` at the prompt.

4. If your license file has another extension, such as .txt or .flx, type:  
`rename license.dat.??? license.dat`

This should rename the file to `license.dat`.

*Solution:* 439

*****



Q: How do I load network files into AvanWaves on a PC?

A: To view network drives in AvanWaves, you must know the path to the drive. To access a file on the server in your home directory, use the path:

\\Server\home\user

Type this in the Open dialog box, in AvanWaves. AvanWaves remembers this path, throughout the current session.

---

**Note:** You must log into the server, and you must have read permission.

---

*Solution: 448*

*****

Q: How do you run Star-Hspice, from a command prompt in Windows?

A: Use the form:

hspice -i input.sp -o output.lis

This opens the Star-Hspice output window (which shows the status of the .stθ file), and runs the input file. When Star-Hspice finishes, it closes the window, and updates the command prompt.

*Solution: 457*

*****

Q: How do I obtain a copy of the Star-Hspice manuals (for Windows)?

A: See this solution in [Documentation on page A-4](#).

*Solution: 460*

*****

Q: If I downloaded Star-Hspice from Windows NT, will it work on all Windows operating systems?

A: Yes, you can run Star-Hspice on Windows 95/98, and NT 3.5 and 4.0.

*Solution: 502*

*****

Q: Are any demo files included with the PC version of the software?

A: Yes. During installation, you can install demos.

*Solution:* 503

*****

---

## Netlist/Options

Q: How do I determine the actual values used for simulation?

A: To determine the actual options used for a simulation, include the following line in your netlist:

```
.OPTION OPTS
```

Star-Hspice prints out the options (and the corresponding values) used in the simulation, into stdout. It also redirects emphasis to a .lis file.

---

**Note:** To find analysis-specific options, you must perform the specified type of analysis, within your netlist.

---

To find the actual element parameters used in the simulation, use element templates, as described in [Element Template Output on page 8-39](#).

*Solution:* 143

```
*****
```

Q: My netlists, which contain both IBIS elements and .ALTER statements, do not always complete successfully, or the answers are odd.

A: This was corrected in Star-Hspice, v1999.4 and later.

*Solution:* 147

```
*****
```

Q: Can I use more than one interface option in my netlist?

A: No. The second interface option overwrites the first. If you first set:

`.OPTION CSDF`

and later in the netlist, you set:

`.OPTION POST`

then Star-Hspice first writes all outputs in Viewlogic format, and later overwrites the outputs in AvanWaves format. In this example, you cannot see the outputs in Viewlogic, but you can see them in AvanWaves.

*Solution: 208*

*****

Q: Why doesn't `.OPTION SCALE` work in my simulation?

A: See this solution in [Models on page A-20](#).

*Solution: 209*

*****

Q: Why does my extracted netlist fail in Star-Hspice?

A: To use an extracted netlist, perform a global search for the colon symbol (:). Replace this character globally, with some unique pattern that is not likely to be in the netlist already. For example, replace the colon symbol with `_c_`.

*Solution: 342*

*****

Q: What is the magnitude difference between the NORM and UNORM options, when you use windowing in the .FFT statement?

A: If you use the UNORM option, instead of the default NORM setting in your .FFT statement, the signal magnitude is different. The reason for this is because different equations govern the different windows. [Table 19-1 on page 19-3](#) shows the equations.

The difference in magnitude is due to *Coherent Power Gain*, which measures the reduction in signal power. The window function suppresses a coherent signal at the end of the measurement interval. Star-Hspice provides this value for each window in the equation column (it is not obvious in all cases). The following table shows the coherent power gain for each case.

Window	Coherent Power Gain	Coherent Power Gain (dB)
Rectangular	1.0	0.0
Bartlett	0.5	-6.021
Hanning	0.5	-6.021
Hamming	0.54	-5.352
Blackman	0.42323	-7.468
Blackman - Harris	0.35875	-8.904
Gaussian (a=3)	0.72	-2.853
Kaiser - Bessel (a=3)	0.4	-7.959

*Solution:* 351

*****

Q: How can I determine what components connect to the same node?

A: The option you probably need is `.OPTION NODE`. This option creates a node cross-reference table, which you can print. The table lists each node, and all elements connected to it. A code indicates the terminal, and a colon (:) separates it from the element name. The codes are:

+	Diode anode
-	Diode cathode
B	BJT base
B	MOSFET or JFET bulk
C	BJT collector
D	MOSFET or JFET drain
E	BJT emitter
G	MOSFET or JFET gate
S	BJT substrate, or MOSFET, JFET source

*Solution:* 360

*****

Q: In PETL, what is the significance of `GRIDFACTOR`?

A: The program uses the predefined number of segments, according to the specified accuracy mode. For higher accuracy than the *accuracy* mode provides, set `GRIDFACTOR` to 2 or 3, to double or triple the number of predefined segments.

*Solution:* 392

*****

Q: Which integration method (TRAP or GEAR) is more accurate?

A: The TRAP (trapezoidal) method is faster and more accurate than GEAR. You can use the GEAR method to remove oscillations that the TRAP algorithm creates. Circuits that do not converge when you use TRAP, often converge when you use GEAR. The GEAR method is suitable for circuits that are inductive in nature, such as switching regulators.

*Solution:* 409

*****

Q: Why doesn't losstangent have any affect on my W Element?

A: To compute the dielectric loss matrix, and to include it in the simulation, you must set .FSOPTIONS computegd in the SPICE deck.

*Solution:* 505

*****

Q: My netlist contains 64 .ALTER statements, but Star-Hspice generates 36 .mtx files. Why?

A: By default, Star-Hspice generates only 36 unique output files for .tr*, .mt*, and so on. After 36 .ALTER statements, Star-Hspice starts from the beginning, and writes over *.mt0 on up. To change this, use either:

.OPTION ALT999

or:

.OPTION ALT9999

The first option lets Star-Hspice write 999 files, before it returns to the 0th file. alt9999 lets you write 9999 output files.

*Solution:* 511

*****

Q: What does the MU option do? The manual says only that it is a trapezoidal integration parameter.

A: The MU option acts as both a flag and a parameter. The equations for trapezoidal integration, and backward-Euler, integration are very similar to each other. The difference between these two is the MU parameter.

- If you set MU to 0.5 (the default), Star-Hspice uses trapezoidal integration.
- If MU=0, then Star-Hspice uses backward-Euler integration.
- If you set MU to a value between 0 and 0.5, then Star-Hspice uses a blend of both methods.

The following is the order of the three integration methods in Star-Hspice (trapezoidal, backward-Euler, Gear), from most accurate to most stable:

Trapezoidal                      (most accurate)

Backward-Euler                .

.

Gear                              (most stable)

*Solution:* 556

*****



# Output

Q: How can I reduce the size of output files?

A: By you add `.OPTION PROBE` to the netlist, only nodes that you explicitly reference in `.PROBE`, `.PRINT`, `.PLOT`, or `.GRAPH` statements, display output values in one of the output files.

*Solution: 43*

*****

Q: The output of my simulation is not what I expected at all. Also, the solution shows instability, or extreme sensitivity to parameter or conductance values.

A: Use `.OPTION PIVOT=1` (or `.OPTION PIVOT=11`). The default pivoting algorithm in Star-Hspice does not select a pivot element. This can lead to instability, if your design uses large conduction ratios (typically when small conductances exist).

*Solution: 135*

*****

Q: I would like `.GRAPH` to write only to a file, and not automatically send the job to the printer.

A: To accomplish this, edit your `$installdir/meta.cfg` file. Comment out all blocks that refer to `PLOTSETUP` and `PRTDEFAULT`. Then enter:

```
PRTDEFAULT = PS
PLOTSETUP PS PSCRIPT
PLOTFILE = /tmp/hspice.ps
END
```

*Solution: 138*

*****

Q: Can I use more than one interface option in my netlist (and how does this affect my output)?

A: See this solution in [Netlist/Options on page A-27](#).

*Solution:* 208

*****

Q: Can I use PRINTDATA in PETL, with only the YES and NO option? When I set PRINTDATA=YES, Star-Hspice did not create an RLGC file.

A: If you specify RLGCFILE, Star-Hspice prints to that file; otherwise, it prints to the standard error output. You can use >& in UNIX, to redirect the standard output. Specify the .PRINTDATA=YES option in .FSOPTIONS, and RLGCFILE in .MODEL cards.

*Solution:* 390

*****

---

## W Element/Field Solver

Q: How do I define an RLGC for a W Element, without R and G?

A: Because the R and G matrix elements are optional, you can leave them empty, or set them to 0, when you define RLGC parameters in the W Element.

*Solution:* 458

*****

Q: Why does the field solver sometimes provide inaccurate results?

A: To receive more accurate results when you use the field solver, use Star-Hspice 1999.4 or later. These newer versions contain many improvements to both the W Element, and the field solver.

*Solution:* 459

*****

Q: Why doesn't losstangent have any affect on my W Element?

A: See this solution in [Netlist/Options on page A-27](#).

*Solution:* 505

*****

---

# Waveform Viewing

Q: How can I view a waveform on an old waveform viewer (such as HSPLLOT), or use third-party tools (such as Viewdraw)?

A: Add the following card to the SPICE deck:

```
.OPTION POST_VERSION = 9007
```

*Solution:* 42

*****

Q: I am performing analysis at different operating temperatures. However, my waveform/measurements are not changing, when I change the temperature of the simulation. Why?

A: The temperature coefficients of most elements defaults to zero. You must specify non-zero coefficients, to derate components in simulation. Do this either on the element instance line, or in the .MODEL card for the elements.

*Solution:* 155

*****

Q: Can I use more than one interface option in my netlist (and if so, how does this affect my waveform viewing)?

A: See this solution in [Netlist/Options on page A-27](#).

*Solution:* 208

*****

Q: How do I create eye diagrams, using Star-Hspice?

A: The following is an example of an eye diagram circuit, in the Star-Hspice distribution:

```
<install_dir>/demo/awaves/demo/eyediag*
```

As a quick reference, an eye diagram imitates the behavior of an oscilloscope (the waveform starts from the left end of the window, and moves to the right at a specified rate). When the electron beam reaches the right end of the window, it moves quickly to the left end of the window, where you can display another *cycle*. You can use this type of display to observe cyclic patterns that vary slightly, from one cycle to another.

To obtain these results in Star-Hspice, include these two statements in your netlist:

```
.PARAM width=5ns phase=0ns
.PROBE TRAN TIME2=par('TIME+phase-int((TIME+phase)/
+ width)*width')
```

These statements create a new TIME2 output variable. You can use the width parameter to adjust the width of the window (in ns). You can also use the phase parameter to adjust the phase of the waveform. To obtain these results in AvanWaves:

- ☐ Open the resulting .tr# file
- ☐ In the *Results Browser* window, under *Types*: select *Params*.
- ☐ Under *Curves*: select *time2*.
- ☐ Under *Current X-Axis*, click on *Apply*.
- ☐ Under *Types*: again, select *Voltages*, *Currents*, or any other type of output variable that you want to display.
- ☐ Double-click on the signals to display.
- ☐ In the viewing window, click on the right mouse button, and select *Monotonic Plot ...*

*Solution:* 611

```
*****
```





## Appendix B

# Interfaces For Design Environments

---

The following design interface products supplement Star-Hspice simulation:

- **AvanLink to Cadence Composer™ and Cadence Analog Artist™**, which interfaces Star-Hspice with Cadence Design Systems' Design Framework II. Using this interface, Star-Hspice can interchange data and cross-probe with Analog Artist and Composer.
- **AvanLink to Mentor Design Architect™**, which interfaces Star-Hspice with Mentor Graphics' Falcon Framework. Using this interface, Star-Hspice can cross-probe with Design Viewpoint Editor, and interchange data with Design Architect (DA) and Design Viewpoint Editor (DVE).

This chapter contains overviews of these interface products:

- [AvanLink to Cadence Composer and Analog Artist](#)
- [AvanLink for Design Architect](#)
- [Viewlogic Links](#)

---

# AvanLink to Cadence Composer and Analog Artist

AvanLink to Cadence Composer and Analog Artist provides:

- A netlister.
- A symbol library.
- Cross-probing.
- Back-annotation capabilities.

AvanLink is an Avant! interface product, which links the Star-Hspice IC circuit simulator with the following products from Cadence:

- Composer, Versions 4.2.1a, 4.2.2, 4.3.2, and 4.3.3
- Analog Artist, Versions 4.2.1a, 4.2.2, 4.3.2, and 4.3.3

## Features

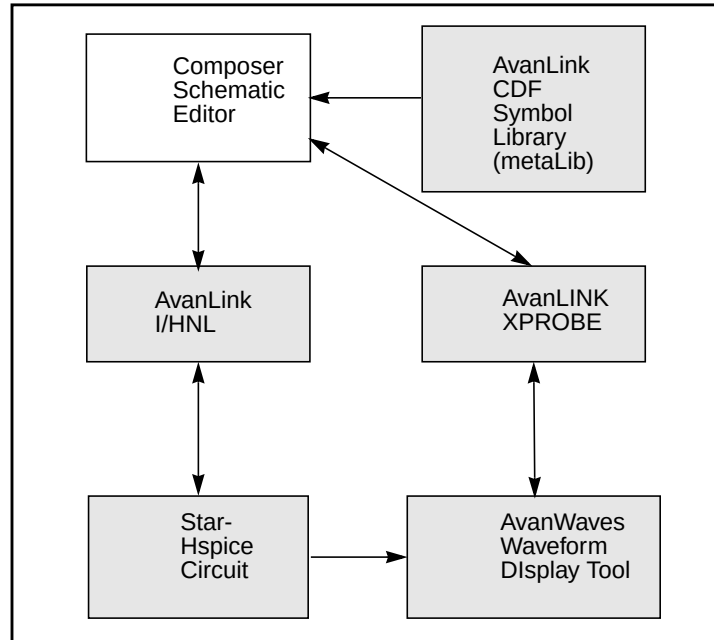
AvanLink provides the following functionality:

- Hierarchical Netlisting (HNL), with incremental capability (IHNL), through the CadenceLink netlister and the Cadence simulation environment.
- Avant! metaLib class-based library, to support all of the Star-Hspice elements. The metaLib library uses the Cadence standard Component Description Format (CDF).
- Cross-probing for both voltage and current, from both Composer and Analog Artist schematics, to the Avant! AvanWaves waveform display tool.
- Hierarchical parameter passing, and algebraic function operations.
- A transparent net name mapping function, which preserves Star-Hspice hierarchical path names.
- Parameter Storage Format (PSF) output program, to display waveforms in Analog Artist.
- Automatic library translation program, for the Cadence sample and analogLib libraries, and for user-defined libraries.
- Back-annotating operating points and element parameters to the schematic.
- Graphical setup of simulation initial conditions.



Figure B-1 shows how AvanLink fits into the design process.

**Figure B-1: AvanLink Architecture for Design Framework II with Composer**

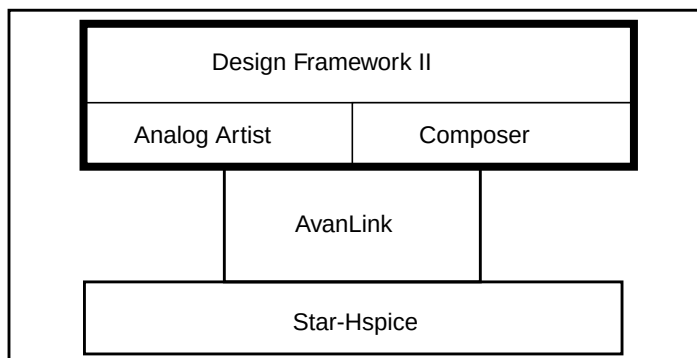


## Environment

AvanLink is an integrated environment, which you can use to configure design, simulation, and display tools, so they support the chosen design flow.

1. Use Composer to develop the design.
2. Use Star-Hspice to simulate it.
3. Use AvanWaves to view it.

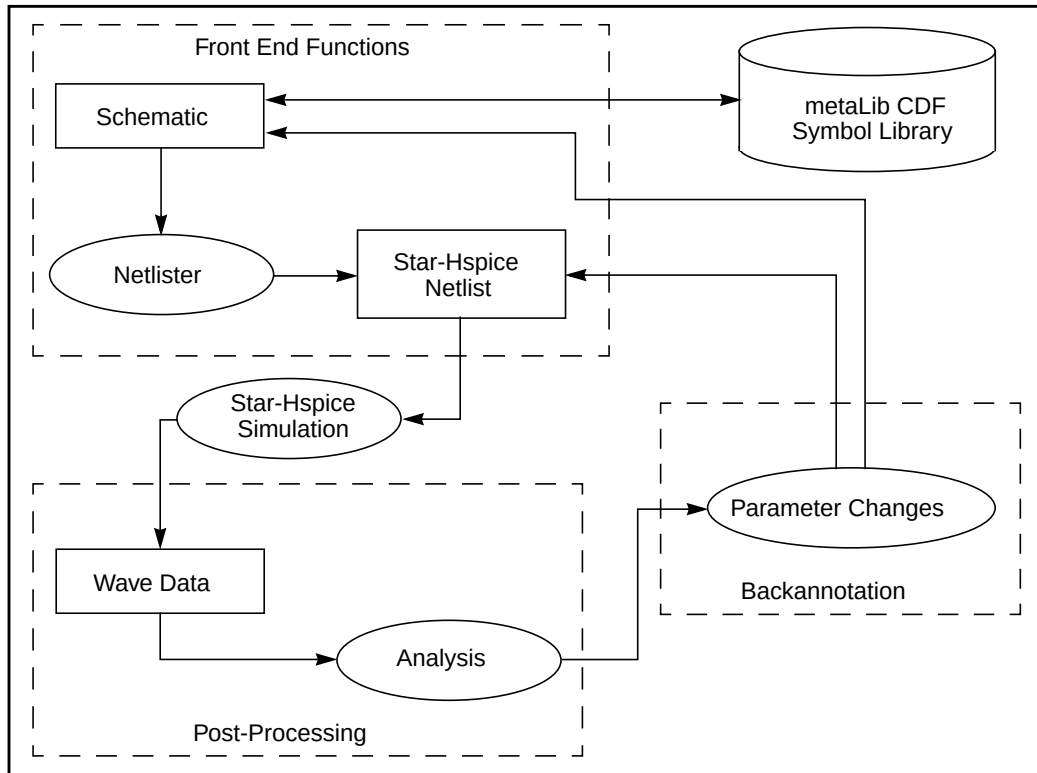
To access AvanLink, either select options on menus, or use specific keys assigned to these options. [Figure B-2 on page B-4](#) shows the operating environment for AvanLink.

**Figure B-2: Star-Hspice AvanLink in a Cadence Composer Environment**

# AvanLink Design Flow

Figure B-3 is an overview of the process of creating a design, using AvanLink.

**Figure B-3: AvanLink Design Flow**

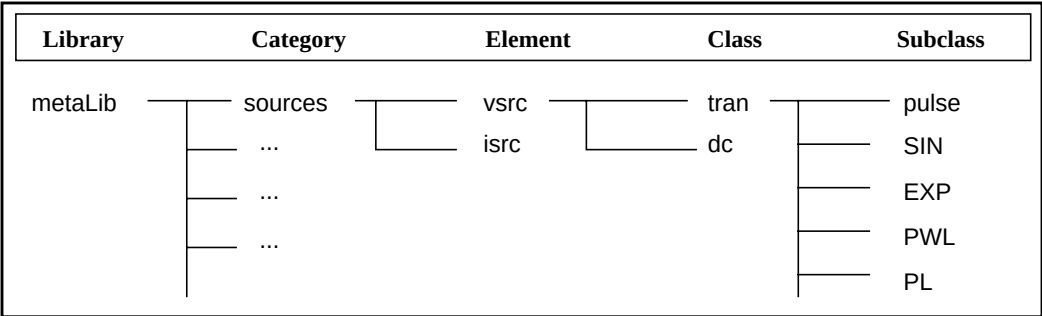


# Schematic Entry and Library Operations

AvanLink helps you design your schematic, using the metaLib Component Description Format (CDF) library. This class-based library includes all of the Star-Hspice elements. A set of parameters, organized into classes and subclasses, describes each element. [Figure B-4](#) shows an example of a metaLib class-based CDF library structure, for a source element.

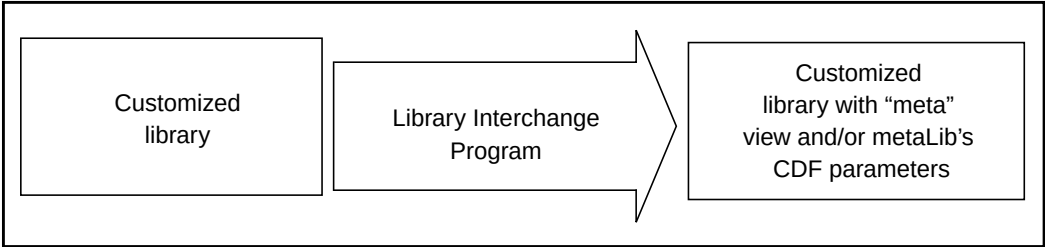
As you add an instance of each element to your circuit design, a form opens, and automatically displays the CDF parameters for that element. You can modify these parameters. AvanLink guarantees that Star-Hspice generates an accurate, and syntactically-correct, netlist for simulation.

**Figure B-4: Example metaLib Library Structure for a Source Element**



Use the AvanLink library interchange program to convert personal or customized symbol libraries, so that these libraries incorporate metaLib CDF parameters. [Figure B-5](#) is a summary of this procedure.

**Figure B-5: AvanLink Library Interchange Function**



## Generating a Netlist

AvanLink uses the Cadence Open Simulation System (OSS) to integrate Star-Hspice into the Cadence Simulation Environment (SE). The AvanLink netlister includes customized functions, in addition to those in the OSS Hierarchical Netlister. The Netlister provides the following features:

- Preserves the pin names in the design.
- Maintains the original design signal names.
- Preserves the design hierarchy.
- Time savings, for large designs that require few modifications.

## Simulation

1. When you run the simulation, specify a simulation directory.  
The Cadence simulation environment initializes this directory, and copies the basic Star-Hspice control files into it.
2. You can modify these files, to add your own custom Star-Hspice commands.
3. You can use the control file to specify all Star-Hspice analysis types, including advanced analyses (such as Optimization and Monte Carlo).

## Waveform Display

1. Use the AvanWaves program to analyze and display output waveforms.
2. You can cross-probe a net in the schematic window, and view its signal in the AvanWaves display window.  
Cross-probing supports signals that transient, DC, or AC analysis generate. You can also cross-probe pins, for branch currents.  
AvanLink also generates waveform format (PSF), to display in the Analog Artist environment.
3. You can also cross-probe and back-annotate the Analog Artist display.

---

# AvanLink for Design Architect

AvanLink for Design Architect is an Avant! interface, which links the Star-Hspice circuit simulator with the following products from Mentor Graphics:

- Design Architect.
- Design Viewpoint Editor (DVE).

AvanLink-DA provides:

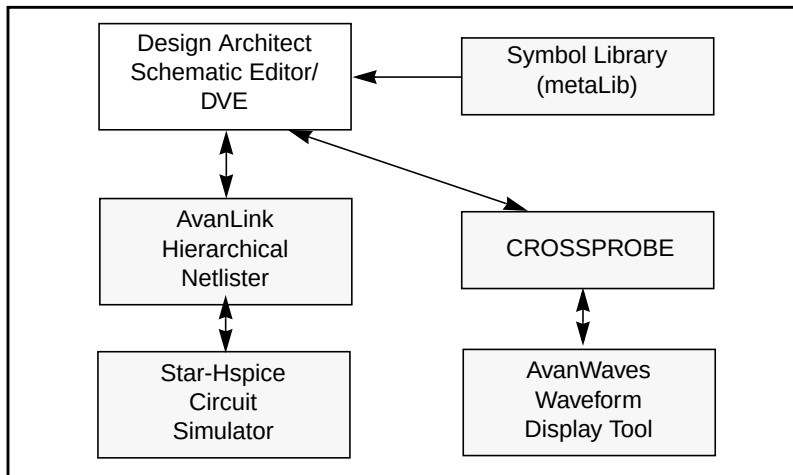
- A netlister.
- A symbol library.
- Cross-probing and back-annotation capabilities.

## Features

AvanLink-DA features include:

- Hierarchical netlisting, provided through the AvanLink-DA netlister and simulation environment.
- Avant! metaLib class-based library, supports all of the Star-Hspice elements, with advanced properties and element syntax.
- Cross-probing, from Design Viewpoint Editor schematics, to the Avant! AvanWaves waveform display tool.
- Hierarchical parameter passing, and algebraic function operations.
- Facility to preserve Star-Hspice hierarchical path names, using a transparent net name mapping function.
- Utilities for migrating customer component libraries, to AvanLink-DA.
- Back-annotation of operating point voltages, back to the Mentor Graphics Design Viewpoint Editor.
- Ability to set up and probe terminal current values.
- Ability to specify initial condition (IC) and nodeset values.
- Support for element parameter back-annotation, onto the schematic within DVE.
- Support for Star-Hspice legacy libraries.

[Figure B-6 on page B-9](#) shows how AvanLink-DA fits into the design process.

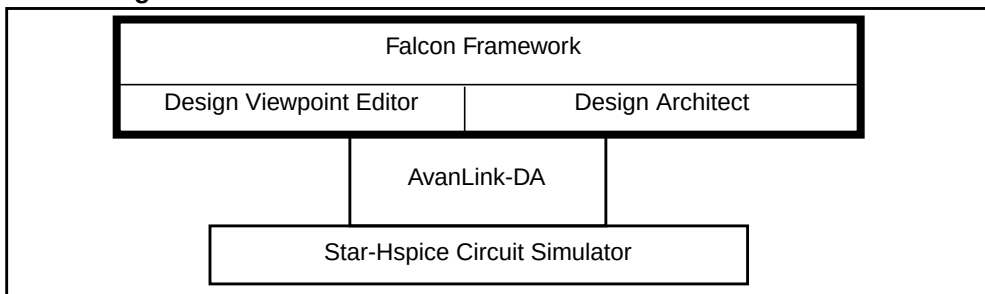
**Figure B-6: AvanLink for Falcon Framework with Design Architect**

## Environment

AvanLink-DA is an integrated environment for circuit design, simulation, and waveform display.

1. Use the Design Architect to develop a design.
2. Use Star-Hspice to simulate the design.
3. Use AvanWaves to view the simulation results.
4. To access AvanLink-DA, select options on menus, or click buttons on the palette in the Mentor Graphics Design Architect window.

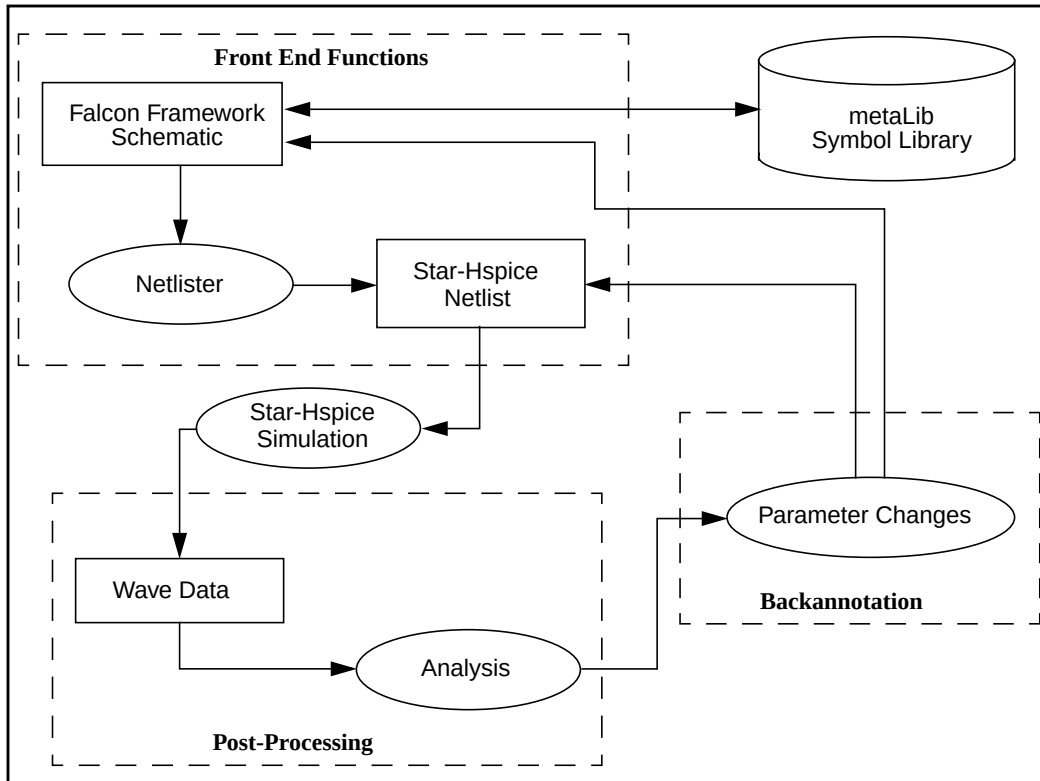
Figure B-7 shows the operating environments for AvanLink-DA.

**Figure B-7: AvanLink-DA in a Falcon Framework Environment**

# AvanLink-DA Design Flow

Figure B-8 is an overview of how to create and simulate a design, using AvanLink-DA.

**Figure B-8: Design Flow for AvanLink for Design Architect**



## Schematic Entry and Library Operations

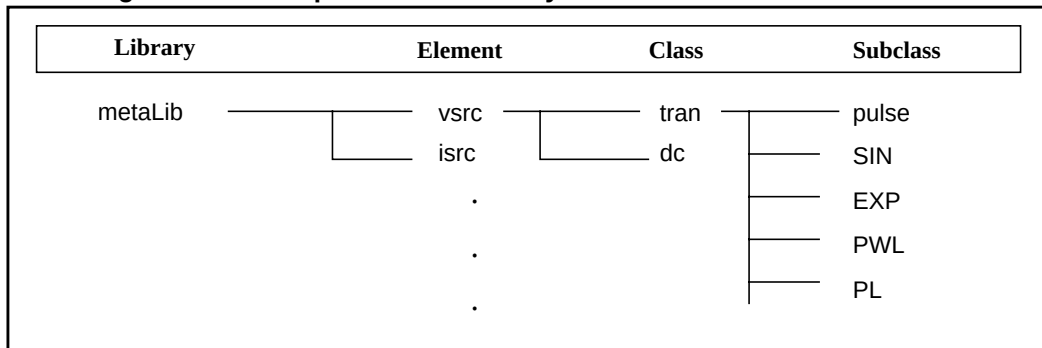
AvanLink-DA helps you to design a schematic, using metaLib. The Avant! metaLib is a class-based library, which includes all of the Star-Hspice elements. A set of parameters, organized into classes and subclasses, describes each element. [Figure B-9 on page B-11](#) shows an example of a metaLib class-based library structure, for a voltage source element.



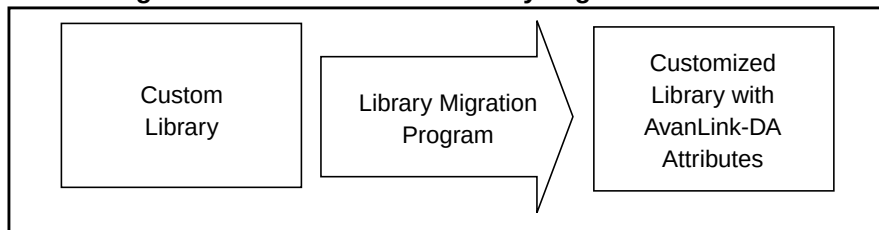
After you add an instance of each design element, you can display and edit the parameters of the element. You can modify these parameters. AvanLink-DA guarantees that Star-Hspice generates an accurate, and syntactically-correct, netlist for simulation.

The library migration program (supplied in AvanLink-DA) converts personal or customized symbol libraries, so that they incorporate metaLib attributes. [Figure B-10](#) diagrams this procedure.

**Figure B-9: Example metaLib Library Structure for a Source Element**



**Figure B-10: AvanLink-DA Library Migration Function**



## Generating a Netlist

The AvanLink-DA Netlister provides the following features:

- Creates a complete hierarchical Star-Hspice netlist.
- Preserves the original design pin names.
- Maintains the original design signal names.
- Preserves the design hierarchy.

## Simulation

1. Specify the directory to run the simulation.  
The AvanLink-DA simulation environment initializes this directory, and copies the basic Star-Hspice control files into it.
2. You can modify these files, to add your own custom Star-Hspice commands.
3. Use the control file to specify all Star-Hspice analysis types, including advanced analyses (such as Optimization and Monte Carlo).

## Waveform Display

1. Use the AvanWaves waveform display tool to display output waveforms.
2. Use the cross-probing feature, to probe a net in the schematic window, inside DVE.
3. View its signal in the AvanWaves display window.

You can cross-probe signals generated in transient, DC, or AC analysis. You can also cross-probe pins, for branch currents.

---

# Viewlogic Links

If you use Viewlogic, you can use the Viewlogic Powerview framework to access Star-Hspice. Viewlogic tools include:

- A netlister.
- A Star-Hspice CSDF option, to display waveforms in Viewtrace.
- Cross-probing.
- A Star-Hspice/Madssim, mixed-signal simulation solution.





## Appendix C

# Library Encryption

---

You can encrypt Star-Hspice libraries, to distribute your own proprietary Star-Hspice custom models, parameters, and circuits to others, without revealing your company's sensitive information. Recipients of an encrypted library can run simulations that use your libraries, but Star-Hspice does not print encrypted parameters, encrypted circuit netlists, or internal node voltages. Your library user sees the devices and circuits as *black boxes*, which provide only terminal functions.

You can use library encryption to distribute circuit blocks that contain embedded transistor models (such as ASIC library cells and I/O buffers). Star-Hspice uses subcircuit calls to read encrypted information. To distribute only the device libraries, create a unique subcircuit file for each device.

This chapter describes the Avant! Library Encryptor, and how to use it to protect your intellectual property. This chapter explains the following topics:

- [Library Encryption](#)
- [Encryption Guidelines](#)
- [Installing and Running the Encryptor](#)
- [Metaencrypt Features](#)
- [Encryption Structure Example](#)

---

# Library Encryption

The library encryption algorithm is based on a five-rotor Enigma machine. You can specify which portions of subcircuits to encrypt. Library encryption uses a key value, which Star-Hspice reconstructs for decryption.

## Controlling the Encryption Process

To control the beginning and end of the encryption process, insert `.PROTECT` and `.UNPROTECT` statements around the text to encrypt, in a Star-Hspice subcircuit. Encryption produces an ASCII text file, which encrypts all text that follows `.PROTECT` and precedes `.UNPROTECT`.

---

**Note:** You can abbreviate `.PROTECT` and `.UNPROTECT` statements, as `.PROT` and `.UNPROT`, in Star-Hspice input files.

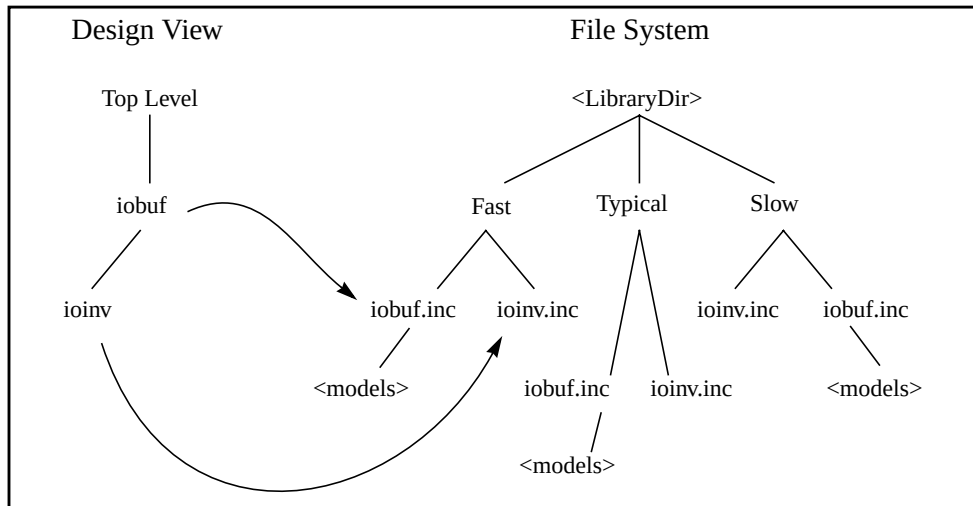
---

## Library Structure

The requirements for encrypted libraries of subcircuits are the same as the requirements for regular subcircuit libraries, as described in [Subcircuit Library Structure on page 3-57](#). To refer to an encrypted subcircuit, use its subcircuit name, in a subcircuit element line of the Star-Hspice netlist.

The following example describes an encrypted I/O buffer library subcircuit. This subcircuit consists of several subcircuits and model statements, which you need to protect with encryption. [Figure C-1](#) shows the organization of subcircuits and models, in the libraries used in this example.

Figure C-1: Encrypted Library Structure



The following input file fragment, from the main circuit level, selects the Fast library. It also creates two instances of the iobuf circuit.

```
...
.Option Search='<LibraryDir>/Fast'      $ Corner Spec
x1 drvin drvout iobuf                   Cload=2pF    $ Driver
u1 drvout 0 recvin 0 PCBModel ...       $ Trace
x2 recvin recvout iobuf                  $ Receiver
...
```

The **<LibraryDir>/Fast/iobuf.inc** file contains:

```
.Subckt iobuf Pin1 Pin2 Cload=1pF
*
* iobuf.inc - model 2001 improved iobuf
*
.PROTECT
cPin1 Pin1 0 1pF          $ Users cannot change this!
x1 Pin1 Pin2 ioinv         $ Italics here means encrypted
.Model pMod pmos Level=28 Vto=...    $ <FastModels>
.Model nMod nmos Level=28 Vto=...    $ <FastModels>
.UNPROTECT
cPin2 Pin2 0 Cload        $ gives you some control
.Ends
```

The <LibraryDir>/Fast/ioinv.inc file contains:

```
.Subckt ioinv Pin1 Pin2
.PROTECT
mp1 Vcc Pin1 Pin2 Vcc pMod      . .  $ Italics=Protected
mn1 Pin2                        Pin1 Gnd Gnd nMod...
$ Italics=Protected
.UNPROTECT
.Ends
```

After encryption, the basic layout of the subcircuits is the same. However, you cannot read the text between .PROTECT and .UNPROTECT statements. Only Star-Hspice can read this text.

Protection statements also suppress printouts of encrypted model information from Star-Hspice. Only Star-Hspice knows how to decrypt the model.



---

# Encryption Guidelines

In general, you use encrypted models the same way you use regular models. However, you *must test your subcircuits, before you encrypt them*. You will not be able to see what is wrong *after* encryption, because Star-Hspice does not let you read the encrypted data.

You can use any legal Star-Hspice statement, inside subcircuits that you encrypt. Refer to [.SUBCKT or .MACRO Statement on page 3-11](#) for more information about how to construct subcircuits. The structure of your libraries can affect how you encrypt them. If your library requires that you change the name of a subcircuit, you must encrypt that circuit again.

If you use `.PROTECT` and `.UNPROTECT` statements, your customers can see portions of your subcircuits. If you protect only device model statements in your subcircuits, other users can set device sizes, or substitute different subcircuits for lead frames, protection circuits, and so on. This requires that users understand the circuits, but it reduces library-management overhead for everyone.

---

**Note:** If you use any version of the encryptor before Star-Hspice Release H93A.03, Star-Hspice cannot correctly decrypt a subcircuit, if that subcircuit contains *any* semicolon (;) characters (even in comments).

---

The following example encrypts the `badsemi.dat` subcircuit into `badsemi.inc`.

```
* Sample semicolon bug
.SubCkt BadSemi A B
.PROT
* Semicolons (;) cause problems!
r1 A B 1k
.UNPROT
.Ends
```

Star-Hspice returns the following message:

```
**reading include file=badsemi.inc
**error**: .ends card missing at read-in
>error ***difficulty in reading input
```

To solve this problem, remove the semicolon from `badsemi.dat`, and encrypt the file again.

Before the 2001.2 version, Metaencrypt cannot encrypt files that contain lines longer than 80 characters. Metaencrypt version 2001.2 and later can encrypt files with lines longer than 80 characters, but cannot correctly encrypt files with lines longer than 254 characters. Avant! strongly recommends that all files to encrypt, have a line length of 254-character or shorter. Then use version 2001.2 or later, to encrypt any files with lines longer than 80 characters. Star-Hspice can recognize all correctly-encrypted files.

You cannot gather the individually-encrypted files into a single file, or include encrypted files directly in the Star-Hspice netlist. Instead, place these files into a separate directory, and use the `.OPTION SEARCH = <dir> named <sub>.inc` to point to the directory, so Star-Hspice can correctly decrypt the files.

---

**Note:** If you divide a data line into more than one line, and use + to link the lines, you cannot add `.PROT` or `.UNPR` among the lines. The following example fails:

---

```
.prot
.Model N1 NMOS Level= 57
+TNOM = 27 TOX = 4.5E-09 TSI = .00000001 TBOX = 8E-08
+MOBMOD = 0 CAPMOD = 2 SHMOD =0
.unpr
+PARAMCHK=0 WINT = 0 LINT = -2E-08
```

---

# Installing and Running the Encryptor

This section describes how to install and run the Encryptor.

## Installing the Encryptor

If you have installed Star-Hspice on your system, to install the Encryptor, place it in the `$installdir/bin` directory. Add the lines that allow the Encryptor to operate, into your `permit.hsp` file (in the `$installdir/bin` directory).

If you have not installed Star-Hspice on your system, first install Star-Hspice according to the *Star-Hspice Installation Guide* and *Star-Hspice Release Notes*, and then follow the instructions in the previous paragraph.

---

**Note:** If you use floating licenses, you must stop and restart the license server, before you can see the changes in the permit file.

---

## Running the Encryptor

The Encryptor requires three parameters, for each subcircuit that you encrypt:

- `<InFileName>`.
- `<OutFileName>`.
- Freelib key type specifier.

To encrypt a file, enter the following line:

```
metaencrypt -i <InFileName> -o <OutFileName> -t Freelib
```

When the Encryptor reads the input file, it looks for `.PROT/.UNPROT` pairs, and encrypts the text between them. You can encrypt only one file at a time.

To encrypt more than one file in a directory, use the following shell script, which encrypts the files as a group. This script produces a .inc encrypted file, for each .dat file in the current directory. Encryption assumes that unencrypted files have a .dat suffix.

```
#!/bin/sh
for i in *.dat
do
Base='basename $i .dat'
metaencrypt -i $Base.dat -o $Base.inc -t Freelib
done

.SUBCKT ioinv Pin1 Pin2
.PROT FREELIB
X34%43*27@#^3rx*34&%^#1
^(*^!^HJHD(*@H$!:&*$
dFE2341&*&)(@@3
$ Encryption starts here ...
$ ... and stops here
$ (.UNPROT is encrypted!)
.ENDS
```

---

# Metaencrypt Features

This section describes new features and enhancements to the metaencrypt functionality, supported in version 2001.2 and later.

## 8-Byte Key Encryption

An 8-byte key encryption feature, based on a 56-bit DES, is available in metaencrypt, with Star-Hspice versions 2001.2 and later. You can insert data into an include file, and use 8-byte key encryption to encrypt this file. The encrypted data is in binary format.

### Syntax

```
metaencrypt -i <infileName> -o <outfileName> -t <keyname>
```

---

**Note:** The keyname can contain alphabetic characters and numbers, and must be no longer than 8 bytes.

---

### Example

```
metaencrypt -i example.dat -o example.inc -t fGi85H9b
```

## Encryption Structure

Star-Hspice supports, .inc encryption, when you use 8-byte key encryption.

1. Insert the data to encrypt, into an include file.
2. Encrypt this file.

In the following example, example.dat contains the data to encrypt.

```
metaencrypt -i example.dat -o example.inc -t h78Gbvni
*example.sp*
.....
.inc example.inc $ example.inc contains encrypted data
.....
.end
```

Follow these rules when you use 8-byte key encryption:

- 8-byte key encryption supports only `.inc` encryption.
- 8-byte key encryption does NOT support `.lib`, `.load` or `.option search` encryption, in version 2001.2.
- Do not include `.PROT` and `.UNPR` with the data, when you perform 8-byte key encryption.
- If `keyname=ddl1`, `ddl2`, `custom`, or `freelib`, then `metaencrypt` uses a *former* algorithm, to perform the encryption.
- If `keyname` is an 8-byte string (combination of characters and numbers), then `metaencrypt` performs the 8-byte key encryption.
- In a `.sp` file, you cannot encrypt the first line, because it is the title. You also cannot encrypt the last line, because it marks the end of the file.

## **.sp File Encryption**

You can encrypt a `.sp` file in Star-Hspice. You can encrypt the data between `.PROT` and `.UNPR`, in a `.sp` file, so that Star-Hspice can recognize it.

---

**Note:** If you use `.sp` encryption, the encrypted data must not use `.inc`, `.lib`, or `.load`, to include another file.

---

The following is an example of an `.sp` file:

```
*sample.sp*
.....
.inc sample1.inc
.prot
.... $ data to be encrypted
.... $ do not include .inc .lib .load in encrypted data
.unpr
.inc sample2.inc
.....
.end
```

## .lib File Encryption

You can place any important information into a .lib file, and encrypt it.

You can place parallel .lib statements into one library file, and encrypt each .lib separately. However, you must place .prot and .unpr between each pair of .lib and .endl.

---

**Note:** You must place .prot and .unpr between .lib and .endl. To find the library name, Star-Hspice searches the .lib file first.

---

The following is an example of a .sp file:

```
*sample.sp*
.....
.lib './sample.lib' test1
.lib './sample.lib' test2
.lib './sample.lib' test3
.....
.end
*sample.lib*
.lib test1 $ .prot , .unpr should be put between
+ .lib and .endl
.prot
..... $ data to be encrypted
.unpr
..... $ data not to be encrypted
.end1 test1

.lib test2 $ .prot , .unpr should be put between
+ .lib and .endl
.prot
..... $ data to be encrypted
.unpr
.end1 test2

.lib test3
..... $ data
.end1 test3
```

## **.inc File Encryption**

You can insert any important data into a `.inc` file, and encrypt the file. You can place `.PROT` and `.UNPR` anywhere in the file.

The following is an example of a `.sp` file:

```
*sample.sp*
.....
.inc sample.inc
.....
.end
*sample.inc*
.prot $ no restriction on the placement of .prot
....$ exclude .inc, .lib .load from data to encrypt
.unpr
.....
```

## **.load Encryption**

To encrypt a `.ic` file, use the same method as for a `.inc` file.

## **Encrypting 80+ Columns**

Star-Hspice, version 2001.2 or later, can encrypt 1 to 255 columns.

## **Statements Not Supported**

- Embedded `.lib` encryption, which causes confusion in decryption.
- Embedded `.inc` encryption, which causes confusion in decryption.
- Data should not contain `.inc`, `.lib`, or `.load` statements, to include another `.inc`, `.lib`, or `.ic` file

## **Additional Recommendations for Encryption**

Do not use `.option search`, when you encrypt models and subcircuits. The old metaencryption functionality supported this method. To directly encrypt subcircuits and model libraries, use the `.inc` and `.lib` encryption method.



# Encryption Structure Example

The following complete example illustrates the *Metaencrypt* encryption structure.

```

file enc.sp:
*test .inc .lib .load encryption
.inc "mm.inc"
.load "xx.ic"
.lib 'kk.lib' pch
.OPTION post list
.tran 2ns 400ns
.end

file mm.inc:
.prot CUSTOM
-hs#y1B]*7[
+t'Y=0$S[t0]ajL
+C :Nx:$.$=<*X:$<#pP=020#ZWP=020x\K:[1:898
y[-x:$-#tRr0($x#4:/[U$<\K:I[U$<J <9 :P#ZQ
6%P2V7D6:]4L/0#+:IXj0#ZWP=020#ZWP/[U$=J++bZ
3[7D6:BxHpg8
/C902P73+26
mh$y#D:bx/$\KwI)U-0R#=-ib+\[
a$0):P.#$<):P.#to)V:\7*K-I1M$#';-[Xz:9qpy
eMDv0%wUoxZ>mzwF*-(3_;W6x.*P!uW.]a+P0.h:n=0>1q+H(J0
o.H#-/B+($;W Me*0x<6#9[UqpH/2h97%;-/B+T35Q
$m;['_-he[uE$%H) 5a:ZxRW9x=*77w$2]=*P!RW%.ahT3VQ
H0[I:[

file xx.ic:
.prot FREELIB
59yUH\$='x.3k77*<]8AT]8
<:7-( :9CV+7x15Xj+h'x=5Xj+(2 +4]8
<:7_D:\[2x9Y>/.7q
59y3\#D$ *y2k=u]PIq:97jH=u1w5Xj+x6
92k#<2FW0'k772<xBU677Q
59y3\#s# r21$],29b72[4'/RW72wd#$:0.U
+ 0sw%5$;[4sv;9=zV7[WFW[(g8#/'']=AH%T5:7Z
[$%C999A2P!8
<:X2o60'$ 06($_#upe1:px8
<5ax/toC n90;<0dw0]23G%C z9$Dh#Sw5a90
ZM*2!M[0
o729!=PAy73x(/1:6[
+ 0%2UT%8
_:-x*$X+q
$9P2y73x(/1:L
T#;*9A27!j+(/z

```

```

$$o#(:/b0
o7ZW-9 -PxJ+y
a9[$0\;n90;<0dw0]23G%C z9$Dh#Sw5a90
Zr      ;6
file kk.lib:
.LIB NCH
.prot FREELIB
H0. T,# %fXz>MZwf*-(3_;w6X.*p!Uw.]A+p0.H:N=o>1Q+h(j0
o.H#-/B+($;W Me*0x<6#9[UqpH/2h97%;-/B+T35Q
$\m;'_-he[uE$%H) 5a:ZxRw9x=*77w$2]=*P!Rw%.ahT3VQ
H0[I:[
.ENDL

.LIB PCH
.prot FREELIB
H0. T,#t%fXz>MZwf*-(3_;w6X.*p!Uw.]A+p0.H:N=o>1Q+h(j0
o.H#-/B+($;W Me*0x<6#9[UqpH/2h97%;-/B+T35Q
$\m;'_-he[uE$%H) 5a:ZxRw9x=*77w$2]=*P!Rw%.ahT3VQ
H0[I:[
.ENDL

```



## Appendix D

# Full Simulation Examples

---

- The examples in this chapter show the basic text and post-processor output, for a sample input netlist. The first example uses AvanWaves to view results.
- The second example uses Cosmos-Scope.

This chapter includes the following sections:

- [Simulation Example, with AvanWaves](#)
- [Simulation Example, with Cosmos-Scope](#)

# Simulation Example, with AvanWaves

## Input Netlist and Circuit

The following example is an input netlist for a linear CMOS amplifier. Comment lines indicate the individual sections of the netlist. Refer to [Simulation Input and Controls on page 3-1](#), for information about the individual commands.

```
* Example HSPICE netlist, using a linear CMOS amplifier
* netlist options
.option post probe brief nomod
* defined parameters
.param analog_voltage=1.0
* global definitions
.global vdd
* source statements
Vinut in gnd SIN ( 0.0v analog_voltage 10x )
Vsupply vdd gnd DC=5.0v
* circuit statements
Rinterm in gnd 51
Cincap in infilt 0.001
Rdamp infilt clamp 100
Dlow gnd clamp diode_mod
Dhigh clamp vdd diode_mod
Xinv1 clamp inv1out inverter
Rpull clamp inv1out 1x
Xinv2 inv1out inv2out inverter
Routterm inv2out gnd 100x
* subcircuit definitions
.subckt inverter in out
Mpmos out in vdd vdd pmos_mod l=1u w=6u
Mnmos out in gnd gnd nmos_mod l=1u w=2u
.ends
* model definitions
.model pmos_mod pmos level=3
.model nmos_mod nmos level=3
.model diode_mod d
* analysis specifications
.TRAN 10n 1u sweep analog_voltage lin 5 1.0 5.0
* output specifications
.probe TRAN v(in) v(clamp) v(inv1out) v(inv2out) i(dlow)
.measure TRAN falltime TRIG v(inv2out) VAL=4.5v FALL=1
+                               TARG V(inv2out) VAL=0.5v FALL=1
.end
```

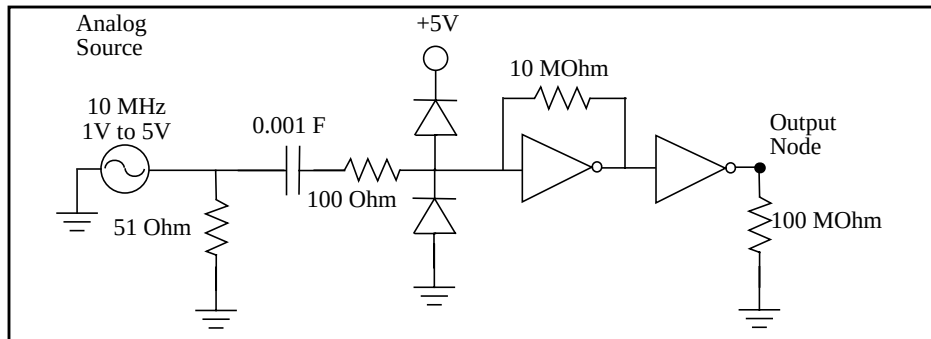
Figure D-1 is a circuit diagram, for the linear CMOS amplifier in the circuit portion of the netlist. The two sources in the diagram are also in the netlist.

---

**Note:** The inverter symbols in the circuit diagram are constructed from two complementary MOSFET elements. Also, the diode and MOSFET models in the netlist do not have non-default parameter values, except to specify Level 3 MOSFET models (empirical model).

---

**Figure D-1: Circuit Diagram for Linear CMOS Inverter**



## Execution and Output Files

The following section displays the output files, from a Star-Hspice simulation of the amplifier, shown in the previous section. To execute the simulation, enter:

```
hspice example.sp > example.lis
```

where the input netlist name is `example.sp`, and the output listing file name is `example.lis`. Simulation creates the following output files:

<code>example.ic</code>	Initial conditions for the circuit.
<code>example.lis</code>	Text simulation output listing.
<code>example.mt0</code>	Post-processor output, for <code>.MEASURE</code> statements.
<code>example.pa0</code>	Subcircuit path table.
<code>example.st0</code>	Run-time statistics.
<code>example.tr0</code>	Post-processor output, for transient analysis.

The following subsections show the text files in their entirety, for simulating the amplifier, using Star-Hspice on a Sun workstation. The example does not show the two post-processor output files, because they are in binary format.

## Example.ic

```
* "simulator" "HSPICE"
* "version" "98.4 (981215) "
* "format" "HSP"
* "rundate" "13:58:43 01/08/1999"
* "netlist" "example.sp "
* "runtitle" "** example hspice netlist using a linear
* cmos amplifier "
* time = 0.
* temperature = 25.0000
*** BEGIN: Saved Operating Point ***
.option gmindc= 1.0000p
.nodeset
+ clamp = 2.6200
+ in = 0.
+ infilt = 2.6200
+ inv1out = 2.6200
+ inv2out = 2.6199
+ vdd = 5.0000
*** END: Saved Operating Point ***
```

## Example.lis

```
Using: /net/sleepy/l0/group/hspice/98.4beta/sol4/hspice
***** Star-HSPICE -- 98.4 (981215) 13:58:43 01/08/1999 solaris
Copyright (C) 1985-1997 by Avant! Corporation.
Unpublished-rights reserved under US copyright laws.
This program is protected by law and is subject to the
terms and conditions of the license agreement found in:
    /afs/rtp.avanticorp.com/product/hspice/current/license.txt
Use of this program is your acceptance to be bound by this
license agreement. Star-HSPICE is a trademark of Avant!
Corporation.
Input File: example.sp
lic:
lic: FLEXlm:v5.12 USER:hspiceuser HOSTNAME:hspiceserv
+ HOSTID:8086420f PID:1459
lic: Using FLEXlm license file:
lic: /afs/rtp/product/distrib/bin/license/license.dat
lic: Checkout hspice; Encryption code: AC34CE559E01F6E05809
lic: License/Maintenance for hspice will expire on 14-apr-
+ 1999/1999.200
```

```

lic: 1(in_use)/10 FLOATING license(s) on SERVER hspiceserv
lic:
*****
* example hspice netlist using a linear cmos amplifier
*****
* netlist options
.option post probe brief nomod
* defined parameters
Opening plot unit= 15
file=./example.pa0
***** Star-HSPICE --      98.4 (981215) 13:58:43
***** 01/08/1999 solaris *****
* example hspice netlist using a linear cmos amplifier
***** transient analysis  tnom= 25.000 temp= 25.000 *****
*** parameter analog_voltage  =    1.000E+00 ***
node    =voltage      node    =voltage      node    =voltage
+0:clamp =    2.6200  0:in    =    0.        0:infilt =    2.6200
+0:inv1out =2.6200  0:inv2out = 2.6199    0:vdd    =    5.0000
Opening plot unit= 15
file=./example.tr0
**warning** negative-mos conductance = 1:mnmos  iter= 2
vds,vgs,vbs =      2.45      2.93      0.
gm,gds,gmb,ids= -3.636E-05    1.744E-04  0.  1.598E-04
*****
* example hspice netlist using a linear cmos amplifier
***** transient analysis  tnom= 25.000 temp= 25.000 *****
falltime= 3.9149E-08 targ= 7.1916E-08  trig= 3.2767E-08
*** Star-HSPICE -- 98.4 (981215) 13:58:43
*** 01/08/1999 solaris ***
* example hspice netlist using a linear cmos amplifier
***** transient analysis  tnom= 25.000 temp= 25.000 *****
*** parameter analog_voltage  =    2.000E+00 ***
node    =voltage      node    =voltage      node    =voltage
+0:clamp = 2.6200    0:in    =    0.        0:infilt =    2.6200
+0:inv1out = 2.6200  0:inv2out = 2.6199  0:vdd    =    5.0000
*****
* example hspice netlist using a linear cmos amplifier
***** transient analysis  tnom= 25.000 temp= 25.000 *****
falltime= 1.5645E-08 targ= 5.7994E-08  trig= 4.2348E-08
**** Star-HSPICE --      98.4 (981215) 13:58:43
**** 01/08/1999 solaris ****

```

```

* example hspice netlist using a linear cmos amplifier
***** transient analysis tnom= 25.000 temp= 25.000 *****
*** parameter analog_voltage = 3.000E+00 ***
node    =voltage      node    =voltage      node    =voltage
+0:clamp = 2.6200 0:in    = 0.      0:infilt = 2.6200
+0:inv1out = 2.6200 0:inv2out = 2.6199 0:vdd    = 5.0000
*****
* example hspice netlist using a linear cmos amplifier
***** transient analysis tnom= 25.000 temp= 25.000 *****
falltime= 1.1917E-08 targ= 5.6075E-08  trig= 4.4158E-08
***** Star-HSPICE -- 98.4 (981215) 13:58:43
***** 01/08/1999 solaris *****
* example hspice netlist using a linear cmos amplifier
***** transient analysis tnom= 25.000 temp= 25.000 *****
*** parameter analog_voltage = 4.000E+00 ***
node    =voltage      node    =voltage      node    =voltage
+0:clamp = 2.6200 0:in    = 0.      0:infilt = 2.6200
+0:inv1out = 2.6200 0:inv2out = 2.6199 0:vdd    = 5.0000
*****
* example hspice netlist using a linear cmos amplifier
***** transient analysis tnom= 25.000 temp= 25.000 *****
falltime= 7.5424E-09 targ= 5.3989E-08  trig= 4.6447E-08
***** Star-HSPICE -- 98.4 (981215) 13:58:43
***** 01/08/1999 solaris *****
* example hspice netlist using a linear cmos amplifier
***** transient analysis tnom= 25.000 temp= 25.000 *****
*** parameter analog_voltage = 5.000E+00 ***
node    =voltage      node    =voltage      node    =voltage
+0:clamp = 2.6200 0:in    = 0.      0:infilt = 2.6200
+0:inv1out = 2.6200 0:inv2out = 2.6199 0:vdd    = 5.0000
*****
* example hspice netlist using a linear cmos amplifier
***** transient analysis tnom= 25.000 temp= 25.000 *****
falltime= 6.1706E-09 targ= 5.3242E-08  trig= 4.7072E-08
meas_variable = falltime
mean = 16.0848n      varian = 1.802e-16
sigma = 13.4237n     avgdev = 9.2256n
max = 39.1488n      min = 6.1706n
***** job concluded

```



```

***** Star-HSPICE -- 98.4 (981215) 13:58:43
***** 01/08/1999 solaris *****
* example hspice netlist using a linear cmos amplifier
*** job statistics summary tnom= 25.000 temp= 25.000 ***

total memory used          155 kbytes
# nodes = 8 # elements= 14
# diodes= 2 # bjts      =      0 # jfets    = 0 # mosfets = 4
analysis      time      # points    tot. iter   conv.iter
op point      0.04       1          23
transient     4.71       505         9322       2624 rev= 664
readin        0.03
errchk        0.01
setup         0.01
output        0.01

total cpu time      4.84 seconds
job started at 13:58:43 01/08/1999
job ended   at 13:58:50 01/08/1999

lic: Release hspice token(s)
HSPICE job example.sp completed.
Fri Jan 8 13:58:50 EST 1999

```

## Example.pa0

```

1 xinv1.
2 xinv2.

```

## Example.st0

```

***** Star-HSPICE --      98.4 (981215) 13:58:43
***** 01/08/1999 solaris
Input File: example.sp
lic: FLEXlm:v5.12 USER:hspiceuser HOSTNAME:hspiceserv
+ HOSTID:8086420f PID:1459
lic: Using FLEXlm license file:
lic: /afs/rtp/product/distrib/bin/license/license.dat
lic: Checkout hspice; Encryption code: AC34CE559E01F6E05809
lic: License/Maintenance for hspice will expire on
+ 14-apr-1999/1999.200
lic: 1(in_use)/10 FLOATING license(s) on SERVER hspiceserv
lic:
init: begin read circuit files, cpu clock= 2.21E+00
      option probe
      option nomod
init: end read circuit files, cpu clock= 2.23E+00
+ memory= 145 kb
init: begin check errors, cpu clock= 2.23E+00
init: end check errors, cpu clock= 2.24E+00 memory= 144 kb
init: begin setup matrix, pivot=      10 cpu clock= 2.24E+00

```

```

establish matrix -- done, cpu clock= 2.24E+00 memory= 146 kb
re-order matrix -- done, cpu clock= 2.24E+00 memory= 146 kb
init: end setup matrix, cpu clock= 2.25E+00 memory= 154 kb
sweep: parameter parameter1          begin, #sweeps= 5
parameter: analog_voltage = 1.00E+00
dcop: begin dcop, cpu clock= 2.25E+00
dcop: end dcop, cpu clock= 2.27E+00 memory= 154 kb
tot_iter= 11
output: ./example.mt0
sweep: tran tran1 begin, stop_t= 1.00E-06 #sweeps= 101
cpu clock= 2.28E+00
tran: time= 1.03750E-07 tot_iter= 78 conv_iter= 24
tran: time= 2.03750E-07 tot_iter= 179 conv_iter= 53
tran: time= 3.03750E-07 tot_iter= 280 conv_iter= 82
tran: time= 4.03750E-07 tot_iter= 381 conv_iter= 111
tran: time= 5.03750E-07 tot_iter= 482 conv_iter= 140
tran: time= 6.03750E-07 tot_iter= 583 conv_iter= 169
tran: time= 7.03750E-07 tot_iter= 684 conv_iter= 198
tran: time= 8.03750E-07 tot_iter= 785 conv_iter= 227
tran: time= 9.03750E-07 tot_iter= 886 conv_iter= 256
tran: time= 1.00000E-06 tot_iter= 987 conv_iter= 285
sweep: tran tran1 end, cpu clock= 2.82E+00 memory= 155 kb
parameter: analog_voltage = 2.00E+00
dcop: begin dcop, cpu clock= 2.83E+00
dcop: end dcop, cpu clock= 2.83E+00 memory= 155 kb
+ tot_iter= 14
output: ./example.mt0
sweep: tran tran2 begin, stop_t= 1.00E-06 #sweeps= 101
+ cpu clock= 2.83E+00
tran: time= 1.01016E-07 tot_iter= 186 conv_iter= 54
tran: time= 2.02642E-07 tot_iter= 338 conv_iter= 98
tran: time= 3.01763E-07 tot_iter= 495 conv_iter= 145
tran: time= 4.04254E-07 tot_iter= 668 conv_iter= 198
tran: time= 5.02594E-07 tot_iter= 841 conv_iter= 248
tran: time= 6.10102E-07 tot_iter= 983 conv_iter= 289
tran: time= 7.01850E-07 tot_iter= 1161 conv_iter= 340
tran: time= 8.01776E-07 tot_iter= 1306 conv_iter= 383
tran: time= 9.04268E-07 tot_iter= 1481 conv_iter= 436
tran: time= 1.00000E-06 tot_iter= 1654 conv_iter= 486
sweep: tran tran2 end, cpu clock= 3.71E+00 memory= 155 kb
parameter: analog_voltage = 3.00E+00
dcop: begin dcop, cpu clock= 3.71E+00
dcop: end dcop, cpu clock= 3.72E+00 memory= 155 kb
+ tot_iter= 17
output: ./example.mt0
sweep: tran tran3 begin, stop_t= 1.00E-06 #sweeps= 101
+ cpu clock= 3.72E+00
tran: time= 1.00313E-07 tot_iter= 143 conv_iter= 42

```

```

tran: time= 2.01211E-07 tot_iter= 340 conv_iter= 100
tran: time= 3.01801E-07 tot_iter= 539 conv_iter= 156
tran: time= 4.02192E-07 tot_iter= 729 conv_iter= 211
tran: time= 5.01997E-07 tot_iter= 917 conv_iter= 265
tran: time= 6.01801E-07 tot_iter= 1088 conv_iter= 314
tran: time= 7.01801E-07 tot_iter= 1221 conv_iter= 351
tran: time= 8.01801E-07 tot_iter= 1362 conv_iter= 392
tran: time= 9.02387E-07 tot_iter= 1515 conv_iter= 435
tran: time= 1.00000E-06 tot_iter= 1674 conv_iter= 479

sweep: tran tran3 end, cpu clock= 4.57E+00 memory= 155 kb
parameter: analog_voltage = 4.00E+00
dcop: begin dcop, cpu clock= 4.57E+00
output: ./example.mt0

sweep: tran tran4 begin, stop_t= 1.00E-06 #sweeps= 101
+ cpu clock= 4.58E+00
tran: time= 1.00110E-07 tot_iter= 236 conv_iter= 70
tran: time= 2.04376E-07 tot_iter= 475 conv_iter= 139
tran: time= 3.07892E-07 tot_iter= 767 conv_iter= 221
tran: time= 4.01056E-07 tot_iter= 951 conv_iter= 273
tran: time= 5.01086E-07 tot_iter= 1250 conv_iter= 353
tran: time= 6.00965E-07 tot_iter= 1541 conv_iter= 432
tran: time= 7.03668E-07 tot_iter= 1805 conv_iter= 506
tran: time= 8.01114E-07 tot_iter= 2046 conv_iter= 571
tran: time= 9.01005E-07 tot_iter= 2308 conv_iter= 640
tran: time= 1.00000E-06 tot_iter= 2528 conv_iter= 703

sweep: tran tran4 end, cpu clock= 5.83E+00 memory= 155 kb
parameter: analog_voltage = 5.00E+00
dcop: begin dcop, cpu clock= 5.83E+00

dcop: end dcop, cpu clock= 5.84E+00 memory= 155 kb
+ tot_iter= 23
output: ./example.mt0

sweep: tran tran5 begin, stop_t= 1.00E-06 #sweeps= 101
+ cpu clock= 5.84E+00
tran: time= 1.00195E-07 tot_iter= 176 conv_iter= 47
tran: time= 2.00617E-07 tot_iter= 431 conv_iter= 115
tran: time= 3.00475E-07 tot_iter= 661 conv_iter= 176
tran: time= 4.00719E-07 tot_iter= 914 conv_iter= 246
tran: time= 5.04084E-07 tot_iter= 1157 conv_iter= 311
tran: time= 6.00666E-07 tot_iter= 1347 conv_iter= 363
tran: time= 7.01830E-07 tot_iter= 1623 conv_iter= 435
tran: time= 8.02418E-07 tot_iter= 1900 conv_iter= 514
tran: time= 9.01178E-07 tot_iter= 2161 conv_iter= 585
tran: time= 1.00000E-06 tot_iter= 2410 conv_iter= 650

sweep: tran tran5 end, cpu clock= 7.03E+00 memory= 155 kb
sweep: parameter parameter 1 end
>info: ***** hspice job concluded
lic: Release hspice token(s)

```

## Simulation Graphical Output in AvanWaves

The plots on the following pages show the six different post-processor outputs from the simulation of the example netlist. The format of these plots is for AvanWaves, the Avant! graphical waveform viewer. These plots are postscript output, from the actual data.

Figure D-2: Plot of Voltage on Node *in*

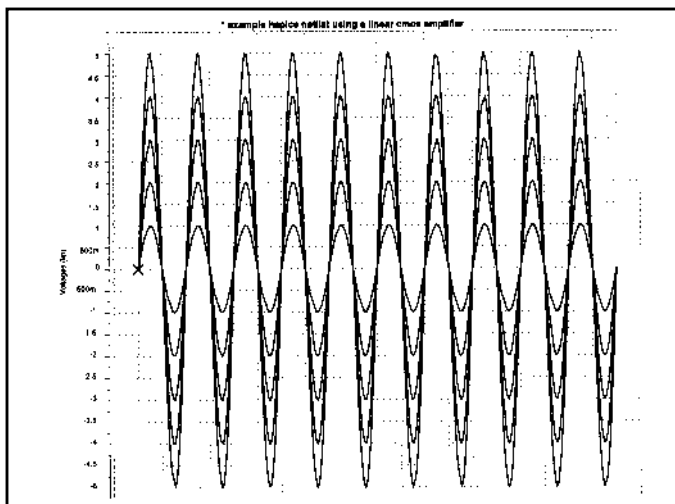
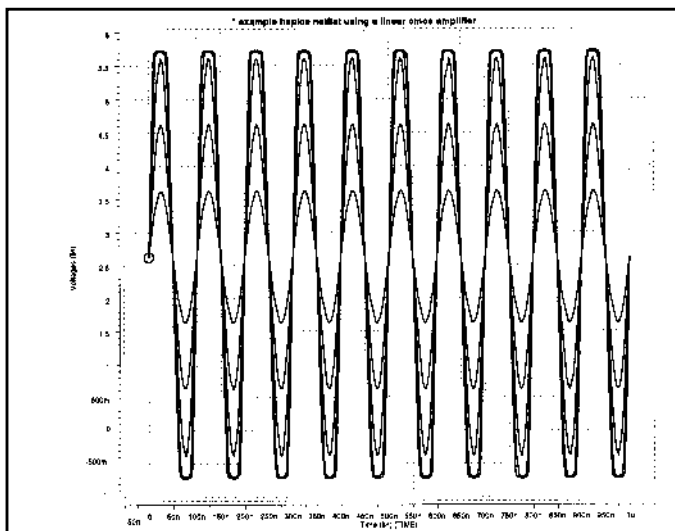
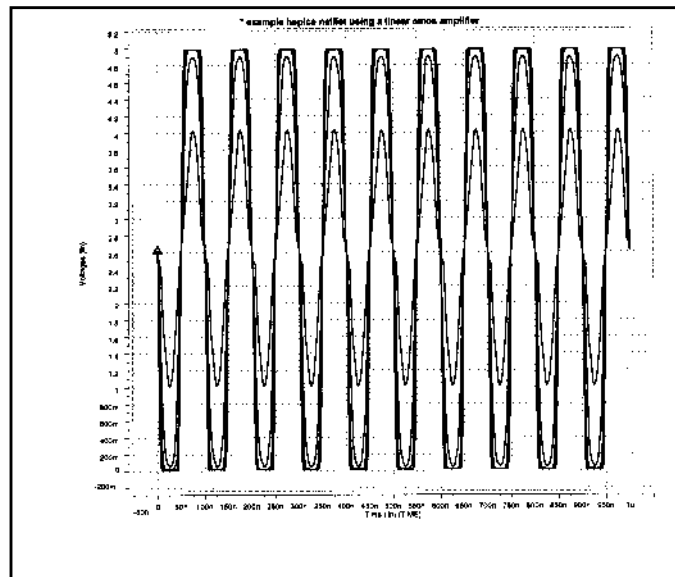
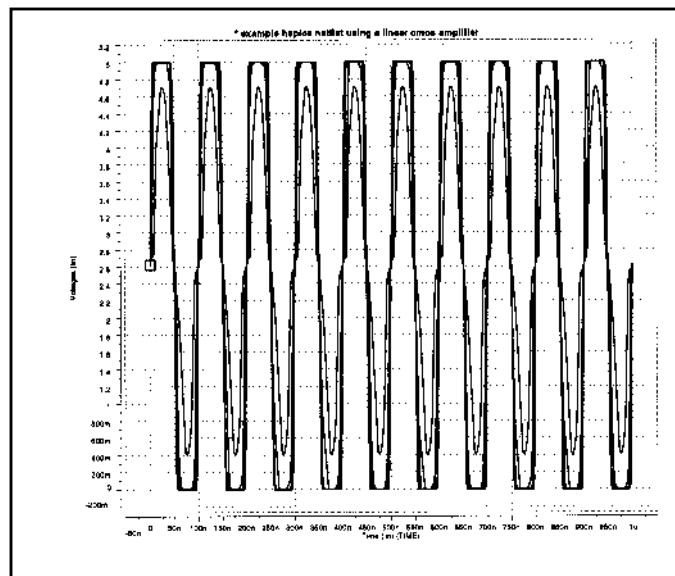
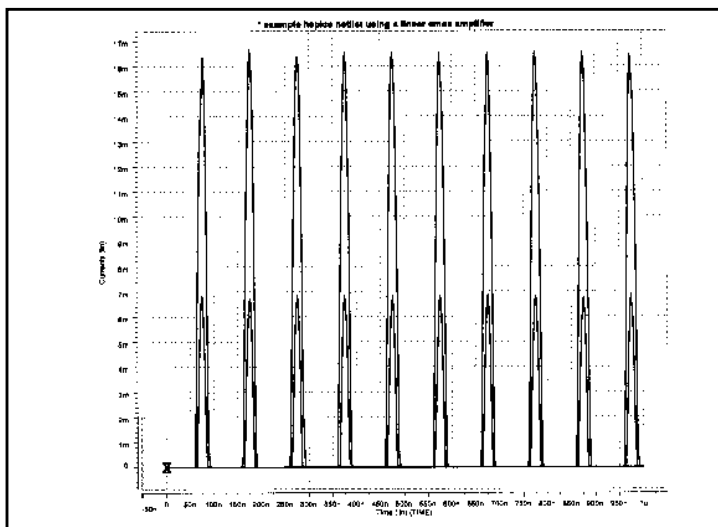
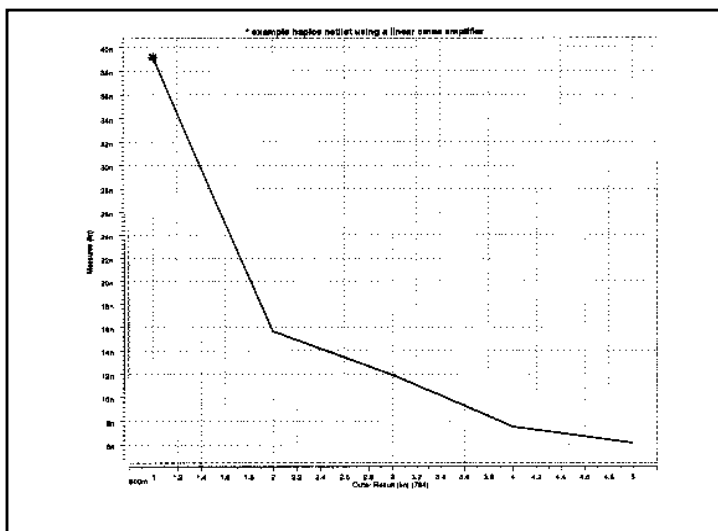


Figure D-3: Plot of Voltage on Node *clamp* vs. Time



**Figure D-4: Plot of Voltage on Node *inv1out* vs. Time****Figure D-5: Plot of Voltage on Node *inv2out* vs. Time**

**Figure D-6: Plot of Current through Diode *d1ow* vs. Time****Figure D-7: Plot of Measured Variable *falltime* vs. Amplifier Input Voltage**

# Simulation Example, with Cosmos-Scope

This example demonstrates the basic steps to perform simulation output, and to view the waveform results, using the Avant! Cosmos-Scope Waveform Viewer.

## Input Netlist and Circuit

The Syntax section below shows the input netlist for a BJT diff amplifier. Comment lines indicate the individual sections of the netlists. See [Simulation Input and Controls on page 3-1](#), for information about the individual commands.

The syntax is:

```
*file: bjtdiff.sp bjt diff amp with every analysis type
**
## ANALYSIS: ac dc tran tf noise new four sens pz disto temp
## OPTIONS: list node ingold=2 measdgt=5 numdgt=8
+ probe post
## TEMPERATURE: 25
*
* netlist options
.OPTION list node ingold=2 measdgt=6 numdgt=8 probe post
* defined parameters
.PARAM rb1x=aunif(20k,1k,30k) rb2x=aunif(20k,1k,30k)
* analysis specifications
.TF v(5) vin
.DC vin -0.20 0.20 0.01 sweep monte=3
.AC dec 10 100k 10meghz
.NOISE v(4) vin 20
.NET v(5) vin rout=10k
.PZ v(5) vin
.DISTRO rc1 20 .9 1m 1.0
.SENS v(4)
.TRAN 5ns 200ns
.FOUR 5meg v(5) v(15)
.TEMP -55 150
* output specifications
.MEAS qa_propdly trig v(1) val=0.09 rise=1
+      targ v(5) val=6.8 rise=1
.MEAS qa_magnitude max v(5)
.MEAS qa_rmspower rms power
.MEAS qa_avgv5      avg v(5)
.MEAS ac qa_bandwidth trig at=100k targ vdb(5) val=36 fall=1
.MEAS ac qa_phase find vp(5) when vm(5)=52.12
```

```

.MEAS ac qa_freq when vm(5)=52.12
.PROBE dc v(4) v(5) v(14) v(15)
.PROBE ac vm(5) vp(5) vm(15) vp(15)
.PROBE ac vt(5) vt(15)
.PROBE noise onoise(m) inoise(m)
.PROBE ac z11(m) z12(m) z22(m) zin(m)
.PROBE disto hd2 hd3 sim2 dim2 dim3
.PROBE tran v(4) v(5) v(14) v(15)
.PROBE tran p(vcc) p(vee) p(vin) power
* source statements
VIN 1 0 sin(0 0.1 5meg) ac 1
VCC 8 0 12
VEE 9 0 -12
* circuit statements
q1 4 2 6 qnl
q11 14 12 16 qpl
q2 5 3 6 qnl
q21 15 13 16 qpl
rs1 1 2 1k
rs11 1 12 1k
rs2 3 0 1k
rs12 13 0 1k
rc1 4 8 10k
rc11 14 9 10k
rc2 5 8 10k
rc12 15 9 10k
q3 6 7 9 qnl
q13 16 17 8 qpl
q4 7 7 9 qnl
q14 17 17 8 qpl
rb1 7 8 rb1x
rb2 17 9 rb2x
* model definitions
.MODEL qnl npn(bf=80 rb=100 ccs=2pf tf=0.3ns tr=6ns cje=3pf
+ cjc=2pf va=50 rc=10 trb=.005 trc=.005)
.MODEL qpl pnp(bf=80 rb=100 ccs=2pf tf=0.3ns tr=6ns cje=3pf
+ cjc=2pf va=50 bulk=0 rc=10)
*
.END

```

Use the previous example (linear CMOS amp) to draw a circuit diagram for this BJT diff amplifier. Also, specify parameter values.



## Execution and Output Files

This section displays the various output files from a Star-Hspice simulation of the BJT diff amplifier example. To execute the simulation, enter:

```
hspice bjtdiff.sp > bjtdiff.lis
```

where the input file is `bjtdiff.sp`, and the output file is `bjtdiff.lis`. Simulation creates the following output files:

<code>bjtdiff.ic</code>	Initial conditions for the circuit.
<code>bjtdiff.lis</code>	Text simulation output listing.
<code>bjtdiff.mt0</code>	Post-processor output, for <code>.MEASURE</code> statements.
<code>bjtdiff.st0</code>	Run-time statistics.
<code>bjtdiff.tr0</code>	Post-processor output, for transient analysis.
<code>bjtdiff.sw0</code>	Post-processor output, for DC analysis.
<code>bjtdiff.ac0</code>	Post-processor output, for AC analysis.
<code>bjtdiff.ma0</code>	Post-processor output, for AC analysis measurements.

## View Star-Hspice Results in Cosmos-Scope

The steps below show how to use the Avant! Cosmos-Scope Waveform Viewer, to view the results of AC, DC, and transient analysis, from the BJT diff amplifier simulation. Refer to previous examples of `.lis`, `.ic`, and `.st0` files.

### Viewing Star-Hspice Transient Analysis Waveforms

1. Invoke Cosmos-Scope.

From a Unix command line:

```
% cscope
```

On a Windows-NT system:

**Programs** > (*user_install_location*) > **Cosmos-Scope** .

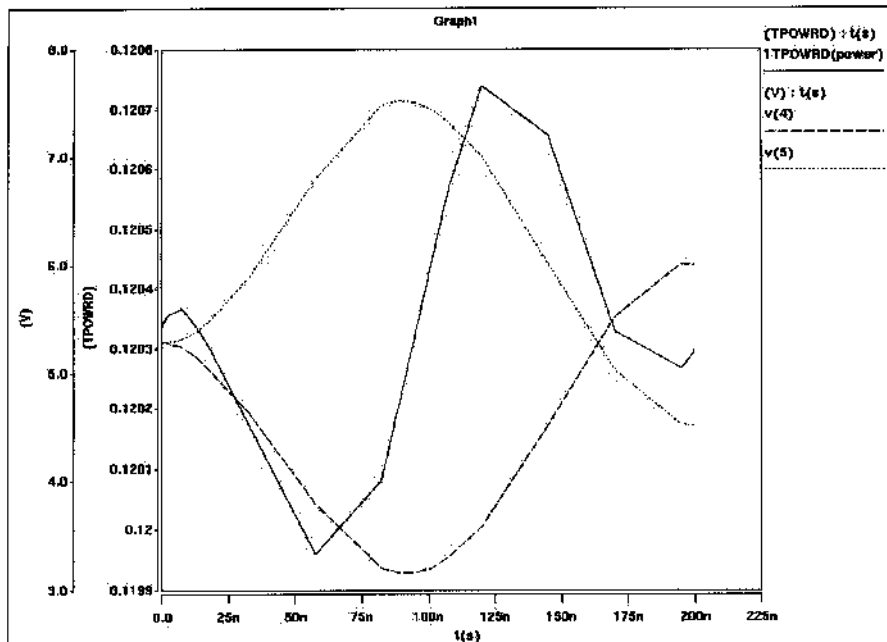
2. Open the Open Plotfiles dialog box:

**File** > **Open** > **Plotfiles** .

3. In the Open Plotfiles dialog box, in the Files of Type fields, select the Hspice Transient (*.tr*) item.
4. In the menu, click on `bjtdiff.tr0`, and click **Open**.  
The Signal Manager and `bjtdiff` Plot File windows open.
5. Hold down the Ctrl key, and select the `v(4)`, `v(5)`, and `ITPOWERD(power)` signals.
6. Click on **Plot** from the `bjtdiff` Plot File window.  
Three cascaded plots open.
7. To see three signals in one plot, right-click on the top-most signal name.  
The **Signal Menu** opens.
8. From the **Signal Menu**, select **Stack Region > Analog 0**.
9. Repeat Step 7 for the next top-most signal.

A plot opens, similar to [Figure D-8](#).

**Figure D-8: Transient Analysis Result: Plot of `v(4)`, `v(5)`, and `ITPOWERD(power)`**



## Viewing Star-Hspice AC Analysis Waveforms

1. From the Signal Manager dialog box, select `bjtdiff(1)`, and click on **Close Plotfiles**.

All transient plots (waveforms) close.

2. In the Signal Manager, click on **Open Plotfiles**.
3. In the **Open Plotfiles** dialog box, in the Files of Type fields, select the Hspice AC (*.ac*) item.
4. Click on `bjtdiff.ac0` in the menu, and click **Open**.

The `bjtdiff` Plot File windows open.

5. Hold down the Ctrl key, and select the `dim2(mag)` and `dim3(mag)` signals.
6. Click on **Plot** from the `bjtdiff` Plot File window.

Two cascaded plots open.

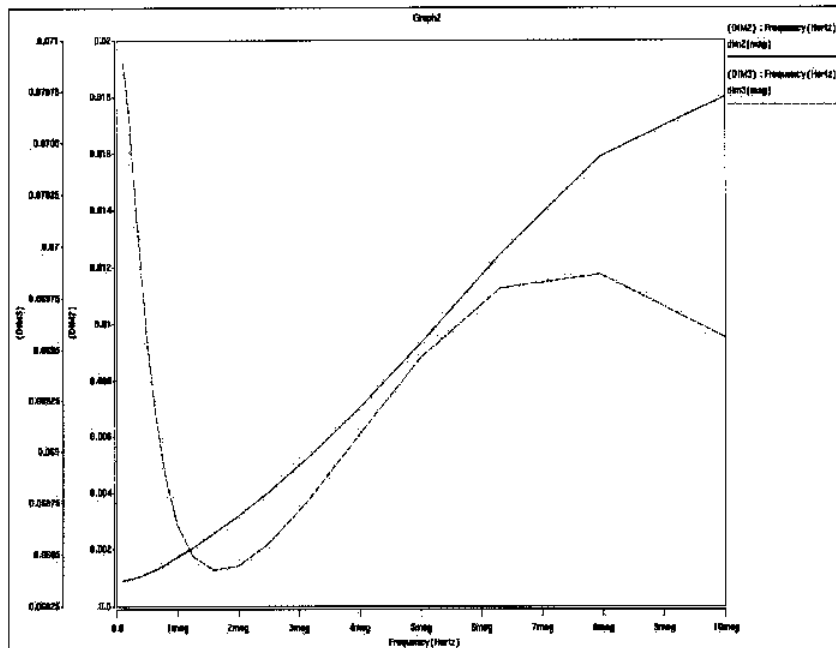
7. For two signals in a plot, right-click on `dim2(mag)`.

A **Signal Menu** opens.

8. From the **Signal Menu**, select **Stack Region > Analog 0**.

A plot opens, similar to [Figure D-9 on page D-18](#).

Figure D-9: AC Analysis Result: Plot of dim2(mag) and dim3(mag) from bjtdiff.ac0



## Viewing Star-Hspice DC Analysis Waveforms

1. From the Signal Manager dialog box, select `bjtdiff(1)`, and click on **Close Plotfiles**.  
All AC plots (waveforms) close.
2. In the Signal Manager, click on **Open Plotfiles**.
3. In the **Open Plotfiles** dialog, **Files of Type** field, select Hspice DC (* .sw*).
4. Click on `bjtdiff.sw0` and **Open** in the menu.  
The Plot File windows open.
5. Hold down the Ctrl key, and select all signals.
6. Click on **Plot** from the `bjtdiff` Plot File window.  
Four cascaded plots open.

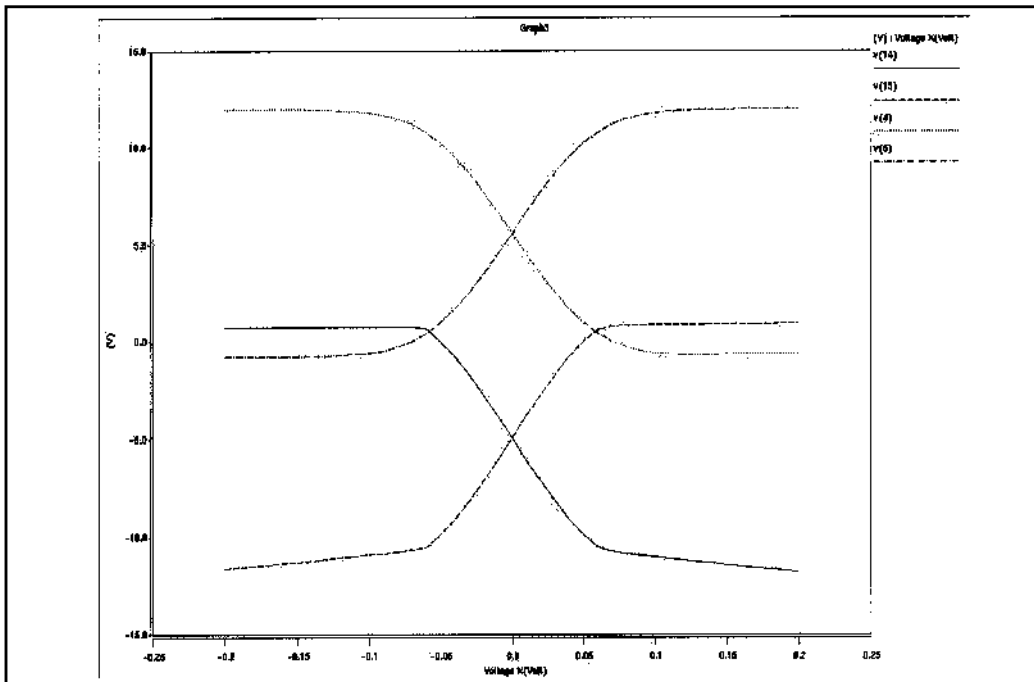
7. To see four signals in one plot, right-click on the name of the top-most signal.

A **Signal Menu** opens.

8. From the **Signal Menu**, select **Stack Region > Analog 0**.
9. Repeat Steps 7 and 8 for the next two top-most signals.

A plot opens, similar to [Figure D-10](#).

**Figure D-10: DC Analysis Result: Plot of v(14), v(15), v(4), and v(5) from bjtdiff.sw0**



The *Cosmos-Scope User's and Reference Manual* includes a full tutorial, information about the various Scope tools, and reference information about the Measure tool. You can also find more information on the Avant! website:

[http:// www.avanticorp.com](http://www.avanticorp.com)





# Index

---

## Symbols

- !GND node 3-16
- \$ comment delimiter 3-8
- \$installdir installation directory 3-55
- * comment delimiter 3-8
- .a2d file 3-78, 3-80, 5-56
- .AC statement 13-5, 13-38
  - external data 3-21
  - keywords 12-6
  - parameters 12-6
  - uses 12-4
- .ac# file 3-78, 3-80
- .ALTER
  - blocks 3-40
  - statement 3-39, 3-40, 3-45, 8-22
- .d2a file 5-56
- .DATA statement 3-20, 15-9
  - data-driven analysis 3-20
  - datanames 3-21
  - external file 3-24, 3-26
  - for sweep data 3-20
  - inline data 3-22
  - inner sweep example 3-23
  - outer sweep 3-23
  - outer sweep example 3-23
- .DC statement 10-14, 13-5, 13-38
  - external data with .DATA 3-21
- .DCVOLT statement 10-9
- .DEL LIB statement 3-6
  - in .ALTER blocks 3-40
  - with .ALTER 3-45
  - with .LIB 3-45
  - with multiple .ALTER statements 3-41
- .DISTO statement 12-13, 12-14
- .END statement
  - for multiple HSPICE runs 3-48
  - in libraries 3-33
  - location 3-48
  - missing 3-2
  - with .ALTER 3-40
- .ENDL statement 3-30, 3-31
- .ENDS statement 3-13
- .EOM statement 3-13
- .FFT statement 19-1, 19-6
- .FOURIER statement 11-44
- .ft# file 3-78, 3-80, 19-10
- .GLOBAL statement 3-18
- .gr# file 3-78, 3-80, 8-12
- .GRAPH statement 8-2, 8-12, 8-22, 22-6
- .ic file 3-78, 10-4
- .IC statement 10-3, 10-4, 10-9, 18-3
  - balancing input nodes 17-66
  - from .SAVE 10-11
- .inc file encryption C-12
- .INCLUDE statement 3-6, 3-28, 3-40, 3-56, 3-57
- .LIB
  - call statement 3-30, 3-31
  - definition statement 3-31
  - file encryption C-11
  - statement 3-6, 3-57
    - in .ALTER blocks 3-30, 3-40
    - nesting 3-32
    - with .DEL LIB 3-45
    - with multiple .ALTER statements 3-41
- .lis file 3-78, 3-79
- .LOAD statement 10-10
- .ma# file 3-78, 3-80

- .MACRO statement 3-11
- .MALIAS
  - statement 3-43
- .MEASURE statement 8-2, 8-3, 8-43, 9-8, 9-13
  - expression 8-61
  - failure message 8-41
  - parameters 7-7
  - performance 8-41
  - TDR files 16-9
- .MODEL statement 3-28, 13-5
  - BISECTION syntax 21-6
  - for .GRAPH 8-14
  - HSPICE version parameter 3-30
  - model name 3-28
  - op-amp 17-67
  - OPT method 21-6
  - optimization syntax 13-40
  - PASSFAIL syntax 21-6
  - syntax 6-5
- .ms# file 3-78, 3-80
- .mt# file 3-78, 3-80
- .NET statement 12-18
- .NODESET statement 9-29, 10-3, 10-23, 18-3
  - balancing input nodes 17-66
  - DC operating point initialization 10-10
  - from .SAVE 10-11
- .NOISE statement 12-15
- .OP statement 10-5, 10-6, 11-3
- .OP statement parameters 10-6
- .OPTION 10-27, 10-30
  - .ALTER blocks 3-40
  - ACCT, summary of job statistics 8-19
  - ALT999 or ALT9999 extension 8-18
  - AUTOSTOP 17-75
  - CO, for printout width 8-17
  - DCSTEP 10-45
  - DI 9-23, 9-38, 10-37, 11-24, 12-9
  - INGOLD, for printout numerical format 8-18
  - keyword application table 9-3
  - POST, for high resolution graphics 8-19
  - SEARCH 3-34
  - SIM_ANALOG 4-17
  - SIM_RAIL 4-17
  - statement 9-2, 10-30
- .pa# file 3-78, 3-80
- .PARAM statement 3-13–3-19, 3-36, 8-42, 8-64, 13-2
  - in .ALTER blocks 3-40
  - optimization 13-39
- .PLOT statement 8-2
  - in .ALTER block 3-39
  - simulation results 8-8, 8-22
- .PRINT statement 8-2
  - in .ALTER 3-39
  - simulation results 8-4, 8-22
- .PROBE statement 8-2, 8-10, 8-22
- .PROTECT statement 3-38, C-2
- .PZ statement 10-19, 18-4, 20-2
- .SAMPLE statement 12-17
- .SAVE statement 10-10
- .SAVE statement parameter 10-12
- .SENS statement 10-19
- .sp file encryption C-10
- .st# file 3-78, 3-81
- .STIM statement 8-2, 8-65
- .SUBCKT statement 3-11, 8-42, 17-66
- .sw# file 3-78, 3-80
- .TEMP statement 3-19–3-20, 13-5, 13-6–13-7
- .TF statement 10-19, 17-48
- .TITLE statement 3-8
- .tr# file 3-78, 3-80
- .TRAN statement 13-5, 13-38
- .UNPROTECT statement 3-38, C-2
- .WIDTH statement 8-18

## A

- A2D
  - function 5-56, 17-4
  - model parameter 5-56
  - output model parameters 5-61
  - See also* mixed mode
- ABS element parameter 17-13, 17-19, 17-24, 17-31
- abs(x) function 7-10
- ABSH option 9-22, 9-37, 10-35, 11-23, 12-9
- ABSI option 9-22, 9-23, 10-25, 10-33, 10-35
- ABSMOS option 9-22, 9-23, 10-25, 10-33, 10-35



- absolute
  - power function 7-10
  - value function 7-10
  - value parameter 17-13, 17-19, 17-24, 17-31
- ABSTOL option 9-23, 10-22
- ABSV option 9-37, 10-33, 11-23
- ABSVAR option 9-44, 9-46, 11-23, 11-26, 11-33
- ABSVDC option 9-23, 10-35
- AC analysis 8-3
  - circuit and pole/zero models 20-43
  - control options 12-9
  - distortion 12-13
  - magnitude 9-5, 9-38, 12-9
  - networks 12-18
  - noise 12-15
  - optimization 12-6
  - output 8-33, 9-5, 9-38, 12-9
  - phase 9-5, 9-38, 12-9
  - RC network 12-10
  - resistance 12-3, 17-75
  - small signals 9-37, 12-2
  - sources 5-5
- accessing customer support viii
- ACCT option 8-19, 9-5
- accuracy
  - control options 10-34
  - element parameter 20-10
  - simulation time 10-34
  - tolerance 10-32, 11-31
- ACCURATE option 9-37, 11-23, 11-33
- ACM model parameter 11-33
- acos(x) function 7-10
- ACOUT option 8-34–8-35, 9-5, 9-38, 12-9
- adder
  - circuit 22-3
  - demo 22-3
  - NAND gate binary 22-4
  - subcircuit 22-3
- admittance
  - AC input 8-38
  - AC output 8-38
  - Y parameters 8-33
- AF model parameter 12-16
- AGAUSS keyword 13-15
- ALFA, .FFT parameter 19-7
- algebraic
  - equations, example 15-11
  - expressions 7-9
  - models 11-33
- algorithms
  - Damped Pseudo Transient algorithm 10-46
  - DVDT 9-44, 9-48, 11-19, 11-23, 11-39, 11-40
  - GEAR 9-48, 11-19, 11-35
  - integration 11-35
  - iteration count 11-39
  - Levenberg-Marquardt 13-52
  - local truncation error 9-39, 9-40, 9-48, 11-19, 11-26, 11-27, 11-39, 11-40
  - Muller 18-3
  - pivoting 9-25, 9-27, 10-26
  - timestep control 9-44, 11-18, 11-38, 11-39, 11-41
  - transient analysis timestep 9-48, 11-19
  - TRAP 9-48, 11-19
  - trapezoidal integration 9-49, 11-20, 11-35
- ALL keyword 10-6, 10-12
- AM
  - behavioral 17-63
  - modulation frequency 19-11
  - source function 5-23, 5-23–5-24
- AMP model parameter 17-67
- amplifiers, pole/zero analysis 18-13, 18-15
- analog
  - circuit simulation of a digital system 16-4
  - elements 17-54
  - modeling 17-4
- Analog Artist interface 9-12, 9-14
  - See also* Artist
- analysis
  - .MEASURE statement 8-3
  - AC 8-3
  - accuracy 10-32–10-34
  - circuit model 20-43
  - data driven 13-2, 13-3, 15-9
  - DC 8-3
  - distortion 12-13
  - element template 8-3
  - FAQ A-2
  - FFT 11-50

analysis (*continued*)

- AM modulation 19-11
- modulator/demodulator 19-14
- test circuit 19-22
- windows 19-2
- Fourier 11-43
- initialization 10-3
- inverter 11-12
- Monte Carlo 13-3, 13-13, 13-13–13-32
- network 12-18
- optimization 13-38
- parametric 8-3
- pole/zero 10-21, 18-1
  - active low-pass filter 18-17
  - CMOS differential amplifier 18-13
  - high-pass Butterworth filter 18-11
  - Kerwin's circuit 18-10
  - low-pass filter 18-7
  - model 20-43
  - overview 18-2
  - simple amplifier 18-15
  - using Muller method 18-3
- pulse width 21-14
- RC network 11-10, 12-10
- setup time 21-9
- spectrum 19-1
- statistical 13-8–13-32
- Taguchi 13-2
- temperature 13-2, 13-5
- timing 21-1
- transfer function (.TF) 17-48
- transient 8-3, 11-3
- worst case 13-2, 13-8–13-32
- yield 13-2

arccos(x) function 7-10

arcsin(x) function 7-10

arctan(x) function 7-10

arithmetic expression measurement 8-61

arithmetic operators 7-10

ARTIST option 9-12, 9-14

ASCII output data 9-12, 9-13, 9-14

ASIC libraries 3-56

asin(x) function 7-10

ASPEC option 9-15

asterisk comment delimiter 3-8

asymptotic waveform evaluation 20-2, 20-33

AT keyword 8-47

atan(x) function 7-10

ATEM characterization system 3-55

AUNIF keyword 13-15

AURORA user's group viii

autoconvergence 9-31, 10-37, 10-40

AUTOSTOP option 9-41, 11-28, 11-31, 15-9, 17-75

AV model parameter 17-67

AV1K model parameter 17-68

AvanLink

- Cadence products B-2
- DA
  - design flow B-10
  - environment B-9
  - Netlister B-11
  - schematic B-10
  - simulation B-12
- design flow B-5
- environment B-3
- library operations B-6
- Mentor Graphics products B-8
- netlist B-7
- schematic B-6
- simulation B-7

Avant! web site viii

AvanWaves waveform display B-7, B-12

AVD model parameter 17-67

average (AVG) measurement 8-48

average deviation 13-3

average value, measuring 8-53

AVG keyword 8-54

AWE *See* asymptotic waveform evaluation

## B

B# node name in CSOS 3-18

backslash continuation character 3-3, 7-9

BADCHR option 3-3, 9-17

BART FFT analysis keyword 19-7

Bartlett FFT analysis window 19-3, 19-5, 19-24

- behavioral
  - 741 op-amp 17-77
  - amplitude modulator 17-63
  - AND and NAND gates 17-34
  - BJTs 17-90, 17-95
  - CMOS inverter 17-48
  - comparator 17-81
  - components 17-42
  - current source 5-45, 17-23
  - data sampler 17-64
  - differentiator 17-56
  - D-Latch 17-36
  - double-edge triggered flip-flop 17-39
  - elements 17-3, 17-54
  - flip-flop 17-39
  - gates 17-34
  - integrator 17-54
  - LC oscillator 17-84
  - look-up tables 17-42
  - n-channel MOSFETs 17-37, 17-44
  - p-channel MOSFETs 17-37
  - phase detector model 17-88
  - phased locked loop 17-87
  - ring oscillator 17-52
  - silicon controlled rectifier 17-60
  - transformer 17-58
  - triode vacuum tube 17-61
  - tunnel diode 17-59
  - VCO model 17-82
  - voltage source 5-33, 17-12
  - VVCAP model 17-85
- Behavioral resistors 4-5
- Behavioral capacitors 4-10
- BETA keyword 12-17
- Biaschk 11-14
- binary output 9-14
- binary search 21-5–21-10
- Bipolar Junction Transistors. *See* BJTs
- bisection
  - data, printing 9-9
  - error tolerance 21-7
  - function 21-1, 21-6
  - methodology 21-4
  - overview 21-2
  - pass-fail method 21-4

- bisection (*continued*)
  - requirements 21-5
  - violation analysis 21-3
- BISECTION model parameter 21-6
- BJTs
  - current flow 8-28
  - element template listings 8-75
  - elements, names 4-21
  - EXPLI 9-21
  - power dissipation 8-30
  - S-parameters, optimization 13-59
- BKPSIZ option 9-41, 11-28
- BLACK FFT analysis keyword 19-7
- Blackman FFT analysis window 19-3, 19-25
- Blackman-Harris FFT analysis window 19-3, 19-26
- bond wire example 22-10
- branch current
  - error 9-22, 9-23, 10-22, 10-35
  - output 8-26
- breakpoint table
  - reducing size 11-42
  - size 9-41, 11-28
- BRIEF option 9-2, 9-6, 9-7, 9-8, 9-9, 10-6
- Broyden update data, printing 9-9
- BSIM model, LEVEL 13 3-30
- BSIM2 LEVEL 39 model 3-30
- buffer 4-40
- bus notation 5-69
- Butterworth filter pole/zero analysis 18-11
- BW model parameter 17-70
- BYPASS option 9-41, 11-18
- BYTOL option 9-41, 11-24

## C

- C Element (capacitor) 4-9
- C2 model parameter 17-68
- Cadence
  - Analog Artist, with AvanLink B-2
  - Composer, with AvanLink B-2
  - Opus 9-12
  - WSF format 9-12

- capacitance
  - charge tolerance, setting 9-38, 11-24
  - CSHUNT node-to-ground 9-38, 11-18
  - CTYPE 4-10
  - element parameter 4-7
  - manufacturing variations 13-23
  - pins 15-6
  - scale factor, setting 9-35, 10-30
  - table of values 9-28, 10-22
  - voltage variable 17-85
- capacitance-voltage plots, generating 9-28, 10-23
- capacitor
  - conductance requirement 10-45
  - current flow 8-27
  - element 4-6, 4-9, 8-71
  - linear 4-9
  - models 3-29, 4-7
  - switched 20-44
  - voltage controlled 5-46, 5-50
- capacitors
  - charge-conserving 4-11
- CAPOP model parameter 11-33
- CAPTAB option 9-28, 10-22
- CCCS element parameter 5-40, 17-19
- CCVS element parameter 5-52, 17-31
- cell characterization 13-2, 15-9
- CENDIF optimization parameter 13-41
- characterization of models 10-14
- charge tolerance, setting 9-38, 11-24
- CHGTOL option 9-38, 11-24, 11-40
- circuit
  - description syntax 2-9
- circuits
  - AC analysis 20-43
  - adder 22-3
  - inverter, MOS 11-12
  - model 20-41
  - nonconvergent 10-49
  - RC line 20-32
  - RC network 12-10
  - reusable 3-51
  - Signetics drivers 16-16
  - subcircuit numbers 3-17
  - temperature 13-5, 13-6
  - circuits (*continued*)
    - test, FFT analysis 19-22
    - transient responses 20-43
    - Xilinx FPGAs 16-19
    - See also* subcircuits
  - CLOAD model parameter 5-61
  - clock skew 16-5
  - CLOSE optimization parameter 13-41
  - CMI
    - function calling protocol 14-32
    - models, simulations 14-5
    - testing 14-12
    - variables 14-17
  - CMI_AssignInstanceParm 14-21
  - CMI_AssignModelParm 14-20
  - CMI_Conclude 14-31
  - CMI_DiodeEval 14-25
  - CMI_Evaluate 14-23
  - CMI_FreeInstance 14-28
  - CMI_FreeModel 14-28
  - CMI_Noise 14-26
  - CMI_PrintModel 14-27
  - CMI_ResetInstance 14-20
  - CMI_ResetModel 14-19
  - CMI_SetupInstance 14-23
  - CMI_SetupModel 14-22
  - CMI_Start 14-30
  - CMI_WriteError 14-29
  - CMOS
    - differential amplifier, pole/zero analysis 18-13
    - output driver demo 22-10
    - tristate buffer, optimization 13-54
  - CMRR model parameter 17-5, 17-68
  - CO option 8-18, 9-6
  - coefficients
    - Laplace 20-23
    - transfer function 20-20
  - column laminated data 3-27
  - commands
    - Hspice 3-60–3-65
    - limit descriptors 8-22
    - output 8-2

- comment line
  - continuation 2-12
  - netlist 2-12
- comment line (digital vector files) 5-85
- comments 3-8
- common
  - emitter gain 22-17
  - model interface (CMI) 14-1
- Common Simulation Data Format 9-12, 9-13
- COMP model parameter 17-66, 17-68
- comparators 17-65, 17-81
- complex poles and zeros 20-28
- compression of input files 3-2
- computer platforms for HSpice 1-6
- concatenated data files 3-25
- conductance
  - current source, initialization 9-32, 10-37
  - for capacitors 10-45
  - minimum, setting 9-38, 11-28
  - models 9-31, 10-23
  - MOSFETs 9-32, 10-38
  - negative, logging 9-17
  - node-to-ground 9-33, 9-38, 10-24, 11-18
  - pn junction 10-52
  - scale, setting 9-35, 10-30
  - sweeping 9-33, 10-24
- configuration file 2-4, 3-59
  - example 2-6
- continuation character, parameter strings 7-9
- continuation of line
  - netlist 2-12
- continuation of line (digital vector file) 5-85
- control characters in input 3-3
- control options
  - accuracy 9-22, 10-34
    - AC 9-37, 9-37–9-40
    - defaults 11-41
  - algorithm 9-47–??
  - algorithm selection 10-22
  - analysis 9-15–9-16
  - convergence 9-29–9-34, 10-22, 10-34
  - DC convergence 10-22
  - DC operating point analysis 10-22
  - defaults 9-2

- control options (*continued*)
  - error 9-17
  - FAQ A-27
  - initialization 10-22
  - input and output 9-5–9-11, 9-28, 9-51
  - interface options 9-12–9-14
  - keyword table 9-3
  - limit 11-28
  - matrix-related 9-25–9-28
  - method 11-18
  - pole/zero 9-35–9-36, 10-30–10-31, 18-4
  - printing 8-17, 9-9
  - setting 9-2
  - speed 9-41–9-42
  - table 9-5
  - timestep 9-44–9-47
  - tolerance 11-23
  - transient analysis
    - limit 11-28–11-30
    - method 11-18–11-20
    - tolerance 11-23–??
  - version 9-17
- controlled sources 5-26, 5-27, 17-4, 17-6, 17-7
- conventions
  - bias polarity 14-37
  - source-drain reversal conventions 14-38
- CONVERGE option 9-29, 9-31, 10-36, 10-39, 10-46
- convergence
  - control options 10-34
  - current 9-22–9-24, 9-37, 9-38, 9-39, 10-35, 10-37, 10-38, 11-23–11-25, 12-9
  - ensuring 10-23
  - for optimization 13-42
  - increasing iterations 10-23
  - problems 10-47
    - .NODESET statement 10-10
    - analyzing 10-47
    - autoconverge process 10-40
    - causes 9-41, 10-49, 11-18
    - changing integration algorithm 9-49, 11-20
    - CONVERGE option 9-29, 9-31, 10-36, 10-46
    - DCON setting 9-30, 10-37, 10-42
    - decreasing the timestep 9-44, 11-28
    - diagnosing 10-47–10-52

- convergence, problems (*continued*)
  - diagnostic tables 10-47
  - floating point overflow 10-46
  - GMINDC ramping 10-42
  - internal timestep too small 9-48, 11-19
  - nonconvergent node listing 9-31, 10-37
  - op-amp models 17-66
  - operating point Debug mode 10-7
  - pole/zero analysis 18-3
  - reducing 10-43
  - setting DCON 9-31, 10-37
  - steady state 9-33, 10-24
- cos(x) function 7-10
- cosh(x) function 7-10
- coupled line noise simulation example 16-23
- CPTIME option 9-12
- CPU time
  - limiting 9-12
  - reducing 9-9
  - setting maximum 9-12
- critical frequency 20-5
- CROSS keyword 8-46
- CSCAL option 9-35, 10-30, 18-4
- CSDF option 9-12, 9-13
- CSHDC option 9-29, 10-22
- CSHUNT option 9-38, 11-18
- CTYPE 4-10
- CUR element parameter 17-24
- current
  - ABSMOS floor value for convergence 9-24, 10-38
  - branch 8-27, 9-22, 10-22, 10-35
  - controlled
    - current sources 5-40, 8-73, 17-7, 17-18
    - elements 17-6
    - sources 5-26
    - voltage sources 5-52, 8-74, 17-6, 17-30
  - in HSPICE elements 8-27
  - operating point table 10-6
  - output 8-26
  - sources 5-44
- CURRENT keyword 10-6
- customer support viii
- CUT optimization parameter 13-41

- C-V plots 9-28, 10-23, 22-6
- CVTOL option 9-20

## D

- D2A
  - function 5-56, 17-4
  - input model parameters 5-58
  - model parameter 5-56
  - See also* mixed mode
- Damped Pseudo Transient algorithm 10-46
- data
  - encryption C-1
  - files, disabling printout 9-6, 9-9
  - flow, overview 1-8
  - sampler, behavioral 17-64
  - sheet parameters 15-1, 15-2
- DATA keyword 3-20, 10-15, 11-6, 12-6
- .DATA statement 3-24
- data-driven analysis 13-2, 13-3, 15-9
  - PWL source function 5-20
- dataname 8-67
- datanames 3-21
- db(x) function 7-11
- DC
  - analysis 8-3, 10-19–10-21
    - capacitor conductances 10-45
    - decade variation 10-16
    - initialization 9-30, 10-22, 10-23
    - iteration limit 9-25, 10-25
    - linear variation 10-16
    - list of points 10-16
    - octave variation 10-16
    - sensitivity 10-19
    - transfer function 10-20
  - convergence control options 10-22
  - errors, reducing 10-43
  - operating point
    - analysis 10-6
    - bypassing 11-3
    - initial conditions file 3-59
    - See also* operating point
  - optimization 10-15
  - sensitivity analysis 10-19
  - small-signal analysis 10-20

- DC (*continued*)
  - sources 5-5
  - sweep 10-14
  - transfer function 10-20
- DCCAP option 9-28, 10-23, 22-6
- DCFOR option 9-29, 10-23
- DCHOLD option 9-30, 10-23
- DCON option 9-30, 10-37, 10-39, 10-40
- DCSTEP option 9-31, 10-23, 10-45
- DCTRAN option 9-32, 10-37
- DDL 3-55, 3-56, 17-5, 22-17
- DDLPATH environment variable 3-56, 22-17
- DEBUG keyword 10-7
- DEC keyword 10-16, 11-7, 12-7
- decibel function 7-11
- decoupling methods 16-6
- DEF model parameter 17-69
- DEFAD option 9-20
- DEFAS option 9-20
- DEFAULT_INCLUDE variable 3-59
- DEFL option 9-20
- DEFNRD option 9-20
- DEFNRS option 9-20
- DEFPD option 9-20
- DEFPS option 9-20
- DEFW option 7-18, 9-20
- DELAY element parameter 17-13, 17-19, 17-25, 17-31
- delays
  - causes 16-3
  - derating curve 15-3
  - element example 5-51
  - Elmore 20-31
  - group 8-36, 9-52
  - measuring 8-43
  - plotting 15-5
  - problems and solutions 16-3
  - simulation example 15-2, 15-9
  - time (TD) 8-36
  - types 16-5
- DEL element parameter 20-6, 20-10
- DELMAX option 9-44, 9-46, 11-28, 11-29, 11-32, 11-33, 11-42, 11-45
- DELPHS 17-69
- DELTA
  - element parameter 17-10, 17-13, 17-19, 17-25, 17-31, 17-59, 20-10
  - internal timestep 9-44, 11-28
  - See also* timestep
- DELVTO model parameter 13-9
- demo files
  - .ALTER statement 22-23
  - 2n2222 BJT's transistor characterization 22-31
  - 2n3330 JFET's transistor characterization 22-31
  - A/D flash converter 22-28
  - A2D 22-27, 22-28
  - AC analysis 22-23
  - acl gate 22-24
  - adders
    - 72-transistor two-bit 22-26
    - BJT NAND gate two-bit 22-24
    - BJT two-bit 22-23
    - D2A 22-27
    - MOS two-bit 22-24
    - NAND gate four-bit binary 22-23
  - air core transformer 22-35
  - algebraic
    - output variables 22-23
    - parameters 22-23
    - transmission lines 22-40
  - AM source 22-38
  - amplifier 22-27
  - amplitude modulator 22-24
  - analog 22-26
  - AND gate 22-24
  - automatic model selection program 22-36
  - behavioral applications 22-24–22-26
  - behavioral models 22-26
    - diode 22-24
    - D-latch 22-25
    - filter 22-23
    - NAND gate 22-25
    - ring oscillator 22-25
    - triode 22-26
    - voltage to frequency converter 22-23
  - benchmarks 22-26
  - bisection 22-26, 22-27

demo files (*continued*)

BJTs

- analog circuit 22-26
- beta plot 22-26
- differential amplifier 22-23, 22-27
- diodes 22-26–22-27
- ft plot 22-26
- gm, gpi plots 22-26
- photocurrent 22-38
- Schmidt trigger 22-23
- sense amplifier 22-23

BSIM3 model, LEVEL=47 22-35

capacitances, MOS models

- LEVEL=13 22-35

- LEVEL=2 22-35

- LEVEL=6 22-35

cell characterization 22-23, 22-25, 22-27

charge conservation, MOS models

- LEVEL=3 22-35

- LEVEL=6 22-36

circuit optimization 22-27–22-28

CMOS

- differential amplifier 22-24

- I/O driver ground bounce 22-23, 22-39

- input buffer 22-27

- inverter macro 22-25

- output buffer 22-27

coax transmission line 22-39

crystal oscillator 22-24

current controlled

- current source 22-25

- voltage source 22-25

D2A 22-27

DC analysis, MOS model LEVEL=34 22-36

DDL 22-28–22-31

delay 22-24, 22-27

device optimization 22-32

differential amplifier 22-23, 22-24

differentiator 22-24

diffusion effects 22-24

diode photocurrent 22-38

D-latch 22-25

E Element 22-24

edge triggered flip-flop 22-24

exponential source 22-38

demo files (*continued*)

FFT

- AM source 22-33

- analysis 22-33–22-34

- Bartlett window 22-33

- Blackman window 22-33

- Blackman-Harris window 22-33

- data-driven transient analysis 22-33

- exponential source 22-33

- Gaussian window 22-33

- Hamming window 22-33

- Hanning window 22-33

- harmonic distortion 22-33

- high frequency detection 22-33

- intermodulation distortion 22-33

- Kaiser window 22-34

- modulated pulse source 22-34

- Monte Carlo, Gaussian distribution 22-33

- product of waveforms 22-33

- pulse source 22-34

- PWL 22-34

- rectangular window 22-34

- single-frequency FM source 22-34

- sinusoidal source 22-33

- small-signal distortion 22-33

- switched capacitor 22-34

- transient 22-33

- window tests 22-34

filter matching 22-28

filters 22-34–22-35

- behavioral 22-23

- fifth-order 22-25, 22-34

- fourth-order Butterworth 22-34

- Kerwin's circuit 22-34

- LCR bandpass 22-34

- matching lossy to ideal 22-28

- ninth-order low-pass 22-25, 22-35

- switched capacitor low-pass 22-24

FR-4 microstrip transmission line 22-35, 22-39

G Element 22-23, 22-24

GaAsFET amplifier 22-24

gamma model LEVEL=6 22-36

general applications 22-23–22-24

ground bounce 22-23, 22-39

group time delay 22-24

impact ionization plot 22-36



demo files (*continued*)

- input 22-23
- installation test 22-26
- integrator 22-25
- inverter 22-23, 22-25, 22-27
  - characterization 22-27
- IRF340 NMOS transistor characterization 22-31
- I-V plots
  - LEVEL=3 22-36
  - MOSFETS model LEVEL=13 22-36
  - SOSFETS model LEVEL=27 22-36
- JFETs photocurrent 22-38
- junction tunnel diode 22-26
- LCR circuit 22-27
- lumped
  - MOS model 22-23
  - transmission lines 22-35, 22-40
- magnetic core transformer 22-35
- magnetics 22-35
- microstrip transmission lines 22-35, 22-39
  - coupled 22-39
  - optimization 22-39
  - series 22-39
- Monte Carlo analysis 22-24
  - Gaussian distribution 22-24
  - limit function 22-24
  - uniform distribution 22-24
- MOS 22-26, 22-27
- MOSFETs 22-35–22-36
  - sigma sweep 22-27
  - sweep 22-24
- NAND gate 22-24, 22-25
- NMOS E-mode model, LEVEL=8 22-38
- noise analysis 22-23
- op-amp 22-23, 22-25
  - characterization 22-28–22-31
  - voltage follower 22-25, 22-39
- optimization 22-25
  - 2n3947 Gummel model 22-32
  - DC 22-32
  - diode 22-32
  - GaAs 22-32
  - group delay 22-27
  - Hfe 22-32
  - I-V 22-32
  - JFETs 22-32

demo files, optimization (*continued*)

- LEVEL=2 model beta 22-32
- LEVEL=28 22-32
- MOS 22-32
- s-parameter 22-32
- speed, power, area 22-27
- width 22-27
- parameters 22-23
- phase
  - detector 22-25
  - locked loop 22-24
- photocurrent 22-36–22-38
  - GaAs device 22-38
- photolithographic effects 22-24
- pll 22-24
- pole/zero analysis 22-23, 22-34
- pulse source 22-38
- PWL 22-38
  - CCCS 22-25
  - CCVS 22-25
  - switch element 22-25
  - VCCS 22-24, 22-26
  - VCO 22-26
  - VCVS 22-25
- radiation effects 22-36–22-38
  - bipolar devices 22-36–22-37
  - DC I-V, JFETs 22-38
  - GaAs differential amplifier 22-38
  - JFETs devices 22-37
  - MOSFETs devices 22-37–22-38
  - NMOS 22-38
- RC circuit optimization 22-28
- resistor temperature coefficients 22-28
- RG58/AU coax test 22-35
- ring oscillator 22-25
- Royer magnetic core oscillator 22-35
- Schmidt trigger 22-23
- sense amplifier 22-23
- series source coupled transmission lines 22-40
- setup 22-26, 22-27
  - characterization 22-27
- shunt terminated transmission lines 22-40
- silicon controlled rectifier 22-25
- sine wave sampling 22-25
- single-frequency FM source 22-39
- sinusoidal source 22-39

- demo files (*continued*)
  - skew models 22-24
  - SNAP to HSPICE conversion 22-27
  - sources 22-38–22-39
  - s-parameters 22-26, 22-34, 22-36
  - sweep 22-24
  - switch 22-25
  - switched capacitor 22-24, 22-25, 22-39
  - temperature effects
    - LEVEL=13 22-36
    - LEVEL=6 22-36
  - timing analysis 22-26
  - total radiation dose 22-38
  - transient analysis 22-23
  - transistor characterization 22-31
  - transmission lines 22-39–22-40
  - triode model 22-26
  - tunnel diodes 22-26, 22-27
  - twinlead transmission line model 22-40
  - U models 22-40
  - unity gain frequency 22-27
  - Viewsim
    - A2D input 22-27
    - D2A input 22-27
  - voltage follower 22-25
  - voltage-controlled
    - current source 22-24, 22-26
    - oscillator 22-23, 22-26
    - resistor inverter 22-39
    - voltage source 22-25
  - voltage-to-frequency converter 22-23
  - voltage-variable capacitor 22-24
  - waveform smoothing 22-25
  - worst case skew model 22-24
- DERIVATIVE keyword 8-56
- derivative, measuring 8-49
- design
  - high speed, problems and solutions 16-3
  - name 3-58
  - partitioning 16-7
  - stability 20-2
  - time, reducing 20-2
- Design Architect, with AvanLink B-8
- Design Viewpoint Editor (DVE) B-8
- deviation, average 13-3
- device characterization 3-55
- DFT 19-1
- DI control option 9-23, 9-38, 10-37, 11-24, 12-9
- DIAGNOSTIC option 9-17
- diagnostic tables 10-47–10-48
- differentiator, behavioral 17-56
- DIFSIZ optimization parameters 13-41
- digital
  - files 5-56, 17-34
  - input 5-56
  - vector file 5-68
- DIM2
  - distortion measure 12-13
  - parameter 8-38
- DIM3
  - distortion measure 12-13
  - parameter 8-38
- diodes
  - breakdown example 5-51
  - CMI_DiodeEval 14-25
  - current flow 8-28
  - elements 8-75
  - equations 5-51
  - junction 4-19
  - models 3-29, 4-18
  - polysilicon capacitor length 4-19
  - power dissipation 8-30
- directories
  - installation directory 3-55
  - tmp 3-63
- DIS 17-69
- Discrete Fourier Transform 19-1
- distortion 8-38, 12-13
- documents
  - conventions v
  - FAQ A-4
- dollar sign (\$) comment delimiter 3-8
- drain-to-source current 9-22, 10-35
- DTEMP parameter 13-5, 13-6, 22-15
- DV option 9-30, 9-32, 10-23, 10-37, 10-39
- DVDT
  - algorithm 9-44, 9-46, 11-23, 11-26, 11-35, 11-39
  - option 9-44, 9-48, 11-18, 11-19, 11-32, 11-39, 11-40, 11-41

DVTR option 9-47, 11-28  
dynamic timestep algorithm 11-40

## E

E Elements 17-6, 17-12, 20-25–20-31  
    applications 17-6  
    controlling voltage 17-15  
    data points 17-14  
    element multiplier 17-14  
    element name 17-13  
    gate type 17-13  
    initial conditions 17-13  
    integrator application 17-56  
    Laplace transform 20-4  
    maximum voltage 17-14  
    minimum voltage 17-14  
    NAND gate 17-34  
    parameters 17-13, 20-11  
    polynomials 17-14  
    sampler application 17-64  
    SCALE parameter 20-36  
    syntax statements 5-33  
    temperature coefficients 17-15  
    time delay keyword 17-15  
    transformer application 17-58  
    triode application 17-62  
    turns ratio 17-14  
    voltage gain 17-13  
electrical measurements 22-17  
Element  
    capacitance  
        CTYPE 4-10  
element  
    active  
        BJTs 4-20  
        diodes 4-18  
        JFETs 4-23  
        MESFETs 4-23  
        MOSFETs 4-25  
    analog behavioral 17-54  
    C (capacitor) 4-9  
    checking, suppression of 9-9  
    current controlled 17-6  
    IC parameter 10-8

element (*continued*)  
    ideal 5-27  
    identifiers 2-11  
    independent source 5-2, 5-7  
    instantiate 2-11  
    L (inductor) 4-16  
    markers, mutual inductors 4-15  
    names 3-16  
    OFF parameter 9-34, 10-4, 10-26  
    parameters *See* element parameters 4-1  
    passive  
        capacitors 4-6  
        inductor 4-11  
        mutual inductor 4-14  
        resistors 4-2  
    R (resistor) 4-4  
    statements 3-9, 3-55  
        current output 8-26  
        G Elements 17-22–17-29  
        independent sources 5-2  
        Laplace 20-6  
        op-amps 17-66  
        pole/zero 20-7  
    subcircuits 3-13  
    temperature 13-6  
    templates 8-39–8-75  
        analysis 8-3  
        BJTs 8-75  
        capacitor 8-71  
        current-controlled 8-73  
        function 7-12  
        independent 8-74  
        inductor 8-72  
        JFETs 8-78  
        MOSFETs 8-79  
        mutual inductor 8-72  
        resistor 8-71  
        saturable core 8-83  
        voltage-controlled 8-72, 8-73  
    transmission line 4-30, 4-33, 4-35  
    voltage-controlled 5-26, 17-6  
element parameters  
    .ALTER blocks 3-40  
    BJTs 4-21  
    capacitors 4-7–4-8

## element parameters (*continued*)

- DELTA 17-10
- DTEMP 13-5
- E Elements 17-13–17-15
- F Elements 17-19–17-20
- G and E Elements 20-10–20-11
- G Elements 17-24–17-27
- H Elements 17-31–17-32
- IBIS buffers 4-40
- independent sources 5-2–5-3
  - data driven PWL function 5-19
  - PULSE function 5-8, 5-11, 5-14, 5-17
  - SFFM function 5-21
- inductors 4-12–4-13
- JFETs and MESFETs 4-23–4-24
- linear inductors 4-12
- MOSFETs 4-25–4-27
- mutual inductors, Kxxx 4-15
- POLY 17-7
- PWL 5-17, 5-20
- resistors 4-2–4-3
- transmission lines
  - T Element 4-33
  - U Element 4-35
  - W Element 4-31, 4-31

Elmore delay 20-31

enable (digital vector file) 5-73

encryption

- 8-byte key C-9
- data C-2
- file organization C-6
- FREELIB keyword C-7
- guidelines C-5
- permit file changes C-7
- semicolon bug C-5
- structure C-13

ENDDATA keyword 3-22, 3-24, 3-26

environment variables 2-2, 3-56, 22-17, A-6

EPSMIN option 9-12, 20-19

equations 8-52, 8-59

ERR function 8-58, 8-59

ERR1 function 8-58, 8-60, 13-36

ERR2 function 8-58, 8-60

ERR3 function 8-58, 8-60

## errors

- cannot open
  - output spool file 8-22
- control options A-27
- current exceeding MAXAMP 9-38, 11-25
- DC 10-43
- digital file has blank first line 5-56
- environment variables A-6
- file open 3-63
- functions 8-58–8-60
- input A-12, A-33
- installation A-13
- internal timestep too small 9-33, 9-38, 9-46,  
9-48, 10-5, 10-24, 10-50, 11-3, 11-18,  
11-19, 11-29
- licensing A-15
- messages A-7
- missing .END statement 3-2
- models A-20
- netlist A-27
- no DC path to ground 10-45
- no input data 3-63
- optimization goal 8-44
- parameter name conflict 8-42
- PC A-23
- special characters in input 3-3
- system resource inaccessible 8-22
- tolerances
  - ABSMOS 9-22
  - branch current 9-22, 9-23, 10-22, 10-35
  - optimization by bisection 21-7
  - pole/zero analysis 9-35, 9-36, 10-30, 10-31
  - relative change 9-24, 9-39, 10-38, 11-25
  - RELMOS 9-22, 10-35
  - voltage 9-24, 9-39, 10-39, 11-26

example

- .DATA 3-23
- .FFT statement 11-49
- .OPTIONS SEARCH 3-34
- AC analysis 8-34, 12-10
- comment line 2-12
- configuration file 2-6
- digital vector file 5-85
- experiments 1-8
- first character 2-11

- example (*continued*)
  - Hspice vs .SPICE methods 8-34
  - Monte Carlo 13-17, 13-25
  - netlist file 2-9
  - network analysis, bipolar transistor 12-25
  - optimization 13-43
  - S parameter 6-8
  - subcircuit 3-14
  - subcircuit node names 2-13
  - subcircuit test 3-12
  - transient analysis 11-10, 11-12
  - worst case 13-25
- EXP source function
  - fall time 5-14
  - initial value 5-14
  - pulsed value 5-14
  - rise time 5-14
- exp(x) function 7-11
- experiment 1-8
- EXPLI option 9-21
- EXPMAX option 9-12
- exponential function 5-14, 7-11
- expressions, algebraic 7-9
- external data files 3-6, 3-21
- EXTRAPOLATION parameter 20-10

## F

- F Elements 17-6, 17-18
  - applications 17-7
  - controlling voltage 17-20
  - current gain 17-19
  - data points 17-20
  - gate type 17-19
  - initial conditions 17-19
  - maximum current 17-19
  - minimum current 17-19
  - multiply parameter 17-19
  - name 17-19
  - parameters 17-19
  - polynomial 17-20
  - syntax statements 5-40
  - temperature coefficients 17-20
  - time delay keyword 17-20
  - value multiplier 17-20

- FALL keyword 8-46
- fall time
  - example 15-9
  - EXP source function 5-14
  - simulation example 15-2
- fanout, plotting 15-5

## FAQ

- analysis A-2
- control options A-27
- environment variables A-6
- error messages A-7
- field solver A-35
- input A-12
- installation A-13
- known limitations A-17
- licensing A-15
- manuals A-4
- models A-20
- MS Windows/PC A-23
- netlist A-27
- output A-33
- W Element A-35
- waveform viewing A-36

## Fast Fourier Transform

*See* FFT

- FAST option 9-42, 11-24, 11-31

## FFT

- analysis
  - alfa control parameter 19-7
  - AM modulation 19-11
  - frequency
    - maximum 19-7
    - minimum 19-7
    - of interest 19-9
    - range 19-9, 19-10
    - specification 19-7
  - harmonic distortion 19-14
  - modulator/demodulator 19-14
  - number of points 19-6
  - output 11-50, 19-6
  - results 19-9–19-10
  - spectral leakage 19-14
  - start point 19-6
  - statement syntax 19-6–19-8
  - stop point 19-6

- FFT, analysis (*continued*)
  - test circuit 19-22
  - window type 19-7
- output 19-9
- windows 19-2–19-4
  - Bartlett 19-3
  - Blackman 19-3
  - Blackman-Harris 19-3
  - Gaussian 19-3
  - Hamming 19-3
  - Hanning 19-3
  - Kaiser-Bessel 19-4
  - rectangular 19-3
- field programmable gate arrays 16-19
- field solver FAQ A-35
- FIL keyword 3-21
- file descriptors limit 8-22
- files
  - .a2d 3-78, 3-80, 5-56
  - .ac# 3-78
  - .ft# 3-78, 3-80, 19-10
  - .gr# 3-78
  - .ic 3-78, 10-4
  - .lis 3-78
  - .ma# 3-78
  - .MEASURE output 15-1
  - .ms# 3-78
  - .mt# 3-78
  - .pa# 3-78
  - .st# 3-78
  - .sw# 3-78
  - .tr# 3-78
  - AC analysis 3-80
  - column lamination 3-27
  - concatenated data files 3-25
  - .d2a 5-56
  - DC analysis 3-80
  - design.ac0 15-1
  - design.mt0 15-1
  - design.sw0 15-1
  - external data 3-6, 3-20
  - filenames 3-21
  - graph data 1-8, 3-80
  - hspice.ini 3-56, 9-13
  - include files 3-6, 3-28, 3-33

- files (*continued*)
  - including 3-59
  - initialization 3-59
  - input 1-8, 3-60
  - limit on number 8-22
  - multiple simulation runs 3-48
  - names 3-58, 3-60–3-62
  - output
    - listing 3-79
    - names 3-61
    - status 3-81
    - version number 3-60
  - scratch files 3-63
  - subcircuit node cross-listing 3-80
  - transient analysis 3-80
  - transition data 17-4
- filters
  - 30 degree phase shift 20-17
  - active low-pass 18-17
  - band reject 20-12
  - low-pass 20-14
  - pole/zero analysis
    - Butterworth 18-11
    - high-pass 18-11
    - low-pass 18-7, 18-17
  - transfer functions 20-29
- FIND keyword 8-49
- first character descriptions 2-10
- floating point overflow
  - CONVERGE setting 9-29, 10-36
  - setting GMINDC 9-32, 10-38
- flush_waveform 2-4
- FMAX 9-35, 10-30, 18-4, 19-7
  - option 10-30
- FMIN .FFT parameter 19-7
- FORMAT .FFT parameter 19-6
- Fourier
  - analysis 11-43
  - coefficients 11-45
  - equation 11-45
  - integral 20-5
  - transfer function  $H(f)$  20-5
  - transform 20-5
- FPGA's 16-19
- FREELIB keyword C-7

## FREQ

- element parameter 20-11
- function 20-8
- keyword 20-5
- model parameter 8-15, 17-70

## frequency

- analysis 19-1, 20-11
- complex 20-4, 20-19
- critical 20-5
- domain to time domain 20-1
- domain, transfer function 20-20
- maximum 20-6
  - frequency analysis 20-11
  - setting 9-35, 10-30
- ratio 12-14
- resolution 20-6, 20-10
- response 20-4
  - analysis 20-2
  - table 20-8, 20-10, 20-11
- setting scale 9-35, 10-30
- sweep 12-7
- table 20-2
- variable 7-14
- weighing functions 19-2

## Frequency Table Model 6-5

## frequency-domain model 6-2

## FROM parameter 8-59, 19-6

## FS option 9-44, 9-46, 11-28, 11-29, 11-42, 12-17

## FSCAL option 9-35, 9-36, 10-30, 10-31, 18-4

## FT option 9-45, 9-46, 11-28, 11-29, 11-41, 11-42

## functions

- A2D 5-56, 17-4
- bisection 21-1
- built-in 7-10–7-14
- D2A 5-56, 17-4
- DERIVATIVE 8-55
- ERR 8-58
- INTEG 8-55
- LAPLACE 18-1, 20-2, 20-6
- NPWL 5-46
- OPTIMIZE 17-3
- POLE 18-1, 20-2, 20-7
- PPWL 5-46
- table 7-10
- See also* independent sources

## FV function value 17-8

## G

## G Elements 17-6, 17-22, 20-25–20-30

- amplitude modulator application 17-63
- AND gate 17-34
- applications 17-7
- controlling voltages 17-25, 17-27
- current 17-25
- curve smoothing 17-26
- data points 17-25
- DELTA parameter 17-59
- element value multiplier 17-26
- gate type 17-25
- initial conditions 17-25
- Laplace transform 20-4
- multiply parameter 17-25, 20-11
- names 17-25
- parameter value multiplier 20-11
- polynomial 17-26
- resistance 17-25
- syntax statements 5-44
- temperature coefficients 17-27
- time delay keyword 17-27
- transconductance 17-27
- triode application 17-62
- tunnel diode application 17-59
- voltage to resistance factor 17-27

## GaAsFET model DC optimization 13-66

## gain, calculating 8-34

## GAUSS

- FFT analysis 19-3, 19-7, 19-26
- functions 13-19
- keyword 13-15
- parameter distribution 13-13

## GBW model parameter 17-70

## GEAR algorithm 9-48, 11-19, 11-35

## GENK option 9-21

## getting customer support viii

## global

- node names 3-18
- parameters 7-16

## GMAX option 9-32, 10-37

## GMIN option 9-32, 9-38, 10-38, 10-52, 11-28

## GMINDC option 9-32, 10-38, 10-39, 10-40, 10-52

## GND node 3-16

- GOAL keyword 8-53, 13-36
- GRAD optimization parameter 13-42
- gradient data, printing 9-9
- GRAMP
  - calculation 9-30, 10-37
  - option 9-32, 9-33, 10-24, 10-38, 10-39, 10-44
- graph data file (Viewlogic format) 1-8, 9-12, 9-13
- ground
  - bounce 16-21
  - node name 3-16
- group delay, calculating 9-52
- GSCAL
  - multiplier 9-35, 10-30
  - option 9-35, 10-30, 18-4, 18-5
- GSHUNT option 9-33, 9-38, 10-24, 11-18
- Gxxx element parameters 17-25

## H

- H Elements 17-6, 17-30
  - applications 17-7
  - controlling voltage 17-32
  - data points 17-32
  - element multiplier 17-32
  - element name 17-31
  - gate type 17-31
  - initial conditions 17-31
  - maximum current 17-31
  - minimum current 17-31
  - polynomial 17-32
  - SCR application 17-60
  - syntax statements 5-52
  - temperature coefficients 17-32
  - time delay keyword 17-32
  - transresistance 17-32
- H parameters 12-25
- H9007 option 9-17
- HAMM FFT analysis keyword 19-7
- Hamming FFT analysis window 19-3, 19-25
- HANN FFT analysis keyword 19-7
- Hanning FFT analysis window 19-3, 19-24
- harmonic distortion 12-13, 19-1
- HARRIS FFT analysis keyword 19-7
- HD2 distortion 8-38, 12-13

- HD3 distortion 8-38, 12-13
- hertz variable 7-14
- hier_delimiter 2-4
- hierarchical designs, flattened 3-6
- hold time 21-3
- HSPICE
  - installation directory 3-55
  - job statistics report 8-19–8-22
  - output, redirecting 3-62
  - starting 3-62
  - version
    - 95.3 compatibility 11-41
    - H9007 compatibility 9-17
    - parameter 3-30
- hspice command 3-60–3-65
- hspice.ini file 3-56, 9-13
- hybrid (H) parameters 8-33

## I

- IB model parameter 17-70
- IBIS buffers 4-40
- IBOS model parameter 17-70
- IC parameter 10-8, 10-9, 10-12, 17-13, 17-19, 17-25, 17-31
- ICSWEEP option 9-33, 10-24
- ideal
  - current sources 10-44
  - delay elements 11-33, 17-6, 17-7
  - op-amp 5-34, 5-37, 17-6, 17-13, 17-14, 17-15
  - transformer 5-34, 5-38, 17-6, 17-13, 17-16, 17-58
- idelay (digital vector file) 5-78
- IGNOR keyword 8-59
- imaginary
  - part of AC voltage 8-34–8-35
  - vs. real component ratio 9-36, 10-31
- IMAX option 9-45, 9-47, 11-29, 11-39
- IMIN option 9-45, 9-47, 11-29, 11-39
- impedance
  - AC 8-37
  - versus time 16-9
  - Z parameters 8-33
- impulse response h(t) 20-5



- inactive devices
  - See latent devices
- include files 3-28, 3-33, 3-59
- independent sources
  - AC 5-3, 5-5
  - AM function 5-23
  - current 5-3, 8-74
  - data driven PWL function 5-19
  - DC 5-3, 5-5
  - elements 5-2
  - EXP function 5-14
  - functions 5-7
  - mixed types 5-6
  - PULSE function 5-7
  - PWL function 5-17
  - SFFM function 5-21
  - SIN function 5-11
  - transient 5-3, 5-6
  - types 5-7
  - voltage 5-3, 8-74
  - See also sources
- indepout 8-68
- indepvar 8-66, 8-67, 8-69
- individual element temperature 13-5
- inductance, sac1e 9-35
- inductance, scale 10-30
- inductors
  - current flow 8-27
  - element 4-11, 8-72
  - GENK 9-21
  - KLIM 9-21
  - mutual models 3-29
  - node names 4-12
- INGOLD option 9-7, 9-8
- initial conditions 10-3
  - file 3-59
  - saving and reusing 9-33, 10-24
  - statement 10-9
  - transient 11-7
- initialization 9-34, 10-3, 10-4, 10-26
  - file 3-59
  - saved operating point 10-11
- inline data 3-22
- inner sweep 3-24

- INOISE parameter 8-38
- input
  - admittance 8-38
  - data
    - adding library data 3-45
    - column laminated 3-27
    - concatenated data files 3-25
    - deleting library data 3-45
    - external, with .DATA statement 3-20
    - filenames on networks 3-27
    - for data driven analysis 3-20
    - formats 3-22, 3-25, 3-27
    - include files 3-28
    - printing 9-7
    - suppressing printout 9-7
  - FAQ A-12
  - files
    - character case 3-2
    - compression 3-2
    - configuration file 3-59
    - control characters 3-3
    - DC operating point 3-59
    - demonstration 22-23
    - initialization 3-59
    - names 3-58, 3-60
    - netlist 3-2
    - structure 3-6
    - table of components 3-6
    - unprintable characters 3-3
  - impedance 8-37
  - netlist 2-9
  - netlist file
    - See also input files
    - 3-8–3-48, D-2
- input stimuli 8-65
- input/output
  - cell modeling 22-20
  - digital vector file 5-71
  - drivers 16-16
- installation directory \$installdir 3-55
- installation FAQ A-13
- int(x) function 7-11
- INTEG keyword 8-54, 8-55
- integer function 7-11

- integration
  - algorithms 11-35
  - backward Euler method 9-48, 11-20
  - order of 9-48, 11-20
- integrator, behavioral 17-54
- interconnect, analyzing 16-2
- interfaces
  - Analog Artist 9-12
  - AvanWaves 8-2
  - Mentor 9-13
  - MSPICE 9-13
  - variables 14-17
  - ZUKEN 9-14
- intermodulation distortion 12-13
- internal
  - nodes, referencing 3-16
  - routines 14-33
- INTERP option 9-51, 11-19
- INTERPOLATION parameter 20-11
- interstage gain 8-34
- inverse Laplace transform 20-31
- inverter
  - analysis, transient 11-12
  - circuit, MOS 11-12
  - gate 17-16
  - lookup table 17-45
- ISC model parameter 17-71
- iterations
  - algorithm 11-37
  - count algorithm 11-39
  - limit 9-25, 10-25
  - maximum number of 9-46, 11-29
  - number 13-53
- ITL1 option 9-25, 9-30, 10-25, 10-37
- ITL2 option 10-25
- ITL3 option 9-45
- ITL4 option 9-46
- ITL5 option 9-46, 11-29
- ITLPZ option 9-42, 10-30, 18-5
- ITROPT optimization parameter 13-42
- ITRPRT option 9-51, 11-19
- I-V and C-V plotting demo 22-6

## J

- Jacobian data, printing 9-9
- JFETs
  - current flow 8-28
  - elements 4-23, 8-78
  - length 4-23
  - power dissipation 8-31
  - width 4-23
- JIS model parameter 17-71

## K

- KAISER FFT analysis keyword 19-7
- Kaiser-Bessel FFT analysis window 19-4, 19-5, 19-27
- KCLTEST 9-23, 10-25
- Kerwin's circuit, pole/zero analysis 18-10
- keyword
  - reserved 2-13
- keywords
  - .AC statement parameters 12-6
  - .DC statement parameters 10-15
  - .FFT statement parameters 19-6
  - .MEASUREMENT statement parameters 8-54
  - .MODEL statement parameters 8-14
  - .TRAN statement parameters 11-6
  - analysis statement syntax 13-39
  - BART 19-7
  - command syntax 21-7
  - DATA statement parameters 3-20
  - DTEMP 13-5
  - ERR1 13-36
  - FREELIB C-7
  - FREQ 20-5
  - FS 12-17
  - GOAL 13-36
  - LAST 8-47, 8-50, 8-51
  - MONTE 13-13
  - optimization syntax 13-37
  - PAR 7-9
  - PP 8-53, 8-54
  - RECT 19-7
  - source functions 5-2
  - target syntax 8-47
  - UNORM 19-6
  - weight 8-54

KF model parameter 12-16  
Kirchhoff's Current Law (KCL) test 9-23, 10-25  
KLIM option 9-21

## L

L Element (inductor) 4-16  
LAM keyword 3-21, 3-27  
laminated data 3-27  
Laplace

- band-reject filter 20-12
- element parameter 20-11
- function 18-1, 20-2, 20-5, 20-6, 20-19, 20-23-20-27
- low-pass filter 20-14
- parameters 20-10
- transfer function 20-2, 20-19
- transform 20-2, 20-6, 20-20, 20-21
  - frequency 20-8, 20-11
  - function call 20-4
  - inverse 20-31
  - modeling 20-19
- POLE (pole/zero) function 20-27

LAST keyword 8-47, 8-50, 8-51  
latent devices

- BYPASS option 9-41, 11-18, 11-24
- BYTOL option 9-41, 11-24
- excluding 9-42, 11-24
- MBYPASS option 9-41, 11-24
- removing from simulation 9-41, 11-24
- VNTOL option 9-41, 11-24

LC oscillator model 17-84  
leadframe example 22-10  
LENGTH model parameter 13-21  
LENNAM option 9-7  
LEVEL parameter 13-42, 20-11  
Levenberg-Marquardt algorithm 13-52  
LEVIN model parameter 17-71  
LEVOUT model parameter 17-71  
libraries

- .END statement 3-33
- adding with .LIB 3-45
- ASIC cells 3-56
- AvanLink B-6

libraries (*continued*)

- building 3-31, 3-33
- configuring 7-19
- creating parameters 7-16
- DDL 3-34, 3-55
- defining macros 3-31
- deleting 3-45
- duplicated parameter names 7-16
- encryption C-1
- integrity 7-16
- private 3-38
- protecting 3-38, C-1
- search 3-56
- selecting 3-35
- subcircuits 3-57
- vendor 3-56

licenses issues, FAQ A-15  
limit descriptors command 8-22  
LIMIT keyword 13-16  
limitations, known A-17  
LIMPTS option 9-15  
LIMTIM option 9-12  
LIN keyword 10-16, 11-7, 12-7  
line 4-17  
linear

- capacitor 4-9
- inductor 4-16
- resistor 4-4

LIST option 9-7  
listing

- page width 8-18
- suppressing 3-38

LM_LICENSE_FILE 2-2  
LMAX model parameter 1-5  
LMIN model parameter 1-5  
local

- parameters 7-16
- truncation error algorithm 9-39, 9-40, 9-48, 11-19, 11-26, 11-27, 11-39, 11-40

log(x) function 7-11  
log10(x) function 7-11  
logarithm function 7-11  
LSCAL option 9-35, 10-30, 18-5  
Lsim models, calibrating 15-1

- LV 8-39
- LV18 model parameter 22-6
- LVLTIM option 9-40, 9-46, 9-48, 11-19, 11-20, 11-26, 11-27, 11-33, 11-39, 11-41
- LX 8-39
- LX7 model parameter 22-6
- LX8 model parameter 22-6
- LX9 model parameter 22-6

## M

- M element parameter 17-19, 17-25, 20-11
- mA741 op-amp 17-77
- macros 3-31, 3-45
- magnetic core models 3-29
- magnitude
  - AC voltage 8-35
- magnitude, AC voltage 8-33, 8-34, 8-35
- MANU model parameter 17-71
- manufacturing tolerances 13-20
- Marquardt scaling parameter 13-52
- mask (digital vector file) 5-78
- matrix
  - calculations 9-25, 10-26
  - minimum pivot values 9-27, 10-27
  - parameters 12-18
  - row/matrix ratio 9-26, 10-27
  - size limitation 9-26, 10-27
- MAX parameter 8-54, 13-42, 17-14, 17-19, 17-25, 17-31, 20-11, 21-4
- max(x,y) function 7-11
- max_waveform_size 2-4
- MAXAMP option 9-23, 9-38, 10-25, 11-25, 12-9
- MAXF parameter 20-6, 20-11
- MAXFLD keyword 12-17
- maximum
  - number size 9-12
  - value, measuring 8-53
- MAXORD option 9-48, 11-20
- MBYPASS option 9-41, 9-42, 11-24, 11-25
- mean, statistical 13-3
- MEASDGT option 9-7, 9-8
- MEASFAIL option 9-51

- measfail value 21-7
- MEASOUT option 9-13
- measure
  - data output formatting 9-8
  - parameter types 8-42
- Mentor interface 9-13
- MENTOR option 9-13
- MER keyword 3-21, 3-25, 3-27
- MESFETs 4-23
- messages
  - pivot change 9-26, 9-28, 10-27, 10-29
  - See also* errors, warnings
- Metaencrypt C-6, C-9
- METHOD option 9-48, 11-20
- Meyer and Charge Conservation parameters 8-81
- MIN parameter 8-54, 17-14, 17-19, 17-25, 17-31
- min(x,y) function 7-11
- minimum
  - number size 9-12
  - value, measuring 8-53
- MINVAL keyword 8-54, 8-59
- mixed mode
  - simulation 5-56, 17-4
  - See also* D2A, A2D
- mixed sources 5-6
- mixed-signal simulation
  - See* mixed mode
- model analysis options 9-18, 9-18–9-20
  - BJTs, EXPLI 9-21
  - DCAP 9-18, 9-19
  - inductors 9-21
  - MODSRH 9-18
  - MOSFETs 9-20
  - SCALM 9-18
  - TNOM 9-19
- MODEL keyword 10-15, 13-39
- model parameters
  - .ALTER blocks 3-40
  - .GRAPH statement parameters 8-15
  - A2D 5-56
  - BISECTION 21-6
  - capacitance distribution 13-23
  - D2A 5-56, 5-58
  - DELVTO 13-9

- model parameters (*continued*)
  - DTEMP 13-5
  - LENGTH 13-21
  - LEVEL 13-42
  - manufacturing tolerances 13-20
  - MONO 8-15
  - op-amps 17-67, 17-67–17-73, 17-74
  - output 8-15
  - PHOTO 13-21
  - RSH 13-9
  - sigma deviations, worst case analysis 13-9
  - skew 13-8
  - suppressing printout of 9-9
  - TEMP 3-19, 13-5
  - temperature analysis 13-5
  - TIC 8-15
  - TOX 13-9
  - TREF 13-4, 13-5, 13-6
  - XPHOTO 13-21
- model parameters *See* model parameters diodes
- models
  - algebraic 11-33
  - BJTs 3-29
  - BSIM LEVEL 13 3-30
  - BSIM2 LEVEL 39 3-30
  - capacitors 3-29
  - characterization 10-14
  - diode 3-29
  - DTEMP parameter 22-15
  - FAQ A-20
  - JFETs 3-29
  - LV18 22-6
  - LX7, LX8, LX9 22-6
  - magnetic core 3-29
  - Monte Carlo analysis 13-13, 13-17, 13-25
  - MOSFETs 3-29
  - mutual inductors 3-29
  - names 3-28
  - op-amps 3-29, 17-65
  - optimization 3-29
  - plot 3-29
  - private 3-38
  - protecting 3-38
  - reference temperature 13-5
  - simulator access 3-33
- models (*continued*)
  - specifying 3-56
  - subcircuit MULTI 3-14
  - types 3-29
  - typical set 13-12
- MONO model parameter 8-15
- Monte Carlo
  - AC analysis 12-6
  - analysis 13-2, 13-3, 13-25–13-32
    - demo files 22-24
    - distribution options 13-15–13-16
  - DC analysis 10-15
  - time analysis 11-5
- MONTE keyword 11-6, 12-6, 13-13
- MOS
  - inverter circuit 11-12
  - op-amp optimization 13-68
- mos_tech 2-2
- MOSFET
  - technology file 2-2
- MOSFETs
  - current flow 8-28
  - drain diffusion area 4-26
  - elements 4-25, 8-79
  - initial conditions 4-26
  - model analysis options 9-20–??
  - node names 4-25
  - perimeter 4-26
  - power dissipation 8-32
  - SCALM 9-20
  - source 4-26, 4-27
  - squares 4-26
  - temperature differential 4-27
  - WL 9-20
  - zero-bias voltage threshold shift 4-27
- MSPICE simulator interface 9-13
- MU option 9-49, 11-25
- Muller algorithm 9-36, 10-31, 18-3
- MULTI 3-14
- multiple .ALTER statements 3-40
- multiplier
  - G and E Element values 20-11
  - GSCAL 9-35, 10-30
- multiply parameter 3-14, 3-52, 4-3, 5-3

- multipoint experiment 1-8
- multi-terminal network 6-1
- multithreading 3-67
- mutual inductor 4-14, 8-72

## N

- namei 8-66, 8-67, 8-68
- NAND gate adder 22-4
- natural
  - frequency 18-2
  - log function 7-11
- n-channel, MOSFET's models 3-29
- NDIM 5-27
- negative conductance, logging 9-17
- nested library calls 3-32
- netlist 3-6
  - AvanLink B-7
  - AvanLink-DA B-11
  - encrypting C-1
  - FAQ A-27
  - file example 2-9
  - flat 3-6
  - input files 3-2
  - schematic 3-6
  - structure 2-9
- network
  - analysis 12-18
  - filenames 3-27
  - multi-terminal 6-1
  - output 8-37, 12-19
  - switched capacitor 20-44
  - variable specification 12-22
- NEWTOL option 9-34, 10-25
- nodal voltage output 8-25, 8-34
- NODE option 3-38, 9-8
- nodes
  - connection requirements 3-16
  - cross-reference table 9-8
  - floating supply 3-16
  - global versus local 3-18
  - internal 3-16
  - MOSFET's substrate 3-16

- nodes (*continued*)
  - names 3-14, 3-16, 3-18, 22-6
    - automatic generation 3-18
    - ground node 3-16
    - period in 3-15
    - subcircuits 3-16
    - zeros in 3-17
  - numbers 3-14, 3-16
  - phase or magnitude difference 8-34
  - printing 9-8
  - shorted 10-44
  - terminators 3-16
  - voltages, encrypting C-1
- NODESET keyword 10-12
- node-to-element list 9-26, 9-28, 10-27, 10-29
- NOELCK option 9-9
- noise
  - calculations 12-16
  - CMI_Noise 14-26
  - coupled line 16-23
  - folding 12-17
  - input 8-38
  - numerical 9-38, 11-18
  - output 8-38, 12-16
  - problems and solutions 16-3
  - quota 16-6, 16-7
  - sampling 12-17
  - sources 16-3, 16-4
- NOISENPT frequency Table Model 6-5
- NOMOD option 9-9
- NONE keyword 10-7, 10-12
- nonlinear elements 20-2
- NOPAGE option 9-9
- NOPIV option 9-25, 10-26
- NORM FFT analysis keyword 19-6
- norm of the gradient 13-52
- NOTOP option 9-9
- NOWARN option 9-17
- NP .FFT parameter 19-6
- NPDELAY element parameter 17-14, 17-20, 17-26, 17-32
- npn BJT models 3-29
- npoints 8-66, 8-67, 8-69

- NPWL
  - element parameter 17-26
  - function 5-46, 17-24
  - NMOS transistor function 17-11
- NT, FAQ A-23
- numbers
  - formatting 9-7, 9-8
  - maximum size 9-12
  - minimum size 9-12
- NUMDGT option 9-9
- numerical integration algorithms 9-48, 11-20
- numerical noise 9-33, 9-38, 10-24, 11-18
- NUMF keyword 12-17
- NXX option 9-6, 9-9
- Nyquist critical frequency 20-5, 20-11

## O

- obtaining customer support viii
- OCT keyword 10-16, 11-7, 12-7
- odelay (digital vector file) 5-78
- OFF parameter 9-34, 10-4, 10-26
- one-dimensional function 5-28
- ONoise parameter 8-38
- OPAMP element parameter 17-14
- op-amps
  - .MODEL statement 17-67
  - automatic generation 17-5
  - behavioral models 17-65
  - characterization 15-7
  - common mode rejection ratio 17-68
  - compensation level selector 17-68
  - diode and BJT saturation current 17-69
  - element statement 17-66
  - excess phase parameter 17-69
  - gain parameter 17-67
  - input
    - bias current 17-70
    - bias offset current 17-70
    - level type selector 17-71
    - offset current 17-70
    - offset voltage 17-73
    - short circuit current 17-71
  - internal capacitance 17-68
  - op-amps (*continued*)
    - JFETs saturation current 17-71
    - mA741 model 17-77
    - manufacturer's name 17-71
    - model
      - generator 17-65
      - names 3-29
      - parameters 17-67, 17-74
      - selector 17-69
    - open loops 10-44
    - optimization 13-68
    - output
      - level type selector 17-71
      - resistance ROAC 17-72
      - slew rate 17-72
      - voltage 17-73
    - phase margin 17-69
    - power
      - dissipation 17-71
      - supply voltage 17-73
    - subcircuit example 17-75
    - subcircuit generator 17-5
    - temperature parameter 17-73
    - unity gain frequency 17-70
- operating point
  - .IC statement initialization 10-9
  - .NODESET statement initialization 10-10
  - capacitance 9-28, 10-22
  - estimate 10-5, 11-3
  - initial conditions 3-59
  - pole/zero analysis 18-3
  - restoring 10-13
  - saving 3-17, 10-11
  - solution 9-34, 10-4, 10-26
  - transient 11-3
  - voltage table 10-7
- operating systems, HSPICE 1-6
- operators 7-10, 20-21
- OPT keyword 13-37, 21-7
- optimization
  - .MODEL statement 13-40
  - .PARAM statement 13-39
  - AC analysis 12-6, 13-59
  - algorithm 13-42
  - analysis statements 13-38

- optimization (*continued*)
  - behavioral models 17-48, 17-52
  - bisection method 21-4, 21-6
  - CMOS tristate buffer 13-54
  - control 13-35
  - convergence options 13-35
  - curve-fit 13-36
  - cv 22-32
  - data-driven vs. s-parameters 13-59
  - DC analysis 10-15, 13-45, 13-47, 13-62, 13-66
  - error function 8-44
  - example 13-43, 22-21
  - goal 13-36
  - incremental 13-62
  - iterations 13-42
  - lengths and widths 13-68
  - MODEL keyword 13-39
  - models 3-29
  - MOS 13-47, 13-68
  - network 13-50, 13-59
  - parameters 13-59
    - .MODEL statement 13-41–13-42
    - magnitude and phase 13-59
    - measured vs. calculated 13-59
  - results
    - function evaluations 13-53
    - iterations 13-53
    - Marquadt scaling parameter 13-52
    - norm of the gradient 13-52
    - residual sum of squares 13-52
  - S parameters 13-59
  - simulation accuracy 13-35
  - simultaneous 13-55, 13-66, 13-68
  - statements 13-37
  - syntax 13-37
  - TDR 16-9, 16-11
  - time
    - analysis 11-5, 13-37
    - required 13-35, 13-41
- OPTIMIZE keyword 10-16, 13-37, 17-3
- options
  - configuration file 2-4
  - EPSMIN 20-19
  - FAQ A-27
  - OPTLST 21-7
  - OPTLST option 9-9, 21-7
  - OPTS option 9-9
  - OPTxxx parameter 13-36, 13-37
  - Opus 9-12
  - oscillation, eliminating 9-49, 11-20
  - oscillators
    - behavioral models 17-65
    - DELMAX option setting 11-33
    - LC 17-84
    - VCO 17-82
  - out (digital vector file) 5-81
  - outer sweep 3-24
  - output
    - .FFT results 19-9
    - .MEASURE results 8-40
    - admittance 8-38
    - commands 8-2
    - current 8-26
    - data
      - format 9-6, 9-8, 9-14
      - limiting 9-14, 9-51, 11-19
      - number format 9-7
      - redirecting 3-62
      - significant digits specification 9-9
      - specifying 9-14, 9-15
      - storing 9-13, 9-14
    - driver example 22-10
    - FAQ A-33
    - files 3-60
      - names 3-58, 3-61
      - redirecting 3-58
      - reducing size of 9-14, 9-17
      - version number, specifying 3-60
    - graphing 8-12
    - impedance 8-37
    - measfail value 21-7
    - network 8-37
    - nodal voltage, AC 8-34
    - noise 8-38, 12-16
    - parameters 8-25
    - plotting 8-9–8-10
    - power 8-29
    - printing 8-6–8-24
    - printout format 8-18
    - reusing 8-65



- output (*continued*)
  - saving 8-10
  - statements 8-2
  - variables 8-3
    - AC formats 8-35
    - function 7-12
    - printing 9-51, 11-19
    - probing 8-11
    - specifying significant digits for 9-9
  - voltage 8-25
- outz (digital vector file) 5-81
- ovari 8-66, 8-67, 8-68
- overview of data flow 1-8
- overview of simulation process 1-10

## P

- packed input files 3-2
- Pade approximation 20-38
- page eject, suppressing 9-9
- PAR keyword 7-5, 7-9, 15-11
- parameters
  - .FFT statement 19-6–19-7
  - AC sweep 12-4
  - ACM 11-33
  - admittance (Y) 8-33
  - AF 12-16
  - algebraic 7-9, 7-10
  - analysis 7-7
  - assignment 7-4
  - CAPOP 11-33
  - cell geometry 7-15
  - constants 7-4
  - data type 7-3
  - data-driven analysis 3-20
  - DC sweep 10-14
  - defaults 7-19
  - defining 7-2, 7-16
  - DIM2 8-38
  - DIM3 8-38
  - encrypting C-1
  - evaluation order 7-3
  - frequency response table 20-10
  - HD2 8-38
  - HD3 8-38

- parameters (*continued*)
  - hierarchical 3-52, 7-15, 8-43
  - hybrid (H) 8-33
  - IC 10-9
  - impedance (Z) 8-33
  - inheritance 7-18, 7-19
  - INOISE 8-38
  - input netlist file 3-5
  - KF 12-16
  - Laplace 20-10
  - libraries 7-16–7-19
  - M 3-52, 7-7
  - matrix 12-18
  - measurement 7-7
  - model 5-58, 5-61
  - modifying 3-20
  - multiply 7-7
  - names 3-29
  - ONOISE 8-38
  - optimization 7-15
  - OPTxxx 13-36, 13-37
  - output 8-25
  - overriding 7-17, 7-20
  - PAR keyword 7-5
  - PARHIER option 7-19
  - passing 7-15–7-22
    - example 15-11
    - order 7-3
    - problems 7-22
    - Release 95.1 and earlier 7-22
  - pole/zero 20-10
  - repeated 8-42
  - scattering (S) 8-33
  - scope 7-15–7-16, 7-22
  - SIM2 8-38
  - simple 7-4
  - simulator access 3-33
  - skew, assigning 3-34
  - subcircuit 3-52, 7-6
  - UIC 10-9
  - user-defined 7-5
  - UTRA 10-43
  - See also* model parameters, optimization parameters
- parametric analysis 8-3

- PARHIER option 7-19
- PARMIN optimization parameter 13-42
- PASSFAIL keyword 21-7
- path names 3-16, 9-10
- path numbers, printing 9-10
- PATHNUM option 9-10
- PC, FAQ A-23
- p-channel
  - JFETs models 3-29
  - MOSFET's models 3-29
- PD model parameter 17-71
- peak measurement 8-48
- peak-to-peak value, measuring 8-53
- period (digital vector file) 5-72
- permit.hsp file, encryption capability C-7
- phase
  - AC voltage 8-35
  - calculating 8-34
  - detector model 17-88, 17-95
  - locked loop, BJT model 17-90
- PHOTO model parameter 13-21
- piecewise linear sources *See* PWL
- pin capacitance, plotting 15-6
- pivot
  - algorithm, selecting 9-25, 9-27, 10-26
  - change message 9-26, 9-28, 10-27, 10-29
  - matrix calculations 9-25, 10-26
  - reference 9-26, 10-27
  - selection 11-30
- PIVOT option 9-25, 9-27, 10-26, 11-30
- PIVREF option 9-26, 10-27
- PIVREL option 9-26, 10-27
- PIVTOL option 9-26, 9-27, 9-28, 10-27, 10-29
- platforms for Hspice 1-6
- PLIM option 9-10
- PLL *See* phase locked loop
- plot
  - limits 8-8
  - models 3-29
  - value calculation method 9-5, 9-38, 12-9
- PLOT keyword 8-14
- pn junction conductance 10-52
- pnp BJT models 3-29
- POI keyword 10-16, 11-7, 12-7
- POLE
  - element parameter 20-11
  - function 18-1, 20-2, 20-7, 20-27–20-31
    - call 20-27
    - highpass filter 20-29
    - low-pass filter 20-30
    - transconductance element statement 20-7
    - voltage gain element statement 20-7
- pole/zero
  - analysis 18-1, 18-4, 20-2
    - absolute tolerance 9-35, 10-30, 18-5
    - active low-pass filter 18-17
    - capacitance scale 18-4
    - CMOS differential amplifier 18-13
    - conductance scale 18-4
    - frequency 9-35, 10-30, 18-4
    - high-pass Butterworth filter 18-11
    - imaginary to real ratio 18-5
    - inductance scale 18-5
    - iteration limit 18-5
    - Kerwin's circuit 18-10
    - low-pass filter 18-7
    - maximum frequency 18-4
    - maximum iterations 9-42, 10-30
    - Muller algorithm 18-3
    - operating point 18-3
    - overview 18-2
    - real to imaginary ratio 9-36, 10-31, 18-5
    - relative error tolerance 18-5
    - simple amplifier 18-15
    - starting points, Muller algorithm 9-36, 10-31
  - complex 20-28
  - conjugate pairs 20-8
  - control options 9-35–9-36, 18-4, 18-4–18-5
  - function, Laplace transform 20-7
  - models 20-41, 20-43
  - parameters 20-10
  - transfer function 20-27
  - transient modeling 20-2
- POLY parameter 5-27, 17-7, 17-14, 17-20, 17-26, 17-32

- polynomial function 5-27, 17-7
  - FV function value 17-8
  - one-dimensional 5-28
  - three-dimensional 5-30
  - two-dimensional 5-29
- POST option 1-8, 9-14
- POST_VERSION option 9-10
- pow(x,y) function 7-10
- power
  - dissipation 8-29, 8-32
  - function 7-10
  - operating point table 10-6
  - output 8-29
  - stored 8-29
- PP keyword 8-53, 8-54
- PPWL
  - element parameter 17-26
  - function 5-46, 17-24
  - PMOS transistor function 17-11
- precision numbers 7-9
- print
  - CMI_PrintModel 14-27
  - control options 8-17
- printed circuit boards 16-2
- printer, device specification 8-12
- printout
  - columns, number 9-6
  - disabling 9-6, 9-9
  - page ejects 9-9
  - suppressing 3-38
  - value calculation method 9-5, 9-38, 12-9
- PROBE option 9-14
- program structure 1-7
- protecting data 3-38
- PRTDEFAULT printer 8-12
- PSF option 9-14
- PSRR specification 17-5
- PULSE source function 5-8, 5-11, 5-14, 5-17
  - delay time 5-8
  - initial value 5-8
  - onset ramp duration 5-8
  - plateau value 5-8
  - recovery ramp duration 5-8
  - repetition period 5-8
  - width 5-8

- pulse width 21-14
- PUTMEAS option 8-41, 9-52
- PWL
  - current controlled gates 17-7
  - data driven 5-19
  - element parameter 17-14, 17-20, 17-26, 17-32
  - functions 5-31, 17-7, 17-10
  - gates 17-7
  - NMOS and PMOS transistor functions 17-11
  - output values 5-17
  - parameters 5-17
  - repeat parameter 5-17
  - segment time values 5-17
  - simulation time 11-42
  - sources, data driven 5-19
  - voltage-controlled capacitors 17-7
  - voltage-controlled gates 17-6
  - See also* data driven PWL source
- PWR model parameter 17-71
- pwr(x,y) function 7-10
- PZABS option 9-35, 10-30, 18-5
- PZTOL option 9-36, 10-31, 18-5

## Q

- quality assurance 13-2

## R

- R Element (resistor) 4-4
- RAC model parameter 17-72
- radix (digital vector file) 5-69
- random limit parameter distribution 13-13
- RC
  - analysis 11-10, 12-10
  - circuit 12-10
  - line modeling 20-31
  - optimizing 13-50
- rcells, reusing 7-16
- real part of AC voltage 8-34–8-35
- real vs. imaginary component ratio 9-36, 10-31
- RECT FFT analysis keyword 19-7
- rectangular FFT window 19-3
- reference temperature 3-19, 13-5

- RELH option 9-24, 9-39, 10-38, 11-25, 12-9
- RELI option 9-23, 9-24, 9-39, 10-25, 10-33, 10-38, 11-25
- RELIN optimization parameter 13-42
- RELMOS option 9-22, 9-23, 9-24, 10-25, 10-33, 10-35, 10-38, 11-33
- RELOUT optimization parameter 13-42
- RELQ option 9-39, 11-26, 11-40
- RELTOL option 9-38, 9-39, 11-24, 11-26
- RELV option 9-24, 9-39, 9-42, 10-33, 10-39, 11-24, 11-25, 11-26
- RELVAR option 9-46, 11-26, 11-33
- RELVDC option 9-24, 10-39
- repeat function 22-3
- residual sum of squares 13-52
- resistance 9-34, 10-28, 12-3
- resistor
  - current flow 8-27
  - element 4-2
  - element template listings 8-71
  - length parameter 4-3
  - linear 4-4
  - model name 4-2
  - node to bulk capacitance 4-3
  - voltage controlled 5-45
  - width parameter 4-3
- RESMIN option 9-34, 10-28
- RESULTS keyword 10-16
- reusing simulation output 8-65
- RIN keyword 12-18
- ripple calculation 15-3
- rise and fall times 8-43
- RISE keyword 8-46
- rise time 15-2, 15-9
- RISETIME option 9-39
- RITOL option 9-36, 10-31, 18-5
- RLOAD model parameter 5-61
- RMAX option 9-46, 11-29, 11-42
- RMIN option 9-46, 11-29, 11-42
- RMS (root mean squared) measurement 8-48
- RMS keyword 8-54
- rms value, measuring 8-53
- roac model parameter 17-72

- ROUT keyword 12-18
- ROUT model parameter 17-72
- row/matrix ratio 9-26, 10-27
- RSH model parameter 13-9
- runtime statistics 9-5

## S

- S Element 6-2
  - Frequency Table Model 6-5
  - syntax 6-3
  - transmission line 6-8
- S parameter
  - example 6-8
  - model type 3-29
  - S Element 6-2
  - transmission line 6-10
- S19NAME model parameter 5-61
- S19VHI model parameter 5-61
- S19VLO model parameter 5-61
- S1NAME model parameter 5-61
- S1VHI model parameter 5-61
- S1VLO model parameter 5-61
- sampling noise 12-17
- saturable core
  - elements 4-15, 8-83
  - models 4-15
  - winding names 8-83
- scale factors 2-14
- SCALE parameter 4-3, 17-14, 17-20, 17-26, 17-32, 20-11, 20-36, 22-6
- scaling, effect on delays 22-20
- SCALM option 9-20
- scattering (S) parameters 6-2, 8-33
- schematic
  - AvanLink B-6
  - AvanLink-DA B-10
  - netlists 3-6
- Schmitt trigger example 10-18
- scope of parameters 7-16
- scratch files 3-63
- SDA option 9-14
- s-domain 20-19, 20-24

- SEARCH option 3-34, 3-57, 9-11, 22-17
- search path, setting 3-35
- SEED option 9-16
- sensitivity analysis 10-19
- setup time 21-3, 21-5, 21-9
- SFFM source function
  - carrier frequency 5-21
  - modulation index 5-21
  - output amplitude 5-21
  - output offset 5-21
  - signal frequency 5-21
- sgn(x) function 7-11
- shorted nodes 10-44
- sigma model parameter, sweeping 15-3
- sign function 7-11
- signal integrity 16-3
- SIGNAME element parameter 5-60
- signed power function 7-10
- Signetics FAST I/O drivers 16-16
- silicon-controlled rectifier 17-60
- silicon-on-sapphire devices 3-18
- SIM_ANALOG option 4-17
- SIM_CONFIG variable 2-2
- SIM_RAIL option 4-17
- SIM2 distortion measure 8-38, 12-13
- simulation
  - ABSVAR option 11-41
  - accuracy 9-37, 9-48, 11-19, 11-23, 11-31, 13-35
    - improvement 9-44, 11-18
    - models 11-33
    - option 11-34, 11-41
    - reduced by BYPASS 9-41, 11-18
    - timestep 11-32
    - tolerances 10-32, 10-33, 11-31
  - AvanLink B-7
  - AvanLink-DA B-12
  - circuits
    - with Signetics drivers 16-16
    - with Xilinx FPGAs 16-19
  - electrical measurements 22-17
  - example D-1
  - graphical output D-10
  - ground bounce example 16-21
  - multiple analyses, .ALTER statement 3-39
  - simulation (*continued*)
    - multiple runs 3-48
    - performance, multithreading 3-67
    - preparation 16-2
    - process, overview 1-10
    - reducing time 3-20, 9-40, 9-41, 9-44, 9-45, 9-47, 11-18, 11-24, 11-26, 11-27, 11-28, 11-29, 11-42, 20-44
    - results
      - graphing 8-12
      - output variables 9-14
      - plotting 8-9–8-10
      - printing 8-6–8-24
      - specifying 8-40–8-43
      - storing 9-14
    - reusing output 8-65
    - speed 9-9, 11-31
    - structure 1-7
    - time
      - reducing 9-41, 9-42, 9-45, 9-47, 11-25
      - RELVAR option 11-41
    - title 3-8
  - SIN source function 5-11
  - sin(x) function 7-10
  - single point experiment 1-8
  - single-frequency FM source function 5-21
  - sinh(x) function 7-10
  - sinusoidal source function 5-11
  - skew
    - file 13-12
    - parameters 3-34, 13-8
  - skip_nrd_nrs 2-5
  - slope (digital vector file) 5-79
  - SLOPETOL option 9-47, 11-26
    - simulation time 11-42
    - timestep control 11-41
  - small-signal
    - DC sensitivity 10-19
    - distortion analysis 12-13
    - transfer function 10-20
  - SMOOTH element parameter 17-26
  - SONAME model parameter 5-61

## source

- AC sweep 12-4
- controlled 17-4, 17-6
- data driven 5-19
- DC sweep 10-14
- element types 17-6
- keywords 5-2
- Laplace transforms 20-20
- statements 3-9
- See also* independent sources

SOVHI model parameter 5-61

SOVLO model parameter 5-61

SPARSE option 10-28

spectral leakage 19-2, 19-14

spectrum analysis 19-1

## SPICE

- compatibility 9-16
  - AC output 8-34–8-35, 9-5, 9-38, 12-9
  - numeric format 9-7
  - plot 8-8, 9-10
- option 9-16

sqrt(x) function 7-10

square root function 7-10

SRN model parameter 17-72

SRNEG model parameter 17-72

SRP model parameter 17-72

SRPOS model parameter 17-72

Star-Hspice

- conventions 2-12

START .FFT parameter 19-6

START keyword 11-6

## statement

- .PARAM 8-64

## statements

- .AC 12-4, 13-5, 13-38
- .ALTER 3-39
- .DATA 3-20, 15-9
  - external file 3-24, 3-26
  - inline 3-22
  - inner sweep example 3-23
  - outer sweep example 3-23
- .DC 10-14, 13-5, 13-38
- .DCVOLT 10-9
- .DEL LIB 3-45

## statements (*continued*)

- .DISTO 12-13
- .END 3-48
- .ENDL 3-30, 3-31
- .ENDS 3-13
- .EOM 3-13
- .FFT 11-47, 19-6
- .FOUR 11-44
- .GLOBAL 3-18, 3-19
- .GRAPH 8-2, 8-12, 8-22
- .IC 10-9
- .INCLUDE 3-28
- .LIB 3-30, 3-31
  - call 3-31
  - nesting 3-32
- .LOAD 10-10, 10-13
- .MACRO 3-11
- .MALIAS 3-43
- .MEASURE 8-2, 8-3, 8-40, 9-8, 9-13
- .MODEL 3-28, 8-14, 13-5, 13-41
- .NET 12-18
- .NODESET 9-29, 10-10, 10-23
- .NOISE 12-15
- .OP 10-6
- .OPTION 9-2, 10-30
  - ACCT 8-19
  - ALT999(9) 8-18
  - CO 8-17
  - INGOLD 8-18
  - POST 8-19
- .OPTIONS SEARCH 3-34
- .PARAM 3-36, 13-39
- .PLOT 8-2, 8-8, 8-22
- .PRINT 8-2, 8-4, 8-22
- .PROBE 8-2, 8-10, 8-22
- .PROTECT 3-38
- .PZ 10-21, 18-4
- .SAMPLE 12-17
- .SAVE 10-10, 10-11
- .SENS 10-19
- .STIM 8-2, 8-65
- .SUBCKT 3-11, 8-42, 17-66
- .TEMP 3-19, 13-5, 13-6
- .TF 10-20
- .TITLE 3-8
- .TRAN 13-5, 13-38

- statements (*continued*)
  - .UNPROTECT 3-38
  - .WIDTH 8-18
  - call subcircuit 3-13
  - DOUT 8-2
  - element 3-9
  - initial conditions 10-9
  - source 3-9
- statistical analysis 13-8–13-32
- statistics
  - calculations 13-3
  - listing 8-19
  - report 9-5
- stimuli 8-65
- stimulus input files 17-4
- STOP .FFT parameter 19-6
- structure simulation 1-7
- subcircuit
  - node names 2-12
- subcircuits
  - .PRINT and .PLOT statements 3-54
  - adder 22-3
  - call statement 3-13
  - calling 3-13
  - calling tree 3-17
  - changing in .ALTER blocks 3-40
  - creating reusable circuits 3-51
  - element names 3-13
  - generator 17-5
  - global versus local nodes 3-18
  - hierarchical parameters 3-52
  - library structure 3-57
  - model names 3-13
  - multiply parameter 3-14
  - multiplying 3-52
  - names 3-12
  - node names 3-13, 3-16, 3-17
  - node numbers 3-12
  - output printing 8-22
  - parameter 3-12, 3-13, 15-11
  - path names 3-16
  - power dissipation computation 8-29
  - printing path numbers 9-10
  - search order 3-54
  - test example 3-12
  - zero prefix 3-17

- sweep
  - data 3-24, 9-13
  - frequency 12-7
  - inner 3-24
  - outer 3-24
  - variables 22-15
- SWEEP keyword 10-16, 11-6, 12-6
- switch example 5-50
- switched capacitor 20-44, 20-46
- switch-level MOSFET's example 5-50
- symbols, reserved 2-13
- Synopsys models, calibrating 15-1
- systems, simulating 16-2

## T

- tabular data 5-73, 5-77
- Taguchi analysis 13-2
- tan(x) function 7-10
- tanh(x) function 7-10
- TARG keyword 8-45
- target specification 8-44
- TC1, TC2 element parameters 17-15, 17-20, 17-27, 17-32, 20-11
- TD parameter 8-36, 8-50, 17-15, 17-20, 17-27, 17-32
- tdelay (digital vector file) 5-78
- TDR (time domain reflectometry) 16-9, 16-10, 16-11
- TEMP
  - keyword 10-16, 12-7
  - model parameter 3-19, 13-5, 17-73
  - sweep variable 22-15
- temper variable 7-14
- temperature
  - AC sweep 12-4
  - circuit 13-4, 13-5, 13-6
  - coefficients 4-3, 20-11, 22-15
  - DC sweep 10-14, 10-17
  - derating 3-19, 3-20, 13-5, 13-7
  - element 13-5, 13-6
  - optimizing coefficients 22-15
  - reference 3-19, 13-5
  - sweeping 22-15
  - variable 7-14
- Temperature Variation Analysis 13-2

- tfall (digital vector file) 5-80
- THD (total harmonic distortion) 19-1
- three-dimensional function 5-30
- TIC model parameter 8-15
- time 10-7
  - delay 8-36
  - domain
    - algorithm 11-36
    - reflectometry 16-9
    - to frequency domain 20-1
  - maximum 20-6
  - resolution 20-5
  - variable 7-14
  - versus impedance 16-9
  - See also* CPU time
- TimeMill models, calibrating 15-1
- TIMERES option 9-47, 11-26
- TIMESCALE model parameter 5-61
- timestep
  - algorithms 9-44, 11-18, 11-39
  - calculation for DVDT=3 9-44, 11-28
  - changing size 9-39, 11-26
  - control 9-40, 9-44, 9-45, 9-46, 11-26, 11-27, 11-28
    - algorithms 11-38–11-41
    - CHGTOL 11-40
    - DELMAX 11-42
    - FS 11-42
    - FT 11-42
    - IMAX 11-39
    - IMIN 11-39
    - minimum internal timestep 11-42
    - Minimum Timestep Coefficient 11-42
    - options 11-32, 11-41
    - RELQ 11-40
    - RMAX 11-42
    - RMIN 11-42
    - TRTOL 11-40
    - TSTEP 11-42
  - default control algorithm 11-35
  - DVDT algorithm 11-40
  - internal 9-44, 11-28
  - local truncation error algorithm 11-40
  - maximum 9-45, 9-46, 9-47, 11-29
  - minimum 9-45, 9-46, 9-47, 11-29
  - reversal 9-44, 11-23, 11-40
- timestep (*continued*)
  - setting initial 9-44, 11-28
  - transient analysis algorithm 9-48, 11-19
  - variation by HSPICE 9-44, 11-28
- TIMESTEP model parameter 5-61
- timing
  - analysis 21-1
  - constraints 21-1
  - failures 21-2
  - hold time 21-3
  - setup time 21-3
  - violation analysis 21-2
- title for simulation 3-8
- tmp directory 3-63
- TMPDIR environment variable 3-63
- TNOM option 3-19, 13-5
- TO .FFT parameter 19-6
- TO keyword 8-53, 8-59
- TOL 12-17
- tolerance options 10-22
- TOP keyword 10-12
- topology 9-9, 14-35
- TOX model parameter 13-9
- transconductance
  - FREQ function 20-8
  - LAPLACE function 20-6
  - POLE function 20-7
- transfer function 10-20
  - algebraic 17-3
  - analysis 17-48
  - coefficients 20-29
  - filters 20-29
  - frequency domain 18-2, 20-20
  - general form 20-19, 20-27
  - inverse Laplace transform 20-31
  - poles 18-2
  - reduced form 20-28
  - roots 18-2
  - voltage gain 20-23
  - zeros 18-2
- transfer sign function 7-11
- TRANSFORMER element parameter 17-15
- transformer, behavioral 17-58
- transforms, Fourier 20-5



- transient
  - analysis 8-3
    - accuracy 9-37
    - Fourier analysis 11-44
    - initial conditions 10-9, 11-3
    - inverter 11-12
    - number of iterations 9-46, 11-29
    - RC network 11-10
    - sources 5-6
  - modeling 20-2
  - output variables 8-25
- transition files 17-4
- transmission lines
  - effects 16-7
  - example 22-10
  - resistive termination 6-8
  - S Element 6-8
  - simulation 16-2
  - U Element 4-35
- TRAP algorithm
  - See* trapezoidal integration
- trapezoidal integration
  - algorithm 9-48, 11-19, 11-35
  - coefficient 9-49, 11-25
- TREF model parameter 13-5, 13-6
- TRIG keyword 8-45
- trigger specification 8-44
- triode tube 5-51, 17-61
- trise (digital vector file) 5-79
- triz (digital vector file) 5-81
- troubleshooting A-1
- TRTOL option 9-40, 11-27, 11-40
- truncation algorithm 11-39
- tskip (digital vector file) 5-72
- TSTEP
  - multiplier 9-46, 11-29
  - option 9-46, 11-29
  - timestep control 11-42
- tunit (digital vector file) 5-71
- tunnel diode behavioral 17-59
- two-dimensional function 5-29

## U

- U Elements 5-56
  - digital input 5-56
  - transmission line model 3-29
- UIC
  - analysis parameter 10-5
  - keyword 11-7
  - parameter 10-9
  - transient analysis parameter 11-3
- UNIF keyword 13-15
- uniform parameter distribution 13-13
- unit_atto 2-5
- unity gain frequency 22-17
- UNORM FFT analysis keyword 19-6
- unprintable characters in input 3-3
- UNWRAP option 9-52, 12-9
- URL for Avant! viii
- UTRA model parameter restriction 10-43

## V

- variables
  - AC formats 8-35
  - changing in .ALTER blocks 3-40
  - DEFAULT_INCLUDE 3-59
  - Hspice-specific 7-14
  - output 8-3
    - AC 8-33
    - DC 8-25
    - transient 8-25
  - plotting 22-6
  - sweeping 22-15
  - TMPDIR 3-63
- variables, environment 2-2
- variance, statistical 13-3
- VCC model parameter 17-73
- VCCAP 5-46, 17-27
- VCCS *See* voltage controlled current source
- VCO *See* voltage controlled oscillator
- VCR *See* voltage controlled resistor
- VCVS *See* voltage controlled voltage source
- vector patterns 5-68
- VEE model parameter 17-73

- vendor libraries 3-56
- VERIFY option 9-7, 9-11
- Verilog models, calibrating 15-1
- Verilog value format 5-76
- version
  - 95.3 compatibility 11-41
  - H9007 compatibility 9-17
  - HSPICE 3-30
- VFLOOR option 9-28, 11-30
- VHDL models, calibrating 15-1
- Viewlogic 5-56
  - graph data file 9-12, 9-13
  - links B-13
- Viewsim simulator 5-56
- vih (digital vector file) 5-81
- vil (digital vector file) 5-82
- vname (digital vector file) 5-69
- Vnn node name in CSOS 3-18
- VNTOL option 9-40, 9-41, 9-42, 11-24, 11-27
- voh (digital vector file) 5-83
- vol (digital vector file) 5-84
- VOL keyword 5-39, 17-17
- voltage
  - error tolerance
    - DC analysis 9-24, 10-39
    - transient analysis 9-39, 11-26
  - failure 10-47
  - gain
    - FREQ function 20-8
    - LAPLACE function 20-6
    - POLE function 20-7
  - initial conditions 10-9
  - iteration-to-iteration change 9-32, 10-23
  - limiting, in transient analysis 9-47, 11-28
  - maximum change 9-44, 11-23
  - minimum
    - DC analysis 9-23, 10-35
    - listing 9-28, 11-30
    - transient analysis 9-37, 9-40, 11-27
  - nodal output DC 8-25
  - operating point table 10-7
  - relative change, setting 9-46, 11-26
  - sources 5-33, 5-52, 8-26
  - summer 5-38, 17-16

- voltage (*continued*)
  - tolerance
    - BYTOL option 9-41, 11-24
    - MBYPASS multiplier 9-42, 11-25
  - value for BYPASS 9-41, 11-24
  - variable capacitance model 17-85
- VOLTAGE keyword 10-7
- voltage-controlled
  - capacitor 5-46, 5-50, 17-24
  - current source 5-40, 5-44, 8-72, 17-7, 17-22, 17-27
  - elements 17-6
  - oscillator 5-39, 17-17, 17-82
  - resistor 5-45, 17-7, 17-23, 17-27
  - sources 5-26
  - voltage source 5-33, 8-73, 17-6, 17-12, 17-15, 17-81
- VON model parameter 17-73
- VONEG model parameter 17-73
- VOP model parameter 17-73
- VOPOS model parameter 17-73
- VOS model parameter 17-73
- Vos specification 17-5
- vref (digital vector file) 5-82
- vth 8-69
- vth (digital vector file) 5-83
- vtl 8-69
- VVCAP model 17-85
- Vxxx source element statement 5-2

## W

- W Elements 4-30
  - FAQ A-35
  - RLGC model definition 6-10
  - transmission line model 3-29
- warnings
  - all nodes connected together 10-44
  - floating power supply nodes 3-16
  - limiting repetitions 9-17
  - MAXF ignored 20-12
  - misuse of VERSION parameter 3-30
  - suppressing 9-17
  - zero diagonal value detected 10-47
- WARNLIMIT option 9-17

- waveform
  - characteristics 5-77, 5-78
  - display (AvanWaves) B-7, B-12
  - FAQ A-36
- WEIGHT keyword 8-54, 8-59
- WHEN keyword 8-49, 22-17
- width of printout 8-18
- wildcard uses 2-7
- wildcard_left_range 2-5
- wildcard_match_all 2-5
- wildcard_match_one 2-5
- wildcard_right_range 2-5
- WINDOW .FFT parameter 19-7
- Windows (MS), FAQ A-23
- wire 16-7
- WL option 9-20
- WMAX model parameter 1-5
- WMIN model parameter 1-5
- worst case analysis 13-8, 13-25, 13-32, 15-9
- Worst Case Corners Analysis 13-2
- WSF output data 9-12
- www site for Avant! viii

## X

- XGRID model parameter 8-15
- XL model parameter 13-9
- XMAX model parameter 8-15
- XMIN model parameter 8-15
- XnR, XnI option 9-36, 10-31
- XPHOTO model parameter 13-21
- XSCAL model parameter 8-15
- XW model parameter 13-9

## Y

- Y parameter
  - line model 20-40
  - modeling 20-36, 20-37
  - network 20-38
- Y parameters 6-2
- YGRID model parameter 8-15
- yield analysis 13-2

- YIN keyword 8-38, 12-19
- YMAX parameter 8-15, 8-59
- YMIN parameter 8-15, 8-59
- YOUT keyword 8-38, 12-19
- YSCAL model parameter 8-15

## Z

- zero delay gate 5-38, 5-50
- ZIN keyword 8-37, 12-19
- ZOUT keyword 8-37, 12-19
- ZUKEN option 9-14

